agena >>

a programming language

---

reference

for version 7.11.4 despina

by alexander walz

october 21, 2026

agena Copyright 2006 to 2026 by alexander walz, rhineland.
All rights reserved. Portions Copyright 1994-2026 Lua.org, All rights reserved.

None of the Agena project members or anyone else connected with this documentation, in any way whatsoever, can be responsible for your use of the information contained in or linked from it.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this manual, and the author was aware of a trademark claim, the designations have been printed in initial caps or all caps.

The latest release of Agena can be found at http://sourceforge.net/projects/agena.

The source files can be browsed on https://github.com/al-agena/Agena.

This manual has been created with Lotus Word Pro 98 running on Sun Microsystems VirtualBox with Microsoft Windows 2000 Professional, Visio 2013, yWorks yEd Graph Editor, and PDF Creator.

Table of Contents

7 The Libraries 13

8 Basics 17

9 Strings 85
9.1 Basic String Functions 85
9.1.1 Operators and Functions 87
9.1.2 The strings Library 92
9.1.3 Patterns 139
9.1.4 Format Strings for Pack and Unpack 141
9.2 memfile - Memory File for Strings & Bytes in General 144
9.3 utf8 - UTF-8 Helpers 159
9.4 iconv - Internationalization 161
9.5 hashes - Hashes 163
9.6 bloom - Bloom Filter 183
9.7 cuckoo - Cuckoo Filter for Strings 187
9.8 regex - Regular Expression Matching 189
9.9 fzy - Fuzzy Search 191
9.10 suffix - libdivsufsort Binding 193
9.11 binpack - Serialising Strings & Numbers 201
9.12 aspell - GNU Aspell 0.60 Spell Checker 204

10 Structures 211
10.1 Tables 211
10.1.1 Operators & Functions 212
10.1.2 tables Library 225
10.2 Sets 239
10.3 Sequences 248
10.3.1 Operators & Functions 248
10.3.2 sequences Library 260
10.4 Registers 267
10.4.1 Operators & Functions 267
10.4.2 registers Library 281
10.5 Pairs 286
10.6 numarray - Numeric C Arrays 288
10.6.1 Introduction 288
10.6.2 General Functions 292
10.6.3 Mathematical Functions 314
10.6.4 Bitwise Functions 318
10.6.5 Units Conversion Functions 318
10.6.6 Metamethods 321
10.7 llist - Linked Lists 322
10.7.1 Introduction and an Example 322
10.7.2 Functions 324
10.7.3 Unrolled Singly-Linked Lists 327

10.7.4 Doubly-Linked Lists 330
10.8 bags - Mulitsets 334
10.9 bimaps - Bi-directional Maps 337
10.10 heaps - Priority Queues 343
10.10.1 Introduction and Examples 343
10.10.2 Metamethods 344
10.10.3 Binary Heap Functions 344
10.10.4 AVL Tree Functions 347
10.10.5 Skew Heap Functions 350
10.11 rbtree - Red-Black Trees 352
10.11.1 Metamethods 353
10.11.2 Functions 353
10.12 bfield - Bit Fields 356
10.13 tuples - Closures Storing Data 359
10.14 lookup - Lookup Tables 363
10.14.1 Metamethods 365
10.14.2 Functions 365
10.15 fifo - First-in First-Out Queues 369
10.15.1 Metamethods 370
10.15.2 Functions 370
10.16 lifo - Last-in First-Out Queues 372
10.16.1 Metamethods 373
10.16.2 Functions 373
10.17 trie - Prefix and Suffix Trees 375
10.17.1 Metamethods 376
10.17.2 Functions 376
10.18 numcuckoo & numfilter - Cuckoo Filters for Numbers 378
10.19 dblhash - Number Hash Sets 381
10.20 uintmap - Integer-to-String Hash Maps 384
10.21 intmap - Integer-to-Integer Hash Maps 388
10.22 dblmap - Number-to-Number Hash Maps 392
10.23 strhash - String Hash Sets 396
10.24 strmap - String-to-Number Hash Maps 400

11 Numbers 405
11.1 Mathematical Functions 405
11.1.1 Operators and Functions 408
11.1.2 math Library 440
11.1.3 fastmath Library 473
11.2 bytes Library 475
11.3 mapm - Arbitrary Precision Library 495
11.3.1 Introduction 495
11.3.2 Real Domain 496
11.3.3 Complex Domain 502
11.3.4 Excursus: Precision 505
11.4 gmp - GNU Multiple Precision Arithmetic Library 507
11.4.1 Creation of Signed and Unsigned Integers 507
11.4.2 Signed and Unsigned Integer Arithmetic 508

11.4.3 Number Theoretic Functions 511
11.4.4 Bitwise Operations 513
11.4.5 Miscellaneous 515
11.4.6 Metamethods 516
11.4.7 Constants 516
11.5 mpfr - GNU Multiple Precision Floating-Point Reliable Library 517
11.6 divs - Library to Process Fractions 525
11.7 dual - Dual Numbers 529
11.8 clock - Clock Package 537
11.9 astro - Astronomy Functions 541
11.10 cordic - Numerical CORDIC Library 569
11.11 zx - Sinclair ZX Spectrum Arithmetic Functions 572
11.12 calc - Calculus Package 581
11.13 linalg - Linear Algebra Package 629
11.14 stats - Statistics 669
11.15 long - 80-Bit Floating-Point Arithmetic 716
11.16 ints - 64-bit Integer Arithmetic 741
11.16.1 Introduction 741
11.16.2 Usage 741
11.17 combinat - Combinatorics 756
11.18 numtheory - Number Theory 761
11.19 kiss - Fast Fourier Fransform (FFT) 767
11.20 ival - Interval Arithmetic 769
11.21 maple - Aliases to Maple Functions 775
11.22 unimath - Common Functions for Floating-Point Data 776
11.23 dd - Double-to-Double Arithmetic 779

12 Input & Output 787
12.1 io - Input and Output Facilities 787
12.2 binio - Binary File Package 807
12.3 xbase - Library to Read and Write xBase Files 820
12.4 ads - Agena Database System 838
12.5 xml - XML Parser 848
12.6 json - JSON Structures 856
12.7 tar - UNIX tar 857
12.8 gzip - Library to Read and Write UNIX gzip Compressed Files 859
12.9 minizip - Library to Read and Write ZIP Compressed Files 862
12.10 ini - Library to Read and Create INI Files 874
12.11 sqlite - Interfacing to SQLite Databases 879
12.12 keyfile - Key Files 880

13 Communication 889
13.1 net - Network Library 889
13.1.1 Introduction and Examples 889
13.1.2 Functions 895
13.2 usb - libusb Binding 909
13.3 com - Serial RS-232 Communication through COM Ports 911

14 System & Environment 917
14.1 os - Access to the Operating System 917
14.2 environ - Access to the Agena Environment 969
14.3 package - Modules 984
14.4 rtable - Remember Tables 986
14.5 registry - Access to the Registry 989
14.6 stack - Built-In Number, Character & Value Stacks 991
14.7 sema - Unique Identifiers 1011
14.8 Coroutines 1015
14.9 debug - Debugging 1016
14.10 history - Logging Your Input 1025

15 Graphics 1029
15.1 gdi - Graphic Device Interface package 1029
15.1.1 Opening a File or Window 1029
15.1.2 Plotting Functions 1029
15.1.3 Colours, Part 1 1030
15.1.4 Closing a File or Window 1030
15.1.5 Supported File Types 1030
15.1.6 Plotting Graphs of Univariate Functions 1031
15.1.7 Plotting Geometric Objects Easily 1031
15.1.8 Colours, Part 2 1032
15.1.9 GDI Functions 1033
15.2 fractals - Library to Create Fractals 1046
15.2.1 Escape-time Iteration Functions 1046
15.2.2 The Drawing Function fractals.draw 1051
15.2.3 Examples 1056
15.3 curses- Library to Create Terminal Applications 1057
15.3.1 Introduction and Example 1057
15.3.2 curses Functions 1058
15.3.3 Window Functions 1058
15.3.4 Constants 1059
15.3.5 Compatibility 1060

16 Utilities 1063
16.1 utils - Utilities 1063
16.2 skycrane & telex - Auxiliary Functions 1083
16.3 factory - Iterators 1091
16.4 units - Physical Unit Conversion 1095

17 C API Functions 1101

Appendix A 1172
A1 Operators 1172
A2 Metamethods 1173
A3 Mathematical Constants 1177
A4 System Variables 1178

A5 Command-Line Usage & Scripting 1180
A5.1 Using the -e Option 1180
A5.2 Using the Internal args Table and Exit Status 1180
A5.3 Running a Script and then Entering Interactive Mode 1182
A5.4 Running Scripts in UNIX and Mac OS X 1182
A5.5 Converting an Agena Script into a Binary Executable 1182
A5.6 Command Line Switches 1183
A6 Define Your Own Printing Rules for Types 1183
A7 The Agena Initialisation File 1185
A8 Escape Sequences 1187
A9 Some Few Technical Notes 1188

Appendix B 1190
B1 Agena Licence 1190
B2 GNU GPL v2 Licence 1190
B3 Sun Microsystems Licence for the fdlibm IEEE 754 Style Arithmetic Library .. 1197
B4 GNU Lesser General Public Licence 1197
B5 SOFA Software Licence 1206
B6 MAPM Copyright Remark (Mike's Arbitrary Precision Math Library) 1208
B7 RSA Security/MD5 Licence 1208
B8 David Schultz's Openlibm Licence 1209
B9 ISC Licence 1209
B10 Other Copyright Remarks 1210

Appendix C 1211
C1: Further Reading 1211

Index 1213

Part Two

Reference

Chapter Seven

The Libraries

7 The Libraries

The standard libraries, mostly taken from the Lua 5.1distribution, provide essential services to the language (e.g., **nextone** and **getmetatable**).

Other libraries provide access to `outside` services (e.g., I/O).

Most libraries are available immediately after start-up of the interpreter. Others, called `plus packages`, must be initialised with the **import** statement or the **readlib** function - just check the respective chapters.

Aside from the basic library, the others are:

Library	**Functionality**
ads	Database specialised on storing and retrieving strings
astro	Astronomical time and date functions
bags	Multisets, Cantor sets that count occurrences
bimaps	Bi-directional maps
binio	Functions for processing binary files
bloom	Bloom filter for strings and numbers
bytes	Bits and bytes twiddling
calc	Undergraduate Calculus package
clock	Functions to process hours, minutes and seconds
com	RS-232 communication via COM ports
combinat	Combinatorial functions
convert	Physical unit conversion (lengths, weights, etc.)
cordic	CORDIC numeric functions
cuckoo	Cuckoo filter for strings
curses	(n)curses binding to build terminal applications
dd	Double-to-double arithmetic
div	Fractions
dual	Dual numbers
environ	Access to the Agena environment
fastmath	Numeric approximations
fifo	First-in first-out queues
fractals	Various fractals & plotting routines, some FRACTINT support
fzy	Fuzzy string matching
gdi	Graphics
gmp	GNU Multiple Precision Arithmetic Library (GMP)
gzip	Read and write UNIX gzip compressed files
hashes	String and number hashes
heaps	Skewed & binary heaps plus AVL trees
iconv	GNU iconv port, to transform strings between codepages
ini	INI file encoding & decoding (iniparser binding)
io	Input/output functions for console and files
ival	Interval arithmetic
json	JSON encoding & decoding

Library	Functionality
kiss	Fast Fourier Transform
lifo	Last-in first-out queues
linalg	Undergraduate Linear Algebra
llist	Linked lists
long	80-Bit Floating-Point arithmetic
lookup	Lookup tables
maple	Aliases to Maple functions
mapm	Mathematical arbitrary precision library
math	Additional mathematical functions
memfile	String memory files
minizip	Read and write ZIP compressed files
mpfr	GNU Multiple Precision Floating-Point Reliable Library (MPFR)
net	IPv4/IPv6-based data exchange over the Internet or LANs
numarray	Numeric C arrays
numtheory	Number Theory
os	Functions to operate with the underlying operating system
rbtree	Red-black binary trees
regex	Regular expression matching (PCRE2)
registers	Functions for register administration
registry	Functions to access the registry
rtable	Administration of remember tables
sema	Unique integer IDs
skycrane	Utilities and easy-to-use wrappers to some functions
sqlite	SQLite 3 database interface
stack	Functions for stack operations
stats	Statistical functions
strings	Various string handling functions
tables	Functions specialised on table processing
tar	Functions to list, read and extract UNIX tar archives
trie	Prefix and suffix trees for strings
unimath	arithmetic functions working with all kinds of numerical data
utils	Utility functions, e.g. CSV import and export
xbase	xBase file support (i.e. dBASE (tm) III+)
xml	XML encoding & decoding (LuaExpat port)
zx	Sinclair ZX Spectrum numeric functions

Except for the basic library, each library provides all its functions as fields of a global table or as methods of its objects. Agena operators have been built into the kernel (the Virtual Machine), so they are not part of any library.

Chapter Eight

Basics

8 Basics

The basic library provides some core functions to Agena. If you do not include this library in your application, you should check carefully whether you need to provide implementations for some of its facilities.

For logical operators, please see Chapter 4.8.

Summary of functions:

Checks

> **$$**, **abs**, **alternate**, **assigned**, **assume**, **binsearch**, **filled**, **has**, **in**, **isequal**, **member**, **notin**, **rawequal**, **recurse**, **satisfy**, **whereis**.

Extraction

> **$**, **bottom**, **columns**, **descend**, **drop**, **duplicates**, **getentry**, **left**, **max**, **min**, **nextone**, **op**, **ops**, **rawget**, **right**, **top**, **unique**, **unity**, **unpack**, **values**.

Types

> **checkoptions**, **checktype**, **fractional**, **gettype**, **isboolean**, **iscomplex**, **isint**, **isnegative**, **isnegint**, **isnonneg**, **isnonnegint**, **isnonposint**, **isnonzeroint**, **isnumber**, **isnumeric**, **ispair**, **isposint**, **ispositive**, **isseq**, **isstring**, **isstructure**, **istable**, **nan**, **optboolean**, **optcomplex**, **optint**, **optnonnegative**, **optnonnegint**, **optnumber**, **optposint**, **optpositive**, **optstring**, **settype**, **type**, **typeof**.

Counting

> **countitems**, **size, tables.numintersect**, **tables.numminus**, **tables.numunion**.

Data Manipulation

> @, **append**, **augment**, **binclude**, **freeze**, **getbit, include**, **map**, **move**, **prepend**, **purge**, **put**, **rawset**, **reduce**, **remove**, **select**, **selectremove**, **setbit**, **sort**, **sorted**, **subs**, **subsop**, **toreg**, **toseq**, **toset**, **totable**, **unfreeze**, **zip**.

Data Generation

> **iterate**, **tables.new**, **sequences.new**, **registers.new**.

Error Handling

> **argerror**, **error**, **protect**, **xpcall**.

Libraries

readlib, **with**.

Files

read, **save**.

Output

print, **printf**, **write**, **writeline**.

Parsing

load, **loadfile**, **loadstring**.

Cantor Operations

bintersect, **bisequal**, **bminus**.

Metatables

addtometatable, **getmetatable**, **setmetatable**.

Miscellaneous

bye, **clear**, **restart**, **time**.

f @ obj

f @ g

In the first form, the operator maps a function `f` to all the values in table, set, sequence, register, string or pair `obj`. `f` should be a univariate function and return only one value. The type of return is the same as of `obj`. If `obj` has metamethods or user-defined types, the return will also have them.

If `obj` is a string, `f` is applied on all of its characters from the left to right. The return is a sequence of the mapping results. If `obj` is **null**, the operator returns **null**.

Examples:

```
> <: x -> x^2 :> @ [1, 2, 3]:
[1, 4, 9]

> <: x -> x > 1 :> @ [1, 2, 3]:
[false, true, true]
```

In the second form, the function creates the composition of two functions `f @ g` = `f(g(x))` and returns it is a new function `(f @ g)(x)`. `f` and `g` may be univariate or multivariate and also return multiple results.

Example:

```
> # first take root, then negate

> h := <: x -> -x :> @ <: x -> sqrt x :>

> h(2):
-1.4142135623731
```

The operator actually calls function **map**.

See also: @ and **$$** operators, **map**, **reduce**, **remove**, **select**, **subs**, **subsop**, **times**, **zip**.

f $ obj

Returns all values in table, set, sequence or register `obj` that satisfy a condition determined by function `f`. `f` should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x > 1 :> $ [1, 2, 3]:
[2, 3]
```

If present, the function also copies the metatable and user-defined type of `obj` to the new structure.

Please note that if `obj` is a table, the return might include holes. With `obj` a register, all values up to the current top pointer are evaluated, and the size of the returned register is equal to the number of the elements in the return.

The operator actually calls function **select**.

If `obj` is a string, the operator returns a string with all the accurate characters.

If `obj` is **null**, the operator returns **null**.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **unique**, **values**, **zip.**

f $$ obj

Checks whether at least one element in table, set, sequence or register `obj` or at least one character in string `obj` satisfies the condition defined by function `f` and returns **true** or **false**. `f` should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 1 :> $$ [1, 2, 3]:
false
```

If `obj` is **null**, the operator returns **null**.

f $$$ obj

Counts the number of elements in table, set, sequence or register `obj` or the number of characters in string `obj` that satisfy the condition defined by function `f`, a univariate function that returns at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 3 :> $$$ [1, 2, 3]:
2
```

With `obj` a register, all values up to the current top pointer are evaluated.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **subsop**, **unique**, **values**, **zip**.

x in obj

Checks whether `x` is included in table, sequence, register or pair `obj` and returns **true** or **false**.

```
> 3 in [1, 2, 3]:
true

> 0 in [1, 2, 3]:
false
```

With `x` and `obj` both strings, returns the position of substring `x` in `obj` or **null** if `x` is not part of `obj`. `x` may consist of one or more characters.

```
> 'c' in 'abc':
3

> 'd' in 'abc':
null

> 'bc' in 'abc':
2
```

x notin obj

Checks whether `x` is not included in table, sequence, register or pair `obj` and returns **true** or **false**.

```
> 3 notin [1, 2, 3]:
false

> 0 notin [1, 2, 3]:
true
```

With x and obj both strings, checks whether substring x is not part of obj and returns **true** or **false**. x may consist of one or more characters.

```
> 'c' notin 'abc':
false

> 'd' notin 'abc':
true
```

abs (x)

If x is a number, the **abs** operator will return the absolute value of x. With complex numbers, the magnitude $\sqrt{\text{real(x)}^2 + \text{imag(x)}^2}$ is evaluated (see also: **cabs**).

If x is a Boolean, it will return 1 for **true**, 0 for **false**, and -1 for **fail**.

If x is null, **abs** will return -2.

If x is a string of only one character, **abs** will return the ASCII value of the character as a number. If x is the empty string or longer than length 1, the function returns **fail**.

addtometatable (obj, metatable [, usertype])

Sets all the metamethods in metatable to the *existing* metatable of obj, possibly overwriting existing metamethods of obj. If the metatable of obj has a '__metatable' or a '__gc' field, raises an error. This function returns the modified metatable.

If usertype, a string, is given, the function in addition sets the specified user-defined type to obj, and if usertype is **null**, deletes it.

See also: **setmetatable**.

alternate (x, y)

Returns x if y evaluates to **null**, else returns y. This is equivalent to **if** y = **null then** x **else** y **fi**, which is not equal to y **or** x.

See also: **or** operator.

append (x, obj)

Adds element x to the end of structure obj, in-place. The function returns the modified structure.

With a table, its hash part is not modified.

With register obj, the function automatically increases its size by one, so you do not have to explicitly enlarge obj before. If obj is a pair, returns obj:x.

See also: **copyadd**, **include**, **prepend**, **put**.

argerror (x, procname, message)

Receives any value `x`, the name of procedure `procname` (a string) where `x` did not satisfy anything, the error message text `message`, and appends the user-defined type or if not defined the basic type of `x`. Thus it returns the error message: 'Error in `procname`: `message`, got <type of `x`>.'.

The function is written in Agena and included in the lib/library.agn file.

See also: **assume**, **error**, **protect**, **xpcall**.

assert (arg)

This is just an alias to **assume**.

assigned (obj)

This Boolean operator checks whether any value different from **null** is assigned to the expression `obj`. If `obj` is already a constant, i.e. a number, boolean including **fail**, or a string, the operator always returns **true**. If `obj` *evaluates* to a constant, the operator also returns **true**. See also: **unassigned**.

assume (arg [, message])

Issues an error when the value of its argument `arg` is **false** (i.e., **null** or **false**); otherwise, returns all its arguments. `message` is an error message; when absent, it defaults to 'assumption failed'.

See also: **try/catch** statement, **argerror**, **assert**, **error**, **protect**, **xpcall**.

augment (obj1, obj2 [, ···])

Joins two or more tables, sequences or registers `obj1`, `obj2` etc. together horizontally. The arguments must either be tables, sequences or registers only. All structures must be of the same size and have the same keys. The type of return is determined by the type of the arguments.
The function is written in Agena and included in the lib/library.agn file.

See also: **columns**, **linalg.augment**, **zip**.

beta (x, y)

Computes the Beta function. `x` and `y` are numbers or complex values. The return may be a number or complex value, even if `x` and `y` are numbers. The Beta function is defined as: Beta(`x`, `y`) $=\frac{\Gamma x * \Gamma y}{\Gamma(x+y)}$, with special treatment if `x` and `y` are integers or are equal.

See also: **math.beta**, **math.gammasign**, **math.lnbeta**.

binclude (obj, x [, options])

Efficiently inserts a number or string `x` into a sorted table, sequence or register `obj`, shifting up other elements to open space, if necessary. `obj` must be either in ascending or descending order, with ascending the default. If it is descending, you must pass the `sorting` option in each call to the function. See **binsearch** for more information on the supported options.

Examples:

```
> a := []

> binclude(a, 10, sorting=<: x, y -> x > y :>)

> a:
[10]

> binclude(a, 20, sorting=<: x, y -> x > y :>)

> a:
[20, 10]

> binclude(a, 5, sorting=<: x, y -> x > y :>)

> a:
[20, 10, 5]

> binclude(a, 15, sorting=<: x, y -> x > y :>)

> a:
[20, 15, 10, 5]
```

binsearch (o, x [, m [, l [, r]]] [, options])

Performs a binary search for number or string `x` in the *sorted* table, sequence or register `o`. You may optionally specify the left border `l` and the right border `r` in `o` where to search for `x`, by default `l` is 1 and `r` is size `o`. The very first element in `o` to be checked is given by `m` which by default is (`l` + `r`) \ 2.

With a non-empty structure `o`, the function returns **true** on success or **false** otherwise. The second return is the index position of the last element checked before the function returns. With an empty structure, the function returns **fail** only.

You may have to sort `o` before invoking the function, otherwise the result would be incorrect, but check out the `sorting` option below. When searching for a string `x`, the function uses the current collation of the system.

The function accepts up to two options:

- `sorting` = *f*: indicates the sorting order of the numbers or strings in a structure, which usually should be numerically or alphabetically ascending or descending. *f* denotes a function: The functions <: x, y -> x > y :> or **math.gt** for example

indicate that the elements in the structure are in descending order, contrary to the default which is ascending:

```
> binsearch([5, 4, 3, 2, 1], 4, sorting = math.gt):
true    2
```

- `compare` = *f*: controls how to check for a hit, with *f* denoting a function such like **approx** to check for approximate equality which is useful when searching for fractional numbers:

```
> binsearch([1, 2, round(Pi, 10), 4, 5], Pi, compare = approx):
true    3
```

See also: **binclude**, **in** operator, **trie** package, **os.setlocale**.

bintersect (obj1, obj2 [, option])

Returns all values of table, sequence or register `obj1` that are also values in table or sequence `obj2`. `obj1` and `obj2` must be of the same type. The function performs a binary search in `obj2` for each value in `obj1`. If no option is given, `obj2` is sorted before starting the search. If you pass an option of any value then `obj2` should already have been sorted, for no correct results would be returned otherwise.

With larger structures, this function is much faster than the **intersect** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bisequal**, **bminus**.

bisequal (obj1, obj2 [, option])

Determines whether the tables, sequences or registers `obj1` and `obj2` contain the same values. The function performs a binary search. If no option is given (any value), `obj1` and `obj2 are` sorted before starting the search. If you pass an option of any type then `obj1` and `obj2` should already have been sorted, for no correct results would be returned otherwise.

With larger structures, this function is much faster than the = operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bminus**.

bminus (obj1, obj2 [, option])

Returns all values of table, sequence or register `obj1` that are not values in table, sequence or register `obj2`. `obj1` and `obj2` must be of the same type. The function performs a binary search in `obj2` for each value in `obj1`.

If no `option` is given, `obj2` is sorted before starting the search.

If you pass a non-**null** `option` then `obj2` should already have been sorted, for no correct results would be returned otherwise.

With larger structures, this function is much faster than the **minus** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bisequal**.

bottom (obj)

With the table array, sequence or register `obj`, the operator returns the element at index 1. If `obj` is empty, it returns **null**.

See also: **top**.

bye

Quits the Agena session. No arguments or brackets are needed. If a procedure has been assigned to the name **environ.onexit**, then this procedure will automatically be run before exiting the interpreter. The function also conducts a final garbage collection fully closes the state of the interpreter before leaving. An example:

```
> environ.onexit := proc() is print('Tschüß !') end

> bye
Tschüß !
```

checkoptions (procname, obj, option [, ···] [, true])

Checks options passed to a given procedure, saving many lines of code in procedures.

Since an option such like `delimiter=';'` in a function call is actually passed as the pair `'delimiter':';'` you have to make sure that `real` pairs containing data (but not options) are not included in the call to **checkoptions**. See Chapter 6.6.

Its first argument `procname` - a string, not the function reference - is the name of the procedure in which the check takes place.

Its second argument `obj` - a table - represents the arguments to be checked.

The third and following arguments are pairs. The respective left operand (a string) will be checked whether one of the right operands of the pairs in `obj` is of the type passed as the right operand (a string, a basic or a pseudo-type). See examples below.

The evaluation of `obj` works as follows: If an entry in `obj` is not a pair, it is not evaluated, ignored and not returned in the resulting table. But if the entry is a pair, the function checks whether the left-hand side is a string, i.e. the name of an option. It then checks whether its right hand side is of the given type in anything passed to `option` or further options of type pair. By default, if an option in `obj` cannot be found in `option` or further options of type pair, an error will be issued. But if the very last argument is the Boolean value **true**, no error will be issued and the `unknown` option is part of the resulting table.

If successful, the return is a table where the respective left-hand side in `obj` is the key and the respective right-hand side in `obj` is the respective entry. You may have a look at the lib/skycrane.agn file in your local Agena installation, function **skycrane.scribe** for an example. User-defined types are properly handled.

Pseudo-numeric types are: **integer**, **posint**, **nonnegint**, **nonzeroint**, **negative**, **nonnegative**, **positive**. Further pseudo-types are: **basic**, **listing** and **anything**, see Chapter 6.8.2.

Thus:

```
> checkoptions('myproc', [1, 'neil':'armstrong'], neil=string):
> # 'neil' must be a string, number 1 will be skipped as not being a pair
[neil ~ armstrong]

> checkoptions('myproc', ['neil':'armstrong'], neil=boolean):
Error in `myproc`: boolean expected for neil option, got string.

> checkoptions('myproc', ['neil':'armstrong', 'james':'lovell'],
>    neil=string, true):
[james ~ lovell, neil ~ armstrong]
```

checktype (obj, main, sub)

Checks whether the structure `obj` is a table, set, pair, sequence or register, and whether it is of the type given by `main` (a string), and whether all its elements are of type `sub` (a string). It returns **true** or **false**. User-defined types are supported.

The function is written in Agena and included in the lib/library.agn file.

See also: **type**, **tables.isall**, **sequences.isall**, **registers.isall**, **tuples.isall**.

cleanse (obj)

Empties a table, set, sequence or register `obj` and returns the emptied structure. With a register, sets all its places to **null** and returns the modified register. With tables, sets and sequences, the memory previously occupied can be reused by the interpreter.

See also: **pack**, **sets.cleanse**, **tables.cleanse**, **tables.reshuffle**, **tables.resize**, **sequences.resize**, **registers.resize**.

clear v1 [, v2, ···]

Deletes the values in variables `v1`, `v2`, `···`, and performs a garbage collection thereafter in order to clear the memory occupied by these values.

columns (obj, p [, ···] [, 'structure'])

Extracts the given columns `p` (etc.) from the two-dimensional table, sequence or register `obj`. The type of return is determined by the type of `obj` and is either a structure of structures if the option `'structure'` is given, or a multiple return of structures.

The function is written in Agena and included in the lib/library.agn file.

See also: **ops**, **select**, **unpack**, **values**, **linalg.col**, **registers.col**, **sequences.col**, **tables.col**.

copy (obj [, option])

The operator copies the entire contents of a table, set, pair or sequence `obj` into a new structure. If `obj` contains structures itself, those structures are also copied (by a `deep copying` method). Structures included more than once are properly aggregated to one single reference to save memory space. Metatables and user-defined types are copied, too.

With tables, if the `'array'` option is given, then the operator will return just the array part of `obj`. Likewise, the `'hash'` option only extracts the hash part of `obj`.

If the option is `'nometa'`, then metatables and user-defined types will not be copied regardless of the data type of `obj`.

The type of return is determined by the type of `obj`.

The operator also treats cycles (structures that directly or indirectly reference to themselves), correctly.

See also: **copyadd**.

copyadd (obj [, ···] [, deepcopy=false] [, inplace=true])

By default, copies all elements in table, set, sequence or register `obj` and any further optional arguments into a new structure and returns it. The result is of the same type as `obj`.

With tables, the array and hash parts are copied 1:1, that is all the keys and entries in the array part of `obj` are copied to the array part of the new table, and the

elements in the hash part of `obj` are copied to the hash part of the new table, with the same keys, too.

Substructures are deep-copied by default. When given the `deepcopy`=**false** option, the function instead creates a new structure, copies all key-value pairs (with tables) or values (with sequences, registers) of its first argument into it - without any deep-copying - and then inserts all the other arguments.

When given the `inplace`=**true** option, then the elements are actually added to `obj`, thus modifying the input structure, as well.

The function may be used when in an expression there is a call to **op**, **unpack** or any other function returning multiple values, which is followed by one or more *subsequent* values: in this case multiple values returned by any function get truncated to the first value if it's not the last trailing expression.

Compare for example:

```
> f := proc(x, y) is
>    return math.sincos(x), y
> end;

> f(0, E):
0       2.718281828459
```

versus

```
> f := proc(x, y) is
>   return op(copyadd([math.sincos(x)], y))
> end;

> f(0, E):
0       1       2.718281828459
```

See also: **append**, **copy**, **include**, **prepend**, **put**, **tables.include**, **union**, **insert** statement.

countitems (item, obj [, option])
countitems (f, obj [, ···])

In the first form, counts the number of occurrences of an `item` in the structure (table, set, sequence, register) `obj`. If you pass any `option`, then with numbers the function will conduct an approximate check, see **approx**. If `obj` is **null**, it is assumed torepresent an empty structure.

In the second form, by passing a function `f` with a Boolean relation as the first argument, all elements in the structure `obj` that satisfy the given relation are counted. If the function has more than one argument, then all arguments *except the first* must be passed right after `obj`.

The return is a number. The function does not invoke metamethods.

See also: **$$$** and **size** operators, **select**, **bags** package, **linalg.countitems**, **numarray.countitems**, **skycrane.obcount**.

descend (f, obj, [, ···] [, option])

Returns all elements in the structure `obj` (a table, set, sequence or register) that satisfy a given Boolean condition expressed by function `f`. The function can be multivariate and must return either **true** or **false**. The optional second and all further arguments of `f` may be passed as the third, etc. argument.

With tables, all the keys and entries are scanned.

With sequences and registers, only the entries (not the keys) are scanned.

The function performs a recursive descent if it detects tables, sets, registers or sequences in `obj` so that it can find elements in deeply nested structures. Pairs, however, are ignored.

If `obj` is a table and the option `skiphash`=**true** has been passed, then the function will ignore all non-numeric keys and their corresponding values, i.e. ignore the hash part of a table.

The function returns a structure with its type depending on the type of `obj` with all the hits in no more than two levels, an example:

```
> s := seq(1, 2, 3, [1, 2, 3], seq(1, 2, 2, 4, {2, 4, 5}));

> descend(<: x -> x = 2 :>, s):
seq(2, [2], seq(2, 2), {2})

> # return all elements greater or equal 3

> ge := proc(x, y) is  # x greater or equal y ?
>    try
>       return x >= y
>    catch  # avoid comparisons of numbers with other data types
>       return false
>    yrt
> end;

> descend(ge, s, 3):
seq(3, [3], seq(4), {4, 5})
```

descend issues an error if `obj` is unassigned.

See also: **has**, **member**, **recurse**, **satisfy**, **select**, **whereis**.

drop (d, r_1 [, ···, r_n])

Drops a number of arguments and returns the remaining. `d` is the number of arguments to drop. r_1 to r_n are the arguments.

If `d` is positive, the function returns r_{d+1} to r_n. This is useful to avoid creation of dummy variables.

if `d` is negative, the function drops the last $|d|$ arguments and returns r_1 to r_{n+d}.

If `d` is 0, arguments r_1 to r_n are returned.

See also: **op**, **ops**, **unpack**.

duplicates (obj [, option])

Returns all the values that are stored more than once to the given table, sequence or register `obj`, and returns them in a new table, sequence or register. Each duplicate will be returned only once. If `option` is not given, the structure is sorted before evaluation since this is needed to determine all duplicates. The original structure is left untouched, however.

If a value of any type is given for `option`, the function assumes that the structure has been already sorted. The values in `obj` should either be strings or numbers if no `option` is given, otherwise the function will fail.

The function is written in Agena and included in the lib/library.agn file.

empty (obj)

This Boolean operator checks whether a table, set, register, sequence or string `obj` does not contain any item and returns **true** if so; otherwise it returns **false**.

See also: **filled**.

error (message [, level])

Terminates the last protected function called and returns `message` as the error message. **error** never returns.

Usually, **error** adds some information about the error position at the beginning of the message. The `level` argument specifies how to get the error position. With level 1 (the default), the error position is where the **error** function was called. Level 2 points the error to where the function that called **error** was called; and so on. Passing a level 0 avoids the addition of error position information to the message.

See also: **argerror**, **assume**, **protect**, **xpcall**.

everyth (n, k)

everyth (obj, k)

In the first form, returns the Agena equivalent `n % k = 0`, a Boolean.

In the second form, returns every given `k`-th element in the table, sequence or register `obj` in a new structure. The type of return is determined by the type of the first argument. With tables, only the array part is traversed.

_G

A global variable (not a function) that holds the global environment (that is, `_G._G = _G`). Agena itself does not use this variable; changing its value does not affect any environment, nor vice-versa. (Use **debug.setfenv** to change environments.)

filled (obj)

This Boolean operator checks whether a table, set, register, sequence or string `obj` contains at least one item and returns **true** if so; otherwise it returns **false**.

See also: **empty**.

freeze (obj)

Write-protects table, set, sequence, register, pair or userdata `obj`. After calling the function, you can still read data from `obj`, but you cannot modify it.

Metatables and user-defined types can also no longer be changed and **getmetatable** will issue an error as it returns a reference into the metatable that would allow tampering its contents.

Note that the function protects a structure from being modified only, it does not prevent a structure - or more exactly its reference - from being completely overwritten or deleted. You can use the **constant** statement to do this in a top-level chunk only, so this won't work in procedures.

Use **unfreeze** to remove write-protection.

See also: **environ.frozen**, **getmetatable**, **gettype**, **setmetatable**, **settype**.

fold (f, obj [, options])

Applies a function `f` on each item of a structure or string `obj` and returns an accumulated result. It works like **reduce** with the `fold=true` option, i.e. the initialiser is always given by the first value or character in `obj`. **fold** supports the options available in **reduce**.

If you need to save memory, the **foreach** operator may be an alternative.

See also: @ operator, **map**, **mulup**, **qmdev**, **qsumup**, **sumup**.

foreach (start, stop [, step], f [, s])

foreach (start, stop [, step], f, n)

foreach (true, start, stop [, step], f, a)

In the first form, the operator traverses a numeric range, starting with `start` (a number) and stopping at `stop` (a number) with step size `step`, applies a univariate function `f` on each intermediate value and puts the result into a given table or sequence `s`. The operator allows to omit the `step` size - defaulting to one - or the structure `s`, in this case returning a table of function values.

For example,

```
> foreach(1, 5, 0.5, <: x -> 2*x :>, seq()):
seq(2, 3, 4, 5, 6, 7, 8, 9, 10)
```

is equivalent to

```
> s, f := seq(),<: x -> 2*x :>;
> for i from 1 to 5 by 0.5 do
>    insert f(i) into s
> end;
```

If the given structure `s` already includes elements, they are not overwritten, and the function values are appended instead.

In the second form, if you pass a number, as the last argument, the operator computes the sum of all function values, with the sum initialised to `n`:

```
> # Pi approximation by Indian mathematician and astronomer
> # Madhava of Sangamagrama, 14th century AD:
> sqrt(12)*foreach(0, 25, <: k -> (-3)^(-k)/(2*k + 1) :>, 0):

3.1415926535898
```

The third flavour resembles the **reduce** function and thus makes the operator more generic and versatile: By passing an interval [`start`, `stop`] with an optional `step` size defaulting to 1, a two-parameter accumulator function `f` and an initialiser `a` of any type, you can create short one-liners that do not consume a lot of memory as there are no structures to be iterated.

Two examples: To compute the tenth factorial 10! = fact(10), issue:

```
> foreach(true, 1, 10, 1, <: x, a -> a*x :>, 1):
3628800
```

In this example, the operator receives the range [1, 10] and optional step size 1. The accumulator function with its first argument x receives the respective iteration value and with its second argument a the accumulator which is initialised to 1, the last argument. The function returns and must return the updated accumulator as its

only result, otherwise an error is thrown. (The value **true** as the first argument to **foreach** is just a flag that switches the operator into this mode.)

In this mode, either real or complex numbers can be processed, that is `f` might be a function returning real or complex numbers and initialiser `a` might be a number or complex number.

To compute a list of the first ten factorials, enter:

```
> foreach(true, 1, 10,
>    proc(x, a) is insert a[size a]*x into a; return a end,
>    [1]):
[1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800]
```

The first value in the return is the initialiser - you can provide a more sophisticated function that omits it from the result. Compared to **reduce** and **fold**, the third form is useful to save a lot of memory for you pass only a left and right border and a step size and not a structure of - often many - individual values.

Generally, in all forms the function uses computationally intensive Kahan-Babuška summation for the iteration control variable only if the start value and/or the step size are fractional.

See also: **fold**, **reduce**, **sumup**, **times**, **calc.fsum**, **stats.sumdata**.

getbit (x, pos)

Checks for the bit at position `pos` ∈ [1, 32] in the integer `x`, and either returns **true** or **false**.

See also: **getbits**, **getnbits**, **setbit**, **setbits**, **setnbits**, **bytes.tobytes**, **numarray.getbit**.

getbits (x [, any])

Returns all 32 bits in the integer `x`, and returns a register of size 32 with values **true** or **false**. If any second argument is given, the register is filled with zeroes or ones instead of booleans.

See also: **getbit**, **getnbits**, **setbit**, **setbits**, **setnbits**, **numarray.getbit**.

getnbits (x, pos, nbits)

From the 32-bit integer `x`, starting from bit position `pos` from the right, retrieves `nbits` bits and returns a decimal value. `pos` should be in [1, 32].

getentry (obj [, k_1, ···, k_n])

Returns the entry obj[k_1, ···, k_n] from the table, sequence or register obj without issuing an error if one of the given indices k_i (second to last argument) does not exist. It conducts a raw access and thus does not invoke any metamethods.

If obj[k_1, ···, k_n] does not exist, **null** will be returned. If only obj is given, it is simply returned.

See also: **..** operator, **{}** indexing (Chapter 4.9.1), **getorset**.

getmetatable (obj [, true])

If obj does not have a metatable, returns **null**. Otherwise, if the obj's metatable has a '__metatable' field, returns the associated value. Otherwise, returns the metatable of the given obj, a table, set, pair, sequence, register, userdata or procedure.

The function issues an error if obj is read-only, see **freeze**, **unfreeze**.

If obj is a string and the Boolean value **true** is given as a second argument, then the function retrieves a metatable directly from the Agena registry, instead of doing so by querying a certain individual value, such as a structure or userdata. Example:

```
> getmetatable('numarray', true):
[__add ~ procedure(01D6FDF8), __aeq ~ procedure(01D64150), ...]
```

With the **true** option, if the given metatable does not exist or is frozen, then the function just returns **null**.

See also: **addtometatable**, **setmetatable**, **registry.get**.

getorset (obj, k_1, ···, k_n, v)

Returns the non-**null** element at index obj[k_1, k_2, ..., k_n], where obj is a table, sequence or register. If any index position is invalid, the function returns **null**.

If obj[k_1, k_2, ..., k_n] = **null**, then the function will assign obj[k_1, k_2, ..., k_n] := v and return v.

See also: **getentry**.

gettype (obj)

Returns the type - set with **settype** - of a function, sequence, set, pair or userdata obj as a string. If no user-defined type has been set, or any other data type has been passed, **null** will be returned.

See also: **settype**, **typeof**.

has (obj, x)

Checks whether the structure `obj` (a table, set, sequence, register or pair) contains element `x`.

If `obj` and `x` are strings, checks whether at least one character in `obj` matches one of the characters in `x`.

With tables, all the entries are scanned. If `x` is not a number then the indices of the table are searched, too.

With sequences and registers, only the entries (not the keys) are scanned. With pairs, both the left and the right item is scanned. The function performs a deep scan so that it can find elements in deeply nested structures.

The function returns **true** if `x` could be found in `obj`, and **false** otherwise. If `obj <> x` and if `obj` is a number, boolean, complex number, string, procedure, thread, userdata or lightuserdata, **has** returns **fail**. Examples:

```
> has([10, 20, 30], 30):
true

> has([10, 20, 30], 40):
false

> has('az', 'abcdefg'):
true

> has('xz', 'abcdefg'):
false
```

See also: **descend**, **in**, **member**, **notin**, **recurse**, **satisfy**, **whereis**.

identity (···)

Returns its arguments. If the argument is a function call that returns nothing, the function opposed to the **unity** operator returns nothing, as well. See also: **unity** operator, **unpack**.

include (obj, x [, ···])

Inserts one or more values `x`, `···` to the end of structure `obj`, not discarding multiple returns if its last argument is a function call. Note that the **insert** statement ignores all but the first return when given a function call:

```
> a := []; f := <: () -> 1, 2, 3 :>;

> insert f() into a;
```

```
> a:
[1]
```

include, however, does not:

```
> a := [];

> include(a, f()):
[1, 2, 3]
```

See also: **copyadd**, **append**, **prepend**, **put**, **insert** statement.

initialise (packagename [, false])

initialise (packagename, key1, key2, ··· [, false])

Assigns short names to package procedures such that:

```
name := packagename.name
```

The function works as follows:

- In both forms, **initialise** first tries to load and run the respective Agena package. The package may reside in a text file with file suffix `.agn`, or in a C dynamic link library with file suffix `.so` in UNIX and `.dll` in Windows, or both in a text file and in a dynamic link library. The function first tries to find the package in the current working directory and if it failed, in the path pointed to by mainlibname; if this fails, too, it traverses all paths in **libname** from left to right until it finds at least the C DLL or the Agena text file, or both. If a package consists of both the C DLL and an Agena text file, then they both must reside in the same folder.

- If the function does not find the package, an error will be returned.

- Next, **initialise** tries to find a package initialisation procedure. If a procedure named `` `packagename.init` `` is present in your package then it is executed if the package has been found successfully.

- In the first form, if only the string `packagename` is given, short names to all functions residing in the global table `packagename` are created.

  If you do not want **initialise** to assign short names for certain functions, their names should be in the format `packagename.aux.procedurename`, e.g. math.aux.errormessage.

- Note that if `packagename.name` is not of type procedure, a short name is not created for this object.

- If you would like to display a welcome message, put it into the string `packagename.initstring`. It is displayed with an empty line before and after the

text. An example:

```
agenapackage.initstring := 'agenapackage v0.1 for Agena as of \
May 23, 1949\n';
```

- In the second form, you may specify which short names are to be assigned by passing them as further arguments in the form of strings. Contrary to the first form, short names are also created for tables stored to table `packagename`.

  As opposed to the first version, **initialise** does not print any short names or welcome messages on screen.

- Further information regarding both forms:

  The function returns a table of all short names assigned.

  If the global environment variable **environ.withverbose** is set to **false**, no messages are displayed on screen except in case of errors. If it is set to any other value or **null**, a list of all the short names loaded and a welcome message is printed.

  If a short name has already been assigned, a warning message is printed. If a short name is protected (see table **environ.withprotected**), it cannot be overwritten by **initialise** and a proper message is displayed on screen. You can control which names are protected by modifying the contents of **environ.withprotected**.

  For information on which folders are checked and how to add new directories to be searched by **initialise**, see **readlib**.

  Note that **initialise** executes any statements (and thus also any assignment) included in the file `packagename`.agn.

The function is written in Agena and included in the lib/library.agn file.

If the last argument is the Boolean **false**, **initialise** does not print the assigned shortcuts at the console.

Note: the **import**/**alias** statement is an interface to the **initialise** function but does not require package names to be put into quotes. For example,

```
> initialise 'clock';
```

is equivalent to

```
> import clock alias;
```

See also: **readlib**, **run**, **register**, and **import/alias** statement.

ipairs (obj)

Returns three values: an iterator function, the table, sequence, register, string or userdata `obj`, and 0, so that the construction

```
for i, v in ipairs(obj) do body od
```

will iterate over the pairs (1, `obj`[1]), (2, `obj`[2]), ⋯, up to the first integer key absent from the data structure.

If you pass userdata, for example a numarray, it must feature a metatable with an '__index' metamethod. Otherwise an error will be issued.

If there is nothing more to iterate, the iterator returns **null**s.

See **nextone** for the caveats of modifying the table during its traversal; and also: **pairs**, **factory.iterate**, **skycrane.iterate**.

Example:

```
> d := numarray.double(3)

> d[1] := Pi; d[2] := 2*Pi; d[3] := 3*Pi;

> f := ipairs(d):
procedure(00410A30)

> idx, val := f(d, 0):    # pass 0 to start the iteration
1       3.1415926535898

> idx, val := f(d, idx):
2       6.2831853071796

> idx, val := f(d, idx):
3       9.4247779607694

> idx, val := f(d, idx):  # nothing left
null    null
```

isall (obj, type)

Checks whether all elements in table, set, sequence or register `obj` are of a given `type` and returns **true** or **false**. Eligible types that the function accepts are `'number'`, `'integer'` (numbers that are all integral), `'complex'`, `'string'` and `'boolean'`. Also supported are `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonzeroint'` (non-zero integers), `'nonnegative'` (non-negative numbers), and `'numeric'` (numbers and complex numbers).

The function is at least fifteen times faster than checking structures with the **satisfy** function.

See also: **sequences.isall**, **sets.isall**, **registers.isall**, **tables.isall**.

isboolean (···)

Checks whether the given arguments are all of type **boolean** and returns **true** or **false**.

iscomplex (···)

Checks whether the given arguments are all of type **complex** and returns **true** or **false**.

isequal (obj1, obj2)

Equivalent to `obj1 = obj2` and returns **true** or **false**.

The function is written in Agena and included in the lib/library.agn file.

isint (···)

Checks whether all of the given arguments are integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **finite**, **fractional**.

isnegative (···)

Checks whether all of its arguments are negative numbers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnegint**, **isposint**, **isnonneg**, **ispositive**.

isnegint (···)

Checks whether all of the given arguments are negative integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnonneg (···)

Checks whether all of its arguments are zero or positive numbers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnegint**, **isposint**, **isnegative**, **ispositive**.

isnonnegint (···)

Checks whether all of the given arguments are zeros or positive integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnonposint (···)

Checks whether all of the given arguments are zeros or negative integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnonzeroint (···)

Checks whether all of the given arguments are non-zero integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnumber (···)

Checks whether the given arguments are all of type **number** and returns **true** or **false**.

isnumeric (···)

Checks whether the given arguments are all of type **number** or of type **complex** and returns **true** or **false**.

ispair (···)

Checks whether the given arguments are all type **pair** and returns **true** or **false**.

isposint (···)

Checks whether all of its arguments are positive integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnonposint**.

ispositive (···)

Checks whether all of its arguments are positive numbers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: i**snonposint**, **isposint**, **isnegative**, **isnonneg**.

isreg (···)

Checks whether all of its arguments are of type **register** and returns **true** or **false**.

isseq (···)

Checks whether all of its arguments are of type **sequence** and returns **true** or **false**.

isstring (···)

Checks whether all of its arguments are of type **string** and returns **true** or **false**.

isstructure (···)

Checks whether all of its arguments are of type **table**, **set**, **sequence** or **pair** and returns **true** or **false**.

istable (···)

Checks whether all of its arguments are of type **table** and returns **true** or **false**.

left (obj)

With the pair `obj`, the operator returns its left operand. This is equals to `obj[1]`.

See also: **right**.

load (f [, chunkname])

Loads a chunk using function `f` to get its pieces. Each call to `f` must return a string that concatenates with previous results. A return of **null** (or no value) signals the end of the chunk.

If there are no errors, returns the compiled chunk as a function; otherwise, returns **null** plus the error message.

The environment of the returned function is the global environment.

`chunkname` is used as the chunk name for error messages and debug information.

loadfile ([filename])

Similar to **load**, but gets the chunk from file `filename` or from standard input, if no file name is given.

loadstring (s [, chunkname])

Similar to **load**, but gets the chunk from the given string `s`. To load and run a given string, use the idiom

```
assume(loadstring(s))(···)
```

Examples:

```
> f := loadstring('a := exp(1)');

> f():

> a:
2.718281828459

> f := loadstring('local x := exp(1); return x')();

> f:
2.718281828459

> f := loadstring('return exp(1)')():
2.718281828459
```

See also: **strings.dump**.

map (f, obj [, ···] [, inplace=true])

map (f, g)

In the first form, the **map** function maps a function `f` to all the values in table, set, sequence, register, string or pair `obj`. `f` usually should return only one value (but see below). The type of return is the same as of `obj`. If `obj` has metamethods or user-defined types, the return will also have them.

If `obj` is a string, `f` is applied on all of its characters from the left to right. The return is a sequence of function values. If `obj` is **null**, the function returns **null**.

If function `f` has only one argument, then only the function and the structure `obj` will be passed to **map**. If the function has more than one argument, then all arguments *except the first* will be passed right after `obj`.

Examples:

```
> map(<: x -> x^2 :>, [1, 2, 3] ):
[1, 4, 9]

> map(<: (x, y) -> x > y :>, [-1, 0, 1], 0 ):  # 0 for y
[false, false, true]
```

If the last argument is the option `inplace=true`, then the operation will be done in-place, modifying the original structure, but saving memory. After completion, the function will return the modified structure.

If you pass a function that does not return anything, the return is an empty structure.

If you pass a function that returns multiple values, then by default all but the first result will be ignored. You can override this behaviour by passing the `multret=true` option. In this case - with tables, sequences, registers and strings only - each element in `obj` is replaced by *all* the return values. In case of tables, all the values are put into a table array with positive integral keys starting from 1; compare:

```
> map(<: x -> 2*x, 0 :>, [1, 2, 3]):
[2, 4, 6]

> map(<: x -> 2*x, 0 :>, [1, 2, 3], multret=true):
[2, 0, 4, 0, 6, 0]
```

With tables only, you can also pass the `reshuffle=true` option. In this case, all holes will be removed from the result before returning it. Just an example:

```
> a[3] := null;  # we create a hole

> a:
[1 ~ 1, 2 ~ 2, 4 ~ 4, 5 ~ 5]
```

With the following statement, there will be no reordering and index 3 is still not assigned a value:

```
> map(<: x -> 2*x :>, a):
[1 ~ 2, 2 ~ 4, 4 ~ 8, 5 ~ 10]
```

With the `reshuffle` option, all holes will be removed and the return is a concise table array with consecutive positive integral indices:

```
> map(<: x -> 2*x :>, a, reshuffle=true):
[2, 4, 8, 10]
```

With (deeply) nested tables, sequences, registers, sets and pairs, **map** can recursively descent into the entire structure and map a function with only just one call, with the `descend=true` option:

```
> map(<: x -> 2*x :>, [1, 2, 3, [4, 5, [6]]], descend=true):
[2, 4, 6, [8, 10, [12]]]
```

With this option, only the first return of `f` will be processed.

In the second form, the function creates the composition of two functions `f @ g` = `f(g(x))` and returns it is a new function (`f @ g`)(x). `f` and `g` may be univariate or multivariate and also return multiple results.

Example:

```
> # first take the root, then negate

> h := map(<: x -> -x :>, <: x -> sqrt x :>) # which is equivalent to:

> h := <: x -> -x :> @ <: x -> sqrt x :>     # which results to:

> h(2):
-1.4142135623731
```

See also: @ operator, **iterate**, **sequences.new**, **registers.new**, **pipeline**, **reduce**, **remove**, **select**, **subs**, **subsop**, **times**, **zip**.

max (obj [, 'sorted'])

max (x, y)

In the first form, returns the maximum of all numeric values in table, set, sequence or register `obj`. If the option `'sorted'` is passed than the function assumes that all values in `obj` are sorted in ascending order and returns the last entry. The function in general returns **null** if it receives an empty table or sequence.

In the second form, the function returns the largest of the two numbers `x` and `y`.

See also: **min**, **math.max**, **stats.max**, **stats.min**, **stats.minmax**.

member (x, obj [, option])

Searches `x` in the table, sequence or register `obj` and if successful returns the index of the first hit, otherwise returns **null**. The function is much faster than **whereis** if you need the index of the first hit only. Note that with respect to **whereis**, the parameters are in reverse order. With `obj` a set, returns **true** if `x` is a member, and **null** otherwise.

Examples:

```
> member(30, [10, 20, 30]):
3

> member(40, [10, 20, 30]):
null
```

If `x` and `obj` are both strings, checks whether all characters in string `x` are part of the characters in string `obj` - the `alphabet` - and returns **true** or **null** (at least one character in `x` is not in `obj`). If `obj` is the empty string, the function returns **null**, as well.

```
> member('agena', 'abcdefghijklmn'):
true

> member('agenax', 'abcdefghijklmn'):
null
```

If you pass any `option`, then with `x` a number the function will conduct an approximate equality check: if `option` is a positive number, it uses it as the maximum tolerance, if `option` is not a positive number, the tolerance will be **Eps**. For more information, see **approx**.

See also: **descend**, **has**, **in**, **notin**, **numarray.member**, **recurse**, **satisfy**, **strings.contains**, **tables.entries**, **tables.indices**, **whereis**.

min (obj [, 'sorted'])

min (x, y)

In the first form, returns the minimum of all numeric values in table, set, sequence or register `obj`. If the option `'sorted'` is passed than the function assumes that all

values in `obj` are sorted in ascending order and returns the first entry. The function in general returns **null** if it receives an empty table or sequence.

In the second form, the function returns the smallest of the two numbers `x` and `y`.

See also: **max**, **math.min**, **stats.max**, **stats.minmax**, **stats.min**.

move (obj1, start, stop, newidx [, obj2])

Copies elements from the table, sequence, register or userdata `obj1` to table, sequence, register or userdata `obj2`, performing the equivalent to the following multiple assignment: `obj2[newidx]`, ··· = `obj1[start]`, ···, `obj1[stop]`. The default for `obj2` is `obj1`, i.e. elements are shifted in the same structure. The destination range can overlap with the source range. `obj1` and `obj2` must be of the same type.

The function returns destination structure `obj2`, which might be simply `obj1`.

Example: The following statement copies four elements in table a from position 3 up to and including 6 to a new table b, starting with index 1:

```
> a := seq('a', 'b', 'c', 'd', 'e', 'f', 'g', 'h');

> b := move(a, 3, 6, 1, seq());

> b:
seq(c, d, e, f)
```

The next statement copies four elements in a to its beginning:

```
> move(a, 3, 6, 1);

> a:
seq(c, d, e, f, e, f, g, h)
```

The function is implemented in the Agena language and is included in the lib/library.agn file.

See also: **purge**, **shift**, **swap**, **tables.move**.

mprint (···)

Similar to **print** but centres the output in the console. Besides the `delim`, `enclose` and `nonewline` options known from **print**, the function also accepts the `simple` option: when set to **true**, structures are not pretty-printed, when set to **false** (default), the whole contents is output.

mulup (obj)

mulup (f, obj [, ···])

mulup (f, start, stop [, step [, ···]])

In the first form, the operator multiplies all numeric values in table, sequence, register or userdata `obj`, using round-off error correction. The return is a number. If `obj` is empty or consists entirely of non-numbers, **fail** will be returned. If the structure contains numbers and other objects, only the numbers are multiplied. With tables, only numeric entries with integral keys will be processed.

In the second form, you pass a function `f` as the first argument and a structure `obj` as the second argument: if `f` returns a number then it is applied on each value in `obj` before multiplication. If `f` returns the Boolean value **true**, then only the values in the structure that satisfy the given condition are multiplied. You can put any second, etc. arguments to `f` right after `obj`.

In the third form, the operator computes the product

$$\prod_{k=\text{start}}^{\text{stop}} f(k, \ldots)$$

Just pass the function `f` representing the product plus the `start` and `stop` value and optionally a `step` size which defaults to one. if `start` > `stop`, the result will be 1.

Multivariate functions are supported - just pass their second, third, etc. arguments right after `step`, so with multivariate function you must always give the step size which should usually be one.

See also: **fact**, **foreach**, **sumup**, **calc.fprod**, **math.fall**, **math.lnfact**, **math.pochhammer**.

nextone (obj [, index [, sentinel]])

Allows a programme to traverse all fields of a table or all items of a set, sequence or register `obj`. With strings, it iterates all its characters. Its first argument is a table, set, string or sequence and its second argument is an index in the structure.

With tables, sequences or registers, **nextone** returns the next index of the structure and its associated value. When called with **null** as its second argument, **nextone** returns an initial index and its associated value. When called with the last index, or with **null** in an empty structure, **nextone** returns **null**.

With sets, **nextone** returns the next item of the set twice. When called with **null** as its second argument, **nextone** returns the initial item twice. When called with the last index, or with **null** in an empty set, **nextone** returns **null**.

With strings, **nextone** returns the position of the respective character (a positive integer) and the character. When called with **null** as its second argument, **nextone** returns the first character. When called with the last index, **nextone** returns **null**.

If the second argument is absent, then it will be interpreted as **null**. In particular, you can use `nextone(t)` to check whether a table or set is empty. However, it is recommended to use the **filled** operator for this purpose.

If the third optional argument `sentinel` is given, and if **nextone** during traversal encounters an element that equals this `sentinel`, the function just returns **null**, and you may start iterating the structure again from its beginning.

With tables, the order in which the indices are enumerated is not specified, *even for numeric indices.* The same applies to set items.

The behaviour of **nextone** is undefined if, during the traversal, you assign any value to a non-existent field in the structure. With tables, you may however modify existing fields. In particular, you may clear existing table fields.

See also: **factory.iterate**, **factory.cycle**, **long.nextone**, **skycrane.iterate**.

op (obj)

op (i, obj [, options])

op (i:j, obj [, options])

op (t, obj [, options])

The function returns one or more elements from a table, set, pair, sequence or register. If `obj` is **null**, the function returns **null**.

In the first form, **op** returns all the values in table, set, pair, sequence or register `obj` and thus works exactly as **unpack** does. With complex numbers, returns both its real and imaginary part. With all other data types, returns just its argument.

```
> op([10, 20, 30]):
10      20      30
```

In the second form, by specifying an integer as the index `i`, **op** returns member `obj`[`i`] from table, sequence, register or pair `obj`.

```
> op(2, ['a', 'b', 'c']):
b
```

In this form, if `i` is negative integer or zero, then by default the type of `obj` is returned and additionally - if existing - the user-defined type. In this mode, `obj` can be of any type, that is it can also be a number, string, boolean, userdata, etc. Specifically, with `obj` a number, the return is either `'integer'` (`obj` is integral) or `'number'` (`obj` is fractional).

```
> tbl := [-1~'-a', 0~'zero', 'a', 'b', 'c'];
```

```
> settype(tbl, 'triple');

> op(0, tbl):
table   triple
```

If you do not want this and want to access an element with an existing zero key, then pass the `gettype`=**false** as the last argument in the call:

```
> op(0, tbl, gettype=false):
zero
```

If you want to retrieve a member with a negative key, then pass the `gettype`=**false** along with the `posrelat`=**false** options as the last two arguments.

```
> op(-1, tbl, gettype=false, posrelat=false):
-a
```

Or, for short, to switch off all the defaults, pass **true** as the last argument in the call, not only in this form, but also in the third and fourth.

```
> op(-1, tbl, true):
-a
```

In the third form, by passing a range `i`:`j`, with `i` and `j` integers, with `i <= j`, returns members `obj`[`i`] to `obj`[`j`] from table, sequence or register `obj`.

```
> op(1:3, tbl):
a       b       c
```

In the fourth form, by receiving a table t of one or more integral indices `i`, `j`, ... returns all the members `obj`[`i`], `obj`[`j`], ... in table, sequence or register `obj`.

```
> op([3, 1, 0], tbl):
c       a       zero
```

With all forms, if a member does not exist, the function returns **null** in the result.

```
> op([4, 3, 1, 0], tbl):
null    c       a       zero
```

By default, in the second, third and fourth form, if you pass a negative index `i`, it represents the |`i`|-th element `from the right side` of `obj`, that is `obj`[**size**(`obj`) + `i` + 1], see **utils.posrelat** for details. If you want to access an element that has a zero or negative integral index - relates to tables only -, then pass the `posrelat`=**false** option as the last argument in the call, or just pass **true**.

```
> op([3, 0, -1, 1], tbl, true):
c       zero    -a      a
```

The function with all its defaults emulates Maple's op() function as much as possible with the exception that if an index is out-of-bounds, Agena will return **null** instead of an error.

The function has been introduced to facilitate porting code from Maple to Agena. In general, access to structure elements by indexing is much faster:

```
> tbl[1]:
a

> tbl[-1]:
-a

> tbl[2 to 3]:
[2 ~ b, 3 ~ c]
```

See also: **drop**, **ops**, **unpack**, **values**.

ops (index, ···)

ops (obj, ···)

In the first form, if `index` is a number, returns all arguments after argument number `index`. Otherwise, `index` must be the string `'#'`, and **ops** returns the total number of extra arguments it received. The function is useful for accessing multiple returns, like:

```
> f := <: () -> 10, 20, 30, 40 :>

> ops(2, f()):
20       30       40
```

If you want to obtain only the element at `index`, put the call to **ops** in brackets.

```
> (ops(2, f())):
20
```

In the second form, the index positions (integers) in table, sequence or register `obj` specify the values to be returned after the first argument to **ops**.

```
> ops([2, 4], f()):
20      40
```

See also: **columns**, **drop**, **identity**, **op**, **unity**, **unpack**, **values**.

optboolean (x, y [, idx [, procname]])

The function checks whether `x` is a Boolean and in this case returns `x`. If `x` is **null**, it returns the Boolean `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name is printed as the function issuing the error.

optcomplex (x, y [, idx [, procname]])

The function checks whether `x` is a number or complex number and in this case returns `x`. If `x` is **null** it returns the number or complex number `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the

position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name is printed as the function issuing the error.

`optint (x, y [, idx [, procname]])`

The function checks whether `x` is an integer and in this case returns `x`. If `x` is **null** it returns the integer `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name is printed as the function issuing the error.

`optnonnegative (x, y [, idx [, procname]])`

The function checks whether `x` is a non-negative number and in this case returns `x`. If `x` is **null** it returns the non-negative number `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name is printed as the function issuing the error.

See also: **optpositive**, **optnumber**.

`optnonnegint (x, y [, idx [, procname]])`

The function checks whether `x` is a non-negative integer and in this case returns `x`. If `x` is **null** it returns the non-negative integer `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name will be printed as the function issuing the error.

See also: **optint**, **optnonzeroint**, **optposint**.

`optnonzeroint (x, y [, idx [, procname]])`

The function checks whether `x` is a non-zero integer and in this case returns `x`. If `x` is **null** it returns the non-zero integer `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name will be printed as the function issuing the error.

See also: **optint**, **optnonnegint**, **optposint**.

`optnumber (x, y [, idx [, procname]])`

The function checks whether `x` is a number and in this case returns `x`. If `x` is **null** it returns the number `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name will be printed as the function issuing the error.

See also: **optpositive**, **optnonnegative**.

optposint (x, y [, idx [, procname]])

The function checks whether `x` is a positive integer and in this case returns `x`. If `x` is **null** it returns the positive integer `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name will be printed as the function issuing the error.

See also: **optint**, **optnonnegint**, **optnonzeroint**.

optpositive (x, y [, idx [, procname]])

The function checks whether `x` is a positive number and in this case returns `x`. If `x` is **null** it returns the positive number `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name will be printed as the function issuing the error.

See also: **optnonnegative**, **optnumber**.

optstring (x, y [, idx [, procname]])

The function checks whether `x` is a string and in this case returns `x`. If `x` is **null** it returns the string `y`, otherwise the function issues an error. If the third argument `idx`, a number, is given, then the position `idx` will be returned in error messages. If the fourth argument `procname` is given, this name will be printed as the function issuing the error.

pack (obj [, newsize])

When called with no second argument, reduces the number of allocated slots in table, sequence or register `obj` to the number of slots actually assigned a value, also freeing memory if possible.

With tables removes holes in the array part and closes the space by shifting down the other elements, if necessary.

If the optional argument `newsize`, a non-negative integer, is given, then the number of pre-allocated slots is reduced to the number of actually assigned values, purging all surplus elements that may exist and giving back memory to the interpreter. With `newsize` given, the function also removes all trailing **null** of `obj` is a register.

The function works in-place and returns nothing. It automatically conducts a garbage collection before returning.

See also: **cleanse**.

pairs (obj)

Returns three values: the **nextone** function, the table `obj`, and **null**, so that the construction

```
for k, v in pairs(obj) do body od
```

will iterate over all key~value pairs or values of table `obj`.

See **nextone** for the caveats of modifying the table during its traversal; and also: **ipairs**, **factory.iterate**, **skycrane.iterate**.

pipeline (f [, ···], obj [, ···])

Maps one or more functions `f`, etc. - from left to right - on a table, set, sequence, register or userdata `obj`, avoiding multiple internal copies of the structure if possible.

If given a userdata `obj`, the function will change its values in-place, whereas with tables, sets, sequences and registers, the original structure `obj` will not be modified.

The return is a new structure, depending on the type of `obj`. If the function has more than one argument, then all arguments except the first will be passed right after the name of object `obj`.

See also: **map**, @ operator.

prepend (x, obj)

Prepends `x` to the beginning of structure `obj`, in-place. The function returns the modified structure.

The new object can always be found at index 1, all other elements have been shifted up one index into open space. With a table, its hash part is not modified.

With a register, the function automatically increases its size by one. If `obj` is a pair, returns `x` : `obj`.

See also: **append**, **copyadd**, **include**, **put**, **insert** statement.

print (··· [, options])

Receives any number of arguments, and prints their values to the console, using the **tostring** function to convert them to strings. **print** is not intended for formatted output, but only as a quick way to show a value, typically for debugging. For formatted output, use **strings.format**.

In Agena, **print** also prints the *contents* of tables and nested tables to stdout if no `__tostring` metamethods are assigned to them. The same applies to sets and sequences.

If the option `'delim':`<string> is given as the last argument, then **print** will separate multiple values with the given <string> delimiter, otherwise `'\t'` is used. If the option `'nonewline':true` is passed, then Agena will not print a final newline when finishing output. The `'enclose':`<string> option will enclose the values in a given substring. All other types will not be enclosed. All options can be combined.

If the kernel setting `environ.kernel('longtable')` is set to **true**, then each key~value pair will be printed on a separate line, and Agena halts after **environ.more** number of lines for the user to press any key for further output. Press 'q', 'Q' or the Escape key to quit. The default for **environ.more** is 40 lines, but you may change this value in the Agena session or in the Agena initialisation file.
You may change the way **print** formats objects by changing the respective **environ.aux.print*** functions in the lib/library.agn file. See Appendix A5 for further details.

See also: **mprint**, **printf**, **io.write**, **io.writeline**, **skycrane.scribe**, **skycrane.tee**.

`printf ([fh,] template, ··· [, options])`

If the first argument `fh` is not given, prints the optional arguments under the control of the template string `template` to stdout, else it writes to the open file denoted by its file handle `fh`. See **strings.format** for information on how to create the template string. Use the `\n` escape sequence to include a line break.

Example:

```
> printf('%-10s %3d %10.2f\n', 'Carbon', 6, 12.0107);
Carbon       6      12.01

> fh := io.open('file.txt', 'w');

> printf(fh, '%-10s %3d %10.2f\n', 'Carbon', 6, 12.0107);

> close(fh);
```

You must not necessarily pass a formatting string:

```
> printf('abc')
abc
```

This way, if the string to be printed includes any formatter such as `%10.2f`, **printf** will complain. In this case, do something like this:

```
> printf('%a', '%s')
%s
```

In the console, that is **io.stdout**, the function supports colour printing. Pass at least the `color` or `colour` option for the foreground colour, or at least the `background` option for the background colour. You may also set both the foreground and background colour.

Supported colours, all to be passed as strings, are: `'black'` (the default), `'white'`, `'red'`, `'green'`, `'blue'`, `'cyan'`, `'gray'`, `'grey'`, `'magenta'`, `'yellow'`. On some platforms, colours might look different.

Examples:

```
> printf('abc', colour = 'red')
```

```
> printf('abc', colour = 'red', background = 'white')
```

You can pass following options, none, one or more, but set them all to **true**, see examples below:

- `blink`: text will slowly blink (if supported in your shell),
- `bold`: print text in bold letters, in some shells the test will be output in a brighter shade of the same colour instead,
- `dark`: dim text,
- `inverse`: see `reverse`,
- `italic`: italic text,
- `native`: Windows only, use native Win32 C API for colouring, switched off by default. Might be needed in older Windows 10 flavours and in Windows 8.1 or earlier, ignored by all other platforms,
- `reverse`: invert foreground and background colour,
- `underline`: underline text,
- `reset`: all attributes become turned off; not needed as **printf** switches back to normal after returning.

Example:

```
> printf('abc', colour = red, bold = true, blink = true, native = true)
```

See also: **mprint**, **print**, **io.write**, **io.writeline**, **skycrane.scribe**, **skycrane.tee**.

`protect (f, arg1, ···)`

Calls function `f` with the given arguments in *protected mode.* This means that any error inside `f` is not propagated; instead, **protect** simply catches the error.

The function either returns all results from the call in case there have been no errors, or returns the error message as a string as the only return. In case of an error, the error message is set to the global variable **lasterror**, otherwise **lasterror** is set to **null**.

lasterror is useful to see whether the call to **protect** resulted in an error:

```
if protect(···) = lasterror then (your error treatment here) fi
```

See also: **argerror**, **assume**, **error**, **xpcall**, **try/catch** statement.

purge (obj [, pos])

purge (obj, a, b)

In the first form, the function removes from table, sequence or register `obj` the element at position `pos`, shifting down other elements to close the space, if necessary. It returns the value of the removed element. The default value for `pos` is n, where n is the length of the table, sequence or register, so that a call `purge(obj)` removes the last element of `obj`. With tables, the function only removes an element from the array part.

In the second form, removes all elements starting from index `a` to index `b` (inclusive), moving excess elements down to close the space; the function automatically performs a garbage collection after shifting. In the 2nd form, nothing will be returned.

Use the **delete** *element* **from** *structure* statement if you want to remove any occurrence of the table value *element* from a table or sequence. You might also consider using a linked list, a data structure which supports much faster operations when inserting or deleting elements, see **llist** package in Chapter 10.7.

Note that with tables, the function only works if the table is an array, i.e. if it has positive integral and consecutive keys only. With registers, the top pointer is reduced by the number of elements removed.

See also: **append**, **move**, **prepend**, **put**, **llist.purge**, **tables.remove**, **ulist.purge**.

put (obj, [pos,] value)

Inserts element `value` at position `pos` in table, sequence or register `obj`, shifting up other elements to open space, if necessary. The default value for `pos` is n+1, where n is the current length of the structure, so that a call `put(obj, value)` inserts `value` at the end of `obj`.

Use the **insert** *element* **into** *structure* statement if you want to add an element at the current end of a structure, for it is much faster. You might also consider using a linked list, a data structure which supports much faster operations when inserting or deleting elements, see **llist** package in Chapter 10.7.

The function returns the modified structure.

See also: **append**, **binclude**, **prepend**, **purge**, **llist.put**, **ulist.put**.

qsumup (obj [, d])

Raises all numbers or complex numbers - or a mix of them - in table, sequence, register or userdata `obj` to the power of 2 and sums up these powers, using a precision-saving method. The return is a number. If `obj` is empty or consists entirely of non-numbers, the operator returns **fail**.

If the structure contains numbers and other objects, only the powers of the numbers are added. With tables, only numeric entries with integral keys will be processed.

If the number or complex number `d` is given, the operator divides each square by `d` before summation takes place, in a way that numeric overflow is avoided.

The operator uses a combination of fused multiply-add and Kahan-Babuška Summation. To improve accuracy, you may sort `obj` before.

See also: **foreach**, **qmdev**, **reduce**, **sort**, **sorted**, **sumup**, **stats.issorted**.

rawequal (obj1, obj2)

Checks whether `obj1` is equal to `obj2`, without invoking any metamethod. Returns a Boolean.

rawget (obj, index)

Gets the real value of `obj[index]`, without invoking any metamethod. `obj` must be a table, set, sequence or pair; `index` may be any value. To get the left element of a pair pass 1 for `index`, for the right element pass 2.

See also: **getentry**, **rawgeti**, **rawset**.

rawgeti (obj, idx)

Gets the real value of `obj[idx]`, without invoking any metamethod. `obj` must be a table, set, sequence or pair; `idx` must be an integer. To get the left element of a pair pass 1 for `idx`, for the right element pass 2.

See also: **rawget**.

rawset (obj, index, value)

rawset (obj, value)

In the first form, sets the real value of `obj[index]` to `value`, without invoking any metamethod. `obj` must be a table, set, register, sequence or pair, `index` any value different from **null**, and `value` any value. To delete a value from any structure, pass **null** for `value`.

In the second form, the function inserts `value` into the next free position in the given structure `obj`. `obj` can be a table, set, sequence or register.

This function returns `obj`.

See also: **rawget**, **rawgeti**, **rawseti**.

`rawseti (obj, idx, value)`

Sets the real value of `obj[idx]` to `value`, without invoking any metamethod. `obj` must be a table, set, register, sequence or pair, and `idx` an integer. To delete a value from any structure, pass **null** for `value`. This function returns `obj` and is 20 % faster than **rawset**.

See also: **rawget**, **rawgeti**.

`read (filename)`

Reads an object stored in the binary file created by **save** and denoted by file name `filename` and returns it. The function is written in Agena and included in the lib/library.agn file.

`readlib (packagename [, packagename2, ···] [, true])`

Loads and runs packages stored to agn text files (with filename `packagename`.agn) or binary C libraries (`packagename`.so in UNIX, `packagename`.dll in Windows), or to both.

If **true** is given as the last argument, the function prints the search path(s). For debugging information use **package.loadclib**, or set:

```
> environ.kernel(debug = true)
```

The function first tries to find the libraries in the current working directory, and thereafter in the path in **mainlibname**. If it fails, it traverses all paths in **libname** until it finds them. If it finds a library and the current user has at least read permissions for it, it is initialised. On successful initialisation, the name of the package is entered into the **package.readlibbed** set.

Note that if a package consists both of a C DLL and an Agena text file, they should both be located in the very same folder as **readlib** does not search for them across multiple paths and may thus initialise a package only partially.

Make sure that on the operating system level the environment variable AGENAPATH has been set, that the individual paths are separated by semicolons and that they do not end in slashes. In UNIX, if AGENAPATH has not been set, **readlib** by default searches in `/usr/agena/lib`.

In OS/2 and Windows, the Agena installation programme automatically sets AGENAPATH. If it failed, or you want to modify its contents, you may manually set the variable like in the following examples, assuming that the Agena libraries are located in the `d:\agena\lib` folder and optionally in the `d:\agena\mypackage` folder.

```
SET AGENAPATH=d:/agena/lib  or
SET AGENAPATH=d:/agena/lib;d:/agena/mypackage
```

In UNIX, you may execute one of the following statements in your shell, assuming that the Agena libraries are located in the `/home/usr/agena/lib` folder and optionally in the `/home/usr/agena/mypackage` folder.

```
SET AGENAPATH=/home/usr/agena/lib  or
SET AGENAPATH=/home/usr/agena/lib;/home/usr/agena/mypackage
```

In DOS, you have to set AGENAPATH in the autoexec.bat file:

```
SET AGENAPATH=d:/agena/lib  or
SET AGENAPATH=d:/agena/lib;d:/agena/mypackage
```

Of course, packages may reside in other directories as well. Just enter further paths to **libname** as you need them.

The function returns **true** if all the packages have been successfully loaded and executed, or **fail** if an error occurred.

Hint: the **import** statement is an interface to **readlib** (and **initialise**), but does not require to put the package names into quotes. For example,

```
> readlib('clock');
```

is equivalent to

```
> import clock;
```

See also: **run**, **initialise**, **import** statement.

recurse (f, obj [, ···][, option])

Checks each element of the structure `obj` (a table, set, pair, sequence or register) by applying a function `f` on each of its elements. `f` can be a multivariate function and must return either **true** or **false**. The optional second and all further arguments to `f` may be passed as the third, etc. argument.

With tables, all the entries and keys are scanned.

With sequences and registers, only the entries (not the keys) are scanned.

The function performs a recursive descent if it detects tables, sets, pairs, registers or sequences in `obj` so that it can find elements in deeply nested structures.

If `obj` is a table and the option `skiphash`=**true** has been passed, then the function will ignore all non-numeric keys and their corresponding values.

The function immediately returns **true** if the function call to any element in `obj` evaluates to **true**, and **false** otherwise. If `obj` is a number, boolean, complex number, string, **null**, procedure, thread, userdata or lightuserdata, **recurse** returns **fail**. It issues an error if `obj` is unassigned. To check whether all elements satisfy a condition, use **satisfy**.

Examples:

```
> recurse(<: x -> x > 3 :>, [1, 2, 3]):
false

> recurse(<: x -> x > 3 :>, [1, 2, 3, [4, 5, 6]]):
true
```

See also: **descend**, **in**, **has**, **member**, **notin**, **satisfy**, **whereis**.

reduce (f, obj [, init [, ··· [, options]]])

reduce (f, obj [, ··· [[, option], fold=true]])

Applies function `f` on each item of a structure or string `obj` and returns an accumulated result.

`f` must have two or more parameters, but at least parameters x, a, where x will represent the respective item in `obj`, and a the accumulator to be updated. If `init` is given, then the accumulator will be initialised with it, otherwise the accumulator will be set to zero at first.

After traversal of `obj`, the accumulator will be returned. The function is equivalent to:

```
> reduce := proc(f, s, init, ?) is
>    local accumulator := init or 0;
>    for item in s do
>       accumulator := f(item, accumulator, unpack(?))
>    od;
>    return accumulator
> end;
```

For example, `reduce(<: x, a -> x + a :>, [1, 2, 3, 4])` computes the sum of the numbers in a table, i.e. 10; and `reduce(<: x, a -> a & x & '|' :>, '1234', '')` appends a pipe to each character, i.e. returns '1|2|3|4|'.

You can pass further arguments to the given accumulator function by just passing them as the fourth and following argument(s) to **reduce**. Example to compute the arithmetic mean of all the numbers in table [10, 20, 30]:

```
> tbl := [10, 20, 30];

> f := <: x, a, len -> a + x/len :>

> a := reduce(f, tbl, 0, size tbl):
20
```

for:

```
> a, len := 0, size tbl;

> for x in tbl do
>    inc a, x/len
> od;
```

A counter can also be used: it can be accessed within the accumulator function by the name `_c` when passing the `_c = true` option - with `_c` starting from 1. The performance penalty, however, may be quite significant:

```
> tbl := seq(3, 3, 3);

> a := reduce(<: x, a -> a + x * 10^(_c - 1) :>, tbl, 0, _c=true):
333
```

may be up to four time slower than

```
> a := 0;

> for _c from 1 to size tbl do
>    inc a, tbl[_c] * 10^(_c - 1)
> od;
```

An alternative is to pass the `counter=true` option: It will assign the value of an internal counter, starting from 1 with step size 1, to the last argument of `f`; it is at least 20 % faster than the `_c` option. In the following example, `c` is assigned the internal counter value:

```
> reduce(<: x, a, c -> a + x * 10^(c - 1) :>, tbl, 0, counter=true):
333
```

The function also supports folding with the `fold=true` option: in this case the first value in a structure or string is taken as the initialiser and you do not need to - and should not - pass an initialiser explicitly as the third argument. Examples:

```
> reduce(<: x, a, c -> a + x * 10^(c - 1) :>, seq(0, 3, 3, 3),
>    counter = true, fold = true):
333
```

The **fold** function is a short-cut:

```
> fold(<: x, a, c -> a + x * 10^(c - 1) :>, seq(0, 3, 3, 3),
>    counter = true):
333
```

With strings, you might place an embedded zero, represented by '\000', at its start:

```
> reduce(<: x, a -> a & x & '|' :>, '\0001234', fold=true):
1|2|3|4|
```

If you need to save memory, the **foreach** operator may be an alternative. See also: @ operator, **fold**, **map**, **mulup**, **qmdev**, **qsumup**, **sumup**.

_RELEASE

A global variable that holds a string containing the language name, the current interpreter main version, the subversion, and the patch level. The format of this variable is: `'AGENA :> <version>.<subversion>.<patchlevel>'`.

See also: global environment variable **environ.release**, **environ.version**.

remove (f, obj [, ··· [, reshuffle=true]] [, inplace=true])

Returns all values in table, set, sequence or register `obj` that do not satisfy a condition determined by function `f`, as a new table, set, sequence or register. The type of return is determined by the type of second argument, depending on the type of `obj`. If `obj` is a string, then `f` is applied on each of its character and the return is a string of the non-matching characters, in the right order.

If the function has only one argument, then only the function and the table/set/register/sequence will be passed to **remove**.

```
> remove(<: x -> x > 1 :>, [1, 2, 3]):
[1]
```

If the function has more than one argument, then all arguments *except the first* will be passed right after the name of the table or set `obj`.

```
> remove(<: x, y -> x > y :>, [1, 2, 3], 1):   # 1 for y
[1]
```

If present, the function also copies the metatable and user-defined type of `obj` to the new structure.

Please note that if `obj` is a table, the return might include holes. If you pass the `reshuffle`=**true** option as the last argument, however, the result will be returned in a table array with consecutive positive integral keys, not preserving the original keys of the respective values determined, and not having holes; for example:

```
> remove(<: x -> x < 2 :>, [1, 2, 3]):
[2 ~ 2, 3 ~ 3]

> remove(<: x -> x < 2 :>, [1, 2, 3], reshuffle = true):
[2, 3]
```

With a register, all values up to the current top pointer are evaluated, and the size of the returned register is equal to the number of the elements in the return.

If the last argument is the option `inplace`=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original structure, but saving memory. After completion, the function will return the modified structure. (You can combine the reshuffle and `inplace` options).

If `obj` is **null**, the function returns **null**.

See also: **cleanse**, **countitems**, **map**, **select**, **selectremove**, **subs**, **subsop**, **unique**, **zip**.

`restart`

Restarts an Agena session. No argument is needed.

If a procedure has been assigned to the name **environ.onexit**, then this procedure will automatically be run before re-initialising the interpreter. An example:

```
> environ.onexit := proc() is print('Tschüß !') end

> restart
Tschüß !
```

During start-up, Agena stores all initial values, e.g. assigned package tables, in a global variable called **_origG**. Tables are copied, too, so their contents cannot be altered in a session.

When the Agena session is reset, all values in the Agena environment are unassigned including the environment variable **_G**. The seeds used by **math.random**/**math.randomseed** are reset, too.

The system variables **_origG**, **libname**, **mainlibname environ.onexit**, the current working directory are not reset. **mainlibname** and **libname**, however, are reset to their original values if you issued the statement `environ.kernel(libnamereset = true)` before.

Then all entries in **_origG** are re-read and assigned to the new environment.

After this, the library base file `library.agn` and thereafter the initialisation file `agena.ini` or `.agenainit` - if present - are read and executed. Finally, restart runs a garbage collection.

`reverse (obj)`

Reverses the order of all elements in sequence or register `obj` in-place. With tables, it reverses the elements in the array part, only. The function returns the modified structure.

See also: **strings.reverse**, **stack.reversed**.

right (obj)

With the pair `obj`, the operator returns its right operand. This is equals to `obj[2]`. See also: **left**.

rsorted (obj)

Takes any table `obj` of strings and/or numbers and then radix-sorts them, where in the resulting new table the items will be in order of length, and then in lexicographic order within each length class (shortlex order). If the item in the input table is a number, it will also a number in the output table. The sorting is done non-destructively, leaving `obj` unchanged.

Example:

```
> strings.rsorted(['1', 100, '2', '11']):
[1, 2, 11, 100]
```

Compare to:

```
> sorted(['1', '100', '2', '11']): # can't sort a mix of numbers & strings
[1, 100, 11, 2]
```

See also: **sort**, **sorted**, **skycrane.sorted**, **strings.strverscmp**.

run (filename)

Opens the named file and executes its contents as a chunk. When called without arguments, **run** executes the contents of the standard input (stdin). Returns all values returned by the chunk.

In case of errors, **run** propagates the error to its caller (that is, **run** does not run in protected mode).

See also: **readlib**, **with**.

satisfy (f, x [, ···] [, option])

satisfy (f, obj [, ···] [, option])

In the first form, with `x` a number, complex number, string, boolean, null or userdata, calls the function `f` which should return **true** or **false**. The result is the return of this call. You may also specify optional arguments to `f`.

With `obj` a structure (second form), checks each element in `obj` by calling function `f` which also should return **true** or **false**. If at least one element in `obj` does not satisfy the condition checked by `f`, the result is **false**, and otherwise **true** - that is, in other words, the function returns **true** if every element satisfies the condition.

The function performs a recursive descent if it detects tables, sets, pairs, registers or sequences in `obj` so that it can find elements in deeply nested structures. If `obj` is a table and the option `skiphash`=**true** has been passed, then the function will ignore all non-numeric keys and their corresponding values. To check whether at least one element satisfies a condition, use **recurse**.

See also: **descend**, **has**, **in**, **notin**, **recurse**, **tables.isall**, **sequences.isall**, **registers.isall**, **sets.isall**.

save (obj, filename)

Saves an object `obj` of almost type into a binary file denoted by file name `filename`.

With tables, sequences, registers, sets and functions, **save** also stores attached metatables if present. Likewise with functions, it saves internal remember tables and stores.

While numbers, strings, and Booleans are stored in a portable fashion so that the data can be read both on Big Endian (e.g SPARCs, PPCs) and Little Endian systems, procedures cannot.

save returns an error if an object that cannot be stored to a file has been passed: threads and userdata. It also returns an error if the object to be written is self-referencing (e.g. **_G**). If `obj` contains one and the same structure multiple times, say n times, then **save** will store it n times.

The function locks the file when writing, avoiding file corruption if another application tries to gain access to it.

Note that **save** overwrites existing files without warning. Use **read** to import the data written by **save**.

The function is written in Agena and included in the lib/library.agn file.

See also: **read**, **io.writefile**.

select (f, obj [, ··· [, reshuffle=true] [, inplace=true]])

Returns all values in table, set, sequence or register `obj` that satisfy a condition determined by function `f`. The type of return is determined by the type of the second argument. If `obj` is a string, then `f` is applied on each of its character and the return is a string of the matching characters, in the right order.

If `f` has only one argument, then only the function and the object will be passed to **select**.

```
> select(<: x -> x > 1 :>, [1, 2, 3]):
[2, 3]
```

If the function has more than one argument, then all arguments *except the first* will be passed right after the name of the object `obj`.

```
> select(<: x, y -> x > y :>, {1, 2, 3}, 1):  # 1 for y
{3, 2}
```

If present, the function also copies the metatable and user-defined type of `obj` to the new structure.

Please note that if `obj` is a table, the return might include holes. If you pass the `reshuffle`=**true** option as the last argument, however, the result will be returned in a table array with consecutive positive integral keys, not preserving the original keys of the respective values determined, and not having holes. Thus,

```
> select(<: x -> x :: number :>, ['a', 10, 20, 30, 'z'], reshuffle=true);
```

returns

```
[10, 20, 30]
```

instead of

```
[2 ~ 10, 3 ~ 20, 4 ~ 30]
```

If the last argument is the option `inplace`=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original structure, but saving memory. After completion, the function will return the modified structure. (You can combine the reshuffle and `inplace` options).

With a register, all values up to the current top pointer are evaluated, and the size of the returned register is equal to the number of the elements in the return.

If `obj` is **null**, the function returns **null**.

See also: **$** and **$$$** operators, **cleanse**, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **subsop**, **unique**, **values**, **zip.**

`selectremove (f, obj [, ··· [, reshuffle=true]])`

Combines the functionality of **select** with the one of **remove**: The first result contains all the elements of a structure `obj` (a table, set, sequence, register or string) that satisfy a given condition, the second result contains the elements of a structure not satisfying the condition. This may speed up computations where you need both results, maybe for post-processing, by around 33 %.

If `obj` is a table, the return might include holes. If you pass the `reshuffle`=**true** option as the last argument, however, the result will be returned in table arrays with

consecutive positive integral keys, not preserving the original keys of the respective values determined, and not having holes. Examples:

```
> a := ['a', 10, 20, 30, 'z'];

> selectremove(<: x -> x :: number :>, a):
[2 ~ 10, 3 ~ 20, 4 ~ 30]        [1 ~ a, 5 ~ z];

> selectremove(<: x -> x :: number :>, a, reshuffle=true):
[10, 20, 30]    [a, z]
```

If `obj` is **null**, the function returns **null** twice.

See also: **remove**, **select**.

setbit (x, pos, bit)

Sets or unsets a bit in an integer `x` at the given bit position `pos`.

Internally, `x` is first converted into its binary representation. Then `bit` is set to the `pos`-th position from the right of this binary representation of `x`. `bit` may be either **true** or **false**, or the numbers 0 or 1. E.g. if `x` is 2 = 0b0010, `pos` is 1, and `bit` is **true**, then the result will be 3 = 0b0011.

`pos` should be an integer in the range $|\texttt{pos}| \in [1 .. 32]$.

See also: **getbit**, **getbits**, **setbits**, **setnbits**, **numarray.setbit**.

setbits (x, r)

Sets or unsets all 32 bits of an integer `x` with the bits given in register `r`. The register must contain a minimum of one, and a maximum of 32 values, either the Booleans **true** or **false**, or the integers 0 and 1.
If the register contains less than 32 elements, and has length n, the first 32 - n bits `to the left` are *not* set.

Example:

```
> setbits(8, reg(1, 0, 0)):
12
```

See also: **getbit**, **getbits**, **setbit**, **setnbits**, **numarray.setbit**.

setmetatable (obj, metatable [, usertype])

Sets the metatable for the given table, set, sequence, pair or function `obj`. (You cannot change the metatable of other types from Agena, only from C.) If `metatable` is **null**, removes the metatable of the given table. If the original metatable has a '__metatable' field, raises an error. The function cannot replace metatables of C library functions, use **addtometatable** instead.

If `usertype`, a string, is given, the function in addition sets the specified user-defined type to `obj`, and if `usertype` is **null**, deletes it.

The function issues an error if `obj` is read-only, see **freeze**, **unfreeze**.

This function returns `obj`.

See also: **getmetatable**, **settype**.

setnbits (x, y [, pos [, nbits [, 'or']]])

Sets `nbits` bits in 32-bit integer `y` into position `pos` of 32-bit integer `x`, and returns the modified value of `x`. `pos` and `nbits` should be in [1, 32]. If `pos` is not given, it is 1 by default (the right-most bit in `x`).

If `nbits` is not given, it is **math.mostsigbit**(`y`) by default.

By default, the bits in `x` are overwritten by the bits in `y`. If the fifth argument `'or'` (the string) is given, the bits are Boolean-OR'ed.

See also: **getbit**, **getbits**, **getnbits**, **setbit**, **setbits**.

settype (obj [, ···], str)
settype (obj [, ···], null)

In the first form the function sets the type of one or more procedures, sequences, tables, sets, pairs, or userdata `obj` to the name denoted by string `str`. **gettype** and **typeof** will then return this string when called with `obj`.

In the second form, by passing the **null** constant, the user-defined type is deleted, and **gettype** thus will return **null** whereas **typeof** will return the basic type of `obj`.

If `obj` has no `__tostring` metamethod, then Agena's pretty printer will output the object in the form `str & '(' & <elements> & ')'` instead of the standard `'seq(' & <elements> & ')'` or `'<element>:<element>'` string.

If given just two arguments, i.e. an object and a string or an object and **null**, the function returns the modified object. In all other cases, the function returns **null**.

The function issues an error if `obj` is read-only, see **freeze**, **unfreeze**.

See also: **gettype**.

shift (obj, a, b)

Moves an element in table, sequence register or userdata structure `obj` from position `old` to `new`, with `old`, `new` integers, shifting all the other elements accordingly - which might also cause a rotation. The function returns nothing.

If passed `obj` is a userdata structure, it must have the '__index' and '__writeindex' metamethods for indexed reading and writing. This holds true for numarray, llist, dlist, ulist and some other data structures, see this Reference.

See also: **move**, **purge**, **put**, **rotate** statement, **swap**.

size (obj)

With tables, the operator returns the number of key~value pairs in table `obj`.

With sets, pairs, and sequences, the operator returns the number of items in `obj`.

With registers, the operator returns the number of elements up to the current top pointer, but not the total number of elements in the registers.

With strings, the operator returns the number of characters in string `obj`, i.e. the length of `obj`. With **null**, returns zero.

See also: **$$$** operator, **countitems**, **environ.attrib**, **strings.strlen**, **strings.utf8size**, **tables.getsize**.

sort (obj [l [, u]] [, f] [, 'number'])

Sorts table, sequence or register elements in a given order, in-place, from `obj[l]` to `obj[u]`, where by default `l` is 1 and `u` is the length of the structure. If `f` is given, then it must be a function that receives two structure elements, and will return **true** when the first is less than the second (so that not `f(obj[i+1], obj[i])` will be **true** after the sort). If `f` is not given, then the standard operator **<** (less than) will be used instead.

The sort algorithm is not stable; that is, elements considered equal by the given order may have their relative positions changed by the sort. Also, the function cannot sort structures featuring values of different types (see **skycrane.sorted** for an alternative). The return is the sorted structure.

If the last argument `'number'` is given, it is assumed that `obj` contains numbers only and the sorting will be four times faster. Note that when given, you need twice the amount of memory as the data is duplicated internally and you cannot provide a sorting function as this mode supports sorting in ascending order only.

See also: **math.gt**, **math.lt**, **rsorted**, **sorted**, **skycrane.sorted**, **stats.issorted**, **stats.sorted**, **strings.gt**, **strings.lt**, **strings.strverscmp**.

Example:

```
> s := [1, 2, 3]
```

```
> sort(s, <: x, y -> x > y :>):
[3, 2, 1]

> s := seq(1:'a', 1.1:'b', 1.2:'c');

> sort(s, <: x, y -> left(x) > left(y) :>):
seq(1.2:c, 1.1:b, 1:a)
```

sorted (obj [l [, u]] [, f], [, 'number'])

Sorts table, sequence or register elements in `obj` in a given order from `obj[l]` to `obj[u]`, but - unlike sort - not in-place, and non-destructively. By default, `l` is 1 and `u` is the length of the structure. Depending on the type of `obj`, the return is a new table or sequence.

If `f` is given, then it must be a function that will receive two structure elements to determine the sorting order. See **sort** for further information.

If the last argument `'number'` is given, it is assumed that `obj` contains numbers only and the sorting will be four times faster. Note that when given, you cannot provide a sorting function as this mode supports sorting in ascending order only.

The function cannot sort structures featuring values of different types (see **skycrane.sorted** for an alternative).

See also: **math.gt**, **math.lt**, **rsorted**, **sort**, **skycrane.sorted**, **stats.issorted**, **stats.sorted**, **strings.gt**, **strings.lt**.

subs (x:v [, ···], obj [, options])

Substitutes all occurrences of the value `x` in the table, set, sequence or register `obj` with the value `v`, by default non-destructively. More than one substitution pair can be given. The substitutions are performed sequentially and by default simultaneously starting with the first pair. The type of return is determined by the type of `obj`.

```
> subs(1:3, 2:4, [1, 2, -1]):
[3, 4, -1]
```

If present, the function also copies the metatable and user-defined type of `obj` to the new structure.

If one of the `options` is `inplace`=**true**, then the operation will be done in-place, modifying the original structure, but saving memory. After completion, the function returns the modified structure.

By default, **subs** is still replacing the elements in a structure with *all* the replacements given, including intermediate substitutions. So if we have an expression like

```
> subs(1:2, 2:3, 3:4, [1, 2, 3])
```

we will get:

```
[4, 4, 4]
```

By passing the `multipass`=**false** option, the rest of the replacement list will be skipped as soon as a substitution has been done:

```
> subs(1:2, 2:3, 3:4, [1, 2, 3], multipass=false):
[2, 3, 4]
```

You can check numbers for approximate instead of strict equality by passing the new `strict`=**false** option.

If you substitute a table value with **null**, then the value will be purged, leaving holes in the table if the value resided in the array part. By passing the `reshuffle` option, these holes will be removed by shifting down other elements to close the space.

Compare:

```
> subs(2:null, [1, 2, 3]):
[1 ~ 1, 3 ~ 3]
```

with

```
> subs(2:null, [1, 2, 3], reshuffle=true):
[1, 3]
```

With (deeply) nested tables, sequences, registers, sets and pairs, **subs** can recursively descent into the entire structure and substitute values with only just one call, with the `descend=true` option:

```
> subs(1:0, 6:10, [1, 2, 3, [4, 5, [6]]], descend=true):
[0, 2, 3, [4, 5, [10]]]
```

See also: **countitems**, **map**, **remove**, **select**, **subsop**, **zip**, **registers.subs**, **sequences.subs**, **tables.hashole**, **tables.reshuffle**, **tables.subs**.

subsop (i:v [, ···], obj [, options])

The function replaces the value in a table, sequence or register `obj` at the given index `i` with the new value `v`, by default non-destructively. More than one substitution pair can be given. The type of return is determined by the type of `obj`.

If present, the function also copies the metatable and user-defined type of `obj` to the new structure.

The function also allows to delete values, by setting `v` to **null**.

Examples:

```
> a := [10, 20, 30]
```

Substitute the value at index 2 with zero and delete the third element:

```
> subsop(2:0, 3:null, a):
[10, 0]
```

The original structure is left unchanged:

```
> a:
[10, 20, 30]
```

If the last argument is the option `inplace`=**true**, then the whole operation will be done in-place, modifying the original structure, but saving memory. After completion, the function returns the modified structure.

```
> subsop(2:0, 3:null, a, true):
[10, 0]
```

```
> a:
[10, 0]
```

If you substitute a table value with **null**, then the value will be purged, leaving holes in the table if the value resided in the array part. By passing the `reshuffle` option, these holes will be removed by shifting down other elements to close the space. Compare:

```
> subsop(2:null, [1, 2, 3]):
[1 ~ 1, 3 ~ 3]
```

with

```
> subsop(2:null, [1, 2, 3], reshuffle=true):
[1, 3]
```

See also: **insert** statement, **prepend**, **purge**, **put**, **subs**, **tables.hashole**, **tables.reshuffle**.

sumup (obj)

sumup (obj, n)

sumup (obj, n, p, xm)

sumup (f, obj [, ···])

sumup (f, start, stop, [step [, ···]])

Sums up all numbers or complex numbers in table, sequence, register or userdata `obj`. depending on the input, the return is either a number or complex number. If `obj` is empty or consists entirely of non-numbers, the operator returns **fail**. If the structure contains (complex) numbers and other objects, only the (complex)

numbers are added. With tables, only numeric entries with integral keys will be processed. The operator uses Kahan-Babuška Summation. To improve accuracy, you may sort `obj` before.

In the first form, all the elements in `obj` a summed up. In the second form, each element in `obj` is divided by `n` before being added to the sum. If `n` is 0 or **undefined**, all the elements are divided by the size of `obj`. In both these two forms, `obj` may contain numbers or complex numbers, or a mix of them.

In the third form, computes the moment `p` - an integer - of the given table, sequence, register or numarray `obj` about any origin x_m for a full population and returns a number. As with the second form, if `n` is **undefined** or 0, the size of the distribution is determined automatically.
In summary, all this is equivalent to:

$$\sum_{i=1}^{n} \left(obj_i - x_m\right)^p / n$$

In the fourth form, a function `f` is applied on each number or complex number in `obj` before adding it to the sum. There are two flavours: if `f` returns a (complex) number, then the result to the call of `f` is added to the sum. If `f` is a function returning a boolean, then only if the call to `f` with a number or complex number x, ... in `obj` results to **true** will x be added to the sum. You can put any second, etc. arguments to `f` right after `obj`.

In the fifth form, the operator computes the sum

$$\sum_{k=start}^{stop} f(k, \ldots)$$

Just pass the function `f` representing the series or sum plus the `start` and `stop` value and optionally a `step` size which defaults to one. If `start` > `stop`, the result will 0.

Multivariate functions are supported - just pass their second, third, etc. arguments right after `step`, so with multivariate function you must always give the step size which should usually be one. In the fifth form, complex arguments are accepted, as well, so in order to approximate, for example,

$$\exp(z) \sim \sum_{n=0}^{b} \frac{z^n}{n!}$$

with z the complex number 1 + I, a = 0, b = 25 and (surely) step size 1, enter:

```
> sumup(<: n, z -> z^n/fact(n) :>, 0, 25, 1, 1 + I):
1.4686939399159+2.2873552871788*I
```

In the fifth form, the computation is done twice as fast as with a **for/to** loop combined with calls to **math.kbadd**.

See also: **foreach**, **mulup**, **qsumup**, **calc.fsum**, **sort**, **sorted**, **stats.cumsum**, **stats.issorted**, **stats.sumdata**, **sumup**, **stats.moment**.

swap (obj, a, b)

In the table array, sequence or register `obj`, swaps the entries at index positions `a` and `b`, with `a`, `b` integers. With `obj` a pair, swaps the components. The function returns nothing.

See also: **move**, **purge**, **shift**.

time ()

Returns UTC time in seconds elapsed since the epoch in seconds as a number. The fractional part of the return represents milliseconds. The epoch usually is January 01, 1970, but this may vary between platforms.

See also: **os.clock**, **os.difftime**, **os.time**, **os.timestamp**, **os.uptime**, **watch**, **skycrane.stopwatch**.

times (f, x, n [, ···])

times (f, x, infinity, eps [, ···])

In the first form, the function takes a start value `x` of any type, applies function `f` to it and repeatedly applies `f` to its previous result `n`-1 times. `n` should be a positive integer. It returns the result of the last call to `f`. The second and further arguments of `f` must be put right after `n`.

If `n` is less than 1, the function returns **null**.

Example:

```
> times(<: x -> 1 + recip x :>, 1, 33) -> 1.6180339887499  # Golden ratio

> f := <: x -> times(<: n -> 0.5*(n + x/n)  :>, 1, 20) :>  # square root
```

You can bail out of the loop prematurely by including a Boolean condition in the function definition. As soon as the expression evaluates to **false**, the iteration will stop and the previous interim result will be returned, e.g.:

```
> times( <: x -> x < 10 and x + 1 :>, 1, 33):
10
```

If **times** should bail out in the first iteration then **false**, i.e. the result of the function call, will be returned:

```
> times( <: x -> x < 0 and x + 1 :>, 1, 33):
false
```

In the second form, takes a start value `x` of any type, applies function `f` to it and repeatedly applies `f` to its previous result until the absolute difference of the last two function calls reaches or drops below the numeric threshold `eps`, a non-negative value.

If a function call evaluates to ±**infinity** or **undefined**, the operator also quits, returning **infinity** or **undefined**, respectively.

The third argument **infinity** just signals that the user wants to use this mode. If `f` is multivariate, all arguments but the first are passed right after `eps`.

Example: Solve the equation $7^3 + 2x - 5 = 0$ using Newton's method.

```
> s := <: x -> 7*x^3 + 2*x - 5 :>

> times(<: x -> x - s(x)/calc.eulerdiff(s, x) :>, 4, infinity, DoubleEps):
0.78792505251729

> s(ans):
0
```

See also: @ operator, **foreach**, **map**, **calc.aitken**.

top (obj)

With the table array, sequence or register `obj`, the operator returns the element with the largest index. If `obj` is empty, it returns **null**.

See also: **bottom**.

toreg (obj)

If `obj` is a string, the function will split it into its characters and return them in a register with each character in `obj` as a register value, and in the same order as the characters in `obj`.

If `obj` is a table, the function puts all its values - but not its keys - into a register.

If `obj` is a set, the function puts all its items into a register. The same applies to sequences.

If `obj` contains structures, then only their references will be copied. Map **copy** to structures if you want to create independent copies of them.

In all other cases, the function issues an error.

See also: **toseq**, **toset**, **totable.**

toseq (obj)

If `obj` is a string, the function will split it into its characters and return them in a sequence with each character in `obj` as a sequence value, and in the same order as the characters in `obj`.

If `obj` is a table, the function puts all its values - but not its keys - into a sequence.

If `obj` is a set, the function puts all its items into a sequence. The same applies to registers.

If `obj` contains structures, then only their references will be copied. Map **copy** to structures if you want to create independent copies of them.

In all other cases, the function issues an error.

See also: **toreg**, **toset**, **totable.**

toset (obj)

If `obj` is a string, the function will split it into its characters and returns them in a set. Note that there is no order in the resulting set.

If `obj` is a table, sequence or register, the function puts all its values - but not its keys - into a new set.

If `obj` contains structures, then only their references will be copied. Map **copy** to structures if you want to create independent copies of them.

In all other cases, the function issues an error.

See also: **toreg**, **toseq**, **totable**.

totable (obj)

If `obj` is a string, the function splits it into its characters, and returns them in a table with each character in `obj` as a table value in the same order as the characters in `obj`.

If `obj` is a sequence, register, or set, the function converts it into a table.

If `obj` contains structures, then only their references will be copied. Map **copy** to structures if you want to create independent copies of them.

In all other cases, the function issues an error.

See also: **toreg**, **toseq**, **toset**.

type (obj)

This operator returns the basic type of its only argument `obj`, coded as a string. The possible results of this function are `'null'` (the string, not the value **null**), `'number'`, `'string'`, `'boolean'`, `'table'`, `'set'`, `'sequence'`, `'register'`, `'pair'`, `'complex'`, `'procedure'`, `'thread'`, `'lightuserdata'`, and `'userdata'`.

If `obj` is a table, set, sequence, pair, or procedure with a user-defined type, then **type** will always return the basic type, e.g. `'sequence'` or `'procedure'`.

See also: **::** and **-:** operators, **checktype**, **gettype**, **typeof**.

typeof (obj)

This operator returns the user-defined type - if it exists - of its only argument `obj`, coded as a string.

A self-declared type can be defined for procedures, tables, pairs, sets, and sequences with the **settype** function. If there is no user-defined type for `obj`, then the basic type will be returned, i.e. `'null'` (the string, not the value **null**), `'number'`, `'string'`, `'boolean'`, `'table'`, `'set'`, `'register'`, `'sequence'`, `'pair'`, `'complex'`, `'procedure'`, `'thread'`, and `'userdata'`.

See also: **::** and **-:** operators, **type**, **gettype**.

unassigned (obj)

This Boolean operator checks whether an expression `obj` evaluates to **null**. If `obj` is a constant, i.e. a number, boolean including **fail**, or a string, the operator always returns **false**.

See also: **assigned**.

unfreeze (obj)

Removes the write-protection from table, set, sequence, register, pair or userdata `obj`. After calling the function, you can also can modify associated metatables and set/unset user-defined types.

Use **freeze** to set write-protection. See also: **environ.frozen**.

unique (obj [, true])
unique (obj [, options])

With a table `obj`, the function removes all holes (`missing keys`) and removes multiple occurrences of the same value in the array part, if present. The hash part of a table is considered to be always unique by definition, so it just copies it to the result. The return is a new table with the original table unchanged.

With a sequence or register `obj`, the **unique** function removes multiple occurrences of the same value, if present. The return is a new sequence or register with the original structure unchanged. If you pass the option `inplace` = **true**, however, the operation will be in-place, changing the input structure.

If **true** is given as an optional second argument, or if you pass the `approx` = **true** option, the function will unify all numbers that are very close to each other to just one value, see **approx** for further details, with the environment variable **Eps** also used here for proximity control. The last value in the structure found approximate will be put into the result.

If `obj` is **null**, the function returns **null**.

See also: **numarray.unique**, **tables.entries**.

unity (x)

The operator returns just its only argument `x`, and with function calls returning multiple results, returns just the first, which is useful to prevent passing additional arguments to other functions that might not expect them.

Example:

tables.entries always returns two results, the table entries and a Boolean, but the **sorted** function does not accept the latter:

```
> tables.entries([3, 2, 1]):
[3, 2, 1]       false

> sorted(tables.entries([3, 2, 1])):
Wrong argument #2 to `sorted`: procedure expected, got boolean.

> sorted(unity tables.entries([3, 2, 1])):
[1, 2, 3]
```

If a function call does not return anything, **unity** returns **null**.

See also: **identity**, **ops**.

unpack (obj, [, i [, j]])

Returns the elements from the given table, set, sequence or register `obj`. This function is equivalent to

```
return obj[i], obj[i+1], ···, obj[j]
```

except that the above code can be written only for a fixed number of elements. By default, `i` is 1 and `j` is the length of the object, as defined by the **size** operator.

Please note that if you put a call to **unpack** into an expression list, only the first return of **unpack** is propagated if the call to **unpack** is not at the final position of the expression list, for example:

```
> s := [unpack([1, 2, 3]), 4, 5]:   # 2 and 3 are discarded
[1, 4, 5]

> s := [-1, 0, unpack([1, 2, 3])]:  # 2 and 3 are included
[-1, 0, 1, 2, 3]
```

Consider **copyadd** in this case.

Note that with sets, but the order of the values returned may vary, depending on how elements are internally stored in the set.

See also: **drop**, **identity**, **op**, **ops**, **unity**, **values**.

values (obj, i_1 [, i_2, ···])

Returns elements i_k from the given table, sequence or register `obj`. This function is equivalent to - for example -

```
return [ i1 ~ obj[i1], i2 ~ obj[i2], ··· ]   or
return seq( obj[i1], obj[i2], ··· )   or
return seq( obj[i1], obj[i2], ··· )
```

The type of return is determined by the first argument `obj`.

See also: **columns**, **op**, **ops**, **select**, **unpack**.

watch ([option])

The function implements a stop watch. With the first call, the function starts counting and returns 0. The second call returns the elapsed time in seconds and milliseconds and restarts the clock. If any argument is given, then the clock will be reset, but it will not start counting.

See also: **time**, **watch**, **os.time**, **skycrane.stopwatch**.

whereis (obj, x], option])

whereis (f, obj, [, ···])

In the first form, returns the indices of a given value `x` in table, sequence or register `obj` as a new table, sequence or register, respectively, dependent on the type of `obj`. If `x` is not in `obj`, returns an empty structure. Examples:

```
> whereis([10, 20, 10, 20], 10):
[1, 3]

> whereis([10, 20, 10, 20], 0):
[]
```

If you pass any `option`, then with numbers the function will conduct an approximate check, see **approx**.

In the second form, **whereis** takes a univariate or multivariate function `f` and a table, sequence or register `obj` and returns all the indices of the structure values that satisfy the Boolean condition defined by `f`. The second, third, etc. arguments to `f` will be given right after `obj`.

Example:

```
> whereis(<: x -> x > 2 :>, [1, 2, 30, 40, 50]):
[3, 4, 5]
```

See also: **descend**, **has**, **in**, **member**, **notin**, **recurse**, **satisfy**, **tables.entries**, **tables.indices**.

write ([fh,] v_1 [, v_2, ···] [, delim = <str>])

This function prints one or more numbers, Booleans or strings v_k to the file denoted by the handle `fh`, or to stdout (i.e. the console) if `fh` is not given.

By default, no character is inserted between neighbouring values. This may be changed by passing the option `'delim':<str>` (e.g. `'delim':'|'` or `delim='|'`) as the last argument to the function with `<str>` being a string of any length. Remember that in the function call, a shortcut to `'delim':<str>` is `delim = <str>`.

The function is an interface to **io.write**.

See also: **printf**, **skycrane.scribe**, **skycrane.tee**.

writeline ([fh,] v_1 [, v_2, ···] [, delim = str])

This function prints one or more numbers, booleans or strings v_k followed by a newline to the file denoted by the handle `fh`, or to stdout (i.e. the console) if `fh` is not given.

By default, no character is inserted between neighbouring values. This may be changed by passing the option `'delim':str` (i.e. a pair, e.g. `'delim':'|'`) as the last argument to the function with `str` being a string of any length. Remember that in the function call, a shortcut to `'delim':str` is `delim=str`.

The function is a wrapper to **io.writeline**.

See also: **printf**, **skycrane.scribe**, **skycrane.tee**.

`xpcall (f, err [, arg1, ···])`

This function is similar to **protect**, except that you can set a new error handler.

xpcall calls function `f` in protected mode, using `err` as the error handler. Arguments to `f` are optional. Any error inside `f` is not propagated; instead, **xpcall** catches the error, calls the `err` function with the original error object, and returns a status code. Its first result is the status code (a Boolean), which is **true** if the call succeeds without errors. In this case, **xpcall** also returns all results from the call, after this first result. In case of any error, **xpcall** returns **false** plus the result from `err`.

See also: **argerror**, **assume**, **error**, **protect**.

`zip (f, obj1, obj2 [, ···])`

`zip (op, obj1, obj2)`

In the first form, the function zips together either two sequences, two registers, or two tables `obj1`, `obj2` by applying function `f` to each of its respective elements. Depending on the type of `obj1` and `obj2`, the result is a new sequence, register, or table `s` where each element `s`[k] is determined by `s`[k] := f(`obj1`[k], `obj2`[k]).

`obj1` and `obj2` must have the same number of elements. If you pass tables, they must have the same keys.

If `f` has more than two arguments, then its fourth to last argument must be given right after `obj2`.

In the second form, `op` depicts an arithmetic operator, represented as a string, that zips together the structure elements:

- `'+'`: addition,
- `'-'`: subtraction,
- `'*'`: multiplication,
- `'/'`: division,
- `'\\'`: integer division,
- `'%'`: modulo,
- `'^'`: exponentiation,
- `'**'`: integer exponentiation.

This method is twice as fast with sequences and registers, and 50 % faster with tables than passing a function (first form).

If `obj1` or `obj2` have user-defined types or metatables, they are copied to the resulting structure, as well. If `obj1` has a metatable, then this metatable will be copied, else the metatable of `obj2` will be used if the latter exists. The same applies to user-defined types.

See also: **augment**, **columns**, **map**, **remove**, **select**, **subs**, **subsop**.

Chapter Nine

Strings

9 Strings

9.1 Basic String Functions

Summary of Functions:

Search

atendof, **has**, **in**, **notin**, **strings.find**, **strings.glob**, **strings.instr**, **strings.match**, **strings.mfind**, **suffix.find**, **suffix.findall**.

Insertion, Substitution, and Deletion

strings.replace, **strings.appendmissing**, **strings.chomp**, **strings.chop**, **strings.gsub**, **strings.include**, **strings.remove**, **strings.unwrap**, **strings.wrap**.

Extraction

split, **strings.advance**, **strings.between**, **strings.charset**, **strings.fields**, **strings.gmatch**, **strings.gmatches**, **strings.gseparate**, **strings.separate**, **strings.strchr**, **strings.strnstr**, **strings.strptime**, **strings.strrchr**, **strings.strstr**, **strings.sub**, **suffix.autocomplete**.

Queries

abs, **empty**, **filled**, **member**, **strings.charmap**, **strings.contains**, **strings.isaligned**, **strings.isalpha**, **strings.isalphanumeric**, **strings.isalphaspace**, **string.isalphaspec**, **strings.isascii**, **strings.isblank**, **strings.iscenumeric**, **strings.iscomplex**, **strings.isconsonant**, **strings.iscontrol**, **strings.isdia**, **strings.isending**, **strings.isfractional**, **strings.isgraph**, **strings.ishex**, **strings.islatin**, **strings.isinstring**, **strings.isintegral**, **strings.isisoalpha**, **strings.isisolower**, **strings.isisoprint**, **strings.isisospace**, **strings.isisoupper**, **strings.islatinnumeric**, **strings.isloweralpha**, **strings.islowerlatin**, **strings.ismagic**, **strings.isnumber**, **strings.isnumeric**, **strings.isnumberspace**, **strings.ispattern**, **strings.isprintable**, **strings.isspace**, **strings.isspec**, **strings.isstarting**, **strings.issubseq**, **strings.isupperalpha**, **strings.isupperlatin**,**strings.isutf8**, **strings.isvowel**, **strings.iswrapped**, **strings.shannon**, **suffix.unique**, **strings.walker**.

Comparing

fzy.filter, **fzy.has**, **fzy.score**, **strings.compare**, **strings.dice**, **strings.diffs**, **strings.dleven**, **strings.eq**, **strings.fuzzy**, **strings.ge**, **strings.gt**, **strings.hirschberg**, **strings.jaro**, **strings.lcs**, **strings.le**, **strings.leven**, **strings.lrs**, **strings.lt**, **strings.neq**, **strings.strcmp**, **strings.stricmp**, **strings.strncmp**, **strings.strverscmp**.

Counting

size, **strings.hits**, **strings.strlen**, **strings.utf8size**, **strings.words**, **suffix.count**.

Formatting

strings.align, **strings.capitalise**, **strings.format**, **strings.isolower**, **strings.isoupper**, **strings.ljustify**, **strings.lower**, **strings.lpad**, **strings.ltrim**, **strings.lrtrim**, **strings.rjustify**, **strings.rpad**, **strings.rtrim**, **strings.trim**, **strings.uncapitalise**,**strings.upper**.

Conversion

&, **fold**, **reduce**, **tonumber**, **tostring**, **strings.a64**, **strings.bigrams**, **strings.diamap**, **strings.hextodec**, **strings.iterate**, **strings.join**, **strings.obfusxor**, **strings.pack**, **strings.packsize**, **strings.reverse**, **strings.strtoul**, **strings.strxfrm**, **strings.tolatin**, **strings.tonumber**, **strings.toutf8**, **strings.transform**, **strings.unpack**, **toreg**, **toseq**, **totable**.

Manipulation

@, **map**, **strings.iterate**, **strings.repeat**, **strings.rotateleft**, **strings.rotateright**, **strings.tobytes**, **strings.tochars**.

Miscellaneous

strings.random.

A note in advance: All operators and **strings** package functions know how to handle many diacritics properly. Thus, the **strings.lower** and **strings.upper** functions know how to convert these diacritics, and various **is*** functions recognise diacritics as alphabetic characters.

Diacritics in this context are the letters:

```
â  Â  ä  Ä  à  À  á  Á  å  Å  æ  Æ  ã  Ã
ê  Ê  ë  è  È  é  É  Ë
ï  Ï  î  Î  ì  Ì  í  Í  ý  Ý  ÿ
ô  Ô  ö  Ö  ò  Ò  ø  Ø  ó  Ó  õ  Õ
û  Û  ù  Ù  ü  Ü  ú  Ú
ç  Ç  ñ  Ñ  ð  Ð  þ  Þ  ß
```

9.1.1 Operators and Functions

s1 & s2

This binary operator concatenates two strings `s1`, `s2` and returns a new string. `s1` or `s2` may also be a number, complex number, or a Boolean; in this case the argument will be converted to a string and then concatenated with the other operand. **null** will be treated like the empty string in this context, too.

See also: **strings.join**.

v &:= s

The compound concatenation operator appends string `s` to the contents of the string variable `v`. It is equivalent to: `v := v & s`.

s1 < s2

The relational operator checks whether string `s1` is less then strings `s2`, taking the locale into account if the platform supports it. See also: =, >, <=, >=.

s - p

The operator removes all occurrences of pattern `p` from string `s` and returns a new string. The functionality supports pattern-matching, see Chapter 9.1.3, and is implemented via the '__sub' metamethod which interfaces to **strings.remove**. Example:

```
> 'agena.a' - '.a$':
agena
```

s || p:t

The operator replaces all occurrences of substring `p` in string `s` with substring `t` and returns a new string. `p` may include patterns, see Chapter 9.1.3. The functionality is implemented via the '__bor' metamethod which calls **strings.gsub** internally. Example:

```
> 'agena.a' || '.a$':'.h':
agena.h
```

s << n

The operator prepends whitespace to the beginning of the string, shifting the text `s` to the right. The integer `n` specifies the target total length of the final string. The functionality is implemented via the '__bshl' metamethod which calls **strings.lpad** internally. Example:

```
> 'folder' << 10:
    folder
```

s >> n

The operator appends whitespace to the end of string `s`. The integer `n` specifies the target total length of the final string. The functionality is implemented via the '__bshr' metamethod which calls **strings.rpad** internally. Example (concatenation of the pipe has only been added to show the resulting string):

```
> 'folder' >> 10 & '|':
folder     |
```

s / t

The operator concatenates strings `s` and `t` with a slash. There is some logic built in to remove leading and trailing slashes in the operands.

```
> 'folder' / 'filename':
folder/filename

> 'folder' / '/filename':
folder/filename
```

The functionality has been implemented via the '__div' metamethod.

s \ n

The operator will return the first `n` characters in string `s`, with `n` a positive integer. Example:

```
> 'going' \ 2:
go
```

The functionality has been implemented via the '__intdiv' metamethod.

s % n

s % d

In the first form the operator will return the final `n` characters in string `s`, with `n` a positive integer. Example:

```
> 'going' % 3:
ing
```

In the second form the operator provides an interface to **strings.format**, with `s` the format string and all values to be formatted put into table `d`:

```
> '%s=%.2f' % ['Agena', 7.11]:
Agena=7.11
```

This is equal to:

```
> strings.format('%s=%.2f', 'Agena', 7.11):
Agena=7.11
```

The functionality has been implemented via the '__mod' metamethod.

s1 atendof s2

This binary operator checks whether string `s2` ends in a substring `s1`. If true, the position of the position of `s1` in `s2` will be returned; otherwise **null** will be returned. The operator also returns **null** if the strings have the same length or at least one of them is the empty string.

See also: **in**, **strings.instr**, **strings.isstarting**, **strings.isending**.

s1 in s2

This binary operator checks whether string `s2` includes `s1` and returns its position as a number, or **null** if `s1` cannot be found. The operator also returns **null** if at least one of the strings is the empty string.

See also: **atendof**, **has**, **notin**, **strings.contains**, **strings.instr**, **strings.isstarting**, **strings.isending**, **suffix.find**.

s1 notin s2

This binary operator checks whether string `s2` does not include `s1` and returns **true** or **false**.

See also: **in** operator.

s1 split s2

Splits the string `s1` into words. The delimiter is given by string `s2`, which may consist of one or more characters. The return of the operator is a sequence. If `s1`= `s2` then an empty sequence will be returned. If `s2` is the empty string, then the operator will split `s1` into its individual characters. The operator does not support pattern matching, use **strings.fields** instead.

See also: **strings.iterate**, **strings.separate**, **strings.tobytes**.

abs (s)

With strings, the operator returns the numeric ASCII value of the given character `s` (a string of length 1). See also: **char**.

char (n)

The function converts integer `n` representing an ASCII code into corresponding character. See also: **abs**.

empty (s)

The operator checks whether the string `s` is empty. The return is **true** or **false**. See also: **filled**.

filled (s)

The operator checks whether the string `s` is non-empty. The return is **true** or **false**. See also: **empty**.

fold (f, s [, options])

Applies a function `f` on each item of string `s` and returns an accumulated result. It works like **reduce** with the `fold=true` option, i.e. the initialiser is always given by the first character in `s`. **fold** supports all the options available for **reduce**.

has (s, chars)

Checks whether at least one character in `s` matches one of the characters in `chars`, a string representing a set of individual characters. If `chars` is the empty string, the function generally returns **false**.

See also: **in** operator, **strings.contains**.

map (f, s [, ···] [, true])

This function maps a function `f` to all characters of string `s` from the left to right. The return is a sequence of function values.

If function `f` has only one argument, then only the function and the string `s` must be passed to **map**. If the function has more than one argument, then all arguments *except the first* are passed right after argument `s`. If the last argument is the option inplace=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original structure, but saving memory. After completion, the function returns the modified structure. For further options, see the function description in Chapter 8.

member (s, t)

If `s` and `t` are both strings, checks whether all characters in string `s` are part of the characters - the `alphabet`- in string `t`, and returns **true** or **null**, the latter indicating that at least one character in `s` is not in `t`. If `t` is the empty string, the function returns **null**, as well.

See also: **has**, **strings.contains**.

reduce (f, s [, init [, ··· [, options]]])

reduce (f, s [, ··· [[, option], fold=true]])

Applies a function `f` on each item of string `s` and returns an accumulated result. See **reduce** in Chapter 8 for more information.

size (s)

With a string `s`, the operator returns its length, i.e. the number of characters in `s`.

See also: **strings.strlen**.

tonumber (e [, base] [, true])

Tries to convert its argument to a number or complex value. If the argument is already a number, complex value, or a string convertible to a number or complex value, then **tonumber** will return this value; otherwise, it will return `e` if `e` is a string, and **fail** otherwise. The function recognises the strings `'undefined'` and `'infinity'` properly, i.e. it converts them to the corresponding numeric values **undefined** and **infinity**, respectively.

If `e` is a complex number and the Boolean **true** is given as the last argument, the function will return the real and the imaginary part of `e` as two numbers.

An optional argument specifies the base to interpret the numeral. The base may be any integer between 2 and 36, inclusive. In bases above 10, the letter 'A' (in either upper or lower case) represents 10, 'B' represents 11, and so forth, with 'Z' representing 35. In base 10 (the default), the number may have a decimal part, as well as an optional exponent part.

In other bases, only unsigned integers are accepted. If an option is passed, `'undefined'` and `'infinity'` are not converted to numbers; and if `e` could not be converted, **fail** will be returned.

See also: **strings.fields**, **strings.strtoul**.

toreg (s)

Splits string `s` it into its characters and returns them in a register with each character in `s` as a separate value, and in the same order as the characters in `s`.

See also: **strings.tochars**.

toseq (s)

Splits string `s` it into its characters and returns them in a sequence with each character in `s` as a separate value, and in the same order as the characters in `s`.

See also: **strings.tochars**.

tostring (e [, anyoption])

Receives an argument `e` of any type and converts it to a string in a reasonable format. For complete control of how numbers are converted, use **strings.format**.

If the metatable of `e` has a '__tostring' field, then the **tostring** function will call the corresponding value with e as argument, and will use the result of the call as its result.

With numbers, the number of digits in the resulting string is dependent on the **kernel/digits** setting. See **environ.kernel** for further information.

If `e` is a complex number, its real and imaginary parts are returned as two strings. If any option is given, the return is one string of the format ±"re+im*I" or ±"re-im*I", depending on the sign of the imaginary part of `e`.

See also: **tostringx** and Appendix A6.

tostringx (e [, option])

Works like **tostring** but also formats structures, userdata and complex numbers the same way as the pretty printer does, or in other words: it returns the argument as a string formatted the same way as the **print** function writes it on screen. This is useful if you want to write structures or complex numbers to a file.

If any `option` is given, then the function encloses any string present in `e` in single quotes and it also properly escapes any embedded quotes. This is useful when writing strings to a text file and reading them back later.

totable (s)

Splits string `s` it into its characters and returns them in a table array with each character in `s` as a separate value, and in the same order as the characters in `s`.

See also: **strings.tochars**.

9.1.2 The strings Library

The **strings** library provides generic functions for string manipulation, such as finding and extracting substrings, and pattern matching. When indexing a string in Agena, the first character is at position 1 (not at 0, as in C). Indices are allowed to be negative and are interpreted as indexing backwards, from the end of the string. Thus, the last character is at position -1, and so on.

The strings library provides all its functions inside the table `strings`, and all functions provided there can also be called as OOP methods, so

```
> str := 'Remember October 7th, 2023'

> strings.upper(str):
REMEMBER OCTOBER 7TH, 2023
```

is equal to:

```
> str@@upper():
REMEMBER OCTOBER 7TH, 2023
```

and likewise:

```
> strings.wrap(str, '**'), str@@wrap('**'):
**Remember October 7th, 2023**   **Remember October 7th, 2023**
```

strings.a64 (x)

The function converts between 32-bit long integers and little-endian Base64 ASCII strings (of length 0 to 6).

If the argument `x` is a Base64 ASCII string, the result is a signed 32-bit integer; if the argument `x` is a number, the result is the Base64 ASCII string, which consists of the characters:

./0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

strings.advance (s, p [, option])

strings.advance (s, pos)

In the first form, the function moves to substring `p` in string `s` and returns `p` up to the end of `s`. If `p` could not be found, the function returns **null**. The function supports pattern matching. If the optional third argument **true** is given, the function returns the rest of `s` following but not including `p`. In this case, if `s` ends with `p`, **null** will be returned.

In the second form, the substring starting at position `pos` (a positive integer) up to the end of `s` will be returned. If `pos` is greater than the length of `s`, the result is **null**.

See also: **strings.find**.

strings.align (s [, n])

Inserts newlines into a string `s` after each `n` character. By default `n` is 79, so a newline is inserted at position 80, 160, and so forth. The return is a string. The function helps with correctly outputting formatted text at the console.

strings.appendmissing (s, t)

Appends suffix `t` (a string) to `s` (a string) if `t` is not already at its end; otherwise returns `s`. If `s` is the empty string, `t` will be returned.

strings.between (s, p, q [, true])

Returns the substring in string `s` that is nested between the prefix string `p` and the suffix string `q`. `p` or `q` may reside within the string. If the Boolean value **true** is given as a fourth argument, the function tries to return the substring found as a number. If nothing could be found, the function returns **null**.

See also: **strings.chomp**, **strings.include**, **strings.unwrap**, **strings.wrap**.

strings.bigrams (s [, option])

The function returns the bigrams for strings `s`. With no `option`, the return is a sequence of strings of length 2 each; with any `option` a sequence with the bigrams encoded to 4-byte signed integers is returned.

See also: **math.hamming**, **bytes** package, **strings.dice**, **strings.ngrams**.

strings.byte (s [, i [, j]])

Returns the internal numeric codes of the characters `s[i]`, `s[i+1]`, ..., `s[j]`. The default value for `i` is 1; the default value for `j` is `i`. If `j` is greater than the length of the string it is auto-corrected to the string length.

Numeric codes are not necessarily portable across platforms.

See also: **strings.tobytes**.

strings.capitalise (s [, sep])

Converts the first character in string `s` to upper case - if possible - and returns the capitalised string. If `s` is the empty string, it is simply returned. It also converts ligatures if the Western European character set is being used. If `sep`, a string, is given, then all the words in `s` - separated by `sep` - will be capitalised.

See also: **strings.upper**, **strings.uncapitalise**.

strings.charmap ()

Queries the internal tables to classify characters. Returns a table of key~value pairs:

Key	Sequence of	Used by/Comment
`alpha`	alphabetical letters	strings.alpha, strings.isalphanumeric, strings.isalphaspace, strings.isalphaspec
`ascii`	table of all ASCII characters	result may vary across platforms and codepages

Key	Sequence of	Used by/Comment
`upper`	upper-case letters	strings.isupperalpha
`lower`	lower-case letters	strings.isloweralpha
`vowel`	vowels	strings.isvowel
`dia`	diacritics	strings.isalpha, strings.isalphanumeric, strings.isalphaspace, strings.isalphaspec, strings.isdia
`digits`	digits 0 to 9	strings.isalphanumeric
`hex`	hexadecimals	strings.ishex
`punct`	punctuations	strings.isspec, strings.isalphaspec
`control`	control characters such as \n, \r, \b	strings.iscontrol
`blank`	white space, tab	strings.isblank
`printable`	printable characters	strings.isprintable

`strings.charset (s)`

Returns a set of all the unique characters included in string `s`.

See also: **has**, **strings.contains**.

`strings.chomp (s, t [,···])`

Removes pattern string `t` from the end of string `s` if it is there, and will return the shortened string `s`; otherwise will return `s` unchanged. If more than one pattern is given, each additional pattern will be removed from the previous result.

The function supports pattern matching. In this case you or may not terminate the pattern with one final `'$'`.

Example:

```
> strings.chomp('agena language', '(%a+)', 'ena '):
ag
```

See also: **strings.chop**.

`strings.chop (s)`
`strings.chop (s, f)`

In the first form, removes the last character from string `s` and returns the shortened string. If `s` is empty, it is simply returned.

In the second form, if a function `f` returning **true** or **false** is given, **chop** checks each character in the string from the right to the left for the given Boolean condition and returns the string from its beginning up to and including the character that no longer satisfies the condition.

Example:

```
> strings.chop('path/file.name', <: x -> x <> '/' :>):
path/
```

See also: **strings.between**, **strings.chomp**.

strings.compare (s1, s2)

When called with no option, returns the first position - an integer - where the two strings `s1` and `s2` differ, or 0 if both strings are equal.

See also: **strings.strcmp**, **strings.strverscmp**, **strings.dice**, **strings.fuzzy**, **strings.jaro**.

strings.contains (s, t)

Checks whether all characters in string `s` are part of the characters in string `t`, and returns **true** or **false**. Embedded zeros, expressed by the character sequence '\000' or the empty string are supported.

See also: **has**, **in** operator, **member**, **strings.charset**, **strings.strspn**.

strings.cut (s, d)

The function takes a string `s` to be split into two pieces, and a string `d` of one or more single-character delimiters, and returns two values: the first part of `s` up to - but not including - the delimiter found, and the rest of `s` also without the delimiter.

If a string cannot be split apart, it will be returned as the first result and the second return is **null**.

See also: **split**, **strings.separate**.

strings.diamap (s [, option])

The function corrects problems in the Solaris, Linux, OS/2, Windows, and DOS consoles running code page 850 (also dubbed `DOS Latin-1`) with diacritics and ligatures read in from the keyboard or a text file by mapping them to code page 1252. It takes a strings `s`, applies the mapping, and returns a new string. All other characters are returned unchanged.

The `'ascii7'` option converts Codepage 850 (DOS-Latin-1) input to 7bit ASCII for the 0x80-0xFF range.

If any other `option` is given, the function transforms a string from code page 1252 to 850.

Example:

```
> strings.diamap('AEIOU-Í_ã+Ï'):
AEIOUÄÖÜÆÅØ
```

Note that the function does not convert all existing special tokens.

Agena is shipped with substitution tables for code page 1252. If you want to use another code page, edit the `_c2f` and `_f2c` tables in the lib/library.agn file accordingly.

See also: **os.codepage**, **os.shellinfo**.

strings.dice (s, t)

The function returns the Dice's coefficient of two strings `s`, `t` by measuring how similar a set and another set are, in terms of the number of common bigrams. The return is the number of matches (where each match counts twice) divided by the combined length of `s` and `t` minus 2.

See also: **strings.bigrams**, **strings.diffs**, **strings.fuzzy**, **strings.jaro**, **strings.ngrams**.

strings.diffs (s, t [, n [, option]])

Counts the differences between the two strings `s` and `t`: substitutions, transpositions, deletions, and insertions.

By default, both strings must contains at least `n`=3 characters. You may change this by passing any other positive number for `n`. The function returns **fail** if at least one of the strings consists of less characters.

If any fourth argument is given, the return is a sequence of strings describing the respective difference found, otherwise the returns is the number of the differences encountered.

The function is at least thrice as fast as **strings.dleven**, but may count differently in odd situations.

See also: **strings.dice**, **strings.dleven**, **strings.fuzzy**, **skycrane.tolerance**.

strings.dleven (s, t [, option])

Returns the Damerau-Levenshtein distance between two strings `s` and `t`. It is a count of the minimum number of insertions, deletions, substitutions of a single character, or transpositions of two neighbouring characters to convert `s` into `t`. The return is a number. If at least one of the strings is empty, **undefined** will be returned.

If `option` is set to **true**, then the Damerau-Levenshtein similarity will be computed: the higher the value, the more similar the strings are. The score is normalised such that 0 equates to no similarities and 1 is an exact match with the function taking

into account that if up to four characters at the start of the strings match, the score will be higher.

The function is very fast even with large input.

See also: **strings.fuzzy**, **strings.dice**, **strings.diffs**, **strings.hirschberg**, **strings.jaro**, **strings.lcs**, **strings.leven**, **skycrane.tolerance**.

strings.dump (f [, strip])

Returns a string containing a binary representation of the given function `f`, so that a later **loadstring** on this string returns a copy of the function. `f` must be an Agena function without upvalues, remember table or internal store table.

If `strip` is a **true** value, the binary representation may not include all debug information about the function, to save space.

The function can also be used to binarily serialise data by defining a function returning data, e.g.:

```
> f := proc() is return [1, 2, 3] end;

> s := strings.dump(f);

> loadstring(s)():
[1, 2, 3]
```

See also: **strings.tobytes**.

strings.eq (x, y)

Checks whether the string `x` is equal to the string `y`, using the collation of the system, and returns **true** or **false**. The function internally calls the C function strcoll() to conduct the test, see **strings.strcoll**.

See also: <, <=, >, >=, <> operators, **strings.ge**, **strings.gt**, **strings.le**, **strings.lt**, **strings.neq**.

strings.fields (s, [i_1 [, i_2, ···]] [, delim] [, true])

strings.fields (s, [o] [, delim] [, true])

strings.fields (s, [i_1 [, i_2, ···]] [, options])

strings.fields (s, [o] [, options])

Extracts the given fields (columns) in string `s`. In the first form, the field positions i_1, i_2, etc. are non-zero integers. The field positions may be negative, denoting fields counted from the right end of `s`. If no position is given, then all the fields will be returned. In the second and fourth form, the field positions are given in the sequence or table `o`.

An optional string `delim` may be passed to denote the character or character sequence that separates the individual fields. The default for `delim` is the white space. The function supports pattern matching for the delimiter.

If the Boolean value **true** is given as the last argument, the function tries to convert the fields into numbers.

In all forms, the function issues an error if a field position does not exist (but check the `bailout` option described below).

In the third and fourth form, you can pass one or more of the following options:

- `delim`=*string* where *string* denotes the non-empty string separating the fields, the default is the white space,
- `unwrap`=*string* where *string* denotes a non-empty string - by default there is no unwrapping; if a field is enclosed by one of the characters in *string* then it is removed from the start and end of the field,
- `convert`=*boolean*: if *boolean* is **true** then the function tries to convert the field into a number or complex number. In the latter case, the value must be of the form "a + I*b" with or without white spaces in between; default is **false**,
- `bailout`=*boolean*: if *boolean* is **true** (default) then the function throws an error, if it is **false**, **fail** will be returned if a field does not exist in string `s`.

The return on success is a sequence of the fields (strings, optionally numbers or complex numbers) or a table if parameter `o` for the field numbers is a table.

Example, the delimiter pattern %A stands for any non-letter, see Chapter 4.7.7:

```
> strings.fields('a bb c dddd',  [2, 3], '%A'):
[bb, c]
```

See also: **split**, **strings.iterate**, **strings.gseparate**, **strings.separate**, **strings.wrap**, **tonumber**.

strings.find (s, pattern [, init [, plain]])

Looks for the first match of string `pattern` in string `s`. If it finds a match, then **find** returns the indices of `s` where this occurrence starts and ends; otherwise, it returns **null**.

The function supports pattern matching facilities (which you can turn off, see below). If `pattern` is a table, set, sequence or register of string patterns, then the function checks whether at least one of the patterns matches `s` and returns the respective result. Check Chapter 4.4.7 and Chapter 9.1.3 for more information.

A third, optional numerical argument `init` specifies where to start the search; its default value is 1 and may be negative. A value of **true** as a fourth, optional argument `plain` turns off the pattern matching facilities (see Chapter 9.1.3), so the

function does a plain `find substring` operation, with no characters in pattern being considered `magic`. Note that if `plain` is given, then `init` must be given as well.

If the pattern has captures, then in a successful match the captured values will also be returned, after the two indices. If `pattern` is the empty string, the function returns **null**.

See also: **in**, **atendof**, **regex.find**, **strings.glob**, **strings.instr**, **strings.mfind**, **suffix.find**.

strings.format (formatstring, ···)

Returns a formatted version of its variable number of arguments following the description given in its first argument (which must be a string). The format string almost follows the same rules as the ISO C function `sprintf`. The only differences are that the conversion specifiers *, l and L are not supported and that there are thirteen extra specifiers: a, A, b, B, h, H, m, n, N, p, P, q, Q, D, and F.

For an overview of all available specifies and examples, see below.

In general a format has the following syntax, where values in square brackets are optional:

%[flags][width][.precision]

'flags' may be one of the specifiers described below, optionally preceded by:

- `-` (minus): left-justify the result,
- `+` (plus): print plus sign in front of positive numbers

You can mix `-` and `+`.

'width' is the minimum *total* length of the output in characters, pre-decimal places plus decimal (fractional) places. 'precision' depicts the minimum number of decimal places to appear, with trailing zeros to be added if necessary.

The following specifiers do not comply to the C standard:

The q specifier formats a string in a form suitable to be safely read back by the Agena interpreter: All double quotes, newlines, embedded zeros, and backslashes in the string are correctly escaped when written, and without trailing zeros in the fractional part when the precision specified in the specifier is greater than the number of significant digits in the argument supplied. The same applies to Q but with single quotes. The a and A specifiers work the same like the q and Q specifiers, respectively, but do not include trailing or leading double quotes. The B specifier prints a string in backquotes. The b specifier prints a Binary value.

For instance, the call

```
> strings.format('%q', 'a string with \"quotes\" and \n new line')
```

will produce the string:

```
 "a string with \"quotes\" and \
 new line"
```

The h and H specifiers print a floating-point number in a hexadecimal fractional notation with the exponent to base 2 represented in decimal digits. On DOS and OS/2, the h and H specifiers are not available.

The p specifier multiplies the given number by 100 and displays it in fixed float ('f') format, followed by a percent sign. The m specifier prints a monetary amount with thousands separators and the decimal point defined by the current locale, the default is the format string '%.2f'.

The specifier P formats the pointer (returned by lua_topointer). That gives a unique string identifier for structures, userdata, threads, strings, and functions. For other values (numbers, **null**, booleans), this specifier results in a string representing the pointer NULL.

The n and N specifiers print a number using the decimal point separator of the locale of the operating system (which may differ from the locale in use by Agena), otherwise they work like the f and F specifiers.

The specifiers D and F prevent quarrels with numerical functions that may return non-numbers in case of errors: D formats an integer like the d specifier if the argument is a number, and the C double representation of **undefined** otherwise if the value is not a number. Likewise, F and N either format a float, or the C double companion piece of **undefined** (e.g. `1.#QNAN0` in Windows) if the value is not a number.

The conversion specifiers c, D, d, E, e, f, F, g, G, h, H, i, m, n, N, o, p, u, X, and x all expect a number as argument, whereas s expects a string, and a, A, P, q, Q and expect anything.

This function does not accept string values containing embedded zeros.

Examples:

```
> strings.format('%+15.9f', 10k*Pi):
+31415.926535898

> strings.format('%15.9f', 3.5):
    3.500000000

> strings.format('%-15.9f', 3.5):
3.500000000
```

```
> strings.format('%015.9f', 3.5):
00003.500000000

> strings.format('%15.0f', 3.5):
              4

> strings.format('%15d', 3.5):
              3

> strings.format('%X', 2^16-1):
FFFF

> strings.format('%c', 97):
a

> strings.format('%s', 'agena >>'):
agena >>

> strings.format('%q', 'agena >>'):
"agena >>"

> strings.format('%d\n%2d\n%02d\n%2.5f\n%+2.5f\n+2.5f\n%s', 1, 1, 1,
>   Pi, Pi, -Pi, 'New Horizons'):
1
 1
01
3.14159
+3.14159
-3.14159
New Horizons
```

Summary:

Specifier	Description	Example
%d, %i	writes as an integer	-1, 1
%o	writes as an octal number	12
%b	writes a binary number in the range [-1023, 1023]	-0b1111111111
%u	writes as an unsigned integer, with a cast to C's uint32_t	10
%x, %X	writes as unsigned hexadecimal number, with a cast to uint32_t. %x uses lower-case, %X upper-case	f, F
%f	writes as a floating-point number in normal, fixed-point notation	3.141593
%lf	writes as a floating-point number in normal, fixed-point notation with 16 fractional digits by default	3.1415926535..
%le	writes as a floating-point number in scientic notation with 16 fractional digits by default	3.141592..e+000
%ld	writes a long double in normal, fixed-point notation with 19 fractional digits by default, see **long** package in Chapter 11.15	
%e, %E	writes a floating-point number in exponential notation (scientific e-notation). %e uses lower-case, %E upper-case, with around six fractional digits	3.141593e+000, 3.141593E+000

Specifier	Description	Example
%g, %G	writes a floating-point number in either normal or exponential notation, depending on its magnitude. %g uses lower-case, %G upper-case	1e-006 1E-006
%h, %H	writes a floating-point number in a hexadecimal fractional notation with the exponent to base 2 represented in decimal digits. %h uses lower-case, %H upper-case	0x1.921fb5p+1, 0X1.921FB5P+1
%p	writes in percent	314.159265%
%m	writes a monetary amount with thousands separators and the decimal point defined by the current locale	31,415.93
%n, %N	writes a number using the decimal point separator of the locale of the operating system	31415,926536
%D	writes an integer if the value is a number, and "undefined" otherwise	3, undefined
%F	writes a floating-point number if the value is a number, and "undefined" otherwise	3.141593, undefined
%q, %Q, %B	writes a string put in double (%q), single (%Q) or backquotes (%B) suitable to be safely read back by the interpreter	"3.1415926535.."
%a, %A	like %q but without surrounding quotes	3.1415926535..
%c	writes the corresponding ASCII character	a
%s	writes a string, numbers are automatically converted properly, other data types are converted like the **tostringx** function does	agena, 3.1415926535..
%%	writes percentage sign	%

See also: **%** operator.

strings.fuzzy (s, t)

Compares two strings case-insensitively and returns an estimate of their similarity as both an absolute and relative score, the latter taking into account the length of the longer string.

One point is given for a matching character. Subsequent matches are given two extra points. A higher score indicates a higher similarity. With the second return, 1 depicts equality, and a lower value the degree of similarity.

If at least one of the strings is empty, the function returns **undefined** twice.

See also: **fzy** package, **strings.dice**, **strings.dleven**, **strings.diffs**, **strings.jaro**, **skycrane.tolerance**.

strings.ge (x, y)

Checks whether the string `x` is lexicographically greater than or equal to the string `y`, using the collation of the system, and returns **true** or **false**. The function internally calls the C function strcoll() to conduct the test, see **strings.strcoll**.

See also: <, <=, >, >=, <> operators, **os.setlocale**, **strings.eq**, **strings.gt**, **strings.le**, **strings.lt**, **strings.neq**.

strings.glob (s, pattern [, true])

Compares a string `s` with a string `pattern`, the latter optionally including the wildcards `?` and `*`, where `?` represents exactly one unknown character, and `*` represents zero or more unknown characters. Other pattern matching facilities are not supported.

The return is **true** if the pattern could be found, and **false** otherwise. If the optional third argument is **true**, then the strings will be compared case-insensitively.

See also: **regex.find**, **strings.find**.

strings.gmatch (s, pattern)

Returns an iterator function that, each time it is called, returns the next captures from `pattern` over string `s`. The function supports pattern matching facilities described in Chapter 9.1.3.

If `pattern` specifies no captures, then the whole match is produced in each call.

As an example, the following loop

```
> s := 'hello world from Lua'

> for w in strings.gmatch(s, '%a+') do
>    print(w)
> od
```

will iterate over all the words from string `s`, printing one per line. The next example collects all pairs key~value from the given string into a table:

```
> create table t;

> s := 'from=world, to=Lua'

> for k, v in strings.gmatch(s, '(%w+)=(%w+)') do
>    t[k] := v
> od
```

See also: **strings.match**, **strings.gmatches**, **strings.wrap**, **strings.unwrap**, **tonumber**, **suffix.gfind**.

strings.gmatches (s, pattern)

Wrapper around **strings.gmatch** which returns all occurrences of a substring `pattern` in string `s` a in a new sequence.

The function is written in Agena and included in the lib/library.agn file.

strings.gseparate (s, pattern [, tonumber [, init]])

strings.gseparate (s, pattern [, options])

The function takes a string `s` to be split apart into its tokens one after another, and a delimiter string `pattern`, and returns an iterator function that each time it is called, returns one token. If the end of `s` has been reached, the function returns **null**. The function supports pattern matching. For an iterator without pattern matching, see **strings.iterate**.

If `s` starts with the delimiter, an empty string will be returned.

In the first form, if the Boolean value **true** is passed to `tonumber`, the function tries to convert the token into a number. If `init`, a positive integer is given, the function searches from the `init`'th character in `s`.

In the second form, you can also pass one or more of the following options:

- `convert`=*boolean*, tries to convert the field into a number if *boolean* is **true**,
- `unwrap`=*string*, remove enclosing characters, given in *string*, e.g. double quotes, if existent,
- `init`=*integer*, with *integer* a positive integer, defines where to start the extraction.

If the iterator function is called with any argument, the function returns the number of tokens returned so far but does not search for the next token.

See also: **split**, **strings.fields**, **strings.gmatch**, **strings.iterate**, **strings.separate**.

strings.gsub (s, pattern, repl [, n])

Returns a copy of `s` in which all occurrences of the `pattern` have been replaced by a replacement string specified by `repl`, which may be a string, a table, or a function. **gsub** also returns, as its second value, the total number of substitutions made. See Chapter 9.1.3 for more information on patterns.

If `repl` is a string, then its value is used for replacement. The character % works as an escape character: any sequence in `repl` of the form %n, with n between 1 and 9, stands for the value of the n-th captured substring (see below). The sequence %0 stands for the whole match. The sequence %% stands for a single %.

If `repl` is a table, then the table is queried for every match, using the first capture as the key; if the pattern specifies no captures, then the whole match is used as the key.

If `repl` is a function, then this function is called every time a match occurs, with all captured substrings passed as arguments, in order; if the pattern specifies no captures, then the whole match is passed as a sole argument.

If the value returned by the table query or by the function call is a string or a number, then it is used as the replacement string; otherwise, if it is **false** or **null**, then there is no replacement (that is, the original match is kept in the string).

The optional last parameter `n` limits the maximum number of substitutions to occur. For instance, when `n` is 1 only the first occurrence of pattern is replaced.

Here are some examples:

```
x := strings.gsub('hello world', '(%w+)', '%1 %1')
--> x = 'hello hello world world'

x := strings.gsub('hello world', '%w+', '%0 %0', 1)
--> x = 'hello hello world'

x := strings.gsub('hello world from Lua', '(%w+)%s*(%w+)', '%2 %1')
--> x = 'world hello Lua from'

x := strings.gsub('home = $HOME, user = $USER', '%$(%w+)', os.getenv)
--> x = 'home = /home/roberto, user = roberto'

x := strings.gsub('4+5 = $return 4+5$', '%$(.-)%$', proc (s)
return loadstring(s)()
end)
--> x = '4+5 = 9'

local t := [name~'lua', version~'5.1']
x = strings.gsub('$name%-$version.tar.gz', '%$(%w+)', t)
--> x = 'lua-5.1.tar.gz'
```

See also: || operator, **strings.replace**.

strings.gt (x, y)

Checks whether the string `x` is lexicographically greater than the string `y`, using the collation of the system, and returns **true** or **false**.

The function internally calls the C function strcoll() to conduct the test, see **strings.strcoll**.

See also: <, <=, >, >=, <> operators, **os.setlocale**, **strings.eq**, **strings.ge**, **strings.le**, **strings.lt**, **strings.neq**.

strings.hextodec (s)

Converts a string of any length representing a hexadecimal integral value to a string representing a decimal integer without overflowing internally. The string may optionally start with the prefix '-0x' or '0x' but this is not obligatory. See also: **math.hextodec**.

strings.hirschberg (s, t)

Takes two strings `s` and `t` and returns their global alignment in five results: the edit sequence, the alignment of `s` and the alignment of `t`, the total length of the edit sequence and the Damerau-Levenshtein distance between `s` and `t`, see **strings.dleven**. The function uses the memory-friendly Hirschberg algorithm which yields a total structural space complexity of strictly O(N + M). The function, however, is not suited for larger strings, you may use **strings.dleven** instead.

```
> strings.hirschberg('Alex', 'Axelander'):
=++=+++=!       A  l   ex       Axelander

> strings.hirschberg('Axelander', 'Alex'):
=--=---=!       Axelander       A  l   ex
```

See also: **strings.diffs**, **strings.fuzzy**, **strings.leven**.

strings.hits (s, pattern [, true])

Returns the number of occurrences of substring `pattern` in string `s`.

If only two arguments are passed, pattern matching facilities (see Chapter 9.1.3) are supported. If the Boolean constant **true** is passed as a third argument, pattern matching is switched off for faster execution.

See also: **strings.words**.

strings.include (s, pos, p)

Inserts the string `p` into the string `s` at position `pos`.

If `pos` ≤ **size** `s`, the character at position `pos` is moved size `p` places to the right.

If `pos` = **size** `s` + 1, `p` is just appended to `s`, equal to the Agena expression `s` **&** `p`.

The function returns the new string and issues an error, if the index `pos` is invalid. p may be the empty string, in this case, `p` will be returned.

See also: **strings.between**, **strings.remove**.

strings.instr (s, pattern [, init] [, plain] [, 'reverse'] [, 'borders'])

Looks for the first match of string `pattern` in the string `s`. If it finds a match, then **strings.instr** will return the index of `s` where this occurrence starts; otherwise, it will return **null**. It also will return **null** if `pattern` is the empty string.

If `pattern` is a set of pattern strings, returns **true** if at least one of the patterns matches `s`; otherwise returns **false**.

If the option `'reverse'` is given, then the search will start from the right end and always runs to its left beginning and the first occurrence of `pattern` with respect to the beginning of `s` will be returned. In the reverse search, pattern matching is not supported.

An optional numerical argument `init` passed anywhere after the second argument specifies where to start the search; its default value is 1 and may be negative. In the latter case, the search is started from the `|init|`'s position from the right end of `s`.

The function by default supports pattern matching, almost similar to regular expressions, see Chapter 9.1.3. **strings.instr** is 45 % faster than **strings.find**. If the optional Boolean argument `plain` is set to the Boolean **true**, pattern matching is switched off and a much faster plain search is conducted instead (speed bonus around 40 %).

The optional argument `'borders'` returns the start and the end position of a match in a pair. The `'borders'` option will be ignored if pattern is a set.

See also: **atendof**, **in**, **strings.isstarting**, **strings.isending**, **strings.find**.

strings.isaligned (s)

Checks whether the string `s` is aligned on the 4-byte word boundary and returns **true** or **false**.

strings.isalpha (s [, t])

Checks whether the string `s` consists entirely of alphabetic letters (including diacritics) and returns **true** or **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might additionally match one of the characters in `t`.

See also: **strings.contains**, **strings.isdia**, **strings.isisoalpha**, **strings.islatin**, **strings.isloweralpha**, **strings.ismagic**, **strings.isupperalpha**.

strings.isalphanumeric (s [, t])

Checks whether the string `s` consists entirely of numbers or alphabetic letters (including diacritics) and returns **true** or **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might additionally match one of the characters in `t`.

See also: **strings.contains**, **strings.islatinnumeric**.

strings.isalphaspace (s [, t])

Checks whether the string `s` consists entirely of alphabetic letters (including diacritics) and/or a white space and returns **true** or **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might additionally match one of the characters in `t`.

See also: **strings.ltrim**, **strings.rtrim**, **strings.trim**.

strings.isalphaspec (s)

Checks whether the string `s` consists entirely of the Latin letters a to z, A to Z, or all characters that are not blanks or alphanumeric, and returns **true** or **false**.

See also: **strings.isascii**, **strings.isspec**, **strings.isalphaspace**.

strings.isascii (s)

Checks whether the string `s` consists entirely of C unsigned char 7-bit characters that fit into the UK/US character set. It is a direct port to the C function `isascii`, and returns **true** or **false**. See also: **strings.isdia**.

strings.isblank (s [, true])

Checks whether the string `s` consists entirely white spaces or tabulators (\t) and returns **true** or **false**. If the option **true** is given, the function checks for tabs, linefeeds, carriage returns, white spaces, vertical tabs, and form feeds.

See also: **strings.isisospace**, **strings.isspace**.

strings.iscenumeric (s [, option])

Checks whether the string `s` consists entirely of the digits 0 to 9 and optionally exactly one decimal comma at any position, and returns **true** or **false**. If `option` is set to **true**, the function ignores preceding signs (+ or -), the default is **false**.

See also: **strings.isfractional**, **strings.isintegral**, **strings.isnumeric**, **os.setlocale**.

strings.iscomplex (···)

Checks whether all string arguments represent complex numbers that have the imaginary unit `I` at their end and returns **true** or **false**. In this context, valid complex numbers are `I`, `0*I`, `0+I`, `-1-I`, `+1-I`, etc. Invalid complex numbers are `1`, `I*0`, `1+I*(-1)`, etc.

See also: **tonumber**, **strings.isnumber**, **strings.isnumeric**.

strings.isconsonant (s)

Checks whether the string `s` consists entirely of lower or upper-case Latin consonants and returns **true** or **false**.

See also: **strings.isalpha**, **strings.isdia**, **strings.isvowel**.

strings.iscontrol (s)

Checks whether the string `s` consists entirely of control characters and returns **true** or **false**. Control characters are: '\0', bell, backspace, tab, linefeed, carriage return, and all other characters between ASCII code 0 and 31, plus the DEL key (ASCII code127). The function is the opposite to **strings.isprintable**.

See also: **strings.isblank**, **strings.isprintable**, **strings.isspec**.

strings.isdia (s [, true])

Checks whether the string `s` consists entirely of diacritics (such as á, â. ø, Ü) and ligatures (such as ß, Æ) and returns **true** or **false**. The function works correctly with the ISO/IEC 8859-1 character set only. If the second argument is **true**, then the function checks whether `s` includes at least one diacritic or ligature.

See also: **strings.isascii**, **strings.isalpha**.

strings.isending (s, pattern [, true])

Determines whether a string `s` is ending in the substring `pattern`, i.e. whether `pattern` fits entirely to the end of the string `s` in case the length of `pattern` is less than that of `s`. The function returns **true** or **false.**

If only two arguments are passed, pattern matching facilities (see Chapter 9.1.3) are supported. If the Boolean constant **true** is passed as a third argument, pattern matching is switched off for faster execution.

If `s` or `pattern` are empty strings or both are the same, the function returns **false**.

The function can be useful in linguistics if you want to check whether a word has a given inflectional ending.

See also: **strings.isstarting**, **atendof**.

strings.isfractional (s [, option [, base]])

Checks whether the string `s` consists entirely of the digits 0 to 9 and exactly one decimal point (or the decimal-point separator at your locale) at any position and returns **true** or **false**. If `option` is set to **true**, the function also ignores preceding signs (+ or -), the default is **false**.

By default, `base` is 10 for decimal numbers, but you may set it to any other base (or radix).

See also: **strings.isintegral**, **strings.isnumeric**, **os.setlocale**.

strings.ishex (s [, option])

Checks whether the string `s` represents a hexadecimal number which consists of the digits 0 to 9 and or the letters 'a' to 'f' or 'A' to 'F', and returns **true** or **false**. If `option` is set to **true**, the function also ignores preceding signs (+ or -), the default is **false**.

See also: **strings.isintegral**, **strings.isnumeric**, **utils.hexlify**.

strings.isgraph (s)

Checks whether the string `s` consists of glyphs only. It is a direct port to the C function `isgraph`, and returns **true** or **false**.

strings.isintegral (s [, option [, base]])

Checks whether the string `s` consists entirely of the digits 0 to 9 and returns **true** or **false**. If `option` is set to **true**, the function also ignores preceding signs (+ or -), the default is **false**. By default, `base` is 10 for decimal integers, you may set it to any other base (or radix).

See also: **strings.isfractional**, **strings.ishex**, **strings.isnumeric**, **os.setlocale**.

strings.isisoalpha (s)

Checks whether the string `s` consists entirely of ISO 8859/1 Latin-1 alphabetic lower and upper-case characters (including diacritics) and returns **true** or **false**. The function only correctly recognises strings read from a file. Mostly, it cannot process ligatures input in a shell, e.g. the Windows NT or Mac console.

See also: **strings.isalpha**, **strings.isolower**, **strings.isoupper**.

strings.isisolower (s)

Checks whether the string `s` consists entirely of ISO 8859/1 Latin-1 alphabetic lower-case characters (including diacritics) and returns **true** or **false**. The function only correctly recognises strings read from a file. Mostly, it cannot process ligatures input in a shell, e.g. the Windows NT or Mac console.

See also: **strings.isalpha**, **strings.isloweralpha**.

strings.isisoprint (s)

Checks whether the string `s` consists entirely of printable ISO 8859/1 Latin-1 letters and returns **true** or **false**.

strings.isisospace (s)

Checks whether the string `s` consists entirely of ISO 8859/1 Latin-1 white spaces and returns **true** or **false**.

See also: **strings.isblank**, **strings.isspace**.

strings.isisoupper (s)

Checks whether the string `s` consists entirely of ISO 8859/1 Latin-1 alphabetic upper-case characters (including diacritics) and returns **true** or **false**. The function only correctly recognises strings read from a file. Mostly, it cannot process ligatures input in a shell, e.g. the Windows NT or Mac console.

See also: **strings.isalpha**, **strings.isisolower**, **strings.isupperalpha**,**strings.isisoupper**.

strings.islatin (s [, t])

Checks whether the string `s` entirely consists of the characters 'a' to 'z', and A' to 'Z'. It returns **true** or **false**. If `s` is the empty string, the result is always **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might match one of the characters in `t`.

See also: **strings.isalpha**, **strings.isascii**, **strings.islowerlatin**, **strings.isupperlatin**.

strings.islatinnumeric (s [, t])

Checks whether the string `s` consists entirely of numbers or Latin letters 'a' to 'z' and 'A' to 'Z', and returns **true** or **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might match one of the characters in `t`.

See also: **strings.isalphanumeric**.

strings.isloweralpha (s [, t])

Checks whether the string `s` consists entirely of the characters a to z and lower-case diacritics, and returns **true** or **false**. If `s` is the empty string, the result is always **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might match one of the characters in `t`.

See also: **strings.isisolower**, **strings.isupperalpha**.

strings.islowerlatin (s [, t])

Checks whether the string `s` consists entirely of the characters 'a' to 'z', and returns **true** or **false**. If `s` is the empty string, the result is always **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might match one of the characters in `t`.

See also: **strings.isupperlatin**.

strings.ismagic (s)

Checks whether the string `s` contains one or more magic characters and returns **true** or **false**. In this function, magic characters are anything unlike the letters 'A' to 'Z', 'a' to 'z', and the diacritics listed at the top of this chapter.

See also: **has**, **strings.isalpha**, **strings.isinstring**.

strings.ismultibyte (s)

Detects whether the given string `s` is in UTF-8 encoding and returns two booleans (**true** or **false**): The first Boolean indicates that `s` is compliant to the UTF-8 standard. Remember that a string in ASCII or ISO 8859 encoding is also a valid UTF-8 string.

The second Boolean indicates that `s` contains at least one multi-byte UTF-8 character, i.e. that at least one character is part of the UTF-8 but not of the ASCII or ISO 8859 standard.

If an integer is returned as a third argument, it will denote the position where the string did not meet UTF-8 criteria.

Please note that the function may not produce correct results with text input in a console. The function can only return correct results if the string to be checked has been read from a file.

See also: **strings.isutf8**, **strings.isisoalpha**.

strings.isnumber (···)

Checks whether one or more strings all represent real or complex numbers and returns **true** or **false**.

For each string s in the argument list it actually performs the check:

strings.isnumeric(s, **true**) or **strings.iscomplex**(s)

as long as the result is **true**.

See also: **tonumber**, **strings.iscomplex**, **strings.isnumeric**.

strings.isnumberspace (s [, option])

Checks whether the string `s` consists entirely of the digits 0 to 9 or white spaces and returns **true** or **false**. The function returns **true** even if it found one or more spaces embedded between two digits:

```
> strings.isnumberspace(' 1 23 '):
true
```

If you want the function to return **false** with embedded spaces, you can set `option` to **true**:

```
> strings.isnumberspace(' 1 23 ', true):
false
```

strings.isnumeric (··· [, option])

Checks whether one or more strings all consist entirely of the digits 0 to 9 or entirely of digits plus exactly one optional decimal point (or the decimal-point separator at your locale) at any position, and returns **true** or **false**. If `option` is set to **true**, the function also checks for an *optional* preceding sign (+ or -), the default is **false**.

See also: **tonumber**, **strings.iscenumeric**, **strings.iscomplex**, **strings.isfractional**, **strings.isintegral**, **strings.isnumber**, **os.setlocale**.

strings.isolower (s)

Receives an ISO 8859/1 Latin-1 string `s` and returns a copy of this string with all upper-case letters changed to lower-case. The operator leaves all other characters unchanged. `s` may include embedded zeros, which are preserved in the output.

See also: **strings.lower**, **strings.isoupper**.

strings.isoupper (s)

Receives an ISO 8859/1 Latin-1 string `s` and returns a copy of this string with all lower-case letters changed to upper-case. The operator leaves all other characters unchanged. `s` may include embedded zeros, which are preserved in the output.

See also: **strings.lower**, **strings.isoupper**.

strings.ispattern (s [, any])

Checks whether the given string `s` represents a valid pattern or capture accepted by **strings.match**, **strings.find**, etc. and returns **true** or **false**. The second return, a Boolean, indicates whether `s` contains at least one special character or meta-character used in patterns or captures, that is anything in `"^$*+?.([%-"`.

If any second argument is given then the functions only checks for meta-characters in string `s` and returns **true** or **false**.

See also: **strings.isspec**.

strings.isprintable (s)

Checks whether the string `s` consists entirely of characters that can be output at the console (characters with ASCII codes 32 to 255 except the backspace) and returns **true** or **false**. The function is the opposite to **strings.iscontrol**.

strings.isspace (s)

Checks whether the string `s` consists entirely white spaces and returns **true** or **false**. Tabulators `\t` are not considered to be white spaces.

See also: **strings.isblank**, **strings.isisospace**.

strings.isspec (s [, any])

Checks whether the string `s` consists entirely of punctuation characters (any printing character that is not a white space or alphanumeric), including

white space `¿ ? ¡ ! " # $ @ § % & ' ` * / + − . , ; ( ) [ ] { } | ¦ \ ^ _ ~ = < >`

and returns **true** or **false**.

See also: **strings.isalphaspec**, **strings.isblank**, **strings.ispattern**, **strings.isspace**, **strings.ismagic**.

strings.isstarting (s, pattern [, true])

Determines whether a string `s` is beginning with the substring `pattern`, i.e. whether `pattern` fits entirely to the beginning of the string `s` in case the length of `pattern` is less than that of `s`. The function returns **true** or **false.**

If only two arguments are passed, pattern matching facilities (see Chapter 9.1.3) are supported. If the Boolean constant **true** is passed as a third argument, pattern matching is switched off for faster execution.

If `s` or `pattern` are empty strings or have the same length, the function returns **false**.

The function can be useful in linguistics if you want to check whether a word has a given prefix.

See also: **strings.isending**, **atendof**.

strings.issubseq (s, t)

The function checks whether `s` represents a subsequence of characters of string `t`.

A subsequence of a string is a new string which is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (that is, "ace" is a subsequence of "abcde" while "aec" is not). Examples:

```
> strings.issubseq('gen', 'agena'):
true

>  strings.issubseq('gn', 'agena'):
true

> strings.issubseq('eg', 'agena'):
false
```

See also: **strings.dleven**, **strings.lcs**, **strings.leven**.

strings.isupperalpha (s [, t])

Checks whether the string `s` consists entirely of the capital letters 'A' to 'Z' and upper-case diacritics, and returns **true** or **false**.

If `s` is the empty string, the result is always **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might match one of the characters in `t`.

See also: **strings.isisoupper**, **strings.isloweralpha**.

strings.isupperlatin (s [, t])

Checks whether the string `s` consists entirely of the capital letters 'A' to 'Z', and returns **true** or **false**. If `s` is the empty string, the result is always **false**.

If the optional string `t` representing a character set is given, the function also checks whether any character in `s` might match one of the characters in `t`.

See also: **strings.islowerlatin**.

strings.isutf8 (s)

Detects whether the given string `s` contains at least one multibyte and return **true** or **false**.

See also: **strings.isisoalpha**, **strings.ismultibyte**, **strings.utf8size**.

strings.isvowel (s [, withy])

By default, checks whether string `s` consists entirely of the letters 'a', 'e', 'i', 'o', 'u' or 'y', or their upper-case equivalents, and returns **true** or **false**. If optional `withy` is **false**, then 'y', 'Y' are not considered vowels.

See also: **strings.isalpha**, **strings.isdia**, **strings.isconsonant**.

strings.iswrapped (s, t)

Checks whether string `s` is enclosed by string `t` and returns **true** or **false**.

See also: **strings.unwrap**, **strings.wrap**.

strings.iterate (s [, pos [, n]])

strings.iterate (s, 0 [, option])

strings.iterate (s, delim [, options])

In the first form, returns an iterator function that, when called returns the next `n` characters in string `str`, starting at position `pos`. `pos` and `n` are 1 by default.

In the second form, when `pos` is zero, returns an iterator function that from the left to right returns each four consecutive characters in string `str` as an unsigned 4-byte integer. The iterator returns Little Endian integers unless the third argument is set to **true** to return Big Endian integers.

In the third form, by passing a string `del` of one or more delimiters as the second argument, returns an iterator function that step-by-step returns a field surrounded by at least one of the delimiters.

With the third form, you can also pass one or more of the following options:

- `convert`=*boolean*, tries to convert the field into a number of *boolean* is **true**,
- `unwrap`=*string*, remove enclosing characters, given in *string*, e.g. double quotes, if existent.

Pattern-matching is not supported, use **strings.gseparate** instead.

If there are no more characters to process, the iterator returns **null**.

See also: **split**, **strings.fields**, **strings.gseparate**, **strings.separate**, **strings.tobytes**.

strings.itouni (n)

Converts an integer `n` in the range [0, 0x10ffff] into its Unicode representation, a string. If `n` is 0, returns '\0', not the empty string. If `n` is out-of-bounds, the function returns **null**.

See also: **strings.tolatin**, **strings.toutf8**.

strings.jaro (s1, s2 [, option])

Computes either the Jaro similarity or Jaro-Winkler similarity, the measure of two strings' `s1`, `s2` similarity: the greater the value, the more similar the strings are. The score is normalised such that 0 equates to no similarities and 1 is an exact match.

By default, the function takes into account that if up to four characters at the start of the strings match, the score will be higher - thus computing the Jaro-Winkler similarity. If `option` is set to **false**, the Jaro similarity is returned which does not check whether strings match at their beginning.

Running times depend on the similarity between the two strings: the more similar they are, the faster the function computes the result.

See also: **strings.dice**, **strings.diff**, **strings.dleven**, **strings.fuzzy**, **strings.lcs**, **strings.leven**.

strings.join (obj [, sep [, i [, j]]])

Concatenates all string, number, complex number and Boolean values in the table, sequence or register `obj` in sequential order and returns a string: `obj[i]` & `sep` & `obj[i+1]` ··· & `sep` & `obj[j]`. The default value for `sep` is the empty string, the default for `i` is 1, and the default for `j` is the length of the sequence.

The function issues an error if `obj` contains non-strings. **null** will be treated like the empty string.

See also: **&** operator.

strings.lcs (s, t)

Computes the longest common subsequence (LCS) of two strings `s`, `t`, that is the longest subsequence of characters that is common to both `s` and `t`. The algorithm observes the order of characters.

The LCS allows for insertions and deletions, but not substitutions and transpositions.

The returns are the length of the LCS plus the LCS itself. The function is very fast even with large `s`, `t`.

See also: **strings.dice**, **strings.diffs**, **strings.dleven**, **strings.fuzzy**, **strings.jaro**, **strings.leven**, **strings.lrs**.

strings.le (x, y)

Checks whether the string `x` is lexicographically less than or equal to the string `y`, using the collation of the system, and returns **true** or **false**. The function internally calls the C function strcoll() to conduct the test, see **strings.strcoll**.

See also: <, <=, >, >=, <> operators, **os.setlocale**, **strings.eq**, **strings.ge**, **strings.gt**, **strings.lt**, **strings.neq**.

strings.leven (s, t [, option])

Returns the Levenshtein distance between two strings `s` and `t`. It is a count of the minimum number of insertions, deletions, substitutions of a single character to convert `s` into `t`, but does not take transpositions into account. For more information on its use, see **strings.dleven**.

The function is very fast even with large input.

```
> strings.leven('alex', 'paely'), strings.dleven('alex', 'paely'):
4   3
```

See also: **strings.fuzzy**, **strings.dice**, **strings.diffs**, **strings.hirschberg**, **strings.jaro**, **strings.lcs**, **skycrane.tolerance**.

strings.ljustify (s, width [, filler])

Adds filling characters to the back of string `s`, as necessary to return a new string of the given `width`. If `s` is a number, it is automatically converted to a string before padding starts.

The filling characters may be denoted by the third optional argument `filler` (number or string), otherwise `filler` is a white space by default. If the resulting string is longer than the given `width`, it is truncated to the first `width` characters.

See also: **strings.lpad**, **strings.rjustify**, **strings.rpad**

strings.lower (s [, option])

The function receives string `s` and returns a copy of this string with all uppercase letters ('A' to 'Z' plus the above mentioned diacritics) changed to lowercase ('a' to 'z' and the diacritics listed at the end of Chapter 9.1). The function leaves all other characters unchanged. Example:

```
> strings.lower('Elektronika MK-61'):
elektronika mk-61
```

If any `option` is given, then only Latin characters A-Z are put to lower-case.

See also: **strings.isolower**, **strings.upper**.

strings.lpad (s, width [, filler])

Adds filling characters to the *left* side (the front) of string `s`, as necessary to return a new string of the given `width`. If `s` is a number, it is automatically converted to a string before padding starts.

The filling characters may be denoted by the third optional argument `filler` (number or string), otherwise `filler` is a white space by default. If the resulting string is longer than the given `width`, it is truncated to the first `width` characters.

The function actually calls **strings.rjustify** to do the job.

See also: << operator, **strings.rpad**.

strings.lrs (s)

Returns the largest substring in string `s`.

The function is written in Agena and included in the lib/library.agn file.

See also: **strings.dice**, **strings.diffs**, **strings.dleven**, **strings.fuzzy**, **strings.jaro**, **strings.lcs**, **strings.leven**.

strings.lrtrim (s [, c])

Returns a new string with all leading and trailing white spaces removed from `s`. If a single character is passed for `c` as an optional second argument, then all leading and trailing characters given by `c` will be removed. If `c` is a multi-character string,

then, if existing, it is removed once from the start and once from the end of `s`. The function supports pattern matching.

It does not remove spaces or the given character(s) within the `actual` part of the string.

See also: **strings.ltrim**, **strings.rtrim**, **strings.trim**, **strings.wrap**.

strings.lt (x, y)

Checks whether the string `x` is lexicographically less than the string `y`, using the collation of the system, and returns **true** or **false**. The function internally calls the C function strcoll() to conduct the test, see **strings.strcoll**.

See also: <, <=, >, >=, <> operators, **os.setlocale**, **strings.eq**, **strings.ge**, **strings.gt**, **strings.lt**, **strings.neq**.

strings.ltrim (s [, c])

Returns a new string with all leading white spaces removed from `s`. If a single character is passed for `c` as an optional second argument, then all leading characters given by `c` will be removed. If `c` is a multi-character string, then, if existing, it is removed once from the start of `s`. The function supports pattern matching.

See also: **strings.lrtrim**, **strings.remove**, **strings.rtrim**, **strings.trim**.

strings.match (s, pattern [, init])

Looks for the first match of `pattern` in string `s`. If it finds one, then match returns the captures from the pattern; otherwise it returns **null**. If `pattern` specifies no captures, then the whole match will be returned. A third, optional numerical argument `init` specifies where to start the search; its default value is 1 and may be negative.

The function supports pattern matching facilities. For examples and help in case of problems, see Chapter 4.7.7 and 9.1.3.

See also: **strings.gmatch**, **strings.matches**, **skycrane.xmlmatch**.

strings.matches (s, pattern [, init])

Works like **strings.match**, but returns all matches in only one call.

Example:

```
> strings.matches('St. Petersburg, Europe', '([äöüßÄÖÜ%a]*)'):
St              Petersburg              Europe
```

strings.mfind (s, pattern [, init [, plain]])

Like **strings.find**, but looks for all the matches of `pattern` in the string `s`.

If it finds at least one match, it returns a sequence with at least one pair indicating where the respective match starts and ends, otherwise, it returns **null**.

A third, optional numerical argument `init` specifies where to start the search; its default value is 1 and may be negative. A value of **true** as a fourth, optional argument `plain` turns off the pattern matching facilities (see Chapters 4.7.7 and 9.1.3), so the function does a plain `find substring` operation, with no characters in `pattern` being considered `magic`. This can significantly speed up the search. Note that if `plain` is given, then `init` must be given as well.

Contrary to **strings.find**, if the pattern has captures, then in a successful match the captured values are not returned.

See also: **in**, **atendof**, **strings.find**, **strings.instr**, **strings.matches**, **suffix.findall**.

strings.neq (x, y)

Checks whether the string `x` is not equal to the string `y`, using the collation of the system, and returns **true** or **false**. The function internally calls the C function strcoll() to conduct the test, see **strings.strcoll**.

See also: <, <=, >, >=, <> operators, **strings.eq**, **strings.ge**, **strings.gt**, **strings.le**, **strings.lt**.

strings.ngrams (s, n)

The function produces n-grams of string `s`. `n` should be a positive integer and not larger than the size of `s`. The return is a sequence of the ordered n-grams of `s`.

See also: **strings.bigrams**.

strings.obfusxor (s, k)

Obfuscates a string `s` using binary-xor and a key string `k`. The output has the same length as the input. If key `k` is shorter than `s`, it is repeated as much as necessary.

To get back the original string, just call the function with the output and the same key again.

See also: **strings.tobytes**, **utils.hexlify**, **utils.unhexlify**.

strings.pack (fmt, v1, v2, ···)

Returns a binary string containing the values `v1`, `v2`, etc. serialized in binary form (packed) according to the format string `fmt`, see Chapter 9.1.4 for the formatters and examples.

See also: **strings.packsize**, **strings.unpack**, **binpack library**.

strings.packsize (fmt)

Returns the size of a string resulting from **strings.pack** with the given format. The format string cannot have the variable-length options `'s'` or `'z'`. For format strings and examples, see Chapter 9.1.4.

strings.random (length [, kind [, l [, u]]])

strings.random (length, alphabet [, option])

In the first form, creates a random string of the given fixed `length`. By default, i.e. `kind` is set to `'base64'`, a Base64 string consisting of the characters

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/

will be returned. If the second argument kind is `'ascii'`, a random ASCII string consisting of characters in the range ASCII 32 to ASCII 126 will be returned. You can change the upper and lower bounds by explicitly passing the non-negative integers `l` and `u`.

In the second form, the random string will consist entirely of `length` characters given in `alphabet`, such as `'ABCDEFG012345'`:

```
> strings.random(16, 'abcdef01234'):
cdbfe314f00af1be
```

When you pass **true** as a third argument, the default, the function will always produce really random strings each time it is called. When setting `option` to **false** and the function is subsequently called in a session, it will always produce the same sequence of random `random` strings.

See also: **math.random**.

strings.remove (s, pos [, len])

strings.remove (s, p [, n])

strings.remove (s, p [, ···])

The function removes a substring from a string. It supports pattern matching, see Chapter 9.1.3.

In the first form, starting from string position `pos`, the function removes `len` characters from string `s`. The return is a new string. If `len` is not given, it defaults to one character to be deleted.

It is not an error if `len` is greater than the actual length of `s`. In this case all characters starting at position `pos` are deleted.

In the second form, substring `p` is removed `n` times from string `s`. The default for `n` is **infinity**, i.e. all occurrences of `p` are removed.

In the third form, one or more substrings `p`, ... are removed from string `s`, in the order of the arguments.

As the function supports pattern matching: If one of the magic characters `^$()%.[]*+-?)` is to be removed, prepend a percentage sign, for example:

```
> strings.remove('[a,b]', '%[', '%]'):
a,b
```

See also: **-** operator, **strings.include**, **strings.replace**, **strings.ltrim**, **strings.rtrim**.

strings.repeat (s, n [, delimiter])

Returns a string that is the concatenation of `n` copies of the string `s`. An optional `delimiter` string may also be given. If `n` is zero, the empty string will be returned.

See also: ***** operator.

strings.replace (s1, s2, s3)

strings.replace (s1, obj)

strings.replace (s1, pos, s2)

In the first form, the function replaces all occurrences of string `s2` in string `s1` by string `s3`.

In the second form, the function receives a string `s1` and a table, sequence or register `obj` of one or more string pairs of the form `s2:s3` and replaces all occurrences of `s2` in string `s1` with the corresponding string `s3`. Thus you can replace multiple patterns simultaneously with only one call to **replace**.

In the third form, the function inserts a new string `s2` into the string `s1` at the given position `pos`, substituting the respective character in `s1` with the new string `s2` which may consist of zero, one or more characters. The return is a new string. If `s2` is the empty string, the character in `s1` is deleted. The return is always a new string.

The function does not support pattern matching, use **strings.gsub** instead.

See also: **utils.singlesubs**.

strings.reverse (s)

Returns a string that is the string `s` reversed. See also: **reverse**, **stack.reversed**.

strings.rjustify (s, width [, filler])

Adds filling characters to the front of string `s`, as necessary to return a new string of the given `width`. If `s` is a number, it is automatically converted to a string before padding begins. The filling characters may be denoted by the third optional argument `filler` (number or string), otherwise `filler` is a white space by default.

If the resulting string is longer than the given `width`, it is truncated to the last `width` characters.

See also: **strings.lpad**, **strings.ljustify**.

strings.rpad (s, width [, filler])

Adds filling characters to the back of string `s`, as necessary to return a new string of the given `width`. If `s` is a number, it is automatically converted to a string before padding starts.

The filling characters may be denoted by the third optional argument `filler` (number or string), otherwise `filler` is a white space by default. If the resulting string is longer than the given `width`, it is truncated to the first `width` characters.

The function actually calls **strings.ljustify** to do the job.

See also: >> operator, **strings.lpad**.

strings.rtrim (s [, c])

Returns a new string with all trailing white spaces removed from `s`.

If a single character is passed for `c` as an optional second argument, then all trailing characters given by `c` will be removed. If `c` is a multi-character string, then if existing it is removed once from the end of `s`.

The function supports pattern matching.

See also: **strings.lrtrim**, **strings.ltrim**, **strings.remove**, **strings.trim**.

strings.rotateleft (s, n [, xorkey [, xorval]])

Rotates all the bits in the string `s` `n` bits to the left, with `n` in range 0 .. 7.

The `n` bits dropping off the beginning of the string will be appended to the resulting string, so that there is no information loss when calling **strings.rotateright** to decrypt it.

For optional arguments `xorkey` and `xorval` see **strings.rotateright**.

strings.rotateright (s, n [, xorkey [, xorval]])

Rotates all the bits in the string `s` `n` bits to the right, with `n` in range 0 .. 7.

The `n` bits dropping off the end of the string will be prepended to the resulting string, so that there is no information loss when calling **strings.rotateleft** to decrypt it.

You can optionally xor the string by passing the third argument `xorkey`, an integer in the range 0 .. 255. (Note that in case the string might be corrupted, the function issues an error.) By explicitly setting the optional fourth argument `xorval` to **true**, you can achieve further obfuscation of the string while xoring.

strings.separate (s, d [, any])

Splits a string `s` into its tokens. `d` is a string that specifies a set of delimiters that may surround the token to be extracted. Thus, the delimiter in front of a token may be different from the delimiter at its end.

All the tokens or returned in a sequence in sequential order. If `s` only consists of characters that are part of `d`, or if `s` or `d` are empty strings, the function returns **fail**.

```
> strings.separate('a word, another word.', ' .,'):
seq(a, word, another, word)
```

If `any` third argument is passed, then a) the function returns a sequence with one empty string if `s` is the empty string instead of **fail**, and b) if none of the delimiters could be found in `s`, returns a sequence with `s` in it instead of **fail**.

See also: **split** operator, **strings.fields**, **strings.iterate**, **strings.gseparate**.

strings.shannon (s)

Returns the normalised specific Shannon entropy, the specific Shannon entropy, and the total information entropy (in bits) for string `s`, in this order.

The function does not look for any patterns that might be available for compression, so its use is quite limited and **gzip.deflate** with the `true` option as third argument might be a better alternative.

See also: **strings.walker**.

strings.strchr (s, i)

strings.strchr (s, c)

The function is an interface to the C strchr() function, searches `s` for a single character represented by its ASCII code `i` or the character `c` (a string of size 0 or 1) and returns a substring starting from the first match to the end of `s`. The second return is the position of the match, starting from 1. It returns **null** and 0 if no match was found and issues an error if needle is non-positive. If `s` is **null**, the function returns **null** to mimic undefined behaviour in C.

See also: **abs**, **strings.strrchr**, **strings.strstr**.

strings.strcmp (s1, s2)

The function calls the C function strcmp() and returns its result, "a value that has the same sign as the difference between the first differing pair of characters" (GNU C Library manual). If the result is negative, then `s1` < `s2`; if it is zero, then `s1` = `s2`, else `s1` > `s2`. If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **strings.compare**, **strings.strcoll**, **strings.stricmp**, **strings.strncmp**, **strings.strstr**, **strings.strverscmp**.

strings.strcoll (s1, s2 [, lang])

Similar to **strings.strcmp**, but dependent on the current locale, see **os.setlocale**. The check may vary across platforms. The function is an interface to the C strcoll() function.

The function accepts a third argument `lang`, a string, that determines the locale to be used for the comparison of two strings, just for the single call and without permanently changing the locale on the system. Examples:

```
> strings.strcoll('aäüßou', 'aausou'):
1

> strings.strcoll('aäüßou', 'aausou', 'German'):
-1
```

On some platforms you may have to pass a combination of the ISO 639-1 language code and the ISO 3166-1 region code instead of the full language name, e.g.:

```
> strings.strcoll('aäüßou', 'aausou', 'de_DE'):
```

See also: **strings.compare**, **strings.strverscmp**, **os.setlocale**, **skycrane.getlocales**.

strings.strcspn (s1, s2)

The function checks `s1` for the first occurrence of any of the characters that are part of `s2` and returns the number of characters of `s1` read before this first occurrence. The function returns the length of string `s1` if no characters of `s1` matched to `s2`. If `s1` or `s2` are empty strings, the function will return 0. The function is an interface to the C strcspn() function. If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

Example:

```
> strings.strcspn('abcdef012', '0123456789'):
6
```

See also: **member**, **strings.contains**, **strings.strspn**.

strings.strftime (seconds, format)

strings.strftime (obj, format)

Takes the number of seconds elapsed since the epoch or a table of date and time components and returns a formatted timestamp, a string.

In the first form, the `seconds` argument represents the number of seconds elapsed since a given epoch (usually January 01, 1970).

In the second form, the function receives a date and optionally time of the form year, month, date [, hour [, minute [, second [, milliseconds]]]], with all values put into the table, sequence or register `obj` in this order, and a format string.

The following date and time specifiers always refer to the current locale, and may not be fully supported by your operating system - if not, an empty string will be returned:

Spec	Meaning	strftime	strptime	Example
%a	with strings.strftime: abbreviated weekday name with strings.strptime: all consecutive Latin letters, digits or characters with ASCII code > 127 will be skipped	yes	yes	'Fri' for Friday
%A	full weekday name (same as above)	yes	yes	'Friday'
%b	abbreviated month name	yes	no	'Apr' for April
%B	full month name (same as above)	yes	no	'April'

Spec	Meaning	strftime	strptime	Example
%c	preferred calendar time representation	yes	no	'05/19/17 20:33:13' for UK
%C	century of the year, greatest integer not greater than year divided by 100.	yes	yes	'19'
%d	day of the month as two-digit integer	yes	yes	'03'
%D	date of the format %m/%d/%y	yes	yes	'01/01/70'
%e	day of the month like with %d, but padded with blank instead of a zero	yes	yes	' 1'
%F	date of the format %Y-%m-%d	yes	yes	'1970-01-01'
%g, %G	year corresponding to the ISO week number, but without the century	yes	no	'70' '1970'
%h	abbreviated name of month	yes	no	'Jan'
%H	two-digit hour in the range 00 .. 23	yes	yes	'01'
%I	two-digit hour in the range 00 .. 12	yes	yes	'01'
%j	three-digit day of year in the range 001 .. 366	yes	yes	'001'
%k	hour in the range 0 .. 23, like %H, padded with a blank	yes	no	' 1'
%l	hour in the range 0 .. 12, like %H, padded with a blank	yes	no	' 1'
%m	month in the range 01 .. 12	yes	yes	'05'
%M	minute in the range 00 .. 59	yes	yes	'01'
%n	newline (\n)	yes	yes	
%p	AM or PM	yes	yes	'PM'
%P	am or pm	yes	yes	'pm'
%Q	milliseconds	yes	yes	'123'
%R	hour and minute, like %H:%M	yes	yes	'01:00'
%s	number of seconds since the epoch; when given a format string for second argument: milliseconds	yes	no	'0' 451
%S	seconds in the range 00 .. 60	yes	yes	'01'
%t	tabulator, incl. blank (strptime: one or more)	yes	yes	(a tab, '\t')', ' ')
%T	time of day of format %H:%M:%S	yes	yes	'01:00:00'
%u	day of week as a decimal number range 1 (Monday) to through 7	yes	no	'4'
%U, %V	week number in the range 00 ..53, starting with the first Sunday as the first day of the first week	yes	no	'18'

Spec	Meaning	strftime	strptime	Example
%w	day of week in the range 0 (Sunday) to 6	yes	yes	'1'
%W	week number of current year in the range 00 .. 53, starting with the first Monday as the first day of the first week	yes	no	'18'
%x	preferred date representation	yes	no	'05/03/17' for UK
%X	preferred time of day representation	yes	no	'21:09:02' for UK
%y	year in the range 00 .. 99	yes	yes	'17'
%Y	full year number	yes	yes	'1949'
%%	character '%'	yes	yes	'%'

Please note that the behaviour is undefined if the date passed is earlier than the epoch.

Examples:

```
> secs := os.datetosecs([2010, 1, 1, 12, 30, 1, 123]):
1262345401

> strings.strftime(secs, '%Y-%m-%dT%H:%M:%SZ'):
2010-01-01T12:30:01Z

> strings.strftime([2010, 1, 1, 12, 30, 1, 123], '%Y-%m-%dT%H:%M:%S.%QZ'):
2010-01-01T12:30:01.123Z
```

The function is not available in DOS and always issues an error on this platform.

See also: **os.date**, **strings.strptime**.

strings.stricmp (s1, s2)

Works like **strings.strcmp**, but compares case-insensitively.

See also: **strings.compare**, **strings.strcoll**, **strings.strncmp**, **strings.strverscmp**.

strings.strlen (s)

Returns the length of string `s`: the first return is the result of the call to the internal C function strlen(), and the second return is the internally stored length of `s`, returned by Agena's **size** operator.

The difference between **strings.strlen** and **size** is that C's strlen only counts the number of characters up to and excluding the first embedded zero (i.e. character '\0'), whereas **size** returns the real length including embedded zeros, but without the terminating zero.

Example:

```
> s := 'abc' & char(0) & 'defgh';
> # 3 chars up to the first embedded zero, 9 chars at all

> strings.strlen(s):
3       9
```

strings.strncmp (s1, s2, n)

Works like **strings.strcmp**, but compares only the first `n` characters, or less if at least one of the string is shorter. If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **strings.compare**, **strings.strcoll**, **strings.stricmp**, **strings.strverscmp**.

strings.strnstr (s1, s2 [, n])

The function is an interface to the C strnstr() function, searches the first `n` characters in string `s1` for the first `n` characters of substring `s2` and returns a substring starting from the first match to the end of `s1`. The second return is the position of the match, starting from 1. `n` defaults to the length of `s1`.

The function returns **null** and 0 if no match was found. If `s2` is an empty string, the function returns `s1`. If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **strings.strchr**, **strings.strcmp**, **strings.strstr**, **strings.strrchr**.

strings.strptime (s, format)

Takes a string `s`, usually a time stamp, and parses it using the format specified by string `format`, see the **strings.strftime** description above.

On success, the function returns a table with the year, month, and optionally hour, minute, second.

The timestamp may consist of date and time, date only or time only.

If there are surplus characters in `s`, the function also returns the portion of `s` starting from the first character not processed in the function call. If there are no surplus characters, the empty string will be returned.

If the function fails to match the format string, the function returns **null**. It also returns **null** with an unknown specifier.

The function provides an interface to the underlying C function strptime(), but ignores the day of month with the `%a` and `%A` specifiers. The function deliberately

does not know about the `%b` and `%B` specifiers for the name of the month and returns **null** in this case.

You may use `%a` or `%A` to skip all alphabetical letters and digits plus all characters with an ASCII code greater than 127 in s.

The non-standard `%Q` specifier denotes milliseconds. If given, they will be returned as the seventh entry in the resulting table.

Examples:

```
> strings.strptime('2025-08-29 12:40:44.123', '%Y-%m-%d %H:%M:%S'):
[2025, 8, 29, 12, 40, 44]       .123

> strings.strptime('12:40:44.123', '%H:%M:%S.%Q'):
[1900, 1, 0, 12, 40, 44, 123]


> strings.strptime('2025-08-29', '%Y-%m-%d'):
[2025, 8, 29, 0, 0, 0]
```

Ignore 'äöü123' (`%a`) and the subsequent white space (`%t`):

```
> strings.strptime('äöü123 12:40:44', '%a%t%H:%M:%S'):
[1900, 1, 0, 12, 40, 44]
```

See also: **strings.strftime**, **utils.timestamp**.

strings.strrchr (s, i)

strings.strrchr (s, c)

The function is an interface to the C strrchr() function, searches `s` backwards from the end for a single character represented by its ASCII code `i` or the character `c` (a string of size 0 or 1) and returns a substring starting from the first match to the end of `s`. The second return is the position of the match, starting from 1. It returns **null** and 0 if no match was found and issues an error if needle is non-positive. If `s1` is **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **abs**, **strings.strchr**, **strings.strstr**.

strings.strspn (s1, s2)

Returns the index of the first character in string `s1` that does not belong to the set of characters in `s2`. If the first character in `s1` is not in `s2`, the function will return 0. If `s1` or `s2` are empty strings, the function will return 0. The function is an interface to the C strspn() function. If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

Example:

```
> strings.strspn('012abcdef', '0123456789'):
3
```

See also: **member**, **strings.contains**, **strings.strcspn**.

strings.strstr (s1, s2)

The function is an interface to the C strstr() function, searches `s1` for a substring `s2` and returns a substring starting from the first match to the end of `s1`. The second return is the position of the match, starting from 1. The function returns **null** and 0 if no match was found. If `s2` is an empty string, the function returns `s1`. If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **strings.strchr**, **strings.strcmp**, **strings.strnstr**, **strings.strrchr**.

strings.strtoul (s, base [, true])

Tries to convert an integer of base `base` represented by string `s` to a (decimal) value and if successful returns it or pushes **undefined** otherwise. If the third argument is **true**, then the decimal value and the empty string, or zero and the rest of `s` where parsing failed, will be returned.

With the **true** option given, a value of 2^{64} -1 may indicate an overflow if the value in `s` is too big - without the option the return will automatically be set **undefined** in these cases.

The function provides an interface to the underlying strtoull() C function. Negative values are auto-corrected internally, as the C function may overflow. Bases greater than 36 are unsupported. If s is **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **math.convertbase**, **tonumber**.

strings.strverscmp (s1, s2)

The function compares two version strings. It is a direct interface to the GNU C strverscmp() function. The following is a summary of the GNU documentation:

"If you have files jan1, jan2, ..., jan9, jan10, ..., it feels wrong when an application orders them jan1, jan10, ..., jan2, ..., jan9, because the expected order is just: jan1, jan2, ..., jan9, jan10, ...

The function returns an integer less than, equal to, or greater than zero if `s1` is found, respectively, to be earlier than, equal to, or later than `s2`.

Both input strings should be in plain ASCII."

If `s1` or `s2` are **null**, the function returns **null** twice to mimic undefined behaviour in C.

See also: **rsorted**, **strings.compare**, **strings.strcmp**, **strings.strcoll**, **strings.stricmp**.

`strings.strxfrm (s)`

Transforms the multibyte string `s` using the collation transformation determined by the locale currently selected for collation, and returns the transformed string. The function is a wrapper to the C function strxfrm() but spares the effort to compute the exact size of the output.

See also: **os.setlocale**.

`strings.sub (s, i [, j])`

Returns the substring of `s` that starts at `i` and continues until `j`; `i` and `j` can be negative. If `j` is absent, then it is assumed to be equal to -1 (which is the same as the string length). In particular, the call **string.sub**(s,1,`j`) returns a prefix of `s` with length `j`, and **string.sub**(s, -`i`) (for a positive `i`) returns a suffix of `s` with length `i`.

If, after the translation of negative indices, `i` is less than 1, it is corrected to 1. If `j` is greater than the string length, it is corrected to that length. If, after these corrections, `i` is greater than `j`, the function returns the empty string.

Note that Agena's substring indexing auto-corrects the right border if it is out-of-range, but does not change the left border, so for example ('agena')[2 to 10] evaluates successfully but ('agena')[0 to 10] does not.

`strings.tobytes (s [, option [, bigendian [, container]]])`

By default converts a string `s` into a sequence of its numeric ASCII codes. If the string is empty, an empty sequence will be returned.

If `option` is **true** or the integer 4, the function returns word-aligned 4-byte unsigned integers instead of individual bytes. If `bigendian` is **true**, then with word-aligned 4-byte unsigned integers the result is in Big Endian notation, otherwise it is Little Endian.

If the fourth optional argument `container` is a sequence or table, then the individual bytes in the given string will be added to it, meaning that if the structure already includes elements, they will be preserved. This is a convenient and fast way of collecting string bytes in sequential order.

Note that numerical codes are not necessarily portable across platforms.

Example:

```
> s := strings.tobytes('agena', 4):  # convert to 4-byte integers
seq(1852139361, 97)

> str := '';

> for i in s do  # convert each 4-byte integer
>    t := bytes.tobytes(i, 4)   # to four single bytes and convert
>    str &:= strings.tochars(t) # back to string
> od

> str:
agena

> # or just simply:

> strings.tochars(s, 4):
```

See also: **split** operator, **strings.iterate**, **bytes.tobytes**, **strings.tochars**, **strings.tonumber**, **utils.hexlify**.

strings.tochars (···)

strings.tochars (s [, nbytes [, little]])

In the first form, receives zero or more integers in the range 0 .. 255 and returns a string with length equal to the number of arguments, in which each character has the internal numerical code equal to its corresponding argument.

In the second form, converts all the integers in sequence `s` to a string. By default, `s` is assumed to contain integers in the range 0 .. 255. If `nbytes` is 4, `s` should include unsigned 4-byte integers. If `little` is the Boolean **true** - the default - the integers are converted to Little Endian before assembling the string, otherwise pass **false**.

Note that numerical codes are not necessarily portable across platforms and that the resulting string may have embedded '\0' characters, for example at its end.

See also: **strings.tobytes**, **toreg**, **toseq**, **totable**.

strings.tolatin (s [, codepage])

Creates a copy of string `s`, changing the encoding from UTF-8 to ISO-8859-15. Unknown code points are returned unchanged. The return is a string. ISO-8859-15 is ISO-8859-1 plus the Euro symbol.

The function can convert from codepage 850 (Western Latin), 852 (Eastern Latin), 857 (Turkey), 860 (Portuguese) or 775 (Baltics) to Latin-1, Latin-2, Latin-4, Latin-1 or Latin-5, respectively, by passing the proper `codepage`, an integer, as a second argument. You may have change the codepage in the Windows or OS/2 shell before, too, see **strings.toutf8** for an example.

See also: **iconv** package, **strings.islatin**, **strings.itouni**, **strings.toutf8**.

strings.tonumber (s [, true])

By default takes a string `s` with the binary representation of a number and converts it into an Agena number, an 8-byte C double. The function also supports strings containing C long doubles, uint16_t's, int32_t's and floats; pass **true** as a second argument to indicate either a C 4-byte float instead of a signed 4-byte integer, or a signed C 8-byte integer instead of a C double.

See also: **bytes.tobytes**, **strings.tochars**, **strings.tobytes**.

strings.toutf8 (s [, codepage])

Creates a copy of string `s`, changing the encoding from ISO-8859-15 to UTF-8. The return is a string. ISO-8859-15 is ISO-8859-1 plus the Euro symbol.

The function supports conversion from codepage 850 (Western Latin), 852 (Eastern Latin), 857 (Turkey), 860 (Portuguese), 775 (Baltics) and Windows-1258 (Vietnamese) to UTF-8. Just pass either the integer 850, 852, 857, 860, 775 or 1258 as a second argument to the function. You might have to change the codepage in use by the Windows or OS/2 shell before calling **strings.toutf8** with the optional argument. Example:

```
> oldinputcp, oldoutputcp := os.codepage();

> os.codepage(input = 852, output = 852);

> strings.toutf8('Lódz', 852):
+ü+¦d+¦
```

Change back to default codepage:

```
> os.codepage(input = oldinputcp, output = oldinputcp);
```

See also: **iconv** package, **strings.isutf8**, **strings.tolatin**, **strings.itouni**, **strings.utf8size**.

strings.transform (f, s)

Applies a function `f` to the ASCII value of each character in string `s` and returns a new string. `f` must return an integer in the range [0, 255], otherwise an error will be issued.

Note that numerical ASCII codes are not necessarily portable across platforms.

strings.trim (s [, c])

The function returns a new string with all leading, trailing and excess embedded white spaces removed from `s`, by default. If character `c` is given, then this character instead of the white space will be removed.

See also: **strings.ltrim**, **strings.lrtrim**, **strings.remove**, **strings.rtrim**.

strings.uncapitalise (s [, sep])

Converts the first character in string `s` to lower case - if possible - and returns the uncapitalised string. If `s` is the empty string, it is simply returned. It also converts ligatures if the Western European character set is being used. If `sep`, a string, is given, then all the words in `s` - separated by `sep` - will be uncapitalised.

See also: **strings.lower**, **strings.capitalise**.

strings.unpack (fmt, s [, pos])

Returns the values packed in string `s` (see **strings.pack**) according to the format string `fmt`, see Chapter 9.1.4. An optional `pos` marks where to start reading in `s` (default is 1). After the read values, this function also returns the index of the first unread byte in `s`. If `s` - depending on the requested transformation - is too short, the function just returns **null** and zero.

See also: **strings.packsize**.

strings.unwrap (s [, delim])

Removes an enclosing character chr from string `s` and returns the modified string i.e. with chr deleted from both the start and end of `s`. One or more potential enclosing characters are given in string `delim`, which defaults to a single and a double quote (`` "' ``).

With the first enclosing character in `delim` found in `s`, the function returns the shortened string.

If `s` is not enclosed by any of the characters in `delim`, `s` will be returned unmodified.

See also: **strings.between**, **strings.iswrapped**, **strings.lrtrim**, **strings.wrap**.

strings.upper (s [, option])

The function receives string `s` and returns a copy of this string with all lowercase letters ('a' to 'z' plus the above mentioned diacritics) changed to uppercase ('A' to 'Z' and the diacritics listed at the end of Chapter 9.1). The function leaves all other characters unchanged. Example:

```
> strings.upper('Elektronika MK-61'):
ELEKTRONIKA MK-61
```

If any `option` is given, then only Latin letters a-z are put to upper case.

See also: **strings.lower**, **strings.capitalise**, **strings.isoupper**.

strings.utf8size (s)

Determines the size of the string `s` in UTF-8 encoding and returns a non-negative integer. The return is not the number of bytes used to represent a UTF-8 string, but the number of single- and multi-byte `UTF-8 characters`. Thus, for example, while `size strings.toutf8('à')` returns 2, `strings.utf8size(strings.toutf8('à'))` returns 1.

Please note that the function may not produce correct results with text input in a console. The function can only return correct results if the string to be checked has been read from a file.

See also: **size**, **strings.isutf8**.

strings.walker (s [, stream [, stats]])

For string `s`, returns the normalised specific Shannon entropy.

By default `s` is considered to be a bit stream, otherwise, if the second argument `stream` is set to **true**, to be a byte stream. If the third argument `stats` is **true**, then the function also returns the serial correlation coefficient, the Chi square distribution, the mean of the ASCII values in `s`, and the Monte Carlo value for Pi, in this order.

See also: **strings.shannon**.

strings.words (s [, delim [, true]])

Counts the number of words in a string `s`. A word is any sequence of characters surrounded by white spaces or its left and/or right borders. The user can define any other delimiter by passing an optional character `delim` (of type string) as a second argument. If the third argument is **true**, then succeeding delimiters are ignored. The return is a number.

See also: **strings.hits**.

strings.wrap (s, t [, true])

Wraps a string `s` with another strings `t`, returning the Agena equivalent of `t` & `s` & `t`.

If `s` is a number, **null** or a Boolean, it is converted to a string before the operation starts.

If the third argument is **true**, then wrapping is done only if `t` is missing at the start and the end of `s`; otherwise simply returns `s`.

See also: **strings.between**, **strings.iswrapped**, **strings.lrtrim**, **strings.unwrap**.

9.1.3 Patterns

Character Class:

A character class is used to represent a set of characters. The following combinations are allowed in describing a character class:

- **x:** (where x is not one of the magic characters `^$()%.[]*+-?`) represents the character x itself.
- **.:** (a dot) represents all characters.
- **%a:** represents all letters.
- **%c:** represents all control characters.
- **%d:** represents all digits.
- **%l:** represents all lowercase letters.
- **%k**: represents all upper and lower-case consonants, y and Y are not considered consonants.
- **%p:** represents all punctuation characters.
- **%s:** represents all space characters, e.g. white spaces, newlines, tabulators, and carriage returns.
- **%u:** represents all uppercase letters.
- **%v**: represents all upper and lower-case vowels including the letters y and Y.
- **%w:** represents all alphanumeric characters.
- **%x:** represents all hexadecimal digits.
- **%z:** represents the character with representation 0.
- **%<y>:** (where <y> is any non-alphanumeric character) represents the character y. This is the standard way to escape the magic characters. Any punctuation character (even the non magic) can be preceded by a '%' when used to represent itself in a pattern.
- **[*set*]:** represents the class which is the union of all characters in *set*. A range of characters may be specified by separating the end characters of the range with a '-'. All classes %y described above may also be used as components in set. All other characters in set represent themselves. For example, [%w_] (or [_%w]) represents all alphanumeric characters plus the underscore, [0-7] represents the octal digits, and [0-7%l%-] represents the octal digits plus the lowercase letters plus the '-' character.
- The interaction between ranges and classes is not defined. Therefore, patterns like [%a-z] or [a-%%] have no meaning.
- **[^*set*]:** represents the complement of *set*, where set is interpreted as above.

For all classes represented by single letters (%a, %c, %v etc.), the corresponding uppercase letter represents the complement of the class. For instance, %S represents all non-space characters.

The definitions of letter, space, and other character groups depend on the current locale. In particular, the class [a-z] may not be equivalent to %l.

Pattern Item:

A *pattern item* may be

- a single character class, which matches any single character in the class;
- a single character class followed by '*', which matches 0 or more repetitions of characters in the class. These repetition items will always match the longest possible sequence;
- a single character class followed by '+', which matches 1 or more repetitions of characters in the class. These repetition items will always match the longest possible sequence;
- a single character class followed by '–', which also matches 0 or more repetitions of characters in the class. Unlike '*', these repetition items will always match the *shortest possible sequence;*
- a single character class followed by '?', which matches 0 or 1 occurrence of a character in the class;
- %n, for n between 1 and 9; such item matches a substring equal to the n-th captured string (see below);
- %b*xy*, where x and y are two distinct characters; such item matches strings that start with x, end in y, and where the x and y are balanced. This means that, if one reads the string from left to right, counting +1 for an x and -1 for a y, the ending y is the first y where the count reaches 0. For instance, the item %b() matches expressions with balanced parentheses;
- %f[set], a frontier pattern; such item matches an empty string at any position such that the next character belongs to set and the previous character does not belong to set. The set `set` is interpreted as previously described. The beginning and the end of the subject are handled as if they were the character '\0'.

Pattern:

A *pattern* is a sequence of pattern items. *A* '^' at the beginning of a pattern anchors the match at the beginning of the subject string. A '$' at the end of a pattern anchors the match at the end of the subject string. At other positions, '^' and '$' have no special meaning and represent themselves.

Captures:

A pattern may contain sub-patterns enclosed in parentheses; they describe captures. When a match succeeds, the substrings of the subject string that match captures are stored (captured) for future use. Captures are numbered according to their left parentheses. For instance, in the pattern '(a*(.)%w(%s*))', the part of the string matching 'a*(.)%w(%s*)' is stored as the first capture (and therefore has number 1); the character matching '.' is captured with number 2, and the part matching '%s*' has number 3.

As a special case, the empty capture () captures the current string position (a number). For instance, if we apply the pattern '()aa()' on the string 'flaaap', there will be two captures: 3 and 5.

A pattern cannot contain embedded zeros. Use `%z` instead.

9.1.4 Format Strings for Pack and Unpack

The first argument to **strings.pack**, **strings.packsize**, and **strings.unpack** is a format string, which describes the layout of the structure being created or read.

A format string is a sequence of conversion options. The conversion options are as follows:

- **`<`**: sets Little Endian
- **`>`**: sets Big Endian
- **`=`**: sets native Endian
- **`![n]`**: sets maximum alignment to n (default is native alignment)
- **`b`**: a signed byte (char)
- **`B`**: an unsigned byte (char)
- **`h`**: a signed short (native size)
- **`H`**: an unsigned short (native size)
- **`l`**: a signed long (native size)
- **`L`**: an unsigned long (native size)
- **`j`**: a lua_Integer
- **`J`**: a lua_Unsigned
- **`T`**: a size_t (native size)
- **`i[n]`**: a signed int with n bytes (default is native size)
- **`I[n]`**: an unsigned int with n bytes (default is native size)
- **`f`**: a float (native size)
- **`d`**: a double (native size)
- **`n`**: an Agena number of C type double = lua_Number
- **`cn`**: a fixed-sized string with n bytes
- **`z`**: a zero-terminated string
- **`s[n]`**: a string preceded by its length coded as an unsigned integer with n bytes (default is a size_t)
- **`x`**: one byte of padding
- **`Xop`**: an empty item that aligns according to option op (which is otherwise ignored)
- **`' '`**: (a white space) ignored

(A "[n]" means an optional integral numeral.) Except for padding, spaces, and configurations (options "xX <=>!"), each option corresponds to an argument in **strings.pack** or a result in **strings.unpack**.

For options '!n', 'sn', 'in', and 'In', n can be any integer between 1 and 16. All integral options check overflows; **strings.pack** checks whether the given value fits in the

given size; **strings.unpack** checks whether the read value fits in an Agena integer. For the unsigned options, Lua integers are treated as unsigned values too.

Any format string starts as if prefixed by "!1=", that is, with maximum alignment of 1 (no alignment) and native endianness.

Native endianness assumes that the whole system is either Big or Little Endian. The packing functions will not emulate correctly the behaviour of mixed-Endian formats.

Alignment works as follows: For each option, the format gets extra padding until the data starts at an offset that is a multiple of the minimum between the option size and the maximum alignment; this minimum must be a power of 2. Options "c" and "z" are not aligned; option "s" follows the alignment of its starting integer.

All padding is filled with zeros by **strings.pack** and ignored by **strings.unpack**.

Examples:

Print the size of a machine's native int, we get four bytes:

```
> struct.packsize('i'):
4
```

To pack and unpack the C structure

struct Str { char b; int i[4]; };

you can use the string `'<!4biiii'`.

```
> x := strings.pack('<!4biiii', abs('A'), 10, 20, 30, 40)

> strings.unpack('<!4biiii', x):
65      10      20      30      40      21
```

If you need to code a structure with a large array, you may use **strings.repeat** to automatically generate part of the string format. For instance, for the structure

struct Str { double x; int i[400]; };

you can build the format string with the code `'d' & strings.repeat('i', 400)`.

Pack **Pi** as an 8-byte C double:

```
> x := strings.pack('d', Pi)

> strings.unpack('d', x):
3.1415926535898 9
```

Code and decode a string:

```
> s := 'agena programming language'

> x := strings.pack('s', s):
?agena programming language

> strings.unpack('s', x):
agena programming language      31
```

9.2 memfile - Memory File for Strings & Bytes in General

The **memfile** library implements character and byte buffers - that is `memory file` userdata structures that either store a string of almost unlimited length or a sequence of any number of bytes - along with functions to administer it.

Let us start with the typical usage of a character buffer: it is useful if you have to iteratively concatenate a lot of strings, being 20 times faster than the **&** operator.

```
> m := memfile.charbuf()           # create a memory file

> memfile.append(m, 'nasa', 'jpl') # put two strings into it

> f := memfile.iterate(m, 1, 2) # from position 1, return 2 chars per call

> f():
na

> f():
sa

> f():
jp

> f():
l

> f():
null
```

Dump the contents of the buffer, making room for new content:

```
> memfile.dump(m):
nasajpl
```

Let us declare a bit field of two bytes:

```
> b := memfile.bitfield(16)
```

and in these two bytes, set the even bits to 1, i.e. twice to 0b10101010 = 170:

```
> for i to 16 do if even(i) then b[i] := 1 fi od
```

The contents of the field is:

```
> b:
bitfield(0b10101010, 0b10101010)
```

Get some bits, the first and the tenth:

```
> b[1]:
0
```

```
> b[10]:
1
```

Clear the bits in the first byte:

```
> for i to 8 do if even(i) then b[i] := 0 fi od

> b:
bitfield(0b00000000, 0b10101010)
```

Alternatively, you can store any number of *bytes* in a buffer, making it more versatile than just storing a string, see **memfile.bytebuf** for further information. Generally the two types are interchangeable, with some very few differences, see especially the `'__index'` metamethod to read data from a memfile and the iterators produced by **memfile.iterate**.

You can use all functions in this package OOP-style, for example `memfile.dump(n)` and `n@@dump()` are equivalent.

Note that you can write-protect memfiles with **freeze** and remove the protection with **unfreeze**.

The package provides the following metamethods:

Metamethod	Functionality
`'__index'`	read operation, e.g. n[p] or n[p to q], with p, q indices, both counting from 1; with bit fields, reads a bit; with a byte buffer returns exactly one byte (`to q` is not supported here); with a character buffer returns a string of one or more characters
`'__writeindex'`	write operation, e.g. n[p] := value, with p the index, counting from 1; with bit fields, sets a bit, not a byte
`'__size'`	**size** operator, number of characters currently stored; with bit fields returns the number of bits in the field, not bytes
`'__in'`	**in** operator
`'__notin'`	**notin** operator
`'__eq'`	= equality operator
`'__empty'`	**empty** operator
`'__filled'`	**filled** operator
`'__tostring'`	formatting for output at the console; with bit fields, returns binary representations
`'__gc'`	garbage collection

The package functions are:

memfile.append (memfile, v [,···] [, delim=str])

memfile.append (memfile, v [,···] [, false])

In the first form, appends one or more (complex) numbers, strings, Booleans or **null**'s `v`, etc. to the end of `memfile`. Anything not a string is converted to one before insertion. The function returns nothing.

In the second form, appends one or more bytes `v`, ..., all unsigned integers in the range 0 .. 255, into *byte* buffer `memfile`. `v` may also be one or more tables, sequences or registers of bytes. Pass **false** as the very last argument if you want `v`, ... to be converted to strings before insertion.

You can specify the optional character delimiter option `` `delim = str' `` that separates each value to be added to the memory file, e.g. `memfile.append(m, 'a', 'b', 'c', delim = ';')` actually adds the string `'a;b;c;'`. You may later on drop the final delimiter by calling **memfile.dump** with the size of the delimiter.

See also: **memfile.move**, **memfile.prepend**, **memfile.purge**, **memfile.put**, **memfile.rewind**, **memfile.setbyte**, **memfile.setbytes**, **memfile.setitem**.

memfile.attrib (memfile)

Returns the total capacity of a `memfile` and the current number of allocated bytes, in this order.

See also: **memfile.getsize**.

memfile.bitfield (n [,···])

Creates a bit field of at least `n` bits and optionally sets zeros or ones into this field. The return is a byte buffer with initially all positions set to zero.

If you pass optional ones or zeros, or the Booleans **true** or **false**, they are set from the right end to the left end of the new bit field, e.g. if we have

```
> b := memfile.bitfield(4, 1, 1, 1, 0)
```

we will store 0b0111 = 7 decimal into the field. If you need it the other way around, execute something like

```
> b := memfile.bitfield(4, unpack(reverse(reg(1, 1, 1, 0))))
```

The number of bits actually allocated is always a multiple of 8, i.e. the field is filled up to whole bytes. In the example above, instead of four bits we created a bit field of eight bits.

Since the memory file created is no different from the one created by **memfile.charbuf**, with the exception of the metatable, you can apply all the other **memfile** functions on it. The bit field metatable `bitfield` contains specialised functions to get, set, determine the size and print bit fields, which work on the bit and not byte level.

See also: **memfile.attrib**, **memfile.bytebuf**, **memfile.charbuf**, **memfile.getbit**, **memfile.getbyte**, **memfile.getfield**, **memfile.resize**. **memfile.setbit**, **memfile.setbyte**, **memfile.setfield**.

memfile.bytebuf ([n [,···]])

Creates a memory file of fixed size `n` bytes and fills it with zeros if `n` > 0. It can also initialise the buffer with bytes (second to last argument). If no argument is given, `n` is set to zero.

Since the memory file created is no different from the one created by **memfile.charbuf**, you can apply all the other **memfile** functions on it.

If `n` is zero, a byte buffer of size zero with no pre-filled slots will be created. This allows to easily insert bytes later with calls to **memfile.append** without having to run **memfile.resize** before.

See also: **memfile.attrib**, **memfile.bitfield**, **memfile.charbuf**, **memfile.getbyte**, **memfile.getbytes**, **memfile.setbyte**, **memfile.setbytes**.

memfile.charbuf ([v [,···] [, delim=str]])

Creates a memory file and optionally puts one or more (complex) numbers, strings, Booleans or **null**'s `v` into it. The function returns the memory file created.

You can specify the optional character delimiter option `delim = str` that separates each value to be added to the memory file, e.g. `memfile.charbuf('a', 'b', 'c', delim = ';')` actually adds the string `'a;b;c;'`. You may drop the final delimiter later on by calling **memfile.dump** with the size of the delimiter.

See also: **memfile.attrib**, **memfile.bytebuf**, **memfile.resize**.

memfile.clearbit (memfile, n)

memfile.clearbit (memfile, pos, n)

In the first form, unsets bit position `n`, from the right end of the field, in `memfile`, i.e. sets it to 0. `n` counts from 1. To set a bit, use **memfile.setbit**.

In the second form, in byte no. `pos` of the `memfile`, unsets the `n`-th bit, i.e. sets it to 0, where `n` > 0.

The return is the modified byte at byte position `pos`.

See also: **memfile.getbit**, **memfile.setbit**, **memfile.setbyte**, **memfile.setchar**, **memfile.setfield**, **memfile.setitem**.

memfile.dump (memfile [, n])

Without a second argument, returns the whole string stored in `memfile` and resets `memfile` completely to its original state, so that it can store a new string. With a byte buffer, the function refills it with zeros again after dumping the contents.

If a positive integer `n` is passed as an optional argument, then the function just dumps `n` bytes from the end (tail) of the memory file and returns them as a string, leaving the rest of the `memfile` untouched. If the `memfile` should be empty after this operation, it is reset, which is equal to calling the function without an optional argument.

The function returns **null**, if `memfile` is empty.

Examples:

```
> m := memfile.bytebuf();
```

```
> memfile.append(m, 'abcd');
```

Cut the buffer to size 2:

```
> memfile.move(m, 2);
```

Append 'xyz' to remaining 'ab':

```
> memfile.append(m, 'xyz'):
```

```
> memfile.substring(m):
abxyz
```

Cut the buffer to zero size:

```
> memfile.move(m, 0):
```

```
> memfile.dump(m):
null
```

See also: **memfile.get**, **memfile.getbytes**, **memfile.rewind**, **memfile.substring**, **memfile.tostring**.

memfile.find (memfile, str [, pos])

memfile.find (memfile, byte [, pos])

memfile.find (memfile, s [, pos])

With a character or byte buffer, searches `memfile` for a substring `str` and returns its position, an integer, or **null** if the string has not been found. The optional argument `pos` indicates the position where to start the search, and is 1 by default.

With a byte buffer, searches for the given `byte` or sequence `s` of bytes, non-negative integers in the range 0 to 255.

The function does not support pattern matching, use **memfile.match** instead.

See also: **memfile.mfind**, **memfile.substring**, **in** metamethod.

memfile.get (memfile [, n])

Without a second argument, returns the whole string stored in `memfile`.

If a positive integer `n` is passed as an optional argument, then the function just returns `n` characters from the end (tail) of the memory file.

The function contrary to **memfile.dump** does not remove any contents and also does not reset or re-size the memory file.

See also: **memfile.dump**, **memfile.getbytes**, **memfile.substring**.

memfile.getbit (memfile, n)

memfile.getbit (memfile, p [, n])

In the first form, returns the bit stored at bit position `n`, from the right end of the field, in `memfile`. `n` counts from 1.

In the second form, from byte no. `p` in the `memfile`, returns the `n`-th bit, where `p` and `n` > 0.

The return is either 1 or 0.

See also: **memfile.clearbit**, **memfile.getfield**, **memfile.setbit**.

memfile.getbyte (memfile, pos [, option])

From `memfile`, returns the byte at position `pos`, with `pos` <> 0. If `pos` is negative, the position is relative to the end of the string or bitfield. The return is an integer in the range [0, 255].

If any `option` is given, then the function returns a string with the binary representation of the byte at `pos`, e.g. `'0b10000000'`.

See also: **memfile.getbit**, **memfile.getbytes**, **memfile.getchar**, **memfile.getitem**.

`memfile.getbytes (memfile [, pos [, n]])`

From `memfile`, starting from byte position `pos` in `memfile`, returns a sequence with all the bytes (integers in the range 0 .. 255) stored in the memory file. `pos` is 1 by default, i.e. all bytes will be returned. If `pos` is negative, the position is relative to the end of the string.

If `n` is given, then only `n` bytes are read, starting from position `pos`, and put into the sequence. `n` may be internally capped to prevent the function from reading beyond the end of the buffer.

See also: **memfile.dump**, **memfile.getbit**, **memfile.getbyte**, **memfile.getchar**, **memfile.getitem**, **memfile.getsize**, **memfile.setbyte**, **size** metamethod.

`memfile.getchar (memfile, pos)`

From `memfile`, returns the character at position `pos`, with `p<> 0`. If `pos` is negative, the position is relative to the end of the string. The return is a string of size 1: the character.

See also: **memfile.getbyte**, **memfile.get**, **memfile.getitem**, **memfile.substring**.

`memfile.getfield (memfile, n)`

Returns the bit stored at absolute bit position `n` in `memfile`. `n` counts from 1.

The return is either 1 or 0.

See also: **memfile.setfield**, **memfile.clearbit**, **memfile.getbit**, **memfile.setbit**.

`memfile.getitem (memfile, p [, n])`

From `memfile`, returns the substring starting at position `p` and of length `n`, with non-zero `p`. If `p` is negative, the position is relative to the end of the string. By default, `n` is 1.

See also: **memfile.getbyte**, **memfile.getchar**, **memfile.getsize**, **memfile.iterate**, **memfile.setitem**, **memfile.substring**.

memfile.getsize (memfile)

From `memfile`, returns the number of characters (bytes) stored in it.

See also: **size** metamethod, **memfile.attrib**.

memfile.iterate (memfile [, pos [, n [, true]]])

Returns an iterator function that when called, returns the next `n` characters or bytes stored in `memfile`, starting at position `pos`. If there are no more characters or bytes, the iterator returns **null**. The type of return of the iterator will depend of the type of memfile:

- with a character buffer, the return will be a string,
- with a byte buffer, the return will be an integer.

By default `pos` and `n` are 1.

If the fourth optional argument **true** is given, then the iterator will return both a running index and the associated value(s). When the end of the memfile has been reached, it returns **null** once and then restarts iteration from the beginning (or the index given when calling the factory).

See also: **memfile.getitem**, **memfile.substring**.

memfile.map (f, memfile [,···])

Maps a function `f` on each character in `memfile`, in-place. `f` must always return a number or a string.

memfile.match (memfile, str [, pos])

With a byte or character buffer, searches `memfile` for a substring `str` and returns its start and end position, both positive integer plus the pattern found, or **null** if the string has not been found. The optional argument `pos` indicates the position where to start the search, and is 1 by default.

The function supports pattern matching, see Chapter 9.1.3.

See also: **memfile.find**, **memfile.mfind**, **memfile.substring**, **in** metamethod.

memfile.mfind (memfile, str [, pos])

memfile.mfind (memfile, byte [, pos])

memfile.mfind (memfile, s [, pos])

With a character or byte buffer, searches `memfile` for all occurrences of substring `str` and returns pairs representing the respective start and end positions, or **null** if the string has not been found at all. The optional argument `pos` indicates the

position where to start the search and is 1 by default. The function supports pattern matching, see Chapter 9.1.3.

```
> m := memfile.charbuf();

> memfile.append(m, 'NASA/JPL');

> memfile.mfind(m, 'A'):  # find letter A
seq(2:2, 4:4)

> memfile.mfind(m, '(%w)'):  # find all characters
seq(1:1, 2:2, 3:3, 4:4, 6:6, 6:6, 7:7, 8:8)
```

With a byte buffer, searches for the given `byte` or sequence `s` of bytes, non-negative integers in the range 0 to 255.

See also: **memfile.match**, **memfile.mfind**, **memfile.substring**, **in** metamethod.

memfile.move (memfile, pos)

Sets the end of the current `memfile` to the given position `pos`, inclusive, with `pos` a non-negative integer, or in other words: changes the size of the `memfile` without reallocating memory. If `pos` is 0, the `memfile` is cleared. If `pos` is, for example 2, then if you call **memfile.append** thereafter with a substring, it will be added starting at position 3, preserving the values at position 1 and 2. The function returns nothing.

See also: **memfile.append**, **memfile.resize**, **memfile.rewind**.

memfile.prepend (memfile, v [,···] [, delim=str])

memfile.prepend (memfile, v [,···], true)

In the first form, inserts one or more (complex) numbers, strings, Booleans or **null**'s `v`, etc. to the start of `memfile`. Anything not a string is converted to one before the insertion. The function returns nothing.

In the second form, by passing **true** as the very last argument, the function prepends one or more bytes `v`, ..., all unsigned integers in the range 0 .. 255, in byte buffer `memfile`. `v` may also be one or more sequences, registers ir tables of bytes.

You can specify the optional character delimiter option `` `delim = <str>' `` that separates each value to be added to the memory file, e.g. `memfile.prepend(m, 'a', 'b', 'c', delim = ';')` actually adds the string `'a;b;c;'`. You may later on drop the final delimiter by calling **memfile.dump** with the size of the delimiter.

See also: **memfile.move**, **memfile.append**, **memfile.rewind**, **memfile.purge**, **memfile.put**, **memfile.setbyte**, **memfile.setitem**.

memfile.purge (memfile, pos [, n])

Starting from position `pos`, removes `n` bytes or characters from memory file `memfile`, shifting down other elements to close the space. `n` is 1 by default. If `pos` is negative then the deletion will start at the |`pos`|-th position from the right. The function returns **true** on success and **false** otherwise.

See also: **memfile.append**, **memfile.prepend**, **memfile.put**, **memfile.setbyte**, **memfile.setitem**.

memfile.put (memfile, pos, str)
memfile.put (memfile, pos, b [,···])

Puts string `str` or sequence of bytes `b`, ... into memory file `memfile` starting from position `pos`, shifting the other values into open space. If `pos` is negative then the deletion will start at the |`pos`|-th position from the right.

The function automatically grows the memory file if needed. It returns nothing.

See also: **memfile.append**, **memfile.prepend**, **memfile.purge**, **memfile.setbyte**, **memfile.setitem**.

memfile.read (fh, memfile [, bufsize])

Reads data from the file denoted by its file handle `fh` into the given `memfile` userdata.

The file should have previously been opened with **binio.open** and should finally be closed with **binio.close**.

By default, the function reads in the entire file if `bufsize` is not given.

If a positive integer has been passed with `bufsize`, the function only reads in the given number of bytes per each call, returns the number of bytes actually read and increments the file position thereafter so that the next bytes in the file can be read with a new call to **memfile.read**.

(Passing the `bufsize` argument may also be necessary if your platform requires that an internal input buffer is aligned to a certain block size.)

If the end of the file has been reached, or there is nothing to read at all, **null** will be returned. In case of an error, it quits with the respective error.

If you want to read in the file again in the current session, call **binio.rewind** with the file handle before.

Example:

```
> m := memfile.charbuf();
```

```
> print(size m);  # should be zero

> fd := binio.open('memfile.bin');

> pos := 1;

> # the following loop is equivalent to the simple call
> # `memfile.read(fd, m)`:

> do
>    pos := memfile.read(fd, m, 512)  # read 512 bytes per each call
> until pos = null;

> binio.close(fd);  # should now be non-zero

> print(size m);
```

See also: **memfile.write**.

memfile.replace (memfile, pattern, repl [, init [, n]])

Replaces `pattern` with `repl` in memory file `memfile`. The return is the number of substitutions made. `pattern` and `repl` may either be strings or byte sequences.

The replacements start at position `init`, which be default is 1, i.e. the start of the buffer. If `n` is given, then the number of substitutions is limited by that number. By default, all occurrences are replaced.

The function supports pattern matching, see Chapter 9.1.3 for more information on patterns.

memfile.resize (memfile, n [, flag])

Resizes the `memfile` to exactly `n` places (bytes), with `n` > 0. It can grow or shrink a memory file and in the latter case preserves the remaining content. If the memory file is to be enlarged, the function optionally fills the new space with zeros if the third argument `flag` **true** is given, otherwise you may just pass **false** which is the default.

You may call **bytes.optsize** before to determine the new optimal number of word-aligned places (bytes) in the memory file.

See also: **memfile.attrib**, **memfile.bytebuf**, **memfile.charbuf**, **memfile.getitem**, **memfile.move**, **memfile.rewind**.

memfile.reverse (memfile)

Reverses the data, i.e. bytes, in a memory file, in-place. The function returns nothing.

memfile.rewind (memfile)

Sets the current size of a `memfile` to zero, effectively clearing the buffer without re-allocating memory. The function returns nothing.

See also: **memfile.append**, **memfile.dump**, **memfile.move**.

memfile.setbit (memfile, n)

memfile.setbit (memfile, pos, n [, val])

In the first form, sets bit position `n`, from the right end of the field, in `memfile` to 1. `n` counts from 1. To clear a bit, use **memfile.clearbit**.

In the second form, in byte no. `pos` of the `memfile`, sets the `n`-th bit to `val`, where `val` is either a Boolean or 0 or 1 and `n` > 0. If `val` is omitted, sets the bit to 1.

The return is the modified byte at byte position `pos`.

See also: **memfile.clearbit**, **memfile.getbit**, **memfile.setbyte**, **memfile.setchar**, **memfile.setfield**, **memfile.setitem**.

memfile.setbyte (memfile, pos, i [, count])

Sets byte `i` of type (non-negative) integer into `memfile` at the *existing* position pos, with `pos` <> 0.

If `pos` is negative, the position is relative to the end of the string or bitfield, otherwise it is relative to the beginning of the string or bitfield. `i` should be an integer in the range [0, 255]. The function returns nothing.

If a fourth argument `count` is given, then the function sets `count` bytes - starting from position `pos` - to the given byte. By default, `count` is 1. If `pos` has not been set before, the function fills all preceding positions with zeros, if they have not yet been already set to any byte.

See also: **memfile.append**, **memfile.getbyte**, **memfile.getchar**, **memfile.setbit**, **memfile.setbytes**, **memfile.setchar**, **memfile.setitem**.

memfile.setbytes (memfile, pos, obj)

Similar to **memfile.setbyte**, but takes a table, sequence or register of one or more bytes (integers) `obj` and sets them into the buffer starting from position `pos`. The function returns nothing.

You must make sure that the buffer `memfile` is properly sized, see **memfile.resize**, otherwise the function will issue an error.

The function returns nothing.

See also: **memfile.append**, **memfile.setbyte**.

memfile.setchar (memfile, pos, c)

Sets character `c` of type string into `memfile` at the *existing* position `pos`, with `pos` <> 0. If `c` consists of multiple characters, only the first character will be written to the memory file.

If `pos` is negative, the position is relative to the end of the string. `i` should be an integer in the range [0, 255]. The function returns nothing.

If a fourth argument `count` is given, then the function will set `count` places - starting from position `pos` - to the given character. By default, `count` is 1. If `pos` has not been set before, the function will fill all preceding positions with white spaces, if they have not yet been already set.

See also: **memfile.getbyte**, **memfile.getchar**, **memfile.setbit**, **memfile.setchar**, **memfile.setitem**.

memfile.setfield (memfile, n, val)

Sets the `n`-th bit in a memory file to `val`, where `val` is either the Boolean **true** or **false**, or 0 or 1. In `n` is negative, then the |`n`|-th bit from the left side of the buffer is set or unset.

The return is the modified byte in which the bit resides.

See also: **memfile.clearbit**, **memfile.getfield**, **memfile.setbit**.

memfile.setitem (memfile, str, pos)

memfile.setitem (memfile, byte, pos)

In the first form, sets a substring `str` into `memfile` at position `pos`, with non-zero `pos`. If the substring is too long, the function issues an error. If `pos` is negative, the position is relative to the end of the string.

In the second form, does the same with the non-negative integer `byte`.

The function returns nothing.

See also: **memfile.append**, **memfile.prepend**, **memfile.purge**, **memfile.getitem**, **memfile.setbyte**, **memfile.setchar**.

memfile.shift (memfile, n)

Rotates the contents of the buffer `memfile` `n` bytes to the right if `n` is positive, and `n` bytes to the left if `n` is negative. The function returns nothing.

See also: **memfile.move**, **memfile.substring**.

memfile.substring (memfile [, p [, q]])

From `memfile`, returns the substring from position `p` to position `q`, with non-zero `p`, `q`. If `p` or `q` are negative, the respective positions are relative to the end of the string. `q` is `p` by default. If `p` and `q` are missing, the whole contents is returned without dumping it. The function returns **null** if the buffer is empty.

See also: **memfile.dump**, **memfile.get**, **memfile.getitem**, **memfile.tostring**.

memfile.tostring (b [,···])

memfile.tostring (memfile)

In the first form, converts one or more bytes `b`, ... into a string, to be used for example as search criteria in **memfile.find**. Each byte should be an unsigned integer in the range 0 .. 255. `b` may also be a sequence of one or more bytes.

In then second form, returns the whole contents of `memfile` as a string without dumping the contents.

See also: **memfile.substring**.

memfile.write (fh, memfile [, pos [, nchars]])

Writes the string in a `memfile` userdata to the file denoted by its numeric file handle `fh`.

The file should be opened with **binio.open** and closed with **binio.close** after completion.

The start position `pos` is 1 by default but can be changed to any other valid position in the `memfile`.

The number of characters (not necessarily bytes) to be written can be changed by passing an optional fourth argument `nchars`, a positive number, and by default equals the total number of characters in `memfile`. (Passing the `nchars` argument may also be necessary if your platform requires that buffers must be aligned to a particular block size.)

The function returns the index of the next start position (an integer) for a further call to **memfile.write** to write further characters, where the return should be passed to the third `pos` argument.

If the end of the string in `memfile` has been reached, the function returns **null** and flushes all unwritten content to the file so that you do not have to call **binio.sync** manually.

No further information is stored to the file created.

Example on how to write a string of 8,000 characters piece-by-piece:

```
> m := memfile.charbuf();

> to 1000 do
>    memfile.append(m, 'nasa/jpl')
> od;

> fd := binio.open('memfile.bin');

> pos := 1;

> # The following is equivalent to "memfile.write(fd, m)":

> do  # write 1024 values per each call
>    pos := memfile.write(fd, m, pos, 1024)
> until pos = null;

> binio.close(fd);
```

Use **binio.sync** if you want to make sure that any unwritten content is written to the file when calling **memfile.write** multiple times.

See also: **memfile.read**.

9.3 utf8 - UTF-8 Helpers

This library provides basic support for UTF-8 encoding. It provides all its functions inside the table utf8. This library does not provide any support for Unicode other than the handling of the encoding. Any operation that needs the meaning of a character, such as character classification, is outside its scope.

Unless stated otherwise, all functions that expect a byte position as a parameter assume that the given position is either the start of a byte sequence or one plus the length of the subject string. As with many string functions, negative indices count from the end of the string.

Functions that create byte sequences accept all values up to 0x7FFFFFFF, as defined in the original UTF-8 specification; that implies byte sequences of up to six bytes.

Functions that interpret byte sequences only accept valid sequences (well formed and not overlong). By default, they only accept byte sequences that result in valid Unicode code points, rejecting values greater than 10FFFF and surrogates. A boolean argument **lax**, when available, lifts these checks, so that all values up to 0x7FFFFFFF are accepted. (Not well formed and overlong sequences are still rejected.)

The UTF-8 encoding of a Unicode character can be inserted in a literal string with the escape sequence \u{XXX} (with mandatory enclosing braces), where XXX is a sequence of one or more hexadecimal digits representing the character code point. This code point can be any value less than 2^{31}. (Agena uses the original UTF-8 specification here, which is not restricted to valid Unicode code points.) Example:

```
> "\u{10FFFF}":
¶Å++
```

utf8.chars (···)

Receives zero or more integers, converts each one to its corresponding UTF-8 byte sequence and returns a string with the concatenation of all these sequences.

utf8.charpattern

The pattern (a string, not a function) "[\0-\x7F\xC2-\xF4][\x80-\xBF]*", which matches exactly one UTF-8 byte sequence, assuming that the subject is a valid UTF-8 string.

utf8.codes (s [, lax])

Returns values so that the construction

```
for p, c in utf8.codes(s) do body od
```

will iterate over all characters in string `s`, with `p` being the position (in bytes) and `c` the code point of each character. It raises an error if it meets any invalid byte sequence.

`utf8.codepoint (s, [, i [, j [, lax]]])`

Returns the codepoints (as integers) from all characters in `s` that start between byte position `i` and `j` (both included). The default for `i` is 1 and for `j` is i. It raises an error if it meets any invalid byte sequence.

`utf8.len (s, [, i [, j [, lax]]])`

Returns the number of UTF-8 characters in string `s` that start between positions i and j (both inclusive). The default for `i` is 1 and for `j` is -1. If it finds any invalid byte sequence, returns a false value plus the position of the first invalid byte.

`utf8.offset (s, n [, i])`

Returns the position (in bytes) where the encoding of the `n`-th character of `s` (counting from position `i`) starts. A negative `n` gets characters before position `i`. The default for `i` is 1 when `n` is non-negative and size `s` + 1 otherwise, so that utf8.offset(`s`, -`n`) gets the offset of the `n`-th character from the end of the string. If the specified character is neither in the subject nor right after its end, the function returns **null**.

As a special case, when `n` is 0 the function returns the start of the encoding of the character that contains the `i`-th byte of `s`.

This function assumes that s is a valid UTF-8 string.

9.4 iconv - Internationalization

As a *plus* package, the **iconv** package is not part of the standard distribution and must be activated with the **import** statement, i.e. `import iconv`.

The **iconv** library allows to convert strings from one code page (character set) to another. For a list of available code pages, see **iconv.list**. It is a port to the GNU iconv package, where iconv stands for `internationalization conversion`.

Typical usage: First open a handle by passing the from code page and the to code page, in this example, we convert a text from Latin-1 to UTF-8:

```
> import iconv

> cd := iconv.open('latin1', 'utf-8');

> iconv.convert(cd, 'äöüß'):
-ä-ö-ü+í
```

After all strings have been converted, the handle must be closed.

```
> iconv.close(cd);
```

Hint for UNIX & MacOS X users: You must have the iconv package installed on your system in order to use this package.

The available functions are:

iconv.open (from, to)

Opens a new conversion descriptor, from the `from` character set (a string) to the `to` character set (also a string). Concatenating "//TRANSLIT" to the first argument will enable character transliteration and concatenating "//IGNORE" to the first argument will cause iconv to ignore any invalid characters found in the input string.

This function returns a new converter or issues an error. For a list of available character sets, see **iconv.list**. `from` and `to` may be given in upper and lower case.

iconv.convert (cd, str)

Converts string `str` to the desired character set. `cd` depicts the converter descriptor. This method always returns the converted string on success, and **null** and an error code otherwise:

- iconv.ERROR_NO_MEMORY
  Failed to allocate enough memory in the conversion process.
- iconv.ERROR_INVALID
  An invalid character was found in the input sequence.

- iconv.ERROR_INCOMPLETE
  An incomplete character was found in the input sequence.
- iconv.ERROR_FINALIZED
  Trying to use an already-finalized converter. This usually means that the user was tweaking the garbage collector private methods.
- iconv.ERROR_UNKNOWN
  There was an unknown error.

The function can also be used like an OOP method, that is **iconv.convert**(cd, str) and cd@@**convert**(str) are equal.

See also: **strings.tolatin**, **strings.toutf8**.

iconv.close (cd [, ···])

Closes one or more converters `cd` and for each converter successfully closed returns **true**, or **false** otherwise.

With only one handle, the function can also be used like an OOP method, that is **iconv.close**(cd) and cd@@**close**() are equal.

iconv.list ()

Returns a table of all supported codepages.

9.5 hashes - Hashes

9.5.1 Introduction

The package computes various hashes for variable-sized strings and for numbers. All the functions require a string or number as the first argument, and - with the exception of the **hashes.md5** function - optionally the maximum number of slots in an assumed hash table as the second argument if you want the modulo of the hash value to be returned. Alternatively, you can tentatively apply **hashes.fibmod** or **hashes.fibmod2** to the resulting hash for more evenly distributed results.

For almost each of the functions listed below an algorithm in the Agena language roughly explaining its mode of operation has been given.

9.5.2 Usefulness

With a dictionary of 517,996 surnames, where each surname consists of 7.55 characters on average, the following table shows the performance of some string hashes, computed on an Intel i-5 6500 CPU, 3.2 GHz.

Hash	Collision quotient	Collisions	Max. values per hash	Running time
adler32	1.000054	28	2	2.46
asu	1.022457	11'377	5	2.33
bkdr	1.000234	121	2	2.37
bp	3.360010	363'831	4'656	2.2
bsd	12.675476	477'130	447	2.55
cksum	1.000054	28	3	2.33
ccitt	7.906887	452'484	23	2.39
crc7	4046.843750	517'868	4'188	2.49
crc8	2023.421875	517'740	2'157	3.28
crc16	7.910631	452'515	23	3.20
crc32	1.000064	33	2	2.27
dek	1.003477	1'795	4	2.30
derpy	1.000044	23	2	2.49
djb	1.000788	408	2	2.35
djb2	1.000122	63	2	2.66
djb2rot	1.000073	38	2	2.46
elf	1.022457	11'377	5	2.41
fletcher	3.447653	367'750	42	2.28
fnv	1.000070	36	2	2.24
internet	10.537798	468'840	121	2.33
ispell	1.017662	8'990	4	2.37
jen	1.000058	30	2	2.39
lua	1.001793	927	3	2.22
md5	1.000000	0	1	6.94

Hash	Collision quotient	Collisions	Max. values per hash	Running time
murmur1	1.000044	23	2	2.39
murmur2	1.000044	23	2	2.30
murmur3	1.000075	39	2	2.30
murmur3128	1.000066	34	2	2.82
murmur3128x86	1.000066	34	2	2.88
oaat	1.000098	51	2	2.17
pjw	1.022457	11'377	5	2.16
pl	1.000056	29	2	2.71
raw	1.003783	1'952	3	2.27
roaat	1.000655	339	2	2.37
rs	1.000073	38	2	2.19
sax	1.000879	455	2	2.29
sdbm	1.000064	33	2	2.31
sth	1.009873	5'064	5	2.25
strval	8.042167	453'586	8'712	2.29
superfast	1.002553	1'319	3	2.34
varlen	1.000000	0	1	5.75

9.5.3 Summary of Operators and Functions

Numeric Hashes and Checksums

%, **modp**, **symmod**, **math.morton**, **math.modulus**, **math.nearmod**, **bytes.parity**, **hashes.damm**, **hashes.digitsum**, **hashes.droot**, **hashes.fibmod**, **hashes.fibmod2**, **hashes.finger**, **hashes.ftok**, **hashes.hashfloat**, **hashes.interweave**, **hashes.j32to32**, **hashes.jinteger**, **hashes.jnumber**, **hashes.luhn**, **hashes.mix**, **hashes.mix64**, **hashes.mix64to32**, **hashes.numlua**, **hashes.parity**, **hashes.reflect**, **hashes.squirrel32**, **hashes.squirrel64**, **hashes.varlen**, **hashes.verhoeff**, **hashes.wang**, **dblhash.hash**, **numcuckoo.hash**, **numfilter.hash**.

String Hashes

hashes.adler32, **hashes.asu**, **hashes.bkdr**, **hashes.bp**, **hashes.bsd**, **hashes.ccitt**, **hashes.cksum**, **hashes.crc7**, **hashes.crc8**, **hashes.crc16**, **hashes.crc32**, **hashes.dek**, **hashes.derpy**, **hashes.djb**, **hashes.djb2**, **hashes.djb2rot**, **hashes.elf**, **hashes.fletcher**, **hashes.fuv**, **hashes.internet**, **hashes.ispell**, **hashes.jen**, **hashes.lua**, **hashes.md5**, **hashes.murmur1**, **hashes.murmur2**, **hashes.murmur3**, **hashes.murmur3128**, **hashes.oaat**, **hashes.pjw**, **hashes.pl**, **hashes.raw**, **hashes.roaat**, **hashes.rs**, **hashes.sax**, **hashes.sdbm**, **hashes.sha256**, **hashes.sha512**, **hashes.sth**, **hashes.strval**, **hashes.sumupchars**, **hashes.superfast**, **hashes.sysv**, **hashes.varlen**

9.5.4 Functions

A note in advance: You may call **numtheory.nextprime** to compute an optional modulus with all the following functions that support it. This helps distribute data in hash tables better, especially if the hash function has any subtle patterns.

hashes.adler32 (s [, n [, h]])

Returns the Adler32 hash for string `s`. If `n` is given and non-zero, the hash is taken modulo `n` before returning. `h` by default is 65521, but may be any other non-negative integer.

See also: **hashes.crc32**, **minizip.adler32**.

hashes.asu (s [, n [, h]])

Returns a hash for string `s` as proposed by A. V. Aho, R. Sethi, J. D. Ullman in their book "Compilers: Principle, Techniques, and Tools", Addison-Wesley, 1988, p. 436.

If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default.

The algorithm used is equivalent to:

```
> asu := proc(s :: string, n, h) is
>    local g;
>    n := n or 0;
>    h := h or 0;
>    for i in s do
>       h := (h << 4) &+ abs i;
>       g := h && 0xf0000000;
>       if g <> 0 then
>          h ^^:= g >> 24;
>          h ^^:= g
>       fi
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

See also: **hashes.elf**.

hashes.bkdr (s [, n [, seed [, h]]])

Computes a hash value published by Brian Kernighan and Dennis Ritchie, for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional integer `seed` determines a salt is 131 by default; you may chose other primes if necessary. If `h` is given, the initial hash value in the computation is set to this non-negative integer which defaults to zero. The return is a number. The algorithm used is equivalent to:

```
> bkdr := proc(s :: string, n, h) is
>    n := n or 0;
>    seed := seed or 131;  # 31, 131, 1313, 13131, 131313, etc.
>    h := 0;
>    for i in s do
>       h := (h &* seed) &+ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.bp (s [, n [, h]])

Computes a hash for string `s`; it may be useful to classify words with common endings since they have the same hash code. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default.

The return is a number. The algorithm used is equivalent to:

```
> bp := proc(s :: string, n, h) is
>    n := n or 0;
>    h := h or 0;
>    for i in s do
>       h := h << 7 ^^ abs i;
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

See also: **hashes.strval**, **math.ndigits**, **math.nthdigit**.

hashes.bsd (s [, mode])

Returns the 8-bit or 16-bit BSD checksum for the given string `s`. The return is a non-negative integer. By default, the function computes the 16-bit checksum, a value between 0 and 65,535 if `mode` is not given or is **true**. If `mode` is set to **false**, the 8-bit checksum is computed, a value between 0 and 255.

hashes.ccitt (s [, n [, init]])

Performs 16-bit cyclic redundancy check for string `s`, using the CCITT algorithm CCITT algorithm $x^{16}+x^{12}+x^5+x$, starting with initial CRC value `init`, which is 0 by default. The return is a non-negative integer.

If `n`, a positive integer, is given, the computed hash is taken modulo `n`.

hashes.cksum (s [, len])

Returns the same checksum as the UNIX cksum utility for the given string `s`. The return is a non-negative integer. By default, the full length of `s` is evaluated, but you may compute the hash for the first `len` characters by passing a second argument (an integer).

The function can be used to validate the integrity of a file but may not always detect hacker manipulation.

hashes.collisions (s, f [, iters [, factor [, returnbag]]])

Takes a table or sequence `s` of strings and one of the hash functions `f` and returns the mean number of collisions (a value of 1 is best), number of total slots (occupied or free), the time it took to run the procedure, and if `returnbag` is **true**, the hashing table (a bag). If `iters`, a positive integer, is not given, then the function determines the hash values only once, otherwise `iters` times. If `factor`, a positive integer or fraction, is not given, the number of slots of the virtual hash table is twice the number of elements in `s`.

The function is written in Agena (see lib/hashes.agn).

hashes.crc7 (s [, n [, init]])

Performs 8-bit cyclic redundancy check for string `s`, using the CRC-7 algorithm x^7+x^3+1, starting with initial CRC value `init`, which is 0 by default. The return is a positive integer in the range 1 .. 255.

If `n`, a positive integer, is given, the computed hash is taken modulo `n`.

hashes.crc8 (s [, init])

Performs 8-bit reversed cyclic redundancy check for string `s`, starting with initial CRC value `init`, which is 0 by default. The return is a non-negative integer in the range 0 .. 255.

hashes.crc16 (s [, init])

Performs 16-bit reversed cyclic redundancy check for string `s`, starting with initial CRC value `init`, which is 0 by default. The return is a non-negative integer in the range 0 .. 65,535.

hashes.crc32 (s [, init])

Performs 32-bit reversed cyclic redundancy check for string `s`, starting with initial CRC value `init`, which is 0 by default. The return is a non-negative integer in the range 0 .. 4,294,967,295.

See also: **hashes.adler32**, **minizip.crc32**.

hashes.damm (x [, true])

If passed no option, computes the checksum of its argument `x` (an integer or string consisting of ciphers), and returns an integer in the range 0 .. 9 using the Damm

algorithm. Contrary to the Luhn algorithm, it detects all single-digit errors and all adjacent transposition errors.

If passed the Boolean option **true**, the function checks whether `x` includes the correct checksum digit at its end.

If you pass an integer `x` and if |`x`| > **math.lastcontint**, then an error will be issued, for `x` cannot be represented accurately on your system. Pass a string instead.

See also: **hashess.luhn**, **hashes.verhoeff**.

hashes.dek (s [, n [, h]])

Computes a hash value for string `s` proposed by Donald E. Knuth in The Art Of Computer Programming Volume 3, under the topic of sorting and search.

If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default. The algorithm used roughly resembles:

```
> dek := proc(s :: string, n, h) is
>    n := n or 0;
>    h := h or size s;
>    for i in s do
>       h := ((h << 5) ^^ (h >> 27)) ^^ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.derpy (s [, seed [, false]])

Takes a string `s` and an optional `seed` and computes the Derpy hash, a 4-byte XOR-interweaved unsigned integer. You can return the original higher and lower parts of the hash by passing the third argument **false**. The `seed` defaults to 0.

hashes.digitsum (x [, n])

Computes the digit sum of the integer `x` to the base `n` and returns an integer. `n` is 10 by default. If `x` is negative, the result is negative, i.e. -**hashes.digitsum**(abs(`x`), `n`) will be returned. The function is written in Agena and included in the lib/hashes.agn file.

hashes.djb (s [, n [, lsh [, h]]])

Computes the Daniel J. Bernstein hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 5381 by default. The return is a number. The algorithm used roughly resembles:

```
> djb := proc(s :: string, n, sh) is
>    local h;
>    h :=  5381;
>    n :=  n or 0;
>    sh := sh or 5;
>    for i in s do
>       h := (h << sh) &+ h &+ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.djb2 (s [, n [, f [, h]]])

Computes a modified Daniel J. Bernstein hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 5381 by default. The return is a number. The algorithm used roughly resembles:

```
> djb2 := proc(s :: string, n, f) is
>    local h;
>    h := 5381;
>    f := f or 33;
>    n := n or 0;
>    for i in s do
>       h := (f &* h) ^^ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.djb2rot (s [, n [, sh [, f [, h]]]])

Like **hashes.djb2**, but using an additional left rotation-bit shift operation; good performance, few collisions. The algorithm used is equivalent to:

```
> djb2rot := proc(s :: string, n, sh, f) is
>    local h;
>    h :=  5381;
>    f :=  f or 33;
>    sh := sh or 17;
>    n :=  n or 0;
>    for i in s do
>       h := h <<< sh;
>       h := (f &* h) ^^ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.droot (x [, b])

Returns the digital root and the additive persistence for the integer `x` and base `b`. By default `b` is 10.

The digital root is the sum of its digits and the sum of the digits of this sum, and so forth, until the respective sum is less than `b`. The additive persistence is the number of summations it took to compute the root.

hashes.elf (s [, n [, h]])

Similar to **hashes.asu**, but optimised for 32-bit CPUs, commonly used in UNIX systems. The code is equivalent to:

```
> elf := proc(s :: string, n, h) is
>    local x;
>    n := n or 0;
>    h := h or 0;
>    for i in s do
>       h :=(h << 4) &+ abs i;
>       x := h && 0xf0000000;
>       if x <> 0 then
>          h ^^:= x >> 24;
>       fi;
>       h := h && ~~(x)
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.fibmod (k, m [, true])

Returns an unsigned 32-bit integer hash value for the non-negative integer `k` and the given number of slots `m` - also a non-negative integer - that *may* be more evenly distributed than just computing `k` % `m`, with every number far removed from the previous and the next number, but not necessarily in every case. The function uses Fibonacci hashing, and returns a value that is equivalent to:

```
> fibmod := proc(k :: nonnegint, m :: nonnegint) is
>    local p := k *(Phi - 1);
>    return bytes.numto32(m * frac(p))
> end;
```

The result is in the range 0 .. `m` - 1 if `m` is odd; if `m` is even, the result is always in the range 1 .. `m` - 1, unless $k = 0$ or $m = 0$ where the function returns 0.

Note that with a < b, fibmod(a, `m`) is not necessarily less than fibmod(b, `m`).

If you pass the optional third argument **true**, then the results are always the same across different platforms - due to performance reasons, the default is **false**.

See also: **%**, **math.fibmod2**, **math.modulus**.

hashes.fibmod2 (k, i)

Similar to **hashes.fibmod**, but computes the Fibonacci modulus for the signed integer `k` over a set of 2^i slots, with $i \geq 0$.

The result is in the range 0 .. 2^i - 1 if `i` is even; if `i` is odd, the result is always in the range 1 .. 2^i - 1, unless $k = 0$ or $i = 0$ where the function returns 0.

The function is 20 % faster than **hashes.fibmod** called with a power of 2.

See also: **hashes.fibmod**, **math.ispow2**.

hashes.finger (hx, lx [, n])

Takes two unsigned 32-bit integers, represented by standard Agena numbers `hx` and `lx`, and returns the unsigned 32-bit and 16-bit integer fingerprints. If the positive integer `n` is given, the result is taken modulo `n`. Example:

```
> hx, lx, fp16, fp32 := numcuckoo.hash(Pi):
2329595718      3120307696      53942   858182326

> hashes.finger(hx, lx):
858182326       53942
```

The result computed is equal to:

```
> h := ints.uint64(hx, lx)

> h := h ^^ (h >> 32)

> ints.getlow(h % (2^32)), ints.getlow(h % (2^16)):
858182326    53942
```

hashes.fletcher (s, [mode [, len]])

If `mode` is not given or is **true**, returns the position-dependent 16-bit checksum of a string s according to Fletcher's algorithm using an internal 32-bit accumulator, and returns an integer. The 360th and all succeeding characters are ignored.

If `mode` is **false**, returns the position-dependent 8-bit checksum using an internal 16-bit accumulator, and returns an integer in the range [257, 65535]. The 21st and all succeeding characters are ignored.

If the option `len` is given, only the first `len` characters are processed.

hashes.fnv (s [, n])

Computes the Fowler-Noll-Vo hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The return is a number. The algorithm used is equivalent to:

```
> fnv := proc(s :: string, n) is
>    local h;
>    h := 2166136261;
>    n := n or 0;
>    for i in s do
>       h := (h &* 16777619) ^^ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.ftok (inode, device [, id, [, n]])

Computes the System V IPC (Inter Process Communications) key. `inode`, `device` and optional `id` are all 4-byte signed integers, with `id` defaulting to 0. The return is an integer equivalent to, in signed bits mode:

(`inode` && 0xffff) || ((`device` && 0xff) << 16) || ((`id` && 0xffu) << 24)

If `n` is given and non-zero, the hash is taken modulo `n` before returning.

See also: **os.ftok**.

hashes.hashfloat (n [, h])

Computes a non-negative integer hash in the range 0 to 2,147,483,647 for any number `n`, as Lua 5.5.0 does internally to access table members indexed by floating-point numbers. The result is taken modulus `h` with `h` = 1 the default. The function is an alternative to **hashes.numlua**.

hashes.internet (s [, n])

Computes the Internet Checksum according to RFC 1077. If `n` is given and non-zero, the hash is taken modulo `n` before returning. RFC 1077 is often used in network implementations although it has one of the worst collision rates.

hashes.interweave (x [, option [, mask [, sh [, n]]]])

Splits a number `x` into its higher and lower unsigned 4-byte words hx and lx and applies one of the following binary operations on them: `'or'` (the default), `'and'`, `'xor'`.

By passing a non-negative `mask` as the optional third argument, the mask is applied to the intermediate result, the default is 0xFFFFFFFF.

If a fourth positive `sh` integer is given, the intermediate result is right-shifted `sh` bits; if `sh` is a negative integer, it is left-shifted `sh` bits. If `sh` is 0 (the default), there is no shift.

If a fifth argument `n` is given, a positive integer, the intermediate result is taken modulo `n`. The default is 1.

Thus, with hx, lx := bytes.numwords(x):

- `option` = `'or'`: (((hx || lx) && `mask`) >> `sh`) % n, if `sh` > 0,
- `option` = `'and'`: (((hx && lx) && `mask`) >> `sh`) % n, if `sh` > 0,
- `option` = `'xor'`: (((hx ^ ^ lx) && `mask`) >> `sh`) % n, if `sh` > 0,

and

- `option = 'or'`: (((hx || lx) && `mask`) << |`sh`|) % n, if `sh` < 0,
- `option = 'and'`: (((hx && lx) && `mask`) << |`sh`|) % n, if `sh` < 0,
- `option = 'xor'`: (((hx ^ ^ lx) && `mask`) << |`sh`|) % n, if `sh` < 0.

hashes.ispell (s [, n [, h]])

Computes the ISpell hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default.

hashes.j32to32 (x [, n])

Hashes an unsigned 4-byte integer `x` (i.e. in the range 0 .. 2^{32} - 1) to yet another integer in the same range, Julia-style.

If a second argument `n` is given, a positive integer, the intermediate result is taken modulo `n`. The default for `n` is 1.

See also: **hashes.jinteger**.

hashes.jen (s [, n])

Computes the Bob Jenkins' hash (96 bit Mix Function) for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The return is a number. Please see the C hashes.c source file for its implementation.

hashes.jinteger (x [, h])

Value-based hashing of an unsigned 4-byte integer `x`, with seed `h` which by default is 4,294,967,295 = 2^{32} - 1, ported from the Julia language.

See also: **hashes.jnumber**, **hashes.j32to32**.

hashes.jnumber (x [, n [, option]])

Maps a number `x` to one or two unsigned 4 byte integers, Julia-style. If `n`, a positive integer, is given, the computed hashes are taken modulo `n`. By default, only one unsigned 4-byte integer will be returned. If you pass **true** for `option` then the function will split `x` into its higher and lower unsigned 4-byte words and returns unsigned 4-byte integer hashes for each of them. In this case, the second return is equal to the result of **hashes.jnumber** when called without this `option`.

See also: **bytes.numwords**, **hashes.jinteger**.

hashes.lua (s [, n])

Returns the hash, an integer, Lua/Agena internally computes for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. It is an adaption of the Shift-Add-XOR hash. This variant chooses the length of the string as its seed, not a fixed value, and scans from the right to the left. See also: **hashes.sax**.

hashes.luhn (x [, true])

If passed no option, computes the checksum of its argument `x` (an integer or string consisting of ciphers), and returns an integer in the range 0 .. 9 using the Luhn formula, which is used to validate credit card numbers, IMEIs or some social security numbers.

If passed the Boolean option **true**, the function checks whether `x` includes the correct checksum digit at its end.

If you pass an integer `x` and if $|x| >$ **math.lastcontint**, then an error will be issued, for `x` cannot be represented accurately on your system. Pass a string instead.

The Luhn formula does not recognise the transposition 09 vs. 90, nor does it detect twin 22 vs. 55, 33 vs. 66, and 44 vs. 77.

See also: **hashess.damm**, **hashes.verhoeff**.

hashes.md5 (fn, option)
hashes.md5 (s [, n])

In the first form, computes the MD5 hash for file `fn` if `option` is any non-numeric option. `fn` is a file name, not a handle.

In the second form, computes the MD5 hash for string `s`. By default, the function internally splits the string into chunks of $n = 64$ bytes each to compute the result. You can change that by passing any other non-negative integer for `n` where 0 stands for the entire length of `s`.

The return is a string of 32 characters that represent 16 pairs of hexagesimal numbers where the alphabetical letter is in upper-case.

See also: **hashes.varlen**.

hashes.mix (a, b, c)

The function mixes three non-negative integers `a`, `b`, `c` assumed to be 32-bit and returns an integer.

hashes.mix64 (x [, n])

Computes the 64-bit mix for number `x`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The return is a number. See also: **hashes.mix64to32**.

hashes.mix64to32 (x [, n])

Computes the 64-bit mix for number `x`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The return is a number. See also: **hashes.mix64**.

hashes.murmur1 (s [, n [, seed]])

Returns MurmurHash1 for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. Note that the function returns different values on little-endian and big-endian machines. By default `seed` is 0x9747b28c.

See also: **hashes.murmur1**, **hashes.murmur3**, **hashes.murmur3128**.

hashes.murmur2 (s [, n [, seed]])

Returns MurmurHash2 for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. Note that the function returns different values on little-endian and big-endian machines. By default `seed` is 0x9747b28c.

See also: **hashes.murmur1**, **hashes.murmur3**, **hashes.murmur3128**.

hashes.murmur3 (s [, n [, seed]])

Computes MurmurHash3 using 32-bit unsigned integers internally, for the given string `s` and returns an integer. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. Note that the function returns different values on little-endian and big-endian machines. By default `seed` is 0x9747b28c.

See also: **hashes.murmur1**, **hashes.murmur2**, **hashes.murmur3128**.

hashes.murmur3128 (s [, n [, seed]])

Computes MurmurHash3 using 128-bit unsigned integers internally and optimized for 64-bit platforms, for the given string `s` and returns four unsigned 32-bit integers. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. `seed`, if not given, is 0x9747b28c by default. Note that the function returns different values on little-endian and big-endian machines. (In OS/2 the function always returns an error.)

See also: **hashes.murmur1**, **hashes.murmur2**, **hashes.murmur**3, **hashes.murmur3128x86**.

hashes.murmur3128x86 (s [, n [, seed]])

Computes MurmurHash3 using 128-bit unsigned integers internally and optimized for 32-bit platforms, for the given string `s` and returns four unsigned 32-bit integers. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. `seed`, if not given, is 0x9747b28c by default. Note that the function returns different values on little-endian and big-endian machines. (In OS/2 the function always returns an error.)

See also: **hashes.murmur3128**.

hashes.numlua (n [, h [, true | method]])

Computes an integral unsigned 4-byte hash for any integral or fractional number `n`, optionally taken modulo `h`, plus a secondary hash. The return, which may differ across platforms, is equivalent to:

```
> hx, lx := bytes.numworda(n); h := (h - 1) || 1;

> mask :=0x5bd1e995;

> return (hx + lx) % h, (hx * lx) % h,
>        ((hx ^^ mask )+ (lx ^^ mask)) % h;
```

Compared to the other numeric hashing functions available in the package, **hashes.numlua** is best with integral numbers, with only half the collisions than with fractional numbers.

If the last argument is **true**, then the function returns the hash used to store numeric keys in the hash parts of Agena tables. If the last argument is the string "Luau", a MurmurHash64B-style hash will be computed. If the last argument is "GemAl" a MurmurHash3 32-bit hash will be returned.

See also: **hashes.hashfloat**.

hashes.oaat (s [, n [, h]])

Computes the One-at-a-Time hash for string `s`. If given, `n` must be a positive integer. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default. The return is a number. The algorithm used is equivalent to:

```
> hashmask := <: n -> (1 << n) - 1 :>

> oaat := proc(s :: string, n) is
>    local h := 0;
>    n := n or 0;
>    for i in s do
>       inc h, abs i;
>       inc h, h << 10;
>       h ^^:= h >> 6
>    od;
>    inc h, h << 3;
>    h ^^:= h >> 11;
```

```
>    inc h, h << 15;
>    return if n <> 0 then h && hashmask(n) else h fi

> end;
```

See also: **hashes.roaat**.

hashes.parity (x)

Returns a byte with even parity for the non-negative integer `x`, and returns an integer in the range [0, 255]. See also: **bytes.parity32**.

hashes.pjw (s [, n [, h]])

Computes the P. J. Weinberger Hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default.

The return is a number.

hashes.pl (s [, n [, f [, h]]])

Computes Paul Larson's hash of Microsoft Research for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default. The return is a number. The algorithm is equivalent to:

```
> pl := proc(s :: string, n, f, h) is
>    local h := 0;
>    f := f or 101;
>    n := n or 0;
>    h := h or 0;
>    for i in s do
>       h :=(h &* f) &+ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

With initial `h`=5381 and `f`=33, emulates the GNU hash.

hashes.raw (s [, n [, h]])

Computes a self-invented hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default. The return is a number. The algorithm used is equivalent to:

```
> raw := proc(s :: string, n, h) is
>    n := n or 0;
>    h := h or 0;
```

```
>    for i in s do
>       h := 38 &* (h << 1) &+ abs i &- 63
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.reflect (x [, n])

Reorders the bits of the `n`-bit integer `x` by reflecting them about the middle position. By default, `n` is 32, but may be any other integer in [1, 32]. The return is an integer.

hashes.roaat (s [, n [, h]])

Like **hashes.oaat**, but uses bit rotation internally instead of simple bit shifts.

hashes.rs (s [, n [, h]])

Computes the Robert Sedgewick hash for string `s`, but without ANDing the result with 0x7FFFFFFF. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default. The return is a number. The algorithm used is equivalent to:

```
> rs := proc(s :: string, n, h) is
>    local a, b := 63689, 378551;
>    n := n or 0;
>    h := h or 0;
>    for i in s do
>       h := h &* a &+ abs i;
>       a := a &* b
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.sax (s [, n [, h]])

Computes the Shift-Add-XOR hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 5381 by default. The return is a number. The algorithm used is equivalent to:

```
> sax := proc(s :: string, n) is
>    local h := 5381;
>    n := n or 0;
>    for i in s do
>       h := h ^^ ((h << 5) + (h >> 2) + abs i)
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.sdbm (s [, n [, h]])

Computes the ndbm database library hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by

default. The return is a number. The algorithm uses a public-domain implementation. The algorithm used is equivalent to:

```
> sdbm := proc(s :: string, n) is
>    local h := 0;
>    n := n or 0;
>    for i in s do
>       h := abs i &+ (h << 6) &+ (h << 16) &- h
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.sha256 (s [, salt [, rounds]])

Calculates a SHA256 cryptographic hash for string `s`, optionally using a `salt` of type string, and the optional number of `rounds` to be taken. `salt` by default is the empty string and `rounds` is 5000.

The first return is the hash itself, and the second return includes the control parameters `salt` and `rounds` plus the first result.

In case of errors, the function returns **fail**.

Due to the algorithm and the resulting long running times this function is not suited for usage with string hash tables.

hashes.sha512 (s [, salt [, rounds]])

Calculates a SHA512 cryptographic hash for string `s`, optionally using a `salt` of type string, and the optional number of `rounds` to be taken. `salt` by default is the empty string and `rounds` is 5000.

The first return is the hash itself, and the second return includes the control parameters `salt` and `rounds` plus the first result. Thus, the second return is the same as the output of the mkpasswd UNIX command.

In case of errors, the function returns **fail**.

Due to the algorithm and the resulting long running times this function is not suited for usage with string hash tables.

hashes.squirrel32 (key [, seed])

Takes any signed integer `key` and applies Squirrel Eiserloh's hash function on it. The return is a signed 4-byte integer. You may optionally add a `seed` as a second argument, a non-negative integer, with 0 the default.

hashes.squirrel64 (key [, seed])

Similar to **bytes.numwords**, but applying Squirrel Eiserloh's hash function on its first argument `key`, any Agena number, integral or fractional, and returning the higher and lower parts of the result as two unsigned 4-byte integers. You may optionally add a `seed` as a second argument, a non-negative integer, with 0 the default.

hashes.sth (s [, n [, h]])

Computes the s-th hash for string `s`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default. The return is a number. The algorithm has been published at StackOverflow. The algorithm is equivalent to:

```
> sth := proc(s :: string, n) is
>    local h := 0;
>    n := n or 0;
>    for i in s do
>       h := (h << 6) ^^ (h >> 26) ^^ abs i
>    od;
>    return if n <> 0 then h % n else h fi
> end;
```

hashes.strval (s [, sh [, h]])

Computes a hash with many collisions, useful to classify words with common endings since they have the same hash code. `s` denotes the string to be hashed, `sh` the left-shift, which is -8 by default; and `h` the initial hash value before computation starts, 0 by default. The algorithm used is equivalent to:

```
> strval := proc(s :: string, sh, h) is
>    sh := n or 8;
>    h := h or 0;
>    for i in s do
>       h <<:= sh;
>       h := h &+ abs i
>    od;
>    return h
> end;
```

See also: **hashes.bp**, **math.ndigits**, **math.nthdigit**.

hashes.sumupchars (s [, f [, g]])

Sums up all the ASCII values in string `s` and returns the result as a positive integer. The sum is expressed as an unsigned 32-bit integer, so keep overflows in mind.

Instead of just adding the plain ASCII values, you might optionally apply function `f` to the first character in `s`, and function `g` to the second character in `s`, then `f` again on the third character, `g` on the fourth character, and so forth. Example:

```
> sum := hashes.sumupchars("agena",
>    <: x ->(x && 0xff) << 8 :>,
>    <: x -> x && 0xff :>):

> ~~(sum + (sum >> 16)):
```

Note that the Internet checksum, even if strictly implemented according to RFC 1071, has by far more collisions than the other hash functions available in this package.

hashes.superfast (s [, n [, h]])

Takes a string `s` and applies the SuperFastHash algorithm on all its characters, returning an integer in the domain 0 through 4294967295 decimal. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional argument `h` determines the initial value of the resulting hash code before the string is evaluated, and is 0 by default.

hashes.sysv (s [, n [, f [, h]]])

For string `s`, computes the System V hash to access libraries via dynamic symbol tables on UNIX. If `n`, a positive integer, is given, the computed hash is taken modulo `n`. The optional positive integer f is the factor to multiply intermediate results, see algorithm below, and is 16 by default. The optional argument seed `h` determines the salt and is 0 by default; you may chose other primes if necessary.

The return is a number.

The algorithm used is equivalent to:

```
> sysv := proc(s :: string, n, f, h) is
>    n := n or 0;
>    f := f or 16;
>    h := h or 0;
>    for i in s do
>       h := f &* h &+ abs i;
>       h ^^:= (h >> 24) && 0xf0
>    od;
>    h := h && 0xffffffff;
>    return if n <> 0 then h % n else h fi
> end;
```

For GNU hash, see **hashes.pl**.

hashes.varlen (x, salt [, n])

Computes a variable-length integer hash for string or number `x` and string `salt`. If the optional positive integer `n` is given, the computed hash is taken modulo `n`. Depending on the given keyword, the number of collisions might be zero, so this function is an alternative to **hashes.md5**.

hashes.verhoeff (x [, true])

If passed no option, computes the checksum of its argument `x` (an integer or string consisting of ciphers), and returns an integer in the range 0 .. 9 using the Verhoeff algorithm. Contrary to the Luhn algorithm, it detects all single-digit errors, and all accidental transposition involving two adjacent ciphers.

If passed the Boolean option **true**, the function checks whether `x` includes the correct checksum digit at its end.

If you pass an integer `x` and if $|x| >$ **math.lastcontint**, then an error will be issued, for `x` cannot be represented accurately on your system. Pass a string instead. The function also returns an error, if a non-digit is included in string `x`.

See also: **hashes.damm**, **hashes.luhn**.

hashes.wang (x [, n])

Computes the Wang hash, an integer, for non-negative integer `x`. If `n`, a positive integer, is given, the computed hash is taken modulo `n`.

9.6 bloom - Bloom Filter

9.6.1 Introduction

This package implements the Bloom filter, a dictionary containing bit signatures of its individual strings (words).

A Bloom filter is a memory-efficient mean to check whether a string *probably* is part of a dictionary or whether it is *definitely not* part of the dictionary, with acceptable query times. It consumes less memory than the original dictionary of strings and can be used to prevent unnecessary access to the file system on which the actual dictionary resides, for example in dBASE III+, binary or text files.

In the context of this package, the term "dictionary" does not depict an Agena table dictionary, but just a list of strings, e.g.: "Akatsuki", "Chandrayaan", "Chang'e", "Mars Express", "Venera", "Voyager".

Depending on the size of the Bloom filter, the hash string function used, and the number of internal iterations - i.e. number of `salts` - when inserting or reading values, around 80 % of memory can be saved with only around 5 % of the words to be actually looked up in the original dictionary. Bloom filter lookup takes around a third more running time than searching Agena built-in data structures.

Technically, the hash value of a string - see **hashes** package for a variety of string hash functions - is converted into a bit signature that is stored to the slots in the Bloom filter. Internally, the Bloom filter implemented here uses four unsigned bytes for each slot (C type uint32_t). The string hash function should produce the least number of collisions, see Chapter 9.5.2 for a rating of the various hash functions that Agena provides.

You cannot delete values from a Bloom filter. Also, you cannot change the number of slots of the bloom filter or the number of salts once the filter has been created.

You may use the package as follows:

1. Determine the number of entries `s` in your original dictionary `d`.

```
d := ['Akatsuki', 'Chandrayaan', 'Mars Express',
      'Venera', 'Voyager', 'Zond'];
```

2. Create a Bloom filter `b` with **size**(d)\4 slots and 4 salts:

```
b := bloom.new(size(d)\4, 4);
```

3. Insert all entries `str` of your dictionary into Bloom filter `b` using a string hash function, e.g.:

```
for str in d do
   bloom.include(b, hashes.sdbm(str))
od;
```

4. Query the Bloom filter for any entry, using the same hash function:

```
result := bloom.find(b, hashes.sdbm('Zond'));

if result = false then
   print('entry definitely not included')
else
   print('entry probably included, search original dictionary.')
fi;
```

5. Query a Bloom filter slot, with an index counting from 1:

```
bloom.get(b, 1):
```

   of just dump all slots with **bloom.toseq**.

6. Check the state of the bloom filter `b`:

```
bloom.attrib(b):
```

For 100 % certainty check the **trie** package described in Chapter 10.17. You may also try a **cuckoo** filter, see next chapter, which allows to remove entries. You can use **environ.used** and **time** to compare memory consumption and speed of both filters, or tries.

The package supports the **in** and **notin** metamethods, see **bloom.find**, as well as the **size** operator, that is the number of allocated slots. Likewise, the **empty** and **filled** operators check whether the filter is empty or includes at least one word.

9.6.2 Functions

With the exception of **bloom.new**, all of the following package functions can be used OOP-style:

`bloom.attrib (b)`

Returns various information on the Bloom filter:

- key `'size'`: number of internal slots of the bloom filter, the first argument to **bloom.new**.
- key `'salts'`: number of internal hash functions (salts) applied to a word when computing the signature, the second argument to **bloom.new**.

- key `'wordsincluded'`: number of words included into the filter. If the signature of a word is already included, it will not be counted.
- key `'collisions'`: number of collisions detected when trying to include a word into the filter, for its internal signature is already present. If a word has already been included in the filter, its collision is being counted nevertheless.
- key `'bytes'`: size of the whole Bloom filter userdata in bytes.

bloom.get (b, i)

With a bloom filter `b`, returns the value stored at `` `b[i]` ``, where `i`, the index, is an integer counting from 1.

See also: **bloom.toseq**.

bloom.find (b, hash)
bloom.find (b, str)

In the first form, checks whether a string converted to the `hash` value is part of a dictionary of strings represented by Bloom filter userdata `b`.

In the second form, the function tries to find string `str` in Bloom filter `b`, converting it to a MurmurHash3 hash value internally.

The function returns **true** or **false**, where **false** means that the string is *definitely not included* in the original dictionary, and **true** means it is *probably part of* the original dictionary.

Example: `bloom.find(b, hashes.pl('Soyuz')).`

See also: **bloom.include**.

bloom.include (b, hash)
bloom.include (b, str)

In the first form, inserts the `hash` value (an integer) of a string into the Bloom filter `b`, a userdata.

In the second form, the function inserts string `str` into Bloom filter `b`, converting it to a MurmurHash3 hash value internally.

By default, the function returns nothing.

If a hash value has already been inserted, nothing happens.

If the optional third argument is **true**, internal information will be returned: the last internal subhash - an integer - computed before inserting the signature of the string into the Bloom filter, and a table with the keys representing the slot indices of the

Bloom filter modified (an integer starting from 1) and the respective bit position set to 1 (counting from 0, from the right of the bit field).

Example: `bloom.include(b, hashes.pl('Soyuz')).`

See also: **bloom.find**.

`bloom.new (n, salts)`

Creates a Bloom filter, of type userdata, consisting of `n` slots. The number of salts internally applied when inserting or searching the hash value of a string is given by `salts`, a positive integer in the range [1, 65]. If `salts` is 1, then no salt is applied, otherwise (`salts` - 1) salts are applied.

With a large list of surnames, for example, `n` should be at least a fourth of the number of words contained in the dictionary, and `salts` should be 4.

See also: **bloom.attrib**, **bloom.toseq**.

`bloom.toseq (b)`

Receives a Bloom filter `b` and converts its internal slots into a sequence of integers, the return.

See also: **bloom.get**.

9.7 cuckoo - Cuckoo Filter for Strings

9.7.1 Introduction

The package implements a Cuckoo filter for string dictionaries, an alternative to a Bloom filter. Both are space-efficient data structures designed to answer approximate membership queries:

- Is a string *definitely not* part of a dictionary ?
- Is a string *likely* to be part of a dictionary ?

Contrary to the **bloom** package, **cuckoo** allows to delete entries from the filter and is around 10 % faster when creating new filters and 15 % faster when querying them. The false positive probability is around 0.001. For 0 % false positive probability check the **trie** package described in Chapter 10.17.

You can use **environ.used** and **time** to compare memory consumption and speed of Bloom and Cuckoo filters and tries.

Create a filter with a maximum of 13,500 hash entries:

```
> c := cuckoo.new(13500);
```

Insert a string:

```
> cuckoo.include(c, 'abc');
```

Find it:

```
> cuckoo.find(c, 'abc'):
true
```

Remove the string:

```
> cuckoo.remove(c, 'abc');
```

And try to find it again:

```
> cuckoo.find(c, 'abc'):
false
```

The package supports the **in** and **notin** metamethods, see **cuckoo.find**, as well as the **size** operator, that is the number of words added to the filter.

Likewise, the **empty** and **filled** operators check whether the filter is empty or includes at least one word.

For a Cuckoo filter handling Agena numbers, see package **numcuckoo** and package **numfilter** described in chapter 10.18.

9.7.2 Functions

The package functions can also be called OOP-style, if not indicated otherwise.

`cuckoo.attrib (c)`

Returns various information on the Cuckoo filter `c`.

`cuckoo.find (c, str)`

Tries to find string `str` in Cuckoo filter `c`. It returns **false** if `str` is definitely not in the filter, and **true** if it is likely that `str` is part of the filter.

`cuckoo.include (c, str)`

Inserts string `str` into Cuckoo filter `c`. It returns **true** on success and **false** otherwise. If the maximum number of entries has been exceeded, the function deliberately issues an error.

`cuckoo.new ([m [, n [, s]])`

Creates and returns a Cuckoo filter, of type userdata.

`m` depicts the maximum number of slots and by default is 500,000. `n` is the maximum number of kicks, defaulting to 100, and `s` is an optional seed which by default is the number of seconds elapsed since the given epoch.

The function cannot be used as a method.

`cuckoo.remove (c, str)`

Deletes string `str` from Cuckoo filter `c`. It returns **true** on success and **false** if `str` has not been found. In all other cases it returns **fail**.

9.8 regex - Regular Expression Matching

The **regex** package is part of the standard distribution.

UNIX users may need to install the pcre2-8 library before using this binding.

The package provides basic functions to work with Perl Compatible Regular expressions, PCRE2 flavour.

It is advised to define regular expressions as strings enclosed in backquotes as this will prevent escaping of backslashes so you do not have to enter a backslash twice in order to become part of a regular expression.

To include a backquote in a string enclosed by backquotes, use the escape sequence `\q` as this is the only escape sequence processed.

Example:

```
> regex.find('agena', `\w*`):
1       5

> p := regex.new(`[a-z]*`)

> regex.find('agena', p):
1       5

> regex.match('agena', p):
agena
```

Some package functions can also be called OOP-style, e.g.:

```
> p@@find('agena'):
1       5
```

For the full documentation, check the regex.txt file in the doc folder of your Agena installation. Frequently used functions are:

regex.count (subj, patt [, cf] [, ef] [, larg...])

This function counts matches of the pattern `patt` in the string `subj`. For the meaning of the arguments, check **regex.find**.

regex.find (subj, patt [, init] [, cf] [, ef] [, larg...])

The function searches for the first match of the regexp `patt` in the string `subj`, starting from offset `init`, subject to flags `cf` and `ef`.

`subj` is a string, `patt` either a string containing a regex or a regex userdata created by **regex.new**.

`init` is the start offset in the subject and defaults to 1. If a negative value is given, the check starts at the `|init|`-th character from the right.

`cf`, `ef` depict compilation and execution flags (bitwise OR), respectively, passed as numbers.

`larg` may contain PCRE2 library-specific arguments.

The function returns **null** on failure, and on success:

1. the start point of the match (a number, counting from 1),
2. the end point of the match (a number),
3. all substring matches ("captures"), in the order they appear in the pattern. **false** is returned for sub-patterns that did not participate in the match.

See also: **in** operator, **regex.flags**, **regex.match**, **regex.new**, **strings.find**, **strings.glob**.

`regex.flags ([tbl])`

The function returns a table containing the numeric values of the constants defined by the used regex library, with the keys being the (string) names of the constants. If the table argument `tbl` is supplied then it is used as the output table, otherwise a new table is created.

The constants contained in the returned table can then be used in most functions and methods where compilation flags or execution flags can be specified. They can also be used for comparing with return codes of some functions and methods for determining the reason of failure. For details, see the PCRE2 library documentation.

`regex.match (subj, patt [, init] [, cf] [, ef] [, larg...])`

The function searches for the first match of the regexp `patt` in the string `subj`, starting from offset `init`, subject to flags `cf` and `ef`. For the meaning of the arguments, see **regex.find**.

See also: **regex.find**, **regex.flags**, **regex.new**.

`regex.new (patt [, cf] [, larg...])`

The function compiles regular expression `patt` into a regular expression object whose internal representation is corresponding to the library used. The returned result then can be used by the functions, e.g. regex.find, regex.match. Regular expression objects are automatically garbage collected. See the library-specific documentation for details of the library-specific arguments `larg`..., if any.

9.9 fzy - Fuzzy Search

The package provides a few functions for fuzzy string matching.

Most other fuzzy matchers sort based on the length of a match. **fzy** tries to find the result as the user intended. It does this by favouring matches on consecutive letters and starts of words. This allows matching using acronyms or different parts of the path.

The functions are:

fzy.filter (haystacks, needle [, true])

Applies the **fzy.has** and **fzy.positions** functions to a table array of `haystacks`. For large numbers of haystacks, this will have better performance than iterating over the individual haystacks and calling those functions for each string.

```
> fzy.filter(['*ab', 'b', 'a*b'], 'ab'):
[1 ~ [.995, [2, 3], 0], 3 ~ [0.89, [1, 3]]]
```

Unlike the other functions, the `needle` does need not be a subsequence of any of the strings in `haystacks`.

Returns a hash table with one entry per matching item in `haystacks`, each entry giving the index of the item in `haystacks` as well as the equivalent to the return value of the positions for that item.

```
> haystacks := ['cab', 'ant/bat/cat', 'ant/bat/ace'];

> needle := 'abc';

> fzy.filter(haystacks, needle):
[2 ~ [2.63, [1, 5, 9]], 3 ~ [1.725, [1, 5, 10]]
```

fzy.has (haystack, needle [, true])

Checks if string `needle` is a string of string `haystack` and returns **true** or **false**. By default, the check is case-insensitive - by passing optional `true` it will be case-sensitive.

```
> fzy.has('acB', 'ab'):
true

> fzy.has('ac', 'ab'):
false

> fzy.has('acB', 'ab', true):
false
```

fzy.positions (haystack, needle [, true])

Determines where each character of string `needle` is matched to string `haystack` in the optimal match.

`needle` must be a substring of `haystack`, or the result will be undefined. You can verify this by calling the **fzy.has** function before.

By default, the processing is case-insensitive, by passing optional `true` it will be case-sensitive.

Returns a table with the location of the k-th character of `needle` in `haystack`, plus the score, the same value as returned by **fzy.score**.

```
> fzy.positions('app/models/user.rb', 'amuser'):
[1, 5, 12, 13, 14, 15]

> fzy.positions('app/models/customer.rb', 'amuser'):
[1, 5, 13, 14, 18, 19]
```

fzy.score (haystack, needle [, true])

Gets a score for string `needle` that matches string `haystack`.

The `needle` must be a substring of `haystack`, or the result is undefined. You can verify this by calling the **fzy.has** function before.

The return is a number, where higher numbers indicate better matches. 0 denotes no match.

These functions are split for performance reasons. Either use **fzy.has** before each use of **fzy.score**, or use the **fzy.filter** function to do it automatically.

By default, the processing is case-insensitive - by passing optional `true` it will be case-sensitive.

```
> fzy.score('app/models/user.rb', 'amuser'):
5.595

> fzy.score('app/models/customer.rb', 'amuser'):
3.655
```

See also: **strings.dleven**, **strings.fuzzy**.

9.10 suffix - libdivsufsort Binding

The **suffix** library provides a high-performance interface to libdivsufsort, enabling fast suffix array and Longest Common Prefix (LCP) array generation. Instead of polluting the garbage collector with large arrays of individual objects, these primitives return contiguous, packed binary strings of raw signed 32-bit integers, allowing near-native processing speeds inside Agena. The **suffix** package is part of the standard distribution.

There are two package functions, **suffix.sa** and **suffix.lcp**, that are the foundational primitives of the suffix engine. By passing the raw text data through those two specific functions, you generate the presorted suffix positions and overlapping boundary scores in a highly compressed binary layout. Once those basic structures live in memory, you can assemble practically any specialized string algorithm - including high-speed keyword searching, predictive autocomplete generation, phrase counting, or duplicate structural hunting.

A note on memory consumption: each suffix array created with **suffix.sa** takes four times the amount of memory consumed by the input string, and each LCP array created with **suffix.lcp** takes another four times of memory. So you might use the package with care if you are dealing with vast amounts of string data. The increase in speed, however, makes good with that as most of the suffix package functions are around 40 % faster than the functions in the **strings** package.

suffix.sa (str)

Computes the Suffix Array (SA) for the given input string `str` using the libdivsufsort engine. `str` can contain arbitrary binary data or null bytes.

The resulting Suffix Array contains a lexicographically sorted list of all starting indices of the suffixes of `str`. Rather than returning an expensive table of integer elements, this function returns a single immutable binary string where each index is stored as a native signed 4-byte (32-bit) integer. If `str` is empty, an empty string '' is returned.

The function returns a packed binary string of length 4***size**(`str`) bytes representing the Suffix Array offsets.

Worst time complexity is O(N logN). Space complexity is O(N).

Example:

```
> doc    := 'banana';

> sa_bin := suffix.sa(doc);
```

The string length is 6, so the binary block size is 24 bytes:

```
> size sa_bin:
24
```

Extract individual offsets using string unpacking:

```
> for i from 0 to (size doc) - 1 do
>    local offset := strings.unpack('i4', sa_bin, 4*i+ 1);
>    print('Suffix index at position ' & i & ' is ' & offset);
> end;
Suffix index at position 0 is 5
Suffix index at position 1 is 3
Suffix index at position 2 is 1
Suffix index at position 3 is 0
Suffix index at position 4 is 4
Suffix index at position 5 is 2
```

suffix.lcp (str, sabin)

Computes the Longest Common Prefix (LCP) Array using the linear-time Kasai Algorithm.

The LCP Array tracks the length of the shared prefix between adjacent entries in the Suffix Array. It acts as the vital mathematical companion to the Suffix Array, allowing users to evaluate structural repeating sequences or locate duplicate strings effortlessly from Agena scripts.

Like **suffix.sa**, the output is packed into an immutable binary string where each common prefix length is represented as a signed 4-byte (32-bit) integer. The final element at index [**size** `str` - 1] is always 0 to serve as a boundary terminator.

The function takes the original input string `str` and the packed binary Suffix Array `sabin` previously generated via **suffix.sa**(`str`). Its byte size must equal exactly 4***size** `str`.

The return is a packed binary string of length 4***size** `str` bytes representing the shared prefix lengths. The time complexity is O(N).

Example:

```
> doc     := 'banana';
> sabin  := suffix.sa(doc);
> lcpbin := suffix.lcp(doc, sabin);
```

Loop through the LCP entries to see shared prefix sizes:

```
> for i from 0 to size doc - 2 do
>    local len := strings.unpack('i4', lcpbin, 4*i + 1);
>    print('Shared prefix length between sorted suffix ' &
>       i & ' and ' & i + 1 & ' is: ' & len);
> end;
```

```
Shared prefix length between sorted suffix 0 and 1 is: 1
Shared prefix length between sorted suffix 1 and 2 is: 3
Shared prefix length between sorted suffix 2 and 3 is: 0
Shared prefix length between sorted suffix 3 and 4 is: 0
Shared prefix length between sorted suffix 4 and 5 is: 2
```

The following two functions allow to dump and load a string and the associated arrays to disk/from disk with:

suffix.save (filename, str, sabin [, lcpbin])

Saves a precomputed text corpus and its corresponding indexing structures directly to a binary file on disk. This function provides a robust caching mechanism, allowing you to bypass time-consuming index regeneration on subsequent application runs. The arguments are:

- `filename`: The target file path string.
- `str`: The original static text corpus string.
- `sabin`: The raw Suffix Array binary data chunk generated by **suffix.sa**.
- `lcpbin`: (Optional) The raw Longest Common Prefix binary data chunk generated by **suffix.lcp**.

The function returns nothing on success and issues an error otherwise.

suffix.read (filename)

Reads and reconstructs a serialized index file with name `filename` back into active memory.

The function returns, in the following order, three results: the original text corpus string, the raw Suffix Array binary chunk and the raw LCP Array binary chunk, or **null** if the file was saved without LCP metadata.

Example:

```
> corpus, sabin, lcpbin := suffix.read('mydata.bin');
```

All the following additional package functions do not support pattern matching:

suffix.autocomplete (str, sabin, prefix [, delimiters])

Generates a unique, de-duplicated list of words or phrases extracted from `str` that match a specified search `prefix`.

The function uses an accelerated dual binary search over the Suffix Array `sabin` to instantly locate the contiguous block of matching index entries in time. It then sweeps through this slice, validates boundaries, and reads ahead to extract string completions.

The Delimiter Parameter Mechanics: By default, if the optional delimiters parameter is omitted, word extractions are bounded by standard whitespace and sentence punctuation characters. If a custom delimiters string is provided, standard word-splitting rules are entirely bypassed:

Forward Scanning: The engine ignores spaces or periods and continues reading until it encounters a character present inside your custom delimiters set. This allows the function to seamlessly extract multi-word terms or complex phrases.

Boundary Validation: The engine ensures that a substring match is only extracted if it occurs immediately after a character in the delimiters set (or at the absolute beginning of the document). This prevents arbitrary mid-word slice matching.

The function parameters are:

- `str`: The primary text corpus string containing all known valid phrases or words.
- `sabin`: The packed binary Suffix Array mapping the reference text.
- `prefix`: The exact character sequence typed by the user to compute completions for.
- `delimiters` (optional): A string containing the exact character set to be treated as structural entry boundaries.

The function returns a table containing all unique word terms matching the input prefix. If no terms exist, an empty table is returned.

Time complexity is O(M logN + K*L) where M is the prefix length, N the corpus size, K the span of matched suffixes and L the average length of the extracted words. Space complexity is O(K*L).

Example: 1. Default mode (splits on standard spaces and punctuation)

```
> corpus :=
> 'banana split,banana bread,banana shake,apple pie,apricot tart,bandanna';

> sabin  := suffix.sa(corpus);
```

Request completions using default delimiters:

```
> suffix.autocomplete(corpus, sabin, 'ba'):
[banana, bandanna]
```

'banana' is collected once because trailing details are cut at the spaces.

Example 2. Structured mode (splits strictly on custom delimiters)

```
> corpus :=
> 'banana split,banana bread,banana shake,apple pie,apricot tart,bandanna';

> sabin  := suffix.sa(corpus);
```

Force the engine to *only* split when hitting a comma:

```
> suffix.autocomplete(corpus, sabin, 'ba', ','):
[banana bread, banana shake, banana split, bandanna]
```

Multi-word sequences are treated as single entries because spaces are ignored.

suffix.count (str, sabin, pattern [, init])

Finds the number of occurrences of string `pattern` in string `str` using Suffix Array `sabin`. A fourth, optional numerical argument `init` specifies where to start the count; its default value is 1 and may be negative to indicate a position from the right end of `str`.

For more information, see **suffix.findall**.

Example:

```
> doc := 'banana split, banana bread, banana shake';

> sabin := suffix.sa(doc);

> suffix.count(doc, sabin, 'banana'):
3
```

suffix.find (str, sabin, pattern [, init [, nocase]])

Performs fast substring search for string `pattern` inside string `str` by executing a binary search over the pre-computed Suffix Array `sabin`.

Instead of sequentially scanning characters from left to right like standard **strings.strstr**, this function matches the pattern against alphabetically sorted suffixes. This changes the execution time to depend primarily on the length of your search term rather than the overall size of your file. It is ideal for indexing static documents that require multiple rapid keyword lookups.

If a match is found, it returns the character position relative to the starting point of the string and the matching slice itself. If the pattern does not exist or an argument is missing, it cleanly returns **null**.

The function takes the original document text string `str`, the packed binary Suffix Array `sabin` matching the document, generated via **suffix.sa**(`str`), and the target search substring `pattern` you want to locate.

A fourth, optional numerical argument `init` specifies where to start the search; its default value is 1 and may be negative to indicate a position from the right end of `str`.

If you pass **true** as a fifth argument `nocase` to the function, the search will be case-insensitive.

The function returns a 1-based integer index pointing to where the first character of the match starts in the string and a copy of the matched text slice.

Time complexity is O(M logN) where M is the pattern length and N is the text length. Space complexity is O(1), constant auxiliary space.

Example:

```
> doc := 'The quick brown banana jumps over the lazy dog';

> sabin := suffix.sa(doc);
```

Locate a word instantly without scanning the whole sentence string:

```
> pos, match := suffix.find(doc, sabin, 'banana'):
17      banana
```

See also: **suffix.findall**, **suffix.gfind**.

`suffix.findall (str, sabin, pattern)`

Locates all sequential occurrences of a specified `pattern`, a string, inside string `str` using an accelerated two-stage search strategy over the Suffix Array `sabin`.

Because identical substring sequences group together tightly in a sorted suffix space, this function uses a fast binary search to establish an initial anchor index. It then performs an immediate linear scan up and down the boundary neighbours to pull all matching offsets in time per instance.

The extracted array indices are automatically sorted chronologically into standard left-to-right reading order and packaged into a natively indexable table.

The function takes the original document text string `str`, the packed binary Suffix Array `sabin` matching the document, generated via **suffix.sa**(`str`), and the target search substring `pattern` you want to locate.

The function returns a table containing 1-based integer index locations specifying exactly where each instance of the pattern begins in the document. If the pattern is not found, an empty table is returned.

The time complexity is O(M logN + K logK) where M is the pattern length, N is the document length, and K is the total count of matches found. Space complexity is O(K).

Example:

```
> doc := 'banana split, banana bread, banana shake';

> sabin := suffix.sa(doc);

> suffix.findall(doc, sabin, 'banana'):
[1, 15, 29]
```

See also: **suffix.find**, **suffix.gfind**.

suffix.gfind (str, sabin, pattern [, init [, nocase]])

Returns an iterator function that, each time it is called, returns the position of the next match of a pattern in a string, or **null** if there is no (further) match.

For the arguments, see **strings.find**. See also: **suffix.findall**.

suffix.unique (str [, position])

Finds the shortest unique substring (SUS) inside string `str` that begins exactly at the specified `position`.

By evaluating the Longest Common Prefix (LCP) values of the target position's alphabetical neighbours, the function calculates the minimum number of characters required to differentiate that specific slice from every other sequence in the document. The required length is always precisely 1 character longer than the maximum overlapping prefix shared with its closest alphabetical neighbours in the Suffix Array index.

If the character sequence remains non-unique all the way to the end of the text block, the entire trailing slice from position to the end of the string is returned.

The function takes the source string `str` to analyze and optionally the `position`: The 1-based integer index pointing to the starting character inside `str`. Must be a positive integer within bounds (1 ≤ position ≤ **size** `str`), the default is 1.

It returns the shortest substring starting at position that appears exactly once across the entire text. Returns an empty string if the position is out of bounds.

Examples:

```
> suffix.unique('alexander'):
al
```

Note: Just 'a' is not unique because 'ander' also starts with 'a'.

Find the shortest unique sequence starting at position 4 ('xander'):

```
> suffix.unique('alexander', 4):
x
```

Note: 'x' appears only once in the entire word, so a single character suffices.

9.11 binpack - Serialising Strings & Numbers

The **binpack** library provides functions to convert Agena numbers, complex numbers, Booleans, **null**, and strings plus signed 64-bit integers, C long doubles and double-to-double numbers into binary strings and back again.

While the functions provided here do not pack data as tightly as **strings.pack** and **strings.unpack**, they save you from having to construct and store a format string for subsequent unpacking.

You can either use **binpack.pack** and **binpack.unpack** to serialise one or more arguments, or use a buffer. Examples:

```
> str := binpack.pack(1, 'agena', 2*Pi, Pi!E, true, false, null,
>    fail, long.Pi, ints.int64(10), dd.new(2.1));

> binpack.unpack(str):
[1 ~ 1, 2 ~ agena, 3 ~ 6.2831853071796,
 4 ~ 3.1415926535898+2.718281828459*I, 5 ~ true, 6 ~ false, 8 ~ fail,
 9 ~ longdouble(3.14159265358979323385), 10 ~ int64(10),
 11 ~ 2.1000000000000000 (0)]
```

Create a buffer,

```
> b := binpack.new()
```

and fill it with data:

```
> b@@chars('agena')

> b@@int32(-10)

> b@@uint16(16)

> b@@uint32(32)

> b@@double(Pi)

> b@@cdouble(E!Pi)

> b@@longdouble(long.Pi)

> b@@int64(ints.int64(10))

> b@@dd(dd.new(2.1))
```

Pack:

```
> str := b@@tostring();
```

and unpack:

```
> binpack.unpack(str):
[agena, -10, 16, 32, 3.1415926535898, 2.718281828459+3.1415926535898*I,
 longdouble(3.1415926535897932385), int64(10), 2.1000000000000001 (0)]
```

Most of the following package functions can also be used OOP-style:

binpack.pack (···)

Serialises one or more numbers, complex numbers, Booleans, **null** and strings into a binary string. Arguments can also be long doubles provided by the **long** package, signed 8-byte integers created with the **ints** package and double-to-double numbers as used in the **dd** package.

Alternatively, you can put the data into a table or sequence and pass it as the first argument.

binpack.unpack (str)

Unpacks a binary string `str` into the components that it represents and returns the data in a table. `str` should have been created with either **binpack.pack**, **binpack.tostring** or **binpack.dump** previously.

binpack.new ()

Creates and returns a new buffer to collect data to be serialised later on.

binpack.tostring (b)

Returns serialised data collected in buffer `b`, see **binpack.new**, as a binary string. The contents of the buffer is not cleared and left untouched.

binpack.dump (b)

Like **binpack.tostring** but clears the buffer entirely for re-use after returning a binary string.

The following functions add numeric data, represented as the second and optionally further arguments, to a buffer represented by the first argument. The buffer must have been created with **binpack.new**. The return is always a reference to the buffer to allow chaining.

Function	Input & Description
binpack.bool	Boolean **true**, **false** or **fail**
binpack.chars	string of any length; strings up to 255 characters in length are packed more tightly
binpack.cdouble	complex number to be stored as a 16-byte C complex double
binpack.dd	double-to-double number created with **dd.new**

Function	Input & Description
binpack.double	number, to be stored as an 8-byte C double
binpack.int32	number in the range -2,147,483,648 to 2,147,483,647
binpack.int64	8-byte signed integer created with **ints.int64**
binpack.longdouble	80-bit number created with **long.new**
binpack.nil	**null**
binpack.uchar	number in the range 0 to 255
binpack.uint16	number in the range 0 to 65,535
binpack.uint32	number in the range 0 to 4,294,967,295

9.12 aspell - GNU Aspell 0.60 Spell Checker

The **aspell** library interfaces to the GNU Aspell 0.60 spell checker. As a plus package, activate it by issuing:

```
> import aspell;
```

The package is available on OS/2, Windows, Solaris, Linux and macOS.

Example usage:

```
> import aspell;

> d := aspell.new('en_US');

> d@@check('Chattanuga'):
false

> d@@suggest('Chattanuga'):
[Chattanooga, Chaitanya, Chatting, Chattanooga's, Charting, Chautauqua]

> d@@close();
```

The package comes with at least an English dictionary, see the data/aspell or data/aspell64 folder of your Agena installation.

The functions, which can also be used OOP-style, are:

aspell.close (speller)

Frees the underlying memory and system resources allocated for Aspell `speller` instance.

See also: **aspell.new**.

aspell.check (speller, word)

Checks whether a specific `word`, a string, is spelled correctly according to the loaded language dictionary `speller` and returns **true** or **false**.

If internal dictionary processing fails (returning **null**), the second return contains the explicit Aspell error description.

See also: **aspell.suggest**.

aspell.config (speller, key [, value])

Retrieves or sets the configuration for the given `speller`. When given two arguments, where `key` is a string, the respective setting will be returned. When also given string `value`, the configuration will be changed for the specific setting denoted by `key`. Not all settings can be changed.

Available settings include:

1. Dictionary Selection

- 'lang': The core language string (e.g., 'en_US', 'fr').
- 'master': The base filename of the master dictionary.
- 'variety': Distinguishes extra dictionary variations if size and language are identical.

2. Spelling & Suggestion Control

- 'sug-mode': Adjusts suggestion speed vs. precision. Acceptable values include 'ultra', 'fast', 'normal', and 'bad-spellers'.
- 'ignore': Tells Aspell to ignore words below a certain length (takes an integer converted to string, e.g., '3').
- 'ignore-case': Determines whether to ignore variations in capitalization ('true' or 'false').
- 'camel-case': Enables or disables checking split words inside camelCase tokens ('true' or 'false').
- 'run-together': Treats run-together words as legal compounds ('true' or 'false').
- 'size': Size metric score rating of the active dictionary word list.

3. File Path Configuration

- 'dict-dir': The directory containing the primary dictionary binaries.
- 'data-dir': The directory containing language prefix rules and .dat configuration files.
- 'personal': The specific filename or absolute path for the user's personal wordlist database.

4. Text Content Processing (Filters)

- 'mode': Adjusts document parsing type rules. Common structural modes include 'url', 'email', 'html', 'tex', and 'markdown'.

5. Encoding

- 'encoding': The encoding the input text is in. Valid values include, but not limited to, 'iso-8859-*', 'utf-8', 'ucs-2', 'ucs-4'.
- 'normalize': Perform Unicode normalization. Enabled by default.

Please consult the Aspell 0.60 documentation on the Internet for more information and settings.

aspell.new (lang)

Creates and initializes a new Aspell speller instance for a specific language.

The function takes a string `lang` depicting the language to initialize (for example, 'en', 'en_UK', 'en_US', 'en_CA') and returns a newly created Aspell speller object if successful. In case of error, the function returns **null** and an error message.

The binding automatically resolves its execution path to locate the dictionary files. If no specific path has been set with **aspell.path**, on 32-bit Agena, it looks for a `data/aspell` folder of your Agena installation, and an `data/aspell64` folder on 64-bit Agena.

Agena comes with a pre-installed English dictionary only. You will find additional dictionaries in the Agena SourceForge download folder "Packages":

https://sourceforge.net/projects/agena/files/Packages/AspellDicts.zip

You may download alternative or additional Aspell 0.60-compliant dictionaries from other sites, but be aware that Aspell 32-bit dictionaries only work with the 32-bit edition of Agena and Aspell 64-bit dictionaries only work with the 64-bit edition.

The following table lists all the dictionaries in the AspellDicts.zip archive. Sometimes you may just pass only the first two characters to **aspell.new**.

Short	**Language**	**Short**	**Language**
af	Afrikaans	la	Latin
ar	Arabic	ml	Malayalam
bg	Bulgarian	ms	Malay
cs	Czech	nb	Norwegian Bokmal
da	Danish	nds	Low German
de	German (Common word list)	nl	Dutch
de_AT	German (Austria specific)	nn	Norwegian Nynorsk
de_CH	German (Switzerland specific)	pa	Punjabi
de_DE	German (Germany specific)	pl	Polish
el	Greek	pt_BR	Portuguese (Brazil)
en	English (Common word list)	ro	Romanian (Modern standard)
en_CA	English (Canada)	ro-classic-only	Romanian (Historical spelling)
en_GB	English (United Kingdom)	ro-common	Romanian (Common word list)
en_US	English (United States)	ru-ye	Russian
es	Spanish	ru-yo	Russian
fa	Farsi	sk	Slovak

Short	Language	Short	Language
fi	Finnish	sr-cyrl-only	Serbian (Cyrillic script)
fr	French	sr-latn-only	Serbian (Latin script)
fr_CH	French (Switzerland specific)	sv	Swedish
fy	Western Frisian	sw	Swahili
he	Hebrew	ta	Tamil
hi	Hindi	tr	Turkish
hu	Hungarian	uz	Uzbek
id	Indonesian	vi	Vietnamese
is	Icelandic	yi	Yiddish
it	Italian	zu	Zulu
ku	Kurdish		

See also: **aspell.close**.

aspell.path ([path])

Gets or sets the custom filesystem directory where the **aspell.new** searches for dictionary files (such as .multi and .rws files).

If you provide a directory `path` string, the function validates that the path exists and is accessible before saving it. You can completely delete a custom directory previously set by the function by passing the empty string.

If you call the function without any arguments, it returns the currently active custom dictionary path. If no path has been set before with **aspell.path**, or if it has been unset, the function returns **null**.

aspell.suggest (speller, word)

Retrieves a list of spelling corrections and alternative suggestions for a given word, a string, in dictionary speller.

If no corrections are found, or if an internal calculation issue occurs, an empty table is returned.

The suggestions in the returned table are ordered by relevance, matching Aspell's internal scoring metric. The very first entry represents the closest, most statistically accurate match calculated by the spelling engine, followed sequentially by decreasingly likely alternatives.

See also: **aspell.check**.

Chapter Ten

Structures

10 Structures

10.1 Tables

Summary of Functions:

General Queries

$$**, **$$$, **bisequal**, **bottom**, **columns**, **countitems**, **empty**, **filled**, **getentry**, **getorset**, **has**, **in**, **member**, **notin**, **recurse**, **size**, **tables.getdim**, **tables.getsize**, **tables.getsizes**, **tables.hashole**, **tables.isall**, **tables.isarray**, **tables.ishash**, **tables.isnullarray**, **tables.isone**, **tables.isrectangular**, **tables.issquare**, **tables.iszero**, **tables.maxn**, **top**, **type**, **typeof**, **values**, **whereis**.

Retrieving Values

$, **columns**, **descend**, **getentry**, **unique**, **unpack**, **values**, **tables.array**, **tables.borders**, **tables.col**, **tables.entries**, **tables.getarray**, **tables.getfield**, **tables.gethash**, **tables.geti**, **tables.gettable**, **tables.hash**, **tables.indices**, **tables.parts**.

Operations

@, **append**, **bintersect**, **bminus**, **cleanse**, **copy**, **copyadd**, **freeze**, **include**, **join**, **map**, **move**, **pack**, **prepend**, **purge**, **put**, **qsumup**, **remove**, **reverse**, **select**, **selectremove**, **shift**, **sort**, **sorted**, **subs**, **subsop**, **sumup**, **swap**, **unique**, **zip**, **tables.cleanse**, **tables.concat**, **tables.extend**, **tables.include**, **tables.move**, **tables.pack**, **tables.reshuffle**, **tables.resize**, **tables.setfield**, **tables.settable**, **tables.subs**, **tables.swapcol**, **tables.swaprow**, **tables.transpose**, **tables.unpack**, **unfreeze**.

Relational Operators

=, ==, ~=, <>. ~<>.

Cantor Operations

intersect, **minus**, **subset**, **union**, **xsubset**.

Miscellaneous

tables.dimension, **tables.allocate**, **tables.auto**, **tables.new**, **tables.newtable**, **tables.tableoftables**.

10.1.1 Operators & Functions

append (x, t)

Adds `x` to the end of table `t`, in-place. The function returns the modified structure. For more information, check Chapter 8.

See also: **copyadd**, **include**, **prepend**, **put**, **insert** statement.

augment (t1, t2 [, ···])

Joins two or more tables `t1`, `t2` etc. together horizontally. All tables must be of the same size and have the same keys. The function is written in Agena and included in the lib/library.agn file.

See also: **columns**, **linalg.augment**, **zip**.

bintersect (t1, t2 [, option])

Returns all values of table `t1` that are also values in table `t2`. The function performs a binary search in `t2` for each value in `t1`. If no option is given, `t2` is sorted before starting the search. If you pass an option of any value then `t2` should already have been sorted, for no correct results would be returned otherwise.

With larger tables, this function is much faster than the **intersect** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bisequal**, **bminus**.

bisequal (t1, t2 [, option])

Determines whether the tables `t1` and `t2` contain the same values. The function performs a binary search. If no option is given (any value), `t1` and `t2` are sorted before starting the search. If you pass an option of any type then `t1` and `t2` should already have been sorted, for no correct results would be returned otherwise.

With larger tables, this function is much faster than the = operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bminus**.

bminus (t1, t2 [, option])

Returns all values of table `t1` that are not values in table `t2`. The function performs a binary search in `t2` for each value in `t1`. If no option is given, `t2` is sorted before starting the search.

If you pass any `option` then `t2` should already have been sorted, for no correct results would be returned otherwise.

With larger tables, this function is much faster than the **minus** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bisequal**.

bottom (t)

With table array `t`, the operator returns the element at index 1. If `t` is empty, it returns **null**.

See also: **top**.

cleanse (t)

Empties table `t`, and returns the emptied structure. The memory previously occupied can be reused by the interpreter. See also: **pack**.

columns (t, p [, ···] [, 'structure'])

Extracts the given columns `p` (etc.) from the two-dimensional table `t`. The return is either a table of structures if the option `'structure'` is given, or a multiple return of tables.

The function is written in Agena and included in the lib/library.agn file.

See also: **ops**, **select**, **unpack**, **values**, **linalg.col**, **utils.readscv**, **zip**.

copy (t)

The operator copies the entire contents of a table `t` into a new table. See Chapter 8 for more information.

copyadd (t [, ···])

Copies all elements in table `t` and any further optional arguments into a new table and returns it.

The array and hash parts are copied 1:1, i.e. the keys and entries in the array part of `t` are copied to the array part of the new table, and the elements in the hash part of `t` are copied to the hash part of the new table, with the same keys, too. For performance reasons, substructures are not deep-copied.

For further information, check the description of **copyadd** in Chapter 8.

countitems (item, t)

countitems (f, t [, ···])

In the first form, counts the number of occurrences of an `item` in the table `t`.

In the second form, by passing a function `f` with a Boolean relation as the first argument, all elements in the structure `t` that satisfy the given relation are counted. If the function has more than one argument, then all arguments *except the first* are passed right after the name of table `t`.

The return is a number. The function may invoke metamethods.

See also: **$$$** and **size** operators, **select**.

descend (f, t, [, ···] [, option])

Returns all elements in table `t` that satisfy a given condition expressed by function `f`. The function can be multivariate and must return either **true** or **false**. The optional second and all further arguments of `f` may be passed as the third, etc. argument.

All the keys and entries are scanned. For more information see the description of **descend** in Chapter 8.

See also: **has**, **recurse**, **select**.

duplicates (t [, option])

Returns all the values that are stored more than once to the given table `t`, and returns them in a new table. Each duplicate will be returned only once.

If `option` is not given, the structure is sorted before evaluation since this is needed to determine all duplicates. The original structure is left untouched, however.

The total size of the new register is equal to the number of the elements in the result.

If a value of any type is given for `option`, the function assumes that the table has been already sorted. Otherwise it is suggested to use **skycrane.sorted** before the call to **duplicates** if the table contains values of different types, to prevent errors.

The function is written in Agena and included in the lib/library.agn file.

empty (t)

Checks whether table `t` does not contain any element. The return is **true** or **false**. The operator works with dictionaries, as well. See also: **filled**.

filled (t)

Checks whether table t contains at least one element. The return is **true** or **false**. The operator works with dictionaries, as well. See also: **empty**.

freeze (t)

Write-protects table t. See Chapter 8 for further information. See also: **unfreeze**.

getentry (t [, k_1, ···, k_n])

Returns the entry $t[k_1, \cdots, k_n]$ from the table t without issuing an error if one of the given indices k_i (second to last argument) does not exist. See also **rawget**.

getmetatable (t)

If t does not have a metatable, returns **null**. Otherwise, if the t's metatable has a '__metatable' field, returns the associated value. Otherwise, returns the metatable of table t.

See also: **setmetatable**.

getorset (t, k_1, ···, k_n, v)

Returns the non-**null** element at index $t[k_1, k_2, ..., k_n]$, where t is a table. If any index position is invalid, the function returns **null**.

If $t[k_1, k_2, ..., k_n]$ = **null**, then the function assigns $t[k_1, k_2, ..., k_n] := v$ and returns v.

See also: **getentry**.

has (t, x)

Checks whether table t contains element x. In general, all the entries are scanned, but if x is not a number then the indices of the table are searched, too. The function performs a deep scan so that it can find elements in deeply nested tables.

The function return **true** if x could be found in t, and **false** otherwise.

See also: **descend**, **in**, **member**, **recurse**, **satisfy**.

include (t, x [, ···])

Inserts one or more values x, ··· to the end of table t, not discarding multiple returns if its last argument is a function call. For more information, see Chapter 8.

See also: **copyadd**, **append**, **prepend**, **put**, **insert** statement.

map (f, t [, ···] [, true])

Maps the function `f` on all elements of a table `t`. See **map** in Chapter 8 for more information.

See also: **countitems**, **remove**, **select**, **selectremove**, **subs**, **subsop**, and **zip**.

member (x, t)

Searches `x` in the table `t` and if successful returns **true** plus the index of the first hit. Otherwise returns **false** and **null**. The function is much faster than **whereis** if you need the index of the first hit only. Note that with respect to **whereis**, the parameters are in reverse order.

See also: **has**, **whereis**, **tables.entries**, **tables.indices**.

move (t1, start, stop, newidx [, t2])

Copies elements from table `t1` to table `t2`, performing the equivalent to the following multiple assignment: `t2`[`newidx`],··· = `t1`[`start`], ···, `t1`[`stop`]. The default for `t2` is `t1`, i.e. elements are shifted in the same table. The destination range can overlap with the source range.

The function returns the destination table `t2`, which might be simply `t1`.

See also: **purge**, **put**, **shift**, **tables.move**.

pack (t [, newsize])

When called with no second argument, reduces the number of allocated slots in table `t` to the number of slots actually assigned a value, also freeing memory if possible.

With registers, also removes all trailing **null**. With tables removes holes in the array part and closes the space by shifting down the other elements, if necessary.

If the optional argument `newsize`, a non-negative integer, is given, then the number of pre-allocated slots is reduced to the number of actually assigned values, purging all surplus elements that may exist and giving back memory to the interpreter.

The function works in-place and returns nothing. It automatically conducts a garbage collection before returning.

See also: **cleanse**.

prepend (x, t)

Prepends `x` to the beginning of table `t`, in-place. The function returns the modified structure. For more information, check Chapter 8.

See also: **append**, **copyadd**, **include**, **put**, **insert** statement.

purge (t [, pos])

purge (t, a, b)

Removes from table `t` the element at position `pos`, shifting down other elements to close the space, if necessary. Returns the value of the removed element. The default value for `pos` is n, where n is the length of the table, so that a call `purge(t)` removes the last element of `t`.

In the second form, removes all elements starting from index `a` to index `b` (inclusive), moving excess elements down to close the space; the function automatically performs a garbage collection after shifting. In the 2nd form, nothing will be returned.

Use the **delete** *element* **from** *table* statement if you want to remove any occurrence of the table value *element* from a table.

Note that the function only works if the table is an array, i.e. if it has positive integral and consecutive keys only.

See also: **move**, **put**, **shift**, **swap**, **tables.remove**.

put (t, [pos,] value)

Inserts element `value` at position `pos` in table `t`, shifting up other elements to open space, if necessary. The default value for `pos` is n+1, where n is the current table size, so that a call `put(t, value)` inserts `value` at the end of `t`.

Use the **insert** *element* **into** *structure* statement if you want to add an element at the current end of a table, for it is much faster.

The function returns the modified structure.

See also: **move**, **prepend**, **purge**, **tables.remove**.

qsumup (t)

Raises all numeric values in table `t` to the power of 2 and sums up these powers. See **qsumup** in Chapter 8 for more information. See also: **sumup**.

recurse (f, t [, ···][, option])

Checks each element of table `t` by applying a function `f` on each of its elements. `f` can be a multivariate function and must return either **true** or **false**. The optional second and all further arguments of `f` may be passed as the third, etc. argument.

All the entries and keys are scanned. For more information, see the description of **recurse** in Chapter 8.

See also: **has**, **descend**, **select**.

remove (f, t [, ··· [, reshuffle=true] [, inplace=true]])

Returns all values in table `t` that do not satisfy a condition determined by function `f`. See **remove** in Chapter 8 for more information.

See also: **map**, **select**, **selectremove**, **subs**, **subsop**, **zip**.

reverse (t)

Reverses the order of all elements in the array part of table `t`. The function returns the modified structure.

See also: **strings.reverse**, **stack.reversed**.

select (f, t [, ··· [, reshuffle=true]])

Returns all values in table `t` that satisfy a condition determined by function `f`. See **select** in Chapter 8 for more information.

See also: **$** and **$$$** operators, **countitems**, **map**, **remove**, **selectremove**, **subs**, **subsop**, **zip**.

selectremove (f, t [, ··· [, reshuffle=true]])

Returns all values in table `t` that satisfy and do not satisfy a condition determined by function `f`, in two tables. See **selectremove** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subs**, **subsop**, **zip**.

setmetatable (t, metatable)

Sets the metatable for the given table `t`. (You cannot change the metatable of other types from Agena, only from C.) If `metatable` is **null**, removes the metatable of the given table. If the original metatable has a '__metatable' field, raises an error. The function cannot assign metatables to C library functions.

This function returns `t`.

See also: **getmetatable**.

shift (t, a, b)

Moves an element in the table array `t` from position `old` to `new`, with `old`, `new` integers, shifting all the other elements accordingly - which might also cause a rotation. The function returns nothing.

See also: **move**, **purge**, **put**, **rotate** statement, **swap**.

size (t)

Returns the number of actual entries in the array and hash parts of table `t`. The operator returns a number and conducts a linear traversal.

See also: **$$$** operator, **countitems**, **environ.attrib**, **tables.getsize**, **tables.getsizes**.

sort (t [, comp] [, 'number'])

Sorts table `t` in a given order, and in-place. See **sort** in Chapter 8 for more information.

See also: **sorted**, **skycrane.sorted**, **stats.issorted**, **stats.sorted**.

sorted (t [, comp] [, 'number'])

Sorts table elements in `t` in a given order, but - unlike sort - not in-place, and non-destructively. See **sorted** in Chapter 8 for more information.

See also: **sort**, **skycrane.sorted**, **stats.issorted**, **stats.sorted**.

subs (x:v [, ···], t [, options])

Substitutes all occurrences of value `x` in table `t` with value `v`. See **subs** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subsop**, **zip**.

subsop (i:v [, ···], t [, options])

Substitutes the value at `t[i]` with value `v`, where `v` can also be **null**, deleting the element in this case. See **subsop** in Chapter 8 for more information.

sumup (t)

Sums up all numeric values in table `t`. See **sumup** in Chapter 8 for more information.

See also: **qsumup**.

swap (t, a, b)

Swaps the table array `t` entries at index positions `a` and `b`, with `a`, `b` integers. The function returns nothing.

See also: **move**, **purge**.

top (t)

With table array `t`, the operator returns the element with the largest index. If `t` is empty, it returns **null**.

See also: **bottom**.

type (t)

Returns the type of table `t`, i.e. the string 'table'.

typeof (t)

Returns either the user-defined type of table `t`, or the basic type 'table'.

unique (t [, true])
unique (t [, options])

The function removes all holes (`missing keys`) in the array part of table `t` and removes multiple occurrences of the same value, if present. See **unique** in Chapter 8 for more information.

values (t, i_1 [, i_2, ···])

Returns the elements from the given table `t` in a new table. This function is equivalent to

```
return [ i1 ~ t[i1], i2 ~ t[i2], ··· ]
```

See also: **ops**, **select**, **unpack**.

zip (f, t1, t2)

This function zips together two tables `t1`, `t2` by applying the function `f` to each of its respective elements. See Chapter 8 for more information.

See also: **augment**, **columns**, **map**, **remove**, **select**, **subs**.

The following functions have been built into the kernel as binary operators.

Please note that the operators returning a Boolean work in the Cantor way, i.e. `{1, 1} = {1}` → true, `{1, 2} xsubset {1, 1, 2, 2, 3, 3}` → true.

t1 = t2

This equality check of two tables `t1`, `t2` first tests whether `t1` and `t2` point to the same table reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `t1` and `t2` contain the same values without regard to their keys, and returns **true** or **false**. In this case, the search is quadratic.

See also: **bisequal**, **environ.isequal**.

t1 == t2

This strict equality check of two tables `t1`, `t2` first tests whether `t1` and `t2` point to the same table reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `t1` and `t2` contain the same number of elements and whether all key~value pairs in the tables are the same. In this case, the search is linear.

See also: **bisequal**, **environ.isequal**.

t1 ~= t2

This approximate equality check of two tables `t1`, `t2` first tests whether `t1` and `t2` point to the same table reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `t1` and `t2` contain the same number of elements and whether all key~value pairs in the tables are approximately equal (please see **approx** for further details). In this case, the search is linear.

t1 <> t2

This inequality check of two tables `t1`, `t2` first tests whether `t1` and `t2` do not point to the same table reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `t1` and `t2` do not contain the same values, and returns **true** or **false**. In this case, the search is quadratic. See also: **bisequal**.

`t1 ~<> t2`

Approximate inequality check, the negation of the ~= operator.

`c in t`

Checks whether the table `t` contains the value `c` and returns **true** or **false**. The search is linear.

See also: **notin** operator, **binsearch** for binary search.

`c notin t`

Checks whether table `t` does not contain the value `c` and returns **true** or **false**. The search is linear.

See also: **in** operator.

`t1 intersect t2`

Searches all values in `t1` that are also values in `t2` and returns them in a new table. The search is quadratic, so you may use **bintersect** instead if you want to compare large tables since **bintersect** performs a binary search. The key ~ value pairs in the *hash* part of a table are treated as being unique. If `t1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `t2`. If `t1` and/or `t2` are **null**, the operator assumes that **null** represents a structure of zero size. See also: **tables.numintersect**.

`t1 minus t2`

Searches all values in table `t1` that are not values in table `t2` and returns them as a new table. The search is quadratic, so you may use **bminus** instead if you want to compare large tables since **bminus** performs a binary search. The key ~ value pairs in the *hash* part of a table are treated as being unique. If `t1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `t2`. If `t1` and/or `t2` are **null**, the operator assumes that **null** represents a structure of zero size. See also: **tables.numminus**.

`t1 subset t2`

Checks whether all values in table `t1` are included in table `t2` and returns **true** or **false**. The operator also returns **true** if `t1 = t2`. The search is quadratic.

t1 union t2

Concatenates two tables `t1` and `t2` simply by copying all its elements - even if they occur multiple times - to a new table. The key ~ value pairs in the *hash* part of a table are treated as being unique. If `t1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `t2`. If `t1` and/or `t2` are **null**, the operator assumes that **null** represents a structure of zero size. See also: **tables.numunion**.

t1 xsubset t2

Checks whether all values in table `t1` are included in table `t2` and whether `t2` contains at least one further element, so that the result is always **false** if `t1` = `t2`. The search is quadratic.

See also: **bintersect**, **bisequal**, **bminus**, **purge**, **put** in Chapter 8 Basic Functions.

f @ t

The operator maps a function `f` to all the values in table `t` and returns a table as the result. `f` must be a univariate function and return only one value. If `t` has metamethods or user-defined types, the return will also have them.

The operator actually calls function **map**.

Examples:

```
> <: x -> x^2 :> @ [1, 2, 3]:
[1, 4, 9]

> <: x -> x > 0 :> @ [1, 2, 3]:
[true, true, true]
```

If `t` is **null**, the operator returns **null**.

See also: **$**, **$$** and **$$$** operators.

f $ t

Returns all values in table `t` that satisfy a condition determined by function `f`. `f` should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored. The return might include holes.

```
> <: x -> x > 1 :> $ [1, 2, 3]:
[2 ~ 2, 3 ~ 3]
```

If present, the function also copies the metatable and user-defined type of `t` to the new table. The operator actually calls function **select**.

If `t` is **null**, the operator returns **null**.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **subsop**, **unique**, **values**, **zip.**

f $$ t

Checks whether at least one element in table `t` satisfies the condition defined by function `f` and returns **true** or **false**. `f` should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 1 :> $$ [1, 2, 3]:
false
```

The return might include holes. If `t` is **null**, the operator returns **null**.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **subsop**, **unique**, **values**, **zip**.

f $$$ t

Counts the number of elements in table `t` that satisfy the condition defined by function `f`, a univariate function that returns at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 3 :> $$$ [1, 2, 3]:
2
```

10.1.2 tables Library

This library provides generic functions for table manipulation. It provides all its functions inside table `tables`.

Most functions in the table library assume that the table represents an array or a list. For these functions, when we talk about the 'length' of a table we mean the result of the length operator.

tables.allocate (t, key$_1$, value$_1$ [, key$_2$, value$_2$, ···, key$_n$, value$_n$])

Sets the specified keys and values to table `t`, i.e. t`[key`$_k$`] := value`$_k$. Note that if a key is given multiple times, then only the first occurrence of the key in the argument sequence is processed. The function returns nothing.

See also: **tables.include**.

tables.array (t)

Returns the array part of table `t` in a new table, with all key~value pairs preserved.

See also: **tables.hash**, **tables.hashole**, **tables.parts**.

tables.auto ([depth])

Creates Perl-style automagic tables (also called autovivication). An automagic table creates subtables on demand, so that you do not have to set them up manually. The last table in the chain generated will itself not be an auto-table.

Example:

```
> a := tables.auto();
> a.b.c.d := 'a.b and a.b.c are automatically created'
```

This is short for

```
> a := []; create table a.b; create table a.b.c;
> a.b.c.d := 'a.b and a.b.c are automatically created'
```

`depth` (optional) controls how deep to auto-generate tables. The default is 0, meaning there is no limit.

See also: **tables.dimension**, **tables.new**, **tables.newtable**.

tables.borders (t [, option])

By default, returns the smallest and largest assigned integral index - in this order - in the array part of a table `t`.

If any `option` is given, then the function determines the smallest and largest assigned integral index in both the array and hash part of table `t`. Note that this is slower since the entire hash part has to be searched linearly.

If zeros are returned, the array or the array and hash part of the table is empty.

See also: **environ.attrib**, **tables.getsize**, **tables.indices**, **tables.maxn**.

tables.cleanse (t)

Removes all values from the array and hash parts of table t. The function does not slim the table size and also does not conduct a garbage collection, but you may call **tables.resize**. See also: **cleanse**.

tables.col (t, p [, ···] [, s:t])

Extracts the given columns `p` etc. from the two-dimensional table `t`. If only one column shall be extracted, the return is a table with all the column elements; otherwise a table of column tables will be returned. You may restrict extraction to rows `s` to `t` by passing the respective index range `s:t` as an optional last argument.

See also: **columns**.

tables.concat (t [, sep [, i [, j]]])

Given a table array where all elements are strings or numbers, returns the string `t[i]` & `sep` & `t[i+1]` ··· `sep` & `t[j]`. The default value for `sep` is the empty string, the default for `i` is 1, and the default for `j` is size(`t`). If `i` is greater than `j`, returns the empty string.

See also: **join**.

tables.dimension (a:b [, c:d, ···] [, default])
tables.dimension (a:b [, c:d, ···] [, init = default])
tables.dimension (a [, b, ···] [, init = default])

In the first form, creates a table of any dimension with arbitrary index ranges `a:b` etc. with `a, b`, etc. integers, and an optional `default` for all its entries. `default` must not be a pair.

In the second form the initialiser may be given as the option "init = `default`", which allows to also use pairs as a default.

In the third form, a table of any dimension with index ranges `1:a` , `1:b` etc. will be returned.

If the initialiser is a structure, i.e. table, set, sequence or register, then individual copies of the initialiser are created to avoid referencing to the same structure.

Examples:

```
> tables.dimension(1:3, 1:2):
[[], [], []]

> tables.dimension(3, 2, 0):
[[0, 0], [0, 0], [0, 0]]

> tables.dimension(2:4, 1:2, 0):
[2 ~ [0, 0], 3 ~ [0, 0], 4 ~ [0, 0]]
```

See also: **tables.auto**, **tables.include**, **tables.new**, **tables.newtable**, **tables.tableoftables**, **create table/dict** statements.

tables.entries (t [, option])

Returns all entries of table `t` (not its keys) in a new table array. Its second result, a Boolean, indicates whether a value has been found in the hash part of `t`.

When given any second argument, the function returns all the table values that have integral keys - in ascending order of these integral keys.

See also: **member**, **unique**, **tables.hashole**, **tables.indices**, **whereis**.

tables.extend (t, addrows, addcols [, options])

Creates a new 2-dimensional table which is a copy of the input 2D table `t` with `addrows` additional rows and `addcols` additional columns.

You can also optionally initialise new entries by passing the `init = <def>` option where `<def>` may be of any type. Also, the `inplace = true` option allows to work in-place, altering the input table and saving memory.

If `addrows` and `addcols` are both zero, a deep copy of the input table is returned.

```
> A := tables.tableoftables(2, 3, 0, init = 10):
[[10, 10, 10], [10, 10, 10]]

> tables.extend(A, 1, 0):  # add one empty row
[[10, 10, 10], [10, 10, 10], []]

# Add one row and two columns, prefilled with 20:
> tables.extend(A, 1, 2, init = 20, inplace = true):
[[10, 10, 10, 20, 20], [10, 10, 10, 20, 20], [20, 20, 20, 20, 20]]

> A:  # by default, the input table is not modified
[[10, 10, 10], [10, 10, 10]]
```

```
> tables.extend(A, 1, 0, init = 10, inplace = true):
[[10, 10, 10], [10, 10, 10], [10, 10, 10]]

> A:  # input has been modified
[[10, 10, 10], [10, 10, 10], [10, 10, 10]]
```

See also: **sequences.extend**.

tables.getarray (t, k)

Tries to find a value with integral index `k` in the array part of table `t` and returns it, otherwise returns **null**. The function might be useful to explore the interpreter's internal table administration.

See also: **tables.getfield**, **tables.gethash**, **tables.geti**.

tables.getdim (t)

Checks table `t` for subtables of the same size. Only elements in the array part of the subtables are taken into account. Returns both the number of rows and the number of columns found if all subtables have the same size and issues an error otherwise.

See also: **tables.isrectangular**, **tables.issquare**.

tables.getfield (t, k)

Returns `t[k]` - where `k` is a string, i.e. a field name - or **null** if there is no value at `t[k]`. The function triggers a metamethod if needed. Useful to explore the C API function lua_getfield.

tables.gethash (t, k)

Tries to find a value with integral index `k` in the hash part of table `t` and returns it, otherwise returns **null**. The function might be useful to explore the interpreter's internal table administration. (Note that any table value with a non-positive integral key is always stored in the hash part.

A table with holes in the array part, always indexed by positive integral keys, might store some of its values in the hash part, too.)

See also: **tables.getarray**.

tables.geti (t, k)

Returns `t[k]` - where `k` is an integer - or **null** if there is no value at `t[k]`, from table `t`. The function triggers a metamethod if needed. The function also returns the type name of the value. Useful to explore the C API function lua_geti.

See also: **tables.gettable**.

tables.getsize (t [, option])

Returns a guess on the number of elements in a table `t`. If any `option` is given, the function additionally returns a Boolean indicator on whether a table contains an allocated hash part, and a Boolean indicator on whether **null** has been assigned to a table. The latter return is not foolproof, especially if a table value has been deleted with a raw assignment, e.g. t[2] := null;

The function is useful to determine the size of a table much more quickly than the **size** operator does, using a logarithmic instead of linear method, but may return incorrect results if the array part of a table has holes. It also does not count the number of elements in the hash part of a table.

See also: **size**, **tables.getsizes**, **tables.isarray**, **tables.ishash**.

tables.getsizes (t [, option])

If any `option` is given, returns the actual number of elements currently stored in the array and hash part. If no `option` is given, then an estimate of the number of elements in the array part will be returned, plus the number of elements actually assigned in the hash part.

Returns two integers: the first for the array part, the second for the hash part.

See also: **size**, **tables.getsize**, **tables.isarray**, **tables.ishash**.

tables.gettable (t, k)

Returns `t[k]` - where `k` is any value - or **null** if there is no value at `t[k]`, with `t` a table. The function triggers a metamethod if needed. Useful to explore the C API function lua_gettable.

See also: **tables.geti**, **tables.getfield**.

tables.hash (t)

Returns the hash part of table `t` in a new table with all key~value pairs preserved.

See also: **tables.array**, **tables.parts**.

tables.hashole (t)

Checks whether the array part of a table contains at least one **null** value, i.e. a hole, and returns **true** in this case and **false** otherwise. The table may or may not have a hash part, this does not influence the result.

See also: **tables.entries**, **tables.isarray**, **tables.isnullarray**.

tables.include (t, key, value [, ···])

Inserts values into a subtable of table `t`. If `t[key]` already represents a table, `value` is added to the end of its array part. If `t[key]` is unassigned, then it creates a new subtable and inserts `value` into it, which is equivalent to the pseudo code:

```
for i from 3 to nargs do
   if assigned t[key] then
      insert <argument_i> into t[key]
   else
       t[key] := [<argument_i>]
   end
fi
```

The function returns nothing.

Example: Add 10 to the end of the first row of a 2-dimensional table:

```
> A := tables.dimension(1:3, 1:2, 0):
[[0, 0], [0, 0], [0, 0]]

> tables.include(A, 1, 10);

> A:
[[0, 0, 10], [0, 0], [0, 0]]
```

See also: **copyadd**, **bags.include**, **tables.allocate**, **tables.dimension**.

tables.indices (t [, option])

Returns all keys of table `t` in an unsorted new table.

If you pass any optional argument, the function will return the integral indices of a table only, in ascending order. In this case, the second result, a Boolean, indicates whether at least one integral key has been found in the hash part, so you might sort the table if needed. This mode is 40 % faster than the standard mode of the function.

See also: **member**, **tables.borders**, **tables.entries**, **whereis**.

tables.isall (t, type [, option])

Checks whether all elements in a table `t` are of a given `type` and returns **true** or **false**. Eligible types that the function accepts are `'number'`, `'integer'` (numbers that are all integral), `'complex'`, `'string'` and `'boolean'`. Also supported are `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonnegative'` (non-negative numbers) and `'numeric'` (numbers and complex numbers).

If you pass **true** for `option`, then the function checks values with integral keys only, both in the array part and the hash part of `t`. All the other values are ignored.

The function is at least fifteen times faster than checking structures with the **satisfy** function.

Examples:

```
> tables.isall([1, 2, Pi], 'number'):
true
```

which is equal to:

```
> tables.isall([1, 2, Pi], number):
true

> tables.isall([1, 2, 3], 'integer'):
true

> tables.isall([1, 2, 3], integer):
true

> satisfy(<: x -> x :: integer :>, [1, 2, 3]):
true
```

See also: **checktype**, **isall**, **sequences.isall**, **sets.isall**, **registers.isall**.

tables.isarray (t)

Checks whether the given table `t` is a pure array, i.e. only contains one or more elements in the array part of the table but none in the hash part, and returns **true** or **false**. The second Boolean return indicates whether the array has holes.

The function checks for elements that are actually assigned, not for slots that have just been allocated. With empty tables, always returns **false**.

See also: **environ.attrib**, **tables.hashole**, **tables.ishash**, **tables.isnullarray**, **tables.getsizes**.

tables.ishash (t)

Checks whether the given table `t` is a pure dictionary, i.e. only contains one or more elements in the hash part of `t` but none in the array part, and returns **true** or **false**. The function checks for elements that are actually assigned, not for slots that have just been allocated. With empty tables, always returns **false**.

See also: **environ.attrib**, **tables.isarray**, **tables.getsizes**.

tables.isnullarray (t)

Checks whether the given table `t` is a pure array, i.e. only contains one or more elements in the array part of the table, but none in the hash part. It then checks whether at least one of the elements in `t` is the **null** value and returns **true** or **false**.

See also: **tables.hashole**, **tables.isarray**.

tables.isone (t [, epsilon])

Checks whether table `t` has only 1-elements in its array part and returns **true** or **false**. It also returns the index of the first non-one element in `t` as a second result, or 0 if there is none. The non-one element is returned as a third result, too.

The function regards any non-numeric values in `t` as non-one. An empty table is non-one, too. The function checks the array part only.

By default, the check is done against strict one. You can change this by passing a positive `epsilon` value as an optional second argument so that all elements x with $|x - 1| \leq$ `epsilon` will be considered one.

See also: **linalg.viszero**, **sequences.iszero**, **tables.iszero**.

tables.isrectangular (t)

Checks the 2-dimensional table `t` for subtables of the same size. Only elements in the array part of the subtables are taken into account.

Returns **true** or **false** and in case of **true**, also returns the number of rows and the number of columns found. With square tables, also returns **true**, see **tables.issquare**.

tables.issquare (t)

Checks whether the 2-dimensional table `t` has the same number of rows as there are elements in each of its subtables (`columns`). The subtables must be of the same size. Only elements in the array part of the subtables are taken into account.

Returns **true** or **false** and in case of **true**, also returns the number of rows and the number of columns found.

tables.iszero (t [, epsilon])

Checks whether table `t` has only zero elements in its array part and returns **true** or **false**. It also returns the index of the first non-zero element in `t` as a second result, or 0 if there is none. The non-zero element is returned as a third result, too.

The function regards any non-numeric values in `t` as non-zero. An empty table is non-zero, too. The function checks the array part only.

By default, the check is done against strict zero. You can change this by passing a positive `epsilon` value as an optional second argument so that all elements x with |x| <= `epsilon` will be considered zero.

See also: **linalg.viszero**, **sequences.iszero**, **tables.isone**.

tables.maxn (t)

Returns the largest positive numerical index of the given table `t`, or zero if the table has no positive numerical indices. (To do its job, this function does a linear traversal of the whole table.)

See also **tables.borders**, which is faster with arrays.

tables.move (t1, start, stop, newidx [, t2])

Copies elements from table `t1` to table `t2`, performing the equivalent to the following multiple assignment: `t2`[`newidx`],··· = `t1`[`start`], ···, `t1`[`stop`]. The default for `t2` is `t1`, i.e. elements are shifted in the same table. The destination range can overlap with the source range.

Returns the destination table `t2`.

Example: The following statement copies four elements in table a from position 3 up to and including 6 to new table b, starting with index 1:

```
> a := ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'];

> b := tables.move(a, 3, 6, 1, []);

> b:
[c, d, e, f]
```

The next statement copies four elements in a to its beginning:

```
> tables.move(a, 3, 6, 1);

> a:
[c, d, e, f, e, f, g, h]
```

See also: **move**, **purge**, **shift**, **swap**.

tables.new ([bool,] a, b [, k])

tables.new ([bool,] f, a, b [, k [, ···]])

tables.new (n, init = default)

tables.new (na:nh [, option])

In the first form, if no Boolean `bool` is given as the very first argument, the function creates a table array [`a`, `a+k`, ···, `b-k`, b], with `a`, `b`, and `k` (the step size) being numbers. The step size is 1 if `k` - a number - is not given. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced table array of `k` numbers in the interval [`a`, `b`].

In the second form, if no Boolean `bool` is given as the very first argument, the function returns a table array [1~`f`(`a`), 2~`f`(`a+k`), ···, (`(b-a)*1/k+1`)~`f`(`b`)], with `f` a function, `a` and `b` numbers. Thus, the function `f` is applied to all numbers between and including `a` and `b`. If `f` requires two or more arguments, the second, third, etc. argument must be passed after `k`.

If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced table array of `k` numbers in the interval [`a`, `b`] with `f` applied to all its members.

The function uses the Kahan-Babuška summation algorithm to prevent round-off errors in case the step size is non-integral.

In the third form, creates a table array of `n` slots, pre-filled with `default` which may be of any type.

In the fourth form, the function takes a pair with both the default number of slots `na` in the array and the default number of slots `nh` in the hash part and an optional argument. If the latter is given, the function creates a table with the given number of slots and optionally adds a metatable so that **tables** package functions can be called in OOP-style with it.

Examples:

```
> tables.new(<: x, y -> x:x^2 + y :>, 1, 5, 1, 10):
[1:11, 2:14, 3:19, 4:26, 5:35]

> p := [0.1, 0.2, 0.1, 0.3, 1]

> tables.new( <: x -> x:p[x] :>, 1, size p):
[1:0.1, 2:0.2, 3:0.1, 4:0.3, 5:1]

> tables.new(true, -4, 4, 6):
[-4, -2.4, -0.8, 0.8, 2.4, 4]

> tables.new(8, init = 0):
[0, 0, 0, 0, 0, 0, 0, 0]

> t := tables.new(10:2, true);
```

```
> insert 10, 20 into t;

> t@@entries():
[10, 20]      false
```

tables.new also accepts functions that may return **null**. Example:

```
> tables.new(<: x -> if x % 3 = 0 then x else null fi :>, 0, 10):
[1 ~ 0, 4 ~ 3, 7 ~ 6, 10 ~ 9]
```

See also: **foreach**, **map**, **registers.new**, **sequences.new**, **tables.auto**, **tables.dimension**, **tables.newtable**, **tables.tableoftables**.

tables.newtable (a, b [, option])

Returns a table with `a` pre-allocated array slots and `b` pre-allocated hash slots. `a` and `b` should be non-negative integers. If `a` or `b` is negative, zero slots will be pre-allocated with no error being issued.

The function is useful only if you have to pass a table initialiser as a function argument, otherwise it is recommended to use the **create table** statement.

If any `option` is given, the function adds a metatable so that **tables** package functions can be called OOP-style with it.

See also: **tables.auto**, **tables.dimension**, **tables.new**, **tables.tableoftables**, **create table/dict** statements.

tables.numintersect (a, b)

Returns the number of elements in the intersection of `a` and `b`, aka **size**(`a` **intersect** `b`), without the overhead of generating the structure.

tables.numminus (a, b)

Returns the number of elements in the difference of `a \ b`, aka **size**(`a` **minus** `b`), without the overhead of generating the structure.

tables.numunion (a, b)

Returns the number of elements in the union of `a` and `b`, aka **size**(`a` **union** `b`), without the overhead of generating the structure.

tables.pack (···)

Returns a new table with all arguments stored into keys 1, 2, etc. and with a field "n" with the total number of arguments. Note that the resulting table may not be a table array, if some arguments are **null**.

See also: **tables.unpack**.

tables.parts (t)

Returns both the array and the hash part of table `t` in two tables, with all key~value pairs preserved.

See also: **tables.array**, **tables.hash**, **tables.hashole**, **tables.isarray**, **tables.ishash**.

tables.remove (t [, pos])

Removes from table `t` the element at position `pos`, returning the value of the removed element. When `pos` is an integer between 1 and **size** `t`, it shifts down the elements `t`[`pos`+1], `t`[`pos`+2], ···, `t`[**size** `t`] and erases element `t`[**size** `t`]; the index `pos` can also be 0 when **size** `t` is 0, or **size** `t` + 1.

The default value for pos is **size** `t`, so that a call **tables.remove**(`t`) removes the last element of `t`.

See also: **move**, **purge**, **put**.

tables.reshuffle (t [, flag])

Removes all **null** values from the array part of a table and moves the remaining non-**null** values in the array part to close the space. If any second argument is not given, also moves all values in the hash part of table `t` to the end of its array part, thus emptying the hash part. The function works in-place, thus destructively, and returns nothing.

See also: **cleanse**, **sort**, **sorted**, **tables.resize**.

tables.resize (t [, newsize])

Resizes the size of the array part of table `t` to `newsize` allocated slots. `newsize` must be a non-negative integer; if not given the function counts the number of elements in the array part and proceeds.

If `newsize` is less than the number of currently stored values in the array part, surplus values are cut off.

If the array part includes embedded **null**'s, they are removed shifting down the other elements to close the space.

`newsize` may be zero, and in this case the function removes the array part of `t` completely.

The function always leaves the hash part unchanged.

See also: **cleanse**, **pack**, **tables.cleanse**, **tables.reshuffle**.

tables.setfield (t, k, v)

The function sets value `v` to field `k` in table `t`, triggering a metamethod if available and necessary. `k` represents the field name, i.e. a string, and `v` any value. This is equivalent to the assignment statement `t[k] := v`. Useful to explore the behaviour of the underlying C API function lua_setfield.

tables.settable (t, k, v)

The function sets value `v` to field `k` in table `t`, triggering a metamethod if available and necessary. `k` represents the index which may be of any type, and `v` any value. This is equivalent to the assignment statement `t[k] := v`. Useful to explore the behaviour of the underlying C API function lua_settable.

tables.subs (t, old, new)

In table `t`, the function replaces every occurrence of the value `old` with value `new`, in-place, and returns the number of substitutions done. If you are substituting with **null**, you may call **tables.reshuffle** afterwards to remove the holes created in the table, or call **tables.remove** instead of **tables.subs**.

The function is four times faster than **subs** in traditional mode, avoiding most of all checks and conversions that **subs** is performing.

tables.swapcol (A, p, q [, s:t] [, true])

Swaps column `p` in the two-dimensional table `A` with column `q`. `p`, `q` must be positive integers. The result is a new table of tables, by default.

If the very last argument is the Boolean **true**, then the operation is done in-place, modifying `A`. In this mode, the modified table `A` is returned, as well.

Whether in-place or not, you may limit the exchange of the table elements to rows `s` to `t` by passing the respective index range `s:t` as an optional fourth argument.

See also: **tables.swaprow**, **tables.transpose**.

tables.swaprow (A, p, q [, s:t] [, true])

Swaps row `p` in two-dimensional table `A` with row `q`. `p`, `q` must be positive integers. The result is a new table of tables.

Whether in-place or not, you may limit the exchange of the table elements to columns `s` to `t` by passing the respective index range `s:t` as an optional fourth argument.

See also: **tables.swapcol**, **tables.transpose**.

tables.tableoftables (n, narray, nhash [, init = default])

The function creates a 2-dimensional table of `n` subtables, each with `narray` preallocated array slots and `nhash` preallocated hash slots. You can fill the array part of each subtable with a default, of any type, `narray` times, by passing the `init` option. `narray` and `nhash` must be non-negative integers each.

Example:

```
> A := tables.tableoftables(5, 3, 2, init = 0):
[[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

See also: **tables.dimension**, **tables.new**, **tables.newtable**.

tables.transpose (A)

Computes the transpose of a rectangular or square two-dimensional table `A` with row dimension m and column dimension n and returns an n x m 2-dimensional table. The [i,j]-th element of the result is equal to the [j,i]-th element of `A`, that is the function swaps rows with columns. The function does not change the input.

By passing the `sparse`=**true** option, all zeros in `A` will not be part of the result (the default is **false**).

See also: **tables.swapcol**, **tables.swaprow**.

tables.unpack (t [, i [, j]])

Returns the elements from the given table array. This function is equivalent to

return `t`[`i`], `t`[`i`+1], ⋯, `t`[`j`].

By default, `i` is 1 and `j` is **size**(`t`). The function works like **unpack**, but only for tables.

10.2 Sets

Summary of Functions:

Queries

> **$$**, **$$$**, **descend**, **empty**, **filled**, **getorset**, **has**, **in**, **member**, **notin**, **recurse**, **size**, **type**, **typeof**, **sets.isall**, **sets.numintersect**, **sets.numminus**, **sets.numunion**.

Retrieving Values

> **descend**, **unpack**.

Operations

> @, **$**, **cleanse**, **copy**, **copyadd**, **freeze**, **map**, **pack**, **remove**, **select**, **selectremove**, **unfreeze**.

Relational Operators

> =, ==, ~=, <>.

Cantor Operations

> **intersect**, **minus**, **subset**, **union**, **xsubset**.

Miscellaneous

> **cleanse**, **sets.cleanse**, **sets.new**, **sets.newset**, **sets.resize**.

A tip in advance: If you want to use a set as some sort of lookup dictionary for numbers, but have too many, resulting in out-of-memory errors, you might use the **dblhash** (Chapter 10.19) or **numcuckoo** (Chapter 10.18) package instead, or even both. For strings, use the **trie** (Chapter 10.17) and/or the **cuckoo** package (Chapter 9.7) alternatively.

The functions are:

cleanse (s)

Empties set `s` and returns the emptied structure. The memory previously occupied can be reused by the interpreter. See also: **sets.cleanse**.

copy (s)

The operator copies the entire contents of a set `s` into a new set. See Chapter 8 for more information.

descend (f, s, [, ···] [, option])

Returns all elements in set `s` that satisfy a given condition expressed by function `f`. The function can be multivariate and must return either **true** or **false**. The optional second and all further arguments of `f` may be passed as the third, etc. argument.

For more information see the description of **descend** in Chapter 8.

copyadd (s [, ···])

Copies all elements in set `s` and any further optional arguments into a new set and returns it. For performance reasons, substructures are not deep-copied.

For further information, check the description of **copyadd** in Chapter 8.

empty (s)

The operator checks whether a set `s` does not contain any element. The return is **true** or **false**.

See also: **filled**.

filled (s)

The operator checks whether set `s` contains at least one element. The return is **true** or **false**.

See also: **empty**.

freeze (s)

Write-protects set `s`. See Chapter 8 for further information. See also: **unfreeze**.

getmetatable (s)

If `t` does not have a metatable, returns **null**. Otherwise, if the `s`'s metatable has a `'__metatable'` field, returns the associated value. Otherwise, returns the metatable of set `s`.

See also: **setmetatable**.

getorset (s, k_1, ···, k_n, v)

Returns the non-**null** element at index s[k_1, k_2, ..., k_n], where s is a set. If any index position is invalid, the function returns **null**.

If s[k_1, k_2, ..., k_n] = **null**, then the function assigns s[k_1, k_2, ..., k_n] := v and returns v.

See also: **getentry**.

has (s, x)

Checks whether set s contains element x. The function performs a recursive scan so that it can find elements in deeply nested structures.

The function return **true** if x could be found in s, and **false** otherwise.

See also: **descend**, **in**, **member**, **recurse**, **satisfy**.

map (f, s [, ···] [, true])

Maps the function f on all elements of a set s. See **map** in Chapter 8 for more information.

See also: **countitems**, **remove**, **select**, **selectremove**, **subs**, **subsop**, and **zip**.

member (x, s)

Searches x in set s and if successful returns **true** and **null** otherwise.

See also: **has**, **whereis**.

recurse (f, s [, ···][, option])

Checks each element of set s by applying a function f on each of its elements. f can be a multivariate function and must return either **true** or **false**. The optional second and all further arguments of f may be passed as the third, etc. argument.

For more information, see the description of **recurse** in Chapter 8.

See also: **has**, **descend**, **select**.

remove (f, s [, ···] [, true])

Returns all values in set s that do not satisfy a condition determined by function f. See **remove** in Chapter 8 for more information.

See also: **map**, **select**, **selectremove**, **subs**, **subsop**, **zip**.

select (f, s [, ···] [, true])

Returns all values in set `s` that satisfy a condition determined by function `f`. See **select** in Chapter 8 for more information.

See also: **$** and **$$$** operators, **countitems**, **map**, **remove**, **selectremove**, **subs**, **subsop**, **zip**.

selectremove (f, s [, ···])

Returns all values in set `s` that satisfy and do not satisfy a condition determined by function `f`, in two sets. See **selectremove** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subs**, **subsop**, **zip**.

size (s)

Returns the number of items in a set `s`.

See also: **$$$** operator, **countitems**.

type (s)

Returns the type of set `s`, i.e. the string 'set'.

typeof (s)

Returns either the user-defined type of set `s`, or the basic type 'set'.

The following functions have been built into the kernel as binary operators.

Please note that the operators returning a Boolean work in a Cantor way, i.e. `{1, 1} = {1}` → true, `{1, 2} xsubset {1, 1, 2, 2, 3, 3}` → true.

s1 = s2

This equality check of two sets `s1`, `s2` first tests whether `s1` and `s2` point to the same set reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `s1` and `s2` contain the same items, and returns **true** or **false**. In this case, the search is linear.

s1 == s2

With sets, the == operator acts exactly as the = operator.

s1 ~= s2

With sets, the ~= operator compares each element in s1 and s2 for approximate equality. See **approx** for further details. The return is either **true** or **false**.

s1 <> s2

This inequality check of two sets s1, s2 first tests whether s1 and s2 do not point to the same set reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether s1 and s2 do not contain the same items, and returns **true** or **false**. In this case, the search is linear.

c in s

Checks whether the set s contains the item c and returns **true** or **false**. The search is constant.

c notin s

Checks whether the set s does not contain the item c and returns **true** or **false**. The search is constant.

s1 intersect s2

Searches all items in set s1 that are also items in set s2 and returns them in a set. The search is linear. If s1 has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from s2.

If s1 and/or s2 are **null**, the operator assumes that **null** represents a set of zero size.

See also: **sets.numintersect**.

s1 minus s2

Searches all items in set s1 that are not items in set s2 and returns them as a set. The search is linear. If s1 has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from s2.

If s1 and/or s2 are **null**, the operator assumes that **null** represents a set of zero size.

See also: **sets.numminus**.

s1 subset s2

Checks whether all items in set s1 are included in set s2 and returns **true** or **false**. The operator also returns **true** if s1 = s2. The search is linear.

s1 union s2

Concatenates two sets `s1` and `s2` simply by copying all its items to a new set. If `s1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `s2`.

If `s1` and/or `s2` are **null**, the operator assumes that **null** represents a set of zero size.

See also: **sets.numunion**.

s1 xsubset s2

Checks whether all items in set `s1` are included in set `s2` and whether `s2` contains at least one further item, so that the result is always **false** if `s1 = s2`. The search is linear.

f @ s

The operator maps a function `f` to all the values in set `s` and returns a set as the result. `f` must be a univariate function and return only one value. If `s` has metamethods or user-defined types, the return will also have them.

Examples:

```
> <: x -> x^2 :> @ {1, 2, 3}:
{1, 4, 9}

> <: x -> x > 1 :> @ {1, 2, 3}:
{false, true}
```

If `s` is **null**, the operator assumes that **null** represents a set of zero size.

See also: **$**, **$$**, **countitems**, **remove**, **select**, **selectremove**, **subs**, **subsop**, and **zip**.

f $ s

Returns all values in set `s` that satisfy a condition determined by function `f`. `f` should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x > 1 :> $ {1, 2, 3}:
{2, 3}
```

If present, the function also copies the metatable and user-defined type of `s` to the new set.

If `s` is **null**, the operator assumes that **null** represents a set of zero size.

See also: @, **$$**, **map**, **remove**, **selectremove**, **subs**, **subsop**, **zip**.

f $$ s

Checks whether at least one element in set `s` satisfies the condition defined by function `f` and returns **true** or **false**. `f` should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 1 :> $$ {1, 2, 3}:
false
```

If `s` is **null**, the operator assumes that **null** represents a set of zero size.

f $$$ s

Counts the number of elements in set `s` that satisfy the condition defined by function `f`, a univariate function that returns at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 3 :> $$$ {1, 2, 3}:
2
```

sets.cleanse (s)

Removes all elements from set `s` and returns nothing. Contrary to **cleanse**, the function does not resize the set, call **sets.resize** afterwards if needed.

sets.isall (s, type)

Checks whether all elements in a set `s` are of a given `type` and returns **true** or **false**. Eligible types that the function accepts are `'number'`, `'integer'` (numbers that are all integral), `'complex'`, `'string'` and `'boolean'`. Also supported are `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonzeroint'` (non-zero integers), `'nonnegative'` (non-negative numbers) and `'numeric'` (numbers and complex numbers).

The function is at least fifteen times faster than checking structures with the **satisfy** function.

See also: **checktype**, **isall**, **tables.isall**, **sequences.isall**, **registers.isall**.

sets.new ([bool,] a, b [, k])
sets.new ([bool,] f, a, b [, k [, ···]])
sets.new (n, init = default)

In the first form, if no Boolean `bool` is given as the very first argument, the function creates a set **{**`a`, `a+k`, ···, `b-k`, `b`**}**, with `a`, `b`, and `k` (the step size) being numbers. The step size is 1 if `k` - a number - is not given. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced set of `k` numbers in the interval [`a`, `b`].

In the second form, if no Boolean `bool` is given as the very first argument, the function returns a set **{**1~`f`(`a`), 2~`f`(`a`+`k`), ···, (`(b-a)*1/k+1`)~`f`(`b`)**}**, with `f` a function, `a` and `b` numbers. Thus, the function `f` is applied to all numbers between and including `a` and `b`. If `f` requires two or more arguments, the second, third, etc. argument must be passed after `k`. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced set of `k` numbers in the interval [`a`, `b`] with `f` applied to all its members.

The function uses the Kahan-Babuška summation algorithm to prevent round-off errors in case the step size is non-integral.

In the third form, creates a set of `n` pre-allocated slots, pre-filled with `default` which may be of any type.

Examples:

```
> sets.new(<: x, y -> x:x^2 + y :>, 1, 5, 1, 10):
{1:11, 2:14, 3:19, 4:26, 5:35}

> sets.new(true, -4, 4, 6):
{-4, -2.4, -0.8, 0.8, 2.4, 4}

> sets.new(8, init = 0):
{0}
```

sets.new does not accept functions that return **null**.

See also: **map**, **sequences.dimension**, **tables.new**, **sequences.new**, **registers.new**.

`sets.newset (n)`

Returns a set with `n` pre-allocated slots. `n` should be a non-negative integer.

The function is useful only if you have to pass a set initialiser as a function argument, otherwise it is recommended to use the **create set** statement.

See also: **tables.new**, **sequences.new**, **registers.new**.

`sets.numintersect (a, b)`

Returns the number of elements in the intersection of `a` and `b`, aka **size**(`a` **intersect** `b`), without the overhead of generating the structure.

`sets.numminus (a, b)`

Returns the number of elements in the difference of `a \ b`, aka **size**(`a` **minus** `b`), without the overhead of generating the structure.

sets.numunion (a, b)

Returns the number of elements in the union of `a` and `b`, aka **size**(`a` **union** `b`), without the overhead of generating the structure.

sets.resize (s [, newsize [, true]])

Resizes set `s` to store at least `newsize` elements. If the last argument is **true** the number of pre-allocated slots will be adjusted to an optimum of the smallest power of 2 greater than or equal to n.

If only `s` is given, the number of pre-allocated slots will be changed to the smallest power of 2 greater than or equal the current size, usually freeing formerly occupied space.

If `newsize` < **size** `s` or the number of pre-allocated slots would not change, the function does nothing and returns without modifying the set.

The function returns the number of allocated elements and the number of pre-allocated slots.

See also: **math.nextpower**, **size**, **environ.attrib** maxsize and size values.

10.3 Sequences

Summary of Functions:

Queries

> **$$**, **$$$**, **bisequal**, **bottom**, **countitems**, **descend**, **duplicates**, **empty**, **filled**, **getentry**, **getorset**, **has**, **in**, **member**, **notin**, **recurse**, **size**, **top**, **type**, **typeof**, **unique**, **values**, **whereis**, **sequences.getdim**, **sequences.isall**, **sequences.isone**, **sequences.isrectangular**, **sequences.issquare**, **sequences.iszero**, **sequences.numitersect**, **sequences.numminus**, **sequences.numunion**.

Retrieving Values

> **columns**, **descend**, **getentry**, **unique**, **unpack**, **values**, **sequences.col**.

Operations

> **@**, **$**, **append**, **augment**, **bintersect**, **bminus**, **cleanse**, **copy**, **copyadd**, **include**, **join**, **map**, **move**, **mulup**, **pack**, **prepend**, **purge**, **put**, **qsumup**, **remove**, **reverse**, **select**, **selectremove**, **sumup**, **shift**, **sort**, **sorted**, **subs**, **subsop**, **sumup**, **swap**, **zip**, **sequences.concat**, **sequences.extend**, **sequences.move**, **sequences.new**, **sequences.subs**, **sequences.swapcol**, **sequences.swaprow**, **sequences.transpose**.

Relational Operators

> =, ==, ~=, <>.

Cantor Operations

> **intersect**, **minus**, **subset**, **union**, **xsubset**.

Creation

> **sequences.dimension**, **sequences.new**, **sequences.newseq**, **sequences.seqofseqs**.

10.3.1 Operators & Functions

append (x, s)

Adds `x` to the end of sequence `s`, in-place. The function returns the modified structure. For more information, check Chapter 8.

See also: **copyadd**, **include**, **prepend**, **put**, **insert** statement.

augment (s1, s2 [, ···])

Joins two or more sequences `s1`, `s2` etc. together horizontally. All sequences must be of the same size. The function is written in Agena and included in the lib/library.agn file.

See also: **columns**, **zip**.

bintersect (s1, s2 [, option])

Returns all values of sequence `s1` that are also values in sequence `s2`. The function performs a binary search in `s2` for each value in `s1`. If no option is given, `s2` is sorted before starting the search. If you pass an option of any value then `s2` should already have been sorted, for no correct results would be returned otherwise.

With larger sequences, this function is much faster than the **intersect** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bisequal**, **bminus**.

bisequal (s1, s2 [, option])

Determines whether the sequences `s1` and `s2` contain the same values. The function performs a binary search. If no option is given (any value), `s1` and `s2` are sorted before starting the search. If you pass an option of any type then `s1` and `s2` should already have been sorted, for no correct results would be returned otherwise.

With larger sequences, this function is much faster than the = operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bminus**.

bminus (s1, s2 [, option])

Returns all values of sequence `s1` that are not values in sequence `s2`. The function performs a binary search in `s2` for each value in `s1`. If no option is given, `s2` is sorted before starting the search. If you pass the option then `s2` should already have been sorted, for no correct results would be returned otherwise.

With larger sequences, this function is much faster than the **minus** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bisequal**.

bottom (s)

With sequence s, the operator returns the element at index 1. If s is empty, it returns **null**.

See also: **top**.

cleanse (s)

Empties a sequence and returns the emptied structure. The memory previously occupied can be reused by the interpreter. See also: **pack**.

columns (s, p [, ···] [, 'structure'])

Extracts the given columns p (etc.) from the two-dimensional sequence s. The return is either a sequence of structures if the option 'structure' is given, or a multiple return of sequences.

The function is written in Agena and included in the lib/library.agn file.

See also: **ops**, **select**, **unpack**, **values**, **utils.readscv**.

copy (s)

The operator copies the entire contents of a sequence s into a new sequence. See Chapter 8 for more information.

copyadd (s [, ···])

Copies all elements in sequence s and any further optional arguments into a new sequence and returns it. For performance reasons, substructures are not deep-copied.

For further information, check the description of **copyadd** in Chapter 8.

countitems (item, s)

countitems (f, s [, ···])

Counts the number of occurrences of an item in the sequence s. For further information, see Chapter 8.

descend (f, s, [, ···] [, option])

Returns all elements in sequence s that satisfy a given condition expressed by function f. The function can be multivariate and must return either **true** or **false**. The optional second and all further arguments of f may be passed as the third, etc. argument.

For more information see the description of **descend** in Chapter 8.

duplicates (s [, option])

Returns all the values that are stored more than once to the given sequence `s`, and returns them in a new sequence. Each duplicate will be returned only once.

If `option` is not given, the structure is sorted before evaluation since this is needed to determine all duplicates. The original structure is left untouched, however.

The total size of the new register is equal to the number of the elements in the result.

If a value of any type is given for `option`, the function assumes that the sequence has been already sorted. Otherwise it is suggested to use **skycrane.sorted** before the call to **duplicates** if the sequence contains values of different types, to prevent errors.

The function is written in Agena and included in the lib/library.agn file.

empty (s)

The operator checks whether the sequence `s` does not contain any element. The return is **true** or **false**.

See also: **filled**.

filled (s)

The operator checks whether the sequence `s` contains at least one element. The return is **true** or **false**.

See also: **empty**.

freeze (s)

Write-protects sequence `s`. See Chapter 8 for further info. See also: **unfreeze**.

getentry (s [, k_1, ···, k_n])

Returns the entry $s[k_1, \cdots, k_n]$ from the sequence `s` without issuing an error if one of the given indices k_i (second to last argument) does not exist.

getmetatable (s)

If `s` does not have a metatable, returns **null**. Otherwise, if the `s`'s metatable has a `'__metatable'` field, returns the associated value. Otherwise, returns the metatable of sequence `s`.

See also: **setmetatable**.

getorset (s, k_1, ···, k_n, v)

Returns the non-**null** element at index r[k_1, k_2, ..., k_n], where r is a register. If any index position is invalid, the function returns **null**.

If r[k_1, k_2, ..., k_n] = **null**, then the function assigns r[k_1, k_2, ..., k_n] := v and returns v.

See also: **getentry**.

has (s, x)

Checks whether sequence s contains element x. The function performs a recursive scan so that it can find elements in deeply nested structures.

The function return **true** if x could be found in s, and **false** otherwise.

See also: **descend**, **in**, **member**, **recurse**, **satisfy**, **whereis**.

include (s, x [, ···])

Inserts one or more values x, ··· to the end of sequence s, not discarding multiple returns if its last argument is a function call. For more information, see Chapter 8.

See also: **copyadd**, **append**, **prepend**, **put**, **insert** statement.

map (f, s [, ···] [, true])

Maps the function f on all elements of a sequence s. See **map** in Chapter 8 for more information.

See also: **remove**, **select**, **subs**, **subsop**, **zip**.

member (x, s)

Searches x in the sequence s and if successful returns **true** plus the index of the first hit. Otherwise returns **false** and **null**. The function is much faster than **whereis** if you need the index of the first hit only. Note that with respect to **whereis**, the parameters are in reverse order.

See also: **has**, **whereis**.

move (s1, start, stop, newidx [, s2])

Copies elements from sequence s1 to sequence s2, performing the equivalent to the following multiple assignment: s2[newidx],··· = s1[start], ···, s1[stop].

The default for `s2` is `s1`, i.e. elements are shifted in the same sequence. The destination range can overlap with the source range.

The function returns destination sequence `s2`, which might be simply `s1`.

See also: **purge**, **put**.

mulup (s)

Multiplies all numeric values in sequence `s`. See **mulup** in Chapter 8 for more information.

See also: **sumup**.

pack (s [, newsize])

When called with no second argument, reduces the number of allocated slots in sequence `s` to the number of slots actually assigned a value, also freeing memory if possible.

If the optional argument `newsize`, a non-negative integer, is given, then the number of pre-allocated slots is reduced to the number of actually assigned values, purging all surplus elements that may exist and giving back memory to the interpreter.

The function works in-place and returns nothing. It automatically conducts a garbage collection before returning.

See also: **cleanse**.

prepend (x, s)

Prepends `x` to the beginning of sequence `s`, in-place. The function returns the modified structure.

See also: **append**, **copyadd**, **include**, **put**, **insert** statement.

purge (s [, pos])

purge (s, a, b)

In the first form, the function removes from sequence `s` the element at position `pos`, shifting down other elements to close the space, if necessary. Returns the value of the removed element, or nothing if `pos` is invalid.

The default value for `pos` is n, where n is the length of the sequence, so that a call `purge(s)` removes the last element of `s`.

In the second form, it removes all elements starting from index `a` to index `b` (inclusive), moving excess elements down to close the space; the function automatically performs a garbage collection after shifting. In the 2nd form, nothing will be returned.

See also: **move**, **put**.

`put (s, [pos,] value)`

Inserts element `value` at position `pos` in sequence `s`, shifting up other elements to open space, if necessary. The default value for `pos` is n+1, where n is the current sequence size, so that a call `put(s, value)` inserts `value` at the end of `s`.

Use the **insert** *element* **into** *structure* statement if you want to add an element at the current end of a sequence, for it is much faster.

The function returns the modified structure.

See also: **move**, **prepend**, **purge**.

`qsumup (s)`

Raises all numeric values in sequence `s` to the power of 2 and sums up these powers. See **qsumup** in Chapter 8 for more information. See also: **sumup**.

`recurse (f, s [, ···][, option])`

Checks each element of sequence `s` by applying a function `f` on each of its elements. `f` can be a multivariate function and must return either **true** or **false**. The optional second and all further arguments of `f` may be passed as the third, etc. argument. For more information, see the description of **recurse** in Chapter 8.

`remove (f, s [, ···] [, true])`

Returns all values in sequence `s` that do not satisfy a condition determined by function `f`. See **remove** in Chapter 8 for more information.

See also: **map**, **select**, **subs**, **subsop**, **zip**.

`reverse (s)`

Reverses the order of all elements in sequence `s` in-place. The function returns the modified structure.

See also: **strings.reverse**, **stack.reversed**.

select (f, s [, ···] [, true])

Returns all values in sequence `s` that satisfy a condition determined by function `f`. See **select** in Chapter 8 for more information.

See also: **$** and **$$$** operators, **countitems**, **map**, **remove**, **subs**, **subsop**, **zip**.

selectremove (f, s [, ···])

Returns all values in sequence `s` that satisfy and do not satisfy a condition determined by function `f`, in two sequences. See **selectremove** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subs**, **subsop**, **zip**.

setmetatable (s, metatable)

Sets the metatable for the given sequence `s`. (You cannot change the metatable of other types from Agena, only from C.) If `metatable` is **null**, removes the metatable of the given sequence. If the original metatable has a '__metatable' field, raises an error. The function cannot assign metatables to C library functions.

This function returns `s`.

See also: **getmetatable**.

shift (s, a, b)

Moves an element in sequence `s` from position `old` to `new`, with `old`, `new` integers, shifting all the other elements accordingly - which might also cause a rotation. The function returns nothing.

See also: **move**, **purge**, **put**, **rotate** statement, **swap**.

size (s)

Returns the number of items in a sequence `s`.

See also: **$$$** operator, **countitems**.

sort (s [, comp] [, 'number'])

Sorts sequence `s` in a given order, and in-place. See **sort** in Chapter 8 for more information.

See also: **sorted**, **skycrane.sorted**, **stats.issorted**, **stats.sorted**.

sorted (s [, comp] [, 'number'])

Sorts sequence elements in s in a given order, but - unlike sort - not in-place, and non-destructively. See **sorted** in Chapter 8 for more information.

See also: **sort**, **skycrane.sorted**, **stats.issorted**, **stats.sorted**.

subs (x:v [, ···], s [, options])

Substitutes all occurrences of the value x in sequence s with the value v. See **subs** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subsop**, **zip**.

subsop (i:v [, ···], s [, options])

Substitutes the value at s[i] with value v, where v can also be **null**, deleting the element in this case. See **subsop** in Chapter 8 for more information.

sumup (s)

Sums up all numeric values in sequence s. See **sumup** in Chapter 8 for more information. See also: **qsumup**.

swap (s, a, b)

In sequence s, swaps the entries at index positions a and b, with a, b integers. The function returns nothing.

See also: **move**, **purge**, **shift**.

top (s)

With sequence s, the operator returns the element with the largest index. If s is empty, it returns **null**.

See also: **bottom**.

type (s)

Returns the type of sequence s, i.e. the string 'sequence'.

typeof (s)

Returns either the user-defined type of sequence s, or the basic type 'sequence'.

unique (s [, true])
unique (s [, options])

With a sequence `s`, the function removes multiple occurrences of the same item, if present in `s`. See **unique** in Chapter 8 for more information.

values (s, i_1 [, i_2, ···])

Returns the elements from the given sequence `s` in a new sequence. This function is equivalent to

```
return seq( s[i1], s[i2], ··· )
```

See also: **ops**, **select**, **unpack**.

whereis (s, x)

Returns the indices of a given value `x` in sequence `s` as a new sequence, respectively, dependent on the type of `s`. If `x` is not in `s`, returns an empty sequence.

See also: **has**, **member**.

zip (f, s1, s2)

This function zips together two sequences `s1`, `s2` by applying the function `f` to each of its respective elements. See Chapter 8 for more information.

See also: **augment**, **columns**, **map**, **remove**, **select**, **subs**.

Following are the binary operators.

Please note that the operators returning a Boolean work in a Cantor way, i.e. `seq(1, 1) = seq(1)` → true, `seq(1, 2) xsubset seq(1, 1, 2, 2, 3, 3)` → true.

s1 ≟ s2

This equality check of two sequences `s1`, `s2` first tests whether `s1` and `s2` point to the same sequence reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `s1` and `s2` contain the same values without regard to their keys, and returns **true** or **false**. In this case, the search is quadratic.

s1 == s2

This strict equality check of two sequences `s1`, `s2` first tests whether `s1` and `s2` point to the same sequence reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `s1` and `s2` contain the same number of elements and whether all entries in the sequences are the same and are in the same order, and returns **true** or **false**. In this case, the search is linear.

s1 ~= s2

This approximate equality check of two sequences `s1`, `s2` first tests whether `s1` and `s2` point to the same sequence reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `s1` and `s2` contain the same number of elements and whether all entries in the sequences are approximately equal and are in the same order, and returns **true** or **false**. In this case, the search is linear. See **approx** for further information on the approximation check.

s1 <> s2

This inequality check of two sequences `s1`, `s2` first tests whether `s1` and `s2` do not point to the same sequence reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `s1` and `s2` do not contain the same values, and returns **true** or **false**. In this case, the search is quadratic.

c in s

Checks whether the sequence `s` contains the value `c` and returns **true** or **false**. The search is linear. See also **binsearch** for binary search. See also: **notin** operator.

c notin s

Checks whether the sequence `s` does not contain the value `c` and returns **true** or **false**. The search is linear. See also: **in** operator.

s1 intersect s2

Searches all values in sequence `s1` that are also values in sequence `s2` and returns them in a sequence. The search is quadratic. If `s1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `s2`. If `s1` and/or `s2` are **null**, the operator assumes that **null** represents a sequence of zero size. See also **sequences.numintersect**.

s1 minus s2

Searches all values in sequence `s1` that are not values in sequence `s2` and returns them as a sequence. The search is quadratic. If `s1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `s2`. If `s1` and/or `s2` are **null**, the operator assumes that **null** represents a sequence of zero size. See also **sequences.numminus**.

s1 subset s2

Checks whether all values in sequence `s1` are included in sequence `s2` and returns **true** or **false**. The operator also returns **true** if `s1 = s2`. The search is quadratic.

s1 union s2

Concatenates two sequences `s1` and `s2` simply by copying all its elements - even if they occur multiple times - to a new sequence. If `s1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `s2`. If `s1` and/or `s2` are **null**, the operator assumes that **null** represents a sequence of zero size. See also **sequences.numunion**.

s1 xsubset s2

Checks whether all values in sequence `s1` are included in sequence `s2` and whether `s2` contains at least one further element, so that the result is always **false** if `s1 = s2`. The search is quadratic.

f @ s

The operator maps a function `f` to all the values in sequence `s` and returns a sequence as the result. `f` must be a univariate function and return only one value. If `s` has metamethods or user-defined types, the return will also have them.

The operator actually calls function **map**.

Examples:

```
> <: x -> x^2 :> @ seq(1, 2, 3):
seq(1, 4, 9)

> <: x -> x > 0 :> @ seq(1, 2, 3):
seq(true, true, true)
```

If `s` is **null**, the operator assumes that **null** represents a sequence of zero size.

See also: **$** and **$$** operators.

f $ s

Returns all values in sequence `s` that satisfy a condition determined by function `f`. `f` should be a univariate function and return at least one value. In the multivariate case, all results but the first are ignored.

```
> <: x -> x > 1 :> $ seq(1, 2, 3):
seq(2, 3)
```

If present, the function also copies the metatable and user-defined type of `obj` to the new sequence.

The operator actually calls function **select**.

If `s` is **null**, the operator assumes that **null** represents a sequence of zero size.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **unique**, **values**, **zip.**

f $$ s

Checks whether at least one element in sequence `s` satisfies the condition defined by function `f` and returns **true** or **false**. `f` should be a univariate function and return at least one value. In the multivariate case, all results but the first are ignored.

```
> <: x -> x < 1 :> $$ seq(1, 2, 3):
false
```

If `s` is **null**, the operator assumes that **null** represents a sequence of zero size.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **unique**, **values**, **zip**.

f $$$ s

Counts the number of elements in sequence `s` that satisfy the condition defined by function `f`, a univariate function that returns at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 3 :> $$$ seq(1, 2, 3):
2
```

10.3.2 sequences Library

This library provides generic functions for sequence manipulation. It provides all its functions inside table `sequences`.

sequences.col (s, p [, ···] [, u:v])

Extracts the given columns `p` etc. from the two-dimensional sequence `s`. If only one column shall be extracted, the return is a sequence with all the column elements; otherwise a sequence of column sequences will be returned. You may restrict extraction to rows `u` to `v` by passing the respective index range `u:v` as an optional last argument.

See also: **columns**.

sequences.concat (s [, sep [, i [, j]]])

Given a sequence where all elements are strings or numbers, returns the string `s[i]` & `sep` & `s[i+1]` ··· `sep` & `s[j]`. The default value for `sep` is the empty string, the default for `i` is 1, and the default for `j` is size(`s`). If `i` is greater than `j`, returns the empty string.

See also: **join**.

sequences.dimension (a:b [, c:d, ···] [, default])

sequences.dimension (a:b [, c:d, ···] [, init = default])

sequences.dimension (a [, b, ···] [, init = default])

In the first form, creates a sequence of any dimension with index ranges `a:b` etc. with `a`, `b`, etc. integers, and an optional `default` for all its entries. `default` must not be a pair. The left-hand side values `a`, `c`, ... of the dimensions must always be 1.

In the second form the initialiser may be given as the option "init = `default`", which allows to also use pairs as a default.

In the third form, a sequence of any dimension with index ranges `1:a` , `1:b` etc. will be returned.

If the initialiser is a structure, i.e. table, set, pair, sequence or register, then individual copies of the initialiser are created to avoid referencing to the same structure.

See also: **sequences.newseq**, **create sequence** statements.

sequences.extend (s, addrows, addcols [, options])

Creates a new 2-dimension sequence which is a copy of the input 2D sequence `s` with `addrows` additional rows and `addcols` additional columns.

You can also optionally initialise new entries by passing the `init = <def>` option where `<def>` may be of any type. Also, the `inplace = true` option allows to work in-place, altering the input sequence and saving memory.

If `addrows` and `addcols` are both zero, a deep copy of the input sequence will be returned.

For examples, see **tables.extend** which works the same, but on tables instead.

sequences.getdim (s)

Checks sequence `s` for sub-sequences of the same size. Returns both the number of rows and the number of columns found if all sub-sequences have the same size and issues an error otherwise.

See also: **sequences.isrectangular**, **sequences.issquare**.

sequences.isall (s, type)

Checks whether all elements in sequence `s` are of a given `type` and returns **true** or **false**. Eligible types that the function accepts are `'number'`, `'integer'` (numbers that are all integral), `'complex'`, `'string'` and `'boolean'`. Also supported are `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonzeroint'` (non-zero integers), `'nonnegative'` (non-negative numbers) and `'numeric'` (numbers and complex numbers).
The function is at least fifteen times faster than checking structures with the **satisfy** function.

See also: **checktype**, **isall**, **sets.isall**, **tables.isall**, **registers.isall**.

sequences.isone (s [, epsilon])

Checks whether sequence `s` consists of ones only and returns **true** or **false**.

By default, the check is done against strict one. You can change this by passing a positive `epsilon` value as an optional second argument so that all elements x with $|x - 1| \leq$ `epsilon` will be considered one. See **sequences.iszero** for further information.

sequences.isrectangular (s)

Checks the 2-dimensional sequence `s` for sub-sequences of the same size.

Returns **true** or **false** and in case of **true**, also returns the number of rows and the number of columns found. With square sequences, also returns **true**, see **sequences.issquare**.

sequences.issquare (s)

Checks whether the 2-dimensional sequence `s` has the same number of rows as there are elements in each of its sub-sequences (`columns`). The sub-sequences must be of the same size.

Returns **true** or **false** and in case of **true**, also returns the number of rows and the number of columns found.

sequences.iszero (s [, epsilon])

Checks whether sequence `s` consists of zeros only and returns **true** or **false**. It returns the index of the first non-zero element in `s` as a second result, or 0 if there is none. The non-zero element is returned as a third result, too.

The function considers any non-numeric values in `s` as non-zero. An empty sequence is non-zero, too.

By default, the check is done against strict zero. You can change this by passing a positive `epsilon` value as an optional second argument so that all elements x with |x| <= `epsilon` will be considered zero.

See also: **sequences.isone**.

sequences.move (s1, start, stop, newidx [, s2])

Copies elements from sequence `s1` to sequence `s2`, performing the equivalent to the following multiple assignment: `s2`[`newidx`],⋯ = `s1`[`start`], ⋯, `s1`[`stop`]. The default for `s2` is `s1`, i.e. elements are shifted in the same sequence. The destination range can overlap with the source range. Specifically, for `newidx` we have 0 < `newidx` ≤ **size**(`s2`) + 1, all other targets will be rejected.

Returns the destination sequence `s2`.

Example: The following statement copies four elements in sequence a from position 3 up to and including 6 to new sequence b, starting with index 1:

```
> a := seq('a', 'b', 'c', 'd', 'e', 'f', 'g', 'h');

> b := sequences.move(a, 3, 6, 1, seq());

> b:
seq(c, d, e, f)
```

See also: **move**, **shift**.

sequences.new ([bool,] a, b [, k])
sequences.new ([bool,] f, a, b [, k [, ···]])
sequences.new (n, init = default)

In the first form, if no Boolean `bool` is given as the very first argument, the function creates a sequence **seq(**`a`, `a`+`k`, ···, `b`-`k`, `b`**)**, with `a`, `b`, and `k` (the step size) being numbers. The step size is 1 if `k` - a number - is not given. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced sequence of `k` numbers in the interval [`a`, `b`].

In the second form, if no Boolean `bool` is given as the very first argument, the function returns a sequence **seq(**`1`~`f(a)`, `2`~`f(a+k)`, ···, `((b-a)*1/k+1)`~`f(b)`**)**, with `f` a function, `a` and `b` numbers. Thus, the function `f` is applied to all numbers between and including `a` and `b`. If `f` requires two or more arguments, the second, third, etc. argument must be passed after `k`. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced sequence of `k` numbers in the interval [`a`, `b`] with `f` applied to all its members.

The function uses the Kahan-Babuška summation algorithm to prevent round-off errors in case the step size is non-integral.

In the third form, creates a sequence of `n` slots, pre-filled with `default` which may be of any type.

Examples:

```
> sequences.new(<: x, y -> x:x^2 + y :>, 1, 5, 1, 10):
seq(1:11, 2:14, 3:19, 4:26, 5:35)

> p := seq(0.1, 0.2, 0.1, 0.3, 1)

> sequences.new( <: x -> x:p[x] :>, 1, size p):
seq(1:0.1, 2:0.2, 3:0.1, 4:0.3, 5:1)

> sequences.new(true, -4, 4, 6):
seq(-4, -2.4, -0.8, 0.8, 2.4, 4)

> sequences.new(8, init = 0):
seq(0, 0, 0, 0, 0, 0, 0, 0)
```

sequences.new also accepts functions that may return **null**. In this case, an element is not added to the resulting structure. Example:

```
> sequences.new(<: x -> if x % 3 = 0 then x else null fi :>, 0, 10):
seq(0, 3, 6, 9)
```

See also: **foreach**, **map**, **sequences.dimension**, **tables.new**, **sets.new**, **registers.new**.

sequences.newseq (n)

Returns a sequence with `n` pre-allocated slots. `n` should be a non-negative integer.

The function is useful only if you have to pass a sequence initialiser as a function argument, otherwise it is recommended to use the **create sequence** statement.

The function is written in Agena and included in the lib/library.agn file.

See also: **sequences.dimension**.

sequences.numintersect (a, b)

Returns the number of elements in the intersection of `a` and `b`, aka **size**(`a` **intersect** `b`), without the overhead of generating the structure.

sequences.numminus (a, b)

Returns the number of elements in the difference of `a \ b`, aka **size**(`a` **minus** `b`), without the overhead of generating the structure.

sequences.numunion (a, b)

Returns the number of elements in the union of `a` and `b`, aka **size**(`a` **union** `b`), without the overhead of generating the structure.

sequences.resize (s [, newsize [, true]])

Resizes sequence `s` to the given number of pre-allocated slots. If you actually shrink a sequence, then it discards any surplus elements.

The function returns the number of allocated elements and the number of pre-allocated slots, which may be vacant.

When you set `newsize` to 0 then all elements are deleted from `s`.

If the optional third argument is **true** and `newsize` is non-zero, then the function aligns by word boundaries, the next multiple of four.

If you pass just `s` without any further arguments, the function automatically extends memory to the optimum number of slots without dropping any values.

Note that Agena automatically enlarges and shrinks a sequence if necessary when adding new or purging existing values, see **environ.kernel/seqautoshrink**.

See also: **math.nextpower**, **pack**, **size**, **environ.attrib** maxsize and size values, **utils.newsize**.

sequences.seqofseqs (n, slots, [, init = default])

The function creates a 2-dimensional sequence of `n` sub-sequences, each with `slots` pre-allocated slots. You can fill each of the sub-sequences with a default, of any type, `slots` times, by passing the `init` option. `slots` must be a non-negative integer.

Example:

```
> A := sequences.seqofseqs(5, 3, init = 0):
seq(seq(0, 0, 0), seq(0, 0, 0), seq(0, 0, 0), seq(0, 0, 0), seq(0, 0, 0))
```

See also: **sequences.dimension**, **sequences.new**, **sequences.newseq**.

sequences.subs (s, old, new)

In sequence `s`, the function replaces every occurrence of the value `old` with value `new`, in-place, and returns the number of substitutions done. Note that if `new` is **null**, the function automatically closes the space by shifting down the other elements as sequences cannot store **null**.

The function is four three faster than **subs** in traditional mode, avoiding most of all checks and conversions that **subs** is performing.

sequences.swapcol (A, p, q [, s:t] [, true])

Swaps column `p` in the two-dimensional sequence `A` with column `q`. `p`, `q` must be positive integers. The result is a new sequence of sequence s, by default.

If the very last argument is the Boolean **true**, then the operation is done in-place, modifying `A`. In this mode, the modified sequence `A` is returned, as well.

Whether in-place or not, you may limit the exchange of the sequence elements to columns `s` to `t` by passing the respective index range `s:t` as an optional fourth argument.

See also: **sequences.swaprow**, **sequences.transpose**.

sequences.swaprow (A, p, q [, s:t] [, true])

Swaps row `p` in two-dimensional sequence `A` with row `q`. `p`, `q` must be positive integers. The result is a new sequence of sequence s.

Whether in-place or not, you may limit the exchange of the sequence elements to columns `s` to `t` by passing the respective index range `s:t` as an optional fourth argument.

See also: **sequences.swapcol**, **sequences.transpose**.

sequences.transpose (A)

Computes the transpose of a rectangular or square two-dimensional sequence `A` with row dimension m and column dimension n and returns an n x m 2-dimensional sequence. The [i,j]-th element of the result is equal to the [j,i]-th element of `A`, that is the function swaps rows with columns. The function does not change the input.

See also: **sequences.swapcol**, **sequences.swaprow**.

10.4 Registers

Summary of Functions:

Queries

> **$$**, **$$$**, **bisequal**, **bottom**, **columns**, **countitems**, **descend**, **duplicates**, **empty**, **filled**, **getentry**, **has**, **in**, **member**, **recurse**, **size**, **type**, **typeof**, **whereis**, **registers.getdim**, **registers.isall**, **registers.isone**, **registers.isrectangular**, **registers.issquare**, **registers.iszero**, **registers.numintersect**, **registers.numminus**, **registers.numunion**.

Retrieving Values

> **columns**, **descend**, **getentry**, **unique**, **unpack**, **values**, **registers.col**.

Operations

> **@**, **$**, **append**, **augment**, **bintersect**, **bminus**, **cleanse**, **copy**, **copyadd**, **freeze**, **getorset**, **include**, **join**, **map**, **move**, **mulup**, **pack**, **prepend**, **purge**, **qsumup**, **remove**, **reverse**, **select**, **selectremove**, **shift**, **sort**, **sorted**, **subs**, **subsop**, **sumup**, **swap**, **top**, **unique**, **zip**, **registers.dimension**, **registers.move**, **registers.new**, **registers.newreg**, **registers.regofregs**, **registers.resize**, **registers.subs**.

Relational Operators

> =, ==, ~=, <>.

Cantor Operations

> **intersect**, **minus**, **subset**, **union**, **xsubset**.

With the exception of **getentry**, **map** and **zip**, the following functions have been built into the kernel as unary operators:

10.4.1 Operators & Functions

`append (x, r)`

Adds `x` to the end of register `r`, in-place. The function returns the modified structure. For more information, check Chapter 8.

See also: **copyadd**, **include**, **prepend**, **put**, **insert** statement.

augment (r1, r2 [, ···])

Joins two or more registers `r1`, `r2` etc. together horizontally. All registers must be of the same size. The function is written in Agena and included in the lib/library.agn file.

See also: **columns**, **zip**.

bintersect (r1, r2 [, option])

Returns all values of register `r1` that are also values in register `r2`. The function performs a binary search in `r2` for each value in `r1`. If no option is given, `r2` is sorted before starting the search. If you pass an option of any value then `r2` should already have been sorted, for no correct results would be returned otherwise.

With larger registers, this function is much faster than the **intersect** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bisequal**, **bminus**.

bisequal (r1, r2 [, option])

Determines whether the registers `r1` and `r2` contain the same values. The function performs a binary search. If no option is given (any value), `r1` and `r2` are sorted before starting the search. If you pass an option of any type then `r1` and `r2` should already have been sorted, for no correct results would be returned otherwise.

With larger registers, this function is much faster than the = operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bminus**.

bminus (r1, r2 [, option])

Returns all values of register `r1` that are not values in register `r2`. The function performs a binary search in `r2` for each value in `r1`. If no option is given, `r2` is sorted before starting the search. If you pass the option then `r2` should already have been sorted, for no correct results would be returned otherwise.

With larger registers, this function is much faster than the **minus** operator.

The function is written in Agena and included in the lib/library.agn file.

See also: **bintersect**, **bisequal**.

bottom (r)

With register r, the operator returns the element at index 1. If r is empty, it returns **null**.

See also: **top**.

cleanse (r)

Empties register r by setting all its places to **null** and returns the modified register.

See also: **pack**.

columns (r, p [, ···] [, 'structure'])

Extracts the given columns p (etc.) from the two-dimensional register r. The return is either a register of structures if the option 'structure' is given, or a multiple return of registers.

The function is written in Agena and included in the lib/library.agn file.

See also: **ops**, **select**, **unpack**, **values**, **utils.readscv**.

copy (r)

The operator deep-copies the entire contents of a register r into a new register. See Chapter 8 for more information.

copyadd (r [, ···])

Copies all elements in register r and any further optional arguments into a new register and returns it. For performance reasons, substructures are not deep-copied.

For further information, check the description of **copyadd** in Chapter 8.

countitems (item, r)

countitems (f, r [, ···])

Counts the number of occurrences of an item in the register r. For further information, see Chapter 8.

descend (f, r, [, ···] [, option])

Returns all elements in register r that satisfy a given condition expressed by function f. The function can be multivariate and must return either **true** or **false**. The optional second and all further arguments of f may be passed as the third, etc. argument.

For more information see the description of **descend** in Chapter 8.

See also: **has**, **recurse**, **select**.

duplicates (r [, option])

Returns all the values that are stored more than once to the given register `r`, and returns them in a new register. Each duplicate will be returned only once.

If `option` is not given, the structure is sorted before evaluation since this is needed to determine all duplicates. The original structure is left untouched, however.

The total size of the new register is equal to the number of the elements in the result.

If a value of any type is given for `option`, the function assumes that the register has been already sorted. Otherwise it is suggested to use **skycrane.sorted** before the call to **duplicates** if the register contains values of different types, to prevent errors.

The function is written in Agena and included in the lib/library.agn file.

empty (r)

The operator checks whether the register `r` does not contain any element. The return is **true** or **false**.

See also: **filled**.

filled (r)

The operator checks whether the register `r` contains at least one element. The return is **true** or **false**.

See also: **empty**.

freeze (r)

Write-protects register `r`. See Chapter 8 for further info. See also: **unfreeze**.

getentry (r [, k_1, ···, k_n])

Returns the entry `r[`k_1`, ···, `k_n`]` from the register `r` without issuing an error if one of the given indices k_i (second to last argument) does not exist.

getmetatable (r)

If `r` does not have a metatable, returns **null**. Otherwise, if the `r`'s metatable has a `'__metatable'` field, returns the associated value. Otherwise, returns the metatable of register `r`.

See also: **setmetatable**.

getorset (r, k_1, ···, k_n, v)

Returns the non-**null** element at index r[k_1, k_2, ..., k_n], where r is a register. If any index position is invalid, the function returns **null**.

If r[k_1, k_2, ..., k_n] = **null**, then the function assigns r[k_1, k_2, ..., k_n] := v and returns v.

See also: **getentry**.

has (r, x)

Checks whether register r contains element x. The function performs a recursive scan so that it can find elements in deeply nested structures.

The function return **true** if x could be found in s, and **false** otherwise.

See also: **descend**, **in**, **member**, **recurse**, **satisfy**, **whereis**.

include (r, x [, ···])

Inserts one or more values x, ··· to the end of register r, not discarding multiple returns if its last argument is a function call. For more information, see Chapter 8.

See also: **copyadd**, **append**, **prepend**, **put**, **insert** statement.

member (x, r)

Searches x in the register r and if successful returns **true** plus the index of the first hit. Otherwise returns **false** and **null**. The function is much faster than **whereis** if you need the index of the first hit only. Note that with respect to **whereis**, the parameters are in reverse order.

See also: **has**, **whereis**.

map (f, r [, ···])

Maps the function f on all elements of a register r. See **map** in Chapter 8 for more information.

See also: @, **has**, **remove**, **select**, **subs**, **zip**.

move (r1, start, stop, newidx [, r2])

Copies elements from register r1 to register r2, performing the equivalent to the following multiple assignment: r2[newidx],··· = r1[start], ···, r1[stop]. The default for r2 is r1, i.e. elements are shifted in the same register. The destination range can overlap with the source range.

The function returns destination register `r2`, which might be simply `r1`.

See also: **purge**, **put**, **swap**, **registers.move**.

`mulup (r)`

Multiplies all numeric values in register `r`. See **mulup** in Chapter 8 for more information.

See also: **sumup**.

`pack (r [, newsize])`

When called with no second argument, reduces the number of allocated slots in register `r` to the number of slots actually assigned a value, also freeing memory if possible.

If the optional argument `newsize`, a non-negative integer, is given, then the number of pre-allocated slots is reduced to the number of actually assigned values, purging all surplus elements that may exist and giving back memory to the interpreter. With `newsize` given, the function also removes all trailing **null**s.

The function works in-place and returns nothing. It automatically conducts a garbage collection before returning.

See also: **cleanse**.

`prepend (x, r)`

Prepends `x` to the beginning of register `r`, in-place. The function returns the modified structure.

See also: **append**, **copyadd**, **include**, **put**, **insert** statement.

`purge (r [, pos])`

`purge (r, a, b)`

In the first form, the function removes from register `r` the element at position `pos`, shifting down other elements to close the space, if necessary. Returns the value of the removed element, or nothing if `pos` is invalid. The default value for `pos` is n, where n is the length of the register, so that a call `purge(r)` removes the last element of `r`.

In the second form, removes all elements starting from index `a` to index `b` (inclusive), moving excess elements down to close the space; the function automatically performs a garbage collection after shifting. In the 2nd form, nothing will be returned.

Note that the function also reduces the top pointer of `r` by the number of elements removed.

See also: **move**, **put**, **shift**, **swap**.

`put (r, [pos,] value)`

Inserts element `value` at position `pos` in register `r`, shifting up other elements to open space, if necessary. The default value for `pos` is n+1, where n is the current register size, so that a call `put(r, value)` inserts `value` at the end of `r`.

Use the **insert** *element* **into** *structure* statement if you want to add an element at the current end of a register, for it is much faster.

The function returns the modified structure.

See also: **move**, **prepend**, **purge**.

`qsumup (r)`

Raises all numeric values in register `r` to the power of 2 and sums up these powers. See **qsumup** in Chapter 8 for more information.

See also: **sumup**.

`recurse (f, r [, ···][, option])`

Checks each element of register `r` by applying a function `f` on each of its elements. `f` can be a multivariate function and must return either **true** or **false**. The optional second and all further arguments of `f` may be passed as the third, etc. argument. For more information, see the description of **recurse** in Chapter 8.

See also: **descend**, **has**, **select**.

`remove (f, r [, ···] [, true])`

Returns all values in register `r` that do not satisfy a condition determined by function `f`. The total size of the new register is equal to the number of the elements in the result. See **remove** in Chapter 8 for more information.

See also: **map**, **select**, **subs**, **zip**.

`reverse (r)`

Reverses the order of all elements in register `r` in-place. The function returns the modified structure.

See also: **strings.reverse**, **stack.reversed**.

select (f, r [, ···] [, true])

Returns all values in register `r` that satisfy a condition determined by function `f`. The total size of the new register is equal to the number of the elements in the result. See **select** in Chapter 8 for more information.

See also: **$** and **$$$** operators, **countitems**, **map**, **remove**, **subs**, **zip**.

selectremove (f, r [, ···])

Returns all values in register `r` that satisfy and do not satisfy a condition determined by function `f`, in two new registers. The total size of the new registers is equal to the number of the elements in the respective results. See **selectremove** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subs**, **zip**.

setmetatable (r, metatable)

Sets the metatable for the given register `r`. (You cannot change the metatable of other types from Agena, only from C.) If `metatable` is **null**, removes the metatable of the given register. If the original metatable has a '__metatable' field, raises an error. The function cannot assign metatables to C library functions.

This function returns `r`.

See also: **getmetatable**.

shift (r, a, b)

Moves an element in register `r` from position `old` to `new`, with `old`, `new` integers, shifting all the other elements accordingly - which might also cause a rotation. The function returns nothing.

See also: **move**, **purge**, **put**, **rotate** statement, **swap**.

size (r)

Returns the total number of items assignable in register `r`.

See also: **$$$** operator, **countitems**.

sort (r [, comp] [, 'number'])

Sorts register `r` in a given order, and in-place. All the values in the register up to the position pointed to by **the size** operator must be of the same type and non-**null**. See **sort** in Chapter 8 for more information. See also: **sorted**.

sorted (r [, comp] [, 'number'])

Sorts register elements in r in a given order, but - unlike sort - not in-place, and non-destructively. All the values in the register up to the position pointed to by the **size** operator must be of the same type and non-**null**. See **sorted** in Chapter 8 for more information.

See also: **sort**.

subs (x:v [, ···], r [, options])

Substitutes all occurrences of the value x in register r with the value v. See **subs** in Chapter 8 for more information.

See also: **map**, **remove**, **select**, **subsop**, **zip**.

subsop (i:v [, ···], r [, options])

Substitutes the value at r[i] with value v, where v can also be **null**, deleting the element in this case. See **subsop** in Chapter 8 for more information.

sumup (r)

Sums up all numeric values in register r. See **sumup** in Chapter 8 for more information.

See also: **qsumup**.

swap (r, a, b)

In register r, swaps the entries at index positions a and b, with a, b integers. The function returns nothing.

See also: **move**, **purge**.

top (r)

With register r, the operator returns the element with the largest index. If r is empty, it returns **null**.

See also: **bottom**.

type (r)

Returns the type of a register r, i.e. the string 'register'.

typeof (r)

Returns either the user-defined type of register `r`, or the basic type 'register'.

unique (r [, true])
unique (r [, options])

With a register `r`, the **unique** function removes multiple occurrences of the same item, if present in `r`, and returns a new register. The total size of the new register is equal to the number of the elements in the result. See **unique** in Chapter 8 for more information.

values (r, i_1 [, i_2, ···])

Returns the elements from the given register `r` in a new register. This function is equivalent to

```
return reg( r[i1], r[i2], ··· )
```

The total size of the new register is equal to the number of the elements in the result.

See also: **ops**, **select**, **unpack**.

whereis (r, x)

Returns the indices of a given value `x` in register `r` as a new register, respectively, dependent on the type of `r`. If `x` is not in `r`, returns an empty register.

See also: **has**, **member**.

zip (f, r1, r2)

This function zips together two registers `r1`, `r2` by applying the function `f` to each of its respective elements. See Chapter 8 for more information. See also: **augment**, **columns**, **map**, **remove**, **select**, **subs**.

The following functions have been built into the kernel as binary operators.

Please note that the operators returning a Boolean work in a Cantor way, i.e. `reg(1, 1) = reg(1)` → true, `reg(1, 2) xsubset reg(1, 1, 2, 2, 3, 3)` → true.

r1 == r2

This equality check of two registers `r1`, `r2` first tests whether `r1` and `r2` point to the same register reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `r1` and `r2` contain the same values without regard to their keys, and returns **true** or **false**. In this case, the search is quadratic.

r1 == r2

This strict equality check of two registers `r1`, `r2` first tests whether `r1` and `r2` point to the same register reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `r1` and `r2` contain the same number of elements and whether all entries in the registers are the same and are in the same order, and returns **true** or **false**. In this case, the search is linear.

r1 ~= r2

This approximate equality check of two registers `r1`, `r2` first tests whether `r1` and `r2` point to the same register reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `r1` and `r2` contain the same number of elements and whether all entries in the registers are approximately equal and are in the same order, and returns **true** or **false**. In this case, the search is linear. See **approx** for further information on the approximation check.

r1 <> r2

This inequality check of two registers `r1`, `r2` first tests whether `r1` and `r2` do not point to the same register reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `r1` and `r2` do not contain the same values, and returns **true** or **false**. In this case, the search is quadratic.

c in r

Checks whether the register `r` contains the value `c` and returns **true** or **false**. The search is linear. See also **binsearch** for binary search.

r1 intersect r2

Searches all values in register `r1` that are also values in register `r2` and returns them in a new register. The search is quadratic. The total size of the new register is equal to the number of the elements in the result. If `r1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `r2`.
If `r1` and/or `r2` are **null**, the operator assumes that **null** represents a register of zero size.

See also **registers.numintersect**.

r1 minus r2

Searches all values in register `r1` that are not values in register `r2` and returns them as a new register. The search is quadratic. The total size of the new register is equal to the number of the elements in the result. If `r1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `r2`. If `r1` and/or `r2` are **null**, the operator assumes that **null** represents a register of zero size.

See also **registers.numminus**.

r1 subset r2

Checks whether all values in register `r1` are included in register `r2` and returns **true** or **false**. The operator also returns **true** if `r1 = r2`. The search is quadratic. The total size of the new register is equal to the number of the elements in the result.

r1 union r2

Concatenates two registers `r1` and `r2` simply by copying all its elements - even if they occur multiple times - to a new register. The total size of the new register is equal to the number of the elements in the result. If `r1` has a metatable and/or a user-defined type, then they will be copied to the result; otherwise the function will try to copy them from `r2`. If `r1` and/or `r2` are **null**, the operator assumes that **null** represents a register of zero size.

See also **registers.numunion**.

r1 xsubset r2

Checks whether all values in register `r1` are included in register `r2` and whether `r2` contains at least one further element, so that the result is always **false** if `r1 = r2`. The search is quadratic. The total size of the new register is equal to the number of the elements in the result.

f @ r

In the first form, the operator maps a function `f` to all the values in register `r`. `f` should be a univariate function and return only one value. The return is a register. If `r` has metamethods or user-defined types, the return will also have them.

The operator actually calls function **map**.

Examples:

```
> <: x -> x^2 :> @ reg(1, 2, 3):
reg(1, 4, 9)

> <: x -> x > 1 :> @ reg(1, 2, 3):
reg(false, true, true)
```

If r1 and/or r2 are **null**, the operator assumes that **null** represents a register of zero size.

See also: @ and **$$** operators, **map**, **reduce**, **remove**, **select**, **subs**, **times**, **zip**.

f $ r

Returns all values in register r that satisfy a condition determined by function f. f should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x > 1 :> $ reg(1, 2, 3):
reg(2, 3)
```

If present, the function also copies the metatable and user-defined type of r to the new register.

All values up to the current top pointer are evaluated, and the size of the returned register is equal to the number of the elements in the return.

The operator actually calls function **select**.

If r is **null**, the operator assumes that **null** represents a register of zero size.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **unique**, **values**, **zip.**

f $$ r

Checks whether at least one element in register r satisfies the condition defined by function f and returns **true** or **false**. f should be a univariate function and return at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 1 :> $$ reg(1, 2, 3):
false
```

All values up to the current top pointer are evaluated.

If r is **null**, the operator assumes that **null** represents a register of zero size.

See also: @ operator, **countitems**, **descend**, **map**, **remove**, **selectremove, subs**, **unique**, **values**, **zip**.

f $$$ r

Counts the number of elements in register `r` that satisfy the condition defined by function `f`, a univariate function that returns at least one value. In the multi-return case, all results but the first are ignored.

```
> <: x -> x < 3 :> $$$ reg(1, 2, 3):
2
```

10.4.2 registers Library

This library provides generic functions for register manipulation. It provides all its functions inside table `registers`.

`registers.col (r, p [, ···] [, u:v])`

Extracts the given columns `p` etc. from the two-dimensional register `r`. If only one column shall be extracted, the return is a register with all the column elements; otherwise a register of column registers will be returned. You may restrict extraction to rows `u` to `v` by passing the respective index range `u:v` as an optional last argument.

See also: **columns**.

`registers.dimension (a:b [, c:d, ···] [, default])`

`registers.dimension (a:b [, c:d, ···] [, init = default])`

`registers.dimension (a [, b, ···] [, init = default])`

In the first form, creates a register of any dimension with index ranges `a:b` etc. with `a`, `b`, etc. integers, and an optional `default` for all its entries. `default` must not be a pair. The left-hand side values `a`, `c`, ... of the dimensions must always be 1.

In the second form the initialiser may be given as the option "init = `default`", which allows to also use pairs as a default.

In the third form, a register of any dimension with index ranges `1:a` , `1:b` etc. will be returned.

If the initialiser is a structure, i.e. table, set, pair, sequence or register, then individual copies of the initialiser are created to avoid referencing to the same structure.

See also: **registers.newreg**, **registers.regofregs**, **create register** statements.

`registers.getdim (r)`

Checks register `r` for sub-registers of the same size. Returns both the number of rows and the number of columns found if all sub-registers have the same size and issues an error otherwise.

See also: **registers.isrectangular**, **registers.issquare**.

`registers.isall (r, type)`

Checks whether all elements in register `r` are of a given `type` and returns **true** or **false**. Eligible types that the function accepts are `'number'`, `'integer'` (numbers that are all integral), `'complex'`, `'string'` and `'boolean'`. Also supported are `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonzeroint'` (non-zero integers), `'nonnegative'`

(non-negative numbers), `'numeric'` (numbers and complex numbers) and `'null'` (for **null** values).

The function is at least fifteen times faster than checking structures with the **satisfy** function.

See also: **checktype**, **isall**, **sequences.isall**, **sets.isall**, **tables.isall**.

registers.isone (r [, epsilon])

Checks whether register `r` consists of ones only and returns **true** or **false**.

By default, the check is done against strict one. You can change this by passing a positive `epsilon` value as an optional second argument so that all elements x with |x - 1| ≤ `epsilon` will be considered one. See **registers.iszero** for further information.

registers.isrectangular (r)

Checks the 2-dimensional register `r` for sub-registers of the same size.

Returns **true** or **false** and in case of **true**, also returns the number of rows and the number of columns found. With square registers, also returns **true**, see **registers.issquare**.

registers.issquare (r)

Checks whether the 2-dimensional register `r` has the same number of rows as there are elements in each of its sub-registers (`columns`). The sub-registers must be of the same size.

Returns **true** or **false** and in case of **true**, also returns the number of rows and the number of columns found.

registers.iszero (r [, epsilon])

Checks whether register `r` consists of zeros only and returns **true** or **false**. It returns the index of the first non-zero element in `r` as a second result, or 0 if there is none. The non-zero element is returned as a third result, too.

The function considers any non-numeric values in `r` as non-zero. An empty register is non-zero, too.

By default, the check is done against strict zero. You can change this by passing a positive `epsilon` value as an optional second argument so that all elements x with |x| <= `epsilon` will be considered zero.

See also: **registers.iszero**.

registers.move (r1, start, stop, newidx [, r2])

Copies elements from register `r1` to register `r2`, performing the equivalent to the following multiple assignment: `r2`[`newidx`],⋯ = `r1`[`start`], ⋯, `r1`[`stop`]. The default for `r2` is `r1`, i.e. elements are shifted in the same register.

The destination range can overlap with the source range. Specifically, for `newidx` we have 0 < `newidx` ≤ **size**(`r2`) + 1, all other targets will be rejected. Take care that the target register `r2` is properly sized.

The function returns the destination register `r2`.

Example: The following statement copies four elements in register a from position 3 up to and including 6 to new register b, starting with index 1:

```
> a := reg('a', 'b', 'c', 'd', 'e', 'f', 'g', 'h');
> b := registers.move(a, 3, 6, 1, reg());
> b:
reg(c, d, e, f)
```

See also: **move**, **shift**.

registers.new ([bool,] a, b [, k])

registers.new ([bool,] f, a, b [, k [, ⋯]])

registers.new (n, init = default)

In the first form, if no Boolean `bool` is given as the very first argument, the function creates a register **reg(**`a`, `a`+`k`, ⋯, `b`-`k`, `b`**)**, with `a`, `b`, and `k` (the step size) being numbers. The step size is 1 if `k` - a number - is not given. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced register of `k` numbers in the interval [`a`, `b`].

In the second form, if no Boolean `bool` is given as the very first argument, the function returns a register **reg(**1~`f`(`a`), 2~`f`(`a`+`k`), ⋯, (`b`-`a`)*1/`k`+1)~`f`(`b`)**)**, with `f` a function, `a` and `b` numbers. Thus, the function `f` is applied to all numbers between and including `a` and `b`. If `f` requires two or more arguments, the second, third, etc. argument must be passed after `k`. If any Boolean `bool` is given as the very first argument, the function generates a linearly spaced register of `k` numbers in the interval [`a`, `b`] with `f` applied to all its members.

The function uses the Kahan-Babuška summation algorithm to prevent round-off errors in case the step size is non-integral.

In the third form, creates a register of `n` slots, pre-filled with `default` which may be of any type.

Examples:

```
> registers.new(<: x, y -> x:x^2 + y :>, 1, 5, 1, 10):
reg(1:11, 2:14, 3:19, 4:26, 5:35)

> p := reg(0.1, 0.2, 0.1, 0.3, 1)

> registers.new( <: x -> x:p[x] :>, 1, size p):
reg(1:0.1, 2:0.2, 3:0.1, 4:0.3, 5:1)

> registers.new(true, -4, 4, 6):
reg(-4, -2.4, -0.8, 0.8, 2.4, 4)

> registers.new(8, init = 0):
reg(0, 0, 0, 0, 0, 0, 0, 0)
```

registers.new also accepts functions that may return **null**. Example:

```
> registers.new(<: x -> if x % 3 = 0 then x else null fi :>, 0, 10):
reg(0, null, null, 3, null, null, 6, null, null, 9, null)
```

See also: **map**, **tables.new**, **sets.new**, **sequences.new**.

registers.newreg (n)

Returns a register with `n` pre-allocated slots. `n` should be a non-negative integer.

The function is useful only if you have to pass a register initialiser as a function argument, otherwise it is recommended to use the **create register** statement.

The function is written in Agena and included in the lib/library.agn file.

See also: **registers.dimension**.

registers.numintersect (a, b)

Returns the number of elements in the intersection of `a` and `b`, aka **size**(`a` **intersect** `b`), without the overhead of generating the structure.

registers.numminus (a, b)

Returns the number of elements in the difference of `a \ b`, aka **size**(`a` **minus** `b`), without the overhead of generating the structure.

registers.numunion (a, b)

Returns the number of elements in the union of `a` and `b`, aka **size**(`a` **union** `b`), without the overhead of generating the structure.

registers.regofregs (n, slots, [, init = default])

The function creates a 2-dimensional register of `n` sub-registers, each with `slots` pre-allocated slots. You can fill each of the sub-registers with a default, of any type, `slots` times, by passing the `init` option. `slots` must be a non-negative integer.

Example:

```
> A := registers.regofregs(5, 3, init = 0):
reg(reg(0, 0, 0), reg(0, 0, 0), reg(0, 0, 0), reg(0, 0, 0), reg(0, 0, 0))
```

See also: **registers.dimension**, **registers.new**, **registers.newreg**.

registers.resize (r, n)

Extends or shrinks the given register `r` to - and not *by* - the given number of elements `n`.

When extending, all the elements already residing in `r` are kept. If `n` is less or equal to the current top (see **size**).

When shrinking, all surplus elements are removed. If the current top pointer is greater than `n`, it is reset to `n`.

The function returns **true** if the register has been successfully extended or shrunk, and **false** otherwise, for example when `n` is equal to the current top.

See also: **pack**.

registers.settop (r, n)

Sets the current position of the pointer to the top of register `r` to the given position `n`, a non-negative integer. Values above this position cannot be altered by any functions and operators. It returns **true** on success, and **false** otherwise. If the return is **false**, the current position of the top pointer has not been changed.

See also: **size**.

registers.subs (r, old, new)

In register `r`, the function replaces every occurrence of the value `old` with value `new`, in-place, and returns the number of substitutions done.

The function is four times faster than **subs** in traditional mode, avoiding most of all checks and conversions that **subs** is performing.

10.5 Pairs

Summary of Functions:

Queries

has, **in**, **notin**, **left**, **right**, **size**, **type**, **typeof**.

Operations

copy, **freeze**, **map**, **unfreeze**.

Relational Operators

=, ==, ~=, <>.

The following functionality has been built into the kernel as unary operators:

copy (p)

The operator deep-copies the entire contents of a pair p into a new pair.

freeze (p)

Write-protects pair p. See Chapter 8 for further information. See also: **unfreeze**.

has (p, x)

Checks whether pair p contains element x. The function performs a recursive scan so that it can find elements in deeply nested structures.

The function return **true** if x could be found in s, and **false** otherwise.

See also: **descend**, **in**, **recurse**, **satisfy**.

map (f, p [, ···])

Maps the function f on both elements of a pair p and returns a new pair. See **map** in Chapter 8 for more information.

size (p)

Returns the number of items in a pair p, i.e. always returns 2.

type (p)

Returns the type of a pair `p`, i.e. the string 'pair'.

typeof (p)

Returns either the user-defined type of the pair `p`, or the basic type 'pair'.

The following functionality has been built into the kernel as binary operators.

p1 = p2

This equality check of two pairs `p1`, `p2` first tests whether `p1` and `p2` point to the same pair reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether the left-hand side of `p1` and the left-hand side of `p2` are equal, and the same with both right-hand sides, and returns **true** or **false**.

p1 == p2

With pairs, the == operator acts exactly as the = operator.

p1 ~= p2

With pairs, the ~= operator compares the left-hand side of `p1` and the left-hand side of `p2` for approximate equality, and the same with both right-hand sides. The return is either **true** or **false**. See **approx** for further details.

p1 <> p2

This inequality check of two pairs `p1`, `p2` first tests whether `p1` and `p2` do not point to the same set reference in memory. If so, it returns **true** and quits.

If not, the operator then checks whether `p1` and `p2` do not contain the same items, and returns **true** or **false**.

c in p

Checks whether the number `c` fits into the closed interval with borders denoted by the numeric elements of pair `p`, and returns **true** or **false**.

c notin p

Checks whether the number `c` does not fit into the closed interval with borders denoted by the numeric elements of pair `p`, and returns **true** or **false**.

10.6 numarray - Numeric C Arrays

10.6.1 Introduction

The numarray package implements arrays of the C data types of either double, complex double, long double, unsigned char, unsigned 2-byte, signed or unsigned 4-byte integers and signed or unsigned 8-byte integers. The unsigned char type also supports bit fields.

The arrays implemented by this package are called numarrays for short.

Since numbers stored to numarrays consume less space, numarrays may be useful if a large amount of numbers have to be processed, but the amount of random-access memory of your system is limited.

While any element in a sequence, for example, occupies 24 bytes of memory, a number in a numeric array takes only eight bytes, and a character in a character array only one byte. On average, in real life with Agena numbers, numarrays occupy only half the memory that a table array of Agena numbers does.

Also, numarrays are useful to store binary data. Operations on numarrays, however, are usually slower than those on Agena's native structures: tables, pairs, sequences or registers - so you will trade speed for memory.

Internally, numarrays are userdata structures that support various metamethods.

You can create numarrays, assign and read numbers, resize arrays, store them to binary files, and read from files.

Functions to convert arrays to Agena's native structures, and vice versa, are provided, as well, see **numarray.toseq**, **numarray.toreg** or **numarray.toarray**.

To create an array of unsigned chars, use **numarray.uchar**, of signed 64-bit doubles use **numarray.double**, of complex doubles use **numarray.cdouble**, of 16-bit unsigned integers use **numarray.ushort**, of 32-bit unsigned integers use **numarray.uint32** and of 32-bit signed integers use **numarray.int32**.

Long double arrays can be created with **numarray.longdouble**, signed 64-bit integer arrays with **numarray.int64** and signed 64-bit integer arrays with **numarray.uint64**.

The number of entries to be stored must be given when calling these three procedures. When creating arrays, all slots are automatically filled with zeros. Array indices count from 1, not 0. You can pass negative indices to access values from the end of an array.

Example: Create an array of five 8-byte floating point numbers, i.e. ordinary Agena real numbers:

```
> a := numarray.double(5);
```

You may initialise an array with numbers by passing them in a table, sequence or register:

```
> a := numarray.double(5, [10, 20, 30, 40, 50]);
```

Determine the size, here we have five elements:

```
> size a:
5
```

The array prints as:

```
> a:
double(5)
```

Arrays can be converted to sequences and registers with **numarray.toseq** and **numarray.toreg**. So to quickly check what is in array `a`, we can just type:

```
> numarray.toseq(a):
seq(10, 20, 30, 40, 50)
```

Set the first element to number 10:

```
> a[1] := 0
```

```
> numarray.toseq(a):
seq(0, 20, 30, 40, 50)
```

Agena's standard indexing functions save and read numbers. So, for example, a[1] := -1 stores the number -1 to index 1 of the array a. a[1] reads the value stored at index 1 of the array a. Alternatively, **numarray.setitem** and **numarray.getitem** save and read numbers, respectively. Furthermore, **numarray.include** (bulk-assigns) numbers.

Get the element at index 1:

```
> a[1]:
0
```

For a much faster way to read and write values, check **numarray.geti** and **numarray.seti**.

The **numarray.resize** function extends or shrinks arrays. If we want to extend the array to store ten elements then issue, preserving the values already stored in it:

```
> numarray.resize(a, 10):
10

> numarray.toseq(a):
seq(10, 20, 30, 40, 50, 0, 0, 0, 0, 0)
```

Functions **numarray.setbit**, **numarray.getbit**, **numarray.iterate** support bit fields with unsigned char arrays.

numarray.whereis searches for numbers, and **numarray.iterate** can sequentially traverse arrays. Search for number 10 which resides at index 1 in array `a`:

```
> numarray.whereis(a, 50):
5
```

Search for a number that is not in the array:

```
> numarray.whereis(a, -1):
null

> f := numarray.iterate(a):
procedure(01DA10C8)

> f():  # 1st element
0

> f():  # 2nd element
20
```

Repeat further seven times, and finally issue

```
> f():  # tenth element
0

> f():  # no more elements
null
```

numarray.toarray creates arrays from tables, sequences and registers.

```
> numarray.toarray([1, 2, 3]):
double(3)
```

numarray.write writes the contents of any array to a binary file. **numarray.readuchars** reads a complete file of unsigned chars, **numarray.readushorts** of unsigned 2-byte integers, **numarray.readdoubles** of doubles, **numarray.readcdoubles** of complex doubles, **numarray.readlongdoubles** of longdoubles (see **long** package) and **numarray.readintegers** of signed integers with only one call. To open and close these files, use **binio.open** and **binio.close**. Most other binio functions, such as **binio.sync**, **binio.rewind**, and **binio.filepos**, are supported, as well, with the exception of the **binio.read*** procedures. The low-level **numarray.read** function is used by the above mentioned **numarray.read*** functions.

The following metamethods exist: standard read and write indexing (see above), **in** and **notin** operators, strict and approximate equality (=, ==, <>, ~=, ~<> operators), **size**, **zero**, **nonzero** and **tostring**. To easily add further metamethods, have a look at the end of the lib/library.agn source file.

The arrays can store status information or other data in a special registry table that is available at pseudo-index position 0. It is an empty table, by default. You can use the index metamethod or **numarray.getitem** to read from or write data into this table, e.g. `n[0]` or `numarray.getitem(n, 0)`. Alternatively, you can also retrieve a reference to the status information by calling the numarray like a function and without arguments, like `n()`, get a value from it by passing a key `k`, like `n(k)`, or set data by passing a key `k` and a value `v`, like `n(k, v)`.

All functions in this package can also be called OOP-style, for example:

```
> n := numarray.double(3)

> numarray.getitem(n, 1):
0

> n@@getitem(1):
0
```

Note that long doubles are not supported on ARM platforms.

Usually a numarray is one-dimensional. When creating an array, however, you can define multiple dimensions, with a maximum of 8. For example to create a two-dimensional array with two rows and five columns, enter:

```
> a := numarray.double(2, 5, [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);
```

To retrieve and assign an item from or to a multidimensional array, use **numarray.geti** and **numarray.seti** - classical indexing with square brackets works with one-dimensional arrays, only.

```
> numarray.geti(a, 2, 1):
6

> numarray.seti(a, 2, 1, 100):

> numarray.geti(a, 2, 1):
100
```

You can re-dimension arrays with **numarray.redim**. To convert 2-dimensional `a` to one dimension, issue:

```
> numarray.redim(a, 10);

> a[6]:
100
```

Hint: Package functions requiring indices usually work with a one-dimensional index only. Use **numarray.one** to convert multidimensional indices to a one-dimensional one.

Note that you can write-protect numarrays with **freeze** and remove the protection with **unfreeze**. You may also be interested in the **numcuckoo** and **numfilter** packages described in Chapter 10.18 and the **dblhash** hash set as a faster alternative to a numarray, see Chapter 10.19.

10.6.2 General Functions

numarray.append (a, x [,···])
numarray.append (a, b)

In the first form, adds one or more numbers `x`, etc. to the end of any numarray `a`.

In the second form adds all elements in numarray `b` to the end of numarray `a`.

The function returns nothing as it works in-place, modifying `a`.

See also: **numarray.include**, **numarray.prepend**, **numarray.resize**, **numarray.setitem**.

numarray.assign (a, str [, pos])

From position `pos` in numarray `a` assigns all numbers represented in the binary string `str` to `a`. `pos`, by default `pos` is 1. Example:

```
> d := numarray.double(2);

> str := bytes.tochars(Pi);

> str &:= bytes.tochars(E);

> numarray.assign(d, str);

> d[1], d[2]:
3.1415926535898 2.718281828459
```

See also: **binio.readchars**, **numarray.read***, **numarray.write** functions.

numarray.attrib (a)

Returns the type of numarray `a`, its current number of slots allocated and the number of bytes occupied, in this order. The number of dimensions and their respective sizes are returned as a fourth and fifth result.

See also: **typeof**, **numarray.used**, **numarray.getsize**.

numarray.binsearch (a, x [, exact])

Searches `x` in the numeric double array `a`, conducting binary search, and returns **true** or **false**. This is around three times faster than calling **numarray.member**. The numarray, however, must already be in ascending order.

By default, the function conducts an approximate equality check - set the optional third argument `exact` to any non-**null** value if you need an exact match.

See also: **numarray.find**, **numarray.member**, **numarray.sort**, **numarray.sorted**, **numarray.whereis**.

numarray.cdouble ([n_1 [, n_2, ···]])
numarray.cdouble (n_1, [, n_2, ···,] init)

In the first form, creates a one-dimensional or multi-dimensional numarray of signed complex doubles (C complex doubles) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0.

Initially, the number of elements can be zero (the default) or more, use **numarray.resize** to extend the array before assigning values.

In the second form, the function creates a numarray of sizes n_k of complex doubles and fills it with the numbers or complex numbers in table, sequence or register `init`. The number of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.int32**, **numarray.double**, **numarray.longdouble**, **numarray.redim**, **numarray.uint32**, **numarray.uchar**, **numarray.ushort**.

numarray.checkarray (··· [, atype])

The function checks whether all its arguments are number arrays, issues an error if one is not and returns the array types otherwise:

```
> a := numarray.new('double', [1, 2, 3]);

> b := numarray.new('int32', 3, <: x -> x :>));

> numarray.checkarray(a, b):
double  int32
```

By passing the optional string `atype`, you check for a specific number array type. Valid settings are 'double', 'cdouble', 'longdouble', 'ushort', 'int32', 'uint32' and 'int64', 'uint64'.

See also: **numarray.isall**, **numarray.isarray**.

numarray.convert (f, a [,···])

Same as **numarray.map**, but processes in-place: Maps a function `f` on each element in the numarray `a` and changes the entries accordingly, i.e. the array elements will be transformed from `a`[1] to `f`(`a`[1]), etc. `f` must always return a number.

numarray.countitems (x, obj [, eps])
numarray.countitems (f, obj [, ···])

In the first form, counts the number of occurrences of number `x` in numarray `obj`. If you pass any positive number for `eps`, then the function will conduct an approximate check with the given epsilon value, see **approx**.

In the second form, by passing a function `f` with a Boolean relation as the first argument, all elements in the structure `obj` that satisfy the given relation are counted. If the function has more than one argument, then all arguments *except the first* must be passed right after `obj`.

The return is a number.

See also: **numarray.find**, **numarray.select**, **linalg.countitems**, **numarray.countitems**.

numarray.cycle (a [, i [, p, [, true]]])

Like **numarry.iterate**, but cycles through the numarray `a`, restarting from the `i`-th element which is 1 by default. For arguments `p` and `true`, see **numarray.iterate**.

numarray.find (a, x [, eps])

Searches x in the numeric array a and if successful returns **true** and **false** otherwise. The search is linear. If you pass any positive value for `eps`, then the function will conduct an approximate check with the given epsilon value, for example **Eps**, see **approx**.

See also: **numarray.binsearch**, **numarray.member**, **numarray.whereis**.

numarray.double ([n_1 [, n_2, ···]])
numarray.double (n_1 [, n_2, ···], init)

In the first form, creates a one-dimensional or multi-dimensional numarray of signed doubles (C doubles) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0.

Initially, the number of elements can be zero (the default) or more, use **numarray.resize** to extend the array before assigning values.

In the second form, the function creates a numarray of sizes n_k of doubles and fills it with the numbers in table, sequence or register `init`. The number of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.int32**, **numarray.cdouble**, **numarray.longdouble**, **numarray.redim**, **numarray.uint32**, **numarray.uchar**, **numarray.ushort**.

numarray.geti (a, i_1 [, i_2, ···])
numarray.geti (a)

In the first form, retrieves the value in multidimensional array `a` by passing its respective indices i_1, i2, ... For example with a two-dimensional array, i_1 denotes the row and i_2 the column. The function works with one-dimensional arrays, as well.

In the second form, by just passing a one-dimensional or multidimensional array `a` produces a factory that each time it is called with one or more indices, returns the corresponding value in the array. This speeds up read access by 33 percent compared to the first form. Example:

```
> a := numarray.double(2, 3)  # a 2-dimensional array, 2 rows, 3 columns

> for row to 2 do  # fill it with 10, 20, .., 60
>    for column to 3 do
>       numarray.seti(a, row, column, row*column*10)
>    od
> od;

> f := numarray.geti(a):  # create the factory
procedure(02A88198)

> f(1, 1), f(2, 3):       # check the contents of the array
10      60
```

Read-access is `call-by-reference`, so you can read or modify the array with all the other proper **numarray** functions or the indexing metamethod and you will always get the recent content.

See also: **numarray.getitem**, **numarray.seti**, **numarray.one**, **numarray.redim**.

numarray.getitem (a, i [, n])

With `a` any numarray, considered to be a 1-dimensional array, returns the value stored at `a[i]`, where `i`, the index, is an integer counting from 1. The function is provided to avoid the index metamethod overhead.

If `n` is given, then besides `a[i]`, the values `a[i+1]` ... `a[i + n - 1]` are also returned as additional results. The default for `n` is 1.

If `i` is zero, then the internal status table of `a` will be returned. You can use the reference to write data into the status table.

To retrieve values from multidimensional arrays, by passing the respective indices, consider **numarray.geti**.

See also: **numarray.getparts**, **numarray.iterate**, **numarray.setitem**, **numarray.replicate**, **numarray.subarray**.

numarray.getparts (a, i [, n])

With `a` a 1-dimensional complex number numarray, returns both the real and imaginary part of the value at `a[i]`, where `i`, the index, is an integer counting from 1. The function is provided to avoid the index metamethod overhead.

If `n` is given, then besides `a[i]`, the real and imaginary parts of the values `a[i+1]` ... `a[i + n - 1]` are also returned as additional results. The default for `n` is 1.

See also: **numarray.geti**, **numarray.iterate**, **numarray.setitem**, **numarray.replicate**, **numarray.subarray**.

numarray.getbit (a, i)

Returns the bit at index position `i` of uchar array `a`. `i` starts from 1, the rightmost bit, not zero.

The return is either 0 or 1.

See also: **getbit**, **numarray.setbit**, **numarray.iterate**.

numarray.getsize (a)

Returns the number of slots used by the numarray.

See also: **numarray.attrib**, **numarray.used**.

numarray.include (a, pos, b)
numarray.include (a, i, x [, ···])

In the first form, copies all values in the numarray `b` into the numarray `a`, starting at index `pos` (a number) of `a` and overwriting the elements previously stored there. The function returns nothing. Both numarrays must by of the same type: either be uchar, integer, or double arrays and `a` must have enough space to accommodate the elements in `b`. See also **numarray.setitem** and **numarray.resize**.

In the second form, inserts one or more numbers `x`, etc. into an array. First, the array is enlarged by the number of values to be inserted, all values starting at position `i` (thus including the value already stored at `a`[`i`]) are pushed to open space and finally the new numbers are assigned to `a`[`i`] etc. You can append new values without overwriting existing elements by passing **size**(`a`) + 1 for `i`, or prepend values by setting `i` to 1.

The function returns nothing.

See also: **numarray.append**, **numarray.prepend**, **numarray.resize**, **numarray.setitem**.

numarray.int32 ([n_1 [, n_2, ···]])
numarray.int32 (n_1 [, n_2, ···], init)

Creates a one-dimensional or multi-dimensional numarray of signed 4-byte integers (C int32_t) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0. Initially, the number of elements can be zero (the default) or more.

In the second form, the function creates a numarray of size `n` of signed 4-byte integers and fills it with the numbers in table, sequence or register `init`. The number of values in `init` must be equal or less than `n`. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.int64**, **numarray.redim**, **numarray.resize**, **numarray.uchar**, **numarray.uint32**, **numarray.uint64**, **numarray.ushort**.

numarray.int64 ([n_1 [, n_2, ···]])
numarray.int64 (n_1 [, n_2, ···], init)

Creates a one-dimensional or multi-dimensional numarray of signed 8-byte integers (C int64_t) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0. Initially, the number of elements can be zero (the default) or more.

In the second form, the function creates a numarray of size `n` of signed 8-byte integers and fills it with 8-byte integers created with the **ints** package in table, sequence or register `init`. The number of values in `init` must be equal or less than `n`. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.int32**, **numarray.redim**, **numarray.resize**, **numarray.uchar**, **numarray.uint32**, **numarray.uint64**, **numarray.ushort**.

numarray.introsort (a)

Sorts a numeric array `a` in ascending order, in-place, using the introsort algorithm which combines quicksort with a final hashsort as suggested by Niklaus Wirth. The function processes all arrays except complex number arrays, see **numarray.sort**.

The function returns nothing.

numarray.isall (a, type)

Checks whether all elements in numeric double array `a` are of a given `type`.

Eligible types that the function accepts are `'integer'` (numbers that are all integral), `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonzeroint'` (non-zero integers) and `'nonnegative'` (non-negative numbers).

The return is either **true** or **false**.

The function is at least four times faster than checking structures with the **numarray.satisfy** function.

See also: **numarray.isarray**, **numarray.checkarray**.

numarray.isarray (··· [, atype])

The function checks whether all its arguments are number arrays, and returns **true** or **false**, respectively:

```
> a := numarray.new('double', [1, 2, 3]);

> numarray.checkarray(a, 'I am a string'):
true  false
```

By passing the optional string `atype`, you check for a specific number array type. Valid settings are 'double', 'cdouble', 'longdouble', 'int32','uint32' and 'ushort'.

See also: **numarray.checkarray**, **numarray.isall**.

numarray.iterate (a [, i [, p, [, bits [, true]]]])

Returns an iterator function that when called returns the next value in the numarray userdata structure `a`, or **null** if there are no further entries in the structure.

If an index `i` is passed, the first call to the iterator function returns the `i`-th element in the numarray list and with subsequent calls, the respective elements after index `i`.

You may also pass a positive integer step `p` to the iterator function: If given, then in subsequent calls the `p`-th element after the respective current one will be returned, equivalent to giving an optional step size in numeric for loops.

Bit Fields can be iterated one after the other by passing the fourth argument, the Boolean value **true**. (You may set `i` and `p` to 1 each to traverse all bits.)

Example 1: C doubles

```
> a := numarray.double(3)
```

```
> for i to 3 do a[i] := i * Pi od

> f := numarray.iterate(a, 2):  # return all values starting with index 2
procedure(01CDC200)

> f():
6.2831853071796

> f():
9.4247779607694

> f():  # no more values in a
null
```

Example 2: Bit Fields

```
> import numarray as n

> a := n.uchar(1)

> for i to 8 do n.setbit(a, i, 1) od

> n.get(a, 1):
255

> f := numarray.iterate(a, 1, 1, true) # iterate each bit, from the right

> f():
1
```

(etc.)

If the fifth argument is set to **true**, then the generated iterator returns both the position (index) and the value with each call. If the end of `a` has been reached, the next call to the iterator results in **null**, and the call after that will restart traversal.

```
> a := numarray.double(2, [10, 20])

> f := numarray.iterate(a, 1, 1, false, true);

> f():
1       10

> f():
2       20

> f():
null

> f():
1       10
```

Actually, **numarray.double**, **numarray.int32**, etc. attach an iterator to the metatable of a numarray, field '__iter', so that **for/in** loops will easily loop over it:

```
> a := numarray.double(2, [10, 20])
```

```
> for i, j in a do print(i, j) od
1       10
2       20
```

This is short for:

```
> f := numarray.iterate(a, 1, 1, false, true);

> for i, j in f do print(i, j) od
1       10
2       20
```

See also: **numarray.cycle**.

numarray.longdouble ([n_1 [, n_2, ···])
numarray.longdouble ([n_1 [, n_2, ···], init)

Creates a one-dimensional or multidimensional numarray of (signed) longdoubles (C long double) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0. Check the **long** package for further information.

Initially, the number of elements can be zero (the default) or more, use **numarray.resize** to extend the array before assigning values.

In the second form, the function creates a numarray of sizes n_k of long doubles and fills it with the numbers in table, sequence or register `init`. The number of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.int32**, **numarray.int**64, **numarray.uint32**, **numarray.uint64**, **numarray.uchar**, **numarray.ushort**.

numarray.map (f, a [,···] [, true])

Maps a function `f` on each element in the numarray `a` and returns a new numarray with the mapped results, i.e. the new array includes the values `f`(`a`[1]), `f`(`a`[2]), etc. `f` must always return a number.

If the last argument is the option inplace=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original array, but saving memory. After completion, the function returns the modified array.

See also: **numarray.convert**, **numarray.subs**.

numarray.member (x, obj [, eps])

Searches `x` in the numeric array `obj` and if successful returns the index of the first hit, otherwise returns **null**.

The function is much faster than **numarray.whereis** if you need the index of the first hit only. Note that with respect to **numarray.whereis**, the parameters are in reverse order.

If you pass any positive value for `eps`, then the function will conduct an approximate check with the given epsilon value, see **approx**. **Eps** may be an appropriate tolerance.

See also: **numarray.binsearch**, **numarray.countitems**, **numarray.find**, **numarray.whereis**.

numarray.minmax (obj [, true])

Returns a sequence with the minimum and maximum of all values in numarray `obj`, in this order.

If the option **true** is passed than the function assumes that all values in `obj` are sorted in ascending order so that execution is much faster.

With complex number arrays, the function computes the modulus to determine the result.

See also: **stats.minmax**.

numarray.new (what, obj)
numarray.new (what, n, f)

The function creates a new number array and fills it with values. `what`, a string, indicates the type of array to create - either 'double', 'cdouble', 'longdouble', 'uchar', 'ushort', 'uint32', 'int32', 'uint64' or 'int64'.

In the first form, the array is filled with all the (complex) numbers in table, sequence or register `obj`.

In the second form, `n` denotes the number of slots to allocate and `f` denotes a function that is called with integers from 1 to `n` and fills the array with its numeric result.

The function is written in Agena, see lib/library.agn.

numarray.one (a, i_1 [, i_2, ···])

Converts co-ordinates i_1, i_2, ... in the multidimensional array `a` to a one-dimensional index, starting from 1. With a two-dimensional array, for example, i_1 denotes the row and i_2 the column. The function works with one-dimensional arrays, as well.

See also: **numarray.geti**, **numarray.seti**, **numarray.redim**.

numarray.prepend (a, x [,···])
numarray.prepend (a, b)

In the first form, adds one or more numbers `x`, etc. to the beginning of any numarray `a`, shifting all elements in `a` into open space so that they are preserved.

In the second form prepends all elements in numarray `b` to the beginning of numarray `a`, also shifting all elements in `a` to open space.

The function returns nothing as it works in-place, modifying `a`.

See also: **numarray.include**, **numarray.prepend**, **numarray.resize**, **numarray.setitem**.

numarray.purge (a, i [, bool])

Removes the value stored at `a[i]`, shifting down other elements to close the space, and by default reduces the size of the array by one slot. If the array already is of size 0, an error will be returned. The function thus works in-place and returns the value deleted.

If `bool` is **false**, the size of the array is not reduced. Instead, the last entry of the array is set to 0. Use **numarray.resize** if you want to finally shrink the array to its new smaller size. Passing the **false** option may be useful to avoid memory re-allocation overhead when deleting a lot of values at one time.

See also: **numarray.include**, **numarray.resize**, **numarray.setitem**.

numarray.read (fh [, bufsize])

Reads data from the file denoted by its file handle `fh` and returns a numarray userdata structure of unsigned C chars.

The file must be opened before with **binio.open** and must finally be closed with **binio.close**.

In general, the function reads in a limited amount of bytes per call. If only `fh` is passed, the number of bytes read is determined by the **environ.kernel**('buffersize') setting, usually 512 bytes.

You can pass the second argument `bufsize`, a positive integer, to read less or more bytes. Passing the `bufsize` argument may also be necessary if your platform requires that an internal input buffer is aligned to a certain block size.

The function increments the file position thereafter so that the next bytes in the file can be read with a new call to **numarray.read**.

If the end of the file has been reached, or there is nothing to read at all, **null** will be returned.

In case of an error, it quits with the respective error. Use one of the following functions to read an entire file with only one call: **numarray.readdoubles**, **numarray.readintegers**, **numarray.readuchars**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

See also: **numarray.write**.

numarray.readcdoubles (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C complex doubles.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena, see lib/library.agn.

See also: **numarray.write**.

numarray.readdoubles (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C doubles.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readint64 (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C signed int64_t's.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readintegers (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C (signed) int32_t's.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readlongdoubles (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C long doubles, see **long** package.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readuchars (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C unsigned chars.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readuint32 (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C unsigned uint32_t's.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readuint64 (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of C unsigned uint64_t's.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.readushorts (fh [, bufsize])

Reads all the numeric data from the file denoted by its file handle `fh` and returns a numarray of 16-bit C unsigned integers.

By default, the function internally uses an input buffer of **environ.kernel**('buffersize') bytes, but you may choose another number of bytes with the `bufsize` option.

The file must be opened before with **binio.open** and finally be closed with **binio.close**.

The resulting array is one-dimensional, you may use **numarray.redim** to transform it to a multidimensional one.

The function is written in Agena (see lib/library.agn).

See also: **numarray.write**.

numarray.redim (a, n_1 [, n_2, ···, n_n])

Re-dimensions numarray a to an n-dimensional array with the respective sizes denoted by n_1, n_2, ... With a two-dimensional array, for example, n_1 denotes the row size and n_2 the column size. The maximum dimension is 8. The function works with one-dimensional arrays, as well, and returns nothing. The function issues an error if the total number of elements in the current array does not match the overall size of the re-dimensioned array.

See also: **numarray.geti**, **numarray.seti**, **numarray.one**, **numarray.resize**.

numarray.remove (f, a [, ···] [, true])

Returns all values in numeric array `a` that do not satisfy a condition determined by function `f` and returns a new array, or **null** if the condition has not been satisfied at all.

If `f` has only one argument, then pass the function and the array only.

```
> numarray.remove(<: x -> x > 1 :>, a);
```

If the function has more than one argument, then all arguments *except the first* are to be passed right after the name of `a`.

```
> numarray.remove(<: x, y -> x > y :>, a, 1):   # 1 for y
```

If the last argument is the option inplace=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original array if the given condition has been satisfied at least once, but saving memory. After completion, the function returns the modified array. If the result is **null**, then the array has not been changed.

See also: **numarray.map**, **numarray.satisfy**, **numarray.select**, **numarray.subs**.

numarray.replicate (a)

Copies the entire contents of numarray `a` into a new array and returns it.

See also: **numarray.getitem**, **numarray.subarray**.

numarray.resize (a, n)

The function re-sizes a numarray userdata structure `a` to the given number of entries `n`. Thus you can extend or shrink a numarray. When extending, the function fills the new array slots with zeros, while existing values are preserved. An array can be reduced to zero entries, as well.

The function returns the new size, an integer.

See also: **numarray.append**, **numarray.include**, **numarray.purge**, **numarray.redim**.

numarray.satisfy (f, a [,···])

With any numarray `a`, checks each element by calling function `f` which should return **true** or **false**. If at least one element in `a` does not satisfy the condition checked by `f`, the result is **false**, and **true** otherwise.

If `f` has more than one argument, then all arguments except the first are passed right after argument `a`.

See also: **numarray.isall**.

numarray.select (f, a [, ···] [, true])

Returns all values in numeric array `a` that satisfy a condition determined by function `f` and returns a new array, or **null** if the condition has not been satisfied at all.

If `f` has only one argument, then pass just the function and the array.

```
> numarray.select(<: x -> x > 1 :>, a);
```

If the function has more than one argument, then all arguments *except the first* are to be passed right after argument `a`.

```
> numarray.select(<: x, y -> x > y :>, a, 1):   # 1 for y
```

If the last argument is the option inplace=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original array if the given condition has been satisfied at least once, but saving memory. After completion, the function returns the modified array. If the result is **null**, then the array has not been changed.

See also: **numarray.map**, **numarray.remove**, **numarray.satisfy**, **numarray.subs**.

numarray.setbit (a, i, n)

Sets bit `n` at index position `i` of unsigned char array `a`. `n` must be either 0 or 1. `i` starts from 1, the rightmost bit, not from position zero. The function returns nothing.

See also: **setbit**, **numarray.getbit**, **numarray.iterate**.

numarray.seti (a, i_1 [, i_2, ···], v)
numarray.seti (a)

In the first form, with numarray `a`, sets number `v` to the slot denoted by its co-ordinates (one or more) i_1, i_2, ...; for example with a two-dimensional array, i_1 denotes the row and i_2 the column.

All indices start from 1. The function works with one-dimensional arrays, as well, and returns nothing.

In the second form, when given just a numeric array `a`, produces a factory that each time it is called with one or more indices plus a value, sets the latter into the array. This is 40 percent faster than the first form. Example:

```
> a := numarray.double(2, 3)  # a 2-dimensional array, 2 rows, 3 columns

> f := numarray.seti(a);      # create the factory

> for row to 2 do  # fill the array with 10, 20, .., 60
>    for column to 3 do
>       f(row, column, row*column*10)
>    od
> od;
```

```
> numarray.toseq(a):  # check the contents
seq(10, 20, 30, 20, 40, 60)
```

Write-access is `call-by-reference`, so you can modify the array with all the other available **numarray** writing functions and the indexing metamethod and you will always have the recent content.

See also: **numarray.geti**, **numarray.one**, **numarray.redim**, **numarray.setitem**.

numarray.setitem (a, i, v)

With `a` any 1-dimensional numarray, sets number `v` to `a[i]`, where `i`, the index, is an integer counting from 1. The function is provided to avoid the __index metamethod overhead.

If `i` is zero, the function sets `v`, which can be any data, as internal administration data, replacing the existing one.

If the size of `a` is zero, then the function will issue an error. Just resize `a` with **numarray.resize** before assigning values.

See also: **numarray.include**, **numarray.purge**.

numarray.setparts (a, i, re, im)

With `a` a 1-dimensional complex number array, sets the complex number `re` + I*`im` to `a[i]`, where `i`, the index, is an integer counting from 1. The function is provided to avoid the __index metamethod overhead.

See also: **numarray.include**, **numarray.purge**, **numarray.setitem**.

numarray.sort (a [, m:n] [, f])
numarray.sort (a [, f] [, m:n])

By default, with only numarray `a` given, sorts the entire array in ascending order, in-place. The function actually passes the call to **numarray.introsort** which can process all numeric arrays except complex number arrays.

You may indicate the elements to be sorted by giving their positions `m` to `n` as a pair. If `m` or `n` is negative, then the position is taken from the end of the array, with -1 denoting the last element in `a`, -2 the element before the last, et cetera. Put negative integers in brackets.

The sorting order can be controlled by passing the two-argument function `f` which should return a Boolean. The default for `f` is ascending, that is <: x, y -> x < y :>. This also allows to sort complex number arrays:

```
> L := seq(1!2, 3!4, 2!0, 8!4, 7!4, 0!1, 1!0, 3!4, 5!6, 9!10)

> a, c := numarray.cdouble(size L), 0; for i in L do a[++c] := i od;
```

Sort the array in descending order, from position 3 to the end:

```
> numarray.sort(a, <: x, y -> abs(x) > abs(y) :>, 3:(-1));

> numarray.toseq(a):
seq(1+2*I, 3+4*I, 9+10*I, 8+4*I, 7+4*I, 5+6*I, 3+4*I, 2, I, 1)
```

The function returns nothing. The function is written in Agena, implements the introsort algorithm which combines quicksort with a finalising hashsort as advised by Niklaus Wirth, and is included in the lib/library.agn file.

See also: **numarray.sorted**, **utils.posrelat**.

numarray.sorted (a)

Sorts a numarray `a` in ascending order, non-destructively, and returns a new array. Complex number arrays are not supported.

numarray.subarray (a, i [, j])

With `a` any numarray, returns the subarray `a[i` **to** `j]`, where `i`, `j`, the indices, are integers counting from 1. The return is a new numarray of the same type as `a`. If `j` is omitted, only `a[i]` will be returned. The function is provided to avoid index metamethod overhead.

See also: **numarray.getitem**.

numarray.subs (x:v [, ···], a [, options])

Substitutes all occurrences of the value `x` in the numarray `a` with the value `v`. More than one substitution pair can be given. The substitutions are performed sequentially and simultaneously starting with the first pair.

```
> numarray.subs(1:3, 2:4, [1, 2, -1]):
[3, 4, -1]
```

If the last argument is the option inplace=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original array, but saving memory. After completion, the function returns the modified array.

You can check numbers for approximate instead of strict equality by passing the new strict=**false** option.

See also: **numarray.map**, **numarray.remove**, **numarray.select**.

numarray.toarray (o [, option])

Writes all data in the table array, sequence or register `o` into a numarray and returns it. By default, a double array will be returned (option `'double'`); if the second argument `option` is the string `'uchar'`, an unsigned char array is created; if it is the string `'int32'`, a signed integer array will be returned. Further options are: `'ushort'` for unsigned 16-bit integer, `'uint32'` for unsigned 32-bit integer, `'longdouble'` for 80-bit floating-point and `'cdouble'` for complex double arrays.

If a value in `o` is not a number, zero is written to the array.

numarray.toreg (a)

Receives numarray `a` and converts it into a register of numbers, the return.

numarray.toseq (a)

Receives numarray `a` and converts it into a sequence of numbers, the return.

numarray.totable (a)

Receives numarray `a` and converts it into a table array of numbers, the return.

numarray.uchar ([n_1 [, n_2, ···]])
numarray.uchar (n_1 [, n_2, ···], init)

Creates a one-dimensional or multidimensional numarray of unsigned 1-byte characters (C unsigned char) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0.

Initially, the number of elements can be zero (the default) or more.

In the second form, the function creates a numarray of sizes n_k of 1-byte characters and fills it with the numbers in table, sequence or register `init`. The number of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.int32**, **numarray.redim**, **numarray.uint32**, **numarray.ushort**, **numarray.resize**.

numarray.uint32 ([n_1 [, n_2, ···]])
numarray.uint32 (n_1 [, n_2, ···], init)

Creates a one-dimensional or multidimensional numarray of unsigned 4-byte integers (C uint32_t) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0. Initially, the number of elements can be zero (the default) or more.

In the second form, the function creates a numarray of sizes n_k of unsigned 4-byte integers and fills it with the numbers in table, sequence or register `init`. The number of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.redim**, **numarray.uchar**, **numarray.int32**, **numarray.readuint32**, **numarray.uint32**, **numarray.ushort**.

numarray.uint64 ([n_1 [, n_2, ···]])
numarray.uint64 (n_1 [, n_2, ···], init)

Creates a one-dimensional or multidimensional numarray of unsigned 8-byte integers (C uint64_t) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0. Initially, the number of elements can be zero (the default) or more.

In the second form, the function creates a numarray of sizes n_k of unsigned 4-byte integers and fills it with 8-byte unsigned integers created with the **ints** package in table, sequence or register `init`. The number of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.redim**, **numarray.uchar**, **numarray.int32**, **numarray.readuint32**, **numarray.uint32**, **numarray.ushort**.

numarray.unique (a)

With numeric array `a`, the function removes multiple occurrences of the same value, if present. The return is a new numeric array with the original structure unchanged.

numarray.used (a)

Returns the estimated number of bytes consumed by the given array `a`.

See also: **numarray.attrib**, **numarray.getsize**.

numarray.ushort ([n_1 [, n_2, ···]])
numarray.ushort (n_1 [, n_2, ···], init)

Creates a one-dimensional or multidimensional numarray of unsigned 2-byte integers (C int16_t) with the given number of respective sizes n_1, ..., etc. with n_1, ... integers, and with each slot set to the number 0. Initially, the number of elements can be zero (the default) or more.

In the second form, the function creates a numarray of sizes n_k of unsigned 2-byte integers and fills it with the numbers in table, sequence or register `init`. The number

of values in `init` must be equal or less than n_k. Missing values in `init` will be set to zero.

See also: **numarray.cdouble**, **numarray.double**, **numarray.int32**, **numarray.int64**, **numarray.redim**, **numarray.uchar**, **numarray.uint32**, **numarray.uint64**.

numarray.whereis (a, what [, pos [, eps]])

Returns the index for a given value `what` in the numarray `a`. By default, the search starts at the beginning of the array, but you may pass any valid position `pos` (a positive integer) to determine where to start the search. The return is the index position, a positive number, or **null** if `what` could not be found in `a`.

By default, the function checks for exact equality to detect the existence of a value. By passing the fourth argument `eps`, a non-negative number, the function also compares the values approximately with the given maximum deviation `eps`. See **approx** for more details.

The `'__in'` metamethod internally uses this function to check for the existence of values.

See also: **numarray.binsearch**, **numarray.countitems**, **numarray.find**, **numarray.member**.

numarray.write (fh, a [, pos, nvalues])

Writes unsigned chars, integers, doubles, long doubles or 64-bit integers stored in a numarray `a` to the file denoted by its numeric file handle `fh`. The file must be opened with **binio.open** and closed with **binio.close**.

The start position `pos` is 1 by default but can be changed to any other valid position in the numarray.

The number of values (not bytes !) `nvalues` to be written can be changed by passing an optional fourth argument, a positive number, and by default equals the total number of entries in `a`, so the function can be called only once to write the entire array.

Passing the `nvalues` argument may also be necessary if your platform requires internal buffers to be aligned to a particular block size. Depending on the type of data stored in `a`, the function automatically computes the number of bytes to be written.

The function returns the index of the next start position (an integer) for a further call, to write the next bunch of data in `a`, or **null**, if the end of the array has been reached. When the function returns **null**, then it also automatically flushes all unwritten content to the file so that you do not have to call **binio.sync** manually if you want to read the file subsequently.

No further information is stored to the file created, so you always must know the type of data you want to read in later.

Example on how to write an entire array of 4,096 integers piece-by-piece:

```
> a := numarray.int32(4 * 1024);

> fd := binio.open('integer.bin');

> pos := 1;

> do  # write 1024 values per each call
>    pos := numarray.write(fd, a, pos, 1024)
> until pos = null;

> binio.close(fd);
```

Use **binio.sync** if you want to make sure that any unwritten content is physically written to the file when calling **numarray.write** multiple times on one array.

If you want to add data to the end of a file later on, pass the `'a'` option to **binio.open**.

Use one of the **numarray.read*** functions to read in the data later on.

`numarray.zip (f, a, b [, ···])`

In the first form, the function zips together either two numeric arrays `a`, `b` of the same kind by applying function `f` to each of its respective elements. Depending on the type of `a` and `b`, the result is a new numarray `s` where each element `s`[k] is determined by `s`[k] := f(`a`[k], `b`[k]).

`a` and `b` must have the same number of elements.

If `f` has more than two arguments, then its fourth to last argument must be given right after `b`.

If the last argument is the option inplace=**true**, or the Boolean **true**, then the operation will be done in-place, modifying the original array, but saving memory.

After completion, the function returns the modified array.

10.6.3 Mathematical Functions

The following functions all work non-destructively by default and return a new numarray, leaving the original input untouched. If given the inplace=**true** option, however, the functions modify the first numarray passed and also return it.

With functions processing two numarrays, both must be of the same type and have the same number of elements, otherwise an error will be issued.

See also Chapter 10.6.6 for supported metamethods.

The following four functions work with all kinds of numarrays:

numarray.xadd (a, b [, inplace=true])

For each number in numarray `a` adds the respective number in numarray `b`. This is equal to `a` **+** `b`.

numarray.xsub (a, b [, inplace=true])

For each number in numarray `a` subtracts the respective number in numarray `b`. This is equal to `a` **-** `b`.

numarray.xmul (a, b [, inplace=true])

Multiplies each number in numarray `a` by the respective number in numarray `b`. This is equal to `a` ***** `b`.

numarray.xdiv (a, b [, inplace=true])

Divides each number in numarray `a` by the respective number in numarray `b`. This is equal to `a` / `b`.

The following functions work with numarrays of type 'double' (C doubles) and 'cdouble' (C complex doubles) only:

numarray.xrecip (a [, inplace=true])

For each number x in numarray `a`, computes 1/x = **recip**(x). In non-destructive mode, this is equal to **recip**(`a`).

numarray.xln (a [, inplace=true])

For each number x in numarray `a`, computes **ln**(x). In non-destructive mode, this is equal to **ln**(`a`).

numarray.xexp (a [, inplace=true])

For each number x in numarray `a`, computes e^x. In non-destructive mode, this is equal to **exp**(`a`).

numarray.xsqrt (a [, inplace=true])

For each number x in numarray a, computes $\sqrt{x}$. In non-destructive mode, this is equal to **sqrt**(a).

numarray.xsquare (a [, inplace=true])

For each number x in numarray a, computes x^2. In non-destructive mode, this is equal to **square**(a).

numarray.xlog2 (a [, inplace=true])

Applies the logarithm to base 2 on each number in numarray a.

numarray.xantilog2 (a [, inplace=true])

For each number x in numarray a, computes 2^x. In non-destructive mode, this is equal to **antilog2**(a).

numarray.xcos (a [, inplace=true])

For each number x in numarray a, computes **cos**(x). In non-destructive mode, this is equal to **cos**(a).

numarray.xsin (a [, inplace=true])

For each number x in numarray a, computes **sin**(x). In non-destructive mode, this is equal to **sin**(a).

numarray.xtan (a [, inplace=true])

For each number x in numarray a, computes **tan**(x). In non-destructive mode, this is equal to **tan**(a).

numarray.xcosh (a [, inplace=true])

For each number x in numarray a, computes **cosh**(x). In non-destructive mode, this is equal to **cosh**(a).

numarray.xsinh (a [, inplace=true])

For each number x in numarray a, computes **sinh**(x). In non-destructive mode, this is equal to **sinh**(a).

numarray.xtanh (a [, inplace=true])

For each number x in numarray a, computes **tanh**(x). In non-destructive mode, this is equal to **tanh**(a).

numarray.xarccos (a [, inplace=true])

For each number x in numarray a, computes **arccos**(x). In non-destructive mode, this is equal to **arccos**(a).

numarray.xarcsin (a [, inplace=true])

For each number x in numarray a, computes **arcsin**(x). In non-destructive mode, this is equal to **arcsin**(a).

numarray.xarctan (a [, inplace=true])

For each number x in numarray a, computes **arctan**(x). In non-destructive mode, this is equal to **arctan**(a).

10.6.4 Bitwise Functions

The following functions all work non-destructively by default and return a new numarray, leaving the original input untouched. If given the inplace=**true** option, however, the functions modify the first numarray passed and also return it.

The numarrays must have unsigned integers only (see **numarray.char**, **numarray.uint32**, **numarray.ushort**) - so double, complex double, longdouble and signed integer arrays are not supported.

With functions processing two numarrays, both must be of the same type and have the same number of elements, otherwise an error will be issued.

See also Chapter 10.6.6 for supported metamethods.

The functions are:

numarray.band (a, b [, inplace=true])

For each integer x in numarray `a` and the respective integer y in numarray `b`, computes x **&&** y (bitwise AND). This is equal to `a` **&&** `b`.

numarray.bor (a, b [, inplace=true])

For each integer x in numarray `a` and the respective integer y in numarray `b`, computes x || y (bitwise OR). This is equal to `a` || `b`.

numarray.bxor (a, b [, inplace=true])

For each integer x in numarray `a` and the respective integer y in numarray `b`, computes x ^ ^ y (bitwise XOR). This is equal to `a` ^ ^ `b`.

numarray.bnot (a [, inplace=true])

For each number x in numarray `a`, computes ~~x. In non-destructive mode, this is equal to ~~`a`.

10.6.5 Units Conversion Functions

The following functions all work non-destructively by default and return a new numarray, leaving the original input untouched. If given the inplace=**true** option, however, the functions modify the first numarray passed and also return it.

The following functions work with numarrays of type 'double' only:

numarray.celsius (a [, inplace=true])

Converts each number in numarray a, considered to be in degrees Fahrenheit, to degrees Celsius.

See also: **numarray.fahren**, **units.celsius**.

numarray.fahren (a [, inplace=true])

Converts each number in numarray a, considered to be in degrees Celsius, to degrees Fahrenheit.

See also: **numarray.celsius**, **units.fahren**.

numarray.km (a [, inplace=true])

Converts each number in numarray a, considered to be in statute miles, to kilometers.

See also: **numarray.mile**, **units.km**.

numarray.mile (a [, inplace=true])

Converts each number in numarray a, considered to be in kilometers, to statute miles.

See also: **numarray.km**, **units.mile**.

numarray.gram (a [, inplace=true])

Converts each number in numarray a, considered to be in avoirdupois ounces, to statute grams.

See also: **numarray.ounce**, **units.gram**.

numarray.ounce (a [, inplace=true])

Converts each number in numarray a, considered to be in grams, to avoirdupois ounces.

See also: **numarray.gram**, **units.ounce**.

numarray.floz (a [, inplace=true])

Converts each number in numarray a, considered to be in litres, to US fluid ounces.

See also: **numarray.gallon**, **numarray.litre**, **units.floz**.

numarray.litre (a [, inplace=true])

Converts each number in numarray `a`, considered to be in US fluid ounces, to litres.

See also: **numarray.floz**, **numarray.gallon**, **units.litre**.

numarray.gallon (a [, inplace=true])

Converts each number in numarray `a`, considered to be in litres, to US liquid gallons.

See also: **numarray.floz**, **numarray.litre**, **units.gallon**.

10.6.6 Metamethods

Metamethod	Functionality
'__index'	read operation, e.g. n[p] or n[p **to** q], with p, q any valid indices
'__writeindex'	write operation, e.g. n[p] := value, with p any valid index
'__size'	**size** operator, number of elements in a numarray
'__in'	**in** operator
'__notin'	**notin** operator
'__empty'	**empty** operator
'__filled'	**filled** operator
'__tostring'	formatting for output at the console
'__aeq'	approximate equality ~= operator
'__eq'	equality operator =
'__eeq'	strict equality operator ==
'__zero'	**zero** operator
'__nonzero'	**nonzero** operator
'__sumup'	**sumup** operator
'__qsumup'	**qsumup** operator
'__mulup'	**mulup** operator
'__qmdev'	**qmdev** operator
'__intersect'	**intersect** operator
'__minus'	**minus** operator
'__union'	**union** operator
'__add'	+ operator
'__sub'	- operator
'__mul'	* operator
'__div'	/ operator
'__recip'	**recip** operator
'__sqrt'	**sqrt** operator
'__square'	**square** operator
'__ln'	**ln** operator
'__exp'	**exp** operator
'__antilog2'	**antilog2** operator
'__sin'	**sin** operator
'__cos'	**cos** operator
'__tan'	**tan** operator
'__sinh'	**sinh** operator
'__cosh'	**cosh** operator
'__tanh'	**tanh** operator
'__arcsin'	**arcsin** operator
'__arccos'	**arccos** operator
'__arctan'	**arctan** operator
'__gc'	garbage collection

10.7 llist - Linked Lists

The **llist** package is built-in and is available right from the start.

10.7.1 Introduction and an Example

Tables and sequences are quite slow if you have to insert or delete a lot of elements during an operation, for with each insertion or deletion, objects have to be shifted upward or downward physically.

To avoid these costly operations, data can also be represented in containers, or `nodes`, where "[e]ach node contains two fields: a "data" field to store whatever element [...], and a "nextone" field which is a pointer used to link one node to the next node.[1]" For example, if you would like to insert a new element at position n, the address of the `next entry` of node n - 1 is changed to the address of this new node containing the element to be inserted, and the `next entry` in the new node is assigned the address of the node containing the original value at position n.

This speeds up write operations by dimensions; read operations, however, are slower, for the linked list has to be traversed linearly. However, linked lists as implemented in this package are around fifteen times faster even when conducting a read operation with each write operation.

Metamethods exist to support printing, indexing, and indexed assignments; the **size**, **in**, **notin**, =, and ~= operators are also supported.

Linked lists can contain **null**s, i.e. putting **null** into the data field of a node does not delete this node from the chain.

Linked list can store status information or other data in a special registry table that is available at pseudo-index position 0. You can use the index metamethod or **llist.getitem** to read from or write data into this table.

Create an empty list.

```
> L := llist.new():
llist()
```

Now add 0 to it

```
> llist.append(L, 0);
```

and also put -2 to its beginning.

```
> llist.prepend(L, -2);
```

[1] For an excellent introduction on implementing linked lists, see "Linked List Basics", Copyright © 1998-2001, Nick Parlante. This quote has been taken from his manual, page 4.

```
> L:
llist(-2, 0)
```

Insert -1 at position 2. As you see, the original element at this position is not deleted but shifted to open space.

```
> llist.put(L, 2, -1);

> L:
llist(-2, -1, 0)
```

To delete an element at a position, enter:

```
> llist.purge(L, 2):
-1
```

Singly-linked lists can be iterated directly in for /in loops:

```
> for i, j in L do print(i, j) od
1       -2
2       0
```

Print the contents of the status table:

```
> print(L[0]);

> print(llist.getitem(L, 0));
```

Assign 16 to status table key 'cursor':

```
> a[0].cursor := 16;
```

Or likewise:

```
> a[0, 'cursor'] := 16;
```

For an example of how to use linked lists, see Chapter 6.28.

Note that the linked list implemented in this package always knows about the position of the top and the bottom element - so read and write access to them is always O(1).

Last but not least, linked lists, unrolled singly-linked lists and doubly-linked lists can be write-protected with the **freeze** function.

With the exception of **llist.new**, all of the following package functions can be used OOP-style:

10.7.2 Functions

`llist.append (l, obj [, ···])`

Appends one or more elements `obj` which may be of any type, to the singly-linked list `l`, in sequential order. There is no return.

See also: **llist.prepend**, **llist.put**.

`llist.checkllist (l)`

Checks whether its argument is a singly-linked list and issues an error otherwise. The function returns nothing.

`llist.dump (l)`

Writes each element in the singly-linked list `l` to a sequence and then deletes it from the list. The linked list thereafter is completely empty and can be re-used later on. The return is the sequence mentioned before.

The function can be used if you have run out of memory.

See also: **llist.toseq**.

`llist.getitem (l, idx [, n])`

Returns the item at index `idx` of the singly-linked list `l`. If the index does not exist, the function returns **null**.

If `idx` is negative, the function returns the value stored at the `-idx`'s position counting from end of the list.

If `n` is given, then besides `a[idx]`, the values `a[idx + 1]` ... `a[idx + n - 1]` are also returned as additional results. The default is 1, that is, `a[idx]`.

See also: **llist.setitem**, **utils.multidim**, **utils.onedim**.

`llist.iterate (l [, n [, p [, true]]])`

Returns an iterator function that when called returns the next value in the singly-linked list `l`, which might also be **null** if one or more **null**s are included in the linked list, or **null** if there are no more entries in the list. Also returns **null** if the linked list is empty.

If an index `n` is passed, the first call to the iterator function returns the `n`-th element in the list and with subsequent calls, the respective elements after index `n`.

You may also pass a non-negative integer `p` to the iterator function: In this case, the next consecutive `p` elements in the list are skipped before determining and returning a value.

If the fourth optional argument **true** is given, then the iterator will return both a running index and the associated value. When the end of the lists has been reached, it returns **null** once and then restarts iteration from the beginning of the list (or the index given when calling the factory).

Example: Since the iterator can return **null** even if the end of the list has not yet been reached, we use a counter:

```
> L := llist.new(1); llist.append(L, null); llist.append(L, 2);

> f := llist.iterate(L);

> c := 0;

> while c++ < size L do
>    print(f())
> od;
1
null
2
```

The function can also process ulists.

See also: **ipairs**.

llist.new ([···])

The function creates a new singly-linked list and optionally stores all of the given elements in it. The return is a userdata of user-type `'llist'`.

llist.prepend (l, obj [, ···])

Prepends an element `obj`, and optionally further elements, which may be of any type, to the singly-linked list `l`. There is no return.

See also: **llist.append**, **llist.put**.

llist.purge (l [, n])

The function removes the element at position `n` from the linked list `l`. All the successors of the element to be deleted are `shifted` downwards. The function returns the value deleted, but issues an error if there is no element (i.e. node) at index `n`.

If `idx` is negative, the function deletes the value stored at the `-idx`'s position counting from end of the list.

If `n` is not given, then the last, i.e. top node is deleted; this is equal to **llist.purge**(`l`, size `l`).

The function can also process ulists.

llist.put (l, n, obj)

The function inserts the given element `obj` into singly-linked list `l` at position `n`. The original element at position `n` is not deleted - it and all its successors are `shifted` to open space. The function returns nothing, and issues an error if the index is out-of-range.

If `idx` is negative, the function inserts the value at the -`idx`'s position counting from end of the list.

The function can also process ulists.

See also: **llist.append**, **llist.prepend**, **utils.multidim**, **utils.onedim**.

llist.replicate (l)

The function creates a copy of the singly-linked list `l` and returns a new linked list. However, if an element in `l` is a structure, it is not deep-copied.

llist.setitem (l, idx, obj)

Stores `obj`, which may be of any type, to position `idx` of the singly-linked list `l`, overwriting the existing value. If size `l` > `idx` + 1 and the index does not yet exist, the function simply quits without an error. If `idx` = size `l` + 1, then the call is equivalent to **llist.append**, just adding `obj` to the end. The function returns nothing.

If `idx` is negative, the function sets value to the -`idx`'s position counting from the end of the list.

See also: **llist.getitem**, **utils.multidim**, **utils.onedim**.

llist.toseq (l)

The function creates a new sequence and copies all elements in the singly-linked list `l` into it, in sequential order. The return is the sequence. If there are no elements in `l`, an empty sequence will be returned. If the list includes **null**s, they are ignored.

llist.totable (l)

The function creates a new table and copies all elements in the singly-linked list `l` into it, in sequential order. The return is the table. If there are no elements in `l`, an empty table will be returned. If the list includes **null**s, the resulting table will contain holes.

10.7.3 Unrolled Singly-Linked Lists

The **llist** package also supports unrolled singly-linked lists. You will find the respective functions in the `package` table **ulist**.

Unrolled singly-linked lists (ulists) internally consist of a singly-linked list storing sequences of the actual values in each of its nodes. Various administrative information - the current number of sequences (i.e. nodes) plus the current and the maximum number of values in each sequence - is stored in a `registry` table at pseudo-index 0. However, this internal structure is hidden from the user, and you can use the same indices as you would do when calling **llist** functions to read or write values.

Insert and delete operations on ulists are twenty times faster when compared to singly-linked lists, with only a small increase of memory consumption. Similarly, simple read and write operations are 15 times faster.

Note that contrary to lllists, ulists cannot store **null**. The ulist package provides the following metamethods:

Functionality	**ulist metamethod**	**ulist alternative**
Pretty printer	**print** function, colon utility	ulist.tostring(L)
Reading values	L[k]	ulist.getitem(L, k)
Saving values	L[k] := v, etc.	ulist.setitem(L, k, v)
Size	**size** operator	ulist.getsize(L)
Existence check	**in** operator	ulist.has(L, v)
Existence check	**notin** operator	ulist.hasnot(L, v)
Equality check	= operator	ulist.isequal(K, L)

The following **ulist** functions work like the **llist** functions of the same name, with the exception of **ulist.new**, and can also be called OOP-style:

ulist.append (ul, obj [, ···])

The function works like **llist.append**.

It is written in Agena and included in the lib/llist.agn file.

ulist.checkulist (ul)

Checks whether its argument is a ulist and issues an error otherwise. The function returns nothing.

See also: **ulist.isulist**.

ulist.dump (ul)

The function works like **llist.dump**.

ulist.getitem (ul, idx [n])

The function works like **llist.getitem**.

ulist.getllist (ul, node)

Returns the sequence stored at `node` (a positive integer) of the underlying llist. If the node does not exist, the function returns **null**.

ulist.getsize (ul)

Returns the number of items in a ulist.

ulist.has (ul, v)

Checks whether the ulist contains item `v` and returns **true** or **false**.

ulist.isulist (ul)

Checks whether its argument is a ulist and returns **true** or **false**.

See also: **ulist.checkulist**.

ulist.iterate (ul [, n [, p [, true]]])

The function works like **llist.iterate**.

It is written in Agena and included in the lib/llist.agn file.

See also: **ipairs**.

ulist.new (n [, fill])

The function creates a new unrolled singly-linked list and internally uses sequences with a maximum size of `n` slots each. The default for `n` is 128.

If the fractional number `fill` is given, with $0 < \texttt{fill} < 1$, each underlying sequence will be filled later on to the given percentage before a new one is created. The default is 0.75 for 75 percent. Reasonable values for `fill` may range between 0.5 to 0.75.

Unrolled singly-linked lists have an embedded iterator function that allows them to be iterated in **for/in** loops.

ulist.prepend (ul, obj [, ···])

The function works like **llist.prepend**.

It is written in Agena and included in the lib/llist.agn file.

ulist.purge (ul, n)

The function works like **llist.purge**, but also returns the element deleted.

It is written in Agena and included in the lib/llist.agn file.

ulist.put (ul, n, obj)

The function works like **llist.put**.

It is written in Agena and included in the lib/llist.agn file.

ulist.setitem (ul, idx, value)

The function works like **llist.setitem**.

ulist.sort (ul [, f])

The function works like **sort** and returns nothing.

ulist.swap (ul, i, j)

Swaps the positions of `ul`[`i`] and `ul`[`j`] in-place. The function returns nothing.

ulist.tostring (ul)

Converts the contents of a ulist to a formatted string that can be output at the prompt.

It is written in Agena and included in the lib/llist.agn file.

ulist.toseq (ul)

The function works like **llist.totable** but returns a sequence instead of a table.

See also: **ulist.dump**.

ulist.totable (ul)

The function works like **llist.totable**.

10.7.4 Doubly-Linked Lists

Finally, the **llist** package features doubly-linked lists. Read and write access to elements in doubly-linked lists is twice as fast as with singly-linked lists.

You find the respective functions in the package table **dlist**.

The functions implemented for doubly-linked lists have the same name, work the same way and have the same syntax as those for singly-linked lists, which are available in package table **llist**. Just replace the prefix `llist` with `dlist`.

With the exception of **dlist.new**, all of the following package functions can be used OOP-style:

dlist.append (l, obj [, ···])

Appends one or more elements `obj` which may be of any type, to the doubly-linked list `l`, in sequential order. There is no return.

See also: **dlist.prepend**, **dlist.put**.

dlist.checkdlist (l)

Checks whether its argument is a doubly-linked list and issues an error otherwise. The function returns nothing.

dlist.dump (l)

Writes each element in the doubly-linked list `l` to a sequence and then deletes it from the list. After completion, the linked list is empty and can be re-used later on. The return is the sequence mentioned before.

The function can be used if you have run out of memory.

See also: **dlist.toseq**.

dlist.getitem (l, idx [, n])

Returns the item at index `idx` of the doubly-linked list `l`. If the index does not exist, the function returns **null**.

If `idx` is negative, the function returns the value stored at the `-idx`'s position counting from end of the list.

If `n` is given, then besides `a[idx]`, the values `a[idx + 1]` ... `a[idx + n - 1]` are also returned as additional results. The default is 1, that is `a[idx]` will be returned.

See also: **dlist.setitem**, **utils.multidim**, **utils.onedim**.

dlist.iterate (l [, n [, p [, true]]])

Returns an iterator function that when called returns the next value in the doubly-linked list `l`, which might also be **null** if one or more **null**s are included in the linked list, or **null** if there are no more entries in the list. Also returns **null** if the linked list is empty.

If an index `n` is passed, the first call to the iterator function returns the `n`-th element in the list and with subsequent calls the respective elements after index `n`.

You may also pass a non-negative integer `p` to the iterator function: In this case, the next consecutive `p` elements in the list are skipped before determining and returning a value.

If the fourth optional argument **true** is given, then the iterator will return both a running index and the associated value. When the end of the lists has been reached, it returns **null** once and then restarts iteration from the beginning of the list (or the index given when calling the factory).

Example: Since the iterator can return **null** even if the end of the list has not yet been reached, we use a counter:

```
> L := dlist.new(1); dlist.append(L, null); dlist.append(L, 2);

> f := dlist.iterate(L);

> c := 0;

> while c++ < size L do
>    print(f())
> od;
1
null
2
```

See also: **ipairs**.

dlist.new ([···])

The function creates a new doubly-linked list and optionally stores all of the given elements in it. The return is a userdata of user-type `'dlist'`. Doubly-linked lists have an embedded iterator function that allows them to be iterated in **for/in** loops.

dlist.prepend (l, obj [, ···])

Prepends an element `obj`, and optionally further elements, which may be of any type, to the doubly-linked list `l`. There is no return.

See also: **dlist.append**, **dlist.put**.

dlist.purge (l [, n])

The function removes the element at position `n` from the doubly-linked list `l`. All the successors of the element to be deleted are `shifted` downwards. The function returns the value deleted, but issues an error if there is no element (i.e. node) at index `n`.

If `idx` is negative, the function deletes the value stored at the `-idx`'s position counting from end of the list.

If `n` is not given, then the last, i.e. top node is deleted; this is equal to **dlist.purge**(`l`, size `l`).

See also: **utils.multidim**, **utils.onedim**.

dlist.put (l, n, obj)

The function inserts the given element `obj` into doubly-linked list `l` at position `n`. The original element at position `n` is not deleted - it and all of its successors are `shifted` to open space. The function returns nothing, and issues an error if the index is out-of-range.

If `idx` is negative, the function inserts the value at the `-idx`'s position counting from end of the list.

See also: **dlist.append**, **dlist.prepend**, **utils.multidim**, **utils.onedim**.

dlist.replicate (l)

The function creates a copy of the doubly-linked list `l` and returns a new linked list. However, if an element in `l` is a structure, it is not deep-copied.

dlist.setitem (l, idx, obj)

Stores `obj`, which may be of any type, to position `idx` of the doubly-linked list `l`, overwriting the existing value. If size `l` > `idx` + 1 and the index does not yet exist, the function simply quits without an error. If `idx` = size `l` + 1, then the call is equivalent to **dlist.append**. The function returns nothing.

If `idx` is negative, the function sets value to the `-idx`'s position counting from the end of the list.

See also: **dlist.getitem**, **utils.multidim**, **utils.onedim**.

dlist.toseq (l)

The function creates a new sequence and copies all elements in the doubly-linked list `l` into it, in sequential order. The return is the sequence. If there are no elements in `l`, an empty sequence will be returned. If the list includes **null**s, they are ignored.

dlist.totable (l)

The function creates a new table and copies all elements in the doubly-linked list `l` into it, in sequential order. The return is the table. If there are no elements in `l`, an empty table will be returned. If the list includes **null**s, the resulting table will contain holes.

10.8 bags - Mulitsets

10.8.1 Introduction and Examples

A bag, also called a multiset, is a kind of Cantor set that stores the number of occurrence along with each unique element.

Consider a bulk of orders of books where each order is reported individually. You may only want to know how many times a book has been sold, instead of storing each individual order (and maybe all its data) to finally count them. You may want to save space and perform the count immediately as soon as the order has been committed.

The package uses tables of the user-defined type 'bag' to implement multisets.

A sequence of orders might look like this:

```
> orders := seq(
>   'Programming in Lua', 'Moon Lander', 'Lost Moon',
>   'Programming in Lua', 'Moon Lander', 'Lost Moon',
>   'C von A bis Z');

> books := bags.new(unpack(orders));

> books['Lost Moon']:
2
```

For a further order, just enter

```
> bags.include(books, 'Agena');

> books:
bag(Agena ~ 1, C von A bis Z ~ 1, Lost Moon ~ 2, Moon Lander ~ 2,
Programming in Lua ~ 2)
```

A customer has cancelled his previous orders:

```
> bags.remove(books, 'Agena'):

> books:
bag(C von A bis Z ~ 1, Lost Moon ~ 2, Moon Lander ~ 2, Programming in Lua ~
2)
```

If you would like to iterate a bag, you can use a **for/in** loop:

```
> for i, j in books do print(i, j) od
Programming in Lua      2
C von A bis Z   1
Lost Moon       2
Moon Lander     2
```

10.8.2 Functions & Metamethods

The package provides the following metamethods:

Metamethod	Functionality
'__index'	read operation, e.g. n[p], with p an index
'__writeindex'	write operation, e.g. n[p] := value, with p the index
'__size'	**size** operator, number of characters currently stored
'__in'	**in** operator
'__notin'	**notin** operator
'__empty'	**empty** operator
'__filled'	**filled** operator
'__union'	**union** operator
'__intersect'	**intersect** operator
'__minus'	**minus** operator
'__tostring'	formatting for output at the console
'__gc'	garbage collection

With the exception of **bags.new**, all of the following package functions can be used OOP-style:

bags.attrib (b)

Returns the number of occurrence of all unique elements in the bag `b` and also the accumulated number of all occurrences of these elements in it. For example, the multiset bag('Curiosity' ~ 2, 'Skycrane' ~ 1) results to 2, 3.

See also: **bags.getsize**.

bags.bagtoset (b)

The function returns all of the unique elements in `b` as a set.

bags.getsize (b)

Returns the number of occurrence of all unique elements in the bag `b` . without the overhead of calling **bags.attrib**. For example, the multiset bag('Curiosity' ~ 2, 'Skycrane' ~ 1) results to 2.

See also: **bags.attrib**.

bags.include (b, obj [, ···])

The function inserts all of the given elements `obj`, etc. into bag `b`.

The function returns nothing.

See also: **bags.minclude**, **bags.remove**, **tables.include**.

bags.minclude (b, obj)

The function inserts all of the given elements in the table, sequence or register `obj` into bag `b`. The function should be used instead of **bags.include** if the number of elements to be inserted exceeds Agena's argument stack.

The function returns nothing.

See also: **bags.include**, **bags.remove**.

bags.new ([···])

The function creates a new bag and optionally stores all of the given elements in it.

See also: **sykcrane.bagtable**.

bags.remove (b, obj [, ···])

The function removes all of the given elements `obj`, etc. from bag `b`. If the number of counts of the removed element reaches 0, the element will be deleted from the bag.

The function returns nothing.

See also: **bags.include**, **bags.minclude**.

10.9 bimaps - Bi-directional Maps

10.9.1 Introduction and Examples

The **bimaps** package implements a bi-directional map through tables. It is intended to hold items, i and j, that have a 1-to-1 relationship and allows to look up item j from table i and look up i from table j.

Examples:

```
> l, r := bimaps.bimap()

> l.foo := 1

> l.bar := 2

> l.spam := 'eggs'

> l:
bimap(bar ~ 2, foo ~ 1, spam ~ eggs)

> r:
bimap(1 ~ foo, 2 ~ bar, eggs ~ spam)

> l = r:
true
```

Note: When you try to assign a *different* value to an existing bimap key and the new value is also existing as a key, then an explicit error will be issued instead of just ignoring the attempted reassignment.

```
> l.foo := 2;
Error, cannot assign value '2' to key 'foo': already assigned to key 'bar'.
```

However, to change a value, you can completely delete the entry by nulling it and then assign a non-**null** value:

```
> l.foo := null

> l.foo := 3
```

For easy identification, all bimaps have the user-defined type `'bimap'`:

```
> typeof(l):
bimap
```

10.9.2 Functions and Metamethods

The functions are:

bimaps.attrib (bm)

Returns administrative information for bimap `bm`, see **environ.attrib** for details.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

bimaps.bimap ([tbl])

Creates two tables representing a bi-directional map. You can initialise the bimap by passing an optional table `tbl` with pre-defined values.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

bimaps.countitems (item, bm [, option])
bimaps.countitems (f, bm [, ···])

In the first form, counts the number of occurrences of an `item` in bimap `bm`. If you pass any `option`, then with numbers the function will conduct an approximate check, see **approx**.

In the second form, by passing a function `f` with a Boolean relation as the first argument, all elements in the bimap that satisfy the given relation are counted. If the function has more than one argument, then all arguments *except the first* must be passed right after `obj`.

The return is a number.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

See also: **countitems**.

bimaps.entries (bm)

Returns all entries in bimap `bm`, without invoking any metamethods.

See also: **bimaps.indices**.

bimaps.indices (bm)

Returns all indices in bimap `bm`, without invoking any metamethods.

See also: **bimaps.entries**.

bimaps.map (f, bm [, ···] [, inplace=true])

Maps a univariate or multivariate function `f` on all values in bimap `bm` and by default returns a new bimap with the result.

If you pass the `inplace` = **true** option, then the operation is done in-place, too, implicitly modifying the input bimap, too.

If f has only one argument, then only the function and the object will be passed to **bimaps.map**.

If function `f` has more than one argument, then all arguments except the first will be passed right after argument `bm`.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

Example:

```
> l, r := bimaps.bimap()

> l.foo, l.bar, l.spam  := 1, 2, 'eggs';
```

Add 10 to all numeric entries:

```
> bimaps.map(<: x -> if x :: number then x + 10 else x fi :>, l):
bimap(bar ~ 12, foo ~ 11, spam ~ eggs)
```

See also: **bimaps.remove**, **bimaps.select**, **bimaps.subs**.

bimaps.rawget (bm [, k])

Returns the underlying table in bimap `bm`, without invoking any metamethods, if no index `k` is given - or if `k` is given, returns entry `bm`[`k`] without invoking any metamethods.

```
> l, r := bimaps.bimap()

> l.foo, l.bar, l.spam  := 1, 2, 'eggs';

> l:
bimap(bar ~ 2, foo ~ 1, spam ~ eggs)

> bimaps.rawget(l):
[bar ~ 2, foo ~ 1, spam ~ eggs]
```

```
> bimaps.rawget(l, 'foo'):
1
```

bimaps.remove (f, bm [, ···] [, inplace=true])

Removes all values along with their keys from bimap `bm` that satisfy a condition determined by function `f` which should return **true** or **false**. By default, the return is a new bimap.

If you pass the `inplace` = **true** option, then the operation is done in-place, too, implicitly modifying the input bimap, and also returning the modified structure.

If `f` has more than one argument, then all arguments except the first will be passed right after argument `bm`.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

For example, to remove all entries from a bimap, in-place, that are not numbers:

```
> l, r := bimaps.bimap()

> l.foo, l.bar, l.spam  := 1, 2, 'eggs';

> bimaps.remove(<: x -> x -: number :>, l, inplace=true);

> l:
bimap(bar ~ 2, foo ~ 1)
```

See also: **bimaps.map**, **bimaps.select**.

bimaps.select (f, bm [, ···] [, inplace=true])

Selects all values along with their keys in bimap `bm` that satisfy a condition determined by function `f` which should return **true** or **false**. By default, the return is a new bimap.

If you pass the `inplace` = **true** option, then the operation is done in-place, too, implicitly modifying the input bimap, and also returning the modified structure.

If `f` has more than one argument, then all arguments except the first will be passed right after argument `bm`.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

For example, to select only those values that are numbers and remove all the other ones, in-place, issue:

```
> l, r := bimaps.bimap()
```

```
> l.foo, l.bar, l.spam  := 1, 2, 'eggs';

> bimaps.select(<: x -> x :: number :>, l, inplace=true);

> l:
bimap(bar ~ 2, foo ~ 1)
```

See also: **bimaps.map**, **bimaps.select**.

bimaps.subs (x:v [, ···], bm [, inplace=true])

Substitutes all occurrences of the value `x` in bimap `bm` with the value `v`, by default non-destructively.

More than one substitution pair can be given. The substitutions are performed sequentially.

If you pass the `inplace`=**true** option, then the operation is done in-place, too, implicitly modifying the input bimap, and also returning the modified structure.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

For example, to replace all values that are one with ten, and those that are two with twenty, non-destructively, issue:

```
> l, r := bimaps.bimap()

> l.foo, l.bar, l.spam  := 1, 2, 'eggs';

> bimaps.subs(1:10, 2:20, l):
bimap(bar ~ 20, foo ~ 10, spam ~ eggs)
```

See also: **bimaps.map**, **bimaps.subsop**.

bimaps.subsop (i:v [, ···], bm [, inplace=true])

The function replaces the value in bimap `bm` at the given index `i` with the new value `v`, by default non-destructively. More than one substitution pair can be given. The type of return is determined by the type of `obj`.

The function also allows to delete values, by setting `v` to **null**.

The function is written in the Agena language and is included in the lib/bimaps.agn file.

See also: **bimaps.subsop**.

The package also provides the following metamethods:

Metamethod	**Functionality**
`'__index'`	read operation, e.g. bm[p], with p any valid index
`'__writeindex'`	write operation, e.g. bm[p] := value, with p any valid index
`'__size'`	**size** operator, number of bi-directional pairs
`'__in'`	**in** operator
`'__notin'`	**notin** operator
`'__empty'`	**empty** operator
`'__filled'`	**filled** operator
`'__tostring'`	formatting for output at the console
`'__call'`	access to the underlying mapping table through a simple function call to the bimap with no arguments, e.g. bm().

10.10 heaps - Priority Queues

10.10.1 Introduction and Examples

The package implements skew heaps, `emulated` binary heaps and AVL trees which for example can be used as priority queues. A skew heap is a mostly unbalanced binary tree, usually avoiding costly reshuffles with each insert, whereas binary heaps and AVL trees are usually balanced - with extra cost for insertion.

The exact amortized complexity O(n) of all operations on a skew heap is known to be log(n, Phi), and the one for AVL and binary heaps is log(n, 2).

The package provides constructors (**avl.new**, **skew.new**, **binary.new**), metamethods (see table below), functions to insert new and replace existing values (**avl.include**, **skew.include**, **binary.include**), functions to remove the entry with the smallest index (**avl.remove**, **skew.remove**, **binary.remove**), and iterators (**avl.iterate**, **skew.iterate**, **binary.iterate**) that traverse the heaps in an ordered fashion - depending on the sorting method chosen at heap creation, which by default is in ascending order of the indices.

All the functions that work with skew heaps reside in the `skew` table, those for binary heaps are in table `binary`, and those for AVL trees in table `avl`.

The package operations on binary heaps and AVL trees are at least 25 times faster than on skew heaps. Many functions can also be used OOP-style.

```
> h := binary.new()

> h@@include(2, 'world')
> h@@include(1, 'hello')
> h@@include(10, 'everybody')

> k1, v1 := h@@remove()
> k2, v2 := h@@remove()
> k3, v3 := h@@remove()

> print(v1, v2, v3)
hello   world   everybody

> binary.include(h, 2, "world"); binary.include(h, 1, "hello");
> binary.include(h, 10, "everybody");

> f := binary.iterate(h);

> f():
1       hello

> f():
2       world

> f():
10      everybody
```

The skew heap functions have the same syntax and work the same.

10.10.2 Metamethods

The package provides the following metamethods for all three heap types:

Metamethod	Functionality
`'__writeindex'`	skew heaps and AVL trees only: write operation, e.g. n[p] := value, with p any valid non-null index
`'__size'`	**size** operator, number of key~value pairs in the heap
`'__in'`	**in** operator
`'__notin'`	**notin** operator
`'__empty'`	**empty** operator
`'__filled'`	**filled** operator

10.10.3 Binary Heap Functions

The package functions can also be called OOP-style, if not indicated otherwise.

`binary.entries (h)`

The function returns all entries in heap `h` in a new table. For the ordering, see **binary.iterate**.

See also: **binary.explore**, **binary.indices**.

`binary.explore (h)`

Explores the internal structure of binary heap h and returns a table with all its levels in various subtables. The key~value pair with the highest priority, at level 1, is always in the first subtable of the result, while all the other entries are at higher levels and will not necessarily be returned in the order when popped by **binary.remove**. All the key~value pairs are represented by pairs with the key at the left-hand side and the value at the right-hand side.

The function cannot be called OOP-style.

`binary.find (h, v)`

Searches for value `v` in the binary heap `h` and returns its index. If `v` is not in `h`, returns **null**.

See also: **binary.entries**, **binary.get**.

binary.get (h, k)

Returns the value stored at index `k` of binary heap `h`. If `k` is not an index in `h`, returns **null**.

See also: **binary.find**, **binary.indices**.

binary.include (h, [k,] v)

Inserts a new key~value pair into heap `h`. The key `k` and value `v` must be non-null. If `k` is omitted, then `v` is appended to `h` at index `h.length + 1`. The function returns nothing.

Note that if a tuple already exists with the same key `k`, it will not be deleted and the new one will just be inserted, but you cannot predict which one will be popped first when calling **binary.remove** unless you know exactly how insertion and deletion works internally.

See also: **binary.remove**.

binary.indices (h)

The function returns all indices in heap `h` in a new table. For the ordering, see **binary.iterate**.

See also: **binary.entries**.

binary.iterate (h)

The factory returns an iterator that with each call returns a key~value pair from heap `h`.

The function returns the key-value pairs in `h` in the same order as **binary.remove** does, with the highest priority pair returned first.

The function cannot be called OOP-style.

See also: **binary.explore**.

binary.new ([comparison])

Creates an empty binary heap with the default comparison method for various operations the `less than` relation, i.e.

<: k1, k2 -> k1 and type k1 = type k2 and k1 < k2 :>.

You might pass another comparison function to be used.

The function cannot be called OOP-style.

`binary.remove (h)`

Removes the key~value pair with the highest priority and returns this key and value.

If the heap is empty, the function just returns **null**.

See also: **binary.include**.

`binary.reorder (h)`

The function deletes all obsolete datasets in binary heap `h` - to free memory.

10.10.4 AVL Tree Functions

Note: AVL trees as implemented here can store values of any type, also mixed, but the keys are always negative, zero or positive integers.

AVL trees have an internal status table which you can access with **avl.getstore** or the `__call` metamthod.

The AVL tree functions and OOP methods are:

`avl.attrib (h)`

The function returns administrative information about an AVL tree: maximum height (key `maxheight`), number of key~value pairs currently included (key `length`) and the balance factor (key `balancefactor`).

`avl.entries (h)`

The function returns all entries in the AVL tree `h` in a new table, in ascending order of its respective indices.

See also: **avl.indices**.

`avl.get (h, k)`

Retrieves the value stored in AVL tree `h` at index `k`, an integer. If no value could be found, the function returns **null**.

See also: **avl.indices**.

`avl.getmax (h)`

The function returns the largest key along with its associated value from AVL tree `h`. The function does not remove the data from `h`.

See also: **avl.getmin**, **getminmax**, **avl.getroot**.

`avl.getmin (h)`

The function returns the smallest key along with its associated value from AVL tree `h`. The function does not remove the data from `h`.

See also: **avl.getmax**, **avl.getminmax**, **avl.getroot**.

`avl.getminmax (h)`

The function returns the smallest and largest key along with their associated values from AVL tree `h`. The function does not remove the data from `h`.

See also: **avl.getmax**, **getminmax**, **avl.getroot**.

avl.getroot (h)

The function returns the key along with its associated value from the root node of AVL tree `h`. The function does not remove the data from `h`.

avl.getstore (h)

The function returns the internal status table of AVL tree `h`, same as the expression `h`().

avl.include (h, [k,] v)

Inserts a new key~value pair into AVL tree `h`. Key `k`, if given, must be an integer; if `v` is **null** then the function will call **avs.remove**(`h`, `k`).

If `k` is not given, `v` is simply appended to `h` with the key set to **avl.getmax**(`h`) + 1.

The function returns nothing.

See also: **avl.remove**.

avl.indices (h)

The function returns all indices in AVL tree `h` in ascending order and returns them in a new table.

See also: **avl.entries**, **avl.get**.

avl.iterate (h)

The factory returns an iterator that with each call returns a key~value pair from AVL tree `h`, in ascending order of its indices.

The function issues an error when called as a method.

avl.new ()

Creates an empty AVL tree.

The function issues an error when called as a method.

avl.remove (h)

avl.remove (h, k)

avl.remove (f, h)

In the first form, removes the key~value pair with the smallest key from AVL tree `h` and returns the key and the value removed.

In the second form, deletes the given value referenced by key `k` from tree `h`, also returning the key and the associated value removed. `k` must be an integer.

If the key does not exist or the tree is empty, the function just returns **null**.

An OOP-style method call of this variant is supported.

In the third form, takes a function `f` expressing a condition and removes all values in tree `h` that satisfy this condition, in-place. There are no returns in this mode. Example:

```
> import heaps;

> h := avl.new();

> for i from 10 downto 0 do h@@include(i, i^2) od;

> avl.remove(<: x -> x > 25 :>, h);

> h@@entries():
[0, 1, 4, 9, 16, 25]
```

In the third form, you cannot call the function OOP-style.

See also: **avl.include**.

10.10.5 Skew Heap Functions

The package functions can also be called OOP-style, if not indicated otherwise.

skew.entries (h)

The function returns all entries in heap `h` in a new table. For the ordering, see **skew.iterate**.

See also: **skew.find**, **skew.indices**.

skew.find (h, v)

Searches for value `v` in the skew heap `h` and returns its index. If `v` is not in `h`, returns **null**.

See also: **skew.get**.

skew.get (h, k)

Returns the value stored at index `k` of skew heap `h`. If `k` is not an index in `h`, returns **null**.

See also: **skew.find**.

skew.height (h, k)

Returns the height of a key `k` in skew heap `h`, with 0 depicting that the key is in the root node.

The function cannot be used OOP-style.

skew.include (h, [k,] v)

Inserts a new key~value pair into the skew heap `h`. The key `k` and value `v` must be non-null. If `k` is omitted, then `v` is appended to `h` at index `h.length` + 1. The function returns nothing.

See also: **skew.remove**.

skew.indices (h)

The function returns all indices in heap `h` in a new table. For the ordering, see **skew.iterate**.

See also: **skew.entries**, **skew.get**.

skew.iterate (h)

The factory returns an iterator that with each call returns a key~value pair from heap `h`, in an ordered fashion.

The ordering is determined by the comparison function passed to **skew.new**, which by default is a `less than` comparison, so that the iterator returns the values in ascending order of its indices.

The function cannot be used as an OOP method.

skew.new ([comparison])

Creates an empty skew heap with the default comparison method for various operations the `less than` relation, i.e.

<: k1, k2 -> k1 and type k1 = type k2 and k1 < k2 :>.

You might pass another comparison function to be used.

The function cannot be used as an OOP method.

skew.remove (h)

Removes the key~value pair with the smallest key from heap `h` and returns the key and the value removed. If the key does not exist, the function just returns **null**.

See also: **skew.include**.

skew.reorder (h)

The function balances skew heap `h` internally by popping the node with the highest priority and then re-inserting it. This is just for maintenance, you do not have to run this function before executing any other package function.

10.11 rbtree - Red-Black Trees

The package allows to create binary search trees for numbers and assures that when inserting a number, all its elements are in ascending order thereafter. The package has been added primarily to guarantee that the internal red-black tree C implementation works fine and is safe, otherwise it might be of use only in some special situations.

The package provides the constructor **rbtree.new**, functions to insert new and remove existing values - **rbtree.include**, **rbtree.remove** -, an iterator **rbtree.iterate** that traverses the tree and some other helpful functions.

A red-black tree as implemented here stores a number only once - all duplicates are ignored when trying to insert them.

Create a new red-black tree a:

```
> a := rbtree.new();
```

Try to insert numbers 1 to 10 in descending order:

```
> for i from 10 downto 1 do
>    rbtree.include(a, i)
> od;
```

All the elements in a are in ascending order:

```
> rbtree.entries(a):
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Check whether ten is in the tree, and whether eleven is, as well:

```
> 10 in a, 11 in a:
true    false
```

Alternatively, use **rbtree.find** to search any number:

```
> rbtree.find(a, 0):
false
```

Is the structure empty or filled ?

```
> empty a, filled a:
false   true
```

Remove ten from a:

```
> rbtree.remove(a, 10);
```

The current number of elements in a now is:

```
> size a:
9
```

Iterate through the entire tree:

```
> f := rbtree.iterate(a);

> f():
1

> f():
2
```

etc.

10.11.1 Metamethods

The package provides the following metamethods for all three heap types:

Metamethod	Functionality
`'__size'`	**size** operator
`'__in'`	**in** operator
`'__notin'`	**notin** operator
`'__empty'`	**empty** operator
`'__filled'`	**filled** operator
`'__tostring'`	pretty-printer

10.11.2 Functions

The package functions can also be called OOP-style, if not indicated otherwise.

`rbtree.entries (t)`

The function returns all the numbers in tree `t` in a new table, in the same order as currently represented by the tree, that is ascending.

See also: **rbtree.iterate**.

`rbtree.find (t, x)`

Searches for number `x` in tree `t` and returns two results: A Boolean indicating whether it has been found, and the height of the number in the tree, which is 0 on failure.

See also: **in** operator.

rbtree.include (t, x)

Inserts number `x` into the tree `t`. Returns **true** on success and **false** otherwise - which should not happen.

See also: **rbtree.remove**.

rbtree.iterate (t)

Returns an iterator function that when called returns one number after another from red-black tree `t`. If there are no more elements left, the iterator function returns **null**.

Example usage:

```
> f := rbtree.iterate(t);

> while x := f() do
>    print(x)
> od;
...
```

The traversal has been finished, there is no more element left.

```
> f():
null
```

The function cannot be called OOP-style.

See also: **rbtree.entries**.

rbtree.min (t)

Returns the smallest number in the tree, in O(1) time.

See also: **rbtree.max**, **rbtree.minmax**.

rbtree.minmax (t)

Returns the smallest and largest number in the tree, in O(1) time.

See also: **rbtree.max**, **rbtree.min**.

rbtree.max (t)

Returns the largest number in the tree, in O(1) time.

See also: **rbtree.min**, **rbtree.minmax**.

rbtree.new ()

The function creates a new red-back tree.

The function cannot be called OOP-style.

See also: **rbtree.include**, **rbtree.remove**.

rbtree.remove (t, x)

rbtree.remove (f, t)

In the first form, deletes number `x` from the tree `t` and returns **true** on success and **false** if the element to be deleted could not be found in the tree. An OOP-style method call of this variant is supported.

In the second form, takes a function `f` expressing a condition and removes all numbers in tree `t` that satisfy this condition, in-place. The return is **true** if at least one number has been deleted, and **false** otherwise. Example:

```
> t := rbtree.new();

> for i from 10 downto 0 do t@@include(i) od;  # insert number 0 to 10

> rbtree.remove(<: x -> x in {0, 10} :>, t):
true

> rbtree.entries(t):
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In the second form, you cannot call the function OOP-style.

See also: **rbtree.include**.

10.12 bfield - Bit Fields

The package provides lean, low-level functions to work with memory-saving bit fields. The functions are generally faster than those implemented in the **memfile** package.

Typical usage:

Create a bit field of at least ten bits, which is internally rounded up to 16 bits, as 16 is a multiple of eight whereas ten is not:

```
> m := bfield.new(10);
```

The field is by default pre-filled with zeros. If you want to preset other values, like 255 to set all bits to 1, enter:

```
> m := bfield.new(10, 0xff);
```

The actual size of the field is:

```
> size m:
16
```

Get some bits:

```
> m[1], m[9]:
1       1
```

Set some bits to zero:

```
> m[1] := 0; m[9] := 0;

> m[1], m[9]:
0       0
```

The package provides the following metamethods:

Metamethod	Functionality
`'__index'`	read operation, e.g. n[p], with p an index counting from 1; reads a bit, not a byte
`'__writeindex'`	write operation, e.g. n[p] := value, with p the index, counting from 1; sets a bit, not a byte
`'__size'`	**size** operator, number of bits in the field
`'__zero'`	**zero** operator, checks whether all field bits are set to zero
`'__tostring'`	formatting for output at the console; returns binary representations
`'__gc'`	garbage collection

The following bit field functions can also be called OOP-style:

bfield.clearbit (bitfield, n)

Clears a bit, i.e. sets absolute bit position `n` in the `bitfield` to 0. `n` counts from 1.

The function returns nothing.

See also: **bfield.flipbit**, **bfield.getbit**, **bfield.setbit**, **bfield.setbitto**, **bfield.setbyte**.

bfield.flipbit (bitfield, n)

Flips the bit stored at absolute bit position `n` in the `bitfield`: if the current bit is 1, it is set to 0, and vice versa. `n` counts from 1.

The return is the bit value after flipping, either 1 or 0.

See also: **bfield.clearbit**, **bfield.setbit**.

bfield.getbit (bitfield, n)

Returns the bit stored at absolute bit position `n` in the `bitfield`. `n` counts from 1.

The return is either 1 or 0.

See also: **bfield.clearbit**, **bfield.setbit**.

bfield.getbyte (bitfield, pos [, option])

From `bitfield`, returns the *byte* at position `pos`, with `pos` > 0. The return is an integer in the range [0, 255].

See also: **bfield.getbit**, **bfield.getbytes**.

bfield.new (n, [, val])

Creates a bit field of at least `n` bits. If val is not given, then every *byte* in the field is set to zero. If `val` - a non-negative integer preferably in the range [0, 255] - is given, then every *byte* in the field is filled with it.

The number of bits actually allocated is always a multiple of 8, i.e. the field is filled up to whole bytes.

Do not call this function OOP-style, it will issue an error.

See also: **bfield.resize**.

bfield.resize (bitfield, n [, val])

Resizes the `bitfield` to exactly `n` bits, with `n` > 0. It can grow or shrink a bit field and in the latter case preserves the remaining content. If the bit field is to be enlarged, the function fills the new space with zeros if the third argument `val` is not given, otherwise the added *bytes* are set to the non-negative integer `val`, which should be in the range [0, 255].

The size of the modified bit field is always a multiple of 8.

See also: **bfield.new**.

bfield.setbit (bitfield, n)

Sets absolute bit position `n` in the `bitfield` to 1. `n` counts from 1. To clear a bit, use **bfield.clearbit**.

The function returns nothing.

See also: **bfield.clearbit**, **bfield.flipbit**, **bfield.getbit**, **bfield.setbyte**.

bfield.setbitto (bitfield, n, val)

Sets absolute bit position `n` in the `bitfield` to 1 the `val`, a non-negative integer preferably in the range [0, 255]. `n` counts from 1.

The function returns nothing.

See also: **bfield.clearbit**, **bfield.getbit**, **bfield.setbyte**.

bfield.setbyte (bitfield, pos, val)

Sets `val`, a non-negative integer preferably in the range [0, 255] into `bitfield` at *byte* position pos, with `pos` > 0.

The function returns nothing.

See also: **bfield.getbyte**, **bfield.setbit**.

10.13 tuples - Closures Storing Data

The package provides functions to work with closures that store any kind of data.

Data stored in closures usually are called `upvalues`. Depending on the platform, the increase in speed when reading or writing upvalues is zero to nine percent compared to tables, with generally less memory required.

To store data in a closure, call **tuples.new**:

```
> t := tuples.new(10, 20, 30);
```

To return all values, just pass no arguments.

```
> t():
10      20      30
```

To retrieve an item at a position, issue

```
> t(1):
10
```

or use square brackets, which is much faster:

```
> t[1]:
10
```

A tuple cannot be extended or shrunk, but values can be replaced by using the notation:

```
> t[1] := 0;
```

However, you may use the **union** operator to concatenate tuples.

The package provides the following metamethods:

Metamethod	Functionality
'__index'	read operation, e.g. n[p], with p an index counting from 1
'__writeindex'	write operation, e.g. n[p] := value, with p the index, counting from 1
'__size'	**size** operator, number of elements in a tuple
'__zero'	**zero** operator, checks whether all tuple elements are set to zero
'__nonzero'	**nonzero** operator, checks whether all elements in the tuple are non-zeros
'__in'	checks whether an element is stored in a tuple
'__notin'	checks whether an element is not part of a tuple
'__intersect'	returns the intersection of two tuples; also copies the metatable and type of the first operand to the result

Metamethod	Functionality
`'__minus'`	returns all elements in the first tuple that are not in the second one; also copies the metatable and type of the first operand to the result; if the right-hand side operand is not a tuple, returns a new tuple without it
`'__union'`	returns a tuple with all the elements of the first and the second tuple; also copies the metatable and type of the first operand to the result; if the left-hand side is not a tuple, returns a new tuple with the right-hand side the first element (prepending); if the right-hand side is not a tuple, returns a new tuple with the right-hand side its last element (appending)
`'__iter'`	iterator for **for/in** loops
`'__eq'`	= equality operator
`'__eeq'`	== equality operator
`'__aeq'`	~= approximate equality operator
`'__tostring'`	formatting for output at the console

There is no __gc method for tuples are automatically collected like functions by Agena.

You can explicitly set a user-defined type to a tuple with **settype**, and add or overwrite existing metamethods with **addtometatable**. You cannot, however, replace the metatable of a tuple with **setmetatable** in order to prevent the garbage collector messing up memory.

With the exception of **tuples.new**, all of the following package functions can be used OOP-style:

`tuples.getitem (a, i)`

With any tuple, returns the value at `a`[`i`], where `i`, the index, is an integer counting from 1. The function has been provided to avoid the index metamethod overhead.

See also: **tuples.setitem**.

`tuples.getsize (a)`

Returns the number of values stored in tuple `a`.

`tuples.isall (a, type)`

Checks whether all elements in tuple `a` are of a given `type`. Eligible types that the function accepts are `'number'`, `'integer'` (numbers that are all integral), `'complex'`, `'string'` and `'boolean'`. Also supported are `'posint'` (positive integers), `'positive'` (positive numbers), `'nonnegint'` (non-negative integers), `'nonzeroint'` (non-zero integers) and `'nonnegative'` (non-negative numbers).

tuples.map (f, a [, ···] [, inplace=true])

Like **map**, applied on tuple `a`. When not given the `inplace = true` option, copies the metatable and user-defined type of `a` to the new tuple.

tuples.iterate (a [, i [, p [, true]]])

Returns an iterator function that when called returns the next value in tuple `a`, or **null** if there are no further entries in the tuple.

If an index `i` is passed, the first call to the iterator function returns the `i`-th element in the tuple and with subsequent calls, the respective elements after index `i`.

You may also pass a positive integer step `p` to the iterator function: If given, then in subsequent calls the `p`-th element after the respective current one will be returned, equivalent to giving an optional step size in numeric for loops. Example:

```
> f := tuples.iterate(t, 3, 2)

> f():
30

> f():
50

> f():
70

> f():
null
```

If the fifth argument **true** is given, the iterator will return both position (index) and value. When the end of a tuple has been reached, it returns **null** with the next call, and then starts all over from the beginning (or `p`).

tuples.new automatically adds an iterator to the tuple generated, so you can directly loop over them, which technically are closures, in **for**/**in** loops:

```
> for i, j in t do print(i, j) od
1       10
2       20
3       30
4       40
```

tuples.member (x, a [, eps])

Searches `x` in tuple `a` and if successful returns the index of the first hit, otherwise returns **null**.

If you pass any positive value for `eps`, then the function will conduct an approximate check with the given epsilon value, see **approx**. **Eps** may be an appropriate tolerance.

See also: **in** operator.

tuples.new ([a [,···]])

Creates a tuple with all the given arguments, limited to 60 entries. The return is a closure. The function can create an empty tuple, as well, if you should need one.

tuples.remove (f, a [, ··· [, inplace=true])

Like **remove**, applied on tuple `a`. When not given the `inplace = true` option, copies the metatable and user-defined type of `a` to the new tuple.

tuples.select (f, a [, ··· [, inplace=true])

Like **select**, applied on tuple `a`. When not given the `inplace = true` option, copies the metatable and user-defined type of `a` to the new tuple.

tuples.setitem (a, i, v)

With any tuple, sets value `v` to `a[i]`, where `i`, the index, is an integer counting from 1.

tuples.subs (x:v [, ···], a [, true])

Like **subs**, applied on tuple `a`.

tuples.toreg (a)

Puts all elements in tuple `a` into a register and returns it.

tuples.toseq (a)

Puts all elements in tuple `a` into a sequence and returns it.

tuples.totable (a)

Puts all elements in tuple `a` into a table and returns it.

tuples.tostring (a)

"Pretty-prints" tuple `a`, returning a string including the number of elements in `a`.

tuples.unpack (a)

Similar to **unpack**, returning all the elements in tuple `a`.

10.14 lookup - Lookup Tables

The package implements simple lookup tables, accompanied by functions to insert and delete values, modify its entries and inspect them. Technically, lookup tables have been implemented as userdata storing size information and an Agena table that contains all the actual data. This table stores key-value pairs as usual with the respective values residing either in subtables or subsets.

The package, similar to the **bags** package, provides an easy means to set up lookup tables without having to care for the details, that is table management, especially when a new key or a new value is to be inserted or deleted. Just set up a lookup table by executing, for example:

```
> zips := lookup.new();
```

for a table storing cities and their various ZIP codes - with the cities, represented as strings, the keys, and the ZIP codes stored in subtables or subsets as values. The statement above creates a lookup table with subtables, if you want to use subsets, issue:

```
> zips := lookup.new(0, 0, 'set');
```

Use **lookup.include** to set up new cities (as keys) and their respective ZIP codes (as values) initially:

```
> lookup.include(zips, 'Antonio', 78201);

> lookup.include(zips, 'Nooga', 37341, 37343, 37350);

> lookup.gettable(zips):
[Antonio ~ {78201}, Nooga ~ {37341, 37343, 37350}]
```

Check what we have at index 'Antonio', in two different ways:

```
> lookup.gettable(zips, 'Antonio'), zips['Antonio']:
{78201}   {78201}
```

Get all the indices:

```
> lookup.indices(zips):
[Antonio, Nooga]
```

Check whether ZIP code 37350 is in one of the subtables:

```
> 37350 in zips:
true
```

Traverse the table:

```
> lookup.nextone(zips, null):
Antonio   {78201}

> lookup.nextone(zips, 'Antonio'):
Nooga     {37341, 37343, 37350}

> lookup.nextone(zips, 'Nooga'):
null
```

See also function **lookup.iterate**.

Map a function on all elements, in-place:

```
> lookup.map(<: x -> 2*x :>, zips):
[Antonio ~ {156402}, Nooga ~ {74682, 74686, 74700}]
```

Substitute values, also in-place:

```
> lookup.subs(156402:0, 74682:0, zips):
[Antonio ~ {0}, Nooga ~ {0, 74686, 74700}]
```

Get the number of all indices and of all table values, summed up, respectively:

```
> lookup.getsizes(zips):
2       4
```

Delete the entire entry indexed by 'Antonio':

```
> lookup.purge(zips, 'Antonio'):
true
```

Delete 0 from zips.Nooga:

```
> lookup.purge(zips, 'Nooga', 0):
true

> lookup.gettable(zips), lookup.getsizes(zips):
[Nooga ~ {74686, 74700}]        1       2
```

lookup.gettable allows to modify the table via the table reference returned. If you add or delete new values this way, do not forget to set the new sizes for the number of indices and entries after each such operation (or better: don't do it, always use **new.include** and **net.purge**):

```
> lookup.setsizes(zips, 1, 4);  # one key, four values
```

10.14.1 Metamethods

The package provides the following metamethods for all three heap types:

Metamethod	Functionality
`'__index'`	read-access indexing
`'__size'`	**size** operator
`'__in'`	**in** operator
`'__notin'`	**notin** operator
`'__empty'`	**empty** operator
`'__filled'`	**filled** operator
`'__tostring'`	pretty-printer

10.14.2 Functions

`lookup.checkarray (···)`

The function checks whether all its arguments are lookup tables, issues an error if one is not and simply returns otherwise, not returning anything.

See also: **lookup.istable**.

`lookup.getsizes (a)`

Returns the number of indices in lookup table `a` and the total number of elements in all its subtable entries, in this order.

See also: **lookup.setsizes**.

`lookup.gettable(a [, k])`

When given just one argument `a`, the lookup table, returns a reference to its internal table data structure. If `k` is given, returns the index entry `a[k]`.

`lookup.include (a, k, v0 [, v1, ···])`

The function sets values into lookup table `a`: `a[k]` := `[v0, v1, ···]` and returns nothing. If `a[k]` already exists, `v0, v1, ···` are appended to the existing subtable or subset.

See also: **lookup.purge**.

`lookup.indices (a [, true])`

The function returns all indices in lookup table `a`. The result is unsorted when given just one argument.

If **true** is given as a second argument, only indices with integral keys are returned, in ascending order.

See also: **lookup.nextone**.

lookup.isarray (···)

The function checks whether all its arguments are lookup tables and returns **true** or **false** for each argument.

See also: **lookup.checktable**.

lookup.iterate (a [, key])

Returns an iterator function that when called returns one key ~ value pair after another from lookup table `a`. If there are no more elements left, the iterator function returns **null**.

If `key` is given, the function starts iteration with the key ~ value pair following `key` in the chain. `key` is **null** by default, meaning an initial key is determined internally to traverse the whole lookup table.

The order in which the indices are enumerated is not specified, *even for numeric indices.*

Example usage:

```
> a := lookup.new();

> lookup.include(a, 'abc', 1, 2, 3, 4)

> lookup.include(a, 'xyz', -1, -2, -3, -4)

> f := lookup.iterate(a);

> while x := [f()], filled x do
>    print(x)
> od;
```

See also: **lookup.nextone**.

lookup.map (f, a, [, ···])

Maps a univariate or multivariate function `f` to all the values in lookup table `a`, in-place. Put the second, third, etc. argument to `f` right after `a`.

The function is written in Agena and included in the lib/library.agn file.

See also: **lookup.subs**.

lookup.new ([la [, lh [, 'set']]])

Creates a new lookup table with `la` pre-allocated slots in the array part and `lh` slots in the hash part. By default, `la` and `lh` are both zero.

By default all the values later on added are stored in subtables. If you pass the optional third argument 'set', a string, then they are stored in subsets, making search and deletion very fast. The return is a userdata value.

See also: **new.include**, **lookup.purge**.

lookup.nextone (a [, index [, sentinel]])

Allows a programme to traverse all fields of lookup table `a`. Its second argument is an index in the structure.

The function returns the next index of the structure and its associated value. When called with **null** as its second argument, **lookup.nextone** returns an initial index and its associated value. When called with the last index, or with **null** in an empty structure, **lookup.nextone** returns **null**.

If the second argument is absent, then it is considered **null**, thus starting with the initial index. In particular, you can use **lookup.nextone**(`a`) to check whether a table is empty. However, it is recommended to use the **filled** operator for this purpose.

If the third optional argument `sentinel` is given, and if **lookup.nextone** during traversal encounters an element that equals this `sentinel`, the function just returns **null**, and you may start iterating the structure again from its beginning.

The order in which the indices are enumerated is not specified, even for numeric indices.

See also: **lookup.indices**.

lookup.purge (a, k [, v])

With only `a`, `k` given, the function deletes lookup table entry `a`[`k`].

If `a`, `k` and `v` are given, deletes all occurrences of value `v` from the lookup table entry `a`[`k`]. If `a`[`k`] after the deletion is empty, it will be entirely removed.

The function returns **false** if `k` is not a valid lookup table key or if `v` has not been found at `a`[`k`]. Otherwise the result is **true** to indicate successful deletion.

See also: **lookup.include**.

lookup.setsizes (a, lk [, lv])

Sets the number `lk` of indices and the total number of elements `lv` in lookup table `a`. If `lv` is not given the total number of elements is not changed.

See also: **lookup.getsizes**.

lookup.subs (x:v [, ···], a)

Substitutes all occurrences of the value `x` in the subtables of lookup table `a` with the value `v`, destructively. More than one substitution pair can be given. The substitutions are performed sequentially and by default simultaneously starting with the first pair.

The function is written in Agena and included in the lib/library.agn file.

See also: **lookup.map**.

10.15 fifo - First-in First-Out Queues

The package implements first-in first-out queues, internally implemented as memory-efficient circular queues which are grown in size automatically if necessary.

Examples:

```
> q := fifo.new();

> fifo.include(q, 10);

> fifo.include(q, 20);

> fifo.include(q, 30);

> fifo.remove(q):
10

> fifo.remove(q):
20

> fifo.remove(q):
30

> fifo.remove(q):
null
```

The package also includes functions to explore and search the queue, for example:

```
> fifo.include(q, 10);

> 10 in q, filled(q), size(q):
true    true    1
```

All the functions except of **fifo.new** can be called OOP-style:

```
> q@@remove():
10
```

Each queue has its own internal status table which you can retrieve in two ways:

```
> fifo.getstore(q):
[]
```

or:

```
> q():
[]
```

Calling the queue like a function also allows to retrieve and write values. Let's write some values into the status table:

```
> q(1, 'a')

> q(2, 'b')
```

Let's read the values:

```
> q(1), q(2):
a       b
```

10.15.1 Metamethods

The package provides the following metamethods:

Metamethod	Functionality
`'__size'`	**size** operator, number of elements currently in the queue
`'__in'`	**in** operator
`'__notin'`	**notin** operator
`'__empty'`	**empty** operator, returns **true** if no element is in the queue
`'__filled'`	**filled** operator, returns **true** if at least one element is in the queue
`'__tostring'`	pretty-printer
`'__call'`	retrieves internal status table
`'__gc'`	garbage collection

All functions but **fifo.new** can be called OOP-style.

10.15.2 Functions

`fifo.attrib (q)`

The function returns information about the state of queue q in a table:

- `'length'`: current number of elements in the queue.
- `'maxsize'`: current number of allocated slots in the queue.
- `'isempty'`: indicator whether the queue is currently empty.
- `'isfull'`: indicator whether the queue is currently full.
- `'head'`: the first element in the queue, counting from zero, and the first one to be popped with the next call to **fifo.remove**.
- `'tail'`: index of the next free slot in the queue, counting from zero.
- `'length'`: current number of elements in the queue.

fifo.getstore (q)

Returns a reference to the queue's internal status table. Same as the expression q(). See example in the introduction.

fifo.has (q, v)

Checks whether queue q has element v and returns **true** or **false**. Same as the expression v **in** q.

fifo.include (q, v)

Inserts the element v into queue q. v may be of any type. The function returns nothing. The function automatically grows the queue if needed.

See also: **fifo.remove**.

fifo.new ([initsize])

The function creates a new first-in first out queue and returns a userdata object. By default, the number of pre-allocated slots is 4, but you can change this to any other positive size by passing initsize, a positive integer, which is always rounded up to the next multiple of 4 if initsize is less than that.

fifo.peek (q)

The function returns the bottom element in queue q but does not pop it.

fifo.remove (q)

The function pops the bottom element from queue q and returns it. The function does not shrink internally allocated memory, but you may call **fifo.shrink** manually.

See also: **fifo.include**.

fifo.shrink (q)

If possible, shrinks the number of pre-allocated slots in queue q to the smallest power of two, but at least eight slots, freeing memory to be reused by the interpreter. Returns **true** on success plus the new number of pre-allocated slots, and **false** otherwise.

The function works non-destructively so all elements currently in the queue will be preserved.

See also: **fifo.remove**.

10.16 lifo - Last-in First-Out Queues

The package implements last-in first-out queues. Examples:

```
> q := lifo.new();

> lifo.include(q, 10);

> lifo.include(q, 20);

> lifo.include(q, 30);

> lifo.remove(q):
30

> lifo.remove(q):
20

> lifo.remove(q):
10

> lifo.remove(q):
null
```

The package also includes functions to explore and search the queue, for example:

```
> lifo.include(q, 10);

> 10 in q, filled(q), size(q):
true    true    1
```

All the functions except of **lifo.new** can be called OOP-style:

```
> q@@remove():
10
```

Each queue has its own internal status table which you can retrieve in two ways:

```
> lifo.getstore(q):
[]
```

or:

```
> q():
[]
```

Calling the queue like a function also allows to retrieve and write values. Let's write some values into the status table:

```
> q(1, 'a')

> q(2, 'b')
```

Let's read the values:

```
> q(1), q(2):
a       b
```

10.16.1 Metamethods

The package provides the following metamethods:

Metamethod	Functionality
'__size'	**size** operator, number of elements currently in the queue
'__in'	**in** operator
'__notin'	**notin** operator
'__empty'	**empty** operator, returns **true** if no element is in the queue
'__filled'	**filled** operator, returns **true** if at least one element is in the queue
'__tostring'	pretty-printer
'__call'	retrieves internal status table
'__gc'	garbage collection

All functions but **lifo.new** can be called OOP-style.

10.16.2 Functions

lifo.attrib (q)

The function returns information about the state of queue q in a table:

- 'top': the next free slot in the queue, counting from zero.
- 'length': current number of elements in the queue.
- 'maxsize': current number of allocated slots in the queue.
- 'isempty': indicator whether the queue is currently empty.
- 'isfull': indicator whether the queue is currently full.

lifo.getstore (q)

Returns a reference to the queue's internal status table. Same as the expression q(). See example in the introduction.

lifo.has (q, v)

Checks whether queue q has element v and returns **true** or **false**. Same as the expression v **in** q.

lifo.include (q, v)

Inserts the element `v` into queue `q`. `v` may be of any type. The function returns nothing. The function automatically grows the queue if needed.

See also: **lifo.remove**, **lifo.shrink**.

lifo.new ([initsize])

The function creates a new last-in first out queue and returns a userdata object. By default, the number of pre-allocated slots is 8, but you can change this to any other positive size by passing `initsize`, a positive integer, which is always rounded up to the next multiple of 4 if `initsize` is less than that.

lifo.peek (q)

The function returns the bottom element in queue `q` but does not pop it.

lifo.remove (q)

The function pops the bottom element from queue `q` and returns it. The function never shrinks the size of the queue, see **lifo.shrink**.

See also: **lifo.include**.

lifo.shrink (q)

Shrinks the maximum size of a queue to the smallest possible multiple of 4, but eight slots at the minimum. All elements in the queue will be preserved so it is working non-destructively.

The function returns **true** on success and **false** otherwise, plus the new number of pre-allocated slots.

10.17 trie - Prefix and Suffix Trees

The package implements prefix and suffix trees, also called `tries`, implemented as binary crit-bit trees. Tries store zero, one or more strings and you can search them for all the entries that start with a given prefix or end in a suffix. The package is part of the main Agena library.

Create a new prefix trie:

```
> t := trie.new();
```

Insert strings into the trie:

```
> for i in ['England', 'Englishman', 'English', 'Edmonton'] do
>    t@@include(i)
> od;
```

Find all the entries that match the prefix 'en':

```
> t@@find('En'):
seq(England, English, Englishman)
```

To create a suffix trie and search for all the words that end in the letter 'n' issue:

```
> t := trie.new('suffix');

> for i in ['England', 'Englishman', 'English', 'Edmonton'] do
>    t@@include(i)
> od;

> t@@find('n'):
seq(Edmonton, Englishman)
```

Tries as implemented here are space-efficient and are relatively fast at finding all strings that share a common prefix. The search time is proportional to the length of the prefix, not the number of strings in the data set. Furthermore, lookups (on prefix trees) yield strings in lexicographical (alphabetical) order making it easy to perform operations on sorted data.

Tries are also an excellent choice for word dictionaries if you just want to lookup entire words, weighing memory consumption and speed. Both the **in** operator and the **trie.has** function, for example, check whether an entire string, and not a prefix or suffix, is part of a dictionary at extraordinary speed in a trie of words that consumes only a fraction of memory that string tables, sequences or registers need.

```
> 'Edmonton' in t, 'Edmon' in t:
true  false
```

However, searching for a string stored to an ordinary set is around six times faster than a lookup in a trie, while a trie consumes only around one percent of memory

with real-world word dictionaries than an Agena set does. You may use **environ.used** to compare memory consumption.

To speed up lookup times, you may consider plugging in a Bloom filter (see Chapter 9.6) into your application that first searches the Bloom filter and depending on its result searches the trie itself. Experiments show, however, that this will only make sense if you expect a very large percentage of mismatches, say 85 percent or more.

10.17.1 Metamethods

The package provides the following metamethods:

Metamethod	Functionality
`'__size'`	**size** operator, number of strings in the trie
`'__in'`	**in** operator, see **trie.has** for further information
`'__notin'`	**notin** operator
`'__empty'`	**empty** operator, returns **true** if no string is in the trie
`'__filled'`	**filled** operator, returns **true** if at least one string is in the trie
`'__tostring'`	pretty-printer
`'__call'`	retrieves internal status table
`'__gc'`	garbage collection

All functions but **trie.new** can be called OOP-style.

10.17.2 Functions

`trie.attrib (t)`

The function returns information about the state of trie `t` in a table:

- `'length'`: current number of strings in the trie.
- `'isempty'`: indicator whether the queue is currently empty.
- `'nodes'`: number of nodes in `t` (experimental).
- `'suffixtree'`: Boolean indicator whether `t` is a suffix tree.

`trie.find (t [, pat [, true]])`

Finds all the strings in trie `t` that match the given prefix or suffix `pat`, a string, and returns them in alphabetical order in a sequence.

When called without `pat`, the function returns all strings in `t`. Alternatively you can pass the empty string for `pat` to also dump the entire contents.

If there is no match, the function returns **null**.

If you pass the third argument **true**, then the function checks for an exact match, returning either **true** or **false**. This can reduce search times by around 33 percent. All this is equivalent to calling **trie.has**.

trie.getstore (t)

Returns a reference to the trie's internal status table. Same as the expression t(). See example in the introduction of the **lifo** package in Chapter 10.16.

trie.has (t, v)

Checks whether the given string or number v is stored in trie t and returns **true** or **false**. If v is a number it is converted to a string before the search starts. The function searches for the entire string, not just a prefix or suffix match. Calling **trie.has** is the same as the expression v **in** t.

See also: **trie.find**.

trie.include (t, v)

Inserts number or string v into trie t. If v is a number, it is converted to a string before insertion.

The function returns 2 if v is a new element, 1 if v has already been stored in t and 0 if there was a failure during insertion, for example because of insufficient memory.

See also: **trie.remove**.

trie.new ([suffix])

With no argument given, the function creates a prefix trie and returns it. With any argument given, returns a suffix trie.

The trie has an internal status table that you can access with **trie.getstore**.

See also: **trie.include**.

trie.remove (t, v)

Removes the number or string v from trie t and returns **true** on success and **false** on failure, that is v is not part of t.

See also: **trie.include**.

10.18 numcuckoo & numfilter - Cuckoo Filters for Numbers

The **numcuckoo** and **numfilter** packages are part of the standard distribution.

10.18.1 Introduction

The **numcuckoo** package implements a Cuckoo filter for Agena numbers. It is a space-efficient data structure using 32-bit fingerprints designed to answer approximate membership queries:

- Is a number *definitely not* part of a dictionary ?
- Is a number *likely* to be part of a dictionary ?

You can add to and delete entries from a **numcuckoo** filter, and it automatically resizes to take more values.

The false positive probability rate is around 0.00255 %. The false negative probability rate is 0 % as long as you do not remove values from the filter, see **numcuckoo.remove**.

Compared to sets and numarrays, you will save around 45 % and 37 % of required memory, respectively. Insertion and lookup is dozens of times faster, too.

Similarly, a **numfilter** also works with Agena numbers, but using 16-bit fingerprints. The filter is of a fixed size, you cannot grow it and cannot remove values. The false positive probability rate is around 0.0122 % (8 times as much as with numcuckoo filters) and compared to sets and numarrays, you will save around 90 % and 75 % of memory. Despite using 16-bit fingerprints, you can store a large amounts of numbers with this type of filter without having too many false positives, provided the filter is properly sized.

Create a filter with an initial number of 1,024 hash entries:

```
> c := numcuckoo.new(1024);
```

Insert a number:

```
> numcuckoo.include(c, Pi);
```

Find it:

```
> numcuckoo.find(c, Pi):
true
```

Remove the number:

```
> numcuckoo.remove(c, Pi);
```

And try to find it again:

```
> numcuckoo.find(c, Pi):
false
```

The package supports the **in** and **notin** metamethods, see **numcuckoo.find**, as well as the **size** operator, that is the number of words added to the filter.

Likewise, the **empty** and **filled** operators check whether the filter is empty or includes at least one word.

For a Cuckoo filter handling strings, see package **cuckoo**.

The **numfilter** package has the same functions as the **numcuckoo** package, with the same names, arguments and returns, except for the **remove** function, so they are not explicitly listed below.

10.18.2 Functions

The package functions can also be called OOP-style, if not indicated otherwise.

numcuckoo.attrib (c)

Returns various information on the NumCuckoo filter `c`.

numcuckoo.find (c, n)

Tries to find number `n` in NumCuckoo filter `c`. It returns **false** if `n` is definitely not in the filter, and **true** if it is likely that `n` is part of the filter.

numcuckoo.hash (n [, m [, fingeronly]])

Returns the high and lower unsigned 32-bit words of the SplitMix64-based hash for number `n`, plus the 16-bit and 32-bit unsigned fingerprints of `n`. These are essential data used internally by the package.

If `m` is given all the four hashes are taken modulo `m`. The default is 0, that is not taking the modulo.

If optional `fingeronly` is set to **true**, only the 16-bit unsigned integer fingerprint of `n` will be returned - the default is **false**.

numcuckoo.include (c, n)

Inserts number `n` into NumCuckoo filter `c`. It returns **true** on success and **false** otherwise. If the maximum number of entries has been exceeded, the function automatically enlarges it.

Note: **numfilter.include** cannot enlarge a filter and will issue an error if the filter is full.

numcuckoo.new ([m [, n])

Creates and returns a NumCuckoo filter, of type userdata.

`m` depicts the initial number of slots and by default is 8. `n` is the maximum number of kicks, defaulting to 500.

The function cannot be used as a method and you cannot give the number of kicks with **numfilter**s.

numcuckoo.remove (c, n)

Deletes number `n` from Cuckoo filter `c`. It returns **true** on success and **false** if `n` has not been found.

Warning: If another number with the same fingerprint is in the filter, the function will subsequently return **false**, too.

Values cannot be removed from **numfilter**s.

10.19 dblhash - Number Hash Sets

The **dblhash** package is part of the standard distribution. It implements hash sets for Agena numbers for fast insertion, deletion and lookup and is an alternative to the **numarray** data structure when order and frequency is not of concern.

Hash sets use around 12 % more memory than numarrays but are dozens of times faster when inserting, deleting or searching numbers.

Compared with sets, double hash sets consume only 40 % of memory with operations around a third slower, so they are an ideal alternative to sets when you have a large number of values to work on. The number hash set implemented here guarantees that all numbers will be reliably stored even if some numbers share the same hash internally.

Example:

```
> d := dblhash.new(1024)

> for i to 2*1024 do
>    dblhash.include(d, sin(i))
> od

> dblhash.has(d, sin(1)), dblhash.has(d, sin(2049)):
true    false

> size d:
2048

> for i to 2*1024 do
>    dblhash.purge(d, sin(i))
> od

> empty d:
true

> dblhash.has(d, sin(1)):
false
```

Besides the functions described below the package supports the following metamethods:

- **size** operator, that is the current number of values in the hash set,
- **empty** and **filled** operators,
- **in** and **notin** operators for lookup,
- read access by indexing ('__index'),
- pretty-printing ('__tostring'),
- garbage collection ('__gc'). Hint: Use **environ.gc**('step', 100) instead of just **environ.gc**() to reliably clean up memory if you allocate and destroy hash sets in short succession.

The package functions can also be called OOP-style, if not indicated otherwise.

dblhash.attrib (d)

Returns various information on double hash set `d`. The return is a table with the following fields:

- 'count': number of elements inserted (and not yet deleted),
- 'deleted': number of values recently deleted,
- 'capacity': total number of allocated slots,
- 'optimised': flag indicating fast mode with high memory consumption (**true**) or slower mode with low memory consumption (**false**),
- 'loadfactor': field 'count' divided by field 'capacity',
- 'utilization': fields ('count' + 'deleted')/'capacity'.

See also: **dblhash.compact**, **dblhash.getindex**.

dblhash.compact (d)

The function rehashes hash set `d` to remove all slots marked deleted and optimizes the memory footprint based on the current number of elements. It aims for a ~50% load factor, with a minimum of eight pre-allocated slots.

The function returns nothing.

See also: **dblhash.attrib**, **dblhash.purge**.

dblhash.getindex (d [, idx])

Returns the actual index number x is stored in double hash set `d`, plus the index position where x could have been stored if it would not have been already occupied by another value (or itself).

All the indices the function will return will start from 0. If `d` would have to be resized before inserting x, then the function returns **null** twice. If the first result is **null** and the second an integer n, it means that x is not part of `d` but could be stored to position n in the hash set if it is vacant.

See also: **dlbhash.attrib**, **dblhash.getitem**.

dblhash.getitem (d [, idx])

Returns the number stored to slot `idx` in double hash set `d`. `idx` is an integer in the range [0, **size** `d` - 1]. The second return is either 'empty' if there is no value stored at idx, 'occupied' if there is an active number at idx, and 'deleted' if the number has recently been removed by **dblhash.purge**.

When idx is omitted, the internal status table of d will be returned so that you can read and write administrative data from/to it if needed.

dblhash.has (d, n [, approx])

Tries to find number n in double hash set d. The function returns **true** or **false**. If approx is **true**, an approximate equality check will be done if a first exact fast built-in lookup is not successful, conducting a linear search through the entire hash set, see function **approx** for details. If approx is **false**, the default, then the function will look for an exact match only, avoiding linear search altogether.

dblhash.hash (n [, m])

Returns the high and lower unsigned 32-bit words of the SplitMix64-based hash for number n. This hash is used by the package internally to write and read data.

If m is given the two hashes are taken modulo m. The default is 0, that is not taking the modulo.

dblhash.include (d, n)

Inserts number n into double hash set d. It returns nothing. If the load in d would exceed 70 %, the function automatically enlarges the hash set.

See also: **dblhash.purge**.

dblhash.iterate (d)

The function iterates over hash set d and returns the current entry.

Example:

```
> d := dblhash.new(1024)

> for i to 3 do
>    dblhash.include(d, sin(i))
> od

> f := dblhash.iterate(d);

> f():
0.8414709848079

> f():
0.90929742682568

> f():
0.14112000805987

> f():
null
```

dblhash.new ([m [, optimised]])

Creates and returns a double hash set, of type userdata.

`m` depicts the initial number of slots and by default is 8. If `m` is not a already a power of two, then it will automatically be rounded up.

The Boolean `optimised` flag controls speed and memory usage: If set to **false** (the default), the function uses as less memory as possible, with insertions, deletions and lookups six times slower than when the flag were **true**: In latter case the double hash set requires much more memory but supports very fast insertions, deletions and lookups.

Specifically, with `optimised` **true**, when a hash set has already reached 70 % capacity and a new element shall be inserted, the size of the table will be doubled. When set to **false**, the table size will grow by around 13 % only when 70 % capacity has been reached.

The function also sets up an internal status table which you can access with **dblhash.getitem**, see there.

The function cannot be used as a method.

dblhash.purge (d, n [, shrink])

Deletes number `n` from double hash set `d`. It returns **true** on success and **false** if `n` has not been found in `d`. The slot may be reused later with subsequent inserts, if appropriate.

If `shrink` has been set to **true**, the function also automatically and carefully slims the overall size of `d` if its load falls below 12.5 %. The default is **false**, that is no shrinking is done at all.

See also: **dblhash.compact**, **dblhash.include**.

dblhash.toseq (d)

Returns the contents of double hash set `d` in a new sequence. Values previously deleted but still available will not be returned.

dblhash.totable (d)

Returns the contents of double hash set d in a new table with the keys the actual integer indices in d and the values the associated numeric data. Values previously deleted but still available will not be returned.

10.20 uintmap - Integer-to-String Hash Maps

The **uintmap** package is part of the standard distribution, so you do not need to initialise it with the **import** statement.

The package implements unsigned 4-byte integer-to-string hash maps, with the Agena integers the keys and the strings the associated values. It allows for fast insertion, lookup and deletion if you are dealing with a very large amount of data and where ordinary Agena tables may sooner or later throw out-of-memory errors.

Compared with tables, integer-to-string hash maps consume only 40 % of memory with operations more or less slower, so they may be alternative to tables when you have a large number of strings to work on. Specifically, inserts into maps are 3.2 time slower, lookups are 3.5 % slower and deletions 4.8 times slower than the corresponding operations on Agena tables.

An integer key can only be associated with exactly one string as the hash map does not support multiple entries with the same key. Example:

```
> h := uintmap.new()

> uintmap.include(h, 10, 'ten')

> uintmap.include(h, 20, 'twenty')

> uintmap.getitem(h, 10):
ten

> uintmap.getitem(h, 20):
twenty

> uintmap.getitem(h, 1):
null

> uintmap.purge(h, 20);

> uintmap.getitem(h, 20):
null

> uintmap.attrib(h):
[capacity ~ 8, count ~ 1, empty ~ 7, loadfactor ~ 0.125, utilization ~ 1,
 void ~ 0]
```

Most of the package functions can be used OOP-style, and the following metamethods are supported:

- **size** operator, that is the current number of values in the hash set,
- **empty** and **filled** operators,
- **in** and **notin** operators, that check whether a string is associated or not associated with any integer key.
- read access by indexing ('__index'),
- pretty-printing ('__tostring'),

- garbage collection ('__gc'). Hint: Use **environ.gc**('step', 100) instead of just **environ.gc**() to reliably clean up memory if you allocate and destroy hash maps in short succession.

uintmap.attrib (h)

Returns various information on the hash map `h`. The return is a table with the following fields:

- 'count': number of elements inserted (and not yet deleted),
- 'empty': number of available slots,
- 'capacity': total number of allocated slots,
- 'loadfactor': field 'count' divided by field 'capacity',
- 'utilization': fields ('count' + 'deleted')/'capacity'.
- 'void': represents the number of deleted or unused slack buckets that are neither actively occupied by a live key-value pair nor marked as purely empty.

uintmap.cleanse (h)

Removes all elements from the hash map without de-allocating or re-sizing the underlying bucket memory. The function returns nothing.

uintmap.getitem (h [, n])

Checks whether integer `n` exists as a key in hash map `h` and returns the associated string.

When `n` is omitted, the internal status table of `h` will be returned so that you can read and write administrative data from/to it if needed.

The function cannot be called OOP-style.

See also: **uintmap.has**.

uintmap.has (h, str)

Checks whether string `str` is associated with any integer key and returns true or false. Usually, the lookup is done in O(1) time. If there is no hit the whole hash map will be iterated to find `str`.

See also: **uintmap.getitem**.

uintmap.hash (str [, n])

Returns the hash value previously used by the **uintmap** package (khash, actually) for string `str`. If `n` is a positive integer the result will be taken modulo `n`.

The function cannot be called OOP-style.

The **uintmap** package is using the CRC32 integer hash, see **hashes.crc32**.

uintmap.include (h, n, str)
uintmap.include (h, str)

In the first form, inserts the integer `n` as a key and the associated string `str` into hash map `h`.

In the second form, inserts string `str` and its associated key k into hash map `h`. k is equal to **hashes.crc32**(`str`). Note that in this mode there is a 0.00637 % collision chance.

The function returns **true** on success and **false** otherwise. The function automatically grows the hash map if deemed necessary.

See also: **uintmap.purge**.

uintmap.iterate (h)

The function iterates over hash map `h` and returns the current key and value.

Example:

```
> h := uintmap.new()

> uintmap.include(h, 10, 'ten')

> uintmap.include(h, 20, 'twenty')

> f := uintmap.iterate(h)

> f():
10      ten

> f():
20      twenty

> f():
null
```

uintmap.new ([m [, optimised]])

Creates and returns an integer-to-string hash map, of type userdata.

`m` depicts the initial number of slots and by default is 8. If `m` is not a already a power of two, then it will automatically be rounded up.

The function also sets up an internal status table which you can access with **uintmap.getitem**, see there.

The function cannot be called OOP-style.

uintmap.purge (h, str [, shrink])
uintmap.purge (h, n [, shrink])

In the first form, deletes string `str` and its associated integer key from hash map `h`. In the second form, by passing integer key `n`, the respective integer-string pair will be removed. It returns **true** on success and **false** if `str` or `n` have not been found in `h`.

The function will automatically shrink the hash map if deemed necessary. You can enforce shrinking by passing **true** as a third argument, but this will only take place if safely possible.

See also: **uintmap.include**.

uintmap.resize (h, newsize)

The function shrinks or expands a hash set `h` to the new size `newsize`, a number. You usually do not need it as the package functions re-size a hash set automatically. However, in order to prevent iterative re-sizing, you may just define a new size to speed up further operations a bit.

The function returns **true** on success or **false** otherwise, plus the new actual hash set size. The function does not allow to reduce `h` to a size that would erase entries.

See also: **uintmap.purge**.

uintmap.totable (h)

Returns the contents of hash map `h` in a new table, with the integer map keys the table keys and the associated strings the table values.

10.21 intmap - Integer-to-Integer Hash Maps

The **intmap** package is part of the standard distribution and does not need to be initialised with the **import** statement.

The package implements integer-to-integer hash maps, with Agena integers both the keys and associated values. Specifically, the integer keys and values are signed 4-byte integers. It allows for fast insertion, lookup and deletion if you are dealing with a very large amount of data and where ordinary Agena tables may sooner or later throw out-of-memory errors.

To map from unsigned integer range to signed integer range and vice versa, call **intmap.remap**.

Compared with tables, signed integer-to-integer hash maps consume only 35 % of memory with operations around two-thirds slower with positive keys and twice as fast with signed and unsigned integer keys (50% non-negative, 50 % non-positive), so they may be an alternative to tables when you have a large number of data to work on. With unsigned integer-to-integer hash maps, insertions, deletions and lookups take roughly twice as long as with Agena table arrays.

A integer key can only be associated with exactly one integer value as the hash map does not support multiple entries with the same key. Example:

```
> h := intmap.new()

> intmap.include(h, 10, 100)

> intmap.include(h, -10, -100)

> intmap.getitem(h, 10):
100

> intmap.getitem(h, -10):
-100

> intmap.getitem(h, 1):
null

> intmap.purge(h, -100);

> intmap.getitem(h, -100):
null

> intmap.attrib(h):
[capacity ~ 8, count ~ 2, empty ~ 6, loadfactor ~ 0.25, utilization ~ 1,
 void ~ 0]
```

Most package functions can be called OOP-style and the following metamethods are supported:

- **size** operator, that is the current number of values in the hash map,
- **empty** and **filled** operators,

- **in** and **notin** operators, that check whether a string is associated or not associated with any number key.
- read access by indexing ('__index'),
- pretty-printing ('__tostring'),
- garbage collection ('__gc'). Hint: Use **environ.gc**('step', 100) instead of just **environ.gc**() to reliably clean up memory if you allocate and destroy hash maps in short succession.

intmap.attrib (h)

Returns various information on the hash map `h`. The return is a table with the following fields:

- 'collisions': number of collisions resolved by moving to the next slot (probing),
- 'count': number of elements inserted (and not yet deleted),
- 'empty': number of available slots,
- 'capacity': total number of allocated slots,
- 'loadfactor': field 'count' divided by field 'capacity',
- 'utilization': fields ('count' + 'deleted')/'capacity'.
- 'void': represents the number of deleted or unused slack buckets that are neither actively occupied by a live key-value pair nor marked as purely empty.

intmap.cleanse (h)

Removes all elements from hash map `h` without de-allocating or re-sizing the underlying bucket memory. The function returns nothing.

intmap.getitem (h [, n])

Checks whether number `n` exists as a key in hash map `h` and returns the associated integer.

When `n` is omitted, the internal status table of `h` will be returned so that you can read and write administrative data from/to it if needed.

The function cannot be called OOP-style.

intmap.include (h, nk, nv)

Inserts the integer `nk` as a key and the associated integer `nv` into hash map `h`. It returns nothing. The function automatically grows the hash map if deemed necessary.

See also: **intmap.purge**.

intmap.iterate (h)

The function iterates over hash map `h` and returns the current key and value.

Example:

```
> h := intmap.new()

> intmap.include(h, 10, 100)

> intmap.include(h, -10, -100)

> f := intmap.iterate(h);

> f():
10      100

> f():
-10     -100

> f():
null
```

intmap.new ([m [, optimised]])

Creates and returns a integer-to-integer hash map, of type userdata.

`m` depicts the initial number of slots and by default is 8. If `m` is not a already a power of two, then it will automatically be rounded up.

The function also sets up an internal status table which you can access with **intmap.getitem**, see there.

The function cannot be called OOP-style.

intmap.purge (h, n)

Deletes integer key `n` and its associated integer from hash map `h`. It returns **true** on success and **false** if `n` has not been found in `h`. The function will automatically shrink the hash map if deemed necessary.

See also: **intmap.include**.

intmap.remap (n [, tosigned])

The function by default maps an integral Agena number `n` from the range 0 .. 4,294,967,295 (2^{31}-1) to range -2,147,483,648 (-2^{31}) .. 2,147,483,647 (2^{31}-1). If `tosigned` is **true**, does the same.

If `tosigned` is **false**, maps `n` from -2,147,483,648 (-2^{31}) .. 2,147,483,647 (2^{31}-1) to 0 .. 4,294,967,295 (2^{31}-1).

If `n` is out-of-range for the respective conversion, the function issues an error.

intmap.resize (h, newsize)

The function shrinks or expands a hash map `h` to the new size `newsize`, a number. You usually do not need it as the package functions re-size a hash map automatically. However, in order to prevent iterative re-sizing, you may just define a new size to speed up further operations a bit.

The function returns **true** on success or **false** otherwise, plus the new actual hash map size. The function does not allow to reduce `h` to a size that would erase entries.

See also: **intmap.purge**.

intmap.totable (h)

Returns the contents of hash map `h` in a new table with both the keys and entries numbers.

10.22 dblmap - Number-to-Number Hash Maps

The **dblmap** package is part of the standard distribution and does not need to be initialised with the **import** statement.

The package implements number-to-number hash maps, with Agena numbers (integral and fractional) both the keys and associated values. It allows for fast insertion, lookup and deletion if you are dealing with a very large amount of data, for example time series, and where ordinary Agena tables may sooner or later run out of memory.

Compared with tables, number-to-number hash maps *with fractional indices* consume only 66 % of the memory, with insert operations 50 % faster than with table arrays [sic !], lookups 33 % slower with table arrays and deletions also 33 % slower with table arrays, so they might be an alternative to table arrays when you have a large number of data to work on.

A number key can only be associated with exactly one number value as the hash map does not support multiple entries with the same key. Example:

```
> h := dblmap.new()

> dblmap.include(h, 10, 100)

> dblmap.include(h, -10, -100)

> dblmap.getitem(h, 10):
100

> dblmap.getitem(h, -10):
-100

> dblmap.getitem(h, 1):
null

> dblmap.purge(h, -100);

> dblmap.getitem(h, -100):
null

> dblmap.attrib(h):
[capacity ~ 8, count ~ 2, empty ~ 6, loadfactor ~ 0.25, utilization ~ 1,
 void ~ 0]
```

Most package functions can be called OOP-style and the following metamethods are supported:

- **size** operator, that is the current number of values in the hash map,
- **empty** and **filled** operators,
- **in** and **notin** operators, that check whether a string is associated or not associated with any number key.
- read access by indexing ('__index'),
- pretty-printing ('__tostring'),

- garbage collection ('__gc'). Hint: Use **environ.gc**('step', 100) instead of just **environ.gc**() to reliably clean up memory if you allocate and destroy hash maps in short succession.

dblmap.attrib (h)

Returns various information on the hash map `h`. The return is a table with the following fields:

- 'collisions': number of collisions resolved by moving to the next slot (probing),
- 'count': number of elements inserted (and not yet deleted),
- 'empty': number of available slots,
- 'capacity': total number of allocated slots,
- 'loadfactor': field 'count' divided by field 'capacity',
- 'utilization': fields ('count' + 'deleted')/'capacity'.
- 'void': represents the number of deleted or unused slack buckets that are neither actively occupied by a live key-value pair nor marked as purely empty.

dblmap.cleanse (h)

Removes all elements from hash map `h` without de-allocating or re-sizing the underlying bucket memory. The function returns nothing.

dblmap.getitem (h [, n])

Checks whether number `n` exists as a key in hash map `h` and returns the associated value.

When `n` is omitted, the internal status table of `h` will be returned so that you can read and write administrative data from/to it if needed.

The function cannot be called OOP-style.

dblmap.include (h, nk, nv)

Inserts the number `nk` as a key and the associated number `nv` into hash map `h`. It returns nothing. The function automatically grows the hash map if deemed necessary.

See also: **dblmap.purge**.

dblmap.iterate (h)

The function iterates over hash map `h` and returns the current key and value.

Example:

```
> h := dblmap.new()

> dblmap.include(h, 10, 100)
```

```
> dblmap.include(h, -10, -100)

> f := dblmap.iterate(h);

> f():
10      100

> f():
-10     -100

> f():
null
```

dblmap.new ([m [, optimised]])

Creates and returns a number-to-number hash map, of type userdata.

`m` depicts the initial number of slots and by default is 8. If `m` is not a already a power of two, then it will automatically be rounded up.

The function also sets up an internal status table which you can access with **dblmap.getitem**, see there.

The function cannot be called OOP-style.

dblmap.purge (h, n)

Deletes number key `n` and its associated number from hash map `h`. It returns **true** on success and **false** if `n` has not been found in `h`. The function will automatically shrink the hash map if deemed necessary.

See also: **dblmap.include**.

dblmap.resize (h, newsize)

The function shrinks or expands a hash map `h` to the new size `newsize`, a number. You usually do not need it as the package functions re-size a hash map automatically. However, in order to prevent iterative re-sizing, you may just define a new size to speed up further operations a bit.

The function returns **true** on success or **false** otherwise, plus the new actual hash map size. The function does not allow to reduce `h` to a size that would erase entries.

See also: **dblmap.purge**.

dblmap.totable (h)

Returns the contents of hash map `h` in a new table with both the keys and entries numbers.

10.23 strhash - String Hash Sets

The **strhash** package is not part of the standard distribution.

Import the package with:

```
> import strhash
```

The package implements hash sets for Agena strings for fast insertion, lookup and deletion.

Compared with sets, string hash sets consume only 40 % of memory with operations around a third slower, so they are an ideal alternative to sets when you have a large number of strings to work on and where ordinary Agena sets may sooner or later throw out-of-memory errors.

Example:

```
> import strhash

> d := strhash.new(1024)

> strhash.include(h, 'apple')

> strhash.include(h, 'banana')

> strhash.getitem(h, 'banana'):
true

> strhash.getitem(h, 'anana'):
false

> strhash.purge(h, 'banana')

> strhash.getitem(h, 'banana'):
false

> strhash.attrib(h):
[capacity ~ 8, count ~ 1, empty ~ 7, loadfactor ~ 0.125, utilization ~ 1,
 void ~ 0]
```

The package supports the following metamethods:

- **size** operator, that is the current number of values in the hash set,
- **empty** and **filled** operators,
- **in** and **notin** operators,
- read access by indexing ('__index'),
- pretty-printing ('__tostring'),
- garbage collection ('__gc'). Hint: Use **environ.gc**('step', 100) instead of just **environ.gc**() to reliably clean up memory if you allocate and destroy hash sets in short succession.

strhash.attrib (h)

Returns various information on string hash set `h`. The return is a table with the following fields:

- 'count': number of elements inserted (and not yet deleted),
- 'empty': number of available slots,
- 'capacity': total number of allocated slots,
- 'loadfactor': field 'count' divided by field 'capacity',
- 'utilization': fields ('count' + 'deleted')/'capacity',
- 'void': represents the number of deleted or unused slack buckets that are neither actively occupied by a live key-value pair nor marked as purely empty.

strhash.cleanse (h)

Removes all elements from the hash set without de-allocating or re-sizing the underlying bucket memory. The function returns nothing.

strhash.getitem (h [, str])

Checks whether string `str` exists in string hash set `h` and returns **true** or **false**.

When `str` is omitted, the internal status table of `h` will be returned so that you can read and write administrative data from/to it if needed.

See also: **in** and **notin** operators, **strhash.has**.

strhash.has (h, str)

Checks whether string `str` exists in string hash set `h` and returns **true** or **false**.

See also: **in** and **notin** operators, **strhash.getitem**.

strhash.include (h, str)

Inserts string `str` into string hash set `h`. It returns nothing. The function automatically grows the hash set if deemed necessary.

See also: **strhash.purge**.

strhash.iterate (h)

The function iterates over hash set `h` and returns the current key.

Example:

```
> h := strhash.new()
> strhash.include(h, 'apple')
> strhash.include(h, 'banana')
```

```
> f := strhash.iterate(h):

> f():
apple

> f():
banana

> f():
null
```

strhash.new ([m [, optimised]])

Creates and returns a string hash set, of type userdata.

`m` depicts the initial number of slots and by default is 8. If `m` is not a already a power of two, then it will automatically be rounded up.

The function also sets up an internal status table which you can access with **strhash.getitem**, see there.

The function cannot be used as a method.

See also: **strhash.resize**.

strhash.purge (h, str)

Deletes string `str` from hash set `h`. It returns **true** on success and **false** if `str` has not been found in `h`. The function will automatically shrink the hash set if deemed necessary.

See also: **strhash.include**.

strhash.resize (h, newsize)

The function shrinks or expands a hash set `h` to the new size `newsize`, a number. You usually do not need it as the package functions re-size a hash set automatically. However, in order to prevent iterative re-sizing, you may just define a new size to speed up further operations a bit.

The function returns **true** on success or **false** otherwise, plus the new actual hash set size. The function does not allow to reduce `h` to a size that would erase entries.

See also: **strhash.purge**.

strhash.toseq (h)

Returns the contents of string hash set `h` in a new sequence. Values previously deleted but still available will not be returned.

strhash.totable (h)

Returns the contents of string hash set `h` in a new table.

10.24 strmap - String-to-Number Hash Maps

The **strmap** package is not part of the standard distribution.

Import the package with:

```
> import strmap
```

The package implements string-to-number hash maps, with the strings the keys and Agena numbers, including fractional numbers, the associated values. It allows for fast insertion, lookup and deletion if you are dealing with a very large amount of data and where ordinary Agena tables may sooner or later throw out-of-memory errors.

Compared with tables, string-to-number hash maps consume only 40 % of memory with operations two-thirds slower, so they may be an alternative to tables when you have a large number of strings to work on.

A string key can only be associated with exactly one number, the hash map does not support multiple entries with the same key. Example:

```
> import strmap

> d := strmap.new(1024);

> strmap.include(h, 'apple', 10)

> strmap.include(h, 'banana', 20)

> strmap.getitem(h, 'banana'):
20

> strmap.getitem(h, 'anana'):
null

> strmap.purge(h, 'banana')

> strmap.getitem(h, 'banana'):
null

> strmap.attrib(h):
[capacity ~ 8, count ~ 1, empty ~ 7, loadfactor ~ 0.125, utilization ~ 1,
 void ~ 0]
```

The package supports the following metamethods:

- **size** operator, that is the current number of values in the hash set,
- **empty** and **filled** operators,
- **in** and **notin** operators, that check whether a number is associated or not associated with any string key.
- read access by indexing ('__index'),
- pretty-printing ('__tostring'),

- garbage collection ('__gc'). Hint: Use **environ.gc**('step', 100) instead of just **environ.gc**() to reliably clean up memory if you allocate and destroy hash maps in short succession.

strmap.attrib (h)

Returns various information on the hash map `h`. The return is a table with the following fields:

- 'count': number of elements inserted (and not yet deleted),
- 'empty': number of available slots,
- 'capacity': total number of allocated slots,
- 'loadfactor': field 'count' divided by field 'capacity',
- 'utilization': fields ('count' + 'deleted')/'capacity'.
- 'void': represents the number of deleted or unused slack buckets that are neither actively occupied by a live key-value pair nor marked as purely empty.

strmap.cleanse (h)

Removes all elements from the hash map without de-allocating or re-sizing the underlying bucket memory. The function returns nothing.

strmap.getitem (h [, str])

Checks whether string `str` exists as key in hash map `h` and returns the associated number.

When `str` is omitted, the internal status table of `h` will be returned so that you can read and write administrative data from/to it if needed.

strmap.include (h, str, num)

Inserts the string `str` as a key and the associated number `num` into hash map `h`. It returns **true** on success and **false** otherwise. The function automatically grows the hash map if deemed necessary.

See also: **strmap.purge**.

strmap.iterate (h)

The function iterates over hash map `h` and returns the current key and value.

Example:

```
> h := strmap.new()

> strmap.include(h, 'apple', 10)

> strmap.include(h, 'banana', 20)

> f := strmap.iterate(h):
```

```
> f():
apple   10

> f():
banana  20

> f():
null
```

strmap.new ([m [, optimised]])

Creates and returns a string-to-double hash map, of type userdata.

`m` depicts the initial number of slots and by default is 8. If `m` is not a already a power of two, then it will automatically be rounded up.

The function also sets up an internal status table which you can access with **strmap.getitem**, see there.

The function cannot be used as a method.

strmap.purge (h, str)

Deletes string `str` and its associated number from hash map `h`. It returns **true** on success and **false** if `str` has not been found in `h`. The function will automatically shrink the hash map if deemed necessary.

See also: **strmap.include**.

strmap.resize (h, newsize)

The function shrinks or expands a hash set `h` to the new size `newsize`, a number. You usually do not need it as the package functions re-size a hash set automatically. However, in order to prevent iterative re-sizing, you may just define a new size to speed up further operations a bit.

The function returns **true** on success or **false** otherwise, plus the new actual hash set size. The function does not allow to reduce `h` to a size that would erase entries.

See also: **strmap.purge**.

strmap.totable (h)

Returns the contents of hash map `h` in a new table with the keys the strings and the numbers the values.

Chapter Eleven

Numbers

11 Numbers

11.1 Mathematical Functions

The mathematical operators and functions explained in this chapter work on both real numbers as well as complex numbers, except if indicated otherwise.

For the sake of speed, basic arithmetic functions have been implemented as operators, whereas all other mathematical functions are implemented as Agena library functions (implemented either in C or Agena). While functions can be overwritten with self-defined versions, operators cannot be overwritten.

Summary of Operators and Functions:

Basic Arithmetic Operators

> +, -, *, /, /*, &+, &-, &*, &/, &\, **foreach**, **math.accu**, **math.fdim**, **math.kbadd**, **math.koadd**.

Relational Operators

> =, ==, <, >, <=, >=, <>, |, **approx**, **math.eq**, **math.ge**, **math.gt**, **math.le**, **math.lt**, **math.neq**.

Integer Division

> \, %, **drem**, **iqr**, **modf**, **modp**, **symmod**, **math.cld**, **math.fld**, **math.modiv**, **math.modulus**, **math.nearmod**.

Exponentiation

> ^, **, **antilog2**, **antilog10**, **cube**, **exp**, **exp2**, **exp10**, **expx2**, **frexp**, **ldexp**, **math.expm1**, **numtheory.iscube**, **numtheory.issquare**, **numtheory.issqrfree**, **square**, **squareadd**.

Roots

> **cathet**, **cbrt**, **hypot**, **hypot2**, **hypot3**, **invsqrt**, **proot**, **pytha**, **pytha4**, **root**, **sqrt**, **math.isqrt**, **math.lnhypot**, **math.pow32**, **math.pow52**, **math.pow72**, **math.powe**, **fastmath.hypotfast**, **fastmath.sqroot**, **fastmath.sqrtfast**.

Logarithms

> **ilog2**, **ln**, **log**, **log2**, **log10**, **math.ceillog2**, **fastmath.lbfast**, **math.lnp1**, **math.xlnp1**.

Trigonometric Functions

cas, **cos**, **cot**, **csc**, **sec**, **sin**, **tan**, **math.cosd**, **math.cotd**, **math.quadrant**, **math.redupi**, **math.sincos**, **math.sind**, **math.tand**, **fastmath.sincosfast**, **math.wrap**.

Inverse Trigonometric Functions

arccos, **arccsc**, **arccot**, **arcsec**, **arcsin**, **arctan**, **arctan2**, **arctanh**.

Hyperbolic Functions

cosh, **coth**, **csch**, **sech**, **sinh**, **tanh**.

Inverse Hyperbolic Functions

arccosh, **arccsch**, **arccoth**, **arcsech**, **arcsinh**, **arctanh**.

Sign

sign, **signum**, **math.copysign**, **math.flipsign**, **math.gammasign**, **math.mulsign**, **math.signbit**.

Miscellaneous

erf, **erfc**, **erfcx**, **erfi**, **inverf**, **inverfc**, **fma**, **sinc**, **cosc**, **tanc**, **math.agm**, **numtheory.fib**, **numtheory.fibinv**, **numtheory.gcd**, **numtheory.isfib**, **numtheory.lcm**, **math.max**, **math.min**, **math.rectangular**, **math.relerror**, **math.triangular**, **muladd**.

Miscellaneous Complex Functions

argument, **bea**, **conjugate**, **cosxx**, **flip**, **polar**.

Gamma, etc.

beta, **binomial**, **fact**, **gamma**, **lngamma**, **math.fall**, **math.lnfact**, **math.pochhammer**, **over**.

Bessel Functions

besselj, **bessely**, **calc.bessel0**, **calc.bessel1**.

Rounding Functions

ceil, **entier**, **int**, **mdf**, **round**, **xdf**, **math.rint**.

Numbers

frac, **frexp**, ++, --, +++, ---, **math.compose**, **math.decompose**, **math.eps**, **math.epsilon**, **math.exponent**, **numtheory.factors**, **math.fraction**, **math.frexp**, **numtheory.jacobi**, **numtheory.kronecker**, **math.mantissa**, **math.ndigits**, **math.nextafter**, **math.nextmultiple**, **math.nextpow2**, **math.nextpower**, **math.nthdigit**, **numtheory.nthpow**, **math.prevmultiple**, **math.prevpow2**, **math.prevpower**, **math.tohex**, **math.uexponent**, **math.ulp**, **multiple**.

Numeric Checks

even, **finite**, **fractional**, **in**, **infinite**, **isint**, **isnegative**, **isnegint**, **isnonneg**, **isnonnegint**, **isnonposint**, **isnonzeroint**, **isnumber**, **isnumeric**, **isposint**, **ispositive**, **nan**, **odd**, **math.fpclassify**, **math.isexceptional**, **math.isinfinity**, **math.isminuszero**, **math.isnormal**, **math.isordered**, **math.ispow2**, **math.issubnormal**.

Range Reduction, Conversion and Interpolation

abs, ||, **heaviside**, **math.branch**, **math.chi**, **math.chop**, **math.clip**, **math.invlerp**, **math.lerp**, **math.lnabs**, **math.norm**, **math.normalise**, **math.piecewise**, **math.ramp**, **math.rectangular**, **math.rempio2**, **math.unitise**, **math.unitstep**, **math.wrap**.

Random Numbers

math.noise, **math.random**, **math.randomseed**.

Bases and Conversion

math.convertbase, **math.dd**, **math.dms**, **math.norm**, **math.splitdms**, **math.tobinary**, **math.todecimal**, **math.tohex**, **math.toradians**, **math.tosgesim**.

Primes

numtheory.congruentprime, **numtheory.ifactor**, **numtheory.ifactors**, **numtheory.isprime**, **numtheory.nextprime**, **numtheory.prevprime**, **numtheory.primes**.

Bitwise Operators, Bit and Byte Twiddling

> **&&**, ~~, ||, ^ ^, <<, >>, <<<, >>>, **implies**, **nand**, **nor**, **xnor**, **xor**, **getbit**, **getbits**, **getnbits**, **setbit**, **setbits**, **setnbits**, **math.inttofpb**, **bytes.numhigh**, **bytes.numlow**, **bytes.gethigh**, **bytes.getlow**, **bytes.numwords**, **math.fpbtoint**, **bytes.leadzeros**, **bytes.leastsigbit**, **bytes.mostsigbit**, **bytes.onebits**, **bytes.reverse**, **bytes.setdouble**, **bytes.sethigh**, **bytes.setnumhigh**, **bytes.setlow**, **bytes.setnumlow**, **bytes.setnumwords**, **bytes.tobytes**.

Boolean Operators

> **and**, **implies**, **nand**, **nor**, **not**, **or**, **xnor**, **xor**

11.1.1 Operators and Functions

x + y

The operator adds two numbers; returns a number. Complex and dual numbers are supported.

See also: **sumup**, **factory.count**, **math.accu**, **math.kbadd**, **inc** operator described in Chapter 4.6.8.

x - y

The operator subtracts two numbers; returns a number. Complex and dual numbers are supported.

See also: **math.fdim**, **dec** operator described in Chapter 4.6.8.

x * y

The operator multiplies two numbers; returns a number. Complex, dual numbers and Booleans are supported. A Boolean operand represents 1 for **true**, and 0 for **false** or **fail**.

See also: **mul** operator described in Chapter 4.6.8.

x / y

The operator divides two numbers; returns a number. Complex and dual numbers are supported.

See also: **recip**,**math.cld**, **math.fld**, **div** operator in Chapter 4.6.8.

x \ y

The operator performs an integer division of two numbers, and returns a number. The integer division is defined as: x \ y = **sign**(x) * **sign**(y) * **entier**(|$\frac{x}{y}$|).

See also: **%**, /, **iqr**, **math.cld**, **math.fld**, **intdiv** operator in Chapter 4.6.8.

x &+ y

The operator adds two signed or unsigned 32-bit numbers; returns a number. Complex numbers are supported, as well. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option.

See also: **bytes.add32**, **factory.count**, **math.accu**, **math.koadd**.

x &- y

The operator subtracts two signed or unsigned 32-bit numbers; returns a number. Complex numbers are supported, as well. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option.

See also: **bytes.sub32**, **math.fdim**.

x &* y

The operator multiplies two signed or unsigned 32-bit numbers; returns a number. Complex numbers are supported. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option.

See also: **bytes.mul32**.

x &/ y

The operator divides two signed or unsigned 32-bit numbers; returns a number. Complex numbers are supported. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option. See also: **bytes.div32**.

x *% y

The operator multiplies two numbers and divides the result by 100; returns a number, the percentage.

x /% y

The operator divides two numbers and multiplies the result by 100; returns a number, the ratio.

x %% y

The operator computes the percentage change from the number x to the number y and returns a number. It is equivalent to y **/%** x - 100.

x +% y

The operator adds the given percentage y to x.

x -% y

The operator subtracts the given percentage y from x.

!n

The factorial function. Returns the same result as the **fact** function, see there.

n over k

The binary operator computes the binomial coefficient $\binom{n}{k}$ for integral n and integral k. Results r that are larger than |r| > **math.lastcontint** = 2^{53} may be imprecise. See also: **binomial**.

z roll r

The binary operator rotates a two-dimensional vector, represented by the complex number z, through the angel r (given in radians) counterclockwise and returns the new complex number z*exp(I*r). To convert degrees to radians, multiply by Pi/180. If z is just a number, it is internally converted to the complex number z + 0*I.

See also: **conjugate**, **flip**.

x % y

The modulo operator conducts the operation x % y = x - **entier**($\frac{x}{y}$)*y. The return is always non-negative.

The function may return surprising results with |x|<1, so calling **numtheory.invmod** with the reciprocal x^{-1} might be an alternative. With large |x|, you might use **numtheory.mulmod** or **numtheory.powmod**.

See also: \, **drem**, **everyth**, **iqr**, **modp**, **symmod**, **hashes.fibmod**, **hashes.fibmod2**, **math.modulus**, **math.wrap**, **mod** operator in Chapter 4.6.8.

x symmod y

The symmetric modulo operator evaluates the remainder of a division x/y (with x, y two Agena numbers). The result has the same sign as the numerator x. Specifically, the return value is x - q * y, where q is the quotient x/y, rounded towards 0 to the next integer. The function works like the C function fmod.

See also: \,%, **drem**, **iqr**, **irem**, **modp**, **numtheory.invmod**, **math.fmod**, **math.modulus**, **math.wrap**.

x ^ y

The operator performs an exponentiation of real, complex or dual x with a rational power y. With numbers, if x is negative and y non-integral, it returns **undefined**.

See also: ^ operator, **antilog2**, **antilog10**, **proot**, **root**, **square**, **squareadd**.

x ** y

The operator exponentiates the real, complex or dual number x with the integer power y. Depending on the platform and with small y, the operator is at least 50 % faster than the ^ operator. If y is **undefined** or ±**infinity**, **undefined** will be returned.

See also: **cube**, **square**, **squareadd**.

x ! y

The operator creates a complex number with real part x and imaginary part y, with x and y number.

x !! y

The operator returns a complex number z in Cartesian notation a + I*b for magnitude/modulus x and argument/phase angle y. x and y must be numbers. The result is equivalent to z = x * **cis**(y).

See also: ||, **abs**, **argument**, **cabs**, **cartesian**, **cis**, **polar**.

z squareadd c

For numeric or complex z, c, computes z ^ 2 + c, preventing round-off errors. Note that the operation comes at the expense of speed and in most real-world situations the results will not be better.

See also: **, **fma**, **square**.

|x|

The operator computes the absolute value of the number, complex or dual number x, i.e. abs(x). The return in both cases is a number.

See also: **|-**, **abs**, **cabs**, **calc.euclddist**, **math.lnabs**, **math.fdim**.

x |- y

The operator computes the absolute difference of the two numbers x and y, i.e. abs(x - y). The return is a number.

See also: **|-**, **abs**, **cabs**, **calc.euclddist**, **math.fdim**.

x && y

Bitwise `and` operation on two numbers x and y. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option. See also: **environ.kernel** in Chapter 14.2. See also: **bytes.and32**.

+++ x

Returns the next representable number larger than x. If given a variable, the operator does *not* change its value. See also: **---**, **math.nextafter**.

--- x

Returns the next representable number smaller than x. If given a variable, the operator does *not* change its value. See also: **+++**, **math.nextafter**.

~~ x

Bitwise 32-bit complementary operation on the number x, i.e. bitwise NOT, flipping all the bits representing x. The operator returns signed results only, regardless of the environ.kernel/signedbits setting. See also: **bytes.not32**.

x || y

Bitwise `or` operation on two numbers x and y. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option. See also: **environ.kernel** in Chapter 14.2.

See also: **bytes.or32**.

x ^^ y

Bitwise 32-bit `exclusive-or` operation on two numbers x and y. By default, the operator internally calculates with unsigned 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option. See also: **environ.kernel** in Chapter 14.2.

See also: **bytes.xor32**.

x << y

Bitwise left-shift operation (multiplication by 2, i.e. x << y = $x*2^y$). By default, the operator internally calculates with signed 32-bit integers. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option. If y >= **environ.kernel**('nbits'), returns 0. Please note that shift by negative y are undefined. Shift by zero is the identity shift.

See also: >>, **environ.kernel**, **bytes.shift32**.

x >> y

Bitwise right-shift operation (division by 2, i.e. x >> y = $x/2^y$). The operator by default calculates with unsigned 32-bit integers internally. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option. If y >= **environ.kernel**('nbits'), returns 0.

Please note that shift by negative y are undefined. However, if x is negative and y positive, an arithmetic right-shift is accomplished, thus preserving the sign of x. A shift by zero is the identity shift.

See also: <<, **environ.kernel**, **bytes.shift32**.

x <<< y

Returns the number x rotated a given number of bits y to the left. Internally it uses unsigned 32-bit integers by default. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option.

See also: >>>, **environ.kernel**, **bytes.rotate32**.

x >>> y

Returns the number x rotated a given number of bits y to the right. Internally it uses unsigned 32-bit integers by default. You can change this to signed integers by calling **environ.kernel** with the **signedbits** option.

See also: <<<, **environ.kernel**, **bytes.rotate32**

x in y

Checks whether the number x is part of the interval defined by the pair y consisting of two numbers. The operator returns **true** or **false**. For a much faster check, chain relational operators, such like a < x <= b.

x ⊥ y

The operator compares two finite numbers x, y, determines whether x is less than y, x is exactly equal to y, or x is greater than y, and returns -1, 0, or 1 respectively.

If at least one of the operators is infinite or **undefined**, the function returns **undefined**.

The operator is twice as fast as **sign**. See also: ~|, **signum**.

To build a piece-wise function, for example the absolute function, you may enter:

```
> my_abs := proc(x) is
>    case x | 0
>       of -1 then return -x
>    else
>       return x
>    esac
> end;
```

With x, y strings, the operator does a version string comparison as **strings.strverscmp** does, that is, for example, 'Jan 1' < 'Jan 2' < ... < 'Jan 10'. The operator, however, returns just one of three values: -1 if x < y, 0 if x = y or 1 else.

x ~⊥ y

The operator compares two finite numbers x, y, determines whether x is approximately equal to y, x is less than y, or x is greater than y, and returns 0, -1, or 1 respectively.

See also: | operator.

abs (z)

If z is a number, the **abs** operator returns the absolute value of z. With a complex number z = x + I*y, it returns the distance between it and the origin as a number, i.e. $\sqrt{x^2+y^2}$. Dual numbers are supported.

See also: ||, |-, **argument**, **cabs**, **math.lnabs**, **polar**.

antilog2 (z)

The operator computes 2^z, i.e. 2 raised to the power of the number, complex or dual number z.

See also: ^ and ** operators, **antilog10**, **log2**.

antilog10 (z)

The operator computes 10^z, i.e. 10 raised to the power of the number, complex or dual number `z`.

See also: ^ and ** operators, **antilog2**, **log10**.

approx (x, y [, eps])

Compares the two numbers or complex values `x` and `y` and checks whether they are approximately equal. If `eps` is omitted, **Eps** is used.

The algorithm uses a combination of simple distance measurement (`|x-y| ≤ eps`) suited for values `near` 0 and a simplified relative approximation algorithm developed by Donald H. Knuth suited for larger values (`|x-y| ≤ eps * max(|x|, |y|)`), that checks whether the relative error is bound to a given tolerance `eps`.

The function returns **true** if `x` and `y` are considered equal or **false** otherwise. If both `a` and `b` are **infinity**, the function returns **true**. The same applies to `a` and `b` being -**infinity** or **undefined**.

See also: **math.eps**, **math.epsilon**.

arccos (x)

Returns the inverse cosine operator (`x` in radians). Complex and dual numbers are supported.

arccosh (x)

Returns the inverse hyperbolic cosine of `x` (in radians). The function works on numbers, complex and dual values. The function is implemented in Agena and included in the lib/library.agn file.

arccsc (x)

Returns the inverse cosecant of `x` (in radians). The function works on numbers, complex and dual values. The function is implemented in Agena and included in the lib/library.agn file.

arccsch (x)

Returns the inverse hyperbolic cosecant of `x` (in radians). The function works on numbers, complex and dual values. The function is implemented in Agena and included in the lib/library.agn file.

arccot (x)

Returns the inverse cotangent of x (in radians). The function works on numbers, complex and dual values. The function is implemented in Agena and included in the lib/library.agn file.

arccoth (x)

Returns the inverse hyperbolic cotangent of x (in radians). The function works on numbers, complex and dual values. With real numbers, returns **undefined** if $x \leq 1$.

arcsec (x)

Returns the inverse secant of x (in radians). The operator works on numbers, complex and dual values.

arcsech (x)

Returns the inverse hyperbolic secant of x (in radians). The function works on numbers, complex and dual values. The function is implemented in Agena and included in the lib/library.agn file.

arcsin (x)

Computes the inverse sine operator (in radians). Complex and dual numbers are supported.

arcsinh (x)

Returns the inverse hyperbolic sine of x (in radians). The function works on numbers, complex and dual values. See also: **math.arcsinh**.

arctan (x)

Computes the inverse tangent operator (in radians). Complex and dual numbers are supported. See also: **arctan2**.

arctan2 (y, x)

Returns the arc tangent of y/x (in radians), but uses the signs of both parameters to find the quadrant of the result. (It also handles correctly the case of y being zero.) x and y must be numbers, complex or dual numbers. See also: **arctan**.

arctanh (x)

Returns the inverse hyperbolic tangent of x (in radians). The function works on numbers, complex and dual values. The function is implemented in Agena and included in the lib/library.agn file.

argument (z)

Returns the argument (the phase angle) of the complex value z in radians, a number. If z is a number, the function returns 0 if $z \geq 0$, and π otherwise.

See also: **abs**, **cabs**, **polar**.

bea (z)

The operator takes the complex number z = x!y and returns the complex number **sin**(x)***sinh**(y) + I***cos**(x)***cosh**(y). This function may be mathematically useless, but it creates beautiful fractals. With numbers, it returns **undefined**.

See also: **cosxx**, **flip**.

beta (x, y)

Computes the Beta function. x and y are numbers, complex or dual values. The return may be a number or complex value. The Beta function is defined as: Beta(x, y) $= \frac{\Gamma x * \Gamma y}{\Gamma(x+y)}$, with special treatment if x and y are integers or are equal.

binomial (n, k)

Returns the binomial coefficient $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ as a number, preventing internal numeric overflow. n, k may also be negative integers, or fractions of any sign.

Results r = |**binomial**(n, k)| with r > **math.lastcontint** = 2^{53} may not be precise.

See also: **fact**, **math.lnbinomial**, **over**.

besselj (n, x)

Returns the Bessel function of the first kind. The order is n given as the first argument, the argument x as the second argument. The return is a number. The function works on both numbers and complex values. (OS/2 and DOS do not support complex numbers).

See also: **bessely**, **calc.bessel0**, **calc.bessel1**, **long.besselj**.

bessely (n, x)

Returns the Bessel function of the second kind. The order n is given as the first argument, the argument x as the second argument. The return is a number. The function works on both numbers and complex values. (OS/2 and DOS do not support complex numbers).

See also: **besselj**, **calc.bessel0**, **calc.bessel1**, **long.bessely**.

cabs (z [, option])

If z is a number, the **cabs** function returns the absolute value of z as a number (default) or abs(z) + I*0 if any option is given.

If z is a complex number z = x + I*y, contrary to the **abs** operator, it returns the real and imaginary absolute value, i.e. |x| + I * |y|.

See also: ||, |-, **abs**, **argument**, **polar.**

cartesian (x, y)

The function returns a complex number z in Cartesian notation a + I*b for magnitude/modulus x and argument/phase angle y. x and y must be numbers. The result is equivalent to z = x * **cis**(y).

See also: ||, **abs**, **argument**, **cabs**, **cis**, **polar**, **!!** operator.

cas (x)

Returns the `casine` of the number, complex or dual number x the efficient way, i.e. sin(x) + cos(x) = $\sqrt{2}\sin(x+\frac{\pi}{4})$.

cathet (x, y)

Returns the number $\sqrt{x^2-y^2}$, with x, y numbers, complex or dual numbers or a mix of them. The function is slower but more precise than using **sqrt** along with **square**, avoiding over- and underflows, treating subnormal numbers accordingly and internally computing with 80-bit precision.

See also: **hypot**, **hypot2**, **hypot3**, **pytha4**, **root**, **sqrt**.

cbrt (x)

Returns the cubic root of the number, complex or dual number x. With complex x, it is equal to x^(1/3), but not to root(x, 3).

See also: ^ operator, **proot**, **root**, **sqrt**, **math.pow32**, **math.pow52**, **math.pow72**, **math.powe**.

ceil (x)

The function rounds upwards to the nearest integer larger than or equal to the number or complex number x. See the **entier** operator for a function that rounds downwards to the nearest integer. For the definition of **ceil**, see **entier**.

See also: **entier**, **floor**, **int**, **round**, **math.rint**.

cis (x)

The operator returns the complex exponential function exp(I*x) = cos(x) + I*sin(x) for any real or complex argument x. It is around 33 % faster than the equivalent expression exp(I*x). Note the equality **abs**(x) * **cis**(**argument**(x)) = x.

See also: **polar**.

conjugate (z)

The operator returns the conjugate x-I*y of the complex value z=x+I*y. If z is of type number, it is simply returned.

See also: **flip**.

cos (x)

The operator returns the cosine of x (in radians). Complex and dual numbers are supported.

See also: **math.cosd**, **math.cospi**, **math.sincos**.

cosc (x [, option])

With no option given, the function returns the un-normalised cardinal cosine of x (in radians), i.e. **cos**(x)/x, with **cosc**(0) = **undefined**. Complex and dual numbers are supported. This flavour is not deemed of much mathematical use by some.

With any option given, evaluates the first derivative of the **sinc** function at real or complex x (in radians):

$$\mathbf{cosc}(x) = \frac{x\cos(x) - \sin(x)}{x^2}, \text{ with } \mathbf{cosc}(0) = 0$$

See also: **math.rectangular**, **sinc**, **tanc.**

cosh (x)

The operator returns the hyperbolic cosine of x (in radians). Complex and dual numbers are supported. See also: **sinh**, **tanh**, **math.sinhcosh**.

cosxx (z)

The operator takes the complex number z = x!y and returns the complex number **cos**(x)*__cosh__(y)+I*__sin__(x)*__sinh__(y). It represents FRACTINT's buggy cos function till v16 where the imaginary part of the result had the wrong sign. This function may be mathematically useless, but it creates beautiful fractals. With the number z, it returns **cos**(z).

See also: **cos**, **bea**, **flip**.

cot (x)

Returns the cotangent -tan($\frac{\pi}{2}+x$) as a number (in radians). The function is implemented in Agena and included in the lib/library.agn file. The function works on numbers, complex and dual values. See also: **csc**, **sec**, **math.cotd**.

coth (x)

Returns the hyperbolic cotangent $\frac{1}{\tanh(x)}$ as a number (in radians). The function is implemented in Agena and included in the lib/library.agn file. The function works on numbers, complex and dual values. See also: **csch**, **sech**.

csc (x)

Returns the cosecant $\frac{1}{\sin(x)}$ as a number (in radians). The function is implemented in Agena and included in the lib/library.agn file. The function works on numbers, complex and dual values. See also: **cot**, **sec**, **math.cscd**.

csch (x)

Returns the hyperbolic cosecant as a number (in radians). The function is implemented in Agena and included in the lib/library.agn file. The function works on numbers, complex and dual values. See also: **coth**, **sech**.

cube (x)

The operator raises the number, complex or dual number x to the power of 3. See also: ******, **^** , **square** operators.

drem (x, y)

Evaluates the remainder of an integer division x/y (with x, y two Agena numbers), but contrary to **symmod**, rounds the internal quotient x/y to the nearest integer instead of towards zero. The function actually is a wrapper to C's remainder.

See also: \,**%**, **iqr**, **modf**, **modp**, **symmod**, **numtheory.invmod**, **math.modulus**.

entier (x)

The operator rounds the number x downwards to the nearest integer. For complex x, the return is:

re = real(x) - entier(real(x)) and im = imag(x) - entier(imag(x)),
then entier(x) = int(real(x)) + I*int(imag(x)) + X , where

$$X = \begin{cases} 0 & \text{if } a+b < 1 \\ 1 & \text{if } a+b \geqslant 1 \wedge a \geqslant b \\ 1 & \text{if } a+b \geqslant 1 \wedge a < b \end{cases}$$

Also: **ceil**(x) = -**entier**(-x). (With numbers, the function internally calls C's floor.)

See also: **ceil**, **floor**, **frac**, **int**, **mdf**, **round**, **xdf**, **fastmath.floor**, **math.rint**.

erf (x [, y])

Returns the error function of x. It is defined by $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_{t=0}^{x} e^{-t^2}$. The function works on numbers, complex and dual values. If two numbers or complex numbers x, y are given, computes the integral of the Gaussian distribution from x to y, with **erf**(x, y) = **erf**(y) - **erf**(x).

See also: **erfc**, **erfcx**, **erfi**, **inverf**.

erfc (x)

Returns the complementary error function of x, a number, complex or dual value. It is defined by erfc(x) = 1 - erf(x). The return is a number, complex or dual value.

See also: **erf**, **erfcx**, **erfi**, **inverfc**.

erfcx (x)

Implements the Scaled Complementary Error Function erfcx(x) = **exp**(x ^ 2)***erfc**(x), with x a number, complex or dual number and - depending on the type of x - a numeric, complex or dual-number result.

See also: **erfc**, **erfcx**, **erfi**, **inverfc**.

erfi (z)

Computes the imaginary error function erfi(z) = -I***erf**(I*z) for real or complex z. The type of return depends on the type of z.

See also: **erf**, **erfc**, **erfcx**, **inverfc**.

even (x)

Checks whether the number x is even. The operator returns **true** if x is even, and **false** otherwise. With non-integral numbers, the operator returns **false**. With the complex value x, the operator returns **fail**. See also: **odd**.

exp (x)

Exponential function; the operator returns the value e^x, with e Euler's number 2.71828... Complex and dual numbers are supported.

See also: **antilog2**, **antilog10**, **cis**, **exp2**, **exp10**, **expx2**, **math.expm1**.

exp2 (x)

The function returns 2^x with `x` any (complex, dual) number.

See also: `^` and **antilog2** operators, **exp**, **exp10**.

exp10 (x)

The function returns 10^x with `x` any (complex, dual) number.

See also: `^` and **antilog10** operators, **exp**, **exp2**.

expx2 (x [, sign])

Computes either $e^{x\hat{}2}$ if `sign` ≥ 0, or $e^{-x\hat{}2}$ if `sign` < 0 while suppressing error amplification that would occur from the in-exactness of the exponential argument x^2. `x` may be a number, complex or dual number, while `sign` must be a number. By default, `sign` is positive.

fact (n)

With non-negative and integral `n`, returns the factorial of `n`, i.e. the product of the values from 1 to `n`, that is $\prod_{k=1}^{n} k$ for `n` > 0, and $0! = 1$. With fractional negative or positive `n`, returns $\Gamma(n + 1)$, otherwise the function returns **undefined**.

See also: unary **!** operator, **math.dblfact**, **math.lnfact**, **math.trifact**, **math.fall**, **math.pochhammer**, **mulup**.

finite (x)

Checks whether the number or complex number `x` is neither ±**infinity** nor **undefined** (C NaN). The operator returns **true** or **false**.

See also: **even**, **fractional**, **infinite**, **nan**, **odd**, **math.isinfinity**, **math.isexceptional**, **math.isordered**.

flip (z)

The operator takes the complex number `z` and returns the new complex number **imag**(`z`)!**real**(`z`), i.e. the real and imaginary parts are swapped. With numbers, always returns 0.

See also: **bea**, **conjugate**, **cosxx**.

floor (x)

The function rounds downwards to the nearest integer larger than or equal to the number or complex number `x`. It works like the **entier** operator.

See also: **ceil**, **entier**, **int**, **fastmath.floor**.

fma (a, b, c)
fma (a, b, [c, d, [, ···]] z)

In the first form, performs the fused multiply-add operation `(a * b) + c`, with the intermediate result *not* rounded to the destination type, to improve the precision of a calculation. `a`, `b`, and `c` must be numbers or complex numbers. Note that the operation takes more time than with basic arithmetic and that in many real-world situations there are no better results.

In the second form, two or more multiply-add operations can be chained. So

```
> fma(2, 3, 4!0, 5!0, 6!1, 7!2, 1):
67+19*I
```

is short for:

```
> fma(2, 3, fma(4!0, 5!0, fma(6!1, 7!2, 1))):
67+19*I
```

See also: **squareadd** and **muladd** operators.

frac (x)

Returns the fractional part of the number `x`, i.e. `x - ` **int**(`x`), thus preserving the sign. With complex numbers a + I*b, returns **frac**(a) + I***frac**(b).

See also: **entier**, **int**, **modf**.

fractional (x)

Checks whether the number `x` has a fractional part so that it is not an integer, and returns **true** or **false**.

With complex numbers, if returns **true** if the real part is fractional and the imaginary part is zero, and **false** otherwise.

If `x` is not a (complex) number, the operator returns **fail**. With +/-**infinity** and **undefined**, returns **false**.

See also: **finite**, **integral**, **isint**.

frexp (x)

Returns the mantissa m and the exponent e of the number `x` such that $x = m2^e$. e is an integer, and the value of m is in the range [0.5, 1) (or zero when `x` is zero). The operation is bijective, i.e. **ldexp**(**frexp**(`x`)) = `x`. With complex `x`, returns m and e both for the real and the imaginary part.

See also: **frexp10**, **ilog2**, **ldexp**, **math.exponent**, **math.frexp**, **math.mantissa**.

frexp10 (x)

Returns the mantissa m and the exponent e of the number `x` such that $x = m10^e$. e is an integer, and the value of m is in the range [0, 1).

Since floats are represented with base 2, and not base 10, the operation is not bijective, i.e. **ldexp**(**frexp**(`x`)) ≠ `x`. With complex `x`, returns m and e both for the real and the imaginary part.

See also: **frexp**.

foreach (start, stop [, step], f, n)

The operator traverses a numeric range, starting with `start` (a number) and stopping at `stop` (a number) with step size `step`, applies a univariate function `f` on each intermediate value and sums them all up. The operator allows to omit the `step` size - defaulting to one in this case. The sum is initialised to `n` and the operator applies Kahan-Babuška summation for the iteration control variable:

```
> # Pi approximation by Indian mathematician and astronomer
> # Madhava of Sangamagrama, 14th century AD:

> sqrt(12)*foreach(0, 25, <: k -> (-3)^(-k)/(2*k + 1) :>, 0):
3.1415926535898
```

See also: **reduce**, **sumup**, **times**, **calc.fsum**, **stats.sumdata**.

gamma ([a ,] x)

With only `x` given, computes the gamma function Γ `x`. `x` may be a number, complex or dual value.

With real non-negative `a` and real non-negative `x` computes the incomplete gamma function equivalent to:

```
> if a = 0.0 then
>    return calc.Ei(1, x)
> elif if x = 0.0 then
>    return gamma(a);
> else
>    return calc.igammac(a, x)*gamma(a);
> fi;
```

See also: **calc.gammainc**, **invgamma**, **lngamma**, **math.gammasign**.

heaviside (x [, z])

The Heaviside function. Returns 0 if $x < 0$, 1 if $x > 0$, and `z` if $x = 0$, where `z` defaults to **undefined**. The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.smoothstep**, **math.clip**, **math.dirac**, **math.ramp**, **math.rectangular**, **math.unitise**, **math.unitstep**.

hypot (x, y)

hypot (z)

Returns $\sqrt{x^2+y^2}$ with `x`, `y` numbers or complex numbers or a mix of them. `x` and `y` can also be dual numbers. With `x`, `y` numbers, it is the length of the hypotenuse of a right triangle with sides of length `x` and `y`, or the distance of the point (`x`, `y`) from the origin. The function is slower but more precise than using **sqrt** along with **square**, avoiding over- and underflows and treating subnormal numbers accordingly. The return is a number or complex number.

With `z` the complex number `x + y*I`, returns $\sqrt{x^2+y^2}$, as well.

See also: **hypot2**, **hypot3**, **cathet**, **invhypot**, **pytha**, **root**, **sqrt**, **calc.euclliddist**, **math.lnhypot**.

hypot2 (x)

Returns the number, complex or dual number $\sqrt{1+x^2}$, with `x` a number, complex or dual number. The function is slower but more precise than using **sqrt** along with **square**, avoiding over- and underflows and treating subnormal numbers accordingly.

See also: **hypot**, **hypot3**, **cathet**, **root**, **sqrt**.

hypot3 (x)

Returns the number $\sqrt{1-x^2}$, with x a number, complex or dual number. The function is slower but more precise than using **sqrt** along with **square**, avoiding over- and underflows and treating subnormal numbers accordingly.

See also: **hypot**, **hypot2**, **cathet**, **root**, **sqrt**.

ilog2 (x)

Extracts the exponent of the number or complex number x (i.e. the integer part of the base-2 logarithm of the positive number x) and returns it as the number **entier**(**log2**(x)).

See also: **frexp**, **ilog10**, **ln**, **log**, **log2**, **log10**, **math.ceillog2**, **math.ispow2**. **utils.ilog2**.

ilog10 (x)

Extracts the exponent of number x and returns it as the number **entier**(**log10**(x)).

See also: **ilog2**, **log10**.

imag (x)

Extracts the imaginary part of the complex number x and returns it as a number. If x is a number, the operator just returns zero.

See also: **real**, **tonumber**.

implies (x, y)

With two booleans, the function computes **not**(x) **or** y. When given **null**, the value is treated like **false**. With numbers returns:

$$(\sim\sim x) \;||\; y.$$

infinite (x)

Checks whether the number or complex number x is ±**infinity**. The operator returns **true** or **false**.

See also: **even**, **fractional**, **finite**, **nan**, **odd**,**math.isexceptional**, **math.isinfinity**, **math.isordered**.

int (x)

Rounds x to the nearest integer towards zero. The operator also supports complex numbers. To round a float to a given decimal place, use **xdf**. To get the fractional part of a number, call **frac**.

See also: \ operator, **ceil**, **entier**, **fractional**, **iqr**, **mdf**, **modf**, **round**, **math.rint**, **xdf**.

integral (x)

Checks whether the number x is an integer, i.e. not a fraction, and returns **true** or **false**.

With complex numbers, if returns **true** if the real part is integral and the imaginary part is zero, and **false** otherwise.

If x is not a (complex) number, the operator returns **fail**. With +/-**infinity** and **undefined**, returns **false**.

See also: **finite**, **fractional**, **isint**, **multiple**.

inverf (x)

Computes the inverse error function $\text{erf}^{-1}(x)$, where x is a number or dual number.

See also: **erf**, **inverfc**.

inverfc (x)

Computes the inverse complimentary error function $\text{erfc}^{-1}(x)$, where x is a number or a dual.

See also: **erfc**, **inverfc**.

invgamma (x)

Computes the inverse gamma function 1/gamma(x), where x is a number.

See also: **gamma**, **lngamma**.

invhypot (x, y)
invhypot (z)

In the first form, computes $1/\sqrt{x^2+y^2} = 1/$**hypot**(x, y), with x, y numbers, complex or dual numbers. In the second form, with complex z = x + I*y, does the same in the complex domain. You can mix numbers with complex numbers.

With x, y numbers, the function is 35 % faster than the naive 1/**hypot** approach and is protected against underflow and overflow. With complex numbers x, y, it is five

percent faster than evaluating the expression 1/**hypot**(x, y) and internally computes with 80-bit precision.

See also: **hypot**, **invpytha**.

invpsi (x)

Evaluates the inverse Psi/Digamma function for number x. See also: **psi**.

invpytha (x, y)
invpytha (z)

In the first form, computes $1/(x^2+y^2) = 1/$**pytha**(x, y), with x, y numbers or complex numbers.
In the second form, with complex z = x + I*y, does the same in the complex domain. You can mix numbers with complex numbers.

With x, y numbers, the function is protected against underflow and overflow. With complex numbers x, y, it internally computes with 80-bit precision.

See also: **invhypot**, **pytha**.

invsqrt (x)

Returns the inverse square root of numeric, complex or dual x, i.e. 1/**sqrt**(x).

See also: **math.isqrt**, **math.pow32**, **math.pow32**, **sqrt**.

iquo (x, y)

Computes the integer quotient of the two integers x and y. See also: **iqr**, **irem**.

iqr (x, y)

Computes both the integer quotient and the integer remainder - rounded toward zero - of the number x divided by the number y and returns them. If x or y are not integers, the function returns **undefined** twice. The function is equivalent to the Agena representation:

```
iqr := proc(x :: number, y :: number) is
   if fractional(x) or fractional(y) then
      return undefined, undefined
   else
      return x \ y, x symmod y
   fi
end;
```

See also: \ and **%** operators, **drem**, **iquo**, **irem**, **modf**, **modp**, **math.cld**, **math.fld**, **numtheory.invmod**, **math.modulus**, **symmod**.

irem (x, y)

The function computes the integer remainder - rounded toward zero - of the integers `x` and `y`. It is equivalent to the **symmod** operator.

See also: **drem**, **iquo**, **irem**.

iscomplex (···)

Checks whether the given arguments are all of type **complex** and returns **true** or **false**.

isint (···)

Checks whether all of the given arguments are integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **fractional**, **integral**.

isnegative (···)

Checks whether all of its arguments are negative numbers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnegint**, **isnegative**, **isnonneg**, **ispositive**.

isnegint (···)

Checks whether all of the given arguments are negative integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnonnegint**, **isposint**, **isnegative**, **ispositive**.

isnonneg (···)

Checks whether all of its arguments are zero or positive numbers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnegint**, **isposint**, **isnegative**, **ispositive**.

isnonnegint (···)

Checks whether all of the given arguments are zeros or positive integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnonzeroint (···)

Checks whether all of the given arguments are non-zero integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnonposint (···)

Checks whether all of the given arguments are zeros or negative integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

isnumber (···)

Checks whether the given arguments are all of type **number** and returns **true** or **false**.

isnumeric (···)

Checks whether the given arguments are all of type **number** or of type **complex** and returns **true** or **false**.

See also: **numeric**.

isposint (···)

Checks whether all of its arguments are positive integers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isnonposint**.

ispositive (···)

Checks whether all of its arguments are positive numbers and returns **true** or **false**. If at least one of its arguments is not a number, the function returns **fail**.

See also: **isposint**, **isnegative**, **isnonneg**.

ldexp (m, e)

Returns $m2^e$ (`e` should be an integer, and `m` must be a number).

See also: **frexp**.

ln (x)

Natural logarithm of `x` with the base e^1. If `x` is non-positive, the operator returns **undefined**. Complex and dual numbers are supported.

See also: **exp**, **lngamma**, **log**, **log2**, **log10**, **math.ln*** functions.

lngamma (x)

Computes ln Γ x. If x is a non-positive number, the operator returns **undefined**. Complex and dual numbers are supported.

See also: **gamma**, **invgamma**, **psi**, **math.gammasign**.

log (x, b)

The operator returns the logarithm of the number or complex number x to the base b, with b a number or a complex number.

See also: **ilog2**, **ilog10**, **ln**, **log2**, **log10**, **math.logs**.

log2 (x)

Returns the base-2 logarithm of the number, complex or dual number x.

See also: **antilog2**, **ilog2**, **ln**, **log**, **log10**, **math.ceillog2**.

log10 (x)

Returns the base-10 logarithm of the number, complex or dual number x.

See also: **antilog2**, **ilog10**, **ln**, **log**, **log2**.

mdf (x [, n])

Rounds up the number x at its n-th decimal place and returns a number. If x is positive, rounds towards $+\infty$; if x is negative, rounds towards $-\infty$. The default of n is 2. With complex x, rounds both the real and imaginary parts.

See also: **entier**, **int**, **round**, **xdf**.

modf (x)

Returns two numbers, the integral part of the number x and its fractional part. The integral part is rounded towards zero. Both the integral and fractional part of the return have the same sign as x. The sum of the two values returned equals x. The function actually is a wrapper to C's modf. With complex x, returns the integral and fractional parts for both its real and the imaginary part.

See also: \, **%**, **frac**, **entier**, **int**, **modp**, **symmod**, **mod** assignment statement.

modp (x, y)

The function emulates Maple's mod()/modp() functions for integral `x`, `y`. The return is equivalent to:

```
> modp := proc(x, y) is
>    local r := x - ceil(x/y)*y;
>    return r + (r < 0)*|y|
> end;
```

See also: \, **%**, **frac**, **entier**, **int**, **symmod**, **mod** assignment statement.

muladd (a, b, c [, true])

muladd (a, b, [c, d, [, ···]] z [, true])

In the first form, the operator computes `a*b + c` , with the intermediate result *not* rounded to the destination type, to improve the precision of a calculation. `a`, `b`, `c` may be all numbers or all complex numbers or a mix of them. **muladd** uses 80-bit precision on Intel platforms to compute the complex result if you pass the optional last argument **true**, the default is **false**.

In the second form, two or more multiply-add operations can be chained. So

```
> muladd(2, 3, 4!0, 5!0, 6!1, 7!2, 1):
67+19*I
```

is short for:

```
> muladd(2, 3, muladd(4!0, 5!0, muladd(6!1, 7!2, 1))):
67+19*I
```

multiple (x, y [, option])

Checks whether numeric or complex `x` is a multiple of numeric `y`, i.e. whether `x`/`y` evaluates to an integral, and returns **true** or **false**.

Also returns **true** with `x` = 0 and any non-zero `y`.

If `y` is zero, **undefined** or +/-**infinity**, the function returns **fail**.

With complex `x`, returns **true** if both **real**(`x`)/`y` and **imag**(`x`)/`y` evaluate to the same integral, or if **real**(`x`)/`y` evaluates to an integral and **imag**(`x`) is zero.

By passing the optional third argument **true**, a tolerant check is done, with subnormal `x` or `y` first converted to zero, and a subsequent approximate equality check to the nearest integer of `x`/`y`. The tolerance value internally used is the value of **DoubleEps** at the time of the function call.

In most cases, it may suffice to just call **integral**(x/y), which is twice as fast as this function.

See also: **math.nextmultiple**, **math.prevmultiple**.

nan (x)

Checks whether the number or complex number x evaluates to **undefined** (NaN). The operator returns **true** or **false**.

See also: **finite**, **fractional**, **infinite**, **math.isexceptional**, **math.isordered**.

x nand y

The operator returns the bitwise complement Boolean `and`, a signed integer: ~~(x **&&** y).

See also: **bytes.nand32**.

x nor y

The operator returns the bitwise complement Boolean `or`, a signed integer: ~~(x || y).

See also: **bytes.nor32**.

nonzero (x)

Checks whether the number or complex number x is neither 0 nor 0+0*I, respectively. The operator returns **true** or **false**.

See also: **math.isminuszero**, **zero**.

odd (x)

Checks whether the number x is odd. The operator returns **true** if x is odd, and **false** otherwise. With non-integral numbers, the operator returns **false**. With the complex value x, the operator returns **fail**.

See also: **even**.

polar (z)

Transforms the complex number z in Cartesian notation or the number z to polar form and returns two numbers: the magnitude (modulus) and the argument (phase angle), in this order. If z is a number and is zero, or if z is complex and its real and imaginary parts equal zero, the function returns zero twice.

See also: ||, **abs**, **argument**, **cabs**, **cartesian**, **cis**, **!!** operator.

proot (x, n)

Returns the principal n-th root of the number or complex value x. n must be a positive integer. The principal n-th root in the complex domain is the first root found starting from the positive real axis going counter-clockwise.

See also: **cbrt**, **hypot**, **hypot2**, **hypot3**, **cathet**, **root**, **sqrt**, **math.pow32**, **math.pow32**.

psi ([n,] x)

In the first form, with only x given, computes the Psi (digamma) function, the logarithmic derivative of the gamma function, for a number, complex or dual number x. The return is the number **calc.xpdiff**(**lngamma**(x), x).

In the second form, with n an integer and x a number, computes:

- n=0: The digamma function, equal to **psi**(x) but since it uses a different algorithm the result may differ slightly.
- n=1: The trigamma function.
- n=2: The tetragamma function.
- n=3: The pentagamma function.

See also: **calc.harmonic**, **invpsi**, **gamma**, **lngamma**, **long.psi**.

pytha (a, b)

pytha (z)

Computes the Pythagorean equation $c^2 = a^2 + b^2$, without undue underflow or overflow and treating subnormal numbers accordingly, for numbers or complex numbers a, b. The function internally computes in 80-bit precision instead of 64-bit precision. Please note that all these features take the function almost twice the time to complete as the naive x**2 + y**2 approach. You can mix numbers with complex numbers.

With z representing the complex number a + I*b, returns $a^2 + b^2$, as well, avoiding undue underflow with $a + b \sim 0$.

Dual numbers are supported.

See also: **hypot**, **invpytha**, **math.lnpytha**, **squareadd**.

pytha4 (a, b)

Computes a^2 - b^2, without undue underflow or overflow and treating subnormal numbers accordingly, for numbers or complex numbers a, b. The function internally computes in 80-bit precision instead of 64-bit precision. Please note that all these features take the function almost twice the time to complete as the naive x**2 -

y**2 approach. You can mix numbers with complex numbers. Dual numbers are supported.

See also: **cathet**, **hypot**, **squareadd**.

qmdev (o)

The operator computes the sum of the squared deviations of each observation o_i in the sequence, register, userdata or table o, from its arithmetic mean μ, i.e.

$$\sum_{i=1}^{n}(o_i - \mu)^2$$

The return should be divided either by the number of elements n in the distribution o to calculate its population variance, or by n - 1 to compute its sample variance. With tables, only numeric entries with integral keys will be processed.

The Knuth-Welford algorithm used by the operator tries to prevent round-off errors as much as possible. The operator returns **fail** if it finds at least one complex number in o and simply ignores any non-numbers otherwise.

Note: If the distribution should contain very large samples, you might better use **stats.meanvar** to better prevent numeric overflow.

See also: **stats.issorted**, **stats.sd**, **stats.var**.

real (x)

Extracts the real part of the complex number x and returns it as a number. If x is a number, the operator just returns x.

See also: **imag**, **tonumber**.

recip (x)

Returns the inverse $1/x$ of a number, complex or dual number x.

See also: /, **fastmath.reciprocal**.

root (x [, n])

Returns the non-principal n-th root of the number or complex value x. n must be an integer and is 2 by default. Note, that since the function computes the non-principal root, with complex x, **root**(x, n) ≠ x^ ($1/n$). In the complex domain, the function returns the n-th root of x whose argument is nearest to the argument of x.

See also: **argument**, **cbrt**, **hypot**, **hypot2**, **hypot3**, **cathet**, **proot**, **sqrt**, **math.pow32**, **math.pow32**.

round (x [, d])

Rounds the number x to its d-th digit, using the round-half-up method. The return is a number. If d is omitted, the number is rounded to the nearest integer. If d is positive, the function rounds to the d-th fractional digit. If d is negative, it rounds to the d-th integral digit.

round treats positive and negative values symmetrically, and is therefore free of sign bias. With complex numbers x=a+I*b returns **round**(**real**(a), d) + I*__round__(**imag**(a), d).

The following Agena code explains the algorithm used:

```
round := proc(x, d) is
   d := d or 0;  # assign zero if d is null
   return int((10^d)*x + sign(x)*0.5) * (10^(-d))
end;
```

See also: **ceil**, **entier**, **int**, **mdf**, **xdf**, **math.rint**.

scalbn (x, n)

Just an alias for **ldexp**.

sec (x)

Returns the secant $\frac{1}{\cos(x)}$ as a number (in radians). The function is implemented in Agena and included in the lib/library.agn file. The function works on both numbers, and complex and dual values. See also: **cot**, **csc**, **math.secd**.

sech (x)

Returns the hyperbolic secant as a number (in radians). The function is implemented in Agena and included in the lib/library.agn file. The function works on numbers, complex and dual numbers. See also: **coth**, **csch**.

sign (x)

Determines the sign of the number or complex value x. The result of the operator is determined as follows:

- 1, if real(x) > 0 or real(x) = 0 and imag(x) > 0
- -1, if real(x) < 0 or real(x) = 0 and imag(x) < 0
- 0 otherwise, even for -0.

If x is **undefined**, **sign** returns **undefined**.

See also: **math.copysign**, **math.flipsign**, **math.mulsign**, **signum**, | operator.

signum (x)

Determines the sign of the number or complex value x. x may also be a Boolean.

If x is a number, the result of the operator is determined as follows:

- 1, if $x \geq 0$
- -1 otherwise.

With complex x, the operator returns $x/|x|$ or 0 if x is zero.

If x is **undefined**, **signum** returns **undefined**.

With Booleans, returns

- 1, if x is **true**,
- -1 otherwise, i.e. **false** or **fail**.

See also: **math.copysign**, **math.mulsign**, **abs**, **sign**, | operator.

sin (x)

The operator returns the sine of x (in radians). Complex and dual numbers are supported.

See also: **math.sincos**, **math.sind**, **math.sinpi**.

sinc (x)

The operator returns the un-normalised cardinal sine of x (in radians), i.e. $\sin(x)/x$, with sinc(0) = 1. Complex and dual numbers are supported.

See also: **cosc**, **math.rectangular**, **tanc.**

sinh (x)

The operator returns the hyperbolic sine of x (in radians). Complex and dual numbers are supported.

See also: **cosh**, **tanh**, **math.sinhcosh**.

sqrt (x)

Returns the square root of x.

If x is a number and negative, the operator returns **undefined**.

With complex numbers, the operator returns the complex square root, in the range of the right halfplane including the imaginary axis. Dual numbers are supported.

See also: **cbrt**, **hypot**, **hypot2**, **hypot3**, **invsqrt**, **proot**, **root**, **fastmath.sqroot**, **fastmath.sqrtfast**, **math.isqrt**, **math.pow32**, **math.pow52**, **math.pow72**, **math.powe**.

square (x)

The operator squares the number, complex or dual number x and returns x**2.

See also: **, ^, **cube** operators, **numtheory.issquare**, **squareadd**.

tan (x)

The operator returns the tangent of x (in radians). Complex and dual numbers are supported.

Note that due to internal argument reduction and resulting slight round-off errors, the function cannot properly detect poles - so instead of **undefined** the function will return a rather big value.

See also: **math.tand**, **math.tanpi**.

tanc (x)

The operator returns the un-normalised cardinal tangent of x (in radians), i.e. tan(x)/x, with tanc(0) = 1. Complex and dual numbers are supported.

See also: **cosc**, **math.rectangular**, **sinc.**

tanh (x)

The operator returns the hyperbolic tangent of x (in radians). Complex and dual numbers are supported.

See also: **cosh**, **sinh**.

xdf (x [, n])

Rounds down the number x at its n-th decimal place towards zero and returns a number. This is equivalent to truncating a float at its n-th decimal place. The default of n is 2.

With complex x, rounds both the real and imaginary parts.

See also: **entier**, **int**, **round**, **mdf**.

x xnor y

With numbers, the operator returns the bitwise complement Boolean `xor`, a signed integer: ~~(x ^^ y). With Booleans, returns **not**(x **xor** y), sometimes also called `if-and-only-if` (iff). See also: **bytes.xnor32**.

x xor y

With Booleans, returns x <> y. With non-booleans: returns the first operand if the second operand evaluates to **null**, otherwise the second operand will be returned.

See also: ^^, **bytes.xor32**.

zero (x)

Checks whether the number or complex number x is 0 or 0+0*I, respectively. The operator returns **true** or **false**.

See also: **math.isexceptional**, **math.isminuszero**, **nonzero**.

11.1.2 math Library

This library is an interface to the standard C math library. It provides all miscellaneous functions inside the table `math`.

`math.accu ([init [, method]])`

Returns a factory that gets a number or complex number with each call, adds it to an internal accumulator, and returns the accumulated sum. If the iterator is called with no argument, the current accumulated sum will be returned.

The function can be used if high accuracy numeric summation is needed. The initial value of the accumulator is 0. If `init`, a number or complex number, is given, the accumulator is set to `init` instead.

The function automatically takes care of storing and processing internal correction values - so the user does not have to worry about this.

By default, Neumaier summations is used. By passing a `method` (of type string), you may use an alternative algorithm to add numbers:

method	algorithm
`'babuska'`	Kahan-Babuška summation, highest accuracy but slowest
`'kahan'`	classic Kahan summation, lowest accuracy but fastest
`'kbn'`	Kahan-Babuška-Neumaier compensated summation, used in the Julia programming language
`'neumaier'`	Neumaier summation, good accuracy and performance (default)
`'ozawa'`	Kahan-Ozawa summation
`'raw'`	no auto-correction

See also: **factory.count**, **math.kbadd**, **math.koadd**, **sumup**.

`math.agm (a, g)`

Approximates the (Gauss') arithmetic–geometric mean of two real or complex numbers `a`, `g`, that is the mutual limit of the sequence:

$a_0 := a$; $g_0 := g$
$a_{n+1} := \frac{1}{2}(a_n + g_n)$; $g_{n+1} := \sqrt{a_n * g_n}$

The return is a number or complex number - depending of the type of `a` and `g` - between the geometric and arithmetic mean of `a` and `g`, thus between `a` and `g`, as well. When calling the function, numbers and complex numbers can be mixed.

See also: **math.hgm**, **stats.amean**, **stats.gmean**.

math.beta (x, y)

Computes the Beta function with `x` and `y` numbers, avoiding internal over- and underflow. The return is a number. The function is used by **beta** when given real arguments.

See also: **beta**, **math.gammasign**, **math.lnbeta**.

math.bintodec (s)

Converts a string `s` representing a binary value to a (decimal) number. The string may or may not start with '0b' or '0B', and also may contain a sign, a fractional part or a `p exponent` part.

If the value represented by `s` is too big, the function will return **undefined**.

Example: `math.bintodec('-0b1111.1111')` ⇒-15.9375.

See also: **math.convertbase**, **math.hextodec**, **math.octtodec**, **math.tohex**, **tonumber**.

math.branch (x [, d [, subs]])

Returns its argument `x` - a number - if `x` is non-negative, otherwise returns 0. By passing any non-negative optional number `d` (the `direction`), the return is the same.

By passing any non-positive optional number `d`, returns `x` if it is negative, otherwise returns 0.

If `x` should be **undefined**, you can return any other number by passing the optional argument `subs`, which is **undefined** by default.

See also: **math.clip**, **math.wrap**, end of Chapter 11.1.2 for a comparison chart.

math.ceillog2 (x)

Returns the smallest exponent to 2 equals or greater than `x`, i.e. **ilog2(**`x` - 1) + 1, where `x` is a positive integer. If `x`= 1, the result is 0; if `x` < 1, **undefined** will be returned.

See also: **ilog2**, **math.ceilpow2**.

math.ceilpow2 (x)

Finds the smallest power of 2 greater than or equal to `x`, where `x` is a non-negative integer. If `x` = 0, the result is 1; if `x` < 0, **undefined** will be returned. Examples: math.ceilpow2(3) ⇒ 4 = 2^2, and math.ceilpow2(8) ⇒ 8 = 2^3.

The function returns **fail** if `x` ≥ 2^{31}.

See also: **ilog2**, **math.ceillog2**, **math.floorpow2**, **math.ispow2**.

math.chi (x [, a [, b]])

The piecewise indicator function, is defined as follows:

- **math.chi**(`x`, `a`, `b`) simplifies to 1 if `a` < `x` < `b`; to 0 if `x` < `a` < `b` or if `a` < `b` < `x`; and to -**math.chi**(`x`, `b`, `a`) if `b` < `a`. Otherwise **math.chi**(`x`, `a`, `b`) simplifies to **sign**(`x` - `a`)/2 -**sign**(`x` - `b`)/2;
- **math.chi**(`x`) simplifies to **math.chi**(x, 0, 1);
- **math.chi**(`x`, `a`) simplifies to **math.chi**(`x`, `a`, 1).

See also: **math.clip**, **math.piecewise**.

math.chop (x [, eps [, method [, n]]])

Shrinks a number or complex number `x` more or less near zero to exactly zero, using one of many methods, passed as an integer. The default for `eps` is **Eps**. The standard `method` is 0 for hard shrinking. `n` is used in the SmoothGarrote `method`.

method	Comment	Value	Domain
0	"Hard", performs hard shrinking	0 x	|x| ≤ eps |x| > eps
1	"Soft", performs soft shrinking	0 sign(x) (|x| - eps)	|x| ≤ eps |x| > eps
3	"PiecewiseGarrote"	0 x - eps^2/x	|x| ≤ eps |x| > eps
4	"SmoothGarrote"; with n → ∞, goes to "Hard" shrinking	$x^{2n+1}/(x^{2n} + eps^{2n})$	any x
5	"Hyperbola"	0 sign(x) $\sqrt{x^2 - eps^2}$	|x| ≤ eps |x| > eps

Method 2 has not been implemented. The function is a port of Mathematica's Chop function.

See also: **math.clip**, **math.unitise**, **math.zeroin**.

math.cld (x, y)

Returns the largest integer less than or equal to the real quotient $\frac{x}{y}$ of the numbers x and y.

See also: \ operator, **math.fld**.

math.clip (x [, a [, b [, f]]])
math.clip (x, a)

In the first form, returns x clipped to be between a and b. The return is x if $a \leq x \leq b$, a if $x < a$, and b if $x > b$. By default $a = -1$, $b = +1$. If function f is given which should return one numeric result, then if x is not in [a, b], the result of f(x) will be returned.

In the second form, returns x clipped to be between -a and +a, -a if $x < a$ and +a if $x > +a$.

See also: **calc.sigmoid**, **heaviside**, **math.branch**, **math.chi**, **math.chop**, **math.rectangular**, **math.unitise**, **math.wrap**, end of Chapter 11.1.2 for a comparison chart.

math.compose (coeffs [, b])

Takes a table, sequence or register of coefficients coeffs and a base b and returns the composed number. In coeffs, the highest-order digit as the first element and the lowest-order digit as the last element. By default, the base is 10. The function does not take care of potential overflows. It is the complement to **math.decompose**.

math.convertbase (s, a, b [, anything])

Converts a number s or a number represented as a string s from base a to base b. a and b must be integers in the range 1 to 36. The number in s must be an integer of any sign. Floats are not allowed. The return is a string. The function is implemented in Agena and included in the lib/library.agn file.

If you pass any fourth argument, then the function internally does not call optimised functions, if appropriate, such as **math.hextodec** or **strings.strtoul** to speed up results.

See also: **math.decompose**, **math.bintodec**, **math.hextodec**, **math.octtodec**, **math.ndigits**, **strings.strtoul**.

math.copysign (x, y)

Returns a number with the magnitude of `x` and the sign of `y`, i.e. abs(`x`) * sign(`y`). If `y` is 0, then its sign is considered to be 1. It is a plain binding to C's copysign function and does not post-process its result.

See also: **math.flipsign**, **math.mulsign**, **math.signbit**, **sign**, **signum**.

math.coscpi (x [, option])

With only `x` given, returns **cos**(π*`x`)/(π*`x`) with number or complex number `x`, preserving accuracy as much as possible especially with larger `x`. With `x` = 0, returns **undefined**.

With any second option, returns the first derivative of **math.sincpi** at real or complex `x`:

$$\frac{d}{dx}\frac{\sin(\pi\, x)}{\pi\, x} = \frac{\cos(\pi\, x)\pi\, x - \sin(\pi\, x)}{\pi x^2}, \text{ 0 if } x = 0.$$

In both ways, the type of return depends on the type of the input.

See also: **cosc**, **sinc**, **tanc**, **math.sincpi**, **math.tancpi**.

math.cosd (x)

Takes `x` in degrees and returns its cosine in radians.

See also: **cos**, **math.cospi**, **math.cscd**, **math.cotd**, **math.secd**, **math.sind**, **math.tand**.

math.cospi (x)

Returns cos(π*`x`) for number or complex number `x` with better precision than calling the respective standard operator.

See also: **cos**, **math.cosd**, **math.sincos**, **math.sincospi**, **math.sinpi**, **math.tanpi**.

math.cotd (x)

Takes `x` in degrees and returns its cotangent in radians.

See also: **cot**, **math.cosd**, **math.cscd**, **math.secd**, **math.sind**, **math.tand**.

math.cscd (x)

Takes `x` in degrees and returns its cosecant in radians.

See also: **csc**, **math.cosd**, **math.cotd**, **math.secd**, **math.sind**, **math.tand**.

math.dblfact (n)

Returns the double factorial n!! = n(n - 2)(n - 4)...3*1 for non-negative integer n.

See also: **fact**, **math.fall**, **math.lnfact**, **math.trifact**.

math.dd (x)

Converts a number x representing a decimal number in TI-30 DMS notation into its decimal representation, and returns a number. For example with 10.3045 representing 10°30'45", the function returns 10.5125.

The complement to **math.dd** is **math.dms**.

See also: **clock** package, **math.splitdms**, **math.todecimal**, **math.tosgesim**.

math.decompose (x [, b])

Splits an integer x to the base b into its digits and returns them in a sequence, with the highest-order digit as the first element and the lowest-order digit as the last element. Any sign of x is ignored. By default, the base is 10, but you may choose any other positive base. Example:

```
> b := 256;

> math.decompose(15 * b^2 + 7 * b + 1, 256):
seq(15, 7, 1)
```

See also: **math.compose**, **math.convertbase**, **math.ndigits**.

math.dirac (x [, eps] [, classic = true])

The Dirac delta function, also known as the impulse function, returns 0 for all numbers x other than 0, and by default **undefined** if x = 0, iff eps is set to zero which is the default. If you pass the `classic = **true**` option, however, the function will return **infinity** with x = 0 and eps = 0.

If eps is set to any positive value x, returns 1/(2*eps)*exp(-|x|/eps) even if x = 0.

See also: **heaviside**.

math.dms (x)

Converts a number representing a decimal number x into its TI-30 decimal DMS notation and returns a number. For example with 10.5125 representing 10°30'45", the function returns 10.3045.

The complement to **math.dms** is **math.dd**.

See also: **clock** package, **math.splitdms**, **math.todecimal**, **math.tosgesim**.

math.eps ([x [, option]])

The function returns the machine epsilon, the relative spacing between the number |x| and its next larger number in the machine's floating point system. If no argument is given, x is set to 1.

On x86 machines and with Agena numbers, i.e. C doubles, eps(1) and eps() return 2.2204460492503e-016 = 2^{-52}, and eps(2) returns 4.4408920985006e-016 = 2^{-51}.

When given any second argument, the function computes a `mathematical` epsilon value that is also dependent on the magnitude of its argument x. It can be used in difference quotients, etc., for it prevents huge precision errors with computations on very small or very large numbers. The mathematical epsilon with respect to x is equal to x* **sqrt**(**math.eps**(x)).

Besides numbers, the function accepts complex numbers as long as their imaginary part is zero.

See also: **calc.eps**, **math.epsilon**, **math.nextafter**.

math.epsilon (x [, method])
math.epsilon (f, x [, ···] [, iters=n])

In the first form, by default returns the relative spacing between |x| and its next larger number on the machine's floating point system, taking into account the magnitude of its argument, thus computing some sort of `mathematical` instead of machine epsilon value. In this case, the function works like **math.eps** with the **true** option set, but is 20 percent faster.

In the first form, you may choose between different methods to determine an epsilon value, where ulp = **math.nextafter**(x, **infinity**) - x:

Method	Formula	Comment
0 (default)	x * **sqrt**(ulp)	**Eps** for |x| < 1
1	x * **cbrt**(ulp)	**Eps** for |x| < 0.0123927159
2	**sqrt**(ulp) * (x +**sqrt**(ulp))	
3	**cbrt**(ulp) * (x +**cbrt**(ulp))	

With methods 0 and 1, the function returns **Eps** with x `around` zero. This prevents a generated epsilon value of exactly zero or very close to zero.

In the second form, by passing a function f and an argument x, the function determines an epsilon value by taking the function value f(x) into account, using a divided difference table. If f is multivariate, pass its further arguments tight after x. In this form, the function returns the original epsilon value h determined, plus 100*h if h is too small and the absolute error, in this order.

Besides numbers, the function accepts complex numbers as long as their imaginary part is zero.

See also: **calc.eps**, **math.eps**.

math.eq (x, y)

Checks whether the number x is equal to the number y and returns **true** or **false**.

See also: <, <=, >, >=, <> operators, **math.ge**, **math.gt**, **math.le**, **math.lt**, **math.neq**.

math.expm1 (x)

Returns a value equivalent to exp(x) - 1, with x a number. It is computed in a way that is accurate even if x is near 0, since exp(~0) and 1 are nearly equal. Dual numbers are supported.

The function can be used, for example, in financial mathematics, to calculate small daily interest rates, among other things. It is a wrapper to the C function expm1.

See also: **expx2**, **math.lnp1**.

math.exponent (x)

Returns the exponent e of a number x such that **math.mantissa**(x) * 2^e equals x. The result is identical to the second result returned by **frexp**. The function is around 20 percent faster but returns correct results only if your system supports IEEE 754 floating-point numbers, whereas **frexp** always works regardless of the internal representation.

See also: **frexp**, **math.mantissa**, **math.uexponent**.

math.fall (x, n)

The falling factorial function computes x*(x - 1)*(x - 2)* ... *(x - n + 1), with x a number and n an integer. If n is negative, the rising factorial function (Pochhammer function)

$$\frac{\Gamma(x+n)}{\Gamma(x)}$$

will be computed. In both cases, if x is a complex number, the function computes in the complex domain and returns a complex result.

See also: **fact**, **math.dblfact**, **math.lnfact**, **math.pochhammer**, **math.trifact**, **mulup**.

math.fdim (x, y [, a])

The function returns x - y if its argument x, a number, is greater than or equal y, else it returns a, which is 0 by default.

math.fld (x, y)

Returns the largest integer less than or equal to the real quotient $\frac{x}{y}$ of the numbers x and y.

See also: \ operator, **math.cld**.

math.flipsign (x, y)

Returns the number x with its sign flipped if y (a number) is negative. For example, abs(x) = flipsign(x, x).

See also: **math.copysign**, **math.signbit**, **sign**, **signum**.

math.floorpow2 (x)

Finds the largest power of 2 less than or equal to x, where x is a non-negative integer. If $x < 2$, the result is x. If $x < 0$, **undefined** will be returned. Examples: math.floorpow2(3) $\Rightarrow 2 = 2^1$, and math.floorpow2(8) $\Rightarrow 8 = 2^3$.

The function returns **fail** if $x \geq 2^{31}$.

See also: **ilog2**, **math.ceillog2**, **math.ceilpow2**, **math.ispow2**.

math.fmod (x, y)

In default mode, works exactly as **symmod**, the symmetric modulo operator, but implemented as a function. If given any option, then the SunPro version of C's fmod will be called internally instead of the standard GCC fmod implementation.

math.fpclassify (x)

For the given number x, returns

- 0 if x is **undefined** (= constant **math.fp_nan**),
- 1 if x is infinite, i.e. +/-**infinity** (= constant **math.fp_infinite**),
- 2 if x is subnormal (= constant **math.fp_subnormal**,
- 3 if x is zero (= constant **math.fp_zero**),
- 4 if x is normal (= constant **math.fp_normal**), including irregular values $\geq 2^{52}$.

Thus, for example, `ordinary` numbers are represented by results greater than 2.

The function returns **fail** if it could not determine the type of floating-point number (of C type double). It is a platform-independent port of C's fpclassify.

See also: **math.isnormal**, **math.issubnormal**.

math.fraction (x [, err])

Given a number `x`, this function outputs two integers and a number: the numerator n, the denominator d, and the accuracy epsilon, such that `x` := n / d to the accuracy epsilon := | (`x` - n/d) / `x` | ≤ `err`.

The error `err` should be a non-negative number, and by default is 0.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **div** package.

math.frexp (x [, option])

Returns the sign s, the mantissa m and the exponent e of the number `x`, in this order, such that $s*m*2^e = x$. The sign is -1 if `x` is negative (including -0) and 1 otherwise. The mantissa is a float in the range [0.5, 1) except for `x` = 0, where the result is 0. The exponent is a negative or positive integer or zero.

If any `option` is given, then instead of the sign the sign bit s will be returned: 1 if `x` is negative or -0, and 0 otherwise. In this case `x` = signum(-s) * $m*2^e$.

The function works correctly only on IEEE 754-compliant systems.

See also: **frexp**, **ldexp**, **ilog2**, **math.exponent**, **math.mantissa**.

math.gammasign (x [, y [, ...]])

When only x is given, returns the sign of the gamma function, i.e. -1 if `x` < 0 and **odd**(**entier**(`x`)), and 1 otherwise.

If further numbers are given, the signs of the gamma function of respective arguments are multiplied with each other, so the result is **math.gammasign**(`x`)*__math.gammasign__(`y`)*__math.gammasign__(...).

See also: **beta**, **gamma**, **math.beta**, **math.gamma**.

math.ge (x, y)

Checks whether the number `x` is greater than or equal to the number `y` and returns **true** or **false**.

See also: <, <=, >, <>, = operators, **math.eq**, **math.gt**, **math.le**, **math.lt**, **math.neq**.

math.gt (x, y)

Checks whether the number `x` is greater than the number `y` and returns **true** or **false**.

See also: <, <=, >=, <>, = operators, **math.eq**, **math.ge**, **math.le**, **math.lt**, **math.neq**.

math.hamming (x, y)

Computes the Hamming distance of two integers `x`, `y` considered as binary values, that is, as sequences of bits. See also: **strings.bigrams**.

math.hextodec (s)

Converts a string `s` representing a hexadecimal value to a (decimal) number. The string may or may not start with '0x' or '0X', and also may contain a sign, a fractional part or a `p exponent` part.

If the value represented by `s` is too big, the function will return **undefined**.

Example: `math.hextodec('0x-F.Fp0')` ⇒ -15.9375.

See also: **math.convertbase**, **math.octtodec**, **math.tohex**, **strings.hextodec**, **strings.strtoul**, **tonumber**.

math.hgm (x, y)

Approximates the harmomic–geometric mean of two positive real or complex numbers `x`, `y`, and returns

$$\frac{1}{\text{math.agm}(1/x,1/y)}$$

The return is a number or complex number - depending of the type of `x` and `y`. When calling the function, numbers and complex numbers can be mixed.

The function is written in Agena and included in the `libary.agn` file.

See also: **math.hgm**, **stats.amean**, **stats.gmean**.

math.invlerp (a, b, x)

The function works in the opposite way to **math.lerp**. Instead of passing a factor `t` for the clipping between `a` and `b`, you pass any value `x`, and the function will return

the corresponding factor, wherever it falls on that spectrum. With `a = b`, returns **undefined**.

For example, to find the value half-way between 50 and 100, enter:

```
> math.lerp(50, 100, .5):
75

> math.invlerp(50, 100, 75):
0.5
```

math.isexceptional (x)

Returns **true** if the number `x` is +0, -0, +**infinity**, -**infinity** or **undefined**. With complex number argument `x` returns **true** if the real *or* the imaginary part is +0, -0, +**infinity**, -**infinity** or **undefined**. Otherwise the function returns **false**.

See also: **finite**, **infinite**, **math.isminuszero**, **nan**, **nonzero**, **zero**.

math.isinfinity (x)

Returns -1 if its numeric argument `x` is **-infinity**, +1 if `x` is +**infinity**, or 0 if neither.

See also: **finite**, **infinite**.

math.isirregular (x)

Checks whether a number or complex number cannot be represented exactly on your system. It returns:

- **false** if $|x| < 2^{52}$: a number with decimal places can internally be represented as a number with decimal places, but not necessarily itself. With n < 52, the spacing between two subsequent representable numbers is the *fraction* 2^{n-52}.
- **fail** if $2^{52} \leq |x| \leq 2^{53}+1$: representable numbers are exactly the integers; spacing between representable numbers is exactly 1.
- **true** if $|x| > 2^{53}+1$: an integer mostly cannot be exactly represented; with n > 52, spacing is the *integer* 2^{n-52}.

With complex `x`, checks whether at least the real or imaginary part evaluates to **false** or **fail** - according to the rule mentioned above - and returns that; otherwise if both parts evaluate to **true**, returns **true**.

See also: **finite**, **math.isexceptional**, **math.isminuszero**, **math.isnormal**, **math.isordered**, **math.issubnormal**.

math.isminuszero (x)

Returns **true** if `x` is -0 (minus zero) and **false** otherwise. See also: **zero**, **math.signbit**.

math.isnormal (x)

Returns **true** returns true if a number is neither +0, -0, +infinity, -infinity, undefined nor subnormal. The result is equal to the expression **math.fpclassify**(x) = **math.fp_normal**.

With complex x, returns **math.isnormal**(**real**(x)) **and math.isnormal**(**imag**(x)).

See also: **finite**, **math.isirregular**, **math.isminuszero**, **math.isordered**, **math.issubnormal**.

math.isordered (x, y)

Returns **false** if at least one of its arguments x and y - two numbers - is **undefined**, and **true** otherwise. See also: **nan**.

math.ispow2 (x)

Checks whether a given non-negative number x is a power of base 2 ($x = 2^{log2(x)}$) and returns **true** or **false**. Also returns **false** if x is negative.

The function returns **fail** if its argument is ±**infinity** or **undefined**.

math.isqrt (x)

Returns the integer square root of the number x: the largest integer m such that m*m ≤ x. See also: **invsqrt**, **sqrt**.

math.issubnormal (x)

Checks whether the number x is subnormal, i.e. whether internally the leading digit of its mantissa is 0. The function returns **true** or **false**. Subnormal numbers are very close to zero, have reduced precision and lead to excessive CPU usage. They are in the range [-2.2250738585072009e-308, -4.9406564584124654e-324] and [4.9406564584124654e-324, 2.2250738585072009e-308]. 0, **undefined** and +/-**infinity** are not subnormal. Please note that the next representable number after 0 (towards +∞) is subnormal.

With complex x, returns **math.issubnormal**(**real**(x)) **or math.issubnormal**(**imag**(x)).

See also: **math.isnormal**, **math.normalise**, **math.smallestnormal**, **math.two54**, **math.zerosubnormal**.

math.kbadd (x, y [, cs, ccs])

The function adds numbers or complex numbers x and y using Kahan-Babuška round-off error prevention and returns three numbers: the uncorrected sum of x and y plus the updated values of the correction variables cs and ccs. The optional correction variables cs and ccs must be either numbers or complex numbers and

should be zero at first invocation, and the values of the correction variables returned by the previous call otherwise - if `cs`, `ccs` are not given, they default to 0.

After the last summation add `cs` and `ccs` to the uncorrected sum to have a corrected final result as shown in the following example:

```
> s, cs, ccs -> 0;

> for i in [Pi, 2*Pi, 3*Pi, 4*Pi] do
>    s, cs, ccs := math.kbadd(s, i, cs, ccs)
> od;

> print(s + cs + ccs);  # the final corrected result
```

Kahan-Babuška summation is generally more accurate than Kahan-Ozawa summation, but slower.

You might consider using **math.accu** as it is roughly 75 faster than this function.

See also: **sumup**, **math.accu**, **math.koadd**.

math.koadd (x, y [, q])

The function adds numbers or complex numbers `x` and `y` using Kahan-Ozawa round-off error prevention and returns two numbers: the sum of `x` and `y` plus the updated value of the correction variable `q`.

The optional correction variable `q` should be a number or complex number and be set to 0 at first invocation or the correction value returned by the previous call otherwise (see example below) - if `q` is not given, it defaults to 0.

The following algorithm used is:

```
math.koadd := proc(s :: number, x :: number, q) is
   local sold, u, v, w, t;
   q := optnumber(q, 0);
   v := x - q;
   sold := s;
   s := s + v;
   if abs(x) < abs(q) then
      x, q := -q, x
   fi;
   u := (v - x) + q;
   if abs(sold) < abs(v) then
      sold, v := v, sold
   fi;
   w := (s - sold) - v;
   q := u + w;
   return s, q
end;
```

A typical usage should look like:

```
x, q -> 0;
y := 0.1;
while x < 1 do
   x, q := math.koadd(x, y, q)  # add 0.1 in each step
od;

print(s + q);
```

You might consider using **math.accu** as it is roughly 75 faster than this function.

See also: **sumup**, **math.kbadd**, **stats.sumdata**.

math.largest

This constant represents the largest positive number representable in Agena. It is computed during start-up and may be different from the setting returned by **environ.system**, the latter statically compiled into the Agena binary. The smallest negative number (towards $-\infty$) is the negative of this constant, i.e. - **math.largest**.

See also: **math.lastcontint**, **math.smallest**.

math.lastcontint

This constant represents the largest integer i on the floating-point system such that i - 1 <> i. In other words: The constant represents the largest integer value that can be stored in an Agena number without loss of precision. On 32-bit systems (and higher), it is equal to $2^{53} = 9{,}007{,}199{,}254{,}740{,}992$.

See also: **math.largest**.

math.le (x, y)

Checks whether the number `x` is less than or equal to the number `y` and returns **true** or **false**.

See also: **<**, **<=**, **>**, **>=**, **=**, **<>** operators, **math.eq**, **math.ge**, **math.gt**, **math.lt**, **math.neq**.

math.length (x)

Emulates Maple's `length' function for any number `x` and counts the number of digits in its integral and fractional parts with the following formula:

math.length(x) = **math.ndigits**(x) + 4 + **math.ndigits**(x, -10)

With `x` = 0, returns 0. Note that with fractional numbers, the return may be different to the original Maple result.

math.lerp (a, b, t [, option])

Returns the linear interpolation between a and b based on factor t. By default, the function uses the formula `a + (b - a)*t` = **fma**(`t, b - a, a`), which has monotonic behaviour. `t` is typically between 0 and 1 but values outside this range are acceptable.

With any fourth argument given, the function returns `a*(1 - t) + t*b` = **fma**(`1 - t, a, t*b`). This variant is monotonic only if `a*b` < 0.

With `t` = 0 returns `a`; with `t` = 1 returns `b`. With `a` = `b` and any `t` returns `a`.

Example: To compute the number that is 35% between 56 and 132, issue:

```
> math.lerp(56, 132, 0.35):
82.6
```

See also: **math.invlerp**.

math.lnabs (x)

Returns **ln**(**abs**(`x`)) for numeric or complex `x`, taking care of underflows. See also: **math.lnhypot**.

math.lnbeta (x, y)

Computes the natural logarithm of the Beta function with `x` and `y` numbers, avoiding internal over- and underflow. The return is a number.

See also: **beta**, **math.gammasign**, **math.beta**.

math.lnbinomial (n, k)

Returns the natural logarithm of the binomial coefficient

$$\ln \binom{n}{k} = \ln \frac{n!}{k!(n-k)!} = \text{lngamma}(n + 1) - \text{lngamma}(k + 1) - \text{lngamma}(n - k + 1)$$

avoiding overflows.

See also: **binomial**, **fact**, **long.lnbinomial**, **math.lnfact**.

math.lnfact (n)

Returns **ln**(**fact**(`n`)) for any non-negative integer or fractional `n`. With integral `n` < 512, the result is taken from a lookup table, with all other arguments the result is equal to **lngamma**(`n + 1`). `n` may also be a complex number, with the result a complex number, as well.

See also: **fact**, **math.dblfact**, **math.fall**, **math.lnbinomial**, **math.trifact**, **mulup**.

math.lnhypot (a, b [, option])
math.lnhypot (z [, option])

With two numbers `a`, `b` returns ln $\sqrt{a^2+b^2}$, avoiding underflow if `a` and `b` are close to zero and trying to avoid overflow with large |a|, |b|. With a complex number z = `a` + I*`b`, also returns ln $\sqrt{a^2+b^2}$, as well, also avoiding underflow and overflow. With `a` = `b` = 0, returns 0.

If any `option` is given, then the function returns ln$\left(1+\sqrt{a^2+b^2}\right)$, guaranteeing a non-negative result in every case.

See also: **hypot**, **ln**, **math.lnabs**, **math.lnpytha**.

math.lnp1 (x)

Returns a value equivalent to **ln**(1 + `x`), with `x` a number. It is computed in a way that is accurate even if `x` is near zero. Dual numbers are supported.

It can be used, for example, in financial calculations, when computing small daily interest rates. The function is a wrapper to the C function log1p.

Example: ln(1.0000000000000001) ⇒ 0, math.lnplus1(0.0000000000000001) ⇒ 1e-016.

See also: **math.expm1**, **math.xlnp1**.

math.lnpytha (a, b [, option])
math.lnpytha (z [, option])

With two numbers `a`, `b` returns ln(a^2+b^2), avoiding underflow if `a` and `b` are close to zero and trying to avoid overflow with large |a|, |b|. With a complex number z = `a` + I*`b`, also returns ln(a^2+b^2), as well, also avoiding underflow and overflow. With `a` = `b` = 0, returns 0.

If any `option` is given, then the function returns ln(1 + a^2+b^2), guaranteeing a non-negative result in every case.

See also: **hypot**, **ln**, **math.lnabs**, **math.lnhypot**, **pytha**.

math.logs (x, b [, eps [, maxiter]])

The iterated logarithm of `x`, log*(`x`) (for `log star`) returns the number of times the logarithm function to a given base `b` must be iteratively applied on `x` until the result reaches or drops below 1. If `x` ≤ 1, returns 0. `eps` will be ignored with real `x`.

The algorithm is equivalent to:

```
> logs := proc(x, b) is
>    for i from 0 while x > 1 do
>        x := log(x, b)
>    od;
>    return i
> end;
```

With out-of-range arguments, **math.logs** returns 0.

If `x` is a complex number, the function iterates until it converges irrespective of any limiting `radius`, with a maximum of `maxiter` cycles taken, with 1024 the default. If `eps` is not given, it defaults to **DoubleEps**, and it controls how much two succeeding iteration results may differ at most in order to declare convergence. This mode is purely experimental but creates interesting fractals.

math.lt (x, y)

Checks whether the number `x` is less than the number `y` and returns **true** or **false**.

See also: <, <=, >, >=, =, <> operators, **math.eq**, **math.ge**, **math.gt**, **math.le**, **math.neq**.

math.mantissa (x)

Returns the mantissa m of a number `x` such that m * 2 ^ **math**.**exponent**(`x`) equals `x`. The result is identical to the first result returned by **frexp**, and is in the range [0.5, 1) (or zero when `x` is zero).

The function is around 20 percent faster but returns correct results only if your system supports IEEE 754 floating-point numbers, whereas **frexp** always works regardless of the internal representation.

See also: **frexp**, **math.exponent**, **math.significand**.

math.max (x [, ···])

Returns the maximum value among its arguments of type number. See also: **math.min**.

math.min (x [, ···])

Returns the minimum value among its arguments of type number. See also **math.max**.

math.modulus (x, y)

The function is a plain binding to the C `%` modulo operator. Both its arguments must be integers. The return is an integer. If `y` = 0, the function returns **undefined**.

See also: **%** operator, **bytes.mod32**, **drem**, **hashes.fibmod**, **hashes.fibmod2**, **iqr**, **modp**, **symmod**, **numtheory.invmod**, **math.nearmod**.

math.morton (x, y)

Interleaves the bits of integers `x` and `y`, so that all of the bits of `x` are in the even positions and `y` in the odd; the function can be used to linearising 2D integer co-ordinates, combining `x` and `y` into a single integer that can be compared easily. It has the property that a number is usually close to another if their `x` and `y` values are close.

math.mulsign (x, y)

Multiplies, not copies, its first argument with the sign of its second, and returns `x` * **signum**(`y`).

See also: **math.copysign**, **math.flipsign**, **math.signbit**, **sign**, **signum**.

math.ndigits (x [, b])

Returns the number of integer digits - without decimal places - in the number `x` to the base `b`. By default, `b` is 10.

If `b` is -10, counts the number of decimal places (fractional digits) in `x`, where `x` is considered to be of base 10. This feature is experimental and not fail-safe.

See also: **math.decompose**, **math.length**, **math.nthdigit**.

math.nearbyint (x [, eps])

Returns `x` rounded to the nearest integer, returns the same result as **round**(`x`, 0) does but is implemented differently and 5 % faster. The function has originally been included for C math library compatibility reasons.

If any second positive number `eps` is given and if non-integral `x` is very close to its nearest integer n, returns this integer n if |`x` - n| <= `eps`, and its argument `x` otherwise.

See also: **int**, **math.rint**, **math.trunc**.

math.nearmod (x, m)

Returns the closest value to the given number `x` divisible by the given modulus `m`, equivalent to round(`x`/`m`) * `m`. See also: **%**, **math.modulus**.

math.neq (x, y)

Checks whether the number x is not equal to the number y and returns **true** or **false**.

See also: <, <=, >, >=, = operators, **math.eq**, **math.ge**, **math.gt**, **math.le**, **math.lt**.

math.nextafter (x, y)

Returns the next machine floating-point number of x in the direction toward y.

See also: +++ and **---** operators, **math.eps**, **math.ulp**.

math.nextmultiple (n, b)

Returns the next multiple of an integer n to the given base b, towards +**infinity** if b is positive, and towards -**infinity**, if b is negative.

See also: **math.nextpower**, **math.prevmultiple**, **math.prevpower**, **multiple**.

math.nextpow2 (x)

Returns the smallest power of 2 greater than non-negative number x. Returns 1 if x is zero.

See also: **math.nextpower**, **math.prevpow2**, **math.prevpower**.

math.nextpower (x, base [, option])

By default returns the smallest power of base greater than x. If the third argument is **true**, then the smallest power of base greater than *or* equal to x will be returned, the default is **false**.

See also: **math.nextpow2**, **math.prevpow2**, **math.prevpower**.

math.noise (x [, y [, z]] [, true])

With numbers x, y and z computes gradient Perlin noise which can be used in applying pseudo-random changes to a variable, procedurally generating terrain, and assisting in the creation of image textures.

If not given, y and z default to zero. All input values should be in [-1, 1].

By default, the return is a number in the range [-1, 1]. By passing the last argument true, the return is a number in [0, 1].

See also: **math.random**, **math.randoms**.

math.norm (x, a1:a2 [, b1:b2])

Converts the number `x` in the scale [`a1`, `a2`] to one in the scale [`b1`, `b2`]. The second and third arguments must be pairs of numbers. If the third argument is missing, then `x` is converted to a number in [0, 1]. The return is a number.

See also: **linalg.scale**, **math.wrap**, **stats.scale**.

math.normalise (x [, option])

Checks whether its numeric argument `x` is subnormal and in this case normalises it, i.e. returns a non-zero normalised value $x*2^{64}$ that is close to `x`; otherwise returns its argument `x` unaltered.
If any `option` is given, the unsigned high 4-byte word of the result will be returned, too.

With complex `x`, normalises both its real and imaginary part if necessary and returns the complex number **math.normalise**(**real**(`x`)) + **I***__math.normalise__(**imag**(`x`)); the `option` is not supported in this case.

It is useful to prevent excessive CPU usage with values very close to zero.

For more information, see **math.issubnormal**.

See also: **math.zeroin**, **math.zerosubnormal**.

math.nthdigit (x, n)

Returns the `n`-th digit of the number `x`, with `n` an integer. To evaluate an integer digit, `n` should be positive; for a decimal place, `n` should be negative.

The function is written in Agena and included in the lib/library.agn file.

See also: **math.ndigits**, **numtheory.ifactor**, **numtheory.ifactors**.

math.octtodec (s)

Converts a string `s` representing an octal value to a (decimal) number. The string may or may not start with '0o' or '0O', and also may contain a sign, a fractional part or a `p exponent` part.

If the value represented by `s` is too big, the function will return **undefined**.

Example: `math.octtodec('-0o17.74')` ⇒-15.9375.

See also: **math.convertbase**, **math.bintodec**, **math.hextodec**, **math.tohex**, **tonumber**.

math.piecewise (cond$_1$, f$_1$, cond$_2$, f$_2$, ..., cond$_n$, f$_n$ [, fotherwise])

Evaluates a piecewise-continuous function. `cond`$_1$, etc. are relations evaluating to Booleans, and `f`$_1$, etc. numeric expressions. The arguments are checked from left to right and as soon as a condition `cond`$_k$ is met, **piecewise** returns the respective value `f`$_k$. If no condition meets, the function returns `fotherwise`, and **undefined** if not given.

The implementation is far from perfect as *all* of its arguments are evaluated before executing the procedure. Better use the Boolean operator **and** and **or**, for example:

- `math.piecewise(x < 2, -1, x < 3, 1, infinity)` and
- `x < 2 and -1 or x < 3 and 1 or infinity`

are equivalent, but the latter is around 15 times faster due to application of the McCarthy Rule.

See also: **math.chi**, **math.clip**.

math.pochhammer (x, n)

Computes the Pochhammer function (rising factorial), where `x` can be a real or complex number and `n` a real number. It returns the number:

$$\frac{\Gamma(x+n)}{\Gamma(x)}$$

See also: **fact**, **math.fall**, **mulup**.

math.pow32 (x)

math.pow52 (x)

math.pow72 (x)

math.powe (x)

Return $x^{1.5} = x\sqrt{x}$ or $x^{2.5} = x^2\sqrt{x}$ or $x^{3.5} = x^3\sqrt{x}$ or $x^{exp(1)} = x^E$, respectively, for number, complex number or dual number `x`. See also: **sqrt**, **cbrt**, ^ operator.

math.prevmultiple (n, b)

Returns the previous multiple of an integer `n` to the given base `b`. If `b` is negative, it is considered positive. The result is always less than `n`.

See also: **math.nextpower**, **math.nextmultiple**, **math.prevpower**, **multiple**.

math.prevpow2 (x)

Returns the smallest power of 2 less than non-negative number `x`. Returns 0 if `x` is zero. See also: **math.nextpow2**.

math.prevpower (x, base [, option])

By default returns the largest power of `base` less than `x`. If the third argument is **true**, then the largest power of `base` less than *or* equal to `x` will be returned, the default is **false**.

See also: **math.nextpower**.

math.quadrant (x)

This function returns the quadrant of an angle `x` given in radians and returns an integer in [1, 4].

math.ramp (x)

For number `x`, gives `x` if `x` > 0 and 0 otherwise.

See also: **heaviside**, **math.rectangular**.

math.random ([m [, n [, any]]])

When called without arguments, returns a pseudo-random float with uniform distribution in the range [0,1).

When called with two integers `m` and `n`, **math.random** returns a pseudo-random integer with uniform distribution in the range [`m`, `n`].

The call `**math.random**(`n`)`, for a positive `n`, is equivalent to `**math.random**(1, `n`)`. The call `**math.random**(0)` produces an integer with all bits (pseudo) random.

When given no third argument, this function uses the xoshiro256** algorithm to produce pseudo-random 64-bit integers, which are the results of calls with argument 0. Other results (ranges and floats) are unbiased extracted from these integers. If you pass any third argument, the function uses a simple XOR-based 32-bit random number generator that is as fast as xoshiro256**.

Agena initialises its pseudo-random generator with the equivalent of a call to **math.randomseed** with no arguments, so that **math.random** should generate different sequences of results each time the program runs.

See also: **math.noise**, **math.randoms**, **strings.random**.

math.randoms ([m [, n]] [, option])

This function creates random numbers as Agena did before version 2.27.10.

When called without arguments, returns a pseudo-random real number in the range (0,1). It can generate up to 2 * **environ.maxlong** unique random numbers in this interval.

When called with a number `m`, the function returns a pseudo-random integer in the range [1, `m`].

When called with two numbers `m` and `n`, returns a pseudo-random integer in the range [`m`, `n`].

If `option`, any Boolean, is given, then the sequence of values returned should be arbitrary, otherwise it is always the same unless **math.randomseed** is called with other values.

See also: **math.noise**, **math.random**, **math.randomseed**, **skycrane.dice**.

math.randomseed ([x, y])

When called with at least one argument, the integer parameters `x` and `y` are joined into a 128-bit seed that is used to reinitialise the pseudo-random generator; equal seeds produce equal sequences of numbers. The default for `y` is zero.

When called with no arguments, Lua generates a seed with a weak attempt for randomness.

This function returns the two seed components that were effectively used, so that setting them again repeats the sequence.

To ensure a required level of randomness to the initial state (or contrarily, to have a deterministic sequence, for instance when debugging a program), you should call **math.randomseed** with explicit arguments.

math.randomseeds ([x, y])

Sets `x` and `y` as the `seeds` for the pseudo-random generator, as Agena did before version 2.27.10: equal seeds produce equal sequences of numbers. `x` and `y` must both be positive integers. It returns two new settings. The function does not check for `x`= 0x464fffff and `y` = 0x9068ffff.

If called without arguments, the function returns the current seeds.

See also: **math.random**.

math.rectangular (x [, pi])

math.rectangular (x [, a [, b [, pi]]])

In the first form, computes the rectangular pulse function for number `x`:

$$\text{math.rectangular(x)} = \begin{cases} 1 & \text{if } |x| < 0.5 \\ 0.5 & \text{if } |x| = 0.5 \\ 0 & \text{if } |x| > 0.5 \end{cases}$$

In the second form, `a` represents the rising edge, and `b` the falling edge of the rectangular pulse function. By default, `a` = -0.5 and `b` = +0.5. The function then returns 0 if `x` < `a` or `x` > `b`; 0.5 if (`x` = `a` or `x` = `b`) and `a` <> `b`, and 1 otherwise.

If `pi` is the Boolean value **true**, the function computes the box distribution Pi(x):

$$\text{Pi(x)} = \begin{cases} 1 & \text{if } |x| < 0.5 \\ \text{undefined} & \text{if } |x| = 0.5 \\ 0 & \text{if } |x| > 0.5 \end{cases}$$

See also: **heaviside**, **math.clip**, **math.ramp**, **math.triangular**, **math.unitise**, **sinc**.

math.redupi (x)

Subtracts the nearest integer multiple of π from its numeric argument `x`.

See also **math.wrap**.

math.relerror (a, b)

Computes the relative error |`b` - `a`|/|`a`|, handling case of **undefined** and **infinity**.

math.rempio2 (x [, option])

Conducts an argument reduction of `x` into the range $|y| < \frac{\pi}{2}$ and returns y = `x` - N*$\frac{\pi}{2}$. If any `option` is given, then the function also returns N, or actually the last three digits of N. The number of operations conducted are independent of the exponent of the input.

The function is 60 percent faster than **math.wrap**, but returns a result different from `x` if its argument |`x`| is already in the range $\frac{\pi}{4}$ .. $\frac{\pi}{2}$.

This function is just a port to the underlying C function rem_pio2 which is used to compute sines, cosines and tangents.

See end of Chapter 11.1.2 for a comparison chart.

math.rint (x)

Rounds a (complex) float to a (complex) integer according to the current rounding method which you can query and set with **environ.kernel/rounding**.

See also: **ceil**, **entier**, **int**, **mdf**, **round**, **xdf**, **math.nearbyint**.

math.secd (x)

Takes x in degrees and returns its secant in radians.

See also: **math.cosd**, **math.cotd**, **math.cscd**, **math.sind**, **math.tand**, **sec**.

math.signbit (x)

Checks whether the number x has its sign bit set and returns **true** or **false**. It is a plain binding to C's copysign function. For example, although -0 = 0, **math.signbit**(-0) ⇒ **true** and **math.signbit**(0) ⇒ **false**.

See also: **math.copysign**, **math.flipsign**, **math.isminuszero**, **sign**.

math.significand (x)

Returns the mantissa of number x in a normalised form, in the range[1, 2), with **math.significand**(x) = 2***math.mantissa**(x) = **ldexp**(x, -**ilog2**(x)). If x is 0, the return is 0.

See also: **math.uexponent**.

math.sincos (x)

Returns both the sine and cosine for number or complex x as two numbers or complex numbers. The function is around 10 to 15 % faster than calling the **sin** and **cos** operators separately.

See also: **cos**, **sin**, **math.sincosfast**, **math.sincospi**.

math.sincospi (x [, option])

Returns both sin(π*x) and cos(π*x) for number or complex number x with better precision than calling the respective standard operators. If option is **true** and x is a number, than the tangent, i.e. tan(π*x) is returned, as well.

See also: **math.sincos**, **math.sinpi**, **math.cospi**, **math.tanpi**.

math.sincpi (x)

Returns **sin**(π*x)/(π*x) with number or complex number x, preserving accuracy especially as much as possible with larger x and with x around the origin. With x = 0, returns 1. The type of return depends on the type of the input.

See also: **cosc**, **sinc**, **tanc**, **math.coscpi**, **math.tancpi**.

math.sind (x)

Takes x in degrees and returns its sine in radians.

See also: **sin**, **math.cosd**, **math.cscd**, **math.cotd**, **math.secd**, **math.sinpi**, **math.tand**.

math.sinhcosh (x)

For number x, returns both the hyperbolic sine and hyperbolic cosine as two numbers. The function is around 30 to 35 % faster than calling the **sinh** and **cosh** operators separately.

With complex x, returns complex results.

See also: **cosh**, **sinh**.

math.sinpi (x)

Returns sin(π*x) for number or complex number x with better precision than calling the respective standard operator.

See also: **sin**, **math.cospi**, **math.sincos**, **math.sincospi**, **math.sind**, **math.tanpi**.

math.smallest

This constant represents the smallest positive number representable in Agena. It is computed during start-up and is different from the setting returned by **environ.system**, the latter statically compiled into the Agena binary.

See also: **math.largest**.

math.smallestnormal

This constant denotes the smallest positive normal number representable on your system.

math.splitdms (x [, option])

Splits the number x representing a sexagesimal number in TI-30 sexagesimal DMS format into its parts and returns three numbers: the degrees, minutes, and seconds. For example, with -10.3045 representing -10°30'45", we get

```
> math.splitdms(-10.3045):
-10     -30     -45
```

If any option is given, the function returns the result in a table.

See also: **clock** package, **math.dd**, **math.dms**, **math.todecimal**, **math.tosgesim**.

math.tancpi (x)

Returns **tan**(π*x)/(π*x) with number or complex number x, preserving accuracy especially as much as possible with larger x and with x around the origin. With x = 0, returns 1. The type of return depends on the type of the input.

See also: **cosc**, **sinc**, **tanc**, **math.coscpi**, **math.sincpi**.

math.tand (x)

Takes x in degrees and returns its tangent in radians.

See also: **tan**, **math.cosd**, **math.cotd**, **math.cscd**, **math.secd**, **math.sind**, **math.tanpi**.

math.tanpi (x)

Returns tan(π*x) for number or complex number x with better precision than calling the respective standard operator.

See also: **tan**, **math.cospi**, **math.sincos**, **math.sincospi**, **math.sinpi**, **math.tand**.

math.tocomplex (x)

Converts number x to the complex number x + I*0. When given a complex number, it is simply returned.

math.todecimal (h [, m [, s]])

math.todecimal (obj)

In the first form, converts a sexagesimal time value given in hours h, minutes m and seconds s into its decimal representation. The optional arguments m and s default to 0. If a sexagesimal value is negative, then at least h should be negative with the function automatically adjusting m and s if necessary.

In the second form, when given a table `obj` of hours and optionally minutes, seconds, does the same.

Example:

```
> math.todecimal(12, 30, 45):  # half past noon and 45 second
12.5125
```

The compliment to **math.todecimal** is **math.tosgesim**.

See also: **clock.todec**, **math.dd**, **math.dms**, **math.splitdms**, **math.todms**

math.todegrees (r [, option])

Converts the number `r` considered to be in radians to degrees. Note that with numeric literals, you can just append the `r` suffix which does the same:

```
> math.todegrees(Pi/2), 1.5707963267949r:
90      90
```

If any `option` is given, the function returns the results in three separate components: the degrees, minutes and seconds.

See also: **math.toradians**.

math.tohex (x [, true])

Converts a non-negative integer `x` to its hexadecimal representation, returned as a string which length is always a multiple of 2, possibly including a leading zero.

By default, the resulting string will have a preceding `0x` and all letters are in lower case. If the second argument is **true**, there will be no prefix and all letters are in upper case.

See also: **math.convertbase**, **math.hextodec**.

math.toradians (d [, m [, s]])

math.toradians (obj)

In the first form, returns the angle given in degrees `d`, minutes `m` and seconds `s`, in radians. The optional arguments `m` and `s` default to 0. If `d` is negative, then `d`, `m` and `s` are automatically converted to positive numbers, while preserving the sign in the result.

In the second form, when given a table `obj` of degrees and optionally minutes, seconds, does the same.

When given the *fractional* number `d`, the function internally splits it apart into integral degrees (`d`), minutes (`m`) and seconds (`s`) to compute the final result. In this case, any other argument, that is `m`, `s`, will be ignored.

See also: **math.todegrees**.

math.tosgesim (d [, option])

Converts a decimal time value given by the number `d` into its sexagesimal representation and returns three numbers: the hours, minutes, and seconds.

Example:

```
> math.tosgesim(12.5125):
12      30      45
```

If given any `option`, the function returns the result in a table.

The compliment to **math.tosgesim** is **math.todecimal**.

See also: **clock** package, **math.dd**, **math.dms**, **math.splitdms**, **math.todecimal**.

math.triangular (x)

math.triangular (x [, a [, b]])

In the first form, computes the triangular function of base length 1 for number `x`:

$$\text{math.triangular(x)} = \begin{cases} 1 - |2x| & \text{if } |x| < 0.5 \\ 0 & \text{if } |x| \geq 0.5 \end{cases}$$

In the second form, by passing a left and a right border `a`, `b`, the function returns non-zero values in this range, and 0 otherwise, with `a` = -0.5 and `b` = +0.5 the defaults. Thus, the general formula used by the function is:

math.triangular(`x`, `a`, `b`) := max(0, 1 - |2*(`x` - offset)/d|),

where d := |`b` - `a`| and offset := `a` + d/2.

See also: **heaviside**, **math.branch**, **math.clip**, **math.rectangular**, **math.unitise**, **math.wrap**, **sinc**.

math.trifact (n)

Returns the triple factorial `n`!!! = `n`(`n` - 3)(`n` - 6)...3 for non-negative integer `n`.

See also: **fact**, **math.dblfact**, **math.fall**, **math.lnfact**, **math.trifact**.

math.trunc (x)

Returns x rounded to the nearest integer towards zero, returns the same result as **int**(x). The function has been included for Maple and C math library compatibility reasons.

See also: **math.nearbyint**.

math.two54

The constant represents 2^{54}, a value with which subnormal numbers can be multiplied in order to become normal. See also: **math.issubnormal**.

math.uexponent (x [, option])

Computes the unbiased base-2 exponent of number x, i.e. returns math.exponent(x) - 1, except for x = 0 and subnormal numbers where the result is -1023, and for x = **undefined** or x = ±**infinity** returns 1024.

If any option is given, then returns **sign**(x)***math.uexponent**(x), but for x = **undefined** returns 0x401= 1025, for x = -**infinity** returns -1024, and for x = **infinity** returns +1024. Due to the definition, returns 0 for x = 0 and subnormal x.

See also: **bytes.getunbiased**, **math.significand**, **frexp**.

math.ulp (x [, eps])

Computes the unit of least precision (ULP), the spacing between floating-point numbers, for number x, as a measure of accuracy in numeric calculations. It is equivalent to **math.nextafter**(x, **infinity**) - x.

If eps is given, the function also returns the number of ULPs - an integer - between x and x + eps.

math.unitise (x [, eps])

Returns 0 if its number argument x is zero or close to zero, and 1 otherwise:

- 0 if |x| ≤ eps,
- 1 if |x| > eps.

With complex numbers x = a + I*b, returns

- 0 if |a| ≤ eps and |b| ≤ eps,
- 1 if |a| > eps and |b| > eps.

By default, eps is set to the constant Eps.

See also: **heaviside**, **math.clip**, **math.rectangular**, **math.unitstep**, **math.zeroin**.

math.unitstep (x [, eps])

For number x, gives 0 for x < 0 and 1 otherwise.

See also: **heaviside**, **math.unitise**.

math.wrap (x [, a [, b]])

Conducts a range reduction of the number x to the interval [a, b) and returns a number. If x ∈ [a, b), x is simply returned. Just add +++0 to b to reduce into range [a, b] if x is a fraction, or 1 to be if x is an integer.

In the second form, if a is not given, a is set to $-\pi$ and b to $+\pi$. If a is given but not b, a is set to -a and b to +a, so a should be positive.

The result is equivalent to:

```
> dec x, a;
> dec b, a;
> a + (b + x symmod b) symmod b:
```

See also: **%** operator, **math.branch**, **math.clip**, **math.norm**, **math.redupi**, **math.rempio2**, **zx.reduce**, end of Chapter 11.1.2 for a comparison chart.

math.xlnp1 (x)

Computes x - ln(1 + x) = x + $\ln\left(\frac{1}{x+1}\right)$ in a way that is accurate even if x is near zero. The algorithm is ten percent faster than simply returning x - **math.lnp1**(x).

math.zeroin (x [, eps])

Returns 0 if for number x we have: |x| ≤ **DoubleEps**, and returns x otherwise. With a complex number x, returns 0+I*0 if its magnitude |x| ≤ **DoubleEps** or sets its respective parts to zero if their respective absolute values are less or equal to **DoubleEps**. Otherwise just returns x. If eps is given, then this threshold is used instead of **DoubleEps**.

See also: **math.chop**, **math.normalise**, **math.zerosubnormal**.

math.zerosubnormal (x)

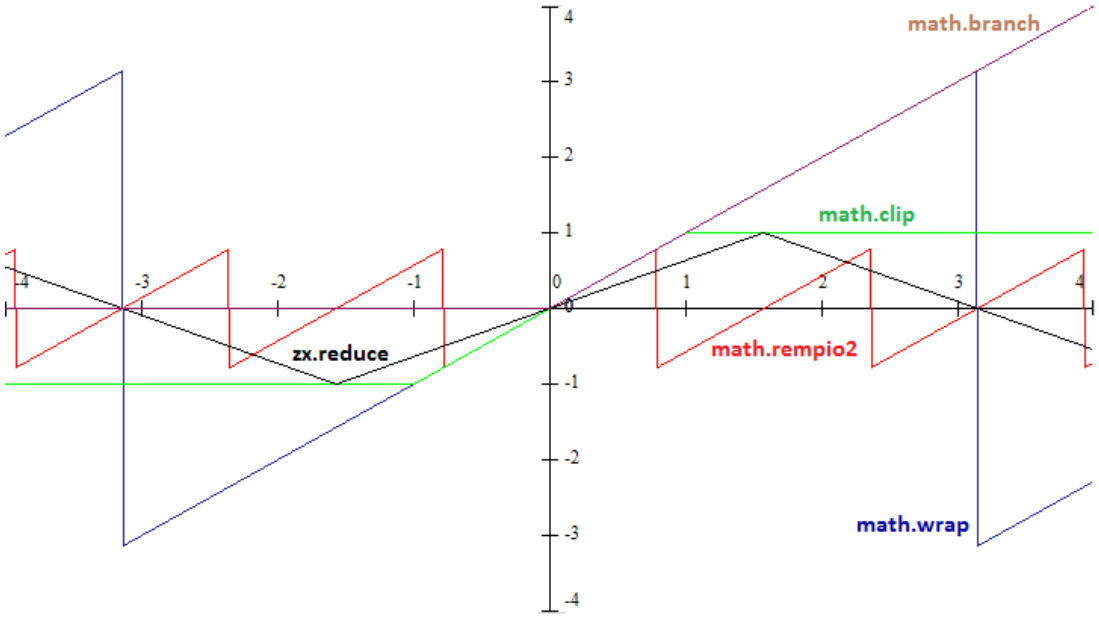
Checks whether its numeric argument x is subnormal and in this case returns 0, otherwise returns its argument x. It is useful to prevent excessive CPU usage in case of arguments very close to zero. Note that result retains the sign of x.

With complex `x`, returns the complex number **math.zerosubnormal**(**real**(`x`)) + **I***__math.zerosubnormal__(**imag**(`x`)).

For more information, see **math.issubnormal**, **math.normalise**, **math.zeroin**.

Comparison of some clipping functions:

```
> gdi.plotfn([<: x -> math.rempio2(x) :>,
>     <: x -> math.wrap(x) :>,
>     <: x -> math.clip(x) :>,
>     <: x -> zx.reduce(x) :> ,
>     <: x -> math.branch(x) :>],
>     -4, 4, -4, 4,
>     colour=['red', 'navy', 'green', 'black', 'maroon']);
```



11.1.3 fastmath Library

As a *plus* package, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import fastmath`.

The library provides procedures to approximate mathematical functions in the real domain. Despite its name, the package functions may not necessarily be faster than the standard functions and operators implemented in Agena.

`fastmath.cosfast (x)`

Approximates **cos**(`x`) for number `x`, and returns a number. It is around 40 percent faster than **cos**.

See also: **cos**.

`fastmath.floor (x)`

Works like the **entier** operator or the **floor** function, that are rounding downwards to the next integer, but is eight percent faster than **floor** (**entier** is always faster). The function may not be portable across platforms.

`fastmath.hypotfast (x, y)`

Returns the hypotenuse of the two numbers x and y; the return is a number. The function is sixty percent faster than **hypot**, but prone two round-off errors.

`fastmath.invroot (x [, degree [, n [, xhalf]]])`

Approximates the inverse root 1/root(`x`, `degree`) using the Quake III method, and returns a number. `x` is the radicand, `degree` the degree-th root which by default is 2. `n` is the number of iterations to be conducted and by default is 2 ^ `degree`. `xhalf` is the internal equivalent of `x`, 0.5*`x` by default. The greater the `degree`, the less accurate is the result.

See also: **fastmath.reciprocal**, **fastmath.sqroot**.

`fastmath.invsqrt (x)`

Approximates the inverse square root 1/**sqrt**(`x`), using Quake's Fast Inverse Square Root method and returns a number. It is five percent faster than the inverse of the **sqrt** operator.

fastmath.lbfast (x)

Approximates **log2**(x) for number x, and returns a number. It is around a third faster than **log2**. If x <= 0, the result will be wrong.

See also: **log2**.

fastmath.reciprocal (x)

Approximates the reciprocal of its argument x of type number. The return is a number. The function is purely experimental.

See also: **fastmath.invroot**, **fastmath.sqroot**.

fastmath.sinfast (x)

Approximates **sin**(x) for number x, and returns a number. It is around 40 percent faster than **sin**.

fastmath.sincosfast (x)

Returns both an approximation of the sine and cosine as two numbers. The function is around 10 % faster than calling **math.sincos**.

fastmath.sqroot (x)

Roughly approximates the square root of its argument x of type number. The returns are two numbers: guesses computed using C doubles and floats, in this order. The function is purely experimental.

See also: **fastmath.invroot**, **fastmath.reciprocal**, **fastmath.sqrtfast**.

fastmath.sqrtfast (x)

Approximates the square root of its argument x of type number. The function is purely experimental.

See also: **fastmath.sqroot**.

fastmath.tanfast (x)

Approximates **tan**(x) for number x, and returns a number. It is around 40 percent faster than **tan**.

See also: **tan**.

11.2 bytes Library

The library provides procedures for bit and byte twiddling.

11.2.1 General Functions

bytes.bcd (n)

Returns the Binary coded decimal (BCD) representation of the non-negative integer `n`. From left to right, each decimal digit is converted to a four-bit representation (0 = 0b0000, 9 = 0b1001), and the resulting bit sequence is then returned as one decimal integer, e.g. decimal 102 = 0001 0000 0010 ⇒ BCD 258.

By default, if only `n` is given, the function converts the decimal integer to BCD. If **true** is passed as a second argument, `n` is converted from BCD to its decimal integer representation.

bytes.castint (x, bits)

Casts number `x` to a C integer. The results may be platform-dependent.

`bits`	**Cast to**
8	uint8_t
16	uint16_t
32	uint32_t
64	uint64_t
-8	int8_t
-16	int16_t
-32	int32_t
-64	int64_t

bytes.fpbtoint (x)

Converts a `floating point byte` generated by **bytes.inttofpb** back. This function is used to evaluate numbers passed to the Lua/Agena virtual machine. Please note that **math.inttofpb(math.fpbtoint**(`x`)**)** does not return `x`.

bytes.numhigh (x)

Returns the higher bytes of a number `x` as an unsigned 4-byte integer. The function does not support complex numbers.

See also: **bytes.numwords**, **bytes.numlow**, **math.tohex**.

bytes.numlow (x)

Returns the lower bytes of a number x as an unsigned 4-byte integer. The function does not support complex numbers.

See also: **bytes.numhigh**, **bytes.numwords**, **bytes.setlow**, **math.tohex**.

bytes.numwords (x [, true])

Returns both **bytes.numhigh**(x), **bytes.numlow**(x) plus the unbiased exponent (i.e. **math.exponent**(**bytes_numhigh**(x)) - 1, except for x = 0 -> -1023) as three results, in this order.

If **true** is given as a second argument, the function returns the upper and lower bytes as hexadecimal strings instead of numbers.

See also: **bytes.setnumwords**, **hashes.squirrel64**, **math.tohex**.

bytes.inttofpb (x)

Converts the integer x to a `floating point byte`, represented as (eeeeexxx), where the real value is (1xxx) * 2^(eeeee - 1) if eeeee <> 0 and (xxx) otherwise. This function is used to transport numbers to the Lua/Agena virtual machine.

See also: **bytes.fpbtoint**.

bytes.leadzeros (x)

Counts the number of leading zeros (clz) in the unsigned 32-bit integer x, and also returns the modified value of x after this operation as a second result, where all bits starting with the first non-zero bit in x are set to 1.

See also: **bytes.leastsigbit**, **bytes.mostsigbit**, **bytes.onebits**, **bytes.trailzeros**.

bytes.leastsigbit (x)

Returns the position of the least significant bit (lsb) in the unsigned 32-bit integer x, here the smallest index of the first 1-bit, counting from bit index 1. If x = 0, returns 0.

See also: **bytes.leadzeros**, **bytes.mostsigbit**, **bytes.onebits**.

bytes.mostsigbit (x)

Returns the position of the most significant bit (msb) in the unsigned 32-bit integer x, i.e. the largest index of a 1-bit, counting from bit index 1. If x = 0, returns 0.

See also: **bytes.leadzeros**, **bytes.leastsigbit**, **bytes.mostsigbit**.

bytes.onebits (x)

Returns the number of bits set in the unsigned 32-bit integer `x`.

See also: **bytes.leadzeros**, **bytes.mostsigbit**.

bytes.optsize (n)

For a given number of bytes `n`, calculates the optimal number of bytes (places) in a C `array` (e.g. a memfile, numarray or even a string) if it shall be aligned on the 4- or 8-byte word boundary.

See also: **strings.strlen**.

bytes.pack (fmt, v1, v2, ···)

Returns a binary string containing the values `v1`, `v2`, etc. packed (that is, serialised in binary form) according to the format string `fmt`.

The first argument to **bytes.pack**, **bytes.packsize**, and **bytes.unpack** is a format string, which describes the layout of the structure being created or read.

A format string is a sequence of conversion options. The conversion options are as follows:

`<`: sets little endian
`>`: sets big endian
`=`: sets native endian
`![n]`: sets maximum alignment to `n` (default is native alignment)
`b`: a signed byte (char)
`B`: an unsigned byte (char)
`h`: a signed short (native size)
`H`: an unsigned short (native size)
`l`: a signed long (native size)
`L`: an unsigned long (native size)
`j`: a lua_Integer
`J`: a lua_Unsigned
`T`: a size_t (native size)
`i[n]`: a signed int with `n` bytes (default is native size)
`I[n]`: an unsigned int with `n` bytes (default is native size)
`f`: a float (native size)
`d`: a double (native size)
`n`: a lua_Number
`cn`: a fixed-sized string with `n` bytes
`z`: a zero-terminated string

`s[n]`: a string preceded by its length coded as an unsigned integer with `n` bytes (default is a size_t)
`x`: one byte of padding
Xop: an empty item that aligns according to option op (which is otherwise ignored)
`' '`: (empty space) ignored

(A `"[n]"` means an optional integral numeral.) Except for padding, spaces, and configurations (options "xX <=>!"), each option corresponds to an argument (in **bytes.pack**) or a result (in **bytes.unpack**).

For options "!n", "sn", "in", and "ln", n can be any integer between 1 and 16. All integral options check overflows; **bytes.pack** checks whether the given value fits in the given size; **bytes.unpack** checks whether the read value fits in a Lua integer.

Any format string starts as if prefixed by "!1=", that is, with maximum alignment of 1 (no alignment) and native endianness.

Alignment works as follows: For each option, the format gets extra padding until the data starts at an offset that is a multiple of the minimum between the option size and the maximum alignment; this minimum must be a power of 2. Options "c" and "z" are not aligned; option "s" follows the alignment of its starting integer.

All padding is filled with zeros by **bytes.pack** (and ignored by **bytes.unpack**).

See also: **bytes.packsize**, **bytes.unpack**, **math.ispow2**.

`bytes.packsize (fmt)`

Returns the size of a string resulting from **bytes.pack** with the given format. The format string cannot have the variable-length options 's' or 'z'.

`bytes.reverse (x)`

Flips all the bits in the unsigned 32-bit integer `x` from 0 to 1 and vice versa.

See also: **bytes.swap**.

`bytes.setnumhigh (x, i)`

The function sets the higher bytes of the number `x` to the unsigned 32-bit integer `i`, and returns the new number. It does not support complex numbers.

See also: **bytes.setnumlow**, **bytes.numhigh**.

bytes.setnumlow (x, i)

The function sets the lower bytes of the number `x` to the unsigned 32-bit integer `i`, and returns the new number. It does not support complex numbers.

See also: **bytes.setnumhigh**, **bytes.numlow**.

bytes.setnumwords (hx, lx)

Returns the number (C double) x represented by the unsigned 32-bit integers `hx` and `lx`, i.e. x = **bytes.setnumwords(math.numhigh**(`x`), **math.numlow**(`x`)).

See also: **bytes.numwords**.

bytes.swap (x)

Swaps all the bytes of the unsigned 4-byte integer `x`. The return is the new integer.

See also: **bytes.reverse**, **bytes.swaplower**, **bytes.swapupper**.

bytes.swaplower (x, n)

Swaps the lower `n` bytes of the unsigned 4-byte integer `x`; bytes above those will be discarded. The return is the new integer. If `n` = 4, all bytes will be exchanged. If `n` is zero or greater than 4, 0 will be returned.

See also: **bytes.reverse**, **bytes.swap**, **bytes.swapupper**.

bytes.swapupper (x, n)

Swaps the upper `n` bytes of the unsigned 4-byte integer `x`; bytes below those will be discarded. The return is the new integer. If `n` = 4, all bytes will be exchanged. If `n` is zero or greater than 4, 0 will be returned.

See also: **bytes.reverse**, **bytes.swap**, **bytes.swaplower**.

bytes.tobig (x [, order])

On Little Endian systems, converts the number `x` into its Big Endian representation and returns it. On Big Endian platforms, just returns `x` unaltered. If `order` is 4, then the function processes `x` as an unsigned 4-byte integer. If `order` is -4, the function treats `x` as a signed 4-byte integer.

See also: **bytes.tolittle**, **os.endian**.

bytes.tobinary (x [, false])

Converts a non-negative integer into its binary representation, a sequence of zeros and ones. By default "leading" zero-bits will be omitted. When passing optional **false**, they will be included in the result.

See also: **math.convertbase**.

bytes.tobytes (x [, nbytes [, false]])

If given no option, returns a sequence of eight bytes representing the number `x` in Little Endian order, i.e. the least-significant byte is the first entry in the resulting sequence. If `nbytes` is the number +4 or -4, `x` is a assumed to be an unsigned 4-byte integer or signed 4-byte integer, respectively, and a sequence of four bytes representing `x` in Little Endian representation will be returned. If `nybtes` is +2 or -2, `x` will be treated as an unsigned or signed 2-byte integer, with a sequence of two bytes to be returned.

With `x` a long double, returns a sequence of ten bytes, see **long** package. With `x` a signed 8-byte integer, returns a sequence of eight bytes, see **ints** package. With `x` a complex number, returns a sequence of 16 bytes. The returns are in Little Endian order, too.

On Big Endian systems, conversion to Little Endian representation can be switched off by passing a third argument, the Boolean value **false**.

See also: **getbit**, **getbits**, **getnbits**, **bytes.tochars**, **bytes.tonumber**, **strings.tobytes**, **strings.tochars**, **strings.tonumber**.

bytes.tochars (x [, nbytes [, false]])

The function works exactly like **bytes.tobytes** but returns a string instead of a sequence. The string may include embedded zeros. To convert the result back to a number, use **strings.tonumber**.

Convert π to a C double and back again:

```
>strings.tonumber(bytes.tochars(Pi)):
3.1415926535898
```

Convert 256 to an unsigned 16-bit integer and back again.

```
> strings.tonumber(bytes.tochars(256, 2)):
256
```

bytes.tolittle (x [, order])

On Big Endian systems, converts the number `x` into its Little Endian representation and returns it. On Little Endian platforms, just returns `x` unaltered. If `order` is 4, then the function processes `x` as an unsigned 4-byte integer. If `order` is -4, the function treats `x` as a signed 4-byte integer.

See also: **bytes.tobig**, **os.endian**.

bytes.tonumber (s [, true])

Takes a sequence `s` of two, four, eight or sixteen numbers representing bytes and converts it into an Agena number. Regardless of your platform, the order of bytes in `s` is assumed to be Little Endian.

Alternatively, the function takes a string `s` of two, four, eight or sixteen characters representing bytes.

If `s` contains eight bytes, it is assumed to represent a C signed double. If it contains four bytes, an unsigned four-byte integer is assumed; and with two bytes, an unsigned two-byte integer is assumed. With ten bytes, the function returns a long double userdata, see **long** package. If `s` contains sixteen bytes, the result will be a complex number.

If is string or sequence `s` contains four bytes and the second argument is **true**, it is taken to represent a C float, returned as an Agena number. If `s` contains eight bytes and the second argument is **true**, a signed 8-byte integer will be returned as an **int64** userdata, see the **ints** package.

See also: **bytes.tobytes**, **strings.tobytes**, **strings.tonumber**.

bytes.trailzeros (x)

Counts the number of trailing zeros (ctz) in the unsigned 32-bit integer `x`, and also returns the modified value of `x` after this operation as a second result, where all bits starting with the least significant bit in `x` are set to 1.

See also: **bytes.leadzeros**, **bytes.leastsigbit**, **bytes.mostsigbit**, **bytes.onebits**.

bytes.unpack (fmt, s [, pos])

Returns the values packed in string `s` (see **bytes.pack**) according to the format string `fmt`. An optional `pos` marks where to start reading in `s` (default is 1). After the read values, this function also returns the index of the first unread byte in `s`.

11.2.2 cast Functions

The bytes package provides the 'cast' userdata data structure representing an Agena number as both a C double (i.a. Agena number) and its two higher and lower 32-bit unsigned integer representations, along with functions to query and assign its individual components.

Example:

```
> a := bytes.cast(-Pi):
cast(-3.1415926535898 : 3221823995, 1413754136)

> hx := bytes.gethigh(a):
3221823995

> hx >> 31:  # sign bit (1 = minus, 0 = plus)
1

> bytes.sethigh(a, hx && 0x7fffffff):  # absolute value
3.1415926535898
```

bytes.cast ([x])

bytes.cast ([hx, lx])

Creates a userdata structure of type 'cast' that stores the Agena number 0 or `x` and its integer representation as two unsigned 32-bit integers. Technically, the userdata represents the C union (see source file src/sunpro.h):

Big-Endian platforms	Little-Endian platforms

typedef union {
  double value;
  struct {
  uint32_t msw;
  uint32_t lsw;
  } parts;
} ieee_double_shape_type;
	typedef union {
	  double value;
	  struct {
	  uint32_t lsw;
	  uint32_t msw;
	  } parts;
	} ieee_double_shape_type;

If no argument is given, then the userdata represents zero (0), alternatively you can set it to number `x` (first form).

You may also initialise the userdata by passing both its unsigned 32-bit integer word `hx` and unsigned 32-bit integer lower word `lx` (second form).

See also: **bytes.getdouble**, **bytes.gethigh**, **bytes.getlow**, **bytes.getwords**, **bytes.setdouble**, **bytes.setwords**.

bytes.getdouble (a)

Returns the floating point element of 'cast' userdata `a`, i.e. a number.

bytes.gethigh (a)

Returns the higher unsigned 32-bit integer representation of a number from 'cast' userdata `a`.

See also: **bytes.getlow**, **bytes.getwords**.

bytes.getlow (a)

Returns the lower unsigned 32-bit integer representation of a number from 'cast' userdata `a`.

See also: **bytes.gethigh**, **bytes.getwords**.

bytes.getunbiased (a)

Returns the unbiased exponent of the double x represented by 'cast' userdata `a`. Equals **math.exponent**(x) - 1, except for x = 0 where the result is -1023. If |x| < 1, the result is always negative.

See also: **bytes.gethigh**, **bytes.getlow**, **bytes.getwords**, **math.uexponent**.

bytes.getwords (a)

Returns both the higher and lower unsigned 32-bit integer representations of a number from 'cast' userdata `a`.

See also: **bytes.getdouble**, **bytes.gethigh**, **bytes.getlow**.

bytes.setdouble (a, x)

Sets the floating point element of 'cast' userdata `a` and returns the higher and lower unsigned 32-bit integer representations, in this order.

See also: **bytes.setwords**.

bytes.sethigh (a, hx)

Sets the higher unsigned 32-bit integer element `hx` of 'cast' userdata `a`. The return is the corresponding floating point representation, i.e. a number.

See also: **bytes.setdouble**, **bytes.setlow**.

bytes.setlow (a, lx)

Sets the lower unsigned 32-bit integer element `lx` of 'cast' userdata `a`. The return is the corresponding floating point representation, i.e. a number.

See also: **bytes.setdouble**, **bytes.sethigh**.

bytes.setwords (a, hx, lx)

Sets the higher and lower unsigned 32-bit integer elements `hx` and `lx` of 'cast' userdata `a`. The return is the corresponding floating point representation, i.e. a number.

See also: **bytes.setdouble**, **bytes.sethigh**, **bytes.setlow**.

11.2.3 IEEE754 Functions

The bytes package provides the 'ieee' userdata data structure representing an Agena number as both a C double (i.a. Agena number) and its components sign bit, biased exponent and high- and low-word mantissa. See **bytes.ieee** for details.

Example:

```
> exp10 := proc(x) is
>    x *:= log2(10);
>    local i := round(x);
>    local f := bytes.ieee(0);
>    bytes.setieee(f, expo = i + 1023);
>    x -:= i;
>    return bytes.getieee(f, 'double') *
>        (1.0 + x*(0.693147180559945309417232121458 18 +
>               x*(0.240226506959100712333551263163 33 +
>               x*(0.055504108664821579953142263768622 +
>               x*(0.009618129107628477161979071573 6589 +
>               x*(0.001333355814642844342341222198 7996 +
>               x*(0.000154035303933816099544370973 32742 +
>               x*(0.000015252733804059840280025439 01201 +
>               x*(0.000001321548679014430948840375 8228288 +
>               x*0.000000101780860092396997274900075977 45))))))))
>end;
```

bytes.ieee ([x])

bytes.ieee ([signbit, exponent, high_mantissa, low_mantissa])

Creates a userdata structure of type 'ieee' that stores the Agena number 0 or `x` and allows read and write access to its components sign bit `signbit`, its biased `exponent`, the high-word part of the mantissa `high_mantissa` and its low-word part `low_mantissa`. Technically, the userdata represents the C union (see source file src/sunpro.h).

Big-Endian platforms	Little-Endian platforms

typedef union {
  double v;
  struct {
  uint64_t sign : 1;
  uint64_t exponent : 11;
  uint64_t mantissa_high : 20;
  uint64_t mantissa_low : 32;
  } c;
} double_ieee754;
	typedef union {
	  double v;
	  struct {
	  uint64_t mantissa_low : 32;
	  uint64_t mantissa_high : 20;
	  uint64_t exponent : 11;
	  uint64_t sign : 1;
	  } c;
	} double_ieee754;

If no argument is given, then the userdata represents zero (0), alternatively you can set it to number `x` (first form).

You may also initialise the userdata by passing both its sign bit, exponent, and the high and low parts of the mantissa (second form).

bytes.setieee (a, options)

Sets components in 'ieee' structure `a`. Accepted `options` are one or more pairs:

- `'double'`: any Agena number,
- `'signbit'`: the sign bit, 1 for minus, 0 for plus,
- `'expo'`: the biased exponent, an unsigned 4-byte integer,
- `'high'`: high-word part of the mantissa,
- `'low'`: low-word part of the mantissa.

Example:

```
> bytes.setieee(a, double=Pi, signbit=1);  # sets -Pi to 'ieee' object a
```

bytes.getieee (a [, options])

Returns the components of 'ieee' structure `a`. If only `a` is given, then its five components number, sign bit, exponent, mantissa high-word and mantissa low-word are returned, in this order.

If one or more of the following strings are passed as `options`, then the requested components will be returned in the order given by the user:

- `'double'`: any Agena number,
- `'signbit'`: the sign bit, 1 for minus, 0 for plus,
- `'expo'`: the biased exponent, an unsigned 4-byte integer,
- `'high'`: high-word part of the mantissa,
- `'low'`: low-word part of the mantissa.

Example:

```
> bytes.getieee(a, 'double', 'signbit'):
-3.1415926535898        1
```

Following are specialised functions for 'ieee' data:

bytes.setieeesignbit (a, bit [, option])

Sets the sign bit of ieee structure `a` to `bit`, which is either 1 for minus or 0 for plus. If any `option` is given, the function returns the updated values for double, sign bit, exponent, high and low-word mantissa, in this order.

bytes.setieeeexpo (a, e [, option])

Sets the biased exponent of ieee structure `a` to non-negative integer `e`. If any `option` is given, the function returns the updated values for double, sign bit, exponent, high and low-word mantissa, in this order.

bytes.setieeehigh (a, hi [, option])

Sets the high-word of the mantissa of ieee structure `a` to non-negative integer `hi`. If any `option` is given, the function returns the updated values for double, sign bit, exponent, high and low-word mantissa, in this order.

bytes.setieeelow (a, lo [, option])

Sets the high-word of the mantissa of ieee structure `a` to non-negative integer `lo`. If any `option` is given, the function returns the updated values for double, sign bit, exponent, high and low-word mantissa, in this order.

bytes.setieeedouble (a, v [, option])

Sets the floating-point component in ieee structure `a` to Agena number `v`. If any `option` is given, the function returns the updated values of the sign bit, exponent, high and low-word mantissa in `a`, in this order.

bytes.getieeedouble (a)

Returns the floating-point component in ieee structure `a`, an Agena number.

11.2.4 32-bit Integer Operations

The following functions process 32-bit signed and unsigned integers.

Please note, that by default, Agena including the functions listed below, work in unsigned mode. You can switch to signed operations by issuing

```
> environ.kernel(signedbits = true);
```

on the command line or in a (library) file.

bytes.add32 (a, b [, ···])

Adds two or more numbers `a`, `b`, ... using 4-byte signed or unsigned integer arithmetic. The return is an integer.

You can switch from unsigned (the default) to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **&+** operator.

bytes.sub32 (a, b [, ···])

Subtracts two or more numbers `a`, `b`, ... using 4-byte sigend or unsigned integer arithmetic. The return is an integer.

You can switch from unsigned (the default) to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **&-** operator.

bytes.mul32 (a, b [, ···])

Multiplies two or more numbers `a`, `b`, ... using 4-byte signed or unsigned integer arithmetic. The return is an integer.

You can switch from unsigned (the default) to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **&*** operator.

bytes.muladd32 (a, b [, ···])

Multiplies two numbers `a`, `b`, and adds further numbers `c`, ... using 4-byte signed or unsigned integer arithmetic. The return is the integer `a*b + c + ...`

You can switch from unsigned (the default) to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **bytes.add32**, **bytes.mul32**.

bytes.div32 (a, b [, ···])

Divides two or more numbers a, b, ... using 4-byte signed or unsigned integer arithmetic. The return is an integer.

You can switch from unsigned (the default) to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **&/** operator.

bytes.mod32 (a, b)

Takes the modulo a % b (with % the C modulo operator, not Agena's **%**), using 4-byte signed or unsigned integer arithmetic. The return is an integer.

You can switch from unsigned (the default) to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **math.modulus**.

bytes.divmod32 (a, b)

Returns the quotient and remainder of the 4-byte division a/b.

See also: **bytes.div32**, **bytes.mod32**.

bytes.mulmod32 (a, b, m)

Conducts modular multiplication (a*b) % m, avoiding overflow. a, b, m must all be unsigned integers. If m is zero, the function returns zero.

See also: **bytes.powmod32**.

bytes.powmod32 (a, b, m)

Conducts modular exponentiation (a ^ b) % m, avoiding overflow. a, b, m must all be unsigned integers. If m is zero, the function returns zero.

See also: **bytes.mulmod32**.

bytes.and32 (···)

Conducts a binary AND operation on all the arguments (none, one or multiple signed or unsigned 32-bit integers) and returns an integer.

See also: **&&** operators, **bytes.interweave**.

bytes.bintogray (n)

Returns converts the Gray code integer representation (32-bit) of `n` to its standard binary representation and returns an integer (an Agena number).

See also: **bytes.graytobin**.

bytes.arshift32 (x, n)

Returns the 32-bit signed or unsigned integer `x` shifted `n` bits to the right. The number `n` may be any representable integer. Negative displacements shift to the left.

This shift operation is what is called arithmetic shift. Vacant bits on the left are filled with copies of the higher bit of `x`, thus preserving the sign of `x`; vacant bits on the right are filled with zeros. In particular, displacements with absolute values higher than 31 result in zero or 0xFFFFFFFF (all original bits are shifted out).

See also: << and >> operators, **bytes.shift32**.

bytes.extract32 (n, field [, width])

Returns the unsigned number formed by the bits field to `field + width - 1` from `n`. Bits are numbered from 0 (least significant) to 31 (most significant). All accessed bits must be in the range [0, 31].

The default for `width` is 1.

Signed 32-bit integers `n` are not supported.

See also: **bytes.replace32**.

bytes.bintogray (n)

Returns converts the Gray code integer representation (32-bit) of `n` to its standard binary representation and returns an integer (an Agena number).

See also: **bytes.bintogray**.

bytes.interweave (hx, lx [, option [, mask [, sh [, n]]]])

Takes two unsigned 4-byte words `hx` and `lx` and applies one of the following binary operations on them: `'xor'` (the default), `'and'`, `'or'`, see third argument `option`.

By passing a non-negative `mask` as the optional fourth argument, the mask is applied to the intermediate result, the default is 0xFFFFFFFF.

If a fifth positive `sh` integer is given, the intermediate result is right-shifted `sh` bits; if `sh` is a negative integer, it is left-shifted `sh` bits. If `sh` is 0 (the default), there is no shift.

If a sixth argument `n` is given, and if n is positive, the intermediate result is taken modulo `n`. If n is negative, the Fibonacci modulus $2^{|n|}$ is computed, see **hashes.fibmod2**.

Thus:

- `option` = `'or'`: (((hx || lx) && `mask`) >> `sh`) % n, if `sh` > 0,
- `option` = `'and'`: (((hx && lx) && `mask`) >> `sh`) % n, if `sh` > 0,
- `option` = `'xor'`: (((hx ^ ^ lx) && `mask`) >> `sh`) % n, if `sh` > 0,

and

- `option` = `'or'`: (((hx || lx) && `mask`) << |`sh`|) % n, if `sh` < 0,
- `option` = `'and'`: (((hx && lx) && `mask`) << |`sh`|) % n, if `sh` < 0,
- `option` = `'xor'`: (((hx ^ ^ lx) && `mask`) << |`sh`|) % n, if `sh` < 0.

bytes.isint32 (n)

Checks whether the given number `n` is in the range of a signed or an unsigned 4-byte integer and returns **true** or **false**.

To check in which mode Agena is, check the **environ.kernel/signedbits** setting. It should usually be unsigned.

If you are in unsigned mode, the argument should be in the range 0 .. **environ.kernel().maxulong** = 0 .. 4'294'967'295.

If you are in signed mode, `n` should be in the range **environ.kernel().minlong** .. **environ.kernel().maxlong** = -2'147'483'647 .. 2'147'483'647.

Example:

```
> environ.kernel().signedbits:  # we are in unsigned mode (C uint32_t's).
false

> bytes.isint32(4'294'967'295):
true
```

```
> bytes.isint32(4'294'967'295 + 1):
false
```

bytes.mask32 (n)

Returns an integer with the first `n` bits set to one, e.g. `bytes.mask32(3)` → 7.

bytes.nand32 (···)

Conducts a binary complementary OR operation on all the arguments (none, one or multiple signed 32-bit integers) and returns an integer. There is no `unsigned` mode available, as the results would be of no use.

See also: **nand**.

bytes.nextbit (mask)

Gets and clears the next bit from the unsigned 4-byte `mask`, starting with the most significant bit. The function returns the modified value of `mask` and the respective bit position 0 .. 31.

bytes.nor32 (···)

Conducts a binary complementary OR operation on all the arguments (none, one or multiple signed 32-bit integers) and returns an integer. There is no `unsigned` mode available, as the results would be of no use.

See also: **nor**.

bytes.not32 (x)

Conducts a binary NOT operation on the signed or unsigned 32-bit integer `x` and returns an integer.

See also: ~~ operator.

bytes.numto32 (x)

Converts a number `x` to its signed or unsigned 4-byte integer representation. Note that very large values (positive or negative) might overflow, e.g. bytes.numto32(2^32+1) ⇒1.

You can switch from unsigned to signed arithmetic by setting **environ.kernel**(signedbits = **true**), and from signed to unsigned arithmetic by **environ.kernel**(signedbits = **false**).

See also: **math.uexponent**.

bytes.or32 (···)

Conducts a binary OR operation on all the arguments (none, one or multiple signed or unsigned 32-bit integers) and returns an integer.

See also: || operator, **bytes.interweave**.

bytes.parity32 (x)

Determines the parity of the unsigned 4-byte integer `x`, i.e. the number of 1-bits in `x` modulo 2.

Returns 0 if `x` is of even parity, and 1 in case of odd parity.

See also: **bytes.parity8**, **hashes.parity**.

bytes.parity8 (x)

Determines the parity of the unsigned byte `x`, i.e. the number of 1-bits in `x` modulo 2.

Returns 0 if `x` is of even parity, and 1 in case of odd parity.

See also: **bytes.parity24**, **hashes.parity**.

bytes.replace32 (n, v, field [, width])

Returns a copy of `n`, an unsigned 32-bit integer, with the bits `field` to `field + width - 1` replaced by the value `v`.

Signed 32-bit integers `n` are not supported.

See **bytes.extract32** for details about `field` and `width`.

bytes.rotate32 (x, n)

Rotates the bits in the 32-bit integer `x n` displacements to the right if `n >= 0`, or `n` places to the left if `n < 0`. The return is a 32-bit integer.

Internally the function uses unsigned 32-bit integers by default. You can change this to signed integers by calling **environ.kernel** with the `'signedbits'` option.

See also: <<< and >>> operators.

bytes.shift32 (x, n)

Shifts the bits in the 32-bit integer `x n` displacements to the left if `n` < 0, and to the right if `n` > 0.

Internally the function uses unsigned 32-bit integers by default. You can change this to signed integers by calling environ.kernel with the `'signedbits'` option.

See also: << and >> operators, **bytes.arshift32**, **bytes.interweave**.

bytes.xnor32 (···)

Conducts a binary complementary exclusive-OR operation on all the arguments (none, one or multiple signed 32-bit integers) and returns an integer. There is no `unsigned` mode available, as the results would be of no use.

See also: **xnor**.

bytes.xor32 (···)

Conducts a binary exclusive-OR operation on all the arguments (none, one or multiple signed or unsigned 32-bit integers) and returns an integer.

See also: ^ ^ operator, **bytes.interweave**.

11.3 mapm - Arbitrary Precision Library

11.3.1 Introduction

As a *plus* package, in Solaris, Linux, Mac OS X, and Windows, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import mapm`.

In OS/2 and DOS, the package is built into the binary executable and does not need to be activated with **import**.

The package provides functions to conduct arbitrary precision mathematics with real and complex numbers. It uses Mike's Arbitrary Precision Math Library, written by Michael C. Ring.

Standard operators like +, -, *, /, \, ^, %, <, =, >, ~=, ~<>, unary minus and the bitwise operators ~~, &&, ||, ^ ^, <<, >> are supported.

All function names in this library usually begin with the letter x for real numbers and the letter c for complex numbers.

The package uses its own kind of real and complex numbers which are different from Agena numbers: use **mapm.xnumber** and **mapm.xtonumber** or **mapm.cnumber** and **mapm.xtocomplex**, respectively, to convert between them. Also, mapm numbers have a `use-defined` type: `xnumber` for real numbers and `cnumber` for complex ones.

Alternatively, use **mapm.new**: If you pass one argument, that is an Agena number, a string representing a number or a real arbitrary-precision `mapm` number, a real number will be created. If you pass two such arguments, a complex number will be created. **mapm.tonumber** returns either a real Agena number stored to a `mapm` number or both the real and imaginary part of a complex `mapm` number. The function, contrary to **mapm.xtonumber**, also accepts **undefined**, +/-**infinity** and just returns them. Note that when passing a number or complex number, the fractional part is cut off after the 14th fractional digit. Thus:

It is always advised to pass numbers *as strings* if possible. This is because Agena uses C doubles which are not always 100 % precise in a mathematical sense. See the excursus in Chapter 11.3.4 for a means to control precision.

By default, the precision for calculations is set to 17 digits, but you can change this any time with the **mapm.xdigits** function, see example below. When a division by zero occurs, all the functions return the Agena constant **undefined**.

Use **mapm.xattrib** which just requires one argument to explore mapm numbers: The function returns a table with the following information:

- 'm_apm_datalength': The number of base 10 digits in the number. In other words, the number 5678 will have a data length of 4. The number 1234567 will have a data length of 7. The value may vary across platforms.
- 'm_apm_exponent': The exponent of the number.
- 'm_apm_sign': The sign of the number. sign = -1 is a negative number. sign = +1 is a positive number. sign = 0 is a number that is exactly 0.
- 'm_apm_data': a raw dump of the internally stored numeric data. The data may vary across platforms.
- 'm_apm_datalength_aligned': number of values in the 'm_apm_data' dump.

In this context, see also: **mapm.xstostring**, for example:

```
> mapm.xstostring(mapm.new(1+DoubleEps), 21, true):
1.00000000000000022045E+0
```

All **mapm** functions can be called OOP-style. You might be interested in the **unimath** package for more functions, check Chapter 11.22.

11.3.2 Real Domain

Let us start with examples from the real domain:

```
> import mapm;

> mapm.xdigits(100);  # precision set to 100 digits

> a := mapm.new('0.5');

> type(a), typeof(a):
userdata, xnumber

> a*mapm.Pi:
1.57079632679489662

> b := mapm.xnumber(0.5):   # same as mapm.new(0.5)
0.50000000000000000

> b*mapm.Pi:
1.57079632679489662
```

You cannot directly compare MAPM numbers with Agena numbers:

```
> a - b = 0:
false

> a - b = mapm.new(0):
true
```

You can input very large numbers that are outside the domain of Agena numbers by passing them as decimal or as hexadecimal strings, e.g. '100.1' or '-0x100'.

See also: The **long** package implementing 80-bit floating-point arithmetic, described in Chapter 11.15.

The mathematical functions are:

Function	Meaning	Function	Meaning
mapm.xabs	absolute value	**mapm.xexp**	exponential function
mapm.xarccos	arc cosine	**mapm.xexp2**	base-2 exponentation
mapm. xarccosh	inverse hyperbolic cosine	**mapm.xexp10**	base-10 exponentation
mapm.xadd	addition	**mapm.xfactorial**	factorial
mapm. xgamma	Gamma function	**mapm.xpsi**	Psi/Digamma function
mapm.xarccos	arcus cosine	**mapm.xidiv**	integer division
mapm.xarccsc	arccosecant	**mapm.xln**	natural logarithm
mapm.xarcsec	arcsecant	**mapm.xlog**	logarithm of given base
mapm.xarcsin	inverse sine	**mapm.xlog2**	base-2 logarithm
mapm.xarcsinh	inverse hyperbolic sine	**mapm.xlog10**	common logarithm
mapm.xarctan	inverse tangent	**mapm.xmul**	multiplication
mapm. xarctan2(x, y)	4 quadrant inverse tangent	**mapm.xpow**	power
mapm.xsec	secant	**mapm.xsech**	hyperbolic secant
mapm. xarctanh	hyperbolic inverse tangent	**mapm.xsign**	sign
mapm.xcbrt	cubic root	**mapm.xsin**	sine
mapm.xcosc	un-normalised cardinal cosine	**mapm.xsinc**	un-normalised cardinal sine
mapm.xcos	cosine	**mapm.xsincos**	sine and cosine
mapm.xcosh	hyperbolic cosine	**mapm.xsinh**	hyperbolic sine
mapm.xcot	cotangent	**mapm. xsinhcosh**	hyperbolic sine and cosine
mapm.xcoth	hyperbolic cotangent	**mapm.xsqrt**	square root
mapm.xcsc	cosecant	**mapm.xsub**	subtraction
mapm.xcsch	hyperbolic cosecant	**mapm.xtan**	tangent
mapm.xdiv	division	**mapm.xtanc**	un-normalised cardinal tangent
mapm.xrecip	reciprocal	**mapm.xtanh**	hyperbolic tangent
mapm.xerf	error function	**mapm.xfma (x, y, z)**	fused multiply-add
mapm.xerfc	complementary error function	**mapm.xterm (c, x, n)**	computes c*x^n

Function	Meaning	Function	Meaning
mapm.xhypot (x, y [, prec])	hypotenuse with optional precision	**mapm.xsquare**	square
mapm.xcathet(x, y [, prec])	sqrt(a^2-b^2) with optional precision	**mapm. xrandom()**	random mapm number (no arg)
mapm.xcube	cube	**mapm. xrandomseed(x)**	seed setting, x a string with digits
mapm.xint	integral part	**mapm.xintfrac**	integral & fractional part
mapm.xfrac	fractional part	**mapm.xipow**	integer power
mapm.xgcd (x, y)	Greatest Common Divisor	**mapm.xlcm (x, y)**	Least Common Multiple
mapm.xinvmod (x, m)	modular multiplicative inverse	**mapm.xmulmod (x, y, m)**	modular multiplication
mapm.xpow-mod(x, y, m)	modular exponentiation	**mapm.max(x, y)**	maximum value of real or complex x, y
mapm.min(x, y)	maximum value of of real or complex x, y	**mapm. minmax(x, y)**	minimum and maximum value of real or complex x, y, in this order
mapm. swap(x, y)	swaps the values of real or complex x, y, in-place	**mapm.approx (x, y [, eps])**	works like **approx**
mapm.xmax (x, y)	maximum of x, y	**mapm.xmin (x, y)**	minimum of x, y

Most of the **mapm** functions accept an optional last argument - a non-negative integer - giving the individual precision.

You do not need to globally change the precision with **mapm.xdigits** or pass a precision to the following functions as they mostly automatically auto-adapt to prevent round-off errors with positive integer exponents: **mapm.xsquare**, **mapm.xcube** **mapm.xipow**, **mapm.xpow**, **mapm.xterm**, **mapm.xexp2**, **mapm.xexp10** and the corresponding operators, plus the bitwise-AND, bitwise-OR and bitwise-XOR operators **&&**, **||**, **^ ^** and complex number exponentiation.

The package provides the following metamethods:

Operator	Name	Description
+	`'__add'`	addition
-	`'__sub'`	subtraction
*	`'__mul'`	multiplication
/	`'__div'`	division
\	`'__intdiv'`	integer division
%	`'__mod'`	modulo
^	`'__pow'`	power with any exponent
**	`'__ipow'`	power with integer exponent
&&	`'__band'`	bitwise AND

Operator	Name	Description
||	'__bor'	bitwise OR
^ ^	'__bxor'	bitwise XOR
~~	'__bnot'	bitwise NOT (32-bit unsigned)
<<	'__bshl'	bitwise left-shift
>>	'__bshr'	bitwise right-shift
-	'__unm'	unary minus
<	'__lt'	less-than
=	'__eq'	equals
~=	'__aeq'	approximately equal
<=	'__le'	less-or-equal
<>	n/a	not equals
~<>	n/a	not approximately equal
abs	'__abs'	absolute value
|-	'__absdiff'	absolute difference
sign	'__sign'	sign
recip	'__recip'	reciprocal
square	'__square'	square (x ^ 2)
cube	'__cube'	cube (x ^ 3)
sqrt	'__sqrt'	square root ($\sqrt{x}$)
invsqrt	'__invsqrt'	inverse square root ($1/\sqrt{x}$)
ln	'__ln'	natural logarithm
exp	'__exp'	exponential function
antilog2	'__antilog2'	exponentiation with base 2: 2 ^ x
antilog10	'__antilog10'	exponentiation with base 10: 10 ^ x
sin	'__sin'	sine
cos	'__cos'	cosine
tan	'__tan'	tangent
sinh	'__sinh'	hyperbolic sine
cosh	'__cosh'	hyperbolic cosine
tanh	'__tanh'	hyperbolic tangent
arcsec	'__arcsec'	arcsecant
arcsin	'__arcsin'	arcus sine
arccos	'__arccos'	arcus cosine
arctan	'__arctan'	arcus tangent
sinh	'__sinh'	hyperbolic sine
cosh	'__cosh'	hyperbolic cosine
tanh	'__tanh'	hyperbolic tangent
sinc	'__sinc'	un-normalised cardinal sine
entier	'__entier'	floor
int	'__int'	integral part
frac	'__frac'	fractional part
integral	'__integral'	integral number check
fractional	'__fractional'	fractional number check
even	'__even'	even number check
odd	'__odd'	odd number check

Operator	Name	Description
zero	`'__zero'`	checks for zero
nonzero	`'__nonzero'`	checks for non-zero
n/a	`'__gc'`	garbage collection
n/a	`'__tostring'`	conversion to a string, e.g. for the pretty printer

Other functions are:

Function	Meaning	Function	Meaning
mapm.xceil	ceil function	**mapm. xexponent**	exponent
mapm.xfloor	floor function	**mapm.xinv**	reciprocal
mapm.xiseven	test for even number	**mapm.xisint**	check for an integral
mapm.xisodd	test for odd number	**mapm.xmod**	modulo
mapm.xround	rounds downwards to the nearest integer	**mapm.xneg**	negates a number
mapm. xcompare(x, y)	comparison, returns -1 if x < y, 0 if x = y, and 1 if x > y	**mapm.xnumber**	converts an Agena number or a string representing a decimal or hexa-decimal number to an arbitrary precision number
mapm.xdigits	sets the number of digits used in all sub-sequent calcula-tions. With no argu-ment, returns the current setting. (default is 17)	**mapm. xtonumber** **mapm. tonumber**	convert an arbitrary precision number to an Agena number. A second optional argument n gives the precision for this specific number, with n > 0.
mapm. xdigitsin	significant digits	**mapm.xtostring (x, [n, [bool]])**	converts an arbitrary precision number to a string, with n > 0
mapm.xispow2	tests for a power of two	**mapm.xisprime**	tests for a prime
mapm. checkxnumber	checks for one or more real numbers, issues an error otherwise	**mapm. checkcnumber**	checks for one or more complex numbers, issues an error otherwise
mapm. isxnumber	tests for one or more real numbers, returns a Boolean	**mapm. iscnumber**	tests for one or more complex numbers, returns a Boolean
mapm. checkinteger	Checks for one or more integral	**mapm. checknonnegint**	Checks for one or more non-negative integral numbers

Function	Meaning	Function	Meaning
	numbers and issues an error otherwise		and issues an error otherwise
mapm. check-nonnegative	Checks for one or more non-negative MAPM numbers and issues an error otherwise	**mapm. checkposint**	Checks for one or more positive integral numbers and issues an error otherwise
mapm. checkpositive	Checks for one or more positive MAPM numbers and issues an error otherwise	**mapm. xisnonnegative**	Tests for one or more non-negative real-valued numbers, returns **true** or **false**
mapm. xispositive	Tests for one or more positive real-valued numbers, returns **true** or **false**	**mapm. xisposint**	Tests for one or more positive integral numbers, returns **true** or **false**
mapm. xisnonnegint	Tests for one or more non-negative integral numbers, returns **true** or **false**	**mapm. xisnegint**	Tests for one or more egative integral numbers, returns **true** or **false**
mapm. xisnegative	Tests for one or more negative integral numbers, returns **true** or **false**	**mapm.new(x) mapm.new(r,i)**	Creates a real or complex number
mapm.xchebyt (n, x)	n-th Chebyshev polynomial of the first kind at point x	**mapm.trim()**	Frees internal memory marked for deletion.

Available constants with precision of 1,000 digits are:

Constant	Value	Comment
mapm.Pi	π	
mapm.Pi2	2π	
mapm.PiO2	$\pi/2$	
mapm.PiO4	$\pi/4$	
mapm.PiO180	$\pi/180$	radians per degree
mapm.InvPi	$1/\pi$	
mapm.InvPi2	$1/(2\pi)$	
mapm.InvPiO4	$4/\pi$	
mapm.InvPiSqO4	$4/\pi^2$	
mapm.Zeta2	$Pi^2/6$	
mapm.E	E = exp(1)	
mapm.sqrt2	$\sqrt{2}$	
mapm.sqrt3	$\sqrt{3}$	

Constant	Value	Comment
mapm.ln2	ln(2)	
mapm.invln2	1/ln(2)	
mapm.invsqrt2	$1/\sqrt{2}$	
mapm.Phi	$(1 + \sqrt{5})/2$	Golden ratio
mapm.InvPhi	$1/((1 + \sqrt{5})/2)$	inverse Golden ratio
mapm.InvPhiSq	$(1/((1 + \sqrt{5})/2))^2$	
mapm.lnPhi	$\ln(1 + \sqrt{5})/2$	logarithm of Golden ratio
mapm.InvlnPhi	$1/\ln(1 + \sqrt{5})/2$	inverse
mapm.EulerGamma	0.57721566...	Euler-Mascheroni constant
mapm.naught **mapm.nought**	0	zero
mapm.one	1	
mapm.two	2	
mapm.three	3	
mapm.four	4	
mapm.five	5	
mapm.six	6	
mapm.seven	7	
mapm.eight	8	
mapm.nine	9	
mapm.ten	10	
mapm.eleven	11	
mapm.twelve	12	
mapm.fifty	50	
mapm.hundred	100	
mapm.thousand	1,000	
mapm.half	0.5	
mapm.quarter	0.25	
mapm.tenth	0.1	
mapm.fifth	0.2	
mapm.hundredth	0.01	
mapm.thousandth	0.001	

These constants have been defined in source file lib/mapm.agn.

11.3.3 Complex Domain

Let us start with some examples:

```
> import mapm;
```

The precision can be set with **mapm.xdigits**:

```
> mapm.xdigits(17);  # precision set to 17 digits (the default)
```

For the complex domain, use **mapm.cnumber** or **mapm.new** to define complex numbers of arbitrary precision. You can pass both two numbers, strings or real mapm numbers to this function:

```
> x := mapm.new(1, 2);

> y := mapm.cnumber(3, 4);
```

Addition:

```
> x + y:
4.00000000000000000+6.00000000000000000*I
```

Convert the result to a complex Agena number:

```
> mapm.ctocomplex(ans):
4+6*I
```

Determine the absolute value, the return is a real mapm number:

```
> abs(x):
2.23606797749978970
```

Get the natural logarithm:

```
> ln(x):
0.80471895621705019+1.10714871779409050*I
```

Get real and complex part of the previous calculation, to be returned as real mapm numbers:

```
> real(ans), imag(ans):
0.80471895621705019     1.10714871779409050
```

Most of the operators support complex mapm numbers. If you use binary operators or complex mapm functions with two arguments, always pass complex mapm numbers - with the exception of the **** operator, you cannot mix operands or arguments of different types.

The metamethods for complex-valued MAPM numbers (operators) are:

Operator	Name	Description
+	'__add'	addition
-	'__sub'	subtraction
*	'__mul'	multiplication
/	'__div'	division
^	'__pow'	power with any exponent
**	'__ipow'	power with integer exponent (of type number)
-	'__unm'	unary minus
=	'__eq'	equals
<>	n/a	not equals

Operator	Name	Description
abs	'__abs'	absolute value
sign	'__sign'	sign
recip	'__recip'	reciprocal
square	'__square'	square (x ^ 2)
cube	'__cube'	cube (x ^ 3)
sqrt	'__sqrt'	square root ($\sqrt{x}$)
ln	'__ln'	natural logarithm
exp	'__exp'	exponential function
sin	'__sin'	sine
cos	'__cos'	cosine
tan	'__tan'	tangent
sinh	'__sinh'	hyperbolic sine
cosh	'__cosh'	hyperbolic cosine
tanh	'__tanh'	hyperbolic tangent
arcsec	'__arcsec'	arcsecant
arcsin	'__arcsin'	arcus sine
arccos	'__arccos'	arcus cosine
arctan	'__arctan'	arcus tangent
sinh	'__sinh'	hyperbolic sine
cosh	'__cosh'	hyperbolic cosine
tanh	'__tanh'	hyperbolic tangent
sinc	'__sinc'	un-normalised cardinal sine
n/a	'__gc'	garbage collection
n/a	'__tostring'	conversion to a string, e.g. for the pretty printer

There are also some few functions for mapm complex numbers. As with the metamethods, all have been implemented in C for the sake of speed, with the exception of **mapm.carctan2** which you can find in the lib/mapm.agn library file.

The imaginary unit is **mapm.I**.

Function	Meaning	Function	Meaning
mapm. cnumber	converts an Agena number or a string representing two numbers to an arbitrary precision complex number	**mapm. carccosh**	inverse hyperbolic cosine
mapm. ctocomplex	converts an arbitrary precision complex number to an Agena complex number	**mapm. carcsinh**	inverse hyperbolic sine
mapm. ctonumber	convert an arbitrary precision complex number to two	**mapm. carctanh**	inverse hyperbolic tangent

Function	Meaning	Function	Meaning
mapm. tonumber	Agena numbers, the real and imaginary parts		
mapm.ctostring	converts an arbitrary precision complex number to two strings representing the real and imaginary parts	**mapm. carctan2**	4 quadrant inverse tangent
mapm.xdigits	sets the number of digits used in all subsequent calculations. With no argument, returns the current setting. (default is 17)	**mapm. cargument**	argument (phase angle)
mapm. xdigitsin	significant digits	**mapm.cfma**	fused multiply-add operation
mapm.csec	secant	**mapm.csech**	hyperbolic secant
mapm.ccot	cotangent	**mapm.ccoth**	hyperbolic cotangent
mapm.ccsc	cosecant	**mapm.ccsch**	hyperbolic cosecant
mapm.csinc	un-normalised cardinal sine	**mapm.ctanc**	un-normalised cardinal tangent
mapm.ccosc	un-normalised cardinal cosine	**mapm.csincos**	sine and cosine
mapm. csinhcosh	hyperbolic sine and hyperbolic cosine	**mapm.cexp2**	power of two
mapm.clog2	base-2 logarithm	**mapm.cexp10**	power of ten
mapm.clog10	base-10 logarithm	**mapm.ccbrt**	cubic root

11.3.4 Excursus: Precision

When passing an Agena number or complex number to **mapm.new**, **mapm.xnumber** and **mapm.cnumber**, the underlying MAPM library by default cuts off the fractional part after 14 digits, sometimes causing unexpected behaviour of the library.

However, all three **mapm** functions support the `digits` option which you can pass a number n between 0 to 21. If you give this option along with an Agena number or a complex Agena number, truncation will take place after the n-th fractional digit. The default is 14, the default of the underlying MAPM library.

This actually does not solve the general issue of inexact storage of numbers that are not a power of two on a computer, but gives you more flexibility on accuracy.

Still, it is strongly advised to pass strings representing numbers to **mapm.new**, **mapm.xnumber** and **mapm.cnumber** instead of (complex) numbers.

```
> import mapm

> x := mapm.new(1):
1.00000000000000000

> y := mapm.new(1 + DoubleEps):
1.00000000000000000

> x = y:
true

> x := mapm.new(1):
1.00000000000000000

> y := mapm.new(1 + DoubleEps, digits = 21):
1.00000000000000022

> x = y:
false
```

When this will not work:

```
> x := mapm.new(2.1, digits = 21):
2.10000000000000009

> x^2:
4.41000000000000037
```

You might expect $x^2 = 4.41$, though:

```
> x := mapm.new(2.1):
2.10000000000000000

> x^2:
4.41000000000000000
```

See also the MPFR and dd packages, Chapters 11.5 and 11.23.

11.4 gmp - GNU Multiple Precision Arithmetic Library

As a *plus* package, the **gmp** package is not part of the standard distribution and must be activated with the **import** statement, i.e. `import gmp`.

The **gmp** library is a binding to the GMP library providing multiple functions to conduct signed and unsigned integer arithmetic of arbitrary precision.

The package provides various metamethods for easy entry of calculations, too. See Chapter 11.4.6 for a list.

Signed and unsigned integers - `mpints` for short in this context - are represented by **gmp** userdata objects which can be passed to the functions and operators described below.

OS/2, Solaris, Linux, Mac OS X and UNIX users may have to install the original GMP 6.x libraries separately in order for this binding to work. In order for this binding to work on Intel CPUs, you may need at least a Sandybridge processor.

```
> import gmp

> a, b := gmp.uint(1), gmp.uint(2)  # unsigned integers

> a + b:
mp(3)

> a, b := gmp.sint(2), gmp.sint(3)  # signed integers

> a * b:
mp(6)
```

Note that most functions in this package accept mpints, non-negative integral Agena numbers and hexadecimal strings as input.

11.4.1 Creation of Signed and Unsigned Integers

gmp.uint (n)

gmp.uint (numstr [, base])

Creates an unsigned integer object (mpz_t GMP userdata object) from an unsigned integer `n`, or a string `numstr` representing an unsigned integer.

If you pass a string you may indicate whether it is in decimal format by passing the optional second argument 10 for `base`, and if its is in hexadecimal encoding, pass 16, which is the default [sic !].

If `n` is already an integer object, it is simply returned.

See also: **gmp.sint**, **gmp.setstring**.

gmp.sint (n)

gmp.sint (numstr [, base])

Like **gmp.uint**, but creates a signed integer mpz_t GMP userdata object.

11.4.2 Signed and Unsigned Integer Arithmetic

The following operators and functions can be applied both on signed and unsigned mpints:

gmp.add (a, b)

Adds two mpints `a`, `b`, and returns a new mpint. Used by `__add` metamethod, i.e.: gmp.add(`a`, `b`) = `a + b`.

gmp.subtract (a, b)

Subtracts two mpints `a`, `b` and returns a new mpint. Used by `__sub` metamethod, i.e.: gmp.subtract(`a`, `b`) = `a - b`.

gmp.multiply (a, b)

Multiplies two mpints `a`, `b` and returns a new mpint. Used by `__mul` metamethod, i.e.: gmp.multiply(`a`, `b`) = `a * b`.

gmp.divide (a, b)

Divides two mpints `a`, `b` and returns a new mpint. Used by `__div` metamethod, i.e.: gmp.divide(`a`, `b`) = `a / b`.

gmp.power (a, b)

Raises mpint `a` to the power of mpint `b` and returns the new mpint a^b. `b` may optionally be a non-negative integral Agena number. Used by the `__pow` and `__ipow` metamethods, i.e.: gmp.power(`a`, `b`) = `a ^ b` = `a**b`. With negative `b`, returns the mpint zero.

gmp.addmul (r, a, b)

Multiplies two mpints `a`, `b`, adds the result to `r` and returns the updated value of `r`: i.e.: gmp.addmul(`r`, `a`, `b`) <=> `r +:= a * b`.

gmp.submul (r, a, b)

Multiplies two mpints `a`, `b`, subtracts the result from `r` and returns the updated value of `r`: i.e.: gmp.submul(`r`, `a`, `b`) <=> `r` -:= `a` * `b`.

gmp.max (a, b)

Determines the maximum of the signed or unsigned integers `a`, `b` and returns it.

See also: **gmp.min**, **gmp.minmax**.

gmp.min (a, b)

Determines the minimum of the signed or unsigned integers `a`, `b` and returns it.

See also: **gmp.max**, **gmp.minmax**.

gmp.minmax (a, b)

Determines the minimum and the maximum of the signed or unsigned integers `a`, `b` and returns them in this order.

See also: **gmp.max**, **gmp.min**.

gmp.modulus (a, b)

Computes the modulo of two mpints `a`, `b` and returns a new mpint. Used by `__mod` metamethod, i.e.: gmp.modulus(`a`, `b`) = `a` % `b`.

gmp.neg (a)

Returns -`a`, with `a` an mpint, as a new mpint. Used by `__unm` metamethod, i.e. gmp.neg(`a`) <=> -`a`. The function does not change its input `a`.

gmp.getd2exp (a)

Similar to **frexp**, converting mpint a to a `mantissa` d, returned as an Agena number, and an `exponent` e, an Agena number by default, such that $a = d * 2^e$, with |d| in the range [0.5, 1). If the exponent e is too big, the function issues an error by default.

If you pass **false** as the second argument, the exponent will be returned as an mpint instead of a number, not issuing an error with very large exponents.

See also: **gmp.mul2exp**, **gmp.tonumber**.

gmp.mul2exp (a, b)

Computes `a * 2^b`, with `a`, `b` mpints, and returns the result in a new mpint. The operation is equivalent to a left shift by `b` bits.

See also: **gmp.getd2exp**.

gmp.tdiv (a, b)

Returns both quotient and remainder of `a / b`, with `a`, `b` units, both rounded towards zero.

gmp.tdivq (a, b)

Divides two mpints `a`, `b` and returns the resulting quotient as a new mpint, rounded towards zero.

gmp.tdivr (a, b)

Divides two mpints `a`, `b` and returns the resulting remainder as a new mpint, rounded towards zero.

gmp.invm (a, m)

Computes the modular multiplicative inverse and returns the result as a new mpint.

See also: **gmp.mulm**, **gmp.powm**.

gmp.mulm (a, b, m)

With three uints, computes `(a * b) % m` (that is conducts modular multiplication) and returns the result as a new mpint.

See also: **gmp.invm**, **gmp.powm**.

gmp.powm (a, b, m [, true])

With three uints, computes `(a ^ b) % m` (that is conducts modular exponentiation) and returns the result as a new mpint.

If you pass **true** as the fourth argument, the function internally calls a GMP function that is intended for cryptographic purposes, where resilience to side-channel attacks is desired, provided that `m` is odd and `b` is positive. Default is **false**.

See also: **gmp.invm**, **gmp.mulm**.

gmp.root (a, n)

Computes the truncated integer part of the n-th root of a, with a an mpint, n a number or an mpint, and returns the result as a new mpint.

gmp.rootrem (a, n)

Computes the truncated integer part of the n-th root of a, with a an mpint, n a number or an mpint, and returns the result as a new mpint. The second result is the remainder, a-root**n, also an mpint.

gmp.log2 (a [, true])

Like **ilog2**, but for mpint a. By default, the return is an Agena number. By passing **true**, the function returns an mpint instead.

gmp.log10 (a [, true])

Like **ilog10**, but for mpint a. By default, the return is an Agena number. By passing **true**, the function returns an mpint instead.

11.4.3 Number Theoretic Functions

gmp.testprime (a [, reps])

Checks whether mpint a is a prime and returns:

- 0 if a is no prime,
- 1 if a is probably prime,
- 2 if a is definitely prime.

The accuracy of the result can be controlled by the optional second parameter reps which by default is 15, with reasonable values between 15 and 50.

gmp.nextprime (a)

Returns the next prime to mpint a, as a new mpint.

gmp.prevprime (a)

Returns the greatest prime less than mpint a, as a new mpint. If a < 3, returns gmp.uint(0).

gmp.gcd (a, b)

Returns the greatest common divisor of mpints a and b, as a new mpint.

gmp.gcdext (s, t, a, b)

Returns the greatest common divisor of mpint `a` and mpint `b` and returns the result.

In addition the function sets mpints `s` and `t` to coefficients satisfying `a*s + b*t =` (the return).

gmp.lcm (a, b)

Returns the least common multiple of mpints `a` and `b`, as a new mpint.

gmp.invert (a, b)

The function has been deprecated, use **gmp.invm** instead.

gmp.jacobi (a, b)

Computes the Jacobi symbol (`a`/`b`) of the mpints `a`, `b`, which is defined only if `b` is odd. The return is an Agena number: either -1, 0 or 1.

See also: **numtheory.jacobi**.

gmp.legendre (a, p)

Computes the Legendre symbol (`a`/`p`) of the mpints `a`, `b`, which is defined only if `p` is an odd positive prime. The return is an Agena number: either -1, 0 or 1.

gmp.kronecker (a, b)

Computes the Jacobi symbol (`a`/`b`) for mpints `a`, `b`, with the Kronecker extension (`a`/2)=(2/`a`) with odd `a`, and (`a`/2)=0 with even `a`. The return is an Agena number: either -1, 0 or 1.

See also: **numtheory.kronecker**.

gmp.remove (a, b)

Removes all occurrences of the factor `a` from `b`, returns the result as a new mpint.

gmp.factorial (n)

Returns the factorial of *number* `n` or mpint as a new mpint.

gmp.fib (n)

Returns the `n`-th Fibonacci number, with `n` a number or mpint, and returns a new mpint.

gmp.lucas (n)

Sets the n-th Lucas number, with n a number or mpint, and returns a new mpint.

gmp.primorial (n)

Returns the primorial of number n or mpint, i.e. the product of all positive prime numbers $\leq$ n. The return is a new mpint.

gmp.binomial (n, k)

Computes the binomial coefficient n (an mpint) over k (a number) and returns the result as a new mpint.

gmp.perfectpower (n)

Returns **true** if n is a perfect power, i.e., if there exist integers a and b, with b > 1, such that n equals a raised to the power b. Returns **false** otherwise.

gmp.perfectsquare (n)

Returns **true** if n is a perfect square, i.e., if the square root of n is an integer. Under this definition both 0 and 1 are considered to be perfect squares. Returns **false** otherwise.

11.4.4 Bitwise Operations

gmp.band (a, b)

Conducts a bitwise-and, `a and b`, with a, b mpints, and returns the result as a new mpint.

gmp.bor (a, b)

Conducts a bitwise-or, `a or b`, with a, b mpints, and returns the result as a new mpint.

gmp.bxor (a, b)

Conducts a bitwise-xor, `a xor b`, with a, b mpints, and returns the result as a new mpint.

gmp.com (a)

Computes the one's complement of mpint a and returns the result as a new mpint.

gmp.scan0 (a, n)

gmp.scan1 (a, n)

Scan mpint `a`, starting from bit `n` (a number, not an mpint), towards more significant bits, until the first 0 or 1 bit (respectively) is found. The functions return the index of the found bit.
Bit positions start from 0.

gmp.popcount (a)

Computes the population count of mpint `a`, i.e. the number of 1 bits in the binary representation, and returns the result a number.

gmp.mostsigbit (a)

Returns the position of the most significant bit (msb) as a number, counting from bit position number 0. If all bits are cleared, i.e. zero, returns -1.

gmp.leastsigbit (a)

Returns the position of the least significant bit (lsb) as a number, counting from bit position number 0. If all bits are cleared, i.e. zero, returns -1.

gmp.hamdist (a, b)

Computes the hamming distance between mpint `a` and mpint `b`, i.e. the number of positions `a` and `b` have different bit values. The count will be returned as a number.

gmp.setbit (r, n)

Sets bit `n` in mpint `r`. The function returns nothing. `n` is an integer position counting from 0.

gmp.getbit (r, n)

Returns bit `n` in mpint `r`. The function returns a number. `n` is an integer position counting from 0.

gmp.clrbit (r, n)

Clears bit `n` in mpint `r`. The function returns nothing. `n` is an integer position counting from 0.

gmp.combit (r, n)

Conducts a bitwise complement (`not` operation) on bit number `n` in mpint `r`. `n` is an integer position counting from 0. The function returns nothing.

11.4.5 Miscellaneous

gmp.tonumber (a)

Returns the numeric value in mpint `a` as a number.

gmp.tostring (a)

Returns the numeric value in mpint `a` as a string.

gmp.swap (a, b)

Swaps the values in mpints `a` and `b`. The function returns nothing.

gmp.cmp (a, b)

Compares `a` and `b` and returns a positive number if `a` > `b`, 0 if `a` = `b`, and a negative number if `a` < `b`.

gmp.cmpabs (a, b)

Compares the absolute values of `a` and `b` and returns a positive number if `a` > `b`, 0 if `a` = `b`, and a negative number if `a` < `b`.

gmp.iseven (a)

Checks whether mpint `a` represents an even integer and returns **true** or **false**.

gmp.isodd (a)

Checks whether mpint `a` represents an odd integer and returns **true** or **false**.

gmp.ispow2 (a)

Checks whether mpint `a` is a power of two and returns **true** or **false**.

gmp.attrib (a)

Returns various information on mpint a, in a dictionary with the following keys: "sizeinbase10" (size of a measured in number of digits in base 10), "limbs" (size measured in number of limbs), "base" (base).

gmp.sizeinbase (a)

Returns the size of `a` measured in number of digits in the given `base`, an integer. `base` can vary from 2 to 62 and should be even.

11.4.6 Metamethods

The supported metamethods are:

Operator	Name	Description
+	'__add'	addition
-	'__sub'	subtraction
*	'__mul'	multiplication
/	'__div'	division
\	'__intdiv'	division
%	'__mod'	modulo
^	'__pow'	power with any exponent
**	'__ipow'	power with integer exponent
-	'__unm'	unary minus
<	'__lt'	less-than
=	'__eq'	equals
<=	'__le'	less-or-equal
<>	n/a	not equals
abs	'__abs'	absolute value
sign	'__sign'	sign
square	'__square'	square (x^2)
cube	'__cube'	cube (x^3)
antilog2	'__antilog2'	exponentiation to base 2, i.e. 2^x
antilog10	'__antilog10'	exponentiation to base 10, i.e. 10^x
sqrt	'__sqrt'	square root ($\sqrt{x}$)
even	'__even'	even number check
odd	'__odd'	odd number check
zero	'__zero'	checks for zero
nonzero	'__nonzero'	checks for non-zero
n/a	'__gc'	garbage collection
n/a	'__tostring'	conversion to a string, e.g. for the pretty printer

11.4.7 Constants

The package features the following constants:

gmp.naught (0), **gmp.nought** (0), **gmp.one** (1), **gmp.two**, **gmp.three**, **gmp.four**, **gmp.five**, **gmp.six**, **gmp.seven**, **gmp.eight**, **gmp.nine**, **gmp.ten**, **gmp.eleven**, **gmp.twelve**, **gmp.fifty**, **gmp.hundred**, **gmp.thousand**.

11.5 mpfr - GNU Multiple Precision Floating-Point Reliable Library

As a *plus* package, the **mpfr** package is not part of the standard distribution and must be activated with the **import** statement, i.e. `import mpfr`.

The **mpfr** library is a binding to the MPFR library providing multiple functions to conduct floating-point arithmetic of arbitrary precision, both in the real and complex domain.

The package provides various metamethods for easy entry of calculations.

MPFR floats are represented by **mpfr** userdata objects which can be passed to the functions and operators described below. You can also pass Agena numbers to all package functions, but, of course, not to the operators.

Solaris, Linux, Mac OS X and UNIX users may have to install the original GMP 6.3 and MPFR 4.2 libraries separately in order for this binding to work. In order for this binding to work on Intel CPUs, you may need at least a Sandybridge processor.

Examples:

```
> import mpfr;

> a, b := mpfr.new(2), mpfr.new(3)  # define 2.0 & 3.0 as two MPFR floats

> a * b:                  # conduct multiplication
mpfr(6.000000000000000000000000000000000000000)

> sin(mpfr.Pi):            # compute sine of Pi
mpfr(1.88304107766078511674590954845603494027 3e-39)

> mpfr.precision(32);      # reduce precision from 128 to 32

> sin(mpfr.Pi):
mpfr(1.8830410775e-39)

> mpfr.log2(mpfr.new(8)):  # compute base-2 logarithm of 8
mpfr(3.0000000000)

> mpfr.log2(8):            # the same
mpfr(3.0000000000)
```

You can create and use complex MPFR numbers like this:

```
> import mpfr;

> x := mpfr.new(1, 2);
```

or, likewise:

```
> x := mpfr.new(mpfr.new(1), mpfr.new(2)):
cmpfr(1.000000000000000000000000000000000000000,
      2.000000000000000000000000000000000000000)
```

```
> sqrt(x):
cmpfr(1.27201964951406896425242246173749149 1716,
      .78615137775742328606955858584295892 95239)
```

You might be interested in the **unimath** package for more functions, check Chapter 11.22.

Following are all available operators, mathematical and general functions. All the operators work with both real and complex MPFR objects, and many functions do so, too.

Operator	Functionality	Operator	Functionality
+	addition	exp	exponential function to the base E = 2.71828..
-	subtraction	sin	sine
*	multiplication	cos	cosine
/	division	tan	tangent
^	exponentiation	sinh	hyperbolic sine
%	modulo	cosh	hyperbolic cosine
(-)	unary minus	tanh	hyperbolic tangent
abs	absolute value	arcsin	inverse sine
sign	sign	arccos	inverse cosine
sqrt	square root	arctan	inverse tangent
ln	natural logarithm	arcsec	inverse secant
square	square, x^2	squareadd	square x & add c
cube	cube, x^3	recip	reciprocal 1/x
antilog2	2^x	antilog10	10^x
a |- b	|a - b|	invsqrt	$1/\sqrt{x}$

Operator	Functionality
nonzero	test for a non-zero
zero	test for zero
even	test for an even integer
odd	test for an odd integer
integral	test for an integral number
fractional	test for a fractional number
nan	test for NaN
finite	test for finite numbers
infinite	test for infinity
=	equality check
~=	approximate equality check
<>	inequality check
~<>	approximate inequality check
<	less-than relation
<=	less-than-or-equal relation
>	greater-than relation
>=	greater-than or equal relation

See also **mpfr.cmpd** to compare an MPFR value with a number. See also the MAPM and dd packages, Chapters 11.3 and 11.23.

Function	Arguments	Functionality
mpfr.agm	2 MPFR values	arithmetic-geometric mean
mpfr.ai	1 MPFR value	Airy function
mpfr.arccosh	1 MPFR value	inverse hyperbolic cosine
mpfr.arccot	1 MPFR value	inverse cotangent
mpfr.arccoth	1 MPFR value	inverse hyperbolic cotangent
mpfr.arccsc	1 MPFR value	inverse cosecant
mpfr.arccsch	1 MPFR value	inverse hyperbolic cosecant
mpfr.arcsech	1 MPFR value	inverse hyperbolic secant
mpfr.arcsinh	1 MPFR value	inverse hyperbolic sine
mpfr.arctanh	1 MPFR value	inverse hyperbolic tangent
mpfr.arctan2	2 MPFR values	inverse tangent
mpfr.argument	1 MPFR value	phase angle (principal argument)
mpfr.beta	2 MPFR values	Beta function
mpfr.cbrt	1 MPFR value	cubic root
mpfr.ceil	1 MPFR value	rounds up to the next higher or equal representable integer
mpfr.copysign	2 MPFR values	like **math.copysign**
mpfr.cot	1 MPFR value	cotangent
mpfr.coth	1 MPFR value	hyperbolic cotangent
mpfr.csc	1 MPFR value	cosecant
mpfr.csch	1 MPFR value	hyperbolic cosecant
mpfr.digamma	1 MPFR value	Digamma function
mpfr.dim	2 MPFR values a, b	returns a-b if a > b, 0 if a <= b, or undefined of a or b is undefined
mpfr.div2exp	2 MPFR values m, e	returns $m/2^e$
mpfr.eint	1 MPFR value	exponential integral
mpfr.erf	1 MPFR value	error function
mpfr.erfc	1 MPFR value	complementary error function
mpfr.exp10	1 MPFR value	exponential to base 10
mpfr.exp2	1 MPFR value	exponential to base 2
mpfr.floor	1 MPFR value	rounds to the next lower or equal representable integer
mpfr.fma	3 MPFR values	fused multiply-addition
mpfr.fmod	2 MPFR values	see: **fmod**; the result is tied to even
mpfr.fms	3 MPFR values	fused multiply-subtraction
mpfr.fraction	1 MPFR value	returns the fractional part
mpfr.frexp	1 MPFR value x	returns mantissa m and exponent e such that $x = m*2^e$
mpfr.gamma	1 MPFR value	Gamma function
mpfr.hypot	2 MPFR values	hypotenuse
mpfr.cathet	2 MPFR values	cathetus, computes $\sqrt{x^2 - y^2}$
mpfr.iseven	1 MPFR value	test for an even integral number

Function	Arguments	Functionality
mpfr.isfinite	1 MPFR value	check for a finite value, unlike undefined or infinity
mpfr.isfractional	1 MPFR value	test for a fractional number
mpfr.isinfinite	1 MPFR value	check for infinity
mpfr.isintegral	1 MPFR value	test for an integral number
mpfr.isodd	1 MPFR value	test for an odd integral number
mpfr.isundefined	1 MPFR value	check for **undefined**
mpfr.j0	1 MPFR value	first kind Bessel function of order 0
mpfr.j1	1 MPFR value	first kind Bessel function of order 1
mpfr.jn	2 MPFR values plus one integer for the order	first kind Bessel function of order n
mpfr.li2	1 MPFR value	real part of the dilogarithm of its argument
mpfr.lgamma	1 MPFR value	logarithm of the Gamma function
mpfr.log10	1 MPFR value	logarithm to the base 10
mpfr.log2	1 MPFR value	logarithm to the base 2
mpfr.max	2 MPFR values	returns the maximum of two values
mpfr.min	2 MPFR values	returns the minimum of two values
mpfr.minmax	2 MPFR values	returns the minimum and maximum of two values, in this order
mpfr.modf	2 MPFR values	see: **modf**; the result is tied to even
mpfr.mul2exp	2 MPFR values m, e	returns $m*2^e$
mpfr.nexttoward	1 MPFR value	works like `math.nextafter`, but for MPFR values; does not change its argument
mpfr.pytha	2 MPFR values	computes x^2+y^2
mpfr.pytha4	2 MPFR values	computes $x^2 - y^2$
mpfr.random	none	returns a uniformly distributed random float on the interval [0, 1]
mpfr.relerror	2 MPFR values	relative difference $\|y - x\|/x$
mpfr.remquo	2 MPFR values	remainder and quotient of a division, 1:1 interface to the MPFR mpfr_remquo() function; the result is tied to even
mpfr.root	1 MPFR value, 1 integer	n-th root $x^{1/n}$
mpfr.round	1 MPFR value	rounds to nearest representable integer, rounding halfway cases away from zero
mpfr.sec	1 MPFR value	secant
mpfr.sech	1 MPFR value	hyperbolic secant
mpfr.signbit	1 MPFR value	checks the sign bit and returns **true** (value is negative) or **false**
mpfr.trunc	1 MPFR value	rounds to the next representable integer toward zero

Function	Arguments	Functionality
mpfr.y0	1 MPFR value	second kind Bessel function of order 0
mpfr.y	1 MPFR value	second kind Bessel function of order 1
mpfr.yn	2 MPFR values plus one integer for the order	second kind Bessel function of order n
mpfr.zeta	1 MPFR value	Riemann Zeta function
mpfr.Zero	1 signed integer	returns MPFR +0 or -0, depending on the sign of its argument
mpfr.Inf	1 signed integer	returns MPFR +**infinity** or -**infinity**, depending on the sign of its argument
mpfr.Nan	none	returns MPFR **undefined**, as a function call
mpfr.I	n/a	MPFR imaginary unit
mpfr.Ln2	n/a	MPFR constant ln(2)
mpfr.Pi	n/a	MPFR constant Pi
mpfr.E	n/a	MPFR constant $e^1 = 2.7182818284...$
mpfr.Euler	n/a	MPFR constant $\gamma = 0.57721566490...$
mpfr.Catalan	n/a	MPFR constant $\lambda = 0.91596559417...$
mpfr.naught mpfr.nought	n/a	0
mpfr.one	n/a	1
mpfr.two	n/a	2
mpfr.three	n/a	3
mpfr.four	n/a	4
mpfr.five	n/a	5
mpfr.six	n/a	6
mpfr.seven	n/a	7
mpfr.eight	n/a	8
mpfr.nine	n/a	9
mpfr.ten	n/a	10
mpfr.eleven	n/a	11
mpfr.twelve	n/a	12
mpfr.fifty	n/a	50
mpfr.hundred	n/a	100
mpfr.thousand	n/a	one thousand
mpfr.half	n/a	0.5
mpfr.quarter	n/a	0.25
mpfr.tenth	n/a	0.1
mpfr.fifth	n/a	0.2
mpfr.hundredth	n/a	0.01
mpfr.thousandth	n/a	0.001
mpfr.threequarter	n/a	0.75
mpfr.third	n/a	1/3
mpfr.sixth	n/a	1/6
mpfr.eighth	n/a	0.125
mpfr.twelfth	n/a	1/12
mpfr.sixteenth	n/a	0.0625

Function	Arguments	Functionality
mpfr.infinity	n/a	infinity
mpfr.undefined	n/a	undefined

The constants all have 128 bits of precision.

See also: The **long** package implementing 80-bit floating-point arithmetic, described in Chapter 11.15.

General functions:

mpfr.approx (x, y [, eps])

Compares real or complex MPFR numbers `x` and `y` and checks whether they are approximately equal. If `eps` is omitted, **Eps** is used. See **long.approx** for further information.

mpfr.checkcmpf (···)

The function issues an error if at least one of its arguments is not a complex MPFR number. Otherwise the function returns nothing.

See also: **mpfr.iscmpfr**.

mpfr.checkmpfr (···)

The function issues an error if at least one of its arguments is not a real MPFR number. Otherwise the function returns nothing.

See also: **mpfr.ismpfr**.

mpfr.clone (x [, prec])

Clones a real or complex MPFR value and returns it. The rounding mode of the MPFR value returned will be the current one, not necessarily the one with which the value to be duplicated has been created. You can optionally set the precision `prec` in bits of the cloned object. See also: **mpfr.new**.

mpfr.cmp (x, y)

Compares the two real MPFR values `x` and `y` and returns -1 if $x < y$, 0 if $x = y$ and 1 if $x > y$. See also relative operators <, <=, =, etc.

mpfr.cmpd (x, y)

Compares the MPFR value `x` and the number `y` and returns -1 if $x < y$, 0 if $x = y$ and 1 if $x > y$. See also relative operators <, <=, =, etc.

mpfr.getparts (x)

Returns the real and imaginary parts, in this order, in complex MPFR floating-point object `x` as two real MPFR floating-point objects.

mpfr.iscmpfr (···)

The function returns **true** if all its arguments are complex MPFR numbers and **false** otherwise. See also: **mpfr.checkcmpfr**, **mpfr.ismpfr**.

mpfr.ismpfr (···)

The function returns **true** if all its arguments are real MPFR numbers and **false** otherwise. See also: **mpfr.checkmpfr**, **mpfr.iscmpfr**.

mpfr.new (x [, y])

With only `x` given, creates a real MPFR floating-point object from a number `x`, or a string `x` representing a number.

For best accuracy, you should pass strings instead of numbers, as numbers are rounded to the next machine-representable floating-point number before being converted to an MPFR value.

If `x` is already an MPFR value , it is simply returned.

With both `x` and `y` given, which must be either numbers, strings or real MPFR floating-point objects, creates a complex MPFR floating-point object with the real part set to `x` and the imaginary part to `y`.

See also: **mpfr.clone**.

mpfr.polygen (t)

Takes a table of coefficients (Agena numbers, real or complex MPFR numbers) in descending order and returns a polynomial function that can be evaluated at any given real or complex point. Example:

```
> import mpf;
> coeff  := [mpfr.new(5, 1), mpfr.new(3, 1), mpfr.new(1, 1)];
```

So the next statement

```
> f := mpfr.polygen(coeff);
```

is equal to "`f := <: x -> coeff[1]*x**2 + coeff[2]*x + coeff[3] :>`".

```
> f(mpfr.new(1, 0)):
cmpf(9.000.., 3.000..)
```

mpfr.precision ([x])

mpfr.precision (x, prec)

In the first form, gets or sets the overall precision, in bits. If an integer `x` in the range 2 .. 2,147,483,647 is being passed, the function sets the precision for all values subsequently allocated.

If `x` is a real MPFR value, the function returns the precision in bits of `x`.

If no argument is given, the current setting will be returned.

The default precision at invocation of the package is 128.

In the second form, sets the precision of MPFR value `x` to `prec` bits and returns `prec`.

mpfr.rounding ([rmode])

Gets or sets the current rounding mode. If a string `rmode` is passed, the function sets the rounding mode for all values subsequently allocated. Valid settings for `rmode` are the strings:

- `'rndn'`, round to nearest, with ties to even;
- `'rndz'`, round toward zero;
- `'rndu'`, round toward +infinity;
- `'rndd'`, round toward -infinity.

If no argument is given, the current rounding mode will be returned.

The default rounding mode at invocation of the package is `'rndn'`, i.e. rounding to nearest.

mpfr.swap (x, y)

Swaps the data in the real or complex MPFR values `x` and `y` in-place. The function returns nothing.

mpfr.tonumber (x)

Converts the real or complex MPFR value `x` into an Agena number(s).

mpfr.tostring (x)

Converts the real or complex MPFR value `x` into an Agena string or two strings.

11.6 divs - Library to Process Fractions

As a *plus* package, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import divs`.

The library provides basic arithmetic to calculate with fractions. To create a fraction, use **divs.divs** which accepts mixed, improper and proper fractions. The package implements metamethods so that the common addition, subtraction, division, and unary minus operators can be used.

The + operator adds two fractions, or a number and a fraction in any order.

The **-** operator subtracts two fractions, or a number and a fraction in any order.

The ***** operator multiplies two fractions, or a number and a fraction in any order.

The / operator divides two fractions, or a number and a fraction in any order.

The ^ operator exponentiates two fractions, or a number and a fraction in any order.

The ****** operator raises a fraction to an integer power, in this order.

The **abs** operator returns the absolute value of a fraction and returns a fraction.

The **sign** operator returns the sign of a fraction and returns a number.

The **sqrt** operator returns the square root of a fraction and returns a fraction. If the resulting fraction could not be evaluated with absolute precision, it returns a number.

The **ln** operator returns the natural logarithm of a fraction and returns a fraction. If the resulting fraction could not be evaluated with absolute precision, it returns a number.

The **exp** operator returns the value of **E** to the power of the given fraction and returns a fraction. If the resulting fraction could not be evaluated with absolute precision, it returns a number.

The **sin** operator returns the sine of a fraction and returns a fraction in radians. If the resulting fraction could not be evaluated with absolute precision, it returns a number (in radians).

The **cos** operator returns the cosine of a fraction and returns a fraction in radians. If the resulting fraction could not be evaluated with absolute precision, it returns a number (in radians).

The **tan** operator returns the tangent of a fraction and returns a fraction in radians. If the resulting fraction could not be evaluated with absolute precision, it returns a number (in radians). It returns **undefined** if poles have been encountered.

The **arctan** operator returns the arcus tangent of a fraction and returns a fraction in radians. If the resulting fraction could not be evaluated with absolute precision, it returns a number (in radians). It returns **undefined** if poles have been encountered.

The **int** operator returns the integer quotient of the numerator of a fraction divided by its denominator.

The numerators and denominators should all be integers.

The return always is an improper fraction. There are also two functions to convert fractions to decimals and vice versa.

Examples:

```
> import divs;

> divs.divs(1, 2, 3) + divs.divs(1, 3):
2

> divs.divs(1, 2) * divs.divs(1, 3):
divs(5, 6)

> divs.divs(1, 2) * divs.divs(1, 3):
divs(1, 6)

> 2 * divs.divs(1, 3):
divs(2, 3)

> divs.todec(divs.divs(1, 2)):
0.5

> divs.todiv(ans):
div(1, 2)       0
```

Relations: Two fractions can be compared with the <, <=, =, ==, ~=, >=, and > operators.

The following operators are also supported: **arcsin**, **arccos**, **arcsec**, **sinh**, **cosh**, **tanh**, **recip**, and ~<>.

Functions:

divs.denom (a)

This function returns the denominator of the fraction `a` of the user-defined type `'divs'` and returns it as a number.

The function is written in Agena and is included in the lib/divs.agn file.

See also: **divs.numer**.

divs.divs ([x,] y, z)

divs.divs ([x:]y:z)

This function defines a fraction and returns it as a value of the user-defined type `'div'` if `z` is not 1, with proper metamethods added. It returns a number if `z` equals 1, and **undefined** if `z` is 0.

In the first form: if all three arguments are given, representing a mixed fraction $x\ \frac{y}{z}$, the function converts it into an improper fraction and returns it. If only `y` and `z` are given, the function returns a reduced improper or proper fraction $\frac{x}{y}$.

The second form allows to pass `x`, `y`, and `z` as a nested pair `x:y:z`, representing a mixed fraction, or the pair `y:z` representing an improper or proper fraction.

In both forms, `x`, `y`, and `z` should be integers.

The function is written in Agena and is included in the lib/divs.agn file.

divs.equals (a, b [, option])

This function checks two fractions `a`, `b` for equality. Alternatively, either `a` or `b` may be simple Agena numbers. The result is either **true** or **false**. If any non-**null** `option` is given, the function checks for approximate equality (see **approx** function). Note that the equality operators =, ==, and ~= cannot check values of different types.

The function is written in Agena and is included in the lib/divs.agn file.

divs.numer (a)

This function returns the numerator of the fraction `a` of the user-defined type `'divs'` and returns it as a number.

The function is written in Agena and is included in the lib/divs.agn file.

See also: **divs.denom**.

divs.todec (a)

This function converts a fraction `a` of the user-defined type `'divs'` to a float and returns it.

The function is written in Agena and is included in the lib/divs.agn file.

See also: **divs.todiv**.

divs.todiv (x)

This function converts a number `x` to an improper fraction of the user-defined type `'divs'` and returns it. The second return is the accuracy (see **math.fraction** for further information).

The function is written in Agena and is included in the lib/divs.agn file.

See also: **divs.todec**, **math.fraction**.

11.7 dual - Dual Numbers

This library provides basic support for dual numbers which are related to complex numbers, but instead of an imaginary unit i with $i^2 = -1$, we have a nilpotent ε unit with $\varepsilon^2 = 0$ and $e \neq 0$.

The package fully supports hyper-dual numbers and third-order dual numbers, too. Dual numbers, hyper-dual numbers and third-order dual numbers have the user-defined type `dual`.

The package provides basic arithmetic operations and transcendent functions.

To define a dual number, e.g. $1+2\varepsilon$, type:

```
> a := dual.new(1, 2)

> a:
1+2e
```

Add a to $3+4\varepsilon$:

```
> b := dual.new(3, 4)

> c := a + b:
4+6e
```

To get the real and the dual part, use classic indexing:

```
> c[1], c[2]:
4       6
```

Dual numbers are used for automatic differentiation. Just pass the dual number $x+1\varepsilon$ to a function or operator supported by this package (see table below) to compute $f(x) + f'(x)\varepsilon$ with a single call.

```
> x := dual.new(2, 1)

> sin(x):
0.90929742682568-0.41614683654714e

> sin(2), cos(2):
0.90929742682568        -0.41614683654714
```

A more elaborate function with automatic application of the Chain Rule: The following differentiates $f(x) = \sin x^2$ at point $x = 2$:

```
> f', x := <: x -> sin(x^2) :>, dual.new(2, 1);

> f'(x)[2]:  # actual result: -2.61457448345444765856
-2.6145744834544478
```

The results are way better than **calc.differ**, **calc.eulerdiff** can produce, with a gain in accuracy between four and seven digits, depending on the `complexity` of the function to differentiate.

Hyper-dual numbers not only allow to compute the function value and the first derivative, but also the second derivative in only one call, fully utilizing Automatic Differentiation. This is the function to differentiate ...

```
> f := <: x -> sin(cos(tan(x))) :>
```

... at point 0.5. To compute the derivatives, always set the second and third argument of **dual.new** to 1 and the fourth argument to 0, or just omit the latter:

```
> x := dual.new(0.5, 1, 1, 0)
```

or

```
> x := dual.new(0.5, 1, 1)

> res := f(x):

0.75421075603261-0.44295355389481e1-0.44295355389481e2-1.7731140332202e1e2
```

So the function value is in res[1], the first derivative is in res[2], and the second derivative in res[4].

```
> res[1], res[2], res[4]:
0.75421075603261        -0.44295355389481       -1.7731140332202
```

For comparison:

```
> f(0.5), calc.differ(f, 0.5, deriv = 1), calc.differ(f, 0.5, deriv = 2):
0.75421075603261        -0.44295355389481       4.2877493006852 [faulty !]
```

Maple V Release 4 gives .754210756032612, -.442953553894815, -1.77311403322025.

Third-order dual numbers allow to compute the function value plus the first three derivatives:

```
> x := dual.new(2, 1, 1, 0, 1, 0, 0, 0)

> res := sqrt(x):
dual(1.414213562373, 0.3535533905933, 0.35355339059327, -0.08838834764832,
0.3535533905933, -0.08838834764832, -0.08838834764832, 0.06629126073624)
```

The function value is in res[1], the first derivative is in res[2], the second derivative in res[4], and the third derivative in res[8].

The function **dual.diff** allows to easily compute function values and the derivatives: It takes a univariate function f and a point x, represented by a number, and by default returns the first derivative of f at x.

The `deriv` option controls what to return:

- `deriv = 0`: the function value f(x),
- `deriv = 1`: the first derivative of f at x, that is f'(x) (the default),
- `deriv = 2`: the second derivative of f at x, that is f''(x),
- `deriv = 3`: the third derivative of f at x, that is f'''(x).

If the option `all = true` is given, then the function returns f(x), f'(x), f''(x) and f'''(x) as four separate values, in this order.

You may set environ.kernel(digits = 17) to print as many fractional digits as possible, but you cannot change the precision used in the computation as this setting only controls the pretty-printer. Examples:

```
> environ.kernel(digits = 17)

> f := <: x -> dual.log2(x) :>

> dual.diff(f, 4):
0.36067376022224085

> f := <: x -> sin(cos(tan(x))) :>

> dual.diff(f, 0.5, deriv = 0):
0.75421075603261

> dual.diff(f, 0.5, all = true):
0.75421075603261  -0.44295355389481  -1.7731140332202  -7.155081684592
```

You can also use **calc.differ** which chooses the best differentiation method, internally calling **dual.diff** with univariate functions. If this internal call should fail, **calc.differ** automatically will revert to one of the other numerical methods available in the **calc** package:

```
> calc.differ(f, 0.5, deriv = 3):
-7.155081684592
```

If you want to use a constant in your function, use **dual.const**. Example:

```
> dual.recipsqrt := <: x :: dual -> dual.const(1)/sqrt(x) :>
```

Have a look at the end of this subchapter for a list of all available constants, such as **dual.one** used in this alternative function definition:

```
> dual.recipsqrt := <: x :: dual -> dual.one/sqrt(x) :>
```

However, binary **dual** operators and **dual** package functions that accept two arguments allow for a mix of a (hyper-/third-order) dual number and an ordinary Agena number, so the above function could also be implemented like this:

```
> dual.recipsqrt := <: x :: dual -> 1/sqrt(x) :>
```

Finally, **dual.generate** allows you to easily create dual-number procedures by simply passing a real-valued (!) function and its real-valued first three derivatives (for generator functions in one real) or nine partial derivatives (for functions in two real), utilizing the whole realm of operators and functions that you find in the base, **calc**, **stats** and other libraries.

Example for a function in one real:

```
dual.recipsqrt :=                       # implements 1/sqrt(x)
   dual.generate(
      <: x -> 1/sqrt(x) :>,             # the function
      <: x -> -0.5/(x*sqrt(x)) :>,      # first derivative
      <: x -> 0.75/(x**2*sqrt(x)) :>,   # second derivative
      <: x -> -1.875/x^3.5 :>,          # third derivative
    'dual.recipsqrt')                   # for error message

dual.diff(dual.recipsqrt, 2, all = true):
0.70710678118655  -0.17677669529664  0.13258252147248  -0.1657281518406
```

Example for a function in two reals:

```
dual.arctan2 := dual.generate(
   <: x, y -> arctan2(x, y) :>,       # f
   <: x, y -> -x/y^2/(1+x^2/y^2) :>, # fx
   <: x, y -> 1/y/(1+x^2/y^2) :>,     # fy
   <: x, y -> 2*x/y^3/(1+x^2/y^2)-2*x^3/y^5/(1+x^2/y^2)^2 :>,    # fxx
   <: x, y -> -2/y^3/(1+x^2/y^2)^2*x :>,                         # fyy
   <: x, y -> -1/y^2/(1+x^2/y^2)+2*x^2/y^4/(1+x^2/y^2)^2 :>,     # fxy
   <: x, y -> -6*x/y^4/(1+x^2/y^2) +                             # fxxx
              14*x^3/y^6/(1+x^2/y^2)^2 -
               8*x^5/y^8/(1+x^2/y^2)^3 :>,
   <: x, y -> 8/y^5/(1+x^2/y^2)^3*x^2 - 2/y^3/(1+x^2/y^2)^2 :>, # fyyy
   <: x, y -> 2/y^3/(1+x^2/y^2) -                                # fxxy
              10*x^2/y^5/(1+x^2/y^2)^2 +
               8*x^4/y^7/(1+x^2/y^2)^3 :>,
  <: x, y -> 6/y^4/(1+x^2/y^2)^2*x-8*x^3/y^6/(1+x^2/y^2)^3 :>,  # fxyy
  'dual.arctan2');

dual.diff(dual.arctan2, 2, 1.5, deriv=3):
-0.14950400000000019
```

For the order:

No.	Symbol	Description	Interaction Category
1	f	Primary function value	Real Part (slot 1)
2	fx	First partial w.r.t. x	Gradient (slots 2, 3, 5)
3	fy	First partial w.r.t. y	Gradient (slots 2, 3, 5)
4	fxx	Second pure partial w.r.t. x	Hessian / 2nd Order (slots 4, 6, 7)
5	fyy	Second pure partial w.r.t. y	Hessian / 2nd Order (slots 4, 6, 7)
6	fxy	Second mixed partial	Hessian / 2nd Order (slots 4, 6, 7)
7	fxxx	Third pure partial w.r.t. x	3rd Order Interaction (slot 8)
8	fyyy	Third pure partial w.r.t. y	3rd Order Interaction (slot 8)
9	fxxy	Mixed: $\partial x2\partial y\partial 3f$	3rd Order Interaction (slot 8)
10	fxyy	Mixed: $\partial x\partial y2\partial 3f$	3rd Order Interaction (slot 8)

The following lists all available operators and functions and the results, with two dual numbers $p = a + b\varepsilon$ and $q = c + d\varepsilon$:

Operation	**Call**	**Real Part**	**Dual Part**
Unary minus	-p	-a	-b
Addition	p + q	a + c	b + d
Subtraction	p - q	a - c	b - d
Multiplication	p * q	a * c	a*d + b*c
Division	p / q	a / c	(b*c - a*d) / c^2
Reciprocal	1 / p	1 / a	-b/(a**2)
Exponentiation	p ^ q	a ^ c	a ^ c*(d*ln a + b*c/a)
Square	square p	a**2	2*a*b
Cube	cube p	a**3	3*b*a^2
Absolute value	abs p	abs a	b*sign(a)
Sign	sign p	1, if a > 0 or a = 0 and b > 0 -1, if a < 0 or a = 0 and b < 0 0, otherwise	n/a
Exponential function	exp p	exp a	b*exp a
Power of two (2^x)	dual. exp2(p)	exp2(a)	b*exp2(a)*ln(2)
Power of ten (10^x)	exp10(p)	exp10(a)	b*exp10(a)*ln(10)
Natural logarithm	ln p	ln a	b/a
Base-2 logarithm	log2(p)	log2(a)	b/a/log(2)
Base-10 logarithm	log10(p)	log10(a)	b/a/log(10)
exp(x)-1	math. expm1(p)	expm1(a)	b*exp(a)
$\exp(x^2)$ & $\exp(-x^2)$	expx2(p, 1) expx2(p, -1)	see source file	see source file
ln(x+1)	math. lnp1(p)	lnp1(a)	b/(1 + a)
Logarithmic Gamma function	lngamma p	see source file	see source file
Gamma function	gamma(p)	see source file	see source file
Beta function	beta (p, q)	see source file	see source file
Psi/Digamma	calc.psi(p)	see source file	see source file
Square root	sqrt p	sqrt a	0.5*b/sqrt a
Inverse square root	invsqrt p	see source file	see source file
Cubic root	cbrt(p)	cbrt(a)	b/(3*cbrt(a)^2)
$x^{1.5}$	math. pow32(p)	see source file	see source file
$x^{2.5}$	math. pow52(p)	see source file	see source file
$x^{3.5}$	math. pow72(p)	see source file	see source file

Operation	Call	Real Part	Dual Part
$x^{exp(1)} = x^E$	math. powe(p)	see source file	see source file
Hypotenuse	hypot(p, q)	hypot(a, c)	(a*b+c*d)/ hypot(a, c)
sqrt(1+p^2)	hypot2(p)	see source code	see source code
sqrt(1-p^2)	hypot3(p)	see source code	see source code
Cathetus sqrt(p^2-q^2)	cathet(p, q)	see source code	see source code
Inverse hypotenuse	invhypot (p, q)	see source code	see source code
Squared sum $p^2 + q^2$	pytha(p, q)	see source code	see source code
Squared difference $p^2 - q^2$	pytha4(p, q)	see source code	see source code
Sine	sin p	sin a	b*cos a
Cosine	cos p	cos a	-b*sin a
Tangent	tan p	tan a	b/(cos(a^2)
Cosecant	cos(p)	see source code	see source code
Cotangent	cot(p)	see source code	see source code
Secant	sec(p)	see source code	see source code
Inverse sine	arcsin p	arcsin a	b/hypot3(a)
Inverse cosine	arccos p	arccos a	-b/hypot3(a)
Inverse tangent	arctan p	arctan a	b/(1 + a^2)
Inverse cotangent	arccot(a)	sun_atanl(1.0L/ a1)	-b/(a^2 + 1)
Inverse cosecant	arccsc(a)	arcsin(1/a)	-b / (|a|*sqrt(a^2 -1))
Inverse secant	arcsec p	arccos(1.0/a)	b/(|a|*sqrt(a^2 -1))
Inverse tangent	arctan2 (p, q)	arctan2(a, c)	(b*c-a*d) / (a^2+c^2))
Hyperbolic sine	sinh p	sinh a	b*cosh a
Hyperbolic cosine	cosh p	cosh a	b*sinh a
Hyperbolic tangent	tanh p	tanh a	b*sech(a^2)
Hyperbolic cosecant	cosh(p)	see source file	see source file
Hyperbolic cotangent	coth(p)	see source file	see source file
Hyperbolic secant	sech(p)	see source file	see source file
Un-normalised cardinal sine	sinc p	see source file	see source file
Un-normalised cardinal cosine	cosc(p)	see source file	see source file
Un-normalised cardinal tangent	tanc(p)	see source file	see source file
Casine sin(p)+cos(p)	cas(p)	see source file	see source file

Operation	Call	Real Part	Dual Part
Inverse hyperbolic sine	arcsinh(p)	arcsinh(a)	b*(1/sqrt(a^2 + 1))
Inverse hyperbolic cosine	arccosh(p)	arccosh(a)	b*(1/sqrt(a^2 - 1))
Inverse hyperbolic tangent	arctanh(p)	arctanh(a)	b*(1/(1 - a^2))
Inverse hyperbolic cosecant	arccsch(p)	arccsch(a)	-b/(|a|*sqrt(1 + a^2))
Inverse hyperbolic cotangent	arccoth(p)	arccoth(a)	b/(-a^2 + 1)
Inverse hyperbolic secant	arcsech(p)	arcsech(a)	-b/(a*sqrt(1 - a^2))
Error function	erf(p)	erf(a)	b*2/sqrt(Pi) * exp(-a^2)
Complementary error function	erfc(p)	erfc(a)	b*-2/sqrt(Pi)*exp(-a^2)
Scaled complementary error function	erfcx(p)	erfcx(a)	2*a * expx2(a) * erfc(a) - 2/sqrt(Pi)
Inverse error function	inverf(p)	see source code	see source code
Inverse complementary error function	inverfc(p)	see source code	see source code
Conversion to string	dual. tostring(p)	n/a	n/a
Conversion to table	dual. totable(p)	n/a	n/a
dual number test	dual. isdual(p,..)	n/a	n/a
hyper-dual number test	dual. ishyper(p, ..)	n/a	n/a
third-order-dual number test	iseight(p, ..)	n/a	n/a
Equality	p = q	a = c and b = d	n/a
Inequality	p <> q	not(a=b)	n/a
Approximate equality	p ~= q	a ~= c and b ~= d	n/a
Approximate inequality	p ~<> q	not(p~=q)	n/a
Relation <	p < q	a < c	n/a
Relation <=	p <= q	a <= c	n/a
Relation >	p > q	a > c	n/a
Relation >=	p >= q	a >= c	n/a

A note on the functions mentioned before: These arithmetic standard functions accept real numbers, complex numbers and dual numbers out-of-the-box.

All the functions listed above, when given a dual number, have been prepared to call alternative implementations written in the Agena language instead of the C language.

If you want to override the dual-number C version for **gamma**, for example, just define a function named **dual.gamma**; to replace the C implementation of **arccsc** just write a function called **dual.arccsc**, and so on. Furthermore, you can replace the underlying metamethods for the operators by injecting your own, see the lib/library.agn file, dual section, on how to do this.

The following constants are available:

dual.naught and **dual.nought** for zero, **dual.one** for one, **dual.two** for two, **dual.three**, **dual.four**, **dual.five**, **dual.six**, **dual.seven**, **dual.eight**, **dual.nine**, **dual.ten**, **dual.eleven**, **dual.twelve**.

Furthermore we have: **dual.half** for 0.5, **dual.quarter** for 0.25, **dual.tenth** for 0.1, **dual.fifth** for 0.2, **dual.hundredth** for 0.01, **dual.thousandth** for 0.001.

dual.threequarter for 0.75, **dual.third** for 1/3, **dual.sixth** for 1/6, **dual.eighth** for 0.125, **dual.twelfth** for 1/12 and **dual.sixteenth** for 0.0625.

To define your own constants, pass an Agena number to **dual.const** or pass it as the first argument to **dual.new**, with the second, third and fourth argument all zero:

```
> dual.twenty := dual.const(20)
```

Likewise:

```
> dual.twenty := dual.new(20, 0, 0, 0)
```

11.8 clock - Clock Package

This package contains mathematical routines to perform basic operations on time values, i.e. hours, minutes, and seconds, or sexagesimal values in general.

As a *plus* package, it is not part of the standard distribution and must be activated with the **import** statement, e.g. `import clock`.

A time value is always defined by the **clock.tm** constructor. You may apply the ordinary `+`, `-`, `*` and / operators in order to add, subtract, multiply or divide values. The relations <, <=, =, >=, and > are also supported.

The **abs** operator determines the absolute value of a sexagesimal and returns a sexagesimal.

The **sign** operator returns the sign of a sexagesimal and returns a number.

The following operators can be used for sexagesimal arithmetic - but please beware of round-off errors, for they convert a sexagesimal argument to decimal, apply the operator, and convert the result back to sexagesimal.

The ^ operator exponentiates sexagesimals, or sexagesimals and numbers, and returns a sexagesimal.

The **sqrt** operator returns the square root of a sexagesimal and returns a sexagesimal. If the sexagesimal is negative, it returns **undefined**.

The **ln** operator returns the natural logarithm of a sexagesimal and returns a sexagesimal. If the sexagesimal is non-negative, it returns **undefined**.

The **exp** operator returns the value of **E** to the power of the given sexagesimal and returns a sexagesimal.

The **sin** operator returns the sine of a sexagesimal and returns a sexagesimal, in radians.

The **cos** operator returns the cosine of a sexagesimal and returns a sexagesimal, in radians.

The **tan** operator returns the tangent of a sexagesimal and returns a sexagesimal, in radians. It returns **undefined** if poles have been encountered.

The **arcsin**, **arctan** and **arctan** operators return the arcus sine, arcus cosine and arcus tangent of a sexagesimal and return a sexagesimal, in radians. With poles and other undefined values, they return simply **undefined**.

By default, all time values are properly adjusted to a normalised representation if the value of the environment variable **_clockAdjust** is not changed. If it

_clockAdjust is set to a value different from **true**, then this normalisation is switched off.

Almost all functions are implemented in Agena and included in the lib/clock.agn file.

A typical example might look like this:

```
> import clock alias
```

Subtract 10 hours and fifteen minutes from 20 hours and 15 minutes:

```
> tm(20, 15, 0) - tm(10, 15, 0):
tm(10, 0, 0)
```

61 seconds are automatically converted to 1 minute and 1 second:

```
> tm(0, 61):
tm(0, 1, 1)
```

Turn off normalisation:

```
> _clockAdjust := null
```

```
> tm(0, 61):
tm(0, 0, 61)
```

Turn on normalisation again:

```
> _clockAdjust := true
```

Alternatively to the clock package, you can use the following functions: **math.dd**, **math.dms**, **math.splitdms**, **math.todecimal** and **math.tosgesim**.

The available **clock** package functions are:

clock.add (t1, t2 [, ···])

The function adds two or more values of type **tm**. The return is a value of type **tm**.

clock.sub (t1, t2 [, ···])

The function subtracts two or more values of type **tm**. The return is a value of type **tm**.

clock.adjust (t [, true])

clock.adjust (h, m, s [, true])

In the first form, the function adjusts the representation of tm values in a time object `t` by applying the rules described in the description of **clock.tm**. The function returns nothing.

In the second form, the function takes Agena numbers `h`, `m` and `s` and returns the normalised values plus the fraction of the day.

Both ways, if the last argument is the Boolean **true**, then the function will wrap hours into a 24-hour cycle. Examples:

```
> clock.adjust(24, 12, 61):
24      13      1       1.0090393518519

> clock.adjust(24, 12, 61, true):
0       13      1       0.0090393518518519
```

See also: **astro.tf2d**.

clock.sgstr (x [, d])

Converts a float or `tm` value `x` into its sexagesimal string representation of the format hh:mm:ss. The colon to separate hours, minutes, and seconds can be changed by passing another optional delimiter `d` of type string.

See also: **clock.totm**.

clock.tm (min)

clock.tm (min, sec)

clock.tm (hrs, min, sec)

This function is used to define time values, where `hrs`, `min`, `sec` are numbers.

In the first form, minutes are defined. The return is a value of type tm of the form tm(0, `min`, 0).

In the second form, both minutes and seconds are defined. The return is a value of type tm of the form tm(0, `min`, `sec`).

In the third form, both hours, minutes, and seconds are defined and returned as a value of type tm of the form tm(`hrs`, `min`, `sec`). `hrs` may be set to 0 or be negative.

By default, if `min` > 59 and / or if `sec` > 59, proper adjustments are made before the time value will be returned. If `min` > 59 the call to **time** returns tm(`hrs` + 1, `min` - 60, `sec`). If `sec` > 59 the call to time returns tm(`hrs`, `min` + 1, `sec` - 60). The default is set by the global variable `_clockAdjust` which is assigned **true** at initialisation of the

package if it has not already been set **false** before the clock package has been loaded.

If `_clockAdjust` is set **false** then no adjustments are being made to the arguments. You can call **clock.adjust** to apply the adjustments described above manually.

clock.todec (t)

Converts a tm value `t` into its decimal representation of type number. Example:

```
> t := clock.tm(1, 30, 0)

> clock.todec(t):
1.5
```

See also: **clock.totm**, **math.todecimal**.

clock.totm (t)

Converts decimal time value `t` (of type number) into its tm representation, of type tm. Example:

```
> clock.totm(1.5):
tm(1, 30, 0)
```

See also: **clock.todec**.

11.9 astro - Astronomy Functions

As a *plus* package, the **astro** package is not part of the standard distribution and must be activated with the **import** statement, e.g. `import astro`.

The package functions are:

astro.a2af (angle [, prec])

Decomposes radians into degrees, arcminutes, arcseconds and fraction. `angle`, an Agena number, represents the angle in radians, and `prec`, an integer, the precision which defaults to -5:

```
prec        resolution
   :      ...0000 00 00
  -7         1000 00 00
  -6          100 00 00
  -5           10 00 00
  -4            1 00 00
  -3            0 10 00
  -2            0 01 00
  -1            0 00 10
   0            0 00 01
   1            0 00 00.1
   2            0 00 00.01
   3            0 00 00.001
   :            0 00 00.000...
```

The largest positive useful value for `prec` is determined by the size of angle, and is typically 9. The fifth result is the sign, a 1-character string. The function provides a 1:1 interface to SOFA's iauA2af() function.

See also **astro.a2tf**, **astro.d2tf**, **astro.dmstodec**, **astro.dectodms**, **astro.tf2d**.

astro.a2tf (rad [, prec])

Decomposes radians into hours, minutes, seconds and fraction. `rad`, an Agena number, represents the value in radians, and `prec`, an integer, the precision which defaults to -5, see **astro.a2af** for the meaning. The fifth result is the sign, a 1-character string. The function provides a 1:1 interface to SOFA's iauA2tf() function.

See also **astro.a2af**, **astro.d2tf**, **astro.dmstodec**, **astro.dectodms**, **astro.tf2d**.

astro.autumn ([year])

The function approximates the Julian Date of the Autumn Equinox for the given Gregorian `year`, an integer. If not given, the current system time year is taken.

See also: **astro.moon**, **astro.seasons**, **astro.spring**, **astro.summer**, **astro.sun**, **astro.winter**.

astro.cal2jd (year, month, day)

astro.cal2jd (obj)

Converts a Gregorian Calendar date given in integral `year`, `month`, `day` to Julian Date and returns four values:

1. the Modified Julian Date (MJD) zero-point, always the number 2400000.5,
2. the MJD for midnight, a number,
3. a numerical status code: 0 for okay, 1 for bad year, 2 for bad month, 3 for bad day, else any other integer,
4. the corresponding status text: 'okay', 'bad year', 'bad month', 'bad day' or 'unknown exception'.

In case of invalid input, the first two results are both **undefined**.

You can alternatively pass year, month and day, in this order, in a table `obj`.

To get the Julian Date for midnight, sum up the first and second result.

The function provides a 1:1 interface to SOFA's iauCal2jd() function.

See also: **astro.tf2d**.

astro.cdate ([jd [, option]])

Converts a Julian Date, represented by the float `jd`, into its Gregorian calendar date representation, returning seven values in the following order: the year, month, day, the decimal fraction of the day - in the range [0, 1) -, the hour, minute, and second.

If any `option` is given, the result is returned in a table without the decimal fraction of the day, plus the fraction as a separate, second result.

If no argument is given, the function returns the current system date and time in UTC. This is equivalent to **astro.cdate**(**astro.jdate**()).

See also: **os.date**, **astro.jdate**, **os.isdst**.

astro.coords (alt, azi, lat, lon, y, m, d, hh, mm, ss [, +1])

astro.coords (ra, dec, lat, lon, y, m, d, hh, mm, ss, -1)

Converts between Local Horizontal Coordinates, altitude `alt` and azimuth `azi` to Equatorial (right ascension/declination) and vice versa for the given location represented by the latitude `lat` and longitude `lon`, year `y`, month `m`, day `d`, hour `hh`, minute `mm`, second ss. Right ascension is expressed by parameter `ra` and declination by parameter `dec`.

Coordinate and location input and the output is in degrees.

The result is a table with fields 'ra' and 'declination' in the first form and fields 'altitude' and 'azimuth' in the second.

astro.cweek (y, m, d)

Returns the calendar week for the given year (`y`), month (`m`), and day (`d`), an integer in the range 1 .. 53, determined according to the ISO 8601 standard, and the corresponding year as the second result. The second result is not necessarily equal to `y`, e.g. the calendar week of January 1, 2016 is calendar week 53 of 2015, and the calendar week of December 31, 2013 is week 1 of 2014.

If the passed date does not exist, the function issues an error.

See also: **astro.lastcweek**, **os.date**.

astro.cweekmonsun (y, cw)

Computes the Gregorian dates of the Monday and Sunday for a given calendar week `cw` and year `y` and returns four numbers: the year, the month, the day, and the fraction of day - in his order. In case of a non-existing calendar week `cw`, the function issues an error.

See also: **astro.cweek**, **astro.shabbat**, **os.date**.

astro.d2tf (days [, prec])

Decomposes days to hours, minutes, seconds, fraction. `days`, an Agena number, represents the day and may be fractional, and `prec`, an integer, the precision which defaults to -5, see **astro.a2af** for the meaning. The fifth result is the sign, a 1-character string. The function provides a 1:1 interface to SOFA's iauD2tf() function.

See also **astro.a2af**, **astro.a2tf**, **astro.tf2d**.

astro.dectodms (x, orientation)

Converts co-ordinates `x` in decimal degrees (a number) to the form degree, minute, second, and their orientation 'N', 'S', 'W', or 'E' (DMS format). You must also specify whether to compute latitude or longitude values, by passing the strings `'lat'` or `'lon'`, respectively, for `orientation`.

The return are three numbers and the orientation, a string.

See also: **astro.dmstodec**.

astro.dmstodec (degree, minute, second, hour, orientation)

Converts co-ordinates in DMS format consisting of `degree`, `minute`, `second`, (all numbers) and their `orientation` 'N', 'S', 'W', or 'E' (a single-character string) to their corresponding decimal degree representation (DegDec format). The return is a number.

See also: **astro.dectodms**.

astro.gethourangle (body, lat, lon, elev, y, m, d, hh, mm, ss)
astro.gethourangle (body, lat, lon, elev, jd)

Finds the hour angle of a body for a given observer and time with the following input:

- `body` may be 'Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune', 'Pluto', 'Sun' or 'Moon'.
- `latitude` in degrees, `longitude` in degrees and `elevation` in meters indicate a location on or near the surface of the Earth where the observer is located.
- The date and time of interest is given by either the Julian Date `jd` or the given year `y`, month `m`, day `d`, hour `hh`, minute `mm`, (fractional) second `ss`.

The hour angle of a celestial body indicates its position in the sky with respect to the Earth's rotation. It depends on the location of the observer on the Earth. The hour angle is 0 when the body's center reaches its highest angle above the horizon in a given day. It increases by 1 unit for every sidereal hour that passes after that point, up to 24 sidereal hours when it reaches the highest point again. So the hour angle indicates the number of hours that have passed since the most recent time that the body has culminated, or reached its highest point.

If successful, the function will return the hour angle in the half-open range [0, 24). Multiply the result by 15 to get degrees (360° / 24 hours = 15° per hour). In case of an error the function returns **fail**.

Example for Chattanooga, TN, for August 13, 2026, noon local time:

```
> import astro;

> lat, lon, elev := 35.0456, -85.3097, 206;

> astro.gethourangle('Sun', lat, lon, elev, 2026, 8, 13, 17, 46, 6):
0.00035289125981042
```

The function provides a 1:1 interface to function Astronomy_HourAngle() of the C library `Astronomy Engine for C/C++`. The description has been taken from the accompanying documentation.

See also: **astro.searchhourangle**.

astro.getposition (body, lat, lon, elev, y, m, d, hh, mm, ss)
astro.getposition (body, lat, lon, elev, jd)

Calculates the exact position of a celestial body for a specific observer location and time.

The input parameters are:

- `body` (string): either 'Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune', 'Pluto', 'Sun' or 'Moon'.
- `lat` (number): Geographic latitude of the observer in degrees (positive for North, negative for South).
- `lon` (number): Geographic longitude of the observer in degrees (positive for East, negative for West).
- `elev` (number): Observer's elevation above sea level in meters.
- `y`, `m`, `d` (integer): The date of the observation (year, month, day).
- `hh`, `mm`, `ss` (integer/number): The time of the observation in UTC (hour, minute, second).
- Alternatively: The Julian Date `jd`.

The function returns a coordinate table, a key-value dictionary containing the positional mapping with the following fields:

- `'azimuth'`: The compass heading in degrees, measured clockwise starting from North (0° = North, 90° = East, 180° = South, 270° = West).
- `'altitude'`: The angle in degrees above (+) or below (-) the observer's local horizon. This value automatically compensates for standard atmospheric refraction on Earth.
- `'ra'`: The Right Ascension in sidereal hours (ranging from 0 to 24). This represents the space-fixed longitude on the celestial sphere.
- `'declination'`: The Declination in degrees (ranging from -90° to +90°), representing celestial latitude relative to Earth's projected equator.

In case of an error the function returns **fail**.

Example for Chattanooga, TN, for August 13, 2026, noon local time:

```
> import astro;

> lat, lon, elev := 35.0456, -85.3097, 206;

> astro.getposition('Sun', lat, lon, elev, 2026, 8, 13, 17, 46, 6):
[altitude ~ 69.454648517606, azimuth ~ 179.99942552291,
declination ~ 14.493921704599, ra ~ 9.559571062088]
```

See also: **astro.locate**.

astro.h2jd (year, month, day [, h [, m [, s]]])
astro.h2jd (obj)

In the first form, converts a Hebrew Date, optionally with hours (`h`), minutes (`m`) and seconds (`s`), to a Julian Date. `year`, `month` and `day` represent the Hebrew date components.

In the second form you can pass the Hebrew year, Hebrew month, Hebrew day and optionally the time in table `obj`.

See also: **astro.hdate**.

astro.hdate (year, month, day [, h, m, s] [, true])
astro.hdate (obj [, true])
astro.hdate ([jd [, true]])

In the first form, converts the Gregorian date, optionally with hours (`h`), minutes (`m`) and seconds (`s`), to the corresponding year, month and day in the Jewish calendar, in this order. The fraction of day, the hour, minute and second are also returned. `year`, `month` and `day` represent the Gregorian date components.

In the second form you can pass the Gregorian year, month, day and optionally time in table `obj`.

In the third form, converts the Julian date `jd` to the Jewish calendar.

If **true** is given as the last argument, the result is put into a table.

If no argument is given, the function returns the Jewish calendar date of the current system time, UTC, in a table. For the Jewish months we have:

Number	Name
1	Tishrei
2	Cheshvan
3	Kislev
4	Tevet
5	Shevat
6	Adar (regular) / Adar I (leap)
7	Nisan
8	Iyar
9	Sivan
10	Tammuz
11	Av
12	Elul
13	Adar II (leap years only)

Example for noon August 10, 2026:

```
> astro.hdate([2026, 8, 10, 12, 0, 0], true):
[5786, 12, 27, 0.5, 12, 0, 0]
```

See also: **astro.h2jd**, **astro.jdate**, **astro.pdate**, **astro.shabbat**, **os.date** with the `'*j'` format.

astro.illumination (body [, y, m, d, hh, mm, ss])
astro.illumination (body [, jd])

Calculates the apparent brightness, illuminated surface area percentage, and phase geometry of a celestial body as seen from Earth at a specific moment. For Saturn, it also returns the tilt angle of its ring system.

The function requires a celestial body name string as the first argument, followed by flexible time tracking options:

- body, a string: The name of the target celestial body to evaluate. Supported options include: 'Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune', 'Pluto', 'Sun', and 'Moon'.
- time parameters (optional variables):
- Omitted (1 argument total): Automatically uses the current system date and time (UTC) as the tracking point.
- Julian Date (2 arguments total): A single number `jd` representing the absolute Julian Date to evaluate.
- Calendar Parameters (7 arguments total): Pass separate numeric arguments matching (year y, month m, day d, hour hh, minute mm, (fractional) second ss) to specify a custom UTC timeline frame.

If successful, the function returns a single Lua table containing the following key-value pairs (or **fail** if the calculations fail):

- field `'fraction'`: The percentage of the body's visible disk illuminated by sunlight, expressed as a fraction between 0.0 (0%, totally dark) and 1.0 (100%, fully lit).
- field `'magnitude'`: The apparent visual brightness of the body as seen from Earth on the logarithmic magnitude scale. Lower or more negative numbers indicate brighter objects (e.g., the Sun is -26.71, Venus peaks around -4.8, and dim outer planets have positive values like Pluto at +14.0).
- field `'phaseangle'`: The phase angle in degrees, measured from the center of the target body between the Sun and the Earth. 0° indicates a fully illuminated phase, while 180° indicates a completely dark phase.
- field `'ringtilt'` (Saturn only): The tilt angle of Saturn's rings in degrees relative to Earth. This value dictates how open or closed the rings appear from our perspective, heavily influencing Saturn's overall visual magnitude.

```
> import astro;
```

```
> astro.illumination('Moon', [2026, 8, 28, 4, 19, 6.085461974144]):
[fraction ~ 0.99998342840944, magnitude ~ -12.647142713872,
 phaseangle ~ 0.46648311109478]
```

astro.isdst (year, month, day [, h [, m [, s]]])
astro.isdst (obj)

In the first form, determines whether Daylight Saving Time is (or was, or will be) in effect at the given date represented by the integers `year`, `month`, `day`, hour (`h`), minute (`m`), second (`s`), and returns **true** or **false**.

Alternatively, in the second form, you may pass the input data in the table `obj`.

In case of an invalid date or an internal time conversion failure, the function issues an error.

astro.isleapyear (x [, any])

Returns **true** if the given year `x` (an integer) is a leap year, and **false** otherwise.

If there is any second argument, then the function checks whether the Persian year `x` is a Persian leap year.

astro.isvaliddate (year, month, day [, h [, m [, s]]])
astro.isvaliddate (obj)

In the first form, checks whether a date and optionally time represents a valid calendar date/a valid time and returns **true** or **false**.

The date is given by the integers `year`, `month` `day`, the optional time by integral hour (`h`) and minute (`m`) and the number second (`s`), the latter which can be fractional.

Alternatively, in the second form, you may pass the input data in the table `obj`.

The function correctly checks for leap years in the Julian (pre-1583) and Gregorian (post-1582) calendars. 24:00:00 hours is not considered a valid time.

See also: **utils.isdate**.

astro.isvalidtime (h [, m [, s]])
astro.isvalidtime (obj)

In the first form, checks for a valid time and returns **true** or **false**.

The time is given by integral hour (`h`), minute (`m`) and the number second (`s`), the latter which can be fractional.

24:00:00 hours is not considered a valid time.

.

Alternatively, in the second form, you may pass the input data in the table `obj`.

See also: **utils.isdate**.

astro.jdate ([year, month, day [, hour [, minute [, second [, ms]]]]], opt)
astro.jdate (obj, opt)

In the first form, converts a Gregorian date represented by `year`, `month`, `day` and optionally `hour`, `minute`, and `second` (all numbers) to the corresponding Julian date (J2000.0). `ms` stands for milliseconds.

In the second form, year, month, day, etc. are passed in a table.

The return is a number, or **fail** if the date or time is invalid.

By definition, the base 0 of the Julian date is January 1, 4713 BC, noon GMT.

The defaults for `hour`, `minute`, `second` and `ms` are 0.

If no argument has been given, the function returns the current time as a Julian Date, GMT.

If `full = **true**` is given for *optional* argument `opt`, then the function returns the Julian Date, GMT, plus a 4-byte integral Julian Date (rounded down), a 4-byte millisecond-since-midnight integer, in this order.

See also: **astro.isvaliddate**, **astro.cdate**, **os.date**, **os.isdst**, **os.date** with the `'*j'` format.

astro.lastcweek (y [, flag])

Computes the last calendar week of the year `y`. If `flag` is not given, the function returns either the number 52 or 53, and the given year `y`. If `flag` is given, then calendar week 1 and `y` + 1 will be returned if December 31 of the given year `y` is either a Monday, Tuesday or Wednesday, otherwise it works as if `flag` has not been passed.

See also: **astro.lastcweek**, **os.date**.

astro.locate (place [, country, options])

Searches the location database stored in the `data` subfolder of the Agena installation for the given `place`, a string, and returns the following fields:

- `'location'`: the name of the place found, a string.
- `'country'`: the country, a string.
- `'latitude'`: the latitude, a number.
- `'longitude'`: the longitude, a number.

- `'population'`: the number of inhabitants, a number.
- `'elevation'`: the elevation above sea level in meters.
- `'timezone'`: the time zone, a string.

You can restrict the search with the optional two-character `country` code, a string, see file `data/country.csv`.

The function supports pattern matching, see Chapter 9.1.3, and also the `*` wildcard. When given a pattern or `*` wildcard for `place`, then the function searches the whole location database for matches, otherwise the function will perform a fast binary search.

You can pass the following options:

- finder = *procedure*: a procedure of two variables for comparing the given `place` with a location in the database file, must return a boolean. The default is <: x, y -> strings.tobytes(x) = strings.tobytes(y) :> which at the time of writing is best suited to compare both pure ASCII and multibyte UTF-8 input.
- codepage = *integer*: In OS/2, DOS and Windows this codepage will be used for the lookup instead of the automatically detected codepage.
- codepages = *table*: A table of one or more codepages represented by positive integers. Used in OS/2, Windows and DOS for extended search only. Default is [], that is no special search, but you may explicitly try [850, 852, 857, 860, 775, 1258] for Western and Eastern Europe, Turkish, Portuguese, Baltic, Vietnamese, in this order.

Please note that St. Petersburg, Russian Federation, is indexed as Saint Petersburg whereas St. Petersburg, Florida, is listed as St. Petersburg. This inconsistency stems from the underlying location database. If you encounter similar issues with other cities, try varying the prefix - such as switching between St., Saint, or San, and adding or removing periods and hyphens. Examples:

```
> astro.locate('Chattanooga'):
[[country ~ US, elevation ~ 208, latitude ~ 35.04563,
 location ~ Chattanooga, longitude ~ -85.30968, population ~ 181099,
 timezone ~ America/New_York]]

> astro.locate('Houma'):
[[country ~ US, elevation ~ 5, latitude ~ 29.59577, location ~ Houma,
  longitude ~ -90.71953, population ~ 34287, timezone ~ America/Chicago],
 [country ~ TO, elevation ~ 24, latitude ~ -21.17005, location ~ Houma,
  longitude ~ -175.30213, population ~ 1932, timezone ~ Pacific/Tongatapu]]

> astro.locate('Houma', 'US'):
[[country ~ US, elevation ~ 5, latitude ~ 29.59577, location ~ Houma,
  longitude ~ -90.71953, population ~ 34287, timezone ~ America/Chicago]]
```

```
> astro.locate('Düsseldorf*', 'DE',
>    finder = <: x, y -> x = y :>, codepages = [850, 852]):
[[country ~ DE, elevation ~ 45, latitude ~ 51.22172, location ~ Düsseldorf,
 longitude ~ 6.77616, population ~ 620523, timezone ~ Europe/Bonn],
 [country ~ DE, elevation ~ 43, latitude ~ 51.23812,
  location ~ Düsseldorf-Pempelfort, longitude ~ 6.78678,
  population ~ 28941, timezone ~ Europe/Bonn]]
```

To convert from decimal degrees (a number) to degrees, minutes and seconds, use **astro.dectodms**.

In Windows and OS/2, the function checks codepages 850 (Western Europe, Scandinavia), 852 (Eastern Europe), 857 (Turkey), 860 (Portuguese), 775 (the Baltics) and 1258 (Vietnam) for an entry. If the location you are interested in includes diacritics or ligatures, such as Düsseldorf or Lódz, you may have to change the codepage before calling the function. Issue **os.codepage**(input=852) and **os.codepage**(output=852), for example, prior to calling **astro.locate** to search for polish locations, and change back later to your default codepage which can be queried with **os.codepage**() at the beginning of a session.

The function is written in Agena and included in the `astro.agn` file.

See also: **astro.moon**, **astro.sun**.

astro.lunareclipse ([jd] [, true])

Computes the next lunar eclipse following the given Julian date `jd`, an Agena number. If `jd` is not given, the function takes the current system time to compute the result.

If the last argument is **true** then the lunar eclipse following the next eclipse will be determined.

The result is a table with the following fields:

- `'time'`, the time of the eclipse as a Julian Date,
- `'obscuration'`, the peak fraction of the Moon's apparent disc that is covered by the Earth's umbra,
- `'kind'`, the type of lunar eclipse found, either 'penumbral', 'partial' or 'total',
- `'penumbral'`, the semi-duration of the penumbral phase in minutes,
- `'partial'`, the semi-duration of the partial phase in minutes, or 0.0 if none,
- `'total'`, the semi-duration of the total phase in minutes, or 0.0 if none.

See also: **astro.moonphase**, **astro.solareclipse**.

astro.lunarlibration (y, m, d, hh, mm, ss)
astro.lunarlibration ([jd])

Calculates the lunar libration (apparent physical `wobble`) and sub-Earth tracking coordinates of the Moon for a specific moment.

Because the Moon's orbit is elliptical and tilted, it appears to rock back and forth and nod up and down over the course of a month as seen from Earth. This function computes that exact physical orientation.

The function accepts either zero arguments, a single Julian Date, or six calendar parameters passed as numbers:

- Omitted (0 arguments): Automatically calculates the libration coordinates for the current system date and time (UTC).
- Julian Date (1 argument): A single number `jd` representing the absolute Julian Date to evaluate.
- Calendar Parameters (6 arguments): Pass separate numeric arguments matching (year `y`, month `m`, day `d`, hour `hh`, min `mm`, (fractional) sec `ss`) to specify a custom UTC timeline frame.

The function returns a dictionary representing the Moon's perspective orientation relative to an observer on Earth:

- field `'latitude'`: The selenographic latitude of the point on the Moon's surface directly beneath the Earth, measured in degrees. Positive (+): The Moon is nodding forward, tilting its North Pole terrain toward Earth. Negative (-): The Moon is nodding backward, tilting its South Pole terrain toward Earth.
- field `'longitude'`: The selenographic longitude of the point on the Moon's surface directly beneath the Earth, measured in degrees. Positive (+): The Moon is twisted east, exposing more features along its eastern edge. Negative (-): The Moon is twisted west, exposing more features along its western edge.
- field `'limbangle'`: The position angle of the Moon's illuminated edge (or its apparent angular diameter frame metric), measured in degrees.

astro.moon (year, month, day, hour, lon, lat)

Provides an easy-to-use interface to **astro.moonriseset**. and **astro.moonphase**.

The first four arguments represent the `year`, `month`, `day`, and `hour`, all of type number. Longitudes and latitudes can be given in form of two tables `lon`, `lat` containing degrees (a number), minutes (a number), seconds (a number), and the orientation (the single character 'N', 'S', 'W', or 'E'). Alternatively, you may give decimal degrees (numbers) for `lon` and `lat`.

The return is a table with the indices 'riseset', containing Moonrise and Moonset in `tm` representation, UTC, and the index 'phase' which holds the computed Lunar phase (a float and an integer).

See **astro.moonriseset** and **astro.moonphase** for further information.

The function uses the `tm` time notation of the **clock** package. You do not have to readlib **clock** before.

The function is written in Agena and included in the `astro.agn` file.

Examples for Chattanooga, Tennessee:

```
> astro.moon(2026, 4, 1, 0, [85, 34.848, 0, 'W'], [35, 2, 44.232, 'N']):
[phase ~ [0.987789, 4, 167.20069], riseset ~ [tm(23, 57, 0), tm(11, 3, 0)]]
```

or

```
> astro.moon(2026, 4, 1, 0, -85.30968, 35.04563):
[phase ~ [0.987789, 4, 167.20069], riseset ~ [tm(23, 56, 0), tm(11, 2, 0)]]
```

See also: **astro.autumn**, **astro.locate**, **astro.spring**, **astro.summer**, **astro.sun**, **astro.winter**.

astro.moonphase (year, month, day [, hour])
astro.moonphase (obj)
astro.moonphase ([jd])

In the first form, takes a `year`, a `month`, a `day`, and optionally an `hour` (all numbers) and returns three values in the following order:

- the moon phase, that is the percentage of the Moon's disc that is illuminated from the Earth's point of view, as a real number in the range [0, 1], where 0 is new moon and 1 is full Moon;
- an integer in the range [0, 7], where 0 indicates new moon and 4 indicates full moon;
- the Moon's ecliptic phase angle, in degrees, where 0 indicates new moon, 90 indicates the first quarter, 180 indicates full moon and 270 indicates the third quarter.

If `hour`, taken to be in UTC, is not given, it is set to 0.

Alternatively, in the second form, you may pass the input data in a table. In the third form you pass the Julian date of interest. If no argument is given, the data for the current date and hour (UTC) will be computed.

See also: **astro.lunareclipse**, **astro.moon**.

astro.moonriseset (year, month, day, lon, lat)

Returns the times of Lunar rise and set in GMT. Receives the `year`, `month day`, the longitude and latitude `lon` and `lat` (all of type number) and returns two numbers: the GMT rise time in a decimal, and the GMT set time also in a decimal.

Use **clock.totm** to convert the rise and set times to sexagesimal format, or try **astro.moon**.

Example for Düsseldorf:

```
> astro.moonriseset(2013, 1, 8,
>   astro.dmstodec(6, 46, 58, 'E'), astro.dmstodec(51, 13, 32, 'N')):
3.7666666666667 12.566666666667
```

See also: **astro.locate**, **astro.searchriseset**, **astro.sunriseset**.

astro.p2cd (year, month, day [, h, m, s])
astro.p2cd (obj)

Like **astro.p2jd**, but converts to Gregorian date, UTC. Contrary to **astro.p2jd**, `month` may either be the Persian month number or Persian month name (a string).

```
> import astro;

> astro.p2cd(1405, 7, 18, 21, 8, 9.5):
2026   10   10   0.73483217600733   17   38   9.50000703335

> astro.p2cd(1405, 'Mehr', 18):
2026   10   9    0.85416666651145   20   30   0
```

astro.p2jd (year, month, day [, h, m, s])
astro.p2jd (obj)

In the first form, converts a Persian year, Persian month and Persian day, and optionally hours, minute and second to the corresponding Julian Date (UTC). Hours, minute and second default to zero. The input is considered to be in local Shiraz Time.

In the second form you can pass the Persian year, Persian month, Persian day and time in table `obj`.

The following example converts Persian year 1405 Solar Hijri, Persian month Mordad (the fifth month of the Persian year, representing mid-summer), Persian day 19, noon Shiraz Time, to Gregorian Date UTC:

```
> astro.cdate(astro.p2jd([1405, 5, 19, 12, 0, 0])):
2026    8       10      0.35416666651145        8       30      0
```

See also: **astro.p2cd**.

astro.pdate (year, month, day [, h, m, s])
astro.pdate (obj)
astro.pdate ([jd])

Converts a Julian Date or a Gregorian Date, both considered UTC, to the corresponding Persian Date.

In the first form, converts the Gregorian date, optionally with hours (`h`), minutes (`m`) and seconds (`s`), to a Persian Date. `year`, `month` and `day` represent the Gregorian date components.

In the second form you can pass the Gregorian year, month, day and time in table `obj`.

In the third form, `jd` represents the Julian Date. If no argument is given, the current system Persian year, Persian month and Persian day, plus Shiraz time will be returned.

With all flavours, the function returns seven values: the Persian year, Persian month, and Persian day, followed by the time of day expressed as a decimal fraction, and finally the hours, minutes, and seconds, all calculated in local Shiraz Time.

Example:

```
> astro.pdate([2026, 3, 21, 12, 0, 0]):  # Nowruz
1405    1       1       0.64583333333333        15      30      0
```

See also: **astro.jdate**, **astro.p2cd**, **astro.p2jd**, **os.date** with the `'*j'` format.

astro.peak ([jd])
astro.peak (year, month, day [, h, m, s, ms])
astro.peak (obj)

Searches for the date and time Venus will next appear brightest as seen from the Earth.

In the first form `jd` is the Julian Date, a number. If `jd` is not given, the function takes the current system time to compute the result.

In the second and third form you pass the year, month, day and time (UTC) as individual arguments or you put them all in a `obj`. `ms` represents milliseconds.

The result is a table with the following fields:

- `'time'`: the date and time of the observation, expressed as a Julian Date.
- `'magnitude'`: the visual magnitude of the body. Smaller values are brighter.
- `'phaseangle'`: the angle in degrees between the Sun and the Earth, as seen from the body. Indicates the body's phase as seen from the Earth.
- `'phasefraction'`: a value in the range [0.0, 1.0] indicating what fraction of the body's apparent disc is illuminated, as seen from the Earth.
- `'heliodist'`: the distance between the Sun and the body at the observation time.

The result is an approximation.

Use **astro.calc** to convert Julian Dates to a Gregorian calendar date, of **astro.hdate** for the Jewish calendar, **astro.pdate** for the Persian Calendar.

astro.pmonth (month)

Returns the Farsi name of integer `month`. `month` must be in [1, 12]. Alternatively, `month` may be the Persian month name, a string, such as 'Dey' or 'dey' and the function will return the corresponding month number, an integer in [1, 12].

astro.pweekday (day [, any])

By default returns the name of the given `day` of week, an integer in the range [1, 7]. The Persian week starts on Saturday.

If any second argument is given, the Zoroastrian name is returned for the given `day` of month, an integer in the range [1, 31].

If `day` is a string depicting a Persian weekday (such as 'Shanbeh' or 'shanbeh') or a Zoroastrian month day (such as 'Vahman' or 'vahman'), its index will be returned: 1 to 7, Saturday-indexed - or 1 to 31, respectively.

astro.searchaltitude (body, lat, lon, elev, y, m, d, hh, mm, ss, angle, dir [, limit])
astro.searchaltitude (body, lat, lon, elev, jd, angle, dir [, limit])

Calculates the exact timestamp when the given celestial `body` reaches a specific altitude angle relative to an observer's horizon. This function utilizes high-precision astronomical algorithms to scan forward from a user-defined starting point. It is widely used to compute local geometric visible set/rise parameters, atmospheric twilight boundaries, and religious calendar transitions (such as the start and end of Shabbat, see **astro.shabbat**).
The parameters are:

- `body` (string): either 'Sun', 'Moon', 'Mercury', 'Venus', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune' or 'Pluto', the ninth planet.
- `lat` (number): The geographic latitude of the observer in degrees (positive for North, negative for South).
- `lon` (number): The geographic longitude of the observer in degrees (positive for East, negative for West).
- `elev` (number): The elevation of the observer above sea level in meters.
- `jd` (number): The Julian Date of interest, or alternatively the six date and time components:
- `y` (integer): The full Gregorian starting year for the search window (e.g., 2026).
- `m` (integer): The starting month of the search (1 to 12).
- `d` (integer): The starting day of the search (1 to 31).
- `hh` (integer): The starting hour of the search (0 to 23, UTC).
- `mm` (integer): The starting minute of the search (0 to 59, UTC).
- `ss` (number): The starting second of the search (0.0 to 59.999, UTC).

- `angle` (number): The target altitude angle of the body in degrees relative to the horizon. Official sunrise/sunset: -0.833 (accounting for solar radius and atmospheric refraction), civil twilight: -6.0 degrees, nautical twilight: -12.0 degrees, astronomical twilight: -18.0 degrees. Use negative values (e.g., -8.5) to track positions below the horizon.
- `dir` (integer): The direction, specifies whether to search for an upward or downward transition. Pass a negative integer (e.g., -1) to track a downward/setting crossing. Pass a positive integer (e.g., 1) to track an upward/rising crossing.
- `limit` (number, optional): The maximum forward search window duration expressed in fractional days. Defaults to 1.0 day if omitted.

If the target altitude angle is successfully found within the specified time window, the function returns:

- the Julian Date of the event computed and
- the corresponding six discrete date and time values of the event, in UTC: year (integer), month (integer), day (integer), hour (integer), minute (integer), second (number), all put into a table.

If the Sun does not cross the target altitude within the designated search limit, the function fails and returns **fail**. The results may vary slightly between platforms.

Example: Find when the Sun drops to 8.5 degrees below the horizon in Jerusalem, marking nightfall, on Saturday evening, August 15, 2026, searching from 12:00:00 UTC. Fetch the second return, that is the Gregorian Date, not the Julian Date.

```
> date := [astro.searchaltitude('Sun', 31.7683, 35.2137, 754.0,
>          2026, 8, 15, 12, 0, 0, -8.5, -1)][2]

> printf('Target met on (UTC): %04d-%02d-%02d %02d:%02d:%02d', op(date))
Target met on (UTC): 2026-08-15 17:01:12
```

See also: **astro.locate**.

astro.searchhourangle (body, lat, lon, elev, y, m, d, hh, mm, ss, angle, direction)

astro.searchhourangle (body, lat, lon, elev, jd, angle, direction)

Searches for the time when the center of a body reaches a specified hour angle as seen by an observer on the Earth. The input parameters are:

- `body` may be 'Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune', 'Pluto', 'Sun' or 'Moon'.
- `latitude` in degrees, `longitude` in degrees and `elevation` in meters indicate a location on or near the surface of the Earth where the observer is located.
- The starting date and time is given by either the Julian Date `jd` or the given year `y`, month `m`, day `d`, hour `hh`, minute `mm`, (fractional) second `ss`.
- The hour `angle` of a celestial body indicates its position in the sky with respect to the Earth's rotation. It depends on the location of the observer on the Earth and

is in the range [0, 24). To find when a body culminates, that is reaches the highest point in the sky pass 0 for hour `angle`. To find when a body reaches its lowest point in the sky, pass 12 for `angle`. The hour angle increases by 1 unit for every sidereal hour that passes after that point, up to 24 sidereal hours when it reaches the highest point again. So the hour angle indicates the number of hours that have passed since the most recent time that the body has culminated, or reached its highest point.

- `direction` indicates the direction in time to perform the search: a positive value searches forward in time, a negative value searches backward in time. `direction` should not be zero.

Note that, especially close to the Earth's poles, a body as seen on a given day may always be above the horizon or always below the horizon, so the caller cannot assume that a culminating object is visible nor that an object is below the horizon at its minimum altitude.

On success, the function reports the date and time as a Julian Date, along with the horizontal coordinates of the body at that time, as seen by the given observer, as a dictionary with the following fields:

- 'azimuth': compass direction around the horizon in degrees. 0=North, 90=East, 180=South, 270=West;
- 'altitude': angle in degrees above (positive) or below (negative) the observer's horizon;
- 'ra': right ascension in sidereal hours;
- 'declination': declination in degrees.

Otherwise, the function returns **fail**.

The function provides a 1:1 interface to function Astronomy_SearchHourAngleEx() of the C library `Astronomy Engine for C/C++`. The description has been taken from the accompanying documentation.

For the solar noon, with the starting date August 13, 2026 00:00:00 UTC, at location Chattanooga, TN, enter:

```
> import astro;

> lat, lon, angle, dir := 35.0456, -85.3097, 0.0, 1.0;

> astro.searchhourangle('Sun', lat, lon, 0,
>    2026, 8, 13, 0, 0, 0, angle, dir):
2461266.2403325 [altitude ~ 69.456447100381, azimuth ~ 179.99998731165,
declination ~ 14.502047100381, ra ~ 9.5592033742828]
```

See also: **astro.gethourangle**, **astro.locate**.

astro.searchlunarapsis ([y, m, d, hh, mm, ss])
astro.searchlunarapsis ([jd])

Finds the very next occurring Lunar Apogee (the point in the Moon's orbit furthest from Earth) or Lunar Perigee (the point closest to Earth) relative to a given starting date and time.

The function accepts either zero arguments, six date-time arguments or a Julian Date, all passed as integers/numbers:

- No arguments: Automatically uses the current system date and time (UTC) as the search starting point.
- Six arguments year `y`, month `m`, day `d`, hour `hh`, minute `mm`, (fractional) second `ss` (these arguments can optionally be passed in a table): Specify a custom UTC date and time to begin searching forward from.
- One argument: The Julian Date `jd` where to search from.

If successful, the function returns exactly 9 values:

The calendar year, month, day, hour, minute, second of the event (UTC) plus the kind 'apogee' if the Moon is furthest, or 'perigee' if it is closest, the distance between the centers of the Earth and the Moon in kilometers and the exact Julian Date timestamp of the event.

In case of an internal error or invalid input date, the function returns **fail**.

The results may vary slightly between platforms. Examples:

```
> import astro;
```

Search for the next event from the current date:

```
> L := [astro.searchlunarapsis()]:
[2026, 8, 22, 8, 20, 46.747706830502, apogee, 404632.01214081,
 2461274.8477633]
```

Search for the event after that:

```
> astro.searchlunarapsis(L[9]):
2026    9       6       20      44      8.9950999617577 perigee
368248.59424929 2461290.363993
```

astro.searchmoonphase ([y, m, d, hh, mm, ss [, options]])
astro.searchmoonphase ([jd [, options]])

Calculates the exact time a specific lunar phase occurs. A moon phase is defined by the geocentric ecliptic longitude angle separation between the Sun and the Moon. It allows you to precisely predict milestones such as New Moons, Full Moons, or any custom crescent/gibbous angle threshold.

By default the function searches for the next Full Moon, for the next 40 days, starting from the date given, if any.

You can give the longitude *x* of interest with the `longitude` = *x* option, or `lon` = *x* for short:

- `longitude` = 0 represents the New Moon (the Moon and Sun are aligned at the same longitude).
- `longitude` = 90 represents the First Quarter (the Moon is 90 degrees ahead of the Sun).
- `longitude` = 180 represents the Full Moon (the Moon and Sun are directly opposite each other).
- `longitude` = 270 represents the Third Quarter (the Moon is 270 degrees ahead of the Sun).

You can specify another search limit *y* in (fractional) days with the `limit` = *y* option, or alternatively the `days` = *y* option with equal meaning. If *y* is positive you search forward, if *y* is negative the search is backward.

You can specify a start date for the search, by either passing the Julian Date `jd` of interest or the Gregorian Date (year `y`, month `m`, day `d`, hour `hh`, minute `mm`, second `ss` - the latter might be fractional to include microseconds), with either six separate arguments or a table of these arguments.

If no start date is given, the current date and time are taken. Input and output time is UTC.

If successful, the function returns two values:

- the precise Julian Date timestamp of the predicted lunar phase event,
- the equivalent Gregorian date in a table with year, month, day, hour, minute and second including sub-second decimal places.

In case of an internal error or invalid input date, the function returns **fail**.

The results may vary slightly between platforms. Examples:

```
> import astro;
```

Search backward:

```
> astro.searchmoonphase(limit = -40, longitude = 180):
2461251.1094261 [2026, 7, 29, 14, 36, 19.011207818985]
```

Search forward:

```
> astro.searchmoonphase(2461252, limit = 40, longitude = 180):
2461280.6808049 [2026, 8, 28, 4, 19, 6.085461974144]
```

which is equal to:

```
> astro.searchmoonphase([2026, 7, 30, 12, 0, 0], limit=40, longitude=180):
2461280.6808049, [2026, 8, 28, 4, 19, 6.085461974144]
```

astro.searchplanetapsis (body [, y, m, d, hh, mm, ss])
astro.searchplanetapsis (body [, jd])

Finds the very next occurring Perihelion (the point in a planet's orbit closest to the Sun) or Aphelion (the point furthest from the Sun) relative to a given starting date and time.
The function requires a target body string as the first argument, followed by flexible time tracking options:

- the `body`, a string: The name of the body orbiting the sun: 'Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune' or 'Pluto'.
- the time (optional):
  • Omitted (1 argument total): Automatically uses the current system date and time (UTC) as the search baseline.
  • Julian Date (2 arguments total): A single number representing the absolute Julian Date `jd` to begin searching forward from.
  • Calendar parameters (7 arguments total): Pass separate numeric parameters matching (`body`, year `y`, month `m`, day `d`, hour `hh`, min `mm`, sec `ss`) to specify a custom date framework.

If successful, the function returns exactly 9 values:

The calendar year, month, day, hour, minute, second of the event (UTC) plus the kind 'perihelion' if closest to the Sun, or 'aphelion' if furthest, distance between the centers of the planet and the Sun measured in Astronomical Units (AU) and the exact Julian Date timestamp of the computed event.

In case of an internal error or invalid input date, the function returns **fail**.

The results may vary slightly between platforms. Examples:

```
> import astro;
```

Search for the next event for the ninth planet:

```
> astro.searchplanetapsis('Pluto', [2026, 8, 11, 12, 0, 0]):
2114   2   20   10   45   39.279797673225   aphelion   49.32025774638
2493232.9483713
```

And the event after that:

```
> astro.searchplanetapsis('Pluto', 2493233):
2237  9  16  17  1  52.103931605816  perihelion  29.645264444715
2538366.2096308
```

```
astro.searchriseset (body, lat, lon, elev, y, m, d,
   hh, mm, ss, direction [, limit])
astro.searchriseset (body, lat, lon, elev, jd, direction [, limit])
```

Searches for the next time a celestial body rises or sets as seen by an observer on the Earth. The function accepts the input parameters:

- `body` may be 'Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune', 'Pluto', 'Sun' or 'Moon'.
- `lat` for latitude in degrees, `lon` for longitude in degrees and `elev` for elevation in meters indicate a location on or near the surface of the Earth where the observer is located.
- The date and time of interest is given by either the Julian Date `jd` or the given year `y`, month `m`, day `d`, hour `hh`, minute `mm`, (fractional) second `ss`.
- Pass +1 for `direction` to search for the time a body begins to rise above the horizon. Pass -1 for `direction` to search for the time a body finishes sinking below the horizon.
- `limit` depicts how many days to search for a rise or set time, and defines the direction in time to search. It defaults to 1.0 day. When `limit` is positive, the search is performed into the future, after the given start time. When negative, the search is performed into the past, before start time. To limit a rise or set time to the same day, you can use a value of 1 day. In cases where you want to find the next rise or set time no matter how far in the future (for example, for an observer near the south pole), you can pass in a larger value like 365.

This function finds the next rise or set time of the Sun, Moon, or planet other than the Earth. Rise time is when the body first starts to be visible above the horizon. For example, sunrise is the moment that the top of the Sun first appears to peek above the horizon. Set time is the moment when the body appears to vanish below the horizon. Therefore, this function adjusts for the apparent angular radius of the observed body (significant only for the Sun and Moon).

This function corrects for a typical value of atmospheric refraction, which causes celestial bodies to appear higher above the horizon than they would if the Earth had no atmosphere. `Astronomy Engine` uses a correction of 34 arcminutes. Real-world refraction varies based on air temperature, pressure, and humidity; such weather-based conditions are outside the scope of `Astronomy Engine`.

Note that rise or set may not occur in every 24 hour period. For example, near the Earth's poles, there are long periods of time where the Sun stays below the horizon, never rising. Also, it is possible for the Moon to rise just before midnight but not set during the subsequent 24-hour day. This is because the Moon sets nearly an hour later each day due to orbiting the Earth a significant amount during each rotation of the Earth. Therefore callers must not assume that the function will always succeed.

On success, the function returns the date and time of the rise or set time as a Julian Date and a table of date time values (UTC), and **fail** in case of an error. Examples:

```
> import astro;
```

For sunrise at Chattanooga on August 13, 2026, enter:

```
> lat, lon, elev := 35.0456, -85.3097, 206;

> astro.searchriseset('Sun', lat, lon, elev, 2026, 8, 13, 0, 0, 0, +1):
2461265.9582471 [2026, 8, 13, 10, 59, 52.548474669456]
```

For sunset:

```
> astro.searchriseset('Sun', lat, lon, elev, 2461266, -1):
2461266.5220485 [2026, 8, 14, 0, 31, 44.994656145573]
```

The function provides a 1:1 interface to function Astronomy_SearchRiseSetEx() of the C library `Astronomy Engine for C/C++`. The description has been taken from the accompanying documentation.

See also: **astro.locate**, **astro.moonriseset**, **astro.sunriseset**.

astro.seasons ([year])

This function calculates the Julian Dates for the solstices and equinoxes of a given Gregorian year. If omitted, it defaults to the current system year. The return is a dictionary with the fields 'spring', 'summer', 'autumn' and 'winter'.

See also: **astro.autumn**, **astro.moon**, **astro.spring**, **astro.summer**, **astro.sun**.

astro.shabbat (y, cw)

Computes the start and end of Shabbat for a given calendar week `cw` and year `y` (both Gregorian) and returns two tables with six numbers each: the year, the month, the day, hour, minute and second for astronomical sunset (begin of Shabbat), nightfall (end of Shabbat) at Jerusalem, State of Israel, both in UTC. In case of a non-existent calendar week `cw`, the function issues an error.

Users should be aware that this function computes absolute geometric and astronomical boundaries for the Sun's position. It does not calculate the actual ritual or community timings observed in practice.

See file lib/astro.agn in your Agena installation folder for the parameters used.

See also: **astro.hdate**, **astro.searchaltitude**.

astro.siderial ([lon,] y, m, d, hh, mm, ss)
astro.siderial ([lon,] jd)

Calculates Greenwich Apparent Sidereal Time (GAST) for the given Gregorian date and time (first form: year `y`, month `m`, day `d`, hour `hh`, minute `mm`, second `ss`) in UTC or the Julian Date `jd` (second form).

Given a date and time, this function calculates the rotation of the Earth, represented by the equatorial angle of the Greenwich prime meridian with respect to distant stars (not the Sun, which moves relative to background stars by almost one degree per day). This angle is called Greenwich Apparent Sidereal Time (GAST). GAST is measured in sidereal hours in the half-open range [0, 24). When GAST = 0, it means the prime meridian is aligned with the of-date equinox, corrected at that time for precession and nutation of the Earth's axis. In this context, the `equinox` is the direction in space where the Earth's orbital plane (the ecliptic) intersects with the plane of the Earth's equator, at the location on the Earth's orbit of the (seasonal) March equinox. As the Earth rotates, GAST increases from 0 up to 24 sidereal hours, then starts over at 0. To convert to degrees, multiply the return value by 15.

For Local Siderial Time, pass the longitude `lon` in decimal degrees as a first argument.

The function provides a 1:1 interface to function Astronomy_SiderealTime() of the C library `Astronomy Engine for C/C++`. The description has been taken from the accompanying documentation.

```
> import astro;
```

GAST:

```
>  y, m, d, fd, hh, mm, ss := 2026, 8, 18, 9, 36, 20.000004172325

> astro.siderial(y, m, d, hh, mm, ss):
7.3902858764316
```

Local Siderial Time for Bonn, Germany:

```
> astro.siderial(7.0955, y, m, d, hh, mm, ss):
7.8633192097649
```

See also: **astro.locate**.

astro.solareclipse ([jd] [, true])

Computes the next solar eclipse following the given Julian date `jd`, an Agena number. If `jd` is not given, the function takes the current system time to compute the result.

If the last argument is **true** then the solar eclipse following the next eclipse will be determined.

The result is a table with the following fields:

- `'time'`, the date and time as a Julian Date when the solar eclipse is darkest. This is the instant when the axis of the Moon's shadow cone passes closest to the Earth's center.

- `'obscuration'`, the peak fraction of the Sun's apparent disc area obscured by the Moon (total and annular eclipses only).
- `'kind'`, the type of lunar eclipse found, either 'annular', 'partial' or 'total',
- `'distance'`, the distance between the Sun/Moon shadow axis and the center of the Earth, in kilometers.
- `'latitude'`, the geographic latitude at the center of the peak eclipse shadow.
- `'longitude'`, the geographic longitude at the center of the peak eclipse shadow.

See also: **astro.lunareclipse**.

astro.spring ([year])

The function approximates the Julian Date of the Spring Equinox for the given Gregorian `year`, an integer (the moment of Persian Nowruz in UTC). If not given, the current system time year is taken.

See also: **astro.autumn**, **astro.moon**, **astro.seasons**, **astro.summer**, **astro.sun**, **astro.winter**.

astro.summer ([year])

The function approximates the Julian Date of the Summer Solstice for the given Gregorian `year`, an integer. If not given, the current system time year is taken.

See also: **astro.autumn**, **astro.moon**, **astro.seasons**, **astro.spring**, **astro.sun**, **astro.winter**.

astro.sun (year, month, day, lon, lat)

Provides an easier-to-use interface to **astro.sunriseset**.

`year`, `month`, and `day` must be integers. Longitudes and latitudes can be given in form of two tables `lon`, `lat`, containing degrees (a number), minutes (a number), seconds (a number), and the orientation (the single-character string 'N', 'S', 'W', or 'E'). Alternatively, you may give decimal degrees (numbers) for `lon` and `lat`.

The return is a table with the indices 'riseset', 'civil', 'astro', and 'nautical' containing the rise and set times in `tm` representation, UTC. The index 'south' holds the time when the Sun is at south.

See **astro.sunriseset** for further information.

The function uses the `tm` time notation of the **clock** package. The function uses the `tm` time notation of the **clock** package. You do not have to readlib **clock** before.

The function is written in Agena and included in the `astro.agn` file.

Example for Düsseldorf:

```
> astro.sun(2013, 1, 7, [6, 46, 58, 'E'], [51, 13, 32, 'N']):
[astro ~ [tm(5, 34, 5.15), tm(17, 44, 22.95)],
civil ~ [tm(6, 56, 25.74), tm(16, 22, 2.36)],
nautical ~ [tm(6, 14, 13.02), tm(17, 4, 15.08)],
riseset ~ [tm(7, 35, 19.78), tm(15, 43, 8.33)],
south ~ tm(11, 39, 14.05)]
```

See also: **astro.locate**, **astro.moon**.

astro.sunriseset (year, month, day, lon, lat [, option])

Returns the sunrise/sunset times in UTC for years starting with 1800 A.D. to 2099 A.D. It is a workhorse function, maybe you would like to use **astro.sun** for a more convenient interface.

`year`, `month` and `day`, all integers, are the values of the day to evaluate. `lon` is the longitude (west/east), and `lat` the latitude (west/east), both in decimal degrees of type float of the location that is of interest. Use **astro.dmstodec** to convert co-ordinates containing degrees (integer), minutes (integer), and seconds (integer or float), and the orientation to decimal degrees.

Example for Düsseldorf:

```
> astro.sunriseset(2013, 1, 7,
>   astro.dmstodec(6, 46, 58, 'E'), astro.dmstodec(51, 13, 32, 'N')):
7.5888265301838 15.718979334935 0    6.9404828811745 16.367322983944 0
6.2369508540273 17.070855011091 0    5.5680967691543 17.739709095964 0
11.653902932559
```

The first and second returns are the sunrise/sunset times which are considered to occur when the Sun's upper limb is 35 arc minutes below the horizon (this accounts for the refraction of the Earth's atmosphere).

The third return is 0, if the rises and sun sets in a day; +1 if the Sun is above the specified `horizon` 24 hours, -1 if the Sun is below the specified `horizon` 24 hours.

The fourth and fifth returns are start and end times of civil twilight. Civil twilight starts/ends when the Sun's centre is 6 degrees below the horizon.

The sixth return is 0, if the rises and sun sets in a day; +1 if the Sun is above the specified `civil twilight horizon` 24 hours, -1 if the Sun is below the specified `horizon` 24 hours.

The seventh and eighth returns are the start and end times of nautical twilight. Nautical twilight starts/ends when the Sun's centre is 12 degrees below the horizon.

The ninth return is 0, if the rises and sun sets in a day; +1 if the Sun is above the specified `nautical twilight horizon` 24 hours, -1 if the Sun is below the specified `horizon` 24 hours.

The tenth and eleventh returns are the start and end times of astronomical twilight. Astronomical twilight starts/ends when the Sun's centre is 18 degrees below the horizon.

The twelfth return is 0, if the rises and sun sets in a day; +1 if the Sun is above the specified `nautical twilight horizon` 24 hours, -1 if the Sun is below the specified `astronomical twilight horizon` 24 hours.

The thirteenth return is the time when the Sun is at south (in decimal UTC).

If any `option` is given, all the results are returned in a table instead of 13 separate returns, in the order mentioned above.

All times returned are given in decimal hours of type number. Use **clock.totm** to convert them into `tm` notation.

See also: **astro.locate**, **astro.moon**, **astro.searchriseset**, **astro.sun**.

astro.taiutc ([jd])

Returns the TAI-UTC lookup table value of leap seconds for a given Julian date `jd`; if no argument is given, then the value for the current system date is computed. TAI stands for International Atomic Time. The function returns 0 for Gregorian dates before 1961.

In the future, you may have to add further values to the source code of this function which also includes the URL of the respective file to be checked. The function is written in Agena and included in the `lib/astro.agn` file.

See also: **astro.jdate**, **os.date** ('*j' format):

astro.tf2d (sign, hours, minutes, seconds)
astro.tf2d (sign, obj)

Converts integral `hours`, `minutes`, `seconds` to fractional, decimal days. If not given, minutes and seconds default to zero. You may also pass the time in a table `obj` in the order mentioned before.

`sign` usually is the single-character string '+', but can also be '-'. The empty string defaults to '+'.

The function returns three values:

- the fractional day(s),
- a numerical status code: 0 for okay, 1 for hours out-of-range, 2 for minutes out-of-range, 3 for seconds out-of-range or any other integer,
- the corresponding status text: 'okay', 'hour outside range [0, 23]', 'minute outside range [0, 59]', 'second outside range [0, 60)' or 'unknown exception'.

The function provides a 1:1 interface to SOFA's iauTf2d() function.

See also: See also **astro.a2af**, **astro.a2tf**, **astro.d2tf**, **astro.cal2jd**, **astro.dmstodec**, **astro.dectodms**, **clock.adjust**.

astro.winter ([year])

The function approximates the Julian Date of the Winter Solstice for the given Gregorian `year`, an integer. If not given, the current system time year is taken.

See also: **astro.autumn**, **astro.moon**, **astro.seasons**, **astro.spring**, **astro.summer**, **astro.sun**.

11.10 cordic - Numerical CORDIC Library

As a *plus* package, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import cordic`.

The CORDIC algorithm (CORDIC stands for COordinate Rotation DIgital Computer) also known as the `Volder's algorithm`, is used to calculate hyperbolic, trigonometric, logarithmic, and root functions, on hardware not featuring multipliers, requiring only addition, subtraction, bitshift and table lookups.

The algorithm, similar to one published by Henry Briggs around 1624, has been developed in 1959 by Kack E. Volder to improve an aviation system. According to Grokipedia, it has not only been used in pocket calculators, but also in x87 FPUs, in CPUs prior to Intel 80486 - and in Motorola's 68881, in signal and image processing, communication systems, robotics, and also 3D graphics - and other applications.

This binding to John Burkardt's CORDIC implementation uses additon, subtraction, table lookups, multiplication, divisions, and the absolute function.

The package accepts and returns Agena numbers only.

Available functions are:

`cordic.carccos (x [, iters])`

Returns the inverse cosine operator in radians.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

`cordic.carcsin (x [, iters])`

Returns the inverse sine operator in radians.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

`cordic.carctan2 (y, x [, iters])`

Returns the arc tangent of `y`/`x` in radians, but uses the signs of both parameters to find the quadrant of the result.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

cordic.carctanh (x [, iters])

Returns the inverse hyperbolic tangent of `x` in radians. By default, `iters` is set to 25 iterations, you can pass any other positive integer.

cordic.ccbrt (x [, iters])

Returns the cubic root of the number `x`.

By default, `iters` is set to 53 iterations, you can pass any other positive integer. The default is essentially the number of binary digits of accuracy, and might go as high as 53.

cordic.ccos (x [, iters])

Returns the cosine of `x` in radians.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

cordic.ccosh (x [, iters])

Returns the hyperbolic cosine of `x` in radians. By default, `iters` is set to 25 iterations, you can pass any other positive integer.

cordic.cexp (x [, iters])

Returns e^x, the exponential function to the base e =2.718281828459 ... By default, `iters` is set to 25 iterations, you can pass any other positive integer.

cordic.chypot (x, y [, iters])

Returns $\sqrt{x^2+y^2}$, the hypotenuse.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

cordic.cln (x [, iters])

Returns the natural logarithm of `x`. By default, `iters` is set to 25 iterations, you can pass any other positive integer.

cordic.cmul (x, y)

Returns `x*y`, i.e. the product of `x` and `y`.

cordic.cpow (x, y)

Returns `x^y`, i.e. `x` risen to the power of `y`.

cordic.csin (x [, iters])

Returns the sine of `x` in radians.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

cordic.csinh (x [, iters])

Returns the hyperbolic sine of `x` in radians. By default, `iters` is set to 25 iterations, you can pass any other positive integer.

cordic.csqrt (x [, iters])

Returns the square root of `x`.

By default, `iters` is set to 53 iterations, you can pass any other positive integer. The default is essentially the number of binary digits of accuracy, and might go as high as 53.

cordic.ctan (x [, iters])

Returns the tangent of `x` in radians.

By default, `iters` is set to 60 iterations, you can pass any other positive integer. A value of 10 is low. Good accuracy is achieved with 20 or more iterations.

cordic.ctanh (x [, iters])

Returns the hyperbolic tangent of `x` in radians. By default, `iters` is set to 25 iterations, you can pass any other positive integer.

11.11 zx - Sinclair ZX Spectrum Arithmetic Functions

As a *plus* package, the **zx** package is not part of the standard distribution and must be activated with the **import** statement, i.e. `import zx`.

11.11.1 Introduction

This package implements various Sinclair ZX Spectrum mathematical functions.

Most of the functions use the same algorithms and Chebyshev polynomials of degree 6, 8, or 12 as implemented in the Sinclair ZX Spectrum ROM, with similar - *but not equal* - accuracy.

All functions are based on those published on the book `The Complete Spectrum ROM Disassembly`, written by Dr. Ian Logan & Dr. Frank O'Hara, pp. 217.

In general, the procedures are mostly slower and also less precise than their Agena pendants. By default, the fully expanded and simplified polynomials are hard-wired into the library's C code. By passing the optional last argument **true**, however, the polynomials are processed iteratively in real-time, using the **zx.genseries** function which imitates the Z80 assembler subprocedure `series generator`.

You may query the respective Chebyshev coefficient vectors by calling **zx.getcoeffs**, and globally change them with **zx.setcoeffs**. Range reduction is performed by **zx.reduce**.

The names of all ZX Spectrum `clones` are written in capital letters, to not collide with Agena's built-in operators. Some functions that do not exist on the ZX Spectrum have been added and are mostly based upon existing ZX Spectrum mathematical functions, thus providing comparable accuracy.

The C source file src/zx.c contains information on the precision of the functions.

11.11.2 Common Functions

`zx.ABS (x)`

Returns the absolute magnitude of the number `x`.

See also: **abs**.

zx.ACS (x)

Computes the ZX Spectrum inverse cosine of its numeric argument x and returns a number. If x ∉ [-1, 1], **undefined** will be returned.

See also: **arccos**, **zx.ASN**, **zx.ATN**.

zx.ACSH (x)

Approximates the inverse hyperbolic cosine of its numeric argument x and returns the number ln(x + $\sqrt{x^2-1}$) for 1 ≤ x < ∞. The function does not exist on the ZX Spectrum.

See also: **arccosh**, **zx.ACSH**, **zx.ATNH**.

zx.ADD (x, y)

Returns x + y in ZX Spectrum machine precision.

See also: **zx.DIV**, **zx.MUL**, **zx.SUB**.

zx.AND (x, y)

Returns x if y is non-zero and the value zero otherwise. Strings are not supported.

See also: **zx.NOT**, **zx.OR**.

zx.ASN (x)

Computes the ZX Spectrum inverse sine of its numeric argument x and returns a number. If x ∉ [-1, 1], **undefined** will be returned.

See also: **arcsin**, **zx.ACS**, **zx.ATN**.

zx.ASNH (x)

Approximates the inverse hyperbolic sine of its numeric argument x and returns the number ln(x + $\sqrt{x^2+1}$) $\forall_x$. The function does not exist on the ZX Spectrum.

See also: **arcsinh**, **zx.ACSH**, **zx.ATNH**.

zx.ATN (x)

Computes the ZX Spectrum inverse tangent of its numeric argument x and returns a number.

See also: **arctan**, **zx.ACS**, **zx.ATN**.

zx.ATNH (x)

Approximates the inverse hyperbolic tangent of its numeric argument x and returns the number $\frac{1}{2}\ln\frac{1+x}{1-x}$ for -1 < x < 1, or **undefined** otherwise. The function does not exist on the ZX Spectrum.

See also: **arctanh**, **zx.ACSH**, **zx.ASNH**.

zx.COS (x)

Computes the ZX Spectrum cosine of its numeric argument x and returns a number.

See also: **cos**, **zx.SIN**, **zx.TAN**.

zx.COSH (x)

Approximates the hyperbolic cosine of its numeric argument x and returns the number 0.5*(**zx.EXP**(x) + **zx.EXP**(-x)). The function does not exist on the ZX Spectrum.

See also: **cosh**, **zx.SINH**, **zx.TANH**, **zx.SECH**, **zx.CSCH**, **zx.COTH**. .

zx.COT (x)

Approximates the cotangent 1/**zx.TAN**(x), or **undefined** if x = 0. The function does not exist on the ZX Spectrum.

See also: **cot**.

zx.COTH (x)

Approximates the hyperbolic cotangent of its numeric argument x and returns the number (**zx.EXP**(x) + **zx.EXP**(-x))/(**zx.EXP**(x) - **zx.EXP**(-x)). The function does not exist on the ZX Spectrum.

See also: **coth**, **zx.SINH**, **zx.COSH**, **zx.TANH**, **zx.SECH**, **zx.CSCH**.

zx.CSC (x)

Returns the cosecant 1/**zx.SIN**(x), or **undefined** if x = 0. The function does not exist on the ZX Spectrum.

See also: **csc**.

zx.CSCH (x)

Approximates the hyperbolic cosecant of its numeric argument x and returns the number 2/(**zx.EXP**(x) - **zx.EXP**(-x)). With x = 0, returns **undefined**. The function does not exist on the ZX Spectrum.

See also: **csch**, **zx.SINH**, **zx.COSH**, **zx.TANH**, **zx.SECH**, **zx.COTH**.

zx.DIV (x, y)

Returns x / y in ZX Spectrum precision. With y = 0, the result is **undefined**.

See also: **zx.ADD**, **zx.MOD**, **zx.MUL**, **zx.SUB**.

zx.E

The constant e = exp(1) in the ZX Spectrum precision, implemented as **zx.EXP**(1).

See also **zx.PI**.

zx.ERF (x)

Computes the error function $\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$. The function does not exist on the ZX Spectrum.

See also: **erf**, **zx.ERFC**.

zx.ERFC (x)

Computes the complimentary error function 1- zx.ERF(x). The function does not exist on the ZX Spectrum.

See also: **erfc**, **zx.ERF**.

zx.EXP (x)

Computes the ZX Spectrum exponential function of the number x to the base **E** = exp(1). It looses precision, however, if its argument is greater than the constant **E**.

See also: **exp**. **zx.LN**, **zx.POW**.

zx.GAM (x)

Approximates the Gamma function $\Gamma x.$ = (x - 1)!. Returns **undefined** if x = 0, but for the sake of speed does not return **undefined** for negative integral x. The function does not exist on the ZX Spectrum.

See also: **gamma**, **zx.LGAM**.

zx.HYP (x, y)

Computes the hypotenuse $\sqrt{x^2+y^2}$ in approximate ZX Spectrum machine precision which equals **zx.SQR**(**zx.ADD**(**zx.MUL**(x, x), **zx.MUL**(y, y))). The function does not exist on the ZX Spectrum.

zx.INT (x)

Rounds its numeric argument x downwards to the nearest integer.

See also: **entier**.

zx.LGAM (x)

Approximates the logarithmic Gamma function ln Γx. = ln(x - 1)!. Returns **undefined** if x = 0, but for the sake of speed does not return **undefined** for negative integral x. The function does not exist on the ZX Spectrum.

See also: **lngamma**, **zx.GAM**.

zx.LN (x)

Computes the ZX Spectrum natural logarithm of the number x. If x $\leq$ 0, **undefined** will be returned.

See also: **ln**, **zx.EXP**.

zx.MOD (x, y)

Computes the modulo x%y in approximate ZX Spectrum machine precision and returns the number **zx.SUB**(x, **zx.MUL**(y, **zx.INT**(**zx.DIV**(x, y)))). The function does not exist on the ZX Spectrum.

See also: **zx.DIV**.

zx.MUL (x, y)

Computes x*y in ZX Spectrum precision.

See also: **zx.ADD**, **zx.DIV**, **zx.SUB**.

zx.NOT (x)

Returns 1 if its numeric argument x is 0, and 0 otherwise.

See also: **not**, **zx.AND**, **zx.OR**.

zx.OR (x, y)

Returns the number x if the number y is 0, and 1 otherwise. Strings are not supported.

See also: **or**, **zx.AND**, **zx.NOT**.

zx.PI

The constant π in the ZX Spectrum precision, implemented as 4***zx.ATN**(1).

See also **zx.E**.

zx.POW (x, y)

Returns the ZX Spectrum exponentiation x ↑ y, with x and y numbers, and returns a number.

Internally, the ZX Spectrum and this function treats x ↑ y like **exp**(**ln**(x)*y). If x < 0 then **undefined** will be returned.

As with **zx.EXP**, the function is quite imprecise if x > **zx.E**, i.e. the constant e^1 = exp(1).

See also: ** and ^ operators, **zx.HYP**, **zx.SQR**.

zx.SEC (x)

Returns the secant 1/**zx.COS(x)**.

See also **sec**.

zx.SECH (x)

Approximates the hyperbolic secant of its numeric argument x and returns the number 2/(**zx.EXP**(x) + **zx.EXP**(-x)). The function does not exist on the ZX Spectrum.

See also: **sech**, **zx.SINH**, **zx.COSH**, **zx.TANH**, **zx.CSCH**, **zx.COTH**.

zx.SGN (x)

Returns -1 if the number x is negative, 0 if x is zero, and 1 if x is positive. If x is **undefined**, **undefined** will be returned.

See also: **sign**, **signum**, **zx.SIG**.

zx.SIG (x)

Returns -1 if the number x is negative, and 1 otherwise. If x is **undefined**, **undefined** will be returned. The function does not exist on the ZX Spectrum.

See also: **sign**, **signum**, **zx.SIG**.

zx.SIN (x)

Computes the ZX Spectrum sine of its numeric argument x and returns a number.

See also: **sin**, **zx.COS**, **zx.TAN**.

zx.SINH (x)

Approximates the hyperbolic sine of its numeric argument x and returns the number 0.5*(**zx.EXP**(x) - **zx.EXP**(-x)). The function does not exist on the ZX Spectrum.

See also: **sinh**, **zx.COSH**, **zx.TANH**, **zx.SECH**, **zx.CSCH**, **zx.COTH**.

zx.SQR (x)

Returns the ZX Spectrum square root of its numeric argument x and returns a number. If $x < 0$, **undefined** will be returned.

See also: **sqrt**, **zx.HYP**, **zx.POW**.

zx.SUB (x, y)

Computes x - y in ZX Spectrum precision.

See also: **zx.ADD**, **zx.DIV**, **zx.MUL**.

zx.TAN (x)

Computes the ZX Spectrum tangent of its numeric argument x and returns a number.

See also: **tan**, **zx.COS**, **zx.SIN**.

zx.TANH (x)

Approximates the hyperbolic tangent of its numeric argument x and returns the number (**zx.EXP**(x) - **zx.EXP**(-x))/(**zx.EXP**(x) + **zx.EXP**(-x)). The function does not exist on the ZX Spectrum.

See also: **tanh**, **zx.COSH**, **zx.SINH**, **zx.SECH**, **zx.CSCH**, **zx.COTH**.

11.11.3 Auxiliary Functions

zx.genseries (x, s)

Receives a number `x` in the range [-1, 1] and a sequence of coefficients and returns the value of the corresponding Chebyshev polynomial. If `x` is out of range, *no* error will be returned. This is an exact clone of the ZX Spectrum ROM `series generator` Z80 assembler subroutine.

zx.getcoeffs ()

The function returns the current Chebyshev coefficient vectors for various package functions. The return is a dictionary of four numeric sequences:

Key	Used by	Indirectly used by	Default size
`'SIN'`	**zx.SIN**	**zx.COS** and **zx.TAN**	6
`'ATN'`	**zx.ATN**	**zx.ACS** and **zx.ASN**	12
`'LN'`	**zx.LN**	**zx.SQR** and **zx.POW**	12
`'EXP'`	**zx.EXP**	**zx.SQR** and **zx.POW**	8

See also: **zx.setcoeffs**.

zx.setcoeffs (n, s)

Globally sets Chebyshev coefficients to the package's environment. You can change existing coefficients, reduce or enlarge their respective number down to one or up to 256 values. Internally, the coefficients are treated as C doubles, the shipped defaults have the precision of C floats.

The first argument `n` must be the string `'SIN'`, `'ATN'`, `'LN'`, or `'EXP'`. The second argument `s` must be a sequence of one to 256 numbers.

For the purpose of the first argument, see **zx.getcoeffs**.

Please note that the respective zx functions must be called with the last argument **true** in order to revert to the (changed) coefficient vectors as they use hard-wired expanded polynomials by default.

See also: **zx.getcoeffs**.

zx.reduce (x)

Reduces a number `x` to another number v in the range [-1, 1] where sin(v) = sin(π *v/2) for multiples or fractions of π, and returns v. Please note that even if `x` $\in$ [-1, 1], v will be calculated.

The function imitates the ZX Spectrum ROM 'reduce argument' Z80 assembler subroutine which is used to prepare calls to ZX Spectrum's sine and cosine subroutines. Example:

```
> zxsin := proc(x) is
>    local w, z;
>    w := zx.reduce(x);
>    z := 2 * w**2 - 1;
>    return w * zx.genseries(z, zx.getcoeffs().SIN)
> end;

> zxcos := <: x -> zxsin(x + Pi/2) :>;
```

See also: **math.wrap**, end of Chapter 11.1.2 for a comparison chart.

11.12 calc - Calculus Package

This package contains mathematical routines to perform basic calculus *numerically*. Since the functions do not work symbolically, please beware of round-off errors.

A typical example might look like this:

Define a function f :=x → sin(x):

```
> f := <: x -> sin(x) :>
```

Determine all its zeros over [-5, 5]:

```
> calc.zeros(f, -5, 5):
seq(-3.1415926535898, 0, 3.1415926535898)
```

Differentiate it at point 0:

```
> calc.differ(f, 0):
1
```

To display more digits, enter:

```
> environ.kernel(digits = 17)
```

Compute the minimum and maximum values on the interval [-10, 10]:

```
> calc.minimum(f, -10, 10):
seq(-7.8539816339744828, -1.5707963267948966, 4.7123889803846897)

> calc.maximum(f, -10, 10):
seq(-4.7123889803846897, 1.5707963267948966, 7.8539816339744828)
```

Integrate it over [0, π]:

```
> calc.integ(f, 0, Pi):
2       4.0000000032552159e-015
```

To facilitate computing the integral of sin(x) a bit you might define a shortcut. We put the function term into brackets to just return the first result but not the error estimate:

```
> F := <: a, b -> (calc.integ(f, a, b)) :>

> F(0, Pi):
2
```

Summary of functions:

Basic Calculus:

calc.aitken, **calc.brent**, **calc. chandrupatla**, **calc.fmaxbr**, **calc.fmaxgs**, **calc.fminbr**, **calc.fmings**, **calc.iscont**, **calc.itp**, **calc.limit**, **calc.maximum**, **calc.minimum**, **calc.regulafalsi**, **calc.sections**, **calc.zeroab**, **calc.zeroin**, **calc.zeros**.

Differentiation:

calc.diff, **calc.differ**, **calc.eulerdiff**, **calc.isdiff**, **calc.xpdiff**.

Integration:

calc.gauleg, **calc.gtrap**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.riesum**, **calc.romberg**, **calc.simaptive**.

Integrals:

calc.ausSiCi, **calc.Ci**, **calc.Cin**, **calc.Chi**, **calc.dawson**, **calc.Ei**, **calc.Ein**, **calc.elliptic1**, **calc.elliptic2**, **calc.En**, **calc.fresnelc**, **calc.fresnels**, **calc.ibeta**, **calc.igamma**, **calc.igammc**, **calc.invibeta**, **calc.scaleddawson**, **calc.Shi**, **calc.Si**, **calc.Ssi**, **calc.w**.

Sums & Products:

calc.prod, **calc.fsum**, **calc.horner**.

Interpolation:

calc.cheby, **calc.chebyt**, **calc.chebycoeffs**, **calc.chebygen**, **calc.clampedspline**, **calc.clampedsplinecoeffs**, **calc.interp**, **calc.linterp**, **calc.naksplinecoeffs**, **calc.neville**, **calc.newtoncoeffs**, **calc.polyfit**, **calc.polygen**, **calc.savgol**, **calc.savgolcoeffs**, **calc.smoothstep**.

Distances

calc.arclen, **calc.eucliddist**, **calc.sinuosity**.

Miscellaneous:

calc.Ai, **calc.bessel0**, **calc.bessel1**, **calc.Bi**, **calc.dct**, **calc.dilog**, **calc.dst**, **calc.eps**, **calc.eta**, **calc.euler**, **calc.expn**, **calc.gammainc**, **calc.gaussian**, **calc.hyp1f1**, **calc.hyp2f1**, **invpsi**, **calc.lambda**, **calc.polylog**, **psi**, **calc.weier**, **calc.zeta**, **calc.zeta2**.

The functions:

calc.Ai (x)

The Airy wave function returns both the first independent solution to the differential equation y''(x) = x*y and its first derivative, for any real `x`.

See also: **calc.Bi**.

calc.aitken (f, x0 [, eps [, iter]])

The function finds the limit of the sequence $x_{n+1} = f(x_n)$ with initial x_0 and tolerance `eps`, with a maximum of `iter` iterations, using Aitken extrapolation. `f` is a univariate function. `eps` by default is **DoubleEps** and `iter` is 20.

It returns either the approximated limit α and the first derivative at α, `f`'(α), if successful, and **undefined** twice otherwise. The third return is the number of iterations taken to compute the result.

Example: **calc.aitken**(<: x -> 1/2*(x + 2/x) :>, 1) ~= **sqrt**(2).

See also: **times**.

calc.arclen (f, a, b [, ···] [, options])

The function returns the arc length (curvilinear length) of a univariate or multivariate function `f` between the points `a` and `b`. The second, etc. arguments to `f` may be given right after argument `b`.

The function by default internally uses a tolerance value of **DoubleEps** which can be changed by passing the option `eps` = *tol*, where *tol* is a positive number, e.g.:

```
> calc.arclen(<: x, a -> sin(x + a) :>, 0, Pi, 0, eps = 1e-5):
3.8201977882313 1.5280690230037e-014
```

The function checks whether `f` has poles in the given range [`a`, `b`] and splits it up accordingly, summing up the various sub-lengths to return a finite result, if possible. You can switch off this feature by passing the `recurse` = **false** option.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.eucliddist**, **calc.sinuosity**.

calc.auxSiCi (x)

Computes the auxiliary sine and cosine integrals and returns the two numbers:

$$\sum_{t=0}^{\infty} \frac{\sin(t)}{t+x}dt, \sum_{t=0}^{\infty} \frac{\cos(t)}{t+x}dt$$

See also: **calc.Ci**, **calc.Cin**, **calc.Chi**, **calc.Shi**, **calc.Si**, **calc.SiCi**, **calc.Ssi**.

calc.bessel0 (x [, any])

When called with just one argument `x`, returns the modified Bessel function of order zero of the argument. The function is defined as **bessel0**(`x`) = **besselj**(0, `x`*I).

With `any` second argument, returns the modified Bessel function of order zero of the argument, exponentially scaled.

See also: **besselj**, **bessely**, **calc.bessel1**.

calc.bessel1 (x [, any])

When called with just one argument `x`, returns the modified Bessel function of order one of the argument. The function is defined as **bessel1**(`x`) = **besselj**(1, `x`*I).

With `any` second argument, returns the modified Bessel function of order one of the argument, exponentially scaled.

See also: **besselj**, **bessely**, **calc.bessel0**.

calc.Bi (x)

The Airy wave function returns both the second independent solution to the differential equation y''(x) = x*y and its first derivative, for any real x.

See also: **calc.Ai**.

calc.brent (f, a, b [, ···] [, options])

The function seeks a zero of a univariate or multivariate function `f` over the interval [`a`, `b`] using a method developed by Richard Brent, a variation of the one designed by G. Forsythe, M. Malcolm and C. Moler and implemented with **calc.zeroin**. You can control the accuracy by passing the `eps`=<positive number> option, with **DoubleEps** the default (**hEps** on ARM).

You can control the maximum number of iterations the function takes until finishing the evaluation by passing the `iters`=<positive integer> option, with the 125 the default.

See also: **calc.chandrupatla**, **calc.itp**, **calc.regulafalsi**, **calc.sections**, **calc.zeroab**, **calc.zeroin**, **calc.zeros**.

calc.Ci (x)

Computes the cosine integral for its numeric argument `x` and returns it as a number:

$$Ci(x) = EulerGamma + ln(|x|) - calc.Cin(x).$$

See also: **calc.auxSiCi**, **calc.Cin**, **calc.Si**, **calc.Chi**, **calc.Shi**, **calc.Ssi**.

calc.Cin (x)

Computes the entire cosine integral for its numeric argument `x` and returns the number:

$$Cin(x) = \sum_{0}^{x} \frac{1-\cos(t)}{t} dt$$

See also: **calc.auxSiCi**, **calc.Ci**, **calc.Si**, **calc.Chi**, **calc.Shi**, **calc.Ssi**.

calc.chandrupatla (f, a, b [, ···] [, options])

The function seeks a zero of a univariate or multivariate function `f` over the interval [`a`, `b`] using Prof. Chandrupatla's algorithm, Prof. Chandrupatla's algorithm, a hybrid quadratic/bisection method for finding the zero of a nonlinear function without using derivatives. It works fine with polyomials, as well. With multivariate `f` put its second, third etc. argument right after `b`.

There are two options with which you can control accuracy: `eps` and `delta`, which are both set to **hEps** by default. To use different values, pass `eps`=<positive number> and/or `delta`=<positive number> as the last arguments. Example:

```
> calc.chandrupatla(<: x -> x^3-2*x^2+1 :>, 1.1, 2,
>    eps=DoubleEps, delta=hEps):
1.6180339887499
```

Since the function finds only one zero, you can pass it to the **calc.zeros** wrapper to get all the roots over a larger interval, for example:

```
> calc.zeros(<: x -> sin x :>, -1000, 1000, finder = calc.chandrupatla):
```

You can control the maximum number of iterations the function takes until finishing the evaluation by passing the `iters`=<positive integer> option, with the 125 the default.

See also: **calc.brent**, **calc.itp**, **calc.regulafalsi**, **calc.sections**, **calc.zeroab**, **calc.zeros**.

calc.cheby (f, a, b, n [, ···] [, option])

Returns a function computing the Chebyshev interpolant for a given point. `f` is the univariate or multivariate function to be interpolated, `a` and `b` represent the domain of the definition, `n` is the order of the interpolant. As a rule of thumb, the wider the domain, the larger `n` should be. If `f` has more than one argument, then all arguments *except the first* are passed right after `n`.

You may optionally pass the `deriv`=k option as the very last argument to compute either the first (k=1), second (k=2), third (k=3), fourth (k=4), fifth (k=5), sixth (k=6), seventh (k=7) or eighth (k=8) derivative, where k defaults to 0, i.e. the function itself.

Using this function may speed up numeric computations significantly if the expression to be evaluated consists of many subexpressions - and if accuracy is not of primary concern. When computing derivatives, however, it is 10 times faster than **calc.xpdiff** and - depending on the expression type - also more accurate. The function uses 80-bit floating-point precision internally.

Example:

```
> # get first derivative of ln(x), i.e. 1/x
> g := calc.cheby(<: x -> ln x :>, 1, 10, 50, deriv = 1);

> g(5):
0.20000000000002
```

See also: **calc.chebycoeffs**, **calc.diff**, **calc.differ**, **calc.savgol**, **calc.xpdiff**.

calc.cheby64 (f, a, b, n [, ···] [, option])

Like **calc.cheby**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.cheby** as they do not have 80-bit arithmetic.

calc.chebycoeffs (f, a, b, n)

Computes Chebyshev interpolation coefficients used internally by **calc.cheby**. `f` is a univariate function for which coefficients shall be computed, `a` and `b` represent the domain of the definition, `n` is the order of the interpolant.

The return is a table of the Chebyshev coefficients, indexed from 1 to `n`, with key `` `domain` `` representing the domain `a:b` (a pair). As a rule of thumb, the larger the domain, the larger `n` should be.

See also: **calc.chebygen**, **calc.savgolcoeffs**.

calc.chebygen (coeffs [, deriv = n])

Takes a table of Chebyshev coefficients as computed by **calc.chebycoeffs** and generates a factory computing approximations of a real function f.

With the optional `deriv = n` option, where `n` is an integer from 1 to 8, the `n`-th derivative of f will be approximated instead of the plain function value, the default is `n` = 0:

```
> coeffs := calc.chebycoeffs(<: x -> ln x :>, 1, 3, 20):
[1 ~ 1.247621, 2 ~ 0.535898, 3 ~ -0.071797, ..., domain ~ 1:3]
```

Return a function that computes the first derivative of ln(x):

```
> f' := calc.chebygen(coeffs, deriv = 1);

> f'(2):
0.49999999999985
```

Note that the table of Chebyshev coefficients must also contain the domain for which the coefficients have been computed, see `domain ~ 1:3` entry above. The generated function, when called with a point, will not complain if the given point is not in the domain, e.g. with a `domain ~ 1:3`, a call `f'(5)` will always be accepted.

See also: **calc.cheby**, **calc.diff**, **calc.savgol**, **calc.xpdiff**.

calc.chebyt (n, x)

Computes the `n`-th Chebyshev polynomial of the first kind, evaluated at `x`, with `n` a non-negative integer and `x` a number. The return is equal to **cos**(`n`***arccos**(`x`)) for $|x| \leq 1$, and **cosh**(`n`***arccosh**(`x`)) otherwise, even for $x < 1$.

calc.Chi (x)

Computes the hyperbolic cosine integral and returns it as a number. `x` must be a number.

See also: **calc.Si**, **calc.Ci**, **calc.Shi**, **calc.Ssi**.

calc.clampedspline (obj, da:db)

calc.clampedspline (obj, da:db, a)

calc.clampedspline (obj, da:db, a, coeffs)

Evaluates the clamped cubic spline for a given table or sequence `obj` of pairs representing the points x_k:y_k, at a single value `a` (a number) of the independent variable x.

The boundary conditions are passed as a pair of numbers `da:db`, where `da` is the derivative of the function at the left border, and `db` is the derivative of the function at the right border.

In the first form, returns a univariate function which can be called with a number to obtain the value of the interpolating polynomial. For best performance, use this first form.

In the second form, the function computes the coefficients of the linear, quadratic, and cubic terms itself in each call.

In the third form, the function expects the coefficients `coeffs` of the linear, quadratic, and cubic terms as a sequence of three sequences, in this order, and each containing numbers. The fourth argument may be obtained by calling **calc.clampedsplinecoeffs**.

In the second and third form, the function returns the value of the interpolating polynomial, a number, at the specified value `a` of the independent variable x.

In general, the function returns **fail** if the structure contains less than two pairs.

See also: **calc.interp**, **calc.clampedsplinecoeffs**, **calc.nakspline**, **calc.neville**.

calc.clampedsplinecoeffs (obj, da:db)

Determines the coefficients for the clamped cubic spline for a given table or sequence `obj` of pairs representing the points x_k:y_k. The return can be used to speed up execution of **calc.clampedspline**.

The boundary conditions are passed as a pair of numbers `da:db`, where `da` is the derivative of the function at the left border, and `db` is the derivative of the function at the right border.

The function returns **fail** if the structure less than two pairs.

See also: **calc.clampedspline**.

calc.curvature (f, x [, ···])

Determines the curvature of a real univariate or multivariate function at a given point. Curvature in this context is defined as the rate of change of *direction* of a point that moves on a curve. The result is the number $f''(x, \cdots)/(1 + f'(x, \cdots)^2)^{3/2}$.

The second, etc. arguments to `f` may be given right after argument `x`.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.sinuosity**.

calc.dawson (x)

Computes Dawson's integral for a number `x`. The return is the number

$$e^{-x^2} \int_0^x e^{t^2} dt.$$

See also: **calc.scaleddawson**, **expx2**.

calc.dct (u [, m])

Finds the Fourier discrete cosine transform (DCT) of type m of a one-dimensional data vector `u`.

By default, DCT-I is computed, but you can do DCT-II, DCT-III and DCT-IV by passing 2, 3 or 4 for m.

The inverse DCTs for types 1, 2, 3, and 4 are 1, 3, 2, and 4, in that order.

Computation is done in 80-bit precision on Intel platforms, and using Kahan-Babuska summation on ARM, to reduce round-off errors as much as possible.

The function returns exactly the same result as Mathematica's FourierDCT function does:

(DCT-I)

$$\sqrt{\frac{2}{n-1}}\left(\frac{u_1}{2}+\sum_{r=2}^{n-1} u_r \cos\left(\frac{\pi}{n-1}(r-1)(s-1)\right)+(-1)^{s-1}\frac{u_n}{2}\right)$$

(DCT-II)

$$\frac{1}{\sqrt{n}}\sum_{r=1}^{n} u_r \cos\left(\frac{\pi}{n}\left(r-\frac{1}{2}\right)(s-1)\right)$$

(DCT-III)

$$\frac{1}{\sqrt{n}}\left(u_1+2\sum_{r=2}^{n} u_r \cos\left(\frac{\pi}{n}(r-1)\left(s-\frac{1}{2}\right)\right)\right)$$

(DCT-IV)

$$\sqrt{\frac{2}{n}}\left(\sum_{r=1}^{n} u_r \cos\left(\frac{\pi}{n}\left(r-\frac{1}{2}\right)\left(s-\frac{1}{2}\right)\right)\right)$$

Image credit: Wolfram Research.

See also: **calc.dst**, **round**.

calc.diff (f, x [, ···] [, options])

Computes the value of the first derivative of a function `f` at a point `x`, and also returns the absolute error as a second return. The second, etc. arguments to `f` may be given right after argument `x`.

If the option `deriv`=n is given, where n may be 1, 2, 3, 4 or 5, the n-th derivative is calculated, with n = 1 the default. If n = 0, then the function value at `f`(`x`) is determined.
If the `eps`=h option is given, the epsilon value h (a positive number preferably close to zero) is used to determine the difference quotient; otherwise it is automatically determined by calling **math.epsilon** with `x`.

If the absolute error is quite large, it may either indicate non-differentiability of `f` at `x`, ···, or that the derivative could not be computed with sufficient precision.

The algorithm is based on Conte and de Boor's `Coefficients of Newton form of polynomial of degree 3`, and computes symmetric difference quotients.

For a function that automatically chooses the best differentiation method, see **calc.differ**. You might also try the **dual** package which supports automatic differentiation to compute the first, second and third derivative, see Chapter 11.7.

See also: **calc.cheby**, **calc.differ**, **calc.isdiff**, **calc.eulerdiff**, **calc.xpdiff**, **dual.diff**.

calc.differ (f, x [, ···] [, options])

Computes the derivative of a univariate or multivariate function `f` at point `x` (a number). The second, etc. arguments to `f` may be given right after argument `x`.

If the option `deriv`=n is given, where `n` may be 1, 2, 3, 4, or 5 the n-th derivative is calculated, with n = 1 the default. If n = 0, then the function value at `f`(`x`) is determined.

If the `eps`=h option is given, the epsilon value h (a positive number preferably close to zero) is used to determine the difference quotient; otherwise it is automatically determined by calling **math.epsilon** for `x`.

This function actually is just a simple wrapper to

- **calc.eulerdiff** if `deriv`=1 or no `deriv` option has been given,
- **calc.xpdiff** with `deriv` < 4 provided that `x` is near an undefined realm or a pole,
- **calc.diff** with `deriv` > 3 if `x` is near an undefined realm or a pole,
- **calc.cheby** otherwise,

thus automatically choosing the best method to compute the derivative. Examples:

```
> calc.differ( <: x -> ln x :>, 2):  # 1st derivative of ln(x) at x=2
0.5
```

which is equivalent to:

```
> calc.differ( <: x -> ln x :>, 2, deriv = 1):
0.5

> calc.differ( <: x -> ln x :>, 2, deriv = 2):  # 2nd derivative
-0.24999999999988       5.3327536830849e-011

> calc.differ( <: x -> ln x :>, 2, deriv = 3):  # 3rd derivative
0.24999999511369        7.8178753495771e-006
```

The `span` option gives control on the computation of Chebyshev coefficients should the logic decide to call **calc.cheby** and also controls the checks for undefined realms in the vicinity of the given point. For example, with the call

```
> calc.differ( <: x -> ln x :>, 2.5, span = 2):
```

and point x = 2.5 (the second argument) the function will compute Chebyshev coefficients over [x - span/2, x + span/2] = [1.5, 3.5] and will call **calc.diff** or **calc.xpdiff** instead of **calc.cheby** if x is less than `span` units away from any undefined realm. The default is `span` = 2.

When given the `poles` option, the function internally checks for poles before choosing and calling the best differentiating function for the given arguments (or better: situation), provided that the second or a higher-order derivative is to be computed. Since searching for poles consumes a lot of computation time, **calc.differ** by default does not do this check as the function is often called by **calc.extrema**, **calc.inflec**, **calc.saddles** to compute characteristic points over an interval. Example:

```
> calc.differ( <: x -> 1/x :>, 0.005, deriv = 2, poles = true):
-40000.000000002
```

The function is implemented in Agena and included in the lib/library.agn file.

You might also try the **dual** package which supports automatic differentiation to compute the first, second and third derivative, producing much better results, see Chapter 11.7.

calc.dilog (x)

Computes the dilogarithm (Spence's) function for a number `x`. The return is the number

$$\mathrm{Li}_2(x) = \sum_{k=1}^{\infty} \frac{z^k}{k^2}$$

See also: **calc.polylog**.

calc.dst (u [, m])

Finds the Fourier discrete sine transform (DST) of type `m` of a one-dimensional data vector `u`. By default, DST-I is computed, but you can do DST-II, DST-III and DST-IV by passing 2, 3 or 4 for `m`.

The inverse DSTs for types 1, 2, 3, and 4 are 1, 3, 2, and 4, in that order.

Computation is done in 80-bit precision on Intel platforms, and using Kahan-Babuska summation on ARM, to reduce round-off errors as much as possible.

The function returns exactly the same result as Mathematica's FourierDST function does:

(DST-I)

$$\sqrt{\frac{2}{n+1}}\ \sum_{r=1}^{n} u_r \sin\left(\frac{\pi}{n+1}\, r\, s\right)$$

(DST-II)

$$\frac{1}{\sqrt{n}}\ \sum_{r=1}^{n} u_r \sin\left(\frac{\pi}{n}\left(r-\frac{1}{2}\right) s\right)$$

(DST-III)

$$\frac{1}{\sqrt{n}}\left(2\sum_{r=1}^{n-1} u_r \sin\left(\frac{\pi}{n}\, r\left(s-\frac{1}{2}\right)\right)+(-1)^{s-1}\, u_n\right)$$

(DST-IV)

$$\sqrt{\frac{2}{n}}\ \sum_{r=1}^{n} u_r \sin\left(\frac{\pi}{n}\left(r-\frac{1}{2}\right)\left(s-\frac{1}{2}\right)\right)$$

Image credit: Wolfram Research.

See also: **calc.dct**, **round**.

calc.Ei ([n,] x)

With only number `x` given, computes the exponential integral as a Cauchy Principal Value integral:

$$\mathrm{Ei}(x) = -\int_{t=-x}^{\infty} \frac{e^{-t}}{t}\,dt$$

The return is a number[2], and **undefined** if `x` = 0.

With `n` a non-negative integer, it computes the generalized exponential integral:

$$\mathrm{Ei}(n, x) = \int_{t=1}^{\infty} \frac{e^{-x*t}}{t^n} dt$$

For the logarithmic integral Li, use **calc.Ei**(**ln** `x`).

See also: **calc.Ein**, **calc.Li**.

calc.Ein (x)

Computes the entire exponential integral

$$\mathrm{Ein}(x) = \int_{-\infty}^{x} \frac{1 - e^{-t}}{t} dt$$

for a number `x`. The return is a number. See also: **calc.Ei**.

calc.elliptic1 (m)

calc.elliptic1 (phi, m)

In the first form, the function approximates the complete elliptic integral of the first kind:

$$K(m) = \int_{0}^{\pi/2} \frac{1}{\sqrt{1-m\sin^2 t}} dt$$

for real `m`. The return is a number.

In the second form, the function approximates the incomplete elliptic integral of the first kind:

$$F(\mathrm{phi}, m) = \int_{0}^{\mathrm{phi}} \frac{1}{\sqrt{1-m\sin^2 t}} dt$$

with amplitude `phi` and modulus `m` both of type number. The return is a number.

See also: **calc.elliptic2**, **calc.jocobian**.

[2] Please note that for -5 ⩽ x < 0, the result is an approximation.

calc.elliptic2 (m)

calc.elliptic2 (phi, m)

In the first form, the function approximates the complete elliptic integral of the second kind:

$$K(m) = \int_0^{\pi/2} \sqrt{1 - m\sin^2 t}\ dt$$

for real `m`. The return is a number.

In the second form, the function approximates the incomplete elliptic integral of the second kind:

$$F(\mathtt{phi}, m) = \int_0^{\mathtt{phi}} \sqrt{1 - m\sin^2 t}\ dt$$

with amplitude `phi` and modulus `m` both of type number. The return is a number.

See also: **calc.elliptic1**, **calc.jocobian**.

calc.En (n, x)

Evaluates the exponential integral

$$E_n(x) = -\int_1^{\infty} \frac{e^{-xt}}{t^n} dt$$

for non-negative `n` (an integer) and real `x`. The return is a number.

calc.eps (f, x [, ···] [, options])

Computes a mathematical epsilon value that takes into account the magnitude of the value of function `f` at point `x`. `f` may be univariate or multivariate. The second, etc. arguments to `f` may be given right after argument `x`.

By default, r = 3 points around (and including) the centre `x` are used to calculate the result. You may choose any other positive integer for r with r < 100 by passing the `samples` = r option, see example below.

By default, the function does not adjust the result if the internal epsilon value computed has become too large to be probably of any use. You can override this by passing the explicit `adjust` = **true** option, e.g.:

```
> calc.eps(<: x -> ln x :>, 1, adjust = true, samples = 5):
1.4901161193848e-06     true
```

The function is useful to analyse functions whose values are very close to zero and where floating point arithmetic may lead to fatal round-off errors as it returns an epsilon value that is larger than the one returned by **math.eps**.

See also: **math.eps**, **math.epsilon**.

calc.eta (n)

For non-negative real `n`, computes the Dirichlet Eta function:

$$\eta(n) = \sum_{1}^{\infty} (-1)^{k-1} / \, k^{n}$$

See also: **calc.zeta**.

calc.euclliddist (f, a, b [, ···])

Computes the Euclidian distance, i.e. the straight-line distance, of two points (`a`, `f`(`a`)) and (`b`, f(`b`)) on a curve defined by a function `f` in one real, in the Euclidean plane. `a`, `b` must be numbers.

If `f` is multivariate, its second, third, etc. argument are passed after `b`.

See also: **|-** operator, **hypot**, **calc.sinuosity**.

calc.eulerdiff (f, x [, ···] [, options])

Computes the first or second derivative of the univariate or multivariate function `f` at real point `x`, a number.

If the option `eps`=h is given, the epsilon value h (a positive number preferably close to zero) is used internally for the computation, its default is **math.epsilon**(`x`).

The second, etc. arguments to `f` may be given right after argument `x`.

The return is the imaginary part of `f`(`x` + I*h)/h, or **fail** if `f` did not evaluate to the complex plane. The function does not check whether `f` is differentiable at `x`, ···, you may call **calc.isdiff** before.

If the `deriv`=2 option is passed, then the second derivative is determined. The quality of the second derivative is close to, but not as good as, the one of **calc.xpdiff**. (The first derivatives of **eulerdiff** are still better than those of **xpdiff**.)

To compute the first and second derivative of ln(`x`) at `x` = 2, enter:

```
> calc.eulerdiff( <: x -> ln x :>, 2):
0.5
```

```
> calc.eulerdiff( <: x -> ln x :>, 2, deriv=2):
-0.2499994448875
```

This function as at least three times faster than **calc.xpdiff**. See also: **calc.diff**.

The idea has been taken from the Euler Math Toolbox, thus its name.

For a function that automatically chooses the best differentiation method, see **calc.differ**.

You might also try the **dual** package which supports automatic differentiation to compute the first, second and third derivative, producing much better results, see Chapter 11.7.

See also: **calc.differ**, **calc.diff**, **calc.xpdiff**, **dual.diff**.

calc.expn (n, x)

Implements the exponential sum function e_n(x), sometimes also denoted exp_n(x):

$$\sum_{k=0}^{n} \frac{x^k}{k!}$$

calc.extrema (f, a, b, [, ···] [, options])

Returns all minimum and maximum locations of the univariate or multivariate function `f` on the interval [`a`, `b`], with `a`, `b` real numbers. `f` should be differentiable on [`a`, `b`].

The first sequence returned includes all local and validated minima, the second one all local validated maxima. So contrary to **calc.minimum** and **calc.maximum** the user does not need to check proposed extrema for existence afterwards. Also, the function with its defaults works much better with highly oscillating functions than **calc.minimum** and **calc.maximum**.

The function actually is an interface to **calc.zeros** and **calc.differ** and supports the following options:

- `eps`: Accuracy of the root-finding method for the first and second derivative, defaulting to **Eps**.
- `step`: Length of the subintervals to be examined, the `step size`. Defaults to 0.1.
- `adaptive`: When given a number n along with this option, conducts n additional divisions of all the sub-intervals to be checked for extrema. Default is n = 0, that is no additional divisions.
- `finder`: The extrema-finding function. Available functions are **calc.zeroin** (the default), **calc.zeroab** or **calc.regulafalsi**, **calc.chandrupatla**, **calc.itp** and **calc.brent**.

- `span`: Control on the computation of Chebyshev coefficients, see **calc.differ** for further details.

See also: **calc.inflect**, **calc.saddle**, **calc.zeros**.

calc.fmaxbr (f, a, b [, ···] [, option])

Estimates the maximum location of a univariate or multivariate function `f` through one-dimensional search over a given range [`a`, `b`], with `a`, `b` numbers, using Golden section search combined with parabolic interpolation. Returns the *abscissa* (x-axis) value where a minimum has been found, a number.

For more information on how to use it, see **calc.fminbr**. See also: **calc.maximum**.

calc.fmaxgs (f, a, b [, ···] [, option])

Like **calc.fmings**, but determines maximum locations. See also: **calc.minimum**.

calc.fminbr (f, a, b [, ···] [, option])

Estimates the minimum location of a univariate or multivariate function `f` through one-dimensional search over a given range [`a`, `b`], with `a`, `b` numbers, using Golden section search combined with parabolic interpolation. Returns the *abscissa* (x-axis) value where a minimum has been found, a number.

The second, etc. arguments to `f` may be given right after argument `b`. The acceptable tolerance defaults to **Eps** and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number, e.g.:

```
> calc.fminbr(<: x -> x**2 :>, -1, 1, eps = DoubleEps):
0
```

If there are multiple minima in the range, the function returns an arbitrary one. This function is rather basic, and the function does not check whether the function is defined at the extremum. See **calc.minimum** and **calc.maximum** for alternatives.

The function uses 80-bit floating-point precision internally.

See also: **calc.fmings**, **ival** package.

calc.fmings (f, a, b [, ···] [, option])

Like **calc.fminbr**, but performs Golden section search only.

Given a univariate or multivariate function `f` with a single local minimum in the interval [`a`, `b`], with `a`, `b` numbers, returns the *abscissa* value (x-axis) where the minimum has been found.

The second, etc. arguments to `f` may be given right after argument `b`. The acceptable tolerance defaults to **Eps** and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number, see **calc.fminbr** for an example.

The function uses 80-bit floating-point precision internally.

calc.fprod (f, a, b [, ···])

Computes the product of `f`(`a`), ···, `f`(`b`), with `f` a function, `a` and `b` numbers. If `f` requires two or more arguments, the second, third, etc. argument must be passed after `b`. If `a` > `b`, then the result is 1. The function accepts complex numbers as arguments of `f`; likewise `f` may compute in the complex domain.

See also: **calc.fsum**, **mulup**.

calc.fresnelc (x)

Computes the Fresnel integral $C(x) = \int_0^x \cos(\frac{\pi}{2} t^2)\, dt$ and returns it as a number.

calc.fresnels (x)

Computes the Fresnel integral $S(x) = \int_0^x \sin(\frac{\pi}{2} t^2)\, dt$ and returns it as a number.

calc.fsum (f, a, b [, ···])

Computes the sum of `f`(`a`, ···) .. `f`(`b`, ···), with `f` a function, `a` and `b` numbers. If `f` requires two or more arguments, the second, third, etc. argument must be passed after `b`. If `a` > `b`, then the result is 0. The function uses Kahan-Babuška round-off error prevention. The function accepts complex numbers as arguments of `f`; likewise, `f` may compute in the complex domain. Examples:

```
> calc.fsum(<: n, x -> (x**n)/fact(n) :>, 0, 100, 1):
2.718281828459

> calc.fsum(<: x, n -> (x**n)/fact(n) :>, 0, 100, 1):
5050
```

See also: **foreach**, **mulup**, **qsumup**, **sumup**, **calc.fprod**, **stats.cumsum**, **stats.fsum**.

calc.gammainc (a, x [, option])

Computes the upper (default)

$$\Gamma(a,x) = \int_{t=x}^{\infty} t^{a-1} e^{-t} dt$$

or lower incomplete gamma function (if any `option` is given) for non-negative argument `x` and non-negative parameter `a`:

$$\gamma(a,x) = \int_{t=0}^{x} t^{a-1}e^{-t}dt$$

With no `option` given, the result is the same as that returned by **gamma**.

calc.gauleg (f, a , b [, ···] [, options])

Performs Gauss-Legendre integration of a real univariate or multivariate function `f` over the interval [`a`, `b`].

The function accepts the following options:

- `eps`, the epsilon value to be used internally, defaults to **Eps**[2].
- `samples`, number of sample points over the whole range; defaults to **int**(`b` - `a`)*20 if `a` - `b` > 1 and to 20 sample points if `a` - `b` ≤ 1.

If `f` requires two or more arguments, the second, third, etc. argument must be passed after `eps`. The function is seven times faster than **calc.integ** with similar precision. It uses 80-bit floating-point precision internally.

Example:

```
> calc.gauleg(<: x -> x :>, 0, 1, eps=DoubleEps, samples=10):
0.5
```

See also: **calc.gauleg64**, **calc.gtrap**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.romberg**, **calc.simaptive**.

calc.gauleg64 (f, a , b [, ···] [, options])

Like **calc.gauleg**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.gauleg** as they do not have 80-bit arithmetic.

calc.gaussian (x [, a [, b [, c]]])

Computes the Gaussian function $\frac{a}{2c^2}\,e^{-(x-b)\hat{}2}$ at a real or complex point `x`, with `a`, `b`, `c` being (real) numbers. By default, `a` = 1, `b` = 0, `c` = $\frac{1}{\sqrt{2}}$. The return depends on the type of `x`.

See also: **expx2**.

calc.gd (x)

Computes Gudermannian function for any number or complex number `x`, i.e.

$$\mathrm{gd}(x) = \int_{t=0}^{x} \frac{1}{\cosh(t)}\,dt = \arctan(\sinh(x)).$$

The type of return depends on x.

Wikipedia: `The Gudermann function relates circular functions and hyperbolic functions without explicitly using complex numbers.` The function is written in the Agena language.

calc.gtrap (f, a, b [, ···] [, option])

Integrates the univariate or multivariate function `f` on the interval [`a`, `b`] using a bisection method based on the trapezoid rule and returns a number.

The second, etc. arguments to `f` may be given right after argument `b`.

By default the function quits after an accuracy of **Eps** has been reached. You may change this by passing the option `eps` = *v*, where *v* is a positive number, for example:

```
> calc.gtrap(<: x -> x :>, 0, 1, eps = DoubleEps):
0.5
```

The function returns **fail** if the algorithm did not converge.

The algorithm is rather slow and imprecise with larger `eps`. It uses 80-bit floating-point precision internally, though.

See also: **calc.gauleg**, **calc.gtrap64**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.romberg**, **calc.simaptive**.

calc.gtrap64 (f, a, b [, ···] [, option])

Like **calc.gtrap**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.gtrap** as they do not have 80-bit arithmetic.

calc.harmonic (x)

Computes the Harmonic function for both real and complex `x` and returns

psi(`x` + 1) + **EulerGamma**.

calc.horner (x, p)

Takes a point `x` and a table, sequence, register or numarray `p` of coefficients in ascending order of power by default and evaluates

$$p[1] + x * (p[2] + x * (....)),$$

i.e. a polynomial via Horner's rule. If the third optional argument is **false**, the coefficients in `p` are in descending order. The function internally uses 80-bit floating-point precision when available for better results.

Example: To evaluate the polynomial $5x^3 + 3x^2 + 2x + 1$ at $x = 2$, issue:

```
> calc.horner(2, [1, 2, 3, 5]):
57

> calc.horner(2, [5, 3, 2, 1], false):
57
```

calc.hyp1f1 (a, b, z)

Computes the confluent hypergeometric function 1F1(`a`, `b`; `z`) aka Kummer's function of the first kind. `a`, `b`, `z` are expected to be numbers. In case of an invalid argument, the function may return **undefined**, otherwise the return is a number. The accuracy is quite good if $0 \leq a < b$, with an average error of about 1e-8, otherwise the error can be up to 1e-2.

calc.hyp2f1 (a, b, c, z)

Computes the Gaussian or ordinary hypergeometric function 2F1(`a`, `b`; `c`; `z`). `a`, `b`, `c`, `z` are expected to be numbers. In case of an invalid argument, the function mostly returns **infinity**, otherwise the return is a number.

calc.ibeta (x, a, b)

Evaluates the incomplete beta integral defined by

$$\frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^x t^{a-1}(1-t)^{b-1} dt$$

from 0 to `x`. Both `a` and `x` must be positive numbers. See also: **calc.invibeta**.

calc.igamma (x, a)

Evaluates the incomplete gamma integral defined by

$$\frac{1}{\Gamma(a)}\int_0^x e^{-t}\, t^{a-1} dt$$

Both `a` and `x` must be positive numbers. See also: **calc.igammac**.

calc.igammac (x, a)

Evaluates the complemented incomplete gamma integral defined by

$$\frac{1}{\Gamma(a)}\int_x^\infty e^{-t}\, t^{a-1} dt$$

Both `a` and `x` must be positive numbers. See also: **calc.igamma**.

calc.inflect (f, a, b, [, ···] [, options])

Returns all points of inflection of the univariate or multivariate function `f` on the interval [`a`, `b`], with `a`, `b` real numbers, in a sequence.

`f` should be differentiable on [`a`, `b`].

The function actually is a wrapper to **calc.extrema** which by itself is an interface to **calc.zeros** and **calc.differ** and supports the following options:

- `eps`: Accuracy of the root-finding method for the first and second derivative, defaulting to **Eps**.
- `step`: Length of the subintervals to be examined, the `step size`. Defaults to 0.1.
- `adaptive`: When given a number n along with this option, conducts n additional divisions of all the sub-intervals to be checked for extrema. Default is n = 0, that is no additional divisions.
- `finder`: The extrema-finding function. Available functions are **calc.zeroin** (the default), **calc.zeroab** or **calc.regulafalsi**, **calc.chandrupatla**, **calc.itp** and **calc.brent**.
- `span`: Control on the computation of Chebyshev coefficients, see **calc.differ** for further details.

See also: **calc.extrema**, **calc.saddle**, **calc.zeros**.

calc.intcc (f, a, b [, ···] [, option])

Integrates the univariate or multivariate function `f` on the interval [`a`, `b`], with `a` and `b` numbers, using Clenshaw-Curtis-Quadrature (CC) which is much faster than Double Exponential (DE) Transformation.

`f` needs to be analytic over [`a`, `b`].

The second, etc. arguments to `f` may be given right after argument `b`. The relative error requested excluding cancellation of significant digits defaults to 1e-15 and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number, e.g.:

```
> calc.intcc(<: x, a -> x^a :>, 0, 1, 2, eps=DoubleEps):
0.33333333333333        1.6653345369377e-016
```

The function returns **undefined** if the algorithm did not converge. It uses 80-bit floating-point precision internally.

For further information on the result, see **calc.intde**. See also: **calc.gauleg**, **calc.gtrap**, **calc.intcc64**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.riesum**, **calc.romberg**, **calc.simaptive**.

calc.intcc64 (f, a, b [, ···] [, option])

Like **calc.intcc**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.intcc** as they do not have 80-bit arithmetic.

calc.intde (f, a, b [, ···] [, option])

Integrates the univariate or multivariate function `f` on the interval [`a`, `b`], with `a` and `b` numbers, using Double Exponential (DE) Transformation, also known as Tanh-sinh quadrature.

`f` needs to be analytic over [`a`, `b`]. If `f` is a multivariate function, then you may pass the second, third, etc. arguments right after `b`.

The relative error requested excluding cancellation of significant digits defaults to 1e-15. Specifically, `eps` means: (absolute error) / ($\int_a^b |f(x)|dx$). It may be changed by passing the option `eps` = *tol*, where *tol* is a positive number.

The return is 1) the approximation to the integral, or **undefined** if evaluation failed, and 2) an estimate err of the absolute error, where

- err ≥ 0: normal termination,
- err < 0: abnormal termination, i.e. an convergent error has been detected: 1) f(x) or $\frac{d}{dx}^n$ f(x) has discontinuous points or sharp peaks over [`a`, `b`] (you must divide

the interval [a, b] at these points). 2) The relative error of f(x) is greater than eps. 3) f(x) has an oscillatory factor and the frequency of the oscillation is very high.

Example:

```
> calc.intde(<: x, a -> x^a :>, 0, 1, 2, eps=DoubleEps):
0.33333333333333        1.4802973660321e-016
```

The function returns **fail** if the algorithm did not converge.

This function is four times faster than **calc.gtrap** and also much more accurate. It works with any polynomial, exponential or trigonometric function, logarithm and most special functions. It uses 80-bit floating-point precision internally.

See also: **calc.gauleg**, **calc.gtrap**, **calc.intcc**, **calc.intde64**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.riesum**, **calc.romberg**, **calc.simaptive**.

calc.intde64 (f, a, b [, ···] [, option])

Like **calc.intde**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.intde** as they do not have 80-bit arithmetic.

calc.intdei (f, a, [, ···] [, option])

Integrates *the non-oscillatory* univariate or multivariate function f on the interval [a, ∞], with a a number, using Double Exponential (DE) Transformation, also known as Tanh-sinh quadrature.

f needs to be analytic over [a, ∞]. If f is a multivariate function, then you may pass the second, third, etc. arguments right after a.

The relative error requested excluding cancellation of significant digits defaults to 1e-15. Specifically, eps means: (absolute error) / ($\int_a^b |f(x)|dx$). It may be changed by passing the option eps = *tol*, where *tol* is a positive number.

The return is either the approximation to the integral, or **undefined** if evaluation failed, and an estimate err of the absolute error. For further information see **calc.intde**.

Example:

```
> calc.intdei(<: x, a -> 1/(x + a)^2 :>, 1, 2, eps=DoubleEps):
0.33333333333333        1.4802973662954e-016
```

The function uses 80-bit floating-point precision internally.

See also: **calc.gtrap**, **calc.intcc**, **calc.intde**, **calc.intdei64**, **calc.intdeo**, **calc.integ**, **calc.riesum**, **calc.romberg**, **calc.simaptive**.

calc.intdei64 (f, a, [, ···] [, option])

Like **calc.intdei**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.intdei** as they do not have 80-bit arithmetic.

calc.intdeo (f, a, [, ···] [, options])

Integrates *the oscillatory* univariate or multivariate function `f` on the interval [`a`, ∞], with `a` a number, using Double Exponential (DE) Transformation, also known as Tanh-sinh quadrature.

`f` needs to be analytic over [`a`, ∞]. If `f` is a multivariate function, then you may pass the second, third, etc. arguments right after `a`.

The function accepts the following options:

- `omega`, the oscillatory non-zero factor of `f`, defaulting to 1.
- `eps`, the relative error requested excluding cancellation of significant digits, defaulting to 1e-15. Specifically, `eps` means: (absolute error)/($\int_a^b |f(x)|dx$).

The return is either the approximation to the integral, or **undefined** if evaluation failed, and an estimate err of the absolute error. For further information see **calc.intde**.

Example:

```
> calc.intdeo(<: x, a -> sin(x)/(x + a) :>, 1, 0, omega=1, eps=Eps):
0.62471325639277        8.3556975456747e-008
```

The function returns **fail** if the algorithm did not converge.

The function uses 80-bit floating-point precision internally.

See also: **calc.gtrap**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo64**, **calc.integ**, **calc.riesum**, **calc.romberg**, **calc.simaptive**.

calc.intdeo64 (f, a, [, ···] [, option])

Like **calc.intdeo**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.intdeo** as they do not have 80-bit arithmetic.

calc.integ (f, a, b [, ···] [, option])

This function is a wrapper around **calc.intde**, **calc.intdei**, **calc.intdeo** and **calc.gauleg**.

If `f` is a multivariate function, then its second, third, etc. arguments must be given after `b`.

The function accepts the following options:

- `eps`, the epsilon value to be used internally, defaults to DoubleEps.
- `omega`, oscillatory factor of `f`, defaults to 1.
- `samples`, number of sample points over the whole range; defaults to **int**(`b - a`)*20 if `a - b` > 1 and to 20 sample points if `a - b` ≤ 1.
- `poles`, to check whether `f` has poles in the given range [`a, b`]. If **true**, splits it up accordingly, summing up the various areas to return a finite result, if possible. The default is **false** as this feature not always returns a correct result.

The return is the integral value and the error margin, both are numbers. Example:

```
> calc.integ(<: x -> x :>, 0, 1, eps = DoubleEps, samples = 10):
0.5     2.2204460491159e-016
```

If `b` is not **infinity**, the function first calls **calc.intde** and returns its results, if **intde** does not evaluate to **undefined** and if the relative error is non-negative. If **calc.intde** returns **undefined**, indicating a pole in over [`a, b`], it searches for that pole, splits up the interval and calls **calc.integ** recursively over the sub-intervals, summing up the results. Otherwise, **calc.gauleg** will be called.

If `b` is **infinity**, the function first calls **calc.intdei** - assuming that `f` is non-oscillatory - and returns its results, if **intdei** does not evaluate to **undefined** and if the relative error is non-negative. Otherwise, **calc.intdeo** will be called, assuming `f` is oscillatory.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.gauleg**, **calc.gtrap**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.riesum**, **calc.romberg**, **calc.simaptive**.

calc.interp (obj)

calc.interp (obj, a)

calc.interp (obj, a, coeffs)

In the first form, computes a Newton interpolating polynomial and returns it as a univariate function. The interpolation points are passed in a table `obj`, with each point being represented by the pair x_k`:`y_k.

Example:

```
> f := calc.interp([ 0:0, 1:3, 2:1, 3:3 ]);
```

Call f at point 10:

```
> f(10):
885
```

In the second and third form, it evaluates the Newton form of the polynomial which interpolates a given table or sequence `obj` of pairs representing the points x_k:y_k, at a single value `a` (a number) of the independent variable.

In the second form, the function computes the coefficients automatically in each call, which slows down this variant.

In the third form, by passing a sequence `coeffs` of coefficients (numbers), the function uses the coefficients passed, avoiding their (re-)computation. The third argument may be obtained by calling **calc.newtoncoeffs**.

Both in second and third form, the function returns the value of the interpolating polynomial, a number, at the specified value `a` of the independent variable. It is advised to use the first form to benefit from maximum speed. Example:

```
> calc.interp([ 0:0, 1:3, 2:1, 3:3 ], 10):
885
```

See also: **calc.cheby**, **calc.clampedspline**, **calc.linterp**, **calc.nakspline**, **calc.neville**, **calc.newtoncoeffs**, **calc.polyfit**, **calc.savgol**.

calc.invibeta (y, a, b)

Evaluates the inverse of the incomplete beta integral such that

y = **calc.ibeta**(x, `a`, `b`).

See also: **calc.ibeta**.

calc.iscont (f, x [, ···] [, option])

The function returns **true** if a real function `f` is continuous at the given point, and **false** otherwise. If `f` requires only one argument, `x` is a number. If `f` requires two or more arguments, the second, third, etc. argument of `f` must be passed right after `x`.

If the option `eps`=h is given as the last argument, the epsilon value h (a positive number preferably close to zero) is used for the approximate equality check with the left and right limit; otherwise it is automatically determined by calling **math.eps** with `x` and any option given. See **calc.limit** for an example.

See also: **calc.isdiff**, **calc.poles**.

calc.isdiff (f, x [, ···] [, option])

The function returns **true** if a real function `f` is differentiable at the given point `x`, of type number, and **false** otherwise. If `f` requires two or more arguments, the second, third, etc. argument (all of type number) - of `f` must be passed right after `x`.

If the option `eps`=h is given as the last argument, the epsilon value h (a positive number preferably close to zero) is used for the approximate equality check with the left and right limit; otherwise it is set to **Eps**. If the difference between the left- and right-sided difference quotients is greater than epsilon, it is taken to the square root, to be more adaptive to functions where the graph is steep around `x`. See **calc.limit** for an example.

The second return is the maximum error value, that is **Eps**, h if the `eps` option has been given, or their square root.

A function is differentiable at `x`, `···` if it is continuous at `x` and if the left- and right-sided difference quotients are equal.

Note that the function may produce wrong results around poles. Example:

```
> calc.isdiff(<: x -> x :>, 0):
true    1.4901161193848e-008

> calc.isdiff(<: x -> 1/x :>, 0, eps = DoubleEps):
false
```

See also: **calc.diff**, **calc.differ**, **calc.iscont**, **calc.poles**, **calc.xpdiff**.

calc.itp (f, a, b, n [, ···] [, options])

The function seeks a zero or a pole of a univariate or multivariate function `f` over the interval [`a`, `b`] using using the ITP algorithm. With multivariate `f` put its second, third etc. argument right after `b`.

There are some options with which you can control accuracy:

- `eps`=<positive number>' : the error tolerance between exact and computed roots, set to **DoubleEps** by default (**hEps** on ARM),
- `k1`: a parameter, with suggested value 0.2/(`b` - `a`) and set to this formula if not given,
- `k2`: a parameter, typically set to 2,
- `n0`: a parameter that can be set to 0 for difficult problems, but by default set to 1, to take more advantage of the secant method,
- `iters`=<positive integer>: maximum number of iterations the function takes until finishing the evaluation, with the 125 the default.

Note that you have to check separately whether the return is a zero or a pole. The function returns **null** if f(a, ...), f(b, ...) have differing signs.

From Wikipedia: "The ITP method, short for Interpolate Truncate and Project, is the first root-finding algorithm that achieves the superlinear convergence of the secant method while retaining the optimal worst-case performance of the bisection method. It is also the first method with guaranteed average performance strictly better than the bisection method under any continuous distribution."

Examples, first a pole and then a zero:

```
> calc.itp(<: x -> tan x :>, 1, 2, eps=Eps, n0=0):
1.5707963258028

> calc.itp(<: x -> tan x :>, 1, 2):
1.5707963267949

> Pi/2:
1.5707963267949

> calc.itp(<: x -> sin x :>, -1, 1):
0
```

The function is around 50 percent slower than the other root-finders. Since the function finds only one zero, you can pass it to the **calc.zeros** wrapper to get all the roots over a larger interval, for example:

```
> calc.zeros(<: x -> sin x :>, -1000, 1000, finder = calc.chandrupatla):
```

Tip: The function cannot find poles of function f if there is no change of sign in its vicinity. In this case pass the first derivative of f, see **calc.differ**.

See also: **calc.chandrupatla**, **calc.poles**, **calc.regulafalsi**, **calc.sections**, **calc.zeroab**, **calc.zeros**.

calc.jacobian (u, m)

Computes the Jacobian elliptic functions sn(u, m), cn(u, m) and dn(u, m) of real parameter m between 0 and 1, and real argument u, in this order and also returns phi, the amplitude of u, as a fourth result.

The relation to the incomplete elliptic integral is as follows: If u = **calc.elliptic1**(phi, m), then sn(u, m) = **sin**(phi), and cn(u, m) = **cos**(phi), with phi the amplitude of u.

See also: **calc.elliptic1**, **calc.elliptic2**.

calc.lambda (v, x [, eps])

Computes the Lambda function and its derivative of (positive) integral order `v` for argument `x`:

$$\Lambda_v(x) = 2^v\Gamma(v+1)$$
$$\Lambda'_v(x) = \frac{2v}{x}(\Lambda_{v-1}(x) - \Lambda_v(x))$$

Γ is the Gamma function, and J_v is the Bessel function of the first kind (Agena function **besselj**). The function also returns the actual order processed, which may differ from the input `v`. `eps` is a bailout value and by default is **DoubleEps**.

calc.lambertw (x [, branch [, offset]])

Approximates the Lambert W function for number `x`. It satisfies **calc.lambertw**(`x`) * **exp**(**calc.lambertw**(`x`)) = `x`. By default the upper branch is taken, pass a non-zero number for the lower `branch`.

If `offset` is 1, `x` will represent the offset from $-e^{-1}$, so the function computes **calc.lambertw**(`x` - **exp**(-1)). The default is zero, that is no offset.

The range requirement for the upper branch is: $-e^{-1} \leq = x$.

The range requirement for the lower branch is: $-e^{-1} < x < 0$.

calc.Li (x)

Computes the logarithmic integral calc.Li(`x`) = **calc.Ei**(**ln**(`x`)). It provides an approximation to the number of primes less than `x` > 1. See also: **calc.Ei**.

calc.limit (f, x [, ···] [, options])

The function returns the limit, a number, of a real function `f` at the given point `x` (a number). If the limit does not exist, **undefined** will be returned.

If `f` is multivariate, the second, third, etc. argument of `f` must be passed right after `x`.

Options may be given as the very last arguments, their order does not matter.

If the `eps`=h option is given, the epsilon value h (a positive number preferably close to zero) is used for the approximate equality check of the left and right limit; otherwise it is automatically determined by calling **math.eps** with `x` and any option.

If the `side='left'` option is given, the left-sided limit is determined. If the `side='right'` option is given, the right-sided limit is determined.

If the `side='both'` option is given, the left and right-sided limit, in this order, will be returned.

If the `side='all'` option is given, the limit, the left-sided, and the right-sided limit will be returned, in this order.

For example, if the function is f(x, y) := |x| + y, with x = 1, y = 3, and eps = 1e-4, the call for the left-sided limit would be:

```
> calc.limit(<: x, y -> abs(x) + y :>, 0, 3, eps = 1e-4, side='left'):
3
```

calc.linterp (obj)

Returns a function that conducts a Lagrange interpolation for a given sequence or table `obj` of numeric pairs x:y where x and y denote a point in the plane. It is often said that Lagrange interpolation is suited for theoretical purposes only, since it is rather slow.

See also: **calc.interp**, **calc.polyfit**.

calc.logistic (x [, max [, k [, x0]]])

Computes the logistic function, having a characteristic `S`-shaped curve or sigmoid curve, for any number `x`, according to the formula

$$L(x) = \frac{max}{1 + e^{-k\,(x - x_0)}},$$

where `max` is the curve's maximum, `k` its steepness and `x0` the x-value of the sigmoid's midpoint. By default, `max` = 1, `k` = 1 and `x0` = 0, thus computing the sigmoid function. If only `x` is given, the function works like **calc.sigmoid**.

The result is a number between - but excluding - 0 and 1.

See also: **calc.gd**, **calc.logit**, **calc.sigmoid**.

calc.logit (x)

Computes the inverse sigmoid or logistic function according to the formula

$$\text{logit}(x) = \ln\left(\frac{x}{1+x}\right) = 2\ \text{arctanh}(2x - 1).$$

If `x`=0, the function will return -**infinity**, and if `x`=1 the result will be +**infinity**.

See also: **calc.logistic**, **calc.probit**, **calc.sigmoid**.

calc.maximum (f, a, b, [, ···] [, options])

Returns all *possible* maximum locations of the univariate or multivariate function `f` on the interval [`a`, `b`]. Works like **calc.minimum**, see there for details.

calc.mean (f, a, b, [, ···] [, option])

Computes the mean of a univariate or multivariate function `f`, that is the average value of `f` over an interval [`a`, `b`].

The result is 1/(`b` - `a`)***calc.intcc**(`f`, `a`, `b`). With `a` = `b`, returns **undefined**. For all the supported options and algorithm used, supported, see **calc.intcc**.

The function uses 80-bit floating-point precision internally.

See also: **calc.integ**, **calc.intde**, **calc.mean64**.

calc.mean64 (f, a, b, [, ···] [, option])

Like **calc.mean**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.mean** as they do not have 80-bit arithmetic.

calc.minimum (f, a, b, [, ···] [, options])

Returns all minimum locations of the univariate or multivariate function `f` on the interval [`a`, `b`], with `a`, `b` real numbers. `f` must not necessarily be differentiable on the given interval. The function by default internally calls **calc.extrema** to fetch the results, in this case the minima, and returns them in one sequence.

If the option `classic` = **true** is given, then the function returns all *approximations* of *possible* minimum locations of the univariate or multivariate function `f` on the interval [`a`, `b`], with `a`, `b` real numbers. The estimate is performed through one-dimensional search over given ranges using "Golden section" search combined with parabolic interpolation. With the `classic` option set to **true**, the function accepts the following options:

- `eps`: accuracy of the extrema-finding method, defaulting to **Eps**.
- `step`: length of the subintervals to be examined, `step size`. Defaults to 0.1.

The function divides the interval [`a`, `b`] into smaller intervals [`a`, `a`+step], [`a`+step, `a`+2*step], ···, [`b`-step, `b`]. It then looks for possible minimum locations `x` in these smaller intervals and checks whether the first derivative of `f` at `x` is 0.

The procedure returns two sequences:

If a possible extreme location `x` matches the condition `f'(x)` = 0 with eps accuracy, it is included in the first sequence that the procedure returns. If the test fails and eps ≤ **Eps**, then an accuracy of 1e-5 is used for a second test. If it

succeeds, `x` is included into both the first and the second sequence, indicating to the user that the first test failed.

The `adaptive` option allows to better cope with arithmetic functions that are highly-oscillating over an interval. When given a number n along with this option, **calc.minimum** conducts n additional divisions of all the sub-intervals to be checked for extrema.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.extrema**, **calc.fminbr**, **calc.fmings**, **calc.maximum**, **ival** package.

calc.naksplinе (obj)

calc.naksplinе (obj, a)

calc.naksplinе (obj, a, coeffs)

Evaluates the `not-a-knot` cubic spline for a given table or sequence `obj` of pairs representing the points x_k:y_k, at a single value `a` (a number) of the independent variable.

In the first form, returns a univariate function which can be called with a number to obtain the value of the interpolating polynomial. This is the recommended usage due to its run-time behaviour.

In the second form, the function computes the coefficients of the linear, quadratic, and cubic terms itself in each call.

In the third form, the function expects the coefficients `coeffs` of the linear, quadratic, and cubic terms as a sequence of three sequences, in this order, and each containing numbers. The third argument may be obtained by calling **calc.naksplinecoeffs**.

In the second and third form, the function returns the value of the interpolating polynomial, a number, at the specified value `a` of the independent variable.

In general, the function returns **fail** if the structure contains less than four pairs.

See also: **calc.clampedspline**, **calc.interp**, **calc.naksplinecoeffs**, **calc.neville**.

calc.naksplinecoeffs (obj)

Determines the coefficients for the `not-a-knot` cubic spline for a given table or sequence `obj` of pairs representing the points x_k:y_k. The return can be used to speed up execution of **calc.nakspline**.

The function returns **fail** if the structure contains less than four pairs.

See also: **calc.nakspline**.

calc.neville (obj)

calc.neville (obj, a)

In the first form, returns a function that conducts an Aitken-Neville interpolation for a given sequence or table `obj` of numeric pairs x_k:y_k where x_k and y_k denote a point in the plane.

In the second form, evaluates the polynomial which interpolates a given sequence or table `obj` of points represented by pairs of the form x_k:y_k at a single value `a` (a number) of the independent variable, using Aitken-Neville interpolation, and returns a number. Example:

```
> calc.neville([1:1, 2:2, 3:3], 2):
2
```

See also: **calc.clampedspline**, **calc.interp**, **calc.nakspline**.

calc.newtoncoeffs (obj)

Returns a sequence of the coefficients of type number of the Newton form of the polynomial which interpolates a given table or sequence `obj` of pairs representing the points x_k:y_k. The return can be used to speed up execution of **calc.interp**.

See also: **calc.interp**.

calc.poles (f, a, b [, ···] [, classic = true])

Returns all poles of the univariate or multivariate function `f` on the interval [`a`, `b`]. The function internally calls **calc.zeros** with the poles option set to **true** and using **calc.itp** for the evaluation, both with the original function `f` and its first derivative.

The function returns **null** if could not find a pole.

If you pass the `` `classic = **true**` `` option, the function uses a faster pole-finder that might be less accurate, though. Default is **false**.

The function is implemented in Agena and included in the lib/library.agn file.

Example:

```
> calc.poles(<: x -> tan(x) :>, -5, 5, classic = true):
seq(-4.7123889803847, -1.5707963267949, 1.5707963267949, 4.7123889803847)
```

See also: **calc.iscont**, **calc.isdiff**.

calc.polyfit (obj, n)

For an n-th-degree polynomial of a sample of Cartesian pairs x_k:y_k, returns a sequence of coefficients of descending degree, using polynomial regression. x_k, y_k are numbers and degree n must be a positive integer. Example:

```
> coeffs := calc.polyfit(seq( 1:0, 2:3, 3:1 ), 2):
seq(-2.5, 10.5, -8.0000000000001)
```

The return may be passed to **calc.polygen** to generate a polynomial function, e.g.:

```
> p := calc.polygen(coeffs);
```

There is no limit on the degree, but a degree of 7 or more is not regarded appropriate.

The function tries to reproduce polynomial trend lines known from spreadsheet applications and internally uses Kahan-Ozawa-Summation for better accuracy.

See also: **calc.horner**, **calc.interp**, **calc.linterp**, **calc.polygen**.

calc.polygen (c_n, c_{n-1}, ···, c_2, c_1)
calc.polygen (obj)

Creates a polynomial $p(x) = c_n*x^{n-1} + c_{n-1}*x^{n-2} + \cdots + c_2*x + c_1$ from the coefficients c_n, c_{n-1}, ···, c_2, c_1 and returns it as a new function <: x -> p(x) :>, where x and the return p(x) represent numbers or complex numbers.

You may alternatively pass the coefficients c_k in a table, sequence or register `obj`.

The function internally uses 80-bit precision floats.

Example: The Taylor series expansion of the sine function, with order 8, is:

$$\sin(x) = x - \frac{1}{3!}x^3 + \frac{1}{5!}x^5 - \frac{1}{7!}x^7 + \ldots$$

So the coefficients in *descending* [sic !] order are:

```
> coeffs := [1, 0, -1/fact(3), 0, +1/fact(5), 0, -1/fact(7)];
```

This generates the function:

```
> taylorsine := calc.polygen(coeffs);
```

```
> taylorsine(1), sin(1):
0.84146825396825  0.8414709848079
```

See also: **calc.horner**, **calc.polyfit**.

calc.polylog (n, x)

Returns the polylogarithm of order `n` (an integer greater or equals -1) at a real point `x`. The return is a number, or **fail** if `n` < -1 as this case is not implemented. The polylogarithm of order `n` is defined by the series:

$$\mathrm{Li}_n(x) = \sum_{k=1}^{\infty} \frac{x^k}{k^n}$$

See also: **calc.dilog**, **calc.Li**.

calc.probit (x)

Computes the inverse of the cumulative distribution function of the standard normal distribution:

$$\mathrm{probit}(x) = \sqrt{2}\,\mathrm{erf}^{-1}(2x-1).$$

If `x`=0, the function will return -**infinity**, and if `x`=1 the result will be +**infinity**.

See also: **calc.logit**, **inverf**.

calc.regulafalsi (f, a, b [, ···] [, options])

Tries to determine the root of the univariate or multivariate function `f` in the borders `a` and `b` and returns it as a number if successful, and **null** otherwise. It also returns **null** if the number of iterations taken exceeded a limit so it cannot go into an infinite loop.

You can control the maximum number of iterations the function takes until finishing the evaluation by passing the `iters`=<positive integer> option, with the 125 the default.

The second, etc. arguments to `f` may be given right after argument `b`.

The precision `eps` by default is **Eps** and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number:

```
> calc.regulafalsi(<: x, a -> sin(x + a) :>, 0, 2, Pi/2, eps=DoubleEps):
1.5707963267949
```

Since the function finds only one zero, you can pass it to the **calc.zeros** wrapper to get all roots over a larger interval, for example:

```
> calc.zeros(<: x -> sin x :>, -1000, 1000, finder = calc.regulafalsi):
```

The function is the fastest root finder, but sometimes, depending on the type of `f`, might miss one.

See also: **calc.brent**, **calc.chandrupatla**, **calc.itp**, **calc.sections**, **calc.zeroab**, **calc.zeroin**, **calc.zeros**.

calc.riesum (f, a, b [, ···] [, options])

Implements numerical integration using the Riemann sum method for the univariate or multivariate real function `f` over the interval [`a`, `b`].

If `f` is a multivariate function, then its second, third, etc. arguments must be given after `b`.

The function accepts the following options:

- `step`: number of subintervals used, defaults to 10.
- `rule`: either `'left'` for left-hand rule, `'right'` for right-hand rule, `'random'` for a random-point rule and `'mid'` (the default) for midpoint rule.
- `absolute`: absolute instead of signed intermediate function values will be summed up, default is **false**.

`f` should be continuous over [`a`, `b`] and the interval carefully chosen. Example:

```
> calc.riesum(<: x, a -> x + a :>, 0, 1, 0,
>    step=5, rule='mid', absolute=true):
0.5
```

The function internally uses 80-bit precision floats.

See also: **calc.gauleg**, **calc.gtrap**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.romberg**, **calc.simaptive**.

calc.riesum64 (f, a, b [, ···] [, options])

Like **calc.riesum**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.riesum** as they do not have 80-bit arithmetic.

calc.romberg (f, a, b [, ···] [, options])

Integrates the univariate or multivariate function `f` on the interval [`a`, `b`] using Romberg Integration combining Trapezoidal Rule with Richardson Extrapolation and returns a number.

The second, etc. arguments to `f` may be given right after argument `b`.

By default the function quits after an accuracy of **Eps** has been reached. You may change this by passing the option `eps` = *v*, where *v* is a positive number, for example:

```
> calc.gtrap(<: x -> x :>, 0, 1, eps = DoubleEps):
0.5
```

Internally, the function uses 26 iterations to compute the result. You can change this by passing the `iters` = *n* option, with *n* a positive integer.

If the algorithm does not converge, which may happen with highly oscillating functions or embedded poles, the function returns **fail**.

See also: **calc.gauleg**, **calc.gtrap**, **calc.gtrap64**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.riesum**, **calc.simaptive**.

calc.saddles (f, a, b, [, ···] [, options])

Returns all saddle points of the univariate or multivariate function `f` on the interval [`a`, `b`], with `a`, `b` real numbers, in a sequence. `f` should be differentiable on [`a`, `b`].

The function actually is a wrapper to **calc.extrema** which by itself is an interface to **calc.zeros** and **calc.differ** and supports the following options:

- `eps`: Accuracy of the root-finding method for the first and second derivative, defaulting to **Eps**.
- `step`: Length of the subintervals to be examined, the `step size`. Defaults to 0.1.
- `adaptive`: When given a number n along with this option, conducts n additional divisions of all the sub-intervals to be checked for extrema. Default is n = 0, that is no additional divisions.
- `finder`: The extrema-finding function. Available functions are **calc.zeroin** (the default), **calc.zeroab** or **calc.regulafalsi**, **calc.chandrupatla**, **calc.itp** and **calc.brent**.
- `span`: Control on the computation of Chebyshev coefficients, see **calc.differ** for further details.

Note that on ARM platforms, the function may miss some saddle points.

See also: **calc.extrema**, **calc.inflect**, **calc.zeros**.

calc.savgol (f [, options])

Computes a Savitzky–Golay filter for the univariate function `f` to `smooth` its data by returning a function interpolating `f` at a given point x_0. preventing large oscillations between sample points.

It fits successive subsets of neighbouring data with a low-degree polynomial using the linear least-square method. By default, 15 equally-spaced points to the left of x_0 and 15 equally-spaced points to the right of x_0 are examined.

You can change this `window` by passing another odd value with the `'points'` option. All adjacent points are separated by distance eps which is 1e-5 by default. You can change the distance with the `'eps'` option, e.g. eps=0.1 which unless you want to compute a derivative, see below, might be a much more useful value.

Note that on ARM systems you may have to use different epsilon values, especially with higher-order derivatives. You may pass the `adaptive` = **false** option, too, so that the epsilon value you pass with the `eps` option will not be modified internally.

Alternatively it can also compute derivatives of any degree `n` by passing the option deriv=`n`. The larger the degree `n` of the derivative, however, the less accurate the results will become.

The degree `d` of the smoothing least-square polynomial is 3 by default and can be changed by the degree=`d` option. Recommended degrees are `d` = 2 or 4, with `d` not exceeding 6.

The function automatically determines the most suitable settings for the window and the spacing eps of its points, but you can switch this off by passing the `adaptive=false` option (default is `adaptive=true`).

Example: Compute the first, second and third derivative of sin(x) and evaluate at x=$\frac{\pi}{2}$:

```
> f := <: x -> sin x :>

> f' := calc.savgol(f, deriv = 1)

> f'' := calc.savgol(f, deriv = 2)

> f''' := calc.savgol(f, deriv = 3)

> f(Pi/2), f'(Pi/2), f''(Pi/2), f'''(Pi/2):
1       7.4115382884751e-017    -0.99999999999765       0
```

See also: **calc.cheby**, **calc.interp**, **calc.savgolcoeffs**.

calc.savgolcoeffs (nleft, nright, deriv, polydeg)

Returns the normalised Savitzky-Golay filter coefficients as a register of 80-bit long doubles. `nleft` is the number of leftward observations to be examined, while `nright` is the number of rightward ones.

`deriv` is the order of the derivative desired (0 for the smoothed function, 1 for the first derivative, asf.).

`polydeg` is the order of the smoothing polynomial, with 2 or 4 being recommended values, but not exceeding 6.

To convert to standard 64-bit Agena numbers, issue:

```
> long.tonumber @  calc.savgolcoeffs(2, 2, 0, 2):
reg(-0.08571429, 0.34285714, 0.48571429, 0.34285714, -0.08571429)
```

See also: **calc.chebycoeffs**, **calc.savgol**.

calc.scaleddawson (x)

Implements the Scaled Dawson Integral w_im(`x`) = 2***calc.dawson**(`x`)/**sqrt**(**Pi**) for real `x`.

See also: **calc.dawson**, **calc.w**.

calc.sections (f, a, b, [, ···] [, option])

Returns all subintervals where the univariate or multivariate function `f` has a change in sign or a pole in interval [`a`, `b`] with `a`, `b` numbers. The second, etc. arguments to `f` may be given right after argument `b`.

The step size by default is 0.1 and can be changed by passing the option `step` = *val*, where *val* is a positive number. Example.

```
> calc.sections(<: x, a -> sin(x + a) :>, -4, 4, Pi/2, step = 0.01):
seq(-1.58:-1.57, 1.57:1.58)
```

The return is a sequence of pairs denoting the subintervals found. The function uses Adapted Neumaier summation to prevent round-off errors, the same algorithm used by numeric **for** loops with fractional step sizes.

See also: **calc.poles**, **calc.zeros**, **unimath.sections**, **unimath.zeros**.

calc.Shi (x)

Computes the hyperbolic sine integral and returns it as the number

$$\mathrm{Shi}(x) = \int_0^x \frac{\sinh t}{t} dt$$

`x` must be a number.

See also: **calc.Ci**, **calc.Chi**, **calc.Si**, **calc.Ssi**.

calc.Si (x)

For `x` a number, computes the sine integral

$$\mathrm{Si}(x) = \int_{t=0}^{x} \frac{\sin(t)}{t}\,dt = \int_{t=0}^{x} \mathrm{sinc}(t)\,dt$$

and returns it as a number.

See also: **calc.Ci**, **calc.Chi**, **calc.Shi**, **calc.SiCi**, **calc.Ssi, sinc**.

calc.SiCi (x)

Returns both the sine and cosine integrals of real `x`, in this order, see **calc.Si**, **calc.Ci** for further details. See also: **calc.auxSiCi**.

calc.sigmoid (x)

Computes the sigmoid, i.e. standard logistic, function,

$$S(x) = \frac{1}{1 + e^{-x}} = \frac{e^{x}}{e^{x} + 1} = 1 - S(-x)$$

having a characteristic `S`-shaped curve or sigmoid curve, for any number `x`. The result is a number S(x) with $0 < S(x) < 1$.

See also: **calc.gd**, **calc.logistic**, **calc.logit**.

calc.simaptive (f, a, b [, ···] [, options])

Integrates the univariate or multivariate function `f` on the interval [`a`, `b`] using Simpson-Simpson Adaptive Quadrature and returns a number.

The second, etc. arguments to `f` may be given right after argument `b`.

The acceptable error eps defaults to **Eps**/2 and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number.

The minimum subinterval length h_min by default is 1e-7 and can be changed by passing the option `length` = *v*, where *v* is a positive number.

```
> calc.simaptive(<: x -> sin x :>, 0, 1, length = 1e-10, eps = Eps):
0.4596976946293
```

The function returns **undefined** if no suitable subinterval of length greater than h_min could be found for which the estimated error falls below eps.

The function is thrice as fast as **calc.integ**, but is not suited with singularities at or within the borders.

The function uses 80-bit floating-point precision internally.

See also: **calc.gauleg**, **calc.gtrap**, **calc.intcc**, **calc.intde**, **calc.intdei**, **calc.intdeo**, **calc.integ**, **calc.riesum**, **calc.romberg**.

calc.simaptive64 (f, a, b [, ···] [, options])

Like **calc.simaptive**, but using 64-bit floating-point precision and Kahan-Babuška summation to prevent round-off errors during summation. On ARM platforms, this version is called by **calc.simaptive** as they do not have 80-bit arithmetic.

calc.sinuosity (f, a, b [, ···] [, option])

Computes the ratio of the curvilinear length (along the curve) and the Euclidean distance (straight line) between the end points `a` and `b`, of the curve defined by a function `f` in one real. `a`, `b` must be numbers. The second, etc. arguments to `f` may be given right after argument `b`.

sinuosity checks whether `f` has poles in the given range [`a`, `b`] and splits it up accordingly, summing up the various sub-lengths to return a finite result, if possible.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.arclen**, **calc.curvature**, **calc.eucliddist**.

calc.smoothstep (x, n)

calc.smoothstep (x, 'perlin')

In the first form, the function receives a non-negative integer `n` and any number `x` and returns 0 if $x < 0$, 1 if $x > 1$, and smoothly interpolates between 0 and 1, using an (2*`n`+1)'th-degree Hermite polynomial otherwise.

The slope of the smoothstep function is zero at both edges, so the result is differentiable over the whole real domain.

Grokipedia: `Smoothstep is a family of sigmoid-like interpolation and clamping functions commonly used in computer graphics, video game engines, and machine learning.`, for example to naturally accelerate or decelerate an object.

In the second form, if the string `'perlin'` is passed, the function computes the `smootherstep` polynomial $6x^5 - 15x^4 + 10x^3$ for $0 < x < 1$, which has zero 1st- and 2nd-order derivatives at $x = 0$ and $x = 1$, as suggested by computer scientist Prof. Kenneth Perlin. If given, `n` may be any integer as it is not evaluated in this case.

See also: **heaviside**, **math.clip**, **math.rectangular**, **math.unitise**.

calc.softsign (x)

Computes the Softsign function $\frac{x}{1+|x|}$.

calc.Ssi (x)

Computes the shifted sine integral and returns it as a number. `x` must be a number.

See also: **calc.Ci**, **calc.Chi**, **calc.Shi**, **calc.Si**.

calc.variance (f, a, b [, ···] [, options])

Returns a positive integer that indicates whether a function `f` in one real changes slowly or rapidly on the given interval [`a`, `b`], with `a`, `b` numbers. The larger the result, the larger is its rate of change. The result is relative, i.e. given per unit on the abscissa.

The second, etc. arguments to `f` may be given right after argument `b`.

The function accepts the following options:

- `samples`, defaults to 10 sample points per unit.
- `eps`, the bail-out value `eps` - a positive value close to zero, defaulting to **Eps**.

Example:

```
> calc.variance(<: x, a -> x^a :>, 0, 2, 3, eps = hEps, samples = 20):
19125
```

Internally, the function uses adaptive integration with trapezoidal rule and counts the number of trapezoids evaluated. Note that the results are estimates.

calc.w (z [, eps])

Implements the scaled complex complementary error function w(`z`) = **exp**(-`z` ^ 2)***erfc**(-**I***`z`) (Faddeeva function) for number or complex number `z`. The return is a complex value. By default, the precision `eps` is **DoubleEps**, and can be any other non-negative number.

See also: **calc.scaleddawson**.

calc.weier (x, a, b [, eps])

Implements the Weierstraß function for the given number `x` and parameters `a`, `b` (also numbers), with $0 < x < 1$ and $ab \geq 1$, `b` an odd positive integer - a function that is continuous but non-differentiable everywhere:

$$\sum_{n=0}^{\infty} a^n \cos(b^n x)$$

The precision is given by its fourth optional argument, `eps`, which is **Eps** by default.

The function internally uses Kahan-Ozawa-Summation for better accuracy.

calc.xpdiff (f, x [, ···] [, options])

Like **calc.diff**, but uses Richardson's extrapolation method to compute symmetric difference quotients. `f` is a univariate or multivariate function to be inspected at point `x` (a number). The second, etc. arguments to `f` may be given right after argument `x`.

The return of the procedure is the derivative of `f` at `x`, `···`, - a number - and the absolute error. If the absolute error is quite large, it may indicate non-differentiability of `f` at `x`, `···`.

If the function could not determine a result if `x` is near an undefined domain, it automatically calls **calc.diff**, which is more robust in this case but returns less precise results.

If the option `deriv`=n is given, where n may be 1, 2, or 3, the n-th derivative is calculated, with n = 1 the default. If n = 0, then the function value at `f`(`x`) is determined.

If the option `eps`=h is given, the epsilon value h (a positive number preferably close to zero) is used for the relative error check. If the option `delta`=g is given, the delta value g (a positive number preferably close to zero) is used for the absolute error check, otherwise they both default to **math.epsilon**(`x`).

Examples:

```
> calc.xpdiff( <: x -> ln x :>, 2):
0.49999999999999        5.7825133570333e-011

> calc.xpdiff( <: x -> ln x :>, 2, deriv=2):
-0.24999999999988       5.3327536830849e-011

> calc.xpdiff( <: x -> ln x :>, 2, deriv=3, eps=hEps):
0.25030038625393        8.7168201300305e-009
```

calc.xpdiff produces better results with powers and trigonometric functions than **calc.diff**. For a function that automatically chooses the best differentiation method, see **calc.differ**.

See also: **calc.diff**, **calc.differ**, **calc.eulerdiff**, **calc.isdiff**, **unimath.xpdiff**.

calc.zeroab (f, a, b [, ···] [, options])

The function determines the root of the univariate or multivariate function `f` in the borders `a` and `b` and returns a number if successful, and **null** otherwise.

It implements the Anderson-Björck root-finding algorithm; it is a modification of the Illinois algorithm that weighs based on ordinate values when one side is not updating, and which exhibits superlinear convergence. According to some reviews, this is considered to be the state-of-the-art algorithm.

The second, etc. arguments to `f` may be given right after argument `b`.

The precision `eps` by default is **Eps** and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number.

You can control the maximum number of iterations the function takes until finishing the evaluation by passing the `iters`=<positive integer> option, with the 40 the default.

Example for finding the root of sin(x^3 - 5) in the interval [1, 2]:

```
> calc.zeroab(<: x -> sin(x^3 - 5) :>, 1, 2, eps=hEps):
1.2294578319581
```

The function might be slightly slower with some types of `f` than **calc.zeroin**. Since the function finds only one zero, you can pass it to the **calc.zeros** wrapper to get all the roots over a larger interval, for example:

```
> calc.zeros(<: x -> sin x :>, -1000, 1000, finder = calc.zeroab):
```

See also: **calc.brent**, **calc.chandrupatla**, **calc.itp**, **calc.regulafalsi**, **calc.zeroin**, **calc.zeros**.

calc.zeroin (f, a, b [, ···] [, options])

Determines the root of the univariate or multivariate function `f` in the borders `a` and `b` and returns a number if successful, and **null** otherwise.

The second, etc. arguments to `f` may be given right after argument `b`.

The precision `eps` by default is **Eps** and may be changed by passing the option `eps` = *tol*, where *tol* is a positive number.

You can control the maximum number of iterations the function takes until finishing the evaluation by passing the `iters`=<positive integer> option, with the 25 the default.

In general, the function will even return accurate results where **calc.regulafalsi** fails to do so - or even cannot find a root at all -, but the runtime behaviour compared to **calc.regulafalsi** depends on the following conditions:

1. the interval should not be too far from the origin,
2. the width of the interval should not be too small.

If both conditions are met, then the function can be faster than **calc.regulafalsi**.

The algorithm uses bisection combined with linear or quadric inverse interpolation, followed by applying Regula Falsi to the estimate done by the previous actions.

Example for finding the root of sin(x^3 - 5) in the interval [1, 2]:

```
> calc.zeroin(<: x -> sin(x^3 - 5) :>, 1, 2, eps=hEps):
1.2294578319581
```

Since the function finds only one zero, you can pass it to the **calc.zeros** wrapper to get all the roots over a larger interval, for example:

```
> calc.zeros(<: x -> sin x :>, -1000, 1000, finder = calc.zeroin):
```

See also: **calc.brent**, **calc.chandrupatla**, **calc.itp**, **calc.regulafalsi**, **calc.zeroab**, **calc.zeros**, **unimath.zeroin**.

calc.zeros (f [, a, b,] [, ···] [, options])

Returns all roots of a function `f` in one or more reals on the interval [`a`, `b`] in a sequence. `f` must be differentiable on [`a`, `b`]. If it could not find a root, it returns **null**.

Alternatively, if only `f` and `options` are given, the function determines all complex roots.

The second, etc. arguments to `f` may be given right after argument `b`.

For real solutions, the function accepts the following options:

- `eps`: accuracy of the root-finding method, defaulting to **Eps**.
- `step`: length of the subintervals to be examined, `step size`. Defaults to 0.1.
- `classic`: when set to **true**, the root-finding will be much faster, but less reliable, default is **false**.

The function divides the interval [`a`, `b`] into smaller intervals [`a`, `a`+step], [`a`+step, `a`+2*step], ···, [`b`-step, `b`]. It then looks for changes in sign in these smaller intervals and if it finds them, determines the roots using a modified regula falsi method.

You can pass a root-finding function with the `finder` option. Available functions are **calc.zeroin** (the default), **calc.brent**, **calc.chandrupatla**, **calc.itp**, **calc.zeroab** or **calc.regulafalsi**:

```
> calc.zeros(<: x -> sin(x^3 - 5) :>, -2, 2,
>    step = 0.05, finder = calc.zeroab, classic = true):
seq(-1.9631909197463, -1.6417127111242, -1.0866669520226, 1.2294578319581,
1.7099759466767, 2.0117304515399)
```

The `adaptive` option allows to better cope with arithmetic functions that are highly-oscillating over an interval. When given a number n along with this option, **calc.zeros** conducts n additional divisions of all the sub-intervals to be checked for a change of sign along the abscissa. Example without subdivision where only some few roots could be found:

```
> calc.zeros(<: x -> sin(x^3 - 5) :>, 9, 10):
seq(9.4730108352616, 9.5996742619696, 9.7008759505391, 9.9503435246497)
```

Examples with subdivision:

```
> calc.zeros(<: x -> sin(x^3 - 5) :>, 9, 10, adaptive = 2):
seq(9.0327657042891, 9.0964888098491, 9.1593314174461, 9.2213233266749,
9.2824927555153, 9.3428664535072, 9.3787193352328, 9.4378721303909,
9.496292585202, 9.5310026375778, 9.5654617002802, 9.6223478503705,
9.6561592089146, 9.6897354281276, 9.7230805540118, 9.7781528646009,
9.8109006964726, 9.8434313575779, 9.8757484129709, 9.9291448830307,
9.9609090575167, 9.9924719293519)
```

And so on ...

To determine complex solutions, you can either pass an explicit guess or use the default method spiralling out from the origin, taking sample points on the spiral and checking whether they are near a complex solution. The following options are supported here:

- `guess`: a complex number, that is an explicit sample point that might be near a complex solution.
- `eps`: accuracy of the root-finding method, defaulting to 1e-15.
- `iters`: maximum number of iterations to check whether each candidate is a root. Default is 50.
- `step`: a number; when spiralling out, increases the angle by this value, defaulting to 0.5.
- `growth`: a number, control the growth of the spiral, default is 0.1.
- `max`: a number, controls the maximum angle of the spiral multiplied by π. Default is 20.
- `principal`: a boolean: all solutions are reduced to the principal value, default is **false**.

Examples:

```
> calc.zeros(<: z -> z^3 - 1 :>):
[1, -0.5-0.86602540378444*I, -0.5+0.86602540378444*I]

> calc.zeros(<: z -> z^3 - 1 :>, guess=0.5+I):
[-0.5-0.86602540378444*I]
```

The function is implemented in Agena and included in the lib/library.agn file.

See also: **calc.brent**, **calc.chandrupatla**, **calc.itp**, **calc.poles**, **calc.regulafalsi**, **calc.sections**, **calc.zeroab**, **calc.zeroin**, **unimath.zeros**.

calc.zeta (x)
calc.zeta (x, q)

In the first form, computes the Riemann Zeta function for real `x` and returns the number:

$$\sum_{k=2}^{\infty} k^{-x} + 1$$

In the second form computes, where `x` > 1 and `q` is not a negative integer or zero:

$$\textbf{calc.zeta}(x, q) = \sum_{k=0}^{\infty} (k+q)^{-x}$$

Note that this second form is not equivalent to the two-argument version of Maple's Zeta() function which computes the n-th derivative at the point `x`.

See also: **calc.eta**.

11.13 linalg - Linear Algebra Package

This package provides basic functions for Linear Algebra.

There are two constructors available to define vectors and matrices, **linalg.vector** and **linalg.matrix**, with **vector** and **matrix** aliases to them to spare some typing. There is also the < ... > syntax available to easily define vectors and matrices with much less typing. The package functions assume that the geometric objects passed have been created with the above mentioned constructors.

The package includes a metatable **linalg.vmt** defined in the lib/library.agn file with metamethods for vector addition, vector subtraction, and various vector multiplication methods. Further functions are provided to compute the length of a vector with the **abs** operator and to apply unary minus to a vector. **size** determines the dimension of a vector, that is the number of elements, including zero values (see below).

The table **linalg.mmt** defines metamethods for matrix addition, subtraction and multiplication.

For both vectors and matrices, there are metamethods to compare matrices with the =, ==, <>, ~= and ~<> operators.

For all the operations implemented by metamethods see the bottom of this chapter.

linalg.vector and other **linalg** functions implement 'sparse' behavior in logic only, not in memory. Unset indices safely return 0 instead of **null**, but the underlying array remains fully allocated. Therefore, this approach provides a convenient default value but offers zero memory savings.

The dimension of the vector and the dimensions of the matrix are indexed with the `'dim'` key of the respective object. You should not change this setting to avoid errors. Existing vector and matrix values can be overwritten.

A sample session:

Define two vectors in three fashions: In the simple form, just pass all components explicitly:

```
> a := vector(1, 2, 3):
< 1, 2, 3 >
```

Or, equally:

```
> a := < 1, 2, 3 >:
< 1, 2, 3 >
```

In a more elaborate form, indicate the dimension of the new vector and pass the non-zero vector components in a table:

```
> b := vector(3, [1 ~ 2]):
< 2, 0, 0 >
```

Check whether a and b are parallel and have the same direction:

```
> abs(a+b) = abs(a) + abs(b):
false
```

Set a vector component by indexing - a negative integral index n depicts the |n|-th element from the right:

```
> b[3] := 1;
```

Now read the modified vector and its rightmost component:

```
> b:
< 2, 0, 1 >

> b[3], b[-1]:
1       1
```

Addition:

```
> a + b:
< 3, 2, 4 >
```

Subtraction:

```
> a - b:
< -1, 2, 2 >
```

Scalar, dot and cross product:

```
> 2 * a:
< 2, 4, 6 >

> a * a:
14

> linalg.crossprod(a, b):
< 2, 5, -4 >
```

Find the vector x which satisfies the matrix equation A x = b. In this example, we will solve the equation $\begin{bmatrix} 1 & 2 & -4 \\ 2 & 1 & 3 \\ -3 & 1 & 6 \end{bmatrix} * x = \begin{bmatrix} -6 \\ 5 \\ -2 \end{bmatrix}$. The **linalg.matrix** constructor expects row vectors.

```
> A := matrix([ 1, 2, -4 ], [ 2, 1, 3 ], [ -3, 1, 6 ]):
[  1, 2, -4 ]
[  2, 1,  3 ]
[ -3, 1,  6 ]

> b := vector(-6, 5, -2):
< -6, 5, -2 >

> linalg.linsolve(A, b):
< 2, -2, 1 >
```

Or, equally:

```
> A :=  < < 1, 2, -4 >, < 2, 1, 3 >, < -3, 1, 6 > >:
[  1, 2, -4 ]
[  2, 1,  3 ]
[ -3, 1,  6 ]

> b := < -6, 5, -2 >:
< -6, 5, -2 >

> linalg.linsolve(A, b):
< 2, -2, 1 >
```

The **linalg** operators and functions are:

v1 + v2

Adds two vectors or matrices `v1`, `v2`. The return is a new vector or matrix. This operation is done by applying the `__add` metamethod.

v1 - v2

Subtracts two vectors or matrices `v1`, `v2`. The return is a new vector or matrix. This operation is done by applying the `__sub` metamethod.

k * v
v * k
v * w
M1 * M2

In the first an second form, computes the scalar product by multiplying a number `k` with each element in vector or matrix `v`. In the first form computes the dot product of vectors `v`, `w`. In the fourth form multiplies the matrix `M1` with matrix `M2`. The return is a scalar, new vector or matrix. This operation is done by applying the `__mul` metamethod.

v / k
k / v

Divides each element in the vector `v` by the number `k`, or vice versa. The return is a new vector. This operation is done by applying the `__div` metamethod.

abs (v)

Determines the length of vector `v`. This operation is done by applying the `__abs` metamethod to `v`.

copy (la)

Creates a deep copy of the matrix or vector `la`.

See also: **linalg.mcopy**, **linalg.vcopy**.

qsumup (v)

Raises all elements in vector `v` to the power of 2. The return is the sum of these powers, i.e. a number. This operation is done by applying the `__qsumup` metamethod to `v`.

size (la)

With `la` a vector, returns its dimension as an integer. With `la` a matrix, returns its dimensions as a pair, see **linalg.dim** for further information.

linalg.add (v, w)

Determines the vector sum of vector `v` and vector `w`. The return is a vector.

See also: **linalg.sub**.

linalg.addcol (A, c1, c2 [, m])
linalg.addrow (A, r1, r2 [, m])

The functions perform linear combinations of matrix rows or columns.

The call **linalg.addrow**(`A`, `r1`, `r2`, `m`) returns a copy of the matrix `A` in which row `r2` is replaced by `m`***linalg.row**(`A`, `r1`) + **linalg.row**(`A`, `r2`).

Similarly linalg.addcol(A, `c1`, `c2`, `m`) returns a copy of the matrix `A` in which column `c2` is replaced by `m`***linalg.col**(`A`, `c1`) + **linalg.col**(`A`, `c2`).

In both cases, if the number `m` is not given, `m` default to 1.

Examples:

```
> a := linalg.matrix(3, 3, [[1, 2, 3], [2, 3, 4], [3, 4, 5]] );

> linalg.addrow(a,1, 2, 10):
[  1,  2,  3 ]
[ 12, 23, 34 ]
[  3,  4,  5 ]
```

```
> linalg.addcol(a, 1, 2, -10):
[ 1,  -8, 3 ]
[ 2, -17, 4 ]
[ 3, -26, 5 ]
```

linalg.adjoint (A)

Finds the co-factor matrix (adjoint) of square matrix A. "A co-factor matrix is a matrix having the co-factors as the elements of the matrix. The co-factor of a matrix element is obtained when the minor M[i, j] of the element is multiplied with $(-1)^{i+j}$. The minor of a matrix is for each element of the matrix and is equal to the part of the matrix remaining after excluding the row and the column containing that particular element." (Cited after cuemath.com.)

See also: **linalg.det**, **linalg.minor**.

linalg.antidiagonal (v)

Creates a square matrix `A` with all vector components in `v` put on the anti-diagonal. The first element in `v` is assigned `A[1][size A]`, the second element in `v` is assigned `A[2][size A - 1]`, etc. Thus the result is a dim(`v`) x dim(`v`)-matrix. All elements off the anti-diagonal are set to **null**, implying zero.

See also: **linalg.diagonal**, **linalg.getantidiagonal**, **linalg.getdiagonal**.

linalg.augment (···)

Joins two or more matrices or vectors together horizontally. Vectors are supposed to be column vectors. The matrices and vectors must have the same number of rows.

The return is a new matrix.

See also: **augment**, **linalg.delcols**, **linalg.delrows**, **linalg.extend**, **linalg.reshape**, **linalg.stack**, **zip**.

linalg.backsub (A)
linalg.backsub (A, v)

Performs backward substitution on a system of linear equations.

In the first form, `A` must be an augmented m x n lower triangular matrix with m+1 = n. In the second form, `A` is an lower triangular square matrix and `v` a right-hand side vector.

The return is the solution vector.

The function issues an error if `A` is not upper triangular. You may change the tolerance to detect `zeros` by setting the global system variable **Eps** to another value.

See also: **linalg.forsub**, **linalg.linsolve**, **linalg.rref**.

linalg.backsubs (A, b)

The function has been deprecated. Please use **linalg.linsolve** instead.

linalg.checkmatrix (A [, B, ···] [, true])

Issues an error if at least one of its arguments is not a matrix. If the last argument is **true**, then the matrix dimensions are returned as a pair, else the function returns nothing.

Contrary to **linalg.checkvector**, the dimensions will not be checked if you pass more than one matrix. See also: **linalg.dim**.

linalg.checksquare (A [, B, ···])

Issues an error if `A` is not a square matrix. You can pass more than one matrix to be checked. The function returns the dimensions on success as single positive integers.

linalg.checkvector (v [, w, ···])

Issues an error if at least one of its arguments is not a vector. In case of two or more vectors it also checks their dimensions and returns an error if they are different.

If everything goes fine, the function will return the dimensions of all vectors passed.

See **linalg.isvector** for information on how the check is being done.

See also: **linalg.dim**.

linalg.col (A, n)

Returns the `n`-th column of the matrix or row vector `A` as a new vector. Example:

```
> A := linalg.matrix(2, 2, [1, 2, 3, 4]):
[ 1, 2 ]
[ 3, 4 ]

> linalg.col(A, 2):
< 2, 4 >
```

See also: **columns**, **linalg.row**, **linalg.submatrix**, **linalg.subvector**.

linalg.coldim (A [, ···])

Determines the column dimension of the matrix `A`. The return is a number.

If you pass more than one argument, then a time-consuming check whether `A` is a matrix, is skipped.

A more direct way of determining the column dimension is `right(A.dim)`.

See also: **linalg.dim**, **linalg.rowdim**.

linalg.colvector (···)

Exactly like **linalg.vector**, but instead of an n-dimensional row vector returns an n x 1 matrix representing a column vector. The function supports all input arguments that **linalg.vector** is accepting. See also: **linalg.augment**.

linalg.copyinto (A, B, m, n)

The function copies the entries of matrix `A` into matrix `B` beginning at index position [`m`, `n`], so `B`[`m`, `n`] is assigned `A`[1, 1], that is, the function modifies its input argument `B`.

The return is the new contents of `B`. Example:

```
> A := linalg.matrix(2, 2, [1, 2, 3, 4]);

> B := linalg.extend(A, 2, 2, 0);

> linalg.copyinto(A,B,3,3):
[ 1, 2, 0, 0 ]
[ 3, 4, 0, 0 ]
[ 0, 0, 1, 2 ]
[ 0, 0, 3, 4 ]
```

See also: **linalg.augment**, **linalg.extend**, **linalg.reshape**, **linalg.stack**, **linalg.submatrix**, **linalg.subvector**.

linalg.countitems (e, v [, option])
linalg.countitems (f, v [, ···])

In the first form, counts the number of occurrences of number `e` in vector `v`. By default, the equality check of e with all the vector components is strict - by giving the `approx`=**true** option, however, the function counts all the elements that are approximately equal, see **approx** and the ~= operator for further information.

In the second form, by passing a function `f` with a Boolean relation as the first argument, all elements in vector `v` that satisfy the given relation are counted. If the function has more than one argument, then all arguments *except the first* must be passed right after `v`.

linalg.crossprod (v, w)

Computes the cross-product of two vectors `v`, `w` of dimension 3. The return is a vector.

See also: **linalg.dotprod**, **linalg.innerprod**,**linalg.multiply**.

linalg.delcols (A, i [, j])

linalg.delrows (A, i [, j])

The functions remove one or more columns or rows from matrix `A`, from position `i` to position `j`. If `j` is not given, it defaults to `i`. The functions return new matrices and do not work in-place:

```
> A := linalg.matrix(
>    [[1, 2, 0, 0], [3, 4, 0, 0], [5, 6, 0, 0], [0, 0, 0, 0]]):
[ 1, 2, 0, 0 ]
[ 3, 4, 0, 0 ]
[ 5, 6, 0, 0 ]
[ 0, 0, 0, 0 ]

> B := linalg.delrows(A, 3, 4):
[ 1, 2, 0, 0 ]
[ 3, 4, 0, 0 ]

> linalg.delcols(B, 3, 4):
[ 1, 2 ]
[ 3, 4 ]
```

See also: **linalg.augment**, **linalg.copyinto**, **linalg.extend**, **linalg.reshape**, **linalg.stack**, **linalg.submatrix**, **linalg.subvector**.

linalg.det (A [, isintegral])

Computes the determinant of the square matrix `A`. The return is a number. With singular matrices, it returns 0. By default, with only `A` given, the function first tries an approach preventing division of matrix components and only switches to an algorithm conducting division if it thinks that `A` is singular but actually is not. If you pass **true** as a second argument, then the function explicitly runs the integral algorithm to compute the determinate, and if it is **false** the fractional one.

See also: **linalg.gausselim**, **linalg.ludecomp**, **linalg.permanent**, **unimath.det**.

linalg.diagonal (v)

Creates a sparse square matrix `A` with all vector components in `v` put on the main diagonal. The first element in `v` is assigned `A[1][1]`, the second element in `v` is assigned `A[2][2]`, etc. Thus the result is a dim(`v`) x dim(`v`)-matrix. All elements off the diagonal are set to **null**, implying zero.

See also: **linalg.antidiagonal**, **linalg.getantidiagonal**, **linalg.getdiagonal**.

linalg.dim (A [, option])

Determines the dimension of a matrix or a vector `A`. If `A` is a matrix, the result is a pair with the left-hand side representing the number of rows and the right-hand side representing the number of columns. If `option` is set to **true**, then instead of a pair, the row and column dimensions of the matrix are returned individually as two integers.

If A is a vector, the size of the vector is determined.

If A is malformed or not a matrix or vector, then the function issues an error.

See also: **linalg.checkmatrix**, **linalg.checkvector**, **linalg.coldim**, **linalg.rowdim**, **linalg.vectdim**, **size**.

linalg.dotprod (v, w)

Computes the vector dot product of two vectors v, w of same dimension. The vectors must consist of Agena numbers. The return is a number.

See also: ***** metamethod, **linalg.crossprod**, **linalg.innerprod**, **linalg.kronprod**, **linalg.multiply**, **linalg.outprodmatrix**.

linalg.eigen (A)

Returns both the eigenvectors and eigenvalues of the square matrix A. The eigenvectors are returned as a matrix and the eigenvalues as a table array of complex numbers, in this order.

See also: **linalg.eigenval**.

linalg.eigenval (A)

Determines the eigenvalues of the square matrix A and returns them as a table array of complex numbers.

Example:

```
> linalg.eigenval( linalg.matrix([1, 2, 4], [3, 7, 2], [5, 6, 9]) ):
[-0.89460254283572, 13.747889058727, 4.1467134841089]
```

See also: **linalg.eigen**.

linalg.extend (A, m, n [, def [, option]])

Creates a new matrix which is a copy of the input matrix A with additional rows m and additional columns n. You can also optionally initialise new entries by passing the number def, which may also be **undefined** (the default, for sparse row vectors) implying zero without actually being physically set.

```
> A := linalg.matrix(3, 2, [1, 2, 3, 4, 5, 6]):
[ 1, 2 ]
[ 3, 4 ]
[ 5, 6 ]
```

Add a further row with all new components explicitly set to zero:

```
> linalg.extend(A, 1, 0, 0):
[ 1, 2 ]
[ 3, 4 ]
[ 5, 6 ]
[ 0, 0 ]
```

Add two further columns, but no new row. Since no initialiser is given, the new components remain unset and this default to zero.

```
> linalg.extend(A, 0, 2):
[ 1, 2, 0, 0 ]
[ 3, 4, 0, 0 ]
[ 5, 6, 0, 0 ]
```

Add two further columns and one new row. As no initialiser is given as the fourth argument, the new components also remain unset, defaulting to zero.

```
> linalg.extend(A, 1, 2):
[ 1, 2, 0, 0 ]
[ 3, 4, 0, 0 ]
[ 5, 6, 0, 0 ]
[ 0, 0, 0, 0 ]
```

If both the second and third arguments are zero, the result is a deep copy of the input.

Finally, you can work in-place by passing the `inplace=true` option as the last argument, modifying the original input structure:

```
> linalg.extend(A, 1, 1, 0, inplace=true):
[ 1, 2, 0 ]
[ 3, 4, 0 ]
[ 5, 6, 0 ]
[ 0, 0, 0 ]

> A:
[ 1, 2, 0 ]
[ 3, 4, 0 ]
[ 5, 6, 0 ]
[ 0, 0, 0 ]
```

See also: **linalg.augment**, **linalg.copyinto**, **linalg.delcols**, **linalg.delrows**, **linalg.reshape**, **linalg.stack**, **linalg.submatrix**, **linalg.subvector**.

linalg.fib (n [, option])

Returns the n-th Fibonacci matrix. If option is set to **true**, then the function works in sparse mode, consuming less memory. Example:

```
> linalg.fib(5):
[ 1, 1, 1, 1, 1, 1, 1, 1 ]
[ 1, 0, 1, 1, 0, 1, 0, 1 ]
[ 1, 1, 0, 1, 1, 1, 1, 0 ]
[ 1, 1, 1, 0, 0, 1, 1, 1 ]
[ 1, 0, 1, 0, 0, 1, 0, 1 ]
[ 1, 1, 1, 1, 1, 0, 0, 0 ]
[ 1, 0, 1, 1, 0, 0, 0, 0 ]
[ 1, 1, 0, 1, 1, 0, 0, 0 ]
```

See also: **combinat.fib**, **numtheory.fib**.

linalg.forsub (A)
linalg.forsub (A, v)

Performs forward substitution on a system of linear equations.

In the first form, A must be an augmented m x n upper triangular matrix with m+1 = n. In the second form, A is an upper triangular square matrix and v a right-hand side vector.

The return is the solution vector.

The function issues an error if A is not upper triangular. You may change the tolerance to detect `zeros` by setting the global system variable **Eps** to another value.

See also: **linalg.backsub**, **linalg.gausselim**, **linalg.gaussjord**, **linalg.linsolve**, **linalg.rref**.

linalg.gausselim (A [, v])

Performs Gaussian elimination with row pivoting on any rectangular or square m x n matrix A and returns the resulting upper triangular matrix, the rank and the determinant of **linalg.submatrix**(A, 1:n).

Example:

```
> A := matrix(3, 3, [3, 1, 0, 0, 0, 1, 1, 2, 1]);

> linalg.gausselim(A):
[ 3,                1, 0 ]
[ 0, 1.6666666666667, 1 ]
[ 0,                0, 1 ]       3       -5
```

The function also accepts a square matrix A and a column vector v:

```
> linalg.gausselim(matrix([[2, 3, 0], [4, 5, -5], [2, 2, 3]]),
>    vector([8, 9, 9])):
[ 2,  3,  0,  8 ]
[ 0, -1, -5, -7 ]
[ 0,  0,  8,  8 ]       3       -16
```

See also: **linalg.backsub**, **linalg.det**, **linalg.forsub**, **linalg.gaussjord**, **linalg.isref**, **linalg.isrref**, **linalg.linsolve**, **linalg.ludecomp**, **linalg.ludoolittle**, **linalg.pivot**, **linalg.rank**.

linalg.gaussjord (A [, v])

Performs Gauss-Jordan elimination with partial pivoting on any rectangular or square m x n matrix A and returns the resulting upper triangular matrix with leading nonzero entries 1, the rank and the determinant of **linalg.submatrix**(A, 1:n).

Example:

```
> A := matrix(3, 3, [3, 1, 0, 0, 0, 1, 1, 2, 1]);

> linalg.gaussjord(A):
[ 1, 0, 0 ]
[ 0, 1, 0 ]
[ 0, 0, 1 ]     3       -5
```

The function also accepts a square matrix A and a column vector v.

See also: **linalg.backsub**, **linalg.forsub**, **linalg.gausselim**, **linalg.linsolve**, **linalg.ludecomp**, **linalg.ludoolittle**.

linalg.getantidiagonal (A)

Returns the anti-diagonal of the square matrix A as a vector. The anti-diagonal runs from the lower left corner to the upper right corner of A.

See also: **linalg.antidiagonal**.

linalg.getdiagonal (A)

Returns the diagonal of the square matrix A as a vector.

See also: **linalg.diagonal**, **linalg.getantidiagonal**.

linalg.getvelem (v, i)

Returns the i-th entry in vector v. The entry can either be a standard Agena real or complex number, 80-bit long double (see **long** package), MAPM and MPFR value or double-to-double number (see **dd** package).

Note that most **linalg** functions will work only with vectors and matrices containing standard Agena (real) numbers.

See also **linalg.setvelem**.

linalg.hilbert (n [, x])

Creates a generalised `n` x `n` Hilbert matrix H, with H[i, j] := 1/(i+j-x). If `x` is not specified, then `x` is 1. `n` and `x` must be numbers.

linalg.identity (m [, n [, option]])
linalg.identity (p [, option])
linalg.identity (M [, option])

Creates an identity matrix of row dimension `m` and column dimension `n` with all components on the main diagonal set to 1 and all other components set to 0. If `n` is not given, creates a square `m` x `m` identity matrix.

In the second form, the pair `m:n` is denoting the row dimension `m` and column dimension `n` of the resulting matrix.

In the third form, takes a matrix `M` and uses its dimensions to create the new matrix.

By default, the function returns a sparse matrix. You can change this by passing the `sparse`=**false** option as the last argument, setting zeros physically into all row vectors.

See also: **linalg.adjoint**, **linalg.ones**.

linalg.infcolnorm (A, c [, p:q])

Calculates the infinity-norm of the column vector given by its index c in matrix `A`. The result is the maximum of the absolute values of all components in the column vector plus the first row index for which the infinity-norm is maximal.

By default, all rows will be processed but you may limit this by passing the lower row bound `p` and the upper row bound `q` as the pair `p:q` for the third argument.

Examples:

```
> A := matrix([[2, 3, 4, 1],[2, 2, 2, 2], [1, 2, 1, 3]]):
[ 2, 3, 4, 1 ]
[ 2, 2, 2, 2 ]
[ 1, 2, 1, 3 ]

> linalg.infcolnorm(A, 4, 2:3):
3       3

> linalg.infcolnorm(A, 4):
3       3
```

See also: **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.infnorm (v [, p:q])

Calculates the infinity-norm of vector `v`. By default, the whole vector is traversed. You can limit this to an index range `p` to `q` by passing the optional pair `p:q` with `p` and `q` valid indices starting from 1.

The return is the maximum absolute vector component, equal to **max**(<: x -> |x| :> @ `v`). The second return is the first row index for which the infinity-norm is maximal.

```
> v := vector(6, [1, 2, 3, 4, 5, 6]):
< 1, 2, 3, 4, 5, 6 >

> linalg.infnorm(v):
6       6

> linalg.infnorm(v, 1:3):
3       3
```

See also: **linalg.infcolnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.innerprod (a, b)

The function calculates the inner product of two matrices, two vectors or a matrix and a vector. The dimension of each matrix and vector must be compatible for multiplication in the order given.

Examples:

```
> A:= linalg.matrix(2, 3, [1, 1, 1, 2, 2, 2]);

> B := linalg.matrix(3, 2, [1, 1, 1, 2, 2, 2]);

> linalg.innerprod(A, B):
[ 4,  5 ]
[ 8, 10 ]

> v, w := linalg.vector(3, [1, 2, 3]), linalg.vector(3, [3, 2, 1]);

> linalg.innerprod(v, w):
10

> linalg.innerprod(A, v):
< 6, 12 >

> linalg.innerprod(v, B):
< 9, 11 >
```

See also: **linalg.crossprod**, **linalg.dotprod**, **linalg.mmul**, **linalg.multiply**.

linalg.inverse (A [, option])

Returns the inverse of the square matrix A. With a singular matrix, issues an error. By default, the resulting matrix is sparse - you can change this by passing the sparse=**false** option as the last argument.

linalg.isantidiagonal (A [, ···])

Checks whether matrix A and optionally further matrices are all anti-diagonal. If so, it returns **true** and **false** otherwise. In an anti-diagonal matrix all the entries except those on the diagonal going from the upper right corner to the lower left (the `anti-diagonal`) are zero. A, etc. do not necessarily be square.

See also: **linalg.isdiagonal**.

linalg.isantisymmetric (A [, ···])

Checks whether the matrix A is an antisymmetric (skew-symmetric, antimetric) matrix. If so, it returns **true** and **false** otherwise.

You can check more than one matrix for antisymmetry. In this case, the function returns **true** if all the matrices are antisymmetric and **false** if at least one is not.

See also: **linalg.issymmetric**.

linalg.isdiagonal (A [, ···])

Checks whether the matrix A is a diagonal matrix. If so, it returns **true** and **false** otherwise. In a diagonal matrix all the entries outside the main diagonal are zero. A, etc. do not necessarily be square.

You can check more than one matrix for diagonality. In this case, the function returns **true** if all the matrices are diagonal and **false** if at least one is not.

See also: **linalg.isantidiagonal**.

linalg.isfractional (A [, ···])

Checks whether m x n matrix A has at least one fractional element and returns **true** or **false**.

You can check more than one matrix. In this case, the function returns **true** if at least one matrix has a fractional number and **false** otherwise.

See also: **linalg.isintegral**.

linalg.isidentity (A [, ···])

Checks whether the square matrix A is an identity matrix. If so, it returns **true** and **false** otherwise. An identity matrix is a square matrix with ones on the main diagonal and zeros elsewhere.

You can check more than one matrix for identity. In this case, the function returns **true** if all the matrices are identical and **false** if at least one is not.

linalg.isintegral (A [, ···])

Checks whether m x n matrix A consists of integral numbers only and returns **true** or **false**.

You can check more than one matrix. In this case, the function returns **true** if all the matrices contain integers and **false** if at least one does not.

See also: **linalg.isfractional**.

linalg.islower (A [, ···])

Returns **true** if A if square matrix A is in lower triangular form, that is all its entries above the main diagonal are zero. Otherwise returns **false**.

You can check more than one matrix. In this case, the function returns **true** if all the matrices are in lower triangular form and **false** if at least one is not.

See also: **linalg.isupper**.

linalg.ismatrix (A [, ···])

Returns **true** if A and optionally further arguments are matrices, and **false** otherwise. To avoid costly checks of the passed object, the function only checks whether the input is a table with the user-defined type 'matrix'.

linalg.isone (A [, ···])

Checks whether the vector or matrix A contains only ones and returns **true** or **false**.

You can check more than one matrix and/or vector. In this case, the function returns **true** if all input arguments have ones only and **false** if at least one does not.

See also: **linalg.iszero**, **linalg.ones**.

linalg.isref (A [, ···])

Checks whether m x n matrix A is in row echelon form and returns **true** or **false**.

A matrix is in row echelon form if it has the following properties:

- Any row consisting entirely of zeros occurs at the bottom of the matrix.
- For two successive (non-zero) rows, the leading non-zero in the higher row is further left than the leading non-zero one in the lower row.

You can check more than one matrix. In this case, the function returns **true** if all matrices are in row echelon form and **false** if at least one is not.

See also: **linalg.gausselim**, **linalg.isrref**, **linalg.rref**.

linalg.isrref (A [, ···])

Checks whether m x n matrix `A` is in reduced row echelon form and returns **true** or **false**.

A matrix is in reduced row echelon form if it has the following properties:

- Any row consisting entirely of zeros occurs at the bottom of the matrix.
- The first non-zero entry of each row is equal to 1 and is the only non-zero entry of its column.
- For two successive (non-zero) rows, the leading one in the higher row is further left than the leading one in the lower row.

You can check more than one matrix. In this case, the function returns **true** if all matrices are in reduced row echelon form and **false** if at least one is not.

See also: **linalg.gausselim**, **linalg.isref**, **linalg.rref**.

linalg.issparse (la)

Checks whether the given vector or matrix contains unassigned components (which default to zero) indicating a sparse vector or a matrix with sparse row vectors. The function returns **true** or **false**. The function also returns the number of all components in a vector or matrix and the number of implicit zeros as a second and third result, with the latter depicting the number of zeros that physically are unset in the **linalg** structure.

See also: **linalg.sparse**, **linalg.iszero**.

linalg.issingular (A [, ···])

Checks whether all of the given matrices are singular and returns **true** or **false**. A matrix is singular if it is square and its determinant equals 0.

See also: **linalg.issquare**, **linalg.det**.

linalg.issquare (A [, ···])

Returns **true** if `A` is a square matrix, i.e. a matrix with equal column and row dimensions, and **false** otherwise. If you pass more than one matrix, the function

checks whether all the matrices are square and returns **true** in this case and **false** otherwise, that is at least one matrix is not square.

linalg.issymmetric (A [, ···])

Checks whether the matrix A is a symmetric matrix. If so, it returns **true** and **false** otherwise.

You can check more than one matrix for symmetry. In this case, the function returns **true** if all the matrices are symmetric and **false** if at least one is not.

See also: **linalg.isantisymmetric**.

linalg.isupper (A [, ···])

Returns **true** if A if square matrix A is in upper triangular form, that is all its entries below the main diagonal are zero. Otherwise returns **false**.

You can check more than one matrix. In this case, the function returns **true** if all the matrices are in upper triangular form and **false** if at least one is not.

See also: **linalg.islower**.

linalg.isvector (A [, ···])

Returns **true** if A is a vector, and **false** otherwise. To avoid costly checks of the passed object, the function only checks whether A is a sequence with the user-defined type 'vector'.

You can check more than one vector. In this case, the function returns **true** if all the arguments are vectors and **false** if at least one is not.

linalg.iszero (A [, ···])

Checks whether the vector or matrix A contains only zeros and returns **true** or **false**.

You can check more than one matrix and/or vector. In this case, the function returns **true** if all the input has zeros only and **false** if at least one does not.

See also: **linalg.isallones**, **linalg.issparse**, **linalg.viszero**.

linalg.kronprod (A, B)

Computes the Kronecker product of m x n matrix A and p x q matrix B and returns an m*p x n*q matrix. Example:

```
> A := < < 1, 2 >, < 3, 4 >, < 1, 0 > >
> B := < < 0, 5, 2 >, < 6, 7, 3 > >
```

```
> linalg.kronprod(A, B):
[  0,  5, 2,  0, 10,  4 ]
[  6,  7, 3, 12, 14,  6 ]
[  0, 15, 6,  0, 20,  8 ]
[ 18, 21, 9, 24, 28, 12 ]
[  0,  5, 2,  0,  0,  0 ]
[  6,  7, 3,  0,  0,  0 ]
```

See also: ***** metamethod, **linalg.crossprod**, **linalg.dotprod**, **linalg.innerprod**, **linalg.multiply**, **linalg.outprodmatrix**.

linalg.linsolve (A [, true])
linalg.linsolve (A, v [, true])

Performs Gaussian elimination on a system of linear equations.

In the first form, `A` must be a square or an augmented m x n matrix with m+1 = n. In the second form, `A` is a square matrix and `v` a right-hand side vector.

The return is the solution vector. It returns **infinity** if an infinite number of solutions has been found, and **undefined** if no solutions exists. It returns **fail** if it could not determine whether no or an infinite number of solutions exist.

If the Boolean value **true** is given as the last argument, the reduced linear system is also returned as an (augmented) upper triangular matrix.

See also: **linalg.backsub**, **linalg.forsub**, **linalg.gausselim**, **linalg.gaussjord**, **linalg.rref**, **unimath.linsolve**.

linalg.ludecomp (A [, options])

Performs LU decomposition on the m x n matrix `A`. By default, the return is an upper triangular factor. If you pass the option`` `all=true` `` then besides the upper triangular factor, the lower triangular factor, the pivot factor, the rank and the determinant will be returned, in this order.

The function tries to prevent as many fractional elements in the resulting upper triangular factor as possible if the input matrix consists of integral values only. If `A` contains at least one fractional value, or the option `` `float=true' `` is given, then a partial row pivoting method is used, otherwise pivoting is done only when a leading entry is zero.

```
> A := matrix([[2, 7, 6, 2], [9, 5, 1, 3], [4, 3, 8, 4], [5, 6, 7, 8]]);

> linalg.ludecomp(A):
[ 2,     7,               6,               2 ]
[ 0, -26.5,             -26,              -6 ]
[ 0,     0, 6.7924528301887, 2.4905660377358 ]
[ 0,     0,               0,             4.4 ]
```

```
> linalg.ludecomp(A, float=true):
[ 9,                5,                1,                3 ]
[ 0, 5.8888888888889, 5.7777777777778, 1.3333333333333 ]
[ 0,                0, 6.7924528301887, 2.4905660377358 ]
[ 0,                0,                0,             4.4 ]
```

The function does not change `A`.

See also: **linalg.gausselim**, **linalg.gaussjord**, **linalg.ludoolittle**.

`linalg.ludoolittle (A [, n [, false]])`

Performs LUP decomposition of the square, non-singular matrix `A` of order `n`. If `n` is missing, it is determined automatically, i.e. `n := left(A.dim)`.

The return is the resulting matrix, the permutation vector as a vector, and a number where this number is either 1 for an even number of row interchanges done during the computation, or -1 if the number of row interchanges was odd. If the matrix is singular, an error will be issued.

By default, the function uses Doolittle's algorithm with pivoting. Pivoting is switched off when passing **false** as the very last argument. Doolittle's method decomposes a nonsingular `n×n` matrix `A` into the product of an `n×n` unit lower triangular matrix L and an `n × n` upper triangular matrix U. A unit triangular matrix is a triangular matrix with 1's along the diagonal.

The function does not change `A`.

```
> A := matrix([[2, 7, 6, 2], [9, 5, 1, 3], [4, 3, 8, 4], [5, 6, 7, 8]]);

> linalg.ludoolittle(A):
[                9,                5,                1,                3 ]
[ 0.2222222222222,  5.8888888888889,  5.7777777777778, 1.3333333333333 ]
[ 0.4444444444444, 0.13207547169811,  6.7924528301887, 2.4905660377358 ]
[ 0.5555555555556, 0.54716981132075, 0.48333333333333,             4.4 ]
[ 2, 2, 3, 4 ]  -1
```

See also: **linalg.gausselim**, **linalg.gaussjord**, **linalg.ludecomp**.

`linalg.maeq (A, B)`

This function checks matrix `A` and matrix `B` for approximate equality. The return is either **true** or **false**. The function uses Donald Knuth's approximation method to compare matrix elements (see the **approx** function for information on how this works).

You can change the accuracy threshold epsilon with the **environ.kernel/eps** function.

See also: ~= and ~<> metamethods, **approx**, **linalg.meeq**, **linalg.vaeq**.

linalg.matinfnorm (A [, p:q [, s:t]])

Calculates the infinity-norm of matrix A. By default the entire matrix will be processed, but you may limit this to rows p to q and/or columns s to t by passing the respective index ranges p:q, s:t as optional second and third arguments, in this order.

The infinity-norm of a matrix is the maximum of the individual one-norms of its row vectors. The second return is the first row index for which the one-norm is maximal.

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.matmat (A, B, i, j [, p:q])

The function computes the scalar product of the i-th row vector in matrix A and the j-th column vector in matrix B. You can limit this to an index range p to q in the two vectors by passing the optional pair p:q with p and q valid indices starting from 1. A must be an m x n matrix and B an m x n matrix, or be both square. If p > q, the function returns 0.

Example:

```
> A := matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]
[ 7, 8, 9 ]
> linalg.matmat(A, A, 1, 3):
42
```

which is equal to:

```
> linalg.row(A, 1)*linalg.col(A, 3):
42
```

See also: **linalg.mattam**.

linalg.matnnorm (n, A [, p:q [, s:t]])

Calculates the n-norm of matrix A. By default the entire matrix will be processed, but you may limit this to rows p to q and/or columns s to t by passing the respective index ranges p:q, s:t as optional second and third arguments, in this order. The n-norm of a matrix is the sum of the n-norms of its row vectors.

With n > 1, the n-norm of a matrix is the sum of the n-norms of its row vectors. With n = 1, the function computes the one-norm of A, which is the maximum of the individual one-norms of its column vectors. With n = 2, returns the Frobenius norm.

Examples:

```
> A := matrix([[2, 3, 4, 1],[2, 2, 2, 2], [1, 2, 1, 3]]):

> linalg.matnnorm(1, A):
7

> linalg.matnnorm(2, A):
7.8102496759067
```

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.matonenorm (A [, p:q [, s:t]])

Calculates the one-norm of matrix `A`. By default the entire matrix will be processed, but you may limit this to rows `p` to `q` and/or columns `s` to `t` by passing the respective index ranges `p:q`, `s:t` as optional second and third arguments, in this order. The one-norm of a matrix is the maximum of the individual one-norms of its column vectors. Example:

```
> A := matrix([[2, 3, 4, 1],[2, 2, 2, 2], [1, 2, 1, 3]]):

> linalg.matonenorm(A):
7
```

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.matrix ([m, n,] obj_1, obj_2, ···, obj_n)
linalg.matrix ([m, n] [, la])
linalg.matrix (m, n, f)
linalg.matrix (str)

In the first form, creates a matrix from the given structures obj_k. The structures are considered to be row vectors. Valid structures are vectors created with **linalg.vector**, tables or sequences. With this style, the row and column dimensions `m`, `n` are purely optional.

In the second form, with `m` and `n` integers, creates an `m` x `n` matrix and optionally fills it row by row with the elements in the table or sequence `la`. If `m` and `n` are not given, the function determines the dimensions by examining `la`. `la` may include tables or sequences representing row vectors, but they must be of the same size. If `la` is not given, the matrix is filled with zeros.

The shorter alias **matrix** refers to **linalg.matrix**.

Examples:

```
> linalg.matrix([1, 2, 3], [4, 5, 6]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]
```

```
> matrix([[1, 2, 3], [4, 5, 6]]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]

> matrix(2, 3, [1, 2, 3, 4, 5, 6]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]

> matrix(2, 3, [[1, 2, 3], [4, 5, 6]]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]

> matrix(2, 3):
[ 0, 0, 0 ]
[ 0, 0, 0 ]
```

In the third form, with `m` and `n` the row and column dimensions and `f` a function of two variables, the function also sets all matrix components, iterating all rows and columns, in this order.

Example:

```
> linalg.matrix(3, 3, <: row, col -> | row - col | :>):

[ 0, 1, 2 ]
[ 1, 0, 1 ]
[ 2, 1, 0 ]
```

So, the call **linalg.matrix**(`m`, `n`, `f`) is equivalent to:

```
L := linalg.newmatrix(m, n);
rowdim, coldim := linalg.dim(L, true);
for i from 1 to rowdim do
   for j from 1 to coldim do
      L[i, j] := f(i, j)
   od
od;
```

In the fourth form the matrix is defined by a string `str` where the row vectors are separated by commas and the vector components by one or more white spaces. Carriage returns and newlines, if any, will be ignored. Example:

```
> linalg.matrix('1 2 3, 4 5 6, 7 8 9'):
[ 1, 2, 3 ]
[ 4, 5, 6 ]
[ 7, 8, 9 ]
```

The return is a table of the user-defined type 'matrix' and a metatable **linalg.mmt** assigned to the matrix. The table key `'dim'` contains a pair with the dimensions of the matrix: the left-hand side specifies the number of rows, the right-hand side the number of columns.

The numeric components of a matrix can either be standard Agena real or complex numbers, 80-bit long doubles (see **long** package), MAPM and MPFR values or double-to-double numbers (see **dd** package). Note that most **linalg**

functions will work only with matrices and vectors containing standard Agena (real) numbers.

See also: **linalg.newmatrix**, **linalg.vector**, **utils.readcsv**.

linalg.mattam (A, B, i, j [, p:q])

The function computes the scalar product of row vector `i` in matrix `A` and row vector `j` in matrix `B`. You can limit this to an index range `p` to `q` in the two rows by passing the optional pair `p:q` with `p` and `q` valid indices starting from 1. If `p` > `q`, the function returns 0.

Examples:

```
> A := matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]
[ 7, 8, 9 ]

> linalg.mattam(A, A, 1, 3):
50

> linalg.mattam(A, A, 1, 3, 2:3):
43

> linalg.mattam(A, A, 1, 3, 1:0):
0
```

See also: **linalg.matmat**.

linalg.mcopy (A)

Like the **copy** operator, this function creates a deep copy of matrix `A`.

See also **linalg.vcopy**.

linalg.meeq (A, B)

This function checks matrix `A` and matrix `B` for strict equality. The return is either **true** or **false**.

See also: == metamethod, **linalg.maeq**, **linalg.veeq**.

linalg.minor (A, r, c)

The function computes the minor of matrix `A` for a given row `r` and column `c`. The determinant of the minor is also returned.

```
> A := matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]]):
[ 1, 2, 3 ]
[ 4, 5, 6 ]
[ 7, 8, 9 ]
```

```
> linalg.minor(A, 1, 1):
[ 5, 6 ]
[ 8, 9 ]        -3
```

See also: **linalg.adjoint**, **linalg.det**, **linalg.submatrix**, **linalg.delcol**, **linalg.delrow**.

linalg.mmap (f, A [, ···])

This function maps a function `f` to all the components in the matrix `A` and returns a new matrix. The function must return only one value. See **linalg.vmap** for further information.

linalg.mmul (A, B)

This function multiplies an m x n matrix `A` with an n x p matrix `B`. The return is an m x p matrix. See also: ***** metamethod.

See also: **linalg.innerprod**, **linalg.multiply**, **linalg.mpow**.

linalg.mpow (A, n)

Raises a square matrix `A` to the power of `n` by multiplying `A` `n` times with itself in $O(\log_{2} n)$ time. `n` must be a positive integer.

See also: ****** and **^** metamethods, **linalg.mmul**, **linalg.multiply**.

linalg.mulrow (A, i, s)

Multiplies each element of row `i` in matrix `A` with the scalar `s` and returns a new matrix.

See also: **linalg.swapcol**, **linalg.swaprow**, **linalg.mulrowadd**.

linalg.mulrowadd (A, i, j, s)

Returns a copy of matrix `A` with each element in row `j` exchanged by the sum of this element and the respective element in row `i` multiplied by the number `s`.

See also: **linalg.swapcol**, **linalg.swaprow**, **linalg.mulrowadd**.

linalg.multiply (A, B)
linalg.multiply (A, v)
linalg.multiply (v, A)
linalg.multiply (v, w)

In the first form, the function multiplies an m x n matrix `A` with an n x p matrix `B`. The return is an m x p matrix.

In the second and third form, with A a matrix and v a vector, calculates the matrix-vector product A * v. The number n of components in v must be equal to the number of columns of A. The result is a row vector with n entries.

In the fourth form, returns the dot product of vectors v and w.

The function is a port of the Maple function of the same name and actually is a comfortable wrapper around **linalg.mmul**.

Examples:

```
>  A := linalg.matrix([[1, 2], [3, 4]]):
[ 1, 2 ]
[ 3, 4 ]

> linalg.multiply(A, A):
[  7, 10 ]
[ 15, 22 ]

>  v := linalg.vector([3, 4]):
< 3, 4 >

> linalg.multiply(A, v):
< 11, 25 >
```

See also: **linalg.crossprod**, **linalg.dotprod**, **linalg.innerprod**.

linalg.mzip (f, A, B [, ···])

This function zips together two matrices A, B by applying the function f to each of its respective components. The result is a new matrix m where each element m[i, j] is determined by m[i, j] := f(A[i, j], B[i, j]). If the f has more than two arguments, then its third to last argument must be given right after B.

A and B must have the same dimension.

See also: **linalg.vzip**, **linalg.mmap**.

linalg.ncolnorm (n, A, c [, p:q])

Calculates the n-norm of the column vector with index c in matrix A, with n a positive integer. By default, all rows will be processed but you may limit this by passing the lower row bound p and the upper row bound q as the pair p:q for the fourth argument.

The result is the sum of the absolute values, raised to the power of n, of all components in the column vector. The sum, finally, is taken to the n-th root before the function returns.

```
> A := matrix([[2, 3, 4, 1], [2, 2, 2, 2], [1, 2, 1, 3]]):
[ 2, 3, 4, 1 ]
[ 2, 2, 2, 2 ]
[ 1, 2, 1, 3 ]
```

```
> linalg.ncolnorm(2, A, 3):
4.5825756949558

> linalg.ncolnorm(2, A, 3, 2:3):
2.2360679774998
```

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.newmatrix (m, n [, def])

Creates a new matrix with `m` rows and `n` columns. If `def` is not **undefined**, explicitly sets `def` into each row vector, otherwise returns a sparse zero matrix. The function is much faster than **linalg.matrix** called with the same arguments.

linalg.nnorm (n, v [, p:q])

Calculates the `n`-norm of vector `v`, with `n` a positive integer. By default, the whole vector is traversed. You can limit this to an index range `p` to `q` by passing the optional pair `p:q` with `p` and `q` valid indices starting from 1. The return is the sum of the absolute values of the vector components, equal to

root(**sumup**(**map**(<: x -> |x|**`n` :>, `v`)), `n`).

Examples:

```
> v := vector([1, 2, 3, 4, 5, 6]);

> linalg.nnorm(1, v):
21

> linalg.nnorm(2, v):
9.5393920141695

> linalg.nnorm(1, v, 3:4):  # 1-norm, positions 3 to 4, only
7
```

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.norm (M [, n])

linalg.norm (v [, n])

The function returns the norm of a matrix or vector.

In the first form, without `n` given, the function returns the infinity norm of a matrix `M`. It is the maximum row sum, where the row sum is the sum of the absolute values of the elements in a given row. If `n` is 1, returns the one-norm and with `n` > 1 returns the n-norm.

In the second form, it returns the n-norm of a vector `v`, where `n` is a positive integer. (The n-norm of a vector is the n-th root of the sum of the magnitudes (absolute

values) of each element in v raised to the n-th power.) If `n` is **infinity**, the return is the infinity norm, i.e. the maximum magnitude of all elements `v`.

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.onecolnorm (A, v [, p:q])

Calculates the one-norm of the column vector given by its index `c` in matrix `A`. By default, all rows will be processed but you may limit this by passing the lower row bound `p` and the upper row bound `q` as the pair `p:q` for the third argument.

The result is the sum of the absolute values of all components in the column vector.

```
> A := matrix([[2, 3, 4, 1],[2, 2, 2, 2], [1, 2, 1, 3]]):
[ 2, 3, 4, 1 ]
[ 2, 2, 2, 2 ]
[ 1, 2, 1, 3 ]

> linalg.onecolnorm(A, 3):
7

> linalg.onecolnorm(A, 3, 2:3):  # column 3, rows 2 to 3
3
```

See also: **linalg.infcolnorm**, **linalg.infnorm**, **linalg.matinfnorm**, **linalg.matnnorm**, **linalg.matonenorm**, **linalg.ncolnorm**, **linalg.nnorm**, **linalg.onecolnorm**, **linalg.onenorm**.

linalg.onenorm (v [, p:q])

Calculates the one-norm of vector `v`. By default, the whole vector is traversed. You can limit this to an index range `p` to `q` by passing the optional pair `p:q` with `p` and `q` valid indices starting from 1. The return is the sum of the absolute values of the vector components, equal to

sumup(**map**(<: x -> |x| :>, `v`)).

Examples:

```
> v := vector([1, 2, 3, 4, 5, 6]);

> linalg.onenorm(v):
21

> linalg.onenorm(v, 3:4):
7
```

linalg.ones (m [, n])
linalg.ones (p)
linalg.ones (M)

Creates an m x n matrix with ones.

In the first form, if n is not given, it is set to m.

In the second form, the pair m:n is denoting the row dimension m and column dimension n.

In the third form, takes a matrix M and uses its dimensions to create the new matrix.

See also: **linalg.identity**, **linalg.zeros**.

linalg.outprodmatrix (v, w)

Creates an outer product matrix P from vectors v and w of any dimension, respectively, by multiplying each element i in v with each element j in w and setting P[i, j] = v[i]*w[j]. Example:

```
> v := < 1, 2, 3, 4 >

> w := < 6, 7, 8 >

> linalg.outprodmatrix(v, w):
[  6,  7,  8 ]
[ 12, 14, 16 ]
[ 18, 21, 24 ]
[ 24, 28, 32 ]
```

See also: * metamethod, **linalg.crossprod**, **linalg.dotprod**, **linalg.innerprod**, **linalg.kronprod**, **linalg.multiply**.

linalg.permanent (M [, false])

The function computes the permanent of square matrix M. By default, it uses an accurate but slow algorithm to compute the result. By passing the second argument **false**, you can use an alternative much faster one with a little less accuracy.

See also: **linalg.det**.

linalg.pivot (M, i, j [, p:q])

The function pivots the matrix M about M[i, j] which must be non-zero.

The call **linalg.pivot**(M, i, j) will add multiples of the i-th row to every other row in the matrix, with the result that the (k, j)-th entry of the matrix M is set to zero for all k not equal to i. That is, the j-th column of the matrix will be all zeros, except for the (i, j)-th element.

The call **linalg.pivot**(M, i, j, p:q) acts like **linalg.pivot**(M, i, j) except that only rows p through q are set to zero in the j-th column. Rows not in the range p:q are not affected.

See also: **linalg.gausselim**.

linalg.randmatrix (m [, n [, type [, borders]])

Creates a random matrix. With just m given, returns a dense square matrix of dimension m x m. With m and n given, computes a dense m x n matrix.

You can specify the type of matrix to be created by passing one of the following strings for type: 'dense' (the default), 'sparse' to create a sparse matrix, 'antisymmetric' or 'symmetric' if m = n and 'unimodular'.

By default, all matrix elements are in the range [-99 .. 99], but you can change this to another range by passing the pair p:q, with p and q integers with $p \leq q$, for fourth argument borders.

linalg.randvector (n [, borders])

Creates a random vector of dimension n, possibly with sparse entries.

By default, all vector elements are in the range [-99 .. 99], but you can change this to another range by passing the pair p:q, with p and q integers with $p \leq q$, for fourth argument borders.

linalg.rank (M)

Computes the rank of any m x n matrix M, by performing Gaussian elimination on the rows of the given matrix. The rank of the matrix M is the number of non-zero rows in the resulting matrix. The function also returns the determinant of **linalg.submatrix**(A, 1:n).

See also: **linalg.gausselim**.

linalg.reshape (M, m [, n])

Returns an m x n matrix whose elements are taken from the matrix M. The elements of the matrix are accessed in column-major order. If n is omitted, it is set to 1.

Example:

```
> a := linalg.matrix(3, 2, [1, 2, 3, 4, 5, 6]):
[ 1, 2 ]
[ 3, 4 ]
[ 5, 6 ]

> linalg.reshape(a, 2, 3):
[ 1, 3, 5 ]
[ 2, 4, 6 ]
```

See also: **linalg.extend**.

linalg.rotcol (A, i, j, c, s [, p:q])

Replaces the i-th column vector x in any m x n matrix A and the j-th column vector y in A by the vectors c*x + s*y and c*y - s*x, respectively.

By default all rows are changed, but you might limit the operation to rows p to q by passing the pair p:q as the sixth argument.

The return is a new matrix, with A left unchanged.

Examples:

```
> A := < < 2, 3, 4, 1 >, < 2, 2, 2, 2 >, < 1, 2, 1, 3 > >:
[ 2, 3, 4, 1 ]
[ 2, 2, 2, 2 ]
[ 1, 2, 1, 3 ]

> linalg.rotcol(A, 2, 4, 2, 3):
[ 2,  9, 4, -7 ]
[ 2, 10, 2, -2 ]
[ 1, 13, 1,  0 ]

> linalg.rotcol(A, 2, 4, 2, 3, 2:3):  # apply to rows 2 to 3 only
[ 2,  3, 4,  1 ]
[ 2, 10, 2, -2 ]
[ 1, 13, 1,  0 ]
```

linalg.rotrow (A, i, j, c, s [, p:q])

Replaces the i-th row vector x given in any m x n array A and the j-th row vector y in A by the vectors c*x + s*y and c*y - s*x. By default all columns are changed, but you might limit this to columns p to q by passing the pair p:q as the sixth argument.

The return is a new matrix, with A left unchanged. Example:

```
> A := < < 2, 3, 4, 1 >, < 2, 2, 2, 2 >, < 1, 2, 1, 3 > >:
[ 2, 3, 4, 1 ]
[ 2, 2, 2, 2 ]
[ 1, 2, 1, 3 ]
```

```
> linalg.rotrow(A, 1, 3, 2, 3):
[  7, 12,  11, 11 ]
[  2,  2,   2,  2 ]
[ -4, -5, -10,  3 ]
```

linalg.row (A, k)

Returns the k-th row from matrix A, as a vector.

See also: **linalg.col**, **linalg.submatrix**, **linalg.subvector**.

linalg.rowdim (A [, ···])

Determines the row dimension of the matrix A. The return is a number.

If you pass more than one argument, then a time-consuming check whether A is a matrix, is skipped.

A more direct way of determining the column dimension is `left(A.dim)`.

See also: **linalg.coldim**, **linalg.dim**.

linalg.rref (A [, v])

Returns the reduced row echelon form (RREF) of any m x n matrix A.

If a vector v is given, the function computes the reduced row echelon form of the augmented matrix A|v. In this case, A and v must have equal dimensions.

See also: **linalg.backsub**, **linalg.forsub**, **linalg.linsolve**.

linalg.scalarmul (v, n)
linalg.scalarmul (n, v)

Performs a scalar multiplication by multiplying each element in vector v by the number n. The result is a new vector.

linalg.scale (A [, p:q])

Normalises the (non-null) columns of a matrix A in such a way that, in each column, an element of maximum absolute value equals 1. The return is a new matrix where the normalised vectors are delivered in the corresponding columns.

p:q is a pair of column numbers where the scaling shall take place (from column p to and including column q), with **linalg.coldim**(A) the default, that is in all columns.

```
> A := < < 2, 3, 4, 1 >, < 2, 2, 2, 2 >, < 1, 2, 1, 3 > >;
```

```
> linalg.scale(A):
[   1,                1,    1, 0.33333333333333 ]
[   1, 0.66666666666667,  0.5, 0.66666666666667 ]
[ 0.5, 0.66666666666667, 0.25,                1 ]

> linalg.scale(A, 2:3):  # columns 2 to 3, only
[ 2,                1,    1, 1 ]
[ 2, 0.66666666666667,  0.5, 2 ]
[ 1, 0.66666666666667, 0.25, 3 ]
```

See also: **math.norm**, **stats.scale**.

linalg.setvelem (v, i, e)

Sets the `i`-th entry in vector `v` to value `e`. `e` can either be a standard Agena real or complex number, 80-bit long double (see **long** package), MAPM and MPFR value or double-to-double number (see **dd** package). Note that most **linalg** functions will work only with vectors and matrices containing standard Agena (real) numbers.

The function returns nothing. See also **linalg.getvelem**.

linalg.sparse (la)

Removes all set zeros from a vector or a matrix `la`, **in-place**, still implying zero but consuming less memory. The function returns the modified input structure.

See also: **linalg.issparse**, **linalg.zeros**.

linalg.stack (···)

Joins two or more matrices or vectors together vertically. Vectors are supposed to be row vectors. The matrices and vectors must have the same number of columns.

The return is a new matrix.

See also: **augment**, **linalg.augment**, **linalg.delcols**, **linalg.delrows**, **linalg.extend**, **linalg.reshape**, **zip**.

linalg.sub (v, w)

Subtracts vector `w` from vector `v`. The result is a new vector.

See also: **linalg.add**.

linalg.submatrix (A, p [, r])
linalg.submatrix (A, p:q [, r:s])

In the first form, returns column `p` from matrix `A` as a new row vector.

In the second form, returns columns `p` to `q` as a new matrix.

An optional third argument may be given to limit the extraction of the columns to the specified row `r` or rows `r` to `s`.

With the second and third arguments, you may mix numbers with pairs.

See also: **linalg.col**, **linalg.subvector**.

`linalg.subvector (la, r, c)`

The function extracts a specified subvector from a matrix or vector `la`. `r` and `c` are a table, sequence or register, a pair of positive integers, or a positive integer.

The return is the vector of `la` selected by the single index in either the row or column entry and the pair or table, sequence or register in the other entry.

Note that either the row or column entry must be a single index specifying the desired row or column.

Examples:

```
> A := < < 1, 2, 3 >, < 4, 5, 6 > >;

> linalg.subvector(A, 1:2, 2):
< 2, 5 >

> linalg.subvector(A, 2, [2, 1]):
< 5, 4 >

> A := < 4, 5, 6 >;

> linalg.subvector(A, 1, [2, 1]):
< 5, 4 >
```

See also: **linalg.submatrix**, **linalg.col**, **linalg.row**.

`linalg.swapcol (A, p, q [, s:t] [, true])`

Swaps column `p` in matrix `A` with column `q`. `p`, `q` must be positive integers. The result is a new matrix, by default.

If the very last argument is the Boolean **true**, then the operation is done in-place, modifying `A`. In this mode, the modified matrix `A` is returned, as well.

Whether in-place or not, you may limit the exchange of the matrix elements to columns `s` to `t` by passing the respective index range `s:t` as an optional fourth argument.

See also: **linalg.swaprow**, **linalg.mulrow**, **linalg.mulrowadd**, **linalg.transpose**.

linalg.swaprow (A, p, q [, s:t] [, true])

Swaps row `p` in matrix `A` with row `q`. `p`, `q` must be positive integers. The result is a new matrix.

Whether in-place or not, you may limit the exchange of the matrix elements to columns `s` to `t` by passing the respective index range `s:t` as an optional fourth argument.

See also: **linalg.swapcol**, **linalg.mulrow**, **linalg.mulrowadd**, **linalg.transpose**.

linalg.totable (la [, sparse=false])

Takes a vector or matrix `la` and converts it to a table with no metamethods and with all sparse elements in the row vector(s) of the input unset by default, that is not set to zero. The function does not change the input structure.

If the option `sparse`=**false** is being passed as a second optional argument, the function sets all sparse elements in a matrix or vector to zero and thus returns a dense table.

linalg.trace (A)

Computes the trace of a square matrix `A` and returns a number.

linalg.transpose (A)

Computes the transpose of an m x n-matrix `A` and thus returns an n x m-matrix. The [i,j]-th element of the result is equal to the [j,i]-th element of `A`, that is the function swaps rows with columns.

See also: **linalg.adjoint**, **linalg.inverse**, **linalg.swapcol**, **linalg.swaprow**.

linalg.unitvector (i, n [, ···])

Returns an `n`-dimensional (row) vector in which the `i`-th entry is one and all other entries are zero. Example:

```
> linalg.unitvector(2, 4):
< 0, 1, 0, 0 >
```

Instead of a row vector the function can alternatively return a column vector, that is an `n` x 1 matrix:

```
> linalg.unitvector(2, 4, column = true):
[ 0 ]
[ 1 ]
[ 0 ]
```

```
[ 0 ]
```

By default, the resulting vector is sparse. By passing the `sparse=**false**' option, all zeros will actually be physically set.

See also: **linalg.colvector**, **linalg.issparse**, **linalg.matrix**, **linalg.vector**, **linalg.vzero**.

linalg.vaeq (a, b)

This function checks vector `a` and vector `b` for approximate equality. The return is either **true** or **false**. The function uses Donald Knuth's approximation method to compare vector elements (see the **approx** function for information on how this works).

You can change the accuracy threshold epsilon with the **environ.kernel/eps** function.

See also: ~= metamethod, **approx**, **linalg.veeq**, **linalg.maeq**.

linalg.vcopy (v)

Like the **copy** operator, this function creates a deep copy of vector `v`.

linalg.vectdim (v)

Checks for a vector and returns its dimension. This is just an alias to **linalg.checkvector**.

See also: **linalg.dim**.

linalg.vector (a1, a2, ···)

linalg.vector (l)

linalg.vector (n, [a1, a2, ···])

linalg.vector (n, [ ])

linalg.vector (str)

Creates a vector with numeric components `a1`, `a2`, etc. (first form). The function also accepts a table, sequence or register `l` of elements `a1`, `a2`, et cetera (second form).

The numeric components `a1`, `a2`, etc. can either be standard Agena real or complex numbers, 80-bit long doubles (see **long** package), MAPM and MPFR values or double-to-double numbers (see **dd** package). Note that most **linalg** functions will work only with vectors and matrices containing standard Agena (real) numbers.

Examples:

```
> linalg.vector(1, 2, 3):
< 1, 2, 3 >

> linalg.vector([1, 2, 3]):
< 1, 2, 3 >
```

Both versions are equivalent to the < ... > notation:

```
> < 1, 2, 3 >
< 1, 2, 3>
```

In the third form, with the second argument a table, `n` denotes the dimension of the vector, and a_k might be single values or key~value pairs. Missing components in the row vector automatically default to zero. This allows you to create memory-efficient sparse vectors and matrices:

The shorter alias **vector** refers to **linalg.vector**.

```
> linalg.vector(3, [2 ~ 1]):  # first and third component are missing
< 0, 1, 0 >
```

In the fourth form, a sparse zero vector of dimension `n` will be returned.

The fifth form allows to define a vector given by string `str` with the vector components separated by one or more white spaces:

```
> vector(' 1  2 3 '):
< 1, 2, 3 >
```

The result is a table of the user-defined type 'vector' and the **linalg.vmt** metatable assigned to allow basic vector operations with the operators **+**, **-**, *****, unary minus and **abs**. The table key `'dim'` contains the dimension of the vector created.

See also: **linalg.colvector**, **linalg.issparse**, **linalg.matrix**, **linalg.unitvector**, **linalg.vzero**.

linalg.veeq (a, b)

This function checks vector `a` and vector `b`. for strict equality. The return is either **true** or **false**.

See also: == metamethod, **linalg.meeq**, **linalg.vaeq**.

linalg.viszero (v [, epsilon])

Checks whether vector `v` has only zero elements and returns **true** or **false**. The function returns the index of the first non-zero element in `v` - if it exists - as a second result, or 0 if `v` is a zero vector. The non-zero element is returned as a third result, too.

By default, the check is done against strict zero. You can change this by passing a positive `epsilon` value as an optional second argument so that all vector elements x with |x| <= `epsilon` will be considered zero.

Example:

```
> linalg.viszero(vector(0, 0, 1)):
false   3

> linalg.viszero(vector(0, 0, +++0), hEps):
true    0
```

See also: **Eps**, **hEps**, **DblEps**, **math.epsilon**, **linalg.iszero**, **tables.iszero**

linalg.vmap (f, v [, ···])

This operator maps a function `f` to all the components in vector `v` and returns a new vector. The function `f` must return only one value.

If function `f` has only one argument, then only the function and the vector are passed to **linalg.vmap**. If the function has more than one argument, then all arguments *except the first* are passed right after the name of the vector.

Examples:

```
> linalg.vmap(<: x -> x^2 :>, < 1, 2, 3 > ):
< 1, 4, 9 >

> linalg.vmap(<: (x, y) -> x > y :>, < 1, 0, 1 >, 0): # 0 for y
< true, false, true >
```

See also: **linalg.vzip**, **linalg.mmap**, **linalg.mzip**.

linalg.vzero (n [, option])

Creates a zero vector of length `n` with all its components physically set to 0 if `option` is set to **false** (the default). The vector is sparse by default, equal to `linalg.vector(n, [])`. If you want to create a dense zero vector of dimension `n`, pass the `sparse`=**false** option as the last argument.

See also: **linalg.issparse**, **linalg.sparse**.

linalg.vzip (f, v1, v2 [, ···])

This function zips together two vectors by applying the function `f` to each of its respective components. The result is a new vector v' where each element v'[k] is determined by v'[k] := f(`v1`[k], `v2`[k]).

`v1` and `v2` must have the same dimension. The third to last argument to `f` must be given right after `v2`.

See also: **linalg.vmap**, **linalg.mmap**.

linalg.zeros (m [, n [, option]])
linalg.zeros (p [, option]])
linalg.zeros (M [, option])

By default, creates an `m` x `n` sparse matrix, with unset elements implicitly representing zero.

In the first form, if `n` is not given, it is set to `m`.

In the second form, the pair `m:n` is denoting the row dimension `m` and column dimension `n`.

In the third form, takes a matrix `M` and uses its dimensions to create the new matrix.

In all forms, if the option `sparse`=**false** is given as the last argument, the function sets zeros explicitly into all the row vectors, thus returning a dense matrix.

See also: **linalg.issparse**, **linalg.ones**, **linalg.sparse**.

The metamethods in the **linalg** package available for vectors are:

Operator	Name	Description
+	'__add'	addition of two vectors
-	'__sub'	subtraction of two vectors
*	'__mul'	multiplication with a scalar, dot product or matrix-vector product
/	'__div'	division by a scalar
=, ==, <>	'__eq', '__eeq'	equals, equals, not equals
~=, ~<>	'__aeq', '__naeq,	approximately equals, approximately not equals
abs	'__abs'	length of a vector
size	'__size'	dimension of a vector
recip	'__recip'	negates all vector components
v[i]	'__index'	returns the i-th component of a vector; see also **linalg.getvelem**
v[i] = val	'__writeindex'	sets vector component i to val; if val is **null**, implicitly sets it to zero; see also **linalg.setvelem**
n/a	'__tostring'	conversion to a string, e.g. for the pretty printer

The metamethods available for matrices are:

Operator	Name	Description
+	'__add'	addition of two matrices
-	'__sub'	subtraction of two matrices
*	'__mul'	multiplication with a scalar, matrix-matrix or matrix-vector multiplication
**, ^	'__ipow', '__pow'	exponentiation with an integer, same as calling **linalg.mpow**
/	'__div'	divides all matrix components by a scalar
=, ==, <>	'__eq', '__eeq'	equals, equals, not equals
~=, ~<>	'__aeq', '__naeq,	approximately equals, approximately not equals
abs	'__abs'	length of a vector
size	'__size'	dimensions of a matrix, returns a pair with the row dimension and the column dimension
recip	'__recip'	inverts a matrix, same as calling **linalg.inverse**
v[i, j]	'__index'	returns an element of a matrix, with i the row position and j the column position
n/a	'__tostring'	conversion to a string, e.g. for the pretty printer

11.14 stats - Statistics

This package contains procedures for statistical calculations and operates completely on tables.

You might want to use **utils.readcsv** to read distributions from a CSV file, see the sample session below.

Summary of functions:

Averages:

stats.accu, **stats.amean**, **stats.ema**, **stats.gema**, **stats.gmean**, **stats.gsma**, **stats.gsmm**, **stats.hmean**, **stats.iqmean**, **stats.median**, **stats.mean**, **stats.midrange**, **stats.qmean**, **stats.sma**, **stats.smm**, **stats.trimean**, **stats.trimmean**.

Deviations:

stats.ad, **stats.chauvenet**, **stats.durbinwatson**, **stats.ios**, **stats.mad**, **stats.md**, **stats.sd**, **stats.spread**, **stats.ssd**, **stats.var**.

Distributions:

stats.laplace, **stats.logistic**, **stats.geometric**, **stats.cauchy**, **stats.chisquare**, **stats.cdfnormald**, **stats.fratio**, **stats.gammacdf**, **stats.gammad**, **stats.gammadc**, **stats.gammapdf**, **stats.hypergeom**, **stats.logseries**, **stats.poissond**, **stats.studentst**.

Density:

stats.cdf, **stats.nde**, **stats.ndf**, **stats.pdf**, **stats.poisson**.

Extrema:

max, **min**, **stats.colnorm**, **stats.extrema**, **stats.minmax**, **stats.peaks**, **stats.rownorm**, **stats.smallest**.

Occurrences:

stats.countentries, **stats.freqd**, **stats.isall**, **stats.isany**, **stats.mode**, **stats.obcount**, **stats.obpart**.

Ranges:

stats.fivenum, **stats.iqr**, **stats.percentile**, **stats.prange**, **stats.qcd**, **stats.quartiles**.

Sums:

qsumup, **sumup**, **stats.cumsum**, **stats.fsum**, **stats.lse**, **stats.moment**, **stats.sumdata**, **stats.sumdataln**, **stats.var**.

Probability density functions:

stats.cauchy, **stats.cdfnormald**, **stats.chisquare**, **stats.F**, **stats.Fc**, **stats.fratio**, **stats.gammad**, **stats.gammadc**, **stats.invF**, **stats.invnormald**, **stats.lognormald**, **stats.normald**, **stats.probit**, **stats.studentst**.

Correlation Functions:

stats.acf, **stats.acv**, **stats.besselj**, **stats.besselk**, **stats.circular**, **stats.constant**, **stats.covar**, **stats.cubic**, **stats.dampedcos**, **stats.dampedsin**, **stats.exponential**, **stats.gaussian**, **stats.hole**, **stats.linear**, **stats.matern**, **stats.penta**, **stats.power**, **stats.ratquad**, **stats.spherical**, **stats.white**.

Miscellaneous:

stats.checkcoordinate, **stats.dbscan**, **stats.deltalist**, **stats.fprod**, **stats.herfindahl**, **stats.issorted**, **stats.kurtosis**, **stats.neighbours**, **stats.scale**, **stats.skewness**, **stats.sorted**, **stats.tovals**.

A sample session:

The **utils.readcsv** function is quite handy to read in data from a comma-separated or other text file.

Suppose we have a file containing data from Viking Lander 2 which touched down on Mars in 1976 to explore the environment:

```
True/No Data (C);Year;Solar Long;Sol;Wind Speed m/s;Wind Dir;Pressure mb;Temp F;Temp C
C;1;117.993;1.02;0;0;0;-459.67;-273.15
T;1;118.012;1.06;1.5;79;7.82;-97.69;-72.05
T;1;118.031;1.1;1.7;65;7.82;-106.44;-76.91
T;1;118.05;1.14;2.5;67;7.83;-111.82;-79.9
```

The first line contains the header describing the various fields, the rest contains valid or invalid data sets. We are only interested in:

- valid samples, the ones not marked with a 'C' character at the start of each line,
- the Sol which is the Martian date (column 4, field name 'Sol') and
- the temperature in Celsius (column 9, field name 'Temp C').

The binary Agena distributions all contain the sample file 'viking2.csv'; you will usually find it in the following paths:

- OS/2: c:/programs/agena/data/viking2.csv,
- Windows: c:/Program Files (x86)/Agena/data/viking2.csv,
- UNIX: /usr/agena/data/viking2.csv.

Let us read in the file:

```
> L := utils.readcsv(
>   'c:/Program Files (x86)/Agena/data/viking2.csv',
>   fields = ['Sol', 'Temp C'],# extract only these fields, or pass fields=[4, 9]
>   header = true,   # the file actually has a header
>   delim = ';',     # the fields are separated by a semicolon (the default)
>   ignore = <: x -> x[1] = 'C' :>  # skip all lines starting with that mark
> );
```

We put the Sol dates and the temperatures into different sequences:

```
> sol := columns(L, 1)

> temp := columns(L, 2)
```

To check whether the observation is consistent, we can use **stats.chauvenet** to search for possible invalid data:

```
> stats.chauvenet(temp):
seq(-273.15, -273.15, -273.15, -273.15, -273.15, -273.15, -273.15, -273.15,
-273.15)
```

There are actually some sets with 0 Kelvin = -273.15 degrees Celsius not marked as invalid. We just remove them and reassign sequences sol and temp again:

```
> remove( <: x -> x[2] = -273.15 :>, L, inplace = true):

> sol := columns(L, 1)

> temp := columns(L, 2)
```

Just to be sure:

```
> stats.chauvenet(temp):
seq()
```

Now let us try some functions. To get the lowest and the highest temperature, just enter:

```
> stats.minmax(temp):
seq(-121.01, -22.24)
```

or separately where the first result is the minimum or maximum and the second result denotes the index position in the distribution:

```
> stats.min(temp):
-121.01 4938

> stats.max(temp):
-22.24  14009
```

We compute the arithmetic mean of the temperature values:

```
> stats.amean(temp):
-82.040489308501
```

The median:

```
> stats.median(temp):
-86.29
```

For short, to get the first quartile, the median and the third quartile of a distribution, along with the minimum, the maximum observation, and the arithmetic mean, in this order, enter:

```
> stats.fivenum(temp):
seq(-103.43, -86.29, -65.19, -121.01, -22.24, -82.040489308501)
```

Standard deviation:

```
> stats.sd(temp):
24.90810361302
```

In many cases we would like to know the corresponding Sol values:

The minimum temperature is at position

```
> whereis(temp, min(temp)):
seq(4938)
```

of the distribution, so the corresponding Sol date is:

```
> sol[4938]:
211.02
```

that is 211.02 Marsian days after landing.

The functions:

A general note: almost all of the statistics functions ignore the **undefined** value should it be part of a distribution. Any non-numeric values in a distribution are replaced with zeros. Most of the following functions also process numarrays.

To reduce round-off errors, it is always a good idea to sort structures before applying **stats** functions that sum up data, see **stats.sorted** and **numarray.sorted**.

Also note that when a distribution is stored to a table and this table has holes, you might get wrong results - so apply **tables.entries** to the respective table before passing it to one of the **stats** functions.

stats.accu ([true])

Returns an iterator that computes the running mean, variance, median, and absolute deviation by mere accumulation of individual observations in the distribution obj.

If the first optional argument **true** is passed, then sample values are computed (division by the number of observations - 1), otherwise population values are computed (division by the number of observations).

If the resulting iterator is called without any argument, the current results are returned in a table.

If the resulting iterator is called with a number, i.e. an observation, it is added to the accumulators.

While the mean and variance computed are correct, the median and the absolute deviation are approximations only. The function ignores non-finite values, i.e. **undefined** and +/-**infinity**.

You may use this function if a distribution is too large to be stored in a structure.

Example:

```
> p := stats.accu()

> p(1), p(2), p(3);

> p():
[ad ~ 0.64395055085938, mean ~ 2, median ~ 2.46875, var ~ 0.66666666666667]

> p(4):

> p():
[ad ~ 0.91597561503675, mean ~ 2.5, median ~ 2.8408203125, var ~ 1.25]
```

The idea has been taken from the Stat package shipped with Digital Equipment Corporation Critical Mass Modula-3. 2.10.1.

stats.acf (obj, lag, [, option [, m [, s]]])

Returns the autocorrelation of a distribution `obj` (a table, sequence, register or numarray) of numbers at a given `lag`, a non-negative integer. If any third argument `option` different from **null** is passed, then the un-normalised autocorrelation will be returned. The return is a number,

$$\sum_{i=1}^{n\text{-}lag} (obj_i - \mu)(obj_{i+lag} - \mu)$$

where n is the number of observations, and μ is the arithmetic mean of the distribution. If no `option` is passed, the sum is divided by the variance of `obj` multiplied by n, yielding a normalised result. The function uses Kahan-Ozawa round-off error prevention.

To speed up computation times significantly, you may also pass a precomputed mean `m` and the sum `s` of all values in the distribution.

It may be used to detect periodicy in a time series.

A distribution is autocorrelated if **stats.acf** returns a negative or positive value significantly different from zero. The - normalised - return is in the range [-1, 1], where +1 denotes perfect autocorrelation and -1 with 1 perfect anti-correlation. A negative correlation indicates that higher values of a distribution are related to lower values.

See also: **stats.acv**.

stats.acv (obj, p, [, option])

Depending on the type of the observation `obj`, returns a table, sequence, register or numarray of autocorrelations starting with lag = 0, through and including the given number `p` of lags. If any third argument `option` is passed, then un-normalised autocorrelations are returned. For the formula and numeric method used, see **stats.acf**.

stats.ad (obj [, options])

Computes the absolute (or mean) deviation of all the numbers or complex numbers in a table, sequence or numarray `obj`, i.e. the mean of the equally likely absolute deviations from the arithmetic mean μ:

$$\frac{1}{n}\sum_{i=1}^{n}\left|obj_i - \mu\right|$$

Regardless of the input, the return is always a number.

If the option `coeff`=**true** is given, then the variation coefficient will be returned:

$$\frac{1}{n}\sum_{i=1}^{n}\left|obj_i - \mu\right| \;/\; |\,\mu\,|$$

Absolute deviation is more robust than standard deviation since it is less sensitive to outliers. The function uses Kahan-Babuška round-off error prevention.

If `obj` is empty or contains less than two elements, **fail** will be returned.

See also: **stats.ios**, **stats.mad**, **stats.md**, **stats.sd**.

stats.amean ([f,] obj)

Divides each number or complex number in a table, sequence or numarray `obj` by the size of `obj` and sums up the quotients to finally return the arithmetic mean. It is equivalent to:

$$\sum_{i=1}^{n}\frac{obj_i}{n}$$

By dividing each element before summation, the function avoids arithmetic overflows and also uses the Kahan-Babuška algorithm to prevent round-off errors during summation. Thus the function is more robust than **stats.mean**.

You may mix numbers and complex numbers in `obj`.

If `obj` is table, it is assumed to be an array, non-positive integral keys (including strings, etc.) are ignored.

The function returns **fail** if `obj` contains less than two elements.

If `obj` is empty, **fail** will be returned. The function does not ignore **undefined**s, so you may delete them from the observation before calling the function.

Please note that if `obj` includes non-numbers, where **undefined** is considered a number, they are interpreted as zeros which might unexpectedly influence the result.

If you pass a function `f` defining a Boolean condition, then only the observations in `obj` that satisfy this condition will be processed, for example:

```
> s := sequences.new(1, 10);

> stats.amean(s):  # mean of all the values
5.5

> stats.amean(<: x -> even x :>, s):  # mean of the even values only
6
```

If you need speed, you might consider using **stats.meanvar** which has the same accuracy as **stats.amean** does.

See also: **remove**, **math.agm**, **stats.accu**, **stats.gmean**, **stats.hmean**, **stats.mean**, **stats.meanvar**, **stats.qmean**, **stats.sma**, **stats.trimmean**.

stats.beta (x, nu1, nu2)

Computes the probability density function 1/**beta**(`nu1`, `nu2`) * x^{nu1-1}* $(1-x)^{nu2-1}$ of the Beta distribution. `x` can be any number and both `nu1` and `nu2` must be positive integers.

stats.binomd (k, n, p)

The function simplifies to the cumulative probability binomial distribution function defined as:

$$\sum_{j=0}^{\min(k,n)} \textbf{stats.binompdf}(j, n, p)$$

using Kahan-Babuška summation to minimise round-off errors.

See also: **stats.invbinomd**, **long.binomd**.

stats.binompdf (k, n, p)

The function simplifies to the binomial probability density defined as:

$$\binom{n}{k} p^k (1-p)^{n-k}.$$

See also: **stats.negbinompdf**.

stats.brownian (s, t)

The function computes the Brownian correlation function. The distributions `s`, `t` can be tables, sequences, registers or numarrays. The output depends on the type of the input and `s`, `t` should include non-negative numbers. `s`, `t` may be of different size.

If `s`, `t` are not numarrays, the function automatically switches to complex arithmetic and you may find complex numbers in the return, which is a two-dimensional m x n structure with the correlations, if at least one sample is negative.

With any numarray as input, the correlation matrix will always be of type 'double' with real results.

The function is written in Agena and included in the lib/library.agn file.

stats.card (obj [, f [, ···]] [, options])

By default, returns the number of samples in table, sequence, register or numeric double array `obj`. The function works in-place to avoid internal duplication of observations, and does not modify the structure.

If you pass a univariate or multivariate function `f` defining a Boolean condition, then only the samples in `obj` that satisfy this condition will be counted. Put any second, third, etc. argument to `f` right after `f`.

You may pass the following `options`:

- With the `meanvar`=**true** option the function returns the cardinality, the arithmetic mean and the population variance (see next), in this order. The default is **false**.
- The `sample`=**true** option forces the function to return the cardinality, arithmetic mean and sample variance instead of the population variance. The default is **false**.
- When given the `optimised`=**true** option, the function more strictly tries to avoid numeric overflow with the variance if there are very large samples in the distribution. The default is **false**.

With the `meanvar` and `sample` options, the arithmetic mean and the variance will be computed using a one-pass `running` algorithm developed by B. P. Welford to prevent round-off errors.

The arithmetic mean and variance returned have very good accuracy, sometimes even beating those computed by **stats.amean** and **stats.var**.

You may sort the distribution prior to the call to improve precision of the results.

See also: **stats.meanmed**, **stats.meanvar**.

stats.cauchy (x, a, b)

The cauchy[a, b] distribution has the probability density function:

$$1/(\pi * b * (1 + ((x-a)/b)^2)),\ b > 0.$$

See also: **stats.chisquare**, **stats.fratio**, **stats.normald**, **stats.studentst**.

stats.cdf (a, b [, μ [, σ]])

Computes the cumulative density function between the lower bound a and the upper bound b. If the mean μ is not given, it defaults to 0; if the standard deviation σ is not given, it defaults to 1.

The return is the number:

$$\frac{1}{\sigma\sqrt{2\pi}} \int_a^b e^{\frac{-(x-\mu)^2}{2\sigma^2}} dx$$

See also: **stats.nde**, **stats.ndf**, **stats.pdf**.

stats.cdfnormald (x)

Implements the cumulative density function for the standard normal distribution. The return is the number:

$$\frac{1}{\sqrt{2\pi}} \int_{t=-\infty}^{x} e^{\frac{-t^2}{2}} dt$$

See also: **long.cdfnormald**, **stats.invnormald**, **stats.normald**.

stats.chauvenet (obj [, x] [, option, ···])

Receives a table or sequence obj of *normally distributed* numbers and checks them for outliers using the formula:

$$p := n * \mathrm{erfc}(|x - \mu| / sd),$$

where n is the number of observations in a distribution, x a sample of it, μ the arithmetic mean $\mu = \sum_{i=1}^{n} \frac{obj_i}{n}$ and sd the standard deviation $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(obj_i - \mu)^2}$.

If at least obj and x is given, the function checks whether the number x is an outlier by conducting a 1-pass check and returns **true** or **false** plus the probability p. The smaller p, the more likely x is an outlier.

If obj but not x is passed, however, the procedure discards outliers non-destructively, recomputes the updated arithmetic mean and standard

deviation as long as it does not find any remaining outliers, and returns them in a structure, its type defined by the type of `obj`.

By default, if p < 0.5, where 0.5 is the magical Chauvenet number, an outlier is detected. If you pass the option `bailout`=p, then p, a non-negative number, will be the threshold.

If you pass the option `jump`=true, as soon as an outlier is detected, it is removed from the distribution and then the whole evaluation process is restarted immediately with a reduced distribution along with a re-computed mean and deviation.

If you do not, all remaining items are also checked according to the current criteria - after the last item has been checked, only then the outliers are removed from the distribution, the mean and deviation are re-computed and another iteration begins.

If you pass the option `mean`=f, where f is a procedure, then the mean μ is determined by f. The default is f = **stats.amean** which computes the arithmetic mean by avoiding numeric overflow.

If you pass the option `dev`=f, where f is a procedure, then the deviation is determined by this procedure f. The default is f = **stats.sd** which computes the standard deviation.

If you pass the option `outlier='lower'` or `outlier='upper'`, then the function only checks for lower or upper outliers, respectively.

With the option `strict`=**false**, the function uses a one-pass approach, computing the mean and standard deviation only once. It computes the limits for regular values analytically by calling **calc.zeros** and then checks each sample in `obj` whether it is an outlier.

Further information: `Cleaning Data the Chauvenet Way`, by Lily Lin and Paul D. Sherman, published at the South East SAS Users Group's website http://www.sesug.org.

The function is implemented in Agena and included in the lib/library.agn file.

stats.checkcoordinate (c [, procname])

The function checks whether the given co-ordinate `c` is a pair x:y with both its left-hand and right-hand side x and y being numbers. If a second argument, a string, is given, then error messages of **stats.checkcoordinate** refer to the given procedure `procname` as the function issuing the error. Otherwise the error message includes a reference to **stats.checkcoordinate**.

The function returns the numbers x and y and issues an error otherwise.

stats.chisquare (x, nu)

The chisquare[nu] distribution has the probability density function:

$$x\,\hat{}\,((nu-2)/2)\ \exp(-x/2)/2\,\hat{}\,(nu/2)\ /\ \Gamma(nu/2),$$

with $x > 0$ and nu a positive integer.

See also: **stats.cauchy**, **stats.fratio**, **stats.normald**, **stats.studentst**.

stats.colnorm (obj)

Returns the largest absolute value of the numbers in the table, sequence, register or numarray obj, and the original value with the largest absolute magnitude. If obj includes **undefined**s, they are ignored. If the structure obj consists entirely of one or more **undefined**s, then the function returns the value **undefined** twice. If the structure is empty, **fail** will be returned.

See also: **stats.scale**, **stats.rownorm**.

stats.countentries (obj [, f [, ···]])

Counts the number of occurrences of each entry in a table, sequence, register or numarray obj and returns a dictionary with its respective key the entry and its value the number of occurrences.

You might optionally pass a procedure f to be mapped on the structure before counting begins on the thus modified structure. If f has more than one argument, then its *second* to last argument must be given right after f.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **countitems**, **bags** package.

stats.covar (x, y [, sample])

Computes the covariance of distributions x and y, which is:

$$\text{cov}(x, y) = [1/n \text{ or } 1/(n-1)] * \sum_{i=1}^{n} (x[i] - \text{mean}(x) * (y[i]-\text{mean}(y)).$$

By default, the population covariance will be returned. If you pass any third argument then the sample covariance will be computed.

The covariance indicates the tendency in the linear relationship between two distributions: if it is positive, the distributions are similar, if it is negative, they are not.

stats.cumsum (obj)

Returns a structure of the cumulative sums of the numbers in the table, sequence, register or numarray `obj`.

The type of return is determined by the type of `obj`.

The function returns **fail** if `obj` contains less than one element. It may also return a structure containing **undefined** and/or **infinity** if `obj` includes non-numbers.

See also: **sumup**, **calc.fsum**, **stats.fsum**, **stats.sumdata**.

stats.dbscan (obj, eps, minpts [, option])

The functions finds clusters in a sequence `obj` of n-dimensional points and returns a table with the individual clusters along with their respective points.

It also returns a register of the size of the whole distribution listing the cluster number associated with each point, where the point in this case is represented by its integral position in the sequence `obj`.

The co-ordinates of points in `obj` may be represented by pairs (2-dimensional space, only), sequences (any space), or vectors created by **linalg.vector** (any space).

`eps` is the maximum allowed distance between two points that shall belong to the same neighbourhood. `minpts` is the minimum number of points that shall constitute a neighbourhood.

By specifying the `'select'` option along with a function returning a Boolean, e.g. `'select':<: x -> right x < 1 :>`, only points satisfying the given criterion are examined.

By specifying the `'method'` option, you can control how the function determines clusters: `'method':'original'` uses the classic one, `'method':'modified'` uses a much faster and memory-saving implementation that contrary to the original method immediately flags neighbours of neighbours as being visited and thus does not examine them again in further passes. The default is `'original'`.

See also: **stats.neighbours**.

stats.deltalist (obj [, option])

Returns a structure of the deltas of neighbouring elements in the table, sequence, register or numarray `obj`. If the value **true** is given as an option, then absolute differences are returned.

The type of return is determined by the type of `obj`.

Please note that the difference between **undefined** and a number is **undefined**, and that the difference between **infinity** and a number is ±**infinity**.

The function returns **fail** if `obj` contains less than two elements.

See also: **stats.ios**.

stats.durbinwatson (obj)

The Durbin-Watson test detects the autocorrelation in the residuals from a linear regression and returns

$$d = \sum_{i=2}^{n} (obj_i - obj_{i-1})^2 \; / \; \sum_{i=1}^{n} obj_i^2$$

If d is equal to 2, it indicated the absence of autocorrelation. If d is less than 2, it indicates positive autocorrelation; if d is greater than 2 it indicates negative autocorrelation and that the observations are very different from each other. If d is less than 1, the regression should be checked. The function uses Kahan-Babuška roundoff prevention. `obj` may be a table, sequence or numarray.

stats.ema (obj, k, alpha [, mode [, y0star]])

Computes the exponential moving average of a table or sequence `obj` up to and including its `k`-th element.

The smoothing factor `alpha` is a rational number in the range [0, 1].

The function supports two algorithms: If `mode` is 1 (the default), then the algorithm

```
r := alpha * obj[k];
s := 1 - alpha;
for i from k - 1 to 1 by -1 do
   r +:= alpha * s ^ i * obj[i]
od;
r := r + s ^ k * y0star;
```

is used to compute the result r. In `mode` 1, you can pass an explicit first estimate `y0star`, otherwise the first value `y0star` is equal to the sample moving average of `obj`. If `mode` is 2, then the formula

```
r := obj[k];
for i from k - 1 to 1 by -1 do
   r +:= alpha * (obj[i] - r)
od;
```

is applied.

The result is a number.

See also: **stats.gema**.

stats.extrema (obj, delta)

Expects a sequence or table `obj` of points x_k:y_k and the number `delta` and determines the local minima and maxima.

A value y_k is considered an extrema if the difference to its surrounding is at least `delta`. The function returns two structures of pairs, i.e. points, the first one including the local minima, the second one the local maxima.

The type of the structures is determined by the type of `obj`.

The function is implemented in Agena and included in the lib/library.agn file.

stats.F (df1, df2, x)

Returns the area from zero to non-negative `x` under the F density function (also known as Snedcor's density, the variance ratio density or F distribution for short). This is the density of x = (u1/`df1`)/(u2/`df2`), where u1 and u2 are random variables having Chi square distributions with `df1` and `df2` degrees of freedom, both positive integers, respectively.

The incomplete beta integral is used, according to the formula

P(x) = **calc.ibeta**(`df1`/2, `df2`/2, (`df1`*x/(`df2` + `df1`*`x`)).

See also: **stats.Fc**, **stats.invF**.

stats.Fc (df1, df2, p)

Computes the complemented F distribution by finding the F density argument x such that the integral from x to infinity of the F density is equal to the given probability `p`, a non-negative number.

This is accomplished using the inverse beta integral function and the relations, with `df1`, `df2` positive integers:

z = **calc.ibeta**(`df2`/2, `df1`/2, `p`)
x = `df2`*(1-z) / (`df1`*z).

See also: **stats.F**, **stats.invF**.

stats.fivenum (obj [, rule])

Returns the first quartile, the median, and the third quartile of a distribution `obj`, in this order. `obj` may be a table, sequence, register or numarray, and the result will be put into a structure of the same type as `obj`. It also includes the minimum and the maximum observation, along with the arithmetic mean, in this order.

The first and third quartiles are computed according to the NIST rule, see **stats.percentile** for further information. Instead of the NIST rule, with the second argument `rule` you can also pass the strings `'excel'` or `'wikipedia'` for the Excel or Wikipedia ways of computing them, respectively.

If the elements in `obj` are not sorted in ascending order, the function automatically sorts them non-destructively, and any non-numeric values are converted to zeros.

See also: **stats.quartiles**.

stats.fprod (f, obj [, a [, b [, ···]]])

Applies the function `f` onto all elements in the table, sequence, register or numarray `obj` and then multiplies the results. The return is the number:

$$\prod_{i=a}^{b} f(obj_i)$$

If `a` is not given, `a` is set to 1. If `b` is not given, `b` is set to the number of elements in `obj`. If `f` is a multivariate function, its second, third, etc. argument must be passed after `b`.

See also: **calc.fsum**, **stats.fsum**, **stats.sumdata**.

stats.fratio (x, nu1, nu2)

The Fisher's F distribution, also known as fratio distribution, has the probability density function

$$\Gamma(\ (nu1+nu2)/2\ )\ /\ \Gamma(nu1/2)/\Gamma(nu2/2)*(nu1/nu2)\ \hat{}\ (nu1/2)\ *$$
$$x\ \hat{}\ ((nu1-2)/2)\ /\ \ (1+\ (nu1/nu2)*x)\ \ \hat{}\ \ ((nu1+nu2)/2)$$

with $x > 0$, `nu1` and `nu2` positive integers.

See also: **stats.cauchy**, **stats.chisquare**, **stats.normald**, **stats.studentst**.

stats.freqd (obj, p [, n])
stats.freqd (obj, p [, option])

For table, sequence or register `obj`, the function computes a frequency distribution function that each time it is called, returns both the start of the respective subinterval (not the class number) defined by pair `p` and step size or number of classes and the number of samples in this subinterval/class. If the distribution has been completely traversed, two **nulls** are returned.

For more information on the arguments to be passed and the values returned, please refer to the description of **stats.obcount**.

Example:

```
> s := seq(-1, 0, 0.1, 0.2, 0.3, 0.4, 1, 1.1, 2, 2.1)

> stats.obcount(s, 0:2, 0.5):
[0 ~ 5, 0.5 ~ 0, 1 ~ 2, 1.5 ~ 1]        [-1, 2.1]

> f := stats.freqd(s, 0:2, 0.5);

> f():
0       5

> f():
0.5     0
```

The function is implemented in Agena and included in the lib/library.agn file.

If `obj` is a register, all its values should be numbers.

See also: **stats.obcount**.

stats.fsum (f, obj [, a [, b [, ···]]])

Applies the function `f` onto all elements in the table, sequence, register or numarray `obj` and then sums up the results using Kahan-Babuška round-off error prevention. The return is the number:

$$\sum_{i=a}^{b} f(obj_i)$$

If `a` is not given, `a` is set to 1. If `b` is not given, `b` is set to the number of elements in `obj`. If `f` is a multivariate function, its second, third, etc. argument must be passed after `b`.

See also: **calc.fsum**, **stats.fprod**, **stats.sumdata**.

stats.gammacdf (x, a, b)

Implements the gamma cumulative distribution function:

$$\frac{\gamma(a,bx)}{\Gamma(a)}$$

See also: **stats.gammapdf**.

stats.gammad (x, a, b)

The Gamma distribution function returns the integral from zero to real `x` of the gamma probability density function and returns the number:

$$\frac{a^b}{\Gamma(b)}\int_0^x t^{b-1}\ e^{-at}\ dt$$

where `a` * `x` > 0, `b` > 0. See also: **stats.gammadc**.

stats.gammadc (x, a, b)

The complemented Gamma distribution function returns the integral from x to infinity of the gamma probability density function and returns the number:

$$\frac{a^b}{\Gamma(b)}\int_x^\infty t^{b-1}\ e^{-at}\ dt$$

where `a` * `x` > 0, `b` > 0. See also: **stats.gammadc**.

stats.gammapdf (x, a, b)

Implements the gamma distribution probability density function:

$$\frac{b^a}{\Gamma(a)}x^{a-1}e^{-bx}$$

See also: **stats.gammacdf**.

stats.gema (obj, k, alpha [, mode [, y0star]])

Like **stats.ema**, but returns a function that, each time it is called, returns the exponential moving average, starting with sample `obj`[`k`], and progressing with sample `obj`[`k`+1], `obj`[`k`+2], etc. with subsequent calls. It return **null** if there are no more samples in `obj`. It is much faster than **stats.ema** with large distributions.

The smoothing factor `alpha` is a rational number in the range [0, 1].

The function supports two algorithms: If `mode` is 1 (the default), then the algorithm

```
r := alpha * obj[k];
s := 1 - alpha;
for i from k - 1 to 1 by -1 do
   r +:= alpha * s ^ i * obj[i]
od;
r +:= s ^ k * y0star;
```

is used to compute the result. In `mode` 1, you can pass an explicit first estimate `y0star`, otherwise the first value `y0star` is equal to the sample moving average of `obj`.

If `mode` is 2, then the formula

```
r := obj[k];
for i from k - 1 to 1 by -1 do
   r +:= alpha * (obj[i] - r)
od;
```

is applied to the period.

The result is a number.

stats.geometric (k, p [, option])

Computes the geometric cumulative distribution F(`k`, `p`) if no `option` is given, and alternatively the geometric probability point distribution f(`k`, `p`) if `option` is **true**:

$$F(k, p) = 1.0 - (1.0 - p)^{k+1}$$

$$f(k, p) = p*(1.0 - p)^{k}$$

where `k` is the number of 0's which occurred before a 1 occurred and `p` is the probability of a 1 on a single trial with $0 \leq p \leq 1$.

See also: **stats.laplace**, **stats.logistic**.

stats.gini (obj [, 'sorted'])

Measures the inequality in a distribution given by the table, sequence, register or numarray `obj` by applying Gini's formula

$$\sum_{i=1}^{n} \sum_{j=1}^{n} |x_i - x_j| \; / \; (2n^2 \mu),$$

where n is the sample size and μ the arithmetic mean.

All members of `obj` should be numbers. **infinity** and **undefined** will be ignored.

It returns a number r indicating the absolute mean of the difference between every pair of observations, divided by the arithmetic mean of the population, with $0 \leq r \leq 1$, where 0 indicates that all observations are equal, and (a theoretical value of) 1 indicates complete inequality. It is assumed that all observations are non-negative.

If the option `'sorted'` is given then the function assumes that all elements in `obj` are already sorted in ascending order - thus computing the result much faster.

To compute the normalised Gini coefficient, multiply the result by n/(n-1).

See also: **stats.herfindahl**.

stats.gmean (obj)

Returns the geometric mean of all numeric values in table, sequence, register or numarray `obj`. It is a measure of central tendency. Its formula is:

$$\left(\prod_{i=1}^{n} obj_i\right)^{1/n}$$

The function returns **fail** if `obj` contains less than two elements.

The geometric mean should be applied on positive values that are interpreted to their products, e.g. rates of growth, instead of their sums, only. Otherwise, **undefined** may be returned.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **math.agm**, **stats.amean**, **stats.hmean**, **stats.mean**, **stats.qmean**.

stats.gsma (obj, k, p)
stats.gsma (obj, k, p, b)

Like **stats.sma**, but returns a function that, each time it is called, returns the simple moving mean, starting with sample `k`, and progressing with sample `k`+1, `k`+2, etc. If `k` > **size** `obj`, then the function returns **null**. It is much faster than **stats.sma** with large distributions.

stats.gsmm (obj, k, p)
stats.gsmm (obj, k, p, b)

Like **stats.smm**, but returns a function that, each time it is called, returns the simple moving median, starting with sample `k`, and progressing with sample `k`+1, `k`+2, etc. If `k` > size(`obj`), then the function returns **null**. It is much faster than **stats.smm** with large distributions.

The function automatically non-destructively sorts the distribution `obj` if it is unsorted.

stats.herfindahl (obj)

Returns the normalised Herfindahl–Hirschman index of a distribution `obj`, a table, sequence, register or numarray, an indicator of the amount of competition in economy. A value of 0 means that there is absolute competition, i.e. that all companies have the same share, and 1 means that there is a monopoly.

The normalised index h is defined as:

$$H = \sum_{i=1}^{n} \left(\frac{obj_i}{s}\right)^2, \text{ where } s = \sum_{i=1}^{n} obj_i, \Rightarrow h = \frac{H - 1/n}{1 - 1/n}$$

It is also a good measure to determine the stability of a distribution, with a value tending to zero indicating that the number of outliers is quite low, and a value tending to 1 that there is at least an extreme outlier.

If `obj` is a register, all its values should be numbers.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.gini**.

stats.hmean (obj)

Returns the harmonic mean of all numbers or complex numbers in table, sequence or numarray `obj`. It is useful with rates and ratios, as it provides the best average. It is defined as follows:

$$n \,/ \sum_{i=1}^{n} \frac{1}{obj_i}$$

Thus, the harmonic mean is the reciprocal of the arithmetic mean of the reciprocals in `obj`. Note that the function processes all the elements of a distribution even if it includes zeros or negative values.

The function returns **fail** if `obj` contains less than two elements. Otherwise if all values in `obj` are numbers, the return is a number. If at least one complex number is in `obj`, then the return is a complex number. The function does not ignore **undefined**s, so you may delete them from the observation before calling the function.

The function internally uses Kahan-Babuška summation to compensate for rounding errors.

See also: **math.hgm**, **remove**, **select**, **stats.amean**, **stats.gmean**, **stats.mean**, **stats.qmean**.

stats.hypergeom (x, N1, N2, n)

Computes the hypergeometric probability density function

binomial(N1, x)***binomial**(N2, n - x)/**binomial**(N1 + N2, n),

avoiding numeric overflow and internally computing with 80-bit precision if supported by the platform. x is non-negative integer, non-negative N1 is the size of the success population, non-negative N2 the size of the failure population, and n is the sample size. If n > N1 + N2 or x > n, the function returns **fail**.

stats.invbinomd (k, n, y)

With three arguments k, n, y given, the function simplifies to the inverse binomial distribution. This is accomplished using the inverse beta integral function and the relation

1 - **stats.invbinomd**(k, n, y) = **calc.invibeta**(y, n - k, k+1).

k, n must be non-negative integers with k < n and y a number in the range [0, 1].

See also: **long.invbinomd**.

stats.invF (df1, df2, p)

Computes the inverse of complemented F distribution by finding the F density argument x such that the integral from x to infinity of the F density is equal to the given probability p, with 0 < p ≤ 1, and df1, df2 positive integers. This is accomplished using the inverse beta integral function and the relations

z = **calc.ibeta**(df2/2, df1/2, p)
x = df2*(1-z) / (df1*z).

See also: **stats.F**, **stats.Fc**.

stats.invnormald (y)

Evaluates the inverse of the Normal distribution function by returning the number x, for which the area under the Gaussian probability density function (integrated from –∞ to x) is equal to number y which be in (0, 1).

See also: **stats.cauchy**, **stats.chisquare**, **stats.fratio**, **stats.normald**, **stats.studentst**.

stats.ios (obj [, option])

Sums up absolute differences between neighbouring entries in a table, sequence, register or numarray obj, divides by the number of its elements minus 1, and returns the number:

$$\frac{1}{n-1}\sum_{i=2}^{n}\left|obj_i - obj_{i-1}\right|$$

The function returns **fail** if `obj` contains less than two elements.

If any second non-**null** argument is given, the function first normalises the distribution to the range (-∞, 1] (see **stats.scale**), determines the difference list, sums up its absolute differences and divides the sum by the sample size minus 1 to make a distribution comparable to other ones.

This indicator is quite useful to find out how stable or volatile a preferably unsorted distribution is.

See also: **stats.ad**, **stats.deltalist**, **stats.sd**, **stats.var**.

stats.iqmean (obj)

Returns the arithmetic mean of the interquartile range of the distribution `obj` using Kahan-Babuška round-off error prevention. The return is a number. `obj` may be a table, sequence or numarray.

If a distribution is unsorted, the function automatically sorts it non-destructively, and any non-numeric observations are converted to zeros.

The interquartile range comprises all observations that reside between the first and third quartiles.

See also: **stats.iqr**, **stats.midrange**, **stats.qmean**.

stats.iqr (obj [, a [, b]])

Without `a` and `b` given, the function determines the interquartile range (IQR), i.e. the difference of the third and first quartile. **stats.iqr** is useful for determining the variability in a distribution `obj`, a table, sequence, register or numarray.

You may optionally pass a lower and upper percentile `a`, `b`, both in the range [0, 100). If a is missing, it is set to 25. If `b` is missing it is set to 100 - `a`.

It returns the number

stats.percentile(`obj`, `b`) - **stats.percentile**(`obj`, `a`).

If `obj` is unsorted, the function sorts it non-destructively. It is implemented in Agena and included in the lib/library.agn file.

See also: **stats.midrange**, **stats.percentile**, **stats.qcd**, **stats.quartiles**.

stats.isall (obj [, eps])

Checks whether all elements in a table, sequence or numarray `obj` are non-zero and returns **true** or **false**. If the second argument `eps`, a non-negative number, is passed, the function returns **true** if all observations x in `obj` satisfies the condition **abs**(x) > `eps`. By default `eps` is 0.

The function returns **fail** with empty structures or structures that contain one element only.

See also: **and** operator, **stats.isany**.

stats.isany (obj [, eps])

Checks whether at least one element in a table, sequence, register or numarray `obj` is non-zero and returns **true** or **false**. If the second argument `eps`, a non-negative number, is passed, the function returns **true** if at least one observations x in `obj` satisfies the condition abs(x) > `eps`. By default `eps` is 0.

The function returns **fail** with empty structures or structures that contain one element only.

See also: **or** operator, **stats.isall**.

stats.issorted (obj [, f])

Checks whether all values in a table, sequence or numarray `obj` of numbers are stored in ascending order and returns **true** or **false**. If a value in `obj` is not a number, it is ignored.

If `obj` is a table, you have to make sure that it does not contain holes, by calling **tables.hashole**. If it contains holes, apply **tables.entries** on `obj`.

If `f` is given, then it must be a function that receives two structure elements to determine the sorting order. See **sort** for further information.

See also: **sort**, **sorted**, **skycrane.sorted**, **stats.sorted**.

stats.kurtosis (obj [, true])

The function determines the population kurtosis, a measure of flatness or peakedness of symmetric and unimodal distributions. `obj` should be a table, sequence, register or numarray.

To quote Grokipedia, `higher kurtosis corresponds to greater extremity of deviations (or outliers), and not the configuration of data near the mean.`

The function computes the result by computing the fourth moment around the mean of a distribution, divided by the fourth power of the standard deviation.

If you pass **true** as a second argument, the sample kurtosis will be returned.

The function returns **fail** if `obj` contains less than two elements.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.skewness**.

stats.laplace (x)

stats.laplace (x, a, b)

In the first form computes both the Laplace distribution F(x) and the corresponding probability density function f(x):

$$F(x) = \frac{e^{x}}{2} \text{ for } x \leq 0,\ 1 - \frac{e^{-x}}{2} \text{ for } x > 0$$

$$f(x) = \frac{e^{-|x|}}{2} \vee x$$

In the second form computes the probability density function of the Logistic[a, b] distribution for numeric x:

1/(2*b) * exp(-abs(x - a)/b)

stats.logistic (x)

stats.logistic (x, a, b)

In the first form computes both the Logistic distribution F(x) and the corresponding probability density function f(x):

$$F(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

In the second form computes the probability density function of the Logistic[a, b] distribution for numeric x:

$$\exp(- (x - a)/b) / b / (1 + \exp(-(x - a)/b))^2.$$

See also: **stats.geometric**, **stats.laplace**.

stats.lognormald (x [, μ [, σ]])

Computes probability density function:

$$\exp(-0.5*(\ln(x) - \mu)^2 / \sigma^2) / \sqrt{2\pi\sigma}$$

σ is the standard deviation and must be positive x. μdefaults to 0, and σ to 1. Both `x` and σ must be positive.

See also: **stats.cauchy**, **stats.chisquare**, **stats.fratio**, **stats.invnormald**, **stats.normald**, **stats.studentst**.

stats.logseries (k, p)

Computes the log(arithmic) series cumulative distribution

$$f(k, p) = \frac{-p^k}{k\ln(1-p)}, \text{ where}$$

- `k`: argument of the logarithmic distribution, `k` >= 1,
- `p`: shape parameter of a logarithmic distribution, 0 < `p` < 1.

stats.lse (obj)

Computes the log-sum-exp function

$$\ln \sum_{i=1}^{n} e^{obj[i]} = a + \ln \sum_{i=1}^{n} e^{obj[i]-a},\ a = \mathbf{max}(obj)$$

and also returns a, the largest sample in `obj` which may either be a table, sequence or register of numbers.

To compute the softmax function, that is the gradient vector of the log-sum-exp function, issue for example:

```
> X := [-1, 2]

> lse := stats.lse(X):
2.0485873515737

> map(<: x, lse -> exp(x)/exp(lse) :>, X, lse):
[0.047425873177567, 0.95257412682243]
```

stats.mad (obj [, option])

Returns the median of the absolute deviations of all numeric values in table, sequence or numarray `obj` from `obj`'s median, and returns the number:

$$\text{stats.median}\left(\bigvee_{i=1}^{\text{size obj}} \left|\text{obj}_i - \text{stats.median(obj)}\right|\right).$$

If any second non-**null** argument is given, then the variation coefficient will be returned:

$$\text{stats.median}\left(\bigvee_{i=1}^{\text{size obj}} \left|\text{obj}_i - \text{stats.median(obj)}\right|\right) / \text{stats.median(obj)}.$$

Median absolute deviation is quite robust if a distribution contains a small number of outliers.

If `obj` is unsorted, it automatically sorts it before determining the result.

If `obj` contains less than two elements or entirely consists of **undefined**s, **fail** will be returned. The function ignores **undefined**s, if `obj` features at least one number.

Please note that if `obj` includes non-numbers, where **undefined** is considered a number, they are interpreted as zeros which might unexpectedly influence the result.

See also: **stats.ad**, **stats.md**, **stats.median**.

stats.max (obj [, 'sorted'])

Returns the maximum of all numeric values in a distribution `obj` (a table, sequence, register or numarray) and its index position, in this order. If the option `'sorted'` is passed than the function assumes that all values in `obj` are sorted in ascending order and returns the last entry plus its index (the size of the distribution). The function in general returns **fail** if it receives an empty distribution.

See also: **max**, **min**, **math.max**, **stats.max**, **stats.minmax**.

stats.md (obj [, option])

Computes the median deviation of all the values in a table, sequence or numarray `obj`, i.e. the mean of the equally likely absolute deviations from the median med:

$$\frac{1}{n}\sum_{i=1}^{n}\left|\text{obj}_i - \text{med}\right|$$

The return is a number.

If any second non-**null** argument is given, then the variation coefficient will be returned:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left|obj_i - med\right|} \;/\; |\; med\;|$$

See also: **stats.mad**.

stats.mean (obj)

Returns the arithmetic mean of all numeric values in table, sequence, register or numarray `obj` as a number. It is equivalent to, where n is the number of samples in `obj`:

$$\frac{1}{n}\sum_{i=1}^{n} obj_i$$

The function is avoiding numeric overflow by dividing each value in `obj` by n before summation.

If `obj` is table, it is assumed to be an array, non-positive integral keys (including strings, etc.) are ignored.

The function returns **fail** if `obj` contains less than two elements.

`obj` may contain numbers, complex numbers or a mix of them.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.accu**, **stats.amean**, **stats.gmean**, **stats.hmean**, **stats.meanmed**, **stats.meanvar**, **stats.qmean**.

stats.meanmed (obj [, option])

Returns both the arithmetic mean and the median of all numeric values in table, sequence, register or numarray `obj` as numbers. If any `option` is given, the quotient of the mean and the median will be returned.

See also: **stats.accu**, **stats.amean**, **stats.meanvar**, **stats.median**.

stats.meanqmdev ([f,] obj [, ···] [, options])

Returns both the arithmetic mean and the quadratic mean deviation - in this order - of the distribution `obj` using a `running` algorithm developed by B. P. Welford to prevent round-off errors.

`obj` may be a table, sequence, register or numarray. The function works in-place to avoid internal duplication of observations, and does not modify the structure.

The arithmetic mean and quadratic mean deviation returned have very good accuracy, sometimes even beating those computed by **stats.amean** and **qmdev**.

When given the `optimised`=**false** option, the function does *not* try to avoid numeric overflow with very large samples. The default is **true**.

The selection function `f` may either be univariate or multivariate. With the latter, put any second, third, etc. argument right after `obj`.

You may sort the distribution prior to the call to improve precision of the results.

See also: **stats.card**, **stats.meanmed**, **stats.meanvar**.

stats.meanvar ([f,] obj [, option])

stats.meanvar ([f,] obj [, ···] [, options])

Returns both the arithmetic mean and the variance - in this order - of the distribution `obj` using a `running` algorithm developed by B. P. Welford to prevent round-off errors.

By default, the population variance will be returned unless you pass the Boolean value **true** for `option` to compute the sample variance. `obj` may be a table, sequence, register or numarray. The function works in-place to avoid internal duplication of observations and does not modify the structure.

The arithmetic mean and variance returned have very good accuracy, sometimes even beating those computed by **stats.amean** and **stats.var**.

If you pass a univariate function `f` defining a Boolean condition, then only the observations in `obj` that satisfy this condition will be processed, for example:

```
> s := sequences.new(1, 10);

> stats.meanvar(s):  # mean & variance of all the values
5.5     8.25

> stats.meanvar(<: x -> even x :>, s):  # mean & variance of even values
6       4
```

In the second form, you may pass the following `options`:

- When given the `optimised`=**true** option, the function more strictly tries to avoid numeric overflow with very large samples. The default is **false**.
- The `sample`=**true** option forces the function to return the sample variance instead of the population variance. The default is **false**.

In the second form the selection function `f` may either be univariate or multivariate. With the latter, put any second, third, etc. argument right after `obj`.

You may sort the distribution prior to the call to improve precision of the results.

See also: **stats.card**, **stats.meanmed**, **stats.meanqmdev**.

stats.median (obj)

Returns the median of all numeric values in table, sequence or numarray `obj` as a number. If `obj` is unsorted, it automatically sorts it before determining the median.

If `obj` contains less than two elements or entirely consists of **undefined**s, **fail** will be returned. The function ignores **undefined**s, if `obj` features at least one number.

Please note that if `obj` includes non-numbers, where **undefined** is considered a number, they are interpreted as zeros which might unexpectedly influence the result.

The median is the middle element of a distribution if its size is odd, or the average of its middle elements it is size is even.

See also: **stats.accu**, **stats.mad**, **stats.meanmed**.

stats.midrange (obj [, option])

Computes the sum of the minimum and maximum value of a distribution `obj`, a table, sequence, register or numarray, divided by two.

If the option `'sorted'` is given, the observation is not traversed; instead the first and the last entry is taken to compute the mean. If the observation is empty or has only one element, **fail** will be returned.

See also: **stats.iqr**, **stats.minmax**.

stats.min (obj [, 'sorted')

Returns the minimum of all numeric values in a distribution `obj` (a table, sequence, register or numarray) and its index position, in this order. If the option `'sorted'` is passed than the function assumes that all values in `obj` are sorted in ascending order and returns the first entry plus its index, that is 1. The function in general returns **fail** if it receives an empty distribution.

See also: **max**, **min**, **math.max**, **stats.min**, **stats.minmax**.

stats.minmax (obj [, 'sorted'])

Depending on the type of `obj`, returns either a table, sequence or register with the minimum of all numeric values in table, sequence, register or numarray `obj` as the first value, and the maximum as the second value. With numarrays containing C doubles, returns a sequence.

If the option `'sorted'` is passed than the function assumes that all values in `obj` are sorted in ascending order so that execution is much faster.

stats.minmax returns **fail** if `obj` has less than two elements. If `obj` consists entirely of **undefined** entries, $[-\infty, \infty]$ or **seq**$(-\infty, \infty)$ are returned, otherwise **undefined** values are simply ignored.

See also: **numarray.minmax**, **stats.max**, **stats.midrange**, **stats.min**.

stats.mode (obj)

Returns all values in the table, sequence, register or numarray `obj` with the largest number of occurrence, i.e. highest frequency. If there is more than one value with the highest frequency, they are all returned.

The type of return is determined by the type of its argument. If the given structure is empty, it is simply returned.

stats.moment (obj [, p [, x_m [, option]]])

Computes the moment `p` of the given table, sequence, register or numarray `obj` about any origin x_m for a full population and returns a number. It is equivalent to:

$$\frac{1}{n}\sum_{i=1}^{n}\left(obj_i - x_m\right)^p$$

If only `obj` is given, the moment `p` defaults to 1, and the origin x_m defaults to 0. If given, the moment `p` and the origin x_m must be numbers. If `obj` contains less then two observations, **fail** will be returned.

If `option` is given and is **true**, the sample moment

$$\frac{1}{n - 1}\sum_{i=1}^{n}\left(obj_i - x_m\right)^p$$

will be computed.

See also: **qmdev**, **stats.sd**, **stats.sumdata**.

stats.nde (x [, [μ, [σ]])

Computes $e^{\frac{-(x-\mu)^2}{2\sigma^2}}$; μ and σ default to 0 and 1, respectively.

See also: **stats.ndf**, **stats.pdf**.

stats.ndf ([σ])

Computes $\frac{1}{\sqrt{2\pi}}$ if σ is not given, and $\frac{1}{\sigma\sqrt{2\pi}}$ otherwise, and issues an error if $\sigma \leq 0$.

See also: **stats.nde**, **stats.pdf**.

stats.negbinompdf (x, n, p)

Computes the probability density function of the negative binomial distribution and is equal to

$$\textbf{binomial}(n + x - 1, x) * p^n * (1 - p)^x,$$

with `x` a non-negative integer no greater than `n`, `n` a positive integer and `p` a number between 0 (inclusive) and 1 (inclusive).

See also: **stats.binompdf**.

stats.neighbours (obj, idx, eps [, power [, indices]])

Determines all neighbours of a given n-dimensional point in a distribution `obj` that lie in a certain Euclidian distance `eps`. `idx` is the position of the point of interest in the distribution - a positive integer -, and not the point itself.

`eps` is any positive number, `power` is a positive integer with which the respective Euclidean distances and `eps` shall be raised before a comparison is conducted, its default is 2.

The return is a sequence with the nearby points. If the fifth argument `indices` is **true**, however, then not the points but their positions in the distribution are returned.

The points may be represented either as pairs (2-dimensional space), sequences of co-ordinates (n-dimensional space), or any n-dimensional vectors created by the **linalg.vector** function.

See also: **linalg.norm**, **stats.dbscan**.

stats.normald (x [, μ [, σ]])

The normal distribution has the probability density function for any number `x`:

$$\exp(-(x - \mu)^2 / 2 / \sigma^2) / \sqrt{2\pi\sigma^2}$$

σ is the standard deviation and must be positive. μdefaults to 0, and σ to 1.

See also: **stats.cauchy**, **stats.chisquare**, **stats.fratio**, **stats.invnormald**, **stats.lognormald**, **stats.studentst**.

stats.obcount (obj, p, n [, f])
stats.obcount (obj, p [, option])

The function counts occurrences in an observation `obj`.

In the first form, it first divides a numeric range defined by the pair `p` and its step size `n` into its respective classes.

In the second form, if the option is `classes=`k, it first divides a numeric range defined by the pair `p` into k classes. If no option is passed, it automatically computes the number of classes according to the formula

$$1 + 3.3 * \ln(\text{size of range } p),$$

with no upper limit. (It is suggested to choose between 5 or 30 classes.)

With both forms, all occurrences in the distribution `obj` (table, sequence, register, numarray) are then sorted into these subranges/classes and the function finally counts all elements in them. If the optional fourth argument `f`, a function, is given, then an occurrence or a part of an occurrence is first converted according to the function definition before the correct subinterval is being determined.

The function returns a table with the keys the respective left borders of the subranges and the values the number of counts in the respective subranges. It always also returns a second table which may include all those elements in `obj` which are not part of the overall range defined by `p`. If all numbers in `obj` fit into `p`, an empty table will be returned.

If an element in `obj` equals the right border of a subinterval, then it is considered to be part of the next subinterval. But if an element in `obj` equals the right border of the overall interval `p`, it is considered part of the last subinterval.

The function issues an error if it encounters a non-number in `obj`, or if the left border in `p` is greater or equals to the right border in `p`.

The function is implemented in Agena and included in the lib/library.agn file.

An example:

```
> s := seq(0.1, 0.2, 0.3, 0.4, 1, 1.1, 2, 2.1);

> stats.obcount(s, 0:2, 1):
[0 ~ 4, 1 ~ 3]  [2.1]
```

See also: **skycrane.obcount**, **stats.freqd**, **stats.obpart**.

stats.obpart (s, p, n [, f [, g]])
stats.obpart (s, p, option [, f [, g]])

The function sorts occurrences into subintervals (classes).

In the first form, it divides a numeric range defined by the pair `p` and its step size `n` into its respective subintervals, and sorts all occurrences in the distribution `s` (a sequence) into these classes.

In the second form, if the option is `class`=k, divides a numeric range defined by the pair `p` into k classes. If the option **null**, the function automatically computes the number of classes according to the formula

$$1 + 3.3 * \ln(\text{size of range } p),$$

with no upper limit. (It is suggested to choose between 5 or 30 classes.)

If the fourth argument `f`, a function, is given, then an occurrence or a part of an occurrence is first converted according to the function definition before the correct subinterval is being determined. If `f` is **null**, no conversion is done.

If the fifth argument `g`, a function, is given, then it is applied on an occurrence or part of it before it is inserted into the subinterval that already has been determined.

The function returns a table with the keys the respective left borders of the subranges and the values sequences with the respective occurrences. It always also returns a second table which may include all those elements in `s` which are not part of the overall range defined by `p`.

If an element in `s` equals the right border of a subinterval, then it is considered to be part of the next subinterval. But if an element in `s` equals the right border of the overall interval `p`, it is considered part of the last subinterval.

The function issues an error if a distribution or part of it is not or could not be converted to a number, or if the left border in `p` is greater or equals to the right border in `p`.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.obcount**.

Examples:

```
> s := seq(1.1, 1.2, 2.4, 2.5, 2.6, 3.1);

> stats.obpart(s, 1:4, 1):
[seq(1.1, 1.2), seq(2.4, 2.5, 2.6), seq(3.1)]   []
```

Given are timestamps and running times in seconds:

```
> s := seq('12:30:05.017':3, '12:31:57.235':4);
```

To convert a timestamp into its decimal representation, so that **stats.obpart** can sort an occurrence into a subinterval, we define the following function:

```
> f := proc(x) is
>    local hrs, min, sec;
>    hrs, min, sec :=
>       strings.match(left(x), '(%d%d):(%d%d):(%d%d\.%d%d%d)');
>    return clock.todec(clock.tm(  # returns a number
>       tonumber(hrs), tonumber(min), tonumber(sec)))
> end;

> stats.obpart(s, 12.4:12.6, 1/60, f):
[12.4 ~ seq(), ..., 12.5 ~ seq(12:30:05.017:3),
 12.516667 ~ seq(12:31:57.235:4), ...]   []
```

We only want to insert the running times in milliseconds, but not the timestamps:

```
> g := <: x -> right(x)*1k :>;

> stats.obpart(s, 12.4:12.6, 1/60, f, g):
[12.4 ~ seq(), ..., 12.5 ~ seq(3000), 12.516667 ~ seq(4000), ...]   []
```

See also: **stats.obcount**.

stats.pdf (x [, μ [, σ]])

Computes the probability density function for the normal distribution at the numeric value `x`. The defaults are $\mu = 0$, with standard deviation $\sigma = 1$, thus determining the standard normal distribution.

The return is the number:

$$\frac{1}{\sigma\sqrt{2\pi}}\, e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$

See also: **stats.cdf**, **stats.nde**, **stats.ndf**.

stats.peaks (obj, delta [, dv])

The function returns all peaks and valleys of a distribution `obj` consisting of two-dimensional numeric co-ordinates represented as pairs xk:yk. `obj` may be a table or sequence. A point is considered an extremum if the `vertical` difference to its surrounding is at least `delta`, a positive number. By default, if `dv` is not given or is 1, the direct neighbours of each point are considered, otherwise the `dv`-th neighbours to the left and the right of each point are checked.

Depending on the type of o, the first return is a structure including all valleys represented as pairs xk:yk, and the second return is a structure of the peaks as pairs xk:yk.

See also: **stats.extrema**.

stats.percentile (obj, p [, option])

Returns the value below which a certain percent `p` of the elements in `obj` fall.

`obj` must be a table, sequence or numarray, `p` an integer in the range $0 \leq p < 100$. If no option is given, then the percentile is determined by computing the nearest rank (rank = p/100 * size `obj` + ½, `Wikpedia method`). If `option` is the string `'nist'`, then the method proposed by NIST is used (rank = p/100 * (size `obj` + 1)); if the string `'excel'` is given for `option`, then the algorithm used by Excel is used (rank = p/100*(size `obj` -1) + 1).

If `obj` is not sorted, the function automatically sorts it non-destructively before computing the indicators.

The function issues an error if `obj` is empty.

See also: **whereis**, **stats.quartiles**.

stats.poisson (k, t)

Simplifies to the Poisson probability density defined as $\frac{e^{-t}t^k}{j!}$, avoiding overflow. `k` must be non-negative, `t` is any number.

stats.poissond (k, t)

The function simplifies to the cumulative probability Poisson distribution function defined as

$$\sum_{j=0}^{k} \frac{e^{-t}t^j}{j!},$$

preventing overflow and using Kahan-Babuška summation to minimise round-off errors.

stats.prange (obj [, a [, b]])

Returns all elements in a table, sequence, register or numarray `obj` from the `a`-th percentile rank up but not including the `b`-th percentile rank. `a` and `b` must be positive integers in the range [0 .. 100). If `a` and `b` are not given, `a` is set to 25, and `b` to 75. If `b` is not given, it is set to 100 - `a`. The type of return is determined by the type of `obj`.

If the elements in `obj` are not sorted in ascending order, the function automatically sorts them non-destructively, and any non-numeric values are converted to zeros.

stats.probit (x)

Implements the quantile function associated with the standard normal distribution, or in other words: the inverse of the cumulative distribution function of the standard normal distribution for number `x`. It returns the number $\sqrt{2}\ \mathrm{erf}^{-1}(2x-1)$.

stats.qcd (obj [, a [, b]])

Without `a` and `b` given, the function determines the interquartile range (IQR) of a distribution `obj` (a table, sequence, register or numarray), i.e. the difference of the third (= Q3) and first (= Q1) quartile divided by the sum of the third and first quartile:

$$\frac{Q_3-Q_1}{Q_3+Q_1}.$$

You may optionally pass a lower and upper percentile `a`, `b`, both in the range [0, 100). If a is missing, it is set to 25. If `b` is missing it is set to 100 - `a`.

If `obj` is unsorted, the function sorts it non-destructively. It is implemented in Agena and included in the lib/library.agn file.

See also: **stats.iqr**, **stats.percentile**, **stats.quartiles**.

stats.qmean (obj)

Computes the quadratic mean (root mean square) of all numbers or complex numbers in table, sequence, register or numarray `obj` and returns a (complex) number. If `obj` is a table, it is assumed to be an array, non-positive integral keys (including strings, etc.) will be ignored. The quadratic mean can be used to measure the magnitude of variates which are positive and negative, e.g. sinusoids.

It is equivalent to:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} obj_i^2}$$

where n is the number of elements in `obj`.

The function returns **fail** if `obj` contains less than two elements.

To avoid arithmetic overflow, the function after squaring always divides by n before conducting summation.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.amean**, **stats.gmean**, **stats.hmean**, **stats.iqmean**, **stats.mean**, **stats.moment**, **stats.sd**.

stats.quartiles (obj)

stats.quartiles (obj [, pos])

In the first form, it returns the first, second, and third quartile of table, sequence, register or numarray `obj`. The first and third quartiles are computed according to the NIST rule, see **stats.percentile** for further information.

It also determines the lower outlier limit L_1, where L_1 = first quartile - 1.5 times the interquartile range of `obj`, and the upper outlier limit U_1, where U_1 = third quartile + 1.5 times the interquartile range of `obj`. If a value x in `obj` is equal to L_1 or U_1, then x will be returned. If L_1 is not included in `obj`, then the next largest value to L_1 will be returned. If U_1 is not included in `obj`, then the next smallest value to U_1 is computed. Finally it computes the interquartile range, i.e. third quartile - first quartile. The order is: first quartile, median, third quartile, `` `L1` ``,`` `U1` ``, and the interquartile range.

In the second form, if either the integer 1, 2, or 3 is passed for the optional second argument `pos`, the first, second or third quartile will be returned as a number, respectively.

If the elements in `obj` are not sorted in ascending order, the function automatically sorts them non-destructively, and any non-numeric values are converted to zeros.

The number of values in `obj` should be at least 12, better are 20 or more values. If the number of values is less than 2, **fail** will be returned.

The returned structure is of the same type as the input structure `obj`.

See also: **whereis**, **stats.fivenum**, **stats.iqr**, **stats.percentile**, **stats.qcd**.

stats.rownorm (obj)

Returns the sum of the absolute values of the numbers in the table, sequence, register or numarray `obj`. If `obj` includes **undefined**s, they are ignored. If the structure consists *entirely* of one or more **undefined**s, then the function returns **undefined**. If the structure is empty, **fail** will be returned.

See also: **stats.scale**, **stats.colnorm**.

stats.scale (obj [, option])

The procedure normalises the numbers in the table, sequence, register or numarray `obj` in such a way that an element of maximum absolute value equals 1, thus scaling a distribution to the range (–∞, 1] by dividing all observations by this maximum element.

When given a second option, the function normalises all its observations to the range [0, 1]. See **math.norm** for further details.

The normalised numbers are returned in a new table, sequence, register or numarray, depending on the type of `obj`.

If the maximum absolute value is 0, the function returns **fail**.

See also: **math.norm**, **linalg.scale**.

stats.sd ([f,] obj [, sample [, option]])

Returns the standard deviation of all numbers or complex numbers in table, sequence, register or numarray `obj`. Numbers and complex numbers can be mixed.

Depending on the values in `obj`, the return is either a number or a complex number.

If `obj` is a table, it is assumed to be an array, non-positive integral keys (including strings, etc.) are ignored.

If `sample` is not given or is not **true**, it returns the population standard deviation:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(obj_i - \mu\right)^2}$$

where μ is the arithmetic mean of a distribution.

If `sample` is given and is **true**, the (unbiased) sample standard deviation will be returned:

$$\sigma = \sqrt{\frac{1}{n\text{-}1}\sum_{i=1}^{n}\left(obj_i - \mu\right)^2}$$

If the return is a small number, it indicates that the points in a distribution are close to its mean m. A large value indicates that its points are rather spread out. Contrary to variance, standard deviation is expressed in the same units as the data.

Standard deviation is less robust to outliers than absolute deviation, see **stats.ad**.

The function returns **fail** if `obj` contains less than two elements.

If any third non-**null** argument is given, then the coefficient σ / | μ| will be returned to make different distributions comparable.

If you pass a function `f` defining a Boolean condition, then only the observations in `obj` that satisfy this condition will be processed, for example:

```
> s := sequences.new(1, 10);

> stats.sd(s):  # standard deviation of all the values
2.872281323269

> stats.sd(<: x -> even x :>, s):  # sd of the even values only
2
```

The function is implemented in Agena and included in the lib/library.agn file.

See also: **qmdev**, **stats.ad**, **stats.chauvenet**, **stats.ios**, **stats.mad**, **stats.moment**, **stats.qmean**, **stats.spread**, **stats.var**.

stats.skewness (obj [, true])

Returns the population skewness, a measure of the asymmetry of the probability distribution represented by the table, sequence, register or numarray `obj` of numbers. Returns 0 if a distribution is symmetric, a negative value if the left tail is longer, and a positive value if the right tail is longer.

It computes the third moment about the mean and divides it by the third power of the standard deviation. If you pass **true** as a second argument, the sample skewness will be returned.

The function returns **fail** if `obj` contains less than two elements.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.kurtosis**.

stats.sma (obj, k, p)
stats.sma (obj, k, p, b)

In the first form, computes the simple moving average of a table, sequence or register `obj` by averaging the last `p` numbers from the structure (`p` is also known as the `period`) including sample `k`, i.e.:

$$\frac{1}{p}\sum_{i=k-p+1}^{k} obj_i \quad \text{(financial form)}$$

In the second form, by passing the Boolean value **true** for argument `b`, the mean is taken from an equal number of values on either side of `k`, including `k`. Thus `p` must be an odd number:

$$\frac{1}{p}\sum_{i=k-p\backslash 2}^{k+p\backslash 2} obj_i \quad \text{(scientific form)}$$

It returns **undefined**, if either the left or right end of the sublist to be evaluated is not part of `obj`. The function does not accept structures including the value **undefined**.

By dividing each element before summation, the function avoids arithmetic overflows and also uses Kahan-Babuška summation to prevent round-off errors during summation.

stats.gsma is the iterator version of this function which traverses large distributions much faster.

See also: **stats.amean**, **stats.gsma**, **stats.gsmm**, **stats.smm**.

stats.smallest (obj [, k])

Returns the `k`-th smallest element in the numeric table, sequence or numarray `obj`. If `k` is not given, it is set to 1.

stats.smm (obj, k, p)
stats.smm (obj, k, p, b)

In the first form, computes the simple moving median of a table, sequence or register `obj` by sorting the last `p` numbers from the structure (`p` is also known as the `period`) including sample `k`, and then taking its median.

In the second form, by passing the Boolean value **true** for argument `b`, the simple moving median is determined by sorting an equal number of values on either side of `k`, including `k`, and then taking the median. Thus `p` must be an odd number.

The function is more robust than **stats.sma** to outliers in a period.

It returns undefined, if either the left or right end of the sublist to be evaluated is not part of `obj`. The function does not accept structures including the value **undefined**.

The function automatically non-destructively sorts the distribution `obj` if it is unsorted.

stats.gsmm is the iterator version of this function which traverses large distributions much faster.

See also: **stats.amean**, **stats.gsma**, **stats.gsmm**, **stats.sma**.

stats.sorted (obj [, true] [, options])

Sorts the table, sequence, register or numarray `obj` of numbers in ascending order and non-destructively up to and around twice as fast as **sort** if the structure contains (around) more than seven elements. It also ignores **undefined**'s. The type of return is defined by the type of the input.

If an element in `obj` is not a number, it is replaced with the number 0 before sorting.

By default, the function internally uses a recursive implementation of the Quicksort algorithm combined with a fallback to Heapsort in ill-conditioned situations, called Introsort.

You may exclusively use an iterative variant of the Quicksort algorithm by passing the second argument **true** or the string `'pixelsort'`, which may be faster on some older systems, especially with elements in completely random or in (nearly) ascending order. If the option `'nrquicksort'` is given, an alternative non-recursive algorithm described by Niklaus Wirth is being used. If the option `'heapsort'` is

passed, the function uses the Heapsort algorithm. If the option `'quicksort'` is given, a traditional recursive Quicksort algorithm is being used.

See also: **sort**, **sorted**, **skycrane.sorted**, **stats.issorted**, **numarray.sort**.

stats.spread (obj)

Computes the population spread, that is the variance, of a distribution `obj` of numbers, and returns a number. The result is equal to:

$$\frac{1}{n}\sum_{i=1}^{n} obj_i^2 - \frac{1}{n^2}\left(\sum_{i=1}^{n} obj_i\right)^2$$

The function is more than twice as fast than **stats.var** but is more susceptible to numeric overflows if the magnitudes of the observations are very large.

`obj` must be a table, sequence, register or numarray. If it is a register, all its values should be numbers.

The function is implemented in Agena and included in the lib/library.agn file.

stats.standardise (obj [, options])

Standardises a distribution by subtracting the arithmetic mean μ from each observation and then dividing by the population standard deviation (default) σ of the distribution:

$$obj_i \longrightarrow \frac{obj_i - \mu}{\sigma}$$

Depending on the type of its argument `obj`, the return is either a new table, sequence or register of the respective quotients, preserving the original order of the observations. You may alternatively divide by the sample standard deviation by passing the optional value **true** as the second argument or the `sample`=**true** option at position two or greater of the argument list. `obj` may be a table, sequence, register or numarray.

If you pass the `inplace`=**true** option then the whole operation is done in-place, thus modifying `obj`, but requiring no additional memory space at a speed gain of around 25 percent. In this case the return is the modified input structure.

You may also pass a numarray for `obj`. If the `inplace` option is not given or not set to **true**, then a *sequence* of the standardised observations will be returned, otherwise the altered numarray will be returned.

stats.studentst (x [, nu])

The Student's t-distribution has the probability density function:

$$\Gamma(\ (\mathrm{nu}+1)/2\ )\ /\ \Gamma(\mathrm{nu}/2)\ /\ \sqrt{\mathrm{nu}*\pi}\ /\ (1+t^2/\mathrm{nu})\ \hat{}\ ((\mathrm{nu}+1)/2),$$

with `nu` a positive integer.

See also: **stats.cauchy**, **stats.chisquare**, **stats.fratio**, **stats.normald**.

stats.sumdata ([f,] obj [, p [, x_m [, ···]]])

Sums up all the powers `p` of the given table, sequence, register or numarray `obj` of n elements about the origin x_m and returns a number. It is equivalent to:

$$\sum_{i=1}^{n}\left(\mathrm{obj}_i - x_m\right)^p$$

If only `obj` is given, the power `p` defaults to 1, and the origin x_m defaults to 0. If given, `p` and x_m must be numbers. If `obj` is empty, the function returns **fail**.

If a function `f` is given, it only sums up the values in `obj` satisfying `f`, which should return a Boolean. If `f` has more than one argument, then its second to last argument must be given right after x_m.

Examples:

```
> stats.sumdata(<: x -> x > 2 :>, seq(1, 2, 3, 4)):
7

> stats.sumdata(<: x, y -> x + y > 2 :>, seq(1, 2, 3, 4), 1, 0, 1):
9
```

The function uses Kahan-Babuška round-off error prevention.

See also: **foreach**, **math.kbadd**, **math.koadd**, **sumup**, **stats.cumsum**, **stats.fsum**, **stats.moment**, **stats.sumdataln**.

stats.sumdataln ([f,] obj [, p [, x_m [, ···]]])

Sums up all the natural logarithms of the powers `p` of the given table, sequence, register or numarray `obj` of n elements about the origin x_m and returns a number. It is equivalent to:

$$\sum_{i=1}^{n}\ln\left(\left(\mathrm{obj}_i - x_m\right)^p\right)$$

If only `obj` is given, the power `p` defaults to 1, and the origin x_m defaults to 0. If given, `p` and x_m must be numbers. If `obj` is empty, the function returns **fail**.

If a function `f` is given, it only sums up the values in `obj` satisfying `f`, which should return a Boolean. If `f` has more than one argument, then its second to last

argument must be given right after x_m. For examples, please see **stats.sumdata**.

stats.tovals (obj)

Converts all string values in the table, set, sequence or register obj to Agena numbers or complex numbers and returns a new structure. The type of return is determined by the type of obj.

stats.trimean (obj [, p])

If p is not given, the function determines the 1st quartile Q1 and the 3rd quartile Q3 along with the median Q2 of a distribution obj and returns the trimean (Q1 + 2*Q2 + Q3)/4 along with the median. obj may be a table, sequence, register or numarray.

If p, an integer in the range [0 .. 100) is given, instead of the first and third quartiles the p-th and 100 - p-th percentile ranks are the lower and upper margins in the computation.

When compared to the median, the trimean is a means to determine whether a distribution is biased in its first or second half. If the distribution is not sorted, it automatically sorts it non-destructively, where any non-numeric elements are set to 0.

See also: **stats.iqr**, **stats.trimmean**, **stats.winsor**.

stats.trimmean (obj, f)

Returns the arithmetic mean of the interior of a distribution obj - a table, sequence, register or numarray - where the number $f \in [0, 1)$ determines the fraction of the data that is to be excluded from the margins.

The number p of data to be excluded from obj is always rounded down to the nearest even number. The function then does not take into account p/2 points from the left margin and p/2 points from the right margin when calculating the average using Kahan-Babuška round-off error prevention. The function does not sort the distribution.

The return is a number. It returns **fail**, if the distribution includes less than two elements.

If obj is a register, all its values should be numbers.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **stats.amean**, **stats.winsor**.

stats.var ([f,] obj [, sample [, option]])

Returns the variance of all the numbers or complex numbers in table, sequence, register or numarray `obj`. Numbers and complex numbers can be mixed.

Depending on the values in `obj`, the return is either a number or a complex number.

If `obj` is a table, it is assumed to be an array, non-positive integral keys (including strings, etc.) are ignored.

If `sample` is not given or does not evaluate to **true,** the population variance will be returned, where μ is the arithmetic mean of a distribution:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(obj_i - \mu)^2$$

If `sample` is given and is **true**, the (unbiased) sample variance will be returned:

$$\sigma^2 = \frac{1}{n - 1}\sum_{i=1}^{n}(obj_i - \mu)^2$$

If `option` of any type is passed, the variation coefficient σ^2 / $|\mu|$ is determined to make different distributions comparable.

The function returns **fail** if `obj` contains less than two elements.

If you pass a function `f` defining a Boolean condition, then only the observations in `obj` that satisfy this condition will be processed, for example:

```
> s := sequences.new(1, 10);

> stats.var(s):  # variance of all the values
8.25

> stats.var(<: x -> even x :>, s):  # variance of the even values only
4
```

The function is implemented in Agena and included in the lib/library.agn file.

For a fast and very precise alternative, check **stats.meanvar**.

See also: **stats.ad**, **stats.ios**, **stats.mad**, **stats.meanvar**, **stats.sd**, **stats.spread**.

stats.weights (obj)

stats.weights (x [, ···])

In the first form, the function inserts all the elements in sequence `obj` into a new sequence and returns it.

In the second form, it inserts all the given arguments into a new sequence and returns it.

In both forms, only numbers and pairs are accepted in the sequence or argument list. In case of a pair x:n, x denotes an observation and n, a non-negative integer, denotes the number of occurrence of x, so x is inserted n times into the new sequence.

stats.winsor (obj [, n])

Returns the winsorised mean of all numeric values in table, sequence, register or numarray `obj`.

The function first replaces the first `n` percent of a distribution at the low and high end with the most extreme remaining values, and then calculates the arithmetic mean of the entire modified distribution. By default, `n` is 10.

The winsorised mean is more resistant to outliers than the traditional arithmetic mean.

See also: **stats.trimean**, **stats.trimmean**..

stats.zscore (obj)

Returns a univariate function `z(x)` computing the z-score (standard score) of a sample x in the table, sequence, register or numarray `obj` - the number of standard deviations x is above or below the mean according to the formula: z(x) = (x - μ)/δ, where μdenotes the arithmetic mean of `obj`, and δ its standard deviation.

The resulting function returns a positive number if x is above the mean and a negative number if it is below. It does, however, not check whether x is part of `obj`. The result is computed using Kahan-Babuška round-off error prevention for $\mu$and δ.

If `obj` is a register, all its values should be numbers.

The function is implemented in Agena and included in the lib/library.agn file.

The package features special correlation functions. Each takes the distribution `rho` as its input distribution (first argument) and the correlation length $\texttt{rho}_0$ (second argument). Depending on the input type of `rho`, the return is a new structure with the result. The input structure is left unchanged.

The functions are:

stats.besselj	Bessel J correlation function
stats.besselk	Bessel K correlation function
stats.circular	circular correlation function
stats.constant	constant correlation function
stats.cubic	cubic correlation function
stats.dampedcos	damped cosine correlation function
stats.dampedsin	damped sine correlation function
stats.exponential	exponential correlation function
stats.gaussian	Gaussian correlation function
stats.hole	hole correlation function
stats.linear	linear correlation function
stats.matern	Matern correlation function
stats.penta	pentaspherical correlation function
stats.power	power correlation function
stats.ratquad	rational quadratic correlation function
stats.spherical	spherical correlation function
stats.white	white noise correlation function

11.15 long - 80-Bit Floating-Point Arithmetic

This package implements 80-bit floating-point numbers and arithmetic. It is at least 80 times faster than the **mapm** package with the same precision.

Note that 80-bit floating-point arithmetic provided by this new package is still at least 12 times slower than Agena's built-in 64-bit arithmetic so it is useful only if you need extended precision with 19 significant digits and a range of approximately 3.65×10^{-4951} to 1.18×10^{4932}.

The package does not work on ARM platforms as ARM processors do not support 80-bit floating point arithmetic.

Example, you can use **long.new** and **long.double** synoymously:

```
> a, b := long.new(1), long.double(2)

> a + b:
longdouble(3.0000000000000000000)

> exp(a):
longdouble(2.7182818284590452354)

> long.tonumber(ans):  # convert back to Agena number (64-bit)
2.718281828459
```

Note that relational operators such like = or <> cannot compare longdoubles with numbers as they are of different type, and the result would always be **false**. Arithmetic binary operators, however, such like +, **%**, however, can process a mix of numbers and longdoubles and return a longdouble as the result. Also, most unary operators and functions with one or two arguments can process both longdoubles and ordinary Agena numbers. Long numbers have the user-defined type `longdouble`.

You might be interested in the **unimath** package for more functions, check Chapter 11.22.

Creation, Checks, Conversion, Iteration:

long.double (x)
long.new (x)

Create a longdouble from the ordinary Agena number `x`, returning a userdata of type longdouble with the longdouble metatable attached.

`x` may also be a string representing a number - with constants, you might prefer this to avoid round-off errors. For a predefined set of constants, also check the end of this subchapter. Strings representing a hexadecimal value are also supported if the strings starts with '-0x' or '0x'.

long.checklong (···)

Checks whether all its arguments are longdoubles and issues an error otherwise.

long.islong (···)

Tests whether all its arguments are longdoubles and returns **true** or **false**.

long.isnegative (···)

Tests whether all its longdouble arguments are negative and returns **true** or **false**.

long.isnegint (···)

Tests whether all its longdouble arguments are negative integers and returns **true** or **false**.

long.isnonnegative (···)

Tests whether all its longdouble arguments are non-negative and returns **true** or **false**.

long.isnonnegint (···)

Tests whether all its longdouble arguments are non-negative integers and returns **true** or **false**.

long.isposint (···)

Tests whether all its longdouble arguments are positive integers and returns **true** or **false**.

long.ispositive (···)

Tests whether all its longdouble arguments are positive and returns **true** or **false**.

long.tonumber (x [, option])

Converts longdouble `x` into an ordinary Agena number, losing precision. If any option is given, then there is a second Boolean return indicating whether the 80-bit value `x` is less or greater the minimum or maximum value an Agena number can represent, i.e. whether there was an overflow during conversion.

See also: **environ.system.numberranges/mindouble** & **maxdouble** settings, **long.overflow**.

long.tostring (x [, format])

Converts the longdouble `x` into a string with 19 fractional digits by default. You can pass a string `format` that includes the `ld` specifier to control the output, e.g.

```
> long.tostring(long.Pi, "%.15ld"):
```

returns

```
3.141592653589793
```

If a value absolutely is less than 1e-10 or greater than 1e20, then it will be formatted in scientific notation, to prevent output with too many digits or just a zero with very small `x`.

`long.count ([start [, step [, stop]]])`

Returns an iterator function that, each time it is called, returns a new longdouble.

If no argument is given, the first number returned by the iterator is 0, the next call returns 1, the next one 2, and so forth. This means that the longdouble returned with each call is increased by 1.

If only `start` is given, the first value returned by the iterator is `start`, the next call returns `start` + 1, the next one `start` + 2, and so forth. This means that the longdouble returned with each call is increased by 1.

If `start` and `step` are given, the first value returned by the iterator is `start`, the next call returns `start` + `step`, the next one start + 2*`step`, and so forth. This means that the number returned with each call is increased by `step`, which may be negative. In the latter case the next value returned will be less than the current returned value.

If `stop` is given, the iterator returns **null** if the counter value exceeds `stop`. Default is +**long.infinity**.

If `start` or `step` are not longdoubles or numbers, the factory issues an error.

If `start` or `step` are non-integral, the function applies Neumaier summation to avoid round-off errors.

Example:

```
> f := long.count(long.double(1), long.double(-0.1), long.double(-1));

> while c := f() do print(c) od; # counts down
```

A note in advance: All the functions and most of the operators are also available as functions which names always start with "long.x". So, for example, the **sin** operator is also available as the function **long.xsin**, and the **math.chop** function has the alias **math.xchop**.

long.overflow (···)

Returns a Boolean indicating whether all 80-bit longdouble arguments are less or greater the minimum or maximum value an Agena number can represent.

See also: **long.tonumber**.

Basic Arithmetic Operations:

x + y

The operator computes x + y, i.e. performs an addition.

x - y

The operator computes x - y, i.e. performs a subtraction.

x |- y

The operator computes |x - y|, i.e. computes the absolute difference.

x * y

The operator computes x * y, i.e. performs a multiplication.

x / y

The operator computes x / y, i.e. performs a division.

x \ y

The operator computes x \ y, i.e. performs an integer division.

x % y

The operator computes x % y, i.e. returns the modulo.

x ^ y

The operator computes x ^ y, i.e. raises x to the power of y, where y represent any arithmetic number.

x ** y

The operator computes x ** y, i.e. raises x to the power of y, where y represent an integer.

recip (x)

The operator returns the inverse 1/x.

abs (x)

The operator returns the absolute value of x.

sign (x)

Determines the sign of x. The result of the operator is determined as follows:

- 1, if $x > 0$,
- -1, if $x < 0$,
- **undefined**, if $x =$ **undefined**,
- 0 otherwise, even for -0.

signum (x)

Determines the sign of x. The result of the operator is determined as follows:

- 1, if $x \geq 0$,
- -1 otherwise.

long.besselj (n, x)

For longdouble x, returns the Bessel function of the first kind of order n, an non-negative integral Agena number. The return is a longdouble.

See also: **besselj**, **long.bessely**.

long.bessely (n, x)

For longdouble x, returns the Bessel function of the second kind of order n, an non-negative integral Agena number. The return is a longdouble.

See also: **bessely**, **long.besselj**.

long.cathet (x)

Returns $\sqrt{x^2 - y^2}$, avoiding over- and underflow.

See also: **long.hypot**, **long.pytha4**.

long.copysign (x, y)

Returns a longdouble with the magnitude of x and the sign of y, i.e. abs(x) * sign(y). If y is 0, then its sign is considered to be 1.

long.exponent (x)

Returns the exponent e of `x` such that **long.mantissa**(`x`) * 2^e equals `x`.

long.fma (x, y, z)

Performs the fused multiply-add operation `(x * y) + z`, with the intermediate result *not* rounded to the destination type, to improve the precision of a calculation.

long.fmod (x, y)

Computes the remainder from the division of numerator `x` by denominator `y`. The return value is `x` - n * `y`, where n is the quotient of `x` divided by `y`, rounded towards zero to an integer. See also: **long.modf**.

long.hypot (x, y)

Returns $\sqrt{x^2+y^2}$. It is the length of the hypotenuse of a right triangle with sides of length `x` and `y`, or the distance of the point (`x`, `y`) from the origin. The function is slower but more precise than using **sqrt** along with **square**, avoiding over- and underflow.

See also **long.cathet**, **long.pytha**.

long.hypot2 (x)

Returns $\sqrt{1+x^2}$, avoiding over- and underflow.

long.hypot3 (x)

Returns $\sqrt{1-x^2}$, avoiding over- and underflow.

long.koadd (x, y [, q])

The function adds `x` and `y` using Kahan-Ozawa round-off error prevention and returns two longdoubles: the sum of `x` and `y` plus the updated value of the correction variable `q`. The optional correction variable `q` should be 0 at first invocation, and the previously returned correction variable otherwise - if `q` is not given, it defaults to 0.

A typical usage should look like:

```
> x, q -> long.double(0);

> y := long.double('0.1');

> while x < long.double(1) do
>    x, q := long.koadd(x, y, q)
> od;
```

```
> print(s + q);
```

long.fdim (x, y)

Computes x - y if x > y, and return 0 otherwise.

long.mantissa (x)

Returns the mantissa m of x such that m * 2 ^ **long.exponent**(x) equals x. See also: **long.significand**.

long.modf (x)

Returns the integral part of x and its fractional part. The integral part is rounded towards zero. Both the integral and fractional part of the return have the same sign as x. The sum of the two values returned equals x. See also: **long.fmod**.

long.polygen (t)

Takes a table of coefficients (Agena numbers or longdoubles) in descending order and returns a polynomial function that can be evaluated at a given point represented by an Agena number or longdouble. Example:

```
> coeff  := [long.new(3), 2, long.new(1)];
```

So the next statement

```
> f := long.polygen(coeff);
```

is equal to "`f := <: x -> coeff[1]*x**2 + coeff[2]*x + coeff[3] :>`".

```
> f(long.new(2)):
longdouble(17)
```

long.pytha (a, b)

Computes the Pythagorean equation $c^2 = a^2 + b^2$, without undue underflow or overflow, for longdoubles or numbers a, b.

See also: **long.hypot**.

long.pytha4 (a, b)

Computes $a^2 - b^2$, without undue underflow or overflow, for longdoubles or numbers a, b.

See also: **long.cathet**.

long.significand (x)

Returns the mantissa of x in a normalised form, in the range[1, 2), with **long.significand**(x) = 2***long.mantissa**(x) = **long.ldexp**(x, -**long.ilog2**(x)). If x is 0, the return is 0.

Relations:

A note in advance: The 80-bit infinity representation is represented by the constant **long.infinity** and not **infinity**, and that of "undefined" is **long.undefined**.

x = y

The binary operator returns **true** if x is exactly equal to y, and **false** otherwise.

x <> y

The binary operator returns **true** if x is not exactly equal to y, and **false** otherwise.

x ~= y

The binary operator returns **true** if x is approximately equal to y, and **false** otherwise.

See also: ~<> operator, **member**, **long.approx**.

x ~<> y

The binary operator returns **false** if x is approximately equal to y, and **true** otherwise.
See also: ~<> operator, **member**.

x < y

The binary operator returns **true** if x is less than y, and **false** otherwise.

x <= y

The binary operator returns **true** if x is less than or equal to y, and **false** otherwise.

x > y

The binary operator returns **true** if x is greater than y, and **false** otherwise.

x >= y

The binary operator returns **true** if x is greater than or equal to y, and **false** otherwise.

long.isequal (x, y)

Compares x with y and returns **true** if x = y, and **false** otherwise.

long.isunequal (x, y)

Compares x with y and returns **true** if x <> y, and **false** otherwise.

long.isless (x, y)

Compares x with y and returns **true** if x < y, and **false** otherwise.

long.islessequal (x, y)

Compares x with y and returns **true** if x <= y, and **false** otherwise.

long.approx (x, y [, eps])

Compares long numbers x and y and checks whether they are approximately equal. If eps is omitted, **Eps** is used.

The algorithm uses a combination of simple distance measurement (|x-y| ≤ eps) suited for values `near` 0 and a simplified relative approximation algorithm developed by Donald H. Knuth suited for larger values (|x-y| ≤ eps * max(|x|, |y|)), that checks whether the relative error is bound to a given tolerance eps.

The function returns **true** if x and y are considered equal or **false** otherwise. If both a and b represent **infinity**, the function returns **true**. The same applies to a and b being -**infinity** or **undefined**.

long.fmax (x, y)

Returns x if x > y, and y otherwise.

long.fmin (x, y)

Returns x if x < y, and y otherwise.

Powers and Roots:

square (x)

The operator squares x and returns x**2.

cube (x)

The operator raises x to the power of 3.

x squareadd c

The operator computes x^2 + c, preventing round-off errors. See also **long.fma**.

sqrt (x)

Returns the square root of x. If x is a number and negative, the operator returns **undefined**.

long.cbrt (x)

Returns the cubic root of the number x.

long.root (x [, n])

Returns the non-principal n-th root of x. n must be an integer and is 2 by default. Note that since the function computes the non-principal root.

invsqrt (x)

The operator computes the inverse square root x, i.e. 1/**sqrt**(x).

Exponentiation & Logarithms:

antilog2 (z)

The operator computes 2^z, i.e. 2 raised to the power of z. See also: **exp**.

antilog10 (z)

The operator computes 10^z, i.e. 10 raised to the power z. See also: **exp**.

exp (x)

Exponential function; the operator returns the value e^x, with e Euler's number.

See also: **antilog2**, **antilog10**.

long.beta (x, y)

Implements the Beta function for longdoubles x, y. See also: **long.gamma**.

long.expm1 (x)

Returns a value equivalent to **exp**(x) - 1. It is computed in a way that is accurate even if x is near 0, since **exp**(~0) and 1 are nearly equal.

The function is a wrapper to the C function expm1l.

See also: **long.lnp1**.

long.expx2 (x [, sign])

Computes either $e^{x\hat{}2}$ if sign $\geq$ 0, or $e^{-x\hat{}2}$ if sign $<$ 0 while suppressing error amplification that would occur from the in-exactness of the exponential argument x^2. x should be a longdouble, while sign must be an Agena number. By default, sign is positive.

long.fact (n)

Returns the factorial of positive integral value n. With n ≥ 171, returns longdouble(**infinity**).

See also: **long.lnfact**.

long.gamma (x)

Implements the Gamma function of its argument x.

See also: **long.beta**, **long.lnfact**, **long.psi**.

ln (x)

Natural logarithm of x (with base e[1]). If x is non-positive, the operator returns **undefined**.

lngamma (x)

Natural logarithm of the Gamma function of x. If x is non-positive, the operator returns **undefined**.

long.lnabs (x)

Returns **ln**(**abs**(x)) for longdouble x.

long.lnbinomial (n, k)

Returns the natural logarithm of the binomial coefficient

$$\ln \binom{n}{k} = \ln \frac{n!}{k!(n-k)!} = \text{lngamma(n + 1) - lngamma(k + 1) - lngamma(n - k + 1)}$$

avoiding overflows.

See also: **binomial**, **math.lnbinomial**, **long.lnfact**.

long.lnfact (n)

Returns the logarithmic factorial ln n!. See **math.lnfact** for details.

See also: **long.lnbinomial**, **long.fact**, **long.gamma**.

long.lnp1 (x)

Returns a value equivalent to **ln**(1 + x). It is computed in a way that is accurate even if x is near zero.

The function is a wrapper to the C function log1pl.

See also: **long.expm1**.

log (x, b)

The operator returns the logarithm of x to the base b. x and b can be Agena numbers or longdoubles.

long.log2 (x)

Returns the base-2 logarithm of x.

long.ilog2 (x)

Returns the integral base-2 logarithm of x.

long.log10 (x)

Returns the base-10 logarithm of x.

long.psi ([n,] x)

With only longdouble or number x given, computes the Psi (digamma) function, the logarithmic derivative of the gamma function.

In the second form, with n an integer and x a longdouble or number, computes:

- n=0: The Psi (digamma) function, equal to **long.psi**(x).
- n=1: The trigamma function.
- n=2: The tetragamma function.
- n=3: The pentagamma function.

See also: **long.gamma**, **psi**.

Trigonometric Functions and Operators:

sin (x)

The operator returns the sine of x (in radians).

cos (x)

The operator returns the cosine of x (in radians).

tan (x)

The operator returns the tangent of x (in radians).

long.cot (x)

The function returns the cotangent of x (in radians), i.e. 1/**tan**(x).

long.csc (x)

The function returns the cosecant of x (in radians), i.e. 1/**sin**(x).

long.sec (x)

The function returns the secant of x (in radians), i.e. 1/**cos**(x).

long.sincos (x)

Returns the sine and the cosine of longdouble x in radians.

sinc (x)

The operator returns the un-normalised cardinal sine of x (in radians), i.e. **sin**(x)/x, with **sinc**(**long.double**(0)) = **long.double**(1).

Hyperbolic Trigonometric Functions and Operators:

sinh (x)

The operator returns the hyperbolic sine of x (in radians).

cosh (x)

The operator returns the hyperbolic cosine of x (in radians).

tanh (x)

The operator returns the hyperbolic tangent of x (in radians).

long.coth (x)

Returns the hyperbolic cotangent of longdouble x in radians.

long.csch (x)

Returns the hyperbolic cosecant of longdouble x in radians.

long.sech (x)

Returns the hyperbolic secant of longdouble x in radians.

long.sinhcosh (x)

Returns both the hyperbolic sine and hyperbolic cosine. The function is 15 percent faster than calling the **sinh** and **cosh** operators separately.

Inverse and Inverse Hyperbolic Trigonometric Functions and Operators:

arccos (x)

The operator returns the inverse cosine operator (x in radians).

long.arccosh (x)

The function computes the inverse hyperbolic cosine of x (in radians).

long.arccsch (x)

Returns the inverse hyperbolic cosecant of longdouble x in radians.

long.arccot (x)

The function returns the inverse cotangent of x (in radians).

long.arccoth (x)

Returns the inverse hyperbolic cotangent of longdouble x in radians.

arcsec (x)

Returns the inverse secant of longdouble x in radians.

long.arcsech (x)

Returns the inverse hyperbolic secant of longdouble x in radians.

arcsin (x)

The operator computes the inverse sine operator (in radians).

long.arcsinh (x)

The function returns the inverse hyperbolic sine of x (in radians).

arctan (x)

The operator computes the inverse tangent operator (in radians).

long.arctanh (x)

The function returns the inverse hyperbolic tangent of x (in radians).

Error Functions:

long.erf (x)

Returns the error function of number x. It is defined by erf(x) = $\frac{2}{\sqrt{\pi}} \int_{t=0}^{x} e^{-t^2}$.

long.erfc (x)

Returns the complementary error function of x. It is defined by erfc(x) = 1 - erf(x).

long.erfcx (x)

Returns the scaled complementary error function of x. It is defined by erfcx(x) = exp(x ^ 2)*erfc(x).

long.inverf (x)

Returns the inverse error function of number x.

long.inverfc (x)

Returns the inverse complementary error function of number x.

Rounding & Related:

long.ceil (x)

Rounds upwards to the nearest integer larger than or equal to x.

entier (x)

The operator rounds x downwards to the nearest integer. Same as **floor**.

long.floor (x)

Rounds x downwards to the nearest integer. Also: long.floor(x) = long.ceil(x) = -entier(-x).

int (x)

The operator rounds x to the nearest integer towards zero.

frac (x)

The operator returns the fractional part of x, i.e. x - **int**(x), preserving the sign.

long.round (x [, d])

Rounds x to its d-th digit, using the round-half-up method. If d is omitted, the longdouble is rounded to the nearest integer. If d is positive, the function rounds to the d-th fractional digit. If d is negative, it rounds to the d-th integral digit. **long.round** treats positive and negative values symmetrically, and is therefore free of sign bias.

long.chop (x [, eps [, method [, n]]])

Shrinks x more or less near zero to exactly zero, using one of several methods, passed as an integer. The default for eps is **DoubleEps**. The standard method is 0 for hard shrinking. Integral n is used in the SmoothGarrote method.

method	Comment	Value	Domain
0	"Hard", performs hard shrinking	0 x	$\|x\| \le eps$ $\|x\| > eps$
1	"Soft", performs soft shrinking	0 sign(x) ($\|x\|$ - eps)	$\|x\| \le eps$ $\|x\| > eps$
3	"PiecewiseGarrote"	0 $x - eps^2/x$	$\|x\| \le eps$ $\|x\| > eps$
4	"SmoothGarrote"; with $n \to \infty$, goes to "Hard" shrinking	$x^{2n+1}/(x^{2n} + eps^{2n})$	any x
5	"Hyperbola"	0 $\text{sign}(x)\sqrt{x^2 - eps^2}$	$\|x\| \le eps$ $\|x\| > eps$

Method 2 has not been implemented.

Checks:

nonzero (x)

Checks whether x is not 0. The operator returns **true** or **false**.

zero (x)

Checks whether x is 0, respectively. The operator returns **true** or **false**.

even (x)

Checks whether x is even. The operator returns **true** if x is even, and **false** otherwise. With non-integral numbers, the operator returns **false**.

odd (x)

The operator checks whether x is odd. The operator returns **true** if x is odd, and **false** otherwise. With non-integral longdoubles, the operator returns **false**.

fractional (x)

Checks whether x has a fraction, i.e. is not integral, and returns **true** or **false**.

With +/-**infinity** and **undefined**, returns **false**.

integral (x)

Checks whether x is integral, i.e. does not have a fraction, and returns **true** or **false**.

With +/-**infinity** and **undefined**, returns **false**.

finite (x)

The operator checks whether x is neither ±**infinity** nor **undefined**. The operator returns **true** or **false**.

See also: **long.isirregular**.

nan (x)

Checks whether x evaluates to **undefined**. The operator returns **true** or **false**.

See also: **long.isirregular**.

long.isirregular (x)

The operator checks whether x is ±**infinity**, **undefined** or $|x| \geq 2^{64}$. The operator returns **true** or **false**. See also: **finite**.

Miscellaneous:

long.binomd (k, n, p [, any])

With three arguments `k`, `n`, `p` given, the function simplifies to the binomial (probability) distribution defined as:

$$\binom{n}{k}\sum_{j=0}^{k} p^j(1-p)^{n-j}.$$

`k`, `n` must be non-negative integers, n ≥ k, and `p` a longdouble in the range [0, 1].

If `any` fourth argument is given, then the complemented binomial distribution is computed:

$$\sum_{j=k+1}^{n}\binom{n}{j} p^j(1-p)^{n-j}.$$

See also: **stats.binomd**, **long.invbinomd**.

long.cdfnormald (x)

Implements the cumulative density function for the standard normal distribution. The return is the number:

$$\frac{1}{\sqrt{2\pi}}\int_{t=-\infty}^{x} e^{\frac{-t^2}{2}}\,dt$$

See also: **long.invnormald**, **stats.cdfnormald**.

long.invbinomd (k, n, y)

With three arguments `k`, `n`, `y` given, the function simplifies to the inverse binomial distribution. This is accomplished using the inverse beta integral function and the relation

1 - **long.invbionmd**(`k`, `n`, `y`) = **calc.invibeta**(`y`, `n` - `k`, `k`+1).

`k`, `n` must be non-negative integers, n > k, and `p` a longdouble in the range [0, 1].

See also: **stats.invbinomd**.

long.invnormald (y)

Computes the inverse of normal distribution function x = **long.invnormald**(`y`) and returns x, for which the area under the Gaussian probability density function (integrated from minus infinity to x) is equal to `y`.

long.unm (x)

Negates a number or longdouble x and returns it, same as the expression -x.

long.eps ([x [, option]])

The function returns the machine epsilon, the relative spacing between |x| and its next larger longdouble in the machine's 80-bit floating point system. If no argument is given, x is set to 1.

When given any second argument, the function computes a `mathematical` epsilon value that is also dependent on the magnitude of its argument x. It can be used in difference quotients, etc., for it prevents huge precision errors with computations on very small or very large numbers. The mathematical epsilon with respect to x is equal to x* **sqrt**(**long.eps**(x)).

long.zeroin (x [, eps])

By default, returns 0 if longdouble |x| ≤ **long.DoubleEps**, and x otherwise. If the longdouble eps is given as a second argument, returns 0 if |x| ≤ eps, otherwise returns its argument.

long.zerosubnormal (x)

Checks whether its longdouble x is subnormal and in this case returns 0, otherwise returns its argument x. It is useful to prevent excessive CPU usage in case of arguments very close to zero. Note that result retains the sign of x.

See also: **long.zerosubnormal**.

long.nextafter (x, y)

Returns the next machine 80-bit floating-point number of x in the direction toward y.

long.gsolve (A)
long.gsolve (A, v)

Performs Gaussian elimination on a system of linear equations.

In the first form, A must be an augmented m x n matrix created with **linalg.matrix** with m+1 = n.

In the second form, A is a square matrix and v a right-hand side vector, created with **linalg.vector**.

linalg.matrix and **linalg.vector** accept standard Agena numbers only, so the input is 64-bit precision but the output is 80-bit.

The return is the solution vector with longdoubles. It returns **infinity** if an infinite number of solutions has been found, and **undefined** if no solutions exists. It returns **fail** if it could not determine whether no or an infinite number of solutions exist.

Example:

```
> A := linalg.matrix([1, 1, 1, 2], [6, -4, 5, 31], [5, 2, 2, 13]):
[ 1,  1, 1,  2 ]
[ 6, -4, 5, 31 ]
[ 5,  2, 2, 13 ]

> long.gsolve(A):
[longdouble(3.000000..), longdouble(-2.000000..), longdouble(0.999999..)]
```

long.norm (x, a1:a2 [, b1:b2])

Converts `x` in the scale [`a1`, `a2`] to one in the scale [`b1`, `b2`]. The second and third arguments must be pairs of longdoubles.

If the third argument is missing, then `x` is converted to a longdouble in [0, 1].

long.redupi (x)

Subtracts the nearest integer multiple of π from longdouble `x`. if `x` is a multiple of π, returns **long.nought**.

See also: **long.rempio2**, **long.wrap**.

long.rempio2 (x [, option])

Conducts an argument reduction of `x` into the range $|y| < \frac{\pi}{2}$ and returns y = `x` - N*$\frac{\pi}{2}$. If any `option` is given, then the function also returns N, or actually the last three digits of N. The number of operations conducted are independent of the exponent of the input.

See also: **long.redupi**, **long.wrap**.

long.wrap (x [, a [, b]])

Conducts a range reduction `x` to the interval [`a`, `b`). If `x` $\in$ [`a`, `b`), `x` is simply returned.

In the second form, if `a` is not given, `a` is set to $-\pi$ and `b` to $+\pi$. If `a` is given but not `b`, `a` is set to -`a` and `b` to +`a`, so `a` should be positive.

See also: **long.redupi**, **long.rempio2**.

long.signbit (x)

Checks whether x has its sign bit set and returns **true** or **false**. For example, although -0 = 0, **long.signbit**(-0) ⇒ **true** and **long.signbit**(0) ⇒ **false**.

long.frexp (x)

Returns the mantissa m and the exponent e of x such that x = $m2^e$. e should be integral, and the value of m is in the range [0.5, 1) (or zero when x is zero). The operation is bijective, i.e. **long.ldexp**(**long.frexp**(x)) = x.

long.ldexp (m, e)

Returns $m2^e$ (e should be integral).

long.multiple (x, y [, option])

Checks whether x is a multiple of y, i.e. whether x/y evaluates to an integral, and returns **true** or **false**.

Also returns **true** with x = 0 and any non-zero y.

If y is zero, **long.undefined** or +/-**long.infinity**, the function returns **fail**.

By passing the optional third argument **true**, a tolerant check is done, with subnormal x or y first converted to zero, and a subsequent approximate equality check to the nearest integer of x/y. The tolerance value internally used is the value of **DoubleEps** at the time of the function call.

In most cases, it may suffice to just call **integral**(x/y).

IEEE:

`long.fpclassify (x)`

For the given x, returns

- **long.fp_nan** if x is **undefined**,
- **long.fp_infinite** if x is infinite, i.e. +/-**infinity**,
- **long.fp_subnormal** if x is subnormal,
- **long.fp_zero** if x is zero,
- **long.fp_normal** if x is normal, including irregular values $\geq 2^{52}$.

`long.isfinite (x)`

Returns **true** if **long.fpclassify**(x) <> **long.fp_nan** and **long.fpclassify**(x) <> **long.fp_infinite**, and **false** otherwise.

`long.isinfinite (x)`

Returns **true** if **long.fpclassify**(x) = **long.fp_infinite**, and **false** otherwise.

`long.isnormal (x)`

Returns **true** if **long.fpclassify**(x) = **long.fp_normal**, and **false** otherwise.

`long.isundefined (x)`

Returns **true** if **long.fpclassify**(x) = **long.fp_nan**, and **false** otherwise.

`long.issubnormal (x)`

Returns **true** if **long.fpclassify**(x) = **long.fp_subnormal**, and **false** otherwise.

See also: **long.normalise**.

`long.iszero (x)`

Returns **true** if **long.fpclassify**(x) = **long.fp_zero**, and **false** otherwise.

`long.normalise (x)`

Returns **long.MinDouble** with the sign of x if x is subnormal, and returns x otherwise.

See also: **long.issubnormal**, **long.zerosubnormal**.

Available constants are:

Constant	Value	Comment
long.Pi	π	
long.Pi2	2π	
long.PiO2	$\pi/2$	
long.PiO4	$\pi/4$	
long.PiO180	$\pi/180$	radians per degree
long.InvPi	$1/\pi$	
long.InvPi2	$1/(2\pi)$	
long.InvPiO4	$4/\pi$	
long.InvPiSqO4	$4//\pi^2$	
long.Zeta2	$Pi^2/6$	
long.E	E = exp(1)	
long.sqrt2	$\sqrt{2}$	
long.sqrt3	$\sqrt{3}$	
long.ln2	ln(2)	
long.Invln2	1/ln(2)	
long.Invsqrt2	$1/\sqrt{2}$	
long.Phi	$(1 + \sqrt{5})/2$	Golden ratio
long.InvPhi	$1/((1 + \sqrt{5})/2)$	inverse Golden ratio
long.InvPhiSq	$(1/((1 + \sqrt{5})/2))^2$	
long.lnPhi	$\ln(1 + \sqrt{5})/2$	logarithm of Golden ratio
long.InvlnPhi	$1/\ln(1 + \sqrt{5})/2$	inverse
long.EulerGamma	0.57721566...	Euler-Mascheroni constant
long.naught **long.nought**	0	zero
long.one	1	
long.two	2	
long.three	3	
long.four	4	
long.five	5	
long.six	6	
long.seven	7	
long.eight	8	
long.nine	9	
long.ten	10	
long.eleven	11	
long.twelve	12	
long.fifty	50	
long.hundred	100	
long.thousand	1,000	
long.half	0.5	
long.quarter	0.25	
long.tenth	0.1	
long.fifth	0.2	
long.hundredth	0.01	

Constant	Value	Comment
long.thousandth	0.001	
long.threequarter	0.75	
long.third	1/3	
long.sixth	1/6	
long.eighth	0.125	
long.twelfth	1/12	
long.sixteenth	0.0625	
long.infinity	infinity	
long.undefined	undefined	
long.DoubleEps	1.084202172485504434e-19	long double machine epsilon at 1
long.ldblmin	3.3621031431120935e-4932	smallest *normalised* positive longdouble value (a longdouble may be smaller, of course, but then it is subnormal)
long.ldblmax	1.189731495357232e+4932	maximum value
long.ldblepsilon	'2.2204460492503131e-016	long double epsilon

These constants - with the exception of **long.ldb*** - have all been defined in source file lib/library.agn.

11.16 ints - 64-bit Integer Arithmetic

11.16.1 Introduction

The package provides functions to conduct mathematics on signed and unsigned 64-bit integers.

Standard operators like +, -, *, /, \, ^, %, <, =, ~=, >, unary minus and the bitwise operators ~~, &&, ||, ^^, <<, >>, <<<, >>>, **sqrt**, etc. are supported, see metamethod table below.

Almost all of the operators and functions do not check for potential overflow and if operations overflow, the results may be system-dependent.

11.16.2 Usage

Let us start with examples. There are two constructors, **ints.uint64** to create unsigned 64-bit integers and **ints.int64** to create signed 64-bit integers. Both functions accept either numbers or - for more accuracy - strings representing numbers which is especially important if you want to handle numbers beyond the range of Agena numbers.

```
> import ints;

> x := ints.uint64('3000'):
uint64(3000)

> y := ints.uint64('0x03E8'):
uint64(1000)
```

Most mathematical integer operations are implemented as metamethods so you can use the common arithmetic operators:

```
> x + y:
uint64(4000)

> x/y:
uint64(3)

> sqrt(y):
uint64(31)
```

However, you cannot directly compare **ints** numbers with Agena numbers, so either use **ints.tonumber** or a constructor, like this:

```
> x - y = 2000:
false

> ints.tonumber(x - y) = 2000:
true

> x - y = ints.uint64(2000):
true
```

You can mix Agena numbers and 64-bit integers with the arithmetic operators **+**, **-**, *****, /, \, **%**, **^**, ******.

The arithmetic 64-bit operators **+**, **-** and ***** detect numeric overflow or underflow. In such a case, they set the global variable "numoverflow" to **true**. You must manually unset the variable if you need up-to-date error information in subsequent arithmetic operations. Example:

```
> numoverflow:
null

> ints.uint64('0xFFFFFFFFFFFFFFFF') * 2:
uint64(18446744073709551614)

> numoverflow:
true

> numoverflow := null
```

See also: The **long** package implementing 80-bit floating-point arithmetic, described in Chapter 11.15, and the **mapm** package for arbitrary-precision math in Chapter 11.3.

The package provides the following metamethods and you can mix ordinary Agena numbers and 64-bit integers with binary operators:

Operator	Name	Description
+	'__add'	addition
-	'__sub'	subtraction
*	'__mul'	multiplication
/	'__div'	division
\	'__intdiv'	dito
%	'__mod'	modulo
^	'__pow'	power with integer exponent
**	'__ipow'	dito
&&	'__band'	bitwise AND
||	'__bor'	bitwise OR
^ ^	'__bxor'	bitwise XOR
~~	'__bnot'	bitwise NOT (32-bit unsigned)
<<	'__bshl'	bitwise left-shift
>>	'__bshr'	bitwise right-shift
<<<	'__brotl'	bitwise left-rotation
>>>	'__brotr'	bitwise right-rotation
-	'__unm'	unary minus
<	'__lt'	less-than
=	'__eq'	equals
~=	'__aeq'	approximately equals
<=	'__le'	less-or-equal
<>	n/a	not equals

Operator	Name	Description
abs	'__abs'	absolute value
|-	'__absdiff'	absolute difference
sign	'__sign'	sign
square	'__square'	square (x ^ 2)
cube	'__cube'	cube (x ^ 3)
antilog2	'__antilog2'	exponentiation to base 2 (2^x)
antilog10	'__antilog10'	exponentiation to base 10 (10^x)
sqrt	'__sqrt'	square root ($\sqrt{x}$)
even	'__even'	even number check
odd	'__odd'	odd number check
zero	'__zero'	test for zero
nonzero	'__nonzero'	test for non-zero
n/a	'__gc'	garbage collection
n/a	'__tostring'	conversion to a string, e.g. for the pretty printer

Unsigned 64-bit integer constants are:

ints.naught and **ints.nought** representing zero, plus **ints.one**, **ints.two**, **ints.three**, **ints.four**, **ints.five**, **ints.six**, **ints.seven**, **ints.eight**, **ints.nine** and **ints.ten** representing one, two, etc.

The constructors and checks are:

ints.uint64 (x [, true])
ints.uint64 (hx, lx)

In the first form creates an unsigned 64-bit integer from number or string `x`.

When given number `x` and the **true** option, **ints.uint64** converts Agena number `x` into an unsigned 64-bit integer, by copying the bytes, so that you can apply standard IEEE 754 operations on it. To check whether **Pi** is a fractional number, we enter:

```
> u := ints.uint64(Pi, true):
uint64(4614256656552045848)

> exponent := ((u >> 52) && 0x7FF) - 1023:
uint64(1)

> mask := (1 << (52 - exponent)) - 1:
uint64(2251799813685247)
```

If the following query is not zero then **Pi** is a fraction:

```
> u && mask:
uint64(3184838310972696)
```

If `x` is a string it may represent a decimal or hexadecimal integer. A hexadecimal string `x` must always start with a leading '0x'. The function checks for potential overflow.

The return is a userdata value. Examples:

```
> ints.uint64(100):
uint64(100)

> ints.uint64('100'):
uint64(100)

> ints.uint64('0x64'):
uint64(100)
```

In the second form, takes two unsigned 32-bit integers `hx`, `lx`, represented by standard Agena numbers and creates an unsigned 64-bit integer. `hx` is the higher word, `lx` the lower word in the resulting unsigned 64-bit integer.

ints.int64 (x)
ints.int64 (hx, lx)

In the first form creates a signed 64-bit integer from number or string `x`.

If `x` is a string it may represent a decimal or hexadecimal integer. A hexadecimal string `x` must always start with a leading '0x' or '-0x'.

The function checks for potential overflow.

In the second form, takes two signed 32-bit integers `hx`, `lx`, represented by standard Agena numbers and creates a signed 64-bit integer. `hx` is the higher word, `lx` the lower word in the resulting signed 64-bit integer.

The return is a userdata value. Examples:

```
> ints.int64(-100):
int64(-100)

> ints.int64('-100'):
int64(-100)

> ints.int64('-0x64'):
int64(-100)

> hx, lx := numcuckoo.hash(Pi);

> h := ints.uint64(hx, lx):
uint64(10005537424831946224)
```

ints.copy (x)

Creates a genuine duplicate of the signed or unsigned 64-bit integer `x`.

ints.checkint64 (··· [, procname])

Checks whether all its arguments are signed 64-bit integers and issues an error otherwise. You may optionally pass the name of the calling function for `procname` so that it is included in the error message. If all arguments passed the test successfully, the function returns nothing.

See also: **ints.checkuint64**, **ints.isint64**, **ints.isuint64**.

ints.checkuint64 (··· [, procname])

Checks whether all its arguments are unsigned 64-bit integers and issues an error otherwise. You may optionally pass the name of the calling function for `procname` so that it is included in the error message. If all arguments passed the test successfully, the function returns nothing.

See also: **ints.checkint64**, **ints.isint64**, **ints.isuint64**.

ints.isint64 (···)

Checks whether all its arguments are signed 64-bit integers and returns **true** or **false**.

See also: **ints.checkint64**, **ints.checkuint64**, **ints.isuint64**.

ints.isuint64 (···)

Checks whether all its arguments are unsigned 64-bit integers and returns **true** or **false**.

See also: **ints.checkint64**, **ints.checkuint64**, **ints.isint64**.

ints.swap (x, y)

Swaps the values of 64-bit integers `x`, `y` in-place. The function returns nothing.

The mathematical functions can also be called OOP-style. They are:

ints.cbrt (x)

Computes the integer cubic root of signed or unsigned 64-bit integer `x`.

See also: **ints.root**, **sqrt** operator.

ints.gcd (x, y)

Calculates the Greatest Common Divisor (GCD) of two signed or unsigned 64-bit integers `x`, `y`. Both `x` and `y` must both be signed or both be unsigned. The type of returns depends on `x`, `y`.

See also: **ints.lcm**.

ints.invmod (a, m)

Calculates the modular multiplicative inverse of `a` modulo `m`. Returns 'x' such that `a`*x = 1 (mod `m`). Returns 0 if the inverse does not exist (**ints.gcd**(a, m) ≠ 1).

`a`, `m` must both either be signed or unsigned 64-bit integers. The return is a signed or unsigned 64-bit integer, depending on `a`, `m`.

See also: **ints.mulmod**, **ints.powmod**.

ints.iscoprime (m, n)

Checks if the signed or unsigned 64-bit integers `m`, `n` are relatively prime (or co-prime) and returns **true** or **false**. `n` may also be a non-negative Agena number or a string representing a non-negative integer.

Two numbers are co-prime if they share no common positive factors other than 1.

See also: **ints.isprime**.

ints.isnegint (···)

Tests whether all its arguments represent negative signed 64-bit integers and returns **true** or **false**. With unsigned 64-bit integers always returns **false**.

ints.isnonnegint (···)

Tests whether all its arguments represent non-negative signed 64-bit integers and returns **true** or **false**. With unsigned 64-bit integers always returns **true**.

ints.isposint (···)

Tests whether all its arguments represent positive signed or unsigned 64-bit integers and returns **true** or **false**.

ints.ispow2 (n)

Checks whether the signed or unsigned 64-bit integer `n` is a power of two and returns **true** or **false**.

The input can also be an Agena number or a string depicting a 64-bit integer.

With signed input `n`, the check is done for the absolute value of `n`.

See also: **ints.nextpow2**.

ints.isprime (n)

Checks whether the signed or unsigned 64-bit integer `n` is a prime and returns **true** or **false**. `n` may also be a non-negative Agena number or a string representing a non-negative integer.

With signed input `n`, the check is done for the absolute value of `n`.

See also: **ints.iscoprime**, **ints.nextprime**, **ints.prevprime**.

ints.lcm (x, y)

Calculates the Least Common Multiple (LCM) of two signed or unsigned 64-bit integers `x`, `y`. Both `x` and `y` must both be signed or both be unsigned. The type of returns depends on `x`, `y`.

See also: **ints.gcd**.

ints.log2 (n)

Computes the base-2 logarithm of the signed or unsigned 64-bit integer `n` and returns a 64-bit integer of the same integer type as `n`.

See also: **ints.log10**.

ints.log10 (n)

Computes the base-10 logarithm of the signed or unsigned 64-bit integer `n` and returns a 64-bit integer of the same integer type as `n`.

See also: **ints.log10**.

ints.max (m, n)

Determines the maximum of the signed or unsigned 64-bit integers m, n and returns it.

See also: **ints.min**, **ints.minmax**.

ints.mean (m, n)

Returns the arithmetic mean of the signed or unsigned 64-bit integers m, n, without overflowing. The return depends on the type of the input.

ints.min (m, n)

Determines the minimum of the signed or unsigned 64-bit integers m, n and returns it.

See also: **ints.max**, **ints.minmax**.

ints.minmax (m, n)

Determines the minimum and the maximum of the signed or unsigned 64-bit integers m, n and returns then in this order.

See also: **ints.max**, **ints.minmax**.

ints.mulmod (a, b, m)

Conducts modular multiplication (a*b) % m, avoiding overflow. a, b, m must either be all unsigned 64-bit integers, non-negative Agena numbers or strings representing a non-negative integer.

Alternatively, a, b, m are all signed 64-bit integers, but m should be positive.

If m is zero, the function returns zero.

See also: **ints.invmod**, **ints.powmod**.

ints.nextpow2 (n)

For the signed or unsigned 64-bit integer n returns the next power of two. n may also be a non-negative Agena number or a string representing a non-negative integer.

With signed input n, the function computes the next power for the absolute value of n.

When overflowing, the function sets the global variable "numoverflow" to **true**. You must manually unset the variable if you need up-to-date error information in subsequent arithmetic operations.

See also: **ints.ispow2**.

ints.nextprime (n)

For the signed or unsigned 64-bit integer `n` returns the next prime. `n` may also be a non-negative Agena number or a string representing a non-negative integer.

With signed input `n`, the function computes the next prime for the absolute value of `n`.

If the return is zero it indicates an overflow. When overflowing, the function sets the global variable "numoverflow" to **true**. You must manually unset the variable if you need up-to-date error information in subsequent arithmetic operations.

See also: **ints.isprime**, **ints.prevprime**.

ints.powmod (a, b, m)

Conducts modular exponentiation (`a ^ b`) % `m`, avoiding overflow. `a`, `b`, `m` must all be unsigned 64-bit integers, non-negative Agena numbers or strings representing a non-negative integer.

Alternatively, `a`, `b`, `m` may all be signed 64-bit integers, with the modulus `m` positive and exponent `b` non-negative.

If `m` is zero, the function returns zero.

See also: **ints.invmod**, **ints.mulmod**.

ints.prevprime (n)

For the signed or unsigned 64-bit integer `n` returns the largest prime less than `n`. The input may also be a non-negative Agena number or a string representing a non-negative integer.

With signed `n`, the function computes the previous prime for the absolute value of `n`.

If the return is zero it indicates an underflow. When underflowing, the function sets the global variable "numoverflow" to **true**. You must manually unset the variable if you need up-to-date error information in subsequent arithmetic operations.

See also: **ints.isprime**, **ints.nextprime**.

ints.random ([seed])

Returns an unsigned 64-bit random integer. If no `seed` is given, it is determined automatically in the first call to the function. Alternatively, you may pass your own seed, a signed or unsigned 64-bit integer `seed` with the first call. The return is not cryptographically secure. Call **ints.toint64** on the result if you need a cast to a signed integer.

ints.root (x, n)

Computes the `n`-th root of signed or unsigned 64-bit integer `x`. `n` is an Agena number and should represent a positive integer.

See also: **ints.cbrt**, **sqrt** operator.

The Bit-twiddling functions are:

ints.getbit (x, pos)

In the signed or unsigned 64-bit integer `x` retrieves the bit currently set at position `pos` and returns the Agena number 0 or 1. `pos` is an integer in the range [1, 64].

See also: **ints.getbits**, **ints.setbit**.

ints.getbits (x)

In the signed or unsigned 64-bit integer `x` retrieves all the bits and returns them in a sequence.

See also: **ints.getbit**, **ints.setbit**.

ints.gethigh (x)

Retrieves the upper 32-bits of a signed or unsigned 64-bit integer `x` and returns them as a signed Agena number.

See also: **ints.getlow**, **ints.getwords**, **ints.sethigh**, **ints.setlow**, **ints.setwords**.

ints.getwords (x)

Retrieves both the higher and the lower 32-bits of a signed or unsigned 64-bit integer `x` and returns them as two Agena numbers. If `x` is signed, then the result may also be signed. If `x` is unsigned, so will be the result.

See also: **bytes.numwords**, **ints.gethigh**, **ints.getlow**, **ints.sethigh**, **ints.setlow**, **ints.setwords**, **ints.tonumber**.

ints.getlow (x)

Retrieves the lower 32-bits of a signed or unsigned 64-bit integer `x` and returns them as a signed Agena number.

See also: **ints.gethigh ints.getwords**, **ints.sethigh**, **ints.setlow**, **ints.setwords**.

ints.leadzeros (x)

Counts the number of leading zeros (clz) in the unsigned 64-bit integer `x`, returned as an Agena number, and also returns the modified value of `x` after this operation as a second result, as an unsigned 64-bit integer, where all bits starting with the first non-zero bit in `x` are set to 1. `x` may also be an Agena number or a string, both representing a non-negative integer.

See also: **ints.leastsigbit**, **ints.mostsigbit**, **ints.onebits**, **ints.trailzeros**.

ints.leastsigbit (x)

Returns the position of the least significant bit (lsb) in the unsigned 64-bit integer `x`, here the smallest index of the first 1-bit, counting from bit index 1. If `x` = 0, returns 0. The return is an Agena number. `x` may also be an Agena number or a string, both representing a non-negative integer.

See also: **ints.leadzeros**, **ints.mostsigbit**, **ints.onebits**.

ints.mostsigbit (x)

Returns the position of the most significant bit (msb) in the unsigned 64-bit integer `x`, i.e. the largest index of a 1-bit, counting from bit index 1. If `x` = 0, returns 0. The return is an Agena number. `x` may also be an Agena number or a string, both representing a non-negative integer.

See also: **ints.leadzeros**, **ints.leastsigbit**, **ints.mostsigbit**.

ints.onebits (x)

Returns the number of bits set in the unsigned 64-bit integer `x`. The return is an Agena number. `x` may also be an Agena number or a string, both representing a non-negative integer.

See also: **ints.leadzeros**, **ints.mostsigbit**.

ints.parity (x)

Determines the parity of the unsigned 64-bit integer `x`, i.e. the number of 1-bits in `x` modulo 2. `x` may also be an Agena number or a string, both representing a non-negative integer.

Returns 0 if `x` is of even parity, and 1 in case of odd parity.

See also: **bytes.parity32**, **hashes.parity**.

ints.reverse (x)

Reverses the order of all 64 bits in the signed or unsigned 64-bit integer `x` and returns a new 64-bit integer, the type depending on the input `x`.

See also: **ints.setbit**.

ints.setbit (x, pos, bit)

In the signed or unsigned integer `x`, sets zero-bit or a one-bit bit in-place at position `pos`. bit can be 0 ore 1. `pos` is an integer in the range [1, 64]. The function returns nothing.

See also: **ints.getbit**, **ints.reverse**.

ints.sethigh (x, h)

Takes a signed or unsigned 64-bit integer `x` and an Agena number `h` and sets `h` to the higher 32 bits in `x`.

The operation is in-place, thus modifying `n`. The function returns nothing.

See also: **ints.gethigh**, **ints.getlow**, **ints.getwords**, **ints.setlow**, **ints.setwords**.

ints.setlow (x, l)

Takes a signed or unsigned 64-bit integer `x` and an Agena number `l` and sets `l` to the lower 32 bits in `x`.

The operation is in-place, thus modifying `n`. The function returns nothing.

See also: **ints.gethigh**, **ints.getlow**, **ints.getwords**, **ints.sethigh**, **ints.setwords**.

ints.setwords (h, l [, sign])

Takes two integers `h`, `l` and sets them as the higher and lower 32-bits of a new signed or unsigned 64-bit integer which is the result. By default or if `sign` is set to **false**, an unsigned integer will be returned. If `sign` is **true** the resulting 64-bit integer will be signed.

See also: **ints.gethigh**, **ints.getlow**, **ints.getwords**, **ints.sethigh**, **ints.setlow**.

ints.trailzeros (x)

Counts the number of trailing zeros (ctz) in the unsigned 64-bit integer `x`, returned as an Agena number, and also returns the modified value of `x` after this operation as a second result, as an unsigned 64-bit integer, where all bits starting with the least significant bit in `x` are set to 1. `x` may also be an Agena number or a string, both representing a non-negative integer.

See also: **ints.leadzeros**, **ints.leastsigbit**, **ints.mostsigbit**, **ints.onebits**.

The conversion functions can also be called OOP-style. They are:

ints.tobytes (x)

Returns all the bytes in the signed or unsigned 64-bit integer `x` in a sequence. On Big Endian systems, the return will be in Little Endian order. `x` may also be an Agena number or a string, both representing a non-negative integer.

ints.tonumber (x)

Converts the signed or unsigned 64-bit integer `x` to an Agena number. The second result is a boolean indicating potential overflow.

See also: **ints.gethigh**, **ints.getlow**, **ints.getwords**.

ints.tostring (x)

Returns the decimal integer representation of the signed or unsigned 64-bit integer `x` as a string.

ints.toint64 (x)
ints.touint64 (x)

Directly copy the raw binary representation of the Agena number `x` (a C double) into a signed or unsigned 64-bit integer (C int64_t, uint64_t). This is commonly known as type punning or viewing the floating-point value as a sequence of bits.

It is way to examine or manipulate the raw IEEE 754 bits of a double-precision float, often used in serialization, hash functions, or low-level bit manipulation of floating-point numbers.

The return is a 64-bit signed or unsigned number, see **ints.int64**, **ints.uint64**. The result depends on the Endianness of the underlying system.

Instead of an Agena number, `x` may also be a signed or unsigned 64-bit integer. In this case, the functions just cast an unsigned 64-bit integer to a signed one and vice versa. If in a call to **ints.toint64** `x` is a signed 64-bit integer, it is just returned. If in a call to **ints.touint64** `x` is an unsigned 64-bit integer, it is just returned.

See also: **ints.todouble**.

ints.todouble (x)

Does the opposite of **ints.touint64**, **ints.toint64** and returns an Agena number for signed or unsigned 64-bit integer `x`.

Example:

```
> sqrtapprox := proc(x :: number) is
>    local one := ints.one;
>    local y := ints.todouble(
>       ((ints.touint64(x) - (one << 52)) >> one) + (one << 61)
>    );
>    return 0.5*(y + x/y);
> end;

> sqrtapprox(2):
1.4166666666667
```

11.17 combinat - Combinatorics

This package features some combinatorial functions. See also: **fact**, **binomial**.

combinat.bell (n [, x])

Returns the `n`-th Bell number, i.e. counts the possible partitions of a set. With `n` > 218, always returns **infinity**.

If you pass the second argument `x`, the function computes the `n`-th Bell polynomial in number `x`.

See also: **combinat.bernoulli**, **combinat.catalan**, **combinat.euler**.

combinat.bernoulli (n [, x] [, option])

Computes the `n`-th Bernoulli number Bn and returns a number. `n` should be a non-negative integer. If you pass the second argument `x`, the function computes the `n`-th Bernoulli polynomial in number `x`.

If you pass the `eps` = <number> option, `eps` represents the internal bailout value and by default equals **DoubleEps**.

Examples:

```
> combinat.bernoulli(4):
-0.033333333333315

> combinat.bernoulli(3, Pi, eps = hEps):
17.772666405461
```

See also: **combinat.bell**, **combinat.catalan**, **combinat.euler**.

combinat.cartprod (l [, flag])

When called with argument `l` only, returns the Cartesian product of a table of two or more tables `l`, or a sequence of two or more sequences `l`. The type of the result is the same as the type of the input. The structures in `l` may be of different size but must be non-empty.

If any non-null second argument is given, the function creates an iterator that each time it is called, returns a tuple in the Cartesian product. A typical usage might look like this:

```
> f := combinat.cartprod([[1, 2, 3], [30], [50, 100]], true);

> for i in f do print(i) od
[1, 30, 50]
[1, 30, 100]
[2, 30, 50]
```

```
[2, 30, 100]
[3, 30, 50]
[3, 30, 100]
```

combinat.catalan (n [, x])

Returns the n-th Catalan number. If you pass the second argument x, the function computes the n-th Catalan polynomial - specifically the Narayana polynomial, the standard polynomial generalization of Catalan polynomials - in number x.

See also: **combinat.bell**, **combinat.bernoulli**, **combinat.euler**.

combinat.choose (n [, m])

The function constructs the combinations of table elements.

If n is a table, then the function returns a table of the combinations of the table elements.

If n is a positive integer, it is interpreted in the same way as a set of the first n integers.

If m is given, then only combinations of size m are generated; otherwise, all combinations are generated, including the empty combination, that is, the power set is generated. Duplicates in table n are taken into account.

See also: **combinat.cartprod**, **combinat.numbcomb**, **combinat.permute**.

combinat.euler (n [, x] [, option])

Computes the n-th Euler number En and returns a number. n should be a non-negative integer.

If you pass the second argument x, the function computes the n-th Euler polynomial in number x.

If you pass the eps = <number> option, eps represents the internal bailout value and by default equals **DoubleEps**.

Example:

```
> combinat.euler(2, 1.5, eps = hEps):
0.75
```

See also: **combinat.bell**, **combinat.bernoulli**, **combinat.catalan**.

combinat.fib (n [, x])

Computes the n-th Fibonacci polynomial in x. If x is not given, returns **numtheory.fib**(n).

Fibonacci polynomials are defined by the recursion:

$$F(n, x) = x\,F(n\text{-}1, x) + F(n - 2, x) \text{ with } F(0, x) = 0,\ F(1, x) = 1.$$

Note that we have $F(n) = F(n, 1)$.

See also: **linalg.fib**, **numtheory.fib**.

combinat.numbcomb (n, r)
combinat.numbcomb (s, r)

In the first form, counts the number of combinations of n things taken r at a time. In the second form, the function counts the number of combinations all the elements in the set s taken r at a time. The set may include data of any type. n, r are numbers and may even be negative or fractions.

See also: **binomial**, **fact**, **combinat.choose**, **combinat.numbperm**.

combinat.numbpart (n [, r])

Computes the number of partitions of n, the partition numbers, taken r at a time. By default, $r = n$.

combinat.numbperm (n, r)
combinat.numbperm (s, r)

In the first form, counts the number of permutations of n things taken r at a time. In the second form, the function counts the number of permutations of all the elements in the set s taken r at a time. The set may include data of any type.

If n or r are non-integral or negative, the function returns **undefined**.

See also: **binomial**, **fact**, **combinat.numbcomb**, **combinat.permute**.

combinat.permute (n, r)

The function constructs the permutations of table elements.

If n is a table, then the function returns a table of all the permutations of the elements taken r at a time.

If n is a non-negative integer, it is interpreted in the same way as a table of the first n integers.

If `r` is not specified, then it is taken to be equal to the number of elements in `n`.

The permutations are generated in order. Duplicates in n are respected.

Examples:

```
> combinat.permute([1, 2, 3], 3):
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

> combinat.permute(3, 2):
[[1, 2], [1, 3], [2, 1], [2, 3], [3, 1], [3, 2]]
```

See also: **combinat.cartprod**, **combinat.choose**, **combinat.numbperm**.

combinat.stirling1 (n, k)

Computes the Stirling number of the first kind for `n`, `k`.

combinat.stirling2 (n, k)

Computes the Stirling number of the second kind for `n`, `k`.

11.18 numtheory - Number Theory

This package features functions for Number Theory.

numtheory.binet (k, n [, heps])

For integral n > 2 , computes the n-th term in the sequence of Metallic numbers of order k which is equal to k times the (n - 1)st term plus the (n - 2)nd term:

numtheory.binet(k, n) = k***numtheory.binet**(k, n - 1) + **numtheory.binet**(k, n - 2)

or in other words determines the n-th Metallic number of order k by multiplying k with the predecessor and adding the term before that predecessor.

The defaults for integral n < 2 are: **numtheory.binet**(k, 0) = 0 and **numtheory.binet**(k, 1) = 1.

n may also be a fractional number or a complex number - in these cases the function uses the generalised version of Binet's formula:

$$\frac{1}{\sqrt{k^2+4}}\left(\left(\frac{k+\sqrt{k^2+4}}{2}\right)^n-\left(\frac{k-\sqrt{k^2+4}}{2}\right)^n\right)$$

Specifically, with k = 1 the function computes Fibonacci numbers, with k = 2 Pell numbers and with k = 3 Bronze Fibonacci numbers.

In general, the result always is a complex number, even for integral n. Very small values close to zero in the real and imaginary parts of the result are automatically set to zero. You can change the threshold for zeroing by passing the third argument heps which is the constant **hEps** = 1.4901161193848e-12 by default.

See also: **numtheory.fib**.

numtheory.congruentprime (n [, a [, b]])

Determines whether integer n is a prime number congruent to a modulo b - or in other words: a prime of the form bn + a, and returns it; otherwise for n, returns the next prime number congruent to a modulo b. By default, a is 3 and b is 4.

The function is implemented in Agena and included in the lib/library.agn file.

See also: **numtheory.isprime**, **numtheory.nextprime**, **numtheory.prevprime**, **numtheory.primes**.

numtheory.factors (n)

Returns all the integers that divide n without remainder. The result is a table.

With |n| < 2, the result will only include n. With |n| > 1, the result will never include 1 and n, and it is also sorted in ascending order.

By default, all the results are non-negative. If you pass the option true, however, the first number in the resulting table will be negative if n is negative.

See also: **numtheory.gcd**, **numtheory.ifactor**, **numtheory.ifactors**, **numtheory.lcm**. **numtheory.primes**.

numtheory.fib (n [, heps])

With n a non-negative integer returns the n-th Fibonacci number. If n is an integer greater then 76, the function switches to complex arithmetic and may return a result that is too large to be accurately represented. The defaults are: **numtheory.fib**(0) = 0 and **numtheory.fib**(1) = 1; with all other values computed by **numtheory.fib**(n) := **numtheory.fib**(n - 2) + **numtheory.fib**(n - 1).

If n is a negative integer, a rational number or any complex number, the function computes the result according to Binet's formula and returns a complex result if the imaginary part of the result is non-zero and an Agena number otherwise:

$$\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right) = \frac{1}{\sqrt{5}}(\text{Phi}^n - \text{InvPhi}^n)$$

Very small values close to zero in the real and imaginary parts of the result are automatically set to zero. You can change the threshold for zeroing by passing the second argument heps which is the constant **hEps** = 1.4901161193848e-12 by default. Results $\geq 2^{53}$ may not be correct since values that large cannot accurately be represented with Agena (complex) numbers.

See also: **combinat.fib**, **linalg.fib**, **numtheory.binet**, **numtheory.fibinv**.

numtheory.fibinv (n)

For any non-negative integer n returns the index i of the Fibonacci number with **fib**(i) <= n < **fib**(i + 1). The function is implemented in Agena and included in the lib/library.agn file.

See also: **numtheory.binet**, **numtheory.fib**.

numtheory.gcd (...)

Returns the greatest common divisor of at least two numbers. If at least one number is fractional, 1 will be returned.

See also: **numtheory.factors**, **numtheory.ifactor**, **numtheory.lcm**, **numtheory.primes**.

numtheory.ifactor (n)

Computes the complete integer factorisation of integer `n`. The return is a table of all the respective primes, and their product equals `n`.

The function mimics the Maple function of the same name as much as possible.

Examples:

```
> numtheory.ifactor(24):
[2, 2, 2, 3]

> numtheory.ifactor(-17):
[-17]
```

The function is written in Agena and included in the lib/library.agn file.

See also: **numtheory.factors**, **numtheory.gcd**, **numtheory.lcm**, **numtheory.primes**.

numtheory.ifactors (n)

Like **numtheory.ifactor**, the function computes the complete integer factorisation of integer `n`. The result, however, is returned in the form [u, [[p_1, e_1], ... , [p_m, e_m]], where n = u*$p_1^{e[1]}$* ... *$p_m^{e[m]}$, p_i is a prime integer, e_i is its exponent (multiplicity) and u is the sign of `n`.

This is an exact clone of the ifactors function in Maple and widely used internally there, so it might facilitate porting code from Maple to Agena. Examples:

```
> numtheory.ifactors(24):  # see numtheory.ifactor
[1, [[2, 3], [3, 1]]]

> numtheory.ifactors(-17):
[-1, [[17, 1]]]
```

The function is written in Agena and included in the lib/library.agn file.

See also: **numtheory.gcd**, **numtheory.lcm**, **numtheory.nthpow**, **numtheory.primes**.

numtheory.invmod (a [, m])

Computes the modular multiplicative inverse of an integer a modulo m and returns an integer x such that $a\,x = 1 \pmod{m}$. The function avoids overflow and underflow. m is 257 by default. If m is not prime or zero, then not every non-zero integer a has a modular inverse - in this case the function returns **undefined**.

See also: **%** operator, **numtheory.mulmod**, **numtheory.powmod**.

numtheory.iscoprime (m, n)

Checks if the integers m, n are relatively prime (or co-prime) and returns **true** or **false**. Two numbers are co-prime if they share no common positive factors other than 1.

See also: **ints.isprime**.

numtheory.iscube (n)

Checks if a given integer n is a perfect cube, i.e. if $\sqrt[3]{n}^3 = n$. If n is a fractional number, the function always returns **false**.

See also: **cbrt**, **cube**, **numtheory.issquare**.

numtheory.isfib (n)

Checks whether the non-negative integer n is a Fibonacci number.

See also: **numtheory.binet**, **numtheory.fib**, **numtheory.fibinv**.

numtheory.isprime (n)

Returns **true**, if integer n is a prime number, and **false** otherwise.

See also: **numtheory.congruentprime**, **numtheory.iscoprime**, **numtheory.nextprime**, **numtheory.prevprime**, **numtheory.primes**.

numtheory.issquare (n)

Checks if a given integer n is a perfect square, that is $\sqrt{n}^2 = n$. Any power of two is a perfect square, for example $n = 1, 4, 9, 16, 25, 36, 49$, etc.

If n is a fractional number, the function always returns **false**.

See also: **sqrt**, **square**, **cube**, **numtheory.iscube**, **math.ispow2**, **numtheory.issqrfree**.

numtheory.issqrfree (n)

Checks if integer n is square free, that is if it is not divisible by a perfect square. If n is a fractional number, returns **false**.

See also: **math.isqrt**, **numtheory.issquare**.

numtheory.jacobi (a, b)

Computes the Jacobi symbol J(a, b). a is an integer relatively prime to b, a positive (odd) number. Returns either -1, 0 or 1. The Jacobi symbol is a generalisation of the Legendre symbol.

See also: **gmp.jacobi**, **numtheory.kronecker**.

numtheory.kronecker (a, b)

Computes Kronecker's symbol (a | b) for any integral a, b. Returns either -1, 0 or 1.

See also: **gmp.kronecker**, **numtheory.jacobi**.

numtheory.lcm (...)

Returns the least common multiple of at least two numbers.

See also: **numtheory.factors**, **numtheory.ifactor**, **numtheory.gcd**, **numtheory.primes**.

numtheory.mulmod (a, b, n)

Performs modular multiplication: a*b % n = ((a % n)*(b % n)) % n, avoiding overflow and underflow. The function is suited to process large a, b. If n is zero, the function returns **undefined**. If $|a^b| < 1$, the function internally calls **numtheory.invmod**.

See also: **%** operator, **numtheory.invmod**, **numtheory.powmod**.

numtheory.nextprime (n)

Returns the smallest prime greater than integer n.

See also: **numtheory.congruentprime**, **numtheory.prevprime**, **numtheory.isprime**.

numtheory.nthpow (m, n)

The function finds the largest n-th power in an integer: It returns b^n where b is the greatest integer such that b^n divides m.

The function is written in Agena and included in the lib/library.agn file.

numtheory.powmod (x, p, m)

Performs modular exponentiation and returns `x^p % m`, avoiding overflow and underflow with large `x`, `p`. If `m` is zero, the function returns **undefined**.

See also: **numtheory.invmod**, **numtheory.mulmod**.

numtheory.prevprime (n)

Returns the largest prime less than integer `n`.

See also: **numtheory.congruentprime**, **numtheory.nextprime**, **numtheory.isprime**, **numtheory.primes**.

numtheory.primes (n)

Performs prime factorisation of a positive integer `n`.

See also: **numtheory.congruentprime**, **numtheory.factors**, **numtheory.ifactor**, **numtheory.isprime**, **numtheory.nextprime**, **numtheory.prevprime**.

11.19 kiss - Fast Fourier Fransform (FFT)

This package features an easy-to-use function, **kiss.fft**, to perform Fast Fourier Transformation. You have to import this package with `import kiss`.

Example usage:

```
> import kiss

> signal := []
> s := kiss.nextsize(2*2048 + 1)
> frequency := 1024
> length := s/frequency
```

Populate a list with random real numbers in complex format:

```
> procedure populate(list, s) is
>    for i to s do
>         list[i] := (sin(2*i/frequency*Pi2) + sin(10*i/frequency*Pi2))!0
>    end
> end
```

Display the fourier spectrum:

```
> procedure display(spectrum) is
>    for i to size(spectrum)/2 do
>       print(strings.format("%.1f Hz\t%1.3f",(i - 1)/length,
>             (abs spectrum[i])))
>    end
> end
```

Create a signal with two sine waves:

```
> populate(signal, s)
```

Carry out fast fourier transformation and store result in "spec".

```
> spec := kiss.fft(signal, false);
```

After re-transformation, we need to divide by the data size s/2.

```
> map(<: x, s -> 2*x/s :>, spec, s, inplace = true);
```

Display the fourier spectrum of the audio signal:

```
> display(spec)
0.0 Hz  0.078
0.2 Hz  0.079
0.5 Hz  0.082
0.7 Hz  0.088
0.9 Hz  0.099
1.2 Hz  0.118
1.4 Hz  0.154
1.7 Hz  0.241
...
```

kiss.fft (input, inverse)

Performs Fast Fourier Transform of the given `input`, a table of points that shall be transformed. The table can include either numbers or complex numbers.

`inverse` controls whether a transformation (**false**) or an inverse transformation (**true**) will be carried out.

The return is a table of complex numbers with the same size as the input and contains the Fourier transformation of the input.

kiss.nextsize (n)

Returns the next possible size for data input. `n` must be a positive integer.

11.20 ival - Interval Arithmetic

This package implements interval arithmetic.

You have to import this package with `import ival` before usage.

Interval arithmetic handles uncertainties and ensures reliable results in numerical computations, by estimating rounding errors, measurement inaccuracies or imprecise data input.

The package provides a port to the `fi_lib` fast interval library (Version 1.2), developed by the Universities of Karlsruhe and Wuppertal.

Example: Let us define an interval [1 .. 2] with the left border 1 and the right border 2.

Do some arithmetic:

```
> import ival

> a := ival.new(1, 2)

> 2*a:
[2 .. 4]

> sin(a):
[0.8414709848079 .. 1]
```

Get the left and right borders of a:

```
> left(a), right(a):
1       2

> ival.extremes(a):
1       2
```

Merge and intersect two intervals:

```
> b := ival.new(1.5, 3):
[1.5 .. 3]

> a union b:
[1 .. 3]

> a intersect b:
[1.5 .. 2]
```

Compare intervals using standard relational operators:

```
> b := ival.new(3, 4)

> a = b, a < b:
false   true
```

Creation, Properties, Checks and Conversion:

`ival.checkival (··· [, true])`

Checks whether all its argument `···` (one or more) are intervals and issues an error if at least one argument is not an interval. If all arguments `···` are intervals, returns nothing.

If the optional last argument **true** is given, then the function additionally also checks whether the respective left border in each interval is less than or equal its right border. Default is **false**.

`ival.contained (a, b)`

Checks whether interval `a` is part of interval `b` and returns **true** or **false**.

`ival.disjoint (a, b)`

Checks whether the two intervals `a`, `b` do not overlap and returns **true** or **false**.

`ival.isival (··· [, true])`

Tests whether all its arguments `···` (one or more) are intervals and returns **true** on success and **false** otherwise.

If the optional last argument **true** is given, then the function additionally also checks whether the respective left border in each interval is less than or equal its right border. Default is **false**.

`ival.new (a, b)`

Creates a new interval [`a` .. `b`] with `a` the left and `b` the right border. The result is a userdata. `a` should be less or equal `b`.

`ival.inf (x)`

Returns the left border of interval `x` and returns a number. Same as **left**(`x`).

`ival.sup (x)`

Returns the right border of interval `x` and returns a number. Same as **right**(`x`).

`ival.extremes (x)`

Returns the left and right border of interval `x` and returns two numbers, in this order.

`ival.diam (x)`

Returns the length of the interval and returns a number. Same as **size**(`x`).

ival.mean (x)

Returns the midpoint of interval `x`, that is with `x` = [a .. b] returns the number (b - a)/2.

ival.member (t, x)

Check whether number `t` is part of interval `x` and returns **true** or **false**. Same as the expression `t` **in** `x`.

ival.tostring (x [, format])

Converts the interval `x` into a string. You can pass a string `format` that includes the `f` specifier to control the output, for example:

```
> ival.tostring(b):
[-3 .. 4]

> ival.tostring(b, '[%.19f, %.19f]'):
[-3.0000000000000000000, 4.0000000000000000000]
```

Arithmetic and Relational Operations:

The package provides the following metamethods:

Metamethod	Functionality
`'__add'(x, y)`	Addition of two intervals, the + operator
`'__sub'(x, y)`	Subtraction of two intervals, the - operator
`'__mul'(x, y)`	Multiplication of interval x by number y, the * operator
`'__div'(x, y)`	Division of interval x by number y, the / operator
`'__recip'(x)`	Reciprocal 1/x for intervals
`'__pow'(x, y)` `'__ipow'(x, y)`	Exponentiation of interval x to the power of number y, the ^ & ** operators; y may be integral or fractional, zero, positive or negative
`'__unm'(x)`	unary - operator, negation of interval x
`'__abs'(x)`	Transformation of interval x (try it out yourself)
`'__square'(x)`	**square** operator, interval x raised to the power of 2
`'__cube'(x)`	**cube** operator, interval x raised to the power of 3
`'__sqrt'(x)`	**sqrt** operator, square root of interval x
`'__ln'(x)`	**ln** operator, logarithm of interval x
`'__exp'(x)`	**exp** operator, **E** raised to the power of interval x
`'__sin'(x)`	**sin** operator, returns an interval with the minimum and maximum sine values as its borders
`'__cos'(x)`	**cos** operator, returns an interval with the minimum and maximum cosine values as its borders
`'__tan'(x)`	**tan** operator, tangent
`'__sinh'(x)`	**sinh** operator, hyperbolic sine
`'__cosh'(x)`	**cosh** operator, returns the minimum and maximum hyperbolic cosine values as its borders

Metamethod	Functionality
`'__tanh'(x)`	**tanh** operator, hyperbolic tangent
`'__arcsin'(x)`	**arcsin** operator, inverse sine
`'__arccos'(x)`	**arccos** operator, inverse cosine
`'__arctan'(x)`	**arctan** operator, inverse tangent
`'__in'(x, y)`	**in** operator checks whether number x is part of y
`'__notin'(x, y)`	**notin** operator, opposite of **in**
`'__intersect'(x, y)`	**intersect** operator, intersection of two intervals x, y
`'__union'(x, y)`	**union** operator, union of two intervals x, y
`'__left'(x)`	**left** operator, left border of interval x
`'__right'(x)`	**right** operator, right border of interval x
`'__size'(x)`	**size** operator, length of interval x
`'__eq'(x, y)`	= equality operator, and thus implicitly <>, too
`'__eeq'(x, y)`	== exact equality operator, actually same as =
`'__aeq'(x, y)`	~= approximate equality operator, see **approx**; thus implicitly ~<>, too
`'__le'(x, y)`	<= equality operator, or > if x, y are flipped
`'__lt'(x, y)`	< equality operator, or >= if x, y are flipped
`'__tostring'(x)`	formatting for output at the console
`'__gc'`	garbage collection

The package provides the following functions:

Metamethod	Functionality
ival.xabs(x)	like the **abs** operator for intervals
ival.xarccos(x)	like the **arccos** operator for intervals
ival.xarccosh(x)	like the **arccosh** function, for intervals
ival.xarccot(x)	like the **arccot** function, for intervals
ival.xarccoth(x)	like the **arccoth** function, for intervals
ival.xadd(x, y)	like the + operator for intervals
ival.xarcsin(x)	like the **arcsin** operator for intervals
ival.xarcsinh(x)	like the **arcsinh** function, for intervals
ival.xarctan(x)	like the **arctan** operator for intervals
ival.xarctanh(x)	like the **arctanh** function, for intervals
ival.xcbrt(x)	like the **cbrt** operator for intervals
ival.xcos(x)	like the **cos** operator for intervals
ival.xcosh(x)	like the **cosh** operator for intervals
ival.xcot(x)	like the **cot** function, for intervals
ival.xcoth(x)	like the **coth** function, for intervals
ival.xcube(x)	like the **cube** function, for intervals
ival.xdiv(x, y)	like the / operator for intervals
ival.xerf(x)	like the **erf** function, for intervals
ival.xerfc(x)	like the **erfc** function, for intervals
ival.xexp(x)	like the **exp** operator for intervals
ival.xexp10(x)	like the **exp10** function, for intervals
ival.xexp2(x)	like the **exp2** function, for intervals
ival.xexpm1(x)	like the **expm1** function, for intervals

Metamethod	Functionality
ival.xhypot(x, y)	like the **hypot** function, for intervals
ival.xln(x)	like the **ln** operator for intervals
ival.xlnp1(x)	like the **lnp1** function, for intervals
ival.xlog10(x)	like the **log10** function, for intervals
ival.xmul(x, y)	like the * operator for intervals
ival.xneg(x)	like the unary minus (-) operator for intervals
ival.xpow(x, y)	like the ^ operator for intervals
ival.xpytha(x, y)	like the **pytha** function, for intervals
ival.xrecip(x)	like the **recip** operator for intervals
ival.xsin(x)	like the **sin** operator for intervals
ival.xsinh(x)	like the **sinh** operator for intervals
ival.xsquare(x)	like the **square** operator for intervals
ival.xsqrt(x)	like the **sqrt** operator for intervals
ival.xsub(x, y)	like the binary - operator for intervals
ival.xtan(x)	like the **tan** operator for intervals
ival.xtanh(x)	like the **tanh** operator for intervals
ival.aeq	like the ~= operator for intervals
ival.eeq	like the == operator for intervals
ival.eq	like the = operator for intervals
ival.le	like the <= operator for intervals
ival.lt	like the < operator for intervals
ival.naught, ival.nought	the origin, that is interval [0 .. 0], as a constant
ival.one	the point 1, that is interval [1 .. 1], as a constant

Extending the Package:

We can define new metamethods for eligable operators or change existing ones. For example, the **ival** package by default does not support the **minus** operator.

We can change this as follows: first we get a reference to the underlying **ival** metatable from the registry:

```
> mt := debug.getregistry('ival')
```

Then we insert a new metamethod into the registry's 'ival' table:

```
> mt.__minus := proc(x, y) is
>   local z;
>   ival.checkival(x, y, true);
>   if z := ival.meet(x, y) then  # we have an intersection
>      if left(x) > left(y) then
>         return ival.new(right(z), right(x))
>      else
>         return ival.new(left(x), left(z))
>      fi
>   fi;
>   return null
> end;
```

Now all newly created intervals feature this new metamethod and you can apply the **minus** operator to them.

```
> a, b := ival.new(1, 2), ival.new(1.5, 3)

> a minus b:
[1 .. 1.5]
```

11.21 maple - Aliases to Maple Functions

This package includes aliases to Maple functions, see package file lib/maple.agn.

You have to import this package with `import maple`.

Maple aliases point to Agena functions or operators. They facilitate porting Maple code to Agena a bit. Many Agena functions have been modelled after their respective pendants and mostly bear their names, so they are not listed here.

Here they are:

Maple	Agena	Functionality
BellB	combinat.bell	Bell polynomial
BesselJ	besselj	Bessel function of the first kind
BesselY	bessely	Bessel function of the second kind
csgn	sign	sign
dilog	calc.dilog	dilogarithm (Spence's) function
Ei	calc.Ei	exponential integral
evalf, evalhf	identity	wrappers, return of the arguments
factorial	fact	factorial
GAMMA	gamma	gamma function
Heaviside	heaviside	Heaviside function
ifactor	numtheory.ifactor	complete integer factorisation
ifactors	numtheory.ifactors	complete integer factorisation
igcd	numtheory.gcd	greatest common divisor
ilcm	numtheory.lcm	least common multiple
Im	imag	imaginary part
isprime	numtheory.isprime	prime check
LambertW	calc.lambertw	Lambert W function
Li	calc.Ei(ln(x))	logarithmic integral function
lnGAMMA	lngamma	logarirhmic gamma function
pochhammer	math.pochhammer	Pochhammer function
polylog	calc.polylog	polylogarithm
power	^	exponentiation
Psi	psi	Psi (digamma) function
Re	real	real part
RETURN	identity	Maple's explicit return statement
Shi	calc.Shi	hyperbolic sine integral
Si	calc.Si	sine integral
Ssi	calc.Ssi	shifted sine integral
surd	root	non-pricipal root
trunc	math.trunc	rounding towards zero

11.22 unimath - Common Functions for Floating-Point Data

The package provides functions that work with either ordinary Agena numbers, MAPM, MPFR, double-to-double or long 80-bit precision floating-point numbers.

Initialise the package with `import unimath`.

You should have already initialised the **mapm** or **mpfr** package, if needed, with the **import** statement, too.

The functions are:

unimath.approx (x, y [, eps])

Checks whether two numbers `x`, `y` are approximately equal taking the magnitude of the input into account, and returns **true** or **false**.

The function works like **approx**. By default, eps is **Eps**. `x` and `y` must be of the same type.

unimath.det (A)

Computes the determinant of the square matrix A. The return is of the same type as in the input.

See also: **linalg.det**, **unimath.linsolve**.

unimath.fma (x, y, z)

Calls the underlying fused multiply-add function computing `x*y + z` with extended accuracy for the Agena numbers, long doubles, double-to-double numbers, MAPM or MPFR values `x`, `y`, `z`, if available, and returns the result which is of the same type as in the input. `x`, `y`, `z` should be of the same type, or an error will be issued.

unimath.linsolve (A, x)

Solves a linear system of equations using Gaussian elimination with partial pivoting, supporting standard Agena numbers, double-to-double numbers, long doubles, numeric MAPM and MPFR values. `A` is a square matrix and `x` a right-hand side vector. The return is the solution vector with the components of the same type as in the input.

See also: **linalg.linsolve**, **unimath.det**.

unimath.max (x, y)

Returns the maximum of `x`, `y` which must be of the same type.

unimath.min (x, y)

Returns the minimum of `x`, `y` which must be of the same type.

unimath.sections (f, a, b [, ...])

Returns all subintervals where the univariate function `f` has a change in sign in interval [`a`, `b`]. The function works like **calc.sections**, see there for further information.

unimath.xpdiff (f, x [, deriv=n] [, eps=v] [, delta=w] [, iters=i])

Computes the n'th derivative of univariate function `f` at point `x`. `n` may be a value from 0 to 5, with 1 the default, for the first derivative. If `n` is zero, the function just returns `f`(`x`).

The `eps` and `delta` options control the accuracy of the result. Their right-hand side values `v`, `w` must be positive.

The `iters` option controls the maximum number of iterations taken to compute the result. Its right-hand side value `i` must be a positive integer and defaults to 30. You can change this default by altering the value stored to **unimath.iters.xpdiff**.

The function works like **calc.xpdiff**, check there for further information.

The type of return is that of argument `x`.

Example:

```
> import mpf, unimath

> unimath.xpdiff(<: x -> sin(x) :>, mpfr.new(1), eps=1e-10):
mpf(.54030230586812691840516648418457240913 74)

> unimath.xpdiff(<: x -> sin(x) :>, mpfr.new(1)):
mpf(.54030230586813971740093660744295560477 58)

> cos(mpfr.new(1)):
mpf(.54030230586813971740093660744297660373 34)
```

unimath.zeroin (f, a, b [, eps=v] [, iters=w])

Determines the root of the univariate function `f` over the interval [`a`, `b`]. The accuracy can be controlled by the `eps` option, which defaults to `v` = **Eps**.

`a`, `b` must be of the same type.

The `iters` option controls the maximum number of iterations taken to compute the result. Its right-hand side value must be a positive integer and defaults to `w` = 25. You can change this default by altering the value stored to **unimath.iters.zeroin**.

The function works like **calc.zeroin**. The type of return is that of arguments `a`, `b`.

Examples:

```
> import mapm, unimath

> unimath.zeroin(<: x -> sin(x) :>, 3, 4):
3.1415926535898

> mapm.xdigits(40)

> mapm.Pi:
3.1415926535897932384626433832795028841972

> unimath.zeroin(<: x -> sin(x) :>, mapm.new(3), mapm.new(4), iters=30):
3.1415926535897932384626433832795028845877
```

unimath.zeros (f, a, b [, ...])

Determines all the roots of the univariate function `f` over the interval [`a`, `b`]. The function works like **calc.zeros**, see there for further information.

11.23 dd - Double-to-Double Arithmetic

The dd package is a high-precision numerical library designed to extend the capabilities of standard IEEE 754 64-bit floating-point arithmetic. By representing numbers as an unevaluated sum of two doubles (a = left a + right a), the package effectively doubles the significand bits from 53 to 106 bits.

This provides approximately 31 decimal digits of precision, filling the critical gap between standard hardware-accelerated double precision and the much slower, arbitrary-precision software libraries like MPFR or MAPM.

A Brief History of Double-Double Arithmetic:

The concept of "Double-Double" arithmetic (also known as "Quad-Double" in its four-part form) was invented by Ole Møller in 1965. It was popularized by pioneers such as Th. J. Dekker and Donald E. Knuth and is based on the fundamental algorithms:

- TwoSum: A method to compute the exact sum of two floating-point numbers.
- TwoProd: A method to compute the exact product.

In the 1990s and early 2000s, researchers like Keith Briggs and Yozo Hida (the author of the qd library) formalized these techniques for modern scientific computing. The primary appeal has always been portability and speed: because the library relies on standard floating-point hardware instructions rather than bit-level integer manipulation, it is remarkably fast compared to other high-precision formats.

Accuracy and Numerical Stability:

The DD Package is engineered for numerical honesty. Standard double precision often fails in "stiff" problems—such as high-order derivatives, solving near-singular matrices, or evaluating functions near their limits.

Setup:

A double-to-double value consists of a high part and a lower part:

High Part: This represents the most significant bits of the number. In a normalized dd_pair, a.hi is equal to the value of the number rounded to a standard 64-bit double. You can query the high part with the **left** operator.

Low Part: This represents the residual or the tail. a.lo contains the precision that was "lost" or rounded away when the number was stored in the high part. You can query the low part with the **right** operator.

dd.new creates a new double-to-double value which is represented by a userdata object of the user-defined type 'dd'. When given a double-to-double value, it

simply returns it. Examples:

```
> d := dd.new(2.5)

> left d, right d:
2.5     0
```

sets the high part to 2.5 and the low part implicitly to zero.

```
> d := dd.new(2.5, 1e-16)

> left d, right d:
2.5     1e-016
```

sets the high part to 2.5 and the low part to 1e-16.

To finally convert a dd value back to a standard Agena number, use **dd.tonumber** which returns the sum of the high and the low part.

Compare the accuracy of operations:

Double-to-double arithmetic	**Standard 64-bit arithmetic**

`> x := dd.naught;`
`> hundredth := dd.one/dd.hundred;`
`> to 100k do`
`>  x +:= hundredth`
`> od;`
`> x:`
`1000 (-1.4941214863799497e-026)`
	`> x := 0;`
	`> hundredth := 1/100;`
	`> to 100k do`
	`>  x +:= hundredth`
	`> od;`
	`> x:`
	`999.99999999924`

See also the MAPM and MPFR packages, see Chapters 11.3 and 11.5.

Following are all available operators, mathematical and general functions.

Operator	Functionality	Operator	Functionality
+	addition	sin	sine
-	subtraction	cos	cosine
*	multiplication	tan	tangent
/	division	sinh	hyperbolic sine
^	integer exponentiation	cosh	hyperbolic cosine
(-)	unary minus	tanh	hyperbolic tangent
abs	absolute value	arcsin	inverse sine
sign	sign	arccos	inverse cosine
sqrt	square root	arctan	inverse tangent
ln	natural logarithm	arcsec	inverse secant
lngamma	logarithmic gamma function	recip	reciprocal 1/x
square	square, x^2	left	high part
cube	cube, x^3	right	low part
antilog2	2^x	exp	exponential function to the base E = 2.71828..
antilog10	10^x	a |- b	dd.new(|left a - left b|, |right a - right b|)

Operator	Functionality
nonzero	test for a non-zero
zero	test for zero
even	test for an even integer
odd	test for an odd integer
integral	test for an integral number
fractional	test for a fractional number
nan	test for undefined
finite	test for finite numbers
infinite	test for infinity
=	equality check
<>	inequality check
~= ~<>	approximate (in)equality checks
<	less-than relation
<=	less-than-or-equal relation
>	greater-than relation
>=	greater-than or equal relation

Following are all available operators, mathematical and general functions.

Function	Arguments	Functionality
dd.approx	3 dd values	checks for approximate equality, see **approx** for further information
dd.arccosh	1 dd value	inverse hyperbolic cosine
dd.arccot	1 dd value	inverse cotangent
dd.arccoth	1 dd value	inverse hyperbolic cotangent
dd.arccsc	1 dd value	inverse cosecant
dd.arccsch	1 dd value	inverse hyperbolic cosecant
dd.arcsech	1 dd value	inverse hyperbolic secant
dd.arcsinh	1 dd value	inverse hyperbolic sine
dd.arctanh	1 dd value	inverse hyperbolic tangent
dd.arctan2	2 dd values	inverse tangent
dd.cathet	2 dd values	cathetus, computes $\sqrt{x^2-y^2}$
dd.cbrt	1 dd value	cubic root
dd.ceil	1 dd value	rounds up to the next higher or equal representable integer
dd.checkdd	1 or more dd values	checks whether all arguments are double-to-double numbers and issues an error otherwise
dd.copysign	2 dd values	like **math.copysign**
dd.cot	1 dd value	cotangent
dd.coth	1 dd value	hyperbolic cotangent
dd.csc	1 dd value	cosecant
dd.csch	1 dd value	hyperbolic cosecant
dd.erf	1 dd value	error function
dd.erfc	1 dd value	complementary error function
dd.floor	1 dd value	rounds to the next lower or equal representable integer
dd.fma	3 dd values	fused multiply-addition
dd.fmod	2 dd values	see: **fmod**; the result is tied to even
dd.frexp	1 dd value x	returns mantissa m and exponent e such that x = m*2^e
dd.get	1 dd value	returns the high part and the low part
dd.hypot	2 dd values	hypotenuse
dd.isdd	1 or more dd values	checks whether all arguments are double-to-double numbers and returns **true** or **false**
dd.log2	1 dd value x	computes **log2**(x)
dd.log10	1 dd value x	computes **log10**(x)
dd.modf	2 dd values	see: **modf**; the result is tied to even
dd.renorm[1]	1 dd value x	ensures that the high and low parts in x are as "separated" as possible; instead of a dd value x you can pass the high and low parts as two standard Agena numbers

Function	Arguments	Functionality
dd.renorm[2]	1 dd value x plus **true**	sums up the high and low part in x and sets the sum as the high component, with the low part set to zero. Returns a new dd value and does *not* change x in-place.
dd.root	1 dd value, 1 integer	n-th root $x^{1/n}$
dd.round	1 dd value	rounds to nearest representable integer, rounding halfway cases away from zero
dd.sec	1 dd value	secant
dd.sech	1 dd value	hyperbolic secant
dd.signbit	1 dd value	checks the sign bit and returns **true** (value is negative) or **false**
dd.trunc	1 dd value	rounds to the next representable integer toward zero
dd.tonumber	1 dd value x	returns **left** x + **right** x as an Agena number
dd.tostring	1 dd value	experimental conversion to a string

The constants are:

Constant	Value
dd.Ln2	dd constant ln(2)
dd.Pi	dd constant Pi
dd.E	dd constant $e^1 = 2.7182818284...$
dd.Euler	dd constant $\gamma = 0.57721566490...$
dd.naught dd.nought	0
dd.one	1
dd.two	2
dd.three	3
dd.four	4
dd.five	5
dd.six	6
dd.seven	7
dd.eight	8
dd.nine	9
dd.ten	10
dd.eleven	11
dd.twelve	12
dd.sixteen	16
dd.hundred	100
dd.thousand	one thousand
dd.half	0.5
dd.quarter	0.25
dd.tenth	0.1
dd.fifth	0.2
dd.hundredth	0.01

Constant	Value
dd.thousandth	0.001
dd.threequarter	0.75
dd.third	1/3
dd.sixth	1/6
dd.eighth	0.125
dd.twelfth	1/12
dd.sixteenth	0.0625
dd.infinity	infinity
dd.undefined	undefined

Chapter Twelve

Input & Output

12 Input & Output

12.1 io - Input and Output Facilities

The I/O library provides two ways for file manipulation.

Summary of functions:

Opening and closing files:

io.open, **io.close**.

Reading data:

io.input, **io.lines**, **io.read**, **io.readfile**, **io.readfrom**, **io.readlines**.

Writing data:

io.output, **io.write**, **io.writefile**, **io.writelines**.

File positions:

io.eof, **io.filepos**, **io.move**, **io.seek**, **io.skiplines**.

File locking:

io.lock, **io.unlock**.

File buffering:

io.setvbuf, **io.sync**

Interaction with applications:

io.pcall, **io.popen**, **io.close**

Keyboard interaction:

io.anykey, **io.getkey**, **io.kbdgetstatus**, **io.keystroke**

Windows clipboard interaction

io.getclip, **io.putclip**.

Miscellaneous:

io.clearerror, **io.ferror**, **io.fileno**, **io.filesize**, **io.isatty**, **io.isfdesc**, **io.mkstemp**, **io.nlines**, **io.isopen**, **io.tmpfile**, **io.truncate**.

Usage:

1. The first one uses *file handles*; that is, there are operations to set a default input file and a default output file, and all input/output operations are over these default files. File handles are values of type userdata and are used as in the following example:

   Open a file and store the file handle to the name `fh`:

   ```
   > fh := io.open('d:/agena/src/change.log'):
   file(7803A6F0)
   ```

   Read 10 characters:

   ```
   > io.read(fh, 10):
   Change Log
   ```

   Close the file:

   ```
   > io.close(fh):
   true
   ```

   In the following descriptions of the **io** functions, file handles are indicated with the argument `filehandle`.

   The table `io` provides three predefined file handles with their usual meanings from C: `io.stdin`, `io.stdout`, and `io.stderr`.

2. The second style uses file names passed as strings like `'d:/agena/lib/library.agn'`. File names are always indicated with the argument `filename` in this chapter.

Unless otherwise stated, all I/O functions return **null** on failure (plus an error message as a second result) and some value different from **null** on success.

The following **io** functions can be called OOP-style: **close**, **read**, **lines**, **write**, **writeline**, **sync**, **filepos**, **seek**, **rewind**, **toend**, **lock**, **unlock**, **eof**, **skiplines** and **setvbuf**. Example:

```
> f := io.open('test.txt', 'r+')

> f@@write('ag', 'ena')

> f@@sync()
```

```
> f@@rewind()

> f@@read():
agena

> f@@close():
true
```

The Functions are:

io.anykey ()

Checks whether a key is being pressed and returns either **true** or **false**. A common usage is as follows:

```
> while io.anykey() = false do od; # wait until a key has been pressed
```

On Mac OS X, the function may sometimes echo the key being pressed.

See also: **io.getkey**, **io.keystroke**, **io.read**.

io.clearerror (filehandle)

Clears the end-of-file and error indicators for the file denoted by `filehandle`. The function returns nothing.

See also: **io.eof**, **io.ferror**.

io.close ([filehandle, ···])

Closes one or more files. Note that files are automatically closed when their handles are garbage collected, but that takes an unpredictable amount of time to happen.

Without a `filehandle`, closes the default output file.

The function also deletes the file handles and the corresponding filenames from the **io.openfiles** table if the files could be properly closed.

The function returns **true** on success and **false** otherwise. With pipes, also returns the exit code (errorlevel) of the application run.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.open**, **io.pclose**, **io.popen**.

io.eof (filehandle)

Checks whether the end of the file denoted by `filehandle` has been reached and returns **true** or **false**. The function returns **fail** if an error occurred during the check.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.clearerror**, **io.ferror**.

io.ferror (filehandle)

Checks the error indicator for the file denoted by `filehandle` and returns **true** if set or **false** if not set. The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.clearerror**, **io.eof**.

io.fileno (filehandle)

Returns the file descriptor, an integer, associated with the stream referenced by `filehandle`, which is of type **userdata/file**. It is useful for informative purposes, only. The return cannot be used as a substitute to `filehandle` in calls to **io** functions, and which require a handle of type **userdata/file**.

The function issues an error if `filehandle` is not of type **userdata/file** or if does not reference an open file. The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.isfdesc**.

io.filepos (filehandle [, offset])

When given only file handle `filehandle`, returns the current position in the file denoted, relative to the beginning of the file as a non-negative integer.

If `offset`, a non-negative number is given, then the function *sets* the file position relative to the beginning of the file.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.seek**.

io.filesize (filehandle)

Returns the size of an open file denoted by its file handle `filehandle` and returns the number of bytes as a non-negative integer.

The function is also available as an OOP method, see Chapter 6.24 about its use.

io.getclip ([options])

Returns the contents of the Windows clipboard as a string. If the clipboard could not be accessed, it returns **fail** plus an error string. It also returns fail and an error string, if the clipboard contains a binary object.

You can pass the following options:

- `retries` = *integer*: maximum number of attempts to successfully get access to the clipboard, default is 5 retries;
- `wait` = *integer*: wait time in milliseconds between retries, default is 25 milliseconds.

Example:

```
> io.getclip(wait=100, retries = 1):
true
```

The function is available in the Windows edition only.

See also: **io.putclip**.

io.getkey ([anything])

If no argument is given, waits until a key is pressed and returns its ASCII number. If any argument is passed, the function waits until a key is pressed, but returns nothing.

See also: **io.anykey**, **io.keystroke**, **io.read**.

io.infile (filename, pattern)

io.infile (filehandle, pattern)

Checks whether the file given by the name `filename` or the file denoted by its descriptor `filehandle` includes a `pattern` of type string, and returns **true** or **false**. The function supports pattern matching.

See also: **io.readfile**, **utils.findfiles**.

io.input (filehandle)

io.input (filename)

io.input ()

When called with a file name, it opens the named file (in text mode), and sets its handle as the default input file. When called with a file handle, it simply sets this file handle as the default input file. When called without parameters, it returns the current default input file.

In case of errors this function raises the error, instead of returning an error code.

io.isatty (filehandle)

Checks whether the file denoted by its handle is associated with an open terminal device and returns **true** or **false**. The handle may also be the **io.stdin**, **io.stdout**, **io.stderr**, usually used for console I/O.

The function is also available as an OOP method, see Chapter 6.24 about its use.

io.isfdesc (filehandle)

Checks whether `filehandle` is a valid file handle. Returns **true** if `filehandle` is an open file handle, or **false** if `filehandle` is not a file handle.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.fileno**, **io.isopen**.

io.isopen (filehandle)

Checks whether `filehandle` references an open file. Returns **true** if `filehandle` is an open file handle, or **false** if `filehandle` is not a file handle. Thus it also returns **false** if `filehandle` is not of type **userdata**/**file**. Contrary to **io.isfdesc**, it also detects invalid file positions caused by files too large or if the stream referenced by `filehandle` does not support file positioning.

Please note that the function cannot detect whether a file has been opened by another application.

The function is five times slower than **io.fdesc**. It is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.fileno**, **io.isfdesc**.

io.kbdgetstatus ()

OS/2 only: Get status information about the keyboard. The function returns a table with the contents of the KBDINFO structure after the call to the C API function KbdGetStatus. See http://www.edm2.com/index.php/KbdSetStatus_(FAPI) for the meaning of the results.

io.keystroke (c)

Windows only: emulates a keystroke for the given ASCII value (an integer) `c` and dumps the character representing `c` to the currently active window. For security, newlines, carriage returns, and CTRL-Z's will not be accepted as input.

See also: **io.anykey**, **io.getkey**, **io.read**.

io.lines (filename)

io.lines (filehandle)

io.lines ()

io.lines (file, [i_1 [, i_2, ···]] [, options])

io.lines (file, [o] [, options])

In the first form, the function opens the given file denoted by string `filename` in read mode and returns an iterator function that, each time it is called, returns a new line from the file.
In the second form, the function opens the given file in read mode and returns an iterator function that, each time it is called, returns a new line from the file.

Therefore, the construction

```
for keys line in io.lines(f) do body od
```

will iterate over all lines of the file denoted by `f`, where `f` is either a file name or file handle. When the iterator function detects the end of file, it returns **null** (to finish the loop) and automatically closes the file if a filename is given. In case of a file handle, the file is not closed.

The call `io.lines()` (without a file name) iterates over the lines of the default input file. In this case it does not close the file when the loop ends.

In the fourth and fifth form, the iterator generated by **io.lines** does not return a string but extracts the given fields from the line just read, where the field positions i_1, i_2, etc. are non-zero integers. The field positions may be negative, denoting fields counted from the right end of the line. If no position is given, then all the fields will be returned. In the fifth form, the field positions are given in the sequence `o`. You can pass one or more of the following options:

- `delim`=*string* where *string* denotes the non-empty string separating the fields, the default is a semicolon,
- `unwrap`=*string* where *string* denotes a non-empty string - by default there is no unwrapping; if a field is enclosed by one of the characters in *string* then it is removed from the start and end of the respective field,
- `convert`=*boolean*: if *boolean* is **true** then the function tries to convert the field into a number or complex number. In the latter case, the value must be of the form "a + I*b" with or without white spaces in between; default is **false**,
- `skipfaulty`=*boolean*: when set to **true**, lines of size zero and all lines with incorrect field numbers are skipped and the function will not return **fail**. Default is: **false**.
- `header`=*boolean*: when given the very first line in a file will be skipped and the function will not return **fail**, default is **false**;
- `ignore`=*i*: applies a function *i* on every line and if *i* evaluates to **true**, the line will be skipped and **io.lines** will not return **fail**.

Alternatively, *i* can be a string. In this case, all lines starting with *i* will be skipped. *i* may include patterns, see Chapter 9.1.3 Patterns.

The return of the iterator function on success is a sequence of the fields (strings, optionally numbers or complex numbers).

Example: Consider a CSV file with US ZIP codes which is set up like this:

```
"Zip";"City";"State Id";"State";"Parish/County/Borough";"Latitude";"Longitude"
"00601";"Adjuntas";"PR";"Puerto Rico";"Adjuntas";"18.1788";"-66.7516"
[further lines]
"70001";"Metairie";"LA";"Louisiana";"Jefferson Parish";"29.987138";"-90.169513"
"70002";"Metairie";"LA";"Louisiana";"Jefferson Parish";"29.987138";"-90.169513"
[further lines]
```

To skip the file header, remove wrapping double quotes and select certain fields for entire Louisiana only, issue:

```
> f := io.lines('uszips.csv', 1, 2, 5,
>    header = true, unwrap = '"',
>    ignore = <: x -> '"LA"' notin x :>);

> f():
seq(70001, Metairie, Jefferson Parish)

> f():
seq(70002, Metairie, Jefferson Parish)
```

To skip empty lines or lines with white spaces only, you can define:

```
> f := io.lines('faulty.txt', skipfaulty = true,
>      ignore = <: x -> strings.isspace(x) :>)
```

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.readlines**.

io.lock (filehandle)

io.lock (filehandle, size)

The function locks the file given by its handle `filehandle` so that it cannot be read or overwritten by other applications.

In the first form, the entire file is locked in UNIX-based systems. In Windows, only 2^{63} bytes are locked, so you have to use the second form described below in Windows after the file has become larger than 2^{63} bytes (= 8,589,934,592 GBytes).

In the second form the function locks `size` bytes from the current file position. Locked blocks in a file may not overlap. `size` may be larger than the current file length.

The function returns **true** on a successful lock, and **false** otherwise.

Note that other applications that do not use the locking protocol may nevertheless have read and write access to the file.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.unlock**.

io.maxopenfiles ([n])

In Solaris, Linux Mac OS X and Windows, returns the maximum number of open files, minus 2 for stdin & stdout, if no argument is given - or sets the maximum number of files that are allowed to be opened simultaneously if an integer `n` is given. In the latter case, the function returns the new maximum number of files that can be opened simultaneously.

On OS/2 and DOS, returns the maximum number of open files, but you cannot change this number.

On the platforms given above, when `n` is not given, returns **undefined** in case of errors. On all other platforms, the function always returns **undefined**.

io.mkstemp (template)

The function creates a unique temporary filename from the given `template`, a string, which must always end with six (capital) `X`'s and returns it as a string. It also creates a file of the same name, but does not open it.

Example: `io.mkstemp('fileXXXXXX')`.

See also: **io.tmpfile**, **os.remove**, **os.tmpdir**, **os.tmpname**.

io.move (filehandle, n)

Moves the current file position of the open file denoted by its `filehandle` either to the left or the right.

If `n` is a positive integer, then the file position is moved `n` characters to the right, if it is a negative integer, it is moved `n` characters to the left. If `n` is zero, the position is not changed at all.

The function returns **true** on success and **false** otherwise.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.seek**.

io.nlines (filename)

io.nlines (filehandle)

The function counts the number of lines in the (text) file denoted by `filename` or `filehandle` and returns a non-negative integer.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.skiplines**.

io.open (filename [, mode])

This function opens a file, given by the string `filename`, in the mode specified in the string `mode`. It returns a new file handle of type **userdata/file**. The function does not lock the file (see **io.lock**).

The function also enters the newly opened file into the **io.openfiles** table in the following format: [filehandle ~ [filename, mode]].

In case of errors, the function quits with an error.

The `mode` string can be any of the following:

- '**r**', '**read**': read mode (the default);
- '**w**', '**write**': write mode only; if the file already exists, it is truncated to zero length;
- '**a**', '**append**': append mode;
- '**r+**', '**rw**': update mode (both reading and writing), all previous data is preserved; the initial file position is at the beginning of the file;
- '**w+**': update mode (reading and writing), all previous data is erased;
- '**a+**': append update mode (reading and appending), previous data is preserved, writing is only allowed at the end of file.

The `mode` string may also have a `'b'` at the end, which is needed in some systems to open the file in binary mode. This string is exactly what is used in the standard C function `fopen`.

See also: **io.close**, **io.lock**.

io.output ([filehandle])

Similar to **io.input** but operates over the default output file.

io.pcall (prog [, mode])

Starts programme `prog` (passed as a string) in a separated process, sends and receives data to this programme via stdout - if `mode` is `'r'`, or `mode` is not given -, or writes data to this programme if `mode` is `'w'`. After communication finishes, the connection is automatically closed.

The return is a sequence of strings containing the result sent back by the application.

The function thus is a combination of **io.popen**, **io.readlines**, and **io.pclose**, has been written in Agena, and is included in the main Agena library (lib/library.agn).

This function is system dependent and is not available for all platforms.

See also: **remove**, **select**, **os.execute**.

io.pclose (pipe)

Closes the given pipe created with **io.popen** and returns **true** on success and **false** otherwise plus the errorlevel returned by the command called with **io.popen**. `pipe` is the file handle returned by **io.popen**. To close a pipe, you can also use **io.close**.

io.popen ([prog [, mode]])

Starts programme `prog` in a separated process and returns a file handle that you can use to read data that is sent from this programme (if `mode` is `'r'`, the default) via stdout, or to write data to this programme (if `mode` is `'w'`). Thus, you can create pipes with this function.

Use **io.close** or **io.pclose** to close the connection.

The following example shows how to receive the output of the UNIX `ls` command:

```
> p := io.popen('ls -l', 'r'):
file(779509B8)

> for keys i in io.lines(p) do print(i) od;
total 1917
drwxrwxrwx   1 user     group           0 Oct 12 17:00 OS2
-rw-rw-rw-   1 user     group       24481 Oct 13 18:23 aauxlib.c
-rw-rw-rw-   1 user     group        6205 Aug 10 02:26 aauxlib.h
-rw-rw-rw-   1 user     group       16067 Oct 12 23:42 aauxlib.o

> io.close(p):
true   0
```

This function is system dependent and is not available for all platforms.

See also: **os.execute**, **io.close**, **io.pcall**.

io.putclip (str [, options])

Copies the string `str` to the Windows clipboard. If the clipboard could not be accessed, it returns **fail** plus an error string. It only returns fail, if something else went wrong, and **true** on success.

You can pass the following options:

- `retries` = *integer*: maximum number of attempts to successfully get access to the clipboard, default is 5 retries;
- `wait` = *integer*: wait time in milliseconds between retries, default is 25 milliseconds.

Example:

```
> io.putclip('agena', wait=50, retries = 5):
true
```

The function is available in the Windows edition only.

See also: **io.getclip**.

io.read (filehandle [, format])

io.read ()

In the first form, reads the file with the given `filehandle`, according to the given formats, which specify what to read. For each format, the function returns a string (or a number) with the characters read, or **null** if it cannot read data with the specified format. When called without formats, it uses a default format that reads the entire next line (see below).

The available formats are:

- `'*n'`: reads a number; this is the only format that returns a number instead of a string. Hexadecimal numbers and numbers in scientific E notation are accepted, too. It also processes floats that include the decimal point separator of the current locale that may be different from a dot.
- `'*a'`: reads the whole file, starting at the current position. On end of file, it returns the empty string[3].
- `'*l'`: reads the next line (skipping the end of line), returning **null** on end of file. This is the default format.
- `'*L'`: reads the next line keeping the end-of-line character.
- **number**: reads a string up to this number of characters, returning **null** on end of file. If **number** is zero, it reads nothing and returns an empty string, or **null** on end of file.

[3] See also **io.readfile** to read a file entirely.

In the second form, the function reads from the default input stream (usually the keyboard) and returns a string or number.

Note that you can read from stdin by passing the constant **io.stdin** as a file handle. Also, the file should have been opened at least in read mode before, or you might get a confusing error message.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.lines**, **io.readfile**, **io.readfrom**, **io.readlines**, **io.readto**, **io.skiplines**, **skycrane.readcsv**, **utils.readcsv**, **utils.readxml**.

io.readfile (filename [, removenlcr [, pattern [, flag]]])

io.readfile (filhandle [, removenlcr [, pattern [, flag]]])

Reads the entire file with name `filename` or the file denoted by its handle `filehandle` in binary mode and returns it as a string. Note that contrary to **io.readlines**, the function also returns carriage returns (ASCII code 13).

If a second argument `removenlcr` is the Boolean value **true**, has been passed, then the function removes all newlines and if existing all carriage returns at the end of each line. If it is **false**, no such deletions are performed.

If the optional third argument `pattern` is given, the function only returns the whole contents of a file if the string `pattern` has been found in the file. Pattern matching is supported.

If the optional fourth argument `flag` is **false**, the function returns the whole file contents file if the string `pattern` has not been found in the file.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.infile**, **io.read**, **io.readfrom**, **io.readlines**, **io.skiplines**, **io.writefile**.

io.readfrom (h, chr [, withmarker])

io.readfrom (h, f [, withmarker])

Reads the remainder of a file starting from or succeeding a specific marker character. Alternatively it reads all the characters once a validator function returns **true**.

You can either pass a filename or a file handle for first argument `h`.

If `h` is a file handle, the search starts from the current file position, if `h` is a filename, the search starts from the beginning of the file.

In the first form, `chr` denotes the marker character which should be a string of length 1. In the second form, `f` is a function returning a Boolean.

The function returns a string, either the contents of the file including or succeeding the marker to the end of the file, or an empty string if the marker is not found.

You may pass **true** for `withmarker` if the marker shall be included in the result. The default is **false**.

This function scans the file from its current position until it encounters the first occurrence of the specified char `chr` or `f` returns **true**. It either skips or consumes the marker, depending on `withmarker`, positions the file cursor immediately after it, and reads the rest of the file into memory.

With a file handle `h`, the function does not close the file when returning, with a filename `h` it does.

See also: **io.infile**, **io.read**, **io.readfile**, **io.readlines**, **io.readto**, **io.skiplines**, **io.writefile**.

io.readlines (filename [, options])

io.readlines (filehandle [, options])

Reads the entire file with name `filename` or file handle `filehandle` and returns all lines in a table.

If a string consisting of one or more characters is given as a further argument, then all lines beginning with this string are ignored. If the option **true** is passed, then diacritics in the file are properly converted to the console character set, provided you use code page 1252. You can mix the options in any order. The function automatically deletes carriage returns (ASCII code 13) if included in the file.

You can also pass a function of one variable: in this case the function is applied on all the lines being read and the function call results are inserted into the resulting table instead of the original lines. A line is only transformed if the line has not been skipped, see the string option mentioned above.
Example:

```
> # convert all lines to upper-case, but skip all lines starting with '1':
> io.readlines(filename, '1', <: x -> upper x :>):
```

An error is issued if the file could not be found.

If you use file handles, you must open the file with **io.open** before applying **io.readlines**, and close it with **io.close** thereafter.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.lines**, **io.read**, **io.readfile**, **utils.readcsv**, **utils.readxml**, **skycrane.readcsv**.

io.readto (h, chr [, withmarker])

io.readto (h, f [, withmarker])

Reads a file until a specific marker character or until a validator function returns **true**.

You can either pass a filename or a file handle for first argument `h`.

If `h` is a file handle, the insertion starts from the current file position, if `h` is a filename, the insertion starts from the beginning of the file.

In the first form, `chr` denotes the marker character which should be a string of length 1. In the second form, `f` is a function returning a Boolean.

The function returns a string, either the contents of the file until it encounters the first occurrence of the specified char `chr` or `f` returns **true**. It returns the entire file contents if the marker has not been found.

You may pass **true** for `withmarker` if the marker shall be included in the result. The default is **false**.

With a file handle `h`, the function does not close the file when returning, with a filename `h` it does.

See also: **io.infile**, **io.read**, **io.readfile**, **io.readfrom**, **io.readlines**, **io.skiplines**, **io.writefile**.

io.rewind (filehandle)

Sets the current file position of the open file denoted by its `filehandle` to the beginning of the file. It returns the current file position, the number 0, at success, and **null** plus an error string otherwise.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.move**, **io.seek**, **io.toend**.

io.seek (filehandle [, whence [, offset]])

Sets and gets the file position, measured from the beginning of the file, to the position given by `offset` plus a base specified by the string `whence`, as follows:

- **`'set'`:** base is position 0 (beginning of the file);
- **`'cur'`:** base is current position;
- **`'end'`:** base is end of file.

In case of success, io.seek returns the final file position, measured in bytes from the beginning of the file. If this function fails, it returns **null**, plus a string describing the error.

The default value for `whence` is `'cur'`, and for `offset` is 0. Therefore, the call `io.seek(file)` returns the current file position, without changing it; the call `io.seek(file, 'set')` sets the position to the beginning of the file (and returns 0); and the call `io.seek(file, 'end')` sets the position to the end of the file, and returns its size.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.move**, **io.rewind**, **io.skiplines**, **io.toend**.

io.setvbuf (filehandle, mode [, size])

Sets the buffering mode for an output file. There are three available modes:

- **'no':** no buffering; the result of any output operation appears immediately.
- **'full':** full buffering; output operation is performed only when the buffer is full or when you explicitly flush the file (see **io.sync**).
- **'line':** line buffering; output is buffered until a newline is output or there is any input from some special files (such as a terminal device).

For the last two cases, `sizes` specifies the size of the buffer, in bytes. The default is an appropriate size.

The function is also available as an OOP method, see Chapter 6.24 about its use.

io.skiplines (filehandle, n)

io.skiplines (filename, n)

The function skips the given number of lines `n` and sets the file position to the beginning of the line that follows the last line skipped.

If a filename is passed, then with each call to **io.skiplines** the search always starts at the very first line in the file. The function automatically closes the file if a filename has been passed and returns the result (see below).

If you use a file handle, then lines can be skipped multiple times, always relative to the current file position. With a file handle, **io.skiplines** does not close the file.

The second argument `n` may be any non-negative number. If `n` is 0, then the function does nothing and does not change the file position.

The function returns two values: the non-negative number of lines actually skipped and the non-negative number of characters skipped in this process, including newlines and carriage returns.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.nlines**, **io.read**, **io.readfrom**, **io.readto**, **io.seek**.

io.sync (filehandle)

io.sync ()

In the first form, saves any written data to the file denoted by `filehandle`. In the second form, the function flushes the default output.

The function is also available as an OOP method, see Chapter 6.24 about its use.

io.tmpfile ()

Returns a handle to a temporary file. The file is opened in update mode and it is automatically removed when the programme ends.

See also: **io.mkstemp**, **os.tmpdir**, **os.tmpname**.

io.toend (filehandle)

Sets the current file position of the open file denoted by its `filehandle` to the end of the file. It returns the current file position, a number indicating the size of the file, at success, and **null** plus an error string otherwise.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.move**, **io.rewind**, **io.seek**.

io.truncate (filehandle)

Truncates an open file denoted by its handle `filehandle` at the current file position. The function returns **true** on success and issues an error otherwise. You may query and change the current file position by calling **io.seek**.

Use the function with care. It is also available as an OOP method.

io.unlock (filehandle [, size])

The function unlocks the file given by its handle `filehandle` so that it can be read or overwritten by other applications again. If size is given, the function, only the given number of bytes is unlocked, starting from the current file position.

The function returns **true** on a successful unlock, and **false** otherwise.

The function is also available as an OOP method, see Chapter 6.24 about its use.

For more information, see **io.lock**.

io.write (···)

io.writeline (···)

Write the value of each of its arguments to standard output if the first argument is not a file handle, or to the file denoted by the first argument, a file handle. Except for the file handle and the `'delim'` option described below, all arguments must be strings, numbers, Booleans or **null**. To write other values, use **tostring**, **tostringx**, or **strings.format**.

io.writeline adds a new line at the end of the data written, whereas **io.write** does not.

By default, no character is inserted between neighbouring values. This may be changed by passing the option `'delim':<str>` (i.e. a pair, e.g. `'delim':'|'`) as the last argument to the functions with `<str>` being a string of any length. Remember that in the function call, a shortcut to `'delim':<str>` is `delim ~ <str>`. The delim option is not supported if the functions are called in OOP mode.

The functions return **true** on success, and **false** otherwise.

Hint: If you work in DOS-like systems, such as DOS, Windows or OS/2, and if the text to be written includes line breaks, you may wonder why the resulting file will be larger than the number of characters in the text. This is because the operating system adds a further control code, i.e. carriage return, in front of each line break. To avoid this, open the file in binary mode, e.g. `io.open(filename, 'wb')`.

Examples:

Write a string to the console. Note that in the first statement, no newline is added to the output, as opposed to the second and third statements.

```
> io.write('Gauden Dach !')
Gauden Dach !

> io.write('Gauden Dach !', '\n')
Gauden Dach !
```

```
> io.writeline('Gauden Dach !')
Gauden Dach !
```

Write strings to the console:

```
> io.writeline('Bet', 'to\'n', '16.', 'Johrhunnert', 'geef', 'dat', 'hier',
>    'baben', 'anne', 'Küst', 'nix', 'anneres', 'as', 'Platt.')
Betto'n16.JohrhunnertgeefdathierbabenanneKüstnixanneresasPlatt.
```

Use a white space as a separator:

```
> io.writeline('Bet', 'to\'n', '16.', 'Johrhunnert', 'geef', 'dat', 'hier',
>    'baben', 'anne', 'Küst', 'nix', 'anneres', 'as', 'Platt.',
>    delim=' ')

Bet to'n 16. Johrhunnert geef dat hier baben anne Küst nix anneres as
Platt.
```

Write a string to a new file called `'d:/newfile.txt'`: First we have to create the new file with **io.open** and the `'w'` (write) option.

```
> fh := io.open('d:/newfile.txt', 'w'):
file(7803A6F0)
```

Write some text to the file.

```
> io.write(fh, 'Gouden Dach !'):
true

> io.writeline(fh, '\nBet', 'to\'n', '16.', 'Johrhunnert', 'geef', 'dat',
>    'hier', 'baben', 'anne', 'Küst', 'nix', 'anneres', 'as', 'Platt.',
>    delim=' '):
true
```

Finally, the file will be closed.

```
> io.close(fh):
true
```

Note that you can also write to stdin, stdout and stderr by passing the constants **io.stdin**, **io.stdout** or **io.stderr** as a file handle.

The function is also available as an OOP method, see Chapter 6.24 about its use.

See also: **io.writefile**, **print**, **skycrane.formatline**, **skycrane.scribe**, **skycrane.tee**.

io.writefile (filename, ···)
io.writefile (filehandle, ···)

In the first form, creates a new file `filename` denoted by its first argument (a string) and writes all of the given strings or numbers starting with the second argument in binary mode to it. To write other values, use **tostring**, **tostringx** or **strings.format**. After writing all the data, the function automatically closes the new file.

In the second form, the function writes its arguments to the open file denoted by its handle `filehandle`.

By default, no character is inserted between neighbouring strings. This may be changed by passing the option 'delim':<str> (i.e. a pair, e.g. 'delim':'|') as the last argument to the function with <str> being a string of any length.

If the file `fn` already exists, it is overwritten without warning.

The function returns the total number of bytes written, and issues an error otherwise. It is around twice as fast than using a combination of **io.open**, **io.write** and **io.close**.

See also: **save**, **io.readfile**.

12.2 binio - Binary File Package

The binio package features functions to conveniently and quickly read and write data: strings, bytes and almost all number types regardless of the Endianess of the underlying platform.

Summary of functions:

Opening and closing files:

binio.open, **binio.close**, **binio.isfdesc**.

Reading data:

binio.lines, **binio.readbytes**, **binio.readchar**, **binio.readlong**, **binio.readlongdouble**, **binio.readint64**, **binio.readnumber**, **binio.readshortstring**, **binio.readstring**, **binio.readuint64**.

Writing data:

binio.writebytes, **binio.writechar**, **binio.writeline**, **binio.writeint64**, **binio.writelong**, **binio.writelongdouble**, **binio.writenumber**, **binio.writeshortstring**, **binio.writestring**, **binio.writeuint64**.

File positions:

binio.eof, **binio.fgetpos**, **binio.fsetpos**, **binio.filepos**, **binio.rewind**, **binio.seek**, **binio.toend**.

File locking:

binio.lock, **binio.unlock**.

File buffering:

binio.sync.

Miscellaneous:

binio.clearerror, **binio.ferror**, **binio.length**, **binio.truncate**.

The package always uses file handles that are positive integers greater than 2. (Note that the **io** package uses file handles of type userdata.) The positive integer will be returned by the **binio.open** function and must be used in all package functions that require a file handle.

A typical example might look like this:

Open a file and return the file handle:

```
> fh := binio.open('c:/agena/lib/library.agn'):
3
```

Determine the size of the file in bytes:

```
> binio.length(fh):
46486
```

Close the file.

```
> binio.close(fh):
true
```

binio supports metamethods. The metatable used by the package is called `BINIOFILE*`. By default, only `__gc` and `__tostring` methods are supported. Check the end of Chapter 6.19 on how to add further methods.

Most **binio** functions can be called OOP-style. Example:

```
> f := binio.open('test.bin')

> f@@writenumber(Pi, E);

> f@@rewind():
0

> f@@readnumber():
3.1415926535898

> f@@readnumber():
2.718281828459

> f@@close()
```

The **binio** functions are:

binio.clearerror (filehandle)

Clears the end-of-file and error indicators for the file denoted by `filehandle`. The function returns nothing.

See also: **binio.eof**, **binio.ferror**.

binio.close (filehandle [, filehandle2, ···])

Closes the files identified by the given file handle(s) and returns **true** if successful, and issues an error otherwise. The function also deletes the file handles and the corresponding filenames from the **binio.openfiles** table if the file could be properly closed.

See also: **binio.open**.

binio.eof (filehandle)

Checks whether the end of the file denoted by `filehandle` has been reached and returns **true** or **false**. The function returns **fail** if an error occurred during the check.

See also **binio.clearerror**, **binio.ferror**.

binio.ferror (filehandle)

Checks the error indicator for the file denoted by `filehandle` and returns **true** if set or **false** if not set.

See also: **binio.clearerror**, **binio.eof**.

binio.fgetpos (filehandle)

Obtains the file position indicator of the file denoted by `filehandle` and returns an `fpos_t' userdata. The value is only meaningful as the input to **binio.fsetpos**.

In case of an error, the function returns **null** plus the error message.

See also: **binio.filepos**, **binio.seek**.

binio.filepos (filehandle [, offset])

When given `filehandle` only, returns the current file position relative to the beginning of the file. In case of an error, it quits with this error.

If `offset`, a non-negative number is given, then the function *sets* the file position relative to the beginning of the file.

See also: **binio.fgetpos**, **binio.seek**.

binio.fsetpos (filehandle, ud)

Sets the file position indicator and the multibyte parsing state (if any) of the file denoted by `filehandle` according to the `fpos_t' userdata value `ud` obtained by **binio.fgetpos**.

Besides establishing a new parse state and position, a call to this function clears the end-of-file state, if it is set.

See also: **binio.filepos**, **binio.seek**.

binio.isfdesc (filehandle)

Checks whether `filehandle` is a valid file handle. Returns **true** if `filehandle` is an open file handle, or **false** if `filehandle` is not a file handle.

binio.length (filehandle)

The function returns the size of the file denoted by `filehandle` in bytes.

binio.lines (filehandle [, n] [, true])

Creates an iterator function that beginning from the current file position, with each call will return a new line from the file pointed to by the handle `filehandle`.

By default, the function traverses the file up to its end. If the second argument `n` is a positive integer, it will read the next `n` characters from the current file position (default is **infinity** = end of file). The function generally ignores carriage returns (ASCII code 13) and does not return newlines (ASCII code 10).

If the last argument is the Boolean value **true**, all embedded zeros (ASCII Code 0) will be replaced with white spaces, and the traversal of the file will continue. By default, zeros are not ignored, so if one is found, the traversal will stop.

The iterator function returns a string, and **null** if the end of the file has been reached. It also returns **null** if the last argument is not **true** and an embedded zero has been found in the file.

The iterator function does not close the file at the end of traversal, use **binio.close** to accomplish this.

The function cannot be called OOP-style.

binio.lock (filehandle)

binio.lock (filehandle, size)

The function locks the file given by its handle `filehandle` so that it cannot be read or overwritten by other applications.

In the first form, the entire file is locked in UNIX-based systems. In Windows, only 2^{63} bytes are locked, so you have to use the second form in Windows after the file has become larger than 2^{63} bytes (= 8,589,934,592 Gbytes).

In the second form the function locks `size` bytes from the current file position. Locked blocks in a file may not overlap. `size` may be larger than the current file length.

The function returns **true** on a successful lock, and **false** otherwise.

Note that other applications that do not use the locking protocol may nevertheless have read and write access to the file.

See also: **binio.unlock**.

binio.open (filename, 'a')
binio.open (filename [, anything else])

Opens the given file denoted by `filename` and returns a file handle (a number).

If it cannot find the file, it creates it and leaves it open for further binio operations.

In the first form, by passing the `'a'` or `'append'` option, and if the file already exists, it opens the file and sets the current file position to the end of the file so that nothing will be overwritten later on.

In the second form, if the file already exists, it opens the file and sets the current file position to the beginning of the file. In subsequent write operations, the contents of the file will thus be overwritten and the programmer has to ensure its integrity himself. (Use **binio.toend** to append to the file in this case or pass the `'a'` option.)

The file is always opened in both read and write modes.

If an optional second argument except `'a'` or `'append'` is given, the file is opened in read mode only. Thus, if the file does not yet exist, the function returns an error.

The function also enters the newly opened file into the **binio.openfiles** table. It cannot be called OOP-style.

See also: **binio.close**, **binio.lock**, **binio.unlock**, **os.exists**.

binio.readbytes (filehandle [, bytes] [, options])

By default, the function reads **environ.kernel('buffersize')** bytes from the file denoted by `filehandle` and returns them as a sequence of integers. You may change the kernel buffer size value to any other values in order to read less or more bytes.

If `bytes` is given, the function reads `bytes` bytes from the file denoted by `filehandle` and returns them as a sequence of integers.

The function increments the file position thereafter so that the next bytes in the file can be read with a new call to various **binio.read*** functions.

If the end of the file has been reached, **null** will be returned. In case of an error, it quits with the respective error.

By default, the function reads in all bytes of a file, including newlines (ASCII 10) or carriage returns (ASCII 13). You can change this by setting the `ignore` option and passing a string of all the explicit bytes that shall be skipped, e.g.:

```
> binio.readbytes(fh, ignore=" .\n");  # skip white space, dot & newline
```

Also by default, the function reads in embedded zeros and treats them as every other byte. If you pass the `eof` option and set it to **true**, then the function quits if it encounters an embedded zero in the file. The file pointer is automatically reset to the position of the embedded zero. The default is **false**, i.e. the whole file is read in.

The function is much faster when working on a larger number of bytes.

See also: **binio.readchars**, **binio.writebytes**, **bytes.tonumber**, **stack.readbytes**, **strings.tochars**.

binio.readchar (filehandle)

binio.readchar (filehandle, offset)

In the first form, the function reads a byte from the file denoted by `filehandle` from the current file position and increments the file position thereafter so that the next byte in the file can be read with a new call to **binio.read*** functions.

In the second form, at first the file position is changed by `offset` bytes (a positive or negative number or zero) relative to the current file position. After that, the byte at the new file position is read. Next, the file position is being incremented thereafter so that the next byte in the file can be read with a new function call.

If the byte is successfully read, it will be returned as a number. If the end of the file has been reached, **null** will be returned. In case of an error, the function quits.

binio.readchars (filehandle [, n] [, true])

Reads in `n` bytes from the file denoted by its handle `fh` and returns them as a string, including embedded zeros if present. By default `n` is environ.kernel('buffersize'). The function is faster with importing data than **binio.readbytes**. The number of characters actually read will also be returned if given **true** as the last argument.

See also: **bytes.tonumber**, **numarray.assign**.

binio.readindex (filehandle, k [, type [, offset]])

The function assumes that all values in the binary file pointed to be `filehandle` are of the same type and reads the `k`-th one. The integral index k starts from 1. You may not necessarily read data in sequential order. For example, you can first read a value at index 4, then 1, then 10 if this fits your needs.

By default, the function reads numbers (C doubles). You may pass the third argument `type` to determine another type. Valid types are the strings 'char' (see **binio.writechar**), 'long' (see **binio.writelong**), 'number' (the default, see **binio.writenumber**), 'shortstring' (see **binio.writeshortstring**) or 'string' (see **binio.writestring**). Longdoubles are not supported.

You may pass an optional offset from the beginning of the file as the fourth argument, which by default is 0. If given, the file position is moved the offset's + 1 byte in the file before searching for the given index and reading the value of interest. This feature supports a self-defined file header.

See also: **binio.readbytes**, **binio.readchar**, **binio.readlong**, **binio.readnumber**, **binio.readshortstring**, **binio.readstring**, **binio.writeindex**.

binio.readint64 (filehandle [, offset])

The function reads a signed 64-bit integer from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**. Note that the signed 64-bit integer to be read should have been written to the file using the **binio.writeint64** function. See the **ints** package for further information on 64-bit integer values.

In the second form, before reading the actual value, at first the file position is changed by `offset` bytes (a positive or negative number or zero) relative to the current file position.

The function is not supported on Big Endian systems.

See also: **binio.readuint64**.

binio.readlong (filehandle [, offset])

The function reads a signed C value of type `int32_t` from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**. Note that the number to be read should have been written to the file using the **binio.writelong** function.

In the second form, before reading the actual value, at first the file position is changed by `offset` bytes (a positive or negative number or zero) relative to the current file position.

See also: **binio.writelong**.

binio.readlongdouble (filehandle [, offset])

The function reads a longdouble (see **long** package) from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**. Note that the longdouble to be read should have been written to the file using the **binio.writelongnumber** function. See the **long** package for further information on 80-bit floating point values.

In the second form, before reading the actual value, at first the file position is changed by `offset` bytes (a positive or negative number or zero) relative to the current file position.

The function is not supported on Big Endian systems.

See also: **binio.writelongdouble**.

binio.readnumber (filehandle [, offset])

The function reads an Agena number from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**. Note that the number to be read should have been written to the file using the **binio.writenumber** function.

In the second form, before reading the actual value, at first the file position is changed by `offset` bytes (a positive or negative number or zero) relative to the current file position.

See also: **binio.writenumber**.

binio.readshortstring (filehandle)

The function reads a string of up to 255 characters from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**.

Note that the string to be read should have been written to the file using the **binio.writeshortstring** function, as this function also stores the length of the string in a special way to the file.

See also: **binio.writeshortstring**.

binio.readstring (filehandle)

The function reads a string of any length from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**.

Note that the string to be read should have been written to the file using the **binio.writestring** function, as this function also stores the length of the string in a special way to the file.

See also: **binio.writestring**.

binio.readuint64 (filehandle [, offset])

The function reads a unsigned 64-bit integer from the file denoted by `filehandle` from the current file position and returns it. If there is nothing to read, the function returns **null**. Note that the unsigned 64-bit integer to be read should have been written to the file using the **binio.writeuint64** function. See the **ints** package for further information on 64-bit integer values.

In the second form, before reading the actual value, at first the file position is changed by `offset` bytes (a positive or negative number or zero) relative to the current file position.

The function is not supported on Big Endian systems.

See also: **binio.readint64**.

binio.rewind (filehandle [, pos])

Sets the file position to the beginning of the file denoted by `filehandle`.

If `pos`, a non-negative integer is given, the function resets the file pointer to the position `pos` relative to the beginning of the file.

The function returns the new file position as a number in case of success, and quits with an error otherwise.

See also: **binio.toend**, **binio.seek**.

binio.seek (filehandle, position)

The function changes the file position of the file denoted by `filehandle position` bytes relative to the current position. `position` may be negative, zero, or positive.

The return is **true** if the file position could be changed successfully, or issues an error otherwise.

See also: **binio.filepos**, **binio.fsetpos**, **binio.rewind**, **binio.toend**.

binio.sync (filehandle)

Flushes all unwritten content to the file denoted by the handle `filehandle`. The function returns **true** if successful, **false** if stdin or stdout should be closed, and issues an error otherwise (e.g. if the file was not opened before or an error during flushing occurred).

binio.toend (filehandle)

Sets the file position to the end of the file denoted by `filehandle` so that data can be appended to the file without overwriting existing data. The function returns the file position as a number in case of success, and issues an error otherwise.

See also: **binio.rewind**, **binio.seek**.

binio.truncate (filehandle)

Truncates an open file denoted by its handle `filehandle` at the current file position. The function returns **true** on success and issues an error otherwise. You may query and change the current file position by calling **binio.seek**.

Use the function with care. It is also available as an OOP method.

binio.unlock (filehandle)

binio.unlock (filehandle, size)

The function unlocks the file given by its handle `filehandle` so that it can be read or overwritten by other applications again.

The function returns **true** on a successful unlock, and **false** otherwise.

For more information, see **binio.lock**.

binio.writebytes (filehandle, s)

The function writes all integers in the sequence `s` to the file denoted by `filehandle` starting at the current position. The function returns **true** in case of success and **fail** if the sequence is empty.

The integers in `s` should be integers `number` with $0 \leq$ `number` < 256, otherwise `number` % 256 will be stored to the file.

Internally, the bytes are stored as C `unsigned char`'s.

See also: **binio.readbytes**, **math.tobytes**, **strings.tobytes**.

binio.writechar (filehandle, number [, ···])

The function writes the given Agena `number`, and optionally more numbers, to the file denoted by `filehandle` at its current position.

The function returns **true** in case of success and **quits with an error** otherwise.

All `number`(s) should be integers with $0 \leq$ `number` < 256, otherwise `number` % 256 will be stored to the file.

Internally, the bytes are stored as a C `unsigned char`.

binio.writeindex (filehandle, k, type, value [, offset])

The function assumes that all values in the binary file pointed to by `filehandle` are of the same type and writes the `k`-th one. The integral index k starts from 1.

Data must not necessarily be written in consecutive order. For example, if you write a number at index 1 and immediately thereafter another number at index 1000, the function will fill the space between the two numbers with the appropriate number of zeros, so that you can easily move between index positions without having to set the current file position backward or forward or fill data manually.

The third argument `type` specifies the type to be written. Valid types are the strings 'char' (see **binio.writechar**), 'long' (see **binio.writelong**), 'number' (see **binio.writenumber**), 'shortstring' (see **binio.writeshortstring**) or 'string' (see **binio.writestring**). Longdoubles are not supported.

The fourth argument specifies the actual `value` to be written.

You may pass an optional `offset` from the beginning of the file as the fifth argument, which by default is 0. If given, the file position is moved to the `offset`'s + 1 byte before writing a value. This feature allows for a self-defined file header.

See also: **binio.writebytes**, **binio.writechar**, **binio.writelong**, **binio.writenumber**, **binio.writeshortstring**, **binio.writestring**, **binio.readindex**.

binio.writeint64 (filehandle, number [, ···])

The function writes the given signed 64-bit integer `number` (see **ints** package), and optionally more signed 64-bit integers, to the file denoted by `filehandle` starting at the current position. The function returns **true** in case of success and issues an error otherwise. The function is not supported on Big Endian systems.

See also: **binio.readint64**, **binio.writeuint64**.

binio.writelong (filehandle, number [, ···])

The function writes the given Agena `number`, and optionally more numbers, to the file denoted by `filehandle` starting at the current position. The `number`(s) should be integers with **environ.minlong** < `number` < **environ.maxlong**, otherwise the result is not defined.

The function returns **true** in case of success and quits with an error otherwise.

Internally, the numbers are stored as signed C `int32_t` in Big Endian notation. Use complement **binio.readlong** to read values written by **binio.writelong** back into Agena as the former transforms the value back into the proper Endian format used by your machine.

binio.writelongdouble (filehandle, number [, ···])

The function writes the given longdouble `number` (see **long** package), and optionally more longdoubles, to the file denoted by `filehandle` starting at the current position. The function returns **true** in case of success and issues an error otherwise. The function is not supported on Big Endian systems.

See also: **binio.readlongdouble**.

binio.writenumber (filehandle, number [, ···])

The function writes the given Agena `number`, and optionally more numbers, to the file denoted by `filehandle` starting at the current position. The function returns **true** in case of success and issues an error otherwise. The numbers are always stored in Big Endian notation.

The complement **binio.readnumber** function conducts proper conversion to Little Endian if Agena runs on a Little Endian machine.

binio.writeshortstring (filehandle, string [, ···])

The function writes the given `string`, and optionally more strings, to the file denoted by `filehandle` starting at the current position. The strings can be of length 0 to 255.

The function returns **true** in case of success and issues an error otherwise. Internally, **writeshortstring** at first writes the length of the respective string as a C unsigned char and after this it stores the string without a trailing null character to the file without unnecessarily wasting disk space. If you call **binio.readstring** later, Agena very efficiently returns the string.

See also: **binio.readshortstring**.

binio.writestring (filehandle, string [, ···])

The function writes the given `string`, and optionally more strings, to the file denoted by `filehandle` starting at the current position.

The function returns **true** in case of success and quits with an error otherwise. Internally, **writestring** first writes the length of the respective string as a C long int and then the string without a null character to the file. This information is then read by the **binio.readstring** function to efficiently return the string.

See also: **binio.readstring**.

binio.writeuint64 (filehandle, number [, ···])

The function writes the given unsigned 64-bit integer `number` (see **ints** package), and optionally more 64-bit unsigned integers, to the file denoted by `filehandle` starting at the current position. The function returns **true** in case of success and issues an error otherwise. The function is not supported on Big Endian systems.

See also: **binio.readint64**, **binio.writeuint64**.

12.3 xbase - Library to Read and Write xBase Files

This package provides basic functions to read and write dBASE III+ compliant files.

A typical session may look like this: First create the database file with its only field 'data' of type 'Number', a floating point value:

```
> xbase.new('test.dbf', data = 'Number');
```

Now open the file:

```
> f := xbase.open('test.dbf', 'write');
```

Write the number Pi to record 1 and field 1

```
> xbase.writenumber(f, 1, 1, Pi);
```

Read back the value just written:

```
> xbase.readvalue(f, 1, 1):
3.1415926535898
```

Finally, close the database file.

```
> xbase.close(f):
true
```

xbase functions, if not stated otherwise below, can also be called OOP-style. Example:

```
> xbase.new('test.dbf', data='Number');

> f := xbase.open('test.dbf', 'write'):
xbase(01E063B0)

> f@@writenumber(1, 1, Pi);

> f@@readvalue(1, 1):
3.1415926535898

> f@@close():
true
```

Limitations:

1. The xBase data types currently supported are: 'Number', 'Float' (dBASE IV 2.0), Binary 'Double' (dBASE 7), 'String', 'Date', and 'Logical'.
2. Only files with extension .dbf are supported. Searching and sorting functions are not available, and any .ndx, or .idx index files or *.dbt files will be ignored.
3. Files with sizes greater than 2 GBytes are not supported.

The package supports the proprietary type 4-byte signed floats, complex numbers, unsigned bytes and 2-byte signed and unsigned integers which are all not part of the dBASE 7 standard.

An elaborate example: Let us create a city database that stores the name of the city in field 1, its latitude and longitude in decimal degrees in fields 2 and 3, and the elevation in meters, which can also be below sea level, in field 4.

If you later write a city name that consists of more than 15 characters, the name will just be truncated.

```
> xbase.new('cities.dbf',
>    City      = 'String':15,  # 15 characters for each city name
>    Latitude  = 'Double',     # location as 8-byte doubles
>    Longitude = 'Double',
>    Elevation = 'Short');     # elevation as a signed 2-byte integer

> f := xbase.open('cities.dbf', 'write');

> f@@writestring(1, 1, 'Chattanooga, TN');
> f@@writedouble(1, 2, 35.04563);
> f@@writedouble(1, 3, -85.30968);
> f@@writeshort (1, 4, -208);

> f@@writestring(2, 1, 'Düsseldorf, NRW');
> f@@writedouble(2, 2, 51.22172);
> f@@writedouble(2, 3, 6.77616);
> f@@writeshort (2, 4, 45);
```

The elevation for Chattanooga was wrong, so we correct that easily:

```
> f@@writeshort(1, 4, 208);
```

Let us traverse all two records with four fields each:

```
> for i to 2 do     # records
>    for j to 4 do  # fields
>       printf('%15s\t', tostring(f@@readvalue(i, j)))
>   od;
>   printf('\n')
>od;
Chattanooga, TN        35.04563       -85.30968             208
Düsseldorf, NRW        51.22172         6.77616              45
```

Get the first record and the second column in the dBASE file and finally close it:

```
> f@@record(1):
seq(Chattanooga, TN, 35.04563, -85.30968, 208)

> xbase.readdbf(f, field = 2):
seq(35.04563, 51.22172)

> f@@close():
```

xbase.attrib (filehandle)

returns a table with various information on the xBase file pointed to by `filehandle`.

Table key	Meaning
`'codepage'`	Code page used.
`'fieldinfo'`	A table of tables that describe the respective fields in consecutive order: title, xBase native type (see below), Agena type, total number of bytes occupied by the field in the file. With numbers, the number of decimals following the decimal point (its scope) given.
`'fields'`	Number of fields in the file.
`'filename'`	Name of the xBase file (relative).
`'headerlength'`	Length of the header in the xBase file.
`'lastmodified'`	UTC date of the last write access, coded as an integer.
`'records'`	Number of records stored in the file.
`'recordlength'`	Number of bytes occupied by each record.
`'version'`	dBASE version number (see **xbase.new**)
`'versionname'`	dBASE version name (a string, see **xbase.new**)

xBase native types recognised are: 'C' for String, 'N' for Number, 'F' for Float, 'L' for Logical, 'D' for Date, and 'O' for a binary Double and 'I' for a binary 4-byte signed integer. 'B' indicates a .DBT block number and 'M' a memo field. See **xbase.new** for further information.

For known version numbers, see **xbase.new**, as well. To check for dBASE 7, binary-and the result with 0b111 and check for result 4, e.g. result **&&** 0b111= 4.

See also: **xbase.fieldtype**, **xbase.filepos**.

xbase.close (filehandle)

Closes a connection to the xBase file pointed to by `filehandle`. No more data can be read or written to the xBase file until you open it again using **xbase.open**. The function returns **true** if the file could be closed, and **false** otherwise.

xbase.eof (filehandle)

Checks whether the end of the file denoted by `filehandle` has been reached and returns **true** or **false**. The function returns **fail** if an error occurred during the check.

xbase.fields (filehandle)

Returns the number of fields per record contained in the xBase file denoted by `filehandle`.

See also: **xbase.attrib**, **xbase.records**.

xbase.fieldtype (filehandle, field)

Determines the dBASE data type of the given `field` in the open file denoted by `filehandle`. The function returns a one-character string, or the string `'?'` if it is unknown. See **xbase.new** for the meaning of the return.

See also: **xbase.attrib**.

xbase.filepos (filehandle)

Returns the current file position in the file denoted by `filehandle` and returns it as a number.

See also: **xbase.attrib**.

xbase.header (filehandle)

Returns three sequences: the header field names of the file denoted by `filehandle`, the corresponding Agena type names, and the respective single-character dBASE types.

See also: **xbase.attrib**.

xbase.ismarked (filehandle, record)

Checks whether a record in a file denoted by `filehandle` has been marked as to be deleted and returns **true** or **false**.

Please make sure that the file has been opened in write, append, or read/write mode before, otherwise the result may be undefined.

See also: **xbase.mark**.

xbase.isopen (filehandle)

Checks whether `filehandle` points to an open xBase file (opened by the same Agena session) and returns **true** or **false**.

xbase.isvoid (filehandle, record, field)

Checks whether the value at record number `record` and field number `field` from the file pointed to by `filehandle` has been deleted.

The function returns either **true** or **false**.

See also: **xbase.ismarked**, **xbase.mark**, **xbase.purge**, **xbase.readvalue**.

xbase.kernel ([options])

The function sets defaults for the binary representation and layout of binary doubles and binary time stamps in dBASE Level 7 files:

Setting	Default	Description
`DoubleIsBigEndian`	**true**	If set to **false**, binary doubles shall be represented in Little Endian notation. By default, binary doubles are Big Endian.
`LongIsBigEndian`	**false**	If set to **true**, binary doubles shall be represented in Big Endian notation. By default, binary integers are Little Endian.
`TimestampIsDouble`	**false**	If set to **true**, binary time stamps are represented as a binary 8-byte double; by default time and date are represented by two 4-byte signed integers, with the endianness depending on the setting of `Long1IsBigEndian`.

If no argument is given, the current settings are returned.

The function cannot be used OOP-style.

xbase.lock (filehandle)

xbase.lock (filehandle, size)

The function locks the file given by its handle `filehandle` so that it cannot be read or overwritten by other applications.

In the first form, the entire file is locked in UNIX-based systems. In Windows, only 2^{63} bytes are locked, so you have to use the second form in Windows after the file has become larger than 2^{63} bytes (= 8,589,934,592 GBytes).

In the second form the function locks `size` bytes from the current file position. Locked blocks in a file may not overlap. `size` may be larger than the current file length.

The function returns **true** on success and **false** otherwise.

Note that other applications that do not use the locking protocol may nevertheless have read and write access to the file.

See also: **xbase.unlock**.

xbase.mark (filehandle, row [, flag])

Marks the record number `row`, an integer, in the file denoted by its `filehandle`, as deleted.

Returns **true** if a record has been marked successfully, **and false** otherwise.

The actual data is not physically deleted, however, **xbase.readvalue**, **xbase.record**, **xbase.field**, and **xbase.readdbf** do not return it. Use **xbase.purge** to delete entries.

If flag is **false**, a formerly marked record is activated (`undeleted`) again.

Please make sure that the file has been opened in write, append, or read/write mode before, otherwise the result may be undefined.

See also: **xbase.ismarked**.

xbase.new (filename, $desc_1$ [, code page] [, version] [, $desc_2$, ···, $desc_k$])

creates a new xBase file with the file name `filename`.

$desc_k$ are `k` fields (columns) the xBase file will contain. code page indicates the code page to be used (see below)[4].

In its header, the function designates the resulting file as a dBASE III+ file without memo .DBT file.

$desc_k$ must be a pair of the following form:

1. field_name : data_type

   where `field_name` is a string and the name of the field to be added, and `data_type` is one of the strings `'Logical'`, `'Date'`, `'Float'`, `'Number'`, `'Double'`, `'Long'`, `'Complex'`, `'Byte'`, `'Decimal'` or `'Character'`, i.e. the xBase data type of the values to be stored later on.

   Examples:

   `new('dbase.dbf', 'logical':'Logical')` or
   `new('dbase.dbf', logical='Logical')` for short for a Boolean.

   A Boolean (which in xBase is equal to a `Logical`) will always consist of one character `'T'`, `'F'` for **true** and **false**.

   An xBase Number will have a standard length of 19 places with a default scale of 15 digits, whereas an xBase Float consists of 20 places with a scale of 18

[4] Note that code pages are a Foxpro extension.

digits (scale: numbers following the decimal point). Numbers are stored in xBase files as strings with ANSI C double precision. The scale may be in [0, 15] with xBase Numbers, and in [0, 18] with xBase Floats.

An xBase Character (string) will have a default length of 64 characters. The minimum length of a string is 1, the maximum length of a string may be 254 characters. Longer strings will be truncated.

A date will always consist of eight digits of the format YYYYMMDD.

A dBase Level 7 Double represents an Agena number (integral or fractional) that is stored in either binary Big Endian or Little Endian format of eight bytes to an xBase file. The default is Big Endian, but you can change this when writing or reading files, see **xbase.kernel**.

A dBase Level 7 Long represents a signed 4-byte integer that is stored in either binary Big Endian or Little Endian format to an xBase file. The default is Little Endian, but you can change this when writing or reading files, see **xbase.kernel**.

There are three proprietary nonstandard types: Type `'Complex'` represents a complex number, stored in 16 bytes, type `'Byte'` represents an unsigned integer in the range 0 .. 255, stored in just one byte, `'UShort'` stores unsigned 2-byte integers, `'Short'` stores signed 2-byte integers, and the experimental type `'Decimal'` stores numbers as signed 4-byte C floats (note that these are highly inaccurate). All of these four types are represented in Little Endian notation.

2. field_name : data_type : length

where `field_name` and `data_type` are the same as mentioned above, and `length` is the maximum length of the item to be added. `length` must be a positive integer. With numbers, `length` denotes the number of digits after the decimal point to be stored.

When passing a length value, you may leave out the quotes for data_type values.

Examples:

`new('dbase.dbf', 'value':'Number':5)` or
`new('dbase.dbf', value=Number:5)` for short for a float with five decimal places.

Supported xBase data types are:

Type	data_type name	Agena type	Write function	dBASE version
Logical	`'Logical'` or `'L'`	boolean	**xbase.writeboolean**	III+
Number	`'Number'` or `'Numeric'` or `'N'`	number	**xbase.writenumber**	III+
Float	`'Float'` or `'F'`	number	**xbase.writefloat**	IV 2.0
Double	`'Double'` or `'O'`	number	**xbase.writedouble**	7
Long	`'Long'` or `'I'`	number	**xbase.writelong**	7
Character	`'Character'` or `'C'`	string	**xbase.writestring**	III+
Date	`'Date'` or `'D'`	string	**xbase.writedate**	III+
Timestamp	`'@'`	numbers	**xbase.writetime**	7
OLE	`'G'`		n/a	?
Memo	`'M'`		n/a	?
Binary	`'B'`		n/a	?
Complex	`'Complex'` or `'c'`	complex	**xbase.writecomplex**	prop.
Byte	`'Byte'` or `'b'`	number	**xbase.writebyte**	prop.
Decimal	`'Decimal'` or `'f'`	number	**xbase.writedecimal**	prop.
Short	`'Short'` or `'s'`	number	**xbase.writeshort**	prop.
UShort	`'UShort'` or `'u'`	number	**xbase.writeushort**	prop.

`codepage` should be a pair of the form `'codepage'`:n, with n an integer in [0, 255].

Valid codepages are:

n	Meaning	Code page
0x01	DOS USA	437
0x02	DOS Multilingual	850
0x03	Windows ANSI	1.252
0x04	Standard Macintosh	10.000
0x64	Eastern Europe DOS	852
0x65	Nordic DOS	865
0x66	Russian DOS	866
0x67	Icelandic DOS	861
0x68	Kamenicky (Czech) DOS	895
0x69	Mazovia (Polish) DOS	620
0x6a	Greek DOS	437G
0x6b	Turkish DOS	857
0x78	Traditional Chinese (Republic of China (Taiwan), Hong Kong SAR)	950
0x79	Korean Windows	949
0x7A	Chinese Simplified (Singapore, PRC)	936
0x7B	Japanese Windows	932
0x7C	Thai Windows	874

n	Meaning	Code page
0x7D	Hebrew Windows	1.255
0x7E	Arabic Windows	1.256
0x96	Russian Macintosh	10.007
0x97	Eastern European Macintosh	10.029
0x98	Greek Macintosh	10.006
0xc8	Eastern Europe Windows	1.250
0xc9	Russian Windows	1.251
0xca	Turkish Windows	1.254
0xcb	Greek Windows	1.253

If no code page has been passed, it is set to 0x00.

Example for Eastern European Macintosh:

```
new('dbase.dbf', text=string:255, code page=0x97);
```

`version` should be a pair of the form `'version'`:n, with n an integer in [0, 255].

dBASE version numbers are:

```
x xxx x 001 = 0x?1 not used
0 000 0 010 = 0x02 FoxBASE
0 000 0 011 = 0x03 FoxBASE+/dBASE III PLUS, no memo
x xxx x 100 = 0x?4 dBASE 7
0 000 0 101 = 0x05 dBASE 5, no memo
0 011 0 000 = 0x30 Visual FoxPro
0 011 0 001 = 0x31 Visual FoxPro, autoincrement enabled
0 011 0 010 = 0x32 Visual FoxPro, Varchar, Varbinary, or Blob-enabled
0 100 0 011 = 0x43 dBASE IV SQL table files, no memo
0 110 0 011 = 0x63 dBASE IV SQL system files, no memo
0 111 1 011 = 0x7B dBASE IV, with memo
1 000 0 011 = 0x83 FoxBASE+/dBASE III PLUS, with memo
1 000 1 011 = 0x8B dBASE IV, with memo
1 000 1 110 = 0x8E dBASE IV with SQL table
1 100 1 011 = 0xCB dBASE IV SQL table files, with memo
1 110 0 101 = 0xE5 Clipper SIX driver, with SMT memo
1 111 0 101 = 0xF5 FoxPro 2.x (or earlier) with memo
1 111 1 011 = 0xFB FoxBASE (with memo?)
| ||| | |||
| ||| | |||   Bit flags (not used in all formats)
| ||| | |||   ----------------------------------
| ||| | +++-- bits 2, 1, 0, version (x03 = level 5, x04 = level 7)
| ||| +------ bit 3, presence of memo file
| +++-------- bits 6, 5, 4, presence of dBASE IV SQL table
+------------ bit 7, presence of .DBT file
```

The default is 0x03 = 3 decimal for dBASE III+. If at least one of the given fields is of dBASE data type 'Double' (= 'B' or 'O') or type 'timestamp', then the version number is automatically changed to 0x?4 = 4 decimal = dBASE Level 7. This allows dBASE files created with Agena and containing binary Doubles to be imported into LibreOffice 5.x. Current versions of Excel still cannot read Visual Fox Pro dbf files with Doubles or Longs, so you might pass the `version` option.

The function cannot be used OOP-style.

See also: **xbase.open**.

xbase.open (filename [, mode])

Opens an xBase file of the name `filename` for reading or writing, or both.

In the first form, the file is opened for reading only.

In the second form, if `mode` is either `'write'`, `'w'`, `'append'`, or `'r+'`, the file is opened for reading while new data sets may be added to the end of the file.

If mode is `'read'` or `'r'`, the file is opened for reading only.

The return is a file handle to be used by all other xBase package functions.

The function cannot be used OOP-style.

See also: **xbase.close**, **xbase.lock**, **xbase.new**.

xbase.purge (filehandle, record, field)

Overwrites the specified `field` in the given `record` of the file denoted by its handle `filehandle` with asterisks, thus physically deleting the original content. The return is **true** if deletion succeeded, and **false** otherwise. After successful completion, a subsequent call to **xbase.isvoid** would return **true**.

See also: **xbase.isvoid**, **xbase.mark**, **xbase.wipe**.

xbase.readdbf (filename [, option])
xbase.readdbf (filehandle [, option])

In the first form, opens an xBase file denoted by its `filename` in read mode, returns all its records and fields, and closes it.

In the second form, it reads the entire contents of the open file denoted by its handle `filehandle`.

If the xBase file contains more than one field, the data will be returned as a sequence of sequences, whereas if the file contains only one field, all values are returned in one sequence only.

If the option `field`=x or `fields`=x with x a positive number is given, only the given column x is extracted, and the return is a sequence of the column values.

If the option `field`=obj or `fields`=obj with obj a table or sequence of positive numbers is given, only the given fields in the records are returned, and the return is a sequence of sequences.

If a record has been marked as being deleted, the function ignores the record.

See also: **xbase.field**, **xbase.ismarked**, **xbase.readvalue**, **xbase.record**.

xbase.readvalue (filehandle, record, field)

Reads a value at record number `record` and field number `field` from the file pointed to by `filehandle`.

Supported values are of xBase type Logical, Number, Float, binary double, Date, Timestamp and String. Also Binary, Memo and OLE .DBT block numbers are supported, as well as type Complex, Byte, Short, UShort and Decimal. If a number could not be read from the file, the function returns 0. On Little Endian systems, you might have to convert Big Endian binary doubles back to Little Endian by calling **bytes.tolittle**; see also **os.endian**.

If `record` has been marked as being deleted, the function returns **null**.

See also: **xbase.field**, **xbase.ismarked**, **xbase.record**, **xbase.isvoid**.

xbase.record (filehandle, line)

Returns all values in the given record `line` (a number) of the file denoted by `filehandle` and returns them in a sequence.

If `record` has been marked as being deleted, the function returns **null**.

See also: **xbase.field**, **xbase.ismarked**, **xbase.readdbf**, **xbase.readvalue**.

xbase.records (filehandle)

Returns the number of records contained in the xBase file denoted by `filehandle`, including the ones marked as to be deleted or being completely void.

See also: **xbase.attrib**, **xbase.fields**.

xbase.sync (filehandle)

Writes any unwritten content to the xBase file pointed to by `filehandle`. The function either returns **true** if flushing succeeded or nothing had be flushed, or **fail** otherwise.

Please make sure that the file has been opened in write, append, or read/write mode before, otherwise the result may be undefined.

xbase.unlock (filehandle)

xbase.unlock (filehandle, size)

The function unlocks the file given by its handle `filehandle` so that it can be read or overwritten by other applications again.

The function returns **true** on success and **false** otherwise.

For more information, see **xbase.lock**.

xbase.wipe (filehandle, record)

In an xBase file denoted by `filehandle`, deletes all fields of the given `record`, a positive integer by overwriting all fields with asterisks. It also marks the record as deleted (see **xbase.mark** for further information).

To ensure performance, the function does not lock the file before deleting data - you may want to manually call **xbase.lock** before and **xbase.unlock** thereafter. Also, it does not flush the file.

The function returns nothing.

The function has been written in Agena, see lib/xbase.agn.

See also: **xbase.mark**, **xbase.purge**.

xbase.write (filehandle, record, field, value)

Writes the number, string or boolean (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

The function automatically determines whether the respective field is of xBASE type Numeric ('N'), Float ('F'), binary Long ('I'), Binary Double ('@'), Character ('C') or Logical ('L').

The return is **true** if writing succeeded, and **false** otherwise, the latter only indicating whether an error may have occurred.

xbase.writeboolean (filehandle, record, field, value)

Writes the Boolean value **true** or **false** (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`. **fail** and **null** are not supported.

When creating the dBASE file with **xbase.new**, pass the `'L'` data type descriptor for the respective fields.

The return is **true** if writing succeeded, and **false** otherwise.

xbase.writebyte (filehandle, record, field, value)

Writes the integer `value` in the range 0 .. 255 (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'b'` data type descriptor for the respective fields.

The integer is stored in binary format of just one byte (unsigned C char).

A proprietary extension, applications that import dBASE files - such as Microsoft Excel - do not support this type.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.writefloat**, **xbase.writelong**, **xbase.writenumber**, **xbase.new**.

xbase.writecomplex (filehandle, record, field, value)

Writes the complex number `value` (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'c'` data type descriptor for the respective fields.

The complex number is stored in Little Endian binary format of sixteen bytes (two C doubles, converted to two C uint64_t).

A proprietary extension, applications that import dBASE files - such as Microsoft Excel - do not support this type.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.writefloat**, **xbase.writelong**, **xbase.writenumber**, **xbase.new**.

xbase.writedate (filehandle, record, field, value)

Writes the string or number `value` (4th argument), representing an integer - or a string representing an integer - in the range$19000101 \leqslant x \leqslant 99991231$and denoting a date, to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'D'` data type descriptor for the respective fields.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.writetime**.

xbase.writedecimal (filehandle, record, field, value)

Writes the number `value` (4th argument) as a signed 4-byte Little Endian float to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'f'` data type descriptor for the respective fields.

A proprietary extension, applications that import dBASE files - such as Microsoft Excel - do not support this type.

Note that 4-byte floats are inherently inaccurate, so when reading them back into Agena you will find significant round-off errors due to stray bits.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

xbase.writedouble (filehandle, record, field, value)

Writes the number `value` (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'O'` (letter O) data type descriptor for the respective fields.

The number is stored in binary format of eight bytes (C double, converted to a C uint64_t). By default, doubles are written in Big Endian representation. You can change this to Little Endian by setting

```
> xbase.kernel(DoubleIsBigEndian = false);
```

A dBASE 7 extension, some applications that import dBASE files - such as Microsoft Excel - do not support binary numbers, but LibreOffice 5.x and beyond does.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.writefloat**, **xbase.writelong**, **xbase.writenumber**, **xbase.new**.

xbase.writefloat (filehandle, record, field, value)

Writes the number `value` (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'F'` data type descriptor for the respective fields.

The number is stored with a total of 20 digits, including a maximum of 18 digits following the decimal point (scale).

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

A proprietary extension, applications that import dBASE files - such as Microsoft Excel - do not support this type.

See also: **xbase.writedouble**, **xbase.writenumber**, **xbase.new**.

xbase.writelong (filehandle, record, field, value)

Writes the number `value` (4th argument) representing a 4-byte signed integer (int32_t) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'I'` data type descriptor for the respective fields.

The function automatically truncates Agena numbers containing decimal places to their integral part and issues an error if the numeric range [-2'147'483'647, +2'147'483'647] is exceeded.

By default, longs are written in Little Endian representation. You can change this to Big Endian by setting

```
> xbase.kernel(LongIsBigEndian = true);
```

A dBASE 7 extension, some applications that import dBASE files - such as Microsoft Excel - do not support binary integers, but LibreOffice 5.x and beyond does.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.new**, **xbase.writedouble**, **xbase.writefloat**, **xbase.writenumber**, **xbase.writeushort**.

xbase.writenumber (filehandle, record, field, value)

Writes the number `value` (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

The function automatically determines whether the respective field is of xBASE type Numeric (`'N'`), Float (`'F'`), binary Long (`'I'`), or Binary Double (`'O'`) (letter O). Concerning `'O'` and `'I'`, read the remarks on Endianness in the description of **xbase.writedouble**.

It also writes the proprietary formats 'b' for byte (see **xbase.writebyte**) and 'c' for complex numbers (see **xbase.writecomplex**).

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.writedouble**, **xbase.writefloat**, **xbase.writelong**, **xbase.new**.

xbase.writeshort (filehandle, record, field, value)

Writes the number `value` (4th argument) representing a 2-byte signed integer (int16_t) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'s'` data type descriptor for the respective fields.

The function automatically truncates Agena numbers containing decimal places to their integral part and issues an error if the numeric range [-32,768, 32,767] is exceeded.

By default, shorts are written in Little Endian representation. You can change this to Big Endian by setting

```
> xbase.kernel(LongIsBigEndian = true);
```

A proprietary extension, applications that import dBASE files - such as Microsoft Excel - do not support this type.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.new**, **xbase.writedouble**, **xbase.writefloat**, **xbase.writelong**, **xbase.writenumber**, **xbase.writeushort**.

xbase.writestring (filehandle, record, field, value)

Writes the string value (4th argument) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'C'` data type descriptor for the respective fields.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error might have occurred.

xbase.writetime (filehandle, record, field,
y, m, d [, h [, m [, s [, ms]]]])

Writes the timestamp given by the year `y`, month `m`, day `d`, and optionally hour `h`, minute `mm`, second `s`, and milliseconds `ms` to the file denoted by `filehandle` to record number `record` and field number `field`.

By default, hours, minutes, seconds, and milliseconds default to 0. Milliseconds must be an integer in the range [0, 999].

When creating the dBASE file with **xbase.new**, pass the `'@'` data type descriptor for the respective fields.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error might have occurred.

By default, the timestamp internally is represented by two signed 32-bit integers in Little Endian representation. You can change this to Big Endian representation by setting:

```
> xbase.kernel(LongIsBigEndian = true);
```

If the timestamp shall be represented by a binary double - the Julian Date, but the documentation available on dBASE Level 7 files is contradictory - you can set:

```
> xbase.kernel(TimestampIsDouble = true);
```

The endianness of the double is Big Endian by default. You can change this to Little Endian by setting:

```
> xbase.kernel(DoubleIsBigEndian = false);
```

Some applications do not recognise a dBASE 7 timestamp. You may encode it as:

- a 4-byte integral Julian Date plus
- a 4-byte millisecond-since-midnight integer, or alternatively:
- a 8-byte Julian Date incl. milliseconds-since-midnight in the fractional part.

astro.jdate returns these three values when given the `full=true` option.

See also: **xbase.writedate**.

xbase.writeushort (filehandle, record, field, value)

Writes the number `value` (4th argument) representing a 2-byte unsigned integer (uint16_t) to the file denoted by `filehandle` to record number `record` and field number `field`.

When creating the dBASE file with **xbase.new**, pass the `'u'` data type descriptor for the respective fields.

The function automatically truncates Agena numbers containing decimal places to their integral part and issues an error if the numeric range [0, 65,535] is exceeded.

By default, shorts are written in Little Endian representation. You can change this to Big Endian by setting

```
> xbase.kernel(LongIsBigEndian = true);
```

A proprietary extension, applications that import dBASE files - such as Microsoft Excel - do not support this type.

The return is **true** if writing succeeded, and **false** otherwise. Note that the return **false** only indicates that an error may have occurred.

See also: **xbase.new**, **xbase.writedouble**, **xbase.writefloat**, **xbase.writelong**, **xbase.writenumber**, **xbase.writeshort**.

12.4 ads - Agena Database System

As a *plus* package, this simple database is not part of the standard distribution and must be activated with the **import** statement, e.g. `import ads`.

Agena is a database for storing and accessing strings, and strings only, and currently supports three `base` types:

1. Sorted `databases` with a key and one or more values,

2. sorted `lists` which store keys only,

3. unsorted `sequences` to hold strings or Agena numbers.

With databases and lists, each record is indexed, so that access to it is very fast. If you store data with the same key multiple times in a database, the index points to the last record stored, so you always get a valid record.

Sequences do not have indexes, so searching in sequence database files is rather slow. However, all values can be read into the Agena environment rapidly with **ads.getall**.

The Agena Database System (ADS) pays attention to both file size and fast I/O operation. To reduce file size, the keys (and values) are stored with their actual lengths (of C type `int32_t`, so keys and values can be of almost unlimited size) and they are not extended to a fixed standard length. To fasten I/O operations, the length of each key (and value) is also stored within the base file.

The following terms are used in this chapter:

Section	Description
header	various information on the data file, including the maximum number of possible records, the actual number of records and the type of the base (database, list, or sequence).
index	only with databases and lists: area containing all file positions of the actual records. The index section is always sorted. Sequence databases do not contain an index section.
records	key-value pairs with databases, and keys with lists or sequences.

A sample session:

First activate the package:

```
> import ads
```

Create a new database (file `test.agb`) including all administration data like number of records, etc.:

```
> ads.new('test.agb');
```

Open the database for access. The variable `fh` is the file handle which refers to the database file `test.agb` and is used in nearly all **ads** functions.

```
> fh := ads.open('test.agb');
```

Put an entry into the database with key `Duck` and value `Donald`.

```
> ads.write(fh, 'Duck', 'Donald');
```

Check what is stored for `Duck`.

```
> ads.read(fh, 'Duck'):
Donald
```

Show information on the database:

```
> ads.attrib(fh):
keylength ~ 31                  # Maximum length for key
type ~ 0                        # database type, 0 for relational database
stamp ~ AGENA DATA SYSTEM       # name of database
indexstart ~ 256                # begin of index section in file
commentpos ~ 0                  # position of a description, 0 because none
                                # was given.
version ~ 300                   # base version, here 3.00
maxsize ~ 4096                  # maximum number of possible records.
indexend ~ 80255                # end of index section
creation ~ 2008/01/18-19:00:50  # number of creation
columns ~ 2                     # number of columns
size ~ 1                        # number of actual entries
```

Close the database. After that you cannot read or write any entries any longer. Use the **ads.open** function if you want to have access again.

```
> ads.close(fh);
```

If you are just interested in reading in a bunch of numbers from a file you might prefer the ADS package over the xbase package as it is more 20 times faster with importing numeric data into a session. Alternatively, the **keyfile** package offers uncapped data storage.

On all **ads** database types, you may use the following procedures, which may all also be called OOP-style except for **ads.new**:

ads.attrib (filehandle)

Returns a table with all attributes of the `base` file. The table includes the following keys:

Key	Description	Type
'columns'	The number of columns in the base.	number
'commentpos'	The position of a comment in the base. If no comment is present, its value is 0.	number
'creation'	The date of creation of the base. The return is a formatted string including date and time.	string
'indexstart'	the first byte in the base file of the index section.	number
'indexend'	the last byte in the base file of the index section.	number
'inthandle'	the internal C integer file handle, used to register an ads file in the **ads.openfiles** table.	number
'keysize'	the maximum length of the record key.	number
'maxsize'	total number of data sets allowed.	number
'size'	the actual number of valid data sets (see **ads.sizeof** as a shortcut).	number
'stamp'	The base stamp at the beginning of the file.	string
'type'	Indicator for database (0), list (1) or sequence (2).	number
'version'	The base version.	number
'description'	The description; if consisting of blanks only, will not be returned.	string

ads.attrib also reads out the following parameters directly from the file handle userdata and not the file header and returns them with the 'handle' field:

- 'maxsize': maximum record capacity,
- 'size': current number of records,
- 'keylength': key length,
- 'columns': number of columns,
- 'type': base type,
- 'lof': length of the ADS file in bytes.

If the file is not open, **attrib** returns **false**.

See also: **ads.free**, **ads.sizeof**.

ads.clean (filehandle)

The function has been deprecated.

ads.close (filehandle [, filehandle2, ···])

Closes the base(s) identified by the given file handle(s) and returns **true** if successful, and **false** otherwise. **false** will be returned if at least one base could not be closed. The function also deletes the file handles and the corresponding filenames from the **ads.openfiles** table.

See also: **ads.open**.

ads.createseq (filename)

Creates a sequence with the given `filename` (a string). The function is written in Agena and can be used after issuing `import ads`.

ads.desc (filehandle)
ads.desc (filehandle, description)

In the first form, returns the description of a base stored in the file header. If no description has explicitly been written, the function returns the empty string.

In the second form, **ads.desc** sets or overwrites the description section of a database or list. Pass the description as a string. If the string is longer than 75 characters, **fail** will be returned and there are no changes to the base file. To delete a description, pass the empty string or **null**. If the file is not open, **fail** will be returned, as well. If it was successful, the return is **true**.

ads.expand (filehandle [, n])

The function has been deprecated.

ads.filepos (filehandle)

Returns the current position of the file denoted by `filehandle`. See also: **ads.attrib**.

ads.find (filehandle, pattern [, column])

With databases, the function searches all entries in the given column for substring `pattern` and returns all respective keys and the matching entries in a table.

If `column` is omitted, the second column will be searched.

If `column` is 0, then all columns but the first will be searched and the return is a table of pairs with the left operand the value found and the right operand the column where it has been found.

The function supports pattern matching, see Chapter 9.1.3.

With lists and sequences, the function always returns **null**. If the base is empty, **null** will be returned, as well.

If the file is not open or the column does not exist, the function will return **fail**.

See also: **ads.read**, **ads.getvalues**.

ads.free (filehandle)

Determines the number of free data sets and returns them as an integer. If the base has not been opened, it returns **fail**. See also: **ads.attrib**.

ads.getall (filehandle [, option])

Converts an ADS string or number sequence to a set and returns this set. The function automatically initialises the set with the number of entries in the ADS sequence. If the file is not open, **fail** will be returned.

If any `option` is given, and if it is a sequence, set or table, then to contents if the ADS sequence will be added to it, meaning that if `option` already includes elements, they will be preserved. If `option` is given but is not a table, sequence or set, then the values will be inserted into a sequence instead of a set.

The values are added/appended in the order of their physical presence in the database file; if one and the same entry is stored multiple times, it will also be returned multiple times in the sequence or table.

See also: **ads.getkeys**, **ads.getvalues**.

ads.getkeys (filehandle)

Gets all valid keys in a database or list and returns them in a table. Argument: file handle (integer). If the file is not open, **fail** will be returned. If the base is empty, **null** will be returned. The function will issue an error if the file represents a sequence.

See also: **ads.get**, **ads.getvalues**.

ads.getvalues (filehandle [, column])

By default, gets all valid entries in the second column in a database referred to by `filehandle` and returns them in a table. If the optional argument `column` is given, the entries in this column will be returned.

If the file is not open or if the column does not exist, **fail** will be returned. If the base is empty, **null** will be returned. With lists, the return is always **null**.

See also: **ads.get**, **ads.getkeys**.

ads.index (filehandle, key)

Searches for the given `key`, a string, in the base pointed to by `filehandle` and returns its file position as a number. If there are no entries for `key`, the function will return **null**. If the file is not open, **fail** will be returned. See also: **ads.indices**.

ads.indices (filehandle)

Returns the file positions of all valid data sets as a table.

If the file is not open, the function will return **fail**. If there are no entries in the base, the return will be an empty table, otherwise a table with the indices will be returned. The function will issue an error if the file represents a sequence.

See also: **ads.retrieve**, **ads.invalids**, **ads.peekin**, **ads.index**.

ads.invalids (filehandle)

Returns the file positions of all invalid records in a database as a table.

If the file is not open, the function will return **fail**. If no invalid entries have been found, the return is an empty table. See also **ads.retrieve**. Note that the function also works with lists. However, since lists never contain invalid records, an empty table will always be returned with lists.

With sequences, the function issues an error.

ads.iterate (filehandle [, key])

Iterates sequentially and in ascending order over all keys in the database or list. With databases, both the next key and its corresponding values are returned, the values enclosed in a table array. With lists, only the next key is returned.

The very first `key` can be accessed with an empty string or **null**, or by only passing `filehandle`. If there are no more keys left, the function will return **null**. If the database is empty, **null** will be returned, as well. If the file is not open, the function will return **fail**.

Example to traverse a US zip code database - you will find the file `uszips.agb` in the Agena test suite:

```
> import ads;
> fh := ads.open('uszips.ads');
> zip, data := ads.iterate(fh, null);
> while zip do
>    zip, data := ads.iterate(fh, zip);
>    print(zip, data)
> od;
> ads.close(fh);
```

With ADS number and string sequences, the function returns an iterator function that when called returns the next entry in it.

The function does not close the file when returning the very last value. Use ads.close to accomplish this.

See also: **ads.read**.

ads.lock (filehandle)

ads.lock (filehandle, size)

The function locks the file given by its handle `filehandle` so that it cannot be read or overwritten by other applications.

In the first form, the entire file is locked in UNIX-based systems. In Windows, only 2^{63} bytes are locked, so you have to use the second form in Windows after the file has become larger than 2^{63} bytes (= 8,589,934,592 GBytes).

In the second form the function locks `size` bytes from the current file position. Locked blocks in a file may not overlap. `size` may be larger than the current file length.

Note that other applications that do not use the locking protocol may nevertheless have read and write access to the file.

See also: **ads.unlock**.

```
ads.new (filename
    [, number_of_records [, type [, number_of_columns
    [, length_of_key [, description]]]]])

ads.new (filename
    [, number_of_records [, type [, length_of_key [, description]]]])

ads.new (filename [, options])
```

Creates and initialises the index section of a new base with the given number of columns. It returns the file handle as a number, and closes the created file.

The first form defines a database, the second form is used to create sequences and lists.

In the third form, options may be one or more pairs, see next table.

Arguments / Options:

filename	The path and full name of the base file.
number_of_records *or the option* records = number_of_records	The maximum number of records in the base. Default is 4,096. If you pass 0, **fail** will be returned and the base is not created. You cannot extend a base or list. Example when passed as an option: `records = 10`. Alternative option name is 'recs' instead of 'records'.
type *or the option* basetype = type	By default, the type is 'database'. If you pass the string 'list', then a list will be created. The string 'string' will create a string sequence, 'number' a number sequence. If the type passed is not known, **fail** will be returned and no base is created. Example when passed as an option: `basetype = 'list'`. Alternative option name is 'base' instead of 'basetype'.
number_of_columns *or the option* columns = number_of_columns	The number of columns in a database. Default: 2 (key and value). If the base is not a database, do not pass any value (see second form). If the number of columns is non-positive, **fail** will be returned and no base will be created. Example when passed as an option: `columns = 3`. Alternative option name is 'cols' instead of 'columns'.
length_of_key *or the option* keylength = length_of_key	The maximum length of the base key. Note that internally, the length is incremented by 1 for the terminating \0 character. Default: 31 including the terminating \0 character. Example when passed as an option: `keylength = 20`. Alternative option name is 'keylen' instead of 'keylength'.
description *or the option* description = (a string)	A string with a description of the contents of the base. A maximum of 75 characters is allowed (including the \0 character). If the string is too long, it will be truncated. Default: 75 spaces. Example when passed as an option: `description = 'my database'`. Alternative option name is 'desc' instead of 'description'.

See also: **ads.open**.

ads.open (filename [, anything])

Opens the base with name `filename`, a string, and returns a file handle (a userdate object). If it cannot find the file, or the base has not the correct version number, the function will return **fail**. The base is opened in both read and write mode.

If an optional second argument is given (any valid Agena value), the base will be opened in read mode only.

The function also enters the integer file handle of the newly opened file into the **ads.openfiles** table.

See also: **ads.close**.

ads.openfiles

A global table containing all files currently open. Its keys are the file handles (integers), the values the file names (strings). If there are no open files, **ads.openfiles** is an empty table.

ads.peekin (filehandle, position)

With a list or sequence, returns both the length of an entry (including the terminating \0 character) and the entry itself at the given file `position` as two values (an integer and a string).

With a database, returns the length of the key including the terminating \0 and the key itself, but not the other entries referred to by the key.

The function is safe, so if you try to access an invalid file position, the function will exit returning **fail**. It will issue an error if the file represents a sequence.

See also: **ads.index**, **ads.retrieve**.

ads.read (filehandle, key)

With databases, the function returns the entry (one or more strings) to the given `key` (also a string). With lists and sequences, the function will return **true** if it finds the key, which may be a string or number, and **false** otherwise.

If the file is not open, read will return **fail**. If the base is empty, **null** will be returned. With databases and lists, the function uses binary search. With sequences the read will be linear.

See also: **ads.find**, **ads.getall**, **ads.iterate**, **ads.write**.

ads.remove (filehandle, key)

With databases, the function deletes a key-value pair from the database referred to by its `filehandle` with the given `key`, a string; with lists, the key is deleted. Physically, only the key to the record is deleted, the data itself will still reside in the record section but cannot be found any longer.

The function returns **true** if it could delete the data set, and **false** if the record to be deleted was not found. If the file is not open, delete will return **fail**. The function will issue an error if the file represents a sequence.

ads.retrieve (filehandle, position)

Gets a key and its value(s) from a database or list referred to by `filehandle` at the given file `position`, an integer.

With a database, the return is a table with the key and the values. With lists, only the key will be returned as a string.

The function is safe, so if you try to access an invalid file position, the function will exit and return **fail**.

If the file is not open, the function will return **fail.** The function will issue an error if the file represents a number or string sequence.

See also **ads.indices**, **ads.invalids**, **ads.peekin**.

ads.sizeof (filehandle)

Returns the number of valid records (an integer) and the maximum capacity (an integer) in the base pointed to be `filehandle`. If the base pointed to by the `filehandle` is not open, the function will return **fail**.

See also: **ads.attrib**, **ads.free**.

ads.sync (filehandle)

Flushes all unwritten content to the base file referred to by `filehandle`. The function returns **true** if successful, and **fail** otherwise (e.g. if the file was not opened before or an error occurred during flushing).

See also: **ads.write**.

ads.unlock (filehandle)

ads.unlock (filehandle, size)

The function unlocks the file given by its handle `filehandle` so that it can be read or overwritten by other applications again. For more information, see **ads.lock**.

ads.write (filehandle, key [, value1, value2, ···])

With databases, the function writes the `key` (a string) and the `value`s (strings, numbers, Booleans or Agena sequences) to the database file pointed to by filehandle (an integer). Numbers and Booleans are automatically converted to strings before writing. Agena sequences should contain one or more integers in the range 0 to 255 representing bytes which will then be written to the file. This allows for numbers, for example, to be stored in a binary way. The values may be of almost any length, only being dependent on available disk space.

Also with databases, if the third and/or following arguments are omitted, then empty strings will be written. If you pass more values than there are columns in the database, then surplus values will be ignored.

With lists, the function writes only the key (a string) to the database file. If you pass values, an error will be issued. If the key already exists, nothing will be written or done and **true** will be returned. Thus, lists never contain invalid records.

In both cases, the index section is updated. If a key already exists, its position in the index section will be replaced by the new index position (in this case there is no re-shifting). This does not remove the actual key-value pair in the record section. The function always writes the new key-value pair to the end of the file. (The file position after the write operation has completed is always 0.)

If the maximum number of eligible records is exceeded, the function issues an error.

With a string sequence the function writes any string `key`, `value1`, etc. to the file. The strings can be of any length, and may be empty.

With a number sequence the function writes the Agena number `key`, `value1`, etc. to the file.

ads.write will return **true** if successful. If the file is not open, the function will return **fail**.

See also: **ads.getall**, **ads.read**, **ads.sync**, **tostring**, **tostringx**, **bytes.tobytes**, **bytes.tonumber** and **strings.tobytes**.

12.5 xml - XML Parser

Since the XML package actually is the LuaExpat binding with some few Agena-specific modifications, large portions of this subchapter have been taken from the LuaExpat documentation.

12.5.1 Introduction

XML/LuaExpat is a SAX XML parser based on the Expat library. SAX is the Simple API for XML and allows programmes to:

- process a XML document incrementally, thus being able to handle huge documents without memory penalties;

- register handler functions which are called by the parser during the processing of the document, handling the document elements or text.

With an event-based API like SAX the XML document can be fed to the parser in chunks, and the parsing begins as soon as the parser receives the first document chunk. XML/LuaExpat reports parsing events (such as the start and end of elements) directly to the application through callbacks. The parsing of huge documents can benefit from this piecemeal operation.

XML/LuaExpat is distributed as a library.

12.5.2 Parser objects

Usually SAX implementations base all operations on the concept of a parser that allows the registration of callback functions. XML/LuaExpat offers the same functionality but uses a different registration method, based on a table of callbacks.

This table contains references to the callback functions which are responsible for the handling of the document parts. The parser will assume no behaviour for any undeclared callbacks.

12.5.3 Shortcuts

xml.decode (str)

Reads a string `str` containing an XML stream and converts it into a dictionary. Its return is rather raw, but it can cope with cases where one and the same XML object is present multiple times on the same hierarchy.

xml.decodexml (str)

Reads a string `str` containing an XML stream and converts it into a dictionary.

The function provides some checking (basic syntax and balanced tags), and supports namespaces, XML and DOCTYPE declarations, comments and processing instructions. If a XML tag includes hyphens or colons, then they are converted to underscores in the corresponding Agena dictionary key.

The data must be included in an envelope.

The function also returns processing instructions in the `xattr` tag.

The function is written in Agena and included in the lib/library.agn file.

See also:.**utils.decodexml**, **xml.decode**.

xml.readxml (filename)

Reads an XML file and returns its data in an Agena dictionary. The data must be included in an envelope.

See also: **utils.readcsv**, **utils.readxml**, **xml.decode**, **xml.decodexml**.

12.5.4 Constructor

xml.new (callbacks [, separator])

The parser is created by a call to the function xml.new, which returns the created parser or raises a Lua error. It receives the callbacks table and optionally the parser separator character used in the namespace expanded element names.

12.5.5 Functions

xml.close (parser)

Closes the parser, freeing all memory used by it. A call to close(parser) without a previous call to parse(parser) could result in an error.

The function is also available as an OOP method, see Chapter 6.24 about its use.

xml.getbase (parser)

Returns the base for resolving relative URIs. The function is also available as an OOP method, see Chapter 6.24 about its use.

xml.getcallbacks (parser)

Returns the callbacks table. The function is also available as an OOP method, see Chapter 6.24 about its use.

xml.parse (parser, s)

Parse some more of the document. The string s contains part (or perhaps all) of the document. When called without arguments the document is closed (but the parser still has to be closed).

The function returns a non **null** value when the parser has been successful, and when the parser finds an error it returns five results: **null**, msg, line, col, and pos, which are the error message, the line number, column number and absolute position of the error in the XML document.

The function is also available as an OOP method, see Chapter 6.24 about its use.

xml.pos (parser)

Returns three results: the current parsing line, column, and absolute position.

The function is also available as an OOP method, see Chapter 6.24 about its use.

xml.setbase (parser, base)

Sets the base to be used for resolving relative URIs in system identifiers.

The function is also available as an OOP method, see Chapter 6.24 about its use.

xml.setencoding (parser, encoding)

Sets the encoding to be used by the parser. There are four built-in encodings, passed as strings: 'US-ASCII', 'UTF-8', 'UTF-16', and 'ISO-8859-1'.

The function is also available as an OOP method, see Chapter 6.24 about its use.

12.5.6 Callbacks

The Agena callbacks define the handlers of the parser events. The use of a table in the parser constructor has some advantages over the registration of callbacks, since there is no need for the API to provide a way to manipulate callbacks.

Another difference lies in the behaviour of the callbacks during the parsing itself. The callback table contains references to the functions that can be redefined at will. The only restriction is that only the callbacks present in the table at creation time will be called.

The callbacks table indices are named after the equivalent Expat callbacks:

CharacterData, Comment, Default, DefaultExpand, EndCDataSection, EndElement, EndNamespaceDecl, ExternalEntityRef, NotStandalone, NotationDecl, ProcessingInstruction, StartCDataSection, StartElement, StartNamespaceDecl, and UnparsedEntityDecl.

These indices can be references to functions with specific signatures, as seen below. The parser constructor also checks the presence of a field called _nonstrict in the callbacks table. If _nonstrict is absent, only valid callback names are accepted as indices in the table (Defaultexpanded would be considered an error for example). If _nonstrict is defined, any other fieldnames can be used (even if not called at all).

The callbacks can optionally be defined as **false**, acting thus as placeholders for future assignment of functions.

Every callback function receives as the first parameter the calling parser itself, thus allowing the same functions to be used for more than one parser for example.

```
callbacks.CharacterData = proc(parser, string)
```

Called when the parser recognises an XML CDATA string.

```
callbacks.Comment = proc(parser, string)
```

Called when the parser recognises an XML comment string.

```
callbacks.Default = proc(parser, string)
```

Called when the parser has a string corresponding to any characters in the document which wouldn't otherwise be handled. Using this handler has the side effect of turning off expansion of references to internally defined general entities. Instead these references are passed to the default handler.

callbacks.DefaultExpand = proc(parser, string)

Called when the parser has a string corresponding to any characters in the document which wouldn't otherwise be handled. Using this handler doesn't affect expansion of internal entity references.

callbacks.EndCdataSection = proc(parser)

Called when the parser detects the end of a CDATA section.

callbacks.EndElement = proc(parser, elementName)

Called when the parser detects the ending of an XML element with elementName.

callbacks.EndNamespaceDecl = proc(parser, namespaceName)

Called when the parser detects the ending of an XML namespace with namespaceName. The handling of the end namespace is done after the handling of the end tag for the element the namespace is associated with.

callbacks.ExternalEntityRef = proc(parser, subparser, base, systemId, publicId)

Called when the parser detects an external entity reference.

The subparser is a XML/LuaExpat parser created with the same callbacks and Expat context as the parser and should be used to parse the external entity.

The base parameter is the base to use for relative system identifiers. It is set by setbase and may be **null**.

The systemId parameter is the system identifier specified in the entity declaration and is never **null**.

The publicId parameter is the public id given in the entity declaration and may be **null**.

callbacks.NotStandalone = proc(parser)

Called when the parser detects that the document is not `standalone`. This happens when there is an external subset or a reference to a parameter entity, but the document does not have standalone set to "yes" in an XML declaration.

callbacks.NotationDecl =
proc(parser, notationName, base, systemId, publicId)

Called when the parser detects XML notation declarations with notationName.

The base parameter is the base to use for relative system identifiers. It is set by setbase and may be **null**.

The systemId parameter is the system identifier specified in the entity declaration and is never **null**.

The publicId parameter is the public id given in the entity declaration and may be **null**.

callbacks.ProcessingInstruction = proc(parser, target, data)

Called when the parser detects XML processing instructions. The target is the first word in the processing instruction. The data is the rest of the characters in it after skipping all whitespace after the initial word.

callbacks.StartCdataSection = proc(parser)

Called when the parser detects the begining of an XML CDATA section.

callbacks.StartElement = proc(parser, elementName, attributes)

Called when the parser detects the begining of an XML element with elementName.

The attributes parameter is a table with all the element attribute names and values. The table contains an entry for every attribute in the element start tag and entries for the default attributes for that element.

The attributes are listed by name (including the inherited ones) and by position (inherited attributes are not considered in the position list).

As an example if the book element has attributes author, title and an optional format attribute (with `printed` as default value),

<book author=\"Ierusalimschy, Roberto\" title=\"Programming in Lua\">

would be represented as

```
[1 ~ 'author',
 2 ~ 'title',
 author ~ 'Ierusalimschy, Roberto',
 format ~ 'printed',
 title ~ 'Programming in Lua']
```

```
callbacks.StartNamespaceDecl = proc(parser, namespaceName)
```

Called when the parser detects an XML namespace declaration with namespaceName. Namespace declarations occur inside start tags, but the StartNamespaceDecl handler is called before the StartElement handler for each namespace declared in that start tag.

```
callbacks.UnparsedEntityDecl =
   proc(parser, entityName, base, systemId, publicId, notationName)
```

Called when the parser receives declarations of unparsed entities. These are entity declarations that have a notation (NDATA) field.

As an example, in the chunk

<!ENTITY logo SYSTEM "images/logo.gif" NDATA gif>

entityName would be "logo", systemId would be "images/logo.gif" and notationName would be "gif". For this example the publicId parameter would be **null**. The base parameter would be whatever has been set with setbase. If not set, it would be **null**.

The separator character:

The optional separator character in the parser constructor defines the character used in the namespace expanded element names. The separator character is optional (if not defined the parser will not handle namespaces) but if defined it must be different from the character '\0'.

12.6 json - JSON Structures

The json package encodes an Agena table to a string representing a JSON object and can also decode a JSON object represented by a string to an Agena table:

```
> data := [
>    'city' ~ 'Tel Aviv',
>    'iscapital' ~ false,
>    'founded' ~ 1909,
>    'details' ~ ['population' ~ 467875, 'area' ~ 52]
> ]

> s := json.encode(data):
{"details":{"population":467875,"area":52},"iscapital":false,
  "city":"Tel Aviv","founded":1909}

> json.decode(s):
[city ~ Tel Aviv, details ~ [area ~ 52, population ~ 467875],
founded ~ 1909, iscapital ~ false]      95
```

The package exposes the functions **json.encode** and **json.decode** presented above, and the `constant` **json.nil** to represent JSON's null value, for example:

```
> json.nil:
null

> void := [ 'nil' ~ json.nil ]:
[nil ~ null]

> s := json.encode(void):
{"nil":null}

> d := json.decode(s):
[nil ~ null]
```

Technically, **json.nil** is a userdata structure representing no data, with a metatable attached for garbage collection, user-defined type checking and pretty-printing and a user-defined type "null". You can check for JSON's null value as follows:

```
> type d.nil, typeof d.nil:
userdata        null
```

or, alternatively:

```
> d.nil :: 'null', d.nil -: 'null':
true   false
```

Actually a reference into memory, you can safely use the constant as often as needed - there is no danger of exhausting memory.

12.7 tar - UNIX tar

As a *plus* package, the tar package is not part of the standard distribution and must be activated with the **import** statement, e.g. `import tar`.

12.7.1 Introduction

This package lists, reads, and extracts individual files from a UNIX tar archive.

See also: **gzip** package.

12.7.2 Functions

`tar.close (fh)`

Closes an archive archived file denoted by its file handle `fh` and returns **true** on success and **false** otherwise.

The function is written in Agena (see lib/tar.agn).

`tar.extract (fn [, pattern])`

Extracts files, directories, and symbolic links from the given tar archive `fn`, a file name of type string to the given current working directory. By default, all files are extracted. If a second argument `pattern` is given, then only the files matching the given pattern - a string - are copied. `pattern` may include wildcards, see **strings.glob**.

The return is a table of all the files extracted.

The function is written in Agena (see lib/tar.agn).

`tar.lines (fh, length)`

Creates an iterator function that with each call returns a new line of a file included in a tar file. The `length` (in bytes) of the archived file pointed to by `fh` must be given as the second argument.

`fh` is a numeric file handle returned by calling **tar.open**. Since **tar.open** also returns the length as a second return, it can be easily passed to **tar.lines**.

If the end of the archived file has been reached, the iterator function returns **null**. The iterator does not close the file connection, use **tar.close** to accomplish this.

The function is written in Agena (see lib/tar.agn).

tar.list (fn [, pattern])

Returns all files in the UNIX tar file fn (a file name), and returns a table of tables with the following information:

- file name (key `'name'`),
- file mode (key `'mode'`),
- start position (key `'start'`, expressed as the offset to the beginning of the file),
- file length in bytes (key `'length'`),
- file timestamp in UNIX time (key, `'timestamp'`, decimal number of seconds since the start of a given epoch, use `os.date` to convert it into calendar date/time),
- ustar indicator (`'ustar'` if set, else the empty string),
- numeric owner id (key `'ownerid'`, decimal),
- numeric group id (key `'groupid'`, decimal),
- and the decimal checksum (key `'checksum'`).

If a second argument `pattern` is given, then only the files matching the given pattern - a string - are returned. `pattern` may include wildcards, see **strings.glob**.

The function is written in Agena (see lib/tar.agn).

tar.open (tarfile, fn)

Opens an archived file `fn` (a file name) in the tar file given by `tarfile` (also a file name), sets the file pointer to the beginning of the actual contents of the archived file (i.e. not its tar header), and returns both a numeric file handle to the archived file and its size.

The function is written in Agena (see lib/tar.agn).

12.8 gzip - Library to Read and Write UNIX gzip Compressed Files

As a *plus* package, in Solaris, Linux, Mac OS X, OS/2, DOS, and Windows, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import gzip`. See also: **tar** package.

A typical session may look like this:

```
> import gzip;

> fd := gzip.open('myfile.gz', 'r'):
gzipfile(0096A9F8)

>for keys i in gzip.lines(fd) do print(i) od;

> gzip.close(fd):
true
```

The following **gzip** package functions can also be called OOP-style: **close**, **read**, **lines**, **write**, **seek**, **sync**.

gzip.close (filehandle [, filehandle, ···])

Closes the files denoted by the given file handles.

The function is also available as an OOP method, see Chapter 6.24 about its use.

gzip.deflate (str [, level] [, stats])

The function compresses the string `str` with compression `level`, an integer in the range 1 .. 9, with 9 the default. The return is the compressed string and its size, in this order.

If `stats` is **true**, then the compression rate, size of `str`, number of allocated bytes, and number of 16K blocks used are returned, too, in this order. The compression rate is determined by dividing the size of the deflated string by the size of `str` with correction for short strings, where 1 means no compression.

See also: **gzip.inflate**.

gzip.inflate (str [, n])

The function decompresses a inflated string `str`. The size of the original uncompressed string is given as the second argument `n`, which by default is size(`str`) + 16383)/16384. The return is the uncompressed string.

See also: **gzip.deflate**.

gzip.lines (filehandle)

gzip.lines (filename)

Returns an iterator function that, each time it is called, returns a new line from the file. Therefore, the construction

```
for keys line in gzip.lines(file) do ... od
```

will iterate over all lines of the file.

If a file name is given, the file is closed when the loop ends. If a file handle is given, the file is not closed.

The function is also available as an OOP method, see Chapter 6.24 about its use.

gzip.open (filename [, mode])

Opens a file name. If mode is not given, a default mode `'rb'` will be used. `mode` can include special modes such as characters `'1'` to `'9'` that will be treated as the compression level when opening a file for writing.

It returns a new file handle, or, in case of errors, **null** plus an error message.

gzip.read (filehandle, format$_1$, ···)

Reads the file with the given file handle, according to the given formats, which specify what to read. For each format, the function returns a string with the characters read, or **null** if it cannot read data with the specified format. When called without formats, it uses a default format that reads the entire next line (see below).

The available formats are:

- '*a' reads the whole file, starting at the current position. On end of file, it returns the empty string.
- '*l' reads the next line (skipping the end of line), returning **null** on end of file. This is the default format.
- number reads a string with up to that number of characters, returning **null** on end of file. If number is zero, it reads nothing and returns an empty string, or **null** on end of file.

Unlike **io.read**, the '*n' format is not available.

The function is also available as an OOP method, see Chapter 6.24 about its use.

gzip.seek (filehandle [, whence] [, offset])

Sets and gets the file position, measured from the beginning of the file, to the position given by `offset` plus a base specified by the string `whence`, as follows:

- `'set'` base is position 0 (beginning of the file),
- `'cur'` base is current position,
- `'end'` is the end of the file.

In case of success, **seek** returns the final file position, measured in bytes from the beginning of the file. If this function fails, it returns **null**, plus a string describing the error.

The default value for `whence` is `'cur'`, and for `offset` is 0. Therefore, the call **gzip.seek**(`filehandle`) returns the current file position, without changing it; the call **gzip.seek**(`filehandle`, `'set'`) sets the position to the beginning of the file (and returns 0); and the call **gzip.seek**(`filehandle`, `'end'`) sets the position to the end of the file, and returns its size.

The function is also available as an OOP method, see Chapter 6.24 about its use.

gzip.sync (filehandle)

This function takes a file handle and flushes all output to the working file.

The function is also available as an OOP method, see Chapter 6.24 about its use.

gzip.write (filehandle, value$_1$, ···)

Writes the value of each of its arguments to the file specified by `filehandle`. The arguments must be strings or numbers. To write other values, use **tostring** or **strings.format** before **write**.

The function is also available as an OOP method, see Chapter 6.24 about its use.

12.9 minizip - Library to Read and Write ZIP Compressed Files

As a *plus* package, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import minizip`.

The package is a binding to the miniz library.

Generally, access to a ZIP archive to be read or written is done by file handles, with the actual read and write operations done by methods. There are a lot of procedures which do the job, as well, see below.

There are read streams, denoted by ZipReader handles, and write streams, denoted by ZipWriter handles.

In the following example, we read the contents of the file 'agena.c' from the archive 'd:/myzip.zip':

```
> import minizip;

> zr := minizip.open('d:/myzip.zip');
```

We need the index of the file to be read:

```
> idx := zr@@index('agena.h'):
2

> zr@@extract(idx):
/*
** $Id: agena.h, v 1.218 2009/03/01 15:38:27 $
...
```

Alternatively, we might pass the filename directly.

```
> zr@@extract('agena.c'):
/*
** $Id: lua.c,v 1.160 2006/06/02 15:34:00 roberto Exp $
...
```

Close the file after operation.

```
> zr@@close();
```

In the next example, we write some content to an archive:

```
> import minizip

> zr := minizip.open('d:/mytest.zip', 'w');

> zr@@add('text.txt', 'contents');

> zr@@finalise();
```

```
> zr@@close():
true
```

The package functions are:

minizip.adler32 (str [, init])

Calculates the adler32 checksum for string `str`. Without `init`, a number, the function returns an initialized sum.

See also: **hashes.adler32**, **minizip.crc32**.

minizip.attribs (fd, idx)

Retrieves information on an archived file, referred to by its integral index `idx`, in the ZIP file denoted by its handle `fd`. See **ZipReader@@stat** for more details.

minizip.close (fd)

Closes an open ZIP file, regardless whether it is in read or write mode. `fd` is the ZIP handle returned previously by **minizip.open**. Returns **true** on success and **false** otherwise.

Example:

```
> import minizip;

> zr := minizip.open('d:/myzip.zip')

> minizip.close(zr):
true
```

See also: **minizip.finalise**.

minizip.compress (str [, level [, windowsize]])
minizip.compress ([level [, windowsize]])

In the first form, compresses string `str` and returns a string. If `windowsize`, a number, is negative, the result will not include a zlib-compatible header. The function also returns a boolean indicating success, plus the number of characters in `str` and in the result.

Example:

```
> minizip.compress('agena'):
x£??· agena?=?² true    5       16
```

In the second form, returns a stream handle, a userdata.

See also: **minizip.decompress**.

minizip.count (fd)

Returns the number of files in a ZIP file. `fd` is the ZIP handle, Example:

```
> import minizip;

> zr := minizip.open('d:/myzip.zip')

> minizip.count(zr):
2
```

Same as:

```
> size zr:
2
```

minizip.crc32 (str [, init])

Calculates the crc32 checksum for string `str`. Without `init`, a number, the function returns an initialized sum.

See also: **hashes.crc32**, **minizip.adler32**.

minizip.decompress (str [, windowbits])
minizip.decompress (n)

In the first form, decompresses string `str` and returns a string. The function also returns a boolean indicating success, plus the number of characters in `str` and in the result.

```
> c := minizip.compress('agena'):
x£??· agena?=?²

> minizip.decompress(c):
agena   true    16      5
```

When given the empty string, the function just returns it.

In the second form, with `n` a number, returns a decompressor stream.

See also: **minizip.compress**.

minizip.finalise (fd)

Finalises the creation of a ZIP file denoted by it ZIP handle `fd`, the one returned previously by **minizip.open**, by flushing the content, building the Central Directory, writing the End of Central Directory Record (EOCD), and fixing all internal offsets and sizes.

You cannot add further content after calling this method. Use **minizip.close** to finally close the file.

minizip.index (fd, filename)

Returns the integral index of the given file, referred by the string `filename`, from the ZIP archive denoted by its handle `fd`, the one previously returned by **minizip.open**.

In the following example, there are two files in the ZIP file, 'agena.c' and 'agena.h':

```
> import minizip;

> zr := minizip.open('d:/myzip.zip')

> minizip.index(zr, 'agena.c'):
1

> minizip.index(zr, 'agena.h'):
2
```

You can use the index to get the filename:

```
> zr[1]:
agena.c

> zr[2]:
agena.h
```

minizip.isdir (fd, idx)

Checks whether the archived file, referred by ZIP handle `fd`, with index `idx` is a directory and returns **true** or **false**.

Example:

```
> import minizip;

> zr := minizip.open('d:/myzip.zip')

> idx := minizip.index(zr, 'agena.c')

> minizip.isdir(zr, idx):
false
```

minizip.iszip (filename)

Checks whether file referred to by filename, a string, is a ZIP archive and returns **true** or **false**.

minizip.open (filename [, [mode] [, flags]])
minizip.open (filename [, [mode] [, inibytes]])

When called with just a `filename`, a string, opens the given ZIP archive in read mode and returns a file handle. The function issues an error if `filename` does not refer to a ZIP archive.

When called with a `filename`, a string, and a `mode`, also a string, opens the given ZIP archive in the following mode and returns a file handle:

- `mode` = 'r': read mode,
- `mode` = 'w': write mode.

In read mode, you may optionally specify one of the following `flags` as a third argument:

- 0x100: case sensitive file name in zip file,
- 0x200: ignore path of file in zip,
- 0x400: file is compressed data,
- 0x800: do not sort central directory in zip file.

In write mode, you can also specify the initial number of bytes `inibytes` to reserve by passing a non-negative integer as a third argument.

Let us open a ZIP archive for reading:

```
> import minizip

> za := minizip.open("d:/myzip.zip")

> for i, file in ipairs(za) do
>    print(i, file)
> end;
1  [bit_flag ~ 2, comment ~ , comp_size ~ 9694, crc32 ~ 501329235,
    external_attr ~ 32, filename ~ agena.c, index ~ 0, internal_attr ~ 0,
    method ~ 8, time ~ 1753524772, uncomp_size ~ 29739, version_made_by ~
    20, version_needed ~ 20]
2   [etc.]
```

As for the time attribute, the number of seconds elapsed since a given epoch, you may convert it to a date, as follows:

```
> os.date('%a, %d %b %Y %H:%M:%S, %z',1753524772):
Mon, 28 Jul 2025 19:01:46, W. Europe Summer Time
```

Let us create a new ZIP file called 'd:/mytest.zip', add a file 'text.txt' to the archive with string 'contents' and finally close the file.

```
> import minizip

> zr := minizip.open('d:/mytest.zip', 'w');

> zr@@add('text.txt', 'contents');

> finalise(zr);

> close(zr):
true
```

See also: **minizip.close**.

minizip.read (fd, idx)
minizip.read (fd, filename)

Reads a file from the zip archive denoted by its ZIP handle `fd`, the one previously returned by **minizip.open**. You can either pass the name `filename`, a string, of the archived file to be read, or its integral index `idx`, see **minizip.index**.

Example: Let us read the file 'agena.h' from the ZIP archive 'd:/myzip'.

```
> import minizip;

> zr := minizip.open('d:/myzip.zip')

> idx := minizip.index(zr, 'agena.h')

> minizip.read(zr, idx):
/*
** $Id: agena.h, v 1.218 2009/03/01 15:38:27 $
...

> minizip.read(zr, 'agena.c'):
/*
** $Id: lua.c,v 1.160 2006/06/02 15:34:00 roberto Exp $
...

> minizip.close(zr);
```

See also: **ZipReader@@extract**, **ZipReader@@rawfd**.

minizip.write (fd, filename, contents)
minizip.write (fd, filename, file = infile)

In the first form, adds the file `filename`, a string, with the given `contents`, also a string, to the open ZIP file denoted by its handle `fd`.

Let us write the word 'contents' to the archive file 'text.txt' in ZIP file 'mytest.zip':

```
> import minizip

> zr := minizip.open('d:/mytest.zip', 'w');

> minizip.write(zr, 'text.txt', 'contents');

> minizip.close(zr):
true
```

In the second form, with the **file** option, adds the file `filename`, a string, to the open ZIP file denoted by its handle `fd`. The contents is read from the file system, with path `infile`, a string.

```
> import minizip;

> fd := minizip.open('d:/new.zip', 'w');

> minizip.write(fd, 'agena.h', file = 'c:/agena/src/agena.h');
```

```
> minizip.close(fd);
```

See also: **ZipWriter@@add**, **ZipWriter@@addfile**, **ZipWriter@@addfrom**.

The following methods for reading are available:

ZipReader@@close ()

Closes an open ZIP file. Returns **true** on success and **false** otherwise.

Example:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'))

> zr@@close():
true
```

See also: **ZipWriter@@finalise**.

ZipReader@@extract (idx)
ZipReader@@extract (filename)

Extracts a file from a zip archive. You can either pass the name `filename`, a string, of the ZIP archive file to be read, or its integral index `idx`, see **ZipReader@@index**.

Example:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'))

> idx := zr@@index('agena.h')

> zr@@extract(idx):
/*
** $Id: agena.h, v 1.218 2009/03/01 15:38:27 $
...

> zr@@extract('agena.c'):
/*
** $Id: lua.c,v 1.160 2006/06/02 15:34:00 roberto Exp $
...

> zr@@close();
```

ZipReader@@index (filename)

Returns the integral index of the given file, referred by the string `filename`. In the following example, there are two files in the ZIP file, 'agena.c' and 'agena.h':

```
> import minizip;
```

```
> zr := assume(minizip.read('d:/myzip.zip'))

> zr@@index('agena.c'):
1

> zr@@index('agena.h'):
2
```

You can use the index to get the filename:

```
> zr[1]:
agena.c

> zr[2]:
agena.h
```

ZipReader@@isdir (idx)

Checks whether the file object with the given index is a directory and returns **true** or **false**.

Example:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'))

> idx := zr@@index('agena.c')

> zr@@isdir(idx):
false
```

ZipReader@@num ()

Returns the number of files in a ZIP file. Example:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'))

> zr@@num():
2
```

Same as:

```
> size zr:
2
```

ZipReader@@offset ()

Returns the start offset of the zip file and its compressed size. Example:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'))
```

```
> zr@@offset():
0       19953
```

ZipReader@@rawfd ()

Returns a handle on the raw contents of a ZIP file. Example:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'));

> fd := zr@@rawfd();
```

Current file position:

```
> fd@@seek(0, 'set'):
0
```

Read the first 32 bytes:

```
> header := fd@@read(32):
PK??¶Üa·ZS¡ß?Ì%
```

Check whether the first two bytes mark a ZIP file:

```
> header@@match('^PK'):
PK
```

ZipReader@@stat (idx)

Retrieves information on a file, referred to by its integral index idx, in a ZIP file. The return is a table with the following fields:

- `index`: index of file,
- `version_made_by`: zip version,
- `version_needed`: extract file need version,
- `bit_flag`: flags of file,
- `method`: compress method,
- `time`: file time,
- `crc32`: file crc32 checksum,
- `comp_size`: compressed size,
- `uncomp_size`: uncompressed size,
- `internal_attr`: internal attribute,
- `external_attr`: external attribute,
- `filename`: filename,
- `comment`: comment.

Example use:

```
> import minizip;

> zr := assume(minizip.read('d:/myzip.zip'))
```

```
> idx := zr@@index('agena.c')

> zr@@stat(idx):
[bit_flag ~ 2, comment ~ , comp_size ~ 9694, crc32 ~ 501329235,
 external_attr ~ 32, filename ~ agena.c, index ~ 0, internal_attr ~ 0,
 method ~ 8, time ~ 1753524772, uncomp_size ~ 29739, version_made_by ~ 20,
 version_needed ~ 20]
```

The methods for writing are:

ZipWriter@@add (filename, contents)

Adds the file referred to by `filename`, a string, to an open ZIP file with the given `contents`, a string.

Example:

```
> import minizip

> zr := minizip.write('d:/mytest.zip');

> zr@@add('text.txt', 'contents');

> zr@@close():
true
```

ZipWriter@@addfile (fullpath)
ZipWriter@@addfile (path, filename)
ZipWriter@@addfile (path, filename, comment, flags)

Adds a file to a ZIP file.

In the first form, the file denoted by its full path is added to an open ZIP archive. The full path is preserved in the archive. In the following example the file 'c:/agena/src/agena.c' is added to archive 'd:/mytest.zip', and you will find 'agena.c' in subdirectory 'agena/src'.

```
> import minizip

> zr := minizip.write('d:/mytest.zip');

> zr@@addfile('c:/agena/src/agena.c');

> zr@@close();
```

In the second form, the file denoted by filename is added with the given path. In the example, you will find 'agena.c' in subdirectory 'agena/src'.

```
> zr := minizip.write('d:/mytest.zip');

> zr@@addfile('c:/agena/src', 'agena.c');

> zr@@close();
```

In the next example, the file denoted by its full path 'c:/agena/src/agena.c' will be written to the root of the archive.

```
> zr := minizip.write('d:/mytest.zip');

> zr@@addfile('agena.c', 'c:/agena/src/agena.c');

> zr@@close();
```

ZipWriter@@addfrom (infile, idx)

Adds a file denoted by integral `idx` from the open read stream `infile`, a file handle, to the archive opened for writing.

```
> import minizip
```

Open the read file:

```
> zin := minizip.read('d:/infile.zip');
```

Get the index of the file we want to write:

```
> idx := zin@@index('agena.c')
```

Create a new ZIP archive:

```
> zout := minizip.write('d:/outfile.zip');
```

Copy file 'agena.c' to the new archive:

```
> zout@@addfrom(zin, idx);
```

Finally, close both streams:

```
> zin@@close(), zout@@close():
```

ZipWriter@@close ()

Finalises the creation of a ZIP file by flushing any unwritten content and closes it. Returns **true** on success and **false** otherwise.

Example:

```
> import minizip

> zr := minizip.write('d:/mytest.zip');

> zr@@close():
true
```

See also: **ZipWriter@@finalise**.

ZipWriter@@finalise ()

Finalises the creation of a ZIP file by flushing the content, building the Central Directory, writing the End of Central Directory Record (EOCD), and fixing all internal offsets and sizes.

You cannot add further content after calling this method. Use **ZipWriter@@close** to finally close the file.

12.10 ini - Library to Read and Create INI Files

The package is built-in and does not need to be initialised with the import statement.

The package processes conventional INI files as found in Pre-Windows 95 configuration files (win.ini for example).

An INI file may look like this:

```
; Pizza general
Taxi=Pizza Cab
State=
; Following is a section ...

[Pizza1]
; ... and now a key~value pair
Ham = yes
Mushrooms = true
Capres = 0
Cheese = "Non" ;
Gravy=false
Price = 3.99
Comment= This \
is a \
multiline string.
Preis=3,99
empty =

[Pizza2]
Ham = no
Mushrooms = false
Capres = -1
Cheese = "Oui" ;
Gravy=true
Price = 0.99
Comment= This \
is a \
second \
multiline string.
Preis=0,99
empty =
```

Internally, INI structures are represented by so-called "dictionaries" represented as userdata.

Sample usage:

Instantiate a dictionary:

```
> d := ini.new()
```

Fill it with data:

```
> ini.read(d, 'sample.ini')
```

Get key-value pairs in section 'Pizza':

```
> ini.getsection(d, 'Pizza', comma = true):
[capres ~ 0, cheese ~ Non, comment ~ This is a multiline string., empty ~ ,
 gravy ~ false, ham ~ yes, mushrooms ~ true, preis ~ 3.99, price ~ 3.99]
```

Get all key-value pairs that are not in any section:

```
> ini.getsection(d, 0):
[state ~ , taxi ~ Pizza Cab]
```

Get info on the dictionary, 'sections' denotes the number of sections, here we only have the `Pizza` section, alternatively execute: **ini.attrib**(d):

```
> d@@attrib():
[allocated ~ 128, assigned ~ 22, sections ~ 1]
```

Drop the dictionary, alternatively execute: **ini.close**(d):

```
> d@@close();
```

The following package functions can also called OOP-style if not indicated otherwise:

ini.attrib (d)

Returns information on dictionary `d`. The result is a table with the following key~value pairs:

- `'assigned'` ~ <an integer>: number of assigned slots,
- `'allocated'` ~ <an integer>: number of allocated slots´,
- `'sections'` ~ <an integer>: number of sections.

ini.close (d)

Empties and discards dictionary `d`, giving back memory to the interpreter. The function returns nothing.

See also: **ini.new**.

ini.dump (d [, options])

Puts all the contents of dictionary `d` into a well-formed table and returns it. `d` is left unchanged.

There are the following options:

- `convert` = **false**: Do not automatically convert strings representing numbers or Booleans to the respective data types. Default is **true**.
- `comma` = **true**: Convert a decimal comma in a string representing a number to a decimal dot. Default is **false**.

- `raw` = **true**: Instead of a table, returns a sequence. Section names are returned as simple strings, key-value pairs as pairs with the key name including the section name and the values as numbers (if the `convert` option has been set to **true**, the default) or strings. Strings representing booleans cannot be converted, they are simply returned as strings.

See also: **utils.readini**.

ini.getitem (d, section [, key] [, options])

The function locates section `section` or key `key` in dictionary `d` and in case of a key-value pair returns the associated value, in case of a section name returns **null** and if `key` could not be found at all explicitly issues an error. The function automatically converts `key` to lower case before searching the dictionary to avoid failures.

There are the following two options:

- `convert` = **false**: Do not automatically convert strings representing numbers or Booleans to the respective data types. Default is **true**.
- `comma` = **true**: Convert a decimal comma in a string representing a number to a decimal dot. Default is **false**.

See also: **ini.getsection**, **ini.setitem**.

ini.getsection (d [, n [, options]])
ini.getsection (d [, str [, options]])

Retrieves all the key-value pairs from dictionary `d` for the given section `n` or `str`. You can either pass the section as a non-negative number `n` or a string `str`.

The return is a table.

If `n` is 0 or `str` is the empty string, then all key-value pairs that are not in any section, if they exist at all, are returned.

If section number `n` or section name `str` do not exist or the dictionary is empty, the function returns **fail**.

There are the following two options:

- `convert` = **false**: Do not automatically convert strings representing numbers or Booleans to the respective data types. Default is **true**.
- `comma` = **true**: Convert a decimal comma in a string representing a number to a decimal dot. Default is **false**.

See also: **utils.readini**.

ini.hash (str [, n])

Computes the hash used internally by the ini package for string `str` and returns an integer. The hash function has been taken from an Article in Dr Dobbs Journal.

If `n`, a positive integer, is given, the computed hash is taken modulo `n`.

ini.new ([n])

Instantiates and returns a new dictionary structure as a Lua userdata.

By default, all the data is stored to so-called `buckets` of $n = 4$ slots each, so that data can speedily be accessed by the modulo of its hash value, and not by traversing each slot from left to right. You can choose another value for `n`, but it must be a multiple of 4. The larger `n`, the faster is the access, but also more memory is needed. As a rule of thumb, the more you store in a dictionary the larger `n` should be.

The function can not be used OOP-style.

See also: **ini.close**, **ini.read**.

ini.read (d, inifile)

Reads all the data in the ini file represented by string `inifile` into dictionary `d`. The function returns nothing.

See also: **ini.close**, **ini.new**, **ini.getsection**, **utils.readini**.

ini.setitem (d, section [, key, val])
ini.setitem (d, key, val)

The function sets data into dictionary `d`.

In the first form, with only `section` given, creates a new section. If `section`, `key` and `val` are given, the function stores a key-value pair to the given section.

In the second form, sets a key-value into the root of dictionary `d`, with no association to a section.

If a `key` is given and it already exists in `d`, then the associated value is replaced by `val`.

The function returns **true** on success and **false** otherwise.

See also: **ini.getitem**, **ini.unset**, **utils.writeini**.

ini.unset (d, section [, key])
ini.unset (d, key)

The function deletes data from dictionary `d`.

In the first form, with just `section` given, removes the section entry. Note that you must also delete all accompanying key-value pairs by calling **ini.unset** again. If both `section` and `key` are given, the function purges the associated key-value pair.

In the second form, removes the key-value pair that resides at the root of `d`.

The function returns **true** on success and **false** otherwise.

The function does not give back the freed space for future reassignment, so you may use the function carefully.

12.11 sqlite - Interfacing to SQLite Databases

The sqlite package provides an interfaces to SQLite 3 and creates, reads and writes databases. It can be initialised as follows:

```
> import sqlite;
```

For the documentation, see the `SQLite.htm` file in the `doc` folder of your Agena installation. You will also find a sample script `SQLite.agn` there.

The package is available in Windows, OS/2, DOS, Linux, Solaris and Mac OS X.

Hint: It is advised that you check the return code of every database operation and only proceed with the next SQL statement if it is **sqlite.OK**.

12.12 keyfile - Key Files

12.12.1 Introduction

The package is built-in and does not need to be initialised with the **import** statement.

The package provides functions to create, change and read key files.

A key file is a specialized database file format designed to provide fast, indexed access to data using a key-value architecture. Unlike a flat text file where you would have to scan the entire document to find a specific record, a key file functions more like an efficient digital index, mapping a unique `key` (such as an ID, a name, or a file path) directly to its associated `value`, that is the data, also called `record` here. You can associate one or more records with one unique key.

To manage the keys efficiently - especially when the file contains millions of records - the package uses a B-Tree (Balanced Tree) structure internally, providing logarithmic efficiency.

In the following example, we create a key file, write data to it, read content from it and finally close it. The key-value pairs are:

Keys	**Values**
Houston	TX
Portland	ON, ME and NSW
Chattanooga	TN

```
> keyfile.new('test.key');

> fh := keyfile.open('test.key');
```

Put the key file into transaction mode. This is optional:

```
> keyfile.start(fh);
```

Now add the key-value pairs:

```
> keyfile.write(fh, 'Houston', 'TX');
> keyfile.write(fh, 'Portland', 'ON');
> keyfile.write(fh, 'Portland', 'ME');
> keyfile.write(fh, 'Portland', 'NSW');
> keyfile.write(fh, 'Chattanooga', 'TN');
```

In transaction mode we mark all data to be physically written to the key file later on:

```
> keyfile.commit(fh, true);  # pass false to roll back
```

Now we flush the data to the file:

```
> keyfile.sync(fh);
```

Let's iterate over all the records with key 'Portland':

```
> index := 'Portland';

> record := null;

> while record := keyfile.read(fh, index, record) do
>    print(index, record)
> od;
Portland        ON
Portland        ME
Portland        NSW
```

We finally close the file.

```
> keyfile.close(fh):
```

The package provides the following metamethods:

Metamethod	Functionality
'__size'	**size** operator, number of keys in the key file
'__in'	**in** operator (literal comparison only)
'__notin'	**notin** operator (literal comparison only)
'__empty'	**empty** operator
'__filled'	**filled** operator
'__tostring'	formatting for output at the console
'__gc'	garbage collection

12.12.2 Functions

With the exception of **keyfile.new** and **keyfile.open**, all of the following package functions can be used OOP-style:

keyfile.allkeys (h [, f] [, true] [, container])
keyfile.allkeys (h, pattern [, true] [, container])

In the first form, returns all the keys in the key file denoted by its handle `h`. The result is a table of strings. If one and the same key is existing in `h` n times, then by default the resulting table will also include the key n times. By passing the very last argument **true**, however, a key will only be included once.

If you pass a second argument, a function `f` returning a boolean, then **keyfile.allkeys** only returns those keys where the call to `f` evaluates to **true**.

In the second form, the function searches all keys for substring `pattern` and returns them. Pattern matching is supported, see Chapter 9.1.3.

Optional `container` represents a table, set, sequence or register in which to store the result. The keys are just added, meaning that if the structure already includes elements, they will be preserved.

Example for filtering all keys starting with minor 'a':

```
> h@@allkeys(<: x -> x[1] = 'a' :>):
```

And equally:

```
> h@@allkeys('^a'):
```

See also: **keyfile.has**.

keyfile.count (h [, f] [, true])
keyfile.count (h, pattern [, true])

In the first form, counts all the keys in the key file denoted by its handle `h`. The return is a number. If one and the same key is existing in `h` n times, then by default the resulting count will also include the key n times. By passing the very last argument **true**, however, a key will be counted only once.

If you pass a second argument, a function `f` returning a boolean, then **keyfile.count** only counts those keys where the call to `f` evaluates to **true**.

In the second form, the function counts all keys that match substring `pattern`. Pattern matching is supported, see Chapter 9.1.3.

In both forms, the count is linear and not constant.

keyfile.close (h [, ···])

Closes one or more key files denoted by their file handles `h`, etc. The function returns **true** on success and issues an error otherwise. The function also returns **true** of at least one of the handles has already been closed.

If you have started transaction mode before (see **keyfile.start**) and have not already committed a transaction (see **keyfile.commit**), then **keyfile.close** will not flush unwritten contents to the key file before closing the key file.

If you are in default non-transactional mode, however, closing a file will automatically flush any unwritten content to the file.

See also: **keyfile.open**.

keyfile.commit (h [, mode])

If `mode` is **true**, flags all previous **keyfile.write** operations to be actually written to the key file denoted by `h` later on. Call **keyfile.sync** to actually flush unwritten content to the file.

If `mode` is **false**, rolls back all previously unsaved **keyfile.write** calls so they will not be actually written finally.

The function returns **true** on success and **false** otherwise.

Before adding content with **keyfile.write** you must have invoked **keyfile.start** to put `h` into transaction mode, otherwise the function may issue errors.

keyfile.has (h, key)

Checks whether the key file denoted by its handle `h` contains the key `key` (a string) and returns **true** or **false**. The function does *not* support pattern matching.

keyfile.isopen (h)

Checks whether the first argument `h`, usually a handle, represents an open key file, and returns **true** or **false**.

keyfile.iterate (h, flag)

Iterates an entire key file denoted by its handle `h`. To start the iteration, pass **null** for `flag`, and pass any non-**null** argument for `flag` in succeeding calls to traverse towards the end of `h`.

The function returns the current key and corresponding record as two strings. If the end of file has been reached, that is there are no more keys left, the function returns **null** twice.

The traversal will be in ascending order of the keys.

Example:

```
> keyfile.new('test.key');

> h := keyfile.open('test.key');

> keyfile.write(h, 'Houston', 'TX');
> keyfile.write(h, 'Portland', 'ON');
> keyfile.write(h, 'Portland', 'ME');
> keyfile.write(h, 'Portland', 'NSW');
> keyfile.write(h, 'Chattanooga', 'TN');

> k := null;
```

```
> do
>    k, v := keyfile.iterate(h, k);
>    break when k = null
>    print(k, v)
> end;
Chattanooga     TN
Houston         TX
Portland        ON
Portland        ME
Portland        NSW

> keyfile.close(h);
```

keyfile.new (fn)

Creates a new key file for the given filename `fn`. It returns nothing if successful and issues an error otherwise.

See also: **keyfile.open**.

keyfile.open (fn [, mode])

Opens the key file denoted by its filename `fn`, a string. The default mode is read-and-write mode. You can open a key file in read-only mode by passing one of the strings `'read'` or `'r'` as a second argument.

The return is the file handle needed to conduct any operation on a key file.

See also: **keyfile.new**, **keyfile.close**.

keyfile.purge (h)

Deletes the current record of the key file denoted by its handle `h`. You may first want to search for a specific record by calling **keyfile.read** one or more times, and if the search is successful invoke **keyfile.purge**.

See also: **keyfile.update**, **keyfile.write**.

keyfile.read (h, key [, flag])

The function searches for the record(s) stored with the given `key`, a string or a number convertable to a string, in the key file denoted by its handle `h`. If the key exists, the return is the record, a string. If `key` does not exist, or there are no more associated records to read, the function returns **null**.

To return all records associated with a given key, `flag` must be **null** at first invocation (the default), and non-**null** with any subsequent call.

With our example database created for **keyfile.iterate**, see above, we have:

```
> fh@@read('Portland'):
ME

> fh@@read('Portland', 'anything'):
NSW

> fh@@read('Portland', 'anything'):
ON

> fh@@read('Portland', 'anything'):
null
```

See also: **keyfile.write**.

keyfile.start (h)

Invokes transaction mode for the key file denoted by its handle `h` and returns **true** on success and **false** otherwise. You must call this function in order to commit or roll back transactions with **keyfile.commit** later on.

See also: **keyfile.sync**.

keyfile.sync (h)

Writes any unwritten content to the key file denoted by its handle `h` and returns **true** on success and **false** otherwise.

If you have started transaction mode before (see **keyfile.start**) and have not already committed a transaction (see **keyfile.commit**), then **keyfile.sync** will not flush unwritten contents to the key file.

keyfile.update (h, data)

Updates the current record of the key file denoted by its handle `h`. `data` must be a string and represents the new, updated record. If `data` is the empty string, the record will be deleted. Note that the function does not update a key.

You may first want to search for a specific record by calling **keyfile.read** one or more times, and if the search is successful invoke **keyfile.update**.

The function returns **true** on success and **false** otherwise.

See also: **keyfile.purge**, **keyfile.write**.

keyfile.write (h, key, data)

The function writes the record `data`, a string, along with its associated `key`, a string, to the key file denoted by its handle `h`. If `key`, `data` are numbers, they are automatically converted to strings before writing them to the file.

The `key` must not be the empty string or consist of more than 255 characters. There are no limits on the length of `data`.

`data` can also be an Agena sequence which should contain one or more integers in the range 0 to 255 representing bytes. This allows for numbers, for example, to be stored in a binary way. Example:

```
> keyfile.new('test.key');

> fh := keyfile.open('test.key');

> keyfile.write(fh, 1, bytes.tobytes(Pi));

> bytes.tonumber(strings.tobytes(keyfile.read(fh, 1))):
3.1415926535898

> keyfile.close(fh);
```

Usually, the content is not immediately written to the file. Call **keyfile.sync** to force a flush. Alternatively, first invoke transaction mode for `h` with a call to **keyfile.start**, then run **keyfile.write** operations and call **keyfile.commit** plus **keyfile.sync**.

See also: **keyfile.purge**, **keyfile.read**, **keyfile.update**.

Chapter Thirteen

Communication

13 Communication

13.1 net - Network Library

As a *plus* package, in Solaris, Linux, Mac OS X, OS/2 and Windows, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import net`.

13.1.1 Introduction and Examples

This package provides basic functions to pass text from a client to a server using the IPv4 or IPv6 protocol. Thus it is suited to exchange information over the Internet and Local Area Networks.

Please remember that the package only supports unencrypted data transfer which might be insecure ! There is no SSL support.

If you do not use this package, no network functionality will be activated.

Please also note that when using *net.accept*, *net.connect*, *net.receive*, *net.send*, and *net.survey*, you will give access to your computer through LANs or the Internet, so please programme handshaking and blacklist/whitelist methods.

Limited white and blacklisting to allow or prohibit connections is supported through the ***net.whitelist*** and ***net.blacklist*** feature.

Communication is performed with `stream sockets` that ensure that data is sent and received in the original order and hopefully without errors. A socket is being created by a call to the **net.open** function.

In the following example, we will set up a one-way communication with the `client` sending and the `server` receiving data.

A typical session might begin by setting up the server. This is because a client cannot connect to a server until the latter is ready for it.

(OS/2 users might want to check the description of **net.open** below which explains how to successfully create a server or client socket.)

```
> import net
```

Create an IPv4 socket: the **net.open** function returns a new socket handle:

```
> s := net.open():
932
```

If you need an IPv6 socket, issue:

```
> s := net.open(ipv = 6):
932
```

From here on, regardless of whether we have an IPv4 or IPv6 socket, all the following functions work the same, you do not have to differentiate between protocols.

Now associate this socket with a port on the server machine[5] by running **net.bind**. In this example we expect data to be received on your own computer on port 1300.

```
> net.bind(s, '127.0.0.1', 1300):
127.0.0.1   1300
```

Now our socket must be converted to a server socket by calling

```
> net.listen(s):
true
```

and be told to get a pending connection by running **net.accept**.

net.accept waits until a client asks the server for a connection, see client example below. It returns a new socket handle which later on manages this specific connection, while the original socket is ready to wait for requests for another connection. **net.accept** also returns the IP address of the client asking for a connection, and its port.

```
> t, ip, port := net.accept(s):
924   127.0.0.1   3230
```

If you do not want **net.accept** to wait indefinitely until something happens, call **net.block** with *the original server socket and* ***false*** as its second argument.

Please note that you should check the incoming connection against a white or black list so that only trusted clients can send you any data. To decline and terminate an incoming connection, either check the incoming caller and just call **net.close** with the handle returned by **net.access**, or use the built-in basic black and whitelist functionality described at the end of this subchapter.

It also a good idea to validate the incoming connection with a handshaking procedure which checks the incoming data for certain information and then automatically decides whether to go on or shut down the connection.

Data received from the client will be returned by calling **net.receive** with the new file handle returned by **net.access**.

---

[5] You may use the operating system commands `ifconfig` (UNIX, Mac) or `ipconfig` (Windows) to determine your own IP address.

```
> net.receive(t):
Kuckuck !  9
```

Finally, close both sockets (or just the handle returned by **net.accept**):

```
> net.close(t, s):
true
```

To open a client session, start Agena in another shell:

```
> import net
```

To connect to a server, first issue:

```
> d := net.open():
932
```

or if you are using the IPv6 protocol:

```
> d := net.open(ipv = 6):
932
```

Now connect to the server by passing the socket handle, the IP address and port number of the server. `'localhost'` means that the server runs on the same machine as the client, regardless of the IPv4 or IPv6 protocol being used.

```
> net.connect(d, 'localhost', 1300):
true
```

Send some text once or more.

```
> net.send(d, 'Kuckuck !'):
9
```

The server immediately returns the text sent. To finish a client session, type:

```
> close(d):
true
```

Call **net.opensockets** to have a look at the state of all open sockets.

Following now is an extended but crude example for a one-way connection which sends one thousand hashes from the client to the server on the local host on port 1300.

Since with one single call, **net.receive** by default processes `only` 512 bytes in Windows and usually 8,192 bytes in UNIX, the server uses a **while** loop to receive all the data until the client closes the connection.

Since **net.receive** returns two results - the string and the number of characters received - its second return will be 0 if the client terminates a network session.

Server	Client
`> import net alias`	`> import net alias`
`> d := open():` `132`	`> d := open():` `352`
`> bind(d, 'localhost', 1300):` `127.0.0.1  1300`	`> connect(d, 'localhost', 1300):` `true`
`> listen(d):` `true`	
`> e, f, g := accept(d);`	
`> print(e, f, g);` `352  127.0.0.1  49178`	
`> x, y := receive(e);`	`> send(d, strings.repeat('#', 1m)):` `1000000`
`> print(x, y);` `##### (512 hashes) #####  512`	`> close(d):` `true`

`> while y <> 0 do`
`>  x, y := receive(e);`
`>  print(x, y);`
`> od;`
`##### (more hashes) ####  488`
`0`	

`> close(e, d):` `true`	

A simple bi-directional connection:

Server

```
> import net alias

> d := open():
124

> bind(d, 'localhost', 1300):
127.0.0.1       1300

> listen(d):
true

> e, f, g := accept(d);

> print(e, f, g);
344     127.0.0.1       49183

> x, y := receive(e);

> print(x, y);
## etc.          512

> send(e, 'Got ' & y & ' bytes');

> while y <> 0 do
>    x, y := receive(e);
>    print(x, y);
>    send(e,
>       'Got ' & y & ' bytes');
> od;
## etc.          488
        0

> close(e, d):
true
```

Client

```
> import net alias

> d := open():
124

> connect(d, 'localhost', 1300):
true

> send(d, strings.repeat('#', 1k)):
1000

> receive(d):
Got 512 bytes   13

> receive(d):
Got 488 bytes   13

> close(d):
true
```

Usage of black and whitelists: First initialise the **net** package.

```
> import net alias
```

Now put one or more a numeric (!) IPs to be blocked into the set **net.blacklist** to prohibit connections to these addresses (valid for both **net.connect** and **net.accept**).

```
> net.blacklist := {'127.0.0.1'}

> d := open():
3

> connect(d, '127.0.0.1', 1300):
Error in `net.connect`: partner in blacklist, closing socket 3.

Stack traceback: in `connect`
   stdin, at line 1 in main chunk
```

Socket d has now been closed:

```
> opensockets():
[]
```

Now define a whitelist with all IPs to which a connection is allowed.

```
> net.whitelist := {'127.0.0.2'}

> d := open():
3

> return connect(d, '127.0.0.3', 1300)
Error in `net.connect`: partner not in whitelist, closing socket 3.

Stack traceback: in `connect`
   stdin, at line 1 in main chunk
```

The socket is closed, as well.

```
> opensockets():
[]
```

13.1.2 Functions

net.accept (s)

Accepts a connection request from a client on the given server socket handle `s`. If the server socket has been set to blocking mode, it waits until there is an incoming connection.

The function returns a new socket handle (a number) for the data to be received later on, and the address (a string) and port (a number) of the client socket.

Please note that the new socket created by **net.accept** must be closed separately to avoid too many open sockets.

The function also checks the global sets **net.blacklist** and **net.whitelist**, in this order, and if they exist. If you are trying to accept a connect from an address that is included in **net.blacklist**, then **net.accept** refuses this connection, closes the new socket that it created (see above), and issues an error. If you are trying to accept a connection from an address that is not in **net.whitelist**, the function does not establish a connection, closes the freshly created socket, and issues an error, as well.

Please note that **net.blacklist** and **net.whitelist must only contain numeric IPs**, and not addresses like 'sunsite.abc.xyz'. However, **net.accept** tries to convert the incoming address to a numeric IP address and then checks both lists[6]. If an address could not be resolved, the function does not allow a connection, and closes the newly created socket, and finally issues an error.

You may use **protect** in order to intercept the errors described above, but you must take care yourself for allowing or prohibiting a connection.

You have to set up **net.blacklist** and/or **net.whitelist** yourself after initialising the **net** package.

The procedure is a binding to C's accept() function.

See also: **net.accept**, **net.bind**, **net.block**, **net.keep**, **net.listen**, **net.receive**, **net.survey**.

[6] Usually, the server that tries to connect sends its numeric IP address, but probably it does not. So this is just a precautionary action.

net.admin

Table containing various operating system-specific administrative network settings:

Key	Meaning
maxnsockets	estimated maximum number of open sockets allowed
protocols	a table containing the supported protocols

net.address (s)

Returns the local address information associated to the given socket s: a string with the local IP address, the port number, and a string with the family ('ipv4' or 'ipv6', the protocol in use). In case of an error, the function returns **false** plus the error description as a string.

The function is the Agena pendant to the C function getsockname().

See also: **net.lookup**, **net.remoteaddress**.

net.bind (s [, address [, port]])

Associates a socket s with an IP address and a port on the local machine and returns its IP address (a string) and the respective port on success or returns **false** and a string containing the error message otherwise.

If address is not given, localhost is bound to the socket (i.e. your own computer), otherwise the numeric IP address or host name is bound.

By default, port 1234 is connected, but you may specify another port (an integer) as a third argument. This might require administrative rights.

If address is the string 'localhost', then depending on the socket's protocol the name will be either converted to '127.0.0.1' (IPv4) or '::1' (IPv6) before the binding attempt starts.

The procedure is a binding to C's bind() function.

To determine your own IP address, open a shell and issue the command ipconfig in Windows, and ifconfig in Solaris, Linux, Mac or other UNIX based platforms.

See also: **net.accept**, **net.listen**, **net.receive**, **net.survey**.

net.block (s, mode)

Sets a socket to blocking or non-blocking mode. The functions expects the socket handle (a number) `s` as its first argument and the `mode` (a Boolean) as its second argument. If the second argument is **true**, the socket is set to blocking mode, else to non-blocking mode. The return is **true** on success and **false** otherwise.

The procedure is a binding to C's fcntl() (UNIX) or ioctlsocket() (Windows) function.

See also: **net.keep**.

net.close (···)

Terminates all the *given* servers or clients denoted by their socket handles and returns **true** on success, or **false** and a string containing an error message otherwise.
The procedure is a binding to C's close() or closesocket() function.

net.closewinsock ([anything])

The function is available only in the Windows edition. It finally terminates the current network session and returns **true** on success, or issues an error otherwise if `anything` is not given. If any value `anything` is passed to the function, in case of an error it returns **fail** plus an error message of type string.

Please note that when you call this function, no further network communication will be possible. Call **net.openwinsock** to enable network communication again.

The procedure is a binding to C's WSACleanup() function.

See also: **net.openwinsock**.

net.connect (s [, address [, port]])

Connects the client denoted by it socket handle `s` (first argument, a number) to a server at the specified IP `address` (second argument, a string) and its `port` (third argument) so that data can be sent later. If `address` is missing, the address is set to `'localhost'`, if `port` is missing, port 1234 will be used.

If the client socket is set to blocking mode, the function waits until the server responds; if the client socket is set to non-blocking mode, it immediately returns without waiting for a server response.

The return is either **true** in case of success or **false** and the error message (a string) at failure.

If `address` is the string 'localhost', then depending on the socket's protocol the name will be either converted to '127.0.0.1' (IPv4) or '::1' (IPv6) before the connection attempt starts.

The function also checks the global sets **net.blacklist** and **net.whitelist**, in this order, and if one of them or both exist, the following will happen:

- If you are trying to connect to an address that is included in **net.blacklist**, then **net.connect** does not establish a connection, closes socket `s`, and issues an error.
- If you are trying to connect to a server that is not in **net.whitelist**, the function does not establish a connection, closes the socket, and issues an error, as well.

Please note that **net.blacklist** and **net.whitelist must only contain numeric IPs**, and not addresses like 'sunsite.abc.yz'. However, **net.connect** tries to convert `address` to a numeric IP address and then checks both lists. If an address could not be resolved, the function does not establish a connection, closes socket `s` and issues an error.

You may use **protect** in order to intercept the errors described above, but you must take care yourself for allowing or prohibiting the connection.

You have to set up **net.blacklist** and/or **net.whitelist** yourself after initialising the **net** package.

The procedure is a binding to C's connect() function.

See also: **net.send**.

net.getaddrinfo (hostname)

Converts from `hostname`, a string, to address which can be an IPv4 or IPv6 address or host name.

The function returns a table with all information returned by the resolver. In case of error, the function returns null followed by an error message.

```
> net.getaddrinfo('localhost'):
[[addr ~ ::1, family ~ ipv6], [addr ~ 127.0.0.1, family ~ ipv4]]

> net.getaddrinfo('www.nasa.gov'):
[[addr ~ 192.0.66.108, family ~ ipv4]]
```

See also: **net.gethostname**, **net.lookup**, **net.tohostname**, **net.toip**.

net.gethostname ()

Gets the host name of the system on which it is called and returns a string. Note that this is not the DNS hostname. In case of error, the function returns **false** plus the error description as a string.

The function calls the underlying C function gethostname().

See also: **net.getaddrinfo**, **os.computername**, **os.netdomain**.

net.htonl (x)

Converts a non-negative 4-byte integer `x` into Big Endian network byte order representation. This is just to play with the underlying C function of the same name. The **net** package takes care of proper byte order representation, so you do not need **net.htons** when calling the package functions.

See also: **net.htons**, **net.ntohl**, **net.ntohs**.

net.htons (x)

Converts a non-negative 2-byte integer `x` into Big Endian network byte order representation. This is just to play with the underlying C function of the same name. The **net** package takes care of proper byte order representation, so you do not need **net.htons** when calling the package functions.

See also: **net.htonl**, **net.ntohl**, **net.ntohs**.

net.isconnected ()

Checks whether you are currently connected to the Internet and returns **true** or **false**. The function checks for a working connection to NASA, www.nasa.gov.

net.isipv4 (address)

Checks whether `address`, represented as a string, is a numeric IPv4 address and returns **true** or **false**.

net.isipv6 (address)

Checks whether `address`, represented as a string, is a hexadecimal IPv6 address and returns **true** or **false**.

net.keep (s, mode)

Sets socket `s` to keep-alive mode or deactivates it. The functions expects the socket handle `s` (a number) as its first argument and the `mode` (a Boolean) as its second argument. If the second argument is **true**, the socket is set to keep-alive mode. If it is **false**, keep-alive mode is turned off. The return is **true** on success and **false** otherwise.

See also: **net.block**.

net.listen (s [, length])

Converts the given socket `s` to a server socket, enabling it to accept connections. You may optionally pass an integer in the range [1, 1024] determining the length of the queue for pending connections.

The return is either **true**, or **false** and a string with an error message if listening failed.

You must first run this function before calling **net.accept** and **net.receive**.

The procedure is a binding to C's listen() function.

net.lookup ([x])

Determines the IP, an optional alias, the official name and the supported protocol of a given URL or numeric IP `x` of type string. With the IPv4 protocol, if no argument is passed, the function will return the information on `'localhost'`. Explicitly pass `'::1'` for localhost for the IPv6 protocol.

An example:

```
> net.lookup('www.nasa.gov'):
[alias ~ [www.nasa.gov], networkaddress ~ [192.0.66.108],
 official ~ nasa-gov.go-vip.net, type ~ IPv4]

> net.lookup('10.137.0.1'):
[networkaddress ~ [10.137.0.1], alias ~ [anything.yz], official ~ Anything,
type ~ IPv4]
```

See also: **net.address**, **net.getaddrinfo**, **net.remoteaddress**, **net.tohostname**, **net.toip**.

net.ntohl (x)

The function converts a non-negative 4-byte integer value from network to host byte order.

See also: **net.htonl**, **net.htons**, **net.ntohs**.

net.ntohs (x)

The function converts a non-negative 2-byte integer value from network to host byte order.

See also: **net.htonl**, **net.htons**, **net.ntohl**.

net.open ([blocking])
net.open ([options])

In the first form, creates a (client) network IPv4 socket. If the optional first argument `blocking` is set to **false**, the socket is set to non-blocking mode.

The return is the socket handle (a number), the default address `'localhost'` (with IPv4) or '::1' (with IPv6) and default port `1234`, the protocol (a number) and a Boolean indicating whether the handle can be reused by the system after the socket has been closed. If a new socket could not be opened, an error will be issued.

net.open does not connect the client to a server - use **net.connect** for this.

To create a server socket waiting for input, use **net.bind**, **net.listen**, and **net.accept.**

In the second form, pass the following options, according to your needs.

Especially in OS/2, you need to pass the mode, address and port options to create a working server or client socket, simply calling **net.open** without them will not work in OS/2 as the operating system will always shut a socket down if it is not immediately connected or bound.

- `ipv` = 4: Creates an IPv4 socket,
- `ipv` = 6: Creates an IPv6 socket (not supported on OS/2),
- `blocking` = <**true** or **false**>: sets or unsets blocking mode, default is **true**.
- `keepalive` = <**true** or **false**>: sets or unsets keep-alive mode, default is **false**.
- `mode` = 'server', creates a server socket (needs address and port, see below), do not call **net.bind** separately,
- `mode` = 'client', creates a client socket (needs address and port, see below), do not call **net.connect** separately,
- `address` = <a string>, the IP address, such as '127.0.0.1',
- `port` = <an integer>, the port, such as 1300.

The return is the socket handle (a number), the address and port the socket has been connected or bound to, the protocol (a number) and a Boolean indicating whether the handle can be reused by the system after the socket has been closed.

If a new socket could not be opened, an error will be issued.

Example:

```
> import net;

> s := net.open(mode = 'client', address = '129.6.15.28', port = 13);

> net.receive(s):
61217 26-06-26 13:59:51 50 0 0 549.9 UTC(NIST) *
        51
```

The procedure is a binding to C's socket() function.

Example to read the time from the NIST server:

See also: **net.close**.

net.opensockets ()

Returns all open sockets along with their respective attributes.

The return is a table with its keys the open socket handles, and their entries tables containing information on whether the socket is a server or client (key `'server'`, **true** or **false**), their own address (key `'address'`, a string), their own port (key `'port'`, a number), the protocol being used (key `'protocol'`, 2 for IPv4, any other number for IPv6), whether the socket works in blocking or non-blocking mode (key `'blocking'`, **true** or **false**), whether the socket has been connected to a server (`'connected'`, **true** or **false**), and whether keep-alive mode has been activated (`'keepalive'`, **true** or **false**).

The table key `'mode'` holds information on the read and write status of the socket:

Value	Meaning
`'none'`	the socket is not connected
`'shutdown'`	the socket no longer can receive or send data
`'read'`	the socket can only receive data, but cannot send any
`'write'`	the socket can only send data, but cannot receive any
`'readwrite'`	the socket can both send and receive data (the default)

Please note that modifying the contents of the table returned will not have any effect on the status of the sockets, so you cannot do any harm.

See also: **net.shutdown**.

net.openwinsock ([anything])

The function is available only in the Windows edition. It re-enables network communication and returns **true** on success, or issues an error otherwise if `anything` is not given. If any value `anything` is passed to the function, in case of an error it returns **fail** plus an error message of type string.

When initialising the **net** package by calling **readlib** or **with**, Agena automatically starts the Winsock daemon, so you do not have to call this function explicitly.

The procedure is a binding to C's WSAStartup() function.

See also: **net.closewinsock**.

net.receive (s [, getall [, maxlength]])

Allows a server socket `s` to receive a string from a client. The function returns this string and its length (a number). `s` should be the socket handle returned by **net.accept**.

If the return is the empty string plus the value 0 (zero) for its length, the client has closed the connection - this is also a proper check on whether a client is still connected with a server socket. Please note that in this case, no further data can be received on this socket and you have to close `s` manually.

If **true** has been passed for the optional argument `getall`, the function reads in all data from the client until the latter closes the connection. If the client does not close the connection, **net.receive** waits infinitely.

The optional argument `maxlength` determines the maximum number of characters to be received. If a client tries to send more data than specified by `maxlength`, the function returns **false** and the string `'too many bytes received'`.

The maximum number of bytes to be read by one stroke is determined by **environ.kernel('buffersize')** which value depends on the operating system and which can also be changed, like this:

```
> environ.kernel(buffersize = 1024): # change from mostly 512 to 1024 bytes
1024
```

If any error occurs during receipt of the data, **net.receive** does not close the socket `s`, but returns **false** and a string containing either the message `'failure during receipt'` or `'too many bytes received'`, the latter if `maxlength` and the number of bytes received exceeded it.

The procedure is an extended binding to C's recv() function.

See also: **net.accept**, **net.bind**, **net.block**, **net.listen**, **net.receive**, **net.send**, **net.survey**.

net.remoteaddress (s)

For socket `s`, returns address information about the remote side of the connection, the `peer`: a string with the IP address of the peer, the port number the peer is using for the connection, and a string with the family ('ipv4' or 'ipv6', the protocol in use). In case of error, the function returns `false` plus the error description as a string.

The function is the Agena pendant to the C function getpeername().

See also: **net.address**, **net.lookup**.

net.send (s, str [, true])

Sends a string `str` (second argument) from the client denoted by its socket handle `s` (first argument, a number) to a server.

The return is the number of the characters actually sent. If the kernel decides not to send all the data in one chunk, the function might not send the complete string. If an optional third argument, the Boolean **true**, is given, **net.send**, however, tries to make sure that the complete string has been sent when it returns.
If `str` is the empty string, it will not be sent to the server.

The function returns **fail** and the string `'socket not connected'` if the socket has not been connected before by either **net.connect** or **net.accept**. It also returns fail and `'socket not connected'` if the connection has been disconnected.

If the number of bytes actually sent is not equal to the length of the string str, the function returns false, the string `'transfer size mismatch'`, and the number of bytes sent.

The procedure is an extended binding to C's send() function.

See also: **net.connect**, **net.receive**.

net.shutdown (s, what)

The function stops further sends and receives on a socket `s`. If `what` is the string `'read'`, then the socket can no longer receive data; if `what` is the string `'write'`, it can lo longer send data; and if `what` is the string `'readwrite'`, it will not do both any longer.

Please note that socket `s` will still be active. Call **net.close** if you want to release the socket completely.

See also: **net.opensockets**.

net.smallping (ip, port [, iters [, delay [, message [, noprint]]]])

Opens a blocking socket, connects to a server given by the string `ip` (either a domain name or a numeric ip) on its port `port`, a number in the [0, 65535], optionally sends a string to the server, and then closes the connection again. It resembles the UNIX ping command, but works on a low-level network connection and does not use ICMP.

By default, only one connection attempt is conducted before the function returns. You can specify the number of connection attempts by the optional argument `iters`, a positive integer.

The function waits one second before connecting to the server again. You can change this by passing a different number of seconds for the argument `delay`, a non-negative integer. If `iters` is set to 1, the `delay` argument is ignored and the function immediately returns.

If `message` is not given, the function does not send any data to the server. You can change this by passing a string as argument `message`, which might also be the empty string.

By default, the function prints the connection results at the console with each iteration. This can be suppressed by passing any non-null value as argument `noprint`. If you specify a value for `noprint` and if you do not want to send a string to the server, just pass a non-string value as argument `message`.

The following data is printed at the console if `noprint` is void: Date and time, round-trip time for the current connection in seconds, average round-trip time, a Boolean indicating whether the connection was successful (true) or not (false), and the number of the current iteration. Example:

```
> net.smallping('www.anything.foo', 80, 4, 2)
> # four iterations, 2-second delay, no message
2014/01/01 13:54:30   0.296   0.296  true  1
2014/01/01 13:54:32   0.031   0.163  true  2
2014/01/01 13:54:34   0.047   0.125  true  3
2014/01/01 13:54:36   0.047   0.105  true  4
```

The function returns the date and time of the final iteration as a number indicating the number of seconds passed since a given `epoch`, the average round-trip time in seconds as a number, and a Boolean indicating whether the last connection attempt was successful (**true**) or not (**false**). Use **skycrane.todate** to convert the numeric date into a readable format.

The function is written in Agena and included in the lib/net.agn file.

See also: **os.ping**.

net.survey ([o], [timeout [, mode [, throw]]])

The function looks for activity on all open sockets, or of specific sockets. If you want to scan only specific sockets, pass a sequence `o` of socket handles as the first argument.

The returns are three sequences and a Boolean: the first sequence with descriptors of sockets ready for reading, the second sequence containing all descriptors of sockets ready for writing, and the third sequence with the descriptors of sockets which encountered exceptional conditions. (Exceptional conditions are not failures.) If the Boolean is **true** then input is available, if it is **false** it indicates a timeout.

By default, **net.survey** waits endlessly and only returns if a network action has been detected (so-called `blocking mode`).

If the positive number `timeout` is passed to the function, the functions will always return after `timeout` seconds even if there was no activity. if `timeout` is **infinity**, it waits endlessly for a connection.

If `mode` is the string `'read'`, then the function only scans sockets ready for reading. If `mode` is the string `'write'`, then the function only scans sockets ready for writing. If `mode` is the string `'except'`, then the function only scans sockets where exceptions occurred. In all three cases, the returns are a sequence of the respective sockets handles and the Boolean **true** if input is available, or **false** at timeout.

If `throw` is set to **false**, then the function does not quit with an error in case the socket status could not be determined.

A socket handle returned can be passed to the **net.accept** function so that an incoming connection can be further processed.

The function is a binding to C's select() function.

See also: **net.accept**, **net.bind**, **net.listen**, **net.receive**.

net.tohostname (address)

Converts from IPv4 address to host name. The input `address` can be an IP address or host name.

The function returns a string with the canonic host name of the given address, followed by a table with all information returned by the resolver. In case of error, the function returns **null** followed by an error message.

Example:

```
> net.tohostname('46.175.8.47'):
www.lua.org     [alias ~ [], ip ~ [46.175.8.47], name ~ www.lua.org]
```

See also: **net.address**, **net.getaddrinfo**, **net.gethostname**, **net.lookup**, **net.toip**.

net.toip (address)

Converts from host name to IPv4 address. The input `address` can be an IP address or host name.

Returns a string with the first IP address found for address, followed by a table with all information returned by the resolver. In case of error, the function returns **null** followed by an error message.

Example:

```
> net.toip('www.lua.org'):
46.175.8.47     [alias ~ [], ip ~ [46.175.8.47], name ~ www.lua.org]
```

See also: **net.address**, **net.getaddrinfo**, **net.gethostname**, **net.lookup**, **net.tohostname**.

net.wget (domain, [path [, port]])

The function downloads an HTML file from a web server.

`domain`, a string, specifies the domain. `path`, also of type string, indicates the absolute path including the HTML file name on the web server. If `port`, a non-negative integer less than 65,535 is given, then the function tries to query this port instead of the standard HTML port 80.

If only `domain` is given, then it may include the absolute path. If you want to download data from a different port than 80, however, you must pass the absolute path as the second argument.

The function uses the HTTP 1.0 protocol along with the GET method.

The function returns the retrieved web page as a string, including its HTTP protocol header.

Examples:

```
> import net

> net.wget('www.lua.org', 'about.html'):
HTTP/1.1 200 OK
Server: Zeus/4.3

> net.wget('www.lua.org/about.html'):
```

The function is written in Agena and included in the lib/net.agn file.

13.2 usb - libusb Binding

As a *plus* package, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import usb`.

The package provides 1:1 access to libusb functions. Please have a look at the libusb man pages and is available in the Windows version of Agena, only.

The functions provided by this binding are:

13.2.1 CTX Functions

Package function name	Corresponding libusb function
usb.event_handler_active	libusb_event_handler_active
usb.event_handling_ok	libusb_event_handling_ok
usb.get_device_list	libusb_get_device_list
usb.get_next_timeout	libusb_get_next_timeout
usb.get_pollfds	libusb_get_pollfds
usb.handle_events	libusb_handle_events
usb.handle_events_locked	libusb_handle_events_locked
usb.handle_events_timeout	libusb_handle_events_timeout
usb.lock_event_waiters	libusb_lock_event_waiters
usb.lock_events	libusb_lock_events
usb.pollfds_handle_timeouts	libusb_pollfds_handle_timeouts
usb.set_debug	libusb_set_debug
usb.set_pollfd_notifiers	libusb_set_pollfd_notifiers
usb.try_lock_events	libusb_try_lock_events
usb.unlock_event_waiters	libusb_unlock_event_waiters
usb.unlock_events	libusb_unlock_events
usb.wait_for_event	libusb_wait_for_event

13.2.2 DEV Functions

Package function name	Corresponding libusb function
usb.get_active_config_descriptor	libusb_get_active_config_descriptor
usb.get_bus_number	libusb_get_bus_number
usb.get_config_descriptor	libusb_get_config_descriptor
usb.get_config_descriptor_by_value	libusb_get_config_descriptor_by_value
usb.get_device_address	libusb_get_device_address
usb.get_device_descriptor	libusb_get_device_descriptor
usb.get_max_iso_packet_size	libusb_get_max_iso_packet_size
usb.get_max_packet_size	libusb_get_max_packet_size
usb.open	libusb_open

13.2.3 Handles

Package function name	Corresponding libusb function
usb.attach_kernel_driver	libusb_attach_kernel_driver
usb.bulk_transfer	libusb_bulk_transfer
usb.claim_interface	libusb_claim_interface
usb.clear_halt	libusb_clear_halt
usb.close	closehandle
usb.control_transfer	libusb_control_transfer
usb.detach_kernel_driver	libusb_detach_kernel_driver
usb.get_configuration	libusb_get_configuration
usb.get_descriptor	libusb_get_descriptor
usb.get_device	libusb_get_device
usb.get_string_descriptor	libusb_get_string_descriptor
usb.get_string_descriptor_ascii	libusb_get_string_descriptor_ascii
usb.get_string_descriptor_utf8	libusb_get_string_descriptor_utf8
usb.interrupt_transfer	libusb_interrupt_transfer
usb.kernel_driver_active	libusb_kernel_driver_active
usb.release_interface	libusb_release_interface
usb.reset_device	libusb_reset_device
usb.set_configuration	libusb_set_configuration
usb.set_interface_alt_setting	libusb_set_interface_alt_setting

13.2.4 Transfer Functions

Package function name	Corresponding libusb function
usb.cancel_transfer	libusb_cancel_transfer
usb.control_transfer_get_data	libusb_control_transfer_get_data
usb.control_transfer_get_setup	libusb_control_transfer_get_setup
usb.fill_bulk_transfer	libusb_fill_bulk_transfer
usb.fill_control_setup	libusb_fill_control_setup
usb.fill_control_transfer	libusb_fill_control_transfer
usb.fill_interrupt_transfer	libusb_fill_interrupt_transfer
usb.fill_iso_transfer	libusb_fill_iso_transfer
usb.get_iso_packet_buffer	libusb_get_iso_packet_buffer
usb.set_iso_packet_buffer	libusb_set_iso_packet_buffer
usb.set_iso_packet_lengths	libusb_set_iso_packet_lengths
usb.submit_transfer	libusb_submit_transfer
usb.transfer_get_data	libusb_transfer_get_data

13.2.5 Miscellaneous Functions

Package function name	Corresponding libusb function
usb.init	libusb_init
usb.open_device_with_vid_pid	libusb_open_device_with_vid_pid
usb.transfer	libusb_transfer

13.3 com - Serial RS-232 Communication through COM Ports

As a *plus* package, the **com** package is not part of the standard distribution and must be activated with the **import** statement, i.e. `import com`.

The **com** library allows to send and receive data through physical and virtual COM ports. It is currently in an experimental state.

Typical usage for sending data to another com port:

```
> import com

> fd := com.open('COM4');

> com.init(fd, 9600, "sb1"):

> com.attrib(fd):

> com.write(fd, 'hallo !');

> com.close(fd);
```

Reading data from a com port:

```
> fd := com.open('COM4');

> com.init(fd, 9600, "sb1"):

> do                            # if nothing has been read, com.read ...
>   str := com.read(fd, 80);              # returns null ...
>   if str :: string then print(str) fi  # and a string otherwise
> od;
```

The following functions can also be called OOP-style:

com.open (port)

Creates a handle to the given COM port `str`, a string, and returns the handle, of type userdata.

In Windows, a COM port may be denoted by the strings `'COM1'`, `'COM2'`, etc.

The function does not configure the port, see **com.init**.

See also: **com.close**.

com.close (hdl)

Closes the given port denoted by its handle `hdl` of type userdata.

See also: **com.open**.

com.write (hdl, str)

Sends the entire string `str` to the port denoted by `hdl`. The number of bytes actually sent will be returned.

See also: **com.read**.

com.read (hdl [, bufsize])

Reads data from the port denoted by `hdl` and returns it as a string. If nothing could be read, the return is **null**.

The (maximum) number of bytes to be read is given by `bufsize`. If not given, the function tries to read the entire in-queue. You should prefer to pass a value for `bufsize`.

See also: **com.write**.

com.attrib (hdl)

Returns the settings of the given COM port, denoted by its handle `hdl`. The function is available in Windows only.

com.init (hdl, option$_1$, options$_2$, ···)

Configures the COM port denoted by its handle `hdl`. The function re-initialises all hardware and control settings, but it does not empty output or input queues.

Supported settings are:

- `'reset'` - reset
- baud rate, a number,
- `'cs5'` to `'cs8'`, - character size,
- `'parno'`, `'parodd'`, `'pareven'` - parity
- `'sb1'`, `'sb2'` - stop bits,
- `'foff'`, `'frtscts'`, `'fxio'` - flow controls.

Example:

```
> com.init(fd, 9600, "sb1");
```

In DOS, you (obviously ?) can only use baud rates 9600, 19200 and 38400. At least in UNIX, supported baud rates are:

9600, 19200, 38400, 57600, 115200, 230400, 460800, 500000, 576000, 921600, 1000000, 1152000, 1500000, 2000000, 2500000, 3000000, 3500000, 4000000.

com.control (hdl, option1, ···)

Sets DTR/DSR or RTS/CTS hardware flow controls to the COM port denoted by its handle `hdl`. Options are:

- `'dtr'` - Data Terminal Ready
- `'dsr'` - Data Set Ready
- `'rts'` - Request To Send
- `'cts'` - Clear To Send

The function does not empty output or input queues.

com.timeout (hdl, msecs)

Sets the time-out parameters for all read and write operations on the specified COM port, denoted by its handle `hdl`. `msecs` is in milliseconds.

com.queues (hdl, in_buffersize, out_buffersize)

Sets buffer sizes for in and out queues for the COM port denoted by its `hdl` handle. *`buffersize`s are integers.

com.purge (hdl, mode)

Discards all characters from the output or input buffer of the given COM port, denoted by it handle `hdl`. It can also terminate pending read or write operations on the resource.

`mode` may be the string: `'rw'`, `'r'` or `'w'`.

com.wait (hdl, event1, ···)

Waits for an event to occur for the COM port denoted by its handle `hdl`. The set of events that are monitored by this function are the strings: `'car'`, `'cts'`, `'dsr'`, `'ring'`.

The function returns a Boolean.

Chapter Fourteen

System & Environment

14 System & Environment

14.1 os - Access to the Operating System

This library is implemented through table `os`.

To determine the operating system and CPU in use by Agena, see the **environ.os** and **environ.cpu** environment variables explained in Appendix A3.

Summary of functions:

File and directory handling:

os.chdir, **os.chmod**, **os.chown**, **os.curdrive**, **os.curdir**, **os.dirname**, **os.exists**, **os.fattrib**, **os.fcopy**, **os.filename**, **os.findinpath**, **os.fstat**, **os.ftok**, **os.getfullpathname**, **os.gettemppath**, **os.inode**, **os.isdir**, **os.isfile**, **os.islink**, **os.isvalidpath**, **os.iterate**, **os.list**, **os.listcore**, **os.mkdir**, **os.mklink**, **os.move**, **os.prefix**, **os.readlink**, **os.realpath**, **os.remove**, **os.rmdir**, **os.suffix**, **os.symlink**, **os.tmpdir**, **os.tmpname**, **os.whereis**.

Hardware access:

os.battery, **os.beep**, **os.cdrom**, **os.endian**, **os.freemem**, **os.getusbdevices**, **os.hasnetwork**, **os.isdocked**, **os.isdriveletter**, **os.ismounted**, **os.isremovable**, **os.isvaliddrive**, **os.meminfo**, **os.screensize**.

Operating System and Hardware Access:

os.bios, **os.cachelinesize**, **os.codepage**, **os.computername**, **os.cpuid**, **os.cpuinfo**, **os.cpuload**, **os.drives**, **os.drivestat**, **os.environ**, **os.execute**, **os.exit**, **os.getcommandline**, **os.getconsolemode**, **os.getdirpathsep**, **os.getenv**, **os.getip**, **os.getlanguage**, **os.getlocale**, **os.getloadeddlls**, **os.getmodulefilename**, **os.getprocesses**, **os.getthreadid**, **os.getthreadseed**, **os.getwinsysdirs**, **os.groupinfo**, **os.isansi**, **os.isarm**, **os.isarm32**, **os.isarm64**, **os.isdos**, **os.isdow**, **os.islinux**, **os.islinux386**, **os.islocale**, **os.ismac**, **os.isos2**, **os.isppc**, **os.isunix**, **os.iswindows**, **os.isx86**, **os.login**, **os.netdomain**, **os.netsend**, **os.netuse**, **os.os2info**, **os.pause**, **os.pid**,**os.ping**, **os.setconsolemode**, **os.setenv**, **os.settime**, **os.setlocale**, **os.shellcolour**, **os.shellgeom**, **os.shellinfo**, **os.system**, **os.terminate**, **os.userinfo**, **os.vendor**, **os.wait**, **os.winver**, **os.xgetbv**.

Date and Time:

os.clock, **os.date**, **os.datetosecs**, **os.difftime**, **os.esd**, **os.isdst**, **os.lsd**, **os.now**, **os.secstodate**, **os.speed**, **os.time**, **os.timestamp**, **os.tzdiff**, **os.uptime**, **os.usd**.

os.battery ()

On Windows 2000 and later, the function returns the current battery status of your system (usually laptops) as a table with the following information:

Key	Meaning
`'acline'`	'on', 'off', or 'unknown'
`'installed'`	**true** if a battery is present, and **false** otherwise
`'life'`	battery life in percent; a value > 100 indicates that a battery is not installed (see `'status'` entry)
`'status'`	either 'low' (capacity < 33%), 'medium' (capacity > 32% and <67 %), 'high' (capacity > 66%), 'critical' (capacity < 5%), 'charging', 'no battery', 'unknown'
`'charging'`	**true** if battery is currently being charged, or **false** otherwise
`'flag'`	the battery flag, a number
`'lifetime'`	the remaining battery lifetime in seconds, a number (or **undefined** if it could not be determined)
`'fulllifetime'`	the battery lifetime in seconds when at full charge, a number (or **undefined** if it could not be determined)

On OS/2 Warp 4 and higher, with APM running, the functions returns the status of the battery as a table with the following information:

Key	Meaning
`'acline'`	'on', 'off', 'unknown', or 'invalid'
`'life'`	battery life in percent, or 'undefined' if not available
`'status'`	either 'high', 'low', 'critical', 'charging', 'unknown', or 'invalid'
`'flags'`	OS/2 power flags
`'power-management'`	**true** if power management is switched on, or **false** if not.

On other operating systems, the function returns **fail**.

os.beep ()

os.beep (freq, dur)

In the first form, the functions sounds the loudspeaker with a short `beep` and returns **null**.

The second form sounds the loudspeaker with frequency `freq` Hz (a positive integer in the range 37 .. 32767) for `dur` seconds (a positive float) in Windows, DOS and OS/2. In UNIX, the loudspeaker beeps `dur` times, and the frequency is ignored (just pass any number to `freq`). Returns **null** if a sound could be created successfully, or **fail** if non-positive arguments were passed.

os.bios ()

In Windows, the function reads out BIOS and mainboard information and returns them in a table with the following fields and associated string entries: 'BIOSVendor', 'BIOSVersion', 'BIOSReleaseDate', 'SystemManufacturer', 'SystemProductName', 'SystemSerialNumber', 'SystemSKU', 'BaseBoardManufacturer', 'BaseBoardProduct', 'BaseBoardVersion'. The fields'BiosMajorRelease' and 'BiosMinorRelease' have number entries.

If the Windows registry could not be accessed or the aforementioned registry keys are all missing, or on all other platforms, the function returns **fail**.

os.cachelinesize ()

Returns the L1 Cache Line Size in bytes. If the return is zero, the size could not be determined. On OS/2 and DOS the function always returns zero.

See also: **os.cpuinfo**.

os.cdrom (d, action)

Opens and closes the tray of an optical disk drive `d`. It can also eject any other removable drive d. If `action` is `'open'` or `'eject'`, the tray is opened or the media is ejected. If `action` is `'close'`, the tray is closed. The function is available in the OS/2, Linux, and Windows edition of Agena only.

See also: **os.drives**, **os.drivestat**, **os.unmount**.

os.chdir ([str [, any]])

Changes into the directory given by string `str` on the file system. Returns **true** on success and issues an error on failure otherwise if any has not been given. If you pass any second argument, the function will not issue an error and will return **false** if the given path does not exist, or **fail** if you have no permission to enter the directory.

If no argument is given or **null** is passed for `str`, the name of the current working directory will be returned as a string. Otherwise, the function will commit the change of directory and return **true**.

See also: **os.curdir**, **os.dirname**, **os.isdir**, **os.iterate**, **os.list**.

os.chmod (fn, m)

Takes a file path `fn` (a filename, thus a string) and a mode `m` (an integer) denoting a three-digit octal number and changes the file permissions accordingly. Contrary to **os.fattrib**, mode `m` must *not* be preceded by the `0o` token.

The function returns **true** on success and issues an error otherwise. It is available in the UNIX versions of Agena, only.

See also: **os.chown**, **os.fattrib**.

os.chown (fn, o [, g])

The function changes the owner of the file `fn` (a filename, thus a string) to owner `o`, and optionally to group `g`.

`o` and `g` may be numbers or strings. If a number is passed for `o` or `g`, it denotes a user id (uid) or group id (gid), respectively. If a string is passed, it denotes a user or group name. If `g` is not given, the default group of user `o` is set.

The function returns true on success and issues an error message otherwise. It is available in the UNIX versions of Agena, only.

See also: **os.chmod**, **os.fattrib**.

os.clock ()

Returns the processor time used by Agena in milliseconds. The function can count beyond 24.8 days and resets only after many years.

See also: **time**, **watch**, **os.time**, **os.timestamp**, **os.uptime**.

os.codepage ()

os.codepage (p)

In Windows, in the first form, returns - in the following order: input code page, output code page, the input code page name and the output code page name.

In OS/2, in the first form, returns - in the following order: input code page and output code page. In DOS, in the first form, returns the output code page.

In OS/2 and Windows only, in the second form, sets the input and output code page. To change the input code page, pass the pair 'input':<code page number>. To change the output code page, pass the pair 'ouput':<code page number>.

In all other operating systems, the function is not available.

See also: **os.getconsolemode**, **os.getlocale**, **os.setconsolemode os.setlocale**, **os.shellinfo**.

os.computername ([option])

Returns the name of the computer in Windows, OS/2, DOS, Mac OS X and UNIX. The return is a string. On other architectures, the function returns **fail**.

If called with any `option`, returns detailed information on the NetBIOS or DNS name associated with the local computer in a table.

See also: **os.netdomain**.

os.countcore (d)

os.countcore (d [, options] [, pattern])

In the first form, counts all occurrences of files, links and directories in the given path `d`. If `d` is void or the string `'.'`, the current working directory is evaluated.

In the second form, by giving at least one of the options `'files'` or `'file'`, `'dirs'` or `'dir'`, or `'links'` or `'link'`, only the files, directories or links are counted, respectively. These three options can be mixed.

Another option may be a `pattern` of type string which can include the wildcards `?` or `*`. If given, the function will only count those entries which match this pattern.

See also: **os.listcore**, **os.whereis**.

os.cpuid (leaf)

Reads out the EAX, EBX, ECX and EDX registers of the x86-compatible CPU. You pass a function number (CPUID leaf) and optionally the number of the desired register, which may be 1 for EAX, 2 for EBX, 3 for ECX, 4 for EDX or 0 for all four registers.

See the descriptions in the Internet on GCC's __cpuid_count() and __cpuid() functions for further information. Example to check whether AVX is supported:

```
> eax, ebx, ecx, edx := os.cpuid(1);

> ecx && (1 << 28) <> 0:
true
```

See also: **os.cpuinfo**, **os.xgetbv**.

os.cpuinfo ()

Returns various information on the CPU in use: its type, frequency, and number of cores. It is available in Windows 2000 and later, OS/2, DOS, Solaris, Linux, and Mac OS X only[7]. The return is a table with the following fields:

[7] In Solaris, you may issue `io.pcall('kstat')` and parse its return.

Field	Meaning	OS/2	Win-dows	Mac	Linux
'bigendian'	endianness: **true** means Big Endian, **false** Little Endian, and **fail** undetermined.	x	x	x	x
'bogomips'	crude measurement of CPU speed				x
'brand'	processor name, a string[8]	x	x	x	x
'cache'	information on L1/L2/L3 caches (Intel only)	x	x		x
'cpuid'	detailed information on the underlying CPU hardware returned by the C function __cpuid.	x	x		x
'cputype'	detailed information on the CPU		x		x
'cycles-perusec'	CPU cycles per microsecond (Intel-compatible CPUs only)	x	x	x	x
'frequency'	clock rate in MHz, a posint	x	x	x	x
'level'	processor level, a posint		x	x	x
'model'	processor model, a posint			x	x
'ncpu'	number of cores, a posint	x	x	x	x
'revision'	processor revision, a posint		x		
'stepping'	processor stepping, a posint			x	x
'support'	supported instruction sets	x	x	x	x
'type'	architecture: in Windows the string: 'x86', 'x64', 'ARM', 'Itanium', or 'unknown'; on a Mac: 'x86', 'x64', 'ppc', 'ppc64', 'MC680x0', 'MC88000', MC98000', HPPA', 'ARM', 'sparc', 'i860', or 'unknown'. In Linux: 'x86', 'x64', 'arm', 'aarch64'.	x	x	x	x
'vendor'	vendor ID, e.g. 'GenuineAMD', 'GenuineIntel'.	x	x	x	x

On all supported operating systems, the data is determined by querying the first processor on the platform, assuming that all other cores have the same features. The returns may be platform-dependent - especially, the return regarding 'level' may have a different meaning.

On other platforms, the function returns **fail**.

The Linux version has been written in both C and Agena, see the lib/library.agn file; the other OS versions have been implemented entirely in C.

See also: **os.cachelinesize**, **os.cpuid**, **os.cpuload**, **os.endian**, **os.xgetbv**.

[8] The return may include leading or trailing blanks.

os.cpuload ()

In OS/2, Linux and Mac OS X, returns the 1, 5 and 15 minute load averages of the computer as a sequence of three numbers in the range [0 , 1]. In Windows Vista and later, it just returns a sequence containing the current average load, the load caused by the kernel and the load caused by user programmes - all three in the range [0, 1] - plus the number of elements in the CPU queue, the number of context switches per second and the number of interrupts per second - in this order.

If the load could not be returned, the function just returns **fail**.

On all other platforms, the function returns **fail**.

See also: **os.cpuinfo**, **os.speed**.

os.curdir ()

Determines the current working directory and returns its absolute path.

See also: **os.chdir**.

os.curdrive ()

In OS/2, DOS, and Windows returns the letter of the current drive, a one-character string with an appending colon.

os.date ([format [, seconds]])

os.date ([format [, obj]])

os.date (format, timestamp)

os.date (format, year, month, day [, hour [, minute [, second]]])

Returns a string or a table containing date and time, formatted according to the given string `format`.

The `seconds` argument represents the number of seconds elapsed since a given epoch (usually January 01, 1970, or try `os.now(0)`). When `seconds` is not given, **os.date** formats the current time. To convert a date and time to seconds, see **os.datetosecs**.

In the second form, receives a `format` and a date and optionally time of the form `year`, `month`, `date` [, `hour` [, `minute` [, `second`]]], with all values put into the table, sequence or register `obj`. Alternatively, in the fourth form, `year`, `month`, `day` and optionally `hour`, `minute`, and `second` can be passed directly.

In the third form, the function splits a `timestamp` string into its components and returns them in a table - see **strings.strptime** for further information.

If `format` starts with `'!'`, then the date is formatted in Co-ordinated Universal Time. After this optional character, if `format` is `*t`, then **os.date** returns a table with the following fields: `year` (four digits), `month` (1..12), `day` (1..31), `hour` (0..23), `min` (0..59), sec (0..59), msec (0..999) - if milliseconds could be determined, `wday` (weekday, Monday is 1, Sunday is 7), `yday` (day of the year, where 1 is January 01, and December 31 either 365 or 366), and `isdst` (daylight saving flag, a boolean). By setting environ.kernel(iso8601 = false), the weekday return 1 means Sunday and 7 Saturday.

If the format is `*j`, the Julian date, a number, will be returned. If the format is `*l`, the Lotus 1-2-3 Serial Date, a number, will be returned. For more information on the Lotus Serial Date value returned, see **os.lsd**. If the format is `*e`, the Excel Serial Date, a number, will be returned, see **os.esd**. `*sdn` computes the Julian date in the Julian calendar (whereas `*j`, `*l`, `*e` , `*t` compute it in the Gregorian calendar).

The `*asc` specifier lets **os.date** return a date formatted like the C function asctime() does, including a finishing newline. `*apa` formats date and time according to the Apache Common Log Format, and `*iso` formats according ISO 8601 standard.

If `format` is not `*t`, `*e`, `*l`, `*j`, or `*sdn`, then **os.date** returns the date as a string, formatted according to the same rules as the C function strftime().

When called without arguments, **os.date** on all supported platforms returns a string of the format 'YYYY/MM/DD mm:hh:ss.xxx', where .xxx denotes milliseconds, if they could be determined; otherwise the return would simply be in the format "YYYY/MM/DD mm:hh:ss'.

Examples:

```
> os.date('%a, %d %b %Y %H:%M:%S, %z'):
Mon, 02 Nov 2015 17:22:09, W. Europe Standard Time

> os.date('%A, %d %B %Y %H:%M:%S, %z'):
Monday, 02 November 2015 01:02:28, W. Europe Standard Time

> os.date("%Y-%m-%dT%H:%M:%S.%s", "2025-08-29T12:40:44.123"):
[2025, 8, 29, 12, 40, 44, 123]

> os.date('*asc', [2048, 1, 1, 23, 0, 0]):
Wed Jan  1 23:00:00 2048

> os.date('*apa', [2048, 1, 1, 23, 0, 0]):
[01/Jan/2048:23:00:00 W. Europe Standard Time]

> os.date('*iso', [2048, 1, 1, 23, 0, 0]):
2048-01-01T23:00:00Z
```

The following date and time specifiers always refer to the current locale, and may not be fully supported by your operating system - if not, an empty string will be returned:

Spec	Meaning	Example
%a	abbreviated weekday name	'Fri' for Friday
%A	full weekday name	'Friday'
%b	abbreviated month name	'Apr' for April
%B	full month name	'April'
%c	preferred calendar time representation	'05/19/17 20:33:13' for UK
%C	century of the year, greatest integer not greater than year divided by 100.	'19'
%d	day of the month as two-digit integer	'03'
%D	date of the format %m/%d/%y	'01/01/70'
%e	day of the month like with %d, but padded with blank instead of a zero	' 1'
%F	date of the format %Y-%m-%d	'1970-01-01'
%g, %G	year corresponding to the ISO week number, but without the century	'70' '1970'
%h	abbreviated name of month	'Jan'
%H	two-digit hour in the range 00 .. 23	'01'
%I	two-digit hour in the range 00 .. 12	'01'
%j	three-digit day of year in the range 001 .. 366	'001'
%k	hour in the range 0 .. 23, like %H, padded with a blank	' 1'
%l	hour in the range 0 .. 12, like %H, padded with a blank	' 1'
%m	month in the range 01 .. 12	'05'
%M	minute in the range 00 .. 59	'01'
%n	newline	(a newline, '\n')
%p	AM or PM	'PM'
%P	am or pm	'pm'
%Q	milliseconds	'210'
%R	hour and minute, like %H:%M	'01:00'
%s	number of seconds since the epoch; when given a format string for second argument: milliseconds	'0' 451
%S	seconds in the range 00 .. 60	'01'
%t	tabulator	(a tab, '\t')
%T	time of day of format %H:%M:%S	'01:00:00'
%u	day of week as a decimal number range 1 (Monday) to through 7	'4'
%U, %V	week number in the range 00 ..53, starting with the first Sunday as the first day of the first week	'18'
%w	day of week in the range 0 (Sunday) to 6	'1'
%W	week number of current year in the range 00 .. 53, starting with the first Monday as the first day of the first week	'18'
%x	preferred date representation	'05/03/17' for UK

Spec	Meaning	Example
%X	preferred time of day representation	'21:09:02' for UK
%y	year in the range 00 .. 99	'17'
%Y	full year number	'1949'
%z	numeric time zone	Windows returns time zone as words
%Z	abbreviation of time zone	Windows returns time zone as words
%%	character '%'	'%'

Please note that the behaviour is undefined if the date passed is earlier than the epoch.

See also: **astro.cweek**, **astro.hdate**, **os.now**, **os.time**, **utils.checkdate**.

os.datetosecs ([obj])

os.datetosecs (year, month, day [, hour [, minute [, second]]])

In the first form, receives a date and optionally time of the form `year`, `month`, `date` [, `hour` [, `minute` [, `second`]]], with all values in table, sequence or register `obj` being integers, and transforms it to the number of seconds elapsed since the start of an `epoch` (usually January 01, 1970 UTC, try also `os.now(0)`). By default, `hour`, `minute`, and `second` are all 0. If no argument is given, returns the number of seconds elapsed from the epoch till the current date and time.

In the second form, receives the given integers, and conducts the same operation.

The time zone acknowledged may depend on your operating system.

The function returns -1 if the date is older than the start of the epoch.

See also: **os.time**, **os.secstodate**, **utils.checkdate**.

os.difftime (t2, t1)

Returns the number of seconds from time `t1` to time `t2`. In POSIX, Windows, and some other systems, this value is exactly `t2-t1`.

See also: **time**, **os.time**.

os.dirname (path)

Returns the directory name of the given `path`, a string. If `path` has no separator, then the function returns `'.'`. If you would like to test relative paths, apply **os.realpath** to `path` before calling this function. In DOS-like systems, when only a drive letter along with a colon is passed, then a trailing slash is appended to the result, e.g. `'c:'` → `'c:/'`.

See also: **os.filename**, **os.findinpath**, **os.isdir**, **os.isvalidpath**, **os.prefix**, **os.suffix**.

os.drives ()

In Windows, OS/2 and DOS, the function returns all the logical drives available at the local computer. The return is a sequence of drive letters. In DOS, floppy drives are not checked to avoid "Insert floppy disk" messages. In other systems, the return is **fail**.

os.drivestat ([drive])

The function returns information of the given logical drive in a table. In DOS-based systems, the drive letter may be followed by a colon. In UNIX/Mac, just pass `'.'`. The following data will be returned:

Key	Meaning
`'label'`	the drive label (Windows, OS/2, DOS)
`'filesystem'`	the file system (e.g. NTFS, FAT32, HFS, JFS, HPFS, etc.)
`'drivetype'`	Windows, OS/2, DOS only: the type of the drive, i.e. 'Removable', 'Fixed', 'Remote', 'CD-ROM', or 'RAMDISK'. 'ResidentCharacter' and 'PseudoCharacter' are OS/2-specific character devices.
`'freesize'`	the number of free space in bytes
`'totalsize'`	the total number of physical bytes
`'usedsize'`	the number of occupied space in bytes (Windows only)
`'totalclusters'`	total number of clusters (OS/2: allocation units)
`'freeclusters'`	number of free clusters (OS/2: allocation units)
`'freeuserclusters'`	number of free clusters to non-superusers (UNIX/Mac only)
`'sectorsper cluster'`	number of sectors per cluster
`'bytespersector'`	number of bytes per sector
`'maxnamelength'`	maximum number of characters in a filename or folder name
`'maxpathlength'`	total absolute path limit
`'totalnodes'`	total number of nodes (UNIX/Mac only)
`'freenodes'`	number of free file nodes (UNIX/Mac only)
`'trim'`	solid-state disk indicator (Windows only)
`'dosdevice'`	DOS device name (Windows only)
`'serialnumber'`	serial number of the drive
`'fatsize'`	number of bits used by the FAT (12, 16 or 32, DOS only)
`'isfat32'`	FAT32-formatted drive (DOS, OS/2, Windows)
`'driveletter'`	drive letter of device with colon (OS/2, DOS, Windows)
`'iType'`	OS/2 only: 1 = Resident character device (e.g. COM1, LPT1, PRN, SCREEN$, KBD), 2 = Pseudo-character device, 3 = Local drive (physical or logical drive directly connected to the system), 4 = Remote drive attached (a drive mounted over the network or from another external subsystem via a file system driver)

Key	Meaning
`'idFileSystem'`	unknown meaning, FSALLOCATE data type item `idFileSystem`(OS/2 only)
`'rgFSAData'`	FSD Attach Data, returned only if available (OS/2 only).

In Windows, the function also returns various flags such as Unicode support, disk quotas, sparse files, compressed volume, read-only volume, etc. in the 'flags' field plus NTFS version information in the 'ntfsver' field.

In other systems, the return is **fail**.

If called without arguments or if `drive` is **null**, the function automatically determines the `current drive`.

Example:

```
> os.drivestat('c:'):  # get information on drive C:\
[bytespersector ~ 512, drivetype ~ Fixed, filesystem ~ NTFS, freeclusters ~
62051077, freesize ~ 254161211392, label ~ <none>, sectorspercluster ~ 8,
totalclusters ~ 122070527, totalsize ~ 500000878592]
```

See also: **os.cdrom**, **os.drives**, **os.ismounted**, **os.isremovable**, **os.isvaliddrive**.

os.endian ()

Determines the endianness of your system. Returns 0 for Little Endian, 1 for Big Endian, and **fail** if the endianness could not be determined.

See also: **os.cpuinfo**.

os.environ ()

Returns all environment variables of the underlying operating system and their current settings as a table of key ~ value pairs of type string.

See also: **os.getenv**, **os.getwinsysdirs**, **os.setenv**.

os.esd ([year, month, day [, hour [, minute [, second]]]])

os.esd (x)

In the first form, the function computes the Excel Serial date for the given date or - if no argument is given - the current date and time. The Excel Serial Date represents the number of days that have elapsed since 31st December 1899, 00:00h, where midnight January 01, 1900 is day 1 and is expressed in local time, taking Daylight Saving Time into account if active.

With any arguments, at least `year`, `month`, and `day` - all numbers - must be given. Optionally, you may add an `hour`, `minute`, or `second`, where all three default to 0. The arguments can also be passed in a table or sequence.

The returns is a number, where the fractional portion represents the decimal time.

In the second form, if the Excel Serial Date `x` - a number - is given, the function returns the corresponding Gregorian year, month, day, the decimal fraction of the day - in the range [0, 1) -, the hour, minute, and second, all numbers. `x` may be 60, returning February 29, 1900.

In case of a non-existing date or if the date is older than the start of the epoch, the function issues an error. The only exception is February 29, 1900 with day 60, that is the bug in the original Lotus 1-2-3 formula.

Note that the Lotus Serial Date computed by **os.lsd** does not take account of daylight saving time.

See also: **astro.jdate**, **os.lsd**, **os.now**, **os.usd**, **utils.checkdate**.

os.execute ([command [, option]])

This function is equivalent to the C function `system`. It passes the string `command` to be executed by an operating system shell. It returns a status code, which is system-dependent. If `command` is absent, then it returns non-zero if a shell is available and zero otherwise.

If any `option` is given, the function runs `command` and returns the entire output of the command as one string. Any carriage returns (`'\r'`) are removed from the result, but keeps newlines (`'\n'`) untouched.

See also: **io.pcall**.

os.exists (filename)

Checks whether the given file or directory (`filename` is of type string) exists and the user has at least read permissions for it. It returns **true** or **false**.

See also: **os.findinpath**, **os.isdir**, **os.isfile**, **os.issysdir**, **os.isvalidpath**.

os.exit ([code])

In the first form, calls the C function `exit`, with an optional `code` to be passed to the environment in which Agena has been started, to terminate the host programme.
The default value for `code` is the success code, usually 0. (In Windows, query ERRORCODE in the shell for the exit status.)

The function by default also closes the interpreter state - this can be prevented by passing the optional Boolean value **false**.

os.faccess (path [, mode])

os.faccess (path [, flags])

Checks whether a directory or file can be accessed. `path` represents the path to the file, directory or symbolic link.
In the first form, `mode` specifies the accessibility through the integer constants:

- **os.f_ok** = check for existence
- **os.x_ok** = file is executable
- **os.w_ok** = write access is granted
- **os.r_ok** = read access is permitted (the default)

You can either specify the constant **os.f_ok** for `mode`, or a bitwise-OR mask of the constants **os.r_ok**, **os.w_ok** and/or **os.x_ok** created by calling the || operator, see example below.

Please note that the integer values these constants represent vary across platforms, so it is recommended to always use the constant names in the calls to the function.

In the second form, a string `flags` of one or more unique characters, in any order, specifies the permissions to be queried:

- `'f'` = check for existence
- `'x'` = file is executable
- `'w'` = write access is granted
- `'r'` = read access is permitted (the default)

You cannot mix the `'f'` flag with the other flags.

In both forms, if at least one bit in the mask asked for a permission is denied, the function returns **false**, and **true** otherwise. If the specified access is not granted, a string describing the kind of error will be returned, too.

Example:

```
> # check for read access to a non-existent folder
> os.faccess('nofolder'):
false   file or directory does not exist

> # check for both read and write access to an existing file
> os.faccess('myfile.txt', os.r_ok || os.w_ok):
true

> # dito
> os.faccess('myfile.txt', 'rw'):
true
```

See also: **os.fattrib**, **os.fstat**, **os.exists**.

os.fattrib (fn, mode)
os.fattrib (fn, oct)
os.fattrib (fn, time)

In the first form, sets or deletes file permission flags given by the `mode` string to the file denoted by the filename `fn`.

The `mode` argument must consist of at least three characters and have the following form:

Character 1	Character 2	Character 3, etc.
`'u'` - user	`'+'` - add permission	`'r'` - read permission
`'g'`- group	`'-'` - remove permission	`'w'` - write permission
`'o'`- other		`'x'` - execute permission
`'a'`- user, group, and other		

The first character in `mode` denotes the owner of the file, the second character indicates whether to set or delete a permission, and the following characters indicate which permissions to set or remove.

In Windows and OS/2 the following permission flags are additionally supported:

Character 3, etc.
`'a'` - archive flag
`'s'` - system flag
`'h'` - hidden flag
`'r'` - read-only flag

In the second form, the file mode is set according to the *octal* number `oct`. This number is the same as the numeric argument to the UNIX chmod command, so - for example - pass 0o444 (instead of 444) to the function to set a file to read-only mode for all users.

In the third form, the function changes the modification and access time of the file denoted by its name `fn` to the date and time given in table `time`. The table must include at least integers representing a year, month, and day. It may optionally include an hour, a minute, and a second. If they are missing, they default to zero.

File timestamps can only be changed in OS/2, UNIX, Windows, Mac OS X, and DOS.

The function returns **true** on success, and **fail** otherwise.

Examples:

```
> os.fattrib('file.txt', 'a-wx');  # deletes write and execute permissions

> os.fattrib('file.txt', 0o444);   # sets read-only for all users

> os.fattrib('file.txt', [2012, 05, 23, 12, 30, 0]); # sets timestamp
```

See also: **os.fstat**, **os.now**.

os.fcopy (infile, outfile [, overwrite])

os.fcopy (infile, dir [, overwrite])

In the first form, copies the file and its permissions denoted by the filename `infile` to the new file called `outfile`. If `outfile` already exists, an error will be issued, but you may overrule this by passing **true** for `overwrite`. The function internally uses **environ.kernel['buffersize']** for the number of bytes to be copied at the same time, which you may change to another positive integer.

In the second form, the function copies the file `infile` to the existing directory `dir`.

The function returns **true** on success, and **fail** and `infile` otherwise. It also returns **fail** and `infile` if the file could be copied, but the file permissions could not be set. The function issues errors if a file could not be read or created, or if the source and target file are identical.

Use **skycrane.fcopy** if you want to use wildcards/file globbing.

os.filename (path [, option])

Returns the filename of the given `path`, a string. This is equivalent to the C basename function. If you would like to test relative paths, apply **os.realpath** to `path` before calling this function.

If any `option` is given in Windows, then the 8.3 DOS filename is also returned if the file exists, otherwise **null** will be returned as the second result.

See also: **os.dirname**, **os.findinpath**, **os.getfullpathname**, **os.isfile**, **os.isvalidpath**, **os.prefix**, **os.realpath**, **os.suffix**.

os.findinpath (path, object [, option])

Searches for the file or folder `object` starting from the fully resolved folder `path` given by and moving upwards toward the file system root. `path` may refer to a file or folder. If found, the function returns the fully resolved path to `object`, or **null** if it could not find it. It also returns **null** if `path` or `object` are empty strings.

The function checks for the existence of files and folders.

If you pass a file for `path` and want to have its name stripped off in the result, set `option` to **true**, see example below. The default is **false**.

Some Windows examples: First get the fully resolved path to the running Agena executable, but any other path will do, too.

```
> s := os.getmodulefilename():
c:/agena/src/agena.exe
```

There definitely is a 'src' folder in s:

```
> os.findinpath(s, 'src'):
c:/agena/src
```

Since on the same file hierarchy level there are a 'lib' and an 'src' folder, we get:

```
> os.findinpath(s, 'lib'):
c:/agena/lib
```

There is no subfolder named 'lib/library':

```
> os.findinpath(s, 'lib/library'):
null
```

But there is the main library file in the 'lib' folder (which is read at Agena startup):

```
> os.findinpath(s, 'lib/library.agn'):
c:/agena/lib/library.agn

> os.findinpath(s, 'lib/library.agn', true):
c:/agena/lib
```

See also: **os.dirname**, **os.exists**, **os.filename**, **os.getmodulefilename**.

`os.freemem ([unit])`

Returns the amount of free physical RAM available on Windows and Mac OS X and UNIX machines. In OS/2, the function returns the amount of free virtual RAM.

If no argument is given, the return is in bytes. If `unit` is the string `'kbytes'`, the return is in kBytes; if `unit` is `'mbytes'`, the return is in Mbytes; if `unit` is `'gbytes'`, the return is in Gigabytes; if `unit` is `'tbytes'`, the return is in TeraBytes. On other architectures, the function returns **fail**.

See also: **environ.used**, **os.meminfo.**

os.fstat (fn)

os.fstat (fh)

Returns information on the file, symbolic link (UNIX and Windows only), or directory given by the string `fn` in a table. You can also pass a file handle `fh` returned by **io.open** or **binio.open**, but not in OS/2.

The table includes the following information:

Key	Meaning
`'mode'`	`'FILE'` if `fn` is a regular file, `'LINK'` if `fn` is a symbolic link (UNIX and Windows only), `'DIR'` if `fn` is a directory, `'CHARSPECFILE'` if `fn` is a character special file (a device like a terminal), `'BLOCKSPECFILE'` if `fn` is a block special file (a device like a disk), or `'OTHER'` otherwise
`'length'`	the size of the file in bytes
`'compressed'`	the compressed size of the file in bytes (Windows only)
`'date'`	last modification date in the form yyyy, mm, dd, hh, mm, ss
`'lastaccess'`	last file access date in the form yyyy, mm, dd, hh, mm, ss
`'attribchange'`	last file attribute change date in the form yyyy, mm, dd, hh, mm, ss
`'perms'`	file attributes coded in a decimal integer, use **math.convertbase** to convert the integer x into its octal representation (from base 10 to base 8).
`'bits'`	The permission bits, a string similar to that in UNIX and DOS, e.g. '-rw-rw-r--:-----' or '----------:-drhas' where the bits to the left of the colon are set in the UNIX and DOS versions of Agena, while in Windows and OS/2, the bits to the right of the colon are set. The letters indicate: 'r' - read permission granted (UNIX & DOS) 'w' - write permission granted (UNIX & DOS) 'x' - execute permission granted (UNIX & DOS) 'd' - indicates directory (OS/2 only) 'r' - readonly file (OS/2 and Windows) 'h' - hidden file (OS/2 and Windows) 'a' - archived file (OS/2 and Windows) 's' - system file (OS/2 and Windows)
`'owner'`, `'group'`, `'other'`	Access permissions to the file or directory are returned with the `owner`, `group` (UNIX only), and `other` (UNIX only) keys which each reference tables with information on `read`, `write`, and `execute` permissions. These tables have the following form: `['read' ~` <boolean>`, 'write' ~` <boolean>`, 'execute' ~` <boolean>`]`, where <boolean> is either **true** or **false**. In OS/2 and Windows, the file attributes `'hidden'`, `'readonly'`, `'archived'`, and `'system'` are also returned in the subtable with key 'owner'.

Key	Meaning
'blocks'	(UNIX only) Disk space occupied by the file, measured in units of 512-byte blocks.
'blocksize'	(UNIX only) Optimal block size for reading or writing this file, in bytes.
'device'	(Windows & UNIX-like OSs only) Volume serial number; only returned when given a file name, not a handle.
'inode'	(OS/2, Windows & UNIX-like OSs only) Unique file serial number/index node. In OS/2 & Windows, a *pair* of two 32-bit integers representing the higher and lower parts is returned. On all other systems it is a pair, where the left-hand side is always 0, and the right-hand side a 32-bit integer depicting the actual inode. Only returned when given a file name, not a handle.
'dosname'	(Windows only) 8.3 DOS name of the file
'binarytype'	binary type of an executable: "Win16", "Win32", "Win64", "DOS", "OS2/16", "PIF", "POSIX" or "unknown"
'uid'	(UNIX only) user ID
'gid'	(UNIX only) group ID

On DOS-based systems append a trailing slash to drive letters if you want to check an entire drive, such as `'c:/'`, otherwise the function will return **fail**.

See also: **os.fattrib**.

os.ftok (path, id)

The function uses the identity of the file, i.e. its index node (inode, see **os.fstat**) named by the given `path` name, a string, and the least significant 8 bits of `id` (which must be non-zero) to generate a signed 4-byte integral System V IPC (Inter Process Communications) key.

On UNIX platforms, the return is one integer, and on OS/2 & Windows two integers, the first for the higher part of the file index and the second for its lower part. On all other systems, the function issues an error.

The result is the same for all `path`names that name the same file, when the same value of `id` is used.

The value returned should be different when the files stored at various locations or the given `id`s differ.

See also: **hashes.ftok**.

os.getadapter ()

On Windows, returns various information on the network adapters installed on the host: MAC addresses, adapter names, IPs, masks, gateways, types, plus DHCP and WINS availability.

On all other platforms, returns **fail**.

See also: **os.getip**, **os.getmac**, **os.netdomain**.

os.getcommandline ()

Returns the complete command-line string used to launch the current Agena process, including the executable name and all passed flags or arguments.

In Windows, the UTF-8 narrow string returned preserves Unicode characters (e.g., diacritics, international scripts). On all other operating systems, the function dynamically reconstructs the command line by iterating through the global `argc` and `argv` C argument vectors, joining them sequentially with space separators.

On all platforms, the programme name is always separated from all the other arguments by two spaces instead of just one, thus fully emulating the Windows C API function GetCommandLineW().

See also: **args** argument system table, **split** operator, **strings.trim**.

os.getconsolemode ()

On Windows only, returns the current output mode of a console screen buffer, an integer. The function gives direct access to the Win32 C API function GetConsoleMode(). See also: **os.codepage**, **os.setconsolemode**.

os.getdirpathsep ()

Returns both the directory and path separators of the underlying platform. Examples are `'\'` and `';'` in DOS-based systems.

os.getenv (varname)

Returns the value of the system environment variable `varname`, or **null** if the variable is not defined.

See also: **os.setenv**, **os.environ**, **os.getwinsysdirs**.

os.getextlibpath ()

In OS/2, returns the current paths to be searched before and after system LIBPATH, when trying to locate DLLs.

The first return is the path to be searched before the LIBPATH, the second one after the LIBPATH.

The function is not available for other operating systems.

See also: **os.setextlibpath**.

os.getfullpathname (path)

This function takes a string `path`, converts it to a wide string, calls the Win32 C API function GetFullPathNameW, and returns the absolute path. The function is available in the Windows edition only.

See also: **os.filename**, **os.realpath**.

os.getip ([domain])

Returns the IPv4 address for a given `domain`, a string. If no `domain` is given, the function tries to retrieve the IP of the host. On Windows, Solaris and Linux, the function also returns the IPv6 address of `domain`. The function does not work in DOS.

See also: **os.getadapter**, **os.getmac**.

os.getlanguage (id)

Returns the name of the language by number `id`. The return is a string. Available in Windows only. See also: **os.getlocale**.

os.getloadeddlls ([pid])

In Windows only, returns Agena's current process id. It also returns all the DLLs along with their absolute paths used by the interpreter as a table. You can also pass any valid Windows process id `pid` to explore the modules loaded by another application. On other platforms or in case of an error, returns **null**.

See also: **os.getmodulefilename**, **os.getprocesses**.

os.getlocale ()

Returns various information on the current locale including decimal point and thousands separators, currency, and monetary formatting suggestions. The return is a table of the key~value pairs listed below. A value of " (the empty string) means `unspecified`.

Key	Value	Value type
`'locale'`	current locale of your system	string
`'charset'`	current character classification category of the C locale	string
`'decimal_point'`	decimal-point separator	string
`'thousands_sep'`	thousands separator	string
`'int_curr_symbol'`	international currency symbol according to international standard ISO 4217 "Codes for the Representation of Currency and Funds"	string
`'currency_symbol'`	local currency symbol like XXX	string

Key	Value	Value type
'mon_decimal_point'	decimal point separator for monetary amounts	string
'mon_thousands_sep'	thousands separator for monetary amounts	string
'positive_sign'	symbol for positive values	string
'negative_sign'	symbol for negative values	string
'int_frac_digits'	recommended number of decimal places of monetary amounts according to international standard	number
'frac_digits'	recommended number of decimal places of monetary amounts according to local standard	number
'p_cs_precedes'	recommendation whether currency symbol precedes positive monetary amount	boolean
'n_cs_precedes'	recommendation whether currency symbol precedes negative monetary amount	boolean
'p_sep_by_space'	recommendation whether currency symbol and non-negative monetary amount are separated by a blank	boolean
'n_sep_by_space'	recommendation whether currency symbol and negative monetary amount are separated by a blank	boolean
'p_sign_posn', 'n_sign_posn'	indicator how to position the sign for non-negative and negative monetary quantities: 0: currency symbol and quantity to be enclosed in parentheses 1: print sign before quantity and currency 2: print sign after quantity and currency 3: print sign right before currency symbol 4: print sign right after currency symbol any other number: unspecified	number
'grouping'	unknown meaning *)	string
'mon_grouping'	ditto	string
'keyboard'	(Windows only) for an attached keyboard, the return has the following fields: 'hex' depicts the language ID as a hexadecimal string; and 'name' contains the associated language name as a string; 'langid' is the language ID, an integer. 'primarylangid' includes the primary language ID as an integer; 'sublangid' represents the sublanguage ID as an integer; 'UserDefaultLangID' and 'SystemDefaultUILangID' are integers. 'UserDefaultLangName' and 'SystemDefaultUILangName' are strings and represent the respective Windows settings.	table

Key	Value	Value type
	Warning: On mixed systems (e.g. default language UK and German keyboard) the information returned may be wrong.	

*) See description of the C function localeconv on the web.

See also: **environ.decpoint**, **os.codepage**, **os.getlanguage**, **os.setlocale**, **os.shellinfo**.

os.getmac (ip)

For the given IPv4 address, returns the associated MAC address on the host or **fail** if it could not be found. The function works in Windows only.

See also: **os.getadapter**, **os.getip**.

os.getmodulefilename ()

Returns the absolute path to the currently executing programme - that is Agena - as a string. On non-supported platforms or in case of an error, the function returns **null**.

See also: **os.findinpath**, **os.getloadeddlls**, **os.pid**.

os.getprocesses ()

The function returns all processes running on the Windows system, per process id (PID). It returns a table with the keys the PIDs and the values tables with the process name, the number of handles and a description.

The function works in Windows 2000 and higher and returns **fail** otherwise.

See also: **os.getloadeddlls**.

os.gettemppath ()

Retrieves the path of the directory designated for temporary files, of type string. Note that on non-Windows systems, the function issues an error if the environment variables TEMP, TMP and TMPDIR are all unassigned.

See also: **os.tmpdir**, **os.tmpname**.

os.getthreadid ()

Returns unique thread IDs, the system-specific value as the first result and the POSIX Thread ID (Pthread ID) as a second.

On Windows, the first ID returned depicts the thread ID assigned by the operating system on which Agena is running. The second return is always zero as POSIX Thread IDs cannot be generated there.

In Mac OS X, the first ID is the Mach thread identifier and the second the POSIX Thread ID.

In Solaris, the first ID is the the Lightweight Process (LWP) ID and the second the POSIX Thread ID.

The results may be zero on other platforms.

You can use the return as a seed. See also: **os.getthreadseed**.

os.getthreadseed ()

Generates an unsigned 32-bit integer seed XORing the platform specific thread ID and the current time in seconds.

See also: **os.getthreadid**.

os.getusbdevices ()

Lists all USB devices currently attached to the host. The function returns a table where each entry is a table representing a USB device.

Each device table contains the properties:

- `'description'`: Device Description (e.g., "USB Composite Device"),
- `'hardware_id'`: Hardware ID (e.g., "USB\VID_045E&PID_00CB"),
- `'friendly_name'`: more user-readable Friendly Name (e.g., "Logitech USB Mouse"),
- `'manufacturer'`: the manufacturer,
- `'service'`: Service (e.g., "usbhub"),
- `'instance_id'`: Device Instance ID (unique identifier for this specific device instance)

The function is available in Windows XP and higher only.

os.getwinsysdirs ()

Returns both the Windows directory and the system directory, in this order. On all other operating systems, returns **fail** twice. See also: **os.environ**, **os.getenv**.

os.groupname (groupname)

os.groupname (gid)

The function receives a group name (a string) or a group id (an integer) and returns a table with keys `'groupname'` denoting the group name (a string) and `'gid'`

denoting the group id (a number). It is available in the UNIX versions of Agena, only.

On all other systems, the function just returns **fail**.

See also: **os.login**, **os.userinfo**.

os.hasnetwork ()

The function returns **true** if the system is connected to any network, and **false** otherwise. The function is available in Windows, only. The result is usually **true**. On all other architectures, the function returns **fail**.

os.inode (path)

On UNIX-like systems and DOS, for the given `path`, a string, the function returns the device ID and the index node (inode), i.e. two unsigned 4-byte integers.

On OS/2 and Windows, the function returns the volume serial number, an unsigned 4-byte integer, along with two additional unsigned 4-byte integers representing the higher and lower parts of the file index.

On all other platforms, the function issues an error.

os.isansi ()

Returns **true** on Agena editions compiled with the LUA_ANSI (strict ANSI C) option, and **false** otherwise.

See also: **os.isdos**, **os.islinux**, **os.ismac**, **os.isos2**, **os.isppc**, **os.isunix**, **os.iswindows**, **os.isx86**.

os.isarm ()

Returns **true** if Agena is run on an ARM CPU, 32- or 64-bit, and **false** otherwise. If run on an ARM CPU, the function might return **fail** if it could not read out CPU data.

See also: **os.isamd**, **os.iscyrix**, **os.isintel**, **os.isnexgen**, **os.isnsc**, **os.isvia**.

os.isarm32 ()

Returns **true** if Agena is run on ARM Raspberry Pi OS 32-bit, and **false** otherwise.

See also: **os.isarm**, **os.isarm64**, **os.israspi**.

os.isarm64 ()

Returns **true** if Agena is run on ARM Raspberry Pi OS 64-bit, and **false** otherwise.

See also: **os.isarm**, **os.isarm32**, **os.israspi**.

os.isdir (path)

Checks whether the given `path` refers to a directory and returns **true** or **false**.

See also: **os.chmod**, **os.chown**, **os.dirname**, **os.exists**, **os.isfile**, **os.islink**, **os.issysdir**, **os.isvalidpath**.

os.isdocked ()

The function returns **true** if the computer is in docking mode, and **false** otherwise. The function is available in Windows, only. On all other architectures, the function returns **fail**.

os.isdos ([any])

Checks whether Agena is run in the DOS environment and returns **true** or **false**.

The procedure is quite dumb: if you are running the DOS version of Agena, it will always return **true** regardless whether it is actually being run in DOS, OS/2 or Windows.

If you pass `any` argument, the function returns additional information in the following order: the name of the DOS edition, the official major and minor version if available, and the internal major and minor version from which you may deduce the actual DOS version. Furthermore, the internal OEM version number if returned. In FreeDOS, the kernel version number will also be given as the last return.

See also: **environ.os**, **os.isansi**, **os.isdow**, **os.islinux**, **os.ismac**, **os.isos2**, **os.isunix**, **os.iswindows**.

os.isdow ()

Checks whether Agena is run in the DOS, OS/2 or Windows environment and returns **true** or **false**.

See also: **os.isdos**, **os.isos2**, **os.iswindows**.

os.isdriveletter (d)

Checks whether its input string `d` represents a drive letter, such as `c:`, `d:/` or `e:\`. The function does not check whether the drive exists on your system.

See also: **os.isvaliddrive**, **os.realpath**.

os.isdst ([year, month, day [, hour [, minute [, second]]]])

Receives a date and optionally time of the form `year`, `month`, `date` [, `hour` [, `minute` [, `second`]]], with all values being integers, checks whether Daylight Saving Time is active for a given date. Alternatively, you may pass the date and time in a table, sequence or register.

The function returns true or false. By default, `hour`, `minute` and `second` are 0.

If no argument is given, returns the number of seconds elapsed from the epoch till the current date and time.

The function issues an error if a non-existing date has been passed.

See also: **os.date**, **os.tzdiff**, **utils.checkdate**.

os.isfile (path)

Checks whether the given `path` (a string) points to a file. The function returns **true** on success, **false** if the path points to another existing file system object (such as a directory), and **fail** if `path` does not exist on the file system at all.

See also: **os.chmod**, **os.chown**, **os.exists**, **os.filename**, **os.fstat**, **os.isdir**, **os.islink**, **os.issysdir**, **os.isvalidpath**.

os.islink (path)

Checks whether the given `path` (a string) points to a file. It returns **true** on success, **false** if the path points to another existing file system object, and **fail** if `path` does not exist on the file system at all.

See also: **os.chmod**, **os.chown**, **os.filename**, **os.fstat**, **os.isfile**, **os.issysdir**, **os.isvalidpath**, **os.mklink**.

os.islinux ()

The function determines whether Agena runs on Linux and returns **true** or **false**.

See also: **environ.os**, **os.isansi**, **os.isdos**, **os.islinux386**, **os.ismac**, **os.isos2**, **os.issolaris**, **os.isunix**, **os.iswindows**.

os.islinux386 ()

The function determines whether the Agena executable has been compiled on 32-bit x86 Linux and returns **true** or **false**.

See also: **os.isarm32**, **os.isarm64**, **os.islinux**, **os.israspi**.

os.islocale (l)

The function checks whether the given locale `l` - represented as a string - is supported by the operating system and returns **true** or **false**. If the locale is supported, a description (of type string) will be returned as a second result.

The function - contrary to **os.setlocale** - never changes the current locale.

Examples for locales are `'UK'` for the United Kingdom (at least in Windows localised to the United Kingdom), `'he_IL'` for Hebrew (Israel), `'de_AT'` for Austrian and `'zh_Hans_SG'` for Simplified Chinese (Singapore).

os.ismac ()

The function determines whether Agena runs on Mac OS X (Darwin) and returns **true** or **false**.

See also: **environ.os**, **os.isansi**, **os.isdos**, **os.islinux**, **os.isos2**, **os.issolaris**, **os.isunix**, **os.iswindows**.

os.ismounted (d)

Checks whether the given drive `d` has been mounted. It is available in the Windows edition of Agena only. `d` may be the single drive letter, optionally combined with a colon.

See also: **os.cdrom**, **os.drived**, **os.drivestat**, **os.isdriveletter**, **os.isremovable**, **os.isvaliddrive**.

os.isos2()

The function determines whether Agena runs on OS/2 and returns **true** or **false**.

See also: **environ.os**, **os.isansi**, **os.isarm**, **os.isdos**, **os.islinux**, **os.ismac**, **os.issolaris**, **os.isunix**, **os.iswindows**.

os.isppc ()

Returns **true** if Agena is run on a PowerPC CPU, 32- or 64-bit, and **false** otherwise.

See also: **os.isarm**, **os.isdos**, **os.islinux**, **os.ismac**, **os.isos2**, **os.isunix**, **os.iswindows**, **os.isx86**.

os.israspi ()

Checks whether Agena is being run on a Raspberry Pi and returns **true** or **false**.

See also: **os.isarm**, **os.isarm32**, **os.isarm64**, **os.islinux**.

os.isremovable (d)

Checks whether the given drive `d` is removable. It is available in the Windows and DOS editions of Agena only. `d` may be the single drive letter, optionally combined with a colon.

See also: **os.cdrom**, **os.drives**, **os.drivestat**, **os.ismounted**, **os.isvaliddrive**.

os.issolaris ()

The function determines whether Agena runs on Sun Solaris and returns **true** or **false**.

See also: **environ.os**, **os.isansi**, **os.isdos**, **os.islinux**, **os.ismac**, **os.isos2**, **os.isunix**, **os.iswindows**.

os.issysdir (path)

Checks whether the given `path` refers to a Windows system directory, such as 'PerfLogs' or 'AppData', and returns **true** or **false**. If `path` is invalid, the function returns **fail**. A whole drive is not considered a system directory. On operating systems other than Windows, the function always returns **false**.

See also: **os.chmod**, **os.chown**, **os.dirname**, **os.exists**, **os.isdir**, **os.isfile**, **os.islink**, **os.isvalidpath**.

os.isunix ()

Returns **true** if Agena is being run in a UNIX environment (i.e. Solaris, Linux, and OpenSolaris), and **false** otherwise.

The function is written in Agena and included in the `libary.agn` file.

See also: **environ.os**, **os.isansi**, **os.isdos**, **os.islinux**, **os.ismac**, **os.isos2**, **os.issolaris**, **os.iswindows**.

os.isvaliddrive (d)

Checks whether the given drive `d` is part of the file system. It is available in the OS/2, DOS and Windows edition of Agena. `d` may be the single drive letter, optionally combined with a colon. On all other platforms, the function returns **fail**.

See also: **os.cdrom**, **os.drivestat**, **os.ismounted**, **os.isremovable**.

os.isvalidpath (path)

Checks whether the given filename or `path`, a string, is syntactically correct. It does *not* check whether the file object actually exists on your file system.

The function returns two values, **true** or **false**, the first result indicating whether `path` is valid in Windows, and the second result whether it is valid in UNIX.

See also: **os.isdir**, **os.isfile**, **os.issysdir**.

os.iswindows ()

Checks whether the Agena version for Windows is being run and returns **true** or **false**.

See also: **environ.os**, **os.isansi**, **os.isarm**, **os.isdos**, **os.islinux**, **os.ismac**, **os.isos2**, **os.issolaris**, **os.isunix**.

os.isx86 ()

Returns **true** if Agena is run on an x86-compatible CPU, 32- or 64-bit, and **false** otherwise. The function is hard-coded with the result determined at compile-time.

See also: **os.isamd**, **os.isarm**, **os.isarm32**, **os.isarm64**, **os.isdos**, **os.isintel**, **os.islinux**, **os.islinux386**, **os.ismac**, **os.isos2**, **os.isppc**, **os.isunix**, **os.iswindows**, **os.vendor**.

os.iterate (path [, option])

Creates an iterator that when called traverses directory `path`. You may also use the `'.'` , `'*'` or `'*.*'` abbreviations for `path`, which all indicate the current working directory. The iterator returns the name of the file, link, directory, etc.

The optional boolean second argument **true** causes the iterator to additionally return the type, i.e. `'FILE'`, `'DIR'` (for directory), `'SYSDIR'` (for Windows system directory), `'LINK'`, `'CHARSPECFILE'`, `'BLOCKSPECFILE'` or `'OTHER'`. After the directory has been completely traversed, the function returns **null**.

The iterator does not return the `'.'` and `'..'` placeholders depicting the current and parent directories, respectively. In Windows, if `path` points to a system directory such as `PerfLogs` or `AppData`, for example, the iterator will just return **null**. The

iterator will also return **null** if the directory exists but you do not have permissions for it.

The iterator cannot recurse into a subdirectory, just create another iterator instead.

See also: **os.chdir**, **os.list**, **os.listcore**.

os.list (d [, options])

Lists the contents of a directory `d` (given as a string) by returning a table of strings denoting the files, subdirectories, and links. The second return is a string with the absolute path to the main directory scanned.

If `d` is **null** or the empty string, the current working directory is evaluated. If the return is **null** and a warning text, then `d` does not exist.

`d` may include the `?` and `*` jokers known from UNIX, OS/2, DOS or Windows to select a subset of files, e.g. `os.list('*.c')` to return all files with suffix `.c`. Jokers can only be used to select files, but not to parse subdirectories if they exist.

If no option is given, files, links, and directories are returned. If the optional argument `'files'` or `'file'` is given, only files are returned. If the optional argument `'dirs'` or `'dir'` is given, directories are returned exclusively. If the optional argument `'links'` or `'link'` is given, links are returned (UNIX only).

The `'r'` option forces a recursive descent into all subfolders of `d`. Multiple options can be given.

If `d` is `'.'`, then the current working directory is examined. If `d` is `'..'`, then the directory one level higher than the current one is searched.

If the string `'r'` is passed as an option, the function traverses all subfolders in `d`.

The function is written in Agena and included in the lib/library.agn file.

See also: **os.iterate**, **os.countcore**, **os.listcore**.

os.listcore (d)

os.listcore (d [, options] [, pattern])

In the first form, returns a table with all the files, links and directories in the given path `d`. If `d` is void or the string `'.'`, the current working directory is evaluated. It is the core function used by **os.list**.

In the second form, by giving at least one of the options `'files'` or `'file'`, `'dirs'` or `'dir'`, or `'links'` or `'link'`, the file, directory name, or link names are returned, respectively. These three options can be mixed.

Another option may be a `pattern` of type string which can include the wildcards `?` or `*`. If given, the function will only return those entries which match this pattern.

The function does not support the file system root, that is when `d` is 'c:\' or '/'. In this case, call **os.list** which conducts a proper conversion before listing the contents by calling **os.listcore**.

See also: **os.countcore**, **os.iterate**, **os.whereis**.

os.login ()

Returns the login name of the current user as a string. The return is a string. In DOS, the function returns **fail**. See also: **os.username**, **os.groupname**.

os.lsd ([year, month, day [, hour [, minute [, second]]]] [, option])

os.lsd (x)

The function computes the Lotus 1-2-3 Serial Date, which is also used in Excel (known there as `Excel Serial Date`). It represents the number of days that have elapsed since 31st December 1899, 00:00h, where midnight January 01, 1900 is day 1.

The function always returns a Standard Time value even if Daylight Saving Time is active for the given date. By passing the `option` **false**, the function takes into account Daylight Saving Time, however, returning the same time as **os.esd** does.

In the first form, if no argument is given, the current Lotus Serial Date is computed. Otherwise, at least `year`, `month`, and `day` - all numbers - must be given. Optionally, you may add an `hour`, `minute`, or `second`, where all three default to 0.

The first return is a number, where the fractional portion represents the decimal time. Also, the second return **true** or **false** indicates whether Daylight Saving Time has been active for the current or given date (if no `option` has been passed) so that you may add 1/24 to the first result to receive a value Microsoft Excel would return when DST is active.

In case of a non-existing date, the function issues an error. The only exception is February 29, 1900, where the function returns 60 to emulate the bug in the original Lotus 1-2-3 formula.

In the second form, if the Lotus Serial Date `x` - a number - is given, the function returns the corresponding Gregorian year, month, day, the decimal fraction of the day - in the range [0, 1) -, the hour, minute, and second, all numbers. `x` may be 60, returning February 29, 1900.

To compute the Julian date from the Lotus Serial Date, add 2415018.5.

See also: **astro.jdate**, **os.esd**, **os.now**, **os.usd**, **utils.checkdate**.

os.meminfo ([unit])

os.memstate ([unit])

Returns a table with information on current memory usage. With no arguments, the return is the respective number of bytes (integers). If unit is the string `'kbytes'`, the return is in kBytes; if unit is `'mbytes'`, the return is in Mbytes; if unit is `'gbytes'`, the return is in Gigabytes, if unit is `'tbytes'`, the return is in Terabytes.

The resulting table will contain the following values, an 'x' indicates which values are returned on your system.

Key	**Description**	OS/2	Win-dows	DOS	UNIX	Mac
`'freephysical'`	free physical RAM	x	x		x	x
`'totalphysical'`	installed physical RAM	x	x	x	x	x
`'freevirtual'`	free virtual memory	x	x		x	x
`'totalvirtual'`	total virtual memory		x		x	x
`'pagesize'`	page size in bytes	x	x		x	x
`'resident'`	occupied resident pages	x				
`'maxprmem'`	maximum number of bytes available for the active process, same as `'freephysical'`	x				
`'maxshmem'`	maximum number of shareable bytes available	x				
`'active'`	memory currently in use by running applications and system processes.					x
`'freepagefile'`	amount of allocated swap space on the hard drive that is currently empty		x		x	x
`'totalpagefile'`	maximum amount of committable memory available to the entire system: physical RAM plus allocated page files		x		x	x
`'inactive'`	data from recently closed applications					x
`'speculative'`	memory used to pre-load data the operating system predicts applications will soon need					x
`'wireddown'`	memory used by the kernel and core drivers and that cannot be paged out or compressed					x

Key	Description	OS/2	Win-dows	DOS	UNIX	Mac
`'reactivated'`	number of memory pages that were moved from the inactive list back to the active list					x

On Mac, the function returns Mach virtual memory statistics. Type `man vm_stat` in a shell to get more information on the meaning of the above mentioned Mac-specific values.

The values returned in DOS should be self-explanatory. `'ems_major'` and `'ems_minor'` are memory manager version numbers.

On other architectures, the function returns **fail**.

See also: **environ.used**, **os.freemem**.

os.mkdir (str)

Creates a directory given by string `str` on the file system. Returns **true** on success, and issues an error on failure otherwise.

The function is available on OS/2, DOS, UNIX, Mac OS X, and Windows based systems only.

os.mklink (obj, linkname [, symbolic])

By default creates a symbolic link to the existing file system object `obj` with link name `linkname`, both arguments of type string. On UNIX-based systems creates a hard link if `symbolic` is set to **false**, otherwise ignores the flag.
In Windows, the function always automatically appends the file suffix '.lnk' to `linkname`.

The function returns **true** on success and **false** otherwise. If `obj` does not exist or the target already exists, the function issues an error and does nothing.

os.monitor (action)

The function switches the monitor on and off (Windows and Linux), and can also put it on stand-by if the monitor supports this feature (Windows only).

Pass the string `'off'` as the only argument to switch off the monitor; pass `'on'` to switch it on, and `'standby'` to put it into stand-by mode if supported. If no argument is given, the Monitor is switched on (which has no effect, if the screen is already active).

On success, the function returns **true**, and **false** and a string containing the error analysis otherwise. The `'on'` switch does not work on every hardware running Windows.

You might also want to call **io.getkey** thereafter in order to ignore any keypress or mouse movement.

os.mouse ([mhd])

In OS/2, DOS and Windows, the function returns various information on the attached mouse by returning a table with the following entries:

Key	Meaning
`'mousebuttons'`	number of mouse buttons; if more than one mouse is attached, the sum of all mouse buttons is computed
`'hmousewheel'`	Windows only: **true** if the mouse features a horizontal mouse wheel, and **false** if not
`'mousewheel'`	Windows only: **true** if the mouse features a vertical mouse wheel, and **false** if not
`'swapbutton'`	Windows only: **true** if the left and right mouse buttons have been swapped
`'speed'`	Windows only: an integer between 1 (slowest) and 20 (fastest)
`'threshold'`	OS/2 & Windows only: the two mouse threshold values, x and y co-ordinates, as a pair of two numbers
`'mickeys'`	OS/2 only: number of mickeys per centimeter; a mickey is the amount that a mouse has to move for it to report that it has moved
`'inmickeys'`	OS/2 only: mouse data in mickeys, not "pels"
`'rowscale'`	OS/2 only: row scaling factor
`'columnscale'`	OS/2 only: column scaling factor

In OS/2 you have to supply a mouse handle `mhd`, see **os.mouseopen**.

On all other platforms, the function returns **fail**.

os.mouseflush ([mhd])

In OS/2 and Windows, flushes the queue of the mouse denoted by `mhd` and returns **true** on success and **false** otherwise.

In Windows, with no argument given, discards all pending mouse and keyboard events currently stored in the console's input buffer. This is useful for clearing `stale` input data (such as clicks or key presses that occurred before a specific action) to ensure that the next input read by the application is fresh.

The function is not available on all the other operating systems.

See also: **os.mouseopen**.

os.mouseopen ()

In OS/2 only, returns a mouse handle - an integer - that is needed with other os.mouse* functions.

os.mouseclose (mhd)

In OS/2 closes the connection to the mouse denoted by `mhd`. See **os.mouseopen**.

os.mousestate ([mhd,] [mask [, threshold]])

In OS/2 and Windows, returns information on the position of the attached mouse and on button clicks plus some operating system-dependent data. In OS/2 you have to supply a mouse handle `mhd`, see **os.mouseopen**.

If the bit-`mask` is 0b0 (equals 0 decimal, the default), the function does not check whether the mouse is being moved at invocation. If `mask` is 0b1 = 1 decimal), movement is being checked. If `mask` is 0b10 (= 2 decimal), the function in Windows tries to transform absolute to window coordinates (default: false, may not work on every Windows version), and if `mask` is 0b11 (=3 decimal) both coordinates are converted and motion is being tracked.

In Windows, the threshold to detect mouse movement is 0.001 seconds. You can choose other values by passing a number to optional `threshold`. In OS/2, the setting is ignored.

The function is not available on all the other operating systems:

Key	Meaning
`'row'`	horizontal position of the mouse
`'column'`	vertical position of the mouse
`'button'`	table of three Booleans: the first for the left mouse button, the second for the middle button, and the third for the right button, where **true** depicts `button clicked` and **false** otherwise (experimental in OS/2)
`'motion'`	mouse is being moved (**true**) or not moved (**false**)
`'eventmask'`	OS/2 only: check EDM/2 Website on the MouGetEventMask API function for a description
`'flush'`	OS/2 only: mouse flush is in progress (**true**, **false** otherwise)
`'blockread'`	OS/2 only: block read is in progress (**true**, **false** otherwise)
`'eventqueuebusy withio'`	OS/2 only: event queue is busy with I/O (**true**, **false** otherwise)

os.move (oldname, newname [, option])

Renames or moves a file or directory named `oldname` to `newname`. The function returns **true** on success, and issues an error on failure otherwise.

If you pass the `option` **true**, then the function does not issue an error if `oldname` does not exist or if `newfile` already exists. Instead, the function just returns **fail**.

See also: **skycrane.move**.

os.netdomain (servername)

On Windows, returns the domain name and the name of the primary domain controller (PDC). If `servername` is **null** or not given, the local computer is used.

See also: **os.computername**, **os.getadapter**, **os.getip**, **os.getmac**.

os.netsend (server, user, message)

On older Windows flavours, sends a `message` (a string) to a `user` (a string) on a given `server` (a string). If `server` is **null** or not given, the local computer is used.

os.netuse (letter [, path])

On Windows, connects or disconnects a `drive` letter to a network `path`. The drive letter must be followed by a colon. For drive letters already in use, see **os.drives**.

Example:

```
> # connect drive with label 'drive_c' on computer TITANIA to drive Z:
> os.netuse('z:', '\\\\TITANIA\\drive_c');

> os.netuse('z:');  # disconnect
```

The function returns **true** on success and issues an error otherwise.

os.now ([secs])

os.now (year, month, day [, hour [, minute [, second]]])

Returns low-level information on the current or given date and time in form of a dictionary.

If no argument is given, the function returns information on the current date and time. If a non-negative number is given which represents the amount of seconds elapsed since the start of the epoch (try `os.now(0)`), information on this date and time will be determined (see **os.datetosecs** to convert a date to seconds).

In the second form, the given date `year`, `month`, `date` and optionally time `hour`, `minute`, `second`, where all the optional values default to 0, is processed. Alternatively, you may pass the date and time in a table, sequence or register.

The `'gmt'` field in the return represents the current date and time in GMT/UTC. The `'localtime'` field includes the same information for your local time zone. Both fields both have an associated table of data in the following order: year, month, day, hour, minute, second, number of weekday (where 0 means Sunday, 1 is Monday, and so forth), the number of full days since the beginning of the year (in the range 0 .. 365), whether daylight saving time is in effect at the time given (0: no, 1: yes), the strings 'AM' or 'PM', the month in English (a string), and the weekday in English (a string).

The `'tz'` field represents the difference between your local time zone and GMT in minutes with daylight saving time cancelled out, and *east* of Greenwich.

The `'td'` field represents the difference between your local time zone and GMT in minutes including daylight saving time, and *east* of Greenwich. `East of Greenwich` means: A positive integer indicates that your computer is located east of Greenwich, a negative value means that you are in a time zone to the west of Greenwich, and 0 means your computer is using GMT.

The `'jd'` field features the Julian date and time, the `'lsd'` field represents the Lotus 1-2-3 Serial Date, see **os.lsd**.

The `'seconds'` field is the number of seconds elapsed since some given start time (the `epoch`), which on most operating systems is January 01, 1970, 00:00:00.

The `'mseconds'` field represents milliseconds; it may be missing if milliseconds could not be determined on your platform.

The `'dst'` field indicates whether daylight saving time is in effect (**true** or **false**).

If the date and time could not be determined, the function returns **fail**.

See also: **astro.cdate**, **astro.hdate**, **astro.jdate**, **astro.pdate**, **utils.calendar**, **utils.checkdate**, **os.datetosecs**, **os.lsd**, **os.secstodate**, **os.time, os.tzdiff**.

`os.os2info ()`

`os.os2info (···)`

In the first form, returns all 31 OS/2 settings that can be queried via the C API function DosQuerySysInfo, in a table.

In the second form, you can pass any of the following options, to individually query the current settings, either in upper or lower case:

"QSV_MAX_PATH_LENGTH"	"QSV_TOTPHYSMEM"
"QSV_MAX_TEXT_SESSIONS"	"QSV_TOTRESMEM"
"QSV_MAX_PM_SESSIONS"	"QSV_TOTAVAILMEM"
"QSV_MAX_VDM_SESSIONS"	"QSV_MAXPRMEM"
"QSV_BOOT_DRIVE"	"QSV_MAXSHMEM"
"QSV_DYN_PRI_VARIATION"	"QSV_TIMER_INTERVAL"
"QSV_MAX_WAIT"	"QSV_MAX_COMP_LENGTH"
"QSV_MIN_SLICE"	"QSV_FOREGROUND_FS_SESSION"
"QSV_MAX_SLICE"	"QSV_FOREGROUND_PROCESS"
"QSV_PAGE_SIZE"	"QSV_NUMPROCESSORS"
"QSV_VERSION_MAJOR"	"QSV_MAXHPRMEM"
"QSV_VERSION_MINOR"	"QSV_MAXHSHMEM"
"QSV_VERSION_REVISION"	"QSV_MAXPROCESSES"
"QSV_MS_COUNT"	"QSV_VIRTUALADDRESSLIMIT"
"QSV_TIME_LOW"	"QSV_INT10ENABLED"
"QSV_TIME_HIGH"	

See: http://www.edm2.com/os2api/Dos/DosQuerySysInfo.html.

os.pause ([n [, msg]])

Waits for an amount of time or any user input and emulates the ZX Spectrum command PAUSE: if no argument is given or `n` = 0, the function waits forever until the user presses any key.

If `n` is a positive number, then the function waits for `n` seconds - `n` may be a fraction - unless the user presses any key - in the latter case, the function quits waiting immediately and exits. If an optional string `msg` is given, the function prints it on screen - you may terminate the string with a newline ('\n') to force a linefeed. The function returns the number of seconds including milliseconds the function waited.

See also: **os.wait**.

os.period ()

Returns the computer's clock timing (period) in a fraction of seconds.

See also: **os.clock**, **os.now**, **os.ticker**, **os.time**, **os.timestamp**, **os.uptime**.

os.pid ()

Returns Agena's process ID as a number. See also: **os.getloadeddlls**.

os.ping (dest [, timeout])

Tries to ping the host given by string `dest` - a domain name or numeric IP address - and returns **true** on success or **false** otherwise. It also returns the number of seconds it took the function to complete. Note that the function is not fail-safe as it depends on the ping command of the underlying operating system and its ability to issue an interpretable errorlevel or an interpretable output.

You may pass an optional `timeout` value to have control how long the function takes to return. By default, on non-OS/2 platforms, the timeout is 0.2 seconds.

As the ping command in OS/2 does not have a timeout switch, `timeout` represents the number of tries the ping command takes before exiting. It is 1 by default.

If `dest` ends in the substring `'.*'` then the function replaces the asterisk with all integers between 0 and 255 and tries to reach all these hosts. The return is a table with the numeric IP addresses of the hosts that responded successfully and the number of seconds the function took to complete. Note that the function may need around two minutes for such a scan.

As the implementation uses the underlying ping command of your platform, it scans its output for one of the following words to determine whether it *failed*:

```
> _store := debug.getstore(os.ping):
```

You may add further terms to the function's internal store by issuing, for example:

```
> for i in ['my failure term #1', 'my failure term #2'] do
>    insert i into _store
> od;
```

The function has been written in the Agena language and is included in lib/library.agn.

See also: **net.smallping**.

os.prefix (filename)

Returns `filename`, a string, without suffix if there is one. If `filename` includes a path, it is included in the return.

The function is written in the Agena language and part of the lib/library.agn file.

See also: **os.dirname**, **os.filename**, **os.suffix**, **strings.chomp**.

os.readlink (linkname)

Returns the target of the symbolic link `linkname` as a string. If the link does not exist or if an error occurred, it returns **fail** and optionally a string indicating the type of error.

In Windows, the function only recognises classical Windows shortcut files, it cannot resolve NTFS symbolic links or junctions.

The function is available in UNIX including Mac OS X, and Windows.

See also: **os.symlink**.

os.realpath (pathname [, option])

Converts the `pathname` argument of type string to an absolute pathname, with symbolic links resolved to their actual targets and no . or .. directory placeholders.

The return is a string.

In Windows, OS/2 and DOS, the function returns the path with slashes instead of backslashes by default. You can override this by passing an optional **false**.

See also: **os.filename**, **os.getfullpathname**.

os.remove (filename [, option])

Deletes the file or directory with the given name. Directories must be empty to be removed. Returns **true** on success, and issues an error on failure otherwise.

If you pass the `option` **true**, then the function does not issue an error if the file to be deleted does not exist. Instead, the function returns **fail**.

os.rmdir (dirname [, option])

Deletes a directory denoted by the string `dirname` on the file system. Returns **true** on success, and issues an error on failure otherwise.

If you pass the `option` **true**, then the function does not issue an error if the directory to be deleted does not exist. Instead, the function returns **fail**.

os.screensize ()

In OS/2 and Windows, returns the current horizontal and vertical resolution of the display as a pair of width:height. On all other platforms, the function issues **fail**.

See also: **os.shellinfo**.

os.secstodate (secs)

Takes the number of seconds `secs` elapsed since the start of an epoch, in UTC, and returns a table of integers in the order: year, month, day, hour, minute, second for your local timezone. In case of an error, **fail** will be returned. See also: **os.datetosec**.

os.setconsolemode ()

On Windows only, sets the current output mode of a console screen buffer, an integer. The function provides direct access to the Win32 C API function

SetConsoleMode(). See Agena function `io.aux.printincolour` in file `lib/library.agn` for an example.

See also: **os.codepage**, **os.getconsolemode**.

os.setenv (var, setting)

Sets the environment variable in the underlying operating system. `var` must be a string. If `setting` is a string or number, the environment variable `var` is set to `setting`. If `var` has already been assigned before, its value is overwritten.

If `setting` is **null**, then the environment variable `var` is deleted (not supported in DOS).

See also: **os.getenv**, **os.environ**.

os.setextlibpath (path [, option])

In OS/2, sets the `path` to be searched before and after system LIBPATH, when trying to locate DLLs. If `option` is 0, then the path is set to the beginning of LIBPATH; if it is non-zero, the path is set to its end.

The function returns **true** on success and **false** otherwise.

The function is not available on other operating systems.

See also: **os.getextlibpath**.

os.setlocale (locale [, category])

Sets the current locale of the programme. `locale` is a string specifying a locale, the empty string returns the locale of your operating system (and not the locale you have set in Agena); `category` is an optional string describing which category to change: `'all'`, `'collate'`, `'ctype'`, `'monetary'`, `'numeric'`, or `'time'`; the default category is `'all'`.

The function returns the name of the new locale, or **null** if the request cannot be honoured.

When called with **null** as the first argument or no argument at all, this function only returns the name of the current locale for the given category.

Hint: You might have to pass the full name of the language instead of its abbreviation to successfully set a locale - so for example 'Czech' instead of 'cz' or 'cs_CZ').

See also: **os.codepage**, **os.islocale**, **os.shellinfo**, **skycrane.getlocales**.

os.settime (secs)

os.settime (year, month, day [, hour [, minute [, second]]])

In the first form, takes the number of seconds `secs` elapsed since the start of an epoch, in your local time zone, and sets the system clock accordingly. In the second form, the given date `year`, `month`, `date` and optionally time `hour`, `minute`, `second`, where all the optional values default to 0, is used. Alternatively, you may pass the date and time in a table, sequence or register.

Agena must be run in root mode in order to change the system time. In case of an error, **fail** will be returned. The function is available only in the Windows, Solaris, OS/2, and Linux versions of Agena.

See also: **os.datetosecs**, **utils.checkdate**.

os.shellcolour ([n])

With no argument given, the function returns the current colour code used in a Windows shell, a non-negative integer usually in the range [0, 255]. If it could not determine the colour code, the function will throw an error.

With a non-negative integer `n` given, sets the given colour code. The function returns **true** on success or **false** and the error code otherwise.

The function gives access to the underlying Win32 C API functions GetConsoleScreenBufferInfo() and SetConsoleTextAttribute(), thus it is available in Windows only.

Example:

```
> old := os.shellcolour();  # save current setting

> for i from 0 to 255 do
>    os.shellcolour(i);
>    print('My text in colour')
> od;

> os.shellcolour(old);  # reset colour
```

See also: **os.getconsolemode**, **os.setconsolemode**.

os.shellgeom ()

Returns the current width and height of the active console, in this order, as two integers. If the settings could not be determined, the function returns **null** twice.

See also: **os.shellinfo**.

os.shellinfo ()

The function reads out the current attributes of the console and returns the data in the following fields:

- `dwCursorPosition`: column and row co-ordinates of the cursor in the console screen buffer (Windows only).
- `dwMaximumWindowSize`: maximum size of the console window, in character columns and rows.
- `dwSize`: the geometry or size of the console screen buffer, in character columns and rows (Windows only).
- `srWindow`: the console screen buffer co-ordinates of the upper-left and lower-right corners of the display window (Windows only).
- `wAttributes`: attributes (colours) of the characters written to a screen buffer (Windows only).
- `inCp`: input codepage (0 on error, Windows only).
- `outCp`: output codepage (0 on error , Windows only).

See also: **os.codepage**, **os.getlocal**, **os.screensize**, **os.setlocale**, **os.shellgeom**, **os.vga**.

os.strerror ([n])

Returns the text message for the given integral error code `n`, or the latest error issued by the underlying operating system if no argument is given. The result varies across platforms.

os.suffix (filename)

Returns the last suffix in `filename` (a string) and also the position (an integer) of the last suffix in `filename`.

If there is no suffix in `filename`, the function returns the empty string and 0 (zero).

The function is written in the Agena language and part of the lib/library.agn file.

See also: **os.dirname**, **os.filename**, **os.prefix**, **strings.chomp**.

os.symlink (target, linkname)

In UNIX, the function creates a symbolic link named `linkname` to the file called `target`. In Windows, the function creates a classical regular Windows shortcut file that points to a real file. It does not create NTFS junctions or NTFS symbolic links.

Both arguments must be strings. The function is not available for DOS.

See also: **os.readlink**.

os.system ()

Returns information on the platform on which Agena is running.

Under Windows, it returns a table containing the string 'Windows', the major version (e.g. 'NT 4.0', '2000', etc.) as a string, the Build (*dwBuildNumber*) as a number, the platform ID (*dwPlatformId*) as a number, the major version (*dwMajorVersion*) as a number, the minor version (*dwMinorVersion*) as a number, the product type (*wProductType*) as a number, maintainence information (*szCSDVersion*) usually depicting an installed service pack as a string, and a summary string combining all the previously mentioned data in a human-readable fashion, all in this order. For an alternative, see: **os.winver**.

In UNIX, Mac OS X, OS/2, and DOS, it returns a table of strings with the name of the operating system (e.g. 'SunOS', 'OS/2' or 'MS-DOS'), the release, the version, and the machine, in this order. Note that Mac OS X is recognised as 'Darwin'. In OS/2, the major and minor revision, along with the revision, are returned as numbers, as well.

In Linux and Windows, the function also checks whether the underlying platform runs in 32 or 64-bit mode, and returns the result with key `'bits'`.

If the function could not determine the platform properly, it returns **fail**.

See also: **environ.os**.

os.terminate (action)

The function halts, reboots, sleeps, or log-offs Windows, OS/2 or Mac OS X. In Windows, it can also lock the current user session or hibernate the system.

The function makes sure that no data loss occurs: if there is any unsaved data, the function does not start termination and just quits.

To put the system into energy-saving sleep mode, pass the string `'sleep'` as the only argument. To hibernate (save the whole system state and then shut off the PC), pass `'hibernate'` (Windows only); to shut down the computer completely, pass `'halt'`; to reboot the system, pass `'reboot'`; to lock the current user session without logging the user off, pass `'lock'` (Windows only); to log-off the open session of the current user, pass `'logoff'`.

The OS/2 version solely supports system halt (`'halt'` argument, without power-off) and reboot.

By default, the function waits for 60 seconds before initiating the termination process. You can change this time-out period to another number of seconds by setting the optional second argument to any non-negative integer.

On all other platforms, the function returns **fail** and does nothing.

os.ticker ()

Returns the number of seconds elapsed since the start of an epoch, usually the time the computer has been switched on, or an Agena session has been started, including fractional seconds. This is equivalent to **os.timestamp**()***os.period**().

See also: **os.clock**, **os.now**, **os.time**, **os.uptime**.

os.time ([obj] [, true])

When called without arguments the function returns the current time, or a value representing the date and time specified by the given table or sequence `obj`.

If `obj` is a table, it must either:

- have fields `year`, `month`, and `day`, and may have fields `hour`, `min`, `sec`, and `isdst`, see example below,
- or at least three integers representing year, month, day, and optionally also hour, minute, second.

If `obj` is a sequence, it must contain a four-digit year, the month, and the day, all integers, in this order. It may additionally include the hour, the minute, and the second, all integers, in this order. See example below.

The values returned are two numbers whose meaning depends on your system. In POSIX, Windows, and some other systems, this number represents the seconds and milliseconds since some given start time (the `epoch`). On other systems, the meaning is not specified, and the number returned by **os.time** can be used only as an argument to **os.date** and **os.difftime**.

If the return is **null**, then the given date lies before the start of the epoch (check `os.now(0)`).

The function works with dates between the start of 1900 and the end of 2099, only.

If a second number is returned, it will denote the millisecond portion of the current time in the range [0, 999].

If the last argument to the function is **true** then it will return the time as one result including milliseconds.

Examples:

```
> os.time(['year' ~ 2013, 'month' ~ 5, 'day' ~ 23,
>    'hour' ~ 1, 'min' ~ 2, 'sec' ~ 3]):
1369263723      791
```

```
> os.time(seq(2013, 5, 23, 1, 2, 3)):
1369267323      791
```

See also: **astro.cdate**, **astro.hdate**, **astro.jdate**, **astro.pdate**, **time**, **watch**, **os.clock**, **os.date**, **os.datetosecs**, **os.difftime**, **os.now**, **os.timestamp**, **os.uptime**, **utils.checkdate**.

os.timestamp ()

Returns the current value of the computer's clock that increments monotonically in tick units.

See also: **time**, **os.clock**, **os.now**, **os.period**, **os.time**, **os.uptime**.

os.tmpdir ([p])

Creates a unique temporary directory from pattern `p` which ends in six 'X' characters and returns its name. By default, the function uses the pattern 'agn_XXXXXX' (non-DOS) or 'agXXXXXX' (DOS).

In case of an error, the function returns **null** and an error message. You have to manually remove the directory if it is not needed any longer.

See also: **os.gettemppath**, **os.tmpname**.

os.tmpname ()

Returns a string with a file name that can be used for a temporary file or directory. The file must be explicitly opened before its use and explicitly removed when no longer needed. The same applies to directories: you have to manually create and remove it. Depending on the platform, the name might denote an absolute or relative path.

See also: **os.gettemppath**, **io.tmpfile**, **io.mkstemp**, **os.tmpdir**.

os.tzdiff ([secs])

os.tzdiff (year, month, day [, hour [, minute [, second]]])

Computes the difference between the system's local time zone and UTC in minutes, taking into account whether Daylight Saving Time is active, plus a Boolean indicating whether Daylight Saving Time is active.

If no argument is being passed, the function will use the current date and time. If a non-negative number `secs` is given - representing the amount of seconds elapsed since the start of the epoch -, this date and time will be used to compute the result.

In the second form, the given date `year`, `month`, `date` and optionally time `hour`, `minute`, `second`, where all the optional values default to 0, is used. You may also pass this date and time data as a table, sequence or register.

See also: **os.isdst**, **os.now**.

os.unmount (fs [, force])

The function unmounts the filesystem `fs`, which has to be passed as a string. If the option **true** is given for the second argument `force`, the function forces a disconnection even if the filesystem is in use by another process. The default is **false**. If your system cannot force a `umount`, this flag is simply ignored.

The function works only if Agena is run with superuser rights. Depending on the operating system, it may only unmount filesystems that the UNIX kernel directly supports (in Linux, look into /proc/filesystems folder), e.g. ntfs-3g filesystems using the FUSE driver may not be unmounted.

On success, **os.unmount** returns **true**, and **false** plus a string indicating the error reason, otherwise.

See also: **os.cdrom**, **os.execute**.

os.uptime ([qpc])

Returns the number of seconds a system has been running. It is available in OS/2, DOS, Windows, Solaris, and Linux.

In Windows XP and earlier, there may be an overflow if the system has been up for more than 49.7 days. You may pass any argument instead in Windows to query the performance counter, a high resolution (1 microsecond or better) time stamp, if any argument `qpc` is given. In this case the return is in microseconds, not seconds.

os.ticker also returns the time elapsed, but including fractional seconds.

See also: **time**, **watch**, **os.clock**, **os.period**, **os.time**, **os.timestamp**, **os.uptime**.

os.usd ([year, month, day [, hour [, minute [, second]]]])

os.usd (x)

The function computes the UTC Serial date, a number, for the given date or - if no argument is given - the current date and time, where the time zone is assumed to be UTC.

The UTC Serial represents the number of days that have elapsed since 31st December 1899, 00:00h, where midnight January 01, 1900 is day 1.

If no argument is given, the UTC Serial Date for the current date and time is computed. Otherwise, at least `year`, `month`, and `day` - all numbers - must be given. Optionally, you may add an `hour`, `minute`, or `second`, where all three default to 0.

In the second form, if the UTC Serial Date `x` - a number - is given, the function returns the corresponding Gregorian year, month, day, the decimal fraction of the day - in the range [0, 1) -, the hour, minute, and second, all numbers. `x` may be 60, returning February 29, 1900.

Since the date and time is considered to be UTC, the function implemented here takes no account of daylight saving time: at Winter time change, it returns the same values for (as an example) 02:00 a.m. before and after time change.

Also, there is a `gap` in the values returned at Summer time change between 02:00 a.m. and 03:00 a.m.

In case of a non-existing date or if the date is older than the start of the epoch, the function issues an error.

See also: **astro.jdate**, **os.esd**, **os.lsd**, **os.now**, **utils.checkdate**.

os.username ([groupname])

os.username ([uid])

The function receives a user name (a string) or a user id (an integer) and returns a table with the following key ~ value pairs:

Key	Value
`'username'`	user's login name
`'uid'`	user ID number
`'gid'`	user's default group ID number
`'realname'`	user's real name, etc.
`'homedir'`	user's home directory, or initial working directory (undefined means system default)
`'shell'`	user's default shell (undefined means system default)

If no argument is given, data for the current user is being determined.

The function is available in the UNIX versions of Agena, only. On all other systems, the function just returns **fail**.

See also: **os.login**, **os.groupinfo**.

os.vendor ()

Checks at runtime for the manufacturer of a specific x86-compatible CPU and returns **true** or **false** in the following fields:

- `'amd'`: Advanced Micro Devices, Inc. (AMD),
- `'centaur'`: Centaur Technology-designed CPU,
- `'cyrix'`: Cyrix Corporation,
- `'intel'`: Intel Corporation,
- `'nexgen'`: NexGen, Inc.,
- `'nsc'`: National Semiconductor,
- `'rise'`: Rise Technology,
- `'sis'`: Silicon Integrated Systems (SiS),
- `'tmx86'`: "Genuine x86 M6 Tine", signature belongs to NexGen,
- `'transmeta'`: Transmeta Corporation,
- `'umc'`: United Microelectronics Corporation (UMC),
- `'via'`: VIA Technologies,
- `'vortex'`: Vortex86 CPU, originally by Rise Technology, now DM&P Electronics

With a non-x86 compatible CPU the values might all be **fail**.

See also: **os.isx86**.

os.vga ()

In OS/2, DOS, Windows, macOS and Linux, the function returns a table with the following information on the display:

Key	Meaning
`'resolution'`	a pair with the horizontal and vertical number of pixels (OS/2, Windows, Linux, Mac, Solaris)
`'dimension'`	a pair with the number of rows and columns of the shell
`'colours'`	number of colours (OS/2 only)
`'depth'`	an integer indicating the colour depth in bits
`'monitors'`	the number of monitors attached to the system (Windows, Linux, Mac)
`'vrefresh'`	the vertical refresh rate in Hertz (Windows, Linux, Mac)
`'mode'`	screen mode, an integer (unknown meaning, DOS only)
`'primarygpu'`	name of the primary GPU (Windows, Linux, Mac, Solaris)

See also: **os.monitor**, **os.screensize**, **os.shellinfo**.

os.wait (x)

Waits for `x` seconds and returns **null**. `x` may be an integer or a float. This function does not strain the CPU, but execution cannot be interrupted. The function is available on OS/2, DOS, UNIX, Mac OS X, and Windows based systems only.

If `x` is negative, the function does not wait and returns **fail** immediately.

On other architectures, the function returns **fail**.

See also: **os.pause**.

os.whereis (file, dir [, options])

Searches case-insensitively for the given `file`, link or directory in directory `dir` and returns a table of all the hits.

If `dir` is `'.'` or `'*'`, the current working directory will be searched. If `option` is `'r'`, the subfolders in `dir` are also scanned, returning the files found with their relative paths. If `option` is `'s'`, the search is case-sensitive. You can mix options.

`file` may include the wildcards `?` and `*`, where `?` represents exactly one unknown character, and `*` represents zero or more unknown characters. In Windows, the functions does not check system directories as it cannot traverse them, see **os.issysdir**.

Example:

```
> os.whereis('agena*', '..', 'r'):
```

See also: **os.chdir**, **os.countcore**, **os.list**, **strings.glob**.

os.winver ([argument])

This function is an alternative to **os.system** and returns the internal Windows release number (a float). If any `argument` is given, it also returns the service pack major (an integer) and minor version (an integer), whether the operating system is a workstation (**true**) or server (**false**) and the build number (an integer), in this order.

On all other platforms other than Windows, the function returns **undefined**, which, if used in a relation, always evaluates to **false**.

Internal Version Number	Official Release
5.0	Windows 2000
5.1	Windows XP
5.2	Windows XP 64-Bit Edition, Windows Server 2003, Server 2003 R2
6.0	Windows Vista, Windows Server 2008
6.1	Windows Server 2008 R2, Windows 7
6.2	Windows Server 2012, Windows 8
6.3	Windows Server 2012 R2, Windows 8.1
10.0	Windows Server 2016 & 2019, Windows 10
11.0	Windows 11

os.xgetbv ([arg])

The function runs the XGETBV (Get Extended Control Register) instruction and returns an unsigned 32-bit integer that can be used to check whether certain CPU features have been enabled by the operating system, like the modern instruction sets AVX, AVX2, and AVX-512.

The function returns **null** if the CPU does not support extended states or the operating system has not explicitly enabled the use of XGETBV. It also returns **null** when run on non-Intel/AMD CPUs or Windows Vista and earlier.

`arg` should be 0 (zero), the default, otherwise the function just returns **null**.

Specifically, the function reads the contents of the extended control register XCR0.

Following is a complete list of the currently defined bits in XCR0, as detailed in Intel's architecture manuals:

- Bit 0 (1): x87 FPU State. This is always required to be set if the OS supports the XSAVE feature set.
- Bit 1 (2): SSE State (XMM registers).
- Bit 2 (4): AVX State (YMM registers).
- Bit 3 (8): MPX (BNDREGS) State. This bit is for the Bound Registers used by the Memory Protection Extensions (MPX).
- Bit 4 (16): MPX (BNDCSR) State. This bit is for the BNDCFGS and BNDSTATUS registers, also part of MPX.
- Bit 5 (32): AVX-512 Opmask State (K registers).
- Bit 6 (64): AVX-512 ZMM_Hi256 State (the upper 256 bits of ZMM0-ZMM15).
- Bit 7 (128): AVX-512 ZMM_Hi16 State (ZMM16-ZMM31).
- Bit 8 (256): PT State (Processor Trace).
- Bit 9 (512): PKRU State (Protection Keys for User-Mode pages).

See also: **os.cpuid**, **os.cpuinfo**.

14.2 environ - Access to the Agena Environment

This package comprises functions to access the Agena environment, explore the internals of data, read settings, and set defaults.

environ.anames ([option])

Returns all global names that are assigned values in the environment. If called without arguments, all global names are returned. If `option` is given and `option` is a string denoting a basic or user-defined type (e.g. '`boolean`', '`table`', etc.), then all variables of that type are returned.

The function is written in Agena and included in the lib/library.agn file.

environ.arithstate ()

The function returns information encoded in a bit field on the kind of numerical exception encountered by the **inc**, **dec**, **mul**, **div**, **intdiv** and **mod** operators:

add, **sub**:
0b0000: no exception
0b0010: very large value to be added to (subtracted from) value close to zero
0b0100: very large values to be added (or subtracted)
0b1000: both operands are close to zero:

mul:
0b0000: no exception
0b0010: very large value to be multiplied by value close to zero
0b0100: very large values to be multiplied
0b1000: both operands are close to zero

div, **intdiv**, **mod**:
0b0000: no exception
0b0001: denominator is zero
0b0010: very large value to be divided by value close to zero
0b1000: indicates both operands are close to zero

See also: **environ.kernel/closetozero** setting.

environ.arity (f)

The function returns the number of parameters of a function `f` and additionally a Boolean indicating whether its parameter list includes a `?` (varargs) token or not, plus the number of upvalues used in functions created by factories. The first return does not count a varargs token.

See also: **debug.getinfo**, **debug.getupvalues**.

environ.attrib (obj)

The function returns various internal status information on structures and procedures.

With the table `obj`, returns a new table with

- the current maximum number of key~value pairs allocable to the array and hash parts of `obj`; in the resulting table, these values are indexed with keys `'array_allocated'` and `'hash_allocated'`, respectively,
- the number of key~value pairs actually assigned to the respective array and hash sections of `obj`; in the resulting table, these values are indexed with keys `'array_assigned'` and `'hash_assigned'`,
- an indicator `'array_hasholes'` stating whether the array part contains at least one hole,
- an indicator `'bytes'` stating the estimated number of bytes reserved for the structure,
- an indicator `'metatable'` denoting whether a metatable has been attached to the structure,
- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- if present, a weak table is indexed by the `'weak'` key, otherwise **fail**,
- the `'length'` entry contains the estimated number of elements in a table (see **tables.getsize** & **tables.getsizes**),
- the `'lowest'` and `'keys'` represent the lowest and highest index positions in the array part; they are set to zero if there is no array part in the table;
- the `'readonly'` entry indicates whether the table is write-protected;
- the `'dummynode'` entry indicates whether a table has no allocated hash part.
- See also: **tables.borders**, **tables.indices**, **tables.maxn**.

With the set `obj`, returns a new table with

- the current maximum number of items allocable to the set; in the resulting table, this value is indexed with the key `'hash_allocated'`.
- the number of items actually assigned to `obj`; in the resulting table, this value is indexed with the key `'hash_assigned'`,
- an indicator `'bytes'` stating the estimated number of bytes reserved for the structure,
- an indicator `'metatable'` depicting whether a metatable has been attached to the structure,
- the `'readonly'` entry indicates whether the set is write-protected;
- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- if present, a weak table is indexed by the `'weak'` key, otherwise **fail**.

With the sequence `obj`, returns a new table with

- the maximum number of items assignable; in the resulting table, this value is indexed with the key `'maxsize'`. If the number of entries is not restricted, `'maxsize'` is **infinity**.

- the current number of items actually assigned to `obj`; in the resulting table, this value is indexed with the key `'size'`,
- an indicator `'bytes'` stating the estimated number of bytes reserved for the structure,
- an indicator `'metatable'` depicting whether a metatable has been attached to the structure,
- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- the `'readonly'` entry indicates whether the sequence is write-protected;
- if present, a weak table is indexed by the `'weak'` key, otherwise **fail**.

With the register `obj`, returns a new table with

- the total number of items assigned; in the resulting table, this value is indexed with the key `'size'`.
- the current top indexed by the key `'top'`,
- an indicator `'bytes'` stating the estimated number of bytes reserved for the structure,
- an indicator `'metatable'` indicating whether a metatable has been attached to the structure,
- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- the `'readonly'` entry indicates whether the register is write-protected;
- if present, a weak table is indexed by the `'weak'` key, otherwise **fail**.

With the pair `obj`, returns a new table with

- an indicator `'bytes'` stating the estimated number of bytes reserved,
- an indicator `'metatable'` betoking whether a metatable has been attached to the structure,
- the `'readonly'` entry indicates whether the pair is write-protected;
- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- if present, a weak table is indexed by the `'weak'` key, otherwise **fail**.

With userdata `obj` returns a new table with

- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- the (estimated) size in bytes with the `'bytes'` entry,
- the `'readonly'` indicating whether the pair is write-protected;
- the `'nuvalue'` entry with unknown meaning.

With function `obj` returns a new table with

- the information whether the function is a C or an Agena function. In the resulting table, this value is indexed with the key `'C'`;
- the information whether a function contains a remember table, indicated by the key 'rtableWritemode', where the entry **true** indicates that it is an rtable (which is updated by the **return** statement), where **false** indicates that it is an rotable (which cannot be updated by the **return** statement), and where **fail** indicates that the function has no remember table at all,

- the information whether an internal storage table is present, in the `'storage'` field, see Chapter 6.2.5,
- an indicator `'bytes'` stating the estimated number of bytes reserved,
- if present, a user-defined type is indexed by the `'utype'` key, otherwise **fail**,
- the number of parameters excluding varargs (?) in the `'arity'` field (with OOP methods, the result includes the **self** variable),
- a Boolean indicating whether the varargs tokern (?) is part of the parameter list, in the `'varargs'` field,
- the number of upvalues in the `'nupvals'` field,
- if present, a weak table is indexed by the `'weak'` key, otherwise **fail**.

See also: **tables.getsizes**, **tables.isarray**, **tables.ishash**.

environ.callable (obj)

Returns `obj` if `obj` is a function. If `obj` is a structure and has a '__call' metamethod, returns this metamethod (see Chapter 6.19). Otherwise returns nothing which is equivalent to **null** if tested. The Agena counterpart is:

```
> environ.callable := proc(x) is
>    if x -: procedure then
>       x := (getmetatable(x) or []).__call
>    end;
>    if x :: procedure then
>       return x
>    end
> end;
```

The function is useful to first check whether a value f is callable like a function before actually running it:

```
> r := environ.callable(f) and f(...);
```

environ.decpoint ()

Returns the decimal point separator used in the current locale. It is an alternative to the expression **os.getlocale.decimal_point**, but is faster.

environ.frozen (obj)

Checks whether a structure or userdata `obj` has been set to read-only mode by the **freeze** function and returns **true** or **false**.

See also: **environ.attrib**, **unfreeze**.

environ.gc ([opt [, arg]])

This function is a generic interface to the garbage collector. It performs different functions according to its first argument, `opt`:

- **'stop':** stops the garbage collector.
- **'restart':** restarts the garbage collector.
- **'collect':** performs a full garbage-collection cycle (if no option is given, this is the default action).
- **'count':** returns the total memory in use by Agena (in Kbytes).
- **'step':** performs a garbage-collection step. The step 'size' is controlled by `arg` (larger values mean more steps) in a non-specified way. If you want to control the step size you must experimentally tune the value of `arg`. Returns **true** if the step finished a collection cycle.
- **'setpause':** sets `arg`/100 as the new value for the pause of the collector.
- **'setstepmul':** sets `arg`/100 as the new value for the step multiplier of the collector.
- **'status':** determines whether the garbage collector is running or has been stopped, and returns **true** - i.e. collection has been activated - or **false**.

See also: **environ.frozen**.

`environ.getfenv (obj)`

If `obj` is a function: Returns the current environment in use. `obj` can be an Agena function or a number that specifies the function at that stack level: Level 1 is the function calling **environ.getfenv**. If the given function is not an Agena function, or if `obj` is 0, **environ.getfenv** returns the global environment. The default for `obj` is 1.

If `obj` is a userdata object: returns its environment table. Note that you should be careful when tampering its contents, so the function might be excluded in sandboxes.

See also: **debug.getfenv**, **debug.setfenv**, **environ.setfenv**.

`environ.getopt (args, format)`

The function parses command-line options passed from the underlying operating system to an Agena script.

Each option (switch) may consist of exactly one letter, preceded by a dash or slash, multi-letter switches are not supported and will be incorrectly processed.

Examples:

Valid: agena script.agn -h
Valid: agena script.agn /h
Valid: agena script.agn -apx (expanded to -a -p -x)
Valid: agena script.agn /apx (expanded to /a /p /x)
Valid: agena script.agn -val 3.141592654
Valid: agena script.agn -val=3.141592654
Invalid: agena script.agn -help (would be split into the switches -h, -e, -l and -p.

The function takes the **args** system table and a `format` string denoting the switches to detect and - if found - returns the switch name without a preceding dash or slash, an optional value if given, and the index of the next **args** entry to be processed in a subsequent call. If **args** is **null**, the function simply returns.

Depending on their position in the call from the operating system, unknown options might be ignored.

Example:

```
> for switch, optarg, nextidx in environ.getopt(args, 'ab:c::d') do
>    print(switch, optarg, nextidx)
> end
```

In this example, the format string `'ab:c:p'` has the following meaning, and you can use combinations in any order:

- `'a'` - check for just the /a or -a switch.
- `'b:'` - check for the /b or -b switch succeeded by a mandatory value; the switch and the value may be separated by a blank or an equals sign ('=');
- `'c::'` - check for the /c or -c switch optionally succeeded by a value, both separated by a blank or an equals sign ('=');
- `'d'` - check for just the /d or -d switch.

For an example script, check file `getopt.agn` in the `share/scripting` folder of your Agena installation.

The function is a port to a modified version of the C library function getopt.

See also: **os.getcommandline**.

`environ.isafunc (f)`

Checks whether the function `f` has been written in Agena and returns **true** or **false**.

See also: **environ.iscfunc**, **environ.kernel**.

`environ.iscfunc (f)`

Checks whether the function `f` has been written in C and returns **true** or **false**.

See also: **environ.isafunc**, **environ.kernel**.

`environ.isequal (obj1, obj2)`

Compares two objects `obj1`, `obj2` for equality and returns **true** or **false**. Note that the function considers two structures (tables, sequences, registers and pairs) a and b of the same type to be different if they do not reference one another. Thus, for

example, with a := [1] and b := [1], the function returns **false**, whereas a and b with a := [1] and b := a are equal.

See also: =, == operators.

environ.isselfref (obj)

Checks whether a structure `obj` (table, set, sequence, or pair) references to itself. It returns **true** if it is self-referencing, and **false** otherwise.

The function is written in Agena and included in the lib/library.agn file.

See also: **environ.pointer**.

environ.kernel ([setting])

environ.kernel (setting:value)

Queries or defines kernel settings that cannot be changed or deleted automatically by the **restart** statement.

In the first form, by passing the given `setting` as a string, the current configuration will be returned. If no argument is given, then all current settings are returned in a table.

In the second form, by passing a pair of the form `setting:value`, where `setting` is a string and `value` the respective setting given in the table below, the kernel is set to the given configuration.

The return is the new configuration.

Settings are:

Setting	Value	Description
`'alignable'`	**true** or **false**	Indicates whether your system aligns data along the 4- or 8-byte word boundary. The check is done at runtime, the setting is not compiled into the interpreter; read-only, in subtable 'types'.
`'bitsint'`	a number	Number of bits in a C int, should mostly be 32, read-only, in subtable 'types'.
`'blocksize'`	a number	String 'long' word-boundary in bytes, read-only, in subtable 'types'.
`'buffersize'`	a number	The default buffer size for file operations for the **os.fcopy**, **net.receive**, and **binio.readlines** functions. Must be set to [256 .. 1024^2] It is equal to the C constant BUFSIZ in stdio.h. Grep LUAL_BUFFERSIZE in the C sources.

Setting	Value	Description
'builtoncpu'	a string	Denotes the CPU architecture Agena has been built on.
'clockspersec'	an integer	Ticks per second measured by the **os.clock** function, read-only.
'closetozero'	a number	Threshold so that the binary **inc**, **dec**, **mul**, **div**, **intdiv** and **mod** operators can recognise operators close to zero. Default is **DoubleEps**.
'constants'	**true** or **false**	Turn constants feature on or off. Default: on (**true**). See also -C command-line switch in Appendix A5.5.
'constanttoobig'	**true** or **false**	Issue a syntax error if a numeric constant (binary, octal, decimal, hexadecimal) is out-of-range. Default: **false**. See also -B command-line switch in Appendix A5.5.
'debug'	**true** or **false**	Prints debugging information if the initialisation of a C dynamic library failed. Also prints information while the **import** statement is being executed.
'digits'	an integer in [1, 17]	Sets the number of digits used in the output of numbers. Note that this setting does not affect the precision of arithmetic operations. The default is 14. See also -D command-line switch in Appendix A5.5 and the **Digits** environment variable described in Appendix A4.
'doubleeps'	a number	Gets and sets **DoubleEps** variable; default is 2.2204460492503131e-16.
'duplicates'	**true** or **false**	Turns duplicate declarations (shadowing) warnings on (**true**) and off. Default: **true**.
'emptyline'	**true** or **false**	If set **true** (the default), two input regions are always separated by an empty line. If set **false**, no empty line is inserted.
'enclose'	**true** or **false**	print strings in single quotes, default: **false**.
'encloseback'	**true** or **false**	print strings in backquotes, default: **false**.
'enclosedouble'	**true** or **false**	print strings in double quotes, default: **false**.
'eps'	a number	Stores the accuracy threshold epsilon used by the ~= operator and the **approx** function.
'errmlinebreak'	a positive integer	Stores the maximum number of characters to be displayed per line in syntax error messages. Default is 70.
'foradjust'	**true** or **false**	Controls auto-correction of iteration values x close to zero in numeric **for** loops with fractional step sizes. Default is **true**, i.e. x will be set to zero if x < hEps.

Setting	Value	Description
'glibc'	a number	When Agena has been compiled with GCC, denotes the major GLIBC version linked during compilation. Otherwise is **null**.
'glibcminor'	a number	Dito, but with respect to the minor GLIBC version.
'gui'	**true** or **false**	If set **true**, tells the interpreter that it has been invoked by AgenaEdit. Default is **false**.
'hEps'	a number	Gets and sets **hEps** variable; default is 1.4901161193847656e-12. Also controls auto-correction features in numerical for loops, see Chapter 5.2.2.
'history'	**true** or **false**	Switches command-line logging on or off. Default is off, that is **false**. See Chapter 14.10.
'histfile'	a string	Gets or sets the full path to the command-line logging file.
'iso8601'	**true** or **false**	If set to **true**, **os.date** determines weekdays according to the ISO 8601 norm. Default is **true**.
'is32bit'	**true** or **false**	If **true**, then Agena has been compiled in 32-bit mode, read-only.
'is64bit'	**true** or **false**	If **true**, then Agena has been compiled in 64-bit mode, read-only.
'is32bitaligned'	**true** or **false**	If **true**, then a memory address in your computer is aligned to 32-bit, read-only.
'isARM'	**true** or **false**	If set to **true**, then Agena is running on an ARM CPU.
'isDOS'	**true** or **false**	If set to **true**, then Agena is running in DOS.
'isIntel'	**true** or **false**	If set to **true**, then Agena is running on an Intel-compatible CPU.
'isLinux'	**true** or **false**	If set to **true**, then Agena is running in Linux.
'isMac'	**true** or **false**	If set to **true**, then Agena is running in Mac OS X.
'isOS/2'	**true** or **false**	If set to **true**, then Agena is running in OS/2 and successors.
'isSolaris'	**true** or **false**	If set to **true**, then Agena is running in Solaris.
'isWindows'	**true** or **false**	If set to **true**, then Agena is running in Windows.
'kahanbabuska'	**true** or **false**	If set to **true**, Kahan-Babuška round-off error prevention in numeric **for** loops instead of the original Kahan algorithm. Default is **false**.

Setting	Value	Description
`'kahanozawa'`	**true** or **false**	If set to **true**, Kahan-Ozawa round-off error prevention in numeric **for** loops instead of the original Kahan algorithm. Default is **false**.
`'lastcontint'`	a number	Largest accurately representable integer (usually 2^{53}), read-only, in subtable 'types'.
`'libnamereset'`	**true** or **false**	If set **true**, the **restart** statement resets **libname** and **mainlibname** to their original values. Default is **false**.
`'loaded'`	a set	Returns the names of all basic libraries initialised at start-up of the interpreter.
`'longmantdigs'`	a number	Number of digits in the floating point mantissa for C data long doubles, read-only, in subtable 'types'.
`'longmaxexp'`	a number	largest possible exponent value in long doubles, read-only, in subtable 'types'.
`'longtable'`	**true** or **false**	If set **true**, then each key~value pair in a table will be printed at a separate line, otherwise a table will be printed like sets or sequences. Default is **false**.
`'maxinteger'`	a number	Maximum representable 4-byte signed integer used internally, read-only, in subtable 'types'.
`'maxlong'`	a number	Maximum value of a signed 32-bit integer, usually 2,147,483,647, read-only, in subtable 'types'.
`'maxulong'`	a number	Maximum value of an unsigned 32-bit integer, usually 4,294,967,295, read-only, in subtable 'types'.
`'minlong'`	a number	Minimum value of a signed 32-bit integer, usually -2,147,483,648, read-only, in subtable 'types'.
`'minstack'`	a number	Minimum internal stack size, i.e. the number of preallocated slots slots. Cannot be changed.
`'nbits'`	a number	Number of bits in a 4-word integer, read-only, in subtable 'types', read-only, in subtable 'types'.
`'nbits64'`	a number	Number of bits in an 8-word integer, read-only, in subtable 'types', read-only, in subtable 'types'.
`'nbytesulong'`	a number	Number of bytes in an unsigned C long, read-only, in subtable 'types'.
`'nulliszero'`	**true** or **false**	If set to true, then in basic arithmetic only (+, -, *, /, ^, **) **null** will be interpreted as zero. Default is **false** to facilitate debugging of code.

Setting	Value	Description
`'pathmax'`	a string	Returns the maximum path length accepted by the operating system, an integer.
`'pathsep'`	a string	The token that separates paths in libname; by default is ';', cannot be changed. Grep LUA_PATHSEP in the C sources.
`'promptnewline'`	**true** or **false**	If set to **true**, prints an empty line between the input and output regions. Default is **false**.
`'readlibbed'`	a set	Returns the names of all libraries manually imported in a session by **import**, **readlib**, **initialise**.
`'regsize'`	a number	Sets the default size of registers, the number must be a non-negative integer.
`'rounding'`	a string	Returns or sets the current rounding method (beware, this may cause unwanted results; see also **math.rint**): `'downward'` rounds down to the next lower integer, `'upward'` rounds up to the next greater integer, `'nearest'` rounds up or down toward whichever integer is nearest (default on most systems), `'zero'` rounds toward zero.
`'rthits'`	an integer	Number of hits in remember tables, read-only.
`'rtmisses'`	an integer	Number of misses in remember tables, read-only.
`'seqautoshrink'`	**true** or **false**	If set to **false**, Agena does not free memory when you remove sequence values; default is **true**.
`'signedbits'`	**true** or **false**	If set to **true**, the bitwise operators **&&**, ~~, ||, ^^, <<, >>, <<< and >>> internally use signed integers (the default), otherwise they use unsigned integers.
`'skipagenapath'`	**true** or **false**	If set to **true**, the interpreter has been started with the -a option, ignoring the setting of the AGENAPATH environment variable. Default is **false**.
`'skipinis'`	**true** or **false**	If set to true, does not read the Agena initialisation files agena.ini / .aganea.init at **restart**. Default is **false**.
`'skipmainlib'`	**true** or **false**	If set to **true**, does not read the main library file **lib/library** at **restart**. Default is **false**.
`'smallestnormal'`	a number	Smallest normal number (usually 2^{-1022}).
`'sse2'`	**true** or **false**	If **true**, then Agena uses the SSE2 instead of the x87 instruction set, read only.

Setting	Value	Description
`'warnings'`	an integer or Boolean	When just reading this setting, returns a number: 0 - warning system is off; 1 - ready to start a new message; 2 - previous message is to be continued. Setting the warning mode requires a Boolean: **true** - switch on warnings, **false** - switch them off.
`'zeroedcomplex'`	**true** or **false**	When set to **true**, real and imaginary parts of complex values close to zero are rounded to zero on output. (Note that internally, complex values are not rounded.) Default is **false**.

Examples:

```
> environ.kernel('signedbits'):
false

> environ.kernel(signedbits = true):
true
```

See also: **history.off**, **history.on**, **environ.arity**, **environ.isfunc**, **environ.system**.

`environ.onexit ()`

If assigned a function to the name **environ.onexit**, this function is automatically called when quitting or restarting Agena. For more information, see **bye**.

`environ.pointer (obj [, option])`

Converts `obj` to a generic C pointer (void*) and returns the result as a string. `obj` may be userdata, a string, a table, set, sequence, register, pair, thread or function; otherwise, **environ.pointer** returns **fail**. Different objects will give different pointers, which gives a unique string identifier.

With any `option` given, **environ.pointer** retrieves the object stored at the given memory address denoted by the string returned by the same function called with no option. Non-collectable objects, that is numbers, complex numbers, booleans and null cannot be retrieved. To make sure that the objects are the same you may stop the garbage collector before and restart it later on. Example:

```
> environ.gc('stop');

> a := [10, 20, 30];

> p := environ.pointer(a):
023052E0

> environ.pointer(p, true):
[10, 20, 30]
```

```
> environ.gc('restart');
```

The search for a given pointer is linear. Note that you cannot just unsafely peak into memory as the function checks whether a collectable object is associated with the given memory address. If a collectable object cannot be found, the function will return **null**.

See also: **environ.isselfref**.

environ.ref (tbl, obj [, option])

Creates a unique integer reference for any argument `obj` and inserts `obj` into table `tbl` at position ref (i.e. `tbl`[ref] :=`obj`).

The function returns ref. Do not manually put any integer keys into `tbl` or delete them, always use **environ.ref** and **environ.unref** to modify `tbl`.

By default, `obj` is always inserted into `tbl`, even if it is already stored there.

If the optional third argument is `'reference'` or `'full'`, then a check is performed to ensure that `obj` has not already been included in `tbl`. If `obj` is already in `tbl`, it is not inserted again and the integer index of `obj` in `tbl` is simply returned.

If `option` is `'reference'`, the function uses **environ.isequal** for the check, whereas with the option `'full'`, the standard = equality operator is being used.

See also: **environ.unref**, **sema.open**, **utils.uuid**.

environ.setfenv (obj, table)

Sets the environment to be used by the given object.

If `obj` represents a function: Sets the environment to be used by `obj`. `obj` can be an Agena function or a number that specifies the function at that stack level: Level 1 is the function calling **environ.setfenv**. **environ.setfenv** returns the given function.

As a special case, when `f` is 0, **environ.setfenv** changes the environment of the running thread. In this case, the function returns no values.

If `obj` is a userdata, the function attaches table `table` as its environment.

To protect the environment of a userdata object from tampering, **environ.setfenv** does not allow to completely delete its environment table, so calls like 'environ.setfenv(ud, null)' will be rejected.

See also: **debug.getfenv**, **debug.setfenv**, **environ.getfenv**.

environ.system ()

Returns a table with the following system information:

- The size of various C types (char, int, long, long long, float, double, long double, complex double, uint16_t, int32_t, int64_t),
- the smallest and largest numeric values for C doubles (fields 'mindouble', 'maxdouble'),
- the largest numeric values for C unsigned short ints (16-bit, 'maxushort'),
- the smallest and largest numeric values for C long ints (32-bit, fields 'minlong', 'maxlong'),
- the smallest and largest numeric values for C long long ints (fields 'minlonglong', 'maxlonglong'),
- the largest numeric values for C unsigned long ints (field 'maxulong'),
- the number of bits in an C unsigned char (field 'bitschar')
- the number of bits in a 32-bit integer (field 'bitsint'),
- the endianness of your platform (field 'endianess'),
- the hardware (field 'hardware') and
- the operating system (field 'OS') for which the Agena executable has been compiled.
- The 'floatradix' key represents the GNU C FLT_RADIX environment variable, which usually is 2.
- The 'doubleradix' key represents the base of C doubles which usually is 2.

See also: **environ.kernel**.

environ.unref (tbl, ref)

With `tbl` a table and `ref` an integer, deletes value `tbl[ref]` and returns it. `ref` should have been created by calling **environ.ref**.

environ.used ([opt])

By default, returns the total memory in use by Agena in Kbytes. If `opt` is the string `'bytes'`, `'kbytes'`, `'mbytes'`, `'gbytes'` or `'tbytes'`, the number will be returned in the given unit.

In Windows, the function also returns the memory committed to Agena by the operating system (second result) and the interpreter's real-time physical memory footprint (third result), both using the same unit as with the first result.

See also: **os.freemem**, **os.memstate**.

environ.userinfo (f, level [, ···])

Writes information to the user of a procedure `f` depending on the given `level`, an integer. The information to be printed is passed as the third, etc. arguments and may be either numbers or strings.

At first the procedure should be registered in the **environ.infolevel** table along with a `level` (an integer) indicating the infolevel setting at which information will be printed, e.g. `environ.infolevel[myfunc] := 1`.

If you do not enter an entry for the function to the **environ.infolevel** table, then nothing is printed.

```
> f := proc(x) is
>    environ.userinfo(f, 1, 'primary info to the user:     ', x, '\n');
>    environ.userinfo(f, 2, 'additional info to the user:  ', x, '\n')
> end;
```

If the `level` argument to **userinfo** is equal or less than the **environ.infolevel** table setting, then the information is printed, otherwise nothing is printed.

```
> environ.infolevel[f] := 2;

> f('hello !');
primary info to the user:     hello !
additional info to the user:  hello !
```

Now the infolevel is decreased such that less information will be output.

```
> environ.infolevel[f] := 1;

> f('hello !');
primary info to the user:     hello !
```

See also: **environ.warn**.

environ.warn (str [, ···])

Emits a warning with a message composed by the concatenation of all its arguments (which should be strings or numbers).

By convention, a one-piece message starting with '@' is intended to be a control message, which is a message to the warning system itself. In particular, the standard warning function in Lua recognizes the control messages "`@off`", to stop the emission of warnings, and "`@on`", to (re)start the emission; it ignores unknown control messages.

If called without arguments, returns a Boolean indicating whether the warning system is on or off, and the current warning state as an integer:

- 0 - warning system is off;
- 1 - ready to start a new message;
- 2 - previous message is to be continued.

See also: **environ.infolevel**, **environ.userinfo**.

14.3 package - Modules

The package library provides a basic facility to inspect which packages have been loaded in a session.

package.checkclib (pkg)

Checks whether the package denoted by the string `pkg` and stored to a C dynamic library has already been initialised. If not, it returns a warning printed on screen and creates an empty package table. Otherwise it does nothing.

package.getcfuncs ([libname])

Returns a sorted table of all the C functions in standard library `libname`, given as a string. If you want the names of the base library, just pass the empty string or no argument at all. The function scans the '_origG' table stored to the registry which contains all the assigned names in the environment after start-up.

See also: **debug.getregistry**, **package.packages**.

package.loadclib (packagename, path)

Loads the C library `packagename` (with extension `.so` in UNIX and Mac, or `.dll` in Windows) residing in the folder denoted by `path`. `path` must be the name of the folder where the C library is stored, and not the absolute path name to the library file. The function returns **true** on success and **false** otherwise. On successful initialisation, the name of the package is entered into the **package.readlibbed** set.

See also: **readlib**, **with**.

package.loaded

A table containing all the names of the packages that have been initialised, either at start-up or later on in a session.

package.packages ()

Returns a set with all the names of Agena's standard libraries that are initialised at start-up. The set does not include packages that have been loaded manually later on in a session, for example by the **import** statement or the **readlib** or with functions.

See also: **package.getcfuncs**.

package.readlibbed

A set with all the names of the packages that have been initialised with the **readlib** and **with** functions, and the **import** statement. This set may be deprecated in future versions of Agena.

14.4 rtable - Remember Tables

This package comprises functions to administer remember tables.

Note that remember tables can hold up to 20,000 entries.

rtable.defaults (f)

rtable.defaults (f, tab)

rtable.defaults (f, null)

Administrates read-only remember tables of functions. As it works exactly like the **remember** function, except that it creates remember tables that cannot be updated by the **return** statement, please refer to the description of the **rtable.remember** function for further details.

rtable.remember (f)

rtable.remember (f, tab)

rtable.remember (f, null)

Administers remember tables.

In the first form, the remember table stored to procedure `f` will be returned. See **rtable.get** for more information.

In the second form, **remember** adds the arguments and returns contained in table `tab` to the remember table of function `f`. If the remember table of `f` has not been initialised before, **remember** creates it. If there are already values in the remember table, they are kept and not deleted.

If `f` has only one argument and one return, the function arguments and returns are passed as key~value pairs in table `tab`.

If `f` has more than one argument, the arguments are passed in a table. If `f` has more than one return, the returns are passed in a table, as well.

Valid calls are:

```
import rtable alias remember;

remember(f, [0 ~ 1]);           # one argument 0 & one return 1
remember(f, [[1, 2] ~ [3, 4]);  # two arguments 1, 2 & two returns 3, 4
remember(f, [1 ~ [3, 4]]);      # one argument 1 & two returns 3, 4
remember(f, [[1, 2] ~ 3]];      # two arguments 1, 2 & one return 3
```

In the third form, by explicitly passing **null** as the second argument, the remember table of `f` is destroyed and a garbage collection run to free up space occupied by the former rtable.

remember always returns **null**. It is written in Agena and included in the lib/library.agn file.

See Chapter 6.18 for examples. See also: **rtable.defaults**.

rtable.forget (f)

Empties the remember table or read-only remember table of procedure `f` entirely, giving back the space formerly occupied to the interpreter. It also enforces an immediate garbage collection. The function returns **null**.

rtable.get (f [, option])

Returns the contents of the current remember table or read-only remember table of procedure `f`. If any value for `option` is given, the internal remember table including all the hash values are returned.

```
> fib := proc(n) is
>    assume(n >= 0);
>    return fib(n - 2) + fib(n - 1)
> end;

> rtable.remember(fib, [0 ~ 0, 1 ~ 1]);

> rtable.get(fib):
[[0] ~ [0], [1] ~ [1]]
```

You cannot destroy the internal remember table by changing the table returned by **rtable.get**.

rtable.init (f)

Creates a remember table (an empty table) for procedure `f`. The procedure must have been written in Agena; reminisce that rtables for C API functions are not supported and that in these cases the function quits with an error.

If there is already a remember function for `f`, it is overwritten. **init** returns **null**.

rtable.mode (f)

Returns the string `'rtable'` if function `f` has a remember table, `'rotable'` if f has a read-only remember table (that cannot be updated by the **return** statement), and the string `'none'` otherwise.

rtable.purge (f)

Deletes the remember table or read-only remember table of procedure `f` entirely. It also enforces an immediate garbage collection. The function returns **null**.

rtable.roinit (f)

Creates a read-only remember table (an empty table) for procedure `f`, which may be either a C function or an Agena procedure.

If there is already a remember function for `f`, it is overwritten. **roinit** returns **null**.

rtable.put (f, arguments, returns)

The function adds one (and only one) function-argument-and-returns `pair` to the already existing remember table or read-only remember table of procedure `f`.

`arguments` should be a table array, `returns` should also be a table array. If the argument(s) already exist(s) in the remember table, then the corresponding result(s) are replaced with `returns`. If `arguments`, `returns` are not tables, the function will put them into tables internally, so you do not need to care for conversion.

Given a function `f := <: x -> x :>` for example, valid calls are:

```
rtable.put(f, [1], [2]);
rtable.put(f, 1, 2);
rtable.put(f, [1], 2);
rtable.put(f, [1, 2], [2]);
rtable.put(f, [1], [1, 2]) etc.
```

14.5 registry - Access to the Registry

This package provides access to the registry, see Chapter 6.31 of the Agena Primer. It should be used carefully.

You may exclude the package from sandboxes, see Chapter 6.15 of the Agena Primer, since tampering the registry might cause Agena to no longer work successfully and even cause memory leaks.

Its library functions are:

registry.anchor (key, value)

Inserts a new key ~ value pair into the registry, where the `key` is a unique string, and the `value` the corresponding data. To delete entries in the registry, pass **null** for `value`.

The function returns nothing.

See also: **registry.get**, **registry.ref**.

registry.anyid (key)

The function has become obsolete and just returns the string `key` given.

See also: **registry.get**, **utils.uuid**.

registry.get (key)

Returns the registry value indexed by `key`, which may be of any type. If the registry entry is occupied by userdata, is referring to loaded libraries or open files, the function just returns **null**. Otherwise, the entry is simply returned.

If a C library metatable contains the `__metatable` read-only metamethod, **null** will be returned, as well.

With metatables defined by C libraries, it is still possible to delete or change metamethods, so extreme care should be taken when referencing to metatables that have been returned by **registry.get**. Especially, the `__gc` metamethod must not be deleted or changed.

The function leaves the registry unchanged.

See also: **getmetatable**, **debug.getregistry**, **registry.anchor**, **registry.unref**.

registry.ref (value)

Inserts `value`, which may be of any type, into the Agena registry and returns a reference to it, an integer.

See also: **registry.get**, **registry.unref**.

registry.unref (ref)

Returns the registry value with reference `ref`, an integer, and deletes it from the registry. `ref` should be the integer reference returned by **registry.ref**.

If you call the function again with the same reference `ref`, you will retrieve an unrelated item from the registry that has almost certainly been created by the previous call to **registry.unref** to ensure uniqueness of the references.

14.6 stack - Built-In Number, Character & Value Stacks

The functions and operators in this package work with one of the seven built-in numerical, character or cache stacks, internal last-in-first-out data structures that can either store numbers in a numeric stack or characters only in a character stack - or any kind of data with a cache stack. The stacks are addressed by their stack number and not variable names.

Stacks 1 to 3 are number stacks, stacks 4 to 6 are character stacks and stack 7 is the cache stack which can store numbers, strings and booleans, or a mix of it. The default stack is stack 1, i.e. a number stack.

Never use or change numeric stack 0 which is used by Agena internally as some sort of fixed-sized register array for numeric and other operations.

You may switch between the stacks by calling switchd(1) for the first stack, switchd(2) which is the second stack, etc.

While any element in - for example - a sequence occupies 24 bytes of memory, a number in a numeric stack takes only eight bytes, and a character in a character stack only one byte. A value in a cache stack takes 24 bytes.

You can push a theoretically unlimited amount of numbers or characters onto the currently selected stack, with 128 pre-allocated slots after Agena initialisation. A cache stack - or `cache` for short -, however, can store a maximum of up to 2,048 values only.

If values, numbers or characters are added, Agena automatically enlarges the current stack if needed. However, if values are removed, Agena itself does not shrink the allocated memory. Call **stack.shrinkd** to accomplish this if required, and **stack.sized** to query the amount of used and internally allocated space. Allocated space in caches cannot be freed.

Example: to convert a decimal number into the binary system, you might use a numerical stack:

```
> tobinary := proc(x :: number) is  # for positive numbers only
>    local base := 2;     # new base
>    local r := '';
>    stack.resetd();      # always clear stack before usage, destructive !
>    while x > 0 do
>       pushd(x % base);  # push the remnant onto the stack (0 or 1)
>       x := x \ base
>    od;
>    while allotted() do  # now traverse the stack from top to bottom
>       r := r & popd()   # and also remove the remnants one after another
>    od;
>    return r             # return result as a string
> end;
```

```
> switchd(1): # select a numeric stack

> tobinary(6):
110
```

The names of functions and operators usually end in a final `d`.

The basic library functions and operators are:

allotted ([n])

If given no argument and if the stack contains one or more elements, the function returns the number of values in the stack, else it returns **null**. It can be easily used in **while** loops to traverse a stack, see example above.

If given a stack number `n`, it either returns the number of elements currently in the given stack or **null** if no values are in this stack.

See also: **stack.isempty**, **stack.sized**.

cell (idx)

The operator returns the element stored at the absolute or relative position `idx` of the current stack. While -1 refers to the top of the stack, -2, to the element just below the top, etc., 0 refers to the first element at the bottom of the stack, 1 one step `above`, etc. If the stack is empty or `idx` is out-of-bounds, **null** will be returned.

popd ([n [, anyoption]])

The function pops `n` numbers, values or characters from the top of the current stack, or if `n` is not given, the element at the current top of the stack. If the stack is empty, **null** will be returned.

By default, the **popd** returns the last value popped. This can be suppressed if any second argument is given.

All slots that are discarded by a call to the operator are nullified, i.e. set to **null**.

See also: **pushd**, **stack.sized**.

pushd (x [, ...])
pushd (s)

In the first form, and depending on the stack type, the statement pushes one or more numbers, booleans, strings or single characters `x` onto the current stack.

In the second form, the command pushes all the numbers, booleans, strings or single characters in the sequence `s`. With strings, all the characters in the string are pushed, with the last character put at the stack top, and all the preceding characters below.

The function returns nothing.

You may or may not enclose the arguments in brackets.

If the current stack is a character stack and `x` or an element in `s` is an integer in the range 0 to 9, both inclusive, the operator converts `x` to a character and inserts this character into the stack. If you insert an empty string and dump the string using - for example - **stack.dumpd** later, the resulting string is terminated at the position you inserted the empty string.

Hint: If the very last argument is the pair `'stack':`<stacknumber>, which is equal to `stack =` <stacknumber>, with <stacknumber> an integer, the values are pushed onto the given stack <stacknumber>. Using this option, however, may slow down the operation since processing an option takes quite some computation time.

See also: **stack.pushstringd**.

switchd (n)

The statement switches to stack `n` with `n` an integer. Valid values for `n` are 1 to 3 for numeric stacks, 4 to 6 for character stacks and 7 for the cache stack.

You may or may not enclose the argument in brackets.

switchd memorises the formerly active stack. To change back to it, just pass -1 for `n`.

See also: **stack.selected**, **stack.switchto**.

The **stack** library features the following auxiliary procedures:

stack.absindex (idx [, option])

With no `option` given, and with the current stack, converts a negative index `idx` into the current non-negative index and returns it. Otherwise returns `idx`. A return of 0 refers to the value at the bottom of the stack, 1 to the value above the bottom, and so on.

With any `option` given, converts a non-negative index `idx` to a negative one, with -1 referring to the element at the top of the stack, -2 to the value just below the top, etc. If `idx` is negative, returns `idx`.

If the stack is empty or `idx` is out-of-bounds, returns **null**.

`stack.attribd ()`

Returns various status information on the current stack in a table with the keys:

`'currentstack'` (number of the current stack), `'defaultsize'` (number of default slots), `'isnumstack'` (**true** if the stack stores Agena numbers, **false** if not), `'iscachestack'` (**true** if the stack is a cache, **false** if not), `'stackmax'` (maximum number of allocated slots), `'stacktop'` (current stack top, counting from 0).

See also: **stack.isempty**, **stack.sized**.

`stack.choosed ()`

Returns the ID of both the number and character stack with the least number of values in them. The first return is the ID of the smallest number stack, the second return the one of the smallest character stack.

See also: **switchd**, **stack.selected**.

`stack.dequeued ()`

Removes the value at the bottom of the current stack, returns it, and moves all elements to close the space. It is equivalent to **stack.removed**(0).

With a cache stack, the former top position is automatically nullified, i.e. set to **null**.

See also: **stack.enqueued**, **stack.removed**.

`stack.dumpd ([n] [, option])`

With a numeric stack or a cache, returns all values in the current stack in a new sequence if given no argument, or the last `n` values pushed onto the stack, and pops them all from the stack. The number of pre-allocated slots is not changed, see **stats.shrinkd**.

With a character stack, when given no argument, all characters are returned as exactly one string, first-in first-out style. If `n` is given, the last `n` characters inserted are returned in exactly one string.

If passed **true** as the very last argument, the selected numbers or characters are returned in reverse order. This saves an expensive call to **stack.reversed**.

If the stack is empty, the function just returns **null**.

All slots that are discarded by a call to the function are nullified, i.e. set to **null**.

See also: **stack.explored**.

stack.enqueued (x)

Inserts the values, number or character `x` at the bottom of the current stack, shifting up other elements to open space.

The function returns nothing. It works like **stack.insertd**(0, `x`).

See also: **stack.dequeued**, **stack.insertd**, **stack.removed**.

stack.explored ()

Returns the entire contents of the current stack without modifying it. If the current stack is numeric, the return is a sequence of numbers. If the current stack is a cache, the return is a sequence of values, with the first value the bottom of the stack. Otherwise the function will return a string.

If the stack is empty, the function just returns **null**.

The function will not modify the stack.

See also: **stack.dumpd**.

stack.insertd (idx, x)

Inserts the value, number or character `x` at the given relative stack position `idx`, shifting up other elements to open space. The function returns nothing.

If `idx` is a non-negative integer, 0 represents the bottom of the stack, 1 the position just above the bottom, etc. If `idx` is negative, -1 represents the stack top, -2 the element just below the top, etc. If no argument is given, `idx` is set to -1 by default, i.e. the top element.

See also: **stack.dequeued**, **stack.enqueued**, **stack.removed**.

stack.isempty ()

Checks whether the current stack is empty and returns **true** or **false**.

See also: **allotted**, **stack.attribd**, **stack.sized**.

stack.mapd ([idx,] f, [···] [, true])
stack.mapd (true, f, [, true])

In the first form, if `idx` is not given or if `idx` = -1, the default, maps function `f` on the top element of the current character or numeric stack. With a character stack, the

result of the call to `f` must be a character, and with a number stack the result must be a number, otherwise the function issues an error. Cache stacks are supported, as well.

If `idx` is a negative integer, the **stack.mapd** applies `f` on the |`idx`|-th value in the stack. If `idx` is zero, it refers to the bottom, 1 to the value above the bottom, etc.

If `f` is a multivariate function, its second, third, etc. argument must be passed right after `f`.

In the second form, by passing the Boolean **true** and a univariate or multivariate function `f` *of n parameters*, **stack.mapd** takes the n top stack values, passes them as arguments to `f`, with the value at the top of the stack the last argument, and returns the result. If the parameter list of `f` contains a ?-varargs token, then all stack elements are passed to `f`. Example:

We have a function of three parameters x, y, z that returns x*y + z. The call puts the top three arguments 3, 2, 4

```
> pushd 3, 2, 4
```

into the argument list, pops them from the stack, pushes the result of the function call onto the stack and returns it:

```
> stack.mapd(true, <: x, y, z -> fma(x, y, z) :>):
10
```

In both the first and the second form, the stack element(s) is (are) replaced by the result of the function call if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the stack element(s) is (are) not replaced and the result is just put onto the stack top.

With a cache stack, all slots that are discarded by a call to the function are nullified, i.e. set to **null**.

stack.pushstringd (str)

Pushes all the characters in string `str` onto a character stack. After the operation, `str`[-1] resides at the top of the stack, `str`[-2] just below, etc. The return is the size of `str`, i.e. **size** `str`.

See also: **pushd**.

stack.pushvalued (idx)

Pushes the value residing at the given relative stack position `idx` to the top of the stack. The original value also remains at position `idx`.

If `idx` is a non-negative integer, 0 represents the bottom of the stack, 1 the position just above the bottom, etc. If `idx` is negative, -1 represents the stack top, -2 the element just below the top, etc. If no argument is given, `idx` is set to -1 by default, i.e. the top element.

stack.readbytes (filehandle [, bytes] [, options])

By default, the function reads **environ.kernel('buffersize')** bytes from the file denoted by `filehandle` and writes them to the current number or character stack.

Open the file with **binio.open** before the first call and when you are finished, call **binio.close** to release the file.

The function increments the file position thereafter so that the next bytes in the file can be read with a new call to various **stack.readbytes**.

You may change the kernel buffer size value to any other values in order to read less or more bytes.

If `bytes` is given, the function writes `bytes` bytes from the file denoted by `filehandle` to the current stack.

To write the file contents to another than the current stack, pass the optional `stack` option with the preferred stack number, e.g.:

```
> stack.readbytes(fh, stack=6);
```

By default, the function ignores any newlines (ASCII 10) or carriage returns (ASCII 13) in the file. You can change this by setting the `ignore` option and passing a string of explicit characters that shall be skipped, e.g.:

```
> stack.readbytes(fh, ignore=" .\n");  # skip white space, dot & newline
```

By default the function reads in embedded zeros and treats them as every other byte. If you pass the `eof` option and set it to **true**, then the function quits if it encounters an embedded zero in the file. The file pointer is automatically reset to the position of the embedded zero. The default is **false**, i.e. the whole file is read in.

The function returns both the number of bytes read from the file and the number of bytes written to the stack. If the end of the file has been reached, **null** will be returned. In case of an error, it quits with the respective error.

Cache stacks are not supported.

See also: **binio.writebytes**.

stack.removed (idx)

Removes the value residing at the given relative stack position `idx`, returns it, and moves all elements to close the space.

If `idx` is a non-negative integer, 0 represents the bottom of the stack, 1 the position just above the bottom, etc. If `idx` is negative, -1 represents the stack top, -2 the element just below the top, etc. If no argument is given, `idx` is set to -1 by default, i.e. the top element.

With a cache stack, the former top position is automatically null ified, i.e. set to **null**.

See also: **stack.insertd**, **stack.dequeued**, **stack.enqueued**.

stack.replaced (idx, x)

Replaces the value at stack position `idx` with the value, number or one-character string `x`. The value replaced will be returned.

If `idx` is a non-negative integer, 0 represents the bottom of the stack, 1 the position just above the bottom, etc. If `idx` is negative, -1 represents the stack top, -2 the element just below the top, etc. If no argument is given, `idx` is set to -1 by default, i.e. the top element.

Note that with character stacks, if `x` should be the empty string, the string returned by **strings.dumpd** will be terminated at position `idx`.

stack.resetd ([···])

If given no arguments, clears the entire stack so that it becomes empty. The functions does not return anything and should be used cautiously as another function might still need elements in the stack.

If passed one or more valid stack numbers (integers), the function conducts the same for the given stack(s).

The number of pre-allocated slots is not reset, however, see **stack.shrinkd**.

With a cache stack, all slots that are discarded by a call to the function are nullified, i.e. set to **null**.

stack.reversed ([n])

If `n` is not given, reverses the positions of all the elements in the current stack. If `n` is given, only the last `n` elements pushed onto the stack are reversed. The function returns nothing.

See also: **strings.reverse**.

stack.rotated ([n])

Moves all the elements in the current stack `n` places from the bottom to the top if `n` is positive, and `n` places from the top to the bottom if `n` is negative. The default is `n` = +1. The function returns nothing.

stack.selected ()

Returns the number of the currently selected stack and the formerly active stack, in this order.

See also: **switchd**, **stack.choosed**, **stack.switchto**.

stack.shrinkd ([any])

With no argument, shrinks the number of pre-allocated but not assigned slots in the current stack if possible. If any argument is given, all stacks are shrunk. Cache stacks are *not* supported.

The function returns the new number(s) of pre-allocated slots but does not pop any elements. The function is useful to reduce memory consumption if a lot of values have been removed from the stack.

See also: **stack.resetd**.

stack.sized ()

Returns the current number of elements in the current stack along with the current maximum number of slots internally allocated by the system.

See also: **allotted**, **stack.attribd**, **stack.isempty**.

stack.sorted ([n])

Sorts all or the last `n` values pushed onto the current number or characher stack in ascending order using the fast Introsort algorithm. If no argument is passed, all elements in the current stack are processed. The function returns nothing. Cache stacks are not supported.

The function works with numeric stacks only.

stack.swapd (i, j)

Swaps the values stored at position `i` and `j` of the current stack.

stack.switchto ([x])

Determines the current stack number p, and then switches to the number stack q with the least number of elements if any number `x` or no argument is given, or to the character stack q with the least number of elements if any string `x` is given.

The function returns p and q, in this order.

You can use p later to switch back to the former stack with the **switchd** statement.

The function issues an error if the current stack is a cache.

The function memorises the formerly active stack. To change back to it, just issue the statement `` `switchd -1` ``.

stack.writebytes (filehandle [, bytes])

By default, writes all the values in a number or character stack to the file denoted by `filehandle`. If `bytes` is given, the number of bytes starting from the bottom of the stack are written. The function does not change the stack.

See also: **stack.readbytes**.

The following functions perform real arithmetic on a numeric stack:

stack.addtod (x [, true])

Adds its argument `x`, a number, to the top element of the current numeric stack.

The top stack element is replaced by the resulting sum if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

The function also returns the sum computed. It uses the Kahan-Babuška algorithm to prevent round-off errors.

See also: **stack.mulby**, **stack.powd**, **stack.sumupd**.

stack.mulbyd (x [, true])

Multiplies its argument `x`, a number, by the top element of the current numeric stack.

The top stack element is replaced by the resulting product if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns computed product.

See also: **stack.addto**, **stack.mulupd**, **stack.powd**, **stack.recipd**.

stack.absd ([true])

Computes the absolute value of the top element of the current numeric stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

stack.negated ([true])

Multiplies the top numeric stack element by -1.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

stack.recipd ([true])

Computes the reciprocal of the top element of the current numeric stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

stack.intd ([true])

Rounds the number on the top of a numeric stack to the nearest integer towards 0.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

stack.fracd ([true])

Converts the number on the stack top to its fractional part.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

stack.intdivd (x [, true])

Integer division of the top element of the current numeric stack and the given number `x`; works like the \ operator.

The top stack element is replaced by the result if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

See also: **stack.modd**.

stack.modd (x [, true])

Modulus of the top element of the current numeric stack and the given number `x`; works like the **%** operator.

The top stack element is replaced by the result if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

See also: **stack.intdivd**.

stack.powd (power [, true])

Raises the top element of the current numeric stack to the given `power`, a number.

The top stack element is replaced by the result if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

The function also returns the result.

See also: **stack.antilogd**, **stack.rootd**.

stack.pythad ([true])

Computes the hypotenuse or inverse hypotenuse of the number below the stack top and the number on the top and returns the computed result. The function avoids over- and underflows and treats subnormal numbers accordingly.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.hypotd**, **stack.invhypotd**.

stack.squared ([true])

Raises the top element of the current numeric stack to the power of 2.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .
The function also returns the result.

See also: **stack.antilogd**, **stack.powd**, **stack.rootd**.

stack.rootd (n [, true])

Computes the non-principal `n`-th root of the top element of the current numeric stack. `n` must be an integer and is 2 by default.

The top stack element is replaced by the result if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.cbrtd**, **stack.powd**, **stack.sqrtd**.

stack.cbrtd ([true])

Computes the cubic root of the top element of the current numeric stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.powd**, **stack.rootd**, **stack.sqrtd**.

stack.sqrtd ([true])

Computes the square root of the top element of the current numeric stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

See also: **stack.cbrtd**, **stack.powd**, **stack.rootd**.

stack.fmad ([true])

Computes x*y + z, where x represents the second number below the stack top, y the number below the stack top and z the number on the top and returns the computed result. The function avoids over- and underflows and treats subnormal numbers accordingly.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.hypotd ([true])

stack.invhypotd ([true])

Computes the hypotenuse or inverse hypotenuse of the number below the stack top and the number on the top and returns the computed result. The function avoids over- and underflows and treats subnormal numbers accordingly.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

See also: **stack.cathetd**, **stack.pythad**.

stack.cathetd ([true])

Computes $\sqrt{x^2 - y^2}$, where x represents the number below the stack top and y the number on the top and returns the computed result. The function avoids over- and underflows and treats subnormal numbers accordingly.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

See also: **stack.hypotd**.

stack.lnd ([true])

Returns the natural logarithm of the top element of the current numeric stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.antilogd**, **stack.expd**, **stack.logd**.

stack.logd (base [, true])

Returns the logarithm of the top element of the current numeric stack to the given `base`, with `base` a number. For the natural logarithm, pass **E** for `base`.

The top stack element is replaced by the result if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.antilogd**, **stack.lnd**.

stack.antilogd (base [, true])

Raises the given `base`, a number, to the power of the top element of the current numeric stack.

To compute the exponential function e^x, x should be the top stack element and pass **E** as function argument `base`.

The top stack element is replaced by the result if **true** is not given as the last argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.expd**, **stack.exp2d**, **stack.exp10d**.

stack.expd ([true])

Raises E = exp(1) to the power of the number at the stack top and returns it.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

See also: **stack.exp2d**, **stack.exp10d**.

stack.exp2d ([true])

Raises 2 to the power of the number at the stack top and returns it.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

See also: **stack.expd**, **stack.exp10d**.

stack.exp10d ([true])

Raises 10 to the power of the number at the stack top and returns it.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

See also: **stack.expd**, **stack.exp2d**.

stack.sind ([true])
stack.cosd ([true])
stack.tand ([true])

Returns the sine, cosine or tangent of the top element of the current numeric stack, in radians.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.sinhd ([true])
stack.coshd ([true])
stack.tanhd ([true])

Returns the hyperbolic sine, hyperbolic cosine or hyperbolic tangent of the top element of the current numeric stack, in radians.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.secd ([true])
stack.cotd ([true])
stack.cscd ([true])

Returns the secant, cotangent or cosecant of the top element of the current numeric stack, in radians.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.arcsind ([true])
stack.arccosd ([true])
stack.arctand ([true])

Returns the arcus sine, arcus cosine or arcus tangent of the top element of the current numeric stack, in radians.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.arcsinhd ([true])
stack.arccoshd ([true])
stack.arctanhd ([true])

Returns the inverse hyperbolic sine, inverse hyperbolic cosine or inverse hyperbolic tangent of the top element of the current numeric stack, in radians.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.arctan2d ([true])

Computes the arc tangent of y/x of the two top elements of the current numeric stack, in radians, where x represents the value just below the top and y the value at the top of the stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.erfd ([true])

Returns the error function of the top element of the current numeric stack.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top.

stack.meand ([n] [, true])

Computes the arithmetic mean of all or a given number `n` of values in the current numeric stack and either pushes the result onto the top of the stack if **true** has been passed as the last argument; or pops all the values processed and pushes the result on top of the stack if only `n` or no argument has been given.

The function in general also returns the arithmetic mean.

By dividing each element before summation, the function avoids arithmetic overflows and also uses the Kahan-Babuška algorithm to prevent round-off errors during summation. Computation is done from top to bottom.

stack.sumupd ([n] [, true])

Sums up all or a given `n` number of values in the built-in numeric stack and either pushes the result onto the top of the stack if **true** has been passed as the last argument; or pops all the values summed-up and pushes the sum on top of the stack if only `n` or no argument has been given.

Summation is done from top to bottom. The function uses the Kahan-Babuška algorithm to prevent round-off errors during summation and also returns the sum.

stack.mulupd ([n] [, true])

Multiplies all or a given `n` number of values in the built-in numeric stack and either pushes the result onto the top of the stack if **true** has been passed as the last argument; or pops all the values multiplied and pushes the product on top of the stack if only `n` or no argument has been given.

Multiplication is done from top to bottom. The function in general also returns the product.

The following functions perform operations on the two values at the top of the stack, i.e. the value at the top and the value just below it. They have been deliberately kept lean for maximum performance:

stack.addtwod ()

Adds up the two numbers on top of the current stack and replaces them with the result; also returns the sum.

stack.subtwod ()

Subtracts the number on top of the current stack from the number below it and replaces them with the result; also returns the difference.

stack.multwod ()

Multiplies the number on top of the current stack by the number below it and replaces them with the result; also returns the product.

stack.divtwod ()

Divides the number below the stack top by the number on the top and replaces them with the result; also returns the quotient.

stack.intdivtwod ()

Performs an integer division of the number below the stack top by the number on the top and replaces them with the result; also returns the integer quotient.

stack.modtwod ()

Like **stack.intdivtwod** but computes the modulo.

stack.powtwod ()

Raises the number below the stack top to the power on the top and replaces both numbers with the result; also returns the power.

The following functions perform operations on a character stack:

stack.absd ()

Returns the ASCII value of the top element of the current numeric stack.

The function does not modify the stack.

stack.lowerd ([true])

Converts the character at the stack top to lower-case and returns it.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

stack.upperd ([true])

Converts the character at the stack top to upper-case and returns it.

The top stack element is replaced by the result if **true** is not given as an argument, but the rest of the stack is left unchanged. Otherwise, if you pass **true**, the top stack element is not replaced and the result is just put onto the stack top .

14.7 sema - Unique Identifiers

The package provides functions to create and administer unique, non-negative integer identifiers, called `semaphore ids` in this context, in a memory-saving fashion (see **sema.state** and **sema.open** for details).

There are two types of `semaphores`: a global built-in one, and the semaphore instance that counts on its own and has its own state. You can create one or more semaphore instances.

Example: Create new global semaphore ids:

```
> sema.open(), sema.open(), sema.open(), sema.open():
0   1   2   3
```

Create a semaphore instance s:

```
> s := sema.new():
sema(01CCD3B8)
```

Now let this new instance create unique ids:

```
> sema.open(s), sema.open(s), sema.open(s):
0   1   2
```

We will `close` id 0 created before by instance s:

```
> sema.close(s, 0):
```

If **sema.open** is called again for instance s then it returns the id just previously `closed` as the latter has been freed before.

```
> sema.open(s):
0
```

And as id 1 and 2 are still active, that is `open`, the next id returned by the call to **sema.open** is:

```
> sema.open(s):
3

> t := sema.new():  # create another semaphore instance t
sema(01CCD298)

> sema.open(t):     # create a new semaphore id for semaphore t
0
```

With instances, the **sema** package functions can also by called OOP-style except where indicated otherwise. A short example:

```
> s := sema.new();

> s@@open(), s@@open(), s@@open():
0   1   2

> s@@close(0):

> s@@open(), s@@open():
  0   3

> s@@state():
[current ~ 3, firstfreeslot ~ 0, maxopen ~ 3, minopen ~ 0, nextfree ~ 4,
 simplecounter ~ 2, simplecounter_active ~ false, slots ~ seq(15, 0, 0, 0,
 0, 0, 0, 0), totalslots ~ 8]
```

sema.close (id, [···])

sema.close (s, id, [···])

In the first form, closes the given global semaphore `id`, and optionally other ids, as well. If **sema.open** is called thereafter, it will return `id`. The function returns nothing. Note that if `id` is not the last semaphore id opened before, then Agena automatically switches to dynamic memory management consuming a linear amount of memory instead of using just one 8-byte integer for the internal counter.

In the second form, the `id` of semaphore `s` is closed.

See also: **sema.open**, **sema.shrink**.

sema.isopen ([s,] id)

Checks whether the given global semaphore `id` or the `id` of semaphore instance `s` is open - that is has been created and is active - and returns **true** or **false**.

See also: **sema.open**, **sema.close**.

sema.limit (s [, newlimit])

Increases the maximum number of eligible concurrent open ids for semaphore instance `s` to `newlimit`. You cannot decrease the limit, but you may close one or more open ids and then call **sema.limit**. If just `s` is given, returns the current limit.

sema.new ([maxopen])

Creates a new semaphore instance that if subsequently passed to **sema.open** returns unique ids.

By default, on all platforms, the maximum number of concurrently open ids is 2^{31} - 1. You can pass any other limit `maxopen` less than this limit. If the number of concurrently open ids would exceed `maxopen`, then **sema.open** in subsequent calls will return **undefined**.

This function cannot be used as an OOP method.

See also: **sema.limit**.

sema.open ([s,] [id])

Without any argument, creates a new global semaphore id, a non-negative integer. With the very first call, returns 0, counting up by 1 subsequently as long as **sema.close** or **sema.reset** are not executed.

If the non-negative global integer id `id` is given and `id` has not yet been created by **sema.open**, the function creates and returns it.

If semaphore instance `s` is being passed, then the function returns either the next free id if `id` is not given, or the given `id`. (To create a semaphore instance, call **sema.new**.)

This feature also allows you to write functions dumping the current semaphore state to a file and re-load it later, see **binio.writechar**, **binio.readchar**, **bytes.tobytes**.

If the function returns **undefined** then the number of simultaneously open ids would have exceeded the limit that has been given at generation of the semaphore instance. You must either close one or more ids with **sema.close**, reset the instance with **sema.reset** or increase the limit with **sema.limit**.

If the current internal administrative memory has been exceeded, the function automatically expands it. Note that if `id` is not the next semaphore id created in the previous call, then Agena automatically switches to dynamic memory management consuming a linear amount of memory instead of using just one single 8-byte integer for the internal counter.

See also: **environ.ref**, **factory.count**, **utils.uuid**.

sema.reset ([s])

Closes all allocated semaphore ids and also shrinks the internal memory used to administer the ids to the default size, if possible. If semaphore `s` is given, then all ids of this semaphore are reset.

The function returns nothing.

See also: **sema.limit**, **sema.shrink**.

sema.shrink ([s])

Without any argument, shrinks the internal memory used to administer all global semaphores ids so that they consume the least necessary space. The same applies solely to semaphore `s` if given. If internal memory size could be reduced, it returns **true** and **false** otherwise. Note that due to performance reasons, **sema.close** does not try to reduce memory consumption.

See also: **sema.reset**.

sema.state ([s])

Returns administrative information on the current global semaphore state if no argument is given or on semaphore instance `s`, in a table.

The semaphore ids are internally stored using an 8 * 32 bits array by default. Each 32-bit chunk is called a `slot`.

The `'firstfreeslot'` field contains the number of the first slot that might harbour a new id. Note that once you have used **sema.close**, this may just be an estimate.

The `'slot'` sequence returns the bits set in all slots, as decimal non-negative integers.

Field `'current'` is an estimate of the last allocated semaphore id.

Key `'minopen'` denotes the semaphore id with the smallest id, `'maxopen'` the one with the largest id.

`'nextfree'` denotes the semaphore id if **sema.open** would be called next.

`'open'` gives the number of currently open ids. `'maxopen'` is the maximum number of eligible open ids.

`'simplecounter_active'` indicates whether the memory-efficient counting is active (**true**) or dynamic memory management of the semaphore ids is in use (**false**).

`'simplecounter'` depicts the last semaphore id created by the simple counter.

For an example, see the start of this subchapter.

14.8 Coroutines

The operations related to coroutines comprise a sub-library of the basic library and come inside the table `coroutine`. To find out what coroutines are, please have a look at the website of the Lua programming language.

coroutine.resume (co [, val1, ···])

Starts or continues the execution of coroutine `co`. The first time you resume a coroutine, it starts running its body. The values `val1`, ··· are passed as the arguments to the body function. If the coroutine has yielded, resume restarts it; the values `val1`, ··· are passed as the results from the yield.

If the coroutine runs without any errors, resume returns **true** plus any values passed to yield (if the coroutine yields) or any values returned by the body function (if the coroutine terminates). If there is any error, resume returns **false** plus the error message.

coroutine.running ()

Returns the running coroutine, or **null** when called by the main thread.

coroutine.setup (f)

Creates a new coroutine, with body `f`. `f` must be an Agena function. Returns this new coroutine, an object with type 'thread'.

coroutine.status (co)

Returns the status of coroutine `co`, as a string: 'running', if the coroutine is running (that is, it called status); 'suspended', if the coroutine is suspended in a call to yield, or if it has not started running yet; 'normal' if the coroutine is active but not running (that is, it has resumed another coroutine); and 'dead' if the coroutine has finished its body function, or if it has stopped with an error.

coroutine.wrap (f)

Creates a new coroutine, with body `f`. `f` must be an Agena function. Returns a function that resumes the coroutine each time it is called. Any arguments passed to the function behave as the extra arguments to resume. Returns the same values returned by **resume**, except the first boolean. In case of error, propagates the error.

coroutine.yield (···)

Suspends the execution of the calling coroutine. The coroutine cannot be running a C function, a metamethod, or an iterator. Any arguments to `yield` are passed as extra results to resume.

14.9 debug - Debugging

This library provides the functionality of the debug interface to Agena programmes. You should exert care when using this library. The functions provided here should be used exclusively for debugging and similar tasks, such as profiling. Please resist the temptation to use them as a usual programming tool: they can be very slow. Moreover, several of its functions violate some assumptions about Agena code (e.g., that variables local to a function cannot be accessed from outside or that userdata metatables cannot be changed by Agena code) and therefore can compromise otherwise secure code.

All functions in this library are provided inside the `debug` table. All functions that operate over a thread have an optional first argument which is the thread to operate over. The default is always the current thread.

debug.debug ()

Enters an interactive mode with the user, running each string that the user enters. Using simple commands and other debug facilities, the user can inspect global and local variables, change their values, evaluate expressions, and so on. A line containing only the word `cont` or `bye` finishes this function, so that the caller continues its execution.

Note that commands for **debug.debug** are not lexically nested within any function, and so have no direct access to local variables.

debug.funcname (level)

Returns the name of the function in which it (i.e. **debug.funcname**) has been called. The return is a string. It is a wrapper for "debug.getinfo(level, "n").name". By default, `level` 1 is used, but you may pass another `level`. If `level` is out of range, then **fail** will be returned. If the function name could not be determined, **null** will be returned. The function may be useful to create more flexible error messages.

debug.getconstants ()

Returns the internal set that stores global constants.

debug.getfenv (obj)

Returns the environment of object `obj`.

See also: **debug.setfenv**.

debug.gethook ([thread])

Returns the current hook settings of the thread, as three values: the current hook function, the current hook mask, and the current hook count (as set by the **debug.sethook** function).

debug.getinfo ([thread,] function [, what])

Returns a table with information about a function. You can give the function directly, or you can give a number as the value of `function`, which means the function running at level `function` of the call stack of the given thread: level 0 is the current function (**getinfo** itself); level 1 is the function that called **getinfo**; and so on. If `function` is a number larger than the number of active functions, then **getinfo** returns **null**.

The returned table may contain all the fields returned by **lua_getinfo**, with the string what describing which fields to fill in. The default for what is to get all information available, except the table of valid lines. If present, the option `'f'` adds a field named `func` with the function itself. If present, the option `'L'` adds a field named `activelines` with the table of valid lines. If present, the option `'g'` adds a field named `globals` with a table of variables that have been globally assigned. The `'a'` option adds a field called `arity` that includes the number of arguments - excluding possible varargs - expected by `function`. When given the `'v'` formatter, the 'varargs' entry indicates whether varargs can be passed to the function (**?** in its parameter list). `'V'` returns a table of all parameters and locally declared variables along with their current values. `'c'` returns activation record ar->i_ci setting. `'k'` returns a table with all the values internally treated as `constants`.

For instance, the expression `debug.getinfo(1, 'n').name` returns a name of the current function, if a reasonable name can be found, and `debug.getinfo(print)` returns a table with all available information about the **print** function.

See also: **debug.funcname**, **debug.getk**.

debug.getk (function)

debug.getk (level)

Returns the values internally treated as `constants` of the function in which it (i.e. **debug.getk**) has been called. It is a wrapper for "debug.getinfo(level, "k")". By default, `level` 1 is used, but you may pass another level. If level is out of range, then fail is returned.

Instead of a `level`, you may pass the function `function` of interest.

debug.getlocal ([thread,] level, local)

This function returns the name and the value of the local variable with index `local` of the function at level `level` of the stack. (The first parameter or local variable has index 1, and so on, until the last active local variable.) The function returns **null** if there is no local variable with the given index, and raises an error when called with a `level` out of range. (You can call **debug.getinfo** to check whether the level is valid.)

Variable names starting with '`(`' (open parentheses) represent internal variables (loop control variables, temporaries, and C function locals).

See also: **debug.getlocals**, **debug.setlocal**.

debug.getlocals (level [, option])

debug.getlocals (function)

In the first form works like **debug.getlocal**, but returns a table of all local variables of a function running at level `level` of the stack. The array part of the table includes the variable names, the hash part the variables and their current values as key ~ value pairs. The other returns, in the following order are: number of local variables including procedure parameters, number of procedure parameters, a boolean indicating whether **?** (varargs) is part of the parameter list, a table with all unassigned local variables.

By passing any `option`, only an array of parameter and local variable names plus the number of parameters (first entries in the array) will be returned.

In the second form, you pass a function `function` instead of a `level`.

See also: **debug.globals**, **debug.unused**.

debug.getmetatable (object)

Returns the metatable of the given `object` or **null** if it does not have a metatable.

See also: **debug.setmetatable**.

debug.getregistry ([idx])

Without any argument, returns the registry table, see Chapter 6.31. Do not change values with integer keys or the '__gc' key - this would destroy information needed by userdata and could lead to undefined behaviour of the interpreter.

With the given index `idx` returns the value associated with the registry field `idx` only, or **null** if there is none. See also: **registry.get**, **registry.unref**.

debug.getrtable (f)

Returns a reference to the internal remember table of procedure `f`. Opposed to **rtable.rget**, the function gives you direct read and write access to the remember table, so use it with care.

debug.getstore (f)

Returns a reference to the internal storage table of procedure `f`. You can both inspect this table as well as inject values into it. See **debug.setstore** or source file lib/mapm.agn or Chapter 6.25 for an example of how to store precomputed values, like Chebyshev coefficients, to a function's store, to be used later at function invocation.

debug.getupvalue (f, up)

This function returns the name and the value of the upvalue with index `up` of the function `f`. The function returns **null** if there is no upvalue with the given index and returns nothing if f is a C closure (use **debug.getupvalues** instead).

See also: **debug.getupvalues**, **debug.nupvalues**, **debug.setupvalue**.

debug.getupvalues (f [, true])

Returns all upvalues of Agena or C closure `f` in a table, plus the number of upvalues. With C functions, the first entry in the table array depicts the first upvalue, and so on. With Agena closures, the upvalue names and associated values are returned in a hash table.

If there are no upvalues, the return is **null** plus zero. Examples:

```
> t := tuples.new(10, 20, 30);  # a C closure

> debug.getupvalues(t):
[10, 20, 30]    3

> f := proc() is  # an Agena closure
>    local count := 0;
>    return proc()
>       count++;
>       return count
>    end
> end;

> debug.getupvalues(f()):
[count ~ 0]     1
```

If you pass the second optional argument **true**, then the function returns all the upvalue names of Agena closures, but not their associated values, in a table array.

See also: **debug.getupvalue**, **debug.nupvalues**, **debug.setupvalue**, **environ.arity**.

debug.globals (f)

Determines[9] whether function `f` uses global variables (names which have not been defined local and are read or assigned). The return is a sequence of pairs: their left-hand side the variable name of type string, the right-hand side the respective line number (of type number). If no global variables could be found, the function returns **null**.

See also: **debug.unused**.

debug.listcode (f)

Breaks down an Agena function into its opcode instructions and returns them in a table. Each line in it starts with the respective code line number in brackets.

Example:

```
> f := <: ()
>       with k := 101;
>       j := k * 2 ->
>       k + j :>

> for i in debug.listcode(f) do print(i) od
(   2)    0 - LOADK          0    0
(   3)    1 - MUL            1    0  257
(   4)    2 - ADD            2    0    1
(   4)    3 - RETURN         2    2    0
(   4)    4 - RETURN         0    1    0
```

debug.nupvalues (f)

Returns the number of upvalues in a C or Agena closure.

See also: **debug.getupvalue**, **debug.getupvalues**.

debug.setfenv (object, t)

Sets the environment of the given `object` to the given table `t`. Returns `object`.

See also: **debug.getfenv**, **environ.setfenv**.

[9] Note that the function not always returns all global names.

debug.sethook ([thread,] hook, mask [, count])

Sets the given function as a hook. The string `mask` and the number `count` describe when the hook will be called. The string mask may have the following characters, with the given meaning:

- `'c'`: The hook is called every time Agena calls a function;
- `'r'`: The hook is called every time Agena returns from a function;
- `'l'`: The hook is called every time Agena enters a new line of code.

With a `count` different from zero, the hook is called after every `count` instructions.

When called without arguments, **debug.sethook** turns off the hook.

When the hook is called, its first parameter is a string describing the event that has triggered its call: `'call'`, `'return'` (or `'tail return'`), `'line'`, and `'count'`. For line events, the hook also gets the new line number as its second parameter. Inside a hook, you can call **debug.getinfo** with level 2 to get more information about the running function (level 0 is the **getinfo** function, and level 1 is the hook function), unless the event is `'tail return'`. In this case, Agena is only simulating the return, and a call to **getinfo** will return invalid data.

debug.setlocal ([thread,] level, local, value)

This function assigns the value `value` to the local variable with index `local` of the function at level `level` of the stack. The function returns **null** if there is no local variable with the given index, and raises an error when called with a `level` out of range. (You can call **debug.getinfo** to check whether the level is valid.) Otherwise, it returns the name of the local variable.

See also: **debug.getlocal**.

debug.setmetatable (object, t)

Sets the metatable for the given `object` to the given table `t` (which can be **null**).

See also: **debug.getmetatable**.

debug.setstore (f, t)

If not yet established, sets up a store for procedure `f` with the contents of table `t`. If already established, adds all the key~value pairs in table `t` to the internal store of procedure `f`. If `t` is **null**, then the store is deleted entirely.

See also: **debug.getstore**.

debug.setupvalue (f, up, value)

This function assigns the value `value` to the upvalue with index `up` of the function `f`. The function returns **null** if there is no upvalue with the given index. Otherwise, it returns the name of the upvalue.

See also: **debug.getupvalue**.

debug.system ()

Returns a table with the following system information: The size of various C types (char, int, long, long long, float, double, int32_t, int64_t), the smallest and largest numeric values for C doubles, C long ints, C long long ints, and C unsigned long ints (all compiled into the Agena binary), the endianness of your platform, the hardware and the operating system for which the Agena executable has been compiled.

See also: **environ.kernel**.

debug.traceback ([thread,] [message])

Returns a string with a traceback of the call stack. An optional `message` string is appended at the beginning of the traceback. This function is typically used with **xpcall** to produce better error messages.

debug.unused (f [, option])

Identifies candidates of local variables that are declared but never referenced within function `f`. The emphasis is on *candidates* as loop control variables will be output as false positives, as will be names that are used with the Boolean operators **or** and **and**. However, at the time of writing, the function proved to be very handy in everyday use.

Verify that candidate locals are truly unused before removal.

When only `f` is given the return is a table of pairs with the left-hand side the name of the local variable and the right-hand side the approximate line number of its first appearance in `f`. If no candidate has been found, the table is empty. If `f` is not an Agena function, the function returns nothing. If any `option` is given, the candidates are printed at the console and the function returns nothing.

Example:

```
> f := proc(x) is
>    local one, two, three;
>    one := 1;
>    two := 2;
>    return one + two
> end;
```

```
> debug.unused(f, true);
Candidate 'x' around line 3, register [0] (PC 0-4)
Candidate 'three' around line 3, register [3] (PC 0-4)
```

A coarse five-liner to check a whole package is:

```
> for i, j in mypackage do
>    unused := debug.unused(j);
>    if unused :: table and filled(unused) then
>       print(i, unused)
>    fi
> od;
```

See also: **debug.getlocals**, **debug.globals**.

Excursion: There is an extension to the **debug** library, **abci** - short for Agena Byte Code Inspector - that wraps various **debug** functions used to inspect functions written in Agena. Try for example:

```
> import abci

> f := proc(x, y) is
>    print(x, '=', y, 23);
>    local z := x + 2*y;
>    local last;
>    X := 245;
>    return inspect
> end

> abci.inspect(debug.getinfo(1).func, true)
header  procedure(01FC31C0)
        upvalues        0
        source  stdin
        functions       0
        instructions    10
        slots   3
        constants       6
        line    0
        lastline        0
        isvararg        true
        locals  0
        params  0
constants       6
        1       abci
        2       inspect
        3       debug
        4       getinfo
        5       1
        6       func
locals  0
upvalues        0
functions       0
instructions    10
        1       1       GETGLOBAL       0       -1      null
>>>     abci
        2       1       GETTABLE        0       0       -2
        3       1       GETGLOBAL       1       -3      null
>>>     debug
        4       1       GETTABLE        1       1       -4
        5       1       LOADK   2       -5      null
```

```
6       1       CALL    1       2       2
7       1       GETTABLE        1       1       -6
8       1       LOADBOOL        2       1       0
9       1       CALL    0       3       1
10      1       RETURN  0       1       null
```

14.10 history - Logging Your Input

This library allows to write all input on the command-line automatically to a file. Only syntactically correct input will be written.

You can switch off logging at any time or write the input to a different file.

If the log file already exists, any subsequent input will be appended. Agena will never delete the log file or overwrite its contents by itself.

By default, there is no command-line logging, that means every time you start a new session, you must explicitly turn it on.

history.on ([logfile])

By default, with no argument given, writes all the statements entered at the Agena prompt to the file `agena.log` in the current working directory.

With the string `logfile` given, the history will be written to the given file, which may include any full path, for example:

```
> history.on('c:/agena/log.txt');
```

If you pass a wrong path or filename, then Agena automatically turns off logging if activated.

See also: **skycrane.tee**.

history.off ()

Switches off command-line logging.

See also: **environ.kernel/history** and **environ.kernel/histfile**.

Chapter Fifteen

Graphics

15 Graphics

15.1 gdi - Graphic Device Interface package

As a *plus* package, this graphics interface is not part of the standard distribution and must be activated with the **import** statement, e.g. `import gdi`.

The gdi package provides functions to plot graphics either to a window or a PNG, GIF, JPEG, FIG, or PostScript file. It is available for the Solaris, Linux, Mac OS X for Intel CPUs, and Windows editions of Agena. There is an experimental OS/2 - ArcaOS version that is still experimental and it can create FIG and PostScript files only.

For required dependencies, check Chapter Two for your platform. In short, Linux, Mac OS X and Solaris users may have to install them before using the gdi package. The OS/2 and Windows installers already include all necessary dependencies.

The gdi package provides procedures to plot basic geometric objects such as points, lines, circles, ellipses, rectangles, etc.

It also provides means to easily plot graphs of univariate functions and geometric objects where the user does not need pay attention for proper axis ranges, mapping to the internal coordinate systems, etc.

15.1.1 Opening a File or Window

Operation starts by opening a device - window or file - with the **gdi.open** function. The function returns a device handle for later reference. Almost all functions provided by the package request this device handle.

```
> import gdi;

> d := gdi.open(640, 480);
```

15.1.2 Plotting Functions

Plot a point to the window at x=200 and y=100:

```
> gdi.point(d, 200, 100);
```

Plot a line between two points [200, 150] and [300, 200]:

```
> gdi.line(d, 200, 150, 300, 200);
```

Draw a circle and a filled circle. Besides giving the device number, pass a centre (x and y co-ordinates) and a radius.

```
> gdi.circle(d, 320, 240, 50);

> gdi.circlefilled(d, 400, 240, 50);
```

15.1.3 Colours, Part 1

All functions accept a colour option passed as an additional - the last - argument.

The colour must be given as an integer that must be determined by a call to the **gdi.ink** function. **gdi.ink** requires the device number, and three RGB colour values in the range [0 .. 1]. Each colour should be determined only once.

There are 26 predefined colours with numbers 0 to 25, automatically set at each invocation of a new device (call to the **gdi.open** function). Thus, these 26 basic colours do not need to be explicitly set with **gdi.ink**.

The default colours are:

0	white	7	light green	14	grey	21	purple
1	black	8	greenish	15	grey-blue	22	dark orange
2	blue	9	light sky-blue	16	bright green	23	purple
3	light blue	10	bordeaux	17	light greenish	24	light lilac
4	greenish	11	lilac	18	light sky-blue	25	yellow
5	cyan	12	light lilac	19	red		
6	sky-blue	13	khaki	20	purple		

```
> cyan := gdi.ink(d, .1, .5, .5);

> gdi.rectanglefilled(d, 200, 200, 400, 400, cyan);
```

If you want to set a default colour for all subsequent drawings, use **gdi.useink**.

15.1.4 Closing a File or Window

To finally close the window, use **gdi.close**.

```
> gdi.close(d);
```

15.1.5 Supported File Types

To create image files, simply pass the name of the file as the third argument to **gdi.open**. Agena determines the type of the image file from its suffix.

If a file name ends in `.png`, it creates a PNG file. If a file name ends in `.gif`, it creates a GIF file. If a file name ends in `.jpg`, it creates a JPEG file. Likewise, the suffix `.fig` creates a FIG, and `.ps` generates a PostScript file.

15.1.6 Plotting Graphs of Univariate Functions

The **gdi.plotfn** function plots graphs of functions in one real to a window or file. It accepts various options for colour, line thickness, line style, sizing, axis type, etc. The function takes care for opening a device, plotting the graph and axes, so that the user does not need to draw them manually. The function requires a function and the left and right border on the x-axis.

```
> import gdi alias

> plotfn(<: x -> x*sin(x) :>, -10, 10);
```

For further details and examples see **gdi.plotn**. For available plot options, see gdi.options. See **calc.nokspline** which along with **gdi.plotfn** generates a smoothed graph through a given list of interpolation points.

15.1.7 Plotting Geometric Objects Easily

Like **gdi.plotfn**, the gdi function **plot** outputs geometric objects in the Cartesian co-ordinate system with the point [0, 0] its centre. It accepts options for user-defined colours, window sizes, axis types, etc. The function opens a device automatically, plots all the objects that are stored in a PLOT data structure optionally along with axes, a user-defined background colour, etc.

The function requires the PLOT structure as the first argument, and any options as additional arguments. Contrary to **gdi.plotfn**, it does not accept left, right, lower or upper borders, for it determines the borders automatically.

A PLOT data structure is a sequence of the user-defined type 'PLOT', and contains the geometric objects with their positions and respective colours.

The following geometric objects can be drawn with **gdi.plot**:

Object	Name	Object	Name
arc	`ARC`	line	`LINE`
filled arc	`ARCFILLED`	point	`POINT`
circle	`CIRCLE`	rectangle	`RECTANGLE`
filled circle	`CIRCLEFILLED`	filled rectangle	`RECTANGLEFILLED`
ellipse	`ELLIPSE`	triangle	`TRIANGLE`
filled ellipse	`ELLIPSEFILLED`	filled triangle	`TRIANGLEFILLED`

A line stretching from [0, 0] to [1, 1] in grey colour (RGB values 0.5, 0.5, 0.5) for example is represented as follows:

LINE(0, 0, 1, 1, [0.5, 0.5, 0.5])

A PLOT structure can be created with the **gdi.structure** function that optionally accepts the minimum number of entries (for speed).

```
> import gdi alias;

> s := structure();
```

Any geometric objects is inserted into the structure with its respective **gdi.set*** function. The line LINE(0, 0, 1, 1, [0.5, 0.5, 0.5]) for example is added with the **gdi.setline** function:

```
> setline(s, 0, 0, 1, 1, [0.5, 0.5, 0.5]);
```

A PLOT structure can include any number of objects:

```
> setcircle(s, 0, 0, 0.5, [1, 0, 0]);
```

Finally, the **plot** statement puts them onto the screen:

```
> plot(s);
```

The following table shows the various functions to create objects:

Object	Function	Object	Function	Object	Function
arc	`setarc`	ellipse	`setellipse`	rectangle	`setrectangle`
filled arc	`setarcfilled`	filled ellipse	`setellipse-filled`	filled rectangle	`setrectangle-filled`
circle	`setcircle`	line	`setline`	triangle	`settriangle`
filled circle	`setcircle-filled`	point	`setpoint`	filled triangle	`settriangle-filled`

15.1.8 Colours, Part 2

The following colour names (of type string) are built in and are accepted by the **gdi.plot** and **gdi.plotfn** functions only, so that you must not define colours with **gdi.useink** or **gdi.ink** when plotting sets of points or graphs of functions:

```
'aquamarine', 'black', 'blue', 'bordeaux', 'brown', 'chartreuse', 'coral',
'cyan', 'darkblue', 'darkcyan', 'darkgrey', 'gold', 'green', 'grey',
'khaki', 'lightgrey','magenta', 'maroon', 'navy', 'orange', 'pink', 'plum',
'red', 'sienna', 'skyblue', 'spring' 'tan', 'turquoise', 'violet', 'wheat',
'white', 'yellow', 'yellow2'.
```

You can access them through the gdi.COLOURS table, too, e.g. gdi.COLOURS.aquamarine, gdi.COLOURS.black, etc.

15.1.9 GDI Functions

gdi.arc (d, x, y, r1, r2, a1, a2 [, colour])

Draws an arc around the centre [x, y] with x radius r1, y radius r2, and the starting and ending angles a1, a2, given in degrees [0 .. 360], on device d. A colour (an integer, see Chapter 15.1.3 Colours, Part 1), may be given optionally.

gdi.arcfilled (d, x, y, r1, r2, a1, a2 [, colour])

Draws a filled arc around the centre [x, y] with x radius r1, y radius r2, and the starting and ending angles a1, a2, given in degrees [0 .. 360], on device d. The arc is filled with either the default colour, or the one given by colour (an integer, see Chapter 15.1.3 Colours, Part 1).

gdi.autoflush (d, state)

Sets the auto flush mode for device d to either **true** or **false** (second argument). If state is **true** (the default), then after each graphical operation the output is flushed so that it is immediately displayed.

This may decrease performance significantly with a large number of graphical operations - Sun Sparcs seem to be the only exceptions -, so it is advised to

1. set state to **false** right after opening device d before calling any other function that plots something,
2. call **gdi.flush** after the graphical operations have been completed,
3. set state to **true** thereafter.

gdi.background (d, c)

Sets the background colour on device d. c must be a number determined by **gdi.ink**, see Chapter 15.1.3 Colours, Part 1. Note that in Windows, the image is also cleared so that the background is properly displayed, whereas in UNIX, the image is not reset.

gdi.circle (d, x, y, r [, colour])

Draws a circle around the centre [x, y] with radius r, on device d. A colour (an integer, see Chapter 15.1.3 Colours, Part 1), may be given optionally.

gdi.circlefilled (d, x, y, r [, colour])

Draws a filled circle around the centre [x, y] with radius r, on device d. The circle is filled with either the default colour, or the one given by colour (an integer, see Chapter 15.1.3 Colours, Part 1).

gdi.clearpalette (d)

Removes all inks on device `d`.

gdi.close (d)

Closes the window or file referred to by device id `d`. If d points to a file, all image contents is saved to it.

gdi.dash (d, s)

Sets the line dash on device id `d`. The sequence `s` includes a vector of dash lengths (black, white, black, ...). If `s` is the empty sequence, a solid line is restored.

gdi.ellipse (d, x, y, r1, r2 [, colour])

Draws an ellipse around the centre [`x`, `y`] with x radius `r1`, and y radius `r2`, on device `d`. A `colour` (an integer, see Chapter 15.1.3 Colours, Part 1), may be given optionally.

gdi.ellipsefilled (d, x, y, r1, r2 [, colour])

Draws a filled ellipse around the centre [`x`, `y`] with x radius `r1`, and y radius `r2`, on device `d`. The ellipse is filled with either the default colour, or the one given by `colour` (an integer, see Chapter 15.1.3 Colours, Part 1).

gdi.flush (d)

Writes all buffered contents to the window or file referred to by device id `d`.

See also: **gdi.autoflush**.

gdi.fontsize (d, s)

Sets the font size `s` for text written by gdi.text, for device `d`.

See also: **gdi.text**.

gdi.hasoption (t, o)

Iterates a table `t` and returns true if one of its keys is equal to `o`.

See also: **gdi.options**.

gdi.initpalette (d)

Sets up basic colours on device `d`.

gdi.ink (d, r, g, b)

Returns a palette colour value - an integer - for the colour given by its RGB values `r` (red), `g` (green), and `b` (blue), for device `d`. `r`, `g`, and `b` must be numbers x with $0 \leq x \leq 1$. The palette colour value can be given as an optional argument in most of the **gdi** functions, or be used in the **gdi.useink** function. Subsequent calls with the same arguments return different palette values.

gdi.lastaccessed ()

Returns the id of the last accessed device as a number.

gdi.line (d, x1, y1, x2, y2 [, colour])

Draws a line from the first point [`x1`, `y1`] to the second point [`x2`, `y2`] on device `d`. A `colour`, an integer (see Chapter 15.1.3 Colours, Part 1), may be given optionally.

gdi.lineplot (p [, options])
gdi.lineplot ([p1 [, p2, ···]], [, options])

Takes one or more tables or sequences consisting of points x_k:y_k and generates a plot with all points connected by lines. x_k and y_k must be finite numbers. The function automatically determines the common proper borders automatically.

For more information see: **gdi.pointplot**, as **gdi.lineplot** is just a wrapper for the former with the 'connect' option set to **true**.

gdi.mouse (d [, offset])

Returns three numbers: the current horizontal and vertical positions of the mouse relative to the screen, and its button state *button_state*. The button state is coded as a positive integer.

By applying a bitmask to the button state, you can query whether the left or the right mouse button has been pressed:

- *button_state* && 0x0100 = 0x0100: left button has been pressed,
- *button_state* && 0x0400 = 0x0400: right button has been pressed.

gdi.open (width, height)

gdi.open (width, height, filename)

In the first form, opens a window with the given `width` and `height` and returns a device number (an integer) for later reference needed by all other **gdi** functions.

In the second form, creates the image file with name `filename`, the given `width` and `height` and returns a device number (an integer) for later reference needed by all other **gdi** functions.

The type of the image file format is determined by the suffix in `filename`:

Suffix	Resulting image file format	Example
.fig	FIG format	`'/export/home/misc/fern.fig'`
.gif	GIF format	`'c:/images/fractal.gif'`
.jpg	JPEG format	`'c:/images/fractal.jpg'`
.png	PNG format	`'c:/images/circle.png'`
.ps	PostScript format (DIN A4 size)	`'output.ps'`

`gdi.options (···)`

Checks the given plotting options for correctness and returns them in a new table, along with the defaults for options that have not been passed to this function. The function currently only works with the **gdi.plot**, **gdi.pointplot**, and **gdi.plotfn** functions.

Valid options (all key~value pairs) are:

Option (key)	Meaning (value)	Example
`'axes'`	`'none'` - do not print axes `'normal'` - print axes with labels and tick marks `'boxed'` - print axes at top and bottom, and at the left and the right side `'frame'` - print axes at the bottom and at the left side	`'axes':'normal'`
`'axescolour'`	defines the colour of the axes (a colour string, see Chapter 15.1.3)	`'axescolour':'red'`
`'bgcolour'`	sets the background colour (a colour string, see Chapter 15.1.3)	`'bgcolour':` `'yellow'`
`'colour'`	sets the default colour (a string, see Chapter 15.1.3) for the objects to be plotted. Note that the individual colour of an object overrides the one given by this option	`'colour':'navy'`
`'colourfn'`	sets a colouring function	`'colourfn':` `<: x -> ... :>`
`'file'`	indicates the name of the file (a string) to be created	`'file':'image.png'`
`'labels'`	if set to false, no labels are printed (default is **true**)	`'labels':false`
`'labelsize'`	sets the font size (a positive number) for axis labels (**gdi.plotfn** function only)	`'labelsize':6`
`'linestyle'`	sets the dash style (a positive number) for the graph to be plotted (**gdi.plotfn**, **gdi.lineplot**, and **gdi.pointplot** functions only)	`'linestyle':10`

Option (key)	Meaning (value)	Example
'maxtickmarks'	sets the maximum number of tickmarks on both axes, by default is (around) 20.	'maxtickmarks':5
'mouse'	prints the current position of the mouse to the console. Click the right mouse button to finish. Default is **false**.	'mouse':true
'res'	resolution of the window or image file in pixels (pair of numbers)	'res':(1024:768)
'square'	in a plot, uses the same scale for the y-axis as given for the x-axis	'square':true
'thickness'	sets the thickness (a positive number) of the line to be plotted (**gdi.plotfn**, **gdi.lineplot**, and **gdi.pointplot** functions only)	'thickness':2
'title'	sets the title (a string) for the plot (**gdi.plotfn** function only)	'title': 'Graph of sin(x)'
'titlecolour'	sets the colour (a string, see Chapter 15.1.3) of the title (**gdi.plotfn** only)	'titlecolour': 'red'
'titlesize'	sets the font size (a positive number) of the title (**gdi.plotfn** function only)	'titlesize':15
'x'	horizontal range (left and right border) over which the plot is displayed	'x':(-2):2
'y'	vertical range (lower and upper border) over which the plot is displayed	'y':0:5
'xscale'	sets the step size for the tick marks on the horizontal axis	'xscale':0.5
'yscale'	sets the step size for the tick marks on the vertical axis	'yscale':0.5

The function is written in Agena and included in the lib/gdi.agn file.

See also: **gdi.setoptions**.

gdi.plot (p [, options])

Plots PLOT structures stored in `p`. PLOT structures are points, lines, circles, triangles, rectangles, arcs, and ellipses, along with the information given by its optional INFO structure.

A PLOT structure is created by a call to **gdi.structure**, and the respective **gdi.set*** functions.

The function accepts all plot options (see **gdi.options**).

Example:

```
> p := gdi.structure();
```

```
> gdi.setline(p, 0, 0, 1, 1, 'navy');

> gdi.setcircle(p, 0, 0, 1, 'red');

> gdi.plot(p);

> gdi.plot(p, axes='normal', square=true, x=-2:2, y=-2:2);
```

The function is written in Agena and included in the lib/gdi.agn file.

gdi.plotfn (f, a, b [[c, d], options])
gdi.plotfn (ft, a, b [[c, d], options])

Plots graphs of one or more functions, with a straight line drawn between neighbouring points, which are automatically computed.

In the first form, the graph of the function `f` is plotted.

In the second form, by passing a table `ft` of functions, the graphs of the functions are plotted on one device - to one file or window.

If the `file` option is missing, the graphs are plotted in a window (UNIX/Mac and Windows, only). If the `file` option is given, the file type is determined by the suffix of the file you pass to this option.

`a` and `b` (both numbers with `a` < `b`) must be given explicitly and specify the horizontal range. If `c` and `d` are missing, the vertical range is determined automatically.

You may specify one or more options for proper layout of the graphs. See **gdi.options** for more details.

If a table of function is passed, you may specify an individual colour, line style, and the thickness for each of their graphs. Just pass a table of settings at the right-hand side of the respective option. See the examples below.

See **gdi.autoflush** if you experience performance problems while plotting.

Examples:

Plot the graph of the sine function on the horizontal range `a` to `b`. The vertical range is computed automatically.

```
> import gdi;

> gdi.plotfn(<: x -> sin(x) :>, -10, 10);
```

Plot the graph of the sine function on the horizontal range `a` to `b` and the vertical range `c` to `d`.

```
> gdi.plotfn(<: x -> sin(x) :>, -10, 10, -2, 2);
```

Specify a colour other than black:

```
> gdi.plotfn(<: x -> sin(x) :>, -10, 10, colour='red');
```

Give a specific thickness for the line:

```
> gdi.plotfn(<: x -> sin(x) :>, -10, 10, thickness=3);
```

Combine the options - their order does not matter:

```
> gdi.plotfn(<: x -> sin(x) :>, -10, 10, thickness=3, colour='red');
```

Plot two and more functions:

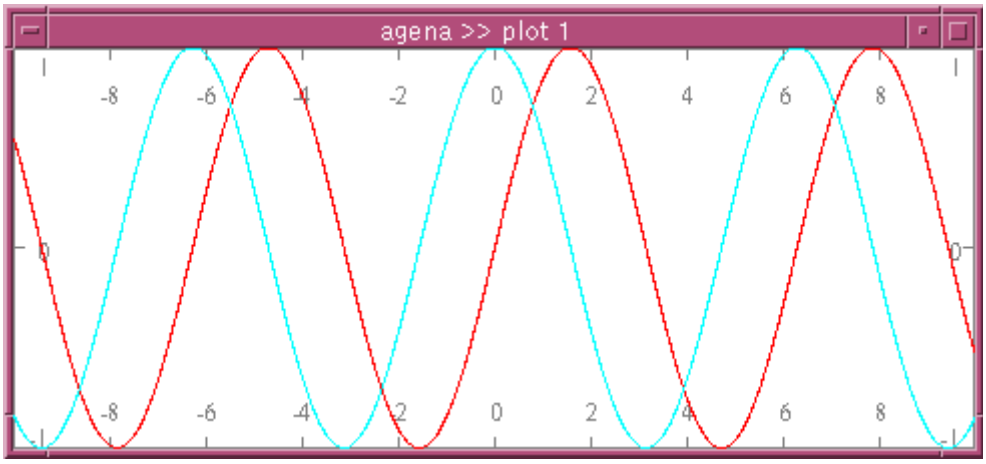
```
> gdi.plotfn([<: x -> sin(x) :>, <: x -> cos(x) :>], -10, 10);
```

Give options, too:

```
> gdi.plotfn([<: x -> sin(x) :>, <: x -> cos(x) :>], -10, 10,
>    colour='navy');
```

Specify individual colours. The graph of the sine function shall be red, the cosine function shall by cyan:

```
>
```




```
> gdi.plotfn([<: x -> sin(x) :>, <: x -> cos(x) :>], -10, 10,
>    colour=['red', 'cyan']);
```

Choose another colour for the axes and another axes style:

```
> gdi.plotfn([<: x -> sin(x) :>, <: x -> cos(x) :>], -10, 10,
>    colour=['red', 'cyan'], axescolour='grey', axes='boxed'
>    res=480:200);
```

Do not draw axes:

```
> gdi.plotfn([<: x -> sin(x) :>, <: x -> cos(x) :>], -10, 10,
>    colour=['red', 'cyan'], axes='none');
```

If you want to set default options that will always be used by **plotfn** and that do not need to be specified with each call to **plotfn**, use **gdi.setoptions**:

```
> gdi.setoptions(colour='red', axescolour='grey');

> gdi.plotfn([<: x -> sin(x) :>, <: x -> cos(x) :>], -10, 10)
```

The function is written in Agena and included in the lib/gdi.agn file.

See also: **calc.clampedspline**, **calc.nakspline**.

gdi.plotfn2d (f, a, b, c, d [, options])

Takes a function of two arguments and creates a plot in the 2-dimensional (!) plane. The function accepts the same arguments and options as **gdi.plotfn** does, but ignores line-styles.

Example:

```
> import gdi

> f := <: x, y -> cos(x * x + y * y) * exp(-0.1 * (x * x + y * y)) :>

> gdi.plotfn2d(f, -2, 2, -1, 1.2, colour='navy')

> sombrero := proc(x, y) is
>    local rho := hypot(x, y);
>    return if rho = 0 then 1 else 2.0*besselj(1, Pi*rho)/(Pi*rho) fi
> end;

> gdi.plotfn2d(sombrero, -2, 2, -0.2, 1.1, colour='maroon')
```

gdi.point (d, x, y [, colour])

Plots a point with co-ordinates [`x`, `y`] on device `d`. A `colour`, an integer (see Chapter 15.1.3 Colours, Part 1), may be given optionally.

gdi.pointplot (p [, options])
gdi.pointplot ([p1 [, p2, ···]], [, options])

Takes one or more tables or sequences consisting of points x_k:y_k and generates a plot with no points connected by lines. x_k and y_k must be finite numbers. The function automatically determines the common proper borders automatically.

By passing the option `colour`=c, where c is either a string denoting a colour, or a table of strings denoting colours, you can set individual colours for the distributions. The default is `'black'`.

By passing the option `symbol`=s, where s is the name of a symbol or a table of strings denoting symbols, each point in a distribution is plotted accordingly. Supported symbols are: `'cross'`, `'circle'`, `'circlefilled'`, `'box'`, `'boxfilled'`, `'triangle'`, `'trianglefilled'`, `'crosscircle'`, and `'dot'`. The default is `'dot'`.

The size of the symbols can be controlled by the `symbolsize` option which denotes a radius in pixels. Only one common size can be set for all distributions passed. The default is 3.

Alternatively, by passing the `connect`=**true** option, you can connect all points in each distribution with a line.

The function supports various plotting options, see **gdi.options**.

In the first form, only one distribution `p` is passed, in the second form you can pass various distributions `p1`, `p2`, etc. by putting them into a table.

The function ignores y-values if they evaluate to **infinity** or **undefined**.

Example:

```
> s := seq(0.1, 0.2, 0.1, 0.3, 1, 2, 5, -1, 0);

> p := sequences.new( <: x -> x:s[x] :>, 1, size s);

> s1 := <: x -> ln(x) :> @ s;

> p1 := sequences.new( <: x -> x:s1[x] :>, 1, size s1);

> gdi.pointplot([p, p1], colour=['red', 'black'],
>    symbol=['circle', 'cross'], symbolsize=5, connect=true);
```

The function is written in Agena and included in the lib/gdi.agn file.

See also: **gdi.lineplot**.

gdi.rectangle (d, x1, y1, x2, y2 [, colour])

Draws a rectangle with the lower left and upper right corners [`x1`, `y1`] and [`x2`, `y2`] on device `d`. A `colour` (an integer, see Chapter 15.1.3 Colours, Part 1), may be given optionally for the lines.

gdi.rectanglefilled (d, x1, y1, x2, y2 [, colour])

Draws a filled rectangle with the lower left and upper right corners [`x1`, `y1`] and [`x2`, `y2`] on device `d`. The rectangle is filled with either the default colour, or the one given by `colour` (an integer, see Chapter 15.1.3).

gdi.reset (d)

Clears the entire window or image file contents of device `d`.

gdi.resetpalette (d)

Clears the colour palette by removing all inks and reallocates basic colours, on device `d`.

gdi.setarc (s, x, y, r1, r2, a1, a2 [, colour [, thickness]])

Inserts an arc around the centre [`x`, `y`] with x radius `r1`, y radius `r2`, and the starting and ending angles `a1`, `a2`, given in degrees [0 .. 360], to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1. `thickness` is the thickness of the arc, with 1 its default.

gdi.setarcfilled (s, x, y, r1, r2, a1, a2 [, colour])

Inserts a filled arc around the centre [`x`, `y`] with x radius `r1`, y radius `r2`, and the starting and ending angles `a1`, `a2`, given in degrees [0 .. 360], to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1.

gdi.setcircle (s, x, y, r [, colour [, thickness]])

Inserts a circle around the centre [`x`, `y`] with radius `r`, to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1. `thickness` is the thickness of the circle, with 1 its default.

gdi.setcirclefilled (s, x, y, r [, colour])

Inserts a filled circle around the centre [`x`, `y`] with radius `r`, to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1.

gdi.setellipse (s, x, y, r1, r2 [, colour [, thickness]])

Inserts an ellipse around the centre [`x`, `y`] with x radius `r1`, and y radius `r2`, to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1. `thickness` is the thickness of the ellipse, with 1 its default.

gdi.setellipsefilled (s, x, y, r1, r2 [, colour])

Inserts a filled ellipse around the centre [`x`, `y`] with x radius `r1`, and y radius `r2`, to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1.

gdi.setinfo (s, ···)

Inserts information on the minimum and maximum values (x- and y values) and their scaling of all the geometric objects included in the PLOT data structure `s` into its INFO substructure. The INFO object always is the last element in `s`.

The options `xdim=a:b` and `ydim=c:d` set the x-range and y-range on which objects will be plotted, respectively, where `a`, `b`, `c`, `d` are numbers (i.e. borders). The `square = true` option scales the x and y dimensions equally, the `square = false` does not.

The information is useful so that **gdi.plot** can automatically determine the proper plotting ranges for `s`.

Example:

```
> gdi.setinfo(s, xdim = 0:10, ydim = -5:5, square = false);
```

gdi.setline (s, x1, y1, x2, y2 [, colour [, thickness]])

Inserts a line drawn from point (`x1`, `y1`) to point (`x2`, `y2`) with the optional `colour` into the PLOT structure `s`. `x1`, `y1`, `x2`, `y2` should be numbers. `colour` may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1. `thickness` is the thickness of the line, with 1 its default.

gdi.setoptions (···)

Checks the given plotting options (all key~value pairs) for correctness and sets them as the respective defaults for subsequent calls to the **gdi.plot** and **gdi.plotfn** functions.

For a list of valid plotting options, see **gdi.options**.

Internally, the function assigns the given options to the global environment variable **environ.gdidefaultoptions** which is checked by **gdi.plot** and **gdi.plotfn**.

gdi.setpoint (s, x, y [, colour])

Inserts a point with co-ordinates [`x`, `y`] to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1.

gdi.setrectangle (s, x1, y1, x2, y2 [, colour [, thickness]])

Inserts a rectangle with the lower left and upper right corners [`x1`, `y1`] and [`x2`, `y2`] to PLOT structure `s`. The optional `colour` argument may be either a string denoting a colour like `'black'`, `'red'`, etc., or a table with three RGB numeric values in the range 0 .. 1. `thickness` is the thickness of the arc, with 1 its default.

gdi.setrectanglefilled (s, x1, y1, x2, y2 [, colour])

Inserts a filled rectangle with the lower left and upper right corners [x1, y1] and [x2, y2] to PLOT structure s. The optional colour argument may be either a string denoting a colour like 'black', 'red', etc., or a table with three RGB numeric values in the range 0 .. 1.

gdi.settriangle (s, x1, y1, x2, y2, x3, y3 [, colour [, thickness]])

Inserts a triangle with the corners [x1, y1], [x2, y2], and [x3, y3] to PLOT structure s. The optional colour argument may be either a string denoting a colour like 'black', 'red', etc., or a table with three RGB numeric values in the range 0 .. 1. thickness is the thickness of the arc, with 1 its default.

gdi.settrianglefilled (s, x1, y1, x2, y2, x3, y3 [, colour])

Inserts a filled triangle with the corners [x1, y1], [x2, y2], and [x3, y3] to PLOT structure s. The optional colour argument may be either a string denoting a colour like 'black', 'red', etc., or a table with three RGB numeric values in the range 0 .. 1.

gdi.structure ([n])

Creates a PLOT data structure with n pre-allocated entries. Of course, the structure may contain less or more entries. If n is not given, no pre-allocation is done which may slow down inserting new objects into s later in a session. The return is the PLOT data structure, that is a sequence of user type 'PLOT'.

See also: **gdi.setinfo**.

gdi.system (d, x, y, xs, ys)

Sets the user's co-ordinate system on device d, where x, y, xs, and ys are numbers. The pixel [x, y] determines the origin. The horizontal unit is given in xs pixels, the vertical unit in ys pixels. The function returns nothing.

```
> d := open(640, 480);

> gdi.system(d, 320, 240, 320, 240);

> gdi.line(d, -1, 0, 1, 0);

> gdi.line(d, 0, -1, 0, 1);
```

gdi.text (d, x, y, str [, colour])

Prints the string str at [x, y] on device d. A text colour (an integer), may be given optionally.

See also: **gdi.fontsize**.

gdi.thickness (d, t)

Sets the default thickness for all lines to `t` pixels, on device `d`.

gdi.triangle (d, x1, y1, x2, y2, x3, y3 [, colour [, thickness]])

Draws a triangle with the corners [`x1`, `y1`], [`x2`, `y2`], and [`x3`, `y3`] on device `d`. A `colour` (an integer, see Chapter 15.1.3 Colours, Part 1), may be given optionally for the lines. `thickness` is the thickness of the triangle, with 1 its default.

gdi.trianglefilled (d, x1, y1, x2, y2, x3, y3 [, colour])

Draws a filled triangle with the corners [`x1`, `y1`], [`x2`, `y2`], and [`x3`, `y3`] on device `d`. The triangle is filled with either the default colour, or the one given by `colour` (an integer, see Chapter 15.1.3 Colours, Part 1).

gdi.useink (d, c)

Sets the default colour `c` (a number, see Chapter 15.1.3 Colours, Part 1) for all subsequent drawings, on device `d`. `c` must be a number determined by **gdi.ink**.

15.2 fractals - Library to Create Fractals

As a *plus* package, in Solaris, Linux, Mac OS X, and Windows, this library is not part of the standard distribution and must be activated with the **import** statement, e.g. `import fractals`.

Since it needs **gdi** graphics functions, it is of no use in DOS. The OS/2 - ArcaOS version is still experimental.

The library creates fractals and includes two types of functions:

1. escape-time iteration functions like **fractals.mandel**,
2. procedure **fractals.draw** to draw fractals using escape-time iteration functions.

See Chapter 15.2.3 for some examples. For one-liners that plot the fractals on screen check the package file `lib/fractals.agn` in your Agena installation folder.

15.2.1 Escape-time Iteration Functions

fractals.amarkmandel (x, y, iter, radius)

This function computes the escape-time fractal created by Mark Peterson of the formula:

$$z := z^2 * c^{0.1} + c$$

It returns the number of iterations a point [`x`, `y`] needs to escape `radius`. The maximum number of iterations conducted is given by `iter`.

See also: **fractals.markmandel**.

fractals.albea (x, y, iter, radius)

This function calculates the Julia set of the formula lambda * bea(z), where lambda is the point 1!0.4 and z =`x`!`y`, and `iter` is the maximum number of iterations. Its return is the number of iterations the function needs to escape `radius`. The function is written in Agena (see lib/fractals.agn).

See also: **fractals.lbea**.

fractals.alcos (x, y, iter, radius)

This function calculates the Julia set of the formula lambda * cos(z), where lambda is the point 1!0.4 and z =`x`!`y`, and `iter` is the maximum number of iterations. Its return is the number of iterations the function needs to escape `radius`. The function is written in Agena (see lib/fractals.agn).

fractals.alcosxx (x, y, iter, radius)

This function calculates the Julia set of the formula lambda * cosxx(z), where lambda is the point 1!0.4 and z =x!y, and `iter` is the maximum number of iterations. Its return is the number of iterations the function needs to escape `radius`. The function is written in Agena (see lib/fractals.agn).

The function implements FRACTINT's buggy cos function till v16, and creates beautiful fractals.

fractals.alsin (x, y, iter, radius)

This function calculates the Julia set of the formula lambda * sin(z), where lambda is the point 1!0.4 and z =x!y, and `iter` is the maximum number of iterations. Its return is the number of iterations the function needs to escape `radius`. The function is written in Agena (see lib/fractals.agn).

fractals.anewton (x, y, iter, radius)

This function implements Newton's formula for finding the roots of $z^3 - 1$, with z = x!y, and returns the number of iterations it takes for an orbit to be captured by a root. The iteration formula itself is

$$z := z - (z^3-1)/(3*z^2)$$

The function stops if $|z^3-1| <$ `radius` or the maximum number of iterations `iter` is reached. The function is written in Agena (see lib/fractals.agn).

See also: **fractals.newton**.

fractals.dragon (p)

Creates the dragon curve fractal, `p`, the recursion depth, should be a maximum of around 20. The function returns the gdi PLOT data for subsequent visualization.

fractals.esctime (f, x, y, l1, l2, iter [, radius])

Computes an escape-time fractal given by procedure `f`. `x` and `y` represent the initial point (`x`, `y`) with `x` and `y` two numbers. The numbers `l1` and `l2` represent the real and complex part of the lambda parameter. `iter` is the number of iterations to be done before bailing out. `radius` by default is 2. The return is the number of iterations it took for the point (`x`, `y`) to move outside `radius`.

Example for the Mandelbrot set, just set `l1` to `x` and `l2` to `y`:

```
> fractals.esctime(<: (z, c) -> z^2 + c :>, -1, 0.5, -1, 0.5, 128, 2):
4
```

Example for the Julia Cantor set, with `l1`, `l2` different from `x`, `y`:

```
> fractals.esctime(<: (z, c) -> z^2 + c :>,
  -1, 0.5, -0.481, 0.532, 128, 2):
128
```

fractals.fern (iters)

The function generates a realistic, organic fern structure (similar to the famous Barnsley Fern) using an Iterated Function System (IFS) driven by a random mutation process. It initializes a coordinate system at (0,0) and loops through a user-defined number of iterations (`iter`). The function returns the gdi PLOT data for subsequent visualization.

See also: **fractals.ifs.**

fractals.henon (iters, a, b)

The function generates a 2D Hénon attractor (Henon map) by iteratively calculating points based on input parameters `a` and `b`, both numbers, over a specified number of iterations `iters`. The function returns the gdi PLOT data for subsequent visualization.

fractals.ifs (iter, params, ps)

The function is a generalized Iterated Function System (IFS) engine designed to compute and plot self-similar fractals - such as the Barnsley Fern or Fractal Trees - using dynamic, randomized affine transformations.

The function automatically detects the number of geometric rules via the size of the cumulative probability table `p`.

For each iteration up to the specified `iter` count, that is the total number of coordinate point iterations to calculate, a random number picker chooses a transformation index. (Higher values yield maximum fractal density.)

The engine then applies a vectorized matrix calculation using six core coefficient arrays (a through f) unpacked from the master `params` table.

The function returns the gdi PLOT data for subsequent visualization.

Barnsley fern:

```
> import fractals;

> a := // 0 0.197 -0.155 0.849 \\;
> b := // 0 -0.226 0.283, 0.037 \\;
> c := // 0 0.226 0.26 -0.0377 \\;
> d := // 0.16 0.197 0.237 0.849 \\;
> e := // 0 0 0 0 \\;
> f := // 0 1.6 0.44 1.6 \\;
```

```
> p := // 0.03 0.14 0.27 1.0 \\;

> gdi.plot(fractals.ifs(100k, [a, b, c, d, e, f], p),
>    bgcolour='darkcyan', colour='yellow', axes='none', res=640:640);
```

6-transformation Pascal perfectly-balanced Symmetric Fractal Tree:

```
> import fractals;

> a := // 0.05, 0.05, 0.46, 0.46, 0.43, 0.43 \\;
> b := // 0.00, 0.00, -0.32, 0.32, -0.28, 0.28 \\;
> c := // 0.00, 0.00, 0.39, -0.39, 0.25, -0.25 \\;
> d := // 0.40, 0.40, 0.38, 0.38, 0.45, 0.45 \\;
> e := // 0.00, 0.00, 0.00, 0.00, 0.00, 0.00 \\;
> f := // 0.00, 1.00, 0.60, 0.60, 1.00, 1.00 \\;

> p := // 0.02, 0.04, 0.36, 0.68, 0.84, 1.00 \\;

> gdi.plot(fractals.ifs(100k, [a, b, c, d, e, f], p),
>    bgcolour='darkcyan', colour='yellow', axes='none', res=640:640);
```

See also: **fractals.fern**.

fractals.juliacantor (x, y, iter, radius)

Computes the escape time (iteration count) of a given coordinate z =x!y under the quadratic map z -> z^2 - s, where s is the fixed complex parameter 0.765 - 0.12i. This is used to visualize a Julia set that forms a disconnected Cantor set (Cantor dust). iter is the maximum number of iterations. The function returns the number of iterations the function needs to escape radius.

The call is equivalent to "fractals.mandel(x, y, iter, radius, -0.765!0.12)".

See also: **fractals.mandel** with 5 or 6 arguments.

fractals.juliapetal (x, y, iter, radius)

Same as **fractals.juliacantor**, but with s set 0.481-0.532i. This is used to visualize a Julia set that forms a Petal structure. The call is equivalent to "fractals.mandel(x, y, iter, radius, -0.481!0.532)".

fractals.koch (x_start, y_start, x_end, y_end, max_depth)

The function generates a Koch curve by recursively breaking down a line segment into smaller self-similar segments up to a specified max_depth.

x_start, y_start are the 2D coordinates of the starting point of the baseline, and x_end, y_end are the 2D coordinates of the ending point of the baseline which is the initial, straight line segment that serves as the foundation for the entire fractal.

The function returns the gdi PLOT data for subsequent visualization.

fractals.lbea (x, y, iter, radius)

This function calculates the Julia set of the formula lambda * bea(z), where lambda is the point 1!0.4 and z =x!y, and `iter` is the maximum number of iteration. Its return is the number of iterations the function needs to escape `radius`. The call is equivalent to fractals.mandel(x, y, iter, radius, -0.481!0.532).

See also: **fractals.albea**.

fractals.logisticmap (r_min, r_max, r_steps, transient, collect)

Creates a Logistic bifurcation diagram generator.

`r_min` is the growth rate start value, `r_max` the growth rate end value, `r_steps` is the horizontal/sampling resolution, `transient` denotes iterations to discard, `collect` determines the points to collect per r value.

The function returns the gdi PLOT data for subsequent visualization.

fractals.mandel (x, y, iter, radius [, s])
fractals.mandel (x, y, iter, radius [, s, t])

By default the function computes the Mandelbrot set of the formula

$$z := z^2 + c$$

using complex arithmetic and with c initially set to x+y*I. It returns the number of iterations a point [`x`, `y`] needs to escape `radius`. The maximum number of iterations conducted is given by `iter`.

If the complex number `s` or the numbers `s`, `t` are given, then the function computes a Julia set with c set to either `s` or `s+t`*I.

The function uses Cardioid/Bulb Checking to speed up computation.

fractals.mandelbrot (x, y, iter, radius)

Like **fractals.mandel**, but written in Agena and using complex arithmetic. The function uses Cardioid/Bulb Checking to speed up computation, too.

fractals.mandelbrotfast (x, y, iter, radius)

Like **fractals.mandel**, but written in Agena and using real arithmetic.

fractals.mandelbrottrig (x, y, iter, radius)

Like **fractals.mandel**, but written in Agena and using real arithmetic and trigonometric functions (see lib/fractals.agn).

fractals.markmandel (x, y, iter, radius)

Like **fractals.amarkmandel**, but implemented in C.

fractals.minkowski (x_start, y_start, side_length, max_depth)

The function generates a Minkowski Island fractal by constructing a closed, square-shaped island layout and deforming each of its four edges. It calculates the corner coordinates of a square based on a starting origin (`x_start`, `y_start`) and a given `side_length`. `max_depth` represents the recursion level that controls the geometric complexity and density of the square-wave patterns.

The function returns the gdi PLOT data for subsequent visualization.

fractals.newton (x, y, iter, radius)

Like **fractals.anewton**, but implemented in C.

15.2.2 The Drawing Function fractals.draw

The function takes an escape-time iterator, various other parameters, and creates either image files or windows of fractals. By default a window is opened (see file option on how to create image files).

fractals.draw (iterator, x_centre, y_centre, x_width [, options])

Draws a fractal given by the escape-time iterator function `iterator` with image centre [`x_centre`, `y_centre`] and of the total length on the x-axis `x_width`. `x_centre` and `y_centre` are numbers whereas `x_width` is a positive number.

Options are:

Option	Meaning	Example
colour ~ f	a colouring function `f` of the form f := <: x -> r, g, b :>. Predefined functions are: red, blue, violet, cyan, cyannew.	colour ~ <: x -> 0, 0, 0.05*x :> colour ~ blue
file~'filename.suf'	creates a GIF, PNG, or JPEG file, if the file suffix is .gif, .png, or .jpg	file ~ 'mandel.gif'
iter ~ n	maximum number of iterations with `n` a positive number; default is 128	iter ~ 512
lambda ~ p	lambda value `p`, a complex number, for fractals.[a]l* functions like **albea**	lambda ~ 1!0.4
map ~ 'filename.map'	FRACTINT colour map to be used to draw the fractal. Additional FRACTINT maps can be downloaded from: http://agena.sourceforge.net/	map ~ 'basic.map'

Option	Meaning	Example
	downloads.html#fractintmaps Put these files into the share folder of your Agena distribution, preserving the subfolder fractint. A valid path may thus be: /usr/agena/share/fractint. Alternatively, set the environment variable environ.fractintcolourmaps to the folder where your map files reside. The map file may include comments introduced with a hash or semicolon. Blank lines will be skipped.	
map ~ tbl	Alternatively, the map option accepts a table of exactly 256 RGB value triples, where each triple is in its own subtable. See also **fractals.genmap** below.	map~ [[0, 0, 255],# Blue [0, 255, 0],# Green [255, 0, 0] # Red ... etc ...]
mouse ~ bool	display pointer co-ordinates on console after image has been finished, if `bool` = **true**. Default: `bool` = **false**. Click the right mouse button to quit printing co-ordinates.	mouse ~ true
radius ~ r	iteration radius `r`, a positive number	radius ~ 2
res ~ width:height	resolution of the window or image, with `width` and `height` positive numbers. Default is 640:480	res ~ 1024:768
update ~ n	with `n` a non-negative number: determines the number of rows after an image is being flushed to a file or window during computation	

Notes on the **update** option:

On all operating systems the default is 1. This behaviour can globally be changed in a session by assigning a non-negative integer to the environment variable **environ.fractscreenupdates**.

In Sun x86 Solaris and Linux, update ~ 0 is the fastest, but when outputting to a window, it does not plot anything while the fractal is being computed (of course, if computation finishes, the fractal will be displayed).

Sparcs do not show any effect when changing the update rate, at least with XVR-1200 VGAs. The same applies to Microsoft Windows XP and 7, as well as Mac OS X 10.5.

fractals.genmap (anchors [, filename])

Creates a colour map of 256 RGB values from the table `anchor` including colour stop RGB values.

In general, each RGB value is a subtable of the red, green and blue colour components, all integers, in the range [0, 255].

The function uses Linear Color Interpolation to create the colour map. If `filename`, a string, is given, then the function writes the colour map to a FRACTINT-compliant map file of that name instead of returning the colour map table.

Example:

```
> anchors := [
>    [0, 0, 255],  # Blue
>    [0, 255, 0],  # Green
>    [255, 0, 0]   # Red
> ]

> fractals.draw(
>   fractals.markmandel,
>   -0.5, 0, 1.5, res = 1280:768,
>   map = fractals.genmap(anchors));
```

If you use colours defined in the gdi.COLOURS table, then multiply each component by 255 to transform it from the 0 .. 1 to the 0 .. 255 range.

```
> anchors := [
>    gdi.COLOURS.bordeaux, gdi.COLOURS.spring, gdi.COLOURS.spring,
>    gdi.COLOURS.yellow, gdi.COLOURS.yellow,
>    gdi.COLOURS.white, gdi.COLOURS.white, gdi.COLOURS.white,
> ]

> for i, j in anchors do
>    anchors[i] := map(<: x -> x*255 :>, j);
> od;

> fractals.genmap(anchors, 'rgb.map');
Successfully generated file `rgb.map`.
```

fractals.sortmap (colmap [, options])

Sorts a colour map represented by table `colmap` non-destructively and either returns the new sorted table or writes it to a file.

The options are:

- `mode` = 'hue': sort by hue (rainbow order).
- `mode` = 'saturation' (or 'sat' for short): sort by saturation, that is vividness: rich colours to greys.
- `file` = *filename*: Write the sorted colour map to file *filename*, a string.

- `header` = *text*, to be used with the `output` option: Write header *text*, a string, to the first line in *filename*. If not given, the resulting file will have no header. *text* should start with a hash or semicolon.

If the `mode` option is not given, then step-sorting (group by colour type, then by brightness) will be done.

Example:

```
> fractals.sortmap(
>   fractals.genmap(anchors),
>   mode   = 'sat',
>   file   = 'mymap.map',
>   header = '# my first saturation-sorted colour map');
```

fractals.readmap (filename)

Reads the FRACTINT colour map file referred to by `filename` and returns a table of RGB values.

In the table returned, each RGB value is a subtable of red, green and blue colour components, all integers, in the range [0, 255].

The map file may include comments starting with a hash or semicolon, and blank lines, which will all be ignored during import.

fractals.writemap (colmap, filename [, header = str])

Writes a colour map represented by table `colmap` non-destructively to file named `filename`.

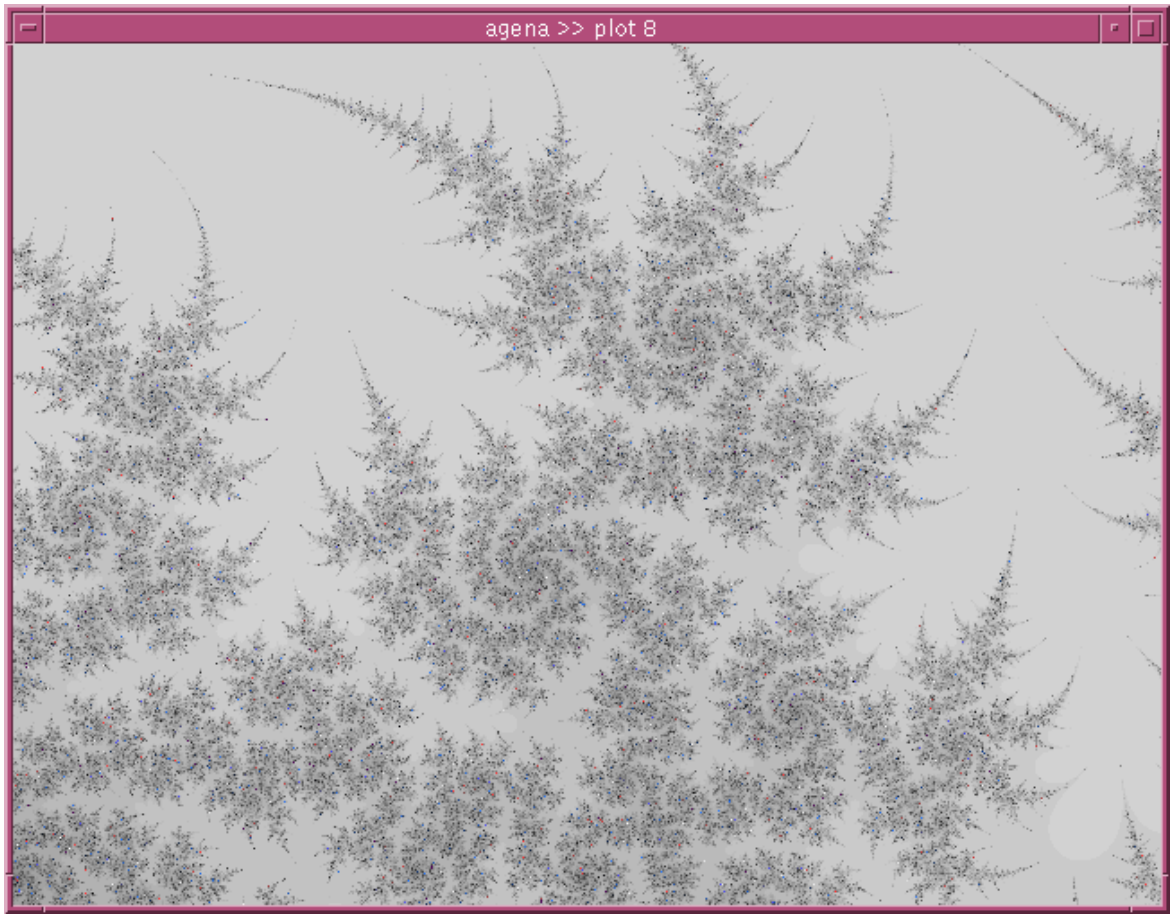
The file generated my include a header with the text defined by string `str`, which should be non-empty and start with either a hash or semicolon - otherwise the function will issue an error.

The function returns nothing.

15.2.3 Examples

```
> import fractals alias

> draw(fractals.lbea, 1.75, 0.5, 0.001, map='grayish.map', radius=1024,
>    iter=1024, lambda=1!0.4);
```



There are further examples at the bottom of the lib/fractals.agn file residing in the main Agena library folder.

```
> draw(mandel, -1.0037855135, 0.2770816775, 0.086686273, iter=255);

> draw(mandel, -1.0037855135, 0.2770816775, 0.086686273, file='out.png',
> iter=255, res=1024:768);  # create a PNG file of the Mandelbrot set
```

15.3 curses- Library to Create Terminal Applications

As a *plus* package, this library is not part of the standard distribution and must be activated with the **import** statement, that is `import curses`.

15.3.1 Introduction and Example

curses is a C library for Agena that wraps the curses API. It is a 1:1 port of the lcurses binding for Lua 5.1 written by Reuben Thomas & Tiago Dionizio.

The following description has been taken entirely from the lcurses documentation. The two minor changes for Agena have been properly marked.

curses allows to create good-looking terminal applications, along with colours, menus, etc. It is the interface between Agena and your system's ncurses or curses library. For descriptions on the usage of a given function or constant, consult your system's documentation, starting with man ncurses or man curses.

Here is an example on its use. Either paste the whole code at the prompt are put it into a script file and run it:

```
import curses;

begin
   stdscr := curses.initscr();
   curses.cbreak();
   curses.echo(false);
   curses.nl(false);
   stdscr@@keypad(true); # activate arrow keys
   stdscr@@cleanse();
   # read table
   a := [];
   for keys k in curses do
      a[size a + 1] := k;
   end;
   stdscr@@mvaddstr(15, 20, "print out curses table (y/n) ? ");
   stdscr@@refresh();
   # wait for keytroke
   n := stdscr@@getch();
   # to print list, press 'y' (ASCII 121)
   if n = 121 then
      stdscr@@cleanse();
      stdscr@@mvaddstr(1, 1, "Available curses functions:");
      for i := 1 to size a do
         if i > 20 then break; end;
         stdscr@@mvaddstr(2 + i, 1, a[i])
      end;
      stdscr@@mvaddstr(24, 1, "Press any key to finish.");
      stdscr@@refresh();
      stdscr@@getch()
   end
   curses.endwin();
end;
```

15.3.2 curses Functions

This list of functions are defined in the static const luaL_reg curseslib section of the curses.c source file:

baudrate	beep	cbreak	color_pair
color_pairs	colors	cols	curs_set
delay_output	doupdate	echo	endwin
erasechar	flash	flushinp	halfdelay
has_colors	has_ic	has_il	init_pair
initscr	isendwin	keyname	killchar
lines	longname	napms	new_chstr
newpad	newwin	nl	pair_content
raw	ripoffline	slk_attroff	slk_attron
slk_attrset	slk_clear	slk_init	slk_label
slk_noutrefresh	slk_refresh	slk_restore	slk_set
slk_touch	start_color	stdscr	termattrs
termname	unctrl	ungetch	

15.3.3 Window Functions

addch	addchstr	addstr	attroff
attron	attrset	border	box
cleanse*)	clearok	clone	close
clrtobot	clrtoeol	copywin	cursyncup
delch	deleteln	derive	echoch
erase	getbegyx	getbkgd	getch
getmaxyx	getparyx	getstr	getyx
hline	idcok	idlok	immedok
insertln	intrflush	is_linetouched	is_wintouched
keypad	leaveok	meta	move
move_derived	move_window	mvaddch	mvaddchstr
mvaddstr	mvdelch	mvgetch	mvgetstr
mvhline	mvvline	mvwinch	mvwinchnstr
mvwinnstr	mvwinsch	mvwinsnstr	mvwinsstr
nodelay	notimeout	noutrefresh	overlay
overwrite	pechochar	pnoutrefresh	prefresh
redrawln	redrawwin	refresh	scrl
scrollok	standend	standout	sub
subpad	syncdown	syncok	syncup
timeout	touch	touchline	vline
wbkgd	wbkgdset	winch	winchnstr
winnstr	winsch	winsdelln	winsnstr
winsstr	wsetscrreg		

*) The function is 'cleanse' instead of 'clear' for the latter is a keyword in Agena.

15.3.4 Constants

These constants only appear after **curses.initscr** is called; they can be seen by:

```
> import curses;

> a := [];

> for keys k in curses do
>    if curses[k] :: number then
>       a[size a + 1] := k
>    end
> end;

> sort(a);

> for k in a do
>    print(k)
> end;
```

and are defined in the static void register_curses_constants section of curses.c

ACS_BLOCK ACS_BOARD ACS_BTEE ACS_BULLET ACS_CKBOARD ACS_DARROW ACS_DEGREE ACS_DIAMOND ACS_HLINE ACS_LANTERN ACS_LARROW ACS_LLCORNER ACS_LRCORNER ACS_LTEE ACS_PLMINUS ACS_PLUS ACS_RARROW ACS_RTEE ACS_S1 ACS_S9 ACS_TTEE ACS_UARROW ACS_ULCORNER ACS_URCORNER ACS_VLINE

A_ALTCHARSET A_ATTRIBUTES A_BLINK A_BOLD A_CHARTEXT A_DIM A_INVIS A_NORMAL A_PROTECT A_REVERSE A_STANDOUT A_UNDERLINE

COLOR_BLACK COLOR_BLUE COLOR_CYAN COLOR_GREEN COLOR_MAGENTA COLOR_RED COLOR_WHITE COLOR_YELLOW

KEY_A1 KEY_A3 KEY_B2 KEY_BACKSPACE KEY_BEG KEY_BREAK KEY_BTAB KEY_C1 KEY_C3 KEY_CANCEL KEY_CATAB KEY_CLEANSE (instead of KEY_CLEAR !) KEY_CLOSE KEY_COMMAND KEY_COPY KEY_CREATE KEY_CTAB KEY_DC KEY_DL KEY_DOWN KEY_EIC KEY_END KEY_ENTER KEY_EOL KEY_EOS KEY_EXIT KEY_F0 KEY_F1 KEY_F10 KEY_F11 KEY_F12 KEY_F2 KEY_F3 KEY_F4 KEY_F5 KEY_F6 KEY_F7 KEY_F8 KEY_F9 KEY_FIND KEY_HELP KEY_HOME KEY_IC KEY_IL KEY_LEFT KEY_LL KEY_MARK KEY_MESSAGE KEY_MOUSE KEY_MOVE KEY_NEXT KEY_NPAGE KEY_OPEN KEY_OPTIONS KEY_PPAGE KEY_PREVIOUS KEY_PRINT KEY_REDO KEY_REFERENCE KEY_REFRESH KEY_REPLACE KEY_RESET KEY_RESIZE KEY_RESTART KEY_RESUME KEY_RIGHT KEY_SAVE KEY_SBEG KEY_SCANCEL KEY_SCOMMAND KEY_SCOPY KEY_SCREATE KEY_SDC KEY_SDL KEY_SELECT KEY_SEND KEY_SEOL KEY_SEXIT KEY_SF KEY_SFIND KEY_SHELP KEY_SHOME KEY_SIC KEY_SLEFT KEY_SMESSAGE KEY_SMOVE KEY_SNEXT KEY_SOPTIONS KEY_SPREVIOUS KEY_SPRINT KEY_SR KEY_SREDO KEY_SREPLACE KEY_SRESET KEY_SRIGHT KEY_SRSUME KEY_SSAVE KEY_SSUSPEND KEY_STAB KEY_SUNDO KEY_SUSPEND KEY_UNDO KEY_UP

15.3.5 Compatibility

In the C library, the following functions:

- getstr() (and wgetstr(), mvgetstr(), and mvwgetstr())
- inchstr() (and winchstr(), mvinchstr(), and mvwinchstr())
- instr() (and winstr(), mvinstr(), and mvwinstr())

are subject to buffer overflow attack. This is because you pass in the buffer to be filled in, which has to be of finite length. But in this Agena package, a buffer is assigned automatically and the function returns the string, so there is no security issue. You may still use the alternate functions:

```
s := stdscr@@getnstr()
s := stdscr@@inchnstr()
s := stdscr@@innstr()
```

which take an extra "size of buffer" argument, in order to impose a maximum length on the string the user may type in.

Some of the C functions beginning with "no" do not exist in Agena. You should use curses.nl(false) and curses.nl(true) instead of nonl() and nl(), and likewise curses.echo(false) and curses.echo(true) instead of noecho() and echo() .

In this module the stdscr@@getch() function always returns an integer. In C, a single character is an integer, but in Agena (and Lua, Perl) a single character is a short string. The Perl Curses function getch() returns a char if it was a char, and a number if it was a constant; to get this behaviour in Agena you have to convert explicitly, e.g.:

```
if c < 256 then c := char(c) end
```

Some Agena functions take a different set of parameters than their C counterparts; for example, you should use str := stdscr.getstr() and y, x := stdscr.getyx() instead of getstr(str) and getyx(y, x), and likewise for getbegyx and getmaxyx and getparyx and pair_content. The Perl Curses module now uses the C-compatible parameters, so be aware of this difference when translating code from Perl into Agena, as well as from C into Agena.

Many curses functions have variants starting with the prefixes w-, mv-, and/or wmv-. These variants differ only in the explicit addition of a window, or by the addition of two coordinates that are used to move the cursor first. For example, addch() has three other variants: waddch(), mvaddch(), and mvwaddch(). The Perl Curses module combines these variants into one "Unified Function" which detects how many arguments it has and behaves accordingly. But with this package you must always specify the window (because it is the Object) e.g.: stdscr@@addch(), and so the mv- variant is retained. If you are translating Perl code into Agena you may wish to use some wrapper functions.

Chapter Sixteen

Utilities

16 Utilities

16.1 utils - Utilities

The **utils** package provides miscellaneous functions.

utils.calendar ([x])

Converts `x` seconds (an integer) elapsed since the beginning of an epoch to a table representing the respective calendar date in your local time. The table contains the following keys with the corresponding values:

- 'year' (integer)
- 'month' (integer)
- 'day' (integer)
- 'hour' (integer)
- 'min' (integer)
- 'sec' (integer)
- 'wday' (integer, day of the week)
- 'yday' (integer, day of the year)
- 'DST' (Boolean, is Daylight Saving Time)

If `x` is **null** or not specified, then the current system time will be returned. If `x` is invalid, the function will issue **fail**.

See also: **os.now**.

utils.decodea85 (str)

Decodes the ASCII85 encoded string `str` and returns the result as a string.

See also: **utils.encodea85**.

utils.decodeb32 (str [, option])

Decodes the Base32 encoded string `str` and returns the result as a string.

If `option` is set to **true**, then the function determines which characters in `str` might be invalid. The default is **false**.

See also: **utils.encodeb32**, **utils.encodeb85**.

utils.decodeb64 (str)

Decodes the Base64 encoded string `str` and returns the result as a string.

See also: **utils.encodeb32**, **utils.encodeb64**.

utils.decodeb85 (str [, option])

Decodes the Base85 encoded string `str` and returns the result as a string. The size of `str` must be a multiple of 5. If `option` is set to **true**, then the function determines which characters in `str` might be invalid. The default is **false**.

See also: **utils.decodez85**, **utils.encodeb32**, **utils.encodeb85**.

utils.decodez85 (str [, option])

Decodes the Base85 Z85 (ZeroMQ) encoded string `str` and returns the result as a string. The size of `str` must be a multiple of 5. If `option` is set to **true**, then the function determines which characters in `str` might be invalid. The default is **false**.

See also: **utils.decodeb85**, **utils.encodeb32**, **utils.encodeb85**.

utils.decodexml (str [, options])

Reads a string `str` containing an XML stream and converts it into a dictionary.

You can pass one or two options in any order:

If the Boolean option **false** is given, the function does not automatically try to convert strings representing numbers, complex numbers and the Booleans **true**, **false**, and **fail** into the proper Agena representation.

If the option `'nocomment'` is given, the function does not return XML comments.

The function provides some checking (basic syntax and balanced tags), and supports namespaces, XML and DOCTYPE declarations, comments and processing instructions. If a XML tag includes hyphens or colons, then they are converted to underscores in the corresponding Agena dictionary key.

Since the function does not return processing instructions, you may want to have a look at the auxiliary utils.aux.decoderawxml function included in the `lib/libary.agn` file which returns a user-defined table containing processing instructions in the `xarg` tag.

The function is written in Agena and included in the `lib/libary.agn` file.

Here is an example:

```
> xmlstr := '<?xml version="1.0"?>
> <Data>
>    <Name1>Agena</Name1>
>    <Name2>1</Name2>
>    <Name3>1.1</Name3>
>    <Name4>1.1+2.2*I</Name4>
> </Data>
```

```
> <Lang:Info-All>
>    <Name action="interpret">Agena</Name>
>    <Version>1.6.1</Version>
> </Lang:Info-All>
> <!-- this is a comment -->
> <Motto>The Power of Procedural Programming</Motto>'

> utils.decodexml(xmlstr):
[Data ~ [Name1 ~ Agena, Name2 ~ 1, Name3 ~ 1.1, Name4 ~ 1.1+2.2*I],
Lang_Info_All ~ [Name ~ Agena, Version ~ 1.6.1], Motto ~ The Power of
Procedural Programming, header ~ <?xml version="1.0"?>]

> for i, j in ans do print(i, j) od
Lang_Info_All   [Name ~ Agena, Version ~ 1.6.1]
Motto   The Power of Procedural Programming
Data    [Name1 ~ Agena, Name2 ~ 1, Name3 ~ 1.1, Name4 ~ 1.1+2.2*I]
header  <?xml version="1.0"?>
```

The function is quite slow when parsing large or deeply nested XML structures, but sometimes is deemed more exact than **xml.decodexml**. If you need to parse only certain portions of an XML stream, just extract them from the string using the **strings.match** function before applying **utils.decodexml**.

See also: **utils.encodexml**, **utils.readxml**.

utils.encodea85 (str)

Encodes a string `str` into ASCII85 format and returns it as a string.

See also: **utils.decodea85**.

utils.encodeb32 (str)

(3.10.3 and later) Encodes a string `str` into Base32 format and returns it as a string.

See also: **utils.decodeb32**.

utils.encodeb64 (str)

Encodes a string `str` into Base64 format and returns it as a string.

See also: **utils.decodeb64**.

utils.encodeb85 (str)

Encodes the string `str` into Base85 format and returns it as a string. The size of `str` should be a multiple of four.

See also: **utils.encodez85**, **utils.decodeb85**.

utils.encodez85 (str)

Encodes the string `str` into Base85 Z85 (ZeroMQ) format and returns it as a string. The size of `str` should be a multiple of four.

See also: **utils.decodeb85**, **utils.encodeb85**.

utils.encodexml (obj [, indent [, flag]])

Encodes a dictionary `obj` of the same format as created by **utils.readxml** into XML format.

If `indent` (a non-negative number) is not given the number of white space indentations is 3.

If any value is given for `flag`, the return is a flat table of substrings, else the return is one concatenated string.

See also: **utils.decodexml**.

utils.findfiles (d, what [, options])

utils.findfiles (obj, what [, options])

Searches a single file - or searches a directory for all the files - that include a certain string or which satisfy a given condition.

In the first form, the directory to be searched is denoted by the first argument `d`, a string, which may include file wildcards. `d` may also denote a single file. In the second form, `obj` is a table of a table with file names of type string, and the absolute path to the directory containing the given files. (**os.list** returns such a table.)

The second argument `what` can either be a string to be searched for, or a procedure of one argument that describes a satisfying condition and which should result in either **true** or **false**.

The returns are two lists: the first list includes all the names of the files where the search has been successful, and the second lists includes all files that could not be read due to errors, for example because of missing read permissions.

By default, the function searches all files line by line for a given search criterion. Pass the option `'whole'` if the search criterion should be applied to the entire file, i.e. to search in the string concatenation of all the lines of a file, so that line breaks do not matter.

By passing the further option `'r'`, the function also searches recursively in all respective subfolders.

Options may be given in any order after the second argument what.

Examples:

```
> utils.findfiles('*.c', '#define'):

> utils.findfiles('*.c', <: x -> '#define' in x = 1 :>, 'whole'):

> utils.findfiles([['a.txt', 'b.txt'], 'c:/text'], 'hello'):
```

See also: **io.infile**, **io.readfile**.

utils.hexlify (str)

Converts a string `str` to its hexadecimal representation and returns a new string where each character in `str` is replaced by a two-digit hexadecimal value. The resulting string is twice as long as `str`.

See also: **utils.unhexlify**.

utils.ilog2 (x)

With non-negative integer `x` returns **entier**(**log2**(`x`)) for `x` > 1 and `x` otherwise. The result thus is suited primarily for an estimate of the required memory space to be reserved in advance for new tables, sequences, registers, numarrays, etc.

See also: **utils.newsize**.

utils.isdate (obj)

utils.isdate (year, month, day [, hour [, minute [, second]]])

In the first form, receives a date of the form year, month, date [, hour [, minute [, second]]], with these values in table or sequence `obj` being integers, and checks whether the given date and optionally time exists and returns **true** or **false**.

In the second form, receives the given integers, and conducts the same operation.

See also: **astro.isvaliddate**, **astro.isvalidtime**.

utils.multidim (dims)

utils.multidim (idx, dims [, 0])

The function returns the multi-dimensional indices for a given one-dimensional index.

In the first form, the function generates a factory that each time it is called with a one-dimensional index returns the corresponding multi-dimensional indices. To represent an array a with two rows and three columns with indices starting from 1, issue:

```
> f := utils.multidim([2, 3]):
procedure(01E06548)

> f(4):
[2, 1]
```

The factory variant is the fastest way to compute the indices.

In the second form, the one-dimensional index is represented by `idx`, and `dims` is a table, sequence or register with one or more dimensions. In the next example we again have a two-dimensional array with two rows and three columns:

```
> utils.multidim(4, [2, 3]):
[2, 1]
```

By default, the indices start from 1. You can change this to zero by passing the third optional argument 0.

See also: **utils.onedim**.

utils.newsize (x)

Returns three suggestions for the optimum size of a structure if at least `x` slots are needed.

The Agena equivalent is:

```
newsize := <: x -> (x + (x >> 3) + 6) && ~~3 :>
x := 0
while x < 1000 do
   y := newsize(x);
   print(x, y, utils.newsize(x))
   x := y
od;
```

In general, the first result is around 113 percent of `x` (median), rounded up to the next multiple of four. Its growth pattern is: 0, 4, 8, 12, 16, 24, 32, 40, 48, 60, 72, 84, 100, 116, 136, etc. The return is always greater than `x`, even if $x = 0$.

The second result is the requested number of slots `x`, rounded up to the next power of two if `x` is not already a power of two, so with powers of two `x` will be returned and otherwise the result is always greater than `x`, even if $x = 0$.

The third result is `x` rounded up to the next multiple of four if `x` is not already a multiple of four. With $x = 0$, the return is 0.

See also: **math.nextmultiple**, **math.nextpower**, **utils.ilog2**.

utils.numiters (a, b [, step])

Returns the number of iterations in the interval [a, b], with a <= b and with an optional positive step size, which is one by default. The result is equal to:

int(|b - a|/step) + 1.

utils.onedim (dims, i [, ···], [, 0])

utils.onedim (inds, dims [, 0])

utils.onedim (dims)

In the first form, computes the one-dimensional index equivalent for multidimensional indices, i, ··· of any structure.

Pass the dimensions of the structure in table dims, followed by one or more indices of interest, see examples below.

By default, the function assumes that all indices start from one. You may append a final zero to the argument list to indicate that all the indices start from zero instead of one.

In the second form, the indices are given in table, sequence or register inds and the dimensions in table, sequence or register dims and the function returns the one-dimensional index. You may append a final zero to the argument list to indicate that all the indices start from zero instead of one.

In the third form, the function generates a factory that each time it is called with the coordinates, which can be passed in a table or as individual arguments, returns the corresponding one-dimensional index. To represent a two-dimensional array a with two rows and three columns with indices starting from 1, issue:

```
> f := utils.onedim([2, 3]):
procedure(01E06818)
```

Get a[2, 2]:

```
> f(2, 2):
5

> f([2, 2]):
5
```

The factory variant is the fastest way to compute a one-dimensional index.

Background: Imagine a two dimensional table array A with two rows and three columns, with all the indices starting from one:

```
A| 1 2 3
-|------
1| 1 2 3
 |
2| 4 5 6
```

This array has dimensions 2, 3.

You can internally represent this two-dimensional array as a one-dimensional one, B, with dimension 6:

```
B| 1 2 3 4 5 6
-|------------
1| 1 2 3 4 5 6
```

utils.onedim easily allows to convert array indices into just one. So, for example, A[2, 1] and B[4] are identical:

```
> utils.onedim([2, 3], 2, 1):
4
```

or

```
> utils.onedim([2, 1], [2, 3]):
4
```

If the indices start from zero, we have:

```
A| 0 1 2       B| 0 1 2 3 4 5
-|------       -|------------
0| 0 1 2       0| 0 1 2 3 4 5
 |
1| 3 4 5
```

The dimensions, of course, still are the same.

```
> utils.onedim([2, 3], 1, 0, 0):
3

> utils.onedim([1, 0], [2, 3], 0):
3
```

See also: **utils.multidim**.

utils.posrelat (pos, len)

If `pos` represents a negative integer index, denoting the |`pos`|-th position from the right side of a structure, returns the respective positive index for the given number `len` of items in the structure, otherwise returns `pos`. Returns zero if `pos` is zero. The function is written in the Agena language and included in the lib/library.file.

utils.readcsv (filename [, options [, fn]])

Reads a comma-separated value (CSV) file and returns its contents in a sequence. The delimiter of the fields in a line by default is a semicolon.

If a line contains more than one field, then the respective fields are returned in a sequence[10]. If a line contains only one field, then it will be returned without including it in a sequence[11]. If a line contains nothing, i.e. '\n', it is by default ignored[12].

Strings containing numbers are automatically converted to numbers.

Options can be passed as pairs:

Left pair element	Right pair element	Example
convert	**true** or **false**: If **false**, do not attempt to convert strings to numbers. Default: **true**.	`convert = true`
comma	**true** or **false**: If a field contains a string recognised as a number by **strings.iscenumeric** - i.e. with a decimal comma instead of a decimal dot - this option automatically transforms the value to an Agena number if the option evaluates to **true**. Default is **false**. This option is applied before checking for the `convert` option.	`comma = true`
delim	A string. Use this string as the delimiter instead of a semicolon which is the default.	`delim = '|'`
dictionary	Returns a dictionary instead of a sequence with the dictionary keys defined by the values in the row passed with this new option, where the row can be depicted by a field number or field id (a string). The values in the `key` row should be unique.	`dictionary = 1` `dictionary = 'ID'`
field	a positive integer: If given, only the given field in the CSV file is extracted, else all fields are returned.	`field = 3`
fields	A table or sequence of positive integers. If given, only the fields	`fields = [3, 1, 5]`

[10] See the `flat` option to override this behaviour.

[11] See the `newseq` option to override this behaviour.

[12] See the `skipemptylines` option to override this behaviour.

Left pair element	Right pair element	Example

	given in this table or sequence are returned, and in the order of the elements in this table or sequence; if not given, all fields are returned.
	
	If a CSV file contains a header, then column numbers or strings denoting the field name can be passed, and column numbers and field names can be mixed.
		`fields =`
		`  ['name', 'phone']`
		
		`fields =`
		`  ['name', 2]`

flat	**true** or **false**: If **true**, do not return values in each line in a new sequence. Default: **false**.	`flat = true`
header	**true** or **false**: If **true**, ignore the very first line. Default: **false**.	`header = true`

ignore	a string or a procedure returning either **true**, **false**, or **fail**. If given a string, each line starting with this string will be skipped, proceeding with the next line. The string may include patterns. If given a procedure, it will be applied to each line of the CSV file and if it evaluates to **true**, it skips the line and proceeds with the next one.	`ignore = 'text'`
		
		`ignore =`
		`  <: x ->`
		`  'text' in x`
		`  <> null`
		`  :>`

ignorespaces	**true** or **false**: all spaces in a line are deleted before returning the fields. Default is **false**.	`ignorespaces = true`

mapfields	A table or sequence of pairs of the form posint:procedure. Applies the given function to a specific field in the CSV file.
	
	If a CSV file contains a header, then column numbers or strings denoting the field name can be passed along with the procedures, and column numbers and field names can be mixed.
		`mapfields =`
		`  [1:f, 3:g]`
		
		`mapfields =`
		`  ['name':f, 2:g]`

newseq	**true** or **false**: if only one field, i.e. one value per line, is stored in the CSV file, always put this single value in each line into a new sequence (**true**), resulting in a sequence of sequences returned by **readcsv**; otherwise simply add it to the flat	`newseq = true`

Left pair element	Right pair element	Example
	sequence returned by the function, which is the default (**false**).	
output	A string. If the right-hand side is `'record'`, then a dictionary will be returned, with its keys being defined by the tokens in the first line of the file (if the header=**true** option is also given), otherwise a table array will be returned.	`output = 'record'`
remove	`'quotes'` or `'doublequotes'`, or both. If `'quotes'` is given, enclosing single quotes are removed from the CSV field. If `'doublequotes'` is given, enclosing double quotes are removed from the CSV field (the default, see removedoublequotes option to prevent this). You cannot remove *both* single *and* double quotes. If a field in single or double quotes includes the field delimiter, then the quotes will not be removed.	`remove = 'quotes'`
remove-doublequotes	If set to **true**, removes enclosing double quotes from a field if present (the default). If set to **false**, enclosing double quotes are not deleted. If a field in double quotes includes the field delimiter, then the quotes will not be removed.	`removedoublequotes = false`
skipemptylines	**true** or **false**: If **true**, do not return empty lines. Default is **true**.	`skipemptylines = true`
skipfaulty skipfaultylines	**true** or **false**; if set to **true**, ignores all lines in a CSV file that do not have the same number of fields as there are in the CSV header, or the first CSV line if a header is non-existent. Default is **false**. If set to **true**, the function also returns all skipped lines as a second result.	`skipfaulty = true`
skipspaces	**true** or **false**: If **true**, do not return lines consisting of spaces only. Default is **false**.	`skipspaces = true`

Left pair element	Right pair element	Example

subs	a pair, or a table or sequence of pairs x:y. For each line read from the CSV file, replaces x with y. If you pass a function as the last argument, substitution is done before finally mapping this function on the return.	`subs = '':undefined`
		`subs = ['':undefined,`
		`'HUGE_VAL':infinity]`

You may also optionally pass a function `fn` - at any position in the argument list - to be mapped on each value of the input to be returned, or mix options given as pairs and a function to be applied to each value to be returned, e.g.:

```
> L := utils.readcsv('data.dat', delim=' ', flat=true, <: x -> x^2 :>);
```

The function is written in Agena and included in the `libary.agn` file.

See also: **columns**, **descend**, **io.lines**, **io.readlines**, **utils.readxml**, **utils.writecsv**, **skycrane.readcsv**, **strings.fields**, **strings.unwrap**.

utils.readini (filename [, options])

Reads a traditional initialisation file and returns its contents as a table. Initialisation files supported look like the following:

```
#
# This is an example of an ini file
#
; Pizza general
Taxi=Pizza Cab
State=

; Following is a section ...
[Pizza]
; ... and now the associated key~value pairs
Ham = yes
Mushrooms = true
Capres = 0
Cheese = "Non" ;
Gravy=false
Price = 3.99
Comment= This \
is a \
multiline string.
Preis=3,99
empty =
```

A line beginning with a hash (#), followed optionally by one or more characters, is completely ignored as are comments initiated at any position with a semicolon.

Key~value pairs are separated by an equals token surrounded by no, one or more white spaces.

The result is a table.

The file is parsed from top to bottom. As long as no section name has been given (here `[Pizza]`), any key~value pairs encountered are entered into the table as such.

If a section name and associated key~value pairs are given, then a subtable of the form sectionname ~ [key ~ value pairs] is stored to the resulting main table.

If a key with no value is given, then the value will be the empty string. Values may also be enclosed in double quotes, but double quotes will be stripped off during import.

By default, any numeric values are automatically converted to numbers, and the strings `'true'`, `'false'`, or `'fail'` are converted to Booleans. All other values are returned as strings. You may prevent any conversion by passing the `convert=false` option.

If the option `comma=true` is given, then all strings representing floating point values containing a decimal comma are converted to a representation with a decimal dot. Default is `comma=false`.

The results of reading the above ini file will look as follows if no option is given:

```
[pizza ~ [capres ~ 0, cheese ~ Non, comment ~ This is a multiline string.,
 empty ~ , gravy ~ false, ham ~ yes, mushrooms ~ true, preis ~ 3,99,
 price ~ 3.99], state ~ , taxi ~ Pizza Cab]
```

As you see, all capital letters in section and key names are converted to lower case.

See also: **ini.dump**, **utils.writeini**.

utils.readxml (filename [, options])

Reads an XML file and returns its data in an Agena dictionary.

You can pass one or two options in any order:

If the Boolean option **false** is given, the function does not automatically try to convert strings representing numbers, complex numbers and the Booleans **true**, **false**, and **fail** into the proper Agena representation.

If the option `'classic'` is given, the function calls **utils.decodexml** to parse the XML file, otherwise the function calls **xml.readxml** which is the more versatile, faster default. If the option `'nocomment'` is given along with `'classic'`, the function does not return XML comments. In no-classic mode comments will always be skipped.

See also: **utils.decodexml**, **utils.readcsv**, **xml.readxml**.

utils.rfc3339 (str [, option])

Reads RFC 3339-compliant timestamps and features some extensions to allow for abridged timestamps. It can either return a table of the various components if only `str` is given or a Lotus Serial Date of `option` has been set to **true**:

```
> utils.rfc3339("2000-01-01T00:00:00.5Z"):
[day ~ 1, gmtoff ~ 0, hour ~ 0, min ~ 0, month ~ 1, msec ~ 500, sec ~ 0,
 year ~ 2000]

> utils.rfc3339("2014-11-12T19:12:14.125-06:30"):
[day ~ 12, gmtoff ~ -23400, hour ~ 19, min ~ 12, month ~ 11, msec ~ 125,
 sec ~ 14, year ~ 2014]

> utils.rfc3339("1980-01-01t01:01:60+01:00"):

> utils.rfc3339("2000-01-01T00:00:00"):  # abridged stamp

> utils.rfc3339("2000-01-01T"):          # abridged stamp

> utils.rfc3339("2000-01-01"):           # abridged stamp

> utils.rfc3339("2000-01-01", true):     # LSD
36526
```

See also: **utils.timestamp**.

utils.singlesubs (str, sp)

Substitutes individual characters in string `str` by corresponding replacements in sequence or register `sp` which should hold ASCII values, that is integers, only. If `sp` is a table, then both integer ASCII values and strings consisting of zero, one or more characters are accepted, but the keys must always be non-negative integers.

The return is a new string. Note that the function tries to find a replacement for a single character in `str` by determining its integer ASCII value n and then accessing index n in `sp`. If an entry is found for index n, then the character is replaced, otherwise the character remains unchanged.

```
> utils.singlesubs('Düsseldorf', [abs('ü') ~ 'ue', abs('Ü') ~ 'Ue']):
Duesseldorf
```

For another example, check the **strings.diamap** procedure in the lib/library.agn file.

See also: **strings.replace**.

utils.speed (n, f [, ···])

Receives a positive integer `n`, a function `f`, and any optional arguments, and executes the function `n` times. The function returns the execution time in seconds.

If you want to check the speed of an operator, you have to enclose it in a function, e.g.:

```
> utils.speed(1k, <: x -> sin x :>, 0):
```

See also: **time**.

utils.timestamp (ts [, options])

Transforms the timestamp string `ts` of the format `'DD.MM.YYYY HH:MM:SS'` or `'YYYY.MM.DD HH:MM:SS'` into a numeric Lotus Serial Date (LSD, a number) and - optionally - the time in DMS notation (a number, see **math.dms** for details).

`ts` does not need to include the number of minutes, hours and seconds. In this case, the missing digits are replaces by zeros. In `ts`, year and day may be swapped, and the month, day, hour, minute and second may be single integers, thus not necessarily preceded by a zero. Seconds can include fractional milliseconds.

If the option `splitup`=**true** is being passed, the function also returns the LSD as a string with a comma as the decimal separator, plus six integers depicting year, month, day, hour, minute and second. See third example below.

The function by default returns the time in Daylight Saving Time if active. You can switch this off (always returning Standard Time) by passing the option `standardtime`=**true**.

The delimiter in `ts` which separates year, month, day, a dot by default, can be changed to another delimiter by passing the `datedelim`=<any character> option, see example below.

The delimiter in `ts` which separates hour, minute and second, a colon by default, can be changed to another delimiter by passing the `timedelim`=<any character> option.

The delimiter in `ts` which separates date and time, a space by default, can be changed to another delimiter by passing the `datetimedelim`=<any character> option.

Optional delimiters may be preceded by a backslash so that the function can parse `ts` successfully.

```
> utils.timestamp('31.12.2017 23:59:01', datedelim='\.'):
43100.99931713  23.5901

> utils.timestamp('01/02/2017 23:59:01', datedelim='\/'):
42767.99931713  23.5901

> utils.timestamp('31-12-2017', datedelim='-'):
43100   0
```

```
> utils.timestamp('2017.2.1 23:59:01.999',
>    datedelim='\.', splitup=true):
42767.99931713  23.5901  42767,99931713  2017   2   1   23   59   1.999
```

The function is written in Agena and included in the lib/library.agn file.

See also: **math.dms**, **os.lsd**, **skycrane.tocomma**, **skycrane.todate**, **strings.strptime**, **utils.rfc3339**.

utils.udata (str [, obj])

Creates an empty userdata structure with an attached metatable for garbage collection (`'__gc'` metamethod), pretty-printing (`'__tostring'` metamethod), type checking (`'__oftype'` metamethod) and the user-defined type `str`, a string. `str` will be used by the pretty-printer.

This data structure is being used in the **json** package to represent JSON's null value. Example:

```
> nil := utils.udata('null')

> type nil, typeof nil:
userdata        null

> nil :: 'null':
true
```

The userdata object returned features *no* accompanying environment table by default. If you need one, pass a table as a second argument or call **environ.setfenv** subsequently. You can read from or write any values to this environment table via standard indexing:

```
> u := utils.udata('any');

> environ.setfenv(u, []);

> u[1], u[2] := 10, 20;

> u[1], u[2]:
10      20
```

utils.udata returns a reference to the environment when given the userdata object as the sole argument:

```
> r := utils.udata(u):
[10, 20]
```

You can read and write values the common way, for example:

```
> insert 30, 40, 50 into r

> utils.udata(u):
[10, 20, 30, 40, 50]
```

Alternatively, you can call the userdata object like a function.

With no argument given, a reference to the accompanying table will be returned, the same way **utils.udata** does:

```
> u():
[10, 20, 30, 40, 50]
```

With one argument, the value for the given key can be retrieved:

```
> u(1):
10
```

With two arguments, the first denoting a key and the second a value, a write operation will be executed:

```
> u(6, 60):

> u():
[10, 20, 30, 40, 50, 60]
```

utils.udata includes a provision to support OOP-style calls. Imagine you have a userdata u of type `'any'`:

```
> u := utils.udata('any');
```

Let's insert some numbers into u's accompanying table first:

```
> u[1], u[2] := 10, 20;
```

Let's define a package function that works with userdata of type `'any'` - the package table name must thus have the same name `'any'`. Here we set up a function `any.include` that adds data to the accompanying table:

```
> any := [
>    include ~ proc(ud :: any, ?) is
>       for i in ? do
>          insert i into ud()  # ud() references to the internal table
>       od
>    end
> ];
```

Call it OOP-style:

```
> u@@include(30, 40)

> u():
> [10, 20, 30, 40]´
```

Voila !

If you pass a second argument `obj`, you can register an alternative data structure as the environment of the userdata value returned: If you pass any set, register,

sequence, pair, table, userdata, it will be used, including its content if it has one. If you pass **null**, there will be no such alternative data container. Let's take a set with some numbers in the following example:

```
> u := utils.udata('any', {10, 20, 30})

> u():
{10, 20, 30}
```

If you register a pair, you may use the **left** and **right** operators as follows:

```
> u := utils.udata('any', 1:2)

> left u, right u:
1       2
```

utils.unhexlify (str)

Does the opposite of **utils.hexlify**.

utils.uuid ([x])

Creates a random version 4 universally unique identifier (UUID) by exclusively producing random numbers, and returns a string of 32 characters. If its argument `x` is **null**, the nil UUID will be returned (i.e. the template), otherwise, `x` has no effect.

See also: **environ.ref**, **factory.count**, **math.random**, **sema.open**.

utils.writecsv (obj, filename [, options])

utils.writecsv (fh, obj [, options])

In the first form, creates a comma-separated value (CSV) file. The function writes all values or keys and value(s) of a table, set, sequence or register `obj` to a text file given by `filename`. If `obj` includes a structure, then each element of the respective structure is written on the same line. Otherwise, each value or key ~ value pair is written on a separate line.

By default only values are written, the keys are ignored, but check the 'key' option below.

In the second form, writes all the data in `obj` to the file denoted by filehandle `fh`, in one line, followed by a terminating newline. Example:

```
> fh := io.open('uszip.csv', 'w');

> utils.writecsv(fh, ['Zip', 'City', 'State Id', 'State'], enclose='\"');

> io.close(fh);
```

The following options can be passed as pairs:

Left pair element	Right pair element	Example
delim	A string. Use this string as the delimiter instead of a semicolon which is the default.	`delim = '|'`
dot	A single character of type string. With numbers, a decimal dot is replaces with the given character. Default: no replacement.	`dot = ','`
enclose	A string. Each value to be written is enclosed with this string.	`enclose = '\"'`
header	A string written to the very first line. Default: no header is written.	`header = 'A;B;C'`
key	A Boolean. If **true**, writes the respective index of the structure at the beginning of each line. Default: **false**, i.e. indices are not written.	`key = true`

The function returns nothing, is written in Agena and included in the lib/library.agn file.

Example:

```
> obj := seq(seq(1.1, 2, 3), seq(4, 5.1, 6), seq(7, 8, 9));

> utils.writecsv(obj, 'c:/out.csv', delim='|', dot=',');
```

creating a file with the contents:

```
1|1,1|2|3
2|4|5,1|6
3|7|8|9
```

See also: **utils.readcsv**, **skycrane.readcsv**.

utils.writeini (obj, filename [, options])

Creates a traditional initialisation file with name `filename` and writes a dictionary `obj` of key~value pairs to it. If values are not tables, they are written at the beginning of the file. If values are tables of key~value pairs, then they are written to the corresponding sections.

By default, the function writes the entries and sections in ascending order. You may change the order of the sections and the specific sections to be written by passing a table array of section names with the sections option, e.g. `sections=['Salad', 'Pizza']` first writes all entries of the Salad section, and then the Pizza section is written.

An optional spacer in front and behind the equals signs may be given by passing the spacer option which accepts any string, e.g. `spacer='\t'`. Default is the empty string.

A floating point value may be written with a decimal comma instead of a decimal dot by passing the `comma=true` option, default is `comma=false`.

The function returns nothing, is written in Agena and included in the lib/library.agn file.

See also: **utils.readini**.

`utils.writexml (obj, filename [, indent])`

Creates an XML file with name `filename` from the dictionary `obj` which should be of the same format as the dictionary returned by **utils.decodexml**.

The function returns nothing, is written in Agena and included in the lib/library.agn file.

See also: **utils.decodexml**, **utils.encodexml**, **utils.readxml**.

16.2 skycrane & telex - Auxiliary Functions

As a *plus* package, the **skycrane** and telex packages are not part of the standard distribution and must be activated with the **import** statement, e.g. `import skycrane` and `import telex`.

The packages contain functions that you might or might not find usefully.

skycrane.bagtable (o)

Creates a table of empty bags with its keys determined by the values in the sequence `o`. `o` may include values of any type. If `o` is empty, an error will be issued.

The function automatically loads the **bags** package if it has not yet been initialised.

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **bags.new**.

skycrane.dice ()

Returns random integers in the range [1 .. 6].

See also: **math.random**, **math.randomseed**.

skycrane.fcopy (a, b [, verbose])

This function is an interface to **os.fcopy** but can also deal with directories. If `a` and `b` are file names, then the function works like **os.fcopy**. If `b` is a directory, then `a` is copied into it. If `a` is a directory, then all files in it are copied into `b`.

If `verbose` is true then the name of the file copied successfully is printed at stdout.

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **os.fcopy**, **skycrane.move**.

skycrane.formatline (l [, ···])

Similar to **io.writeline**, but all the strings, numbers or Booleans to be formatted must be passed in a table, sequence or register `l`. The function returns a string instead of writing the values to a file. The function accepts the following options:

- `delim` = *string*
  the delimiter, a semicolon by default, can also be the empty string;
- `enclose` = *string*
  each value will be enclosed by *string*, double quotes by default, can also be the empty string.

Example:

```
> skycrane.formatline([1, 'agena', true], delim = '|', enclose = '\''):
'1'|'agena'|'true'
```

skycrane.getlocales ([lang])

When given no argument, returns all locales available on your operating system. The return is a table with the keys being valid arguments to **os.setlocale**, and the entries the result of the respective call to **os.setlocale**.

The function is very slow when called the first time in a session in this mode as it internally calls **os.setlocale** to check for valid locales. In UNIX, it would be better to issue the command 'locale -a' in a shell to determine the locales supported on your system.

When given a string as the single argument `lang`, then the function returns the full name of the language and country for a combination of the ISO 639 language code and the ISO 3166 region code. Examples:

```
> skycrane.getlocales('he'):
Hebrew

> skycrane.getlocales('he_IL'):
Hebrew (Israel)
```

If `lang` is any Boolean, then the function determines whether the locales included in an internal mapping list (see file lib/skycrane.lib) are supported by the operating system, and returns the supported ones in a table.

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **os.setlocale**.

skycrane.isemail (str [, strict])

Checks whether a string `str` represents a valid E-mail address and returns **true** or **false**. The algorithm used does not cover all rules defined in RFC 3696, but should suffice with standard E-mail addresses of syntax "local-part@domain".

Note that the domain may not necessarily include a dot. You can override this rule by passing any non-**null** value for `strict`.

skycrane.iterate (o)

Returns an iterator function traversing a table, set, sequence or register `o` always in strict ascending order.

If o is a table, the function first sorts its keys and returns a function which if called, returns the table values of o in the ascending order of these sorted keys.

If o is a set, the function first sorts its entries and returns a function that if called, returns the elements one by one in ascending sorted order.

Although unnecessary: if o is a sequence or register, the function returns a function that if called, returns each value in o one by one in their original order.

The function is written in Agena and included in the lib/skycrane.agn file. For the order how keys or values will be sorted, see **sorted**.

A note: This function is utterly slow compared to the **for**/**in** statement. But there may be few situations demanding loops iterating in the strict ascending order of its (numeric or string) indices, or set, register, and sequence values.

See also: **ipairs**, **nextone**, **sorted**, **factory.iterate**, **factory.count**.

skycrane.move (a, b [, verbose])

This function is an interface to **os.move** but can also deal with directories. If a and b are file names, then the function works like **os.move**. If b is a directory, then a is moved into it. If a is a directory, then all files in it are moved into b.

The function is written in Agena and included in the lib/skycrane.agn file.

If verbose is true then the file copied successfully moved is printed at stdout.

See also: **os.move**, **skycrane.fcopy**.

skycrane.obcount (obj)

Takes a table, sequence or register obj and counts the number of occurrences of each of its values. The return is a table with the key the respective value in obj and the value the associated count. Example:

```
> import skycrane
```

1 is included 4 times, 2 is included twice, 3 only once, etc.:

```
> skycrane.obcount([1, 1, 1, 2, 2, 1, 3, 8, 9, 8, 9]):
[1 ~ 4, 2 ~ 2, 3 ~ 1, 8 ~ 2, 9 ~ 2]
```

See also: **bags** package, **countitems**, **stats.obcount**.

skycrane.readcsv (filename [, ···])

Like **utils.readcsv**, but with the following default options, which can be overridden:

convert=false, ignorespaces=false, remove='doublequotes'.

The function is written in Agena and included in the lib/skycrane.agn file.

skycrane.replaceinfile (fn, oldstring, newstring [, true])

skycrane.replaceinfile (fn, subslist [, true])

In the first form, replaces all occurrences of string `oldstring` in file `fn` (a filename) in-place with string `newstring`. The function supports pattern matching.

In the second form, one or more substitutions can be given by passing a table or sequence `sublist` of pairs of the form `oldstring` : `newstring`.

By default, a backup of the file to be modified is created with the additional suffix `.bup`. You can suppress any backup by passing the Boolean value **true** as the very last argument.

The function is written in Agena and included in the lib/skycrane.agn file.

skycrane.scribe (fh, obj [, ···])

skycrane.scribe (obj [, ···])

skycrane.scribe (···)

Like **io.write** and **io.writeline**, but if a table, sequence or register `obj` is being passed, it writes the values in the structure to the file denoted by its handle `fh` (first form) or the console (second form) instead of throwing an exception. `fh` is a file handle, not a file name.

The values in the structure `obj` must either be numbers or strings.

The function accepts the following options of type pair:

- If the `delim` option (third to last argument) has been passed, all values are separated by the given string. Default is a semicolon. Examples: `delim='|'`: use a pipe instead of a semicolon, `delim=''` (i.e. the empty string): do not include a delimiter.

- If the `newline` or `nl` option has been passed, and if its value is **false**, then no newline is included after the elements have been written. (Include a trailing delimiter - if needed - by calling **io.write**.) Default is **true**. Example: `newline=false`.

If no structure has been passed (third form), the function just behaves like **io.write** or **io.writeline**.

Examples:

```
> import skycrane;

> skycrane.scribe('men ne cunnon hwyder helrunan hwyrftum scriþað'):
men ne cunnon hwyder helrunan hwyrftum scriþað

> fd := io.open('Depeche Mode','wb');

> skycrane.scribe(fd,
>    'Enjoy the silence,
>     words are very unnecessary,
>     they can only do harm.');

> io.close(fd);

> fd := io.open('c:/wulfila.txt', 'w');

> paternoster[13] := (/
>    atta unsar þu in himinam
>    weihnai namo þein
>    qimai þiudinassus þeins
>    wairþai wilja þeins
>    swe in himina jah ana airþai
>    hlaif unsarana þana sinteinan
>    gif uns himma daga \);

> skycrane.scribe(fd, paternoster, delim = ' ');

> io.close(fd);
'

> LeeMiller := io.open("Lee Miller); io.close(LeeMiller);
```

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **print**, **printf**, **io.write**, **io.writeline**, **skycrane.tee**.

skycrane.sorted (obj [, f])

Sorts a table, sequence or register `obj` non-destructively but contrary to **sort** and **sorted** can cope with structures including values of different types. First, numbers are sorted, then strings, the others are not. The function, however, is slower than **sorted**.

If `f` is given, then it must be a function that receives two structure elements, and returns **true** when the first is less than the second (so that not `f(obj[i+1], obj[i])` will be **true** after the sort). If `f` is not given, then the standard operator `<` (less than) is used instead.

[13] Taken from the Gothic Language Wulfila Bible edited by Wilhelm Streitberg.

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **rsorted**, **sort**, **sorted**, **stats.issorted**, **stats.sorted**.

skycrane.stopwatch ()

Implements a stopwatch. Just follow the instructions when calling skycrane.stopwatch(). The function returns nothing.

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **watch**.

skycrane.tee (fh, x [,···] [, 'delim':str])

skycrane.tee (fh, x [,···], 'format':str)

In the first form, the function writes one or more numbers or strings `x` to both the console (stdout), and a file denoted by its handle `fh` to the current working directory. (Use **io.open** to create a handle and **io.close** to finally close the file as **skycrane.tee** always leaves the file open.) By default, the values are separated with a tabulator (`\t`). It finally puts a line feed at the end of the output. By passing the option `'delim':str`, as the last argument, the delimiter is given by the string `str`.

In the second form, one or more numbers or strings `x` are written to both the console (stdout), and a file denoted by its handle `fh` to the current working directory. The resulting string is formatted according to the printf-like template information in `str` passed with the `format` option. See **strings.format** for more information on the template string. It does not put a line feed at the end of the output, but to do so, you may add a `\n` control character to the end of the format string.

The function returns nothing.

The function is written in Agena and included in the lib/skycrane.agn file.

See also: **print**, **printf**, **history.on**, **skycrane.scribe**.

skycrane.tocomma (x)

If `x` is a number, the function converts `x` to a string. If `x` is a float (a number with a fraction), the decimal dot is replaced by a comma. If `x` is a string and represents an integer or float, an optional decimal-dot is replaced by a comma.

The return is a string.

skycrane.todate (x)

Returns the calendar date and time represented by the number `x`, which should hold the number of seconds (and optionally milliseconds) elapsed since the start of the given epoch. The return is a string of the format `` `YYYY/MM/DD hh:mm:ss` ``.

If no argument is given, the current system date and time will be returned. You may pass an optional format string if you prefer another representation of the date and time.

See also: **strings.format**, **os.now**, **os.time**, **utils.timestamp**.

skycrane.tolerance (x, a)

Returns **math.branch**(**ceil**(a * **log10**(`x` + 1) - 1)), a maximum tolerance value especially suited for comparing similar strings where `x` may denote the size of a string. A good value for `a` might be a number greater than 3.

See also: **strings.dleven**, **strings.diffs**, **strings.dice**, **strings.fuzzy**, **strings.jaro**.

skycrane.trimpath (str)

Converts backslashes in the string `str` to slashes and then removes, if existing, one trailing slash, and returns the modified string. If `str` does not include backslashes or trailing slashes/backslashes, `str` will be returned unmodified.

skycrane.xmlmatch (str, tag [, tag_2, ···])

Like **strings.match**, but returns the contents of the string `str` enclosed by the given XML search tag '<`tag`>(.-)</`tag`>'. If further tags tag_2 .. tag_k are given, then the result of the previous search with tag_{k-1} is checked for tag_k. If `tag` is a number, it will be converted to a string before matching. If any tag is not a string or number, it will be ignored.

The function returns **null** if any tag could not be found.

Examples:

```
> data := '<data>
>             <name>abc</name>
>             <info>
>                <name>def</name>
>             </info>
>         </data>'

> skycrane.xmlmatch(data, 'name'):
abc

> skycrane.xmlmatch(data, 'info', 'name'):
def
```

telex.decode (str)

Takes the sequence of non-negative integers or the string `str` generated by **telex.encode** and converts it back to the original string.

telex.encode (str)

Takes a string of printable characters and converts it into a sequence of non-negative integers. The string is packed to around 70 percent of its size. The second result is the representation of the encoded string as a non-human readable string.

The function uses a modified version of the 5-bit International Telegraphy Alphabet No. 2 of 1924, also called Baudot-Murray code, to denote 8-bit characters in 5 bits. To work well with ASCII characters greater than ASCII code 127 in the Windows console, change the codepage to your local ANSI character set, e.g.:

```
> os.codepage(input = 1252, output = 1252);
```

It may be better to convert diacritics and umlauts according to your own rules, e.g. Norse `Þ` to `&Thorn;`, `<Th>` or `<T>`, the shorter their encoding, the better.

16.3 factory - Iterators

The package provides functional programming-style iterators. The functions are:

factory.count ([start [, step [, stop [, method]]]])

Returns an iterator function that, each time it is called, returns a new number.

If no argument is given, the first number returned by the iterator is 0, the next call returns 1, the next one 2, and so forth. This means that the number returned with each call is increased by 1.

If only `start` is given, the first number returned by the iterator is `start`, the next call returns `start` + 1, the next one `start` + 2, and so forth. This means that the number returned with each call is increased by 1.

If `start` and `step` are given, the first number returned by the iterator is `start`, the next call returns `start` + `step`, the next one start + 2*`step`, and so forth. This means that the number returned with each call is increased by `step`, which may be negative. In the latter case the next number returned will be less than the current returned number.

If `stop` is given, the iterator returns **null** if the counter value exceeds `stop`. Default is +**infinity**.

If `start` or `step` are not numbers, the factory issues an error.

If `start` or `step` is a non-integer, the function by default automatically applies Kahan-Babuška summation to avoid round-off errors. You can choose between the following summation `method`s:

`method`	algorithm
`'babuska'`	Kahan-Babuška summation, highest accuracy but slowest (default)
`'kbn'`	Kahan-Babuška-Neumaier compensated summation, used in the Julia programming language
`'neumaier'`	Neumaier summation, good accuracy and performance
`'ozawa'`	Kahan-Ozawa summation

The generator automatically adds **hEps** to the `stop` value to avoid the iterator from leaving prematurely. If the absolute value of the `step` size is less than or equal to **hEps**, the generator will issue an error. You can entirely switch off this feature by setting **math.Eps** to zero, but only by calling **environ.kernel**:

```
> environ.kernel(hEps = 0);
```

The current setting of **hEps** can be queried by:

```
> environ.kernel('hEps'):
0
```

Note that **hEps** also controls numeric **for** loops with fractional step sizes. You might want to reset its value after generating the iterator to its default.

Example:

```
> f := factory.count(1, -0.1, -1, 'ozawa');  # count down

> while c := f() do print(c) od;
```

See also: **factory.reset**, **math.accu**, **skycrane.iterate**, **utils.uuid**.

factory.cycle (obj [, firstkey [, sentinel]])

Like **factory.iterate**, but when `sentinel` is encountered during traversal or the end of the structure has been reached, it does not return **null** but simply restarts iteration with the first element in `obj`.

See also: **nextone**, **factory.reset**.

factory.iterate (obj [, firstkey [, sentinel]])

Creates a function that when called, starting with index `firstkey`, iterates each element in a table, sequence or register `obj`, or each character in a string `obj` one by one, returning the respective index and value as two results.

`firstkey` and `sentinel` by default are **null**. With tables, `firstkey` may be any value and should be **null** if the table is to be iterated from its beginning. With any other data type, `firstkey` is a positive integer. If `sentinel` is encountered during traversal, the iterator returns **null**.

If there is nothing left, the function returns **null** and if called again re-starts iteration.

With strings, sequences and registers, if `firstkey` is out of range, the iterator simply returns **null**. With tables, if `firstkey` is a non-existent key, an error will be issued.

See also: **nextone**, **factory.cycle**, **factory.reset**, **skycrane.iterate**.

factory.reset (f, index)

Sets the current index of the iterator `f` created by **factory.cycle**, **factory.iterate** or **factory.count** to `index`.

If you want to re-iterate an object from its beginning, pass **null** for `index` if you traverse tables, and number 1 with every other object.

The package also features the following more general functions:

factory.anyof (f, [, ···])

Creates a function that when called tries each function `f, ···` with the arguments passed to the generated function. **factory.anyof** also accepts structures that have a '__call' metamethod (see Chapter 6.19).

The generated function quits if one of the calls does not return **null**, **false** or **fail**. If a call results in no result at all, then the next function or structure with '__call' metamethod will be run. The return is either **null** if none of the calls was successful, or the full result of the successful call, that is including all returns.

The very first result of a call is taken to determine whether it was successful or not.

The functionality is more or less equal to:

```
> anyof := proc(?) is
>    local args := ?;
>    return proc(?) is
>       for fn in args do
>          if r := fn(unpack(?)) then
>             return r
>          fi
>       od
>    end
> end;
```

Example:

```
> isalphanumeric := factory.anyof(strings.isnumber, strings.islatin);

> isalphanumeric('1'):
true

> isalphanumeric('i'):
true

> isalphanumeric('ü'):
null
```

factory.curry (f, [a [, ···]])

Transforms a function with multiple arguments into a sequence of single-argument functions, i.e. `f`(`a`, b, c, ...) becomes `f`(`a`)(b)(c)... Depending on its usage, it can also create functions with partially filled arguments, ala `f`(`a`, b)(c), see example below.

Example usage:

```
> import factory alias curry;

> f := <: x, y, z -> x*y + z :>

> t := curry(f);  # returns f
```

```
> t(10, 20, 30):
230

> t := curry(f, 10);  # returns f(10, y, z)

> t(20, 30):
230

> u := curry(t, 20);  # returns t(10, 20)(z) = f(10, 20, z)

> u(30):
230
```

The return is a closure, a function with the argument(s) stored as so-called upvalues. If only `f` is given, returns `f`. See also Chapter 6.22 discussing closures.

Note that the flexibility of **curry** comes at the price of performance: a call to the resulting closure internally will issue one or more calls, finally to `f` itself, depending on the number of arguments in `f`. It may sometimes be better to define a dedicated wrapper function.

factory.pick (f, i [, ···])

The function picks only given results from a function call, by taking a function `f` and the positions `i`, `···` of the results to be returned and producing a function that when called delivers the results of interest. Imagine a function

```
> f := proc() is return 10, 11, 12, 13, end;
```

where we want to have only the first and third result of its call, that is numbers 10 and 12. We define

```
> g := factory.pick(f, 1, 3);

> g():
10      12
```

16.4 units - Physical Unit Conversion

The package provides functions to convert between temperatures, lengths, weights and volumes.

units.celsius (f)

Takes a number `f` in degrees Fahrenheit and converts it to degrees Celsius.

units.fahren (c)

Takes a number `c` in degrees Celsius and converts it to degrees Fahrenheit.

units.foot (x [, option])

Takes a value `x` in meters and converts it to International foot, the default. US, UK, Indian and historical Rhineland feet are supported by providing the option `'US'`, `'UK'`, `'India'` or `'Rhineland'`, respectively.

units.meter (x [, option])

units.metre (x [, option])

Takes a value `x` in International foot (default) and converts it to metres. US and Indian survey feet, UK and historical Rhineland feet are supported by providing the option `'US'`, `'India'`, `'UK'` or `'Rhineland'`, respectively. If `option` is `'yard'`, then `x` is taken to be in (Canadian, that is standard) yards and metres are returned.

units.km (x [, option])

Takes a number `x` in statute miles and converts it to kilometres. If any `option` is given then `x` is in nautical miles.

units.mile (x [, option])

Takes a number `x` in kilometres and converts it to statute miles. If any `option` is given then `x` is converted to nautical miles.

units.yard (x)

Takes a number `x` in metres and converts it to (Canadian) yards.

units.cm (x [, option])

By default, takes a number `x` in inches and converts it to centimetres.

- If `option` is the string `'piad'`, `x` is taken to be in Russian handspans (piads) with 1 piad = 17.8 cm.
- If `option` is the string `'span'`, `x` is taken to be in US spans with 1 span = 22.86005 cm.
- If `option` is the string `'hand'`, `x` is taken to be in US hands with 1 hand = 4 inches = 10.16002 cm.
- If `option` is the string `'link'`, `x` is taken to be in US links with 1 link = $\frac{2}{3}$ feet = 20.11684 cm.

units.inch (x)

By default, takes a number `x` in centimetres and converts it to inches.

- If `option` is the string `'piad'`, `x` is taken to be in Russian handspans (piads) with 1 piad = 7.01 inches.
- If `option` is the string `'span'`, `x` is taken to be in US spans with 1 span = 9 inches.
- If `option` is the string `'hand'`, `x` is taken to be in US hands with 1 hand = 4 inches.
- If `option` is the string `'link'`, `x` is taken to be in US links with 1 link = $\frac{2}{3}$ feet = 7.92 inches.

units.gram (x)

Takes a number `x` in grams and converts it to (International) avoirdupois ounces - these are the United States customary and British imperial ounces.

units.ounce (x)

Takes a number `x` in grams and converts it to (International) avoirdupois ounces - these are the United States customary and British imperial ounces.

units.floz (x [, option])

Takes a number `x` in litres and converts it to US fluid ounces (default) or to imperial fluid ounces if any `option` is given.

units.liter (x [, option])

units.litre (x [, option])

Takes a number `x` in fluid ounces or gallons and converts it to litres:

- If no `option` is given, `x` is in US fluid ounce.
- If `option` is the string `'imp floz'`, `x` is in Imperial fluid ounces.
- If `option` is the string `'gallon'`, `x` is in US liquid gallons.
- If `option` is the string `'dry gallon'`, `x` is in US dry gallons.

- If `option` is the string `'imp gallon'`, `x` is in British imperial gallons.

`units.gallon (x [, option])`

Takes a number `x` in litres converts it to gallons:

- If no `option` is given or option is `'gallon'`, then `x` is converted to US liquid gallons.
- If `option` is the string `'dry gallon'`, then `x` is converted to US dry gallons.
- If `option` is the string `'imp gallon'`, then `x` is converted to British imperial gallons.

Chapter Seventeen

C API Functions

17 C API Functions

As already noted in Chapter 1, Agena features the same C API as Lua 5.1 so you are able to easily integrate your C packages and functions written for Lua 5.1 in Agena. Actually, Agena's C API is a superset of Lua's C API[14]. For a description of the API functions taken from Lua, see its Lua 5.1 manual.

C API compatibility functions for Lua 5.2, 5.3 and 5.4 have also been added to facilitate porting C functions from these Lua editions to Agena.

Furthermore, Luiz Henrique de Figueiredo's `ae` library for evaluating mathematical expressions in C programs based on Agena is availabe via the Agena C API. See bottom of this chapter.

The functions listed cannot be used in your Agena procedures - they have been created to access Agena's features from within C code. It generally supports GCC 3.4.6 and above.

If you would like to compile a Lua C package for Agena, usually only the names of following header files have to be changed:

Lua Header File	Corresponding Agena Header File
lua.h	agena.h
lauxlib.h	agnxlib.h
lualib.h	agenalib.h
luaconf.h	agnconf.h

The following Agena-specific header files exist:

Agena Header	Functionality
agncfg.h	This file will be created when executing `make config`. It determines the Endianness of your system, extends C long ints to eight bytes, and determines the date and time for the Agena build. It is advised to not change the contents of this header file.
agncmpt.h	Establishes cross-platform compatibility for certain mathematical C functions, a few 64-bit C types, and functions to work with files beyond the 2 GBytes size limit. Applicable primarily to Solaris, but also Linux, OS/2, Windows, and GCC.
agnhlps.h	Provides C helper functions and definitions, primarily for file access, further 64-bit types, quicksort, IEEE, Endian, mathematical operations & constants, cross-platform keyboard access, and fast and secure string concatenation and search-and-replace functions. Useful to compile Agena on SPARCs, PPCs, other RISC systems, and also on Little Endian

[14] Full compatibility to Lua's API has been established with Agena 1.6.0 in May 2012.

Agena Header	Functionality
	architectures, since the **binio** package, **read,** and **save** work in Big Endian mode.
agnt64.h, agnt64_c.h, agnt64_l.h	Year 2038-fix headers for 32-bit systems.
cephes.h	Interface to Stephen L. Moshier's mathematical functions.
rlhmath.h	API to exponential integral functions written by RLH.
interp.h	Interface to Professor Brian Bradie's various interpolation and spline functions.
sofa.h	Interface to the IAU Standards of Fundamental Astronomy (SOFA) Libraries.
moon.h, sunriset.h	Miscellaneous astronomical C functions
xbase.h	Interface to dBASE III file support of the Shapelib library.
charbuf.h	Small string character buffer library
luasys.h	Nodir Temirkhodjaev's Lua System (LuaSys v1.8)

Agena features a macro **agn_Complex** which is a shortcut for complex double.

The following API functions have been added (see files `lapi.c` and `agena.h`):

agn_absindex

```
LUA_API int agn_absindex (lua_State *L, int index, int gettop)
```

Returns the absolute positive stack index number for a given non-zero index `i` and the number of arguments `gettop` passed to a function.

agn_arrayborders

```
void agn_arrayborders (lua_State *L, int idx, size_t a[])
```

Returns the lowest and highest indices in the array part of the table at `idx`, where the lowest index may start at 1 (not 0 as in C). `a` must be an array of 2 slots. If `a`[0], `a`[1] are 0, there is no array part in the table.

agn_arraypart

```
void agn_arraypart (lua_State *L, int idx)
```

Pushes a table with all the values in the array part of the table at index `idx` onto the top of the stack.

agn_asize

```
size_t agn_asize (lua_State *L, int idx);
```

Returns the number of items actually currently stored to the array part of the table at stack index `idx`, using a linear method. The search is done in O(n) time as each integer index will be checked. For an O(log2) alternative, see **luaL_getn**.

See also: **agn_hsize**, **agn_size**.

agn_hsize

```
size_t agn_hsize (lua_State *L, int idx);
```

Returns the number of items actually currently stored to the hash part of the table at stack index `idx`, using a linear method. See also: **agn_asize**, **agn_size**.

agn_borders

```
void agn_borders (lua_State *L, int idx, size_t a[]);
```

Returns the smallest and largest assigned index - in this order - in the array and hash part of a table, in two-element array a. If zeros are returned, the array and hash parts of the table are empty.

agn_ccall

```
agn_Complex agn_ccall (lua_State *L, int nargs);
```
(Non-ANSI)

```
agn_Complex agn_ccall (lua_State *L, int nargs,
   lua_Number *real, lua_Number *imag);
```
(ANSI)

There are two different versions of this API function available. The first form supports Non-ANSI versions of Agena, e.g. Solaris, OS/2, etc. The second form can be used in the ANSI versions of Agena (compiled with the LUA_ANSI option).

Non-ANSI version: Exactly like **lua_call**, but returns a complex value as its result, so a subsequent conversion to a complex number via stack operation is avoided. If the result of the function call is not a number or complex value, an error will be issued. **agn_ccall** pops the function and its arguments from the stack.

ANSI version: Like **lua_call**, but returns the real and imaginary parts of the complex result through the parameters `real` and `imag`. If the result of the function call is not a number or complex value, an error will be issued. **agn_ccall** pops the function and its arguments from the stack.

If the result of the function call is a number, it is automatically converted to a complex value.

The function always returns the first result of the function call.

The function does *not* reserve its own stack space so you must call **lua_checkstack** or **luaL_checkstack** before.

agn_checkboolean

```
int agn_checkboolean(lua_State *L, int idx);
```

Checks whether the value at index `idx` is a Boolean and returns 1 for **true**, and 0 for **false** or **fail**. An error will be raised if the value at `idx` is none of them.

agn_checkcomplex

```
LUALIB_API agn_Complex agn_checkcomplex (lua_State *L, int idx)
```

Checks whether the value at index `idx` is a complex value and returns it. An error is raised if the value at `idx` is not of type complex.

agn_checkinteger

```
lua_Integer agn_checkinteger (lua_State *L, int idx);
```

Checks whether the value at index `idx` is a number and an integer and returns this integer. An error is raised if the value at `idx` is not a number, or if it is fractional.

See also: agn_checknonposint, agn_checkposint, lua_checkinteger.

agn_checklstring

```
const char *agn_checklstring (lua_State *L, int idx, size_t *len);
```

Works exactly like luaL_checklstring but does not perform a conversion of numbers to strings. See also luaL_checklstringornil.

agn_checknonnegint

```
lua_Integer agn_checknonnegint (lua_State *L, int idx);
```

Checks whether the value at index `idx` is a number and a non-negative integer and returns this integer. An error is raised if the value at `idx` is not a number, or if it is a float or is negative.

See also: **agn_checkinteger**, **agn_checkposint**, **agn_checkuint16_t**, **agn_checkuint32_t**.

agn_checknonnegative

```
lua_Number agn_checknonnegative (lua_State *L, int idx);
```

Like **agn_checknumber**, but checks whether the number at `idx` is non-negative.

See also: **agn_checkpositive**.

agn_checknumber

```
lua_Number agn_checknumber (lua_State *L, int idx);
```

Checks whether the value at index `idx` is a number and returns this number. An error is raised if the value at `idx` is not a number. This procedure is an alternative to **luaL_checknumber** for it is around 14 % faster in execution while providing the same functionality by avoiding different calls to internal Auxiliary Library functions.

See also: **agn_checkpositive**, **agn_checknonnegative**.

agn_checkposint

```
lua_Integer agn_checkposint (lua_State *L, int idx);
```

Checks whether the value at index `idx` is a number and a positive integer and returns this integer. An error is raised if the value at `idx` is not a number, or if it is a float or is non-positive.

See also: **agn_checkinteger**, **agn_checknonnegint**.

agn_checkpositive

```
lua_Number agn_checkpositive (lua_State *L, int idx);
```

Like **agn_checknumber**, but checks whether the number at `idx` is positive.
See also: **agn_checknonnegative**.

agn_checkuint16_t

```
uint16_t agn_checkuint16_t (lua_State *L, int idx);
```

Checks whether its argument at stack position `idx` is an unsigned integer and whether it fits into the range 0 .. 2^{16} - 1.

See also: **agn_checknonnegint**, **agn_checkuint32_t**, **agnL_optuint32_t**.

agn_checkuint32_t

```
uint32_t agn_checkuint32_t (lua_State *L, int idx);
```

Checks whether its argument at stack position `idx` is an unsigned integer and whether it fits into the range 0 .. 2^{32} - 1.

See also: **agn_checknonnegint**, **agn_checkuint16_t**.

agn_checkstring

```
const char *agn_checkstring (lua_State *L, int idx);
```

Works exactly like luaL_checkstring but does not perform a conversion of numbers to strings. An error is raised if `idx` is not a string.

If `idx` is negative: due to garbage collection, there is no guarantee that the pointer returned will be valid after the corresponding value is removed from the stack.

agn_cleanse

```
LUA_API void agn_cleanse (lua_State *L, int idx, int gc)
```

Empties the entire contents of the table at index `idx`, but does not delete it. If `gc` is 1, then a garbage collection is performed, as well. Set it to 0 of no garbage collection shall be triggered. The function does not change the stack.

agn_cleanseset

```
LUA_API void agn_cleanseset (lua_State *L, int idx, int gc)
```

Empties the entire contents of the set at index `idx`, but does not delete it. If `gc` is 1, then a garbage collection is performed, as well. Set it to 0 of no garbage collection shall be triggered. The function does not change the stack.

agn_compleximag

```
lua_Number agn_compleximag (lua_State *L, int idx)
```

Returns the imaginary part of the complex value at stack index `idx` as a lua_Number. See also: **agn_complexreal**.

agn_complexreal

```
lua_Number agn_complexreal (lua_State *L, int idx)
```

Returns the real part of the complex value at stack index `idx` as a lua_Number. See also: **agn_compleximag**.

agn_copy

```
LUA_API void agn_copy (lua_State *L, int idx, int mode)
```

Returns a true copy of the table, set, or sequence at stack index `idx`. The copy is put on top of the stack, but the original structure is not removed. `mode` controls what to do with tables: `mode` = 1: copy array part only; `mode` = 2: copy hash part only; `mode` = 3: copy both array and hash part. With all structures, if `mode` = 7 then no metatables or user-defined types are copied. The function performs a garbage collection.

agn_createcomplex

```
LUA_API void agn_createcomplex (lua_State *L, agn_Complex c)
```

Pushes a value of type complex onto the stack with its complex value given by `c`.

agn_createpair

```
void agn_createpair (lua_State *L, int idxleft, int idxright);
```

Pushes a pair onto the stack with the left operand determined by the value at index `idxleft`, and the right operand by the value at index `idxright`. The left and right values are *not* popped from the stack. The function performs a garbage collection.

agn_createpairnumbers

```
void agn_createpairnumbers (lua_State *L, lua_Number l, lua_Number r);
```

Pushes a pair onto the stack with the left operand set to number `l`, and the right operand set to number `r`. The function performs a garbage collection.

agn_createpairstrings

```
void agn_createpairstrings (lua_State *L,
                            const char *l, const char *r);
```

Pushes a pair onto the stack with the left-hand part set to string `l`, and the right-hand side to string `r`. The function performs a garbage collection.

agn_createreg

```
LUA_API void agn_createreg (lua_State *L, int nrec)
```

Pushes a register onto the top of the stack with `nrec` pre-allocated places. `nrec` may be zero, a default which is currently 16.

agn_createtable

```
LUA_API void agn_createtable (lua_State *L, int idx)
```

Creates an empty remember table for the function at stack index `idx`. It does not change the stack.

agn_createseq

```
void agn_createseq (lua_State *L, int nrec);
```

Pushes a sequence onto the top of the stack with `nrec` pre-allocated places (`nrec` may be zero).

agn_createset

```
void agn_createset (lua_State *L, int nrec);
```

Pushes an empty set onto the top of the stack. The new set has space pre-allocated for `nrec` items.

agn_deletefield

```
LUA_API void agn_deletefield (lua_State *L, int idx, const char *key)
```

Deletes the field `key` from the table at index `idx` without invoking metamethods. The function leaves the stack unchanged.

agn_deletertable

```
LUA_API void agn_deletertable (lua_State *L, int objindex)
```

Deletes the remember table of the procedure at stack index `idx`. If the procedure has no remember table, nothing happens. The function leaves the stack unchanged.

agn_equalref

```
int lua_equalref (lua_State *L, int idx1, int idx2);
```

Compares any two values at stack indices `idx1` and `idx2`, and returns 0 if they are different and 1 if they are equal. See **environ.isequal** for more information. The function does not change the stack.

agn_entries

```
void agn_entries (lua_State *L, int idx, int *flag);
```

Returns all the values stored to the table at stack index `idx` in a new table and sets it to the top of the stack. `flag` is set to 0 if no value are residing in the hash part, and to 1 if there is at least one element in the hash part.

agn_fnext

```
int agn_fnext (lua_State *L, int indextable, indexfunction, int mode);
```

Pops a key from the stack, and pushes three or four values in the following order: the key of a table given by `indextable`, its corresponding value (if mode = 1), the function at stack number `indexfunction`, and the value from the table at the given `indextable`. If there are no more elements in the table, then **agn_fnext** returns 0 (and pushes nothing).

The function is useful to avoid duplicating values on the stack for **lua_call** and the iterator to work correctly.

A typical traversal looks like this:

```
/* table is in the stack at index 't', function is at stack index 'f' */
lua_pushnil(L); /* first key */
while (lua_fnext(L, t, f, 0) != 0) {
  /* 'key' is at index -3, function at -2, and 'value' at -1 */
  lua_call(L, 1, 1);  /* call the function with one arg & one result */
  lua_pop(L, 1);      /* removes result of lua_call;
                         keeps 'key' for next iteration */
}
```

While traversing a table, do not call **lua_tolstring** directly on a key, unless you know that the key is actually a string. Recall that **lua_tolstring** changes the value at the given index; this confuses the next call to **lua_next**.

agn_getbitwise

```
int agn_getbitwise (lua_State *L)
```

Returns the current mode for bitwise arithmetic: 0 if the bitwise operators (**&&**, **||**, **^^**, **~~**, and **shift**), internally calculate with unsigned integers, and 1 if signed integers are used.

See also: **agn_setbitwise**.

agn_getcmplxparts

```
void agn_getcmplxparts (lua_State *L, int idx,
   lua_Number *re, lua_Number *im)
```

Expects a number or complex number at stack position `idx` and returns its real and imaginary part in `re` and `im`. If the value at `idx` is a number or complex number, returns 1 and 0 otherwise. With numbers, `im` will always be 0.

agn_getconstants

```
int agn_getconstants (lua_State *L)
```

Returns a non-zero if constant declarations are active, and 0 otherwise.

See also: **agn_setconstants**.

agn_getdblepsilon

```
lua_Number agn_getdblepsilon (lua_State *L)
```

The macro returns the setting of the double-accuracy threshold epsilon, i.e. system variable "DoubleEps". See also: **agn_getepsilon**, **agn_setdblepsilon**.

agn_getduplicates

```
int agn_getduplicates (lua_State *L)
```

Returns a non-zero if parser duplicate declaration warnings are turned on, and 0 otherwise.

See also: **agn_setduplicates**.

agn_getemptyline

```
int agn_getemptyline (lua_State *L)
```

Returns the current setting for two input prompts always being separated by an empty line and pushes a Boolean on the stack.

See also: **agn_setemptyline**.

agn_getepsilon

```
lua_Number agn_getepsilon (lua_State *L)
```

The macro returns the setting of the accuracy threshold epsilon used by the ~= operator and the **approx** function. See also: **agn_getdblepsilon**, **agn_setepsilon**.

agn_getfunctiontype

```
LUA_API int agn_getfunctiontype (lua_State *L, int idx)
```

Returns 1 if the function at stack index idx is a C function, 0 if the function at idx is an Agena function, and -1 of the value at idx is no function at all.

agn_gethepsilon

```
lua_Number agn_gethepsilon (lua_State *L)
```

The macro returns the setting **hEps**. See also: Chapter 5.2.2 and **agn_sethepsilon**.

agn_getinumber

```
lua_Number agn_getinumber (lua_State *L, int idx, int n);
```

Returns the value t[n] *as a* lua_Number, where t is a table at the given valid index `idx`. If t[n] is not a number, the return is 0. The access is raw; that is, it does not invoke metamethods. See also: lua_rawsetinumber.

agn_getlibnamereset

```
int agn_getlibnamereset (lua_State *L)
```

Returns the current setting for the **restart** statement to also reset **libname** and either pushes a non-zero integer (= **true**) or zero (= **false**).

See also: **agn_setlibnamereset**.

agn_getlongtable

```
int agn_getlongtable (lua_State *L)
```

Returns the current setting for key~value pairs in tables being output line by line instead of just a single line and puts a Boolean on the stack. A non-zero integer denotes the feature is switched on, and 0 it is switched off.

See also: **agn_setlongtable**.

agn_getround

```
LUA_API void agn_getround (lua_State *L)
```

Gets the current rounding mode. Pushes the string "downward" for FE_DOWNWARD, "upward" for FE_UPWARD, "nearest" for FE_TONEAREST, and "towardzero" for FE_TOWARDZERO onto the stack. If the rounding mode could not be determined, **undefined** is pushed. If any other FE_* value is determined, **fail** will be pushed. Not available in DOS.

See also: **agn_setround**.

agn_getrtable

```
LUA_API int agn_getrtable (lua_State *L, int idx)
```

Pushes the remember table of the function at stack index `idx` onto the stack and returns 1. If the function does not have a remember table, it pushes nothing and returns 0.

See also: **agn_getstorage**.

agn_getrtablewritemode

```
int agn_getrtablewritemode (lua_State *L, int idx)
```

Returns 0 if the remember table of the function at stack index idx cannot be updated by the **return** statement (i.e. if it is an rotable), 1 if it can (i.e. if it is an rtable), 2 if the function at idx has no remember table at all, and -1 if the value at idx is not a function.

agn_getseqlstring

```
const char *agn_getseqlstring (lua_State *L, int idx, int n, size_t *l);
```

Gets the string at index `n` in the sequence at stack index `idx`. The length of the string is stored to `l`.

agn_getstorage

```
LUA_API int agn_getstorage (lua_State *L, int idx)
```

Pushes the internal 'store' table of the function at stack index `idx` onto the stack and returns 1. If the function does not have a store, it pushes nothing and returns 0. See also: Chapter 6.25 and **agn_getrtable**. See also: **agn_setstorage**.

agn_getutype

```
int agn_getutype (lua_State *L, int idx);
```

Returns the user-defined type of a procedure, table, sequence, set, userdata, or pair at stack position `idx` as a string, pushes it onto the top of the stack and returns 1. If no user-defined type has been defined, the function returns 0 and pushes nothing onto the stack.

See also: **agn_isutype**, **agn_setutype**.

agn_hasarraypart

```
int agn_hasarraypart (lua_State *L, int idx);
```

Checks whether the table at stack index `idx` has at least one element assigned in its array part. The return is either 1 at least one element is in the array part) or 0 otherwise. The function pushes nothing and leaves the stack unchanged.

agn_hashashpart

```
int agn_hashashpart (lua_State *L, int idx);
```

Checks whether the table at stack index `idx` has at least one element assigned in its hash part. The return is either 1 at least one element is in the hash part) or 0 otherwise. The function pushes nothing and leaves the stack unchanged.

agn_hashpart

```
void agn_hashpart (lua_State *L, int idx)
```

Pushes a table with all the values in the hash part of the table at index `idx` onto the top of the stack.

agn_in

```
int agn_in (lua_State *L, int idxv, int idxt, int mode)
```

Checks whether the value at stack index `idxv` is included in the structure at `idxt`. If `idxt` does not refer to a structure, the function triggers an error. The function returns 1 if the element has been found and 0 otherwise. If mode is 1 then the function additionally pushes **true** or **false** on the stack.

agn_initmethodcall

```
int agn_initmethodcall (lua_State *L,
  const char *tablename, int lentablename)
```

The function prepares an OOP method call and should be to be embedded into the function that is called by the `__index' mt handler.

With `tablename.procname` representing a function, the C API function works as follows: the tag string (`procname`) is at stack position -1, the tablename userdata on -2, and the method arguments, zero, one or more, are looming somewhere else. The function actually does not call the OOP `function` as this is done by the VM automatically when the `__index' mt returns.

`tablename` is the name of the package, and `lentablename` the length of `tablename` in characters without a finalising \0.

Example usage:

```
static int tablename_get (lua_State *L) {
  < ... >
  if (<condition for procedural call>) {
    < do something >
  } else {
    return agn_initmethodcall(L, "tablename", 3);
  }
}
```

agn_intentries

```
void agn_intentries (lua_State *L, int idx, int *flag);
```

Returns all the values in the table t at stack index `idx` that have integral indices in a new table and puts it onto the top of the stack. It also sets `flag` to 0 if there are no integer indices in hash part of t, and `flag` to 1 if there is at least one integer index in the hash part.

agn_intindices

```
void agn_intindices (lua_State *L, int idx, int *flag);
```

Returns all integer indices of the table t at stack index `idx` in a new table and puts it onto the top of the stack. It also sets `flag` to 0 if there are no integer indices in hash part of t, and `flag` to 1 if there is at least one integer index in the hash part.

agn_isfail / lua_isfail

```
int agn_isfail (lua_State *L, int idx);
int lua_isfail (lua_State *L, int idx);
```

Returns 1 if the Boolean value at the given acceptable index results to **fail**, 0 otherwise (**true** and **false**). **lua_fail** first checks whether the value at the index is a Boolean to avoid crashes if it is not, and then whether it represents **fail**. The functions actually are C macros. See also: **agn_istrue**, **agn_isfail**, **lua_istrue**, **lua_isfalse**.

agn_isfalse / lua_isfalse

```
int agn_isfalse (lua_State *L, int idx);
int lua_isfalse (lua_State *L, int idx);
```

Returns 1 if the Boolean value at the given acceptable index results to **false**, 0 otherwise (**true** and **fail**). **lua_false** first checks whether the value at the index is a Boolean to avoid crashes if it is not, and then whether it represents **false**.

The functions actually are C macros. See also: **agn_istrue**, **agn_isfail**, **lua_istrue**, **lua_isfail**.

lua_isfalseorfail

```
int lua_isfalseorfail (lua_State *L, int idx);
```

Checks whether the value at the index is a Boolean to avoid crashes, and then whether it represents **false** or **fail**. The function actually are C macros.

lua_isnilfalseorfail

```
int lua_isnilfalseorfail (lua_State *L, int idx);
```

Checks whether the value at the index is a Boolean to avoid crashes, and then whether it represents **null**, **false** or **fail**. The function actually are C macros.

agn_isinteger

```
int agn_isinteger (lua_State *L, int idx);
```

The function returns 1 if the value at the given acceptable index is a number representing an integer, and 0 otherwise. The function makes sure that the stack value to be checked actually is a number so there will be no segmentation faults.

See also: **agn_isnumber**, **agn_isposint**, **agn_isnonnegint**.

agn_islinalgvector

```
int agn_islinalgvector (lua_State *L, int idx, size_t *dim)
```

Tests if a value at the given acceptable index is a vector created with the **linalg** package, and returns 1 if true and 0 otherwise. It also stores the dimension of the vector in `dim`.

agn_isnonnegint

```
int agn_isnonnegint (lua_State *L, int idx);
```

The function returns 1 if the value at the given acceptable index is a number representing a non-negative integer, and 0 otherwise. The function makes sure that the stack value to be checked actually is a number so there will be no segmentation faults.

See also: **agn_isinteger**, **agn_isposint**.

agn_isnumber

```
agn_isnumber (lua_State *L, int idx);
```

This macro returns 1 if the value at the given acceptable index is a number, and 0 otherwise. See also: **agn_isfloat**, **agn_isinteger**.

agn_isposint

```
int agn_isposint (lua_State *L, int idx);
```

The function returns 1 if the value at the given acceptable index is a number representing a positive integer, and 0 otherwise. The function makes sure that the stack value to be checked actually is a number so there will be no segmentation faults.

See also: **agn_isinteger**, **agn_isnonnegint**.

agn_issequtype

```
int *agn_issequtype (lua_State *L, int idx, const char *str);
```

Checks whether the type at stack index `idx` is a sequence and whether the sequence has the user-defined type denoted by `str`. It returns 1 if the above condition is true, and 0 otherwise.

agn_issetutype

```
int *agn_issetutype (lua_State *L, int idx, const char *str);
```

Checks whether the type at stack index `idx` is a set and whether this set has the user-defined type denoted by `str`. It returns 1 if the above condition is true, and 0 otherwise.

agn_isstring

```
agn_isstring (lua_State *L, int idx);
```

This macro returns 1 if the value at the given acceptable index `idx` is a string, and 0 otherwise.

agn_istableutype

```
int *agn_istableutype (lua_State *L, int idx, const char *str);
```

Checks whether the type at stack index `idx` is a table and whether the table has the user-defined type denoted by `str`. It returns 1 if the above condition is true, and 0 otherwise.

agn_istrue / lua_istrue

```
int agn_istrue (lua_State *L, int idx);
int lua_istrue (lua_State *L, int idx);
```

Returns 1 if the Boolean value at the given acceptable index `idx` results to **true**, 0 otherwise (**false** and **fail**). **lua_true** first checks whether the value at the index is a Boolean to avoid crashes if it is not, and then whether it represents **true**.

The functions actually are C macros.

See also: **agn_isfalse**, **agn_isfail**, **lua_isfalse**, **lua_isfail**.

agn_isutype

```
int *agn_isutype (lua_State *L, int idx, const char *str);
```

Checks whether a user-defined type `str` has been set for the given table, set, sequence, pair, or procedure at stack position `idx`. It returns 1 if the user-defined type has been set, and 0 otherwise.

agn_isutypeset

```
int *agn_isutypeset (lua_State *L, int idx, const char *str);
```

Checks whether a user-defined type has been set for the given object at stack position `idx`. It returns 1 if a user-defined type has been set, and 0 otherwise. The function accepts any Agena types. By default, if the object is not a table, sequence, a pair, set, or procedure, it returns 0.

agn_malloc

```
void *agn_malloc (lua_State *L, size_t size, const char *procname, ...);
```

Allocates `size` bytes of memory and returns a pointer to the newly allocated block. In case memory could not be allocated, it returns an error message including `procname` that called **agn_malloc**. The function optionally can free one or more objects referenced by their pointers in case memory allocation failed.

In all cases, the last argument must be **NULL**.

See also: **agn_free**.

agn_ncall

```
lua_Number agn_ncall (lua_State *L, int nargs, *int error, int quit);
```

Exactly like **lua_call**, but returns a numeric result as an Agena number, so a subsequent conversion to a number via stack operations is avoided. If the result of the function call is not numeric, an error will be issued. **agn_ncall** pops the function, its arguments and the result from the stack, leaving it leveled. It always returns the first result of the function call.

If the function call does not evaluate to a number, `error` is set to 1 and 0 otherwise. If `quit` is 1, the function will automatically issue an error if the result is not a number; otherwise `quit` should be set to 0. The function does *not* allocate its own stack space, so you must call **lua_checkstack** or **luaL_checkstack** before.

agn_nops

```
size_t agn_nops (lua_State *L, int idx);
```

Determines the number of actual table, set, sequence or register entries of the structure or the size of a string at stack index `idx`. If the value at `idx` is not one of the mentioned data types, it returns 0. With tables, this procedure is an alternative to **lua_objlen** if you want to get the size of a table since **lua_objlen** does not return correct results if there are holes in the table or if the table is a dictionary .

agn_numintersect

```
int agn_numintersect (lua_State *L, int idxa, int idxb);
```

Counts the number of elements in the intersection of the structure at stack index `idxa` and the structure at stack index `idxb`. Both structures must be of the same kind. The function does not change the stack.

agn_numminus

```
int agn_numminus (lua_State *L, int idxa, int idxb);
```

Counts the number of elements in the difference of the structure at stack index `idxa` and the structure at stack index `idxb`. Both structures must be of the same kind. The function does not change the stack.

agn_numunion

```
int agn_numunion (lua_State *L, int idxa, int idxb);
```

Counts the number of elements in the union of the structure at stack index `idxa` and the structure at stack index `idxb`. Both structures must be of the same kind. The function does not change the stack.

agn_onexit

```
LUA_API void agn_onexit (lua_State *L)
```

Pushes the function **environ.onexit** if it exists, and calls it. The function leaves the stack unchanged.

agn_optcomplex

```
agn_Complex agn_optcomplex (lua_State *L, int narg, agn_Complex z);
```

If the value at index `narg` is a complex number, it returns this number. If this argument is absent or is **null**, the function returns complex `z`. Otherwise, raises an error.

agn_pairgeti

```
int agn_pairgeti (lua_State *L, int idx, int n);
```

Puts the left operand of a pair at stack index `idx` if `n` is 1, and the right operand if `n` is 2, and puts it onto the top of the stack. You have to make sure that `n` is either 1 or 2. The function returns the type of the value pushed.

agn_pairgetiall

```
void agn_pairgetiall (lua_State *L, int idx);
```

Puts the left and the right operand of the pair at stack index `idx`, onto the top of the stack, in this order. The function makes sure that the two stack slots are available, you do not have to call **lua_checkstack**, **luaL_checkstack** before.

agn_pairgetiintegers

```
void agn_pairgeti (lua_State *L, int idx, int *x, int *y);
```

Returns in `x` and `y` the integers stored to a pair, at stack index `idx`, and returns 1 if successful or returns 0 if the object at `idx` is not a pair or if any of the pair components is not an integer. In the latter case, `x` and `y` are not assigned anything.

agn_pairgetinumbers

```
void agn_pairgeti (lua_State *L, int idx, lua_Number *x, lua_Number *y);
```

Returns in `x` and `y` the numbers stored to a pair, at stack index `idx`, and returns 1 on success or 0 if the object at `idx` is not a pair or if any of the pair components is not a number. In the latter case, `x` and `y` are not assigned anything.

agn_pairrawget

```
void agn_pairrawget (lua_State *L, int idx);
```

Pushes onto the stack the left or the right hand value of a pair `t`, where `t` is the value at the given valid index `idx` and the number `k` (`k`=1 for the left hand side, `k`=2 for the right hand side) is the value at the top of the stack. It does not invoke any metamethods. This function pops both `k` from the stack.

agn_pairrawset

```
void agn_pairrawset (lua_State *L, int idx);
```

Does the equivalent to p[k] := v, where p is a pair at the given valid index `idx`, v is the value at the top of the stack, and k is the value just below the top.

This function pops both the key and the value from the stack. It does not invoke any metamethods.

See also: **agn_pairseti**.

agn_pairseti

```
LUA_API int agn_pairseti (lua_State *L, int idx, int pos)
```

Sets the value at the stack top to the pair at index `idx` and pops the value. If `pos` is 1, then the value is put into the left part of the pair, if `pos` is 2, then the right part is set. Returns 0 if the `pos` is out of range and 1 on success.

See also: **agn_pairrawset**.

agn_pairstate

```
LUA_API void agn_pairstate (lua_State *L, int idx, size_t a[])
```

Returns a flag indicating whether a metatable has been assigned to the pair at index `idx` in `a`, with `a` a C array with one entry, where 1 indicates that the pair features a metatable, and 0 means it does not.

agn_parts

```
void agn_parts (lua_State *L, int idx)
```

Pushes two tables onto the top of the stack: one with all the values in the array part of the table at index `idx`, and one with the values in its hash part.

agn_poptop

```
void agn_poptop (lua_State *L);
```

Pops the top element from the stack. The function is more efficient than lua_pop(L, 1).

agn_poptoptwo

```
void agn_poptoptwo (lua_State *L);
```

Pops the top element and the value just below the top from the stack. The function is more efficient than lua_pop(L, 2).

agn_pushboolean

```
void agn_pushboolean (lua_State *L, int b);
```

Pushes **true** onto the stack if `b` is 1 or larger, and pushes **false** onto the stack if `b` is 0. If `b` is -1, it pushes **fail** onto the stack.

agn_pushcomplex

```
void agn_pushcomplex (lua_State *L, lua_Number re, lua_Number im);
```

Pushes the complex value `re` + I*`im` onto the stack. The macro can be used both with Agena versions using standard complex.h complex functions and those using proprietary complex arithmetic. See also: lua_pushcomplex.

agn_qsumupdiv

```
void agn_qsumupdiv (lua_State *L, int idx, lua_number d,
   const char *procname)
```

Divides all the numbers or complex numbers x in the distribution at `idx` by d, then multiplies by x and sums up the quotients. The distribution may be either a table, sequence or register. The function pushes a number or complex number onto the stack. With all other values at `idx`, issues an error including the name `procname`, the function that called agn_sumupdiv. See also agn_qsumupdiv.

agn_rawgetfield

```
LUA_API void agn_rawgetfield (lua_State *L, int idx, const char *field)
```

Returns t[`field`], where table t resides at stack index `idx`.

agn_rawgeticomplex

```
LUA_API void agn_rawgeticomplex (lua_State *L, int idx, int n,
  lua_Number *z, int *rc)
```

Returns the real and imaginary parts of the number or complex number that resides at index `n` of table t at stack index `idx`. The real part is stored in `z`[0], the imaginary part in `z`[1].

The function also returns the type of value found at t[`n`] in `rc`: LUA_TNUMBER if t[`n`] is a number, LUA_TCOMPLEX if t[`n`] is a complex number or LUA_TNIL if t[`n`] is any other value. In the latter case (LUA_TNIL), `z`[0] = `z`[1] = 0. If t[`n`] is a number, `z`[1] is set to zero.

agn_rawgetifield

```
LUA_API void agn_rawgetifield (lua_State *L, int idx, int kidx)
```

Returns t[i], where table t resides at stack index `idx` and the key i at stack index `kidx`.

agn_rawgetiinteger

```
lua_Number agn_rawgetiinteger (lua_State *L, int idx, int n, int *rc);
```

Returns the integral value t[n] *as a* lua_Number, where t is a table at the given valid index `idx`. If t[n] is not a number or not an integer, the return is 0. The access is raw; that is, it does not invoke metamethods. `rc` includes the result of the retrieval and is 1 on success and 0 otherwise, i.e. t[n] is not a integer or a string convertible to an integer.

See also: agn_seqrawgetiinteger, agn_regrawgetiinteger.

agn_rawgetinumber

```
lua_Number agn_rawgetinumber (lua_State *L, int idx, int n, int *rc);
```

Returns the value t[`n`] *as a* lua_Number, where t is a table at the given valid index `idx`. If t[`n`] is not a number, the return is 0. The access is raw; that is, it does not invoke metamethods. `rc` includes the result of the retrieval and is 1 on success and 0 otherwise, i.e. t[`n`] is not a number or a string convertible to a number.

agn_rawgetilstring

```
const char *agn_rawgetilstring (lua_State *L, int idx, int n,
  size_t *l, int *rc);
```

Returns the value t[`n`] as a string, where t is a table at the given valid index `idx`. If t[`n`] is not a string, the return is NULL. The access is raw; that is, it does not invoke metamethods. `rc` includes the result of the retrieval and is 1 on success and 0 otherwise, i.e. t[`n`] is not a string. `l` will be assigned the string size or zero otherwise.

agn_rawgetistring

```
const char *agn_rawgetistring (lua_State *L, int idx, int n);
```

Returns the value t[n] as *a const char **, where t is a table at the given valid index idx. If t[n] is not a string, the return is NULL. The access is raw; that is, it does not invoke metamethods. See also: agn_rawgetilstring.

agn_rawinsert

```
LUA_API void agn_rawinsert (lua_State *L, int idx)
```

Inserts the value at the top of the stack to the table at index `idx`, more precisely, it is added to the end of the array part of the table. The value is popped from the stack.

agn_rawinsertfrom

```
LUA_API void agn_rawinsertfrom (lua_State *L, int tidx, int vidx)
```

Inserts the value at stack index `vidx` to the table residing at index `idx`, more precisely, it is added to the end of the array part of the table.

If `vidx` = -1, then the value is popped from the stack, otherwise the stack is left untouched.

agn_rawsetfield

```
LUA_API void agn_rawsetfield (lua_State *L, int idx, const char *field)
```

Does the equivalent to t[`field`] := v, where t is a table at the given valid index `idx`, and v is the value at the top of the stack, without invoking metamethods.

This function pops the value from the stack. It does not invoke any metamethods.

See also: **agn_deletefield**.

agn_reggetinoerr

```
LUA_API void agn_reggetinoerr (lua_State *L, int idx, size_t n)
```

Like **agn_reggeti**, but pushes **null** onto the stack if `n` is out of range, instead of throwing an error.

agn_reggetinoerrrange

```
LUA_API void agn_reggetinoerrrange (lua_State *L, int idx, int p, int q)
```

Like **agn_reggetinoerr**, but pushes elements t[`p`] to t[`q`], with `p` >= `q`, onto the stack. You must ensure that enough stack space has been reserved. `p`, `q` should be positive.

agn_reggetinumber

```
LUA_API lua_Number agn_reggeti (lua_State *L, int idx, size_t n)
```

The function the number stored at position `n` of the register located at stack index `idx`. If `n` is out-of-range, or larger than the position of the top pointer, it issues an error. It returns **infinity** if the value at `n` is non-numeric.

agn_reggettop

```
LUA_API size_t agn_reggettop (lua_State *L, int idx)
```

Returns the position of the top pointer of a register at stack index `idx`. See also: **agn_regsettop**.

agn_regpurge

```
LUA_API void agn_regpurge (lua_State *L, int idx, int n)
```

Removes the value at position `n` of the register at stack index `idx` and shifts down all values beyond `n` if necessary. The function does not reduce the size of the register, but decrements the top pointer by 1.

agn_regrawget

```
LUA_API void agn_regrawget (lua_State *L, int idx)
```

Pushes onto the stack the value t[`k`], where t is the register at the given valid index `idx` and `k` is the value at the top of the stack.

This function pops the key from the stack (putting the resulting value in its place). It does not invoke metamethods.

agn_regrawgeti

```
LUA_API void agn_regrawgeti (lua_State *L, int idx, size_t n)
```

Pushes the value stored at position `n` of the register located at stack index `idx` to the top of the stack. If `n` is out-of-range, or larger than the position of the top pointer, it issues an error.

agn_regrawgeticomplex

```
LUA_API void agn_regrawgeticomplex (lua_State *L, int idx, int n,
  lua_Number *z, int *rc)
```

Returns the real and imaginary parts of the number or complex number that resides at index `n` of table t at stack index `idx`. The real part is stored in `z`[0], the imaginary part in `z`[1].

The function also returns the type of value found at t[`n`] in `rc`: LUA_TNUMBER if t[`n`] is a number, LUA_TCOMPLEX if t[`n`] is a complex number or LUA_TNIL if t[`n`] is any other value. In the latter case (LUA_TNIL), `z`[0] = `z`[1] = 0. If t[`n`] is a number, `z`[1] is set to zero.

agn_regrawgetiinteger

```
lua_Number agn_regrawgetiinteger (lua_State *L, int idx, int n, int *rc);
```

Returns the integer in the register at stack index `idx` at position `n`, with `n` starting from 1. If successful, sets `rc` to 1 and 0 otherwise. The return is zero if the element is not an integer.

See also: **agn_rawgetiinteger**, **agn_regrawgetiinteger**.

agn_regrawgetinumber

```
lua_Number agn_regrawgetinumber (lua_State *L, int idx, int n, int *rc);
```

Returns the value t[`n`] *as a* lua_Number, where t is a register at the given valid index `idx`. If t[`n`] is not a number, the return is 0. The access is raw; that is, it does not invoke metamethods. If t[`n`] is a lua_Number, `rc` is set to 1 and 0 otherwise.

agn_regrawgetilstring

```
const char *agn_regrawgetilstring (lua_State *L, int idx, int n,
  size_t *l, int *rc);
```

Returns the value t[`n`] as a string, where t is a register at the given valid index `idx`. If t[`n`] is not a string, the return is NULL. The access is raw; that is, it does not invoke metamethods. `rc` includes the result of the retrieval and is 1 on success and 0 otherwise, i.e. t[`n`] is not a string. `l` will be assigned the string size or zero otherwise.

agn_regresize

```
LUA_API int agn_regresize (lua_State *L, int idx, size_t newsize, int nil)
```

Either reduces or extends the size of the register residing at stack index `idx` to `newsize` entries.

When reducing and if `nil` is 1, then all values residing at positions larger then `newindex`, are **null**'ed before, otherwise set `nil` to 0.

When extending, fills the newly created slots with **null**. If the current top pointer already refers to the total size of the register, it is set to `newsize`, otherwise it is left unchanged.

Returns 1 if the register has been shrunk or extended, and 0 if `n` is equal to the maximum size (not the value of the top pointer).

agn_regset

```
LUA_API void agn_regset (lua_State *L, int idx)
```

Assumes that the value to be set to a register residing at stack position `idx` is at the top of the stack and the numeric key just below the stack and conducts the assignment.

agn_regseti

```
LUA_API void agn_regseti (lua_State *L, int idx, int n)
```

Sets the value residing at the top of the stack to position `n` of the register at index `idx` and pops the inserted value from the stack.

agn_regsetinumber

```
void agn_regsetinumber (lua_State *L, int idx, int n, lua_Number num);
```

This macro sets the given Agena number `num` to the non-zero and positive index `n` of the register at positive or negative stack index `idx`.

agn_regsettop

```
LUA_API int agn_regsettop (lua_State *L, int idx)
```

Sets the current top pointer of a register residing at index `idx` to the number stored at the top of the stack. The number at the top of the stack is popped therafter. See also: **agn_reggettop**.

agn_regstate

```
LUA_API void agn_regstate (lua_State *L, int idx, size_t a[])
```

Returns the current top pointer, the total number of items, and a flag indicating whether a metatable has been assigned to the register at index `idx` in a, a C array with three entries. The position of the top pointer is stored to a[0], the total number of entries to a[1]. The metatable flag is stored to a[2], where 1 indicates that the sequence features a metatable, and 0 means it does not.

agn_regsubs

```
int agn_regsubs (lua_State *L, int idx, int idxold, int idxnew)
```

In the register at index `idx`, replaces all occurrences of the value at index `idxold` with the value at `idxnew`. The function does not change the stack and returns the number of substitutions made.

agn_seqrawgeticomplex

```
LUA_API void agn_seqrawgeticomplex (lua_State *L, int idx, int n,
  lua_Number *z, int *rc)
```

Returns the real and imaginary parts of the number or complex number that resides at index `n` of table t at stack index `idx`.

The real part is stored in `z`[0], the imaginary part in `z`[1].

The function also returns the type of value found at t[`n`] in `rc`: LUA_TNUMBER if t[`n`] is a number, LUA_TCOMPLEX if t[`n`] is a complex number or LUA_TNIL if t[`n`] is any other value. In the latter case (LUA_TNIL), `z`[0] = `z`[1] = 0. If t[`n`] is a number, `z`[1] is set to zero.

agn_seqrawgetiinteger

```
lua_Number agn_seqrawgetiinteger (lua_State *L, int idx, int n, int *rc);
```

Returns the integer in the sequence at stack index `idx` at position `n`, with `n` starting from 1. If successful, sets `rc` to 1 and 0 otherwise. The return is zero if the element is not an integer.

See also: **agn_rawgetiinteger**, **agn_regrawgetiinteger**.

agn_seqrawgetinumber

```
lua_Number agn_seqrawgetinumber (lua_State *L, int idx, int n, int *rc);
```

Returns the value t[`n`] *as a* lua_Number, where t is a sequence at the given valid index `idx`. If t[`n`] is not a number, the return is 0. The access is raw; that is, it does not invoke metamethods. If t[`n`] is a lua_Number, `rc` is set to 1 and 0 otherwise.

See also: **lua_seqrawgetinumber**.

agn_seqrawgetilstring

```
const char *agn_seqrawgetilstring (lua_State *L, int idx, int n,
  size_t *l, int *rc);
```

Returns the value t[n] as a string, where t is a sequence at the given valid index `idx`. If t[n] is not a string, the return is NULL. The access is raw; that is, it does not invoke metamethods. `rc` includes the result of the retrieval and is 1 on success and 0 otherwise, i.e. t[n] is not a string. `l` will be assigned the string size or zero otherwise.

agn_seqresize

```
int agn_seqresize (lua_State *L, int idx, size_t newsize);
```

Shrinks or expands the sequence at stack index `idx`, i.e. adds or deletes the number of pre-allocated slots to exactly `newsize`. The function takes care to nullify all surplus values before shrinking the sequence or adding **null**s when expanding it.

agn_seqsize

```
size_t agn_seqsize (lua_State *L, int idx);
```

Returns the number of items currently stored to the sequence at stack index `idx`.

agn_seqstate

```
void agn_seqstate (lua_State *L, int idx, size_t a[])
```

Returns the actual number of items, the maximum number of items assignable to, and a flag indicating whether a metatable has been assigned to the sequence at index `idx` in `a`, a C array with three entries. The actual number of items is stored to a[0], the maximum number of entries to a[1]. If a[1] is 0, then the number of possible entries is infinite. The metatable flag is stored in a[2], where 1 indicates that the sequence features a metatable, and 0 means it does not.

agn_seqsubs

```
int agn_seqsubs (lua_State *L, int idx, int idxold, int idxnew)
```

In the sequence at index `idx`, replaces all occurrences of the value at index `idxold` with the value at `idxnew`. The function does not change the stack and returns the number of substitutions made.

agn_setbitwise

```
void agn_setbitwise (lua_State *L, int value)
```

Sets the mode for bitwise arithmetic. If `value` is greater than 0, the bitwise functions (**&&**, **||**, **^^**, **~~**, and **shift**) internally calculate with signed integers, otherwise Agena calculates with unsigned integers.

See also: **agn_getbitwise**.

agn_setconstants

```
void agn_setconstants (lua_State *L, int value)
```

Switches on constants mode if `value` is non-zero, and switches it off if 0.

See also **agn_getconstants**.

agn_setdblepsilon

```
lua_Number agn_setdblepsilon (lua_State *L, lua_Number x)
```

Sets the double-accuracy threshold epsilon to system variable "DoubleEps".

See also: **agn_setepsilon**.

agn_setduplicates

```
void agn_setduplicates (lua_State *L, int value)
```

Switches on duplicate declaration warnings (shadowing) if `value` is non-zero, and switches it off if 0.

See also **agn_getduplicates**.

agn_setemptyline

```
void agn_setemptyline (lua_State *L, int value)
```

If `value` is greater than 0, then two input prompts are always separated by an empty line. If set **false**, no empty line is inserted.

See also: **agn_getemptyline**.

agn_setepsilon

```
lua_Number agn_setepsilon (lua_State *L, lua_Number x)
```

Sets the accuracy threshold epsilon used by the ~= operator and the **approx** function to the number `x`. See also: **agn_getepsilon**.

agn_sethepsilon

```
lua_Number agn_sethepsilon (lua_State *L, lua_Number x)
```

Sets the **hEps** constant. See also: Chapter 5.2.2 and **agn_gethepsilon**.

agn_setinumber

```
void agn_setinumber (lua_State *L, int idx, int i, lua_Number x)
```

Sets number `x` to key `i` of the table at positive or negative stack position `idx`. See also: **agn_getinumber**.

agn_setlibnamereset

```
void agn_setlibnamereset (lua_State *L, int value)
```

If `value` is greater than 0, then the **restart** statement resets libname to its default. If `value` is non-positive, then libname is not changed with a **restart**.

See also: **agn_getlibnamereset**.

agn_setlongtable

```
void agn_setlongtable (lua_State *L, int value)
```

If `value` is greater than 0, then the **print** function outputs key~value pairs in tables line-by-line. If `value` is non-positive, then the print function prints all pairs in a single consecutive line.

See also: **agn_getlongtable**.

agn_setreadlibbed

```
int agn_setreadlibbed (lua_State *L, const char *name)
```

Inserts name into the global set **package.readlibbed**.

agn_setresize

```
void agn_setresize (lua_State *L, int idx, size_t newsize, int protect)
```

Resizes the set at stack index `idx` to `newsize` pre-allocated slots. `protect` controls whether to allow only a set to be shrunk without dropping any elements (`protect == 1`), shrunk or enlarged without dropping any elements (`protect == 2`), or whether to have full control what may happen: shrinking or expanding, dropping or not dropping any elements (`protect == 0`). With `protect == 0`, it is advised that the set is empty.

agn_setround

```
int agn_setreadlibbed (lua_State *L, const char *name)
```

Sets the rounding mode. `what` may be "downward" for FE_DOWNWARD, "upward" for FE_UPWARD, "nearest" for FE_TONEAREST, and "towardzero" for FE_TOWARDZERO. Returns 1 on success, and 0 otherwise. In case of failure, the former rounding mode is re-established. Not available in DOS.

See also: **agn_getround**.

agn_setrtable

```
LUA_API void agn_setrtable (lua_State *L, int find, int kind, int vind)
```

Sets argument~return values to the function at stack index `find`. The argument list reside at a table array at stack index `kind`, the return list are in another table at stack index `vind`. See the description for the **rset** function for more information.

agn_setstorage

```
LUA_API int agn_setstorage (lua_State *L, int idx)
```

If a store has not been established for a function at stack index idx - which may either be implemented in C or Agena -, sets up the store for with the table at the top of the stack and pops this table thereafter.

If the store already exists, then the function adds all the entries in the table at the stack top to the store and then pops the table. If the value at the stack top is **null**, then the store is entirely deleted (not just emptied). The function always returns 1.

See also: Chapter 6.25 and **agn_getrtable**. See also: **agn_setstorage**.

agn_setudmetatable

```
LUA_API void agn_setudmetatable (lua_State *L, int idx)
```

Expects a valid userdata metatable at the top of the stack, assigns it to the userdata residing at stack index idx, and pops the value at the top of the stack thereafter. If the value at the top of the stack is null, then a metatable assigned to a userdatum is deleted, and null is popped from the stack.

agn_setutype

```
void agn_setutype (lua_State *L, int idxobj, int idxtype);
```

Sets a user-defined type of a procedure, table, sequence, set, userdata, or pair. The object is at stack index `idxobj`, the type (a string) is at position `idxtype`. The function leaves the stack unchanged.

If **null** is at `idxtype`, the function deletes the user-defined type.

Setting the type of a sequence, set, table, procedure, or pair also causes the pretty printer to display the string passed to the function instead of the usual output at the console.

See also: **agn_getutype**.

agn_size

```
int agn_size (lua_State *L, int idx);
```

Returns the number of items currently stored to the array and the hash part of the table at stack index `idx`. See also: **agn_asize**.

agn_ssize

```
int agn_ssize (lua_State *L, int idx);
```

Returns the number of items currently stored to the set at stack index `idx`.

agn_sstate

```
void agn_sstate (lua_State *L, int idx, size_t a[])
```

Returns the actual number of items and the current maximum number of items allocable to the set at index `idx` in `a`, a C array with three entries. The actual number of items is stored to a[0], the current allocable size to a[1]. a[2] indicates whether a metatable has been assigned to the set, where 0 means it does not, and 1 that it does.

agn_stralloc

```
char *agn_stralloc (lua_State *L, size_t l, const char *procname, ...);
```

Allocates a string buffer by internally determining its most efficient size, aligned along the "long" boundary. The return is a char* pointer to the beginning of the string. The function zeros only the last few bytes and assumes that the `trailing` rest will be filled by real characters later on. Just pass `l` as the number of characters, excluding the terminating \0, and do not multiply it by sizeof(char). The function automatically adds a terminating \0.

The function can optionally free variables passed after `procname` in case memory allocation fails internally. In case of an error, the function issues the name of the procedure `procname` from which it was called.

In any case, the last argument must always be NULL.

agn_strmatch

```
const char *agn_strmatch (lua_State *L, const char *s, size_t s_len,
    const char *p, ptrdiff_t init, ptrdiff_t *start, ptrdiff_t *end)
```

Searches string `s` of size `s_len` for pattern `p`. `init` is the position from where to start the search and by default is 1, the first character in `s`. `start` and `end` will include the start and end position in case of a match, always counting from 1. The return is the string starting at position `start` in case of a match, or NULL if there was no hit.

agn_structinsert

```
void agn_structinsert (lua_State *L, int idxs, int idxv)
```

Inserts the object at stack index `idxv` into the table, set, sequence or register at stack index `idxs`. The function does not change the stack.

agn_sumup

```
void agn_sumup (lua_State *L, int idx, const char *procname)
```

Sums up all the numbers or complex numbers in the table, sequence or register the at stack index `idx` and pushes a number or complex number onto the stack. The distribution may be either a a table, sequence or register. With all other values at `idx`, issues an error including the name `procname`, the function that called agn_sumup.

agn_sumupdiv

```
void agn_sumupdiv (lua_State *L, int idx, lua_number d,
   const char *procname)
```

Sums up all the numbers or complex numbers divided by `d` in the table, sequence or register at `idx` and pushes a number or complex number onto the stack. With all other at `idx`, issues an error including the name `procname`, the function that called agn_sumupdiv. See also agn_qsumupdiv.

agn_tablesize

```
void agn_tablesize (lua_State *L, int idx, size_t a[])
```

Returns a guess on the number of elements in the array part of the table at stack index `idx` in `a[0]`, an indicator on whether a table contains an allocated hash part `a[1]`, and the number of elements actually assigned to the hash part in `a[2]`.

The function is useful to determine the size of a table much more quickly than the **size** operator does, using a logarithmic instead of linear method, but may return incorrect results if the array part of a table has holes, so the programmer should make sure there are none. See also: **agn_tablestate**.

agn_tablestate

```
void agn_tablestate (lua_State *L, int idx, size_t a[], int mode)
```

Returns the number of key~value pairs allocable and actually assigned to the respective array and hash sections of the table at index `idx` by storing the result in `a`, a C array with nine entries.

The number of key~value pairs currently stored in the array part is stored to a[0], the number of pairs currently stored in the hash part to a[1]. a[2] contains the information whether the array part has holes (1) or not (0). The number of allocable key~value pairs to the array part is stored to a[3], and the number of allocable key~value pairs to the hash part is stored to a[4]. a[5] indicates whether **null** has been set to the table, where 0 = false, and 1 = true. If a[6] is 0, then the table does not feature a metatable, if it is 1 then a metatable has been assigned. a[7]

contains information on whether the hash part of a table does not have an allocated node (no dummynode), a[8] contains a guess on the number of elements in the array part of a table (see **agn_tablesize** for further information). a[9] contains the smallest integral index, and a[10] the largest integral index of the table.

If mode is not 1, then the number of pairs actually assigned is not determined, which may save time. In this case a[0] = a[1] = a[2] = 0.

agn_tabpurge

```
LUA_API int agn_tabpurge (lua_State *L, int idx, int len, int pos)
```

Removes a value at position `pos` from the array part of a table at stack index `idx` and closes the space by shifting down other elements, if necessary. `pos` must be positive. `len` is the length of the array part; if `len` is -1, then the function automatically determines it. The function returns 1 on success and 0 otherwise.

agn_tabresize

```
LUA_API void agn_tabresize (lua_State *L, int idx, size_t newsize,
  int checkholes)
```

Resizes the array part of a table at stack index `idx` to the given number `newsize` of pre-allocated slots.

If you do not know whether there are still non-null values beyond the new size in the array part, then set `checkholes` to 1 instead of 0 as otherwise Agena will crash in such situations.

If set to 1, the function will not resize the table if there are any excess non-null values.

agn_tabsubs

```
LUA_API int agn_tabsubs (lua_State *L, int idx, int idxold, int idxnew)
```

In the table at index `idx`, replaces all occurrences of the value at index `idxold` with the value at `idxnew`. The function does not change the stack and returns the number of substitutions made.

agn_tocomplex (non-ANSI versions only)

```
agn_Complex agn_tocomplex (lua_State *L, int idx)
```

Assumes that the value at stack index `idx` is a complex value and returns it as a lua_Number. It does not check whether the value is a complex number.

agn_tointeger

```
lua_Integer agn_tointeger (lua_State *L, int idx)
```

Assumes that the value at stack index `idx` is a number and returns it as an integer, not a float. It does not check whether the value is a number.

The function does not change the stack.

agn_tonumber

```
lua_Number agn_tonumber (lua_State *L, int idx)
```

Assumes that the value at stack index `idx` is a number and returns it as a lua_Number. It does not check whether the value is a number. The values **undefined** and **infinity** are recognised properly.

The function does not change the stack.

agn_tonumberx

```
lua_Number agn_tonumberx (lua_State *L, int idx, int *exception)
```

If the value at stack index `idx` is a number or a string containing a number, it returns it as a lua_Number. The strings or names `'undefined'` and `'infinity'` are recognised properly. If successful, `exception` is assigned to 0.

If the value could not be converted to a number, 0 will be returned, and `exception` is assigned to 1. See also: **agnL_strtonumber**.

agn_tostring

```
const char *agn_tostring (lua_State *L, int idx)
```

Assumes that the value at stack index `idx` is an Agena string and returns it as a C string of type const char *. It does not check whether the value is a string.

If `idx` is negative: due to garbage collection, there is no guarantee that the pointer returned will be valid after the corresponding value is removed from the stack.

agn_udfreeze

```
LUA_API int agn_udfreeze (lua_State *L, int idx, int readonly)
```

Sets the userdata at stack index `idx` into read-only mode if `readonly` is 1, unsets read-only mode if `readonly` is 0, and just returns the current read-only status (1 or 0) if `readonly` is negative.

agn_usedbytes

```
LUAI_UMEM agn_usedbytes (lua_State *L)
```

Returns the number of bytes used by the interpreter.

agnL_checkoption

```
LUALIB_API int agnL_checkoption (lua_State *L, int idx, const char *def,
                                 const char *const lst[], int ignorecase)
```

Like **luaL_checkoption**, but returns an error if a given option at index `idx` is not a string. Returns the index of the option found in `lst`, which may be the position of the default `def` if there is no value at idx. Otherwise issues an error if the option at `idx` is not part of `lst`[], or if `def` is not in `lst`[]. `def` must not be NULL. If `ignorecase` is 1, the function compares option names case-insensitively, and case-sensitively if it is 0. Example:

```
static const char *const datatypes[] = {"uchar", "double", "int32", NULL};
position = agnL_checkoption(L, 2, "double", datatypes, 0);
```

agnL_createpairofnumbers

```
void agnL_createpairofnumber (lua_State *L, lua_Number l, lua_Number r);
```

Deprecated, see **agn_createpairnumbers**.

agnL_datetosecs

```
LUA_API Time64_T agnL_datetosecs (lua_State *L, int idx,
   const char *procname)
```

Takes either

- a table, register, or a sequence of date time values of the form [yy, mm, dd, hh, mm, seconds], or
- six numbers yy, mm, dd, hh, mm, seconds,

and returns the number of seconds elapsed since the begin of the epoch (usually January 01, 1970) as a Time64_t value; idx must be a _positive_ index number.

agnL_fncall

```
lua_Number agnL_fncall (lua_State *L, int idx, lua_Number x,
                        int optstart, int optstop);
```

Pushes the mathematical function at index `idx` and number `x` onto the stack, optionally pushes the numbers at stack positions `optstart` through `optstop` (including) and then calls the function with the values pushed. It always returns the first result of the function call.

The function does not change the stack and reserves its own stack space, so you do not have to call **lua_checkstack** or **luaL_checkstack**.

The function at `idx` should return one number, otherwise an error will be issued. If the function at `idx` is not multivariate, then pass values for `optstart` and `optstop` such that `optstart` > `optstop`.

See also: **agn_ncall**, **agnL_fncallx**.

agnL_fncallx

```
lua_Number agnL_fncallx (lua_State *L, int idx, lua_Number x,
                         int optstart, int optstop, int *error, int quit);
```

Like agnL_fncall, but with more control of error situations: If the result of the function call is not a number, the function returns 0, and sets `error` to 1. If `quit` is 0 the function always returns, returning zero, without issuing an error. If `quit` is 1, the function issues an error if the result of the function call is not a number. If everything went smooth, `error` is 0.

agnL_fneps

```
lua_Number agnL_fneps (lua_State *L, int fidx, lua_Number x, int n, int p,
int q, lua_Number *origh, lua_Number *abserr);
```

Determines an epsilon value by taking the function value f(x) into account, using a divided difference table. Also returns original epsilon estimate before correction in parameter `origh`, and the absolute error in parameter `abserr`.

The function must be at index position `idx`. `n` is the number of iterations and must be positive. `p` is the first index of further arguments to f, `q` the last index of further arguments to f. if `p > q`, no further arguments are evaluated.

agnL_geti

```
int agnL_geti (lua_State *L, int idx, int i);
```

Returns the `i`'th entry in the string, table array, pair, register, sequence or numarray at index `idx` and pushes it onto the top of the stack. The index `i` counts from 1.

agnL_getmetafield

```
int agnL_getmetafield (lua_State *L, int idx, const char *event);
```

Pushes onto the stack the metamethod `event` from the metatable of the object at index `idx`. If the object does not have a metatable, or if the metatable does not have this field, or - unlike **luaL_getmetafield** - if the field does not refer to a procedure, returns 0 and pushes nothing. Examples for `event` are the metamethods `'__index'`, `'__tostring'`, etc.

agnL_getsetting

```
int agnL_getsetting (lua_State *L, int idx,
  const char *const *modenames, const int *mode, const char *procname);
```

For a given string in `modenames` at stack index `idx`, returns the respective integral representation in `mode`. If the string passed cannot be found in `modenames`, issues an error.

Example:

```
static const int mode[] =
  {OP_ADD, OP_SUB, OP_MUL, OP_DIV, OP_INTDIV, OP_MOD, OP_POW, OP_IPOW};

static const char *const modenames[] =
  {"+", "-", "*", "/", "\\", "%", "^", "**", NULL};

int op = agnL_getsetting(L, 1, modenames, mode, "zip");
```

agnL_gettablefield

```
int agnL_gettablefield (lua_State *L, const char *table, const char *field,
   const char *procname, int issueerror);
```

Determines the entry from the table field <`table`>.<`field`> and puts it on top of the stack. `procname` is the name of the function that calls **agnL_gettablefield**.

If `issueerror` is set to 1, then an error will be issued if `table` is not a table. If `issueerror` is set to 0 and `table` is not a table, then no such error will be issued and the global value found is pushed on the stack. In the latter case, the function returns LUA_TNONE-1.

The function returns the Lua/Agena type, an integer (e.g. LUA_TBOOLEAN), in case of success. If the `field` does not exist, LUA_TNIL will be returned and the function instead pushes **null** on top of the stack. See the agena.h source file for the proper type mapping (grep "basic types").

A typical call might look like this:

```
type = agnL_gettablefield(L, "environ", "infolevel",
  "environ.userinfo", 1);

if (type != LUA_TTABLE) {
  /* do something */
}
```

agnL_gettop

```
int agnL_gettop (lua_State *L, const char *message, const char *procname);
```

Returns the number of arguments passed to a function, or issues an error with the given `message` if no argument has been passed.

agnL_iscallable

```
int agnL_iscallable (lua_State *L, int idx)
```

Checks whether the object at stack index `idx` is either a function or a structure with a '__call' metamethod. It returns 1 if it is and 0 otherwise. The function never changes the stack.

agnL_isdlong

```
int agnL_isdlong (lua_State *L, int idx)
```

Checks whether the value at stack index `idx` is a longdouble (see the **long** package) and returns 1 if it is and 0 otherwise.

agnL_optboolean

```
int agnL_optboolean (lua_State *L, int narg, int def)
```

If the value at stack index `narg` is a Boolean, returns this Boolean as an integer: -1 for **fail**, 0 for **false**, and 1 for **true**. If there is no value at index `narg` or if it is **null**, returns `def`. Otherwise, raises an error.

agnL_optinteger

```
lua_Integer agnL_optinteger (lua_State *L, int narg, lua_Integer def)
```

If the function argument `narg` is a number, returns this number cast to a lua_Integer. If this argument is absent or is NULL, returns `def`. Otherwise, raises an error.

The function internally uses **agn_checknumber** which avoids internal calls to other C API auxiliary library functions and thus is somewhat faster than **luaL_optinteger**.

See also: **agnL_optposint**, **agnL_optnonnegint**, **agnL_optuint32_t**.

agnL_optnonnegative

```
lua_Number agnL_optnonnegative (lua_State *L, int narg, lua_Integer def)
```

If the function argument `narg` is a non-negative number (integer or float), returns this number cast to a lua_Number. If this argument is absent or is NULL, returns `def`. Otherwise, raises an error.

See also: **agnL_optpositive**.

agnL_optnonnegint

```
lua_Integer agnL_optnonnegint (lua_State *L, int narg, lua_Integer def)
```

If the function argument `narg` is a positive integer or zero, returns this number cast to a lua_Integer. If this argument is absent or is NULL, returns `def`. Otherwise, raises an error.

See also: **agnL_optinteger**, **agnL_optposint**.

agnL_optnumber

```
lua_Number agnL_optnumber(lua_State *L, int narg, lua_Number d)
```

If the value at stack index `narg` is a number, returns this number. If this stack value is absent or is NULL, returns `d`. Otherwise, raises an error. Contrary to **luaL_optnumber**, **agnL_optnumber** does not try to convert a string to a number.

agnL_optposint

```
lua_Integer agnL_optposint (lua_State *L, int narg, lua_Integer def)
```

If the function argument `narg` is a positive integer, returns this number cast to a lua_Integer. If this argument is absent or is NULL, returns `def`. Otherwise, raises an error.

See also: **agnL_optinteger**, **agnL_optnonnegint**, **agnL_optuint32_t**.

agnL_optpositive

```
lua_Number agnL_optpositive (lua_State *L, int narg, lua_Integer def)
```

If the function argument `narg` is a positive number (integer or float), returns this number cast to a lua_Number. If this argument is absent or is NULL, returns `def`. Otherwise, raises an error.

See also: **agnL_optnonnegative**, **agnL_optuint32_t**.

agnL_optstring

```
const char *agnL_optstring (lua_State *L, int narg,
                                          const char *def)
```

Similar to **luaL_optstring**, but returns an error if a given option is not a string. The length of the optional string is not determined.

agnL_optuint32_t

```
lua_Integer agnL_optuint32_t (lua_State *L, int narg, uint32_t def)
```

Checks for an optional argument at stack position `narg` and returns it if it is a non-negative number; if the given argument is not a non-positive number, issues an error. If `narg` is null, returns `def`.
See also: **agn_checkuint32_t**, **agnL_optinteger**, **agnL_optnonnegint**, **agnL_optposint**.

agnL_pairgetiintegers

```
LUALIB_API void agnL_pairgetiintegers (lua_State *L, const char *procname,
int idx, int validrange, int *x, int *y)
```

Checks for a pair of integers at stack index `idx` for procedure `procname` and returns them in `x` and `y`. If the object at `idx` is not a pair or the pair does not consist of integers, the function issues an error. If `validrange` is set to 1, then the function also checks whether `x <= y` and throws an error otherwise. Regardless of success or failure, the function does not change the stack.

agnL_pairgetiposints

```
LUALIB_API void agnL_pairgetiposints (lua_State *L, const char *procname,
int idx, int validrange, int *x, int *y)
```

Checks for a pair of positive integers at stack index `idx` for procedure `procname` and returns them in `x` and `y`. If the object at `idx` is not a pair or the pair does not consist of integers, the function issues an error. If `validrange` is set to 1, then the function also checks whether `x <= y` and throws an error otherwise. Regardless of success or failure, the function does not change the stack.

agnL_pairgetinonnegints

```
LUALIB_API void agnL_pairgetinonnegints (lua_State *L, const char
*procname, int idx, int validrange, int *x, int *y)
```

Checks for a pair of non-negative integers at stack index `idx` for procedure `procname` and returns them in `x` and `y`. If the object at `idx` is not a pair or the pair does not consist of integers, the function issues an error. If `validrange` is set to 1, then the function also checks whether `x <= y` and throws an error otherwise. Regardless of success or failure, the function does not change the stack.

agnL_pairgetinumbers

```
void agnL_pairgetinumbers (lua_State *L, const char *procname, int idx,
   lua_Number *x, lua_Number *y)
```

For the given Agena procedure `procname`, checks whether the value at *stack index* `idx` is a pair. It then checks whether the left-hand and right-hand side are numbers and returns these numbers in `x` and `y`. Finally, if `idx` is negative, the function pops the pair from the stack, and leaves the stack unchanged otherwise.

If the value at `idx` is not a pair, or if at least one of its operands is not a number, it issues an error.

agnL_pexecute

```
int agnL_pexecute (lua_State *L, const char *str, const char *procname)
```

Executes the operating system command represented by string `str` and puts the output - a string - onto the top of the stack.

The string `procname` indicates the name of the function from which it is called. The function also removes any carriage returns(`'\r'`) from the output.

agnL_pushcoeffs

```
void agnL_pushcoeffs (lua_State *L, LongDoubleArray *a, size_t nops,
   const char *procname)
```

Takes a structure of numbers or long double userdata residing at stack index 1 and pushes onto the stack a userdata LongDoubleArray. Instead of a structure, numbers or long double userdata might be given as separate arguments to the function that calls this API procedure provided they reside on positive stack positions.

agnL_pushhex

```
void agnL_pushhex (lua_State *L, int idx, int classic)
```

Takes the integer residing at stack index `idx` and pushes the hexadecimal representation of this number as a string onto the stack. The integer at `idx` will be removed if `idx` is negative. If `classic` is set to 0, then the string will have a preceding `0x` and all letters are in lower case. If `classic` is 1, there will be no prefix and all letters are in upper case.

agnL_pushvstring

```
int agnL_pushvstring (lua_State *L, const char *str, ...)
```

Takes one or more strings and pushes their concatenation onto the top of the stack The resulting string does not automatically have a newline at its end, so you may have to explicitly add it if you need it.

The last argument must always be NULL !

agnL_readlines

```
void agnL_readlines (lua_State *L, FILE *f,
   const char *procname, int ispipe)
```

Reads in the file or pipe depicted by its handle `f` and pushes its entire contents or output as a string onto the top of the stack. The string `procname` indicates the name of the function from which it is called.

The function also removes any carriage returns (`'\r'`) from the output.

If you read from a pipe, pass 1 for `ispipe`, and 0 otherwise.

Note that you have to open the file or pipe before, and that the function does not close the file or pipe automatically.

agnL_strtocomplex

```
int agnL_strtocomplex(lua_State *L, int idx)
```

Tries to convert the string at stack position `idx`, with `idx` always negative, to a number and if successful replaces the string with that number, otherwise leaves the stack unchanged. Returns 1 if it could convert the string, and 0 otherwise.

See also: **agn_tonumberx**, **agnL_strtonumber**.

agnL_strtonumber

```
int agnL_strtonumber (lua_State *L, int idx)
```

Tries to convert the string at stack position `idx`, with `idx` always negative, to a number and if successful replaces the string with that number, otherwise leaves the stack unchanged. Returns 1 if it could convert the string, and 0 otherwise.

See also: **agn_tonumberx**, **agnL_strtocomplex**.

agnL_strunwrap

```
int agnL_strunwrap(lua_State *L, int idx,
                   const char *delim, size_t delimlen)
```

Removes the wrapping substring `delim` of length `delimlen` from the string at stack position `idx` and replaces it with the resulting string. `idx` must be negative. If the string could be unwrapped, returns 1 and 0 otherwise, where 0 means there was no string at `idx` with the enclosing substring `delim`.

agnL_tonumarray

```
lua_Number *agnL_tonumarray (lua_State *L, int idx, size_t *size,
                             const char *procname, int duplicate,
                             int throwerr)
```

Creates a C double array and puts all numbers in the table, sequence, register or numeric double array at index `idx` into it. If a non-number is part of the structure, an error will be issued for function `procname`. `size` will contain the number of elements in the C array after returning.

With a numeric array, if `duplicate` is set to 1, the function creates a new array, if set to 0 just returns a reference to the input array.

If `throwerr` is set to 1, the function issues an error if it finds a non-number in the input table, sequence or register. You may have to free the return later on.

See also **agnL_fillarray**.

agnL_tostringx

```
const char *(agnL_tostringx) (lua_State *L, int idx, const char *procname);
```

Converts the number, Boolean, **null** or string at index `idx` to a string. If a wrong argument has been passed, the function issues an error with the name of the procedure causing the exception given by `procname`.

lua_absindex

```
int lua_absindex (lua_State *L, int idx);
```

Converts the acceptable index `idx` into an equivalent absolute index (that is, one that does not depend on the stack top).

lua_arith

```
void lua_arith (lua_State *L, int op);
```

Performs an arithmetic operation `op` over the two values (or one, in the case of negations) at the top of the stack, with the value at the top being the second operand, pops these values, and pushes the result of the operation. The function follows the semantics of the corresponding Agena operator (that is, it may call metamethods).

The value of `op` must be one of the following constants:

- OP_ADD: performs addition (+)
- OP_SUB: performs subtraction (-)
- OP_MUL: performs multiplication (*)
- OP_DIV: performs float division (/)
- OP_INTDIV: performs integer division (\)
- OP_MOD: performs modulo (%)
- OP_POW: performs exponentiation (^)
- OP_IPOW: performs integer exponentiation (**)
- OP_UNM: performs mathematical negation (unary -)

See also: **lua_compare**.

lua_checkinteger

```
lua_Number agn_checkinteger (lua_State *L, int idx);
```

Checks whether the value at index `idx` is a number and represents an integral, fraction-free value and returns this number, avoiding cast to a 4-byte integer. An error is raised if the value at `idx` is not a number, or if it is fractional.

See also: agn_checknonposint, agn_checkposint, agn_checkinteger.

lua_compare

```
void lua_compare (lua_State *L, int idx1, int idx2, int op);
```

Compares two Agena values. Returns 1 if the value at index `idx1` satisfies `op` when compared with the value at index `idx2`, following the semantics of the corresponding Agena operator (that is, it may call metamethods). Otherwise returns 0. Also returns 0 if any of the indices is not valid.
The value of `op` must be one of the following constants:

- OP_EQ: compares for equality (==),
- OP_LT: compares for less than (<),
- OP_LE: compares for less or equal (<=).

See also: **lua_arith**.

lua_copy

```
void lua_copy (lua_State *L, int fromidx, int toidx);
```

Copies the element at index `fromidx` into the valid index `toidx`, replacing the value at that position. Values at other positions are not affected.

lua_getfenvi

```
int lua_getfenvi (lua_State *L, int idx, int i);
```

Pushes the value at integral index `i` in the environment table of the userdata or function at index `idx` onto the stack.

lua_geti

```
int lua_geti (lua_State *L, int idx, lua_Integer i);
```

Pushes onto the stack the value t[i], where t is the value at the given index. This function may trigger a metamethod for the '__index' event. Returns the type of the pushed value. See also: **lua_seti**.

lua_getiuservalue

```
void lua_getiuservalue (lua_State *L, int idx, int n);
```

Pushes onto the stack the n-th user value associated with the full userdata at the given index and returns the type of the pushed value.

If the userdata does not have that value or the value at idx is not a userdata, pushes **null** and returns LUA_TNONE.

n should always be 1 as the function would always return LUA_TNONE otherwise.

lua_getwarnf

```
int lua_setwarnf (lua_State *L);
```

Returns the current warning state:

- 0 - warning system is off;
- 1 - ready to start a new message;
- 2 - previous message is to be continued.

See also: **lua_setwarnf**.

lua_hasfield

```
void lua_hasfield (lua_State *L, int idx, const char *k);
```

Checks whether a table at index idx includes a given field, a string key, and returns 0 (false) or 1 (true). Metamethods are ignored. The stack is left unchanged.

See also: **lua_getfield**, **lua_shas**.

lua_iscomplex

```
void lua_iscomplex (lua_State *L, int idx);
```

This macro checks whether the value at stack index `idx` is a complex number. It returns 1 if the value is a complex number, and 0 otherwise. It does *not* pop anything.

lua_newuserdatuv

```
void *lua_newuserdatauv (lua_State *L, size_t size, int nuvalue);
```

This function creates and pushes on the stack a new full userdata, with `nuvalue` associated Agena values, called user values, plus an associated block of raw memory with size bytes. (The user values can be set and read with the functions **lua_setiuservalue** and **lua_getiuservalue**.) In Agena, `nuvalue` should always be 1, otherwise the function will issue an error to avoid segmentation faults.

The function returns the address of the block of memory.

lua_isnone

```
void lua_isnone (lua_State *L, int idx);
```

This macro returns 1 if the given index is acceptable, but not valid, and 0 otherwise.

lua_isreg

```
void lua_isreg (lua_State *L, int idx);
```

This macro checks whether the value at stack index `idx` is a register. It returns 1 if the value is a pair, and 0 otherwise. It does *not* pop anything.

lua_ispair

```
void lua_ispair (lua_State *L, int idx);
```

This macro checks whether the value at stack index `idx` is a pair. It returns 1 if the value is a pair, and 0 otherwise. It does *not* pop anything.

lua_isseq

```
void lua_isseq (lua_State *L, int idx);
```

This macro checks whether the value at stack index `idx` is a sequence. It returns 1 if the value is a sequence, and 0 otherwise. It does *not* pop anything.

lua_isset

```
void lua_isset (lua_State *L, int idx);
```

This macro checks whether the value at stack index `idx` is a set. It returns 1 if the value is a set, and 0 otherwise. It does *not* pop anything.

lua_isyieldable

```
void lua_isyieldable (lua_State *L);
```

This macro returns 1 if the given coroutine can yield, and 0 otherwise.

lua_numbertointeger

```
int lua_numbertointeger (lua_Number n, lua_Integer *p);
```

Tries to convert a float to an integer; the floating-point value `n` must have an integral value. If that value is within the range of integers, it is converted to an integer and assigned to *`p`. The macro results in a boolean indicating whether the conversion was successful. (Note that this range test can be tricky to do correctly without this macro, due to rounding.)

This macro may evaluate its arguments more than once.

lua_pushchar

```
void lua_pushchar (lua_State *L, char c);
```

Pushes character `c` as a string of length 1 onto the stack.

lua_pushcomplex

```
void lua_pushcomplex (lua_State *L, agn_Complex c);
void lua_pushcomplex (lua_State *L, lua_Number c[]);
```

Pushes the complex number `c` onto the stack. The first form is for platforms that have built-in complex numbers (agn_Complex = complex double) and the

second one for propriety complex math (DOS) where a complex number is represented by a C array of two lua_Numbers. See also: agn_pushcomplex.

lua_pushfail

```
void lua_pushfail (lua_State *L);
```

This macro pushes the Boolean value **fail** onto the stack.

lua_pushfalse

```
void lua_pushfalse (lua_State *L);
```

This macro pushes the Boolean value **false** onto the stack.

lua_pushglobaltable

```
void lua_pushglobaltable (lua_State *L);
```

Pushes the global environment onto the stack.

lua_pushundefined

```
void lua_pushundefined (lua_State *L);
```

Pushes the value **undefined** onto the stack.

lua_pushunsigned

```
LUA_API void lua_pushunsigned (lua_State *L, lua_Unsigned u);
```

Pushes an unsigned int (if you do not change the defaults) onto the stack.

lua_pushtrue

```
void lua_pushtrue (lua_State *L);
```

This macro pushes the Boolean value **true** onto the stack.

lua_rawaequal

```
int lua_rawaequal (lua_State *L, int index1, int index2);
```

Returns 1 if the two values in acceptable indices `index1` and `index2` are primitively approximately equal (that is, without calling metamethods, see also **approx**, ~=). Otherwise returns 0. Also returns 0 if any of the indices are non valid.

lua_rawgetiposrelat

```
void lua_rawgetiposrelat (lua_State *L, int idx, size_t i, int n,
  int posrelat);
```

Similar to **lua_rawgeti**, but if key `i` is negative - referring to the `|i|`-th element from the right side of a table array -, then if `posrelat` is 1, it is converted to the respective positive integer position, see **utils.posrelat** for details. `n` is the size of the table array, see **luaL_getn**. If `i` is 0, returns 0.

lua_rawgeti

```
int lua_rawgeti (lua_State *L, int index, lua_Integer n);
```

Pushes onto the stack the value t[`n`], where t is the table at the given index. The access is raw, that is, it does not use the `__index` metavalue.

Returns the type of the pushed value.

lua_rawgetirange

```
void lua_rawgetirange (lua_State *L, int idx, int p, int q);
```

Similar to **lua_rawgeti**, but pushes elements t[`p`] to t[`q`], with `p >= q`, onto the stack. You must ensure that enough stack space has been reserved. `p`, `q` can be negative, 0 or positive.

lua_rawgetp

```
int lua_rawgetp (lua_State *L, int index, const void *p);
```

Pushes onto the stack the value t[k], where t is the table at the given index and k is the pointer `p` represented as a light userdata. The access is raw; that is, it does not use the `__index` metavalue.

Returns the type of the pushed value.

lua_rawset2

```
void lua_rawset2 (lua_State *L, int idx);
```

Similar to **lua_settable**, but does a raw assignment (i.e., without metamethods).

Contrary to **lua_rawset**, only the value is deleted from the stack, the key is kept, thus you save one call to **lua_pop**. This makes it useful with **lua_next** which needs a key in order to iterate successfully.

lua_rawsetiboolean

```
void lua_rawsetiboolean (lua_State *L, int idx, int n, int num);
```

This macro does the equivalent of t[`n`] := `fail`, if num is -1, t[`n`] := `false`, if num is 0, and t[`n`] := `true`, if num is 1. t is the table at the given valid index `idx`, `n` is an integer, and `num` an integer.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetilstring

```
void lua_rawsetilstring (lua_State *L, int idx, int n, const char *str,
   int len);
```

This macro does the equivalent of t[`n`] := string, where t is the table at the given valid index `idx`, `n` is an integer, `str` the string to be inserted and `len` the length of then string.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetikey

```
void lua_rawsetikey (lua_State *L, int idx, int n);
```

Does the equivalent of t[`n`] := k, where t is the table at the given valid index `idx` and k is the value just below the top of the stack.

This function pops the topmost value from the stack and leaves everything else untouched. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetinumber

```
void lua_rawsetinumber (lua_State *L, int idx, int n, lua_Number num);
```

This macro does the equivalent of t[n] := num, where t is the table at the given valid and *negative* index idx, n is an integer, and num an Agena number (a C double).

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

See also: **agn_setinumber**.

lua_rawsetistring

```
void lua_rawsetistring (lua_State *L, int idx, int n, const char *str);
```

This macro does the equivalent of t[n] = str, where t is the table at the given valid index idx, n is an integer, and str a string.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetp

```
void lua_rawsetp (lua_State *L, int index, const void *p);
```

Does the equivalent of t[p] = v, where t is the table at the given index, p is encoded as a light userdata, and v is the value on the top of the stack.

This function pops the value from the stack. The assignment is raw, that is, it does not use the __newindex metamethod.

lua_rawsetstringboolean

```
void lua_rawsetstringboolean
   (lua_State *L, int idx, const char *str, int n);
```

This macro does the equivalent of t[str] := (n == 1), where t is the value at the given valid index idx, str a string, and n an integer.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetstringchar

```
void lua_rawsetstringchar
   (lua_State *L, int idx, const char *str, int v);
```

This macro does the equivalent of t[`str`] := `v`, where t is the value at the given valid index `idx`, `str` a string, and `v` is an integer.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods. See also **lua_rawsetstringstring**.

lua_rawsetstringnumber

```
void lua_rawsetstringnumber
   (lua_State *L, int idx, const char *str, lua_Number n);
```

This macro does the equivalent of t[`str`] := `n`, where t is the value at the given valid index `idx`, `str` a string, and `n` a number.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetstringpairnumbers

```
void lua_rawsetstringpairnumbers
   (lua_State *L, int idx, const char *str, lua_Number x, lua_Number y);
```

This macro does the equivalent of t[`str`] := `x`:`y`, where t is the value at the given valid index `idx`, `str` a string, and `x`:`y` is a pair of the numbers `x` and `y`.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_rawsetstringstring

```
void lua_rawsetstringstring
   (lua_State *L, int idx, const char *str, const char *text);
```

This macro does the equivalent of t[`str`] := `text`, where t is the value at the given valid index `idx`, `str` a string, and `text` is a string.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods. See also **lua_rawsetstringchar**.

lua_reginsert

```
void lua_reginsert (lua_State *L, int idx);
```

Puts the element on top of the Lua stack into the register at stack index `idx`. on the first slot that is currently set to **null**. If there is no free slot, an error will be issued. The value added to the register is popped from the stack thereafter.

lua_regnext

```
int lua_regnext (lua_State *L, int idx);
```

Pops a key from the stack, and pushes the next key~value pair from the register at the given index `idx`. If there are no more elements in the register or the position of the top pointer has been exceeded, then **lua_regnext** returns 0 (and pushes nothing). To access the very first item in a register, put **null** on the stack before (with **lua_pushnil**).

While traversing a register, do not call **lua_tolstring** directly on the key. Recall that **lua_tolstring** changes the value at the given index; this confuses the next call to **lua_regnext**.

lua_rotate

```
void lua_rotate (lua_State *L, int idx, int n);
```

Rotates the stack elements between the valid index `idx` and the top of the stack. The elements are rotated `n` positions in the direction of the top, for a positive `n`, or `-n` positions in the direction of the bottom, for a negative `n`.

The absolute value of n must not be greater than the size of the slice being rotated.

This function cannot be called with a pseudo-index, because a pseudo-index is not an actual stack position.

lua_sdelete

```
void lua_sdelete (lua_State *L, int idx);
```

Deletes the element residing at the top of the stack from the set at stack position `idx`. The element at the stack top is popped thereafter.

Note that you should not delete a value while traversing a set with lua_usnext. This code, however, works:

```
lua_pushvalue(L, idx);
```

```
if (lua_isset(L, -1)) {
  lua_pushnil(L);
  while (lua_usnext(L, -2) != 0) {
    lua_sdelete(L, -3);  /* delete value in the set and pop it */
    agn_poptop(L);       /* pop the key from stack, too, since it cannot
                            be used for next iteration as it has been
                            purged from the set */
    lua_pushnil(L);      /* restart iteration */
  }
}

agn_poptop(L);
```

See also: **lua_sinsert**.

lua_seqgeti

```
void lua_seqgeti (lua_State *L, int idx, int n);
```

Gets the `n`-th item from the sequence at stack index `idx` and pushes it onto the stack. You have to make sure that the index is valid, otherwise there may be segmentation faults.

See also: **lua_seqseti**.

lua_seqrawgetinumber

```
lua_Number lua_seqrawgetinumber (lua_State *L, int idx, int n);
```

Returns the value t[`n`] *as a* lua_Number, where t is a sequence at the given valid index `idx`. If t[`n`] is not a number, the return is `HUGE_VAL`. The access is raw; that is, it does not invoke metamethods.

See also: **agn_seqgetinumber**.

lua_seqinsert

```
void lua_seqinsert (lua_State *L, int idx);
```

Inserts the element on top of the Lua stack into the sequence at stack index `idx`. The element is inserted at the end of the sequence. The value added to the sequence is popped from the stack thereafter.

lua_seqnext

```
int lua_seqnext (lua_State *L, int idx);
```

Pops a key from the stack, and pushes the next key~value pair from the sequence at the given index `idx`. If there are no more elements in the sequence, then **lua_seqnext** returns 0 (and pushes nothing). To access the very first item in a sequence, put **null** on the stack before (with **lua_pushnil**).

While traversing a sequence, do not call **lua_tolstring** directly on the key. Recall that **lua_tolstring** changes the value at the given index; this confuses the next call to **lua_seqnext**.

lua_seqrawget

```
void lua_seqrawget (lua_State *L, int idx, int pushnil);
```

Pushes onto the stack the sequence value t[k], where t is the sequence at the given valid index `idx` and k is the value at the top of the stack. If t[k] does not exist, an error will be issued if `pushnil` = 0, and **null** will be pushed if `pushnil` is non-zero.

This function pops the key from the stack (putting the resulting value in its place). The function does not invoke any metamethods.

lua_seqrawgeti

```
void lua_seqrawgeti (lua_State *L, int idx, size_t n);
```

Pushes onto the stack the sequence value t[`n`], where t is the sequence at the given valid index `idx`.

The function does not invoke any metamethods. Contrary to **lua_rawgeti**, it issues an error if `n` is out of range.

lua_seqrawgetinoerr

```
void lua_seqrawgetinoerr (lua_State *L, int idx, size_t n);
```

Like **lua_seqrawgeti**, but does not throw an error if `n` is out of range and pushes **null** instead.

lua_seqrawgetinoerrrange

```
void lua_seqrawgetinoerrrange (lua_State *L, int idx, int p, int q);
```

Like **lua_seqrawgetinoerr**, but pushes elements t[p] to t[q], with `p >= q`, onto the stack. You must ensure that enough stack space has been reserved. `p`, `q` should be positive.

lua_seqrawset

```
void lua_seqrawset (lua_State *L, int idx);
```

Does the equivalent to s[k] := v, where s is a sequence at the given valid index `idx`, v is the value at the top of the stack, and k is the value just below the top.

This function pops both the key and the value from the stack. It does not invoke any metamethods.

lua_seqrawsetilstring

```
void lua_seqrawsetilstring (lua_State *L, int idx, int n, const char *str,
   int len);
```

This macro does the equivalent of s[`n`] = `string`, where s is the sequence at the given valid index `idx`, `n` is an integer, `str` the string to be inserted and `len` the length of then string.

This function leaves the stack unchanged. The assignment is raw; that is, it does not invoke metamethods.

lua_seqrawseti

```
int lua_seqrawseti (lua_State *L, int idx, int n);
```

Sets the value at the top of the stack to the non-zero and positive index `n` of the sequence at stack index `idx`.

If the value added is **null**, the entry at sequence index `n` is deleted and all elements to the right of the value deleted are shifted to the left, so that their index positions get changed, as well.

The function pops the value from the top of the stack.

If there is already an item at position `n` in the sequence, it is overwritten.

If you want to extend a current sequence, the function allows to add a new item only at the next free index position. Larger index positions are ignored, but the value to be added is popped from the stack, as well. The function returns:

- -2: attempt to add **null** to sequence
- -1: memory allocation error
- 0: index out of range
- 1: success

See also: **lua_seqrawgeti**.

agn_seqsetinumber

```
void agn_seqsetinumber (lua_State *L, int idx, int n, lua_Number num);
```

The function sets the given Agena number `num` to the non-zero and positive index `n` of the sequence at positive or negative stack index `idx`.

lua_seqsetistring

```
void lua_seqsetistring (lua_State *L, int idx, int n, const char *str);
```

This macro sets the given string `str` to the non-zero and positive index index `n` of the sequence at stack index `idx`.

lua_seti

```
void lua_seti (lua_State *L, int idx, lua_Integer n);
```

Does the equivalent to t[n] = v, where t is the value at the given index `idx` and v is the value at the top of the stack.
This function pops the value from the stack. This function may trigger a metamethod for the `'__writeindex'` event. See also: lua_geti.

lua_setmetatabletoobject

```
void lua_setmetatabletoobject (lua_State *L, int idx, const char *k,
  int settype);
```

Sets metatable `k` to the structure at index position `idx`. The function does not change the stack. If `settype` is 1 the user-defined type `k` will also be set, if `settype` is 0 no user-defined type will be set. If `k` is NULL, the metatable will be deleted and if `settype` is 1, the user-defined type will also be removed.

lua_setwarnf

```
void lua_setwarnf (lua_State *L, lua_WarnFunction f, void *ud);
```

Sets the warning function to be used by Lua to emit warnings (see lua_WarnFunction). The `ud` parameter sets the value `ud` passed to the warning function. See also: **lua_getwarnf**.

lua_sinsert

```
void lua_sinsert (lua_State *L, int idx);
```

This macro to **lua_srawset** inserts an item into a set. The set is at the given index `idx`, and the item is at the top of the stack.

This function pops the item from the stack. See also: **lua_sdelete**.

lua_sinsertlstring

```
void lua_sinsertlstring (lua_State *L, int idx, const char *str, size_t l);
```

This macro sets the first l characters of the string denoted by `str` into the set at the given index `idx`.

lua_sinsertnumber

```
void lua_sinsertnumber (lua_State *L, int idx, lua_Number n);
```

This macro sets the number denoted by `n` into the set at the given index `idx`.

lua_shas

```
void lua_shas (lua_State *L, int idx, int pop);
```

Checks whether the value at the stack top exists in the set at stack index `idx`. Returns 0 or 1. If `pop` is 1, pops the value, otherwise leaves the stack unchanged.

See also **lua_hasfield**.

lua_sinsertstring

```
void lua_sinsertstring (lua_State *L, int idx, const char *str);
```

This macro sets the string denoted by `str` into the set at the given index `idx`.

lua_srawget

```
int lua_srawget (lua_State *L, int idx);
```

Checks whether the set at index `idx` contains the value at the top of the stack. The function pops the value from the stack putting the Boolean value **true** or **false** in its place. It returns 1 if the element has been found, and 0 otherwise.

It does not invoke any metamethods.

lua_srawset

```
void lua_srawset (lua_State *L, int idx);
```

Does the equivalent to `insert` v `into` s, where s is the set at the given valid index `idx`, v is the value at the top of the stack.

This function pops the value from the stack. It does not invoke any metamethods. To delete entries, see **lua_sdelete**.

lua_stringtonumber

```
void lua_stringtonumber (lua_State *L, const char *s);
```

Converts the zero-terminated string `s` to a number, pushes that number into the stack, and returns the total size of the string, that is, its length plus one. The conversion can result in an integer or a float, according to the lexical conventions of Agena. The string may have leading and trailing white spaces and a sign.

If the string is not a valid numeral, returns 0 and pushes nothing. (Note that the result can be used as a Boolean, true if the conversion succeeds.)

lua_toboolean

```
int lua_toboolean (lua_State *L, int idx)
```

Converts the value at the given acceptable index to an integer value (-1, 0 or 1).

If the value at `idx` is **null** or **false**, the functions returns 0.
If the value at `idx` is **fail**, the function returns -1.
If the value at `idx` is different from **false**, **fail**, and **null**, the function returns 1.

The function also returns 0 when called with a non-valid index. (If you want to accept only actual Boolean values, use **lua_isboolean** to test the value's type.)

lua_toint32_t

```
int32_t lua_toint32_t (lua_State *L, int idx)
```

Converts the value at the given acceptable index to the signed integral type int32_t. The value must be a number or a string convertible to a number; otherwise, **lua_toint32_t** returns 0.

If the number is not an integer, it is truncated in some non-specified way.

See also: **agnL_optuint32_t**.

lua_usnext

```
int lua_usnext (lua_State *L, int idx);
```

Pops a key from the stack, and pushes the next item twice (!) from the set at the given `idx`. If there are no more elements in the set, then **lua_usnext** returns 0 (and pushes nothing). To access the very first item in a set, put **null** on the stack before (with **lua_pushnil**).

While traversing a set, do not call **lua_tolstring** directly on an item, unless you know that the item is actually a string. Recall that **lua_tolstring** changes the value at the given index; this confuses the next call to **lua_usnext**.

lua_warning

```
int lua_warning (lua_State *L, const char *msg, int tocont);
```

Emits a warning with the given message. A message in a call with `tocont` true should be continued in another call to this function.

luaL_addgsub

```
const void luaL_addgsub (luaL_Buffer *B, const char *s,
                         const char *p, const char *r);
```

In string `s`, replaces any occurrence of the string `p` with the string `r` and adds the resulting string to buffer `B`. Search patterns are ignored.

luaL_argexpected

```
void luaL_argexpected (lua_State *L, int cond, int arg, const char *tname);
```

Checks whether `cond` is true. If it is not, raises an error about the type of the argument `arg` received and the type expected (`tname`) with a standard message (see **luaL_typeerror**).

luaL_checkcache

```
void luaL_checkcache (lua_State *L, int n, const char *procname);
```

Checks whether the cache stack has enough space for further `n` values and extends it if necessary. It issues an error if the stack would exceed LUAI_MAXCSTACK slots or memory allocation failed. Pass L as first argument, not L->C.

luaL_checkint32_t

```
int32_t luaL_checkint32_t (lua_state *L, int narg)
```

Checks whether the function argument `narg` is a number and returns this number cast to an int32_t.

luaL_checklstringornil

```
const char *luaL_checklstringornil (lua_State *L, int idx, size_t *len);
```

Works exactly like luaL_checklstring but also accepts null at stack index `narg`. In this case, the function returns the empty string and len is set to zero.

luaL_checksetting

```
int luaL_checksetting (lua_State *L, int idx,
  const char *const lst[], const char *errmsg);
```

Checks whether the string at stack index `idx` is included in the list `lst`, and returns its position in `lst`, counting from 0.

If it does not find the string in the list, issues the error `errmsg`.

luaL_clearbuffer

```
void luaL_clearbuffer (luaL_Buffer *B)
```

Clears a luaL_Buffer and resets it, does not leave anything on the stack

luaL_isudata

```
int luaL_isudata (lua_State *L, int ud, const char *tname);
```

Checks whether the object at stack position `ud` is a userdata object. If `ud` depicts the position of a userdata, the function also checks whether the metatable - if available - attached to it complies with metatable `tname` which the programmer originally intended to be used by the specific userdata.

The function returns 1 if all checks have been successful, and 0 otherwise.

See also: **luaL_getudata**.

luaL_getsubtable

```
int luaL_getsubtable (lua_State *L, int idx, const char *fname);
```

Ensures that the value t[`fname`], where t is the value at index `idx`, is a table, and pushes that table onto the stack. Otherwise creates a new table, assigned to t[`fname`], and pushes it onto the top of the stack. Returns true (1) if it finds a previous table and false (0) if it creates a new table.

luaL_getudata

```
void *luaL_getudata (lua_State *L, int narg, const char *tname,
                     int *result);
```

Checks whether the function argument `narg` is a userdata of the type `tname`. Contrary to **luaL_checkudata**, it does not issue an error if the argument is not a userdata, and also stores 1 to `result` if the check was successful, and 0 otherwise.

luaL_setfuncs

```
void luaL_setfuncs (lua_State *L, const luaL_Reg *l, int nup);
```

Registers all functions in the array `l` (see `luaL_Reg`) into the table on the top of the stack (below optional upvalues, see next).

When `nup` is not zero, all functions are created with `nup` upvalues, initialized with copies of the `nup` values previously pushed on the stack on top of the library table. These values are popped from the stack after the registration.

luaL_setmetatype

```
void luaL_setfuncs (lua_State *L, int idx);
```

Sets the metatable (if present) and the user-defined type (if existing) of the object at stack index `idx` to the object at the top of the stack. The function does not change the stack.

The following functions have originally been written by Rici Lake for Lua 5.x:

luaL_newref

```
luaRef *luaL_newref (lua_State *L, int idx);
```

Creates a new C reference to the object at stack index `idx`.

luaL_pushref

```
void luaL_pushref (lua_State *L, luaRef *r)
```

Pushes a referenced object onto the stack.

luaL_freeref

```
void luaL_freeref (lua_State *L, luaRef *r)
```

Frees a reference.

luaL_str2d

```
lua_Number luaL_str2d (const char *s, int *overflow)
```

Converts a string `s` to a Agena number.

Following are the C API functions to easily use Luiz Henrique de Figueiredo's `ae` library for evaluating mathematical expressions in C programs based on Agena:

ae_open

```
void ae_open (void)
```

Opens ae to be used. Call it once before calling the others. Does nothing if ae is already open.

ae_close

```
void ae_close (void)
```

Closes ae after use. All variables are deleted. All memory is released. Does nothing if ae is already closed.

ae_set

```
void ae_set (const char *name, double value)
```

Sets the value of a variable. The value persists until it is set again or ae is closed. Undefined variables evaluate to 0.

ae_eval

```
double ae_eval (const char *expression)
```

Evaluates the given expression and returns its value as a double. Once ae has seen an expression, ae can evaluate it repeatedly quickly as it is internally cached.

Returns 0 if there is an error. ae_error returns the error message.

ae_error

```
const char *ae_error (void)
```

Returns the last error message or NULL if there is none.

Sample program:

```
int main (void) {
   double x;
   ae_open();
   ae_set("a", 1.0);
   ae_set("b", -5.0);
   ae_set("c", 6.0);
   printf("%s\t%g\n", "a", ae_eval("a"));
   printf("%s\t%g\n", "b", ae_eval("b"));
   printf("%s\t%g\n", "c", ae_eval("c"));
   printf("%s\t%s\n", "x", "a*x^2+b*x+c");


   for (x=0.0; x < 4.0; x += 0.25) {
     ae_set("x", x);
     printf("%g\t%g\n", x, ae_eval("a*x^2+b*x+c"));
   }
```

```
   ae_close();
   return 0;
}
```

Appendices

Appendix A

A1 Operators

Unary operators are:

`&&`, `~~`, `||`, `^^`, `abs`, `antilog2`, `antilog10`, `arccos`, `arcsec`, `arcsin`, `arctan`, `assigned`, `atendof`, `bea`, `cis`, `conjugate`, `copy`, `cos`, `cosh`, `cosxx`, `cube`, `empty`, `entier`, `even`, `exp`, `filled`, `finite`, `first`, `flip`, `float`, `lngamma`, `imag`, `infinite`, `int`, `integral`, `last`, `left`, `ln`, `mulup`, `nan`, `nargs`, `nonzero`, `not`, `odd`, `qmdev`, `qsumup`, `real`, `recip`, `reg`, `right`, `sumup`, `seq`, `sign`, `signum`, `sin`, `sinc`, `sinh`, `size`, `square`, `sqrt`, `tan`, `tanh`, `type`, `unassigned`, `unique`, `times`, `typeof`, `values`, `zero`, `-` (unary minus), `~~` (bitwise complement).

Binary operators are:

`and`, `in`, `intersect`, `minus`, `nand`, `nor`, `or`, `roll`, `split`, `squareadd`, `subset`, `symmod`, `union`, `xor`, `xnor`, `xsubset`, `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `*%` (percentage) `/%` (ratio), `+%` (add percentage), `-%` (subtract percentage), %% (percentage change), `\` (integer division), `%` (modulo), `^` (exponentiation), ** (integer exponentiation), `&` (concatenation), `=` (equality), `~=` (approximate equality), `~<>` (approximate inequality), `<` (less than), `<=` (less or equal), `>` (greater than), `>=` (greater or equal), `@` (mapping), `$` (selection), `$$` (fulfillment), `:` (pair constructor), `!` (complex constructor), `&&` (bitwise and), `||` (bitwise or), `^^` (bitwise xor), `<<` (bitwise left-shift), `>>` (right-shift), `<<<` (bitwise left-rotation), `>>>` (right-rotation), `&+` (add 4-byte integer), `&-` (subtract 4-byte integer), `&*` (multiply 4-byte integer), and `&/` (devide 4-byte integer), `|` (comparison), `~|` (approximate comparison), `|-` (absolute difference).

A2 Metamethods

The following metamethods were inherited from Lua 5.1:

Index to metatable	Meaning
'__index'	read operation using indices, e.g. if n[1] = 0 then ... or if n[2 to 3] = 'JP' then ...
'__gc'	Garbage collection, for userdata only
'__add'	Addition of two values
'__sub'	Subtraction of two values
'__mul'	Multiplication of two values
'__div'	Division of two values
'__mod'	Modulus
'__pow'	Exponentiation
'__unm'	Unary minus
'__eq'	Equality operation
'__lt'	Less-than operation
'__le'	Less-than or equals operation
'__concat'	Concatenation
'__call'	Lets a structure a be called as a function, e.g. a(...)
'__tostring'	Method for pretty printing values at stdout
'__metatable'	Write-protection for metatables
'__weak'	Declaration of weak tables, sets, and sequences

Table 20: Metamethods taken from Lua

The **__len** metamethod in Lua 5.1 to determine the size of an object was replaced with the **__size** metamethod. Lua's **__mode** metamethod which sets the weakness of a table has been renamed **__weak**.

The following methods are new in Agena:

Index to metatable	Meaning
'__abs'	**abs** operator
'__absdiff'	**|-** binary operator
'__aeq'	approximate equality operator (~=)
'__antilog2'	**antilog2** operator
'__antilog10'	**antilog10** operator
'__arccos'	**arccos** operator
'__arcsec'	**arcsec** operator
'__arcsin'	**arcsin** operator
'__arctan'	**arctan** operator
'__band'	**&&** operator
'__bor'	**||** operator
'__bxor'	**^ ^** operator
'__bnot'	**~~** operator
'__bshl'	**<<** operator
'__bshr'	**>>** operator
'__brotl'	**<<<** operator

Index to metatable	Meaning
'__brotr'	>>> operator
'__cis'	**cis** operator
'__conjugate'	**conjugate** operator
'__cos'	**cos** operator
'__cosh'	**cosh** operator
'__cube'	**cube** operator
'__eeq'	strict equality operator (==)
'__empty'	**empty** operator
'__entier'	**entier** operator
'__even'	**even** operator
'__exp'	**exp** operator
'__filled'	**filled** operator
'__finite'	**finite** operator
'__flip'	**flip** operator
'__frac'	**frac** operator
'__fractional'	**fractional** operator
'__imag'	**imag** operator
'__in'	**in** binary operator
'__infinite'	**infinite** operator
'__int'	**int** operator
'__intdiv'	integer division
'__intersect'	**intersect** operator
'__integral'	**integral** operator
'__invsqrt'	**invsqrt** operator
'__ipow'	exponentiation with an integer power
'__iter'	iteration of userdata and closures in **for/in** loops
'__left'	**left** operator
'__ln'	**ln** operator
'__lngamma'	**lngamma** operator
'__log'	**log** operator
'__minus'	**minus** operator
'__mulup'	**mulup** operator for table, register or sequence-based user-defined types or userdata
'__nan'	**nan** operator
'__nonzero'	**nonzero** operator
'__notin'	**notin** operator
'__oftype'	self-defined type check for **::**, **-:**, and parameter lists of procedures
'__qmdev'	**qmdev** operator for table, register or sequence-based user-defined types or userdata
'__qsumup'	**qsumup** operator for table, register or sequence-based user-defined types or userdata
'__real'	**real** operator
'__recip'	**recip** operator
'__right'	**right** operator
'__sign'	**sign** operator

Index to metatable	Meaning
'__signum'	**signum** operator
'__sin'	**sin** operator
'__sinc'	**sinc** operator
'__sinh'	**sinh** operator
'__size'	**size** operator
'__sqrt'	**sqrt** operator
'__square'	**square** operator
'__squareadd'	**squareadd** operator
'__sumup'	**sumup** operator for table, register or sequence-based user-defined types or userdata
'__tan'	**tan** operator
'__tanh'	**tanh** operator
'__union'	**union** operator
'__writeindex'	write operation using indices, e.g. n[1] := 0
'__zero'	**zero** operator

Table 21: Metamethods introduced with Agena

Procedures support the '__index', '__writeindex', '__tostring', '__in', '__notin', '__filled', '__empty', '__union', '__minus', '__intersect', '__eq', '__aeq', '__eeq' and '__size' metamethods only.

Let us rehearse how to use metamethods: Imagine you want to add and multiply each element in one table t1 with the respective element in another table t2:

```
> t1, t2 := [1, 2, 3], [4, 5, 6];
```

We define two operations for addition (__add metamethod) and multiplication (__mul metamethod) and put them into metatable mt. In this example we use the short procedure definition for addition and the full procedure definition for multiplication, but both are equivalent:

```
> mt := [
>   __add = <: u :: table, v :: table -> zip(<: x, y -> x + y :>, u, v) :>,
>   __mul = proc(u :: table, v :: table) is
>               return zip(<: x, y -> x * y :>, u, v)
>           end
> ]
```

We register metatable mt with the two tables t1, t2. Actually it suffices to do this with only one of them, t1 or t2:

```
> setmetatable(t1, mt);

> setmetatable(t2, mt);
```

Here we go:

```
> t1 + t2:
[5, 7, 9]

> t1 * t2:
[4, 10, 18]
```

For more information on metamethods and metatables, see Chapter 6.19 Overloading Operators with Metamethods.

A3 Mathematical Constants

Constant	Meaning
degrees	Factor 1/π*180 to convert radians to degrees
DoubleEps	Equals 2.2204460492503131E-16
Eps	Equals 1.4901161193847656e-08
hEps	Equals 1.4901161193847656e-12
EulerGamma	Euler-Mascheroni constant, equals 0.57721566490153286061
E, Exp	Constant e = exp(1) = 2.71828182845904523536
I	Imaginary unit $\sqrt{-1}$
infinity	Infinity ∞
Pi	Constant π = 3.14159265358979323846
Pi2	Constant 2π = 6.28318530717958647692
PiO2	Constant $\pi/2$ = 1.57079632679489661923
PiO4	Constant $\pi/4$ = 0.78539816339744830962
radians	Factor π/180 to convert degrees to radians
undefined	An expression stating that it is undefined, e.g. a singularity
Phi	Golden ratio $(1+\sqrt{5})/2$
InvPhi	Inverse Golden ratio $1/((1+\sqrt{5})/2)$
math.largest	Largest representable number; the smallest negative one nearest to $-\infty$ is the negative of this constant
math.smallest	Smallest positive representable number
math.smallest-normal	Smallest positive normal number
math. lastcontint	Largest integer i representable on the floating-point system with enough precision, such that i - 1 <> i; equals 2^{53}
math.exa	10^{18}
math.peta	10^{15}
math.tera	10^{12}
math.giga	10^{9}
math.mega	10^{6}
math.kilo	10^{3}
math.deka	10^{1}
math.deci	10^{-1}
math.centi	10^{-2}
math.milli	10^{-3}
math.micro	10^{-6}
math.nano	10^{-9}
math.pico	10^{-12}
math.femto	10^{-15}
math.atto	10^{-18}

Table 27: Constants

A4 System Variables

Agena lets you configure the following settings, where `n/e` means `no effect`.

System variable	Meaning	Write
libname	The paths to Agena libraries.	yes
mainlibname	The path to the main Agena directory.	yes
environ.cpu	Contains the name of the CPU in use as a lower-case string, e.g. `'sparc'`, `'ppc'` for PowerPC, or `'x86'` for Intel 386-compatible processors. See also system variable **environ.os**.	no
environ.homedir	The path to the user's home directory.	yes
environ.gdidefaultoptions	A table with all default plotting options for some functions in the **gdi** package. This table is set by **gdi.setoptions**.	no
environ.libpatchlevel	The update version of the main Agena library (in lib/library.agn). Mostly defaults to **null**.	no
environ.maxpathlength	The maximum number of characters for a file path (excluding C's \0 character).	no
environ.more	The number of entries in tables and sets printed by **print** and the end-colon functionality before issuing the `press any key` prompt. Default is 40.	yes
environ.os	Contains the name of the operating system in use as a lower-case string, e.g. `'windows'`, `'macosx'`, `'solaris'`, `'os/2'`, `'haiku'`, `'dos'`, or `'linux'`. Do not change this value. See also system variable **environ.cpu**.	no
environ.release	A sequence containing the string `AGENA`, the main interpreter version as a number, the subversion as a number, and the C patch number as a number, as well. The lib/library.agn patch level is denoted by the fourth entry, or 0 if non-existent. Do not change **environ.release**. See also system variables **_RELEASE** and **environ.version**.	no
environ.version	Similar to **environ.release**, but contains version information represented by a float, not including the lib/library.agn patch level.	no
environ.withprotected	A set of names (passed as strings) that cannot by overwritten by the **with** function. Currently the names `nextone`, `print`, `with`, `write`, `read`, `writeline` have been assigned.	yes

System variable	Meaning	Write
environ.withverbose	If set to **false**, the **with** function will not display warnings, the initialisation string, and the short names assigned. Default is **true**.	yes
Digits	Number of digits in the *output* of Agena numbers. If set to a non-negative integer by the user, it will always supersede the environ.kernel('digits') setting in the entire session.	yes

_G	A table holding all currently assigned global names and their values, and itself. You can add or delete entries by simple table assignment or unassignment, e.g. to delete the **print** function in the current session, just enter:
	`> delete print from _G`
	`> print('Klöße !')`
	`Error in stdin, at line 1:`
	`attempt to call global `print` (a null value)`
		yes

_PROMPT	Defines the prompt Agena displays at the console. If unassigned, by default the prompt is `'> '`.	yes
_RELEASE	Release information on the installed Agena release, returned as a string, e.g. 'AGENA >> 2.2.0'. See also system variables **environ.release** and **environ.version**.	no
nargs	Number of arguments actually passed in a function call, including varargs	no
procname	Refers to the function currently invoked, can be used in recursive calls	no

Table 22: System variables

All environ.* settings are reset by the **restart** statement to their original defaults, whereas those settings the user defines with the **environ.kernel** function will never be modified or deleted by a **restart**.

Some of the default settings can be found at the bottom of the lib/library.agn file.

See also:

- Chapter 14.2 for a description of the **kernel** functions for other settings.
- Appendix A5 for settings that control how Agena outputs data at the console.

A5 Command-Line Usage & Scripting

Agena can be used in the command line as follows:

```
agena [options] [script [arguments]]
```

This means that any option, a script name, and the arguments are all optional. If you just enter

```
shell> agena
```

Agena is started in interactive mode immediately.

There are two ways to run an Agena script with some arguments and then return to the command line immediately without entering interactive mode:

A5.1 Using the `-e` Option

We may write a script with a text editor, e.g. one to print the sine of a number, for example the following two lines:

```
n := n or Pi;  # if n is not set from the shell, just assign Pi to n
writeline(sin(n));
```

When using the `-e` option, we first assign the desired number to a variable and then call the script by its name:

```
shell> agena -e "n := Pi/2" sin.agn
1
```

Note that you first have to enter the `-e` option along with the assignment statement, and then the name of the script.

A much better alternative is this:

A5.2 Using the Internal args Table and Exit Status

When you start Agena with a script or by giving the `-e` option at the command line, everything you pass to the interpreter from the command line is stored in the **args** table.

The name of the script is always stored at index 0, the arguments are stored at the positive indices 1, 2, etc., in the order given by the user. The name of the Agena binary and any options are accessible via negative keys. The name of the interpreter binary is always at the smallest index.

Consider the following script called 'args.agn':

```
for i, j in args do
   writeline(i, j, delim='\t')
od;
```

If you run it, the output will be:

```
shell> agena -m args.agn 1.1 2.2 3.3
4'194'303 KBytes of physical RAM free.

1       1.1
2       2.2
3       3.3
-2      agena
-1      -m
0       args.agn
```

Just play around with this a little bit.

Let us use our new knowledge: The script 'ln.agn' requires at least one number and calculates the corresponding natural logarithm. The numbers entered at the command line are put into the **args** table as strings, so you should convert them back into numbers first. The number of actual arguments - without script name, options and Agena binary file name - are stored in **nargs**. **os.exit** passes an exit code to the shell, if needed.

```
# Evaluate the natural logarithm of given numbers

if nargs < 1 then
   print('Error, need at least one number');
   os.exit(-1)  # just return error exit code
fi

rc := 0;  # exit/return code of the script

for i to nargs do  # iterate each argument at the command line
   x := tonumber(args[i]);
   if x -: number then x := 0 fi;  # if conversion failed, set x to 0
   r := ln(x);
   if r = undefined then rc := 1 fi;  # there was an arithmetic error
   writeline('ln(', x, ') = ', r)
od;

/* clear interpreter state, perform garbage collection & return exit code:
   -1 = no arguments given
    0 = okay
    1 = domain error or wrong type of argument */

os.exit(rc, true);
```

Use it:

```
shell> agena ln.agn 0 1 2
ln(0) = undefined
ln(1) = 0
ln(2) = 0.69314718055995
```

You will find sample scripts in the 'share/scripting' directory of your Agena distribution. The folder also includes batch files to start the scripts from a shell in OS/2

Warp 4.5 (e.g. 'whereis.cmd'), DOS and Windows (e.g. 'whereis.bat'). You may also be interested in the **os.getcommandline** function.

A5.3 Running a Script and then Entering Interactive Mode

The `-i` option allows you to enter the interactive level after running a script or passing other options to Agena. The position of the `-i` option is free. The following shell statement resets the Agena prompt and starts the interpreter:

```
shell> agena -i -e "_PROMPT := 'AGENA> '"
AGENA>
```

A5.4 Running Scripts in UNIX and Mac OS X

If you use Agena in UNIX and Mac OS X, then you can execute Agena scripts directly by just entering the name of the script followed by any arguments (if needed).

Just insert the following line at the head, i.e. the very first line, of each script:

```
#!/usr/local/bin/agena
```

and set the appropriate rights for the script file (e.g. `chmod a+x scriptname`).
An example:

```
bash> ./sin.agn 1
0.8414709848079
```

In all other operating systems, the first line is ignored by the interpreter, so you do not have to delete the first line of the script in order to use scripts you have originally written under UNIX or Mac.

Please make sure that the file is stored in UNIX line break format - and not Windows line breaks.

A5.5 Converting an Agena Script into a Binary Executable

The Windows, Solaris, Linux and Mac OS X binary installers feature a tool originally written by Luiz Henrique de Figueiredo for Lua 5.1, converting an Agena script into a binary executable, for example for Windows and UNIX:

```
srglue sragena whereis.agn whereis.exe
```

```
srglue /usr/local/bin/sragena whereis.agn whereis ; chmod +x whereis
```

In Windows, you find the two utilities in the `bin` folder of your Agena installation or in the `usr/local/bin` folder in UNIX. Make sure you have the `libagena.so` (UNIX) or `agena.dll` C library file somewhere in your path. Note that all the functions in the

main lib/library.agn file are not available but you may copy procedures from there into your script if necessary.

A5.6 Command Line Switches

The available switches are:

Option	Function
-e "stat"	execute string "stat" (double quotes needed)
-h	help information
-i	enter interactive mode after executing `script` or other options
-l	print licence information
-m	print the amount of free RAM at start-up
-n	do not run initialisation file(s) agena.ini / .agenainit at start-up or **restart**
-p path	sets <path> to **libname**, overriding the standard initialisation procedure for this environment variable. The path does not need to be put in quotes if it does not contain spaces.
-r name	readlib library <name>. The name of the library does not need to be put in quotes.
-a	ignore AGENAPATH environment variable, setting **libname** by searching the file system.
-s "text"	issue the slogan "text" at start-up
-S	do not display copyright notice at start-up
-B	throw a syntax error when a numeric constant is too big
-C	allow constants to be overwritten
-D integer	sets the number of digits used in the output of numbers. Note that this setting does not affect the precision of arithmetic operations. The default is 14.
-b	print strings in backquotes
-q	print strings in single quotes
-Q	print strings in double quotes
-v	show version information and compilation time
-x	does not read the main library file lib/library.agn art start-up or **restart**
--	stop handling options
-	execute stdin and stop handling options

Table 23: Command line options

Instead of a preceding hyphen you can also use a slash, e.g. `agena /d` and `agena -d` for debugging mode are accepted.

A6 Define Your Own Printing Rules for Types

You can tell Agena how to output strings, tables, sets, sequences, pairs, and complex values at the console.

With each call to the internal printing routine, the interpreter uses the respective **environ.aux.print*** function or settings defined in the lib/library.agn file. You may change these functions or settings according to your needs.

Table index	Type	Functionality
environ.aux.printtable	function	defines how to print a table, overriding the built-in default
environ.aux.printlongtable	function	defines how to print a table if **kernel/longtable** has been set **true**
environ.aux.printset	function	defines how to print a set, overriding the built-in default
environ.aux.printsequence	function	defines how to print a sequence, overriding the built-in default
environ.aux.printpair	function	defines how to print a pair, overriding the built-in default
environ.aux.printcomplex	function	defines how to print a complex value, overriding the built-in default
environ.aux. printenclosestrings	string	if set, Agena encloses strings with this string; see also: **tostringx**
environ.aux.printprocedure	function	defines how to print a procedure, overriding the built-in default

Table 24: Printing functions

Alternative **environ.aux.print*** functions might look like the following one:

```
> environ.aux.printset := proc(s) is
>   write('set(');
>   if size s > 0 then
>      for i in s do
>         write(i, ', ');
>      od;
>      write('\b\b');
>   fi;
>   write(')');
> end;

> environ.aux.printcomplex := proc(s) is
>    write('cmplx(', real(s), ', ', imag(s), ')');
> end;

> {1, 2}:
set(1, 2)

> 1*2*I:
cmplx(1, 2)
```

A7 The Agena Initialisation File

You can customise your personal Agena environment via special initialisation files.

The initialisation files may include code written agena and will always be executed when Agena is started or **restart**ed. They can include definitions or redefinitions of predefined (environment) variables, and feature self-written procedures or statements to be executed at start-up.

Two kinds of initialisation files are supported:

1. a global initialisation file, and
2. a personal initialisation file for the current user.

Agena first tries to read the global initialisation file, and then the user's initialisation file. If the initialisation files do not exist, nothing happens and Agena starts without errors.

The global initialisation file should reside in the `lib` folder of your Agena installation and is always named `agena.ini` for all operating systems. You may find your Agena installation in `/usr/agena` on UNIX platforms, and usually in `<drive:>/Program Files/Agena` or `<drive:>/Program Files(x86)/Agena` on Windows systems.

In Solaris, Linux, Mac OS X, the personal initialisation file resides in the folder pointed to be the `HOME` environment variable. The personal Agena initialisation file on UNIX machines is called `.agenainit` (not `agena.ini`). Thus the path is `$HOME/.agenainit`.

In Windows, the system environment variable `UserProfile` points to the user's home folder, and the personal initialisation file is called `agena.ini`, (not `.agenainit`), thus the file path is `%UserProfile%/agena.ini`.

On Windows platforms, the user's initialisation file should be put into the user's respective home folder:

Windows version	Path to user's home directory
NT 4.0	`<drive:>\WINNT\Profiles\<username>`
2000, XP, 2003	`<drive:>\Documents and Settings\<username>`
Vista and later	`<drive:>\Users\<username>`

Table 25: Windows' `home` paths

In OS/2 and DOS, Agena tries to find the user's personal `agena.ini` file in the directory pointed to by the environment variable `HOME`, if it has been defined. If HOME has not been defined, it searches in the folder pointed to by the environment variable `USER`, if the latter has been defined. Otherwise, the personal file is not read.

Agena is shipped with a file called `agena.ini.sample` that resides in the `lib` folder of your installation. You can rename it to `agena.ini` or `.agenainit` and play with it - but beware not to overwrite the initialisation which you may already have created.

Here is a sample file:

```
##############################################################################
#                                                                            #
# Agena initialisation file                                                  #
#                                                                            #
##############################################################################

# assign short names for the following library functions:
execute := os.execute;

##############################################################################
# Extend libname to include paths to additional libraries (but only          #
# if directories exist)                                                      #
##############################################################################

if os.isWin() or os.isOS2() or os.isDOS() then
   addpaths := seq(
      'd:/agena/phq',
      'd:/agena/pcomp'
   )
elif os.isSolaris() then
   addpaths := seq(
      '/export/home/proglang/agena/phq',
      '/export/home/proglang/agena/pcomp'
   )
elif os.isLinux() then
   addpaths := seq(
      '~/agena/phq',
      '~/agena/pcomp'
   )
fi;

for i in addpaths do
   if os.exists(i) and i in libname = null then
      libname := libname & ';' & i
   fi
od;

clear addpaths;

writeline('Have fun with Agena !\n');


##############################################################################
# Set default plotting options for gdi.plotfn                                #
##############################################################################

import gdi;
gdi.setoptions(colour~'red', axescolour~'grey');
```

A8 Escape Sequences

Agena supports the following escape sequences known from ANSI C:

Sequence	Meaning
\a	alert
\b	backspace
\e	ESC character (ASCII 27, a POSIX extension)
\f	formfeed
\n	new line
\q	backquote (in backquoted strings only)
\r	carriage return
\t	horizontal tabulator
\u	\u{XXX} (with mandatory enclosing braces), where XXX is a sequence of one or more hexadecimal digits representing the character code point; example: '\u{10FFFF}'
\v	vertical tabulator
\xAB	hex escape
\z	skips subsequent white-space characters, including line breaks; it is particularly useful to break and indent a long literal string into multiple lines without adding the newlines and spaces into the string contents

Table 26: Escape sequences

```
> print('\z
>    1234\z
>    5678')
12345678

> print('{\n\z
>       \x20   \"\104ello\": \"world\"\n\z
>       }')
{
    "hello": "world"
}
```

A9 Some Few Technical Notes

All Solaris binaries of Agena have been created with GCC 9.5.0, the RedHat binaries with GCC 4.9.4, the Debian x86 binaries with GCC 12.2.0 and the AMD64 binaries with GCC 11.2.

All OS/2 binaries have been created with Paul Smedly's GCC great 9.2.0 implementation.

The Windows 32-bit binaries of Agena have been created with MinGW/GCC 9.2.0, the Windows 64-bit binaries have been built with MSYS2/UCRT64 and GCC 16.2.0.

To ensure full backward compatibility and stability down to Windows 2000, recent GCC compilers are not being used for the 32-bit edition. Especially, the **mpfr** MPFR binding is prone to crashes in Windows Vista and earlier when compiled with GCC 10.x or later. GCC 9.5.0 has been found to slow down the interpreter by 3 %, compared to GCC 9.2.0.

The Windows 64-bit edition works fully on Windows 10 and 11. On Windows 8.1 you cannot use the `gdi` package, but otherwise Agena runs flawlessly there, too. Windows 8 and earlier are not supported by the 64-bit edition.

All Mac OS X binaries of Agena have been created with Apple's GCC 4.2.1.

The DOS version is being compiled with DJGPP/GCC 14.2.0.

The C Sources should be ANSI C99 compatible, mostly due to Agena's support of complex arithmetic.

Appendix B

B1 Agena Licence

The Agena source code is licenced under the terms of the following licence:

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicence, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notices and this permission notice shall be included in all copies or portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS' WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

B2 GNU GPL v2 Licence

The Solaris, Linux, Windows, OS/2, Mac OS X, and DOS binaries are distributed under the GNU GPL v2 licence reproduced below:

GNU GENERAL PUBLIC LICENSE
Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA

Everyone is permitted to copy and distribute verbatim copies of this licence document, but changing it is not allowed.

Preamble

The licences for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licence is intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users. This General Public Licence applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by

the GNU Lesser General Public Licence instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licences are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this licence which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licences, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licenced for everyone's free use or not licenced at all.

The precise terms and conditions for copying, distribution and modification follow.

GNU GENERAL PUBLIC LICENSE
TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This Licence applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public Licence. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licencee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this Licence; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this Licence and to the absence of any warranty; and give any other recipients of the Program a copy of this Licence along with the Program.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

a. You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.
b. You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licenced as a whole at no charge to all third parties under the terms of this Licence.
c. If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this Licence. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this Licence, and its terms, do not apply to those sections when you distribute them as separate works. But when

you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this Licence, whose permissions for other licencees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this Licence.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

a. Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
b. Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
c. Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for non-commercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sublicence, or distribute the Program except as expressly provided under this Licence. Any attempt otherwise to copy, modify, sublicence or distribute the Program is void, and will automatically terminate your

rights under this Licence. However, parties who have received copies, or rights, from you under this Licence will not have their licences terminated so long as such parties remain in full compliance.

5. You are not required to accept this Licence, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this Licence. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this Licence to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a licence from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this Licence.

7. If, as a consequence of a court judgement or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this Licence, they do not excuse you from the conditions of this Licence. If you cannot distribute so as to satisfy simultaneously your obligations under this Licence and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent licence would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this Licence would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public licence practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licencee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this Licence.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who

places the Program under this Licence may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this Licence incorporates the limitation as if written in the body of this Licence.

9. The Free Software Foundation may publish revised and/or new versions of the General Public Licence from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of this Licence which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this Licence, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS
How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public Licence as published by
the Free Software Foundation; either version 2 of the Licence, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public Licence for more details.

You should have received a copy of the GNU General Public Licence along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

Gnomovision version 69, Copyright (C) year name of author
Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public Licence. Of course, the commands you use may be called something other than `show w' and `show c'; they could even be mouse-clicks or menu items--whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here is a sample; alter the names:

Yoyodyne, Inc., hereby disclaims all copyright interest in the program
`Gnomovision' (which makes passes at compilers) written by James Hacker.

<signature of Ty Coon>, 1 April 1989
Ty Coon, President of Vice

This General Public Licence does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public Licence instead of this Licence.

B3 Sun Microsystems Licence for the fdlibm IEEE 754 Style Arithmetic Library

```
* ====================================================
* Copyright (C) 1993 by Sun Microsystems, Inc. All rights reserved.
*
* Developed at SunPro, a Sun Microsystems, Inc. business.
* Permission to use, copy, modify, and distribute this
* software is freely granted, provided that this notice
* is preserved.
* ====================================================
```

B4 GNU Lesser General Public Licence

Agena uses the g2 graphic library which is distributed under the GNU LGPL v2.1 licence reproduced below:

GNU LESSER GENERAL PUBLIC LICENSE
Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc.
59 Temple Place, Suite 330, Boston, MA 02111-1307 USA
Everyone is permitted to copy and distribute verbatim copies
of this licence document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public Licence, version 2, hence the version number 2.1.]

Preamble

The licences for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licences are intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users.

This licence, the Lesser General Public Licence, applies to some specially designated software packages--typically libraries--of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this licence or the ordinary General Public Licence is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licences are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this licence, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive licence from a patent holder. Therefore, we insist that any patent licence obtained for a version of the library must be consistent with the full freedom of use specified in this licence.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public Licence. This licence, the GNU Lesser General Public Licence, applies to certain designated libraries, and is quite different from the ordinary General Public Licence. We use this licence for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public Licence therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public Licence permits more lax criteria for linking other code with the library.

We call this licence the "Lesser" General Public Licence because it does Less to protect the user's freedom than the ordinary General Public Licence. It also

provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public Licence for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public Licence.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public Licence is Less protective of the users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

GNU LESSER GENERAL PUBLIC LICENSE
TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This Licence Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public Licence (also called "this Licence"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code

for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this Licence; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this Licence and to the absence of any warranty; and distribute a copy of this Licence along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

a. The modified work must itself be a software library.
b. You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
c. You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this Licence.
d. If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

   (For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this Licence, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this Licence, whose

permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this Licence.

3. You may opt to apply the terms of the ordinary GNU General Public Licence instead of this Licence to a given copy of the Library. To do this, you must alter all the notices that refer to this Licence, so that they refer to the ordinary GNU General Public Licence, version 2, instead of to this Licence. (If a newer version than version 2 of the ordinary GNU General Public Licence has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.
Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public Licence applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this Licence.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this Licence. Section 6 states terms for distribution of such executables.
When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if

the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this Licence. You must supply a copy of this Licence. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this Licence. Also, you must do one of these things:

a. Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)
b. Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.
c. Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.
d. If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

e. Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this Licence, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

a. Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.
b. Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this Licence. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this Licence. However, parties who have received copies, or rights, from you under this Licence will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this Licence, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this Licence. Therefore, by modifying or distributing the Library (or any work based on the
Library), you indicate your acceptance of this Licence to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights

granted herein. You are not responsible for enforcing compliance by third parties with this Licence.

11. If, as a consequence of a court judgement or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this Licence, they do not excuse you from the conditions of this Licence. If you cannot distribute so as to satisfy simultaneously your obligations under this Licence and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this Licence would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this Licence.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this Licence may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this Licence incorporates the limitation as if written in the body of this Licence.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public Licence from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this Licence which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW.
EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Libraries

If you develop a new library, and you want it to be of the greatest possible use to the public, we recommend making it free software that everyone can redistribute and change. You can do so by permitting redistribution under these terms (or, alternatively, under the terms of the ordinary General Public Licence).

To apply these terms, attach the following notices to the library. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the library's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

This library is free software; you can redistribute it and/or

modify it under the terms of the GNU Lesser General Public
Licence as published by the Free Software Foundation; either
version 2.1 of the Licence, or (at your option) any later version.

This library is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
Lesser General Public Licence for more details.
You should have received a copy of the GNU Lesser General Public
Licence along with this library; if not, write to the Free Software
Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the library, if necessary. Here is a sample; alter the names:

Yoyodyne, Inc., hereby disclaims all copyright interest in the
library `Frob' (a library for tweaking knobs) written by James Random Hacker.

<signature of Ty Coon>, 1 April 1990
Ty Coon, President of Vice

That's all there is to it!

B5 SOFA Software Licence

Copyright (C) 2012
Standards Of Fundamental Astronomy Board
of the International Astronomical Union.

==========
SOFA Software Licence
==========

NOTICE TO USER:

BY USING THIS SOFTWARE YOU ACCEPT THE FOLLOWING SIX TERMS AND CONDITIONS WHICH APPLY TO ITS USE.

1. The Software is owned by the IAU SOFA Board ("SOFA").

2. Permission is granted to anyone to use the SOFA software for any purpose, including commercial applications, free of charge and without payment of royalties, subject to the conditions and restrictions listed below.

3. You (the user) may copy and distribute SOFA source code to others, and use and adapt its code and algorithms in your own software, on a world-wide, royalty-free basis. That portion of your distribution that does not consist of intact and unchanged copies of SOFA source code files is a "derived work" that must comply with the following requirements:

a) Your work shall be marked or carry a statement that it (i) uses routines and computations derived by you from software provided by SOFA under license to you; and (ii) does not itself constitute software provided by and/or endorsed by SOFA.

b) The source code of your derived work must contain descriptions of how the derived work is based upon, contains and/or differs from the original SOFA software.

c) The names of all routines in your derived work shall not include the prefix "iau" or "sofa" or trivial modifications thereof such as changes of case.

d) The origin of the SOFA components of your derived work must not be misrepresented; you must not claim that you wrote the original software, nor file a patent application for SOFA software or algorithms embedded in the SOFA software.

e) These requirements must be reproduced intact in any source distribution and shall apply to anyone to whom you have granted a further right to modify the source code of your derived work.

Note that, as originally distributed, the SOFA software is intended to be a definitive implementation of the IAU standards, and consequently third-party modifications are discouraged. All variations, no matter how minor, must be explicitly marked as such, as explained above.

4. You shall not cause the SOFA software to be brought into disrepute, either by misuse, or use for inappropriate tasks, or by inappropriate modification.

5. The SOFA software is provided "as is" and SOFA makes no warranty as to its use or performance. SOFA does not and cannot warrant the performance or results which the user may obtain by using the SOFA software. SOFA makes no warranties, express or implied, as to non-infringement of third party rights, merchantability, or fitness for any particular purpose. In no event will SOFA be liable to the user for any consequential, incidental, or special damages, including any lost profits or lost savings, even if a SOFA representative has been advised of such damages, or for any claim by any third party.

6. The provision of any version of the SOFA software under the terms and conditions specified herein does not imply that future versions will also be made available under the same terms and conditions.

In any published work or commercial product which uses the SOFA software directly, acknowledgement (see www.iausofa.org) is appreciated.

Correspondence concerning SOFA software should be addressed as follows:

By email: sofa@ukho.gov.uk
By post: IAU SOFA Center
HM Nautical Almanac Office
UK Hydrographic Office
Admiralty Way, Taunton
Somerset, TA1 2DN
United Kingdom

B6 MAPM Copyright Remark (Mike's Arbitrary Precision Math Library)

Copyright (C) 1999 - 2007 Michael C. Ring

This software is Freeware.

Permission to use, copy, and distribute this software and its documentation for any purpose with or without fee is hereby granted, provided that the above copyright notice appear in all copies and that both that copyright notice and this permission notice appear in supporting documentation.

Permission to modify the software is granted. Permission to distribute the modified code is granted. Modifications are to be distributed by using the file 'license.txt' as a template to modify the file header. 'license.txt' is available in the official MAPM distribution.

To distribute modified source code, insert the file 'license.txt' at the top of all modified source code files and edit accordingly.

This software is provided "as is" without express or implied warranty.

B7 RSA Security/MD5 Licence

Copyright (C) 1990, RSA Data Security, Inc. All rights reserved.

License to copy and use this software is granted provided that it is identified as the "RSA Data Security, Inc. MD5 Message Digest Algorithm" in all material mentioning or referencing this software or this function.

License is also granted to make and use derivative works provided that such works are identified as "derived from the RSA Data Security, Inc. MD5 Message Digest Algorithm" in all material mentioning or referencing the derived work.

RSA Data Security, Inc. makes no representations concerning either the merchantability of this software or the suitability of this software for any particular purpose. It is provided "as is" without express or implied warranty of any kind.

These notices must be retained in any copies of any part of this documentation and/or software.

B8 David Schultz's Openlibm Licence

Copyright (c) 2011 David Schultz <das@FreeBSD.ORG>
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

B9 ISC Licence

Copyright (c) 2005-2008, Simon Howard

Permission to use, copy, modify, and/or distribute this software for any purpose with or without fee is hereby granted, provided that the above copyright notice and this permission notice appear in all copies.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

B10 Other Copyright Remarks

The Solaris, Linux, Mac OS X, and Windows binaries include code from the gd package which has been published with the following copyright notices:

Portions copyright 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002 by Cold Spring Harbor Laboratory. Funded under Grant P41-RR02188 by the National Institutes of Health.

Portions copyright 1996, 1997, 1998, 1999, 2000, 2001, 2002 by Boutell.Com, Inc.

Portions relating to GD2 format copyright 1999, 2000, 2001, 2002 Philip Warner.

Portions relating to PNG copyright 1999, 2000, 2001, 2002 Greg Roelofs.

Portions relating to gdttf.c copyright 1999, 2000, 2001, 2002 John Ellson (ellson@lucent.com).

Portions relating to gdft.c copyright 2001, 2002 John Ellson (ellson@lucent.com).

Portions copyright 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007 Pierre-Alain Joye (pierre@libgd.org).

Portions relating to JPEG and to color quantization copyright 2000, 2001, 2002, Doug Becker and copyright (C) 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, Thomas G. Lane. This software is based in part on the work of the Independent JPEG Group. See the file README-JPEG.TXT for more information.

Portions relating to WBMP copyright 2000, 2001, 2002 Maurice Szmurlo and Johan Van den Brande.

Permission has been granted to copy, distribute and modify gd in any context without fee, including a commercial application, provided that this notice is present in user-accessible supporting documentation.

This does not affect your ownership of the derived work itself, and the intent is to assure proper credit for the authors of gd, not to interfere with your productive use of gd. If you have questions, ask. "Derived works" includes all programs that utilise the library. Credit must be given in user-accessible documentation.

This software is provided "AS IS." The copyright holders disclaim all warranties, either express or implied, including but not limited to implied warranties of merchantability and fitness for a particular purpose, with respect to this code and accompanying documentation.

Although their code does not appear in gd, the authors wish to thank David Koblas, David Rowley, and Hutchison Avenue Software Corporation for their prior contributions.

Appendix C

C1: Further Reading

A selection of books that helped a lot in recent years when developing Agena:

- Niklaus Wirth: Algorithmen und Datenstrukturen mit Modula-2
- Roberto Ierusalimschy: Programming in Lua
- Roberto Ierusalimschy, Luiz Henrique de Figueirido, Waldemar Celes: Lua 5.1 Reference Manual
- Kurt Jung & Aaron Brown: Beginning Lua Programming
- Jürgen Wolf: C von A bis Z
- Brian W. Kernighan & Dennis M. Ritchie: The C Programming Language
- Federico Biancuzzi & Shane Warden (Ed.): Masterminds of Programming
- Michael. B. Monagan, Keith O. Geddes, K. M. Heal, G. Labahn, S. M. Vorkoetter, J. McCarron, P. DeMarco: Maple 7 Programming Guide
- Brian "Beej Jorgensen" Hall, Beej's Guide to Network Programming, Using Internet Sockets
- Jan Jones: QL SuperBASIC - The Definitive Handbook
- Frank G. Pagan: A Practical Guide to Algol68

Index

A

AgenaEdit, 977
ArcaOS, 58, 96, 792, 918, 936, 942, 944, 951, 952, 954, 958, 977, 1101, 1188
Arguments
  Dropping, 30
  Extracting, 49
Arithmetic, 405
  80-bit floating-point, 716
  Absolute Difference, 412
  Absolute Value, 412, 414, 418, 497, 499, 504, 516, 533, 572, 720, 743, 1001
  Addition, 408, 409, 488, 497, 498, 503, 508, 516, 518, 533, 573, 719, 742, 781, 1000, 1008
  Add-on, 410
  Airy Function, 519
  Approximations, 473
  Arbitrary Precision, 495, 507, 517
  Arithmetic-Geometric Mean, 440, 519
  Automatic Differentiation, 529
  Auxiliary Cosine Integral, 584
  Auxiliary Sine Integral, 584
  Bessel Functions, 417, 520, 720
  Beta Function, 417, 441, 455, 519, 726
  Binary Coded Decimal (BCD), 475
  Binomial Coefficient, 410, 417, 455, 513, 727
  Bitwise Operators, 33, 66, 67, 296, 308, 412, 413, 426, 433, 439, 476, 477, 481, 492, 751, 752, 753, 979
  Box Distribution, 464
  Bronze Fibonacci Numbers, 761
  bytes Library, 475
  Cardinal Cosine, 419, 497, 505
  Cardinal Sine, 437, 497, 499, 504, 505, 729
  Cardinal Tangent, 438, 497, 505
  Cathetus, 418, 519, 720, 1004
  Checking Integers and Floats, 39, 40, 429, 430, 451
  Combinatorics, 756
  Complemented Incomplete Gamma Integral, 602
  Complete Integer Factorisation, 763
  Complex Exponential Function, 419
  Complex Math, 411, 418, 436
  Complex Number Functions, 417, 419, 433
  Complex Values, 467
  Confluent Hypergeometric Function, 601
  Conjugate, 419
  Constants, 454, 466, 470, 976, 977, 1177
  Conversion Functions, 133, 430, 441, 443, 445, 449, 450, 460, 464, 467, 468, 469, 471, 476, 735, 736, 737, 761, 763
  Co-Prime Test, 746, 764
  cordic Library, 569
  Cosecant, 420, 444, 497, 505, 519, 574, 729, 782
  Cosine, 419, 444, 465, 474, 499, 504, 518, 534, 574, 729, 781
  Cosine Integral, 585, 621
  Cotangent, 420, 444, 497, 505, 519, 574, 729, 782
  Cube, 420, 498, 499, 504, 516, 533, 725, 743
  Cubic Root, 418, 519, 533, 726, 746, 782
  Dawson's Integral, 589, 620
  Degrees, 468
  Degrees & Radians, 1177
  Difference or Zero, 448
  Digamma Function, 434, 519, 728
  Dilogarithm, 520, 591
  Dirac Delta Function, 445
  Dirichlet Eta Function, 595
  Discount, 410
  Division, 408, 409, 435, 489, 497, 498, 500, 503, 508, 516, 518, 527, 533, 575, 719, 720, 742, 781, 1009
  divs Library, 525
  Entire Cosine Integral, 585
  Entire Exponential Integral, 593
  Epsilon, 446, 594, 735, 1177
  Error Functions, 421, 427, 497, 519, 535, 575, 623, 731, 782, 1007
  Even Number, 499, 515, 516, 743
  Exponent, 424, 447, 470, 721, 737
  Exponential Functions, 414, 415, 421, 422, 424, 426, 430, 447, 449, 499, 504, 533, 575, 726, 737
  Exponential Integral, 519, 592, 594
  Exponentiation, 411, 420, 423, 438, 447, 497, 508, 518, 533, 575, 577, 719, 721, 725, 726, 781, 1002, 1003, 1005, 1006, 1009
  Factorial, 410, 422, 445, 447, 455, 461, 469, 497, 512, 727

Falling Factorial, 447
Fast Fourier Transform, 767
fastmath Library, 473
Fibonacci Numbers, 512, 761, 762
Floating Point Functions, 423, 444, 446, 452, 453, 459, 465, 720, 721, 732, 733, 735, 737
Floating-Point Classification, 448, 738
Fraction, 423, 525, 732
Fresnel Integral, 598
Fused Multiply-Add, 411, 423, 432, 497, 505, 519, 721, 776, 782, 1004
Gamma Function, 424, 431, 519, 520, 575, 576, 727
Gaussian Function, 599, 601
Generalised Binet Formula, 761
Generalized Exponential Integral, 593
Greatest Common Divisor, 498, 511, 746, 763, 765
Hamming Distance, 450
Harmonic Function, 600, 601
Harmonic-Geometric Mean, 450
Heaviside Function, 425
Higher & Lower Bits, 436, 478, 479
Higher & Lower Bytes, 475, 476, 479
Hyperbolic Cosecant, 420, 497, 505, 519, 575, 730, 782
Hyperbolic Cosine, 419, 466, 499, 504, 518, 534, 574, 729, 730, 781
Hyperbolic Cosine Integral, 587
Hyperbolic Cotangent, 420, 497, 505, 519, 574, 730, 782
Hyperbolic Secant, 436, 497, 505, 520, 577, 730, 783
Hyperbolic Sine, 437, 466, 499, 504, 518, 534, 578, 729, 730, 781
Hyperbolic Tangent, 438, 497, 499, 504, 518, 534, 578, 729, 781
Hypotenuse, 425, 426, 456, 473, 498, 519, 534, 576, 721, 782, 1004
Incomplete Beta integral, 601
Incomplete Gamma integral, 602
Integer Division, 409, 420, 427, 428, 429, 497, 732, 1002, 1009
Integer Factorisation, 762
Integer Functions, 431, 432, 449, 473, 474, 722, 765
Interval Arithmetic, 769
ints Library, 741
Inverse, 435, 474, 509, 720
Inverse Cosecant, 415, 497, 534
Inverse Cosine, 415, 497, 499, 504, 518, 534, 573, 730, 781
Inverse Cotangent, 416, 534, 730
Inverse Gamma Function, 427
Inverse Hyperbolic Cosecant, 415, 519, 730, 782
Inverse Hyperbolic Cosine, 415, 519, 535, 573, 730, 782
Inverse Hyperbolic Cotangent, 416, 519, 730, 782
Inverse Hyperbolic Secant, 416, 519, 730, 782
Inverse Hyperbolic Sine, 416, 519, 535, 573, 731, 782
Inverse Hyperbolic Tangent, 416, 519, 535, 574, 782
Inverse incomplete Beta integral, 607
Inverse Psi Function, 428
Inverse Root, 473
Inverse Secant, 416, 497, 499, 504, 534, 730
Inverse Sine, 416, 499, 504, 518, 534, 573, 731, 781
Inverse Tangent, 416, 499, 504, 518, 519, 534, 573, 731, 781, 782
Iterated Logarithm, 456
Jacobi Symbol, 512, 765
Kahan Compensated Summation, 440
Kahan-Babuška Summation, 56, 72, 440, 452, 675, 682, 685, 689, 691, 708, 711, 712, 714
Kahan-Ozawa Summation, 234, 246, 264, 283, 440, 453, 598, 674, 721, 978
Kronecker Symbol, 512, 765
Kummer's Function, 601
Lambda Function, 610
Lambert W Function, 610
LCM, 454
Least Common Multiple, 498, 512, 747
Least Significant Bit, 476, 514, 752
Legendre Symbol, 512
Logarithmic Factorial, 455, 727
Logarithmic Functions, 426, 430, 431, 455, 456, 471, 474, 497, 511, 520, 533, 576, 727, 728, 747, 754, 782, 1005, 1067
Logarithmic Integral, 593, 610
Lucas Number, 513
Machine Epsilon, 446, 735
Mantissa, 424, 457, 465, 722, 737
mapm Library, 495

math Library, 440
Mathematical Epsilon, 446, 594, 735
Maximum, 748
Mean, 748
Metallic Numbers, 761
MiniMax Functions, 44, 457, 683, 725, 776
Minimum, 748
Modular Exponentiation, 489, 498, 510, 748, 766
Modular Multiplication, 489, 498, 510, 749, 765
Modular Multiplicative Inverse, 498, 510, 764
Modulo, 410, 411, 432, 457, 458, 489, 509, 519, 520, 576, 719, 721, 742, 764, 765, 782, 1002, 1009
Most Significant Bit, 476, 514, 752
mp Library, 507
Multiple, 432, 459, 461, 737
Multiplication, 46, 253, 272, 408, 409, 488, 497, 498, 503, 508, 516, 518, 533, 576, 598, 684, 719, 742, 781, 1000, 1009
Natural Logarithm, 430, 499, 504, 533
Neumaier Summation, 440
Next Power, 459
Non-principle Root, 435, 726
Normal Numbers, 452
Normalisation, 579
Number Evaluation Functions, 421, 422, 433, 439, 458, 460, 732, 733
Odd Number, 499, 515, 516, 743
Operators, 408
Ordinary Hypergeometric Function, 601
Parity, 177, 421, 433, 493, 733, 752
Pell Numbers, 761
Percentage, 409
Percentage Change, 410
Perfect Power, 513
Perfect Square, 513, 764, 765
Perlin Noise, 459
Piecewise-Continuous Function, 461
Pochhammer Function, 447, 461
Polygamma Functions, 434
Polylogarithm, 616
Polynomial, 523, 601, 604, 606, 615, 722
Powers, 411, 452, 459, 461, 462, 497, 498, 503, 516, 533, 742, 747, 748
Premium, 410
Previous Power, 462
Prime Factorisation, 766
Prime Test, 747, 764
Primes, 511, 746, 747, 749, 764, 765, 766
Primorial, 513
Principle Root, 434
Product function, 598
Psi Function, 434, 728
Pythagorean Equation, 434, 456, 722, 1003
Quake III Method, 473
Quotient, 443, 448, 510
Radians, 468
Random Number Generator, 459, 462, 463, 498, 750
Ranges, 414, 441, 443, 460, 464, 471, 579, 660, 706, 736
Ratio, 410
Reciprocal, 435, 474, 497, 499, 504, 518, 533, 720, 781, 1001
Rectangular Pulse Function, 464
Remainder Function, 420, 510
Riemann Zeta Function, 521, 628
Rising Factorial, 447
Root Functions, 433, 434, 435, 438, 461, 497, 511, 725, 750, 1003, 1004
Rotation, 410
Rounding Functions, 39, 418, 419, 420, 423, 427, 431, 436, 439, 441, 443, 458, 465, 470, 519, 520, 576, 722, 731, 732, 782, 783, 979
Rounding Mode, 979
Round-Off Errors, 440, 452, 453, 721
Secant, 436, 465, 497, 505, 520, 577, 729, 783
Sexagesimal Values, 467, 469
Shrinking to Zero, 442, 732
Sign, 414, 436, 437, 444, 448, 449, 458, 465, 497, 499, 504, 516, 519, 520, 533, 577, 578, 720, 737, 743, 782, 783
Sine, 437, 465, 466, 474, 499, 504, 518, 534, 578, 729, 781
Sine Integral, 620, 621, 623, 624
Spence's Function, 591
Square, 423, 438, 498, 499, 504, 516, 518, 533, 721, 725, 743, 781
Square Root, 428, 438, 452, 474, 497, 499, 504, 516, 533, 578, 725, 726, 743
Subnormal Numbers, 452, 460, 471, 735, 738

Subtraction, 408, 409, 412, 448, 488, 497, 498, 503, 508, 516, 518, 533, 578, 719, 742, 781, 1009
Summation, 56, 71, 217, 220, 253, 254, 256, 272, 273, 275, 408, 409, 632, 685, 711
Surd, 435
Symmetric Modulo, 411
Symmetric Modulus, 429, 448
Tangent, 438, 467, 499, 504, 518, 534, 578, 729, 781
Tetragamma Function, 434, 728
Trailing Zeros, 753
Triangular Function, 469
Trigamma Function, 434, 728
Trigonometric & Related Functions, 415, 416, 418, 419, 420, 436, 437, 438, 462, 465, 466, 473, 474, 497, 504, 505, 729, 730, 731, 1006, 1007
Truncation, 39, 420, 423, 427, 431, 436, 439, 458, 465, 470, 732
Type Punning, 754
Unit of Least Precision (ULP), 470
Zeroing, 442, 732
ASCII85
Decoding, 1063
Encoding, 1065
Assignment
Checking for Assigned Names, 22, 76
Assumptions, 22

B

Base32
Decoding, 1063
Encoding, 1065
Base64, 93
Decoding, 1063
Encoding, 1065
Base85
Decoding, 1064
Encoding, 1065
Base85 Z85 (ZeroMQ)
Decoding, 1064
Encoding, 1066
Bit Fields, 144, 147, 150, 155
Booleans, 38, 39
Bitwise Complement and, 433
Bitwise Complement or, 433
Bitwise Complement xor, 439
Relational Operators, 98, 104, 106, 119, 121, 122, 447, 449, 450, 452, 454, 457, 459

C

C API Functions, 1101
agn_absindex, 1102
agn_arrayborders, 1102
agn_arraypart, 1102
agn_borders, 1103
agn_ccall, 1103
agn_checkboolean, 1104
agn_checkcomplex, 1104
agn_checkinteger, 1104
agn_checklstring, 1104
agn_checknonnegative, 1105
agn_checknonnegint, 1105
agn_checknumber, 1105
agn_checkposint, 1105
agn_checkpositive, 1105
agn_checkstring, 1105, 1106
agn_checkuint16_t, 1106
agn_checkuint32_t, 1106
agn_cleanse, 1106
agn_cleanseset, 1106
agn_complexiimag, 1107
agn_complexreal, 1107
agn_copy, 1107
agn_createcomplex, 1107
agn_createpair, 1107
agn_createpairnumbers, 1107
agn_createpairstrings, 1108
agn_createreg, 1108
agn_createrable, 1108
agn_createseq, 1108
agn_createset, 1108
agn_deletefield, 1108
agn_deletertable, 1109
agn_entries, 1109
agn_equalref, 1109
agn_fnext, 1109
agn_getbitwise, 1110
agn_getcmplxparts, 1110
agn_getconstants, 1110
agn_getdblepsilon, 1110
agn_getduplicates, 1110
agn_getempytline, 1111
agn_getepsilon, 1111
agn_getfunctiontype, 1111
agn_gethepsilon, 1111

agn_getinumber, 1111
agn_getlibnamereset, 1111
agn_getlongtable, 1112
agn_getround, 1112
agn_getrtable, 1112
agn_getrtablewritemode, 1112
agn_getseqlstring, 1112
agn_getstorage, 1113
agn_getutype, 1113
agn_hasarraypart, 1113
agn_hashashpart, 1113
agn_hashpart, 1113
agn_hsize, 1103
agn_in, 1114
agn_initmethodcall, 1114
agn_intindices, 1114, 1115
agn_isfail, 1115
agn_isfalse, 1115
agn_isinteger, 1116
agn_islinalgvector, 1116
agn_isnonnegint, 1116
agn_isnumber, 1116
agn_isposint, 1116
agn_issequtype, 1117
agn_issetutype, 1117
agn_isstring, 1117
agn_istableutype, 1117
agn_istrue, 1117
agn_isutype, 1118
agn_isutypeset, 1118
agn_malloc, 1118
agn_ncall, 1118
agn_numintersect, 1119
agn_numminus, 1119
agn_numunion, 1119
agn_onexit, 1119
agn_optcomplex, 1119
agn_pairgeti, 1120
agn_pairgetiall, 1120
agn_pairgetiintegers, 1120
agn_pairgetinumbers, 1120
agn_pairrawget, 1120
agn_pairrawset, 1120
agn_pairseti, 1121
agn_pairstate, 1121
agn_parts, 1121
agn_poptop, 1121
agn_poptoptwo, 1121
agn_pushboolean, 1122
agn_pushcomplex, 1122
agn_qsumupdiv, 1122
agn_rawgetfield, 1122
agn_rawgeticomplex, 1122
agn_rawgetifield, 1123
agn_rawgetiinteger, 1123
agn_rawgetinumber, 1123
agn_rawgetistring, 1123
agn_rawinsert, 1124
agn_rawinsertfrom, 1124
agn_rawsetfield, 1124
agn_reggetinoerr, 1124
agn_reggetinoerrrange, 1124
agn_reggetinumber, 1124
agn_reggettop, 1125
agn_regpurge, 1125
agn_regrawget, 1125
agn_regrawgeti, 1125
agn_regrawgeticomplex, 1125
agn_regrawgetiinteger, 1126
agn_regrawgetinumber, 1126
agn_regrawgetistring, 1126
agn_regresize, 1126
agn_regset, 1127
agn_regseti, 1127
agn_regsetinumber, 1127
agn_regsettop, 1127
agn_regstate, 1127
agn_regsubs, 1128
agn_seqrawgeticomplex, 1128
agn_seqrawgetiinteger, 1128
agn_seqrawgetinumber, 1128
agn_seqrawgetistring, 1129
agn_seqresize, 1129
agn_seqsize, 1129
agn_seqstate, 1129
agn_seqsubs, 1129
agn_setbitwise, 1130
agn_setconstants, 1130
agn_setdblepsilon, 1130
agn_setduplicates, 1130
agn_setemptyline, 1130
agn_setepsilon, 1131
agn_sethepsilon, 1131
agn_setinumber, 1131
agn_setlibnamereset, 1131
agn_setlongtable, 1131
agn_setreadlibbed, 1132
agn_setresize, 1132
agn_setround, 1132
agn_setrtable, 1132
agn_setstorage, 1132
agn_setudmetatable, 1133
agn_setutype, 1133
agn_size, 1133

agn_ssize, 1103, 1133
agn_sstate, 1134
agn_stralloc, 1134
agn_strmatch, 1134
agn_structinsert, 1134
agn_sumup, 1135
agn_sumupdiv, 1135
agn_tablesize, 1135
agn_tablestate, 1135
agn_tabpurge, 1136
agn_tabresize, 1136
agn_tabsubs, 1136
agn_tocomplex, 1137
agn_tointeger, 1137
agn_tonumber, 1137
agn_tonumberx, 1137
agn_tostring, 1137
agn_udfreeze, 1138
agn_usedbytes, 1138
agnL_checkoption, 1138
agnL_createpairofnumbers, 1138
agnl_datetosecs, 1138
agnL_fncall, 1139
agnL_fncallx, 1139
agnL_fneps, 1139
agnL_geti, 1140
agnL_getmetafield, 1140
agnL_getsetting, 1140
agnL_gettablefield, 1140
agnL_gettop, 1141
agnL_iscallable, 1141
agnL_isdlong, 1141
agnL_optboolean, 1142
agnL_optinteger, 1142
agnL_optnonnegative, 1142
agnL_optnonnegint, 1142
agnL_optnumber, 1143
agnL_optposint, 1143
agnL_optpositive, 1143
agnL_optstring, 1143
agnL_optuint32_t, 1143
agnL_pairgetiintegers, 1144
agnL_pairgetinonnegints, 1144
agnL_pairgetinumbers, 1144
agnL_pairgetiposints, 1144
agnL_pexecute, 1145
agnL_pushcoeffs, 1145
agnL_pushhex, 1145
agnL_pushvstring, 1145
agnL_readlines, 1145
agnL_strtocomplex, 1146
agnL_strtonumber, 1146
agnL_strunwrap, 1146
agnL_tonumarray, 1146
agnL_tostringx, 1147
lua_absindex, 1147
lua_arith, 1147
lua_checkinteger, 1148
lua_compare, 1148
lua_copy, 1148
lua_getfenvi, 1148
lua_geti, 1149
lua_getiuservalue, 1149
lua_getwarnf, 1149
lua_hasfield, 1149
lua_iscomplex, 1150
lua_isfail, 1115
lua_isfalse, 1115
lua_isfalseorfail, 1115
lua_isnilfalseorfail, 1115
lua_isnone, 1150
lua_ispair, 1150
lua_isreg, 1150
lua_isseq, 1151
lua_isset, 1151
lua_istrue, 1117
lua_isyieldable, 1151
lua_newuserdatauv, 1150
lua_numbertointeger, 1151
lua_pushchar, 1151
lua_pushcomplex, 1151
lua_pushfail, 1152
lua_pushfalse, 1152
lua_pushglobaltable, 1152
lua_pushtrue, 1152
lua_pushundefined, 1152
lua_pushunsigned, 1152
lua_rawaequal, 1153
lua_rawgeti, 1153
lua_rawgetiposrelat, 1153
lua_rawgetirange, 1153
lua_rawgetp, 1153
lua_rawset2, 1154
lua_rawsetiboolean, 1154
lua_rawsetikey, 1154
lua_rawsetilstring, 1154
lua_rawsetinumber, 1155
lua_rawsetistring, 1155
lua_rawsetp, 1155
lua_rawsetstringboolean, 1155
lua_rawsetstringchar, 1156
lua_rawsetstringnumber, 1156
lua_rawsetstringpairnumbers, 1156
lua_rawsetstringstring, 1156

lua_reginsert, 1157
lua_regnext, 1157
lua_rotate, 1157
lua_sdelete, 1157
lua_seqgeti, 1158
lua_seqinsert, 1128, 1158
lua_seqnext, 1159
lua_seqrawgeti, 1159
lua_seqrawgetinoerr, 1159
lua_seqrawgetinoerrrange, 1160
lua_seqrawgetinumber, 1158
lua_seqrawset, 1160
lua_seqrawseti, 1160
lua_seqrawsetilstring, 1160
lua_seqsetinumber, 1161
lua_seqsetistring, 1161
lua_seti, 1161
lua_setmetatabletoobject, 1161
lua_setwarnf, 1162
lua_shas, 1162
lua_sinsert, 1162
lua_sinsertlstring, 1162
lua_sinsertnumber, 1162
lua_sinsertstring, 1163
lua_srawget, 1163
lua_srawset, 1163
lua_stringtonumber, 1163
lua_toboolean, 1163
lua_toint32_t, 1164
lua_usnext, 1164
lua_warning, 1164
luaL_addgsub, 1165
luaL_argexpected, 1165
luaL_checkcache, 1165
luaL_checkint32_t, 1165
luaL_checklstringornil, 1165
luaL_checksetting, 1165
luaL_clearbuffer, 1166
luaL_freeref, 1167
luaL_getsubtable, 1166
luaL_getudata, 1166
luaL_isudata, 1166
luaL_newref, 1167
luaL_pushref, 1167
luaL_setfuncs, 1167
luaL_setmetatype, 1167
luaL_str2d, 1167
Calculus, 581
Airy Wave Functions, 583, 584
Arc Length, 583
Automatic Differentiation, 529
Chandrupatla's Algorithm, 585
Chebyshev Coefficients, 586
Chebyshev Interpolant, 586, 587
Chebyshev Polynomial, 587
Clenshaw-Curtis-Quadrature, 603
Complete Elliptic Integral 1st Kind, 593
Complete Elliptic Integral 2nd Kind, 594
Continuity, 607
Curvature, 588, 622
Differentiability, 608
Differentiation, 586, 587, 590, 595, 618, 624, 777
Double Exponential Transformation, 603
Euclidian Distance, 595
Exponential Sum Function, 596
Extrema, 596, 597, 612
Fresnel Integral, 598
Gauss-Legendre Integration, 599
Golden Section Search, 597
Gudermannian Function, 600
Horner's Rule, 601
Incomplete Elliptic Integral 1st Kind, 593
Incomplete Elliptic Integral 2nd Kind, 594
Incomplete Gamma Function, 598
Integration, 599, 600, 603, 604, 605, 606, 617, 618, 621, 622
Interpolation, 450, 455, 587, 588, 606, 611, 613, 614, 615
Inverse Logistic Function, 611
Inverse Sigmoid Functions, 611
Jacobian Elliptic Functions, 609
Limit, 583, 610
Logistic Function, 611, 621
Lower Incomplete Gamma Function, 598
Maximum, 596, 597
Mean of a Function, 612
Minimum, 596, 597
Modified Bessel Function of Order One, 584
Modified Bessel Function of Order Zero, 584
Points of Inflection, 602
Pole-Finding, 608, 614
Polynomial Coefficients, 523, 614, 615, 722
Products, 46, 598
Riemann Sum, 617
Romberg Integration, 617
Root-Finding, 584, 585, 608, 616, 625, 626, 777, 778
Saddle Points, 618
Savitzky-Golay Filter, 618, 619

Series, 72, 598, 599
Sigmoid Functions, 416, 421, 600, 611, 621, 623, 731
Smoothstep Function, 622
Softsign Function, 623
Spline, 587, 613
Standard Logistic Function, 621
Summation, 598
Sums, 72, 598, 599
Upper Incomplete Gamma Function, 598
Weierstraß Function, 623
Weight Function, 426
Zeros, 584, 585, 608, 616, 620, 625, 626
Checksum
BSD Checksum, 166
Damm Algorithm, 167
Luhn Algorithm, 174
UNIX cksum, 166
Verhoeff Algorithm, 182
clear Statement, 27
Codepages, 135, 136, 161, 920
1252, 96
850, 96
Combinatorics
Bell Number, 756
Bell Polynomial, 756
Bernjoulli Polynomial, 756
Bernoulli Number, 756
Cartesian Product, 756
Catalan Number, 757
Catalan Polynomial, 757
Combinations, 757
Euler Number, 757
Euler Polynomial, 757
Fibonacci Polynomial, 758
Number of Combinations, 758
Number of Partitions, 758
Number of Permutations, 758
Permutations, 758
Stirling Number of 1st Kind, 759
Stirling Number of 2nd Kind, 759
Command Line Switches, 1183
Command Line Usage, 1180
Arguments, 936, 974, 1180
Complex Numbers, 39, 423, 1103
Argument, 417, 505
Cartesian Notation, 411, 418
Conjugate, 419
Creation, 411
Escape-time Fractals, 1047
Imaginary Error Function, 421
Imaginary Part, 426
Imaginary Unit, 1177
Magnitude, 21, 411, 418, 433
Phase Angle, 411, 417, 505
Polar Notation, 433
Printing Values Close to Zero, 980
Real Part, 435
Rotation, 410
Scaled Complementary Error Function, 421
Compression
gzip Package, 859
minizip Package, 862
Configuration, 975, 1178, 1183, 1185
Complex Number Output, 1184
Debugging Information, 975, 976
Number of Digits on Output, 976
Pair Output, 1184
Procedure Output, 1184
Prompt, 976, 979
Sequence Output, 1184
Set Output, 1184
Table Output, 978, 1178, 1184
console, 52, 58, 79, 96, 111, 804, 929, 950, 1037, 1052, 1084, 1133, 1179, 1182, 1183
Colour, 54
Command Line Switches, 1183
Command Line Usage, 1180
Configuring the Output, 1183
Geometry, 959, 960
Output Mode, 936, 957
Running a Script, 1182
Constants
DoubleEps, 1177
Eps, 1177
EulerGamma, 1177
Euler-Mascheroni, 1177
Euler's, 575
Exp (e), 1177
Golden Ratio, 73, 502, 739, 1177
hEps, 1177
I, 1177
infinity, 1177
Inverse Golden Ratio, 1177
Pi, 577, 1177
Pi2, 1177
PiO2, 1177
PiO4, 1177
radians, 1177
undefined, 1177

Coordinates
Altitude, 542
Azimuth, 542
Declination, 542
Equatorial, 542
Horizontal, 542
Right Ascension, 542
CORDIC, 569
Coroutines, 1015
Counting
in Structures, 20, 224, 245, 260, 280
CPU
ARM, 941, 942
eax, ebx, ecx, edx Registers, 921
Endianness, 928
Features, 921
Load Averages, 923
PowerPC, 944
Vendor, 966
x86-compatible, 946
xgetbv Register Instruction, 968
CSV Files
skycrane.readcsv, 1086
utils.readcsv, 1071
utils.writecsv, 1080

D

Data Types
AVL Trees, 343
Bags/Multisets, 334
Bit Fields, 356
C, 982
Doubly-linked Lists, 330
First-in First-out Queues, 369
Last-in First-out Queues, 372
Lookup Tables, 363
Numeric Arrays, 288
Tuples, 359
Unrolled Singly-linked Lists, 327
Database, 838
dBASE III/IV, 820
SQLite, 879
Date & Time, 219, 445, 467, 469, 543, 548, 923, 926, 953, 955, 961, 962, 963, 1063, 1067, 1089
Autumn Equinox, 541
Calendar week, 543, 549
CPU Time, 920
Current System Time, 542
Daylight Saving Time, 548, 943
Excel Serial Date, 924, 928, 948, 954
Fractional Day, 567
Greenwich Apparent Sidereal Time, 563
Gregorian Date, 549
Gregorian Leap Year, 548
Hebrew Calendar, 546
Julian Date, 542, 549, 554, 924, 954
Leap Seconds, 567
Local Shiraz Time, 554
Local Siderial Time, 564
Lotus Serial Date, 924, 928, 948, 954, 1077
Moon Altitude, 556
Moon Phase, 553
Moonrise & Moonset, 552, 553
Nowruz Persian New Year, 565
Persian Calendar, 554, 556
Persian Day of Month, 556
Persian Day of Week, 556
Persian Leap Year, 548
Persian Month, 556
Planet Altitude, 556
Seasons, 563
Setting System Clock, 959
Spring Equinox, 565
Summer Solstice, 565
Sun Altitude, 556
Sunrise & Sunset, 565, 566
Timestamps, 128, 131, 1077
UTC, 924, 964
Validation, 548
Winter Solstice, 568
dBASE Files
xbase.readdbf, 829
dBASE III/IV, 820
Debugging, 1016
Default Input File
Files, 791
DOS, 58, 96, 942, 948, 977

E

Endianness, 479, 481, 818, 928, 982, 1022
Environment
Exit Handler, 25
Quitting the Interpreter, 25
Restart Handler, 62
Restarting the Interpreter, 62
Errors
Assumptions, 22

Catching Errors, 54, 80
Issuing Errors, 22, 30
Escape Sequences, 1187

F

File System Access
Attributes, 934
Changing Directories, 919
Changing Mode, 919
Changing Owner, 920
Creating Directories, 950
Creating Symbolic or Hard Links, 950
Current Working Directory, 919
Directories, 923, 926, 934, 942, 945, 950, 957
Drives, 927
Files, 929, 930, 931, 932, 953, 956, 957, 960
Inode, 941
IPC ID, 935
Windows System Directories, 945
Files
Attributes, 931, 934
Changing Time Stamp, 931
Closing Files, 789, 808, 809
Compressed Files, 859, 862
Copying Files, 932, 1083
Counting, 921
CSV Files, 1080, 1085
DBF Files, 820
Default Input File, 791
Directory Listing, 946, 947
End Of File, 790, 809, 822
Existence, 929, 930
File Descriptor, 790
File Handles, 788, 792, 810
File Name Syntax Validation, 946
Flushing Files, 816
Getting and Setting File Positions, 790, 795, 801, 802, 803, 809, 815, 816
INI Files, 874, 1074, 1081
Key Files, 880
Locking Files, 794, 803, 810, 816
Maximum Path Length, 979
Moving Files, 953, 1085
Opening Files, 792, 796, 808, 811
Path Separator, 979
Reading Files, 793, 798, 800, 811, 812, 813, 814, 815
Removing Files, 957
Rewinding Files, 801
Searching in Files, 791
Size, 790, 796
SQLite Files, 879
Streams, 788
Symbolic Links, 956, 960
TAR Files, 857
Temporary Filename, 963
Terminal Device, 792
Truncation, 803, 816
utils.readcsv, 1071
utils.readxml, 1075
utils.writecsv, 1080
utils.writexml, 1082
Writing Files, 804, 817, 818, 819
XML Files, 849
ZIP Files, 862
Filters
Bloom Filter for Strings, 183
Cuckoo Filter for Numbers, 378
Cuckoo Filter for Strings, 187
Fractals
Colour Map, 1051, 1052, 1054
Dragon Curve, 1047
Escape-time Fractals, 1047
Fern, 1048
Henon Map, 1048
Iterated Function System, 1048
Julia Set, 1049
Julia Sets, 1046
Koch Curve, 1049
Logistic Bifurcation, 1050
Mandelbrot Set, 1050
Minkowski Island, 1051
Newton's Formula, 1047
Functional Programming, 31, 59, 90, 91, 1093
$ Operator, 19, 223, 224, 244, 245, 260, 279
$$ Operator, 19, 260, 279
$$$ Operator, 20, 224, 245, 260, 280
@ Operator, 18, 278
calc.fprod, 598
calc.fsum, 598
Currying, 1093
descend, 29, 214, 240, 250, 269
factory.anyof, 1093
factory.count, 1091
factory.cycle, 1092
factory.iterate, 1092
factory.pick, 1094
factory.reset, 1092

fold, 31
foreach Operator, 32
has, 35, 215, 241, 252, 271, 286
ints.reverse, 752
long.count, 718
map, 42
mulup Operator, 46
numarray.remove, 306
numarray.select, 308
pipeline, 52
recurse, 58, 218, 241, 254, 273
reduce, 59
remove, 61
satisfy, 63
select, 64
selectremove, 65
stats.fprod, 684
stats.fsum, 685
sumup Operator, 71
times Operator, 73
zip, 80, 314
Functions & Operators
-, 408, 631
---, 412
!, 410
$, 223, 244, 260, 279
$ Operator, 19
$$, 224, 245, 260, 279
$$ Operator, 19
$$$ Operator, 20
%, 88, 410
-%, 410
%%, 410
&, 87
&-, 409
&&, 412
&*, 409
&/, 409
&+, 409
*, 408, 631
*%, 409
**, 411
/, 88, 408
/%, 410
@, 42, 223, 244, 259, 278
@ Operator, 18
\, 88, 409
^, 411
^ ^, 413
|, 414
|-, 412
||, 87, 412
~~, 412
~<>, 222
~=, 221, 243, 258, 277, 287
+, 408, 631
+%, 410
+++, 412
<, 87
<<, 87, 413
<<<, 413
<>, 221, 243, 258, 277, 287
=, 221, 242, 257, 276, 287
==, 221, 242, 258, 277, 287
>>, 88, 413
>>>, 413
abs, 21, 89, 414, 518, 632, 720, 781
addtometatable, 21
ads.attrib, 840
ads.close, 841
ads.createseq, 841
ads.desc, 841
ads.filepos, 841
ads.find, 841
ads.free, 842
ads.getall, 842
ads.getkeys, 842
ads.getvalues, 842
ads.index, 843
ads.indices, 843
ads.invalids, 843
ads.iterate, 843
ads.lock, 844
ads.new, 844
ads.open, 846
ads.openfiles, 846
ads.peekin, 846
ads.read, 846
ads.remove, 847
ads.retrieve, 847
ads.sizeof, 847
ads.sync, 847
ads.unlock, 847
ads.write, 848
allotted, 992
alternate, 21
antilog10, 415, 726, 781
antilog2, 414, 726, 781
append, 21
approx, 415
arccos, 415, 518, 730, 781
arccosh, 415
arccot, 416, 730
arccoth, 416

arccsc, 415
arccsch, 415
arcsec, 416
arcsech, 416
arcsin, 416, 518, 731, 781
arcsinh, 416
arctan, 416, 518, 731, 781
arctan2, 416
arctanh, 416
argument, 417
aspell.check, 204
aspell.close, 204
aspell.config, 204
aspell.new, 206
aspell.pass, 207
aspell.suggest, 207
assert, 22
assigned, 22
assume, 22
astro.a2af, 541
astro.a2tf, 541
astro.autumn, 541
astro.cal2jd, 542
astro.cdate, 542
astro.coords, 542
astro.cweek, 543
astro.cweekmonsun, 543
astro.d2tf, 543
astro.dectodms, 543
astro.dmstodec, 544
astro.gethourangle, 544
astro.getposition, 545
astro.h2jd, 546
astro.hdate, 546
astro.illumination, 547
astro.isdst, 548
astro.isleapyear, 548
astro.isvaliddate, 548
astro.isvalidtime, 548
astro.jdate, 549
astro.lastcweek, 549
astro.locate, 549
astro.lunareclipse, 551
astro.lunarlibration, 552
astro.moon, 552
astro.moonphase, 553
astro.moonriseset, 553
astro.p2jd, 554
astro.pdate, 554
astro.peak, 555
astro.pmonth, 556
astro.pweekday, 556
astro.searchaltitude, 556, 559
astro.searchhourangle, 557
astro.searchlunarapsis, 559
astro.searchplanetapsis, 561
astro.searchriseset, 562
astro.seasons, 563
astro.shabbat, 563
astro.siderial, 563
astro.solareclipse, 564
astro.spring, 565
astro.summer, 565
astro.sun, 565
astro.sunriseset, 566
astro.taiutc, 567
astro.tf2d, 567
astro.winter, 568
atendof, 87, 89
augment, 22, 212, 249, 268
avl.attrib, 347
avl.entries, 347
avl.get, 347
avl.getmax, 347
avl.getmin, 347
avl.getminmax, 347
avl.getroot, 348
avl.getstore, 348
avl.include, 348
avl.new, 348
avl.remove, 349
bags.attrib, 335
bags.bagtoset, 335
bags.getsize, 335
bags.include, 335
bags.minclude, 336
bags.new, 336
bags.remove, 336
bea, 417
besselj, 417
bessely, 417
beta, 22, 417
bfield.clearbit, 357
bfield.flipbit, 357
bfield.getbit, 357
bfield.getbyte, 357
bfield.new, 357
bfield.resize, 358
bfield.setbit, 358
bfield.setbitto, 358
bfield.setbyte, 358
bimaps.attrib, 338
bimaps.bimap, 338
bimaps.countitems, 338

bimaps.entries, 338
bimaps.indices, 339
bimaps.map, 339
bimaps.rawget, 339
bimaps.remove, 340
bimaps.select, 340
bimaps.subs, 341
bimaps.subsop, 341
binary.entries, 344
binary.explore, 344
binary.find, 344
binary.get, 345
binary.include, 345
binary.indices, 345, 348
binary.iterate, 345, 348
binary.new, 345
binary.remove, 346
binary.reorder, 346
binio.chars, 812
binio.clearerror, 808
binio.close, 809
binio.eof, 809
binio.ferror, 809
binio.fgetpos, 809
binio.filepos, 809
binio.fsetpos, 809
binio.isfdesc, 810
binio.length, 810
binio.lines, 810
binio.lock, 810
binio.open, 811
binio.readbytes, 811
binio.readchar, 812
binio.readindex, 813
binio.readint64, 813
binio.readlong, 813
binio.readlongdouble, 814
binio.readshortstring, 814
binio.readstring, 815
binio.readuint64, 815
binio.rewind, 815
binio.seek, 815
binio.sync, 816
binio.toend, 816
binio.truncate, 816
binio.unlock, 816
binio.writebytes, 816
binio.writechar, 817
binio.writeindex, 817
binio.writeint64, 817
binio.writelong, 818
binio.writelongdouble, 818
binio.writenumber, 818
binio.writeshortstring, 818
binio.writestring, 819
binio.writeuint64, 819
binomial, 417
binpack.bool, 202
binpack.cdouble, 202
binpack.chars, 202
binpack.dd, 202
binpack.double, 203
binpack.dump, 202
binpack.int32, 203
binpack.int64, 203
binpack.longdouble, 203
binpack.new, 202
binpack.nil, 203
binpack.pack, 202
binpack.tostring, 202
binpack.uchar, 203
binpack.uint16, 203
binpack.uint32, 203
binpack.unpack, 202
binsearch, 23
bintersect, 24, 212, 249, 268
bisequal, 24, 212, 249, 268
bloom.attrib, 184
bloom.find, 185
bloom.get, 185
bloom.include, 185
bloom.new, 186
bloom.toseq, 186
bminus, 24, 212, 249, 268
bnor, 433
bottom, 25, 213
bye, 25
bytes.add32, 488
bytes.and32, 490
bytes.arshift32, 490
bytes.bcd, 475
bytes.bintogray, 490
bytes.cast, 482
bytes.castint, 475
bytes.div32, 489
bytes.divmod32, 489
bytes.extract32, 490
bytes.fpbtoint, 475
bytes.getdouble, 482
bytes.gethigh, 483
bytes.getieee, 486
bytes.getieeedouble, 487
bytes.getlow, 483
bytes.getunbiased, 483

bytes.getwords, 483
bytes.graytobin, 490
bytes.ieee, 485
bytes.interweave, 491
bytes.inttofpb, 476
bytes.isint32, 491
bytes.leastsigbit, 476
bytes.mask32, 492
bytes.mod32, 489
bytes.mostsigbit, 476
bytes.mul32, 488
bytes.muladd32, 488
bytes.mulmod32, 489
bytes.nand32, 492
bytes.nextbit, 492
bytes.nor32, 492
bytes.not32, 492
bytes.numhigh, 475
bytes.numlow, 476
bytes.numto32, 492
bytes.numwords, 476
bytes.onebits, 477
bytes.optsize, 477
bytes.or32, 493
bytes.pack, 477
bytes.packsize, 478
bytes.parity32, 493
bytes.parity8, 493
bytes.powmod, 489
bytes.replace32, 493
bytes.rotate32, 493
bytes.setdouble, 483
bytes.sethigh, 483
bytes.setieee, 486
bytes.setieeedouble, 487
bytes.setieeeexpo, 486
bytes.setieeehigh, 487
bytes.setieeelow, 487
bytes.setieeesignbit, 486
bytes.setlow, 484
bytes.setnumhigh, 478
bytes.setnumlow, 479
bytes.setnumwords, 479
bytes.setwords, 484
bytes.shift32, 494
bytes.sub32, 488
bytes.swap, 479
bytes.swaplower, 479
bytes.swapupper, 479
bytes.tobig, 479
bytes.tobinary, 480
bytes.tobytes, 480
bytes.tochars, 480
bytes.tolittle, 481
bytes.tonumber, 481
bytes.unpack, 481
bytes.xnor32, 494
bytes.xor32, 494
cabs, 418
calc.Ai, 583
calc.aitken, 583
calc.arclen, 583
calc.ausSiCi, 584
calc.bessel0, 584
calc.bessel1, 584
calc.Bi, 584
calc.brent, 584
calc.chandrupatla, 585
calc.cheby, 586
calc.cheby64, 586
calc.chebycoeffs, 586
calc.chebygen, 587
calc.chebyt, 587
calc.Chi, 587
calc.Ci, 585
calc.Cin, 585
calc.clampedspline, 587
calc.clampedsplinecoeffs, 588
calc.curvature, 588
calc.dawson, 589
calc.dct, 589
calc.diff, 590
calc.differ, 590
calc.dilog, 591
calc.dst, 592
calc.Ei, 592
calc.Ein, 593
calc.elliptic1, 593
calc.elliptic2, 594
calc.En, 594
calc.eps, 594
calc.eta, 595
calc.eucliddist, 595
calc.eulerdiff, 595
calc.expn, 596
calc.extrema, 596
calc.fmaxbr, 597
calc.fmaxgs, 597
calc.fminbr, 597
calc.fmings, 597
calc.fprod, 598
calc.fresnelc, 598
calc.fresnels, 598
calc.fsum, 598

calc.gammainc, 598
calc.gauleg, 599
calc.gauleg64, 599
calc.gaussian, 599
calc.gtrap, 600
calc.gtrap64, 600
calc.harmonic, 600
calc.horner, 601
calc.hyp1f1, 601
calc.hyp2f1, 601
calc.ibeta, 601
calc.igamma, 602
calc.igammac, 602
calc.inflect, 602
calc.intcc, 603, 618
calc.intcc64, 603
calc.intde, 603
calc.intde64, 604
calc.intdei, 604
calc.intdei64, 605
calc.intdeo, 605
calc.intdeo64, 605
calc.integ, 606
calc.interp, 606
calc.invibeta, 607
calc.iscont, 607
calc.isdiff, 608
calc.itp, 608
calc.jacobian, 609
calc.lambertw, 610
calc.Li, 610
calc.limit, 610
calc.linterp, 611
calc.logistic, 611
calc.logit, 611
calc.maximum, 612
calc.mean, 612
calc.mean64, 612
calc.minimum, 612
calc.nakspline, 613
calc.naksplinecoeffs, 613
calc.neville, 614
calc.newtoncoeffs, 614
calc.poles, 614
calc.polyfit, 615
calc.polygen, 615
calc.polylog, 616
calc.probit, 616
calc.regulafalsi, 616
calc.riesum, 617
calc.riesum64, 617
calc.romberg, 617
calc.saddles, 618
calc.savgol, 618
calc.savgolcoeffs, 619
calc.scaleddawson, 620
calc.sections, 620, 777
calc.Shi, 620
calc.Si, 621
calc.SiCi, 621
calc.sigmoid, 621
calc.simaptive, 621
calc.simaptive64, 622
calc.sinuosity, 622
calc.smoothstep, 622
calc.softsign, 623
calc.Ssi, 623
calc.variance, 623
calc.w, 623
calc.weier, 623
calc.xpdiff, 624
calc.zeroab, 625
calc.zeros, 626
cartesian, 418
cas, 418
cathet, 418
cbrt, 418
ceil, 418
cell, 992
char, 89
checkoptions, 25
checktype, 26
cis, 419
cleanse, 26, 213, 239, 250, 269
clear, 27
clock.add, 538
clock.adjust, 539
clock.sgstr, 539
clock.sub, 538
clock.tm, 539
clock.todec, 540
clock.totm, 540
columns, 27, 213, 250, 269
com.attrib, 912
com.close, 911
com.control, 913
com.init, 912
com.open, 911
com.purge, 913
com.queues, 913
com.read, 912
com.timeout, 913
com.wait, 913
com.write, 912

combinat.bell, 756
combinat.bernoulli, 756
combinat.cartprod, 756
combinat.catalan, 757
combinat.choose, 757
combinat.euler, 757
combinat.fib, 758
combinat.numbcomb, 758
combinat.numbpart, 758
combinat.numbperm, 758
combinat.permute, 758
combinat.stirling1, 759
combinat.stirling2, 759
conjugate, 419
copy, 27, 213, 240, 250, 269, 286, 632
copyadd, 27, 213, 240, 250, 269
cordic.carccos, 569
cordic.carcsin, 569
cordic.carctan2, 569
cordic.carctanh, 570
cordic.ccbrt, 570
cordic.ccos, 570
cordic.ccosh, 570
cordic.cexp, 570
cordic.chypot, 570
cordic.cln, 570
cordic.cmul, 570
cordic.cpow, 570
cordic.csin, 571
cordic.csinh, 571
cordic.csqrt, 571
cordic.ctan, 571
cordic.ctanh, 571
coroutine.resume, 1015
coroutine.running, 1015
coroutine.setup, 1015
coroutine.status, 1015
coroutine.wrap, 1015
coroutine.yield, 1015
cos, 419, 518, 729, 781
cosc, 419
cosh, 419, 518, 729, 781
cosxx, 419
cot, 420
coth, 420
countitems, 28, 214, 250, 269
csc, 420
csch, 420
cube, 420, 518, 725, 781
cuckoo.attrib, 188
cuckoo.find, 188
cuckoo.include, 188
cuckoo.new, 188
cuckoo.remove, 188
dblhash.attrib, 382
dblhash.compact, 382
dblhash.getindex, 382
dblhash.getitem, 382
dblhash.has, 383
dblhash.hash, 383
dblhash.include, 383
dblhash.iterate, 383
dblhash.new, 384
dblhash.purge, 384
dblhash.toseq, 384
dblhash.totable, 384
dblmap.attrib, 394
dblmap.cleanse, 394
dblmap.getitem, 394
dblmap.include, 394
dblmap.iterate, 394
dblmap.new, 395
dblmap.purge, 395
dblmap.resize, 395
dblmap.totable, 395
dd.arccosh, 782
dd.arccot, 782
dd.arccoth, 782
dd.arccsc, 782
dd.arccsch, 782
dd.arcsech, 782
dd.arcsinh, 782
dd.arctan2, 782
dd.arctanh, 782
dd.cbrt, 782
dd.ceil, 782
dd.checkdd, 782
dd.copysign, 782
dd.cot, 782
dd.coth, 782
dd.csc, 782
dd.csch, 782
dd.erf, 782
dd.erfc, 782
dd.floor, 782
dd.fma, 782
dd.fmod, 782
dd.frexp, 782
dd.get, 782
dd.hypot, 782
dd.isdd, 782
dd.log10, 782
dd.log2, 782
dd.modf, 782

dd.renorm, 782, 783
dd.root, 783
dd.round, 783
dd.sec, 783
dd.sech, 783
dd.signbit, 783
dd.tonumber, 783
dd.tostring, 783
dd.trunc, 783
debug.debug, 1016
debug.funcname, 1016
debug.getconstants, 1016
debug.getfenv, 1016
debug.gethook, 1017
debug.getinfo, 1017
debug.getk, 1017
debug.getlocal, 1018
debug.getlocals, 1018
debug.getmetatable, 1018
debug.getregistry, 1018
debug.getrtable, 1019
debug.getstore, 1019
debug.getupvalue, 1019
debug.getupvalues, 1019
debug.globals, 1020
debug.listcode, 1020
debug.nupvalues, 1020
debug.setfenv, 1020
debug.sethook, 1021
debug.setlocal, 1021
debug.setmetatable, 1021
debug.setstore, 1021
debug.setupvalue, 1022
debug.system, 1022
debug.traceback, 1022
debug.unused, 1022
descend, 29, 214, 240, 250, 269
divs.denom, 527
divs.divs, 527
divs.equals, 527
divs.numer, 527
divs.todec, 528
divs.todiv, 528
dlist.append, 330
dlist.checkdlist, 330
dlist.dump, 330
dlist.getitem, 330
dlist.iterate, 331
dlist.new, 331
dlist.prepend, 331
dlist.purge, 332
dlist.put, 332
dlist.replicate, 332
dlist.setitem, 332
dlist.toseq, 332
dlist.totable, 333
drem, 420
drop, 30
duplicates, 30, 214, 251, 270
empty, 30, 90, 214, 240, 270
entier, 420, 731
environ.anames, 969
environ.arithstate, 969
environ.arity, 969
environ.attrib, 970
environ.callable, 972
environ.decpoint, 972
environ.frozen, 972
environ.gc, 972
environ.getfenv, 973
environ.getopt, 973
environ.isafunc, 974
environ.iscfunc, 974
environ.isequal, 974
environ.isselfref, 975
environ.kernel, 975
environ.onexit, 25, 980
environ.pointer, 980
environ.ref, 981
environ.setfenv, 981
environ.system, 982
environ.unref, 982
environ.used, 982
environ.userinfo, 982
environ.warn, 983
erf, 421
erfc, 421
erfcx, 421
erfi, 421
error, 30
even, 421, 733
everyth, 31
exp, 421, 518, 726, 781
exp10, 422
exp2, 422
expx2, 422, 447
fact, 422
factory.anyof, 1093
factory.count, 1091
factory.curry, 1093
factory.cycle, 1092
factory.iterate, 1092
factory.pick, 1094
factory.reset, 1092

fastmath.cosfast, 473
fastmath.floor, 473
fastmath.hypotfast, 473
fastmath.invroot, 473
fastmath.invsqrt, 473
fastmath.lbfast, 474
fastmath.reciprocal, 474
fastmath.sincosfast, 474
fastmath.sinfast, 474
fastmath.sqroot, 474
fastmath.sqrtfast, 474
fastmath.tanfast, 474
fifo.attrib, 370
fifo.getstore, 371
fifo.has, 371
fifo.include, 371
fifo.new, 371
fifo.peek, 371
fifo.remove, 371
fifo.shrink, 371
filled, 31, 90, 215, 240, 251, 270
finite, 422, 733
flip, 423
floor, 423
fma, 423
fold, 31, 90
foreach, 32, 424
frac, 423, 732
fractals.albea, 1046
fractals.alcos, 1046
fractals.alcosxx, 1047
fractals.alsin, 1047
fractals.amarkmandel, 1046
fractals.anewton, 1047
fractals.dragon, 1047
fractals.draw, 1051
fractals.esctime, 1047
fractals.fern, 1048
fractals.genmap, 1054
fractals.henon, 1048
fractals.ifs, 1048
fractals.juliacantor, 1049
fractals.juliapetal, 1049
fractals.koch, 1049
fractals.lbea, 1050
fractals.logisticmap, 1050
fractals.mandel, 1050
fractals.mandelbrot, 1050
fractals.mandelbrotfast, 1050
fractals.mandelbrottrig, 1050
fractals.markmandel, 1051
fractals.minkowski, 1051
fractals.newton, 1051
fractals.readmap, 1055
fractals.sortmap, 1054
fractals.writemap, 1055
fractional, 423, 733
freeze, 31
frexp, 424
frexp10, 424
fzy.filter, 191
fzy.has, 191
fzy.positions, 192
fzy.score, 192
gamma, 424
gdi.arc, 1033
gdi.arcfilled, 1033
gdi.autoflush, 1033
gdi.background, 1033
gdi.circle, 1033
gdi.circlefilled, 1033
gdi.clearpalette, 1034
gdi.close, 1034
gdi.dash, 1034
gdi.ellipse, 1034
gdi.ellipsefilled, 1034
gdi.flush, 1034
gdi.fontsize, 1034
gdi.hasoption, 1034
gdi.initpalette, 1034
gdi.ink, 1035
gdi.lastaccessed, 1035
gdi.line, 1035
gdi.lineplot, 1035
gdi.mouse, 1035
gdi.open, 1035
gdi.options, 1036
gdi.plot, 1037
gdi.plotfn, 1038
gdi.plotfn2d, 1040
gdi.point, 1040
gdi.pointplot, 1040
gdi.rectangle, 1041
gdi.rectanglefilled, 1041
gdi.reset, 1042
gdi.resetpalette, 1042
gdi.setarc, 1042
gdi.setarcfilled, 1042
gdi.setcircle, 1042
gdi.setcirclefilled, 1042
gdi.setellipse, 1042
gdi.setellipsefilled, 1042
gdi.setinfo, 1043
gdi.setline, 1043

gdi.setoptions, 1043
gdi.setpoint, 1043
gdi.setrectangle, 1043
gdi.setrectanglefilled, 1044
gdi.settriangle, 1044
gdi.settrianglefilled, 1044
gdi.structure, 1044
gdi.system, 1044
gdi.text, 1044
gdi.thickness, 1045
gdi.triangle, 1045
gdi.trianglefilled, 1045
gdi.useink, 1045
getbit, 31, 33
getentry, 34, 215, 251, 270
getmetatable, 34, 215, 240, 251, 270
getnbits, 33
getorset, 34, 215, 241, 252, 271
gettype, 34
gmp.add, 508
gmp.addmul, 508
gmp.attrib, 515
gmp.band, 513
gmp.binomial, 513
gmp.bor, 513
gmp.bxor, 513
gmp.clrbit, 514
gmp.cmp, 515
gmp.cmpabs, 515
gmp.com, 513
gmp.combit, 514
gmp.divide, 508
gmp.factorial, 512
gmp.fib, 512
gmp.gcd, 511
gmp.gcdext, 512
gmp.getbit, 514
gmp.getd2exp, 509
gmp.hamdist, 514
gmp.invert, 512
gmp.invm, 510
gmp.iseven, 515
gmp.isodd, 515
gmp.ispow2, 515
gmp.jacobi, 512
gmp.kronecker, 512
gmp.lcm, 512
gmp.leastsigbit, 514
gmp.legendre, 512
gmp.log10, 511
gmp.log2, 511
gmp.lucas, 513
gmp.max, 509
gmp.min, 509
gmp.minmax, 509
gmp.modulus, 509
gmp.mostsigbit, 514
gmp.mul2exp, 510
gmp.mulm, 510
gmp.multiply, 508
gmp.neg, 509
gmp.nextprime, 511
gmp.perfectpower, 513
gmp.perfectsquare, 513
gmp.popcount, 514
gmp.power, 508
gmp.powm, 510
gmp.prevprime, 511
gmp.primorial, 513
gmp.remove, 512
gmp.root, 511
gmp.rootrem, 511
gmp.scan0, 514
gmp.scan1, 514
gmp.setbit, 514
gmp.sint, 508
gmp.sizeinbase, 515
gmp.submul, 509
gmp.subtract, 508
gmp.swap, 515
gmp.tdiv, 510
gmp.tdivq, 510
gmp.tdivr, 510
gmp.testprime, 511
gmp.tonumber, 515
gmp.tostring, 515
gmp.uint, 507
gzip.close, 859
gzip.deflate, 859
gzip.inflate, 859
gzip.lines, 860
gzip.open, 860
gzip.read, 860
gzip.seek, 861
gzip.sync, 861
gzip.write, 861
has, 90
hashes.adler32, 165
hashes.asu, 165
hashes.bkdr, 165
hashes.bp, 166
hashes.bsd, 166
hashes.ccitt, 166
hashes.cksum, 166

hashes.collisions, 167
hashes.crc16, 167
hashes.crc32, 167
hashes.crc7, 167
hashes.crc8, 167
hashes.damm, 167
hashes.dek, 168
hashes.derpy, 168
hashes.digitsum, 168
hashes.djb, 168
hashes.djb2, 169
hashes.djb2rot, 169
hashes.droot, 169
hashes.elf, 170
hashes.fibmod, 170
hashes.fibmod2, 170
hashes.finger, 171
hashes.fletcher, 171
hashes.fnv, 171
hashes.ftok, 172
hashes.hashfloat, 172
hashes.internet, 172
hashes.interweave, 172
hashes.ispell, 173
hashes.j32to32, 173
hashes.jen, 173
hashes.jinteger, 173
hashes.jnumber, 173
hashes.lua, 174
hashes.luhn, 174
hashes.md5, 174
hashes.mix, 174
hashes.mix64, 175
hashes.mix64to32, 175
hashes.murmur1, 175
hashes.murmur2, 175
hashes.murmur3, 175
hashes.murmur3128, 175
hashes.murmur3128x86, 176
hashes.numlua, 176
hashes.oaat, 176
hashes.parity, 177
hashes.pjw, 177
hashes.pl, 177
hashes.raw, 177
hashes.reflect, 178
hashes.roaat, 178
hashes.rs, 178
hashes.sax, 178
hashes.sdbm, 178
hashes.sha256, 179
hashes.sha512, 179
hashes.squirrel32, 179
hashes.squirrel64, 180
hashes.sth, 180
hashes.strval, 180
hashes.sumupchars, 180
hashes.superfast, 181
hashes.sysv, 181
hashes.varlen, 181
hashes.verhoeff, 182
hashes.wang, 182
heaviside, 425
history.off, 1025
history.on, 1025
hypot, 425
hypot2, 425
hypot3, 426
iconv.close, 162
iconv.convert, 161
iconv.list, 162
iconv.open, 161
identity, 35, 215, 252, 271
ilog10, 426
ilog2, 426
imag, 426
implies, 426
in, 89, 222, 243, 258, 277, 287, 414
in Operator, 20
infinite, 426
ini.attrib, 875
ini.close, 875
ini.dump, 875
ini.getitem, 876
ini.getsection, 876
ini.hash, 877
ini.new, 877
ini.read, 877
ini.setitem, 877
ini.unset, 878
initialise, 36
int, 427, 470, 732
integral, 427, 733
intersect, 222, 243, 258, 277
inthash.remap, 391
intmap.attrib, 390
intmap.cleanse, 390
intmap.getitem, 390
intmap.include, 390
intmap.iterate, 390
intmap.new, 391
intmap.purge, 391
intmap.resize, 392
ints Package Functions, 746

ints.cbrt, 746
ints.checkint64, 745
ints.checkuint64, 745
ints.copy, 744
ints.gcd, 746
ints.getbit, 751
ints.getbits, 751
ints.getlow, 751
ints.getwords, 751
ints.int64, 744, 745
ints.invmod, 746
ints.iscoprime, 746
ints.isint64, 745
ints.isnegint, 746
ints.isnonnegint, 746
ints.isposint, 747
ints.ispow2, 747
ints.isprime, 747
ints.isuint64, 745
ints.lcm, 747
ints.leadzeros, 751
ints.leastsigbit, 752
ints.log10, 747
ints.log2, 747
ints.max, 748
ints.mean, 748
ints.min, 748
ints.minmax, 748
ints.mostsigbit, 752
ints.mulmod, 748
ints.nextpow2, 748
ints.nextprime, 749
ints.onebits, 752
ints.parity, 752
ints.powmod, 749
ints.prevprime, 749
ints.random, 750
ints.root, 750
ints.setbit, 753
ints.sethigh, 753
ints.setlow, 753
ints.setwords, 753
ints.swap, 745
ints.tobytes, 754
ints.todouble, 754
ints.toint64, 754
ints.tonumber, 754
ints.tostring, 754
ints.touint64, 754
ints.trailzeros, 753
ints.uint64, 743
inverf, 427
inverfc, 427
invgamma, 427
invhypot, 427
invpsi, 428
invpytha, 428
invsqrt, 428, 726
io.anykey, 789
io.clearerror, 789
io.close, 788, 789, 797
io.eof, 790
io.ferror, 790
io.fileno, 790
io.filepos, 790
io.filesize, 790
io.getclip, 791
io.getkey, 791
io.infile, 791
io.input, 791
io.isatty, 792
io.isfdesc, 792
io.isopen, 792
io.kbdgetstatus, 792
io.keystroke, 792
io.lines, 793, 797
io.lock, 794
io.maxopenfiles, 795
io.mkstemp, 795
io.move, 795
io.nlines, 796
io.open, 788, 796
io.output, 796
io.pcall, 797
io.popen, 797
io.putclip, 798
io.read, 788, 798
io.readfile, 799
io.readfrom, 799
io.readlines, 800
io.readto, 801
io.rewind, 801
io.seek, 802
io.setvbuf, 802
io.skiplines, 802
io.sync, 802, 803
io.tmpfile, 803
io.toend, 803
io.truncate, 803
io.unlock, 803
io.write, 804
io.writefile, 805
io.writeline, 804
ipairs, 38

iqr, 428
iquo, 428
irem, 429
isall, 38
isboolean, 39
iscomplex, 39, 429
isequal, 39
isint, 39, 429
isnegative, 39, 429
isnegint, 39, 429
isnonneg, 39, 429
isnonnegint, 40, 429
isnonposint, 40, 430
isnonzeroint, 40, 430
isnumber, 40, 430
isnumeric, 40, 430
ispair, 40
isposint, 40, 430
ispositive, 40, 430
isreg, 40
isseq, 41
isstring, 41
isstructure, 41
ival.checkival, 770
ival.contained, 770
ival.diam, 770
ival.disjoint, 770
ival.extremes, 770
ival.inf, 770
ival.isival, 770
ival.mean, 771
ival.member, 771
ival.new, 769, 770
ival.sup, 770
ival.tostring, 771
json.decode, 856
json.encode, 856
keyfile.close, 882
keyfile.commit, 883
keyfile.has, 883
keyfile.isopen, 883
keyfile.iterate, 883
keyfile.new, 884
keyfile.open, 884
keyfile.purge, 884
keyfile.read, 884
keyfile.start, 885
keyfile.sync, 885
keyfile.update, 885
keyfile.write, 885
kiss.fft, 768
kiss.nextsize, 768
ldexp, 430
left, 41
lifo.attrib, 373
lifo.getstore, 373
lifo.has, 373
lifo.include, 374
lifo.new, 374
lifo.peek, 374
lifo.remove, 374
lifo.shrink, 374
linalg.add, 632
linalg.addcol, 632
linalg.addrow, 632
linalg.adjoint, 633
linalg.antidiagonal, 633
linalg.augment, 633
linalg.backsub, 633
linalg.backsubs, 634
linalg.checkmatrix, 634
linalg.checksquare, 634
linalg.checkvector, 634
linalg.col, 634
linalg.coldim, 634
linalg.colvector, 635
linalg.copyinto, 635
linalg.crossprod, 635
linalg.delcols, 636
linalg.delrows, 636
linalg.det, 636
linalg.diagonal, 636
linalg.dim, 636
linalg.dotprod, 637
linalg.eigen, 637
linalg.eigenval, 637
linalg.extend, 637
linalg.fib, 639
linalg.forsub, 639
linalg.gausselim, 639
linalg.gaussjord, 640
linalg.getantidiagonal, 640
linalg.getdiagonal, 640
linalg.getvelem, 640
linalg.hilbert, 641
linalg.identity, 641
linalg.infnorm, 641, 642
linalg.innerprod, 642
linalg.inverse, 643
linalg.isantidiagonal, 643
linalg.isantisymmetric, 643
linalg.isdiagonal, 643
linalg.isfractional, 643
linalg.isidentity, 644

linalg.isintegral, 644
linalg.islower, 644
linalg.ismatrix, 644
linalg.isone, 644
linalg.isref, 644
linalg.isrref, 645
linalg.issingular, 645
linalg.issparse, 645
linalg.issquare, 645
linalg.issymmetric, 646
linalg.isupper, 646
linalg.isvector, 646
linalg.iszero, 646
linalg.kronprod, 646
linalg.linsolve, 647
linalg.ludecomp, 647
linalg.ludoolittle, 648
linalg.maeq, 648
linalg.matinfnorm, 649
linalg.matmat, 649
linalg.matnnorm, 649
linalg.matonenorm, 650
linalg.matrix, 650
linalg.mattam, 652
linalg.mcopy, 652
linalg.meeq, 652
linalg.minor, 652
linalg.mmap, 653, 666
linalg.mmul, 653
linalg.mulrow, 653
linalg.mulrowadd, 653
linalg.multiply, 653
linalg.mzip, 648, 654
linalg.ncolnorm, 654
linalg.newmatrix, 655
linalg.nnorm, 655
linalg.norm, 655
linalg.onecolnorm, 656
linalg.onenorm, 656
linalg.ones, 657
linalg.outprodmatrix, 657
linalg.permanent, 657
linalg.pivot, 657
linalg.randmatrix, 658
linalg.randvector, 658
linalg.rank, 658
linalg.reshape, 658
linalg.rotcol, 659
linalg.rotrow, 659
linalg.row, 660
linalg.rowdim, 660
linalg.rref, 660
linalg.scalarmul, 660
linalg.scale, 660
linalg.setvelem, 661
linalg.sparse, 661
linalg.stack, 661
linalg.sub, 661
linalg.submatrix, 661
linalg.subvector, 662
linalg.swapcol, 662
linalg.swaprow, 663
linalg.totable, 663
linalg.trace, 663
linalg.transpose, 663
linalg.unitvector, 663
linalg.vaeq, 664
linalg.vcopy, 664
linalg.vectdim, 664
linalg.vector, 664
linalg.veeq, 665
linalg.viszero, 666
linalg.vmap, 666
linalg.vzero, 666
linalg.vzip, 667
linalg.zeros, 667
llist.append, 324
llist.checkllist, 324
llist.dump, 324
llist.getitem, 324
llist.iterate, 324
llist.new, 325
llist.prepend, 325
llist.purge, 325
llist.put, 326
llist.replicate, 326
llist.setitem, 326
llist.toseq, 326
llist.totable, 326
ln, 430, 518, 727, 781
lngamma, 431
load, 41
loadfile, 41
loadstring, 41
log, 430, 431, 728
log10, 431
log2, 431
long.approx, 725
long.arccosh, 730
long.arccot, 730
long.arccoth, 730
long.arccsch, 730
long.arcsec, 730
long.arcsech, 730

long.arcsinh, 731
long.arctanh, 731
long.besselj, 720
long.bessely, 720
long.beta, 726
long.binomd, 734
long.cathet, 720
long.cbrt, 726
long.cdfnormald, 734
long.ceil, 731
long.checklong, 717
long.chop, 732
long.copysign, 720
long.cot, 729
long.coth, 730
long.count, 718
long.csc, 729
long.csch, 730
long.double, 716
long.eps, 735
long.erf, 731
long.erfc, 731
long.erfcx, 731
long.expm1, 726
long.exponent, 721
long.expx2, 726
long.fdim, 722
long.floor, 732
long.fma, 721
long.fmin, 725
long.fmod, 721
long.fpclassify, 738
long.frexp, 737
long.gamma, 727
long.gsolve, 735
long.hypot, 721
long.hypot2, 721
long.hypot3, 721
long.ilog2, 728
long.invbinomd, 734
long.inverf, 731
long.inverfc, 731
long.invnormald, 734
long.isequal, 724
long.isfinite, 738
long.isinfinite, 738
long.isirregular, 733
long.isless, 725
long.islessequal, 725
long.islong, 717
long.isnegative, 717
long.isnegint, 717
long.isnonnegative, 717
long.isnonnegint, 717
long.isnormal, 738
long.isposint, 717
long.ispositive, 717
long.issubnormal, 738
long.isundefined, 738
long.isunequal, 724
long.iszero, 738
long.ldexp, 737
long.lnabs, 727
long.lnbinomial, 727
long.lnfact, 727
long.lnp1, 727, 728
long.log10, 728
long.log2, 728
long.mantissa, 722
long.modf, 722
long.multiple, 737
long.new, 716
long.nextafter, 735
long.norm, 736
long.normalise, 738
long.overflow, 719
long.polygen, 722
long.pytha, 722
long.pytha4, 722
long.redupi, 736
long.rempio2, 736
long.root, 726
long.round, 732
long.sec, 729
long.sech, 730
long.signbit, 737
long.significand, 723
long.sincos, 729
long.sinhcosh, 730
long.tonumber, 717
long.tostring, 717
long.unm, 735
long.wrap, 736
long.zeroin, 735
long.zerosubnormal, 735
lookup.checktable, 365
lookup.getsizes, 365
lookup.gettable, 365
lookup.include, 365
lookup.indices, 365
lookup.isarray, 366
lookup.iterate, 366
lookup.map, 366
lookup.new, 367

lookup.next, 367
lookup.purge, 367
lookup.setsizes, 368
lookup.subs, 368
lower, 120
map, 42, 90, 216, 223, 241, 244, 252, 271, 286
mapm Package Functions, 497
mapm.approx, 498
mapm.carccosh, 504
mapm.carcsinh, 504
mapm.carctan2, 505
mapm.carctanh, 504
mapm.cargument, 505
mapm.ccosc, 505
mapm.ccot, 505
mapm.ccoth, 505
mapm.ccsc, 505
mapm.ccsch, 505
mapm.cexp10, 505
mapm.cexp2, 505
mapm.cfma, 505
mapm.checkcnumber, 500
mapm.checkinteger, 500
mapm.checknonnegative, 501
mapm.checknonnegint, 500
mapm.checkposint, 501
mapm.checkpositive, 501
mapm.checkxnumber, 500
mapm.clog10, 505
mapm.clog2, 505
mapm.cnumber, 495, 503, 504
mapm.csec, 505
mapm.csech, 505
mapm.csinc, 505
mapm.csincos, 505
mapm.csinhcosh, 505
mapm.ctanc, 505
mapm.ctocomplex, 495, 504
mapm.ctonumber, 504
mapm.ctostring, 505
mapm.iscnumber, 500
mapm.isxnumber, 500
mapm.max, 498
mapm.min, 498
mapm.minmax, 498
mapm.new, 501
mapm.swap, 498
mapm.tonumber, 495
mapm.trim, 501
mapm.xabs, 497
mapm.xadd, 497
mapm.xarccos, 497
mapm.xarccosh, 497
mapm.xarccsc, 497
mapm.xarcsec, 497
mapm.xarcsin, 497
mapm.xarcsinh, 497
mapm.xarctan, 497
mapm.xarctan2, 497
mapm.xarctanh, 497
mapm.xcathet, 498
mapm.xcbrt, 497
mapm.xceil, 500
mapm.xchebyt, 501
mapm.xcompare, 500
mapm.xcos, 497
mapm.xcosc, 497
mapm.xcosh, 497
mapm.xcot, 497
mapm.xcoth, 497
mapm.xcsc, 497
mapm.xcsch, 497
mapm.xcube, 498
mapm.xdigits, 495, 500, 502, 505
mapm.xdigitsin, 500, 505
mapm.xdiv, 497
mapm.xerf, 497
mapm.xerfc, 497
mapm.xexp, 497
mapm.xexp10, 497
mapm.xexp2, 497
mapm.xexponent, 500
mapm.xfactorial, 497
mapm.xfloor, 500
mapm.xfma, 497
mapm.xfrac, 498
mapm.xgamma, 497
mapm.xgcd, 498
mapm.xhypot, 498
mapm.xidiv, 497
mapm.xintfrac, 498
mapm.xinv, 500
mapm.xinvmod, 498
mapm.xipow, 498
mapm.xiseven, 500
mapm.xisint, 500
mapm.xisnegative, 501
mapm.xisnegint, 501
mapm.xisnonnegative, 501
mapm.xisnonnegint, 501
mapm.xisodd, 500
mapm.xisposint, 501
mapm.xispositive, 501

mapm.xisprime, 500
mapm.xlcm, 498
mapm.xln, 497
mapm.xlog, 497
mapm.xlog10, 497
mapm.xlog2, 497
mapm.xmax, 498
mapm.xmin, 498
mapm.xmod, 500
mapm.xmul, 497
mapm.xmulmod, 498
mapm.xneg, 500
mapm.xnumber, 495, 500
mapm.xpow, 497
mapm.xpowmod, 498
mapm.xpsi, 497
mapm.xrandom, 498
mapm.xrandomseed, 498
mapm.xrecip, 497
mapm.xround, 500
mapm.xsec, 497
mapm.xsech, 497
mapm.xsign, 497
mapm.xsin, 497
mapm.xsinc, 497
mapm.xsincos, 497
mapm.xsinh, 497
mapm.xsinhcosh, 497
mapm.xsqrt, 497
mapm.xsquare, 498
mapm.xsub, 497
mapm.xtan, 497
mapm.xtanc, 497
mapm.xterm, 497
mapm.xtonumber, 495, 500
mapm.xtostring, 500
math.accu, 440
math.agm, 440
math.beta, 441
math.bintodec, 441
math.branch, 441
math.ceillog2, 441
math.ceilpow2, 442
math.chi, 442
math.chop, 442
math.cld, 443
math.clip, 443
math.compose, 443
math.convertbase, 443
math.copysign, 444
math.coscpi, 444
math.cosd, 444
math.cospi, 444
math.cotd, 444
math.cscd, 444
math.dblfact, 445
math.dd, 445
math.decompose, 445
math.dirac, 445
math.dms, 445
math.eps, 446
math.epsilon, 446
math.eq, 447
math.expm1, 447
math.exponent, 447
math.fall, 447
math.fdim, 448
math.fld, 448
math.flipsign, 448
math.floorpow2, 448
math.fpclassify, 448
math.fraction, 449
math.frexp, 449
math.gammasign, 449
math.ge, 449
math.gt, 450
math.hamming, 450
math.hextodec, 450
math.hgm, 450
math.invlerp, 450
math.isexceptional, 451
math.isinfinity, 451
math.isirregular, 451
math.isminuszero, 451
math.isnormal, 452
math.isordered, 452
math.ispow2, 452
math.isqrt, 452
math.kbadd, 452
math.koadd, 453
math.largest, 454
math.le, 454
math.length, 454
math.lerp, 455
math.lnabs, 455
math.lnbeta, 455
math.lnbinomial, 455
math.lnfact, 455
math.lnhypot, 456
math.lnp1, 456
math.lnpytha, 456
math.logs, 456
math.lt, 457
math.mantissa, 457

math.max, 457
math.min, 457
math.modulus, 457
math.morton, 458
math.mulsign, 458
math.ndigits, 458
math.nearbyint, 458
math.nearmod, 458
math.neq, 459
math.nextafter, 459
math.nextmultiple, 459
math.nextpow2, 459
math.nextpower, 459
math.noise, 459
math.norm, 460
math.normalise, 460
math.nthdigit, 460
math.octtodec, 460
math.piecewise, 461
math.pochhammer, 461
math.pow32, 461
math.pow52, 461
math.pow72, 461
math.powe, 461
math.prevmultiple, 461
math.prevpow2, 462
math.prevpower, 462
math.quadrant, 462
math.ramp, 462
math.random, 462
math.randoms, 463
math.randomseed, 463
math.randomseeds, 463
math.rectangular, 464
math.redupi, 464
math.relerror, 464
math.rempio2, 464
math.rint, 465
math.secd, 465
math.signbit, 465
math.significand, 465
math.sincos, 465
math.sincospi, 465
math.sincpi, 466
math.sind, 466
math.sinhcosh, 466
math.sinpi, 466
math.smallest, 466
math.smallestnormal, 466
math.splitdms, 467
math.tancpi, 467
math.tand, 467
math.tanpi, 467
math.tocomplex, 467
math.todecimal, 467
math.todegrees, 468
math.toradians, 468
math.tosgesim, 469
math.triangular, 469
math.trifact, 469
math.trunc, 470
math.two54, 470
math.uexponent, 470
math.ulp, 470
math.unitise, 470
math.unitstep, 471
math.wrap, 471
math.xlnp1, 471
math.zerosubnormal, 471
max, 44
mdf, 431
member, 44, 216, 241, 252, 271
memfile.append, 146
memfile.attrib, 146
memfile.bitfield, 146
memfile.bytebuf, 147
memfile.charbuf, 147
memfile.clearbit, 147
memfile.dump, 148
memfile.find, 149
memfile.gefield, 150
memfile.get, 149
memfile.getbit, 149
memfile.getbyte, 149
memfile.getbytes, 150
memfile.getchar, 150
memfile.getitem, 150
memfile.getsize, 151
memfile.iterate, 151
memfile.map, 151
memfile.match, 151
memfile.mfind, 151
memfile.move, 152
memfile.prepend, 152
memfile.purge, 153
memfile.put, 153
memfile.read, 153
memfile.replace, 154
memfile.resize, 154
memfile.reverse, 154
memfile.rewind, 155
memfile.setbit, 155
memfile.setbyte, 155
memfile.setbytes, 155

memfile.setchar, 156
memfile.setfield, 156
memfile.setitem, 156
memfile.shift, 157
memfile.substring, 157
memfile.tostring, 157
memfile.write, 157
min, 44
minizip add method, 871
minizip addfile method, 871
minizip addfrom method, 872
minizip close method, 868, 872
minizip extract method, 868
minizip finalise method, 873
minizip index method, 868
minizip isdir method, 869
minizip num method, 869
minizip offset method, 869
minizip rawfd method, 870
minizip stat method, 870
minizip.adler32, 863
minizip.attribs, 863
minizip.close, 863
minizip.compress, 863
minizip.count, 864
minizip.crc32, 864
minizip.decompress, 864
minizip.finalise, 864
minizip.index, 865
minizip.isdir, 865
minizip.iszip, 865
minizip.open, 865
minizip.read, 867
minizip.write, 867
minus, 222, 243, 259, 278
modf, 431
modp, 432
move, 45, 216, 252, 271
mpfr.agm, 519
mpfr.ai, 519
mpfr.approx, 522
mpfr.arccosh, 519
mpfr.arccot, 519
mpfr.arccoth, 519
mpfr.arccsc, 519
mpfr.arccsch, 519
mpfr.arcsech, 519
mpfr.arcsinh, 519
mpfr.arctan2, 519
mpfr.arctanh, 519
mpfr.argument, 519
mpfr.beta, 519
mpfr.cathet, 519
mpfr.cbrt, 519
mpfr.ceil, 519
mpfr.checkcmpf, 522
mpfr.checkmpf, 522
mpfr.clone, 522
mpfr.cmpd, 522
mpfr.copysign, 519
mpfr.cot, 519
mpfr.coth, 519
mpfr.csc, 519
mpfr.csch, 519
mpfr.digamma, 519
mpfr.dim, 519
mpfr.div2exp, 519
mpfr.eint, 519
mpfr.erf, 519
mpfr.erfc, 519
mpfr.exp10, 519
mpfr.exp2, 519
mpfr.floor, 519
mpfr.fma, 519
mpfr.fmod, 519
mpfr.fms, 519
mpfr.fraction, 519
mpfr.frexp, 519
mpfr.gamma, 519
mpfr.getparts, 523
mpfr.hypot, 519
mpfr.Inf, 521
mpfr.iscmpf, 523
mpfr.iseven, 519
mpfr.isfinite, 520
mpfr.isfractional, 520
mpfr.isinfinite, 520
mpfr.isintegral, 520
mpfr.isiundefined, 520
mpfr.ismpf, 523
mpfr.isodd, 520
mpfr.j0, 520
mpfr.j1, 520
mpfr.jn, 520
mpfr.lgamma, 520
mpfr.li2, 520
mpfr.log10, 520
mpfr.log2, 520
mpfr.max, 520
mpfr.min, 520
mpfr.minmax, 520
mpfr.modf, 520
mpfr.mul2exp, 520
mpfr.Nan, 521

mpfr.new, 523
mpfr.nexttoward, 520
mpfr.polygen, 523
mpfr.precision, 524
mpfr.pytha, 520
mpfr.pytha4, 520
mpfr.random, 520
mpfr.relerror, 520
mpfr.root, 520
mpfr.round, 520
mpfr.rounding, 524
mpfr.sec, 520
mpfr.sech, 520
mpfr.signbit, 520
mpfr.swap, 524
mpfr.tonumber, 524
mpfr.tostring, 524
mpfr.trunc, 520
mpfr.y0, 521
mpfr.yn, 521
mpfr.Zero, 521
mpfr.zeta, 521
mprint, 45
muladd, 432
multiple, 432
mulup, 46, 253, 272
nan, 433, 733
nand, 433
net.accept, 895
net.address, 896
net.admin Table, 896
net.bind, 896
net.block, 897
net.close, 897
net.closewinsock, 897
net.connect, 897
net.getaddrinfo, 898
net.gethostname, 899
net.htonl, 899
net.htons, 899
net.isconnected, 899
net.isipv4, 899
net.isipv6, 899
net.keep, 900
net.listen, 900
net.lookup, 900
net.ntohl, 900
net.ntohs, 901
net.open, 901
net.opensockets, 902
net.openwinsock, 903
net.receive, 903
net.remoteaddress, 904
net.send, 904
net.shutdown, 905
net.smallping, 905
net.survey, 906
net.tohostname, 907
net.toip, 907
net.wget, 907
next, 46
nonzero, 433, 732
nor, 433
notin, 89, 222, 243, 258, 287
notin Operator, 20
numarray.append, 292
numarray.assign, 292
numarray.attrib, 292
numarray.band, 318
numarray.binsearch, 293
numarray.bnot, 318
numarray.bor, 318
numarray.bxor, 318
numarray.cdouble, 293
numarray.celsius, 319
numarray.checkarray, 293
numarray.convert, 294
numarray.countitems, 294
numarray.cycle, 294
numarray.double, 294
numarray.fahren, 319
numarray.find, 294
numarray.floz, 319
numarray.gallon, 320
numarray.getbit, 296
numarray.geti, 295
numarray.getitem, 295
numarray.getparts, 296
numarray.getsize, 296
numarray.gram, 319
numarray.include, 296
numarray.int32, 297
numarray.int64, 297
numarray.introsort, 297
numarray.isall, 298
numarray.isarray, 298
numarray.iterate, 298
numarray.km, 319
numarray.litre, 320
numarray.longdouble, 300
numarray.map, 300
numarray.member, 300
numarray.mile, 319
numarray.minmax, 301

numarray.new, 301
numarray.one, 301
numarray.ounce, 319
numarray.prepend, 302
numarray.purge, 302
numarray.read, 302
numarray.readcdoubles, 303
numarray.readdoubles, 303
numarray.readint64, 304
numarray.readintegers, 304
numarray.readlongdoubles, 304
numarray.readuchars, 305
numarray.readuint32, 305
numarray.readuint64, 305
numarray.readushorts, 306
numarray.redim, 306
numarray.remove, 306
numarray.replicate, 307
numarray.resize, 307
numarray.satisfy, 307
numarray.select, 308
numarray.setbit, 308
numarray.seti, 308
numarray.setitem, 309
numarray.setparts, 309
numarray.sort, 309
numarray.sorted, 310
numarray.subarray, 310
numarray.subs, 310
numarray.toarray, 311
numarray.toreg, 311
numarray.toseq, 311
numarray.totable, 311
numarray.uchar, 311
numarray.uint32, 311
numarray.uint64, 312
numarray.unique, 312
numarray.used, 312
numarray.ushort, 312
numarray.whereis, 313
numarray.write, 313
numarray.xadd, 315
numarray.xantilog2, 316
numarray.xarccos, 317
numarray.xarcsin, 317
numarray.xarctan, 317
numarray.xcos, 316
numarray.xcosh, 316
numarray.xdiv, 315
numarray.xexp, 315
numarray.xln, 315
numarray.xlog2, 316
numarray.xmul, 315
numarray.xrecip, 315
numarray.xsin, 316
numarray.xsinh, 316
numarray.xsqrt, 316
numarray.xsquare, 316
numarray.xsub, 315
numarray.xtan, 316
numarray.xtanh, 316
numarray.zip, 314
numcuckoo.attrib, 379
numcuckoo.find, 379
numcuckoo.hash, 379
numcuckoo.include, 379
numcuckoo.new, 380
numcuckoo.remove, 380
numtheory.binet, 761
numtheory.congruentprime, 761
numtheory.factors, 762
numtheory.fib, 762
numtheory.fibinv, 762
numtheory.gcd, 763
numtheory.ifactors, 763
numtheory.invmod, 764
numtheory.iscoprime, 764
numtheory.iscube, 764
numtheory.isfib, 764
numtheory.isprime, 764
numtheory.issqrfree, 765
numtheory.issquare, 764
numtheory.jacobi, 765
numtheory.kronecker, 765
numtheory.lcm, 765
numtheory.mulmod, 765
numtheory.nextprime, 765
numtheory.nthpow, 765
numtheory.powmod, 766
numtheory.prevprime, 766
numtheory.primes, 766
odd, 433, 733
op, 47
ops, 49
optboolean, 49
optcomplex, 49
optint, 50
optnonnegative, 50
optnonnegint, 50
optnonzeroint, 50
optnumber, 50
optposint, 51
optpositive, 51
optstring, 51

os.battery, 918
os.beep, 918
os.bios, 919
os.cachelinesize, 919
os.cdrom, 919
os.chdir, 919
os.chmod, 919
os.chown, 920
os.clock, 920
os.codepage, 920
os.computername, 921
os.countcore, 921
os.cpuid, 921
os.cpuinfo, 921
os.cpuload, 923
os.curdir, 923
os.curdrive, 923
os.date, 923
os.datetosecs, 926
os.difftime, 926
os.dirname, 926
os.drives, 927
os.drivestat, 927
os.endian, 928
os.environ, 928
os.esd, 928
os.execute, 929
os.exists, 929, 930
os.exit, 929
os.fattrib, 931
os.fcopy, 932
os.filename, 932
os.freemem, 933
os.fstat, 934
os.ftok, 935
os.getadapter, 935
os.getcommandline, 936
os.getconsolemode, 936
os.getdirpathsep, 936
os.getenv, 936
os.getextlibpath, 936
os.getfullpathname, 937
os.getip, 937
os.getlanguage, 937
os.getloadeddlls, 937
os.getlocale, 937
os.getmac, 939
os.getmodulefilename, 939
os.getprocesses, 939
os.gettemppath, 939
os.getthreadid, 939
os.getthreadseed, 940
os.getusbdevices, 940
os.getwinsysdirs, 940
os.groupname, 940
os.hasnetwork, 941
os.inode, 941
os.isansi, 941
os.isarm, 941
os.isarm32, 941
os.isarm64, 942
os.isdir, 942
os.isdos, 942
os.isdow, 942
os.isdriveletter, 943
os.isdst, 943
os.isfile, 943
os.islink, 943
os.islinux, 943, 944
os.islinux386, 944
os.islocale, 944
os.ismac, 944
os.ismounted, 944
os.isppc, 944
os.israspi, 945
os.isremovable, 945
os.issolaris, 945
os.issysdir, 945
os.isunix, 945
os.isvaliddrive, 946
os.isvalidpath, 946
os.iswindows, 946
os.isx86, 946
os.iterate, 946
os.list, 947
os.listcore, 947
os.login, 948
os.lsd, 948
os.meminfo, 949
os.memstate, 949
os.mkdir, 950
os.mklink, 950
os.monitor, 950
os.mouse, 951
os.mouseclose, 952
os.mouseflush, 951
os.mouseopen, 952
os.mousestate, 952
os.move, 953
os.netdomain, 953
os.netsend, 953
os.netuse, 953
os.now, 953
os.os2info, 954

os.pause, 955
os.period, 955
os.pid, 955
os.prefix, 956
os.readlink, 956
os.realpath, 957
os.remove, 957
os.rmdir, 957
os.screensize, 957
os.secstodate, 957
os.setconsolemode, 957
os.setenv, 958
os.setextlibpath, 958
os.setlocale, 958
os.settime, 959
os.shellcolour, 959
os.shellgeom, 959
os.shellinfo, 960
os.strerror, 960
os.suffix, 960
os.symlink, 960
os.system, 961
os.terminate, 961
os.ticker, 962
os.time, 962
os.timestamp, 963
os.tmpdir, 963
os.tmpname, 963
os.tzdiff, 963
os.unmount, 964
os.uptime, 964
os.usd, 964
os.username, 965
os.vendor, 966
os.vga, 966
os.wait, 966
os.whereis, 967
os.winver, 967, 968
over, 410
pack, 51, 216, 253, 272
package.checkclib, 984
package.getcfuncs, 984
package.loadclib, 984
package.loaded, 984
package.packages, 984
package.readlibbed, 985
pairs, 52
pipeline, 52
polar, 433
popd, 992
prepend, 52, 212, 217, 248, 253, 267, 272
print, 52
printf, 53
proot, 434
protect, 54
purge, 55, 217, 272
pushd, 992
put, 55, 217, 254, 273
pytha, 434
pytha4, 434
qmdev, 435
qsumup, 56, 217, 254, 273, 632
rawequal, 56
rawget, 56
rawgeti, 56
rawset, 56
rawseti, 57
rbtree.entries, 353
rbtree.find, 353
rbtree.include, 354
rbtree.iterate, 354
rbtree.max, 354
rbtree.min, 354
rbtree.minmax, 354
rbtree.new, 355
read, 57
readlib, 57, 985
real, 435
recip, 435, 518, 720, 781
recurse, 58, 218, 241, 254, 273
reduce, 59, 91
regex.count, 189
regex.find, 189
regex.flags, 190
regex.match, 190
regex.new, 190
registers.col, 281
registers.dimension, 281
registers.extend, 285
registers.getdim, 281
registers.isall, 281
registers.isone, 282
registers.isrectangular, 282
registers.issquare, 282
registers.iszero, 282
registers.move, 283
registers.new, 283
registers.newreg, 284
registers.numintersect, 284
registers.numminus, 284
registers.numunion, 284
registers.regofregs, 285
registers.settop, 285

registers.subs, 285
registry.anchor, 989
registry.anyid, 989
registry.get, 989
registry.ref, 990
registry.unref, 990
remove, 61, 218, 241, 254, 273
restart, 62
reverse, 62, 218, 254, 273
right, 63
roll, 410
root, 435
round, 436, 458
rsorted, 63
rtable.defaults, 986
rtable.forget, 987
rtable.get, 987
rtable.init, 987
rtable.mode, 987
rtable.purge, 987
rtable.put, 988
rtable.remember, 61, 986
rtable.rget, 986
rtable.roinit, 988
run, 63
satisfy, 63
save, 64
scalbn, 436
sec, 436
sech, 436
select, 19, 64, 218, 223, 242, 244, 255, 260, 274, 279
selectremove, 65, 218, 242, 255, 274
sema.close, 1012
sema.isopen, 1012
sema.limit, 1012
sema.new, 1012
sema.open, 1013
sema.reset, 1013
sema.shrink, 1014
sema.state, 1014
sequences.col, 261
sequences.concat, 261
sequences.dimension, 261
sequences.extend, 261
sequences.getdim, 262
sequences.isall, 262
sequences.isone, 262
sequences.isrectangular, 262
sequences.issquare, 262
sequences.iszero, 263
sequences.move, 263
sequences.new, 263
sequences.newseq, 264
sequences.numintersect, 265
sequences.numminus, 265
sequences.numunion, 265
sequences.resize, 265
sequences.seqofseqs, 265
sequences.subs, 266
sequences.swapcol, 266
sequences.swaprow, 266
sequences.transpose, 266
setbit, 66
setbits, 66
setmetatable, 66, 218, 255, 274
setnbits, 67
sets.cleanse, 245
sets.isall, 245
sets.new, 245
sets.newset, 246
sets.numintersect, 246
sets.numminus, 246
sets.numunion, 247
sets.resize, 247
settype, 67
shift, 67, 219, 255, 274
sign, 436, 518, 720, 781
signum, 437, 720
sin, 437, 518, 729, 781
sinc, 437, 729
sinh, 437, 518, 729, 781
size, 68, 91, 219, 242, 255, 274, 286, 632
skew.entries, 350
skew.find, 350
skew.get, 350
skew.height, 350
skew.include, 350
skew.indices, 350
skew.iterate, 351
skew.new, 350, 351
skew.remove, 351
skew.reorder, 351
skycrane.bagtable, 1083
skycrane.dice, 1083
skycrane.fcopy, 1083
skycrane.formatline, 1083
skycrane.getlocales, 1084
skycrane.isemail, 1084
skycrane.iterate, 1084
skycrane.move, 1085
skycrane.readcsv, 1085
skycrane.replaceinfile, 1086

skycrane.scribe, 1086
skycrane.sorted, 1087
skycrane.stopwatch, 1088
skycrane.tee, 1088
skycrane.tocomma, 1088
skycrane.todate, 1089
skycrane.tolerance, 1089
skycrane.trimpath, 1089
skycrane.xmlmatch, 1089
sort, 68, 219, 255, 274
sorted, 69, 219, 256, 275
split, 89
sqrt, 438, 518, 725, 781
square, 518, 725, 781
squareadd, 411, 725
stack.absd, 1001, 1009
stack.absindex, 993
stack.addtod, 1000
stack.addtwod, 1008
stack.antilogd, 1005
stack.arccosd, 1007
stack.arccoshd, 1007
stack.arcsind, 1007
stack.arcsinhd, 1007
stack.arctan2d, 1007
stack.arctand, 1007
stack.arctanhd, 1007
stack.attribd, 994
stack.cathetd, 1004
stack.cbrtd, 1003
stack.choosed, 994
stack.cosd, 1006
stack.coshd, 1006
stack.cotd, 1007
stack.cscd, 1007
stack.dequeued, 994
stack.divtwod, 1009
stack.dumpd, 994
stack.enqueued, 995
stack.erfd, 1007
stack.exp10d, 1006
stack.exp2d, 1006
stack.expd, 1005
stack.explored, 995
stack.fmad, 1004
stack.fracd, 1002
stack.hypotd, 1004
stack.insertd, 995
stack.intd, 1001
stack.intdivd, 1002
stack.intdivtwod, 1009
stack.invhypotd, 1004
stack.isempty, 995
stack.lnd, 1005
stack.logd, 1005
stack.lowerd, 1009
stack.mapd, 995
stack.meand, 1008
stack.modd, 1002
stack.modtwod, 1009
stack.mulbyd, 1000
stack.multwod, 1009
stack.mulupd, 1008
stack.negated, 1001
stack.powd, 1002
stack.powtwod, 1009
stack.pushstringd, 996
stack.pushvalued, 996
stack.pythad, 1003
stack.readbytes, 997
stack.recipd, 1001
stack.removed, 998
stack.replaced, 998
stack.resetd, 998
stack.reversed, 998
stack.rootd, 1003
stack.rotated, 999
stack.secd, 1007
stack.selected, 999
stack.shrinkd, 999
stack.sind, 1006
stack.sinhd, 1006
stack.sized, 999
stack.sorted, 999
stack.sqrtd, 1004
stack.squared, 1003
stack.subtwod, 1009
stack.sumupd, 1008
stack.swapd, 999
stack.switchto, 1000
stack.tand, 1006
stack.tanhd, 1006
stack.upperd, 1010
stack.writebytes, 1000
stats.accu, 673
stats.acf, 674
stats.acv, 674
stats.ad, 675
stats.amean, 675
stats.besselj, 715
stats.besselk, 715
stats.beta, 676
stats.binomd, 676
stats.binompdf, 676

stats.brownian, 677
stats.card, 677
stats.cauchy, 678
stats.cdf, 678
stats.cdfnormald, 678
stats.chauvenet, 678
stats.checkcoordinate, 679
stats.chisquare, 680
stats.circular, 715
stats.colnorm, 680
stats.constant, 715
stats.countentries, 680
stats.covar, 680
stats.cubic, 715
stats.cumsum, 681
stats.dampedcos, 715
stats.dampedsin, 715
stats.dbscan, 681
stats.deltalist, 682
stats.durbinwatson, 682
stats.ema, 682
stats.exponential, 715
stats.extrema, 683
stats.F, 683
stats.Fc, 683
stats.fivenum, 684
stats.fprod, 684
stats.fratio, 684
stats.freqd, 685
stats.fsum, 685
stats.gammacdf, 686
stats.gammad, 686
stats.gammadc, 686
stats.gammapdf, 686
stats.gaussian, 715
stats.gema, 686
stats.geometric, 687
stats.gini, 687
stats.gmean, 688
stats.gsma, 688
stats.gsmm, 688
stats.herfindahl, 689
stats.hmean, 689
stats.hole, 715
stats.hypergeom, 690
stats.invbinomd, 690
stats.invF, 690
stats.invnormald, 690
stats.ios, 690
stats.iqmean, 691
stats.iqr, 691
stats.isall, 692
stats.isany, 692
stats.issorted, 692
stats.kurtosis, 692
stats.laplace, 693
stats.linear, 715
stats.logistic, 693
stats.lognormald, 694
stats.logseries, 694
stats.lse, 694
stats.mad, 694
stats.matern, 715
stats.max, 695
stats.md, 695
stats.mean, 696
stats.meanmed, 696
stats.meanqmdev, 696
stats.meanvar, 697
stats.median, 698
stats.midrange, 698
stats.min, 698
stats.minmax, 698
stats.mode, 699
stats.moment, 699
stats.nde, 699
stats.ndf, 699
stats.negbinompdf, 700
stats.neighbours, 700
stats.normald, 700
stats.obcount, 700
stats.obpart, 701
stats.pdf, 703
stats.peaks, 703
stats.penta, 715
stats.percentile, 703
stats.poisson, 704
stats.poissond, 704
stats.power, 715
stats.prange, 704
stats.probit, 704
stats.qcd, 705
stats.qmean, 705
stats.quartiles, 705
stats.ratquad, 715
stats.rownorm, 706
stats.scale, 706
stats.sd, 707
stats.skewness, 708
stats.sma, 708
stats.smallest, 709
stats.smm, 709
stats.sorted, 709
stats.spherical, 715

stats.spread, 710
stats.standardise, 710
stats.studentst, 710
stats.sumdata, 711
stats.sumdataln, 711
stats.tovals, 712
stats.trimean, 712
stats.trimmean, 712
stats.var, 713
stats.weights, 714
stats.white, 715
stats.winsor, 714
stats.zscore, 714
strhash.attrib, 397
strhash.cleanse, 397
strhash.getitem, 397
strhash.has, 397
strhash.include, 397
strhash.iterate, 397
strhash.new, 398
strhash.purge, 398
strhash.resize, 398
strhash.toseq, 399
strhash.totable, 399
strings.a64, 93
strings.advance, 93
strings.align, 93
strings.appendmissing, 94
strings.between, 94
strings.bigrams, 94
strings.byte, 94
strings.capitalise, 94
strings.charmap, 94
strings.charset, 95
strings.chomp, 95
strings.chop, 95
strings.compare, 96
strings.contains, 96
strings.cut, 96
strings.diamap, 96
strings.dice, 97
strings.diffs, 97
strings.dleven, 97
strings.dump, 98
strings.eq, 98
strings.fields, 98, 793
strings.find, 99
strings.format, 100
strings.fuzzy, 102, 103
strings.ge, 104
strings.glob, 104
strings.gmatch, 104
strings.gmatches, 105
strings.gseparate, 105
strings.gsub, 105
strings.gt, 106
strings.hextodec, 107
strings.hirschberg, 107
strings.hits, 107
strings.include, 107
strings.instr, 108
strings.isaligned, 108
strings.isalpha, 108
strings.isalphanumeric, 108, 109
strings.isalphaspace, 109
strings.isalphaspec, 109
strings.isascii, 109
strings.isblank, 109
strings.iscenumeric, 109
strings.iscomplex, 110
strings.isconsonant, 110
strings.iscontrol, 110
strings.isdia, 110
strings.isending, 110
strings.isfractional, 111
strings.isgraph, 111
strings.ishex, 111
strings.isintegral, 111
strings.isisoalpha, 111
strings.isisolower, 112
strings.isisoprint, 112
strings.isisospace, 112
strings.isisoupper, 112
strings.islatin, 112
strings.islatinnumeric, 112
strings.isloweralpha, 113
strings.islowerlatin, 113
strings.ismagic, 113
strings.ismultibyte, 113
strings.isnumber, 114
strings.isnumberspace, 114
strings.isnumeric, 114
strings.isolower, 114
strings.isoupper, 115
strings.ispattern, 115
strings.isprintable, 115
strings.isspace, 115
strings.isspec, 115
strings.isstarting, 116
strings.issubseq, 116
strings.isupperalpha, 116
strings.isupperlatin, 117
strings.isutf8, 117
strings.isvowel, 117

strings.iswrapped, 117
strings.iterate, 117
strings.itouni, 118
strings.jaro, 118
strings.join, 118
strings.lcs, 119
strings.le, 119
strings.leven, 119
strings.ljustify, 119
strings.lpad, 120
strings.lrs, 120
strings.lrtrim, 120
strings.lt, 121
strings.ltrim, 121
strings.match, 121
strings.matches, 121
strings.mfind, 122
strings.neq, 122
strings.ngrams, 122
strings.obfusxor, 122
strings.pack, 123
strings.packsize, 123
strings.random, 123
strings.remove, 123
strings.repeat, 124
strings.replace, 124
strings.reverse, 125
strings.rjustify, 125
strings.rotateleft, 126
strings.rotateright, 126
strings.rpad, 125
strings.rtrim, 125
strings.separate, 126
strings.shannon, 126
strings.strchr, 127
strings.strcmp, 127
strings.strcoll, 127
strings.strcspn, 128
strings.strftime, 128
strings.stricmp, 130
strings.strlen, 130
strings.strncmp, 131
strings.strnstr, 131
strings.strptime, 131
strings.strrchr, 132
strings.strspn, 132
strings.strstr, 133
strings.strtoul, 133
strings.strverscmp, 133
strings.strxfrm, 134
strings.sub, 134
strings.tobytes, 134
strings.tochars, 135
strings.tolatin, 135
strings.tonumber, 136
strings.toutf8, 136
strings.transform, 136
strings.uncapitalise, 137
strings.unpack, 137
strings.unwrap, 137
strings.utf8size, 138
strings.walker, 138
strings.words, 138
strings.wrap, 138
strmap.attrib, 401
strmap.cleanse, 401
strmap.getitem, 401
strmap.include, 401
strmap.iterate, 401
strmap.new, 402
strmap.purge, 402
strmap.resize, 402
strmap.totable, 402
subs, 69, 219, 256, 275
subset, 222, 243, 259, 278
subsop, 70, 219, 256, 275
suffix.autocomplete, 196
suffix.count, 197
suffix.find, 197
suffix.findall, 198
suffix.gfind, 199
suffix.lcp, 194
suffix.read, 195
suffix.sa, 193
suffix.save, 195
suffix.unique, 199
sumup, 71, 220, 256, 275
swap, 73, 220, 256, 275
switchd, 993
symmod, 411
tables.allocate, 225
tables.array, 225
tables.borders, 226
tables.col, 226
tables.concat, 226
tables.dimension, 226
tables.entries, 227
tables.extend, 227
tables.getarray, 228
tables.getdim, 228
tables.getfield, 228
tables.gethash, 228
tables.getsize, 229
tables.getsizes, 229

tables.gettable, 229
tables.hash, 229
tables.hashole, 229
tables.indices, 230
tables.isall, 230
tables.isarray, 231
tables.ishash, 231
tables.isnullarray, 232
tables.isone, 232
tables.isrectangular, 232
tables.issquare, 232
tables.iszero, 232
tables.maxn, 233
tables.move, 233
tables.new, 233
tables.newtable, 235
tables.numintersect, 235
tables.numminus, 235
tables.numunion, 235
tables.pack, 235
tables.parts, 236
tables.remove, 236
tables.reshuffle, 236
tables.resize, 236
tables.setfield, 237
tables.settable, 237
tables.subs, 237
tables.swapcol, 237
tables.swaprow, 237
tables.tableoftables, 238
tables.transpose, 238
tables.unpack, 238
tan, 438, 518, 729, 781
tanc, 438
tanh, 438, 518, 729, 781
tar.close, 857
tar.extract, 857
tar.lines, 857
tar.list, 858
tar.open, 858
telex.decode, 1090
telex.encode, 1090
time, 73, 219
times, 73
tonumber, 91
top, 74, 220, 256, 275
toreg, 74, 91
toseq, 75, 91
toset, 75
tostring, 92
tostringx, 92
totable, 75, 92
trie.attrib, 376
trie.getstore, 377
trie.has, 376, 377
trie.include, 377
trie.new, 377
trie.remove, 377
trim, 136
tuples.getitem, 360
tuples.getsize, 360
tuples.isall, 360
tuples.iterate, 361
tuples.map, 361
tuples.member, 361
tuples.new, 362
tuples.remove, 362
tuples.select, 362
tuples.setitem, 362
tuples.subs, 362
tuples.toreg, 362
tuples.toseq, 362
tuples.tostring, 362
tuples.totable, 362
tuples.unpack, 362
type, 220, 242, 256, 275, 287
typeof, 76, 220, 242, 256, 276, 287
uintmap.attrib, 386
uintmap.cleanse, 386
uintmap.getitem, 386
uintmap.has, 386
uintmap.include, 386, 387
uintmap.iterate, 387
uintmap.new, 387
uintmap.purge, 388
uintmap.resize, 388
uintmap.totable, 388
ulist.append, 327
ulist.checkulist, 327
ulist.dump, 327
ulist.getitem, 328
ulist.getllist, 328
ulist.getsize, 328
ulist.has, 328
ulist.isulist, 328
ulist.iterate, 328
ulist.new, 328
ulist.prepend, 328
ulist.purge, 329
ulist.put, 329
ulist.setitem, 329
ulist.sort, 329
ulist.swap, 329
ulist.toseq, 329

ulist.tostring, 329
ulist.totable, 329
unassigned, 76
unfreeze, 76
unimath.approx, 776
unimath.det, 776
unimath.fma, 776
unimath.linsolve, 776
unimath.max, 776
unimath.min, 777
unimath.sections, 777
unimath.xpdiff, 777
unimath.zeroin, 777
unimath.zeros, 778
union, 223, 244, 259, 278
unique, 77, 220, 257, 276
units.celsius, 1095
units.cm, 1096
units.fahren, 1095
units.floz, 1096
units.foot, 1095
units.gallon, 1097
units.gram, 1096
units.inch, 1096
units.km, 1095
units.liter, 1096
units.litre, 1096
units.meter, 1095
units.mile, 1095
units.ounce, 1096
units.yard, 1095
unity, 77
unpack, 78
upper, 137
utf8.charpattern, 159
utf8.chars, 159
utf8.codepoint, 160
utf8.codes, 159
utf8.len, 160
utf8.offset, 160
utils.calendar, 1063
utils.decodea85, 1063
utils.decodeb32, 1063
utils.decodeb64, 1063
utils.decodeb85, 1064
utils.decodexml, 1064
utils.decodez85, 1064
utils.encodea85, 1065
utils.encodeb32, 1065
utils.encodeb64, 1065
utils.encodeb85, 1065
utils.encodexml, 1066
utils.encodez85, 1066
utils.findfiles, 1066
utils.hexlify, 1067
utils.ilog2, 1067
utils.isdate, 1067
utils.metre, 1095
utils.newsize, 1068
utils.numiters, 1069
utils.onedim, 1069
utils.posrelat, 1070
utils.readini, 1074
utils.readxml, 1075
utils.rfc3339, 1076
utils.singlesubs, 1076
utils.speed, 1076
utils.timestamp, 1077
utils.udata, 1078
utils.unhexlify, 1080
utils.uuid, 1080
utils.writecsv, 1080
utils.writeini, 1081
values, 78, 220, 257, 276
watch, 78
whereis, 79, 257, 276
with, 985
write, 79
writeline, 79
xbase.attrib, 822
xbase.close, 822
xbase.eof, 822
xbase.fields, 822
xbase.fieldtype, 823
xbase.filepos, 823
xbase.header, 823
xbase.ismarked, 823
xbase.isopen, 823
xbase.isvoid, 823
xbase.kernel, 824
xbase.lock, 824
xbase.mark, 825
xbase.new, 825
xbase.open, 829
xbase.purge, 829
xbase.readdbf, 829
xbase.readvalue, 830
xbase.record, 830
xbase.records, 830
xbase.sync, 830
xbase.unlock, 831
xbase.wipe, 831
xbase.write, 831
xbase.writeboolean, 831

xbase.writebyte, 832
xbase.writecomplex, 832
xbase.writedate, 832
xbase.writedecimal, 833
xbase.writedouble, 833
xbase.writefloat, 834
xbase.writelong, 834
xbase.writenumber, 835
xbase.writeshort, 835
xbase.writestring, 836
xbase.writetime, 836
xbase.writeushort, 837
xdf, 439
xml.close, 851
xml.decode, 850
xml.decodexml, 850
xml.getbase, 851
xml.getcallbacks, 851
xml.new, 850
xml.parse, 851
xml.pos, 851
xml.readxml, 850
xml.setbase, 851
xml.setencoding, 851
xnor, 439
xor, 439
xpcall, 80, 1022
xsubset, 223, 244, 259, 278
zero, 439, 733
zip, 80, 257, 276
zx.ABS, 572
zx.ACS, 573
zx.ADD, 573
zx.AND, 573
zx.ASN, 573
zx.ATN, 573
zx.ATNH, 574
zx.COS, 574
zx.COSH, 574
zx.COT, 574
zx.COTH, 574
zx.CSC, 574
zx.CSCH, 575
zx.DIV, 575
zx.E, 575
zx.ERF, 575
zx.ERFC, 575
zx.EXP, 575
zx.GAM, 575
zx.genseries, 579
zx.getcoeffs, 579
zx.HYP, 576
zx.INT, 576
zx.LGAM, 576
zx.LN, 576
zx.MOD, 576
zx.MUL, 576
zx.NOT, 576
zx.OR, 577
zx.PI, 577
zx.POW, 577
zx.reduce, 579
zx.SEC, 577
zx.SECH, 577
zx.setcoeffs, 579
zx.SGN, 577
zx.SIG, 578
zx.SIN, 578
zx.SINH, 578
zx.SQR, 578
zx.SUB, 578
zx.TAN, 578
zx.TANH, 578

G

Garbage Collection, 27, 62, 500, 504, 516, 668, 743, 972, 986, 1173
Global Environment, 31
Golden ratio, 1177
Graphics, 1029
Arc, 1033, 1042
Background Colour, 1033
Circle, 1033, 1042
Colour Palette, 1034, 1042
Colours, 1030, 1032, 1035, 1045
Ellipse, 1034, 1042
File Formats, 1036
Flushing, 1033, 1034
Font, 1034, 1044
Line, 1035, 1043
Line Dash, 1034
Line Thickness, 1045
Plotting, 1029, 1031, 1035, 1040
Point, 1040, 1043
Rectangle, 1041, 1043, 1044
Triangle, 1044, 1045

H

Handlers
Exit, 25

Restart, 62
Hardware
Battery Status, 918
BIOS, 919
Cache Line Size, 919
Clock, 964
CPU, 921
Drives, 919, 927, 944, 945, 946, 964
Endianness, 922, 928, 982, 1022
GPU, 966
Keyboard, 789, 791
Mainboard, 919
Memory, 933, 949
Mouse, 951, 952
Network Adapters, 935
Reboot, etc., 961
RS-232, 911
Screen, 957, 966
Serial Ports, 911
Sound, 918
USB, 909
USB Devices, 940
VGA, 966
Hashes
Bit Mix, 174, 175
Bob Jenkins' Hash, 173
Daniel J. Bernstein Hash, 168, 169
Digit Sum, 168
Fletcher's Algorithm, 171
Fowler-Noll-Vo Hash, 171
GNU Hash, 177
Internet Checksum, 180
Internet Checksum RFC 1077, 172
ISpell, 173
MD5 Hash, 174
MurmurHash1, 175
MurmurHash2, 175
MurmurHash3, 175
ndbm Hash, 178
One-at-a-Time Hash, 176
Seed, 939, 940
SHA256, 179
SHA512, 179
Shift-Add-XOR Hash, 178
SuperFastHash, 181
System V Hash, 181
Variable-Length Hash, 181
Wang, 182
Home Directory, 1178

I

I/O, 787, 789, 807, 808
Applications, 797, 929
Base32, 1065
Base64, 1063, 1065
Base85, 1064, 1065, 1066
Buffering, 802
Closing Files, 789
CSV Files, 1085
Errors, 789, 790, 808, 809
Flushing, 803
INI Files, 1074, 1081
io Library, 787
Keyboard, 789, 798
Locks, 794
Opening Files, 788
Output, 52, 53, 804, 1086
Pipes, 797
Temporary Files or Directories, 795, 803, 939
Text Files, 798, 800
Windows Clipboard, 791, 798
XML Files, 850, 1064, 1066, 1075
iconv Port, 161
Indices
Conversion between 1-dim & n-dim, 1067, 1069
infinity, 1177
INI Files, 874
Initialisation, 36, 53, 62, 979, 1183, 1185
Internet
(please see Network), 889
ISO 8859/1 Latin-1, 111, 112
Iterator, 718, 1084, 1091

J

JSON
Decoding, 856
Encoding, 856

L

LANs
(please see Network), 889
Latin-1/15
(please see Strings), 92

Libraries
80-Bit Floating-Point, 716
Agena Environment, 969
Algebra, 405, 473
Analysis, 581
Arbitrary Precision, 473, 495, 507, 517
Astronomy, 541
AVL Trees, 347
Bags, 334
Basic Library, 17
Bi-directional Maps, 337
Bit Fields, 356
Bit Twiddling, 475
bloom, 183
Calculus, 581
Combinatorics, 756
Command-Line History, 1025
CORDIC, 569
Coroutines, 1015
cuckoo, 187
curses Binding, 1057
Data Serialisation, 201
Databases, 838
dBASE III/IV, 820
Debugging, 1016
Double-Linked Lists, 330
Fast Fourier Transform, 767
FIFO Queues, 369
Fractals, 1046
Fractions, 525
Fuzzy Search, 191
GMP Binding, 507
Graphics, 1029
gzip Compression, 859
hashes, 163
Heaps, 343
I/O, 787
iconv, 161
Initialisation, 984
initialise Function, 36
Integer-to-Integer Hash Map, 389
Integer-to-String Hash Map, 385
Interval Arithmetic, 769
libdivsufsort Binding, 193
libusb Binding, 909
LIFO Queues, 372
Linear Algebra, 629
Lookup Tables, 363
Maple Aliases, 775
MAPM Binding, 495
Memory File, 144
Modules, 984
MPFR Binding, 517
Networking via IPv4/IPv6, 889
Number Arrays, 288
Number Hash Table, 381
Number Theory, 761
Number-to-Number Hash Map, 393
numcuckoo, 378
numfilter, 378
Operating System, 917
Package Management, 984
readlib Function, 985
Registers, 267, 281
Registry Access, 989
Regular Expressions, 189
Remember Tables, 986
Sequences, 248, 260
Sets, 239, 286
Sexagesimals, 537
Sinclair ZX Spectrum Functions, 572
Singly-Linked Lists, 322
Skew Heaps, 350
Spell Check, 204
Statistics, 669
String Hash Set, 396
Strings, 85
String-to-Number Hash Map, 400
Tables, 211
Tries, 375
Tuples, 359
UNIX tar, 857
Unrolled Singly-Linked Lists, 327
UTF-8, 159
Utilities, I, 1063
Utilities, II, 1083
zip Compression, 862
library.agn, 53, 97, 1179, 1184
Licence, 1211
Linear Algebra, 629
Addition, 668
All-ones Matrix, 644
Anti-Diagonal, 633, 640
Back Substitution, 631, 647, 735
Backward Substitution, 633, 639
Column Dimension, 634, 636
Copying Matrices and Vectors, 632, 635, 652, 663, 664
Cross Product, 630, 635
Determinant, 636, 639, 640, 658, 776
Diagonal, 636, 640
Dimensions, 632, 634, 636, 660
Division, 668
Eigenvalues, 637

Eigenvectors, 637
Equality Check, 648, 652, 664, 665
Fibonacci Matrix, 639
Gaussian Elimination, 633, 639, 647, 735, 776
Gauss-Jordan Elimination, 640
Hilbert Matrix, 641
Identity Matrix, 641
Inner Product, 642
Inverse Matrix, 643
Lower Triangular Form, Check for, 644
Matrix, 650
Matrix Exponentiation, 653, 668
Matrix Multiplication, 642, 653
Multiplication, 668
Norm, 655
Normalisation, 660
Permanent, 657
Random Matrix, 658
Random Vector, 658
Rank, 639, 640, 658
Reduced Row Echelon Form (RREF), 645, 660
Row Dimension, 636, 660
Row Echeclon Form, 644
Scalar Multiplication, 660
Solving Linear Equations, 647, 648
Sparse, 645, 661, 666
Subtraction, 668
Trace, 663
Transpose, 663
Unit Vector, 663
Vector, 664
Vector Dot Product, 637
Zero Matrix, 645, 646, 667
Zero Vector, 644, 645, 646, 666
Linked Lists, 322
Doubly-linked, 330
Singly-linked, 324
Linux, 96, 889, 943, 945, 977, 1185, 1210
Loca, 944
Locale, 937, 958, 959, 960, 1084
for String Comparison, 127
Log Files, 1025

M

Mac, 889, 944, 977, 1053, 1182, 1185, 1210
Maple
Aliases, 775
ifactor alias, 775
ifactors alias, 775
isprime alias, 775
power alias, 775
trunc alias, 775
Mapping & Zipping, 18, 42, 80, 90, 216, 223, 241, 244, 252, 257, 259, 271, 276, 278, 286, 314, 318, 653, 654, 666, 667
Matrices, 226, 246, 261, 281, 629, 650, 655
Adjoint, 633
Adjugate, 633
Anti-Diagonal Matrix, 633, 640, 643
Antimetric Matrix, 643
Antisymmetric Matrix, 643
Co-Factor Matrix, 633
Column Dimension, 634
Column Rotation, 659
Deleting Rows and Columns, 636
Determinant, 636, 658
Diagonal Matrix, 636, 640, 643
Doolittle LU Decomposition, 648
Eigenvalues, 637
Eigenvectors, 637
Fibonacci Matrix, 639
Frobenius Norm, 649
Hilbert Matrix, 641
Identity Matrix, 641, 644
Infinity-Norm, 649
Inner Product, 642
Inverse Matrix, 643
Joining and Extending, 633, 637, 661
Kronecker Product, 646
Linear Combinations of Rows or Columns, 632
Lower Triangular Form, 644
LU Decomposition, 647
LUP Decomposition, 648
Minor, 652
n-Norm, 649
Norm, 655
Normalisation, 660
One-Norm, 650
One-Norm of a Column, 656
Outer Product Matrix, 657
Permanent, 657
Pivoting, 657
Random, 658
Rank, 658
Reduced Row Echelon Form, 645

Reduced Row Echelon Form (RREF), 660
Reshaping, 658
Row Dimension, 660
Row Echelon Form, 644
Row Rotation, 659
Scalar Product, 649, 652
Singularity, 644, 645
Skew-Symmetric Matrix, 643
Sparse Matrix, 645, 661
Square Matrix, 645
Submatrix, 661
Subvector, 662
Swapping Columns, 662
Swapping Rows, 663
Symmetric Matrix, 646
Trace, 663
Transpose, 663
Zero Matrix, 667
Memory, 933, 982
Aligning to Word Boundaries, 1068
Optimum Size, 265, 1067, 1068
Metamethods, 21, 34, 56, 66, 215, 218, 240, 251, 255, 270, 274, 1173
__call, 972, 1093
Multisets, 334, 1083

N

Network, 889
Accepting, 895
Accepting Connections, 890
Adapters, 935
Administrative Information, 896
Bi-directional Connections, 893
Binding Sockets, 890, 896
Black and White Lists, 893, 895, 898
Blocking Mode, 897
Closing Connections, 891, 897
Connecting to a Network Drive, 953
Connecting to a Server, 891, 897
Creating Sockets, 889, 901
DNS, 898, 900, 907
Domain Name, 937, 953
Hexadecimal IPv6 Address, 899
Hostname, 899
HTTP, 907
IP Address, 937
Listening for Incoming Connections, 890, 900
Lookups, 900
MAC Address, 935, 939
Maximum Number of Sockets, 896
Network Byte Order, 899, 900, 901
Numerical IPv4 Address, 899
Peer Connection, 904
Ping, 905
Receiving Data, 890, 903
Sending Data, 891, 904
Socket Activities, 906
Socket Status Information, 891, 902
Sockets, 889
Windows & Winsock, 897, 903
Numbers, 40, 76, 92, 101, 430
Binary, 441
Conversion to a Base, 133, 443
Conversion to String, 92
Decimal Comma, 1088
Hexadecimal, 450
Minus Zero, 436, 451, 465, 720, 737
Octal, 460
Smallest and Largest, 454, 466

O

OOP
aconv Package, 162
bloom Package, 184, 323, 330, 335, 360
cuckoo Package, 188
dblhash Package, 382
gzip Package, 859
heaps Package, 347, 350
io Package, 788
keyfile Package, 881
memfile Package, 145
minizip Package, 868
numarray Package, 291
numcuckoo Package, 379
rbtree Package, 353
regex Package, 189
sema Package, 1011
Strings, 93
xbase Package, 820
xml Package, 851
Opcodes
of a Function, 1020
Opening Files
Files, 792
OpenSolaris, 945
Operating System Access
Group Name, 940
os Library, 917

Thread ID, 939
User Name, 965
Waiting, 966
Operators
Binary, 1172
Unary, 1172
OS/2 Warp 4, 58, 792, 918, 936, 942, 944, 951, 952, 954, 958, 977, 1101, 1188
Output
Formatting, 53, 100
printf Function, 53, 100
Printing Results, 52
Printing Tables, 53
Writing to Console or File, 79, 1088
Writing to CSV Files, 1080
Writing to DBF Files, 831
Writing to XML Files, 1082

P

Pairs, 25, 40
Deep Copying, 286
Entries, 47
Read-Only, 31, 76
Size, 286
Type, 220, 242, 256, 275, 287
User-defined Type, 220, 242, 256, 276, 287
Physical Units
Celsius, 1095
Centimetres, 1096
Fahrenheit, 1095
Feet, 1095
Fluid Ounces, 1096
Gallons, 1097
Grams, 1096
Kilometres, 1095
Litres, 1096
Metres, 1095
Miles, 1095
Ounces, 1096
Russian Handspans (Piad), 1096
US Hands, 1096
US Links, 1096
US Spans, 1096
Yards, 1095
Pipes
Closing, 797
Exchanging Data, 797
Opening, 797
Printing
Colour, 54
io.write, 805
io.writeline, 805
print, 52
printf, 53
skycrane.scribe, 1086
skycrane.tee, 1088
strings.format, 100
Procedures
Attributes, 971
C or Agena, 974
Error Handling, 22, 30, 54, 80
Exception Handling, 54
Global Variables, 1020
Local Variables, 1018
Metamethods, 1018
Optional Arguments, 25
Persistent Storage, 1019
Protected Mode, 54, 80
Returning Procedure Names, 1016
User Information, 982
Programmes
Running, 41, 63

R

Registers, 40
Cleansing, 26
Counting Items, 274, 284, 1119
Creation, 281, 283
Deletion, 276
Entries, 47, 49, 276
Equality, 276, 277
Indexing, 270
Indices, 1067, 1069
Inequality, 277
Numeric Registers, 283
registers.new Function, 283
Set Operations, 277, 278
Shift, 67
Size, 274
Subset Check, 278
Substitutions, 69, 285
Swapping Elements, 73
Two-Dimensional, 282
Registry, 989, 1018
Remember Tables
Access, 1019
Functions, 987, 988
restart Statement, 978

RS-232, 911

S

Scripting, 1180
  Conversion to Binary Executable, 1182
  Exit Status, 929, 1181
Search
  in Files, 791
  in Strings, 20, 21, 89, 107, 108, 116, 121, 122, 138, 189, 191, 1089
  in Structures, 19, 20, 23, 28, 29, 30, 35, 44, 58, 61, 63, 64, 65, 78, 79, 99, 104, 105, 214, 215, 218, 223, 224, 230, 241, 242, 243, 244, 245, 251, 252, 254, 255, 257, 258, 260, 270, 271, 273, 274, 276, 277, 279, 286, 287, 294, 338, 635
  in Word Dictionaries, 375
  Regular Expressions, 189
Search and Replace
  in Strings, 105, 107, 124
  in Structures, 55, 217, 218, 219, 241, 253, 254, 256, 272, 273, 275
Sequences, 41
  Attributes, 970, 971
  Cleansing, 26
  Counting Items, 28, 255, 265, 635, 680, 1119
  Creation, 263, 264
  Deletion, 77
  Duplicate Entries, 77
  Entries, 47, 49, 78, 257
  Equality, 257, 258
  Extending, 261
  Indexing, 251
  Indices, 78, 79, 257, 276, 1067, 1069
  Inequality, 258
  Numeric Sequences, 245, 263
  Read-Only, 31, 76
  sequences.new, 263
  Set Operations, 258, 259
  Shift, 67, 263
  Size, 68, 255
  Sorting, 68, 69, 692
  Subset Check, 259
  Substitution, 69, 70, 266
  Swapping Columns, 266
  Swapping Elements, 73
  Swapping Rows, 266
  Transpose, 266
  Two-Dimensional, 262, 265, 285
Serialisation, 41, 98, 477, 481
Sets
  Attributes, 970, 972
  Bags, 334
  Cleansing, 26, 245
  Counting Items, 28, 214, 242, 246, 635, 1119
  Creation, 245
  Deep Copying, 240
  Multisets, 334
  Read-Only, 31, 76
  Size, 68
  Substitution, 69, 70
Shell, 111, 929, 959, 1084
Signal Processing
  Discrete Cosine Transform (DCT), 589
  Discrete Sine Transform (DST), 592
  Gray Code, 490
Size
  Files, 796
Sockets, 889
  (please see Network), 889
  Accepting Connections, 895
  Binding, 896
  Blocking Mode, 897
  Client Socket, 901
  Closing, 897
  Connecting, 897
  Creating, 901
  Keeping Alive, 900
  Listening, 900
  Receiving, 903
  Selecting, 905, 906
  Sending, 904
  Server Socket, 901
  Shutting Down, 905
  Winsock, 897, 903
Solaris, 96, 889, 945, 977, 1185, 1210
Sorting, 63, 68, 69, 709
  Check, 692
  Destructive, 68, 219, 255
  Heapsort, 709
  Internal Numeric Stack, 999
  Introsort, 709
  Non-destructive, 63, 69, 256, 709, 1087
  Pixelsort, 709
  Quicksort, 709
  Radix-Sort, 63
  Shortlex Order, 63
Sound, 918

Sparc, 1033
SQLite, 879
Stack Programming
  Built-in Numerical Stack, 991
Statistics, 669
  Absolute Deviation, 675, 1089
  Arithmetic Mean, 684
  Arithmetic-Geometric Mean, 440, 519
  Autocorrelation, 674
  Bessel J Correlation, 715
  Bessel K Correlation, 715
  Beta Distribution, 676
  Binomial Probability Density, 676
  Binomial Probability Distribution, 734
  Brownian Correlation, 677
  Cardinality, 677
  Cauchy Distribution, 678
  Chisquare Distribution, 680
  Circular Correlation, 715
  Clusters, 681
  Combinations, 417
  Complemented Binomial Probability Distribution, 734
  Complemented F Distribution, 683
  Constant Correlation, 715
  Covariance, 680
  Cubic Correlation, 715
  Cumulative Density Function, 678, 734
  Cumulative Probability Binomial Distribution, 676
  Cumulative Probability Poisson Distribution, 704
  Cumulative Sum, 681
  Damped Cosine Correlation, 715
  Damped Sine Correlation, 715
  Durbin-Watson Test, 682
  Exponential Correlation, 715
  Exponential Moving Average, 682, 686
  F Distribution, 683
  Fisher's F Distribution, 684
  Five-number Summary, 684, 705
  Gamma Cumulative Distribution, 686
  Gamma Distribution Function, 686
  Gamma Distribution PDF, 686
  Gaussian Correlation, 715
  Geometric Cumulative Distribution, 687
  Geometric Mean, 688
  Harmonic Mean, 689
  Harmonic-Geometric Mean, 450
  Herfindahl-Hirschman Index, 689
  Hole Correlation, 715
  Hypergeometric Probability Density Function, 690
  Interquartile Range, 691, 705
  Inverse Binomial Distribution, 690, 734
  Inverse Normal Distribution, 690
  Inverse of Complemented F Distribution, 690
  Inverse of Normal Distribution, 734
  Kurtosis, 692
  Laplace Distribution, 693
  Linear Correlation, 715
  Local Extrema, 683, 703
  Log Series Cumulative Distribution, 694
  Logarithmic Normal Density Function, 694
  Logistic Distribution, 611, 693
  Log-sum-exp Function, 694
  Matern Correlation, 715
  Maximum Observation, 684, 695, 698
  Mean, 691, 696, 705, 712
  Mean Deviation, 675, 695
  Median, 684, 696, 698
  Median Absolute Deviation, 694
  Median Deviation, 695
  Minimum Observation, 684, 698
  Mode, 699
  Moment, 72, 435, 699
  Moving Average, 682, 686, 708
  Negative Binomial Distribution, 700
  Neighbourhoods, 700
  Normal Distribution, 700, 704
  Normalisation, 680, 706
  Observation, 701
  Outlier, 678, 705
  Pentaspherical Correlation, 715
  Percentile, 703, 704
  Periodicy, 674
  Poisson Probability Density, 704
  Power Correlation, 715
  Probability Density Function, 703
  Quadratic Mean, 705
  Quadratic Mean Deviation, 435, 696
  Quartile, 684
  Rational Quadratic Correlation, 715
  Simple Moving Average, 688, 708
  Simple Moving Median, 688, 709
  Skewness, 708
  Softmax Function, 694
  Spherical Correlation, 715
  Standard Deviation, 435, 707, 1089
  Standard Normal Distribution, 616, 703
  Standard Score, 714

Standardisation, 710
Student's t-Distribution, 710
Summation Function, 56, 71, 217, 220, 253, 272, 711
Trimean, 712
Variance, 435, 696, 697, 1089
Volatility, 690, 1089
Weights, 714
White Noise Correlation, 715
Winsorised Mean, 714
Z-Score, 714
stdin, stdout, stderr, 788
Strings, 41, 76, 85, 126
Alignment, 93, 119, 120, 125
ASCII Code, 21, 89, 94, 134, 791
Autocomplete, 196
Bigrams, 94
Blanks, 109
Captures, 140
Character Classes, 139
Checks, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117
Comparison, 87, 96, 97, 98, 103, 104, 106, 119, 121, 122, 127, 133, 1089
Concatenation, 87, 88, 107, 118, 125, 226, 261, 1173
Control Characters, 110
Conversion to Number, 91, 712
Counting, 107, 138, 197
Counting Characters, 91, 138
Damerau-Levenshtein Distance & Similarity, 97
Deletion, 124
Diacritics, 86, 110
Diacritics and Ligatures, 96
Dice Coefficient, 97
Edit Distance, 97, 107
Embedded Zero, 61
Escape Sequences, 1187
Formatting, 88, 100, 102
Fuzzy Matching, 103, 107, 191
Hexadecimal, 107
Insertion, 107
ISO 8859/1 Latin-1, 111, 112, 114, 115, 135, 136, 161
Jaro Similarity, 118
Jaro-Winkler Similarity, 118
Largest Repeating Substring, 120
Levenshtein Distance & Similarity, 107, 119
Longest Common Prefix Array, 194
Longest Common Subsequence (LCS), 116, 119
Lower & Upper Case, 86, 94, 113, 116, 117, 120, 137
Mapping a Function, 90, 136
n-Grams, 122
Normalised Pair Distance, 97
Obfuscation, 122
Padding, 87, 88, 119, 120, 125
Pattern Items, 140
Pattern Matching, 105, 107, 110, 115, 116, 122, 139
Prefix Trees, 375
Printable Characters, 115
Punctuation Characters, 115
Regular Expressions, 189
Repetition, 124
Search & Replace Functions, 87, 89, 90, 99, 104, 105, 108, 110, 116, 121, 122, 123, 124, 127, 128, 131, 132, 133, 197, 198, 199, 375, 1076, 1086, 1089
Serialisation, 123, 137, 202
Shannon Entropy, 126, 138
Shortest Unique Substring, 199
Size, 91, 130, 138
Special Characters, 109, 115
Splitting into Characters, 74, 75, 91, 92
Splitting into Words, 89, 96, 105, 126
strings Library, 92
Substrings, 88, 93, 134
Suffix Array, 193
Suffix Trees, 375
Trimming, 120, 121, 125, 136
Unicode, 159
UTF-8, 113, 117, 135, 136, 138, 159, 160, 161
UTF-8 Encoding, 159
Structures
Read-Only, 76
Recursive Descent, 29, 35, 58, 214, 215, 218, 240, 241, 250, 252, 254, 269, 271, 273, 286
Write-Protection, 31
System Information, 948, 957, 961
System Settings, 975, 1178, 1183
System Variables, 936, 1178
_G, 31, 62, 1179
_origG, 62
_PROMPT, 1179
_RELEASE, 61
AGENAPATH, 57, 58

environ.buffersize, 811, 997
environ.homedir, 62, 1178
environ.kernel/debug, 975, 976
environ.kernel/digits, 976
environ.kernel/emptyline, 976
environ.kernel/gui, 977
environ.kernel/libnamereset, 978
environ.kernel/longtable, 978
environ.kernel/promptnewline, 979
environ.kernel/signeddigits, 979
environ.kernel/skipagenapath, 979
environ.kernel/zeroedcomplex, 980
environ.withprotected, 37
environ.withverbose, 37
Getting Environment Variables, 928
lasterror, 54
libname, 36, 57, 58, 62, 978, 1178, 1183, 1186
mainlibname, 36, 57, 62, 978, 1178
Setting Environment Variables, 958

T

Tables, 211
Array Part, 225, 228, 231, 236
Arrays, 231, 232
Assignment, 225, 285
Attributes, 231, 969, 970
Automatic Tables, 225
Cleansing, 26
Counting Items, 28, 68, 214, 229, 233, 235, 635, 680, 1119
Creation, 226, 234, 235
Deep Copying, 213, 240, 250, 269
Deletion, 55, 77, 217, 220, 236, 272
Dictionaries, 231
Duplicate Entries, 30, 77, 214, 251, 270
Entries, 47, 49, 78, 215, 227
Equality, 221
Extending, 227
Functions, 18, 19, 61, 64, 69, 220, 223, 278, 368
Hash Part, 228, 229, 231, 236
Holes, 229, 232, 1135
Holes, Removing, 77, 220
Indexing, 215
Indices, 78, 79, 226, 257, 276, 983, 1067, 1069
Inequality, 221
Insertion, 55, 225
Nested Tables, 43
Numeric Tables, 233
Read-Only, 31
Recursive Descent, 43
References, 975
Removing Holes, 43, 70, 227
Set Operations, 222, 223
Shift, 67, 233
Size, 68, 219, 229, 231, 1135
Sorting, 63, 68, 69, 692
Subset Check, 222, 223
Substitution, 69, 70, 237, 368
Swapping Columns, 237
Swapping Elements, 73
Swapping Rows, 237
tables Library, 225
Transpose, 238
Two-Dimensional, 228, 232, 238
TCP
(please see Network), 889
Terminal Applications, 1057
Terminal Device, 792
Threads, 1015
TI-30, 445, 467, 469, 537
Timestamp, 702, 836, 1077
Apache Common Log Format, 924
asctime(), 924
Decomposition, 131
ISO 8601, 924
RFC 3339, 1076
Types, 26, 34, 49, 50, 51, 67, 76, 220, 1111
Casting, 475, 482, 754, 1142

U

Unassignment
clear Statement, 27
undefined, 1177
UNIX, 36, 57, 58, 794, 797, 810, 824, 844, 918, 934, 945, 984, 1033, 1038, 1182, 1185
Userdata, 1078
UTF-8
(please see Strings), 92
UUID, 981, 1080

V

Values
Assigned Names, 22, 76
Comparisons, 56, 221, 222, 242, 243, 257, 258, 276, 277, 287
Reading Values from File, 57
Saving Values to File, 64
Vectors, 629, 664
1-Norm, 656
Infinity-Norm, 642
Infinity-Norm of Column, 641
n-Norm, 655
Random, 658
Subvector, 662

W

Windows, 36, 57, 58, 96, 791, 792, 794, 798, 810, 824, 844, 889, 918, 926, 942, 946, 961, 962, 977, 984, 1033, 1038, 1053, 1185, 1210
Clipboard, 791, 798
Loaded Modules (DLLs), 937
Network Adapters, 935
Process Id, 937, 939, 955
Processes, 937, 939
System Directory, 940
USB Devices, 940
Windows Directory, 940

X

XML, 1089
expat Binding, 849
Reading XML Streams, 849, 850, 1064, 1075
Writing XML Streams, 1066, 1082